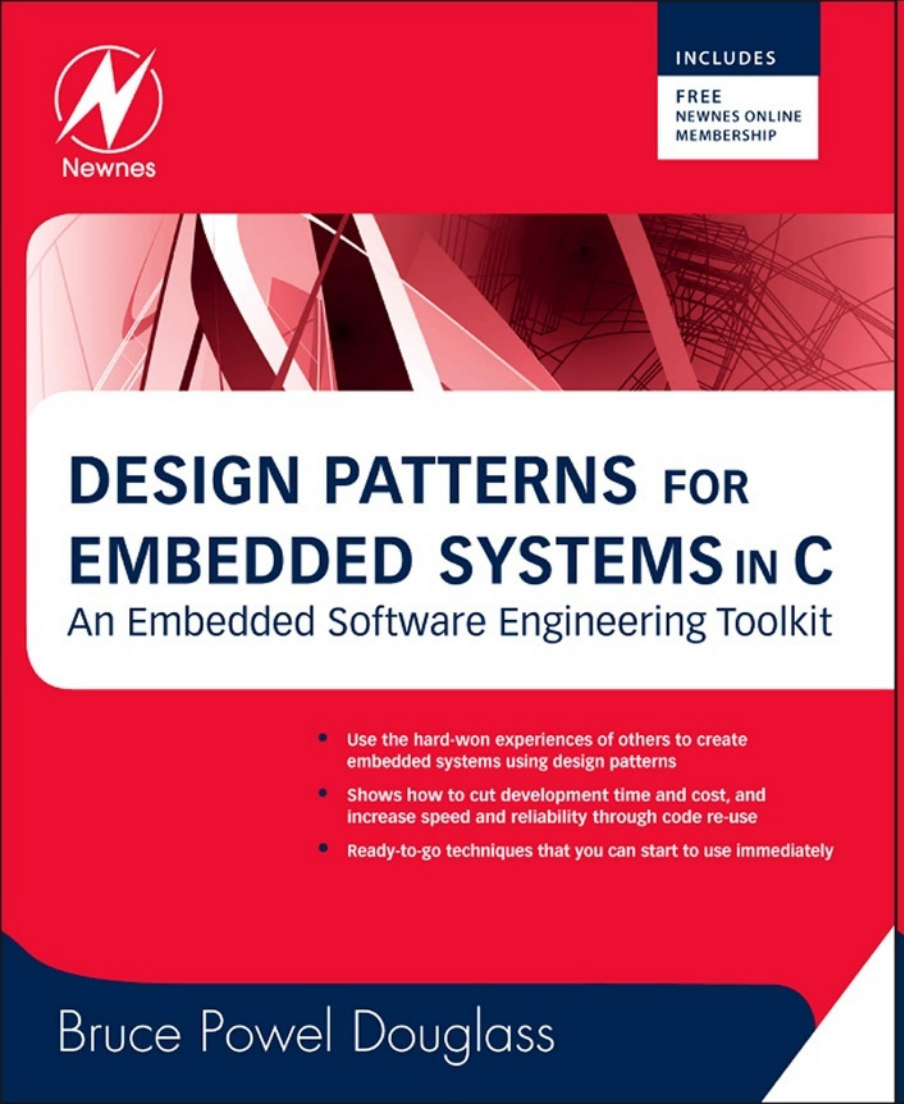


Newnes
INCLUDES
FREE
NEWNES ONLINE
MEMBERSHIP
DESIGN PATTERNS FOR
EMBEDDED SYSTEMS IN C
An Embedded Software Engineering Toolkit
Use the hard-won experiences of others to create embedded systems using design patterns
Shows how to cut development time and cost, and increase speed and reliability through code re-use
Ready-to-go techniques that you can start to use immediately
Bruce Powel Douglass

Design Patterns for Embedded Systems in C

C语言嵌入式系统设计模式

This page intentionally left blank

本页有意留白

Design Patterns for Embedded Systems in C

An Embedded Software Engineering Toolkit

Bruce Powel Douglass, PhD

C语言嵌入式系统设计模式

嵌入式软件工程工具包

Bruce Powel Douglass，博士

ELSEVIER

AMSTERDAM • BOSTON • HEIDELBERG • LONDON
NEW YORK • OXFORD • PARIS • SAN DIEGO
SAN FRANCISCO • SINGAPORE • SYDNEY • TOKYO
Newnes is an imprint of Elsevier

Newnes

ELSEVIER

阿姆斯特丹 •波士顿 • 海德堡 • 伦敦 纽约 • 牛津 • 巴黎 • 圣迭戈 旧金山 • 新加坡 • 悉尼 • 东京 Newnes 是爱思唯尔的出版品牌

Newnes

Newnes is an imprint of Elsevier
Linacre House, Jordan Hill, Oxford OX2 8DP, UK
30 Corporate Drive, Suite 400, Burlington, MA 01803, USA

First edition 2011

Copyright © 2011 Elsevier Inc. All rights reserved.

No part of this publication may be reproduced, stored in a retrieval system or transmitted in any form or by any means electronic, mechanical, photocopying, recording or otherwise without the prior written permission of the publisher.

Permissions may be sought directly from Elsevier's Science & Technology Rights Department in Oxford, UK: phone (+44) (0) 1865 843830; fax (+44) (0) 1865 853333; email: permissions@elsevier.com. Alternatively you can submit your request online by visiting the Elsevier web site at http://www.elsevier.com/locate/permissions, and selecting *Obtaining permission to use Elsevier material.*

Notice:
No responsibility is assumed by the publisher for any injury and/or damage to persons or property as a matter of products liability, negligence or otherwise, or from any use or operation of any methods, products, instructions or ideas contained in the material herein. Because of rapid advances in the medical sciences, in particular, independent verification of diagnoses and drug dosages should be made.

Library of Congress Cataloging-in-Publication Data
Douglass, Bruce Powel.
Design patterns for embedded C: an embedded software engineering toolkit / Bruce Powel Douglass – 1st ed.
p. cm.
Includes index.
ISBN 978-1-85617-707-8
1. C (Computer program language) 2. Embedded computer systems–Programming. 3. Software patterns. I. Title.
QA76.73.C15D685 2011
005.13'3–dc22

2010027721

A catalog record for this book is available from the Library of Congress

British Library Cataloguing in Publication Data
A catalogue record for this book is available from the British Library

For information on all Newnes publications
visit our website at books.elsevier.com

ISBN: 978-1-85617-707-8

Printed and bound in USA
13 12 11 10 9 8 7 6 5 4 3 2 1

Working together to grow
libraries in developing countries
www.elsevier.com | www.bookaid.org | www.sabre.org
ELSEVIER BOOK AID International Sabre Foundation

Newnes 是爱思唯尔旗下的出版品牌，地址：Linacre House, Jordan Hill, 牛津 OX2 8DP, 英国；30 Corporate Drive, Suite 400, 伯灵顿, 马萨诸塞州 01803, 美国

第一版 2011

版权所有 © 2011 Elsevier Inc. 保留所有权利。

未经出版商事先书面许可，不得以任何形式或任何手段（电子、机械、影印、录制或其他方式）复制、存储于检索系统或传播本出版物的任何部分。

可通过以下方式直接向英国牛津的Elsevier科学与技术版权部申请许可：电话 (+44) (0) 1865 843830；传真 (+44) (0) 1865 853333；电子邮件：permissions@elsevier.com。或者，您也可以在线提交请求，访问Elsevier网站 http://www.elsevier.com/locate/permissions，并选择"获取使用Elsevier材料的许可"。

注意：
对于因产品责任、疏忽或其他原因，或因使用或操作本文所含任何方法、产品、说明或想法而导致的人身伤害和/或财产损失，出版商不承担任何责任。特别是由于医学科学的快速发展，应对诊断和药物剂量进行独立验证。

美国国会图书馆在版编目数据 Douglass, Bruce Powel. 嵌入式C设计模式：嵌入式软件工程工具包 / Bruce Powel Douglass – 第1版. p. cm. 含索引. ISBN 978-1-85617-707-8 1. C（计算机程序语言） 2. 嵌入式计算机系统–编程. 3. 软件模式. I. 书名. QA76.73.C15D685 2011 005.13'3–dc22

2010027721

本书的编目记录可从美国国会图书馆获取

大英图书馆在版编目数据 本书的编目记录可从大英图书馆获取

如需了解所有Newnes出版物的信息，请访问我们的网站 books.elsevier.com

ISBN：978-1-85617-707-8

在美国印刷和装订
13 12 11 10 9 8 7 6 5 4 3 2 1

携手共进，助力发展中国家图书馆发展www.elsevier.com | www.bookaid.org | www.sabre.org
ELSEVIER BOOK AID International Sabre Foundation

Dedication

This book is dedicated to my elder son Scott, who graduated with an almost-4.0 average in physics and is going on to graduate school – nicely done Scott!

献词

本书献给我的长子斯科特，他以接近4.0的平均成绩毕业于物理学专业，即将进入研究生院深造——干得漂亮，斯科特！

This page intentionally left blank

本页有意留白

Contents

Preface ..... *xv*
Acknowledgements ..... *xvii*
About the Author ..... *xviii*

Chapter 1 What Is Embedded Programming? *1*
1.1 What's Special About Embedded Systems? 1
1.1.1 Embedded Design Constraints 3
1.1.2 The Embedded Tool Chain 4
1.1.3 OS, RTOS, or Bareback? 5
1.1.4 Embedded Middleware 6
1.1.5 Codevelopment with Hardware 7
1.1.6 Debugging and Testing 8
1.2 OO or Structured – It's Your Choice 9
1.2.1 Classes 10
1.2.2 Objects 13
1.2.3 Polymorphism and Virtual Functions 14
1.2.4 Subclassing 15
1.2.5 Finite State Machines 25
1.3 What Did We Learn? 33

Chapter 2 Embedded Programming with The Harmony™ for Embedded RealTime Process *35*
2.1 Basic Elements of the Harmony Process 36
2.1.1 A Quick Overview of the Development Workflow 36
2.1.2 What Is a Design Pattern? 44
2.1.3 Basic Structure of Design Patterns 45
2.1.4 How to Read Design Patterns in This Book 46
2.1.5 Using Design Patterns in Development 47
2.1.6 Observer Pattern 66
2.2 The Approach 77
2.3 What's Coming Up 78

Chapter 3 Design Patterns for Accessing Hardware *79*
3.1 Basic Hardware Access Concepts 81

目录

前言.xv
致谢.xvii
关于作者.xviii

第1章 什么是嵌入式编程？.1
1.1 嵌入式系统有何特殊之处？.1
1.1.1 嵌入式设计约束.3
1.1.2 嵌入式工具链.4
1.1.3 操作系统、实时操作系统还是裸机？.5
1.1.4 嵌入式中间件.6
1.1.5 软硬件协同开发.7
1.1.6 调试与测试.8
1.2 面向对象还是结构化——由你选择.9
1.2.1 类.10
1.2.2 对象.13
1.2.3 多态与虚函数.14
1.2.4 子类化.15
1.2.5 有限状态机.25
1.3 我们学到了什么？.33

第2章 使用Harmony™进行嵌入式编程——面向嵌入式实时处理.35

2.1 Harmony过程的基本要素.36
2.1.1 开发工作流快速概览.36
2.1.2 什么是设计模式？.44
2.1.3 设计模式的基本结构.45
2.1.4 如何阅读本书中的设计模式.46
2.1.5 在开发中使用设计模式.47
2.1.6 观察者模式.66
2.2 方法.77
2.3 后续内容.78

第3章 设计 访问硬件的模式.79
3.1 基础 Hardw是访问概念.81

3.2 Hardware Proxy Pattern 85
3.2.1 Abstract 85
3.2.2 Problem 86
3.2.3 Pattern Structure 86
3.2.4 Collaboration Roles 87
3.2.5 Consequences 88
3.2.6 Implementation Strategies 89
3.2.7 Related Patterns 89
3.2.8 Example 89
3.3 Hardware Adapter Pattern 96
3.3.1 Abstract 96
3.3.2 Problem 96
3.3.3 Pattern Structure 96
3.3.4 Collaboration Roles 96
3.3.5 Consequences 97
3.3.6 Implementation Strategies 98
3.3.7 Related Patterns 98
3.3.8 Example 98
3.4 Mediator Pattern 100
3.4.1 Abstract 100
3.4.2 Problem 100
3.4.3 Pattern Structure 101
3.4.4 Collaboration Roles 101
3.4.5 Consequences 102
3.4.6 Implementation Strategies 102
3.4.7 Related Patterns 102
3.4.8 Example 103
3.5 Observer Pattern 111
3.5.1 Abstract 111
3.5.2 Problem 112
3.5.3 Pattern Structure 112
3.5.4 Collaboration Roles 112
3.5.5 Consequences 114
3.5.6 Implementation Strategies 114
3.5.7 Related Patterns 115
3.5.8 Example 115
3.6 Debouncing Pattern 122
3.6.1 Abstract 123
3.6.2 Problem 123
3.6.3 Pattern Structure 123
3.6.4 Collaboration Roles 123
3.6.5 Consequences 125

3.2 硬件代理模式.85
3.2.1 摘要.85
3.2.2 问题.86
3.2.3 模式结构.86
3.2.4 协作角色.87
3.2.5 结果.88
3.2.6 实现策略.89
3.2.7 相关模式.89
3.2.8 示例.89
3.3 硬件适配器模式.96
3.3.1 摘要.96
3.3.2 问题.96
3.3.3 模式结构.96
3.3.4 协作角色.96
3.3.5 结果.97
3.3.6 实现策略.98
3.3.7 相关模式.98
3.3.8 示例.98
3.4 中介者模式.100
3.4.1 摘要.100
3.4.2 问题.100
3.4.3 模式结构.101
3.4.4 协作角色.101
3.4.5 结果.102
3.4.6 实现策略.102
3.4.7 相关模式.102
3.4.8 考试 ple.103
3.5 观察者模式.111
3.5.1 摘要.111
3.5.2 问题.112
3.5.3 模式结构.112
3.5.4 协作角色.112
3.5.5 结果.114
3.5.6 实现策略.114
3.5.7 相关模式.115
3.5.8 示例.115
3.6 去抖模式.122
3.6.1 摘要.123
3.6.2 问题.123
3.6.3 模式结构.123
3.6.4 协作角色.123
3.6.5 结果.125

3.6.6 Implementation Strategies ... 125
3.6.7 Related Patterns ... 126
3.6.8 Example ... 126
3.7 Interrupt Pattern ... 130
3.7.1 Abstract ... 130
3.7.2 Problem ... 130
3.7.3 Pattern Structure ... 131
3.7.4 Collaboration Roles ... 131
3.7.5 Consequences ... 132
3.7.6 Implementation Strategies ... 134
3.7.7 Related Patterns ... 135
3.7.8 Example ... 135
3.8 Polling Pattern ... 138
3.8.1 Abstract ... 138
3.8.2 Problem ... 138
3.8.3 Pattern Structure ... 138
3.8.4 Collaboration Roles ... 138
3.8.5 Consequences ... 141
3.8.6 Implementation Strategies ... 141
3.8.7 Related Patterns ... 142
3.8.8 Example ... 142
3.9 So, What Did We Learn? ... 147

Chapter 4 Design Patterns for Embedding Concurrency and Resource Management ... 149

4.1 Basic Concurrency Concepts ... 152
4.1.1 Identifying Tasks ... 160
4.1.2 Concurrency in the UML ... 161
4.1.3 Real-Time Operating Systems ... 163
Scheduling Patterns ... 164
4.2 Cyclic Executive Pattern ... 164
4.2.1 Abstract ... 165
4.2.2 Problem ... 165
4.2.3 Pattern Structure ... 165
4.2.4 Collaboration Roles ... 165
4.2.5 ConcreteCEThread ... 166
4.2.6 Consequences ... 167
4.2.7 Implementation Strategies ... 167
4.2.8 Related Patterns ... 167
4.2.9 Example ... 167
4.3 Static Priority Pattern ... 170
4.3.1 Abstract ... 170
4.3.2 Problem ... 171

3.6.6 实现策略.125
3.6.7 相关模式.126
3.6.8 示例.126
3.7 中断模式.130
3.7.1 摘要.130
3.7.2 问题.130
3.7.3 模式结构.131
3.7.4 协作角色.131
3.7.5 结果.132
3.7.6 实现策略.134
3.7.7 相关模式.135
3.7.8 示例.135
3.8 轮询模式.138
3.8.1 摘要.138
3.8.2 问题.138
3.8.3 模式结构.138
3.8.4 协作角色.138
3.8.5 结果.141
3.8.6 实现策略.141
3.8.7 相关模式.142
3.8.8 示例.142
3.9 那么，我们学到了什么？.147

第4章 用于嵌入并发和资源
*M*管理的设计模式. *149*

4.1 基本并发概念.152
4.1.1 识别任务.160
4.1.2 UML中的并发.161
4.1.3 实时操作系统.163
调度模式.164
4.2 循环执行模式.164
4.2.1 摘要.165
4.2.2 问题.165
4.2.3 模式结构.165
4.2.4 协作角色.165
4.2.5 ConcreteCEThread.166
4.2.6 结果.167
4.2.7 实现策略.167
4.2.8 相关模式.167
4.2.9 示例.167
4.3 静态优先级模式.170
4.3.1 摘要.170
4.3.2 问题.171

4.3.3 Pattern Structure 171
4.3.4 Collaboration Roles 172
4.3.5 Consequences 173
4.3.6 Implementation Strategies 173
4.3.7 Related Patterns 174
4.3.8 Example 175
Task Coordination Patterns 182
4.4 Critical Region Pattern 182
4.4.1 Abstract 182
4.4.2 Problem 182
4.4.3 Pattern Structure 182
4.4.4 Collaboration Roles 183
4.4.5 Consequences 184
4.4.6 Implementation Strategies 184
4.4.7 Related Patterns 184
4.4.8 Example 184
4.5 Guarded Call Pattern 190
4.5.1 Abstract 190
4.5.2 Problem 190
4.5.3 Pattern Structure 190
4.5.4 Collaboration Roles 191
4.5.5 Consequences 192
4.5.6 Implementation Strategies 192
4.5.7 Related Patterns 193
4.5.8 Example 194
4.6 Queuing Pattern 207
4.6.1 Abstract 207
4.6.2 Problem 207
4.6.3 Pattern Structure 208
4.6.4 Collaboration Roles 208
4.6.5 Consequences 209
4.6.6 Implementation Strategies 210
4.6.7 Related Patterns 210
4.6.8 Example 210
4.7 Rendezvous Pattern 224
4.7.1 Abstract 225
4.7.2 Problem 226
4.7.3 Pattern Structure 226
4.7.4 Collaboration Roles 226
4.7.5 Consequences 227
4.7.6 Implementation Strategies 228

4.3.3 模式结构.171
4.3.4 协作角色.172
4.3.5 结果.173
4.3.6 实现策略.173
4.3.7 相关模式.174
4.3.8 示例.175
任务协调模式.182
4.4 临界区模式.182
4.4.1 摘要.182
4.4.2 问题.182
4.4.3 模式结构.182
4.4.4 协作角色.183
4.4.5 结果.184
4.4.6 实现策略.184
4.4.7 相关模式.184
4.4.8 示例.184
4.5 受保护调用模式.190
4.5.1 摘要.190
4.5.2 问题.190
4.5.3 模式结构.190
4.5.4 协作角色.191
4.5.5 结果.192
4.5.6 实现策略.192
4.5.7 相关模式.193
4.5.8 示例.194
4.6 排队模式.207
4.6.1 摘要.207
4.6.2 问题.207
4.6.3 模式结构.208
4.6.4 协作角色.208
4.6.5 结果.209
4.6.6 实现策略.210
4.6.7 相关模式.210
4.6.8 示例.210
4.7 会合模式.224
4.7.1 摘要.225
4.7.2 问题.226
4.7.3 模式结构.226
4.7.4 协作角色.226
4.7.5 结果.227
4.7.6 实现策略.228

4.7.7 Related Patterns ... 228
4.7.8 Example ... 228
Deadlock Avoidance Patterns ... 231
4.8 Simultaneous Locking Pattern ... 232
4.8.1 Abstract ... 232
4.8.2 Problem ... 233
4.8.3 Pattern Structure ... 233
4.8.4 Collaboration Roles ... 234
4.8.5 Consequences ... 236
4.8.6 Implementation Strategies ... 236
4.8.7 Related Patterns ... 236
4.8.8 Example ... 236
4.9 Ordered Locking ... 242
4.9.1 Abstract ... 242
4.9.2 Problem ... 243
4.9.3 Pattern Structure ... 243
4.9.4 Collaboration Roles ... 244
4.9.5 Consequences ... 247
4.9.6 Implementation Strategies ... 248
4.9.7 Related Patterns ... 248
4.9.8 Example ... 248
4.10 So, What Have We Learned? ... 255

Chapter 5 Design Patterns for State Machines ... 257

5.1 Oh Behave! ... 259
5.2 Basic State Machine Concepts ... 261
5.2.1 OR-States ... 263
5.2.2 AND-States ... 265
5.2.3 Special Stuff: Timeouts, Conditionals, and More ... 267
5.2.4 Synchronous versus Asynchronous? ... 270
5.3 Single Event Receptor Pattern ... 274
5.3.1 Abstract ... 274
5.3.2 Problem ... 274
5.3.3 Pattern Structure ... 274
5.3.4 Collaboration Roles ... 275
5.3.5 Consequences ... 278
5.3.6 Implementation Strategies ... 278
5.3.7 Related Patterns ... 278
5.3.8 Example ... 278
5.4 Multiple Event Receptor Pattern ... 287
5.4.1 Abstract ... 287
5.4.2 Problem ... 287

4.7.7 相关模式.228
4.7.8 示例.228
死锁避免模式.231
4.8 同时锁定模式.232
4.8.1 摘要.232
4.8.2 问题.233
4.8.3 模式结构.233
4.8.4 协作角色.234
4.8.5 结果.236
4.8.6 实现策略.236
4.8.7 相关模式.236
4.8.8 示例.236
4.9 有序锁定.242
4.9.1 摘要.242
4.9.2 问题.243
4.9.3 模式结构.243
4.9.4 协作角色.244
4.9.5 结果.247
4.9.6 实现策略.248
4.9.7 相关模式.248
4.9.8 示例.248
4.10 那么，我们学到了什么？.255

第5章 状态机设计模式.257

5.1 Oh Behave!.259
5.2 基本状态机概念.261
5.2.1 或状态.263
5.2.2 AND状态.265
5.2.3 特殊内容：超时、条件及其他.267
5.2.4 同步与异步？.270
5.3 单事件接收器模式.274
5.3.1 摘要.274
5.3.2 问题.274
5.3.3 模式结构.274
5.3.4 协作角色.275
5.3.5 结果.278
5.3.6 实现策略.278
5.3.7 相关模式.278
5.3.8 示例.278
5.4 多事件接收器模式.287
5.4.1 摘要.287
5.4.2 问题.287

5.4.3 Pattern Structure 288
5.4.4 Collaboration Roles 288
5.4.5 Consequences 290
5.4.6 Implementation Strategies 290
5.4.7 Related Patterns 290
5.4.8 Example 290
5.5 State Table Pattern 296
5.5.1 Abstract 296
5.5.2 Problem 296
5.5.3 Pattern Structure 296
5.5.4 Collaboration Roles 297
5.5.5 Consequences 299
5.5.6 Implementation Strategies 299
5.5.7 Related Patterns 299
5.5.8 Example 300
5.6 State Pattern 308
5.6.1 Abstract 308
5.6.2 Problem 308
5.6.3 Pattern Structure 308
5.6.4 Collaboration Roles 310
5.6.5 Consequences 312
5.6.6 Implementation Strategies 313
5.6.7 Related Patterns 313
5.6.8 Example 314
5.7 AND-States 327
5.8 Decomposed AND-State Pattern 330
5.8.1 Abstract 331
5.8.2 Problem 331
5.8.3 Pattern Structure 331
5.8.4 Collaboration Roles 331
5.8.5 Consequences 333
5.8.6 Implementation Strategies 333
5.8.7 Related Patterns 334
5.8.8 Example 334
5.9 OK, What Have We Learned? 356

Chapter 6 Safety and Reliability Patterns 357

6.1 A Little Bit About Safety and Reliability 359
6.1.1 Safety and Reliability Related Faults 360
6.1.2 Achieving Safety and Reliability 361
6.2 One's Complement Pattern 362
6.2.1 Abstract 362
6.2.2 Problem 362

5.4.3 模式结构.288
5.4.4 协作角色.288
5.4.5 结果.290
5.4.6 实现策略.290
5.4.7 相关模式.290
5.4.8 示例.290
5.5 状态表模式.296
5.5.1 摘要.296
5.5.2 问题.296
5.5.3 模式结构.296
5.5.4 协作角色.297
5.5.5 结果.299
5.5.6 实现策略.299
5.5.7 相关模式.299
5.5.8 示例.300
5.6 状态模式.308
5.6.1 摘要.308
5.6.2 问题.308
5.6.3 模式结构.308
5.6.4 协作角色.310
5.6.5 结果.312
5.6.6 实现策略.313
5.6.7 相关模式.313
5.6.8 示例.314
5.7 AND状态.327
5.8 分解AND状态模式.330
5.8.1 摘要.331
5.8.2 问题.331
5.8.3 模式结构.331
5.8.4 协作角色.331
5.8.5 结果.333
5.8.6 实现策略.333
5.8.7 相关模式.334
5.8.8 示例.334
5.9 好的，我们学到了什么？.356

第6章 安全性与可靠性模式.357

6.1 关于安全性与可靠性的一点介绍.359
6.1.1 安全性与可靠性相关故障.360
6.1.2 实现安全性与可靠性.361
6.2 反码模式.362
6.2.1 摘要.362
6.2.2 问题.362

6.2.3 Pattern Structure ... 362
6.2.4 Collaboration Roles ... 362
6.2.5 Consequences ... 363
6.2.6 Implementation Strategies ... 363
6.2.7 Related Patterns ... 363
6.2.8 Example ... 363
6.3 CRC Pattern ... 367
6.3.1 Abstract ... 367
6.3.2 Problem ... 367
6.3.3 Pattern Structure ... 368
6.3.4 Collaboration Roles ... 368
6.3.5 Consequences ... 369
6.3.6 Implementation Strategies ... 369
6.3.7 Related Patterns ... 369
6.3.8 Example ... 369
6.4 Smart Data Pattern ... 380
6.4.1 Abstract ... 380
6.4.2 Problem ... 381
6.4.3 Pattern Structure ... 381
6.4.4 Collaboration Roles ... 382
6.4.5 Consequences ... 382
6.4.6 Implementation Strategies ... 383
6.4.7 Related Patterns ... 383
6.4.8 Example ... 383
6.5 Channel Pattern ... 395
6.5.1 Abstract ... 395
6.5.2 Problem ... 395
6.5.3 Pattern Structure ... 395
6.5.4 Collaboration Roles ... 395
6.5.5 Consequences ... 396
6.5.6 Implementation Strategies ... 397
6.5.7 Related Patterns ... 397
6.5.8 Example ... 397
6.6 Protected Single Channel Pattern ... 402
6.6.1 Abstract ... 402
6.6.2 Problem ... 402
6.6.3 Pattern Structure ... 403
6.6.4 Collaboration Roles ... 404
6.6.5 Implementation Strategies ... 405
6.6.6 Related Patterns ... 405
6.6.7 Example ... 405

6.2.3 模式结构.362
6.2.4 协作角色.362
6.2.5 结果.363
6.2.6 实现策略.363
6.2.7 相关模式.363
6.2.8 示例.363
6.3 CRC模式.367
6.3.1 摘要.367
6.3.2 问题.367
6.3.3 模式结构.368
6.3.4 协作角色.368
6.3.5 结果.369
6.3.6 实现策略.369
6.3.7 相关模式.369
6.3.8 示例.369
6.4 智能数据模式.380
6.4.1 摘要.380
6.4.2 问题.381
6.4.3 模式结构.381
6.4.4 协作角色.382
6.4.5 结果.382
6.4.6 实现策略.383
6.4.7 相关模式.383
6.4.8 示例.383
6.5 通道模式.395
6.5.1 摘要.395
6.5.2 问题.395
6.5.3 模式结构.395
6.5.4 协作角色.395
6.5.5 结果.396
6.5.6 实现策略.397
6.5.7 相关模式.397
6.5.8 示例.397
6.6 受保护单通道模式.402
6.6.1 摘要.402
6.6.2 问题.402
6.6.3 模式结构.403
6.6.4 协作角色.404
6.6.5 实现策略.405
6.6.6 相关模式.405
6.6.7 示例.405

6.7 Dual Channel Pattern 413
6.7.1 Abstract 413
6.7.2 Problem 414
6.7.3 Pattern Structure 414
6.7.4 Collaboration Roles 414
6.7.5 Consequences 417
6.7.6 Implementation Strategies 417
6.7.7 Related Patterns 417
6.7.8 Example 421
6.8 Summary 422

Appendix A UML Notation 425

1.1 Class Diagram 425
1.1.1 Semantic Elements 425
1.2 Sequence Diagram 429
1.2.1 Semantic Elements 429
1.3 State Diagram 430
1.3.1 Semantic Elements 430

Index 437

6.7 双通道模式.413
6.7.1 摘要.413
6.7.2 问题.414
6.7.3 模式结构.414
6.7.4 协作角色.414
6.7.5 结果.417
6.7.6 实现策略.417
6.7.7 相关模式.417
6.7.8 示例.421
6.8 小结.422

附录A UML表示法.425

1.1 类图.425
1.1.1 语义元素.425
1.2 序列图.429
1.2.1 语义元素.429
1.3 状态图.430
1.3.1 语义元素.430

索引.

xiv

Preface

The predominate language for the development of embedded systems is clearly C. Other languages certainly have their allure, but over 80% of all embedded systems are developed in this classic language. Many of the advances in the industry assume the use of object-oriented languages, web clients, and technologies that are either unavailable in the bare-metal development environment of embedded systems, or are too memory or resource intensive to be effectively deployed.

Design patterns are one of these profitable areas of advancement. A design pattern is a generalized solution to a recurring problem. Design patterns allow several advantages. First, they allow users to reason about solutions in a more abstract way and to codify their important properties. Since all design patterns are about optimization of some set of design criteria at the expense of others, different design patterns might address the same operational context but provide different benefits and costs. By reifying design patterns as a basic concept, we can reason about the best ways to optimize our systems and the technologies and approaches to accomplish that.

Secondly, design patterns allow us to reuse solutions that have proven themselves to be effective in other, similar circumstances. This is certainly a larger scope of reuse than reusing lines of code or individual functions. Because design patterns can be analyzed for their performance and optimization properties, we can select the best patterns for our particular problems.

Additionally, design patterns give us a larger set of reusable building blocks with which to describe our system. If you say the system uses a “symmetric deployment pattern with RMS multitasking scheduling and a triple modular redundancy approach” that sums up a significant portion of the architectural optimization decisions about your system. Design patterns make our vocabulary larger and more expressive.

Lastly, design patterns provide a unit of reference. If you Google “design patterns” you get over 3,000,000 hits. If you search the Amazon book department online, you get a list of 793 books. There is a great deal of work that goes on in the area of defining and using design patterns, so we are exposed to an enormous wealth of reusable and proven solutions from which we can select, mix, and bring to bear appropriate solutions.

前言

嵌入式系统开发的主导语言显然是 C 语言。其他语言固然有其吸引力，但超过 80% 的嵌入式系统都是用这门经典语言开发的。行业中的许多进步都假设使用面向对象语言、Web客户端以及相关技术，而这些技术在嵌入式系统的裸机开发环境中要么不可用，要么内存或资源消耗过大而无法有效部署。

设计模式正是这些有价值的进步领域之一。设计模式是对反复出现的问题的泛化解决方案。设计模式具有若干优势。首先，它们允许用户以更抽象的方式对解决方案进行推理，并将其重要属性编纂成文。由于所有设计模式都是以牺牲其他设计准则为代价来优化某一组设计准则，不同的设计模式可能针对相同的运行上下文，但提供不同的收益和成本。通过将设计模式作为基本概念进行具体化，我们可以推理出优化系统的最佳方式以及实现这一目标的技术和方法。

其次，设计模式允许我们重用那些在其他类似情况下已被证明有效的解决方案。这无疑比重用代码行或单个函数具有更大的复用范围。由于可以分析设计模式的性能和优化属性，我们可以为特定问题选择最佳的模式。

此外，设计模式为我们提供了一组更大的可复用构建块，用于描述我们的系统。如果你说系统使用了“采用RMS多任务调度和三模冗余方法的对称部署模式”，这就概括了关于系统架构优化决策的重要部分。设计模式使我们的词汇更加丰富、更具表现力。

最后，设计模式提供了一个参考单元。如果你在Google上搜索“设计模式”，你会得到超过3,000,000条结果。如果你在网上搜索Amazon图书部门，你会得到793本书的列表。在定义和使用设计模式方面有大量的工作在进行，因此我们能够接触到大量可复用且经过验证的解决方案，从中我们可以选择、混合并运用适当的解决方案。

This book brings the power of design patterns to the embedded C developer. Where appropriate, we use an object-based implementation policy with code examples to see how the patterns are implemented and used. The patterns are divided up into different areas of relevance to the embedded developer, including:

- Hardware access
- Concurrency
- State machine implementation
- Safety and reliability

In each area, different patterns are provided that address common concerns.

Audience

The book is oriented toward the practicing professional software developer and the computer science major, in the junior or senior year. It focuses on practical experience by solving posed problems from the context of the examples. The book assumes a reasonable proficiency in the C programming language. UML is used to graphically represent the structure and behavior of the patterns, but code examples are given to clearly show the design and implementation concepts.

Goals

The goal of this book is to provide a toolbox for the embedded C developer and to train him or her on the appropriate use of those tools. An agile development workflow, called the Harmony Process™ (developed by the author) is discussed in Chapter 2 to provide a process framework in which patterns have a definite role. However, the use of patterns transcends any individual process, no matter how coolly named.

By the time you've reached the end of the book, you will hopefully have the expertise to tackle the real design issues that you face in your life as a professional developer of embedded systems.

Bruce Powel Douglass, Ph.D.

本书将设计模式的力量带给嵌入式C开发者。在适当的地方，我们采用基于对象的实现策略，并附上代码示例，以了解这些模式是如何实现和使用的。这些模式按照与嵌入式开发者相关的不同领域进行划分，包括：

- 硬件访问• 并发• 状态机

实现• 安全性与可靠性

在每个领域中，都提供了不同的模式来解决常见的问题。

受众

本书面向从业的专业软件开发人员以及计算机科学专业大三或大四的学生。它侧重于通过解决示例情境中提出的问题来积累实践经验。本书假定读者具备C编程语言的合理熟练程度。UML用于以图形方式表示模式的结构和行为，但同时也给出了代码示例，以清晰地展示设计和实现概念。

目标

本书的目标是为嵌入式C开发者提供一个工具箱，并训练他们恰当地使用这些工具。第2章讨论了一种称为Harmony ProcessTM（由作者开发）的敏捷开发工作流，以提供一个模式在其中具有明确角色的过程框架。然而，模式的使用超越了任何单个过程，无论其名称多么酷炫。

当你读到本书末尾时，希望你已具备应对作为嵌入式系统专业开发者所面临的真实设计问题的专业能力。

Bruce Powel Douglass，哲学博士

Acknowledgements

I want to thank my wife Sarah for being supportive even though I work too much, and my great kids – Scott, Blake, and Tamera – for being awesome even though I work too much.

致谢

我要感谢我的妻子 Sarah，尽管我工作过于繁忙，她依然一直支持我；还要感谢我出色的孩子们——斯科特、Blake 和 Tamera——尽管我工作过于繁忙，他们依然非常棒。

About the Author

Bruce Powel Douglass was raised by wolves in the Oregon wilderness. He taught himself to read at age 3 and learned calculus before age 12. He dropped out of school when he was 14 and traveled around the U.S. for a few years before entering the University of Oregon as a mathematics major. He eventually received his M.S. in exercise physiology from the University of Oregon and his Ph.D. in neurophysiology from the USD Medical School, where he developed a branch of mathematics called autocorrelative factor analysis for studying information processing in multicellular biological neural systems.

Bruce has worked as a software developer in real-time systems for over 30 years and is a well-known speaker, author, and consultant in the area of real-time embedded systems and systems engineering. He is on the Advisory Board of the *Embedded Systems* conference and has taught courses in software estimation and scheduling, project management, object-oriented analysis and design, communications protocols, finite state machines, design patterns, and safety-critical systems design. He develops and teaches courses and consults in real-time object-oriented analysis and design and project management and has done so for many years. He has authored articles for a many journals and periodicals, especially in the real-time domain.

He is the Chief Evangelist[1] for IBM Rational, a leading producer of tools for real-time systems development, including the widely-used Rhapsody modeling tool. Bruce worked with other UML partners on the specification of the UML. He is a former co-chair of the Object Management Group's Real-Time Analysis and Design Working Group. He is the author of several other books on software, including

- *Doing Hard Time: Developing Real-Time Systems with UML, Objects, Frameworks, and Patterns* (Addison-Wesley, 1999)
- *Real-Time Design Patterns: Robust Scalable Architecture for Real-Time Systems* (Addison-Wesley, 2002)
- *Real-Time UML Third Edition: Advances in the UML for Real-Time Systems* (Addison-Wesley, 2004)

[1] Being a Chief Evangelist is much like being a Chief Scientist, except for the burning bushes and stone tablets.

关于作者

Bruce Powel Douglass 在俄勒冈荒野中被狼群抚养长大。他三岁时自学识字，十二岁前学会了微积分。十四岁时辍学，在美国各地游历了几年，之后进入俄勒冈大学攻读数学专业。他最终获得了俄勒冈大学的运动生理学理学硕士学位，以及USD医学院的神经生理学哲学博士学位，并在那里开发了一个名为自相关因子分析的数学分支，用于研究多细胞生物神经系统中的信息处理。

Bruce 在实时系统领域担任软件开发人员超过30年，是实时嵌入式系统和系统工程领域知名的演讲者、作家和顾问。他是嵌入式系统会议顾问委员会的成员，并讲授过软件估算与调度、项目管理、面向对象分析与设计、通信协议、有限状态机、设计模式以及安全关键系统设计等课程。他开发并讲授实时面向对象分析与设计及项目管理方面的课程，并提供相关咨询，已从事这项工作多年。他为许多期刊和杂志撰写过文章，尤其是在实时领域。

他是IBM Rational的首席布道师，该公司是实时系统开发工具的领先生产商，包括广泛使用的Rhapsody建模工具。Bruce与其他UML合作伙伴共同参与了UML规范的制定。他曾是对象管理组织实时分析与设计工作组的联合主席。他还撰写了其他几本关于软件的书籍，包括

• Doing Hard Time: Developing Real-Time Systems with UML, Objects, Frameworks, and Patterns（Addison-Wesley，1999）• 实时设计模式：面向实时系统的健壮可扩展架构（Addison-Wesley，2002）• Real-Time UML Third Edition: Advances in the UML for Real-Time Systems（Addison-Wesley，2004）

[1] 作为首席Evan　gelist与成为首席科学家非常相似，除了燃烧的灌木和石桌　ts.

- *Real-Time UML Workshop for Embedded Systems* (Elsevier, 2007)
- *Real-Time Agility* (Addison-Wesley, 2009)

and several others, including a short textbook on table tennis.

Bruce enjoys classical music and has played classical guitar professionally. He has competed in several sports, including table tennis, bicycle racing, running, triathlons, and full-contact Tae Kwon Do, although he currently only fights inanimate objects that don't hit back.

• 嵌入式系统实时UML工作坊（爱思唯尔，2007年）• 实时敏捷（
Addison-Wesley，2009年）

以及其他几本书，包括一本简短的乒乓球教科书。

Bruce 热爱古典音乐，并曾以古典吉他演奏为职业。他参加过多种运动，包括乒乓球、自行车竞赛、跑步、铁人三项和全接触跆拳道，不过目前他只与不会还手的无生命物体对战。

This page intentionally left blank

本页有意留白

CHAPTER 1

What Is Embedded Programming?

Chapter Outline

1.1 What's Special About Embedded Systems? 1
- 1.1.1 Embedded Design Constraints 3
- 1.1.2 The Embedded Tool Chain 4
- 1.1.3 OS, RTOS, or Bareback? 5
- 1.1.4 Embedded Middleware 6
- 1.1.5 Codevelopment with Hardware 7
- 1.1.6 Debugging and Testing 8

1.2 OO or Structured – It's Your Choice 9
- 1.2.1 Classes 10
- 1.2.2 Objects 13
- 1.2.3 Polymorphism and Virtual Functions 14
- 1.2.4 Subclassing 15
- 1.2.5 Finite State Machines 25

1.3 What Did We Learn? 33

What you will learn

- Basics of embedded systems
- OO versus structured programming
- Implementing classes, inheritance, and state machines in *C*

1.1 What's Special About Embedded Systems?

This book focuses solely on embedded systems development. In doing so, it is drawing a distinction between "embedded" systems and "others." Before we get into the depth of the discussion, we need to understand what this difference is so that we can appreciate what it forebodes with respect to the patterns and technologies we will use to develop embedded systems.

I define an embedded system as "a computerized system dedicated to performing a specific set of real-world functions, rather than to providing a generalized computing environment." Clearly, this is a broad categorization that includes tiny 8-bit computers embedded in cardiac pacemakers, linked 32-bit computers controlling avionics, communications, fire control for

Design Patterns for Embedded Systems in C
DOI:10.1016/B978-1-85617-707-8.00001-7

Copyright © 2011 Elsevier Inc.

第 1 章

什么是嵌入式编程?

本章大纲

1.1 嵌入式系统有何特殊之处? 1
- 1.1.1 嵌入式设计约束 3 1.1.2 嵌入式工具链 4 1.1.3 操作系统、实时操作系统还是裸机? 5 1.1.4 嵌入式中间件 6
- 1.1.5 软硬件协同开发 7 1.1.6 调试与测试 8

1.2 面向对象还是结构化——由你选择 9
- 1.2.1 类 10 1.2.2 对象 13 1.2.3 多态与虚函数 14 1.2.4 子类化 15 1.2.5 有限状态机 25

1.3 我们学到了什么? 33

你将学到什么

- 嵌入式系统基础
- 面向对象与结构化编程
- 在C中实现类、继承和状态机

1.1 嵌入式系统有何特殊之处?

本书仅聚焦于嵌入式系统开发。在此过程中，本书对"嵌入式"系统与"其他"系统进行了区分。在深入讨论之前，我们需要理解这一差异究竟是什么，以便我们能够领会它对于我们开发嵌入式系统所使用的模式和技术的预示意义。

我将嵌入式系统定义为"一种专用于执行特定的一组现实世界功能、而非提供通用计算环境的计算机化系统"。显然，这是一个宽泛的分类，既包括嵌入心脏起搏器中的微型8位计算机，也包括控制航空电子、通信、火控的互联32位计算机，

C语言嵌入式系统设计模式
DOI:10.1016/B978-1-85617-707-8.00001-7 版权所有

© 2011 Elsevier Inc.

aircraft, and wide-area networks composed of hundreds of powerful computer systems for battlefield management in C^4ISR (Command, Control, Communications, Computers, Intelligence, Surveillance, and Reconnaissance) systems. Many embedded systems have no disks, human interface, and barely any memory but the scope of the embedded systems market is far broader than such simple devices.

Embedded systems are everywhere:

- In the medical field, embedded systems include implantable devices (e.g., cardiac pacemakers, defibrillators, and insulin pumps), monitoring equipment (e.g., ECG/EKG monitors, blood gas monitors, blood pressure monitors, EMG monitors), imaging systems (e.g., CT, SPECT, PET, TEM, and x-ray imagers), and therapy delivery devices (e.g., patient ventilator, drug vaporizers, and infusion pumps).
- In the telecom market, there are devices ranging from cell phones, switching systems, routers, modems, and satellites.
- In automotive environments, embedded systems optimize engine combustion, manage power delivery in transmissions, monitor sensor data, control anti-lock braking, provide security, and offer infotainment services such as CD and DVD players, and GPS routing (in some locations, they can offer radar and laser detection and even active radar and laser countermeasures).
- In the office, embedded systems manage phones, printers, copies, fax machines, lights, digital projectors, security systems, and fire detection and suppression systems.
- In the home, examples include ovens, televisions, radios, washing machines, and even some vacuum cleaners.

Embedded systems already control, augment, monitor, and manage virtually every high-tech device we have from televisions to trains to factory automation, and their use is on the rise.

An important subset of embedded systems are real-time systems. Many people have the mistaken impression that "real time" means "real fast" but that is not true. A real-time system is one in which timeliness constraints must be satisfied for system correctness. A common, if simplistic, categorization of real-time systems is into two groups. "Hard" real-time systems are ones in which timeliness constraints are modeled as *deadlines*, points in time by which the execution of specific actions are required to be complete. "Soft" real-time systems are those that are not "hard"[1]; that is, some other (usually stochastic) measure than deadlines is used to determine timeliness. This may include average throughput, average execution time, maximum burst length, or some other measure. All systems may be modeled as hard real-time systems, but this often results in "over-designing" the system to be faster or have more available resources than is necessary, raising the recurring cost (approximately "manufacturing cost") of the system.

[1] Although, as the saying goes, "Hard real-time systems are hard. Soft real-time systems are harder."

以及由数百台强大计算机系统组成的广域网，用于C^4ISR（指挥、控制、通信、计算机、情报、监视与侦察）系统中的战场管理。许多嵌入式系统没有磁盘、没有人机接口，内存也几乎没有，但嵌入式系统市场的范围远比这类简单设备广泛得多。

嵌入式系统无处不在：

- 在医疗领域，嵌入式系统包括植入式设备（例如心脏起搏器、除颤器和胰岛素泵）、监护设备（例如心电图监护仪、血气监护仪、血压监护仪、肌电图监护仪）、成像系统（例如CT、SPECT、PET、TEM和X射线成像仪）以及治疗输送设备（例如患者呼吸机、药物雾化器和输液泵）。
- 在电信市场，有从手机、交换系统、路由器、调制解调器到卫星等各种设备。
- 在汽车环境中，嵌入式系统优化发动机燃烧、管理变速器中的动力传输、监测传感器数据、控制防抱死制动、提供安全保障，并提供信息娱乐服务，如CD和DVD播放器以及GPS导航（在某些地区，它们还可以提供雷达和激光探测，甚至主动雷达和激光对抗）。
- 在办公室中，嵌入式系统管理电话、打印机、复印机、传真机、指示灯、数字投影仪、安全系统以及火灾探测与抑制系统。
- 在家庭中，示例包括烤箱、电视、收音机、洗衣机，甚至一些吸尘器。

嵌入式系统已经控制、增强、监控和管理着我们拥有的几乎所有高科技设备，从电视到火车再到工厂自动化，并且它们的使用正在不断增加。

嵌入式系统的一个重要子集是实时系统。许多人错误地认为"实时"意味着"非常快"，但事实并非如此。实时系统是指必须满足时效性约束才能保证系统正确性的系统。一种常见（尽管过于简化）的实时系统分类将其分为两组。"硬"实时系统是指将时效性约束建模为截止时间的系统，即特定动作的执行必须完成的时间点。"软"实时系统是指那些不属于"硬"实时系统[1]的系统；也就是说，使用截止时间以外的其他（通常是随机性的）度量来确定时效性。这可能包括平均吞吐量、平均执行时间、最大突发长度或其他度量。所有系统都可以建模为硬实时系统，但这通常会导致对系统"过度设计"，使其比必要的更快或拥有更多可用资源，从而提高系统的经常性成本（大约相当于"制造成本"）。

[1] 尽管，一个 正如俗话所说，"硬实时系统很难。软实时系统更难。"

Even though all systems may be modeled as hard real-time systems, in actual fact, most are not. If the response is occasionally delayed or even an entire input event is missed, most systems will continue to function properly. The primary reason for modeling real-time systems as "hard" is because it eases the assurance of the system's timeliness through mathematical analysis.

1.1.1 Embedded Design Constraints

From the inside, one of the most striking characteristics of embedded systems is severity of their constraints. Unlike writing software for a general-purpose computer, an embedded system is usually shipped already integrated with all the hardware it needs. The hardware platform is not usually user-extensible, so resources such as memory, power, cooling, or computing power contribute to the per-unit cost (known as *recurring cost*). To maintain profitability, there is almost always tremendous pressure on the developer to minimize the use of such hardware resources. This means that embedded systems often require additional optimization efforts far beyond that required for desktop applications.

Beyond the need to minimize hardware, performance concerns are often critical to the success of a system. There are many aspects to performance, and different systems value these aspects differently. In some systems, throughput is a critical criterion. *Throughput* is normally measured in terms of the number of transactions, samples, connections, or messages that can be processed per unit time. In other systems, handling each request as quickly as possible is more important, a quality known as *responsiveness*, usually captured as a worst case execution time. Other systems value *predictability* of performance over maximum throughput or responsiveness. Predictability is usually measured as occurring within a range or as being drawn from a probability density function.

Reliability, robustness, and safety are other kinds of constraints levied on embedded systems. The reliability of a system is a (stochastic) measure of the likelihood that the system will deliver the correct functionality. Robustness refers to the ability of a system to deliver services properly when its preconditions (such as operating conditions or input data rates) are violated. Safety denotes the level of risk of a system, that is, the likelihood that using the system will result in an accident or loss. These concerns often require additional hardware and software measures to maintain the operation of the system within acceptable limits. For example, most embedded systems have a power on self-test (POST) as well as a periodic or continuous built-in test (BIT).

Collectively, these constraints on the system are known as the qualities of services (QoS) provided by the system. In addition to the various QoS constraints, to reduce recurring cost, it is common to create custom hardware that requires specialized device driver software.

尽管所有系统都可以建模为硬实时系统，但实际上大多数并非如此。如果响应偶尔延迟，甚至完全错过某个输入事件，大多数系统仍将继续正常运行。将实时系统建模为"硬"实时系统的主要原因是，这样做可以通过数学分析更容易地保障系统的及时性。

1.1.1 嵌入式设计约束

从内部来看，嵌入式系统最显著的特征之一是其约束的严重性。与为通用计算机编写软件不同，嵌入式系统通常在出厂时就已经集成了所需的所有硬件。硬件平台通常不可由用户扩展，因此内存、功耗、散热或计算能力等资源都会计入单位成本（称为经常性成本）。为了保持盈利能力，开发者几乎总是面临着巨大的压力，要尽量减少此类硬件资源的使用。这意味着嵌入式系统通常需要额外的优化工作，远远超出桌面应用所需的程度。

除了需要最小化硬件之外，性能问题往往对系统的成功至关重要。性能涉及许多方面，不同的系统对这些方面的重视程度各不相同。在某些系统中，吞吐量是关键标准。吞吐量通常以每单位时间内可处理的事务、样本、连接或消息的数量来衡量。在其他系统中，尽快处理每个请求更为重要，这种品质称为响应性，通常以最坏情况执行时间来衡量。还有一些系统更看重性能的可预测性，而非最大吞吐量或响应性。可预测性通常以落在某个范围内或服从概率密度函数来衡量。

可靠性、鲁棒性和安全性是对嵌入式系统施加的其他类型的约束。系统的可靠性是对系统将提供正确功能性的可能性的一种（随机）度量。鲁棒性是指系统在其前置条件（如运行条件或输入数据速率）被违反时仍能正确提供服务的能力。安全性表示系统的风险水平，即使用该系统将导致事故或损失的可能性。这些问题通常需要额外的硬件和软件措施来将系统的运行维持在可接受的限值内。例如，大多数嵌入式系统具有上电自检（POST）以及周期性或连续的内置测试（BIT）。

总体而言，这些对系统的约束被称为系统提供的服务质量（QoS）。除了各种QoS约束之外，为了降低经常性成本，通常会创建需要专用设备驱动程序软件的自定义硬件。

1.1.2 The Embedded Tool Chain

Most embedded systems are created on a different system (the "host") than they execute on (the "target"). This has a number of effects on the developer and the tools used in the creation of embedded systems. The most obvious such tool is the cross-compiler. This is a compiler that runs on the host but creates executable code that runs on a different computer and operating environment. Many real-time operating systems (RTOSs) provide their own proprietary compilers or provide customizations for open-source compilers such as GCC (Gnu Compiler Collection)[2].

A linker is a program that combines a set of executable codes together into an executable for a target. Some operating systems don't require an explicit link step because the OS loader dynamically links the program elements as it loads into memory. This is true of Unix-style embedded operating systems but most embedded OSs require an explicit linking step. The linker often relocates the program as well, meaning that the start address is specified during the link step and assembly language jump instructions must be updated to reflect the actual starting base address.

A loader is a tool that loads the object image output from the linking step into the memory of the target environment. This may be done via a serial or network link or by burning the software image into nonvolatile memory such as Flash or EPROM. As an alternative to loading the software image on a target platform, many developers use simulators for the target that execute on their host development systems. It is common, for example, to use Z80 or 8051 simulators running on Windows to begin to run, debug, and test your software even before target boards are available.

So far, the tools mentioned in the tool chain have been limited to getting software onto the system. Beyond that, we need to ensure that the software works properly. The next set of tools are debuggers – tools that give us a great measure over the execution of the software, including the ability to step into (execute line by line) or step over (execute in entirety) functions, set breakpoints, and to examine and modify variables. These debuggers may work over standard serial and network links or over JTAG[3] ports.

Modern-day Integrated Development Environments (IDEs) link together most or all of the tools in the embedded tool chain to facilitate and automate the development process. The latest industry trend is to host the IDEs in the Eclipse platform[4] because of the power of the integrated environment and the availability of third-party and open-source plug-ins. The Jazz Foundation

[2] See *www.gnu.org* for releases and supported targets.
[3] Joint Test Architecture Group, an electronic and software standard interface used by many developers for testing on embedded targets. See *www.freelabs.com/%26whitis/electronics/jtag/* or *www.embedded.com/story/OEG20021028S0049* for more information.
[4] *www.eclipse.org*

1.1.2 嵌入式工具链

大多数嵌入式系统是在与执行系统（“目标机”）不同的系统（“主机”）上创建的。这对开发人员以及创建嵌入式系统所使用的工具产生了诸多影响。最明显的此类工具是交叉编译器。这是一种在主机上运行但生成在另一台计算机和操作环境中运行的可执行代码的编译器。许多实时操作系统（RTOS）提供自己的专有编译器，或为GCC（GNU编译器集合）[2]等开源编译器提供定制化支持。

链接器是一种将一组可执行代码组合成目标机可执行文件的程序。有些操作系统不需要显式的链接步骤，因为操作系统加载器在将程序加载到内存时会动态链接程序元素。Unix风格嵌入式操作系统就是如此，但大多数嵌入式操作系统需要显式的链接步骤。链接器通常还会对程序进行重定位，这意味着起始地址在链接步骤中指定，并且汇编语言跳转指令必须更新以反映实际的起始基地址。

加载器是一种将链接步骤输出的目标映像加载到目标环境内存中的工具。这可以通过串行或网络链接完成，或者通过将软件映像烧录到非易失性存储器（如闪存或EPROM）中完成。作为在目标平台上加载软件映像的替代方案，许多开发者使用在主机开发系统上运行的目标机模拟器。例如，常见做法是使用在Windows上运行的Z80或8051模拟器，甚至在目标板可用之前就开始运行、调试和测试软件。

到目前为止，工具链中提到的工具仅限于将软件加载到系统上。除此之外，我们还需要确保软件能够正常工作。下一组工具是调试器——这些工具让我们能够很好地掌控软件的执行，包括单步进入（逐行执行）或单步跳过（整体执行）函数、设置断点以及检查和修改变量的能力。这些调试器可以通过标准串行和网络链接或通过JTAG[3] 端口工作。

现代集成开发环境（IDE）将嵌入式工具链中的大部分或全部工具链接在一起，以促进和自动化开发过程。最新的行业趋势是将IDE托管在Eclipse平台[4] 上，因为集成环境的强大功能以及第三方和开源插件的可用性。Jazz基金会

[2] 有关发布版本和支持的目标机，请参见 www.gnu.org。[3] 联合测试架构组，一种电子和软件标准接口，许多开发人员用于在嵌入式目标机上进行测试。更多信息请参见 www.freelabs.com/%26 2002 10280049 whitis/electronics/jtag/ 或 www.embedded.com/story/OEG S。[4] www.eclipse.org

takes the Eclipse platform further to integrate management, measurement, reporting, and other tools to better support collaborative software development and delivery.

1.1.3 OS, RTOS, or Bareback?

An *operating system* (OS) provides a set of system and platform execution services for the application developer, especially around the management and use of target system resources. These resources include memory, concurrency units (processes, tasks, or threads), event queues, interrupts, hardware, and application programs. Most OSs do not provide any guarantees about timeliness, and desktop OSs may invoke unpredictable delays due to internal processing, memory management, or garbage collection at unforeseeable times. This unpredictability, and the fact that most desktop OSs are huge (compared to available memory), makes them unsuitable for real-time and embedded environments that are constrained in both time and resources.

A *real-time operating system* (RTOS) is a multitasking operating system intended for real-time and embedded applications. RTOSs are generally written to provide services with good efficiency and performance, but usually the predictability of the performance is more important than the maximum throughput. In addition, RTOSs are smaller and often less capable than desktop OSs. RTOSs don't guarantee real-time performance but they provide an application environment so that appropriate developed applications can achieve real-time performance.

RTOSs run applications and tasks using one of three basic design schemas. Event-driven systems handle events as they arise and schedule tasks to handle the processing. Most such systems use task *priority* as a quantitative means by which to determine which task will run if multiple tasks are ready to run. Task priorities are most often static (i.e., specified at design time as such with *rate-monotonic scheduling*) but some are dynamic, varying the task priorities to account for current operating conditions (such as *earliest deadline first scheduling*). The other two approaches to task scheduling implement a "fairness doctrine" either by giving all tasks a periodic time slice in which to run (time-base schemas, such as *round robin scheduling*) or by running the task set cyclically (sequence-based schemas, such as *cyclic executive scheduling*).

Twenty years ago, most development houses specializing in real-time systems wrote their own proprietary RTOSs but the RTOS market has undergone a consolidation, enabling such houses to spend a greater percentage of their development efforts on their in-house specialties by buying a commercial RTOS instead.

Some of the challenges of using RTOSs include the fact that while there are some big players, such as Wind River (maker of VxWorks™) and Green Hills Software (maker of Integrity™), there are literally hundreds of RTOSs in use today, and each has its own Application Program Interface (API). Further, although RTOSs have a smaller footprint and timeliness impact than their desktop brethren, that is not the same as having *no* impact. Many embedded systems run

进一步扩展了Eclipse平台，集成了管理、度量、报告和其他工具，以更好地支持协作式软件开发和交付。

1.1.3 操作系统、实时操作系统，还是裸机?

操作系统（OS）为应用开发者提供了一组系统和平台执行服务，尤其是在目标系统资源的管理和使用方面。这些资源包括内存、并发单元（进程、任务或线程）、事件队列、中断、硬件和应用程序。大多数操作系统不提供任何关于及时性的保证，而桌面操作系统可能会由于内部处理、内存管理或垃圾回收在不可预见的时间调用不可预测的延迟。这种不可预测性，以及大多数桌面操作系统体积庞大（相对于可用内存而言），使得它们不适合在时间和资源都受限的实时和嵌入式环境中使用。

实时操作系统（RTOS）是一种面向实时和嵌入式应用的多任务操作系统。实时操作系统通常被编写为以良好的效率和性能提供服务，但通常性能的可预测性比最大吞吐量更重要。此外，实时操作系统比桌面操作系统更小，且通常功能更少。实时操作系统不保证实时性能，但它们提供了一个应用环境，使得适当开发的应用程序能够实现实时性能。

实时操作系统使用三种基本设计模式之一来运行应用程序和任务。事件驱动系统在事件出现时对其进行处理，并调度任务来处理相应的处理工作。大多数此类系统使用任务优先级作为一种量化的手段，以确定在多个任务就绪时哪个任务将运行。任务优先级通常是静态的（即，在设计时通过速率单调调度等方式指定），但有些是动态的，会根据当前运行条件改变任务优先级（例如最早截止时间优先调度）。另外两种任务调度方法实现了一种"公平原则"，要么为所有任务分配一个周期性的时间片来运行（时间片方案，如轮转调度），要么循环运行任务集（顺序方案，如循环执行调度）。

二十年前，大多数专门从事实时系统开发的开发公司都编写自己的专有实时操作系统，但实时操作系统市场经历了整合，使得这些公司能够通过购买商用实时操作系统，将更大比例的开发工作量投入到自身的内部专长上。

使用实时操作系统的一些挑战包括：虽然有一些大型厂商，例如 Wind River（VxWorksTM 的制造商）和 Green Hills Software（IntegrityTM 的制造商），但如今实际使用的实时操作系统有数百种之多，而且每种都有其自己的应用程序接口（API）。此外，尽管实时操作系统在占用空间和及时性影响方面比桌面同类系统更小，但这并不等同于没有影响。许多嵌入式系统运行

on small 8-bit processors with only a few tens of kilobytes of memory – too little for a standard RTOS. Many RTOSs support a scalable set of services (based on the Microkernel Architecture Pattern[5]) but the degree of fidelity of that scalability may not meet the needs of a tightly-constrained application.

Some systems having too few resources to support both an RTOS and the actual application software opt to go *bareback* – that is, function without a commercial or proprietary RTOS at all. This means that the application code itself must replicate any RTOS services or functionality needed by the application. With the drop in hardware costs, systems formerly done with tiny 8-bit processors have moved up to larger 16- and 32-bit processors. This is needed to add the additional complexity of behavior required in modern embedded devices and is enabled by the lower recurring costs of parts and manufacturing.

For very simple embedded systems, there may be no explicit operating system functionality at all. The application may simply be a set of interrupt handlers that communicate via a shared resource scheme such as queuing or shared memory. Alternatively, a simple task loop implementation of a cyclic executive might suffice. Somewhat more complex systems may introduce operating system features such as memory or task management as they need them.

From one point of view, an operating system is an integrated set of design patterns that provides certain kinds of services and resources to applications. Chapter 3 in this book discusses patterns for accessing hardware resources, such as timers and memory while Chapter 4 focuses on ways of managing concurrency in your embedded system.

1.1.4 Embedded Middleware

Middleware is software that connects software components together in some way. Middleware as a term dates back to the 1968 NATO Software Engineering Conference[6]. Like operating systems, there is commercial support for middleware, but in small systems it may be developed as a part of the application software. The most common middlewares for real-time and embedded systems include Common Object Request Broker Architecture (CORBA) and its many variants, and the Data Distribution Service (DDS). Both these middleware architectures are based on standards published by the Object Management Group (OMG)[7]. In the case of CORBA, there are many specialized variant standards from which to choose including Real-Time CORBA, Embedded CORBA, and Minimum CORBA.

As you might guess, middleware is appropriate for larger-scale embedded applications – those consisting of multiple software components or applications that may be distributed across

[5] See my book *Real-Time Design Patterns: Robust Scalable Architecture for Real-Time Systems* (Addison-Wesley, 2002).
[6] *http://homepages.cs.ncl.ac.uk/brian.randell/NATO/nato1968.PDF*
[7] *www.omg.org*

在只有几十千字节内存的小型8位处理器上——这对于标准实时操作系统来说太小了。许多实时操作系统支持一组可扩展的服务（基于微内核架构模式[5]），但这种可扩展性的保真程度可能无法满足严格受限应用的需求。

有些系统资源太少，无法同时支持实时操作系统和实际应用软件，因此选择裸机运行——即完全不使用商用或专有的实时操作系统。这意味着应用代码本身必须复制应用所需的任何实时操作系统服务或功能。随着硬件成本的下降，以前使用小型8位处理器的系统已升级到更大的16位和32位处理器。这是为了增加现代嵌入式设备所需的额外行为复杂性，并且得益于零件和制造经常性成本的降低而成为可能。

对于非常简单的嵌入式系统，可能根本没有显式的操作系统功能。应用可能只是一组中断处理程序，通过排队或共享内存等共享资源方案进行通信。或者，一个简单的任务循环实现的循环执行程序可能就足够了。稍微复杂的系统可能会在需要时引入内存或任务管理等操作系统特性。

从某个角度来看，操作系统是一组集成的设计模式，为应用提供某些类型的服务和资源。本书第3章讨论访问硬件资源（如定时器和内存）的模式，而第4章则侧重于管理嵌入式系统中并发的方法。

1.1.4 嵌入式中间件

中间件是一种以某种方式将软件组件连接在一起的软件。中间件这一术语可追溯到1968年的北约软件工程会议[6]。与操作系统类似，中间件有商业支持，但在小型系统中，它可能作为应用软件的一部分进行开发。实时和嵌入式系统中最常见的中间件包括公共对象请求代理体系结构（CORBA）及其众多变体，以及数据分发服务（DDS）。这两种中间件架构均基于对象管理组织（OMG）发布的标准[7]。就CORBA而言，有许多专门的变体标准可供选择，包括实时CORBA、嵌入式CORBA和最小CORBA。

正如你可能猜到的那样，中间件适用于较大规模的嵌入式应用——即由多个软件组件或应用组成、可能分布在

[5] 参见 我的 著作 实时 设计 模式： 健壮 可扩展 架构 用于 实时 系统 （Addison-Wesley， 2002）。 [6] *http://homepages.cs.ncl.ac.uk/brian.randell/NATO/nato1968.PDF* [7] *www.omg.org*

multiple processors and networks. Particularly when the components are to be developed by different organizations, the system will be extended by different organizations, or when the system is particularly long-lived, using standard middleware provides significant benefit to the developer

These middleware architectures, again like operating systems, are integrated collections of design patterns, such as Proxy, Data Bus, and Broker Pattern.

1.1.5 Codevelopment with Hardware

Many embedded projects involve the development of electronic and mechanical hardware simultaneously with the software. This introduces special challenges for the software developer who must develop software solely on the basis of predesign specifications of how the hardware is intended to work. All too often, those specifications are nothing more than notions of functionality, making it impossible to create the correct software until hardware is eventually delivered. Any slips in the hardware schedule directly induce slips in the software schedule, for which the software developers "take the heat."

I am reminded of working against a hardware specification for a telemetry system in the 1980s. In this system, the hardware was supposed to pulse encode bits – eight pulses for a "0" bit and 15 pulses for a "1" bit, with a specific time period between pulses and a longer specific time between bits. I wrote the software predicated on that specification but much to my surprise[8], when the actual hardware arrived I could only intermittently communicate with the device. Despite intensive debugging, I couldn't figure out where the software was failing – until I got out the oscilloscope, that is. With the oscilloscope I could see that when the software put a "0" bit out, the coil pulsed anywhere from six to 12 times. For a "1" bit, it pulsed anywhere from 10 to 18 times (note that the ranges for "0" and "1" bits actually overlap). The solution was to write predictor-corrector algorithm that estimated probabilities of bit error and corrected on-the-fly based on message CRC, the bit values, and their certainties. The software was certainly more complex than I had expected it to be! Such is the life of the embedded programmer.

Although much of the hardware-software integration pain can be eliminated with good specifications and early delivery of prototype hardware, much of it cannot. One of the truths of software development is that reengineering is a necessary consequence of codevelopment.

Agile methods address this concern in a couple of different ways[9]. First, with agile methods, software is constantly being developed throughout the software lifecycle. Each day, software is being written, debugged, unit tested, and delivered. And each day, different software units are integrated together and tested as a cohesive whole in a workflow known as *continuous*

[8] I was naïve back then. ;-)
[9] See my book *Real-Time Agility* (Addison-Wesley, 2009) for a thorough discussion of the topic.

多个处理器和网络上的应用。特别是当组件将由不同组织开发、系统将由不同组织扩展，或者系统特别长寿命时，使用标准中间件为开发者提供了显著的好处

这些中间件架构，同样像操作系统一样，是设计模式的集成集合，例如代理模式、数据总线和代理者模式。

1.1.5 软硬件协同开发

许多嵌入式项目涉及电子和机械硬件与软件的同时开发。这给软件开发人员带来了特殊的挑战，他们必须仅根据硬件预期工作方式的预先设计规格来开发软件。很多时候，这些规格只不过是一些功能性的概念，导致在硬件最终交付之前无法创建正确的软件。硬件进度的任何延误都会直接导致软件进度的延误，而软件开发人员则要为此"背锅"。

这让我想起20世纪80年代为一个遥测系统按照硬件规格进行开发工作的经历。在这个系统中，硬件本应以脉冲编码比特——一个"0"比特对应八个脉冲，一个"1"比特对应15个脉冲，脉冲之间有特定的时间周期，比特之间有更长的特定时间。我基于该规格编写了软件，但令我大为惊讶的是[8],当实际硬件到达时，我只能间歇性地与设备通信。尽管进行了密集的调试，我仍无法找出软件在哪里出错——直到我拿出示波器。通过示波器我可以看到，当软件输出一个"0"比特时，线圈脉冲了六到十二次不等。对于一个"1"比特，它脉冲了十到十八次不等（注意"0"和"1"比特的范围实际上有重叠）。解决方案是编写一个预测-校正算法，根据消息CRC、比特值及其确定性来估计比特错误的概率并进行实时纠正。这个软件确实比我预期的要复杂得多！这就是嵌入式程序员的生活。

尽管通过良好的规格说明和原型硬件的早期交付可以消除大部分硬件-软件集成带来的痛苦，但很多痛苦仍然无法消除。软件开发的一个真理是，再工程是协同开发的必然结果。

敏捷方法通过几种不同的方式来解决这一问题[9]。首先，在敏捷方法中，软件在整个软件生命周期中持续开发。每天都有软件被编写、调试、单元测试和交付。而且每天，不同的软件单元被集成在一起，并在称为持续集成的工作流中作为一个内聚的整体进行测试

[8] 那时我还很天真。 ;-) [9] 关于该主题的深入讨论，请参阅我的著作《实时敏捷》（Addison-Wesley，2009年）。

integration. This allows software defects to be found as early as possible. This integration can (and should) include early hardware as well.

1.1.6 Debugging and Testing

The Prussian general von Clausewitz wrote "Everything in war is very simple, but the simplest thing is very difficult[10]." He might as well have said it about software!

The simplest thing that is very difficult about software isn't writing the software – it's writing the *right software with the right functionality.* The state of the art in developing defect-free software is an agile practice known as *test-driven development* (TDD). In TDD, the unit tests for a piece of software are written simultaneously with, or even slightly before, the software it will verify. All too commonly, unit testing is skipped entirely or performed far too late to have any beneficial effect on the software.

There are many different kinds of unit tests that can be applied to software in general and embedded software in particular. These include:

- Functional – tests the behavior or functionality of a system or system element
- Quality of Service – tests the "performance" of a system or system element, often to measure the performance of the system or element against its performance requirements
- Precondition tests – tests that the behavior of the system or system element is correct in the case that the preconditional invariants are met and in the case that the preconditional invariants are violated
- Range – tests values within a data range
- Statistical – tests values within a range by selecting them stochastically from a probability density function (PDF)
- Boundary – tests values just at the edges of, just inside, and just outside a data range
- Coverage – tests that all execution paths are executed during a test suite
- Stress – tests data that exceeds the expected bandwidth of a system or system element
- Volume – also known as "load testing" – tests the system with large amounts of data that meet or exceed its design load
- Fault Seeding – tests in which a fault is intentionally introduced to the system to ensure the system handles it properly
- Regression tests – normally a subset of previously passed tests to ensure that modification to a system did not introduce errors into previously correctly functioning systems

In a desktop system, unit testing software with test cases that explore all of these concerns is rare (and difficult) enough. When you embedded that software on a less-accessible target

[10] von Clausewitz, C., 1976. *On War*. Howard, M., Paret, P. (Eds, tr.), Princeton University Press; published originally in 1832 as *Vom Kriege*.

。这使得软件缺陷能够尽早被发现。这种集成可以（也应该）包括早期的硬件。

1.1.6 调试与测试

普鲁士将军冯·克劳塞维茨曾写道："战争中的一切都很简单，但最简单的事情却非常困难[10]。"他同样可以用这句话来形容软件！

软件中最简单却又非常困难的事情，不是编写软件——而是编写具有正确功能的正确软件。开发无缺陷软件的最先进方法是被称为测试驱动开发（TDD）的敏捷实践。在TDD中，针对某段软件的单元测试与将要验证的软件同时编写，甚至略微提前编写。但通常情况下，单元测试往往被完全跳过，或者执行得太晚，以至于对软件没有任何有益的影响。

可以应用于一般软件、尤其是嵌入式软件的单元测试有很多不同类型。这些包括：

- 功能性——测试系统或系统元素的行为或功能
- 服务质量——测试系统或系统元素的"性能"，通常用于衡量系统或系统元素的性能是否满足其性能需求
- 前置条件测试——测试系统或系统元素在满足前置条件不变量以及违反前置条件不变量的情况下行为是否正确

- 范围——测试数据范围内的值
- 统计性——通过从概率密度函数（PDF）中随机选取值来测试范围内的值

- 边界——测试恰好位于数据范围边缘、紧邻内侧以及紧邻外部的值
- 覆盖率——测试在测试套件执行期间所有执行路径均被执行
- 压力测试——测试超出系统或系统元素预期带宽的数据
- 容量测试——也称为"负载测试"——用大量达到或超过其设计负载的数据测试系统

- 故障注入——在测试中故意向系统引入故障，以确保系统能够正确处理

- 回归测试——通常是先前已通过测试的子集，用于确保对系统的修改未将错误引入先前正常运行的系统中

在桌面系统中，用测试用例对软件进行单元测试并覆盖所有这些关注点已经足够罕见（且困难）。当你将该软件嵌入到不易访问的目标机

[10] 冯·克劳塞维茨，C.，1976年。《战争论》。霍华德，M.，帕雷特，P.（编、译），普林斯顿大学出版社；原于1832年以《Vom Kriege》出版。

platform it becomes even more difficult. A number of different strategies can be employed to execute test cases, such as

- "printf" testing – tests the system by writing to a file or to stdout
- "Test buddies" – writing test fixtures that embed the test cases in their own functionality
- Testing on host – performing most of the tests on the host platforms using a host native complier and a critical subset on the target platform using a cross compiler
- Simulating on host – simulating the target platform on the host with cross-compiled software and retesting a critical subset on the target with the same object code
- Commercial software testing tools – using software testing tools, such as TestRT™, LDRA™, or VectorCAST™
- Commercial hardware-software integrated tools – this includes tools such as logic analyzers, in-circuit emulators, JTAG-compliant testing tools, and ROM emulators

Of the various kinds of tests, performance tests are usually the most difficult to adequately perform. This is because most of the common methods for executing test cases involve running addition software on the same target platform, which affects the timing and performance of the software under test. Where possible, it is clearly best to use hardware-assisted debugging with in-circuit emulators or ROM emulators for performance testing.

1.2 OO or Structured – It's Your Choice

Structured programming is a disciplined form of software development that emphasizes two separate and distinct aspects.

On one hand, functions or procedures form the foundation of behavioral programming: a procedure is a collection of primitive actions with a single entry point that performs a coherence behavioral goal; a function is simply a procedure that returns a value. Procedures can be broken up into call trees by one procedure invoking another, permitting algorithmic decomposition. Procedures are usually synchronously invoked (i.e., called) but by adding additional means, asynchronous invocations can be invoked as well.

The other side of structured programming is the notion of data structuring. All third-generation computer languages have the notion of building complex data structures from more primitive elements, ultimately basing them on the basic types provided by the computer language. These may be homogeneous collections, such as arrays, or heterogeneous ones, as with C structs.

Object-oriented programming is based on an orthogonal paradigm. Rather than have two separate taxonomies, object-oriented programming has a single one based on the notion of a class. A *class* combines together both data (stored in *attributes*) and procedures (known as *operations*) that operate on that data, because they tend to be inherently tightly bound anyway.

平台时，就变得更加困难。可以采用多种不同的策略来执行测试用例，例如

• "printf"测试——通过写入文件或标准输出来测试系统
• "测试伙伴"——编写测试夹具，将测试用例嵌入其自身功能中
• 主机测试——使用宿主原生编译器在宿主平台上执行大部分测试，并使用交叉编译器在目标平台上执行关键子集测试
• 主机仿真——使用交叉编译的软件在主机上仿真目标平台，并使用相同的目标代码在目标机上重新测试关键子集
• 商业软件测试工具——使用软件测试工具，如TestRTTM、LDRATM或VectorCASTTM
• 商用软硬件集成工具——包括逻辑分析仪、在线仿真器、符合JTAG标准的测试工具和ROM仿真器等工具

在各种类型的测试中，性能测试通常是最难以充分执行的。这是因为大多数执行测试用例的常见方法都涉及在同一目标平台上运行额外的软件，这会影响被测软件的时序和性能。在可能的情况下，使用硬件辅助调试配合在线仿真器或ROM仿真器来进行性能测试显然是最佳选择。

1.2 面向对象还是结构化——由你选择

结构化编程是一种有纪律的软件开发形式，它强调两个独立且不同的方面。

一方面，函数或过程构成了行为编程的基础：过程是一组原语动作的集合，具有单一入口点，执行一个连贯的行为目标；函数只是一个返回值的过程。过程可以通过一个过程调用另一个过程来分解为调用树，从而实现算法分解。过程通常是同步调用的（即被调用），但通过添加额外的手段，也可以进行异步调用。

结构化编程的另一方面是数据结构化的概念。所有第三代计算机语言都具有从更基本元素构建复杂数据结构的概念，最终将它们建立在计算机语言所提供的基本类型之上。这些数据结构可以是同构集合，如数组，也可以是异构集合，如C结构体。

面向对象编程基于一种正交范式。面向对象编程并非采用两个独立的分类体系，而是基于类的概念采用单一分类体系。类将数据（存储在属性中）和操作该数据的过程（称为操作）结合在一起，因为它们本身往往就紧密绑定在一起。

An *object* is an instance of a class. This makes an object equivalent to, but more powerful than, a variable in a structured language because the object provides both the values held in its attributes and the operations that manipulate them.

A *structured language* is a language that directly supports structured programming. In the context of this book, C is clearly a structured language. C is by far the most prevalent language for creating embedded systems and C is an inherently "structured" language. It has all the earmarks – functions, variables, and so forth. However, an interesting question arises: can object-oriented programming be done in a structured language, such as C? And even if it *can* be done, *should* it be done?

I am reminded of a similar discussion back when the most common programming languages were assembly languages. Was it possible to write structured programs in assembly code? After all, assembly languages do not provide structured language features intrinsically. As it happens, the answer is clearly, "Yes!" It is not only possible but profitable to write structured assembly language programs. It requires some discipline on the part of the programmer and decisions about the mapping of assembly language instructions to structured concepts (such as passing parameters on the stack), but it is easy enough to do.

The same is true of object-oriented programming in structured languages such as C. In this book, we will take a primarily object-based perspective in the programming (object-oriented without subclassing) because I believe there are benefits to doing so. However, the creation of object-based or object-oriented programs in C is pretty straightforward. Let's briefly discuss the important aspects of object-oriented programming and how to implement those concepts in C.

1.2.1 Classes

A class is really nothing more than a C struct, but what is special about it is that it contains two different kinds of features: data (attributes) and behavioral (operations).

The simplest way to implement classes is simply to use the file as the encapsulating boundary; public variables and functions can be made visible in the header file, while the implementation file holds the function bodies and private variables and functions. Multiple files can be linked with «Usage» dependencies to support the call tree, such as in Figure 1-1. In this case, a "class" is represented as a pair of files and its implementation uses some features (variables or functions) of the Display class (file), in this case, the `displayMsg()` function.

A somewhat more flexible approach uses structs within the files to represent the classes. The operations of the class are defined as functions located within the same file as the struct. To make sure that the function has access to the correct object data, we need to pass in a `me` pointer.

对象是类的实例。这使得对象等同于结构化语言中的变量，但比后者更强大，因为对象既提供了其属性中保存的值，也提供了操作这些值的操作。

结构化语言是直接支持结构化编程的语言。在本书的上下文中，C 显然是一种结构化语言。C 是迄今为止用于创建嵌入式系统最普遍的语言，并且 C 本质上是一种"结构化"语言。它具备所有特征——函数、变量等等。然而，一个有趣的问题出现了：面向对象编程能否在结构化语言（如 C）中进行？即使可以做到，是否应该这样做？

这让我想起当最常见的编程语言还是汇编语言时的一场类似讨论。是否可以用汇编代码编写结构化程序？毕竟，汇编语言本身并不提供结构化语言特性。事实证明，答案显然是："可以！"编写结构化的汇编语言程序不仅可行，而且有益。这需要程序员具备一定的纪律性，并就汇编语言指令到结构化概念（如在栈上传递参数）的映射做出决策，但做起来并不难。

在C等结构化语言中进行面向对象编程也是如此。在本书中，我们将主要从基于对象的视角来进行编程（即不涉及子类化的面向对象编程），因为我相信这样做是有益的。然而，在C中创建基于对象或面向对象的程序是相当直接的。让我们简要讨论一下面向对象编程的重要方面，以及如何在C中实现这些概念。

1.2.1 类

类实际上不过是一个C结构体，但其特殊之处在于它包含两种不同类型的特性：数据（属性）和行为（操作）。

实现类的最简单方式就是直接使用文件作为封装边界；公共变量和函数可以在头文件中可见，而实现文件则包含函数体以及私有变量和函数。多个文件可以通过«Usage»依赖关系进行链接，以支持调用树，如图1-1所示。在这种情况下，"类"被表示为一对文件，其实现使用了Display类（文件）的某些特性（变量或函数），在此例中即displayMsg()函数。

一种更为灵活的方法是在文件中使用结构体来表示类。类的操作被定义为位于与结构体相同文件中的函数。为了确保函数能够访问正确的对象数据，我们需要传入一个 `me` 指针。

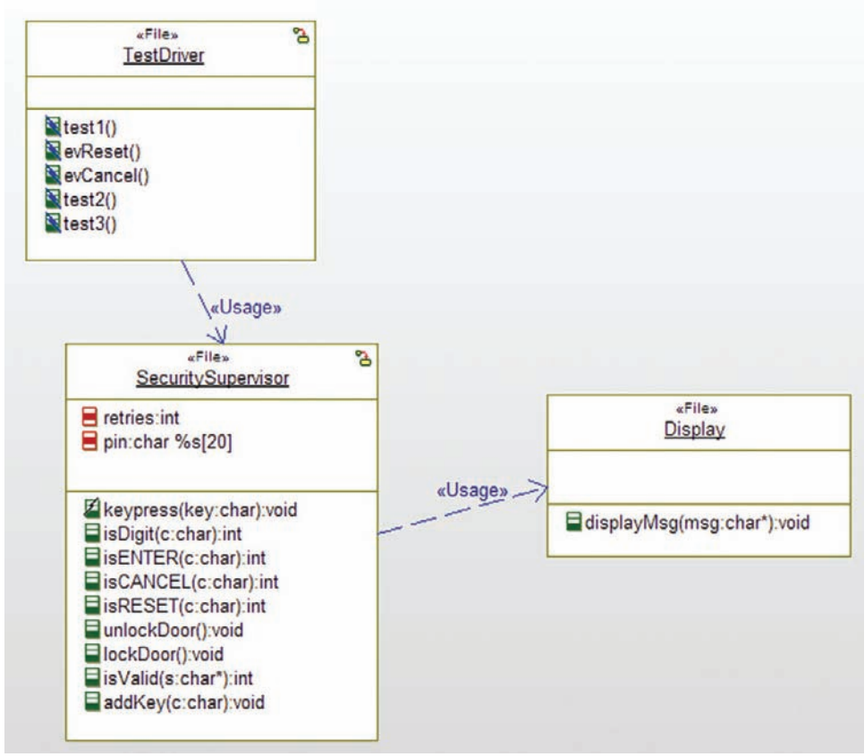


Figure 1.1: Classes represented as files in UML

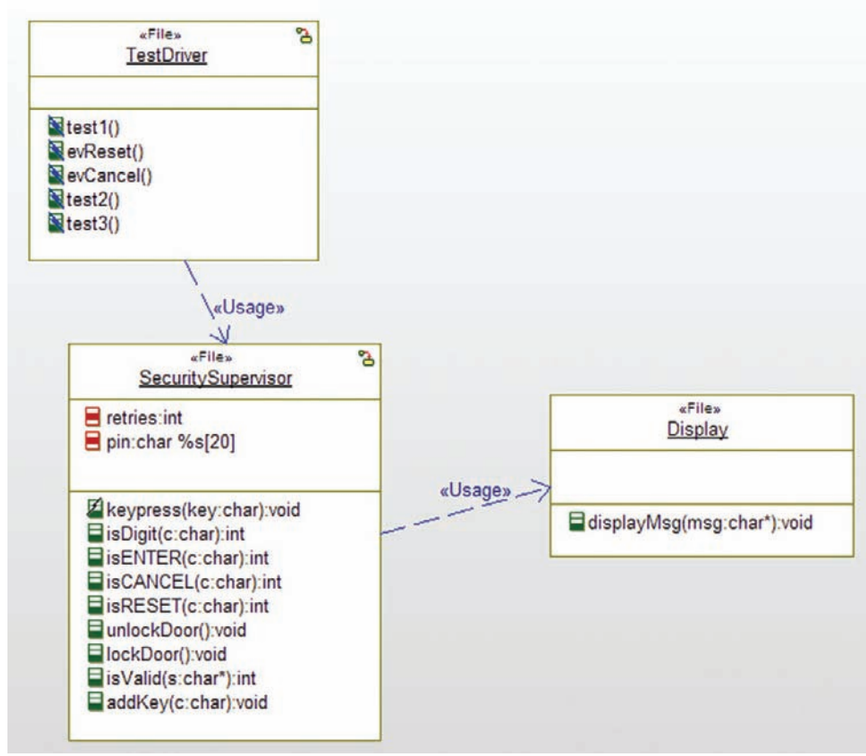


图 1.1：在 UML 中表示为类的文件

This allows us to have multiple instances (objects) of the same class and make sure that the member functions work on the correct copies of the data. In addition, classes are endowed with "special" operations. A constructor creates an object of the class. An initializer (optionally) initializes the object and its attributes. A destructor destroys the class and releases the memory used.

Consider a simple class `Sensor` with data elements `value`, `updateFrequency`, and `filterFrequency` and operations, `getValue()`, `setValue(v: int)`, `setUpdateFreq (r:int)`, `getUpdateFreq()`, `setFilterFreq(ff: int)`, and `getFilterFreq()`. The header file would look something like Code Listing 1.

```
#ifndef Sensor_H
#define Sensor_H

/*## class Sensor */
typedef struct Sensor Sensor;
```

这允许我们拥有同一个类的多个实例（对象），并确保成员函数作用于正确的数据副本。此外，类还被赋予了"特殊"操作。构造函数创建类的对象。初始化器（可选地）初始化对象及其属性。析构函数销毁类并释放所使用的内存。

考虑一个简单的类 Sensor，其数据元素为 value、updateFrequency 和 filterFrequency，操作为 getValue()、setValue(v: int)、setUpdateFreq (r:int)、getUpdateFreq()、setFilterFreq(ff: int) 和 getFilterFreq()。头文件大致如代码清单 1 所示。

```
#ifndef Sensor_H #define Sensor_
H /*## 类 Sensor */ typedef struct
Sensor Sensor;
```

```
struct Sensor {
    int filterFrequency;
    int updateFrequency;
    int value;

};

int Sensor_getFilterFrequency(const Sensor* const me);

void Sensor_setFilterFrequency(Sensor* const me, int p_filterFrequency);

int Sensor_getUpdateFrequency(const Sensor* const me);

void Sensor_setUpdateFrequency(Sensor* const me, int p_updateFrequency);

int Sensor_getValue(const Sensor* const me);

Sensor * Sensor_Create(void);

void Sensor_Destroy(Sensor* const me);
```

Code Listing 1-1: Sensor Header File

The associated implement file is shown in Code Listing 2.

```
#include "Sensor.h"

void Sensor_Init(Sensor* const me) {
}

void Sensor_Cleanup(Sensor* const me) {
}

int Sensor_getFilterFrequency(const Sensor* const me) {
    return me->filterFrequency;
}

void Sensor_setFilterFrequency(Sensor* const me, int p_filterFrequency) {
    me->filterFrequency = p_filterFrequency;
}
int Sensor_getUpdateFrequency(const Sensor* const me) {
    return me->updateFrequency;
}

void Sensor_setUpdateFrequency(Sensor* const me, int p_updateFrequency) {
    me->updateFrequency = p_updateFrequency;
}

int Sensor_getValue(const Sensor* const me) {
    return me->value;
}

Sensor * Sensor_Create(void) {
    Sensor* me = (Sensor *) malloc(sizeof(Sensor));
```


struct Sensor { int
filterFrequency; int
updateFrequency; int value;

}; int Sensor_getFilterFrequency(const Sensor* const me); void Sensor_
setFilterFrequency(Sensor* const me, int p_filterFrequency); int Sensor_
getUpdateFrequency(const Sensor* const me); void Sensor_
setUpdateFrequency(Sensor* const me, int p_updateFrequency); int Sensor_
getValue(const Sensor* const me); Sensor * Sensor_Create(void); void Sensor_
Destroy(Sensor* const me);

代码清单 1-1：传感器头文件

相关的实现文件如代码清单 2 所示。

#include "Sensor.h" void Sensor_Init(Sensor* const me) { } void Sensor_
Cleanup(Sensor* const me) { } int Sensor_getFilterFrequency(const Sensor* const me) {
return me->filterFrequency; } void Sensor_setFilterFrequency(Sensor* const me, int p_
filterFrequency) { me->filterFrequency = p_filterFrequency; } int Sensor_
getUpdateFrequency(const Sensor* const me) { return me->updateFrequency; } void
Sensor_setUpdateFrequency(Sensor* const me, int p_updateFrequency) { me->
updateFrequency = p_updateFrequency; } int Sensor_getValue(const Sensor* const
me) { return me->value; } Sensor * Sensor_Create(void) { Sensor* me = (Sensor *)
malloc(sizeof(Sensor));

```
    if(me!=NULL)
        {
            Sensor_Init(me);
        }
    return me;
}

void Sensor_Destroy(Sensor* const me) {
    if(me!=NULL)
        {
            Sensor_Cleanup(me);
        }
    free(me);
}
```

Code Listing 2: Sensor Implementation File

In this case, the file serves as the encapsulation boundary, and stylistically two files (a header .h file and an implementation .c file) group together the elements within a single class. This approach supports *object-based* programming very well, but doesn't implement virtual functions in a fashion that can be easily overridden in subclasses.

An alternative approach that supports *object-oriented* programming is to embed function pointers within the struct itself. This will be discussed shortly in Section 1.2.3.

1.2.2 Objects

Objects are instances of classes, in the same way that a specific variable `x` (of type `int`) is an instance of its type (`int`). The same set of operators (such as arithmetic operators) apply to all variables of that type, but the specific values of one instance differ from another instance (say, `y`).

In standard C programming, complex algorithms can still be embedded in classes, but it is common that these classes are *singletons*, meaning that there is exactly one instance of the class in the application. A singleton `Printer` class, for example, may have variables such as `currentPrinter` and operations like `print()`, but the application would have only a single instance. There are still benefits to encapsulating the data used by a class with the operations that act on that data, even if there is only one instance ever running. In other cases, usually more data-centric (as opposed to algorithm- or service-centric) classes will have multiple instances.

To create an instance of a class, simply create an instance of the struct. Consider one possible `main()` that creates and uses two instances of the previous Sensor class, shown in Code Listing 3:


```
    if(me!=空) { 传感器_Init(me); } return
me; } void 传感器_Destroy(传感器* const
me) { if(me!=空) { 传感器_清理(me); }
free(me); }
```

代码清单 2：传感器实现文件

在这种情况下，文件充当封装边界，并且在风格上，两个文件（一个头文件 .h 文件和一个实现 .c 文件）将单个类中的元素组合在一起。这种方法很好地支持基于对象的编程，但无法以在子类中易于重写的方式实现虚函数。

支持面向对象编程的另一种方法是将函数指针嵌入结构体本身。这将在第 1.2.3 节中简要讨论。

1.2.2 对象

对象是类的实例，就像特定变量 `x` （类型为 `int`）是其类型（`int`）的实例一样。同一组运算符（例如算术运算符）适用于该类型的所有变量，但一个实例的特定值不同于另一个实例（例如，`y`）。

在标准C编程中，复杂算法仍然可以嵌入到类中，但常见的情况是这些类是单例，这意味着应用程序中该类恰好只有一个实例。例如，一个单例`Printer`类可能拥有诸如`currentPrinter`这样的变量以及像`print()`这样的操作，但应用程序只会有一个实例。即使永远只有一个实例在运行，将类所使用的数据与作用于该数据的操作封装在一起仍然有好处。在其他情况下，通常更以数据为中心（而非以算法或服务为中心）的类会有多个实例。

要创建类的实例，只需创建结构体的实例即可。考虑一个可能的 `main()`，它创建并使用前面 Sensor 类的两个实例，如代码清单 3 所示：

```
#include "Sensor.h"
#include <stdlib.h>
#include <stdio.h>
int main(int argc, char* argv[]) {
    Sensor * p_Sensor0, * p_Sensor1;
    p_Sensor0 = Sensor_Create();
    p_Sensor1 = Sensor_Create();

    /* do stuff with the sensors ere */
    p_Sensor0->value = 99;
    p_Sensor1->value = -1;
    printf("The current value from Sensor 0 is %d\n",
        Sensor_getValue(p_Sensor0));
    printf("The current value from Sensor 1 is %d\n",
        Sensor_getValue(p_Sensor1));
    /* done with sensors */

    Sensor_Destroy(p_Sensor0);
    Sensor_Destroy(p_Sensor1);
    return 0;
}
```

Code Listing 3: Sensor main()

1.2.3 Polymorphism and Virtual Functions

Polymorphism is a valuable feature of object-oriented languages. It allows for the same function name to represent one function in one context and another function in a different context. In practice, this means that when either the static or dynamic context of an element changes, the appropriate operation can be called.

Of course, the standard way to do this in C is to use conditional statements such as if or switch. The problem with the standard approach is that it quickly becomes unwieldy when many different contexts are available. Additionally, the approach requires that all possible contexts must be known when the original function is written, or at least the function must be modified to allow a new context. With polymorphic functions, prescience isn't required. When the new context is discovered, the polymorphic function may be created and added without requiring changes to the original function.

Consider a sensor in which the actual interface may differ. The `acquireValue` function in one context may require accessing a memory-mapped sensor, waiting for a while and then reading a different address to acquire the value. A different context may use a port-mapped sensor. We would definitely prefer that the client of the sensor doesn't know how the interface to the sensor works – requiring such knowledge makes the application far more fragile and difficult to maintain. With the standard C approach, we'd write code something like Code Listing 4 with the attributes `whatKindOfInterface` added to the `Sensor struct`.

#include "Sensor.h" #include <stdlib.h>#include <stdio.h>int main(int argc, char* argv[]) { Sensor * p_Sensor0, * p_Sensor1; p_Sensor0 = Sensor_Create(); p_Sensor1 = Sensor_Create(); /* 在此处对传感器进行操作 */ p_Sensor0->value = 99; p_Sensor1->value = -1; printf("来自 Sensor 0 的当前值为 %d\n", Sensor_getValue(p_Sensor0)); printf("来自 Sensor 1 的当前值为 %d\n", Sensor_getValue(p_Sensor1)); /* 传感器使用完毕 */ Sensor_Destroy(p_Sensor0); Sensor_Destroy(p_Sensor1); return 0; }

代码清单 3：Sensor main()

1.2.3 多态与虚函数

多态是面向对象语言的一项宝贵特性。它允许同一个函数名在一个上下文中表示一个函数，而在另一个上下文中表示另一个函数。在实践中，这意味着当元素的静态或动态上下文发生变化时，可以调用相应的操作。

当然，在C语言中实现这一点的标准方法是使用条件语句，例如if或switch。标准方法的问题在于，当存在许多不同的上下文时，它会迅速变得难以驾驭。此外，该方法要求在编写原始函数时就必须知道所有可能的上下文，或者至少必须修改函数以允许新的上下文。使用多态函数则不需要预知。当发现新的上下文时，可以创建并添加多态函数，而无需更改原始函数。

考虑一个实际接口可能有所不同的传感器。在一种上下文中，`acquireValue` 函数可能需要访问内存映射传感器，等待一段时间，然后读取不同的地址以获取值。另一种上下文可能使用端口映射传感器。我们当然希望传感器的客户端不知道传感器接口是如何工作的——要求客户端具备这种知识会使应用程序更加脆弱且难以维护。使用标准C方法，我们会编写类似代码清单4的代码，并将属性`whatKindOfInterface` 添加到`Sensor` 结构体中。

```
int acquireValue(Sensor *me) {
    int *r, *w; /* read and write addresses */
    int j;
    switch(me->whatKindOfInterface) {
    case MEMORYMAPPED:
        w = (int*)WRITEADDR; /* address to write to sensor */
        *w = WRITEMASK; /* sensor command to force a read */
        for (j=0;j<100;j++) { /* wait loop */ };
        r = (int *)READADDR; /* address of returned value */
        me->value = *r;
        break;
    case PORTMAPPED:
        me->value = inp(SENSORPORT);
        /* inp() is a compiler-specific port function */
        break;
    }; /* end switch */
    return me->value;
    };
```

Code Listing 4: Polymorphism the hard way

1.2.4 Subclassing

Subclassing (known as *generalization* in the Unified Modeling Language [UML] or as *inheritance*) is a way of representing classes that are a "specialized kind of" another class. The basic rule is that the specialized class inherits the features of the more general class. That is, all of the features of the more general class (also known as the *base* or *parent class*) are also features of the specialized class (also known as the *derived* or *child class*), but the latter is allowed to both specialize and extend the more general class.

One of the key advantages of subclassing is the ability to reuse design and code. If I design a class for a specific context and want to reuse it later in a different context, I can just design and code the specializations and extensions without touching the base class. Further, in usage, I can write algorithms that manipulate an instance of a type, without regard to which subclass it might be, when that is appropriate and modify the behavior to be semantically correct with subclass when that is necessary.

By "specialize," I mean that the specialized class can redefine operations provided by the base class. Because a subclass is a specialized form of a base class, any operations that make sense for an instance of the base class should also make sense with an instance of the derived class. However, it often happens that the implementation of that operation must be slightly different. For example, if I write the code for queuing data from a sensor, it is likely to have operations such as `insert(int value)`, `int remove()`, `int getSize()`, and so on. If I create a specialized kind of a queue that can store large amounts of data out to a mass storage device such as a flash drive, those operations still make sense but their implementation will be different. Note that specialization refers exclusively to changing the implementation of operations and not at all to the redefinition of data.

int acquireValue(Sensor *me) { int *r, *w; /* 读取和写入地址 */ int j; switch(me->whatKindOfInterface) { case MEMORYMAPPED: w = (int*)WRITEADDR; /* 要写入传感器的地址 */ *w = WRITEMASK; /* 强制读取的传感器命令 */ for (j=0;j<100;j++) { /* 等待循环 */ }; r = (int *)READADDR; /* 返回值的地址 */ me->value = *r; break; case PORTMAPPED: me->value = inp(SENSORPORT); /* inp() 是编译器特定的端口函数 */ break; }; /* 结束 switch */ return me->value; };

代码清单 4：以困难方式实现多态

1.2.4 子类化

子类化（在统一建模语言[UML] 中称为泛化，或称为继承）是一种表示“某种特殊类型”的另一个类的方式。基本规则是，特化类继承更一般类的特性。也就是说，更一般类（也称为基类或父类）的所有特性也是特化类（也称为派生类或子类）的特性，但后者既可以对更一般的类进行特化，也可以对其进行扩展。

子类化的关键优势之一是能够复用设计和代码。如果我为一个特定上下文设计了一个类，并希望以后在不同的上下文中复用它，我只需设计和编写特化与扩展部分，而无需改动基类。此外，在使用中，我可以编写操作某个类型实例的算法，而不必关心它可能是哪个子类——在合适的时候如此；而在必要时，也可以通过子类修改行为，使其在语义上正确。

所谓“特化”，是指特化类可以重新定义基类提供的操作。由于子类是基类的一种特化形式，因此对基类实例有意义的任何操作，对派生类实例也应当有意义。然而，该操作的实现往往必须略有不同。例如，如果我编写从传感器排队数据的代码，它很可能具有诸如 `insert(intvalue)`、`int remove()`、`int getSize()` 等操作。如果我创建一种特化的队列，能够将大量数据存储到大容量存储设备（如闪存驱动器）中，这些操作仍然有意义，但它们的实现将会不同。请注意，特化仅指更改操作的实现，而完全不涉及数据的重新定义。

Specialization can be easily done with the switch-case statement approach mentioned in the previous section, but it is much more versatile if function pointers are used instead. Then the designer can simply write the specialized functions and create a new constructor that points to the new functions instead of the old. That is, subclasses can override the operations by inserting pointers to different functions that provide specialized behaviors. The downside is that function pointers are a bit tricky and pointers are the leading cause of programmer-induced defects in C programs.

By "extend," I mean that the child class will have new features, such as new attributes or operations. In the case of the data queue, this means that I can add new attributes such as the name of the data file used to store cached data to the flash drive as well as new operations such as flush() and load() operations for writing out the in-memory buffer to flash or reading data from the flash drive into the in-memory buffer, respectively. The UML class diagram in Figure 1-2 shows both the Queue and CachedQueue classes. To simplify the problem, we assume the queue is never misused (so we don't have to handle over and underflow conditions) and we want to store a maximum of 100 integers in the queue[11].

The CachedQueue is conceptually straightforward. If there is room in the insert buffer, then insert the data as with the normal queue. However, if you fill up the internal buffer, then the insert() operation will call flush() to write the insert buffer out to disk, then the internal buffer will be reset. For removing data, it is only slightly more complicated. If the internal buffer is empty, then remove() calls the load() function to read the oldest data off the disk.

Reading the UML Class diagram

The class diagram shown has a number of classic UML class diagram features. The boxes represent the classes while the lines represent relations between the classes. The class box is (optionally) divided into three segments. The upper-most segment holds the name of the class. The middle segment lists the attributes (data members), along with their types. The bottom-most segment shows the operations within the class.

Two different relations are shown in the figure. The line with the closed arrowhead is the generalization relation; the line points to the more base class. The other line is known as composition. The composition is relation that implies strong ownership, and the responsibility for creation and destruction of the instance of the class at the other end of the relation. The open arrowhead depicts navigation (the owner has a pointer to the part in this case, but not the other way around). The name near the arrowhead is the name of the pointer within the CachedQueue class. The number near the arrowhead is the multiplicity – the number of instances needed to play the role (in this case, a single instance is needed).

[11] If you're feeling a little ambitious, feel free to elaborate the example to handle under flow and overflow as well as use C preprocessor macros to be able to change the type and number of elements for the data being queued. Don't worry – I'll wait until you're done.

使用上一节提到的 switch-case 语句方法可以轻松实现特化，但如果改用函数指针，则要灵活得多。这样，设计者只需编写特化函数，并创建一个新的构造函数，使其指向新函数而非旧函数。也就是说，子类可以通过插入指向提供特化行为的不同函数的指针来覆盖这些操作。缺点在于函数指针有些棘手，而指针是 C 程序中程序员引入缺陷的主要原因。

所谓"扩展"，是指子类将具有新的特性，例如新的属性或操作。就数据队列而言，这意味着我可以添加新的属性（例如用于将缓存数据存储到闪存驱动器的数据文件的名称），以及新的操作（例如 flush() 和 load() 操作，分别用于将内存缓冲区写出到闪存，或将数据从闪存驱动器读入内存缓冲区）。图1-2中的UML类图展示了Queue类和CachedQueue类。为简化问题，我们假设队列永远不会被误用（因此我们不必处理上溢和下溢情况），并且我们希望队列中最多存储100个整数[11]。

CachedQueue在概念上很直观。如果插入缓冲区中有空间，则像普通队列一样插入数据。然而，如果内部缓冲区已满，则insert()操作将调用flush()将插入缓冲区写出到磁盘，然后内部缓冲区将被重置。对于移除数据，则稍微复杂一点。如果内部缓冲区为空，则remove()调用load()函数从磁盘读取最旧的数据。

阅读UML类图

所示的类图包含若干经典的UML类图特性。方框代表类，而线条代表类之间的关系。类框（可选地）分为三个部分。最上面的部分包含类的名称。中间部分列出属性（数据成员）及其类型。最下面的部分显示类中的操作。

图中展示了两种不同的关系。带有闭合箭头的线是泛化关系；该线指向更基类的类。另一条线称为组合。组合是一种蕴含强所有权的关系，以及对关系另一端类实例的创建和销毁的责任。开放箭头表示导航（在此情况下，所有者拥有指向部分的指针，但反之则不然）。箭头附近的名称是CachedQueue类中指针的名称。箭头附近的数字是多重性——扮演该角色所需的实例数量（在此情况下，需要一个单一实例）。

[11] 如果你觉得有点雄心勃勃，不妨自由地扩展这个示例，以处理下溢和上溢，并使用C预处理器宏来能够更改排队数据的类型和元素数量。别担心——我会等你完成。

This book will show patterns and examples with class diagrams, even though the primary emphasis will be code representations. Appendix A has a brief overview of UML. If you'd like more detail, see any of a number of books on UML, such as my *Real-Time UML 3rd Edition: Advances in the UML for Real-Time Systems* (Addison-Wesley, 2004).

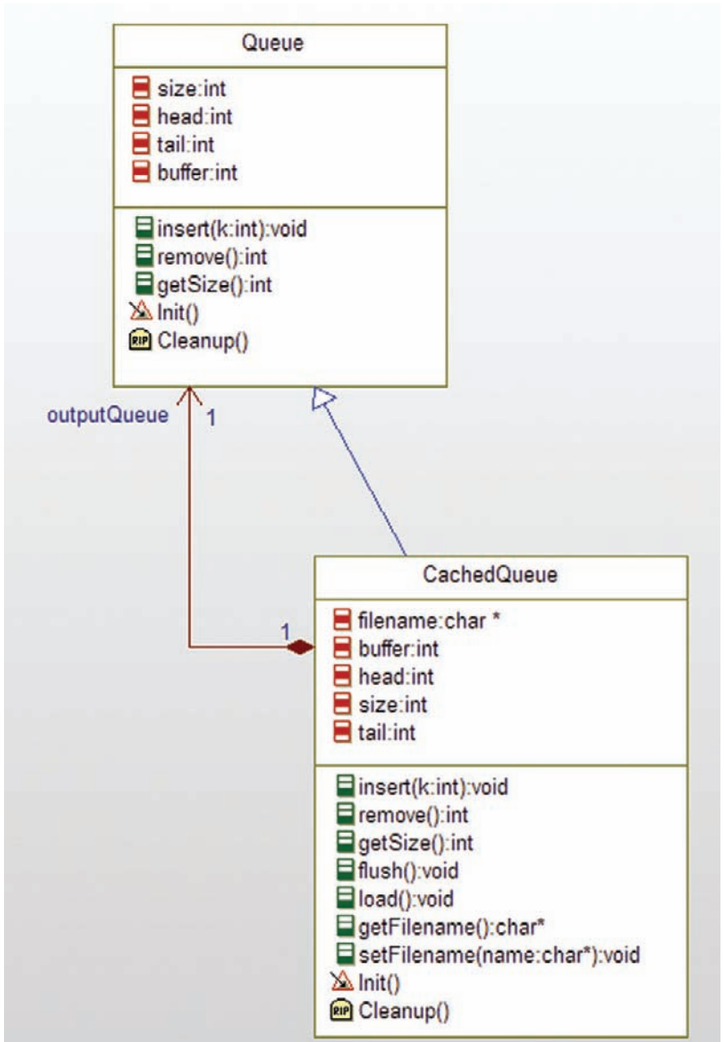


Figure 1.2: Queue and Cached Queue

本书将展示带有类图的模式和示例，尽管主要重点将是代码表示。附录A包含UML的简要概述。如果您需要更多细节，请参阅众多关于UML的书籍中的任何一本，例如我的《Real-Time UML 3rd Edition: Advances in the UML for Real-Time Systems》（Addison-Wesley，2004年）。

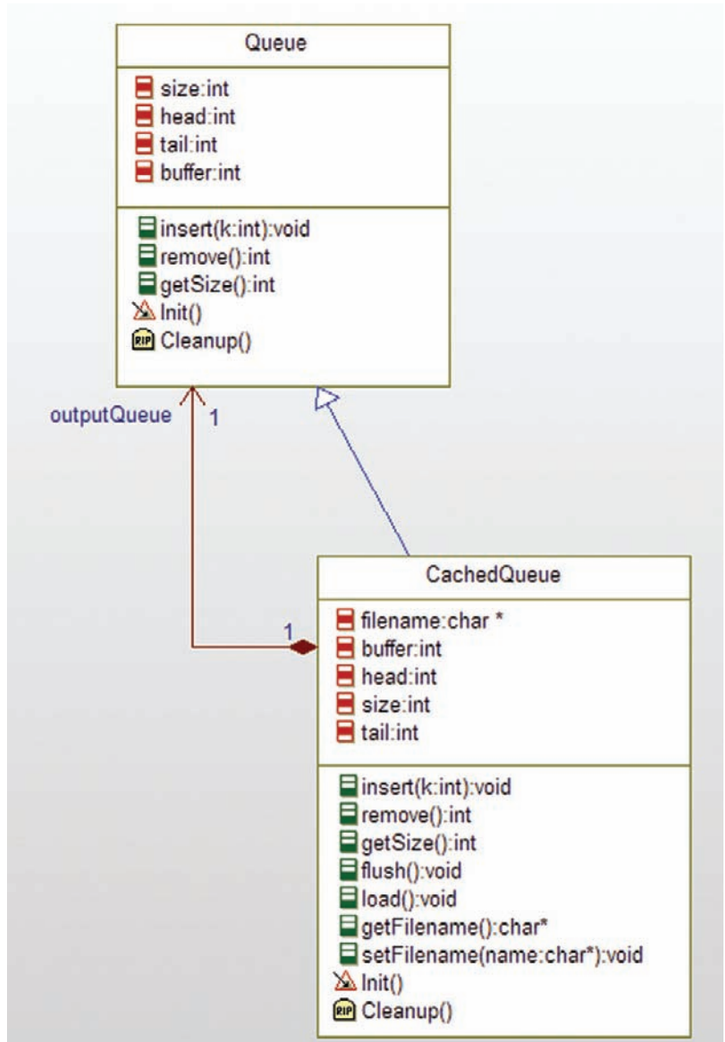


图 1.2：队列和缓存队列

There is certainly more than one way to implement subclassing in C. In the approach used here, to handle specialization, we will employ the member function pointers described in the previous section. For extension, we will simply embed the base class as a struct within the child class.

For example, let's look at the code for the queue example of Figure 1-2. Code Listing 5 is the header file for the Queue class, the structure of which should be pretty much as you expect.

```
#ifndef QUEUE_H_
#define QUEUE_H_

#define QUEUE_SIZE 10

/* class Queue */
typedef struct Queue Queue;
struct Queue {
    int buffer[QUEUE_SIZE];     /* where the data things are */
    int head;
    int size;
    int tail;
    int (*isFull)(Queue* const me);
    int (*isEmpty)(Queue* const me);
    int (*getSize)(Queue* const me);
    void (*insert)(Queue* const me, int k);
    int (*remove)(Queue* const me);
};

/* Constructors and destructors:*/
void Queue_Init(Queue* const me,int (*isFullfunction)(Queue* const me),
        int (*isEmptyfunction)(Queue* const me),
        int (*getSizefunction)(Queue* const me),
        void (*insertfunction)(Queue* const me, int k),
        int (*removefunction)(Queue* const me) );

void Queue_Cleanup(Queue* const me);

/* Operations */
int Queue_isFull(Queue* const me);
int Queue_isEmpty(Queue* const me);
int Queue_getSize(Queue* const me);
void Queue_insert(Queue* const me, int k);
int Queue_remove(Queue* const me);

Queue * Queue_Create(void);
void Queue_Destroy(Queue* const me);

#endif /*QUEUE_H_*/
```

Code Listing 5: Queue Header

Code Listing 6 shows the (simple) implementation of the Queue operations and Code Listing 7 provides a simple little test program that shows elements inserted and removed into and from the queue.

在C语言中实现子类化肯定不止一种方法。在这里使用的方法中，为了处理特化，我们将采用上一节中描述的成员函数指针。对于扩展，我们将简单地将基类作为结构体嵌入到子类中。

例如，让我们看一下图1-2中队列示例的代码。代码清单5是Queue类的头文件，其结构应该与您预期的基本一致。

```
#ifndef QUEUE_H_
#define QUEUE_H_

#define QUEUE_SIZE 10

/* 类 Queue */
typedef struct Queue Queue;
struct Queue {
    int buffer[QUEUE_size];          /* 数据内容所在的位置 */
    int head;
    int size;
    int tail;
    int (*isFull)(Queue* const me);
    int (*isEmpty)(Queue* const me);
    int (*getSize)(Queue* const me);
    void (*insert)(Queue* const me      , int k);
    int (*remove)(Queue* const me);
};

/* 构造函数与析构函数：*/ void Queue_Init(Queue* const me,int
(*isFullfunction)(Queue* const me), int (*isEmptyfunction)(Queue* const me), int
(*getSizefunction)(Queue* const me), void (*insertfunction)(Queue* const me, int k),
int (*removefunction)(Queue* const me) );

void Queue_Cleanup(Queue* const me);

/* 操作 */
int Queue_isFull(Queue* const me); int Queue_
isEmpty(Queue* const me); int Queue_
getSize(Queue* const me); void Queue_插入(Queue*
const me, int k); int Queue_移除(Queue* const me);

Queue * Queue_Create(void); void Queue_
Destroy(Queue* const me);

#endif /*QUEUE_H_*/
```

代码清单5：队列头文件

代码清单 6 展示了队列操作的（简单）实现，代码清单 7 提供了一个简单的测试程序，展示了向队列中插入元素以及从队列中移除元素的过程。

```
#include <stdio.h>
#include <stdlib.h>
#include "queue.h"

void Queue_Init(Queue* const me,int (*isFullfunction)(Queue* const me),
           int (*isEmptyfunction)(Queue* const me),
           int (*getSizefunction)(Queue* const me),
           void (*insertfunction)(Queue* const me, int k),
           int (*removefunction)(Queue* const me) ){
       /* initialize attributes */
    me->head = 0;
    me->size = 0;
    me->tail = 0;
    /* initialize member function pointers */
    me->isFull = isFullfunction;
    me->isEmpty = isEmptyfunction;
    me->getSize = getSizefunction;
    me->insert = insertfunction;
    me->remove = removefunction;
}
/* operation Cleanup() */
void Queue_Cleanup(Queue* const me) {
}

/* operation isFull() */
int Queue_isFull(Queue* const me){
    return (me->head+1) % QUEUE_SIZE == me->tail;
}

/* operation isEmpty() */
int Queue_isEmpty(Queue* const me){
    return (me->head == me->tail);
}
/* operation getSize() */
int Queue_getSize(Queue* const me) {
    return me->size;
}
/* operation insert(int) */
void Queue_insert(Queue* const me, int k) {
    if (!me->isFull(me)) {
          me->buffer[me->head] = k;
          me->head = (me->head+1) % QUEUE_SIZE;
          ++me->size;
    }
}

/* operation remove */
int Queue_remove(Queue* const me) {
    int value = -9999; /* sentinel value */
    if (!me->isEmpty(me)) {
          value = me->buffer[me->tail];
          me->tail = (me->tail+1) % QUEUE_SIZE;
          -me->size;
```


#include <stdio.h>#include <stdlib.h>#include "queue.h"
void Queue_Init(Queue* const me,int (*isFullfunct

ion)(Queue* const me),
int (*isEmptyfunction)(Queue* const me
int (*getSizefunction)(Queue* const me
void (*insertfunction)(Queue* const me
int (*removefunction)(Queue* const me)
),
),
, int k),
){

/* 初始化属性 */ me->head = 0; me->size = 0; me->tail = 0; /* 初始化成员函数指针 */ me->isFull = isFullfunction; me->isEmpty = isEmptyfunction; me->getSize = getSizefunction; me->insert = insertfunction; me->remove = removefunction; } /* 操作 Cleanup() */ void Queue_Cleanup(Queue* const me) { } /* 操作 isFull() */ int Queue_isFull(Queue* const me){ return (me->head+1) % QUEUE_SIZE == me->tail; } /* 操作 isEmpty() */ int Queue_isEmpty(Queue* const me){ return (me->head == me->tail); } /* 操作 getSize() */ int Queue_getSize(Queue* const me) { return me->size; } /* 操作 insert(int) */ void Queue_insert(Queue* const me, int k) { if (!me->isFull(me)) { me->buffer[me->head] = k; me->head = (me->head+1) % QUEUE_SIZE;++me->size; } } /* 操作 remove */ int Queue_remove(Queue* const me) { int value = -9999; /* 哨兵值 */

if (!me->isEmpty(me)) { value = me->buffer[me->tail]; me->tail = (me->tail+1) % QUEUE_SIZE; –me->size;

```
    }
    return value;
}

Queue * Queue_Create(void) {
    Queue* me = (Queue *) malloc(sizeof(Queue));
    if(me!=NULL)
        {
            Queue_Init(me, Queue_isFull, Queue_isEmpty, Queue_getSize,
            Queue_insert, Queue_remove);
        }
    return me;
}

void Queue_Destroy(Queue* const me) {
    if(me!=NULL)
        {
            Queue_Cleanup(me);
        }
    free(me);
}
```

Code Listing 6: Queue Implementation

```
#include <stdio.h>
#include <stdlib.h>
#include "queue.h"

int main(void) {
    int j,k, h, t;

    /* test normal queue */
    Queue * myQ;
    myQ = Queue_Create();
    k = 1000;

    for (j=0;j<QUEUE_SIZE;j++) {
        h = myQ->head;
        myQ->insert(myQ,k);
        printf("inserting %d at position %d, size =%d\n",k--,h, myQ->getSize(myQ));
    };
    printf("Inserted %d elements\n",myQ->getSize(myQ));
    for (j=0;j<QUEUE_SIZE;j++) {
        t = myQ->tail;
        k = myQ->remove(myQ);
        printf("REMOVING %d at position %d, size =%d\n",k,t, myQ->getSize(myQ);
    };

    printf("Last item removed = %d\n", k);
    printf("Current queue size %d\n", myQ->getSize(myQ));
puts("Queue test program");

    return EXIT_SUCCESS;
}
```

Code Listing 7: Queue Test Program

} return value; } Queue * Queue_Create(void) { Queue* me = (Queue *) malloc(sizeof(Queue)); if(me!=空) { Queue_Init(me, Queue_isFull, Queue_isEmpty, Queue_getSize, Queue_插入, Queue_移除); } return me; } void Queue_Destroy(Queue* const me) { if(me!=空) { Queue_清理(me); } free(me); }

代码清单 6：队列实现

#include <stdio.h>#include <stdlib.h>#include "queue.h" int main(void) { int j,k, h, t; /* 测试普通队列 */ Queue * myQ; myQ = Queue_Create(); k = 1000; for (j=0;j<QUEUE_SIZE;j++) { h = myQ->head; myQ->insert(myQ,k); printf("inserting %d at position %d, size =%d\n",k--,h, myQ->getSize(myQ)); }; printf("Inserted %d elements\n",myQ->getSize(myQ)); for (j=0;j<QUEUE_SIZE;j++) { t = myQ->tail; k = myQ->remove(myQ); printf("REMOVING %d at position %d, size =%d\n",k,t, myQ->getSize(myQ); }; printf("Last item removed = %d\n", k); printf("Current queue size %d\n", myQ->getSize(myQ)); puts("队列测试程序"); return EXIT_SUCCESS; }

代码清单 7：队列测试程序

Now let us suppose that memory is very tight and so we need to reduce the size of the in-memory buffer and store most elements to disk. Our CachedQueue fits the bill.

Code Listing 8 shows the header file for the class. You can see some important aspects of the definition of the class. First, note that the base class is retained in the subclass as an attribute named queue. This brings in the operations and attributes from the base class within the subclass. Secondly, note that the operations defined for the base class are replicated as member function pointers in the subclass. In that way, they can be directly invoked on the subclass instance. Each such virtual function can then either deal entirely with the request or deal with some aspect of it and delegate the rest to the original function defined on the Queue class (still held within its virtual functions). Lastly, note that the aggregate `outputQueue` is also an attribute of the subclass. This allows the CachedQueue to manage two in-memory buffers. The "normal" one (represented by the base class) is where new data gets inserted. The other one (represented by the aggregated `outputQueue`) is where the data comes from for a remove.

```
#ifndef CACHEDQUEUE_H_
#define CACHEDQUEUE_H_

#include "queue.h"

typedef struct CachedQueue CachedQueue;
struct CachedQueue {
    Queue* queue;        /* base class */
    /* new attributes */
    char filename[80];

    int numberElementsOnDisk;
    /* aggregation in subclass */
    Queue* outputQueue;

    /* inherited virtual functions */
    int (*isFull)(CachedQueue* const me);
    int (*isEmpty)(CachedQueue* const me);
    int (*getSize)(CachedQueue* const me);
    void (*insert)(CachedQueue* const me, int k);
    int (*remove)(CachedQueue* const me);
    /* new virtual functions */
    void (*flush)(CachedQueue* const me);
    void (*load)(CachedQueue* const me);
};

/* Constructors and destructors:*/
void CachedQueue_Init(CachedQueue* const me, char* fName,
       int (*isFullfunction)(CachedQueue* const me),
       int (*isEmptyfunction)(CachedQueue* const me),
```


现在假设内存非常紧张，因此我们需要减小内存缓冲区的大小，并将大多数元素存储到磁盘上。我们的 CachedQueue 正好满足这一需求。

代码清单8展示了该类的头文件。你可以看到该类定义的一些重要方面。首先，注意基类在子类中保留为一个名为queue的属性。这将基类的操作和属性引入到子类中。其次，注意为基类定义的操作在子类中被复制为成员函数指针。这样，它们就可以直接在子类实例上调用。每个这样的虚函数随后可以要么完全处理该请求，要么处理其某个方面并将剩余部分委托给在Queue类上定义的原始函数（仍保留在其虚函数中）。最后，注意聚合的`outputQueue` 也是子类的一个属性。这允许CachedQueue管理两个内存缓冲区。"正常的"那个（由基类表示）是新数据插入的地方。另一个（由聚合的`outputQueue`表示）是移除操作获取数据的地方。

```
#ifndef CACHEDQUEUE_H_
#define CACHEDQUEUE_H_

#include "queue.h"

typedef struct CachedQueue CachedQueue;
struct CachedQueue { Queue* queue; /* 基类
*/ /* 新属性 */ char filename[80];

    int numberElementsOnDisk; /* 子类
    中的聚合 */ Queue* outputQueue;

    /* 继承的虚函数 */ int (*isFull)(CachedQueue* const
me); int (*isEmpty)(CachedQueue* const me); int
(*getSize)(CachedQueue* const me); void
(*insert)(CachedQueue* const me, int k); int
(*remove)(CachedQueue* const me); /* 新增虚函数 */
void (*flush)(CachedQueue* const me); void
(*load)(CachedQueue* const me); };

/* 构造函数与析构函数：*/ void CachedQueue_Init(CachedQueue*
const me, char* fName, int (*isFullfunction)(CachedQueue* const
me), int (*isEmptyfunction)(CachedQueue* const me),
```

```
        int (*getSizefunction)(CachedQueue* const me),
        void (*insertfunction)(CachedQueue* const me, int k),
        int (*removefunction)(CachedQueue* const me),
        void (*flushfunction)(CachedQueue* const me),
        void (*loadfunction)(CachedQueue* const me));

void CachedQueue_Cleanup(CachedQueue* const me);

/* Operations */
int CachedQueue_isFull(CachedQueue* const me);
int CachedQueue_isEmpty(CachedQueue* const me);
int CachedQueue_getSize(CachedQueue* const me);
void CachedQueue_insert(CachedQueue* const me, int k);
int CachedQueue_remove(CachedQueue* const me);
void CachedQueue_flush(CachedQueue* const me);
void CachedQueue_load(CachedQueue* const me);

CachedQueue * CachedQueue_Create(void);
void CachedQueue_Destroy(CachedQueue* const me);
#endif /*CACHEDQUEUE_H_*/
```

Code Listing 8: CachedQueue header file

Code Listing 9 shows the implementation (sans file I/O, just to simplify the example a bit). You can see how the `CachedQueue_Init()` function constructs the subclass instance (with a call to `Queue_Create()`) and then sets up its own attributes and virtual member functions. You can also see how the `CachedQueue_getSize()` function computes the number of elements held: it is the sum of the data held in the `queue`, the `outputQueue`, and the number of elements stored on disk. While the implementation is not quite up to shippable standards (I'd want to add error handling, for example), it does illustrate one way to create classes and instances of those classes in the C language.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "cachedQueue.h"

void CachedQueue_Init(CachedQueue* const me, char* fName,
          int (*isFullfunction)(CachedQueue* const me),
          int (*isEmptyfunction)(CachedQueue* const me),
          int (*getSizefunction)(CachedQueue* const me),
          void (*insertfunction)(CachedQueue* const me, int k),
          int (*removefunction)(CachedQueue* const me),
          void (*flushfunction)(CachedQueue* const me),
          void (*loadfunction)(CachedQueue* const me)) {

    /* initialize base class */
        me->queue = Queue_Create(); /* queue member must use its original functions */

    /* initialize subclass attributes */
        me->numberElementsOnDisk = 0;
        strcpy(me->filename, fName);
```



```
        int (*getSizefunction)(CachedQueue* const me), void
        (*insertfunction)(CachedQueue* const me, int k), int
        (*removefunction)(CachedQueue* const me), void
        (*flushfunction)(CachedQueue* const me), void
        (*loadfunction)(CachedQueue* const me));

void CachedQueue_Cleanup(CachedQueue* const me);

/* 操作 */
int CachedQueue_isFull(CachedQueue* const me); int CachedQueue_
isEmpty(CachedQueue* const me); int CachedQueue_
getSize(CachedQueue* const me); void CachedQueue_插入
(CachedQueue* const me, int k); int CachedQueue_移除(
CachedQueue* const me); void CachedQueue_刷新(CachedQueue*
const me); void CachedQueue_load(CachedQueue* const me);


CachedQueue * CachedQueue_Create(void); void
CachedQueue_Destroy(CachedQueue* const me); #endif /
*CACHEDQUEUE_H_*/
```

代码清单8：CachedQueue 头文件

代码清单9展示了其实现（为简化示例，省略了文件输入输出）。你可以看到 `CachedQueue_Init()` 函数如何构造子类实例（通过调用`Queue_Create()`），然后设置其自身的属性和虚成员函数。你还可以看到 `CachedQueue_getSize()` 函数如何计算所持有的元素数量：它是 队列、`outputQueue` 中持有的数据以及存储在磁盘上的元素数量之和。虽然该实现尚未达到可发布标准（例如，我希望添加错误处理），但它确实展示了在 C 语言中创建类及其实例的一种方式。

```
#include <stdio.h>#include <
stdlib.h>#include <string.h>#
include "cachedQueue.h"


void CachedQueue_Init(CachedQueue* const me, char* fName, int
(*isFullfunction)(CachedQueue* const me), int (*isEmptyfunction)(CachedQueue*
const me), int (*getSizefunction)(CachedQueue* const me), void
(*insertfunction)(CachedQueue* const me, int k), int
(*removefunction)(CachedQueue* const me), void (*flushfunction)(CachedQueue*
const me), void (*loadfunction)(CachedQueue* const me)) {



    /* 初始化基类 */
        me->queue = Queue_Create(); /* 队列成员必须使用其原始函数 *                                /

    /* 初始化子类属性 */ me->
    numberElementsOnDisk = 0; strcpy(me->
    filename, fName);
```

```
        /* initialize aggregates */
    me->outputQueue = Queue_Create();

    /* initialize subclass virtual operations ptrs */
        me->isFull = isFullfunction;
        me->isEmpty = isEmptyfunction;
        me->getSize = getSizefunction;
        me->insert = insertfunction;
        me->remove = removefunction;
    me->flush = flushfunction;
    me->load = loadfunction;
}

/* operation Cleanup() */
void CachedQueue_Cleanup(CachedQueue* const me) {
        Queue_Cleanup(me->queue);
}

/* operation isFull() */
int CachedQueue_isFull(CachedQueue* const me){
    return    me->queue->isFull(me->queue) &&
              me->outputQueue->isFull(me->outputQueue);
}

/* operation isEmpty() */
int CachedQueue_isEmpty(CachedQueue* const me){
    return    me->queue->isEmpty(me->queue) &&
              me->outputQueue->isEmpty(me->outputQueue) &&
              (me->numberElementsOnDisk == 0);
}

/* operation getSize() */
int CachedQueue_getSize(CachedQueue* const me) {
    return    me->queue->getSize(me->queue) +
              me->outputQueue->getSize(me->outputQueue) +
              me->numberElementsOnDisk;
}

/* operation insert(int) */
// Imsert Algorithm:
// if the queue is full,
//        call flush to write out the queue to disk and reset the queue
// end if
// insert the data into the queue
void CachedQueue_insert(CachedQueue* const me, int k) {
    if (me->queue->isFull(me->queue)) me->flush(me);
    me->queue->insert(me->queue, k);
}

/* operation remove */
// remove algorithm
// if there is data in the outputQueue,
//        remove it from the outputQueue
```


/* 初始化聚合对象 */ me->outputQueue = Queue_Create(); /* 初始化子类虚操作指针 */ me->isFull = isFullfunction; me->isEmpty = isEmptyfunction; me->getSize = getSizefunction; me->insert = insertfunction; me->remove = removefunction; me->flush = flushfunction; me->load = loadfunction; } /* 操作 Cleanup() */ void CachedQueue_Cleanup(CachedQueue* const me) { Queue_Cleanup(me->queue); } /* 操作 isFull() */ int CachedQueue_isFull(CachedQueue* const me){ return me->queue->isFull(me->queue) && me->outputQueue->isFull(me->outputQueue); } /* 操作 isEmpty() */ int CachedQueue_isEmpty(CachedQueue* const me){ return me->queue->isEmpty(me->queue) && me->outputQueue->isEmpty(me->outputQueue) && (me->numberElementsOnDisk == 0); } /* 操作 getSize() */ int CachedQueue_getSize(CachedQueue* const me) { return me->queue->getSize(me->queue) +me->outputQueue->getSize(me->outputQueue) +me->numberElementsOnDisk; } /* 操作 insert(int) */ // Imsert算法： // 如果队列已满， // 调用flush将队列写出到磁盘并重置

队列

// end if // 将数据插入队列 void CachedQueue_insert(CachedQueue* const me, int k) { if (me->queue->isFull(me->queue)) me->flush(me); me->queue->insert(me->queue, k); } /* 操作 remove */ // remove 算法 // 如果 outputQueue 中有数据， // 则从 outputQueue 中移除它

```
// else if there is data on disk
//        call load to bring it into the outputQueue
//        remove it from the outputQueue
// else if there is data in the queue
//        remove it from there
//        (if there is no data to remove then return sentinel value)
int CachedQueue_remove(CachedQueue* const me) {
    if (!me->outputQueue->isEmpty(me->outputQueue))
        return me->outputQueue->remove(me->outputQueue);
    else if (me->numberElementsOnDisk>0) {
        me->load(me);
        return me->queue->remove(me->queue);
    }
    else
        return me->queue->remove(me->queue);
}
/* operation flush */
// Precondition: this is called only when queue is full
//        and filename is valid
// flush algorithm
// if file is not open, then open file
// while not queue->isEmpty()
//        queue->remove()
//        write data to disk
//        numberElementsOnDisk++
// end while
void CachedQueue_flush(CachedQueue* const me){
    // write file I/O statements here ...

}

/* operation load */
// Precondition: this is called only when outputQueue is empty
//    and filename is valid
// load algorithm
// while (!outputQueue->isFull() && (numberElementsOnDisk>0)
//        read from start of file (i.e., oldest datum)
//        numberElementsOnDisk--;
//        outputQueue->insert()
// end while
void CachedQueue_load(CachedQueue* const me) {
    // write file I/O statements here ...

}

CachedQueue * CachedQueue_Create(void) {
    CachedQueue* me = (CachedQueue *)
malloc(sizeof(CachedQueue));
    if(me!=NULL)
        {
            CachedQueue_Init(me, "C:\\queuebuffer.dat",
                        CachedQueue_isFull, CachedQueue_isEmpty,                                                    y,
```


// 否则如果磁盘上有数据 // 调用 load 将其加载到 outputQueue 中 // 从 outputQueue 中移除它 // 否则如果队列中有数据 // 则从那里移除它 // （如果没有数据可移除，则返回哨兵值

e)

int CachedQueue_remove(CachedQueue* const me) { if (!me-> outputQueue->isEmpty(me->outputQueue)) return me->outputQueue-> remove(me->outputQueue); else if (me->numberElementsOnDisk>0) { me-> load(me); return me->queue->remove(me->queue); } else return me-> queue->remove(me->queue); } /* 操作 flush */ // 前置条件：仅在队列已满 // 且 filename 有效时调用 // flush 算法 // 如果文件未打开，则打开文件 // while not queue->isEmpty() // queue->remove() // 将数据写入磁盘 // numberElementsOnDisk++// end while void CachedQueue_ flush(CachedQueue* const me){ // 在此处编写文件输入输出语句 . .. } /* 操作 load */ // 前置条件：仅在 outputQueue 为空 // 且 filename 有效时调用 // load 算法 // while (!outputQueue->isFull() && (numberElementsOnDisk>0) // 从文件开头读取（即最旧的数据） // numberElementsOnDisk– –; // outputQueue-> insert() // end while void CachedQueue_load(CachedQueue* const me) { // 在此处编写文件输入输出语句 . .. } CachedQueue * CachedQueue_ Create(void) { CachedQueue* me = (CachedQueue *) malloc(sizeof(CachedQueue)); if(me!=NULL) { CachedQueue_Init(me, " C:\\queuebuffer.dat", CachedQueue_isFull, CachedQueue_isEmpt

```
                    CachedQueue_getSize, CachedQueue_insert, CachedQueue_remove,
                    CachedQueue_flush, CachedQueue_load);
        }

    return me;
}
void CachedQueue_Destroy(CachedQueue* const me) {
    if(me!=NULL)
        {
            CachedQueue_Cleanup(me);
        }
    free(me);
}
```

Code Listing 9: CachedQueue Implementation

The capability to use inheritance (subclassing) will be useful in some of the design patterns I will discuss later in this book, besides being of value all on its own.

1.2.5 Finite State Machines

A finite state machine (FSM) is a machine specified by a finite set of conditions of existence (called "states") and a likewise finite set of transitions among the states triggered by events. An FSM differs from an activity diagram or flow chart in that the transitions are triggered by events (primarily) rather than being triggered when the work done in the previous state is complete. Statecharts are primarily used to model the behavior of *reactive* elements, such as classes and use cases, that wait in a state until an event of interest occurs. At that point, the event is processed, actions are performed, and the element transitions to a new state.

Actions, such as the execution of a primitive C language statement or the invocation of an operation, may be specified to be executed when a state is entered or exited, or when a transition is taken. The order of execution of actions is exit actions of the predecessor state, followed by the transition actions, followed by the entry actions of the subsequent state.

The UML uses statecharts as their formal FSM representation, which are significantly more expressive and scalable than "classical" Mealy-Moore FSMs. UML state machines, based on Dr. David Harel's statechart semantics and notation, have a number of extensions beyond Mealy-Moore state machines, including:

- Nested states for specifying hierarchical state membership
- AND-states for specifying logical independence and concurrency
- Pseudostates for annotating commonly-needed specific dynamic semantics

Figure 1-3 shows some of the basic elements of a statechart for the SecuritySupervisor class shown previously in Figure 1-1. It includes basic OR-states and transitions, as well as a few less-elementary concepts, including nested states and conditional and initial pseudostates.

CachedQueue_getSize, CachedQueue_insert, CachedQueue_remove, CachedQueue_flush, CachedQueue_load); } return me; } void CachedQueue_Destroy(CachedQueue* const me) { if(me!= NULL) { CachedQueue_Cleanup(me); } free(me); }

代码清单9：CachedQueue实现

使用继承（子类化）的能力将在本书后面讨论的一些设计模式中很有用，而且它本身也具有价值。

1.2.5 有限状态机

有限状态机 (FSM) 是一种由有限的存在条件集合（称为"状态"）以及同样有限的、由事件触发的状态间转换集合所指定的机器。FSM 与活动图或流程图的不同之处在于，转换（主要）由事件触发，而不是在上一状态中完成的工作结束时触发。状态图主要用于对反应式元素（如类和用例）的行为进行建模，这些元素在某个状态中等待，直到发生感兴趣的事件。此时，事件被处理，动作被执行，并且该元素转换到新状态。

动作（例如执行 C语言基本语句或操作调用）可以被指定在进入或退出某个状态时执行，或者在执行某个转换时执行。动作的执行顺序是：前驱状态的退出动作，然后是转换动作，然后是后继状态的进入动作。

UML使用状态图作为其形式化的有限状态机表示，这比"经典"的Mealy-Moore有限状态机更具表达力和可扩展性。UML状态机基于David Harel博士的状态图语义和表示法，在Mealy-Moore状态机之外有许多扩展，包括：

- 用于指定层次状态成员关系的嵌套状态
- 用于指定逻辑独立性和并发的AND状态
- 用于标注常见特定动态语义的伪状态

图1-3展示了之前在图1-1中所示的SecuritySupervisor类的状态图的一些基本元素。它包括基本的或状态和迁移，以及一些不太基础的概念，包括嵌套状态、条件伪状态和初始伪状态。

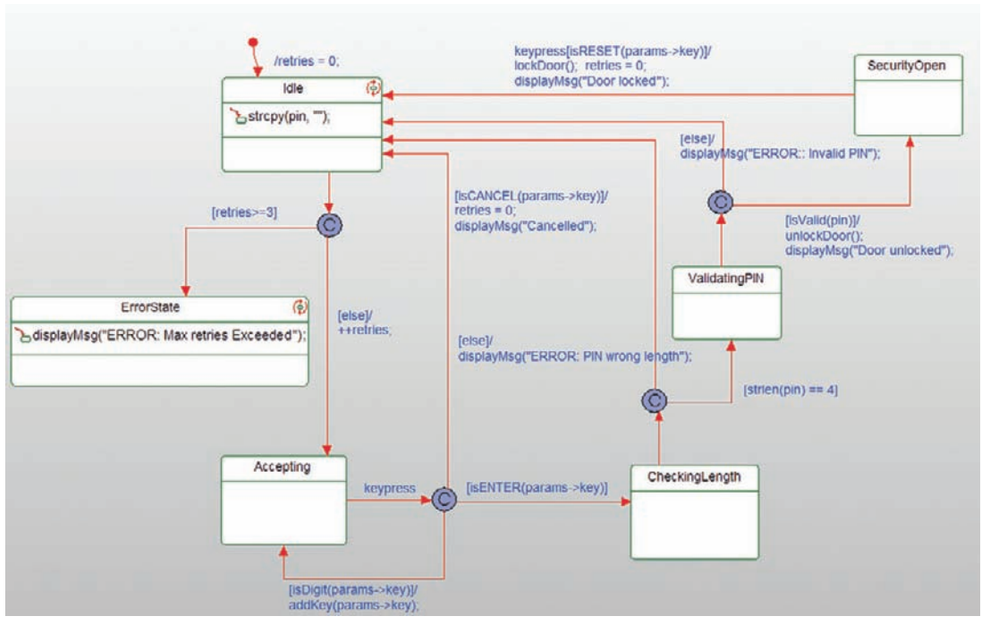


Figure 1-3: Basic State Machine

Transitions are arrowed lines coming from a predecessor state and terminating on a subsequent state. Transitions usually have the optional *event signature* and *action list*. The basic form of an event signature is

event-name '(' parameter-list ')' '[' guard ']' '/' action-list

The event-name is simply the logical name of the event class that may be sent to an instance of the Classifier at run-time, such as `keypress` in Figure 1-3. The UML defines four distinct kinds of events that may be passed or handled:

- SignalEvent – an asynchronously sent event
- CallEvent – a synchronously sent event
- TimeEvent – an event due to the passage of an interval of time (most common) or arrival of an epoch
- ChangeEvent – a change in a state variable or attribute of the Classifier

Asynchronous event transfer is always implemented via queuing of the event until the element is ready to process it. That is, the sender "sends and forgets" the event and goes on about its business, ignorant of whether or not the event has been processed. Synchronous event transfer executes the state processing of the event in the thread of the sender, with the sender blocked from continuing until that state processing is complete. This is commonly implemented by invoking a

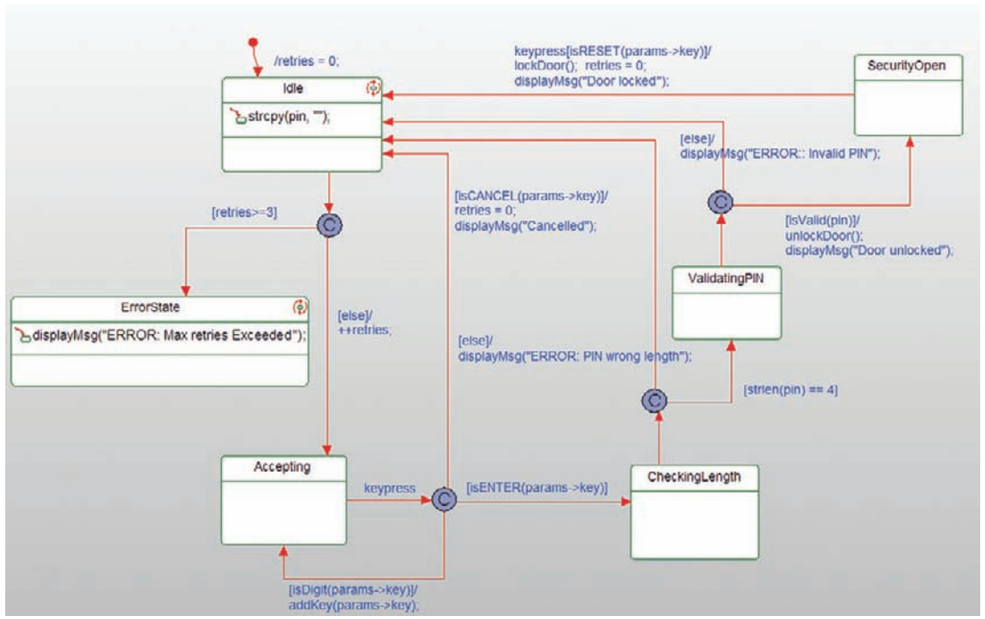


图 1-3：基本状态机

迁移是从前驱状态出发并终止于后继状态的带箭头的线。迁移通常具有可选的事件签名和动作列表。事件签名的基本形式是

事件名 '(' 参数列表 ')' '[' 守卫条件 ']' '/' 动作列表

事件名就是事件类的逻辑名称，该事件类可在运行时发送给分类器的实例，例如图 1-3 中的按键。UML 定义了四种可被传递或处理的不同类型的事件：

- 信号事件 – 异步发送的事件
- 调用事件 – 同步发送的事件
- 时间事件——由于经过一段时间间隔（最常见）或纪元到达而引起的事件
- 变更事件——分类器的状态变量或属性发生变化

异步事件传输始终通过将事件排队来实现，直到该元素准备好处理它。也就是说，发送者"发送后即遗忘"该事件，继续自己的事务，而不知道事件是否已被处理。同步事件传输在发送者的线程中执行事件的状态处理，发送者被阻塞，直到该状态处理完成才能继续。这通常通过调用一个

class method called an *event handler* that executes the relevant part of the state machine, returning control to the sender only when the event processing is complete. Such "triggered operations" don't have a standard method body; their method body is the action list on the state machine.

Events may have parameters – typed values accepted by the state machine which may then be used in the guard and actions in the processing of the event. The statechart specifies the formal parameter list, while the object that sends the event must provide the necessary actual parameters to bind to the formal parameter list. We will use a slightly peculiar syntax for passing events. We will create a struct named *params* that contains the named parameters for every event that carries data. If an event *e* has two parameters, x and y, to use these in a guard, for example, you would dereference the params struct to access their value. So a transition triggered by event *e* with a guard that specified that x must be greater than 0 and y must be less than or equal to 10 for the transition to be taken would look like:

```
e[params->x>0 && params->y<=10]
```

Time events are almost always relative to the entry to a state. A common way to name such an event (and what we will use here) is "tm(interval)," where "interval" is the time interval parameter for the timeout event[12]. If the timeout occurs before another specified event occurs, then the transition triggered by the timeout event will be taken; if another event is sent to the object prior to the triggering of the timeout, then the time is discarded; if the state is reentered, the timeout interval is started over from the beginning.

If a transition does not provide a named event trigger, then this is activated by the "completion" or "null" event. This event occurs either as soon as the state is entered (which includes the execution of entry actions for the state) or when the state activities complete.

A guard is a Boolean expression contained within square brackets that follows the event trigger. The guard should return only TRUE or FALSE and not have side effects. If a guard is specified for a transition and the event trigger (if any) occurs, then the transition will be taken if and only if the guard evaluates to TRUE. If the guard evaluates to FALSE, then the triggering event is quietly discarded and no actions are executed.

The action list for the transition is executed if and only if the transition is taken; that is, the named event is received by the object while it is in the predecessor state and the guard, if any, evaluates to TRUE. The entire set of exit actions–transition actions–entry actions is executed in that order and is executed using run-to-completion semantics. Figure 1-3 shows actions on a number of different transitions as well as on entry into some of the states.

A run of the three "classes" from Figure 1-1 using test1 from the TestDriver (which sends '1', '2', '3', '4', 'e' (the code for the ENTER key), followed by 'r' (the code for the RESET key) is shown in Figure 1-4 in a UML sequence diagram.

[12] Another common way is to use the term "after(interval)."

称为事件处理器的类方法来实现，该方法执行状态机的相关部分，仅在事件处理完成后才将控制权返回给发送者。这种"触发操作"没有标准的方法体；其方法体是状态机上的动作列表。

事件可以带有参数——即状态机所接受的类型化值，这些值随后可在事件处理过程中的守卫条件和动作中使用。状态图规定了形式参数列表，而发送事件的对象必须提供必要的实际参数以绑定到形式参数列表。我们将使用一种略显特殊的语法来传递事件。我们将创建一个名为 params 的结构体，其中包含每个携带数据的事件的命名参数。例如，如果事件 e 有两个参数 x 和 y，要在守卫条件中使用它们，就需要解引用 params 结构体来访问它们的值。因此，由事件 e 触发、且守卫条件规定 x 必须大于 0 且 y 必须小于或等于 10 才能进行迁移的迁移，将如下所示：

```
e[params->x>0 && params->y<=10]
```

时间事件几乎总是相对于进入某个状态而言的。命名此类事件的一种常见方式（也是我们在此将采用的方式）是"tm(interval)"，其中"interval"是超时事件[12]的时间间隔参数。如果超时在另一个指定事件发生之前发生，则由超时事件触发的迁移将被执行；如果在超时触发之前有另一个事件被发送给该对象，则时间被丢弃；如果重新进入该状态，则超时间隔从头开始重新计时。

如果迁移未提供命名的事件触发器，则该迁移由"完成"或"空"事件激活。此事件要么在进入状态时立即发生（包括执行该状态的进入动作），要么在状态活动完成时发生。

守卫条件是包含在方括号内的布尔表达式，位于事件触发器之后。守卫条件应仅返回 TRUE 或 FALSE，且不应产生副作用。如果为迁移指定了守卫条件，并且事件触发器（如果有）发生，则当且仅当守卫条件求值为 TRUE 时，该迁移才会被执行。如果守卫条件求值为 FALSE，则触发事件被静默丢弃，且不执行任何动作。

迁移的动作列表当且仅当迁移被执行时才会执行；也就是说，对象在前驱状态时接收到命名事件，并且守卫条件（如果有）求值为 TRUE。整个退出动作–转换动作–进入动作集合按该顺序执行，并使用运行至完成语义执行。图 1-3 展示了多个不同迁移上的动作，以及进入某些状态时的动作。

使用TestDriver中的test1（它依次发送'1'、'2'、'3'、'4'、'e'（ENTER键的代码），随后发送'r'（RESET键的代码））对图1-1中的三个"类"进行的一次运行，以UML顺序图的形式展示在图1-4中。

[12] 另一种常见的方式是使用术语"after(interval)"。

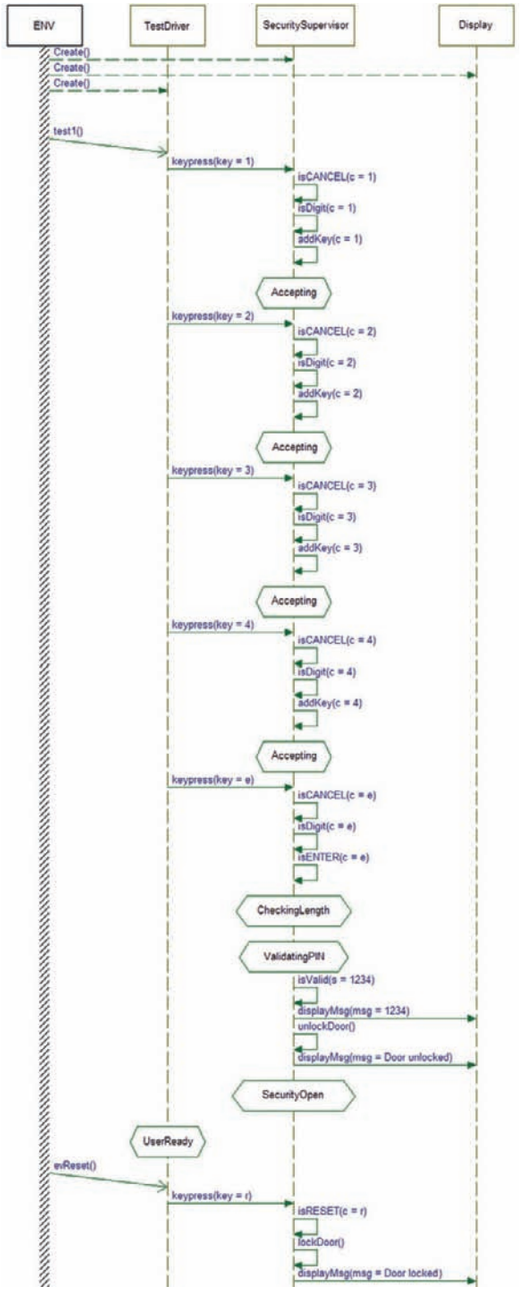


Figure 1-4: Running the state behavior

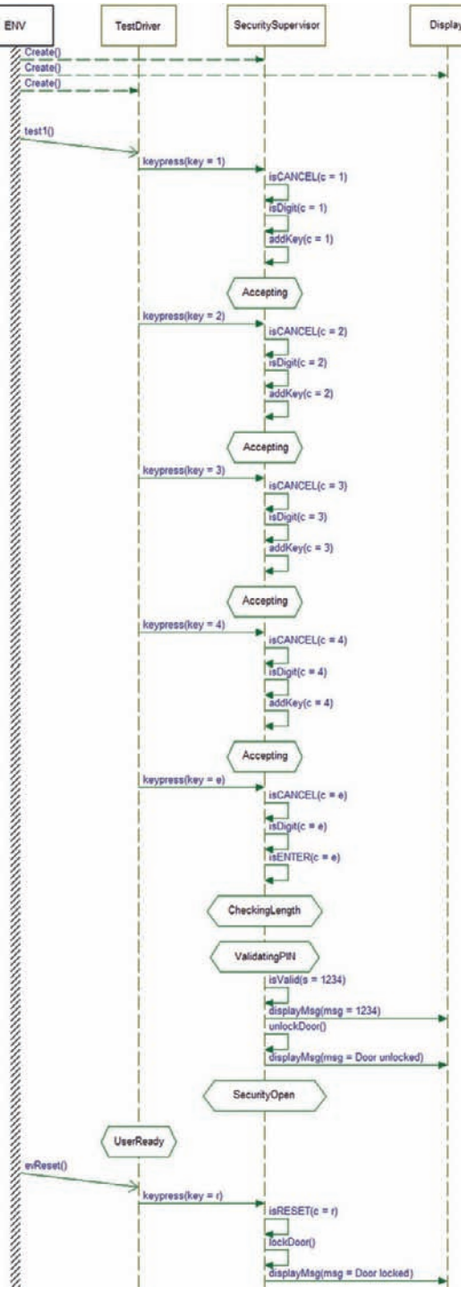


图1-4：运行状态行为

While I will later discuss design patterns for implementation of state machines, the most common implementation is to generate simple if-then or switch-case statements. For example, a common and easy implementation scheme is to

- Add a state variable (e.g., `activeState`)
- For each event to the "class," add an event receptor function and pass in any data that it needs as parameters
- Create an event dispatcher function, called by each event receptor; this processes the incoming event

The structure of the event dispatcher can be as simple as

```
switch(activeState) {
    /* for each state */
    case state1:
        /* for each event */
        Switch (eventID) {
            event1:
                /* check each guard */
                If (guard1()) {
                    action1();
                ] else if (guard2()) {
                    action2();
                };
                break;
            event2:
                if (guard3()) {
                    action3();
                } else if (guard4()) {
                    action5();
                };
                break;
        };
        break;
    case state2:
        // etc
}
```

For example, given the SecuritySupervisor class, you might implement the state machine event dispatcher as shown in Code Listing 10. Adding asynchronous events requires adding the queuing of the events and their data. Note: the Null_id represents so-called "null events" – transitions that are not triggered by explicit events, but simply "fire" when the actions in the previous state complete.

```
static eventStatus dispatchEvent(short id) {
    eventStatus res = eventNotConsumed;
    switch (activeState) {
                /* are we in the Idle state? */
```


虽然我稍后会讨论用于实现状态机的设计模式，但最常见的实现方式是生成简单的 if-then 或 switch-case 语句。例如，一种常见且简单的实现方案是

- 添加一个状态变量（例如 `activeState`）
- 对于发送到"类"的每个事件，添加一个事件接收函数，并将其所需的任何数据作为参数传入
- 创建一个事件分发函数，由每个事件接收函数调用；该函数处理传入事件

事件分发器的结构可以简单到如下所示

```
switch(activeState) { /* 对每个状态 */ case
state1: /* 对每个事件 */ switch (eventID) {
event1: /* 检查每个守卫条件 */ if (
guard1()) { action1();] else if (guard2()) {
action2(); }; break; event2: if (guard3()) {
action3(); } else if (guard4()) { action5(); };
break; }; break; case state2: // 等等 }
```

例如，给定SecuritySupervisor类，你可以如代码清单10所示实现状态机事件分发器。添加异步事件需要添加事件的排队及其数据。注意：Null_id表示所谓的"空事件"——不是由显式事件触发，而是在前一状态中的动作完成时直接"触发"的迁移。

```
static eventStatus dispatchEvent(short id) {
eventStatus res = eventNotConsumed; switch
(activeState) { /* 我们是否处于空闲状态？ */
```

```
case SecuritySupervisor_Idle:
{
    if(id == Null_id) /* null triggered event? */
        {
            if(retries>=3)
                {
                    activeState = SecuritySupervisor_ErrorState;
                        displayMsg("ERROR: Max retries Exceeded");
                    res = eventConsumed;
                }
            else
                {
                    ++retries;
                    activeState = SecuritySupervisor_Accepting;
                    res = eventConsumed;
                }
        }
}
break;
        /* are we in the Accepting State? */
case SecuritySupervisor_Accepting:
{
    if(id == keypress_SecuritySupervisor_Event_id)
        {
            /* params struct has the data in the attribute 'key' */
          /* transition 1 */
if(isCANCEL(params->key))
        {
            retries = 0;
            displayMsg("Cancelled");
            activeState = SecuritySupervisor_Idle;
            {
                /* state ROOT.Idle.(Entry) */
                strcpy(pin, "");
            }
            res = eventConsumed;
        }
      else
        {
            /* transition 3 */
            if(isDigit(params->key))
                {
                    /* transition 3 */
                    addKey(params->key);
                    activeState = SecuritySupervisor_Accepting;
                    res = eventConsumed;
                }
              else
                {
                    /* transition 5 */
```


case SecuritySupervisor_Idle: { if(id == Null_id) /* 空触发事件? */ { if(retries >=3) { activeState = SecuritySupervisor_ErrorState; displayMsg("错误: 超过最大重试次数"); res = eventConsumed; } else {++retries; activeState = SecuritySupervisor_Accepting; res = eventConsumed; } } } break; /* 我们是否处于接受状态? */ case SecuritySupervisor_Accepting: { if(id == keypress_SecuritySupervisor_Event_id) { /* params 结构体在属性 'key' 中包含数据 */ /* 迁移 1 */ if(isCANCEL(params->key)) { retries = 0; displayMsg("已取消"); activeState = SecuritySupervisor_Idle; { /* 状态 ROOT.Idle.(Entry) */ strcpy(pin, ""); } res = eventConsumed; } else { /* 迁移 3 */ if(isDigit(params->key)) { /* 迁移 3 */ addKey(params->key); activeState = SecuritySupervisor_Accep

ting;

res = eventConsumed; }

else { /* 迁移 5 */

```
                        if(isENTER(params->key))
                            {
                                activeState =
                                        SecuritySupervisor_CheckingLength;
                              res = eventConsumed;
                            }
                    }
                }
        }
}
break;
case SecuritySupervisor_CheckingLength:
{
    if(id == Null_id)
        {
            /* transition 10 */
            if(strlen(pin) == 4)
                {
                    activeState = SecuritySupervisor_ValidatingPIN;
                    res = eventConsumed;
                }
              else
                {
                    {
                        /* transition 9 */
                        displayMsg("ERROR: PIN wrong length");
                    }
                    activeState = SecuritySupervisor_Idle;
                    {
                        /* state ROOT.Idle.(Entry) */
                        strcpy(pin, "");
                    }
                    res = eventConsumed;
                }
        }
}
break;
case SecuritySupervisor_ValidatingPIN:
{
    if(id == Null_id)
        {
            /* transition 13 */
            if(isValid(pin))
                {
                    {
                        /* transition 13 */
                        unlockDoor();
                        displayMsg("Door unlocked");
                    }
                    activeState = SecuritySupervisor_SecurityOpen;
                    res = eventConsumed;
```



```
                        if(isENTER(params->key)) { activeState =
                        SecuritySupervisor_Checkin

                                                                    gLength;
        res = eventConsumed; } } } } break; case SecuritySupervisor_
CheckingLength: { if(id == Null_id) { /* 迁移 10 */ if(strlen(pin) == 4) {
activeState = SecuritySupervisor_ValidatingPIN; res = eventConsumed; } else {
{ /* 迁移 9 */ displayMsg("ERROR: PIN wrong length"); } activeState =
SecuritySupervisor_Idle; { /* 状态 ROOT.Idle.(Entry) */ strcpy(pin, ""); } res =
eventConsumed; } } } break; case SecuritySupervisor_ValidatingPIN: { if(id ==
Null_id) { /* 迁移 13 */ if(isValid(pin)) { { /* 迁移 13 */ unlockDoor();
displayMsg("Door unlocked"); } activeState = SecuritySupervisor_
SecurityOpen; res = eventConsumed;
```

```
                    }
                  else
                    {
                        {
                            /* transition 12 */
                            displayMsg("ERROR:: Invalid PIN");
                        }
                        activeState = SecuritySupervisor_Idle;
                        {
                            /* state ROOT.Idle.(Entry) */
                            strcpy(pin, "");
                        }
                        res = eventConsumed;
                    }
            }
    }
    break;
    case SecuritySupervisor_SecurityOpen:
    {
        if(id == keypress_SecuritySupervisor_Event_id)
            {
                    /* params->key has the data passed with the event */
                /* transition 14 */
                if(isRESET(params->key))
                    {
                        {
                            /* transition 14 */
                            lockDoor(); retries = 0;
                            displayMsg("Door locked.");
                        }
                        activeState = SecuritySupervisor_Idle;
                        {
                            /* state ROOT.Idle.(Entry) */
                            strcpy(pin, "");
                        }
                        res = eventConsumed;
                    }
            }
        }
        break;
        default:
            break;
        }
        return res;
    }
```

Code Listing 10: SecuritySupervisor Event Dispatcher

} else { { /* 迁移 12 */ displayMsg("ERROR:: Invalid PIN"); } activeState = SecuritySupervisor_Idle; { /* 状态 ROOT.Idle.(Entry) */ strcpy(pin, ""); } res = eventConsumed; } } } break; case SecuritySupervisor_SecurityOpen: { if(id == keypress_SecuritySupervisor_Event_id) { /* params-key 包含随事件传递的数据 */ /* 迁移 14 */ if(isRESET(params->key)) { { /* 迁移 14 */ lockDoor(); retries = 0; displayMsg("Door locked."); } activeState = SecuritySupervisor_Idle; { /* 状态 ROOT.Idle.(Entry) */ strcpy(pin, ""); } res = eventConsumed; } } } break; default: break; } return res;

}

Code Listing 10: SecurityS$_{uperv}$ isor Event Di$_{spatc}$ her

There is certainly more to state machines than we've introduced so far, but those are discussed later in Chapter 5. For a more detailed discussion of UML state machines, refer to Chapter 3 of *Real-Time UML 3rd Edition*[13] and Chapters 7 and 12 in *Doing Hard Time*[14].

1.3 What Did We Learn?

This chapter discussed the special characteristics of embedded systems – working within tight design constraints while meeting functional and quality-of-service constraints. To do this, embedded developers use tools from the embedded tool chain to develop on a more powerful host computer while targeting an often smaller and less capable target computing environment. Target environments run the gamut from small eight-bit processors with a few kilobytes of memory to networked collections of 64-bit systems. At the small scale, many systems are too tightly constrained in memory, performance, or cost to permit the use of a commercial RTOS, while at the large scale, the systems may include a heterogeneous set of operating systems, middleware, and databases. Often, embedded software must be developed simultaneously with the hardware, making it difficult to test and debug since the target environments may not exist when the software is being developed.

By far, the most common language for developing embedded systems is C. C has the advantages of high availability of compilers for a wide range of target processors and a well-deserved reputation for run-time efficiency. Nevertheless, the pressures to add capabilities while simultaneously reducing costs mean that embedded developers must be continually looking for ways to improve their designs, and their ability to design efficiently. Structured methods organize the software into parallel taxonomies, one for data and the other for behavior. Object-oriented methods combine the two to improve cohesion for inherently tightly-coupled elements and encapsulation of content when loose coupling is appropriate. While C is not an object-oriented language, it can be (and has been) used to develop object-based and object-oriented embedded systems.

The next chapter will discuss the role of process and developer workflows in development. These workflows will determine when and where developers employ design and design patterns in the course of creation of embedded software. It will go on to define what design patterns are, how they will be discussed and organized within this book, and how they can be applied into your designs.

[13] Douglass, B.P., 2004. *Real-Time UML: Advances in the UML for Real-Time Systems*, third ed. Addison-Wesley.
[14] Douglass, B.P., 1999. *Doing Hard Time: Developing Real-Time Systems with UML, Objects, Frameworks, and Patterns*. Addison-Wesley.

关于状态机，我们目前所介绍的还远不止这些，但那些内容将在第5章中讨论。有关UML状态机的更详细讨论，请参阅《实时UML第三版》[13] 的第3章以及《Doing Hard Time》[14]中的第7章和第12章。

1.3 我们学到了什么？

本章讨论了嵌入式系统的特殊特性——在严格的设计约束下工作，同时满足功能性和服务质量约束。为此，嵌入式开发者使用嵌入式工具链中的工具，在功能更强大的主机上进行开发，而目标通常是更小、能力更弱的目标计算环境。目标环境涵盖从仅有几千字节内存的小型八位处理器到由64位系统组成的网络化集合。在小规模场景下，许多系统在内存、性能或成本方面受到严格约束，无法使用商用实时操作系统；而在大规模场景下，系统可能包含异构的操作系统、中间件和数据库。通常，嵌入式软件必须与硬件同步开发，这使得测试和调试变得困难，因为在软件开发时目标环境可能尚不存在。

到目前为止，开发嵌入式系统最常用的语言是C。C具有为广泛目标处理器提供编译器的优势，并且在运行时效率方面享有当之无愧的声誉。然而，在降低成本的同时增加功能的压力意味着嵌入式开发者必须不断寻找改进其设计的方法，以及提高高效设计能力的方法。结构化方法将软件组织为并行分类体系，一个用于数据，另一个用于行为。面向对象方法将两者结合起来，以提高本质上紧密耦合元素的内聚性，并在适合松耦合时实现内容的封装。虽然C不是面向对象语言，但它可以（并且已经）被用于开发基于对象和面向对象的嵌入式系统。

下一章将讨论进程和开发者工作流在开发中的作用。这些工作流将决定开发人员在创建嵌入式软件的过程中何时何地使用设计和设计模式。接着将定义什么是设计模式、本书将如何讨论和组织它们，以及如何将它们应用到你的设计中。

[13] Douglass, B.P., 2004. 实时UML：UML在实时系统中的进展，第三版. Addison-Wesley. [14] Douglass, B.P., 1999. Doing Hard Time: Developing Real-Time Systems with UML, Objects, Frameworks, and Patterns. Addison-Wesley.

Subsequent chapters will list a variety of design patterns that can be applied to optimize your designs. Chapter 3 provides a number of patterns that address access of hardware, such as keyboards, timers, memory, sensors, and actuators. Chapter 4 discusses concurrency patterns both for managing and executing concurrent threads as well as sharing resources among those threads. The last two chapters provide a number of patterns for the implementation and use of state machines in embedded systems: Chapter 5 provides basic state machine implementation-focused patterns, while Chapter 6 addresses the concerns of safety and high-reliability software.

后续章节将列出各种可用于优化设计的设计模式。第3章提供了若干模式，涉及硬件访问，如键盘、定时器、内存、传感器和执行器。第4章讨论并发模式，既包括管理和执行并发线程，也包括在这些线程之间共享资源。最后两章提供了若干用于在嵌入式系统中实现和使用状态机的模式：第5章提供以基本状态机实现为重点的模式，而第6章讨论安全性和高可靠性软件的问题。

CHAPTER 2

Embedded Programming with The Harmony™ for Embedded RealTime Process

Chapter Outline

2.1 Basic Elements of the Harmony Process 36
- 2.1.1 A Quick Overview of the Development Workflow 36
  - *2.1.1.1 Construct the Initial Model 37*
  - *2.1.1.2 Identify Important Design Criteria 40*
  - *2.1.1.3 Rank Design Criteria 42*
  - *2.1.1.4 Select Design Patterns and Technologies 42*
  - *2.1.1.5 Apply Design Patterns and Technologies 44*
  - *2.1.1.6 Validate Design Solution 44*
- 2.1.2 What Is a Design Pattern? 44
- 2.1.3 Basic Structure of Design Patterns 45
- 2.1.4 How to Read Design Patterns in This Book 46
- 2.1.5 Using Design Patterns in Development 47
  - *2.1.5.1 Pattern Hatching – Locating the Right Patterns 47*
  - *2.1.5.2 Pattern Mining – Rolling Your Own Patterns 49*
  - *2.1.5.3 Pattern Instantiation – Applying Patterns in Your Designs 50*
  - *2.1.5.4 An Example: The Observer Pattern 51*
- 2.1.6 Observer Pattern 66

2.2 The Approach 77

2.3 What's Coming Up 78

What you will learn

- Agile process principles and workflows for embedded applications development
- The three levels of design
- The five key views of architecture
- Performing trade studies
- What are design patterns?
- How do I apply design patterns?
- Example design pattern instantiation

Design Patterns for Embedded Systems in C
DOI:10.1016/B978-1-85617-707-8.00002-9

Copyright © 2011 Elsevier Inc.

第 2 章

使用 Harmony™ 进行嵌入式编程：面向嵌入式实时处理

本章大纲

2.1 Harmony 过程的基本要素 36 2.1.1 开发工作流快速概览 36 2.1.1.1 构建初始模型 37 2.1.1.2 识别重要的设计准则 40 2.1.1.3 设计准则排序 42 2.1.1.4 选择设计模式与技术 42 2.1.1.5 应用设计模式与技术 44 2.1.1.6 验证设计方案 44 2.1.2 什么是设计模式? 44 2.1.3 设计模式的基本结构 45 2.1.4 如何阅读本书中的设计模式 46 2.1.5 在开发中使用设计模式 47 2.1.5.1 模式孵化——定位合适的模式 47 2.1.5.2 模式挖掘——打造你自己的模式 49 2.1.5.3 模式实例化——在设计中的应用模式 50 2.1.5.4 一个示例: 观察者模式 51 2.1.6 观察者模式 66 2.2 方法 77 2.3 接下来

g Up 78

你将学到什么

- 嵌入式应用开发的敏捷过程原则和工作流
- 设计的三个层次
- 架构的五个关键视图
- 执行权衡研究
- 什么是设计模式?
- 如何应用设计模式?
- 示例设计模式实例化

C语言嵌入式系统设计模式
DOI:10.1016/B978-1-85617-707-8.00002-9 版权所有

© 2011 Elsevier Inc.

2.1 Basic Elements of the Harmony Process

Let's now turn our attention to the basic questions of how we, as embedded software developers, perform design. While I don't want to belabor process in this book, we perform design activities by following (formal or informal) workflows. To understand how to do design, we must have at least a basic agreement as to the workflow.

2.1.1 A Quick Overview of the Development Workflow

I am the author of the Harmony™ for Embedded RealTime (Harmony/ERT[1]) process from IBM Rational[2]. This process consists of a set of best practices for the development of real-time and embedded software, and is in use in many different embedded vertical markets, from medical to aerospace to consumer electronics. It is an agile, model-based approach to software development that emphasizes continual execution of software throughout its development, test-driven development, and the use of design patterns.

The Harmony™ for Embedded RealTime process is an incremental process, similar in some ways to Scrum[3]. The Harmony/ERT process focuses on three timescales, as shown in Figure 2-1. The *macrocycle* focuses on the overall project timeline and major milestones. The *microcycle* focuses on the production of a single validated increment (a.k.a. a build or prototype). The *nanocycle* timeframe is in the range of minutes to hours and concentrates on continual execution, debug, and unit test of the software.

The microcycle, which is iterated many times during the course of a typical project, is broken up into several short phases, as you can see in Figure 2-2. These phases contain the workflows relevant to the current discussion. These are briefly described in Table 2-1, where we can see a couple of key points. First, design is something that occurs only *after* we have correctly functioning software from the *Object Analysis* (or "Software Analysis" if you prefer) phase. Secondly, design focuses on *optimizing* this correctly functioning software. This latter point is important because that's what design patterns – the theme of this book – do; they optimize software. Design patterns are not about getting the software to work correctly; they are about getting the software to work optimally.

As you can see from Table 2-1, the Harmony/ERT process uses three levels of design. Architectural design looks at design decisions that optimize the system overall. Architectural design is subdivided into five key views that have the greatest impact on the structure and behavior of the software (Figure 2-3). Each of these key architectural views is described in Table 2-2.

[1] Formerly know as Harmony for Embedded SoftWare (Harmony/ESW. Don't ask; I didn't change the name this time).
[2] See *www-01.ibm.com/software/awdtools/rmc/* for information on tools and process content.
[3] See *www.scrumalliance.org/*.

2.1 Harmony过程的基本要素

现在让我们将注意力转向基本问题：作为嵌入式软件开发者，我们如何进行设计。虽然我不想在本书中过多赘述进程，但我们是通过遵循（正式或非正式的）工作流来执行设计活动的。要理解如何做设计，我们至少必须对工作流有基本的共识。

2.1.1 开发工作流快速概览

我是IBM Rational的HarmonyTM for Embedded RealTime（Harmony/ERT[1]）过程的作者。该过程由一组用于实时和嵌入式软件开发的较佳实践组成，已在从医疗到航空航天再到消费电子的众多不同嵌入式垂直市场中使用。它是一种敏捷的、基于模型的软件开发方法，强调在软件开发全过程中持续执行软件、测试驱动开发以及设计模式的使用。

HarmonyTM for Embedded RealTime过程是一个增量过程，在某些方面与Scrum[3]类似。Harmony/ERT过程关注三个时间尺度，如图2-1所示。宏周期关注整体项目时间线和主要里程碑。微周期关注单个已验证增量（也称为构建或原型）的产出。纳周期的时间范围在分钟到小时级别，专注于软件的持续执行、调试和单元测试。

微周期在一个典型项目的过程中会迭代多次，它被分解为几个短阶段，如图2-2所示。这些阶段包含与当前讨论相关的工作流。表2-1对这些阶段进行了简要描述，从中我们可以看到几个关键点。首先，设计是在我们通过对象分析（或者如果你愿意，也可以称为"软件分析"）阶段获得功能正确的软件之后才会发生的事情。其次，设计侧重于优化这个功能正确的软件。后一点很重要，因为这正是设计模式——本书的主题——所做的；它们优化软件。设计模式不是关于让软件正确工作；而是关于让软件以最优方式工作。

从表2-1可以看出，Harmony/ERT过程使用三个层次的设计。架构设计关注的是优化系统整体的设计决策。架构设计细分为五个关键视图，这些视图对软件的结构和行为影响最大（图2-3）。表2-2描述了每个关键架构视图。

[1] 以前称为Harmony for Embedded SoftWare（Harmony/ESW。别问；这次我没有改名字）。[2] 有关工具和过程内容的信息，请参见www-01.ibm.com/software/awdtools/rmc/。[3] 请参见www.scrumalliance.org/。

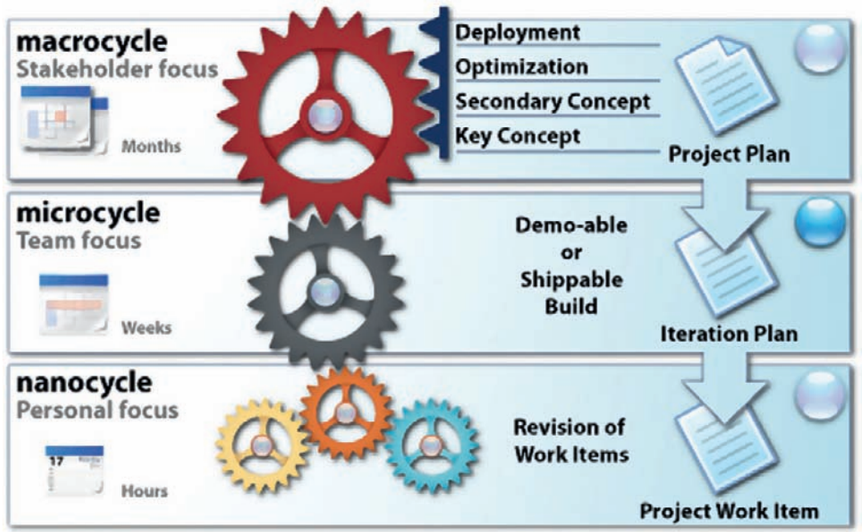


Figure 2-1: Harmony/ERT timescales

Mechanistic design optimizes at the level of the collaboration, where a *collaboration* is a collection of simpler elements (types, functions, variables, and classes) interacting to achieve a single use case or system capability. Each system has usually from as few as half a dozen to as many as several dozen such capabilities; as such, the scope of mechanistic design is an order of magnitude smaller than architectural concerns.

Detailed design limits itself to the smallest level of concern – the insides of functions, variables, types, and classes. This level of design is perhaps the most common level of focus by developers but often has the least impact on overall system performance.

At any level of focus, the basic workflow for design is provided by Figure 2-4. Because these steps will form the basis of how we use design patterns, let's discuss each of them in turn.

2.1.1.1 Construct the Initial Model

This step is crucial but often omitted in the developer's haste to optimize. The step means that before optimization is considered, a working model and code base is created. By "working" I mean not only that it compiles and runs, but also that it is robust and functionally correct. I also mean that it is debugged and unit-tested for functional correctness. It may be tested for performance to determine whether optimization is required but that is optional at this point.

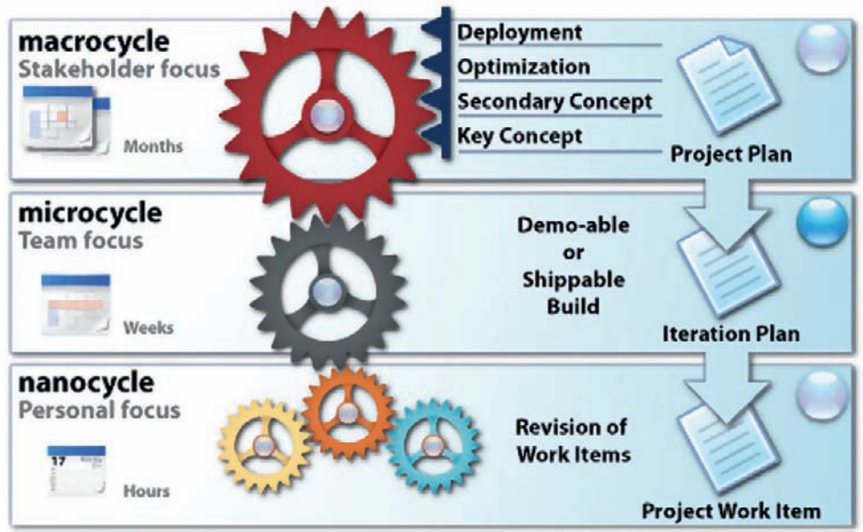


图 2-1：Harmony/ERT 时间尺度

机制设计在协作层面进行优化，其中协作是一组更简单的元素（类型、函数、变量和类）相互作用以实现单个用例或系统能力。每个系统通常具有从少至六个到多达几十个这样的能力；因此，机制设计的范围比架构关注点小一个数量级。

详细设计仅限于最小关注层面——函数、变量、类型和类的内部。这一设计层面或许是开发者最常关注的层面，但对整体系统性能的影响往往最小。

在任何关注层面上，设计的基本工作流程均如图 2-4 所示。由于这些步骤将构成我们使用设计模式的基础，下面逐一讨论。

2.1.1.1 构建初始模型

这一步至关重要，但开发者往往因急于优化而将其省略。这一步意味着在考虑优化之前，先创建一个可工作的模型和代码库。所谓"可工作"，我不仅指它能编译和运行，还指它健壮且功能正确。我还指它经过调试和单元测试以确保功能正确性。此时可以选择性地进行性能测试，以确定是否需要进行优化，但这一步是可选的。

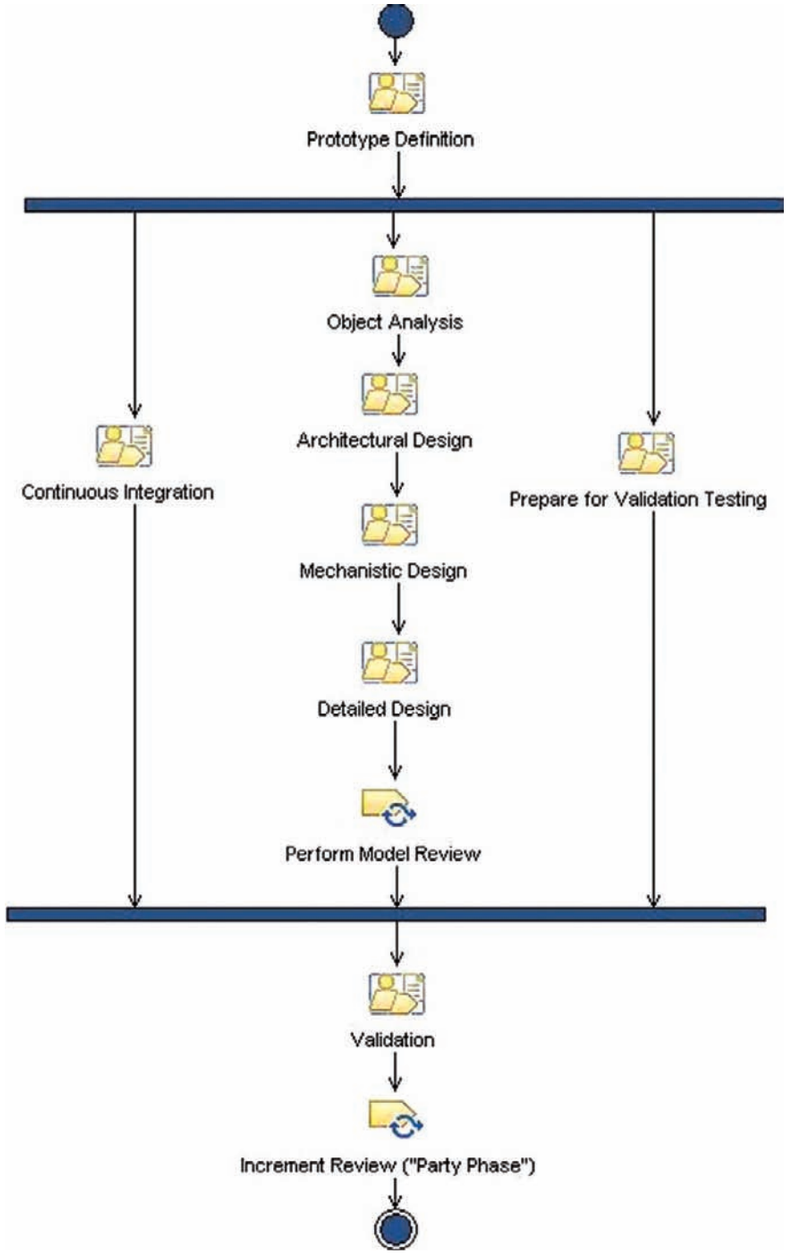


Figure 2-2: Harmony/ERT microcycle

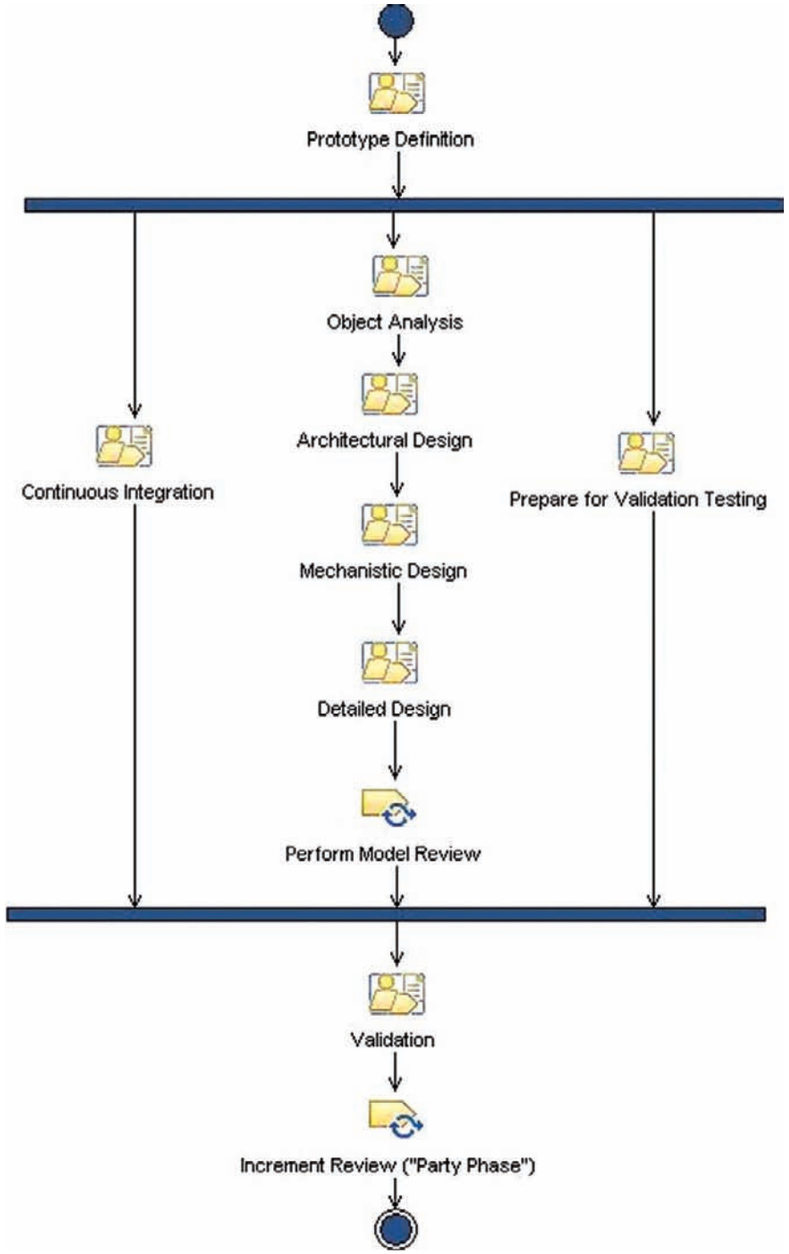


图 2-2：Harmony/ERT 微周期

Table 2-1: Harmony/ERT microcycle phases.

Phase	Primary Inputs	Primary Outputs	Description
Prototype definition	Stakeholder requirements	System requirements use case model	Transforms a subset of stakeholder requirements into system requirements for development in the current microcycle
Object analysis	System requirements	Functionally correct model and source code	Develops software that realizes the functional requirements; includes coding, debugging, and unit testing
Architectural design	Functionally correct code and models	Architecturally-optimized code and models	Optimizes the code based on explicit design decisions meant to optimize the system overall; includes coding, debugging, and unit testing
Mechanistic design	Architecturally-optimized code and models	Collaboration-optimized code and models	Optimizes the code based on explicit design decisions optimizing collaborations of elements meant to realize a feature or use case; includes coding, debugging, and unit testing
Detailed design	Collaboration-optimized code and models	Optimized functions and data structures	Optimizes the code based on the explicit design decisions at the function, variable, and type level; includes coding, debugging, and unit testing
Model/code review	Models and unit tested code	Change requests based on review	Reviews the analysis, design, and code for consistency, completeness, correctness
Validation	Optimized source code	Validated system and change requests based on defects identified	Tests the system against the requirements implemented so far, including regression testing of requirements from previous microcycles
Increment review	Schedule, risk management plan, reuse plan, hazard analysis, team questionnaire	Change requests to process, plans, or development environment	Evaluates progress against plan and identifies where plans must be updated; also identifies team, environment, and development issues that need to be improved
Continuous integration	Models and unit-tested source code	Evolving software baseline	Executed at least daily, this activity integrates source code from multiple developers, applies integration tests, and updates the evolving software baseline
Prepare for validation	System requirements and use case models	Test plans, test cases, test fixtures	Prepares for validation testing by developing plans, test cases, and test environments

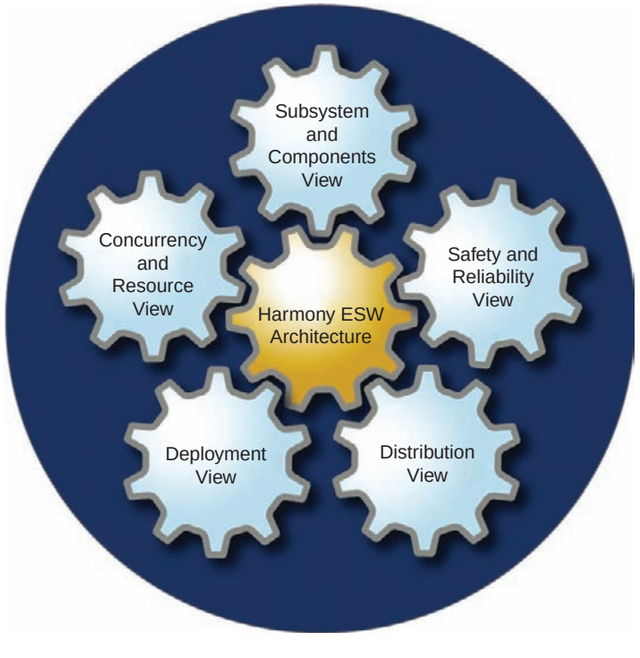


Figure 2-3: Architectural design views

Table 2-2: Key architectural views.

Key Architectural View	Description
Subsystem and components view	Specifies the set of system subsystems and components, their responsibilities, connections to other subsystems, and interfaces
Concurrency and resource view	Identifies the concurrency units, how primitive elements (functions, variables, and objects) map to the concurrency units, how those concurrency units are scheduled, and how the concurrency units synchronize and share resources
Distribution view	Describes how elements are distributed across multiple address spaces and how those elements find and collaborate with each other
Safety and reliability view	Defines how faults are identified, isolated, and corrected during system execution to maintain safety and reliability
Deployment view	Characterizes how responsibilities are split across elements from different engineering disciplines (e.g., software, electronic, mechanical, and chemical) and the interfaces between those disciplines

2.1.1.2 Identify Important Design Criteria

This is another commonly omitted step. It is important to remember that whenever some system aspects are optimized, it is almost always true that some other aspects are deoptimized. In order

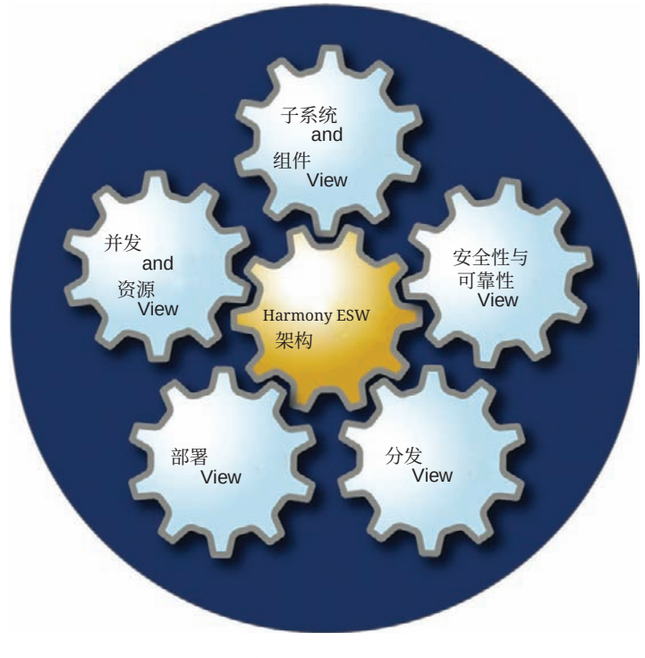


图2-3：架构设计视图

表 2-2：关键架构视图。

关键架构视图	描述
子系统与组件视图	指定系统子系统和组件的集合、它们的职责，与其他子系统的连接及接口
并发与资源视图	标识并发单元、基本元素（函数、变量、和对象）如何映射到并发单元、这些并发单元如何被调度，以及并发单元如何同步和共享资源
分布视图	描述元素如何分布在多个地址空间中，以及这些元素如何相互发现并协作
安全与可靠性视图	定义在系统执行期间如何识别、隔离和纠正故障，以维持安全性与可靠性
部署视图	刻画职责如何在不同工程学科（例如软件、电子、机械和化学）的要素之间进行划分以及这些学科之间的接口

2.1.1.2 确定重要的设计准则

这是另一个常被遗漏的步骤。重要的是要记住，每当某些系统方面被优化时，几乎总是会有另一些方面被去优化。为了

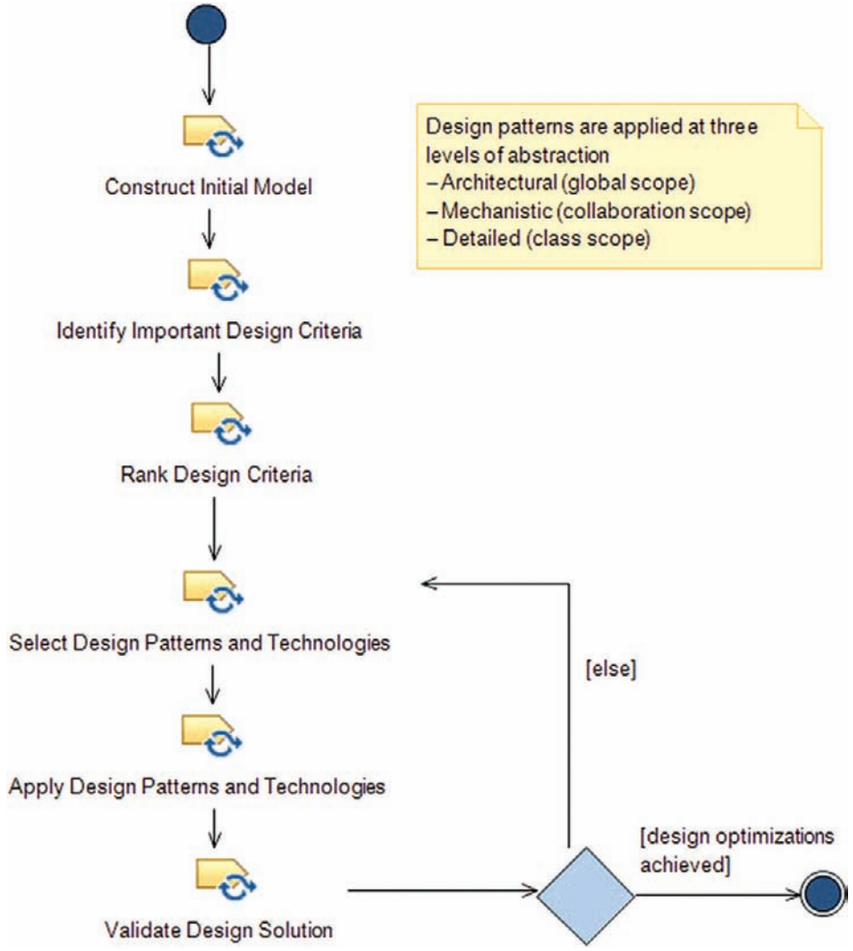


Figure 2-4: Basic design workflow

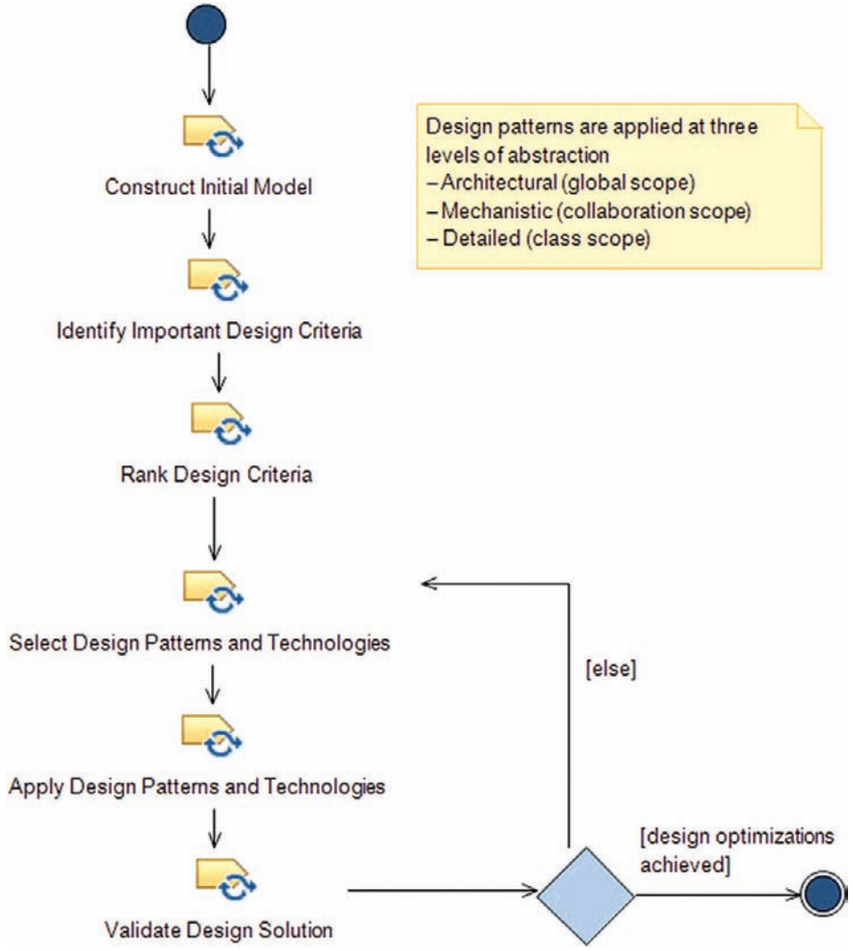


图2-4：基本设计工作流

to select appropriate design patterns and technologies to achieve optimization, we need to first understand what are the important aspects to optimize, so we can trade them off appropriately. Some common design optimization criteria are:

- Performance
  - Worst case
  - Average case
- Predictability
- Schedulability

选择合适的设计模式和技术来实现优化，我们需要首先了解哪些是需要优化的重要方面，以便能够适当地进行权衡。一些常见的设计优化标准包括：

- 性能
  - 最坏情况
  - 平均情况
- 可预测性
- 可调度性

- Throughput
  - Average
  - Sustained
  - Burst
- Reliability
  - With respect to errors
  - With respect to failures
- Safety
- Reusability
- Distributability
- Portability
- Maintainability
- Scalability
- Complexity
- Resource usage, e.g., memory
- Energy consumption
- Recurring cost, i.e., hardware
- Development effort and cost

2.1.1.3 Rank Design Criteria

The ranking of the criteria is necessary for us to make appropriate tradeoffs when we select design patterns. Some patterns will optimize worst-case performance at the expense of using more memory; others will optimize robustness at the expense of average case performance. Others may optimize energy economy at the expense of responsiveness. The ranking of the design criteria makes their relative importance explicit, so that we can reason about which design solutions are best.

2.1.1.4 Select Design Patterns and Technologies

There is a wealth of effective design solutions and technologies available. A *trade study* is a comparison of technologic solutions based on their effectiveness at achieving a balanced set of optimization goals. Although we may choose to be less formal in our approach, the FAA recommends a formalized decision analysis matrix[4]. Basically, this is a spreadsheet that summarizes the "goodness" of a design solution based on the degree to which it matches our weighted design criteria. For example, Table 2-3 shows the kind of analysis that we might do formally in the spreadsheet or informally in our heads. The weight for each criteria is the relative importance of the criteria on a scale of 0 to 10. Each alternative – a combination of design patterns and technologies – is assigned a score (degree to which the criteria is optimized)

[4] *www.faa.gov/about/office_org/headquarters_offices/ato/service_units/operations/sysengsaf/seman/SEM3.1/Section%204.6.pdf*

- 吞吐量
  - 平均值
  - 持续
  - 突发
- 可靠性
  - 关于错误
  - 关于故障
- 安全性
- 可复用性
- 可分布性
- 可移植性
- 可维护性
- 可扩展性
- 复杂性
- 资源使用，例如内存
- 能耗
- 经常性成本，即硬件
- 开发工作量和成本

2.1.1.3 设计准则排序

对准则进行排序对于我们选择设计模式时做出适当的权衡是必要的。有些模式会以使用更多内存为代价来优化最坏情况性能；另一些则会以牺牲平均情况性能为代价来优化鲁棒性。还有一些可能会以牺牲响应性为代价来优化能源经济性。设计准则的排序明确了它们的相对重要性，从而使我们能够推理出哪些设计方案是最佳的。

2.1.1.4 选择设计模式与技术

有大量有效的设计方案和技术可供使用。权衡研究是基于技术解决方案在实现一组平衡的优化目标方面的有效性而对它们进行的比较。尽管我们可能选择采用不太正式的方法，但FAA建议使用正式的决策分析矩阵[4]。基本上，这是一个电子表格，它根据设计方案与我们的加权设计准则的匹配程度来总结该方案的"优良度"。例如，表2-3展示了我们可能在电子表格中正式进行或在头脑中非正式进行的那种分析。每个准则的权重是该准则在0到10分制上的相对重要性。每个备选方案——设计模式与技术的组合——都会被赋予一个评分（该准则被优化的程度）

[4] www.faa.gov/about/office_org/headquarters_offices/ato/service_units/operations/sysengsaf/seman/SEM3.1/Sec- tion%204.6.pdf

Table 2-3: Design tradeoff spreadsheet.

Design Solution	Design Criteria					Total Weighted Score	al Weighted Score
	Criteria 1	Criteria 2	Criteria 3	Criteria 4	Criteria 5		
	Weight = 7	Weight = 5	Weight = 3	Weight = 2	Weight = 1.5		
	Score	Score	Score	Score	Score		
Alternative 1	7	3	6	9	4	106	106
Alternative 2	4	8	5	3	4	95	95
Alternative 3	10	2	4	8	8	**120**	**120**
Alternative 4	2	4	9	7	6	84	84

in the range 0 to 10. Each alternative is then assigned a computed total weighted score equal to the sum of the criteria weights times their relative scores. The alternative with the largest total weighted score provides the best balanced design solution for the set of design criteria.

2.1.1.5 Apply Design Patterns and Technologies

This step refers to the application of the selected design alternative. In design pattern terms, this is known as *pattern instantiation*. As we will see, this is primarily a matter of replacing the formal pattern parameters with the relevant elements from the working analysis model.

2.1.1.6 Validate Design Solution

The validation step ensures that the solution works and checks two different facets of the solution. First, it ensures that the functionality of the system continues to be correct; that is, adding in the design solution didn't break the running model and code. Secondly, since we selected the design patterns to optimize the system, we must ensure that we've achieved our optimization goals. This means that we will rerun the previous functional and unit tests and also add quality of service tests to measure performance, memory usage, or whatever measures are appropriate to verify our design criteria are met.

2.1.2 What Is a Design Pattern?

A design pattern is "a generalized solution to a commonly occurring problem." To be a pattern, the problem must recur often enough to be usefully generalizable and the solution must be general enough to be applied in a wide set of application domains. If it only applies to a single application domain, then it is probably an *analysis pattern*[5]. Analysis patterns define ways for organizing problem-specific models and code for a particular application domain.

Remember that analysis is driven by *what* the system must do while design is driven by *how well* the system must achieve its requirements. A design pattern is a way of organizing a design that improves the optimality of a design with respect to one or a small set of design criteria, such as qualities of service (QoS).

A design pattern always has a focused purpose – which is to take one or a small set of these QoS properties and optimize them at the expense of the others. Each pattern optimizes some aspect of a system at the cost of deoptimizing some other aspects. These constitute a set of pros and cons.

A good design is one composed of a set of design patterns applied to a piece of functional software that achieves a balanced optimization of the design criteria and incurs an acceptable cost in doing so. That is, a good design optimizes the important design criteria at a cost that we are willing to pay. Note that a design always encompasses several different design patterns, so

[5] A coherent set of analysis patterns is also known as a *reference model.*

在0到10的范围内。然后为每个设计方案计算总加权分数，该分数等于各准则权重与其相对得分的乘积之和。总加权分数最大的设计方案为这组设计准则提供了最佳的平衡设计方案。

2.1.1.5 应用设计模式与技术

此步骤指的是应用所选的设计方案。用设计模式的术语来说，这称为模式实例化。正如我们将看到的，这主要是用工作分析模型中的相关元素替换形式化的模式参数。

2.1.1.6 验证设计方案

验证步骤确保解决方案能够正常工作，并检查解决方案的两个不同方面。首先，它确保系统的功能持续正确；也就是说，加入设计方案并没有破坏运行模型和代码。其次，由于我们选择设计模式是为了优化系统，因此必须确保已经实现了优化目标。这意味着我们将重新运行之前的功能测试和单元测试，并添加服务质量测试来测量性能、内存使用或任何适合验证设计准则是否得到满足的指标。

2.1.2 什么是设计模式?

设计模式是"对常见问题的泛化解决方案"。要成为一种模式，该问题必须频繁出现，足以被有效地泛化，并且解决方案必须足够通用，能够应用于广泛的应用领域。如果它只适用于单一应用领域，那么它可能是一种分析模式[5]。分析模式定义了为特定应用领域组织问题特定模型和代码的方式。

请记住，分析是由系统必须做什么驱动的，而设计是由系统必须多好地实现其需求驱动的。设计模式是一种组织设计的方式，它针对一个或一小组设计准则（如服务质量（QoS））来提升设计的优化程度。

设计模式始终具有明确的目的——即选取一个或一小组这些QoS属性，并以牺牲其他属性为代价来优化它们。每种模式都以降低某些其他方面的优化程度为代价来优化系统的某个方面。这些构成了一组优缺点。

好的设计是由一组设计模式构成的，这些设计模式应用于功能性软件，实现对设计准则的平衡优化，并且在此过程中产生可接受的成本。也就是说，好的设计以我们愿意付出的成本来优化重要的设计准则。请注意，一个设计总是包含若干不同的设计模式，因此

[5] 一组连贯的分析模式也称为参考模型。

each alternative in our trade study must also examine how well the patterns work together, in addition to how each works in isolation.

As mentioned previously, patterns may be applied at the different levels of design abstraction. Architectural patterns have systemic scope and apply mostly to only one of the five key views of architecture. At the next level down in design abstraction, mechanistic design patterns apply to individual collaborations, optimizing the same criteria, but in a more narrow focus. Detailed design patterns – often referred to as design idioms – optimize the primitive elements of the software.

2.1.3 Basic Structure of Design Patterns

According to Gamma, et. al.[6], a pattern has four important aspects:

- Name
  The name provides a "handle" or means to reference the pattern.
- Purpose
  The purpose provides the problem context and the QoS aspects the pattern seeks to optimize. The purpose identifies the kinds of problem contexts where the pattern might be particularly appropriate.
- Solution
  The solution is the pattern itself.
- Consequences
  The consequences are the set of pros and cons of the use of the pattern.

The pattern *name* brings us two things. First, it allows us to reference the pattern in a clear, unambiguous way, with the details present, but unstated. Secondly, it gives us a more abstract vocabulary to speak about our designs. The statement "The system uses a Homogeneous Channel redundant architecture with Cyclic Executive Concurrency distributed across a set of processors with a Proxy Pattern" has a lot of information about the overall structure of the system because we can discuss the architecture in terms of these patterns.

The *purpose* of the pattern identifies the essential problem contexts required for the pattern to be applicable and what design criteria the pattern is attempting to optimize. This section specifies under which situations the pattern is appropriate and under which situations it should be avoided.

The *solution*, of course, specifies the structure and behavior of the pattern itself. It identifies the elements of the pattern and their roles in the context of the pattern. This pattern solution is integrated into your design in a process known as *pattern instantiation*.

[6] Gamma, E., Helm, R., Johnson, R., Vlissides, J.M., 1995. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.

在我们的权衡研究中，每个备选方案除了考察每个模式单独工作的情况外，还必须考察这些模式协同工作的效果。

如前所述，模式可以应用于不同的设计抽象层次。架构模式具有系统性范围，并且通常仅适用于架构的五个关键视图之一。在设计抽象的下一个层次，机制设计模式适用于单个协作，优化相同的标准，但焦点更为狭窄。详细设计模式——通常称为设计惯用法——优化软件的基本元素。

2.1.3 设计模式的基本结构

根据 Gamma 等人 的 观点[6], 一个 模式 具有 四个 重要 方面:

- 名称 名称提供了一个"句柄"或用于引用该模式的手段。
- 目的 目的提供了问题上下文以及该模式所寻求优化的QoS方面。目的标识了该模式可能特别适合的问题上下文的类型。
- 解决方案 解决方案就是模式本身。
- 结果 结果是使用该模式的优缺点集合。

模式名称为我们带来两件事。首先，它允许我们以清晰、明确的方式引用该模式，细节存在但未被明说。其次，它为我们提供了一种更抽象的词汇来讨论我们的设计。"该系统使用同构通道冗余架构，采用循环执行并发，分布在一组处理器上，并使用代理模式"这一陈述包含了大量关于系统整体结构的信息，因为我们可以用这些模式来讨论架构。

模式的目的是确定模式适用所必需的基本问题上下文，以及模式试图优化的设计准则。本节说明了模式在哪些情况下适用，以及在哪些情况下应避免使用。

当然，解决方案规定了模式本身的结构和行为。它标识了模式的元素及其在模式上下文中的角色。这个模式解决方案通过一个称为模式实例化的过程集成到你的设计中。

[6] Gamma, E., Helm, R., Johnson, R., Vlissides, J.M., 1995. 设计模式：可复用面向对象软件基础. Addison-Wesley.

The *consequences* are important because we always make tradeoffs when we select one pattern over another. We must understand the pros and cons of the pattern to apply it effectively. The pros and cons are usually couched in terms of improvement or degradation of some design properties, as well as a possible elaboration of problem contexts in which these consequences apply.

2.1.4 How to Read Design Patterns in This Book

All the patterns of this book are organized in the same fashion to improve the usability of the patterns. Each pattern is provided in the following format:

- Abstract
  The abstract gives a brief description of the pattern use or justification. This is meant as an overview of the problem, solution, and consequences.
- Problem
  The problem section gives a statement of the problem context and the qualities of service addressed by the pattern.
- Pattern structure
  This section provides a structural UML diagram of the pattern showing the important elements of the pattern. These elements are the places into which you will substitute your own specific application elements to instantiate the pattern. Relations among elements of the pattern are shown as well.
- Consequences
  The consequences section describes the tradeoffs made when the pattern is used.
- Implementation strategies and source code
  This section discusses issues around the implementation of the pattern on different computing platforms or in different source level languages.
- Example
  Each pattern is shown in an example, illustrating how the pattern is applied in some particular case. This usually involves the presentation of a UML structural diagram showing particular application elements fulfilling the pattern collaboration roles and relevant code snippets showing the pattern implementation.

Each pattern in this book is shown using both generic, standard UML, and C source code. The UML graphical notation provides a higher-level overview of the pattern, but if you don't find this helpful, you can skip directly to the source code. If you're interested, Appendix A gives a UML notational summary. For a more detailed understanding of the UML itself, see *Real-Time UML 3rd Edition*[7].

[7] Douglass, B.P., 2004. *Real-Time UML: Advances in the UML for Real-Time Systems*, third ed. Addison-Wesley.

结果很重要，因为当我们选择一个模式而非另一个时，我们总是在做权衡。我们必须理解模式的优缺点才能有效地应用它。优缺点通常以某些设计属性的改进或退化来表述，同时也可能对这些结果所适用的问题上下文进行详细阐述。

2.1.4 如何阅读本书中的设计模式

本书中的所有模式均以相同的方式组织，以提高模式的可用性。每个模式均按以下格式提供:

- 摘要 摘要简要描述了模式的用途或理由。这旨在作为问题、解决方案和结果的概述。
- 问题 问题部分陈述了问题上下文以及该模式所解决的服务质量。
- 模式结构 本节提供了模式的结构UML图，展示了模式的重要元素。这些元素是您将替换为自己特定应用元素以实例化模式的位置。模式元素之间的关系也会一并展示。
- 结果 结果部分描述了使用该模式时所做的权衡。
- 实现策略与源代码 本节讨论在不同计算平台或不同源码级语言中实现该模式的相关问题。
- 示例 每个模式都通过一个示例来展示，说明该模式在特定情况下的应用方式。这通常包括展示一个UML结构图，显示满足模式协作角色的特定应用元素，以及展示模式实现的相关代码片段。

本书中的每个模式都同时使用通用的标准UML和C源代码来展示。UML图形表示法提供了模式的高层概览，但如果您觉得这没有帮助，可以直接跳到源代码部分。如果您感兴趣，附录A给出了UML符号摘要。如需更详细地了解UML本身，请参阅实时UML第三版[7]。

[7] Douglass, B.P.，2004年。Real-Time UML: UML在实时系统中的进展，第三版。Addison-Wesley .

2.1.5 Using Design Patterns in Development

By this point, you should have a reasonably good understanding of what a pattern is and how it is organized. You should also have at least a vague grasp of the Harmony/ERT process and our view of where design fits into the overall scheme of things. Earlier in this chapter, you got a grounding in what we mean by the term *architecture* and the five views of architecture. At this point, you are almost ready to read the patterns in this book and apply them to your applications development. But first, let's briefly discuss how we might use patterns in our daily work lives.

2.1.5.1 Pattern Hatching – Locating the Right Patterns

You're facing a design problem. How do you find the patterns that can be applied to solve your particular design issues? We recommend a multistep approach, as shown in Figure 2-5.

1. Familiarize yourself with the patterns
   First, before starting your design, familiarize yourself with the patterns literature[8]. There are a number of books, papers, and websites devoted to patterns in many application domains. Some of those patterns are given here and others are given in the references. Once you have increased your vocabulary to include patterns likely to be relevant to your application domain, you have more intellectual ammunition to face your design challenges.
2. Identify design criteria
   In parallel with reading about potential design alternatives, you must identify design criteria. There are many (often conflicting) optimizations you can make to your system. This step identifies the design optimizations that are important for this particular system in its operational and development contexts.
3. Rank design criteria
   Not all optimizations are equally important. For this reason, the design criteria must be ranked to enable good tradeoff decisions to be made.
4. Identify potential design alternatives
   This step finds patterns that optimize the design criteria of importance at a cost that seems reasonable. While it is possible to add a single pattern at a time in this way and repeat the workflow, it is also common to consider a set of compatible patterns together as a single design alternative.
5. Evaluate the design alternatives
   Using the trade study approach suggested earlier, this step compares the design alternatives (consisting of one or more design patterns) against each other on the basis of their effectiveness against the design criteria, weighted with the relative importance of each.

[8] That means read the book!

2.1.5 在开发中使用设计模式

至此，您应该对什么是模式以及如何组织模式有了相当不错的理解。您也应该至少对Harmony/ERT过程以及我们对设计在整体方案中所处位置的看法有了初步的把握。在本章前面，您已经了解了我们所说的架构一词的含义以及架构的五个视图。此时，您几乎已经准备好阅读本书中的模式并将其应用于您的应用程序开发了。但首先，让我们简要讨论一下如何在日常工作中使用模式。

2.1.5.1 模式孵化——定位正确的模式

你正面临一个设计问题。如何找到可用于解决你特定设计问题的模式？我们推荐一种多步骤方法，如图 2-5 所示。

1. 熟悉模式 首先，在开始设计之前，请熟悉模式文献[8]。有许多书籍、论文和网站致力于介绍众多应用领域中的模式。其中一些模式在此给出，另一些则在参考文献中给出。一旦你扩充了词汇量，将可能与你应用领域相关的模式纳入其中，你就拥有了更多的智力弹药来面对设计挑战。 2. 确定设计准则
   在阅读潜在设计方案的同时，你必须确定设计准则。你可以对系统进行许多（往往相互冲突的）优化。此步骤确定对于该特定系统在其运行与开发环境中重要的设计优化。
3. 设计准则排序
   并非所有优化都同等重要。因此，必须对设计准则进行排序，以便做出良好的权衡决策。
4. 识别潜在的设计方案 此步骤寻找能够以合理成本优化重要设计准则的模式。虽然可以通过这种方式一次添加一个模式并重复工作流程，但通常也会将一组兼容的模式作为一个单一设计方案一起考虑。 5. 评估设计方案

   使用前面建议的权衡研究方法，此步骤基于设计方案（由一个或多个设计模式组成）针对设计准则的有效性，并结合每个准则的相对重要性进行加权，将各设计方案相互比较。

[8] 这意味着要读这本书！

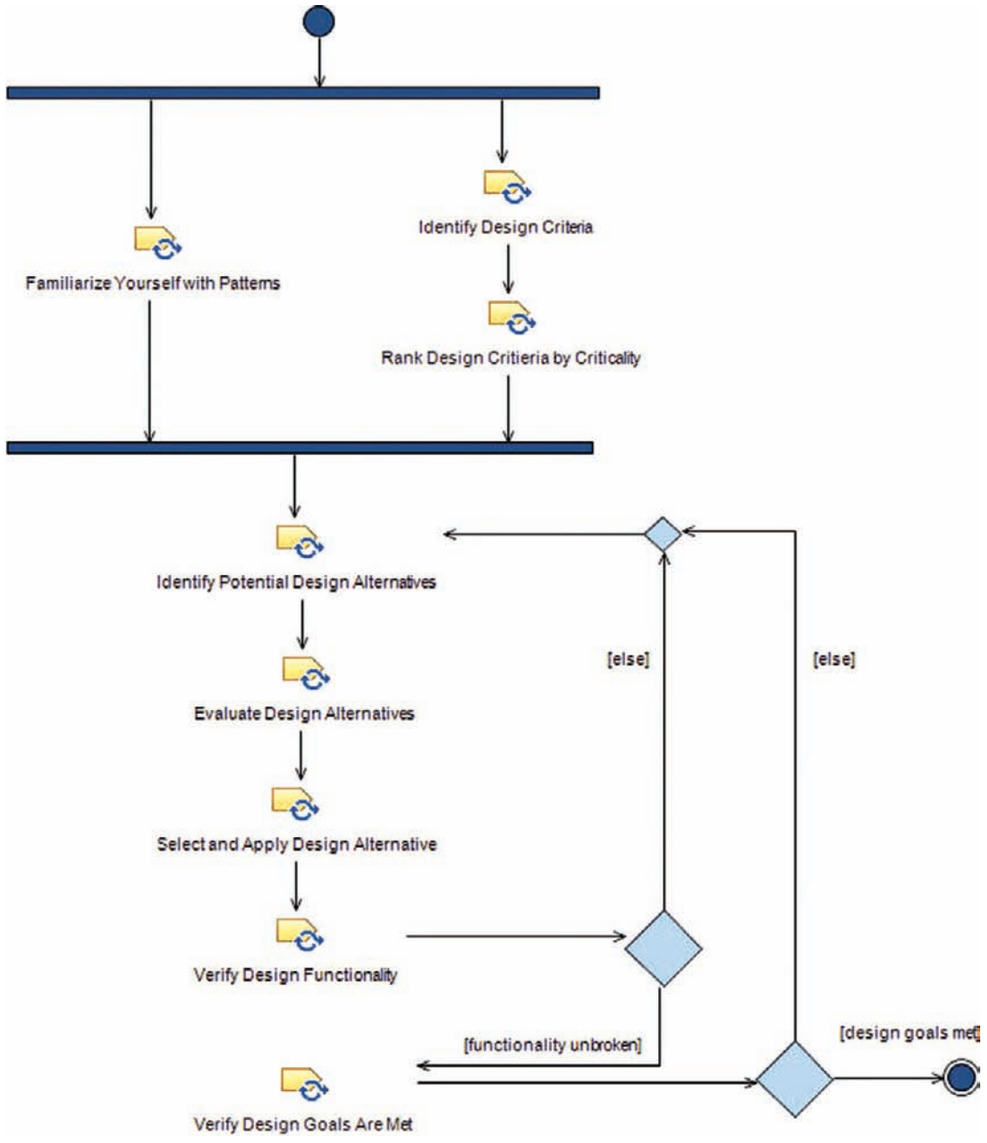


Figure 2-5: Pattern hatching

6. Select and apply design alternative
   Once a design alternative is selected, the patterns are usually added to the collaboration one at a time, including verification of the efficacy of the solution.

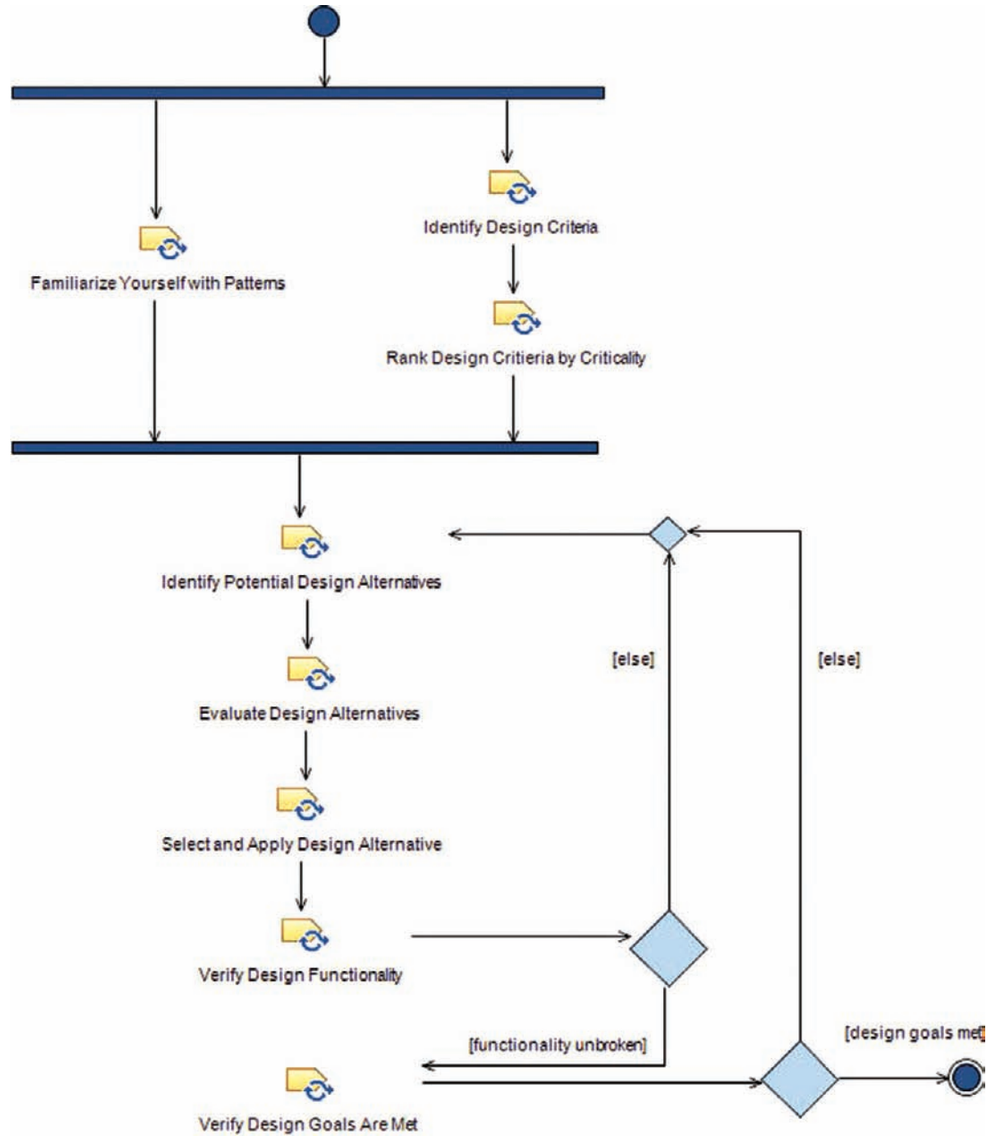


图2-5：模式孵化

6. 选择并应用设计方案
   一旦选定了设计方案，模式通常会被逐个添加到协作中，包括验证解决方案的有效性。

7. Verify design functionality
   The collaboration of software elements worked before applying the design patterns. This task ensures that it still does. Because the application of design patterns usually requires some (possibly small) amount of refactoring, some unit tests may require updates. If the pattern application has broken the functionality, the software must be repaired so that it passes its functional unit tests.
8. Verify design goals are met
   Once the design functionality is verified, we must also ensure that the optimization goals of the design decisions result in the right tradeoffs. If we've optimized the code for memory usage, the usage should be measured as the software runs. If we've optimized worst-case execution time, then that time should be measured.

2.1.5.2 Pattern Mining – Rolling Your Own Patterns

Creating your own pattern is useful, especially when you have a depth of experience to understand the optimization issues in a particular area, and sufficient breadth to understand the general properties of the solutions enough to abstract them into a generalized solution. We call this *pattern mining* (see Figure 2-6). Pattern mining isn't so much a matter of invention as it is

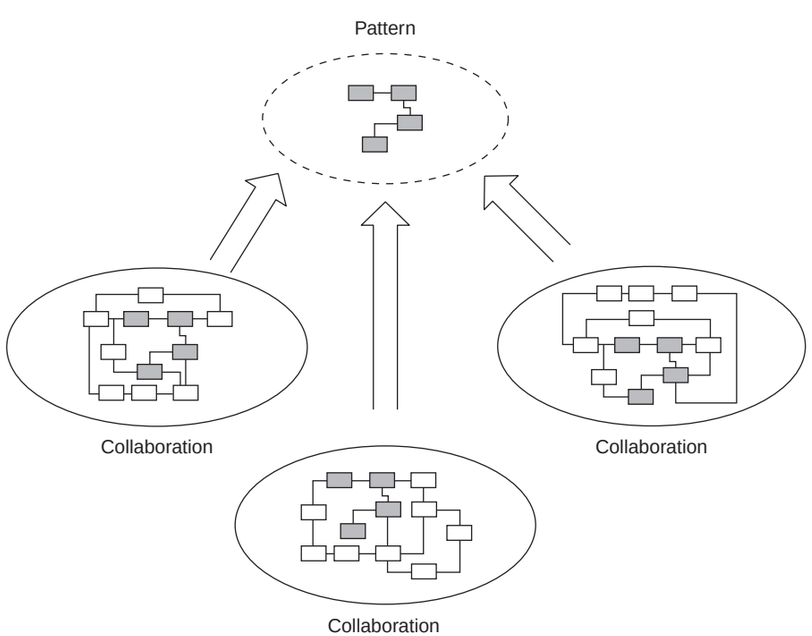


Figure 2-6: Pattern mining

7. 验证设计功能
   软件元素的协作在应用设计模式之前是正常工作的。此任务确保它仍然正常工作。由于应用设计模式通常需要一些（可能少量的）重构，某些单元测试可能需要更新。如果模式的应用破坏了功能，则必须修复软件，使其通过功能单元测试。

8. 验证设计目标是否达成
   一旦设计功能得到验证，我们还必须确保设计决策的优化目标能够带来正确的权衡。如果我们针对内存使用优化了代码，那么就应该在软件运行时测量内存使用情况。如果我们优化了最坏情况执行时间，那么就应该测量该时间。

2.1.5.2 模式挖掘——打造你自己的模式

创建自己的模式是有用的，尤其是当你拥有深厚的经验来理解特定领域中的优化问题，并且具备足够的广度来理解解决方案的一般属性，从而能够将它们抽象为泛化解决方案时。我们称之为模式挖掘（见图2-6）。模式挖掘与其说是发明，不如说是

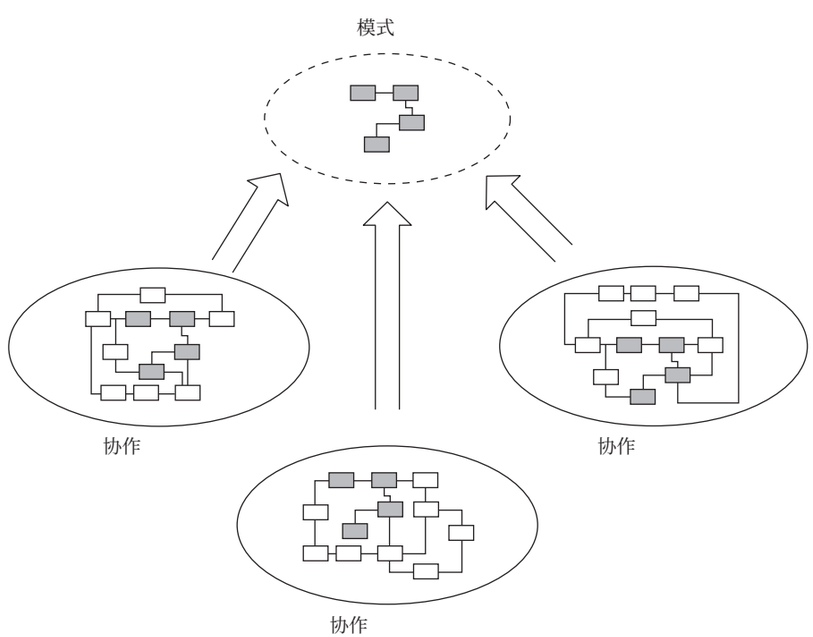


图 2-6：模式挖掘

discovery – seeing that this solution in some context is similar to another solution in another context and abstracting away the specifics of the solutions. Keep in mind that to be a useful pattern, it must occur in different contexts and perform a useful optimization of one or more design criteria.

2.1.5.3 Pattern Instantiation – Applying Patterns in Your Designs

Pattern instantiation is the opposite of pattern mining. It is applying the pattern to a particular collaboration to gain the benefits of the pattern (see Figure 2-7). Patterns are normally applied to a collaboration of software elements, such as classes, types, functions, and/or variables. As mentioned previously, a collaboration is a set of software elements (at some scope these may be small low-level functions or classes, while at others they may be large-grain assemblies such as subsystems and components). The purpose is to organize, and possibly elaborate, this already existing collaboration with the selected pattern.

The application or instantiation of a pattern in your design is a matter of identifying the elements in your collaboration that fulfill the pattern parameter roles. For some of the patterns you may create the role as a super class from which you subclass to instantiate that pattern. In other patterns, you may simply replace the pattern element with one from your application

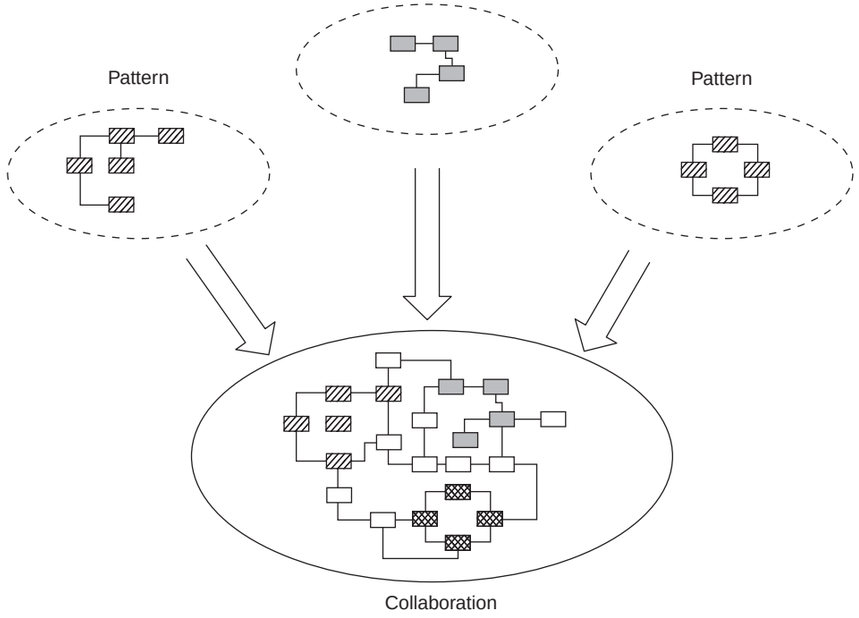


Figure 2-7: Pattern instantiation

发现——看到某个上下文中的这个解决方案与另一个上下文中的另一个解决方案相似，并抽象掉解决方案的具体细节。请记住，要成为一个有用的模式，它必须出现在不同的上下文中，并对一个或多个设计准则执行有用的优化。

2.1.5.3 模式实例化——在设计中应用模式

模式实例化是模式挖掘的逆过程。它是将模式应用于特定的协作，以获得模式带来的好处（见图2-7）。模式通常应用于软件元素的协作，例如类、类型、函数和/或变量。如前所述，协作是一组软件元素（在某些范围内，这些元素可能是小型的低级函数或类，而在其他范围内，它们可能是子系统、组件等大粒度组装件）。其目的是用选定的模式来组织、并可能细化这一已有的协作。

在设计中应用或实例化模式，关键在于识别协作中承担模式参数角色的元素。对于某些模式，你可以将角色创建为超类，再通过子类化来实例化该模式。在其他模式中，你只需将模式元素替换为应用中的元素

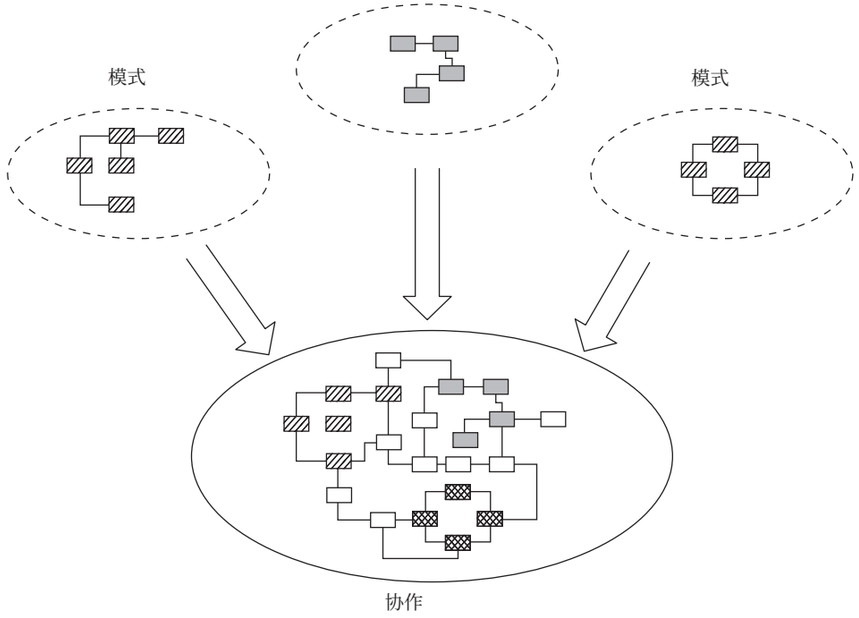


图2-7：模式实例化

domain, adding in the required operations and behavior. Or, you may choose to create elements just as they exist in the pattern itself.

2.1.5.4 An Example: The Observer Pattern

Let's consider a simple example. Figure 2-8 shows a system that contains an ECG module data source that pipes data into a queue for consumption of a number of different clients. The data include both a time interval marker (of type `long`) and the `dataValue`, a measurement of the potential difference between the selected ECG lead pair. The queue is known as a "leaky queue" in that data are never explicitly removed from the circular buffer, but are overwritten as the buffer inserts wraparound to the start of the queue.

The source code for a simple implementation of the elements is straightforward. The ECGPkh.h header file defines the queue size and the `boolean` type.

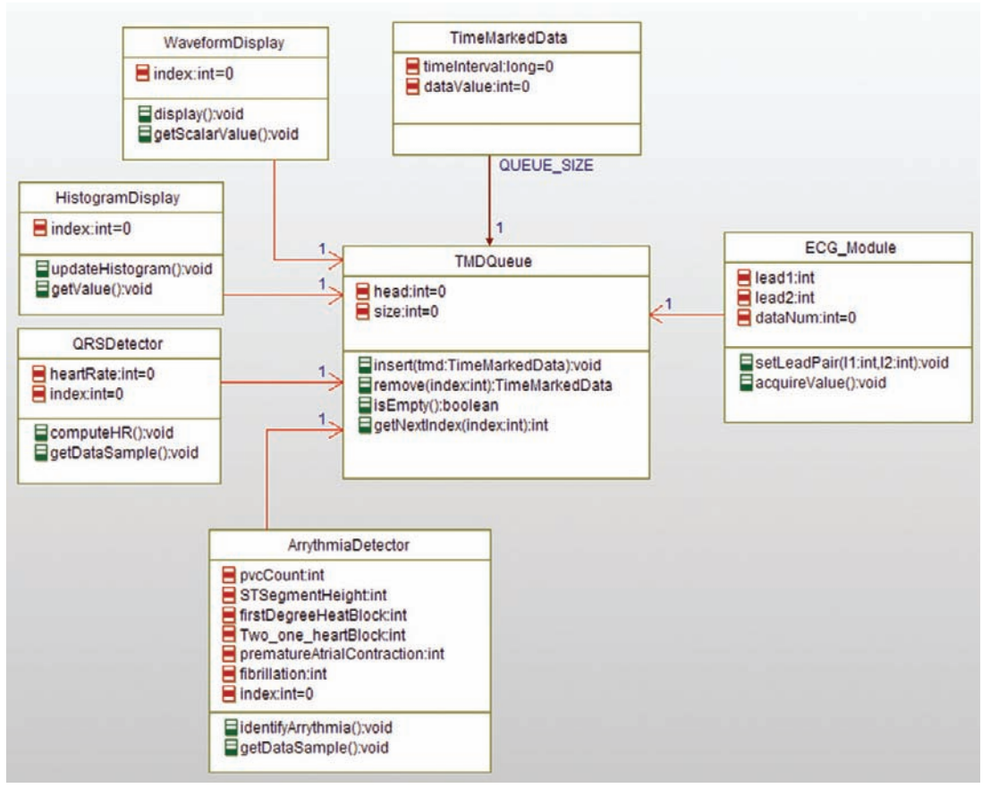


Figure 2-8: ECG monitoring system

领域，并添加所需的操作和行为。或者，你也可以选择完全按照模式本身所存在的方式创建元素。

2.1.5.4 示例：观察者模式

让我们 考虑 一个 简单的示例。 图 2-8 展示了一个系统，其中包含一个ECG模块数据源，该数据源将数据管道传输到队列中，供多个不同的客户端消费。数据包括一个时间间隔标记（类型为长整型）和`dataValue`，即所选ECG导联对之间电位差的测量值。该队列被称为“泄漏队列”，因为数据从未被显式地从环形缓冲区中移除，而是在缓冲区插入回绕到队列起始位置时被覆盖。

这些元素的简单实现的源代码很直接。ECGPkh.h 头文件定义了队列大小和 布尔值 类型。

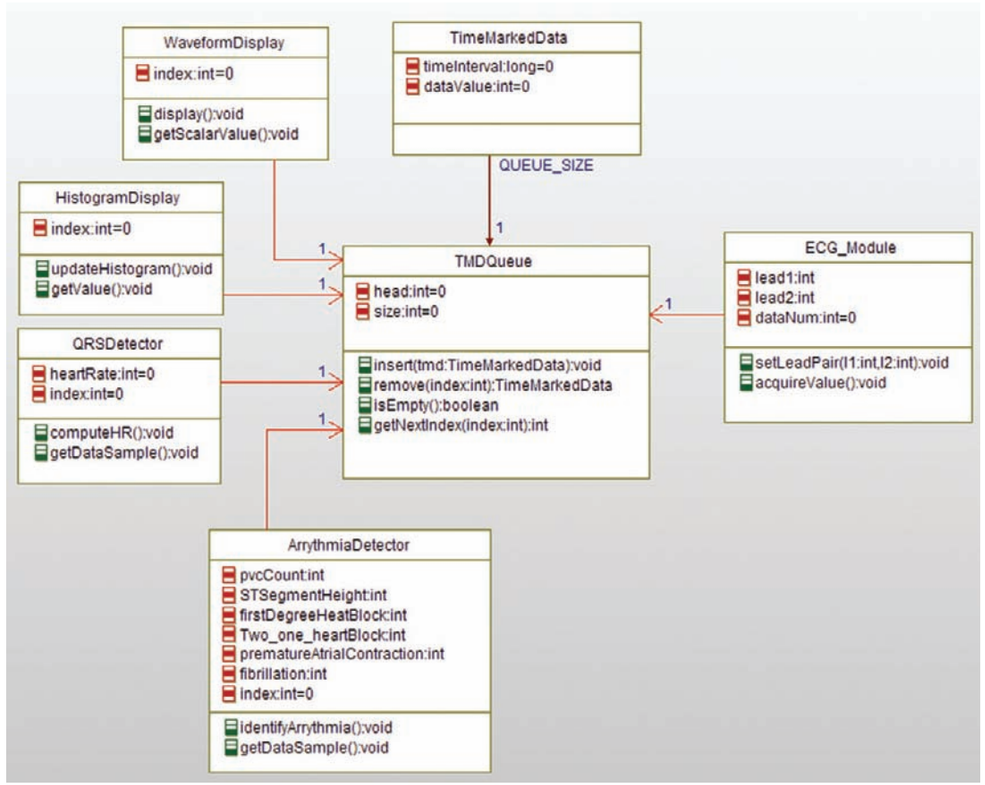


图2-8：ECG监护系统

```
#ifndef ECGPkg_H
#define ECGPkg_H

#include <stdio.h>

/* forward declarations */
struct ArrythmiaDetector;
struct ECG_Module;
struct HistogramDisplay;
struct QRSDetector;
struct TMDQueue;
struct TestBuilder;
struct TimeMarkedData;
struct WaveformDisplay;

typedef unsigned char boolean;
#define QUEUE_SIZE (20000)

#endif
```

Code Listing 2-1: ECGPkg.h Header File

The second header file (Code Listing 2-2) is the Timed Marked Data queue (TMDQueue). It holds the data inserted by the ECG_Module for its various data clients.

```
#ifndef TMDQueue_H
#define TMDQueue_H

/*## auto_generated */
#include <stdio.h>
#include "ECGPkg.h"
#include "TimeMarkedData.h"

typedef struct TMDQueue TMDQueue;
/*
This queue is meant to operate as a "leaky" queue. In this queue, data are never removed per
se, but are instead overwritten when the buffer pointer wraps around. This allows for many
clients to read the same data from the queue. */

struct TMDQueue {
    int head;
    int size;
    struct TimeMarkedData buffer[QUEUE_SIZE];
};

/* Constructors and destructors:*/
void TMDQueue_Init(TMDQueue* const me);
void TMDQueue_Cleanup(TMDQueue* const me);

/* Operations */
int TMDQueue_getNextIndex(TMDQueue* const me, int index);
```



```
#ifndef ECGPkg_H #
define ECGPkg_H

#include <stdio.h>

/* 前向声明 */ struct
ArrythmiaDetector; struct ECG_
Module; struct
HistogramDisplay; struct
QRSDetector; struct TMDQueue;
struct TestBuilder; struct
TimeMarkedData; struct
WaveformDisplay;


typedef 无符号字符 布尔值; #define
QUEUE_SIZE (20000)

#endif
```

代码清单 2-1：ECGPkg.h 头文件

第二个头文件（代码清单 2-2）是时间标记数据队列（TMDQueue）。它保存由心电图_模块为其各类数据客户端插入的数据。

```
#ifndef TMDQueue_H #
define TMDQueue_H

/*## auto_generated */ #include
<stdio.h>#include "ECGPkg.h" #
include "TimeMarkedData.h"


typedef struct TMDQueue TMDQueue;
/*
该队列被设计为一种“泄漏”队列。在此队列中，数据本身永远不会被移除，而是在缓冲区指针回绕时被覆盖。这使
得多个客户端可以从队列中读取相同的数据。*/


struct TMDQueue { int head; int size; struct
TimeMarkedData buffer[QUEUE_SIZE]; };



/* 构造函数与析构函数：*/ void TMDQueue_
Init(TMDQueue* const me); void TMDQueue_
Cleanup(TMDQueue* const me);

/* 操作 */ int TMDQueue_getNextIndex(TMDQueue* const me, int
index);
```

```
    void TMDQueue_insert(TMDQueue* const me, const struct TimeMarkedData tmd);
boolean TMDQueue_isEmpty(TMDQueue* const me);
struct TimeMarkedData TMDQueue_remove(TMDQueue* const me, int index);
int TMDQueue_getBuffer(const TMDQueue* const me);
TMDQueue * TMDQueue_Create(void);
void TMDQueue_Destroy(TMDQueue* const me);
#endif
```

Code Listing 2-2: TMDQueue Header File

Code Listing 2-3 shows the header file for the ECG_Module. The code here acquires the data across the selected lead pair and calls the `TMDQueue_insert()` function to store them.

```
#ifndef ECG_Module_H
#define ECG_Module_H
#include <stdio.h>
#include "ECGPkg.h"

struct TMDQueue;

/* class ECG_Module */
typedef struct ECG_Module ECG_Module;
struct ECG_Module {
    int dataNum;
    int lead1;
    int lead2;
    struct TMDQueue* itsTMDQueue;
};

/* Constructors and destructors:*/
void ECG_Module_Init(ECG_Module* const me);
void ECG_Module_Cleanup(ECG_Module* const me);

/* Operations */
void ECG_Module_acquireValue(ECG_Module* const me);
void ECG_Module_setLeadPair(ECG_Module* const me,
       int l1, int l2);

struct TMDQueue* ECG_Module_getItsTMDQueue(const ECG_Module* const me);

void ECG_Module_setItsTMDQueue(ECG_Module* const me, struct TMDQueue* p_TMDQueue);

ECG_Module * ECG_Module_Create(void);

void ECG_Module_Destroy(ECG_Module* const me);

#endif
```


void TMDQueue_insert(TMDQueue* const me, const struct TimeMarkedData tmd); boolean TMDQueue_isEmpty(TMDQueue* const me); struct TimeMarkedData TMDQueue_ remove(TMDQueue* const me, int index); int TMDQueue_getBuffer(const TMDQueue* c onst me); TMDQueue * TMDQueue_Create(void); void TMDQueue_Destroy(TMDQueue* const me); #endif

代码清单 2-2：TMDQueue头文件

代码清单 2-3 展示了心电图_模块的头文件。此处的代码获取所选导联对的数据，并调用 `TMDQueue_insert()` 函数来存储它们。

#ifndef ECG_Module_H #define ECG_Module_H #include <stdio.h># include "ECGPkg.h" struct TMDQueue; /* 类 ECG Module */_typedef struct ECG_Module ECG_Module; struct ECG_Module { int dataNum; int lead1; int lead2; struct TMDQueue* itsTMDQueue; }; /* 构造函数与析构函数: */ void ECG_Module_Init(ECG_Module* const me); void ECG_Module_ Cleanup(ECG_Module* const me); /* 操作 */ void ECG_Module_ acquireValue(ECG_Module* const me); void ECG_Module_ setLeadPair(ECG_Module* const me, int l1, int l2); struct TMDQueue* ECG_Module_getItsTMDQueue(const ECG_Module*

const me);

void 心电图_模块_setItsTMDQueue(心电图_模块* const me, 结构体 T

MDQueue* p_TMDQueue);

心电图_Module * 心电图_Module_Create(void);

void 心电图_Module_Destroy(心电图_Module* const me);

#endif

Code Listing 2-3: ECG_Module Header File

The clients, for the purpose of this discussion, look pretty much the same. They all have a get_xxx() function that pulls data out of the queue and does something interesting with them. Each client is required to maintain its own position within the queue so that it doesn't miss any data. For this reason, Code Listing 2-4 shows only the header file for the HistogramDisplay class.

```
#ifndef HistogramDisplay_H
#define HistogramDisplay_H

#include <stdio.h>
#include "ECGPkg.h"
struct TMDQueue;

/* class HistogramDisplay */
typedef struct HistogramDisplay HistogramDisplay;
struct HistogramDisplay {
    int index;
    struct TMDQueue* itsTMDQueue;
};

/* Constructors and destructors:*/
void HistogramDisplay_Init(HistogramDisplay* const me);
void HistogramDisplay_Cleanup(HistogramDisplay* const me);

/* Operations */
void HistogramDisplay_getValue(HistogramDisplay* const me);
void HistogramDisplay_updateHistogram(HistogramDisplay* const me);

struct TMDQueue* HistogramDisplay_getItsTMDQueue(const HistogramDisplay* const me);

void  HistogramDisplay_setItsTMDQueue(HistogramDisplay*  const  me,  struct  TMDQueue*
p_TMDQueue);

HistogramDisplay * HistogramDisplay_Create(void);

void HistogramDisplay_Destroy(HistogramDisplay* const me);

#endif
```

Code Listing 2-4: HistogramDisplay Header File

The TMDQueue.c implementation file is likewise simple enough, as shown in Code Listing 2-5.

```
#include "TMDQueue.h"

static void initRelations(TMDQueue* const me);

static void cleanUpRelations(TMDQueue* const me);
```


代码清单 2-3：ECG_模块头文件

就本次讨论而言，客户端看起来大致相同。它们都有一个 get_xxx() 函数，从队列中取出数据并对其进行一些有趣的处理。每个客户端都需要维护自己在队列中的位置，以免遗漏任何数据。因此，代码清单 2-4 仅展示了 HistogramDisplay 类的头文件。

```
#ifndef HistogramDisplay_H #define HistogramDisplay_H #include <stdio.h># include "ECGPkg.h" struct TMDQueue; /* 类 HistogramDisplay */ typedef struct HistogramDisplay HistogramDisplay; struct HistogramDisplay { int index; str uct TMDQueue* itsTMDQueue; }; /* 构造函数与析构函数：*/ void HistogramDisplay_Init(HistogramDisplay* const me); void HistogramDisplay_ Cleanup(HistogramDisplay* const me); /* 操作 */ void HistogramDisplay_ getValue(HistogramDisplay* const me); void HistogramDisplay_ updateHistogram(HistogramDisplay* const me);

struct TMDQueue* HistogramDisplay_getItsTMDQueue(const HistogramDisplay* const me);

void HistogramDisplay_setItsTMDQueue(HistogramDisplay* const me, 结构体 TMDQueue* p_ TMDQueue);

HistogramDisplay * HistogramDisplay_Create(void);

void HistogramDisplay_Destroy(HistogramDisplay* const me); #endif
```

代码清单 2-4：HistogramDisplay头文件

TMDQueue.c实现文件同样足够简单，如代码清单 2-5 所示。

```
#include "TMDQueue.h" static void
initRelations(TMDQueue* const me); static void
cleanUpRelations(TMDQueue* const me);
```

```
void TMDQueue_Init(TMDQueue* const me) {
    me->head=0;
    me->size=0;
    initRelations(me);
}

void TMDQueue_Cleanup(TMDQueue* const me) {
    cleanUpRelations(me);
}

/* operation getNextIndex(int) */
int TMDQueue_getNextIndex(TMDQueue* const me, int index) {
    /* this operation computes the next index from the first using modulo arithmetic
    */
    return (index+1) % QUEUE_SIZE;
}

/* operation insert(TimeMarkedData) */
void TMDQueue_insert(TMDQueue* const me, const struct TimeMarkedData tmd) {
   /* note that because we never 'remove' data from this leaky queue, size only increases to
   the queue size and then stops increasing. Insertion always takes place at the head.
    */
    printf("Inserting at: %d Data #: %d", me->head,
    tmd.timeInterval);
    me->buffer[me->head]=tmd;
    me->head=TMDQueue_getNextIndex(me, me->head);
    if (me->size < QUEUE_SIZE) ++me->size;
    printf(" Storing data value: %d\n", tmd.dataValue);
}

/* operation isEmpty() */
boolean TMDQueue_isEmpty(TMDQueue* const me) {
    return (boolean)(me->size == 0);
}

/* operation remove(int) */
struct TimeMarkedData TMDQueue_remove(TMDQueue* const me, int index) {
    TimeMarkedData tmd;
    tmd.timeInterval=-1; /* sentinel values */
    tmd.dataValue=-9999;

    if (!TMDQueue_isEmpty(me) &&
        (index>=0) && (index < QUEUE_SIZE)
        && (index < me->size)) {
        tmd=me->buffer[index];
    }
    return tmd;
}
```


void TMDQueue_Init(TMDQueue* const me) { me->head = 0; me->size = 0; initRelations(me); } void TMDQueue_Cleanup(TMDQueue* const me) { cleanUpRelations(me); } /* 操作 getNextIndex(int) */ int TMDQueue_getNextIndex(TMDQueue* const me, int index) { /* 此操作使用模运算从第一个索引计算下一个索引 */ return (index+1) % QUEUE_SIZE; } /* 操作 insert(TimeMarkedData) */ void TMDQueue_insert(TMDQueue* const me, const struct TimeMarkedData tmd) { /* 注意，由于我们从不从这个泄漏队列中‘移除’数据，size 只会增加到队列大小然后停止增加。插入始终发生在 head 处。 */ printf("Inserting at: %d Data #: %d", me->head, tmd.timeInterval); me->buffer[me->head] = tmd; me->head = TMDQueue_getNextIndex(me, me->head); if (me->size < QUEUE_SIZE) ++me->size; printf(" Storing data value: %d\n", tmd.dataValue); } /* 操作 isEmpty() */ boolean TMDQueue_isEmpty(TMDQueue* const me) { return (boolean)(me->size == 0); } /* 操作 remove(int) */ struct TimeMarkedData TMDQueue_remove(TMDQueue* const me, int index) { TimeMarkedData tmd; tmd.timeInterval=-1; /* 哨兵值 */ tmd.dataValue=-9999; if (!TMDQueue_isEmpty(me) && (index>=0) && (index < QUEUE_SIZE) && (index < me->size)) { tmd = me->buffer[index]; } return tmd; }

```
int TMDQueue_getBuffer(const TMDQueue* const me) {
    int iter = 0;
    return iter;
}

TMDQueue * TMDQueue_Create(void) {
    TMDQueue* me = (TMDQueue *) malloc(sizeof(TMDQueue));
    if(me!=NULL)
        {
            TMDQueue_Init(me);
        }
    return me;
}

void TMDQueue_Destroy(TMDQueue* const me) {
    if(me!=NULL)
        {
            TMDQueue_Cleanup(me);
        }
    free(me);
}

static void initRelations(TMDQueue* const me) {
    {
        int iter = 0;
        while (iter < QUEUE_SIZE){
            TimeMarkedData_Init(&((me->buffer)[iter]));
            TimeMarkedData__setItsTMDQueue(&((me->buffer)[iter]), me);
                iter++;
            }
        }
}

static void cleanUpRelations(TMDQueue* const me) {
    {
        int iter = 0;
        while (iter < QUEUE_SIZE){
                TimeMarkedData_Cleanup(&((me->buffer)[iter]));
                iter++;
            }
        }
}
```

Code Listing 2-5: TMDQueue.c Implementation File

The ECG_Module.c implementation file shows how data are inserted into the queue (Code Listing 2-6).

```
#include "ECG_Module.h"
#include "TMDQueue.h"
```


int TMDQueue_getBuffer(const TMDQueue* const me) { int iter = 0; return iter; } TMDQueue * TMDQueue_Create(void) { TMDQueue* me = (TMDQueue *) malloc(sizeof(TMDQueue)); if(me!=空) { TMDQueue_Init(me); } return me; } void TMDQueue_Destroy(TMDQueue* const me) { if(me!=空) { TMDQueue_Cleanup(me); } free(me); } static void initRelations(TMDQueue* const me) { { int iter = 0; while (iter < QUEUE_SIZE){ TimeMarkedData_Init(&((me->缓冲区)[iter])); TimeMarkedData__ setItsTMDQueue(&((me->缓冲区)[iter]), me); iter++; } } } static void cleanUpRelations(TMDQueue* const me) { { int iter = 0; while (iter < QUEUE_SIZE){ TimeMarkedData_Cleanup(&((me->缓冲区)[iter])); iter++; } } }

代码清单 2-5：TMDQueue.c 实现文件

ECG_Module.c 实现文件展示了如何将数据插入队列（代码清单 2-6）。

#include "ECG_Module.h" # include "TMDQueue.h"

```
#include <stdlib.h>

static void cleanUpRelations(ECG_Module* const me);

void ECG_Module_Init(ECG_Module* const me) {
    me->dataNum=0;
    me->itsTMDQueue=NULL;
}

void ECG_Module_Cleanup(ECG_Module* const me) {
    cleanUpRelations(me);
}

/* operation acquireValue() */
void ECG_Module_acquireValue(ECG_Module* const me) {
    // in actual implementation, this would return the
    // measured voltage across the lead pair
    TimeMarkedData tmd;
    tmd.dataValue=rand();
    tmd.timeInterval=++me->dataNum;
    TMDQueue_insert(me->itsTMDQueue, tmd);
}

/* operation setLeadPair(int,int) */
void ECG_Module_setLeadPair(ECG_Module* const me, int l1, int l2)
{
    me->lead1=l1;
    me->lead2=l2;
}

struct TMDQueue* ECG_Module_getItsTMDQueue(const ECG_Module* const me) {
    return (struct TMDQueue*)me->itsTMDQueue;
}

void ECG_Module_setItsTMDQueue(ECG_Module* const me, struct TMDQueue* p_TMDQueue) {
    me->itsTMDQueue=p_TMDQueue;
}

ECG_Module * ECG_Module_Create(void) {
    ECG_Module* me=(ECG_Module *) malloc(sizeof(ECG_Module));
    if(me!=NULL)
        {
            ECG_Module_Init(me);
        }
    return me;
}

void ECG_Module_Destroy(ECG_Module* const me) {
    if(me!=NULL)
        {
            ECG_Module_Cleanup(me);
        }
    free(me);
}
```


#include <stdlib.h>static void cleanUpRelations(ECG_Module* const me); void ECG_Module_Init(ECG_Module* const me) { me->dataNum = 0; me->itsTMDQueue = NULL; } void ECG_Module_Cleanup(ECG_Module* const me) { cleanUpRelations(me); } /* 操作 acquireValue() */ void ECG_Module_acquireValue(ECG_Module* const me) { // 在实际实现中，这里会返回 // 导联对上测得的电压 TimeMarkedData tmd; tmd.dataValue = rand(); tmd.timeInterval = ++me->dataNum; TMDQueue_insert(me->itsTMDQueue, tmd); } /* 操作 setLeadPair(int,int) */ void ECG_Module_setLeadPair(ECG_Module* const me, int l1, int l2) { me->lead1 = l1; me->lead2 = l2; } struct TMDQueue* ECG_Module_getItsTMDQueue(const ECG_Module* const me) { return (struct TMDQueue*)me->itsTMDQueue; } void ECG_Module_setItsTMDQueue(ECG_Module* const me, struct TMDQueue* p_TMDQueue) { me->itsTMDQueue = p_TMDQueue; } ECG_Module * ECG_Module_Create(void) { ECG_Module* me = (ECG_Module *) malloc(sizeof(ECG_Module)); if(me!=NULL) { ECG_Module_Init(me); } return me; } void ECG_Module_Destroy(ECG_Module* const me) { if(me!=NULL) { ECG_Module_Cleanup(me); } free(me); }

```
static void cleanUpRelations(ECG_Module* const me) {
    if(me->itsTMDQueue != NULL)
        {
            me->itsTMDQueue=NULL;
        }
}
```

Code Listing 2-6: ECG_Module.c

The HistogramDisplay.c implementation file in Code Listing 2-7 shows the use of the `TMDQueue_remove()` function.

```
#include "HistogramDisplay.h"
#include "TMDQueue.h"

static void cleanUpRelations(HistogramDisplay* const me);

void HistogramDisplay_Init(HistogramDisplay* const me) {
    me->index=0;
    me->itsTMDQueue=NULL;
}

void HistogramDisplay_Cleanup(HistogramDisplay* const me) {
    cleanUpRelations(me);
}

/* operation getValue() */
void HistogramDisplay_getValue(HistogramDisplay* const me) {
    TimeMarkedData tmd;
    tmd=TMDQueue_remove(me->itsTMDQueue, me->index);
    printf("    Histogram index: %d TimeInterval: %d DataValue:
        %d\n", me->index, tmd.timeInterval, tmd.dataValue);
    me->index=TMDQueue_getNextIndex(me->itsTMDQueue,
        me->index);
}

/* operation updateHistogram() */
void HistogramDisplay_updateHistogram(HistogramDisplay* const me)
 {
    /* put some histogram stuff here… */
}

struct TMDQueue* HistogramDisplay_getItsTMDQueue(const HistogramDisplay* const me) {
    return (struct TMDQueue*)me->itsTMDQueue;
}

void  HistogramDisplay_setItsTMDQueue(HistogramDisplay*  const  me,  struct  TMDQueue*
p_TMDQueue) {
    me->itsTMDQueue=p_TMDQueue;
}
```


static void cleanUpRelations(ECG_Module* const me) { if(me->itsTMDQueue != 空) { me->itsTMDQueue = 空; } }

代码清单 2-6：ECG_Module.c

代码清单 2-7 中的 HistogramDisplay.c 实现文件展示了 `TMDQueue_remove()` 函数的使用。

#include "HistogramDisplay.h" #include "TMDQueue.h" static void cleanUpRelations(HistogramDisplay* const me); void HistogramDisplay_ Init(HistogramDisplay* const me) { me->index = 0; me->itsTMDQueue = NULL; } void HistogramDisplay_Cleanup(HistogramDisplay* const me) { cleanUpRelations(me); } /* 操作 getValue() */ void HistogramDisplay_getValue(HistogramDisplay* const me) { TimeMarkedData tmd; tmd = TMDQueue_remove(me->itsTMDQueue, me->index); printf(" Histogram index: %d TimeInterval: %d DataValue: %d\n", me->index, tmd.timeInterval, tmd.dataValue); me->index = TMDQueue_getNextIndex(me->itsTMDQueue, me->index); } /* 操作 updateHistogram() */ void HistogramDisplay_updateHistogram(HistogramDisplay* const me) { /* 在此处放置一些直方图相关内容... */ } struct TMDQueue* HistogramDisplay_ getItsTMDQueue(const HistogramDisplay* co nst me) { return (struct TMDQueue*)me->itsTMDQueue; } void HistogramDisplay_ setItsTMDQueue(HistogramDisplay* const me, struct TMDQueue* p_TMDQueue) { me->itsTMDQueue = p_TMDQueue; }

```
HistogramDisplay * HistogramDisplay_Create(void) {
    HistogramDisplay* me = (HistogramDisplay *)
    malloc(sizeof(HistogramDisplay));
    if(me!=NULL)
        {
            HistogramDisplay_Init(me);
        }
    return me;
}

void HistogramDisplay_Destroy(HistogramDisplay* const me) {
    if(me!=NULL)
        {
            HistogramDisplay_Cleanup(me);
        }
    free(me);
}

static void cleanUpRelations(HistogramDisplay* const me) {
    if(me->itsTMDQueue != NULL)
        {
            me->itsTMDQueue = NULL;
        }
}
```

Code Listing 2-7: HistogramDisplay.c Implementation File

For testing, we can put instances of the classes into a TestBuilder class as shown in Figure 2-9.

```
#ifndef TestBuilder_H
#define TestBuilder_H

#include <stdio.h>
#include "ECGPkg.h"
#include "ArrythmiaDetector.h"
#include "ECG_Module.h"
#include "HistogramDisplay.h"
#include "QRSDetector.h"
#include "TMDQueue.h"
#include "WaveformDisplay.h"

/* class TestBuilder */
typedef struct TestBuilder TestBuilder;
struct TestBuilder {
    struct ArrythmiaDetector itsArrythmiaDetector;
    struct ECG_Module itsECG_Module;
    struct HistogramDisplay itsHistogramDisplay;
    struct QRSDetector itsQRSDetector;
    struct TMDQueue itsTMDQueue;
    struct WaveformDisplay itsWaveformDisplay;
};
```



```
HistogramDisplay * HistogramDisplay_Create(void) {
HistogramDisplay* me = (HistogramDisplay *)
malloc(sizeof(HistogramDisplay)); if(me!=NULL) { HistogramDisplay_
Init(me); } return me; } void HistogramDisplay_
Destroy(HistogramDisplay* const me) { if(me!=NULL) {
HistogramDisplay_Cleanup(me); } free(me); } static void
cleanUpRelations(HistogramDisplay* const me) { if(me->itsT
MDQueue != NULL) { me->itsTMDQueue = NULL; } }
```

代码清单 2-7：HistogramDisplay.c 实现文件

为了 测试， 我们 可以 将 实例 放入 这些 类 中， 即 TestBuilder 类 中， 如 图 2-9 所示。

```
#ifndef TestBuilder_H #define TestBuilder_H #include <
stdio.h>#include "ECGPkg.h" #include "
ArrythmiaDetector.h" #include "ECG_Module.h" #include "
HistogramDisplay.h" #include "QRSDetector.h" #include "
TMDQueue.h" #include "WaveformDisplay.h" /* 类
TestBuilder */ typedef struct TestBuilder TestBuilder; struct
TestBuilder { struct ArrythmiaDetector
itsArrythmiaDetector; struct ECG_Module itsECG_Module;
struct HistogramDisplay itsHistogramDisplay; struct
QRSDetector itsQRSDetector; struct TMDQueue
itsTMDQueue; struct WaveformDisplay
itsWaveformDisplay; };
```

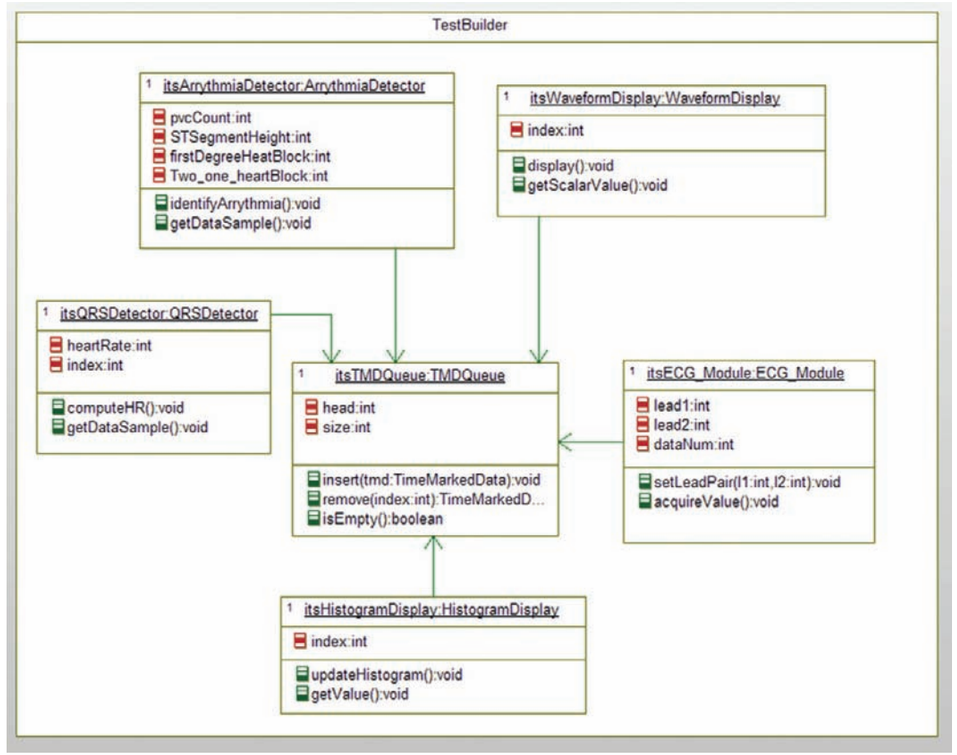


Figure 2-9: ECG system test environment builder class

```
/* Constructors and destructors:*/
void TestBuilder_Init(TestBuilder* const me);
void TestBuilder_Cleanup(TestBuilder* const me);

struct ArrythmiaDetector* TestBuilder_getItsArrythmiaDetector(const TestBuilder*
const me);

struct ECG_Module* TestBuilder_getItsECG_Module(const TestBuilder* const me);

struct HistogramDisplay* TestBuilder_getItsHistogramDisplay(const TestBuilder* const
me);

struct QRSDetector* TestBuilder_getItsQRSDetector(const TestBuilder* const me);

struct TMDQueue* TestBuilder_getItsTMDQueue(const TestBuilder* const me);

struct WaveformDisplay* TestBuilder_getItsWaveformDisplay(const TestBuilder* const
me);
```

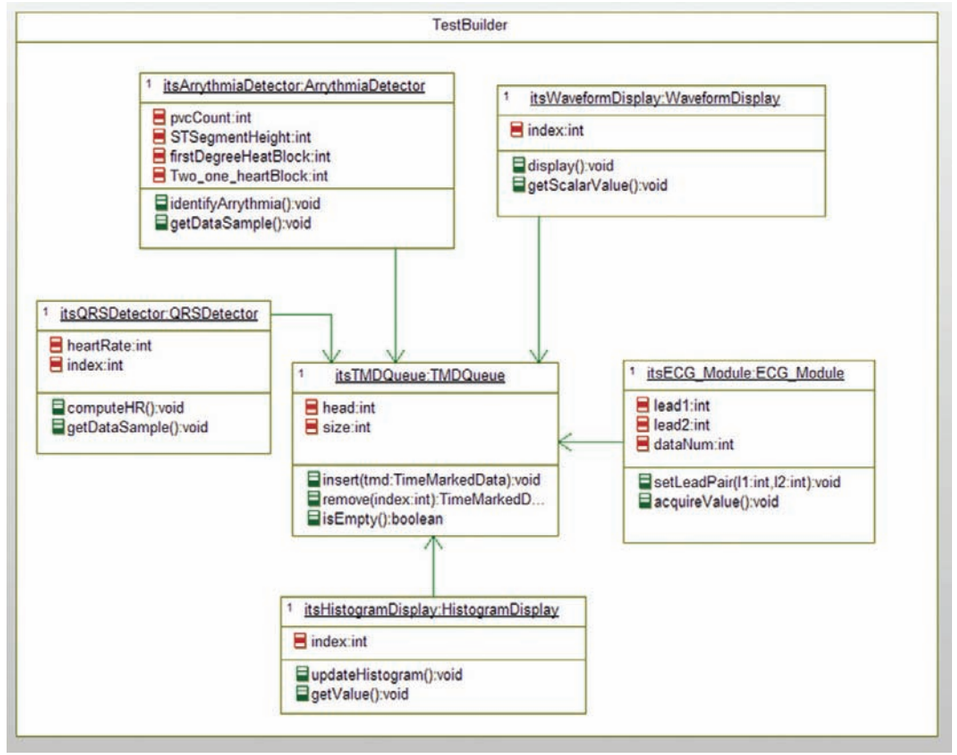



图2-9：ECG系统测试环境构建器类

```
/* 构造函数与析构函数：*/ void TestBuilder_I
nit(TestBuilder* const me); void TestBuilder_
Cleanup(TestBuilder* const me);

struct ArrythmiaDetector* TestBuilder_getItsArrythmiaDetector(const TestBuilder* const me);

struct EC   G_Module* TestBuilder_getItsECG_Module(const TestBuilder* const me);

struct HistogramDisplay* TestBuilder_getItsHistogramDisplay(const TestBuilder* const me);

struct QRSDetector* TestBuilder_getItsQRSDetector(const TestBuilder* const me);

struct TMDQueue* TestBuilder_getItsTMDQueue(const TestBuilder* const me)          ;

struct WaveformDisplay* TestBuilder_getItsWaveformDisplay(const TestBuilder* const me);
```

```
TestBuilder * TestBuilder_Create(void);

void TestBuilder_Destroy(TestBuilder* const me);

#endif
```

Code Listing 2-8: TestBuilder.h Header File

```
#include "TestBuilder.h"

static void initRelations(TestBuilder* const me);
static void cleanUpRelations(TestBuilder* const me);

void TestBuilder_Init(TestBuilder* const me) {
    initRelations(me);
}

void TestBuilder_Cleanup(TestBuilder* const me) {
    cleanUpRelations(me);
}

struct ArrythmiaDetector* TestBuilder_getItsArrythmiaDetector(const TestBuilder*
const me)
{
    return (struct ArrythmiaDetector*)&(me->itsArrythmiaDetector);
}

struct ECG_Module* TestBuilder_getItsECG_Module(const TestBuilder* const me) {
    return (struct ECG_Module*)&(me->itsECG_Module);
}

struct HistogramDisplay* TestBuilder_getItsHistogramDisplay(const TestBuilder* const
me) {
    return (struct HistogramDisplay*)&(me->itsHistogramDisplay);
}

struct QRSDetector* TestBuilder_getItsQRSDetector(const TestBuilder* const me) {
    return (struct QRSDetector*)&(me->itsQRSDetector);
}

struct TMDQueue* TestBuilder_getItsTMDQueue(const TestBuilder* const me) {
    return (struct TMDQueue*)&(me->itsTMDQueue);
}

struct WaveformDisplay* TestBuilder_getItsWaveformDisplay(const TestBuilder* const me)
{
    return (struct WaveformDisplay*)&(me->itsWaveformDisplay);
}

TestBuilder * TestBuilder_Create(void) {
    TestBuilder* me = (TestBuilder *)
malloc(sizeof(TestBuilder));
    if(me!=NULL)
```


TestBuilder * TestBuilder_Create(void); void TestBuilder_
Destroy(TestBuilder* const me);

#endif

代码清单 2-8：TestBuilder.h 头文件

#include "TestBuilder.h" static void initRelations(TestBuilder* const me); static void cleanUpRelations(TestBuilder* const me); void TestBuilder_Init(TestBuilder* const me) { initRelations(me); } void TestBuilder_Cleanup(TestBuilder* const me) { cleanUpRelations(me); } struct ArrythmiaDetector* TestBuilder_getItsArrythmiaDetector(const TestBuilder* const me) { return (struct ArrythmiaDetector*)&(me->itsArrythmiaDetector); } struct ECG_Module* TestBuilder_getItsECG_Module(const TestBuilder* const me) { return (struct ECG_Module*)&(me->itsECG_Module); } struct HistogramDisplay* TestBuilder_getItsHistogramDisplay(const TestBuilder* const me) { return (struct HistogramDisplay*)&(me->itsHistogramDisplay); } struct QRSDetector* TestBuilder_getItsQRSDetector(const TestBuilder* const me) { return (struct QRSDetector*)&(me->itsQRSDetector); } struct TMDQueue* TestBuilder_getItsTMDQueue(const TestBuilder* const me) { return (struct TMDQueue*)&(me->itsTMDQueue); } struct WaveformDisplay* TestBuilder_getItsWaveformDisplay(const TestBuilder* const me) { return (struct WaveformDisplay*)&(me->itsWaveformDisplay); } TestBuilder * TestBuilder_Create(void) { TestBuilder* me = (TestBuilder *) malloc(sizeof(TestBuilder)); if(me!=NULL)

```
        {
            TestBuilder_Init(me);
        }
    return me;
}

void TestBuilder_Destroy(TestBuilder* const me) {
    if(me!=NULL)
        {
            TestBuilder_Cleanup(me);
        }
    free(me);
}

static void initRelations(TestBuilder* const me) {
    ArrythmiaDetector_Init(&(me->itsArrythmiaDetector));
    ECG_Module_Init(&(me->itsECG_Module));
    HistogramDisplay_Init(&(me->itsHistogramDisplay));
    QRSDetector_Init(&(me->itsQRSDetector));
    TMDQueue_Init(&(me->itsTMDQueue));
    WaveformDisplay_Init(&(me->itsWaveformDisplay));
    ECG_Module_setItsTMDQueue(&(me->itsECG_Module),
        &(me->itsTMDQueue));
    HistogramDisplay_setItsTMDQueue(&(me->itsHistogramDisplay),
        &(me->itsTMDQueue));
    QRSDetector_setItsTMDQueue(&(me->itsQRSDetector),
        &(me->itsTMDQueue));
    WaveformDisplay_setItsTMDQueue(&(me->itsWaveformDisplay),
        &(me->itsTMDQueue));
    ArrythmiaDetector_setItsTMDQueue(&(me->itsArrythmiaDetector),
        &(me->itsTMDQueue));
}

static void cleanUpRelations(TestBuilder* const me) {
    WaveformDisplay_Cleanup(&(me->itsWaveformDisplay));
    TMDQueue_Cleanup(&(me->itsTMDQueue));
    QRSDetector_Cleanup(&(me->itsQRSDetector));
    HistogramDisplay_Cleanup(&(me->itsHistogramDisplay));
    ECG_Module_Cleanup(&(me->itsECG_Module));
    ArrythmiaDetector_Cleanup(&(me->itsArrythmiaDetector));
}
```

Code Listing 2-9: TestBuilder.c Implementation File

Lastly, Code Listing 2-10 shows a simple use of the various classes to insert some data into the queue and have the clients pull them out for their use.

```
int main(int argc, char* argv[]) {
    int status = 0;

    TestBuilder * p_TestBuilder;
```



```
{ TestBuilder_Init(me); } return me; } void TestBuilder_
Destroy(TestBuilder* const me) { if(me!=空) { TestBuilder_Cleanup(me); }
free(me); } static void initRelations(TestBuilder* const me) { Arryth
miaDetector_Init(&(me->itsArrythmiaDetector)); ECG_Module_Init(&(me->
itsECG_Module)); HistogramDisplay_Init(&(me->itsHistogramDisplay));
QRSDetector_Init(&(me->itsQRSDetector)); TMDQueue_Init(&(me->
itsTMDQueue)); WaveformDisplay_Init(&(me->itsWaveformDisplay)); ECG_
Module_setItsTMDQueue(&(me->itsECG_Module), &(me->itsTMDQueue));
HistogramDisplay_setItsTMDQueue(&(me->itsHistogramDisplay), &(me->
itsTMDQueue)); QRSDetector_setItsTMDQueue(&(me->itsQRSDetector), &(me->
itsTMDQueue)); WaveformDisplay_setItsTMDQueue(&(me->
itsWaveformDisplay), &(me->itsTMDQueue)); ArrythmiaDetector_
setItsTMDQueue(&(me->itsArrythmiaDetector), &(me->itsTMDQueue)); } static
void cleanUpRelations(TestBuilder* const me) { WaveformDisplay_Cleanup(&
(me->itsWaveformDisplay)); TMDQueue_Cleanup(&(me->itsTMDQueue));
QRSDetector_Cleanup(&(me->itsQRSDetector)); HistogramDisplay_Cleanup(&
(me->itsHistogramDisplay)); ECG_Module_Cleanup(&(me->itsECG_Module));
ArrythmiaDetector_Cleanup(&(me->itsArrythmiaDetector)); }
```

代码清单 2-9：TestBuilder.c 实现文件

最后，代码清单 2-10 展示了各种类的简单用法，将一些数据插入队列，并让客户端将其取出以供使用。

```
int main(int argc, char* argv[]) { int
status = 0; TestBuilder * p_
TestBuilder;
```

```
    p_TestBuilder=TestBuilder_Create();
    ECG_Module_acquireValue(&(p_TestBuilder->itsECG_Module));
    ECG_Module_acquireValue(&(p_TestBuilder->itsECG_Module));
    ECG_Module_acquireValue(&(p_TestBuilder->itsECG_Module));
    ECG_Module_acquireValue(&(p_TestBuilder->itsECG_Module));
    ECG_Module_acquireValue(&(p_TestBuilder->itsECG_Module));

    HistogramDisplay_getValue(&(p_TestBuilder
        ->itsHistogramDisplay));
    HistogramDisplay_getValue(&(p_TestBuilder
        ->itsHistogramDisplay));
    HistogramDisplay_getValue(&(p_TestBuilder
        ->itsHistogramDisplay));
    HistogramDisplay_getValue(&(p_TestBuilder
        ->itsHistogramDisplay));
    HistogramDisplay_getValue(&(p_TestBuilder
        ->itsHistogramDisplay));

    QRSDetector_getDataSample(&(p_TestBuilder->itsQRSDetector));
    QRSDetector_getDataSample(&(p_TestBuilder->itsQRSDetector));
    QRSDetector_getDataSample(&(p_TestBuilder->itsQRSDetector));

    WaveformDisplay_getScalarValue(&(p_TestBuilder
        ->itsWaveformDisplay));
    WaveformDisplay_getScalarValue(&(p_TestBuilder
        ->itsWaveformDisplay));
    WaveformDisplay_getScalarValue(&(p_TestBuilder
        ->itsWaveformDisplay));
    WaveformDisplay_getScalarValue(&(p_TestBuilder
        ->itsWaveformDisplay));
    WaveformDisplay_getScalarValue(&(p_TestBuilder
        ->itsWaveformDisplay));

    ArrythmiaDetector_getDataSample(&(p_TestBuilder
        ->itsArrythmiaDetector));
    ArrythmiaDetector_getDataSample(&(p_TestBuilder
        ->itsArrythmiaDetector));
    ArrythmiaDetector_getDataSample(&(p_TestBuilder
        ->itsArrythmiaDetector));
    ArrythmiaDetector_getDataSample(&(p_TestBuilder
        ->itsArrythmiaDetector));
    ArrythmiaDetector_getDataSample(&(p_TestBuilder
        ->itsArrythmiaDetector));

    printf("Done\n");
    TestBuilder_Destroy(p_TestBuilder);
    return 0;
}
```

Code Listing 2-10: Sample main.c

The code presented above works in a classic client-server approach. The `ECG_Monitor` puts data into the queue when it thinks it should and the various clients all individually decide when


```
p_TestBuilder = TestBuilder_Create(); 心电图_Module_acquireValue(&(p_
TestBuilder->itsECG_Module)); 心电图_Module_acquireValue(&(p_
TestBuilder->itsECG_Module)); 心电图_Module_acquireValue(&(p_
TestBuilder->itsECG_Module)); 心电图_Module_acquireValue(&(p_
TestBuilder->itsECG_Module)); 心电图_Module_acquireValue(&(p_
TestBuilder->itsECG_Module)); HistogramDisplay_getValue(&(p_
TestBuilder ->itsHistogramDisplay)); HistogramDisplay_getValue(&(p_
TestBuilder ->itsHistogramDisplay)); HistogramDisplay_getValue(&(p_
TestBuilder ->itsHistogramDisplay)); HistogramDisplay_getValue(&(p_
TestBuilder ->itsHistogramDisplay)); HistogramDisplay_getValue(&(p_
TestBuilder ->itsHistogramDisplay)); QRSDetector_getDataSample(&(p_
TestBuilder->itsQRSDetector)); QRSDetector_getDataSample(&(p_
TestBuilder->itsQRSDetector)); QRSDetector_getDataSample(&(p_
TestBuilder->itsQRSDetector)); WaveformDisplay_getScalarValue(&(p_
TestBuilder ->itsWaveformDisplay)); WaveformDisplay_getScalarValue(&
(p_TestBuilder ->itsWaveformDisplay)); WaveformDisplay_
getScalarValue(&(p_TestBuilder ->itsWaveformDisplay));
WaveformDisplay_getScalarValue(&(p_TestBuilder ->
itsWaveformDisplay)); WaveformDisplay_getScalarValue(&(p_
TestBuilder ->itsWaveformDisplay)); ArrythmiaDetector_getDataSample(&
(p_TestBuilder ->itsArrythmiaDetector)); ArrythmiaDetector_
getDataSample(&(p_TestBuilder ->itsArrythmiaDetector));
ArrythmiaDetector_getDataSample(&(p_TestBuilder ->
itsArrythmiaDetector)); ArrythmiaDetector_getDataSample(&(p_
TestBuilder ->itsArrythmiaDetector)); ArrythmiaDetector_get
DataSample(&(p_TestBuilder ->itsArrythmiaDetector)); printf("Done\n");
TestBuilder_Destroy(p_TestBuilder); return 0;
}
```

代码清单 2-10：示例 main.c

上面展示的代码采用经典的客户端-服务器方式工作。当心电图_Monitor 认为应该时，就将数据放入队列，而各个客户端都各自决定何时

it is time to pull the data from the queue and do their work. In this case, the `TMDQueue` is the server of the data for the clients, and the `WaveformDisplay`, `HistogramDisplay`, `QRSDetector`, and `ArrythmiaDetector` are the clients.

We have a functioning model (with functioning code) for the system. Let's now think about how we'd like to optimize the system. In this case, we'd like to optimize execution time because we'd like to free up processor cycles for other work. In addition, we'd like to optimize maintainability of the system, limiting the number of places we'd like to make changes to the system when it needs to be updated. We'd also like to be able to dynamically (at run-time) vary the number of clients without wasting extra memory by allocating a potentially large array of pointers (1 per client) that might never be used. So our design criteria (with scaled criticality weights) are: 1) execution efficiency (weight = 7), 2) maintainability (weight = 5), 3) flexibility to add new clients easily at run-time (weight = 4), and 4) memory efficiency at the larger scale (weight = 7). Let's consider three alternative strategies (design patterns).

First, we have the current client-server approach:

1. Execution efficiency
   This solution is not particularly optimal because the clients need to check frequently if there are new data, even if there are not. On a scale of 0 to 10, let's call that a 3. This is the primary weakness of the approach.
2. Maintainability
   In a classic client-server architecture, the clients know about the servers (so they can invoke the services) but the servers don't know about the clients. That limitation on knowledge makes the system more maintainable because knowledge is encapsulated in one place. It earns a score of 7 here.
3. Run-time flexibility
   In this context flexibility primarily translates to the ability to create new clients and easily link them into the server. Since we can adapt it to handle those concerns, it earns an 8 on this criterion.
4. Memory usage
   In this criterion, each client must maintain its position in the list, which requires additional memory in each client. Therefore, the classic client-server approach only rates a 5 on this criterion.

The second alternative approach is to create a "push" implementation in which the responsibility for notifying the clients is given to the TMDQueue. To implement this, the direction of the associations in Figure 2-8 must be reversed, and the TMDQueue must explicitly know about all of its clients to notify them when data become available. The client list would probably be maintained as a list of pointers to clients.

该从队列中拉取数据并执行其工作。在这种情况下，`TMDQueue` 是客户端的服务器，而 `WaveformDisplay`、`HistogramDisplay`、`QRSDetector`和`ArrythmiaDetector` 则是客户端。

我们有一个可运行的系统模型（带有可运行的代码）。现在让我们思考如何优化该系统。在本例中，我们希望优化执行时间，因为我们希望释放处理器周期用于其他工作。此外，我们希望优化系统的可维护性，减少系统需要更新时需要修改的地方数量。我们还希望能够在运行时动态改变客户端数量，而不通过分配一个可能永远不会使用的大型指针数组（每个客户端一个指针）来浪费额外内存。因此，我们的设计准则（带有缩放的关键性权重）是：1）执行效率（权重＝7），2）可维护性（权重＝5），3）在运行时轻松添加新客户端的灵活性（权重＝4），以及4）在较大规模下的内存效率（权重＝7）。让我们考虑三种替代策略（设计模式）。

首先，我们来看当前的客户端-服务器方法：

1. 执行效率 该解决方案并非特别理想，因为客户端需要频繁检查是否有新数据，即使没有新数据也是如此。在0到10的评分标准下，我们给它打3分。这是该方法的主要弱点。2. 可维护性 在经典的客户端-服务器架构中，客户端了解服务器（因此可以调用服务），但服务器不了解客户端。这种知识上的限制使系统更易于维护，因为知识被封装在一个地方。此项得分为7分。3. 运行时灵活性 在此上下文中，灵活性主要体现为创建新客户端并将其轻松链接到服务器的能力。由于我们可以调整它以处理这些关注点，因此在该标准上得8分。4. 内存使用 在该标准下，每个客户端必须维护其在列表中的位置，这需要在每个客户端中占用额外内存。因此，经典的客户端-服务器方法在该标准上仅得5分。

第二种替代方法是创建一种“推送”实现，其中通知客户端的责任被赋予TMDQueue。为实现这一点，必须反转图2-8中关联的方向，并且TMDQueue必须显式地了解其所有客户端，以便在数据可用时通知它们。客户端列表可能被维护为一个指向客户端的指针列表。

1. Execution efficiency
   This approach is efficient in the sense that the clients are notified when data are available and don't need to poll to find discover new data. Let's give this approach an 8 here.
2. Maintainability
   The push approach breaks the classic client-server architecture (the servers know about the clients), so it only rates a 4 on this criterion. It will be harder to maintain if we add more clients.
3. Run-time flexibility
   The push approach has reasonably good run-time flexibility because it means that the server must be modified to accept new clients. It rates a 7 here.
4. Memory usage
   The approach uses memory only when a client is added, so it gets a 9 on this criterion.

The last alternative is the observer pattern[9]. The observer pattern is shown in Figure 2-10.

Before we use the pattern, let's look at its description and properties. Note that in the description that follows, the classic observer pattern has been modified slightly to fit into the example problem easier.

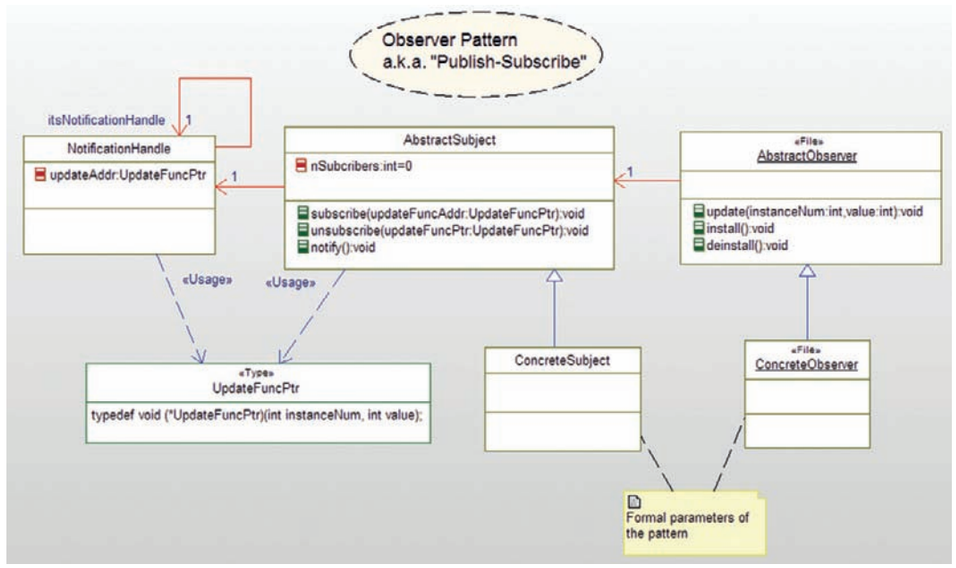


Figure 2-10: Observer pattern structure

[9] See, for example, Douglass, B.P., 2002. *Real-Time Design Patterns: Robust Scalable Architecture for Real-Time Systems*. Addison-Wesley; or Gamma, E., Helm, R., Johnson, R.,Vlissides, J.M., 1995. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.

1. 执行效率 这种方法在某种意义上是高效的，即客户端在数据可用时被通知，无需轮询来发现新数据。我们给这种方法打8分。2. 可维护性 推送方式打破了经典的客户端-服务器架构（服务器了解客户端），因此在这一标准上仅得4分。如果添加更多客户端，维护将更加困难。3. 运行时灵活性 推送方式具有相当好的运行时灵活性，因为它意味着必须修改服务器以接受新客户端。它在此得7分。4. 内存使用 该方法仅在添加客户端时使用内存，因此在这一标准上得9分。

最后一种替代 方案 是 观察者 模式[9]。 观察者 模式 如 图 2-10 所示 .

在使用该模式之前，让我们先看看它的描述和属性。请注意，在接下来的描述中，经典的观察者模式已经做了轻微修改，以便更容易地融入示例问题中。

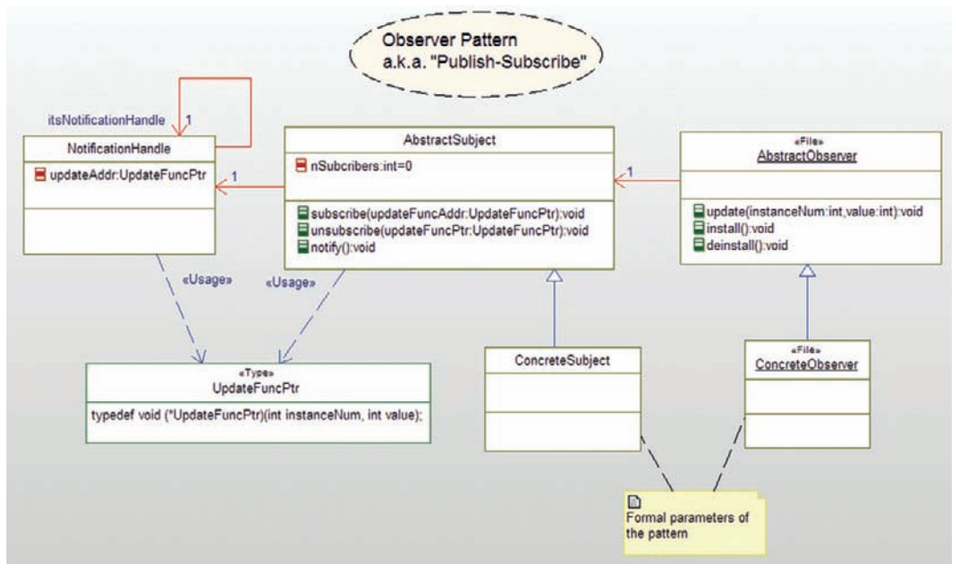


图 2-10：观察者模式结构

[9] 参见，例如，Douglass, B.P., 2002. Real-Time Design Patterns: Robust Scalable Architecture for Real-Time Systems. Addison-Wesley；或 Gamma, E., Helm, R., Johnson, R., Vlissides, J.M., 1995. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley。

2.1.6 Observer Pattern

- Abstract
  The observer pattern addresses the concern of dynamically adding and removing clients of data or services, as well as optimizing the processing associated with updating those clients in a timely way.
- Problem
  This pattern maintains the class client-server architecture in which the clients know about a server that has either data or services of interest, but that knowledge is one way. In spite of this, the responsibility for updating the clients in an efficient and timely way is given to the server. In addition, because the server maintains a list of interested clients, it notifies all clients when it is appropriate to do so, such as periodically or when the data changes. Clients can be dynamically added to or removed from the server as the system runs, while not requiring the server to have any knowledge of the clients (other than the location of an operation of the correct signature).
- Pattern structure
  This pattern defines the following roles:
  - Abstract Observer
    This element has a one-way association (link) to the Abstract Subject so that it can request to be added to the list of interested clients (a process known as *subscribing*) or removed from the list (known as *unsubscribing*). It provides an operation with a specific signature to be invoked by the subject when appropriate.
  - Abstract Subject
    This element will serve as the base class for the server in the client-server architecture; it has information or services needed by the Abstract Observer. It provides operations to add or remove clients (subscribe and unsubscribe, respectively) and can notify all currently subscribed clients when appropriate. Most common strategies for notification are either periodically or when the data change. This element creates a new notification handle for each subscribed client, forming a list of notification handles. While this list can be managed in any number of ways, in this case it will be handled as a linked list. Notification of the clients is simply a matter of "walking the list" and invoking the update operation for each Abstract Observer currently subscribed.
  - Notification handle
    This element provides a dynamic callback link for the Abstract Subject to the Abstract Observer; specifically, it has, as a data element, a pointer to the update operation of the server. It also has a link to the next element in the list which is NULL at the end of the list.
  - Concrete Observer (formal parameter)
    This element is the subclass of the Abstract Observer; it inherits the link to the Abstract Subject and provides a (nonimplemented) update operation that will be invoked by the Concrete Subject.

2.1.6 观察者模式

- 摘要
  观察者模式解决了动态添加和移除数据或服务的客户端的问题，以及优化与及时更新这些客户端相关的处理。

- 问题
  该模式保持了类客户端-服务器架构，其中客户端知道拥有感兴趣的数据或服务的服务器，但这种知识是单向的。尽管如此，以高效且及时的方式更新客户端的责任被赋予服务器。此外，由于服务器维护着感兴趣客户端的列表，它在适当的时候通知所有客户端，例如周期性地或当数据发生变化时。

  客户端可以在系统运行时动态地添加到服务器或从服务器移除，同时不要求服务器对客户端有任何了解（除了具有正确签名的操作的位置）。

- 模式结构
  该模式定义了以下角色：
  - 抽象观察者 该元素与抽象主题之间具有单向关联（链接），以便它可以请求加入感兴趣客户端列表（这一过程称为订阅）或从列表中移除（称为取消订阅）。它提供一个具有特定签名的操作，供主题在适当时机调用。
  - 抽象主题
    该元素将作为客户端-服务器架构中服务器的基类；它拥有抽象观察者所需的信息或服务。它提供添加或移除客户端的操作（分别为订阅和取消订阅），并可在适当时机通知所有当前已订阅客户端。最常见的通知策略是周期性通知或数据变化时通知。该元素为每个已订阅客户端创建一个新的通知句柄，形成通知句柄链表。虽然该列表可以通过多种方式管理，但在本例中将作为链表处理。通知客户端只需"遍历链表"并为每个当前已订阅的抽象观察者调用更新操作即可。
  - 通知句柄 该元素为抽象主题到抽象观察者提供动态回调链接；具体而言，它包含一个数据元素，即指向服务器更新操作的指针。它还包含一个指向列表中下一个元素的链接，该链接在列表末尾为空。
  - 具体观察者（形式参数）此元素是抽象观察者的子类；它继承指向抽象主题的链接，并提供一个（未实现的）更新操作，该操作将由具体主题调用。

- Concrete Subject (formal parameter)
  This element is the subclass of the Abstract Subject; it inherits the ability to add and remove clients and notify the clients when appropriate. It also provides the application-specific data or services of interest to the Concrete Observer.
- Consequences
  This pattern simplifies the process for dynamically adding and removing clients, especially ones that may not have even been defined when the server was created. In addition, because the server can decide when it is appropriate to update the clients, it results in timely delivery of information and services. This delivery is also efficient because the clients need not poll the server for updated data only to find the data are the same.

Now, let's look at how this solution stacks up.

1. Execution efficiency
   This approach is efficient in the sense that the client gets data functionality only when there is a need similar to the push approach. It rates an 8 here.
2. Maintainability
   This solution maintains the classic client-server architecture data encapsulation, improving its maintainability. The solution earns a score of 7 here.
3. Run-time flexibility
   The observer pattern has good run-time flexibility; the only requirement on the client is that it provides an update method with the appropriate signature. It rates a 9 here.
4. Memory usage
   Similar to the push approach, this pattern uses less memory because the clients need not track their position within the data buffer, so it gets a 9 on this criterion.

The results of the analysis are shown in Table 2-4. Based on the weighted criteria, the observer pattern looks to be the best of the design alternatives examined.

Table 2-4: Design tradeoffs for ECG monitor system.

Design Solution	Design Criteria				Total Weighted Score
	Efficiency	Maintainability	Flexibility	Memory Usage	
	Weight = 7	Weight = 5	Weight = 4	Weight = 7	
	Score	Score	Score	Score	
Client Server	3	7	8	5	123
Push	8	4	7	9	167
Observer	**8**	**7**	**9**	**9**	**190**

- 具体主题（形式参数）此元素是抽象主题的子类；它继承添加和移除客户端以及在适当时通知客户端的能力。它还提供具体观察者所关注的应用程序特定数据或服务。

- 结果
  该模式简化了动态添加和移除客户端的过程，尤其是那些在创建服务器时可能尚未定义的客户端。此外，由于服务器可以决定何时适合更新客户端，因此能够及时交付信息和服务。这种交付也很高效，因为客户端无需轮询服务器以获取更新数据，却只发现数据并未改变。

现在，让我们看看这个解决方案的表现如何。

1. 执行效率 这种方法在效率上是高的，因为客户端仅在需要时获取数据功能，类似于推送方式。它在此项评分为8。2. 可维护性 该解决方案保持了经典的客户端-服务器架构的数据封装，提高了其可维护性。该方案在此项得分为7。3. 运行时灵活性 观察者模式具有良好的运行时灵活性；对客户端的唯一要求是它提供一个具有适当签名的更新方法。它在此项评分为9。4. 内存使用 与推送方式类似，该模式使用较少的内存，因为客户端无需跟踪其在数据缓冲区中的位置，因此在此标准上得分为9。

分析结果如表2-4所示。基于加权标准，观察者模式似乎是所考察的设计方案中最好的一个。

表 2-4：ECG监护系统的设计权衡。

设计方案	设计准则				总加权分数
	效率	可维护性	灵活性	内存使用	
	Weight = 7	Weight = 5	Weight = 4	Weight = 7	
	评分	评分	评分	评分	
客户端-服务器	3	7	8	5	123
Push	8	4	7	9	167
观察者	**8**	**7**	**9**	**9**	**190**

Once selected, the next step is to instantiate the pattern and, as a part of that, to make decisions about how to best implement the pattern. Instantiation is a matter of replacing the formal parameters of the pattern with elements from your original model. In this case, formal parameter Concrete Subject will be replaced by the `TMDQueue` and the formal parameter Concrete Observer will be replaced by the clients `WaveformDisplay`, `HistogramDisplay`, `QRSDetector`, and `ArrythmiaDetector`. The resulting design model is shown in Figure 2-11. In this diagram you can see the original environmental control elements, the pattern elements, and their relations.

The relations, attributes, and operations of the Abstract Subject are inherited by the `TMDQueue` class while the features of the Abstract Observer are inherited by the various clients of the `TMDQueue`. Inheritance can be implemented in C in a number of ways, such as:

- Copying from the super class implementation into the subclass implementation
- Using a "mix-in"; that is, delegate service requests from an instance of the subclass to an instance of the super class
- Implementing a virtual function table, as discussed in Chapter 1

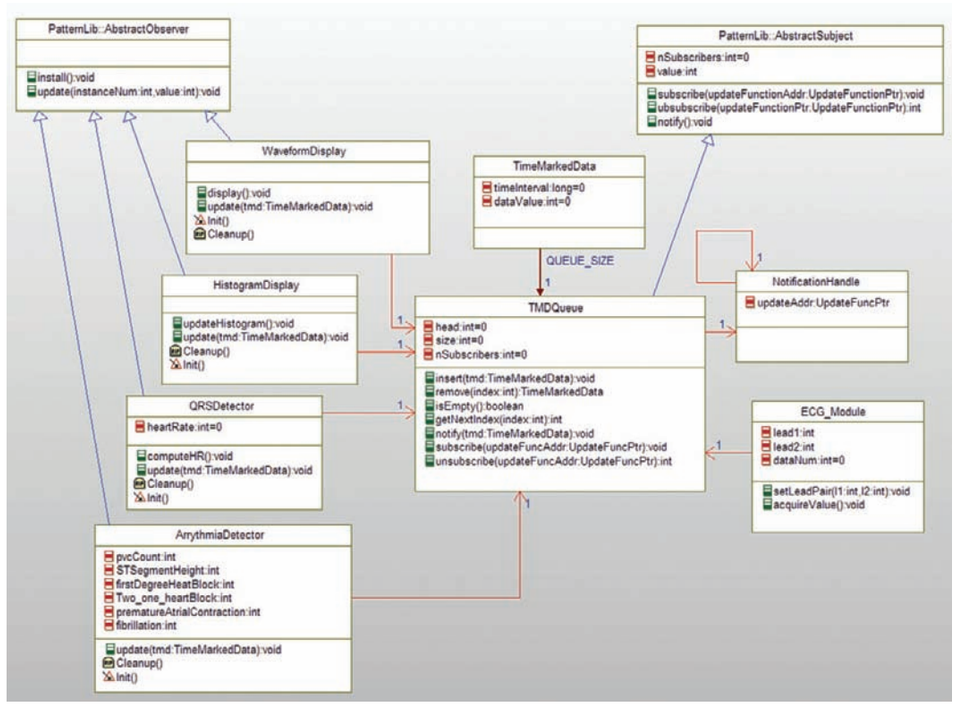


Figure 2-11: ECG design model

一旦选定，下一步就是实例化该模式，并且作为其中的一部分，需要决定如何最好地实现该模式。实例化就是用原始模型中的元素替换模式的形式参数。在本例中，形式参数具体主题将由TMDQueue替换，形式参数具体观察者将由客户端WaveformDisplay、HistogramDisplay、QRSDetector和ArrythmiaDetector替换。得到的设计模型如图2-11所示。在此图中，你可以看到原始的环境控制元素、模式元素及其关系。

抽象主题的关系、属性和操作由 `TMDQueue` 类继承，而抽象观察者的特性则由 `TMDQueue` 的各个客户端继承。继承在 C 中可以通过多种方式实现，例如：

- 将超类实现复制到子类实现中
- 使用"混入"；即，将服务请求从子类的实例委托给超类的实例

- 实现虚函数表，如第1章所述

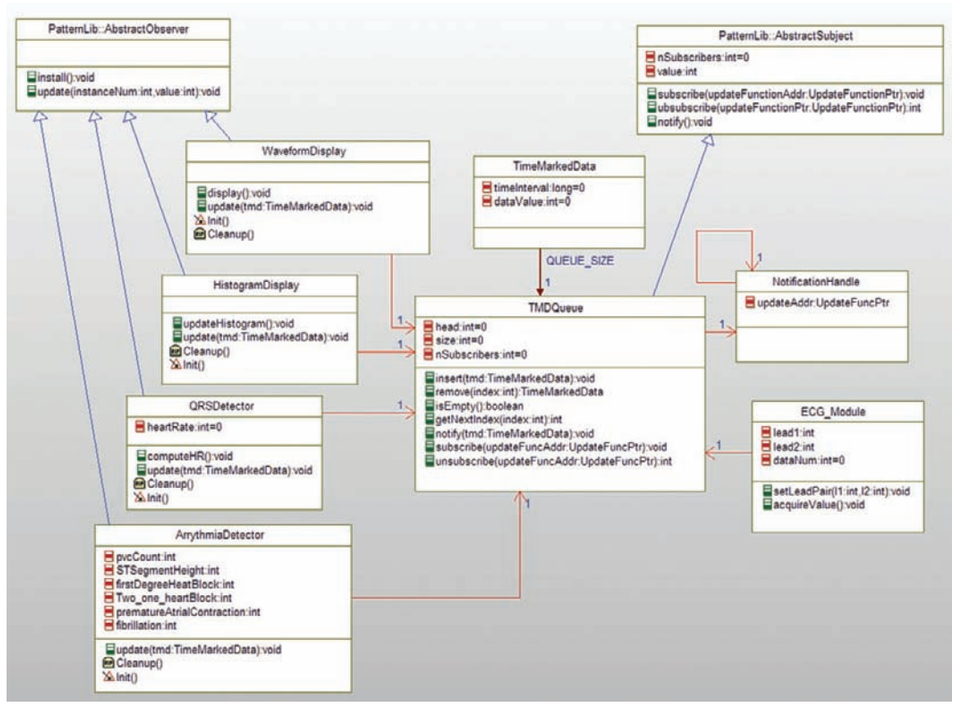


图 2-11：心电图设计模型

In our code implementation, we will take the simplest possible solution just to illustrate the pattern; we will copy and implement code from the Abstract Subject into the `TMDQueue` and do the same for code from the Abstract Observer into the clients. Other implementation strategies can be used, if desired.

The net result can be seen in Figure 2-11. Note that the clients have lost their index attribute and gained an `update()` member function, as well as an `Init()` function (that calls `subscribe()` on the `TMDQueue`) and a `Cleanup()` function that calls `unsubscribe()`. The `TMDQueue` has added `subscribe()`, `unsubscribe()`, and `notify()` operations and an association to the list of `NotificationHandle` classes. The notification handle holds the address for the relevant class's `update()` function. Calling `notify()` walks the linear linked list of notification handles.

Code Listing 2-11 and Code Listing 2-12 show the header and implementation of the `TMDQueue` class updated to include the server side of the observer pattern. You can see the addition of the link for the `NotificationHandle` list to manage the subscriptions of interested observers.

```
#ifndef TMDQueue_H
#define TMDQueue_H

/*## auto_generated */
#include <stdio.h>
#include "ECGPkg.h"
#include "TimeMarkedData.h"

struct NotificationHandle;

typedef struct TMDQueue TMDQueue;
/*
This queue is meant to operate as a "leaky" queue. In this queue,
data are never removed per se, but are instead overwritten when the
buffer pointer wraps around. This allows for many clients to read
the same data from the queue. */
struct TMDQueue {
    int head;
    int nSubscribers;
    int size;
    struct TimeMarkedData buffer[QUEUE_SIZE];
    struct NotificationHandle* itsNotificationHandle;
};

/* Constructors and destructors:*/

void TMDQueue_Init(TMDQueue* const me);
void TMDQueue_Cleanup(TMDQueue* const me);

/* Operations */

int TMDQueue_getNextIndex(TMDQueue* const me, int index);

void TMDQueue_insert(TMDQueue* const me, const struct TimeMarkedData tmd);
```


在我们的代码实现中，我们将采用最简单的解决方案来演示该模式；我们将把抽象主题中的代码复制并实现到`TMDQueue` 中，并对客户端中的抽象观察者代码做同样的处理。如有需要，也可以使用其他实现策略。

最终 结果 可以 在 图 2-11 中 看到 。请注意，客户端已经失去了它们的索引属性，并获得了一个 `update()` 成员函数，以及一个 `Init()` 函数（该函数在 TMDQueue 上调用 subscribe()）和一个 `Cleanup()` 函数，该函数调用 `unsubscribe()`。`TMDQueue` 添加了 `subscribe()`、unsubscribe() 和 `notify()` 操作，以及与 `NotificationHandle` 类列表的关联。通知句柄保存了相关类的 `update()` 函数的地址。调用 `notify()` 会遍历通知句柄的线性链表。

代码清单 2-11 和代码清单 2-12 展示了更新后的 `TMDQueue` 类的头文件和实现，其中包含了观察者模式的服务端部分。您可以看到新增了用于 `NotificationHandle` 列表的链接，以管理感兴趣观察者的订阅。

```
#ifndef TMDQueue_H #
define TMDQueue_H

/*## 自动_生成 */ #include <
stdio.h>#include "ECGPkg.h" #
include "TimeMarkedData.h"

struct NotificationHandle;

typedef struct TMDQueue TMDQueue; /* 该队列被设计为一种“泄漏”队列。在此队列中，数据本身永远不会被移除，而是在缓冲区指针回绕时被覆盖。这使得多个客户端可以从队列中读取相同的数据。 */ struct TMDQueue { int head; int nSubscribers; int size; struct TimeMarkedData buffer[QUEUE_SIZE]; struct NotificationHandle* itsNotificationHandle; };

/* 构造函数与析构函数：  */

void TMDQueue_Init(TMDQueue* const me); void
TMDQueue_Cleanup(TMDQueue* const me);

/* 操作 */

int TMDQueue_获取下一个索引(TMDQueue* const me, int index);

void TMDQueue_insert(TMDQueue* const me, const struct TimeMarkedData tmd);
```

```
boolean TMDQueue_isEmpty(TMDQueue* const me);

void TMDQueue_notify(TMDQueue* const me, const struct TimeMarkedData tmd);

struct TimeMarkedData TMDQueue_remove(TMDQueue* const me, int index);

/* The NotificationHandle is managed as a linked list, with insertions coming at the end. */
void TMDQueue_subscribe(TMDQueue* const me, const UpdateFuncPtr updateFuncAddr);

int TMDQueue_unsubscribe(TMDQueue* const me, const UpdateFuncPtr updateFuncAddr);

int TMDQueue_getBuffer(const TMDQueue* const me);

struct NotificationHandle*
TMDQueue_getItsNotificationHandle(const TMDQueue* const me);
void TMDQueue_setItsNotificationHandle(TMDQueue* const me, struct NotificationHandle*
p_NotificationHandle);

TMDQueue * TMDQueue_Create(void);

void TMDQueue_Destroy(TMDQueue* const me);

#endif
```

Code Listing 2-11: TMDQueue Header File with Observer Pattern

```
#include "TMDQueue.h"
#include "NotificationHandle.h"

static void initRelations(TMDQueue* const me);
static void cleanUpRelations(TMDQueue* const me);

void TMDQueue_Init(TMDQueue* const me) {
    me->head=0;
    me->nSubscribers=0;
    me->size=0;
    me->itsNotificationHandle=NULL;
    initRelations(me);
}

void TMDQueue_Cleanup(TMDQueue* const me) {
    cleanUpRelations(me);
}

int TMDQueue_getNextIndex(TMDQueue* const me, int index) {
    /* this operation computes the next index from the
    first using modulo arithmetic
    */
    return (index+1) % QUEUE_SIZE;
}

void TMDQueue_insert(TMDQueue* const me, const struct TimeMarkedData tmd) {
    /* note that because we never 'remove' data from this
    leaky queue, size only increases to the queue size and
    then stops increasing. Insertion always takes place at the head.
    */
```


布尔值 TMDQueue_isEmpty(TMDQueue* const me);

void TMDQueue_notify(TMDQueue* const me, const struct TimeMarkedData tmd);

struct TimeMarkedData TMDQueue_remove(TMDQueue* const me, int 索引) ;

/* NotificationHandle 以链表方式管理，插入操作在末尾进行。 */ void TMDQueue_subscribe(TMDQueue* const me, const UpdateFuncPtr updateFuncAddr);

int TMDQueue_unsubscribe(TMDQueue* const me, const UpdateFuncPtr updateFuncAddr);

int TMDQueue_getBuffer(const TMDQueue* const me);

struct NotificationHandle* TMDQueue_getItsNotificationHandle(const TMDQueue* const me); void TMDQueue_setItsNotificationHandle(TMDQueue* const me, struct NotificationHandle* p_NotificationHandle); TMDQueue * TMDQueue_Create(void); void TMDQueue_Destroy(TMDQueue* const me); #endif

代码清单 2-11：采用观察者模式的TMDQueue头文件

#include "TMDQueue.h" #include "NotificationHandle.h"

static void initRelations(TMDQueue* const me); static void cleanUpRelations(TMDQueue* const me); void TMDQueue_Init(TMDQueue* const me) { me->head = 0; me->nSubscribers = 0; me->size = 0; me->itsNotificationHandle = NULL; initRelations(me); } void TMDQueue_Cleanup(TMDQueue* const me) { cleanUpRelations(me); } int TMDQueue_getNextIndex(TMDQueue* const me, int index) { /* 此操作使用模运算从第一个索引计算下一个索引 */ return (index+1) % QUEUE_SIZE; }

void TMDQueue_insert(TMDQueue* const me, const struct TimeMarkedData tmd) { /* 注意，由于我们从不从这个泄漏队列中“移除”数据，size 只会增加到队列大小，然后停止增加。插入始终发生在 head 处。 */

```
    printf("Indenting at: %d Data #: %d", me->head, tmd.timeInterval);
    me->buffer[me->head]=tmd;
    me->head=TMDQueue_getNextIndex(me, me->head);
    if (me->size < QUEUE_SIZE) ++me->size;
    printf(" Storing data value: %d\n", tmd.dataValue);
    TMDQueue_notify(me, tmd);
}

boolean TMDQueue_isEmpty(TMDQueue* const me) {
    return (boolean)(me->size==0);
}

void TMDQueue_notify(TMDQueue* const me, const struct TimeMarkedData tmd) {
    NotificationHandle *pNH;
    pNH= me->itsNotificationHandle;
    while (pNH) {
        printf("----->> calling updateAddr on pNH %d\n",pNH);
        pNH->updateAddr(NULL, tmd);
        pNH=pNH->itsNotificationHandle;
        };
}

struct TimeMarkedData TMDQueue_remove(TMDQueue* const me, int index) {
        TimeMarkedData tmd;
        tmd.timeInterval=-1; /* sentinel values */
        tmd.dataValue=-9999;

        if (!TMDQueue_isEmpty(me) && (index>=0) && (index < QUEUE_SIZE)
            && (index < me->size)) {
            tmd=me->buffer[index];
        }
        return tmd;
}

void TMDQueue_subscribe(TMDQueue* const me, const UpdateFuncPtr updateFuncAddr) {
    struct NotificationHandle *pNH;
    pNH=me->itsNotificationHandle;

    if (!pNH) { /* empty list? */
        /* create a new Notification Handle, initialize it, and point to it */
        printf("-----> Added to a new list\n");
        me->itsNotificationHandle=NotificationHandle_Create();
        printf("-----> Called NH_Create()\n");
        pNH=me->itsNotificationHandle;
        }
    else {
        /* search list to find end */
        printf("-----> Adding to an existing list\n");
        while (pNH->itsNotificationHandle != NULL) {
          printf("Getting ready to augment ptr %d to %d\n", pNH, pNH->itsNotificationHandle);
            pNH=pNH->itsNotificationHandle; /* get next element in list */
            printf("-----> augmenting ptr\n");
```



```
printf("Indenting at: %d Data #: %d", me->head, tmd.timeInterval); me->buffer[me->
head] = tmd; me->head = TMDQueue_getNextIndex(me, me->head); if (me->size <
QUEUE_SIZE) ++me->size; printf(" Storing data value: %d\n", tmd.dataValue);
TMDQueue_notify(me, tmd); } boolean TMDQueue_isEmpty(TMDQueue* const me) {
return (boolean)(me->size == 0); } void TMDQueue_notify(TMDQueue* const me, const
struct TimeMarkedData tmd) { NotificationHandle *pNH; pNH= me->
itsNotificationHandle; while (pNH) { printf("----->> calling updateAddr on pNH %d\n",
pNH); pNH->updateAddr(NULL, tmd); pNH = pNH->itsNotificationHandle; }; } struct
TimeMarkedData TMDQueue_remove(TMDQueue* const me, int index) {
TimeMarkedData tmd; tmd.timeInterval=-1; /* 哨兵值 */ tmd.dataValue=-9999; if (!
TMDQueue_isEmpty(me) && (index>=0) && (index < QUEUE_SIZE) && (index < me->
size)) { tmd = me->buffer[index]; } return tmd; }

void TMDQueue_subscribe(TMDQueue* const me, const UpdateFuncPtr updateFuncAddr) { struct
NotificationHandle *pNH; pNH = me->itsNotificationHandle; if (!pNH) { /* 空列表?  */ /* 创建一个新的通知
句柄，初始化它，并指向它 */ printf("-----> 已添加到新列表\n"); me->itsNotificationHandle =
NotificationHandle_Create(); printf("-----> 已调用 NH_Create()\n"); pNH = me->itsNotificationHandle; }
else { /* 搜索列表以找到末尾 */ printf("-----> 正在添加到现有列表\n"); while (pNH->itsNotificationHandle !=
NULL) { printf("准备将指针 %d 扩展到 %d\n", pNH, pNH->itsNotificationHandle); pNH = pNH->
itsNotificationHandle; /* 获取列表中的下一个元素 */ printf("-----> 正在扩展指针\n");
```

```
            };
        printf("-----> calling NH_Create\n");
            pNH->itsNotificationHandle=NotificationHandle_Create();
            pNH=pNH->itsNotificationHandle; /* pt to the new instance */
            printf("-----> called NH_Create()\n");
    }; /* end if */

    /* pNH now points to an constructed Notification Handle */
    pNH->updateAddr=updateFuncAddr; /* set callback address */
    ++me->nSubscribers;
    printf("-----> wrote updateAddr \n");
    if (pNH->itsNotificationHandle)
        printf("xxxxxxx> next Ptr not null!\n\n");
    else
        printf("-----> next ptr null\n\n");
}

int TMDQueue_unsubscribe(TMDQueue* const me, const UpdateFuncPtr updateFuncAddr) {
    struct NotificationHandle *pNH, *pBack;
    pNH=pBack=me->itsNotificationHandle;

    if (pNH==NULL) { /* empty list? */
        return 0; /* can't delete it from an empty list */
        }
    else { /* is it the first one? */
        if (pNH->updateAddr==updateFuncAddr) {
            me->itsNotificationHandle=pNH->itsNotificationHandle;
            free(pNH);
            printf(">>>>>> Removing the first element\n");
            --me->nSubscribers;
            return 1;
            }
        else { /* search list to find element */
            printf(">>>>>> Searching....\n");
            while (pNH!=NULL) {
                if (pNH->updateAddr!=updateFuncAddr) {
                    pBack->itsNotificationHandle=pNH->itsNotificationHandle;
                    free(pNH);
                    printf(">>>>>> Removing subscriber in list\n");
                    --me->nSubscribers;
                    return 1;
                    }; /* end if found */
                pBack=pNH; /* points to the list element before pNH */
                pNH=pNH->itsNotificationHandle; /* get next element in list */
                }; /* end while */
            }; /* end else */
        printf(">>>>>> Didn't remove any subscribers\n");
        return 0;
        }; /* non-empty list */
}
```



```
    }; printf("-----> 正在调用 NH_Create\n"); pNH->itsNotificationHandle=
NotificationHandle_Create(); pNH=pNH->itsNotificationHandle; /* 指向新实例 */
printf("-----> 已调用 NH_Create()\n"); }; /* 结束 if */ /* pNH 现在指向一个已构造的通知句
柄 */ pNH->updateAddr=updateFuncAddr; /* 设置回调地址 */++me->nSubscribers;
printf("-----> 已写入 updateAddr \n"); if (pNH->itsNotificationHandle) printf("
xxxxxxx> 下一个指针不为空！\n\n"); else printf("-----> 下一个指针为空\n\n"); }
```

```
int TMDQueue_unsubscribe(TMDQueue* const me, const UpdateFuncPtr updateFuncAddr)        {
    struct NotificationHandle *pNH, *pBack; pNH=pBack=me->itsNotificationHandle; if
    (pNH==NULL) { /* 空列表?  */ return 0; /* 无法从空列表中删除 */ } else { /* 是第一个元素吗?
    */ if (pNH->updateAddr==updateFuncAddr) { me->itsNotificationHandle=pNH->
    itsNotificationHandle; free(pNH); printf(">>>>>> 正在移除第一个元素\n"); --me->
    nSubscribers; return 1; } else { /* 搜索列表以查找元素 */ printf(">>>>>> 正在搜索. . ..\n");
    while (pNH!=NULL) { if (pNH->updateAddr!=updateFuncAddr) { pBack->
    itsNotificationHandle=pNH->itsNotificationHandle; free(pNH); printf(">>>>>> 正在移除
    列表中的订阅者\n"); --me->nSubscribers; return 1; }; /* 结束 if 找到 */ pBack=pNH; /* 指向
    pNH 之前的列表元素 */ pNH=pNH->itsNotificationHandle; /* 获取列表中的下一个元素 */ }; /
    * 结束 while */ }; /* 结束 else */ printf(">>>>>> 未移除任何订阅者\n"); return 0; }; /* 非空列
    表 */
}
```

```
int TMDQueue_getBuffer(const TMDQueue* const me) {
    int iter = 0;
    return iter;
}

struct NotificationHandle*
TMDQueue_getItsNotificationHandle(const TMDQueue* const me) {
    return (struct NotificationHandle*)me->itsNotificationHandle;
}
void TMDQueue_setItsNotificationHandle(TMDQueue* const me, struct NotificationHandle*
p_NotificationHandle) {
        me->itsNotificationHandle = p_NotificationHandle;
}

TMDQueue * TMDQueue_Create(void) {
    TMDQueue* me = (TMDQueue *) malloc(sizeof(TMDQueue));
    if(me!=NULL)
        {
            TMDQueue_Init(me);
        }
    return me;
}

void TMDQueue_Destroy(TMDQueue* const me) {
    if(me!=NULL)
        {
            TMDQueue_Cleanup(me);
        }
    free(me);
}

static void initRelations(TMDQueue* const me) {
    {
        int iter=0;
        while (iter < QUEUE_SIZE){
            TimeMarkedData_Init(&((me->buffer)[iter]));
            TimeMarkedData__setItsTMDQueue(&((me->buffer)[iter]), me);
            iter++;
        }
    }
}

static void cleanUpRelations(TMDQueue* const me) {
    {
        int iter=0;
        while (iter < QUEUE_SIZE){
            TimeMarkedData_Cleanup(&((me->buffer)[iter]));
            iter++;
        }
    }
```


int TMDQueue_getBuffer(const TMDQueue* const me) { int iter = 0; return iter; } 结构体 NotificationHandle* TMDQueue_getItsNotificationHandle(const TMDQueue* const me) { return (结构体 NotificationHandle*)me->itsNotificationHandle; } void TMDQueue_setItsNotificationHandle(TMDQueue* const me, 结构体 Notifi

cationHandle* p_NotificationHandle) { me->itsNotificationHandle = p_NotificationHandle; } TMDQueue * TMDQueue_Create(void) { TMDQueue* me = (TMDQueue *) malloc(sizeof(TMDQueue)); if(me!=空) { TMDQueue_Init(me); } return me; } void TMDQueue_Destroy(TMDQueue* const me) { if(me!=空) { TMDQueue_清理(me); } free(me); } static void initRelations(TMDQueue* const me) { { int iter = 0; while (iter < QUEUE_SIZE){ TimeMarkedData_Init(&((me->缓冲区)[iter])); TimeMarkedData__setItsTMDQueue(&((me->缓冲区)[iter]), me); iter++; } } } static void cleanUpRelations(TMDQueue* const me) { { int iter = 0; while (iter < QUEUE_SIZE){ TimeMarkedData_清理(&((me->缓冲区)[iter])); iter++; } }

```
    if(me->itsNotificationHandle != NULL)
        {
            me->itsNotificationHandle=NULL;
        }
}
```

Code Listing 2-12: TMDQueue Implementation File with Observer Pattern

The code for the notification handle class is straightforward, as can be seen in Code Listing 2-13 and Code Listing 2-14.

```
#ifndef NotificationHandle_H
#define NotificationHandle_H

#include <stdio.h>
#include "ECGPkg.h"
typedef struct NotificationHandle NotificationHandle;
struct NotificationHandle {
    UpdateFuncPtr updateAddr;
    struct NotificationHandle* itsNotificationHandle;
};

/* Constructors and destructors:*/
void NotificationHandle_Init(NotificationHandle* const me);

void NotificationHandle_Cleanup(NotificationHandle* const me);

struct NotificationHandle*
NotificationHandle_getItsNotificationHandle(const
NotificationHandle* const me);

void
NotificationHandle_setItsNotificationHandle(NotificationHandle*
const me, struct NotificationHandle* p_NotificationHandle);

NotificationHandle * NotificationHandle_Create(void);

void NotificationHandle_Destroy(NotificationHandle* const me);

#endif
```

Code Listing 2-13: NotificationHandle.h Header File

```
#include "NotificationHandle.h"

static void cleanUpRelations(NotificationHandle* const me);

void NotificationHandle_Init(NotificationHandle* const me) {
    me->itsNotificationHandle=NULL;
}
```



```
    if(me->itsNotificationHandle != 空) { me->
itsNotificationHandle = 空; } }
```

Code 代码清单 2-12：采用观察者模式的 TMDQueue 实现文件

通知句柄类的代码很直接，如代码清单 2-13 和代码清单 2-14 所示。

```
#ifndef NotificationHandle_H #define NotificationHandle_H #include <stdio.h>
#include "ECGPkg.h" typedef struct NotificationHandle NotificationHandle;
struct NotificationHandle { UpdateFuncPtr updateAddr; struct
NotificationHandle* itsNotificationHandle; }; /* 构造函数与析构函数：*/ void
NotificationHandle_Init(NotificationHandle* const me); void
NotificationHandle_Cleanup(NotificationHandle* const me); struct
NotificationHandle* NotificationHandle_getItsNotificationHandle(const
NotificationHandle* const me); void NotificationHandle_
setItsNotificationHandle(NotificationHandle* const me, struct
NotificationHandle* p_NotificationHandle); NotificationHandle *
NotificationHandle_Create(void); void NotificationHandle_
Destroy(NotificationHandle* const me); #endif
```

代码清单 2-13：NotificationHandle.h 头文件

```
#include "NotificationHandle.h" static void
cleanUpRelations(NotificationHandle* const me); void
NotificationHandle_Init(NotificationHandle* const me) { me->
itsNotificationHandle = 空; }
```

```
void NotificationHandle_Cleanup(NotificationHandle* const me) {
    cleanUpRelations(me);
}

struct NotificationHandle*
NotificationHandle_getItsNotificationHandle(const
NotificationHandle* const me) {
    return (struct NotificationHandle*)me->itsNotificationHandle;
}

void
NotificationHandle_setItsNotificationHandle(NotificationHandle*
const me, struct NotificationHandle* p_NotificationHandle) {
    me->itsNotificationHandle=p_NotificationHandle;
}

NotificationHandle * NotificationHandle_Create(void) {
    NotificationHandle* me=(NotificationHandle *)
malloc(sizeof(NotificationHandle));
    if(me!=NULL)
        {
            NotificationHandle_Init(me);
        }
    return me;
}

void NotificationHandle_Destroy(NotificationHandle* const me) {
    if(me!=NULL)
        {
            NotificationHandle_Cleanup(me);
        }
    free(me);
}

static void cleanUpRelations(NotificationHandle* const me) {
    if(me->itsNotificationHandle != NULL)
        {
            me->itsNotificationHandle=NULL;
        }
}
```

Code Listing 2-14: NotificationHandle.c Implementation File

Finally, let's look at just one of the clients to see how the client side of the observer pattern is implemented. Code Listing 2-15 shows the implementation file for the `HistogramDisplay` class with the client side of the observer pattern (specifically, it calls the `TMDQueue_subscribe()` operation in its initializer).

```
#include "HistogramDisplay.h"
#include "TimeMarkedData.h"
#include "TMDQueue.h"
```



```
void NotificationHandle_Cleanup(NotificationHandle* const me) {
cleanUpRelations(me); } struct NotificationHandle* NotificationHandle_
getItsNotificationHandle(const NotificationHandle* const me) { return (struct
NotificationHandle*)me->itsNotificationHandle; } void NotificationHandle_
setItsNotificationHandle(NotificationHandle* const me, struct
NotificationHandle* p_NotificationHandle) { me->itsNotificationHandle = p_
NotificationHandle; } NotificationHandle * NotificationHandle_Create(void) {
NotificationHandle* me = (NotificationHandle *)
malloc(sizeof(NotificationHandle)); if(me!=NULL) { NotificationHandle_
Init(me); } return me; } void NotificationHandle_Destroy(NotificationHandle*
const me) { if(me!=NULL) { NotificationHandle_Cleanup(me); } free(me); }
static void cleanUpRelations(NotificationHandle* const me) { if(me->
itsNotificationHandle != NULL) { me->itsNotificationHandle = NULL; } }
```

代码清单 2-14：NotificationHandle.c 实现文件

最后，让我们只看一个客户端，以了解观察者模式的客户端侧是如何实现的。代码清单 2-15 展示了 `HistogramDisplay` 类的实现文件，其中包含观察者模式的客户端侧（具体来说，它在其初始化器中调用了 `TMDQueue_subscribe()`操作）。

```
#include "HistogramDisplay.h" #
include "TimeMarkedData.h" #
include "TMDQueue.h"
```

```
static void cleanUpRelations(HistogramDisplay* const me);

void HistogramDisplay_Init(HistogramDisplay* const me) {
    me->itsTMDQueue = NULL;
    {
        /* call subscribe to connect to the server */
        TMDQueue_subscribe(me->itsTMDQueue,
HistogramDisplay_update);
    }
}

void HistogramDisplay_Cleanup(HistogramDisplay* const me) {
    /* remove yourself from server subscription list */
    TMDQueue_unsubscribe(me->itsTMDQueue,
HistogramDisplay_update);
    cleanUpRelations(me);
}

void HistogramDisplay_update(HistogramDisplay* const me, const struct TimeMarkedData
tmd) {
      printf(" Histogram -> TimeInterval: %d DataValue: %d\n", tmd.timeInterval, tmd.
dataValue);
}

void HistogramDisplay_updateHistogram(HistogramDisplay* const me) {
}

struct TMDQueue* HistogramDisplay_getItsTMDQueue(const HistogramDisplay* const me) {
    return (struct TMDQueue*)me->itsTMDQueue;
}

void HistogramDisplay_setItsTMDQueue(HistogramDisplay* const me,
struct TMDQueue* p_TMDQueue) {
    me->itsTMDQueue = p_TMDQueue;
}

HistogramDisplay * HistogramDisplay_Create(void) {
    HistogramDisplay* me = (HistogramDisplay *)
malloc(sizeof(HistogramDisplay));
    if(me!=NULL)
        {
            HistogramDisplay_Init(me);
        }
    return me;
}

void HistogramDisplay_Destroy(HistogramDisplay* const me) {
    if(me!=NULL)
        {
            HistogramDisplay_Cleanup(me);
        }
        free(me);
}
```


static void cleanUpRelations(HistogramDisplay* const me); void HistogramDisplay_Init(HistogramDisplay* const me) { me->itsTMDQueue = NULL; { /* 调用 subscribe 以连接到服务器 */ TMDQueue_subscribe(me->itsTMDQueue, HistogramDisplay_update); } } void HistogramDisplay_Cleanup(HistogramDisplay* const me) { /* 将自身从服务器订阅列表中移除 */ TMDQueue_unsubscribe(me->itsTMDQueue, HistogramDisplay_update); cleanUpRelations(me); } void HistogramDisplay_update(HistogramDisplay* const me, const struct TimeMarkedData tmd) { printf(" Histogram -> TimeInterval: %d DataValue: %d\n", tmd.timeInterval, tmd. dataValue); } void HistogramDisplay_updateHistogram(HistogramDisplay* const me) { } struct TMDQueue* HistogramDisplay_getItsTMDQueue(const HistogramDisplay* const me) { return (struct TMDQueue*)me->itsTMDQueue; } void HistogramDisplay_setItsTMDQueue(HistogramDisplay* const me, struct TMDQueue* p_TMDQueue) { me->itsTMDQueue = p_TMDQueue; } HistogramDisplay * HistogramDisplay_Create(void) { HistogramDisplay* me = (HistogramDisplay *) malloc(sizeof(HistogramDisplay)); if(me!=NULL) { HistogramDisplay_Init(me); } return me; } void HistogramDisplay_Destroy(HistogramDisplay* const me) { if(me!=NULL) { HistogramDisplay_Cleanup(me); } free(me); }

```
static void cleanUpRelations(HistogramDisplay* const me) {
    if(me->itsTMDQueue != NULL)
        {
            me->itsTMDQueue=NULL;
        }
}
```

Code Listing 2-15: HistogramDisplay.c Implementation File with Observer Pattern

To summarize, we created a correctly functioning code base and then decided how we wanted to optimize it. We did a miniature trade study to look at some different alternatives and how differently they optimized our weighted set of design criteria. In this case, the observer pattern had the best results, so we updated our code base to include the pattern. Simple, eh?

2.2 The Approach

This book provides many different patterns for use in embedded systems, as you will see in the following chapters. In each chapter, a number of patterns are presented. Each pattern follows the same basic presentation style and is presented with nine basic elements:

- Abstract
  The abstract provides an overview of the pattern and why you might care, including an overview of the problems it addresses, the solution structure, and the key consequences.
- Problem
  This section discusses the problem context and the optimizations provided by the pattern.
- Applicability
  This section gives a list of conditions required for the pattern to be applicable.
- Pattern structure
  The pattern structure shows the structural elements within the pattern, their roles, and their relations. Each pattern will be shown using a UML class diagram.
- Collaboration role definitions
  This section details the role each structural element plays in the pattern and its responsibilities.
- Consequences
  The consequences are a key aspect of the pattern description. This section identifies both the benefits and the costs of using the pattern.
- Implementation strategies
  Patterns can be implemented in different ways. This section discusses some implementation alternatives.


```
static void cleanUpRelations(HistogramDisplay* const me) { if(me->
itsTMDQueue != NULL) { me->itsTMDQueue = NULL; } }
```

代码清单　2-15：使用观察者模式的HistogramDisplay.c实现文件

总而言之，我们创建了一个功能正确的代码库，然后决定了我们希望如何对其进行优化。我们进行了一项小型的权衡研究，考察了一些不同的替代方案以及它们如何以不同方式优化我们加权后的设计准则集合。在本例中，观察者模式取得了最佳结果，因此我们更新了代码库以包含该模式。很简单，对吧?

2.2 方法

本书提供了许多可用于嵌入式系统的不同模式，你将在后续章节中看到。每一章都会介绍若干模式。每个模式都遵循相同的基本呈现风格，并通过九个基本要素进行呈现：

- 摘要 摘要部分概述了该模式以及你可能关注它的原因，包括它所解决的问题概述、解决方案结构以及关键结果。
- 问题 本节讨论问题上下文以及该模式所提供的优化。
- 适用性 本节列出了该模式适用所需的条件。
- 模式结构 模式结构展示了模式中的结构元素、它们的角色以及它们之间的关系。每个模式都将使用UML类图来展示。
- 协作角色定义 本节详细说明每个结构元素在模式中所扮演的角色及其职责。
- 结果 结果是模式描述的一个关键方面。本节既指出了使用该模式的好处，也指出了其成本。
- 实现策略 模式可以通过不同的方式实现。本节讨论一些实现替代方案。

- Related patterns
  This section provides references to other patterns in the book that address similar problems, possibly with different applicability criteria and consequences.
- Example
  Each pattern will be illustrated with an example. Both a UML diagram and source code will be provided with the pattern.

2.3 What's Coming Up

The remaining chapters of this book present design patterns in a number of subject areas of particular interest to embedded C developers. Chapter 3 focuses on design patterns for controlling hardware. Chapter 4 is about concurrency with embedded C, including not only basic scheduling patterns but patterns to share resources across threads and to avoid deadlock. Chapter 5 deals with state machines, an important but underutilized tool in the embedded toolbox, while Chapter 6 looks at safety and reliability.

- 相关模式 本节提供了对书中其他模式的引用，这些模式解决类似的问题，可能具有不同的适用性标准和结果。
- 示例 每个模式都将通过一个示例进行说明。每个模式都会提供UML图和源代码。

2.3 后续内容

本书其余章节将介绍嵌入式C开发者特别关注的一些主题领域中的设计模式。第3章重点介绍硬件控制的设计模式。第4章讨论嵌入式C中的并发，不仅包括基本的调度模式，还包括跨线程共享资源和避免死锁的模式。第5章讨论状态机——嵌入式工具箱中一个重要但未被充分利用的工具，而第6章则探讨安全性与可靠性。

CHAPTER 3

Design Patterns for Accessing Hardware

Chapter Outline

3.1 Basic Hardware Access Concepts 81
3.2 Hardware Proxy Pattern 85
3.2.1 Abstract 85
3.2.2 Problem 86
3.2.3 Pattern Structure 86
3.2.4 Collaboration Roles 86
3.2.4.1 Hardware Device 86
3.2.4.2 Hardware Proxy 86
3.2.4.3 Proxy Client 88
3.2.5 Consequences 88
3.2.6 Implementation Strategies 89
3.2.7 Related Patterns 89
3.2.8 Example 89
3.3 Hardware Adapter Pattern 96
3.3.1 Abstract 96
3.3.2 Problem 96
3.3.3 Pattern Structure 96
3.3.4 Collaboration Roles 96
3.3.4.1 Adapter Client 96
3.3.4.2 Hardware Adapter 96
3.3.4.3 Hardware Interface to Client 97
3.3.4.4 Hardware Device 97
3.3.4.5 Hardware Proxy 97
3.3.5 Consequences 97
3.3.6 Implementation Strategies 98
3.3.7 Related Patterns 98
3.3.8 Example 98
3.4 Mediator Pattern 100
3.4.1 Abstract 100
3.4.2 Problem 100
3.4.3 Pattern Structure 101
3.4.4 Collaboration Roles 101
3.4.4.1 Collaborator Interface 101
3.4.4.2 Mediator 101
3.4.4.3 Specific Collaborator 102
3.4.5 Consequences 102

Design Patterns for Embedded Systems in C
DOI:10.1016/B978-1-85617-707-8.00003-0

Copyright © 2011 Elsevier Inc.

第 3 章

访问硬件的设计模式

第 章 大纲 3.1 基本 硬件 访问 概念
81 3.2 硬件 代理模式 **85** 3 2 1 85 . . 摘要
3.2.2 问题 86 3.2.3 模式结构 86 3.2.4 协作角色 86 3.2.4.1 硬件设备 86 3.2.4.2 硬件代理 86 3.2.4.3 代理客户端 88 3.2.5 结果 88 3.2.6 实现策略 89 3.2.7 相关模式 89 3.2.8 示例 89

3.3 硬件 适配器 模式 **96** 3 3 1 96 . . 摘要
3.3.2 问题 96 3.3.3 模式结构 96 3.3.4 协作角色 96 3.3.4.1 适配器客户端 96 3.3.4.2 硬件适配器 96 3.3.4.3 面向客户端的硬件接口 97 3.3.4.4 硬件设备 97 3.3.4.5 硬件代理 97
3.3.5 结果 97 3.3.6 实现策略 98
3.3.7 相关模式 98 3.3.8 示例 98

3.4 中介者 模式 **100**3.4.1 摘要 100
3.4.2 问题 100 3.4.3 模式结构 101 3.4.4 协作角色 101 3.4.4.1 协作者接口 101 3.4.4.2 中介者 101 3.4.4.3 特定协作者 102 3.4.5 结果 102

C语言嵌入式系统设计模式
DOI:10.1016/B978-1-85617-707-8.00003-0 版权所

有 © 2011 Elsevier Inc.

3.4.6 Implementation Strategies 102
3.4.7 Related Patterns 102
3.4.8 Example 103
3.5 Observer Pattern 111
3.5.1 Abstract 111
3.5.2 Problem 112
3.5.3 Pattern Structure 112
3.5.4 Collaboration Roles 112
3.5.4.1 AbstractClient Interface 112
3.5.4.2 AbstractSubject Interface 113
3.5.4.3 ConcreteClient 113
3.5.4.4 ConcreteSubject 113
3.5.4.5 Datum 114
3.5.4.6 NotificationHandle 114
3.5.5 Consequences 114
3.5.6 Implementation Strategies 114
3.5.7 Related Patterns 115
3.5.8 Example 115
3.6 Debouncing Pattern 122
3.6.1 Abstract 123
3.6.2 Problem 123
3.6.3 Pattern Structure 123
3.6.4 Collaboration Roles 123
3.6.4.1 ApplicationClient 124
3.6.4.2 BouncingDevice 124
3.6.4.3 Debouncer 124
3.6.4.4 DebouncingTimer 125
3.6.5 Consequences 125
3.6.6 Implementation Strategies 125
3.6.7 Related Patterns 126
3.6.8 Example 126
3.7 Interrupt Pattern 130
3.7.1 Abstract 130
3.7.2 Problem 130
3.7.3 Pattern Structure 131
3.7.4 Collaboration Roles 131
3.7.4.1 InterruptHandler 131
3.7.4.2 InterruptVectorTable 132
3.7.4.3 vectorPtr 132
3.7.5 Consequences 132
3.7.6 Implementation Strategies 134
3.7.7 Related Patterns 135
3.7.8 Example 135
3.8 Polling Pattern 138
3.8.1 Abstract 138
3.8.2 Problem 138

3.4.6 实现策略 102 3.4.7 相关模式
102 3.4.8 示例 103

3.5 观察者模式 111
3.5.1 摘要 111
3.5.2 问题 112
3.5.3 模式结构 112
3.5.4 协作角色 112
3.5.4.1 抽象客户端接口 *112*
3.5.4.2 抽象主题接口 *113*
3.5.4.3 具体客户端 *113*
3.5.4.4 具体主题 *113*
3.5.4.5 数据 *114*
3.5.4.6 NotificationHandle *114*
3.5.5 结果 114 3.5.6 实现策略 114
3.5.7 相关模式 115 3.5.8 示例 115

3.6 去抖 模式 **122**。。
6 123
3.1 摘要
3.6.2 问题 123
3.6.3 模式结构 123
3.6.4 协作角色 123
3.6.4.1 应用客户端 *124*
3.6.4.2 弹跳设备 *124*
3.6.4.3 去抖器 *124*
3.6.4.4 去抖定时器 *125*
3.6.5 结果 125
3.6.6 实现策略 125
3.6.7 相关模式 126
3.6.8 示例 126
3.7 中断 模式 **130** 3 7 1 130 . . 摘要
3.7.2 问题 130 3.7.3 模式结构 131 3.7.4
协作角色 131 3.7.4.1 I
nterruptHandler 131 3.7.4.2 中断向量
表 132 3.7.4.3 vectorPtr 132 3.7.5 结果
132 3.7.6 实现策略 134 3.7.7 相关模式
135 3.7.8 示例 135

3.8 轮询 模式 **138** 3 8 1
138 . . 摘要 3.8.2 问题 138

3.8.3 Pattern Structure 138
3.8.4 Collaboration Roles 138
3.8.4.1 ApplicationProcessingElement 139
3.8.4.2 Device 139
3.8.4.3 OpportunisticPoller 140
3.8.4.4 PeriodicPoller 140
3.8.4.5 PollDataClient 141
3.8.4.6 PollTimer 141
3.8.5 Consequences 141
3.8.6 Implementation Strategies 141
3.8.7 Related Patterns 142
3.8.8 Example 142
3.9 So, What Did We Learn? 147

Patterns in this chapter:

Hardware Proxy Pattern – Encapsulate the hardware into a class or struct

Hardware Adapter Pattern – Adapt between a required and a provided interface

Mediator Pattern – Coordinate complex interactions

Observer Pattern – Support efficient sensor data distribition

Debouncing Pattern – Reject intermittent hardware signals

Interrupt Pattern – Handle high-ugency hardware signals

Polling Pattern – Periodically check for new sensor data

3.1 Basic Hardware Access Concepts

Probably the most distinguishing property of embedded systems is that they must access hardware directly. The whole point of an embedded system is that the software is embedded in a "smart device" that provides some specific kind of service, and that requires accessing hardware. Broadly, software-accessible hardware can be categorized into four kinds – infrastructure, communications, sensors, and actuators.

Infrastructure hardware refers to the computing infrastructure and devices on which the software is executing. This includes not only the CPU and memory, but also storage devices, timers, input devices (such as keyboards, knobs, and buttons), user output devices (printers, displays, and lights), ports, and interrupts. Most of this hardware is not particularly application specific and may be located on the same motherboard, on daughterboards with high-speed access connections, or as other boards sharing a common backplane.

Communications hardware refers to hardware used to facilitate communication between different computing devices, such as standard (nonembedded) computers, other embedded systems,

3.8.3 模式结构 138 3.8.4 协作角色
138 3.8.4.1
ApplicationProcessingElement 139
3.8.4.2 设备 139 3.8.4.3
OpportunisticPoller 140 3.8.4.4
PeriodicPoller 140 3.8.4.5
PollDataClient 141 3.8.4.6 PollTimer
141 3.8.5 结果 141 3.8.6 实现策略 141
3.8.7 相关模式 142 3.8.8 示例 142 3.9 那么,
我们学到了什么? 147

本章中的模式：硬件代理模式 – 将硬件封装到类或结构体中 硬件适配器模式 – 在所需接口和提供的接口之间进行适配 中介者模式 – 协调复杂的交互 观察者模式 – 支持高效的传感器数据分发 去抖模式 – 拒绝间歇性硬件信号 中断模式 – 处理高紧急性的硬件信号 轮询模式 – 定期检查新的传感器数据

3.1 基本硬件访问概念

嵌入式系统最显著的特性可能就是它们必须直接访问硬件。嵌入式系统的核心意义在于软件嵌入在提供某种特定服务的"智能设备"中，而这需要访问硬件。广义上，软件可访问的硬件可分为四类——基础设施、通信、传感器和执行器。

基础设施硬件是指软件运行所依赖的计算基础设施和设备。这不仅包括CPU和内存，还包括存储设备、定时器、输入设备（如键盘、旋钮和按钮）、用户输出设备（打印机、显示器和指示灯）、端口和中断。这些硬件大多不是特定于应用的，可能位于同一主板上、通过高速访问连接位于子板上，或作为共享公共背板的其他板卡。

通信硬件是指用于促进不同计算设备之间通信的硬件，例如标准（非嵌入式）计算机、其他嵌入式系统、

sensors, and actuators. Usually this means wired or wireless communications subsystems such as RS-232, RS-485, Ethernet, USB, 802.11x, or some other such facility. In embedded systems, DMA (Direct Memory Access) and multiported memory is sometimes used as well for this purpose, somewhat blurring the distinction between infrastructure and communications hardware.

Lastly, we have devices that monitor or manipulate the physical universe – these are the sensors and actuators. Sensors use electronic, mechanical, or chemical means for monitoring the status of physical phenomena, such as the rate of a heart beat, position of an aircraft, the mass of an object, or the concentration of a chemical. Actuators, on the other hand, change the physical state of some real-world element. Typical actuators are motors, heaters, generators, pumps, and switches.

All of these kinds of hardware are usually initialized, tested, and configured – tasks normally performed by the embedded software either on start-up or during execution, or both. All of these hardware elements are managed by the embedded software, provide commands or data to the software, or receive commands or data from the software.

This chapter will assume that you have enough hardware background to understand the hardware terms used in this book such as bus, CPU, RAM, ROM, EPROM, EEPROM, and Flash. If you want to know more, there any many excellent books on the topic, including books by David Simons[1], Michael Barr and Anthony Massa[2], Michael Pont[3], and Jack Ganssle[4]. Instead, we will focus on common ways to develop embedded C applications to solve problems that commonly arise with the management and manipulation of such hardware elements. Some hardware-related concepts that we will talk briefly about are bit fields, ports, and interrupts.

It is exceedingly common to have hardware that uses bit fields to specify commands to the hardware or to return data. A bit field is a continuous block of bits (one or more) within an addressable memory element (e.g., a byte or word) that together has some semantic meaning to the hardware device. For example, an 8-bit byte might be mapped to a hardware device into four different fields:

```
0 0 0000 00
```

The bits represent the following information to or from the memory-mapped hardware device:

Bit Range	Access	Name	Description
0	Write only	Enable bit	0 = disable device, 1 = enable device
1	Read only	Error status bit	0 = no error, 1 = error present
2–5	Write only	Motor speed	Range 0000 = no speed to 1111 (16d) top speed
6–7	Write only	LED color	00 (0d) = OFF 01 (1d) = GREEN 10 (2d) = YELLOW 11 (3d) = RED

[1] Simon, D., 1999. *An Embedded Software Primer.* Addison-Wesley.
[2] Barr, M. Massa, A., 2007. *Programming Embedded Systems with C and GNU Development Tools*. O'Reilly.
[3] Pont, M.J., 2002. *Embedded C*. Addison-Wesley.
[4] Ganssle, J., 2007. *Embedded Systems: World Class Designs*. Newnes.

传感器和执行器。通常这意味着有线或无线通信子系统，如RS-232、RS-485、以太网、USB、802.11x或其他类似设施。在嵌入式系统中，直接内存访问（DMA）和多端口存储器有时也用于此目的，这在一定程度上模糊了基础设施与通信硬件之间的界限。

最后，我们还有用于监测或操纵物理世界的设备——即传感器和执行器。传感器使用电子、机械或化学手段来监测物理现象的状态，例如心率、飞机位置、物体质量或化学物质浓度。另一方面，执行器改变某些现实世界元素的物理状态。典型的执行器包括电机、加热器、发电机、泵和开关。

所有这些类型的硬件通常都需要初始化、测试和配置——这些任务通常由嵌入式软件在启动时或执行期间或两者兼有时完成。所有这些硬件元素都由嵌入式软件管理，向软件提供命令或数据，或从软件接收命令或数据。

本章将假定您具备足够的硬件背景知识，能够理解本书中使用的硬件术语，如总线、CPU、RAM、ROM、EPROM、EEPROM和闪存。如果您想了解更多，有许多关于该主题的优秀书籍，包括David Simons[1], Michael Barr和Anthony Massa[2], Michael Pont[3], 以及Jack Ganssle[4]的著作。相反，我们将重点关注开发嵌入式C应用程序以解决管理和操纵此类硬件元素时常见问题的常用方法。我们将简要讨论的一些硬件相关概念包括位域、端口和中断。

硬件使用位域来指定发送给硬件的命令或返回数据，这是极其常见的。位域是可寻址存储单元（例如，一个字节或字）内一个连续的位块（一个或多个位），这些位共同对硬件设备具有某种语义含义。例如，一个8位字节可能被映射到硬件设备中，分为四个不同的字段：

```
0 0 0000 00
```

这些位表示与内存映射硬件设备之间传输的以下信息：

Bit 范围	访问	Name	描述
0	只写	使能位	0 = 禁用设备，1 = 启用设备
1	只读	错误状态位	0 = 无错误，1 = 存在错误
2–5	只写	电机速度	范围 0000 = 无速度到 1111 (16d) 最高速度
6–7	只写	LED颜色	00 (0d) = 关闭 01 (1d) = 绿色 10 (2d) = 黄色 11 (3d) = 红色

[1] Simon, D., 1999. 嵌入式软件入门. Addison-Wesley. [2] Barr, M. Massa, A., 2007. 使用C和GNU开发工具进行嵌入式系统编程. O'Reilly. [3] Pont, M.J., 2002. 嵌入式C. Addison-Wesley. [4] Ganssle, J., 2007. 嵌入式系统：世界级设计. Newnes.

Bit fields are manipulated with C's bit-wise operators & (bit-wise AND), | (bit-wise OR), ~ (bit-wise NOT), ^ (bit-wise XOR), >> (right shift), and << (left shift).

Here are some examples of the results of using these bit-wise operators with different masks on a specific value:

```
Value    01010101    Value    01010101    Value    01010101
Mask     11111111    Mask     00000000    Mask     00001111
ANDED    01010101    ANDED    00000000    ANDED    00000101

Value    01010101    Value    01010101    Value    01010101
Mask     11111111    Mask     00000000    Mask     00001111
ORED     11111111    ORED     01010101    ORED     01011111

Value    01010101    Value    01010101    Value    01010101
Mask     11111111    Mask     00000000    Mask     00001111
XORED    10101010    XORED    01010101    XORED    01011010
```

A common related C idiom is to use `#define` to create bit masks for ANDing and ORing and give them meaningful names. For example, we might create a set of bit masks for common manipulations of the memory mapped device above, such as:

```
#include <stdlib.h>
#include <stdio.h>
#define TURN_OFF (0x00)
#define INITIALIZE (0x61)
#define RUN (0x69)
#define CHECK_ERROR (0x02)
#define DEVICE_ADDRESS (0x01FFAFD0)

void emergencyShutDown(void){
    printf("OMG We're all gonna die!\n");
}

int main() {
    unsigned char* pDevice;

    pDevice = (unsigned char *)DEVICE_ADDRESS; // pt to device
    // for testing you can replace the above line with
    // pDevice = malloc(1);
    *pDevice = 0xFF;     // start with all bits on
    printf ("Device bits %X\n", *pDevice);
    *pDevice = *pDevice & INITIALIZE; // and the bits into
    printf ("Device bits %X\n", *pDevice);
    if (*pDevice & CHECK_ERROR) { // system fail bit on?
            emergencyShutDown();
            abort();
            }
    else    {
            *pDevice = *pDevice & RUN;
```


位域使用C的位运算符进行操作：&（按位与）、|（按位或）、~（按位NOT）、^（按位异或）、>>（右移）和<<（左移）。

以下是一些示例，展示了在特定值上使用这些位运算符与不同掩码的结果：

值 01010101 值 01010101 值 01010101 掩码 11111111 掩码 00000000 掩码 00001111 与运算 01010101 与运算 00000000 与运算 00000101 值 01010101 值 01010101 值 01010101 掩码 11111111 掩码 00000000 掩码 00001111 或运算 11111111 或运算 01010101 或运算 01011111 值 01010101 值 01010101 值 01010101 掩码 11111111 掩码 00000000 掩码 00001111 异或运算 10101010 异或运算 01010101 异或运算 01011010

一个常见的相关 C 惯用法是使用 `#define` 来创建用于与运算和或运算的位掩码，并赋予它们有意义的名称。例如，我们可以为上述内存映射设备的常见操作创建一组位掩码，例如：

#include <stdlib.h>#include <stdio.h>#define TURN_OFF (0x00) #define INITIALIZE (0x61) #define RUN (0x69) #define CHECK_ERROR (0x02) # define DEVICE_ADDRESS (0x01FFAFD0) void emergencyShutDown(void){ printf("OMG 我们都要死了！ \n"); } int main() { unsigned char* pDevice; pDevice = (unsigned char *)DEVICE_ADDRESS; // 指向设备 // 测试时你可以将上面一行替换为 // pDevice = malloc(1); *pDevice = 0xFF; // 从所有位开启开始 printf ("设备位 %X\n", *pDevice); *pDevice = *pDevice & INITIALIZE; // 将位与运算 printf ("设备位 %X\n", *pDevice); if (*pDevice & CHECK_ERROR) { // 系统故障位开启? emergencyShutDown(); abort(); } else { *pDevice = *pDevice & RUN;

```
        printf ("Device bits %X\n", *pDevice);
        };
    return 0;
};
```

Code Listing 3-1: Manipulating bit-oriented memory mapped hardware

The left and right shift operators are useful for setting or testing particular bits in isolation as well as for processing serial bit data. For example, the expression "1<<3" sets bit 3 resulting in the value 8. This is sometimes used in the definition of bits in a mask. For example, the mask CHECK_ERROR in Code Listing 3-1 could have been written:

```
#define CHECKERROR (1<<1)
```

Bit fields in C offer another way to represent bit-mapped fields in device interfaces. This syntax represents variable-length bit fields as fields within a struct. The ':' operator separates the name of the field from its length in the declaration. For example, see Code Listing 3-2.

```
#include <stdlib.h>
#include <stdio.h>

int main() {
    typedef struct _statusBits {
            unsigned enable : 1;
            unsigned errorStatus : 1;
            unsigned motorSpeed : 4;
            unsigned LEDColor : 2;
    } statusBits;
    statusBits status;

    printf("size = %d\n",sizeof(status));
    status.enable = 1;
    status.errorStatus = 0;
    status.motorSpeed = 3;
    status.LEDColor = 2;

    if (status.enable) printf("Enabled\n");
         else printf ("Disabled\n");
    if (status.errorStatus) printf("ERROR!\n");
         else printf("No error\n");
    printf ("Motor speed %d\n",status.motorSpeed);
    printf ("Color %d\n",status.LEDColor);
    return 0;
};
```

Code Listing 3-2: Bit fields in C


```
printf ("设备位 %X\n", *pDevice); }; return 0; };
```

代码清单3-1：操作面向位的内存映射硬件

左移和右移运算符对于单独设置或测试特定位以及处理串行位数据都很有用。例如，表达式"1<<3"设置第3位，结果为值8。这有时用于掩码中位的定义。例如，代码清单3-1中的掩码CHECK_ERROR可以写成：

```
#define CHECKERROR (1<<1)
```

C语言中的位域提供了另一种在设备接口中表示位映射字段的方法。这种语法将可变长度的位域表示为结构体中的字段。':'运算符在声明中将字段名称与其长度分开。例如，参见代码清单3-2。

```
#include <stdlib.h>#include <stdio.h>int main() { typedef
struct _statusBits { unsigned enable : 1; unsigned
errorStatus : 1; unsigned motorSpeed : 4; unsigned
LEDColor : 2; } statusBits; statusBits status; printf("size = %
d\n",sizeof(status)); status.enable = 1; status.errorStatus = 0;
status.motorSpeed = 3; status.LEDColor = 2; if
(status.enable) printf("已启用\n"); else printf ("已禁用\n"); if
(status.errorStatus) printf("错误！\n"); else printf("无错误\n");
printf (" 电机速度 %d\n",status.motorSpeed); printf (" 颜色 %
d\n",status.LEDColor); return 0; };
```

代码清单3-2：C语言中的位域

There are a couple of problems with bit fields in C. First, bit ordering is compiler- and processor-dependent. Further, the compiler may enforce byte padding rules, meaning that it may not map onto a memory mapped device as you expect. For example, the GNU C compiler returns size of `status` in Code Listing 3-2 to be 4 bytes even though the size of an unsigned char is only 1 byte.

Even worse, because most CPUs must write a byte or word at a time, bit fields may not be written in an atomic step, leading to thread safety issues if separate mutexes are used for different bit fields.

Another potential problem with using bit fields is that it is impossible to cast between a scalar and a user-defined struct. Thus, the following is disallowed[5]:

```
Unsigned char f;
f = 0xF0;
status = (statusBits) f;
```

Let's turn our attention now to a number of design patterns that have proven themselves useful for the manipulation of hardware. The Hardware Proxy Pattern, discussed next, is an archetypal pattern for the abstraction of hardware for the purpose of encapsulating details that are likely to change from the usage of the information provided to or by the hardware. The Hardware Adapter Pattern extends the Hardware Proxy Pattern to provide the ability to support different hardware interfaces. The Mediator Pattern supports coordination of multiple hardware devices to achieve a system level behavior. The Observer Pattern is a way of distributing sensed data to the software elements that need it. The Debouncing and Interrupt Patterns are simple reusable approaches to interface with hardware devices. The Timer Interrupt Pattern extends the Interrupt timer to provide accurate timing for embedded systems.

3.2 Hardware Proxy Pattern

The *Hardware Proxy Pattern* creates a software element responsible for access to a piece of hardware and encapsulation of hardware compression and coding implementation.

3.2.1 Abstract

The *Hardware Proxy Pattern* uses a class (or struct) to encapsulate all access to a hardware device, regardless of its physical interface. The hardware may be memory, port, or interrupt

[5] Although you *can* get around that by doing something as circuitous as

`status = *(statusBits*)&f`

(that is, casting the address of f (unsigned char) to be an address of the structure type statusBits) but I can hardly recommend that!

C语言中的位域存在几个问题。首先，位序取决于编译器和处理器。此外，编译器可能会强制执行字节填充规则，这意味着它可能不会像你预期的那样映射到内存映射设备上。例如，GNU C编译器将代码清单3-2中`status`的大小返回为4字节，尽管无符号字符的大小仅为1字节。

更糟糕的是，由于大多数 CPU 每次必须写入一个字节或一个字，位域可能无法在原子步骤中写入，如果为不同的位域使用不同的互斥锁，就会导致线程安全问题。

使用位域的另一个潜在问题是无法在标量和用户定义结构体之间进行强制转换。因此，以下写法是不允许的[5]:

```
无符号字符 f; f = 0xF0;
status = (statusBits) f;
```

现在让我们将注意力转向一些已被证明对硬件操作有用的设计模式。接下来讨论的硬件代理模式是一种典型的模式，用于对硬件进行抽象，以封装那些可能因所提供或由硬件提供的信息的使用而发生变化的具体细节。硬件适配器模式扩展了硬件代理模式，以提供支持不同硬件接口的能力。中介者模式支持多个硬件设备的协调，以实现系统级行为。观察者模式是一种将感知数据分发给需要它的软件元素的方式。去抖模式和中断模式是与硬件设备接口的简单可复用方法。定时器中断模式扩展了中断定时器，为嵌入式系统提供精确的时序。

3.2 硬件代理模式

硬件代理模式创建一个软件元素，负责访问某块硬件，并封装硬件压缩和编码实现。

3.2.1 摘要

硬件代理模式使用一个类（或结构体）来封装对硬件设备的所有访问，无论其物理接口是什么。硬件可以是内存、端口或中断

[5] 尽管你可以通过像`status = *(statusBits*)&f`这样迂回的方式绕过它（也就是说，将 f（无符号字符）的地址强制转换为结构体类型 statusBits 的地址），但我几乎不能推荐这种做法!

mapped, or may even be mapped via a serial connection, bus, network, or wireless link. The proxy publishes services that allow values to be read from and written to the device, as well as initialize, configure, and shut down the device as appropriate. The proxy provides an encoding and connection-independent interface for clients and so promotes easy modification should the nature of the device interface or connection change.

3.2.2 Problem

If every client accesses a hardware device directly, problems due to hardware changes are exacerbated. If the bit encoding, memory address, or connection technology changes, then every client must be tracked down and modified. By providing a proxy to sit between the clients and the actual hardware, the impact of hardware changes is greatly limited, easing such modifications. For easiest maintenance, the clients should be unaware of bit encoding, encryption, and compression used by the device; these details should be managed by the Hardware Proxy with internal private functions.

3.2.3 Pattern Structure

The pattern structure is quite simple as can be seen in Figure 3-1[6]. There may be many clients but a single Hardware Proxy per device being controlled. The proxy contains both public functions and private, encapsulated functions and data. The device is represented as a `void*` in the figure, but it should be appropriately typed for the device.

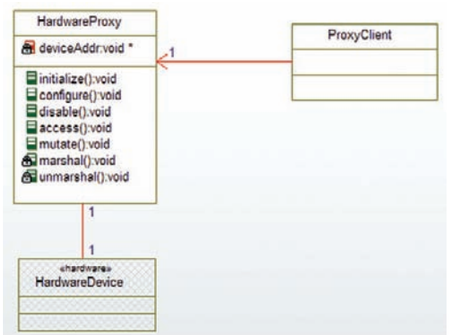


Figure 3-1 Hardware Proxy Pattern

[6] Each of these patterns will be shown as a UML diagram and code will be provided for the examples. The Appendix provides a short overview of the UML notation.

映射的，甚至可以通过串行连接、总线、网络或无线链接进行映射。代理发布服务，允许从设备读取值并向设备写入值，以及适当地初始化、配置和关闭设备。代理为客户端提供了一个与编码和连接无关的接口，因此当设备接口或连接的性质发生变化时，可以轻松进行修改。

3.2.2 问题

如果每个客户端都直接访问硬件设备，那么由硬件变更引起的问题会加剧。如果位编码、内存地址或连接技术发生变化，那么必须逐一找到并修改每个客户端。通过在客户端与实际硬件之间提供一个代理，硬件变更的影响被大大限制，从而简化了此类修改。为了实现最简便的维护，客户端不应感知设备所使用的位编码、加密和压缩；这些细节应由硬件代理通过内部私有函数来管理。

3.2.3 模式结构

该 模式的结构 如 图 3-1[6] 所示， 相当 简单。 可能有 多个客户端，但每个被控设备只有一个硬件代理。代理包含公共函数以及私有的、封装的函数和数据。图中设备表示为 `void*`，但应根据设备类型进行适当的类型化。

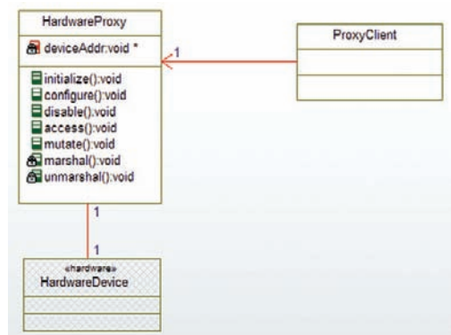


图3-1 硬件代理模式

[6] 这些模式中的每一个都将以UML图的形式展示，并且将为示例提供代码。附录提供了UML表示法的简要概述。

3.2.4 Collaboration Roles

This section describes the roles for this pattern.

3.2.4.1 Hardware Device

This element represents the actual hardware. As such, you will not write C code for this element, but it is present just to aid with understanding the diagram. The association shown between the Hardware Device and the Hardware Proxy is realized via a software-addressable hardware interface, such as a port address, memory address, or interrupt.

3.2.4.2 Hardware Proxy

This is the primary class (or function) in the system. It has both data and functions that are customized to the device at hand, although usually each one has an initialize(), configure(), and disable() function. The other public functions provide read access to values from the device or set values sent to the device. Although the pattern structure has a single access() and mutate() function, it is common to have several such functions, each bearing the name of the value being read or set.

The key features[7] of the class include:

access()

This public function returns a particular value from the device. In most cases, the class will contain a separate function for each separately-identifiable piece of information desired from the device. This operation usually calls the `unmarshal()` function prior to returning the retrieved values to the client.

configure()

This public function provides a means by which the device may be configured. Although in the general pattern no parameter list can be provided, this function almost always takes a list of parameters that specify the proper operating state of the device.

disable()

This public function provides a means by which the device may be safely turned off or disabled. It may or may not have parameters depending upon the device characteristics.

deviceAddr

This private variable provides low-level direct access to the hardware. In the pattern it is shown as a `void*` but it may be a memory mapped integer (`int*`) or other primitive type, a port-mapped device port number, or some other location identified. If more complex means are used

[7] By the term *feature*, I am referring primarily to data elements (attributes or variables), operations and functions, states, and event receptions.

3.2.4 协作角色

本节描述该模式的角色。

3.2.4.1 硬件设备

该元素代表实际的硬件。因此，你不会为该元素编写C代码，它的存在只是为了帮助理解图示。硬件设备与硬件代理之间所示的关联是通过软件可寻址硬件接口实现的，例如端口地址、内存地址或中断。

3.2.4.2 硬件代理

这是系统中的主要类（或函数）。它包含针对当前设备定制的数据和函数，不过通常每个都包含 initialize()、configure() 和 disable() 函数。其他公共函数提供对设备值的读取访问，或设置发送到设备的值。尽管模式结构只有一个 access() 和 mutate() 函数，但通常会有多个此类函数，每个都以所读取或设置的值的名称命名。

该 类的关键[7] 特性 包括：

access()

该公共函数从设备返回特定值。在大多数情况下，该类会为从设备获取的每一项可单独识别的信息包含一个单独的函数。此操作通常在将检索到的值返回给客户端之前调用`unmarshal()`函数。

configure()

该公共函数提供了一种配置设备的方式。尽管在通用模式中无法提供参数列表，但此函数几乎总是接受一组指定设备适当操作状态的参数。

disable()

该公共函数提供了一种可以安全关闭或禁用设备的方法。根据设备特性，它可能带有参数，也可能不带参数。

deviceAddr

这个私有变量提供对硬件的低层直接访问。在模式中它显示为 `void*`，但它可能是内存映射整数（`int*`）或其他基本类型、端口映射的设备端口号，或某些其他标识的位置。如果使用更复杂的方式

[7] 所谓特性，我主要指数据元素（属性或变量）、操作和函数、状态以及事件接收。

to access the device, such as an RS232 serial port or an Ethernet connection, then this data type and its access method will be more complex. In any event, the public functions provided by the Hardware Proxy completely hide how the proxy connects to the actual device. This variable is not directly accessible by the client.

initialize()

This public function enables and initializes the device prior to first use. It is common for this function to have no parameters but in some cases it may need values provided by the client.

marshal()

This private function takes parameters from various other functions and performs any required encryption, compression, or bit-packing required to send the data to the device. This ensures that the peculiarities of the device interface are hidden from the client. Data in the format required by the actual device are known as "marshaled data" or are said to be in "native format." Data that are in a form that is easily manipulated by software are said to be "unmarshaled" or in "presentation format." This function is not accessible by the client since native format is hidden from the clients.

mutate()

This public function writes data values to the device. This function always has one or more input parameters. It usually calls the `marshal()` operation prior to writing the value to the device.

unmarshal()

This private function performs any necessary unpacking, decryption, and decompression of the data retrieved from the device prior to returning them to the client in presentation format. This ensures that the peculiarities of the device interface are hidden from the client. This function is not accessible by the client.

3.2.4.3 Proxy Client

The proxy client "knows about" the Hardware Proxy and invokes its services to access the hardware device.

3.2.5 Consequences

The pattern is extremely common and provides all the benefits of encapsulation of the hardware interface and encoding details. It provides flexibility for the actual hardware interface to change radically with absolutely no changes in the clients. This is because the hardware details are all encapsulated within the Hardware Proxy. This means that the proxy clients are usually unaware of the native format of the data and manipulate them only in presentation format.

来访问设备，例如RS232串行端口或以太网连接，那么这个数据类型及其访问方法将更加复杂。无论如何，硬件代理提供的公共函数完全隐藏了代理如何连接到实际设备。该变量不能被客户端直接访问。

initialize()

该公共函数在首次使用前启用并初始化设备。通常该函数没有参数，但在某些情况下可能需要客户端提供值。

marshal()

该私有函数从其他各种函数接收参数，并执行将数据发送到设备所需的任何加密、压缩或位打包操作。这确保设备接口的特殊性对客户端隐藏。以实际设备所需格式表示的数据称为"封送数据"，或称处于"原生格式"。以易于软件操作的形式表示的数据称为"未封送"或处于"表示格式"。该函数不可由客户端访问，因为原生格式对客户端是隐藏的。

mutate()

该公共函数将数据值写入设备。该函数始终有一个或多个输入参数。它通常在将值写入设备之前调用`marshal()`操作。

unmarshal()

该私有函数对从设备检索到的数据执行任何必要的解包、解密和解压缩，然后以表示格式将其返回给客户端。这确保设备接口的特殊性对客户端隐藏。该函数不可由客户端访问。

3.2.4.3 代理客户端

代理客户端"了解"硬件代理并调用其服务来访问硬件设备。

3.2.5 结果

该模式极为常见，并提供了封装硬件接口和编码细节的全部优势。它为实际硬件接口的彻底变更提供了灵活性，而客户端完全无需改动。这是因为硬件细节全部封装在硬件代理内部。这意味着代理客户端通常不了解数据的原生格式，仅以表示格式对其进行操作。

This can, however, have a negative impact on run-time performance. It may sometimes be more time efficient for the clients to know the encoding details and manipulate the data in their native format. However, this degrades the maintainability of the system because now the clients need to be modified should the hardware interface or encoding change.

3.2.6 Implementation Strategies

As discussed in Chapter 1, classes may be implemented in different ways in C ranging from simple files to using struct to store class attributes, to using virtual function tables that support true polymorphism. All of those approaches can be used to implement this very simple pattern.

Usually, a Hardware Proxy supports all features of a specific device and a different Hardware Proxy is used for each separate device, but this isn't an absolute rule. Separation into different devices means that the devices can follow independent maintenance paths and so is very flexible for the future.

In general, it is preferable to hide the bit-encoding, encryption, and data compression of the actual hardware away from the application software[8]. However, it is possible to implement the pattern without the `marshal()` and `unmarshal()` functions by making the native format visible to the clients.

3.2.7 Related Patterns

The simple implementation of this pattern doesn't implement anything for thread safety. It can be combined with the *Critical Region, Guarded Call*, or *Queuing* patterns to provide thread safety. For deadlock avoidance, it can be combined with the *Ordered Locking* or *Simultaneous Locking* patterns.

3.2.8 Example

The example, shown in Figure 3-2, is of a system with a motor with a memory-mapped interface. The interface is 16 bits wide and is detailed in the figure comment. In addition to the motor proxy, there are two clients in the example. The first of these is a motor controller than makes high-level decisions about what to do with the motor direction, speed, and how to handle errors. The motor display client is concerned with providing user feedback of the speed, direction, and status of the motor. Most of the action, of course, is in the Hardware Proxy *per se*.

The intention of the motor proxy is to provide the services to access the hardware in a hardware interface-independent fashion so that the two clients needn't worry about – or even be aware of

[8] It may not be in high-security applications however. In those cases, it may be necessary to always have the data stored in encrypted formats to make it more difficult to violate data security protocols.

然而，这可能对运行时性能产生负面影响。有时，让客户端了解编码细节并以原生格式操作数据可能在时间上更为高效。但这会降低系统的可维护性，因为一旦硬件接口或编码发生变化，客户端就需要被修改。

3.2.6 实现策略

如第1章所述，类在C语言中可以通过多种方式实现，从简单的文件到使用结构体存储类属性，再到使用支持真正多态的虚函数表。所有这些方法都可以用来实现这个非常简单的模式。

通常，一个硬件代理支持特定设备的所有特性，并且每个独立设备使用不同的硬件代理，但这并非绝对规则。分离为不同的设备意味着这些设备可以遵循独立的维护路径，因此对未来非常灵活。

一般来说，最好将实际硬件的位编码、加密和数据压缩对应用软件[8]隐藏起来。然而，通过使原生格式对客户端可见，也可以在不使用marshal()和unmarshal()函数的情况下实现该模式。

3.2.7 相关模式

该模式的简单实现并未实现任何线程安全功能。它可以与临界区、受保护调用或排队模式结合使用，以提供线程安全。对于死锁避免，它可以与有序锁定或同时锁定模式结合使用。

3.2.8 示例

该示例（如图3-2所示）是一个具有内存映射接口的电机的系统。该接口为16位宽，详细信息见图中的注释。除电机代理外，示例中还有两个客户端。第一个客户端是电机控制器，它负责对电机方向、速度以及如何处理错误做出高层决策。电机显示客户端则负责向用户提供电机速度、方向和状态的反馈。当然，大部分动作都发生在硬件代理本身。

电机代理模式的意图是以独立于硬件接口的方式提供服务来访问硬件，从而使两个客户端无需担心——甚至无需了解

[8] 然而，在高安全性应用中可能并非如此。在那些情况下，可能有必要始终以加密格式存储数据，以增加违反数据安全协议的难度。

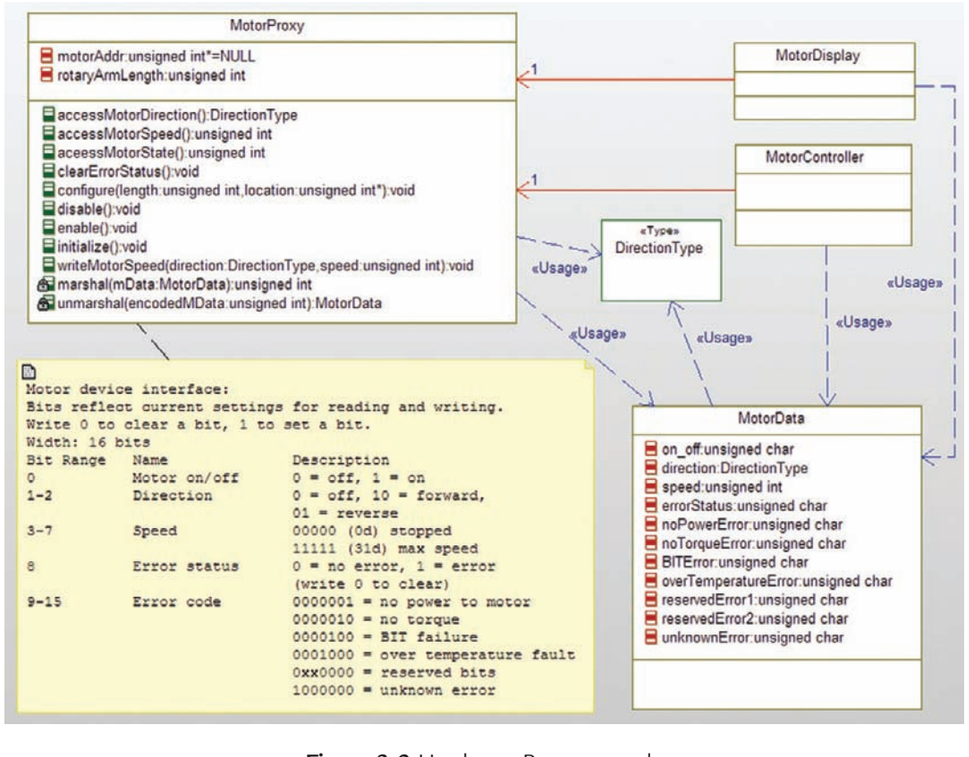


Figure 3-2 Hardware Proxy example

– the specific encoding of the bits. To this end, several types of services are offered by the motor proxy:

Motor Management Functions

- configure() – this function sets up the memory-mapped address of the motor and the length of the rotary arm; this function must be called first
- disable() – this function turns off the motor but keeps the set values intact
- enable() – this function turns the motor on with the current set values
- initialize() – this function turns on the motor to default set values (off)

Motor Status Functions

- accessMotorDirection() – this function returns the current motor direction (off, forward, or reverse)
- accessMotorSpeed() – this function returns the speed of motor

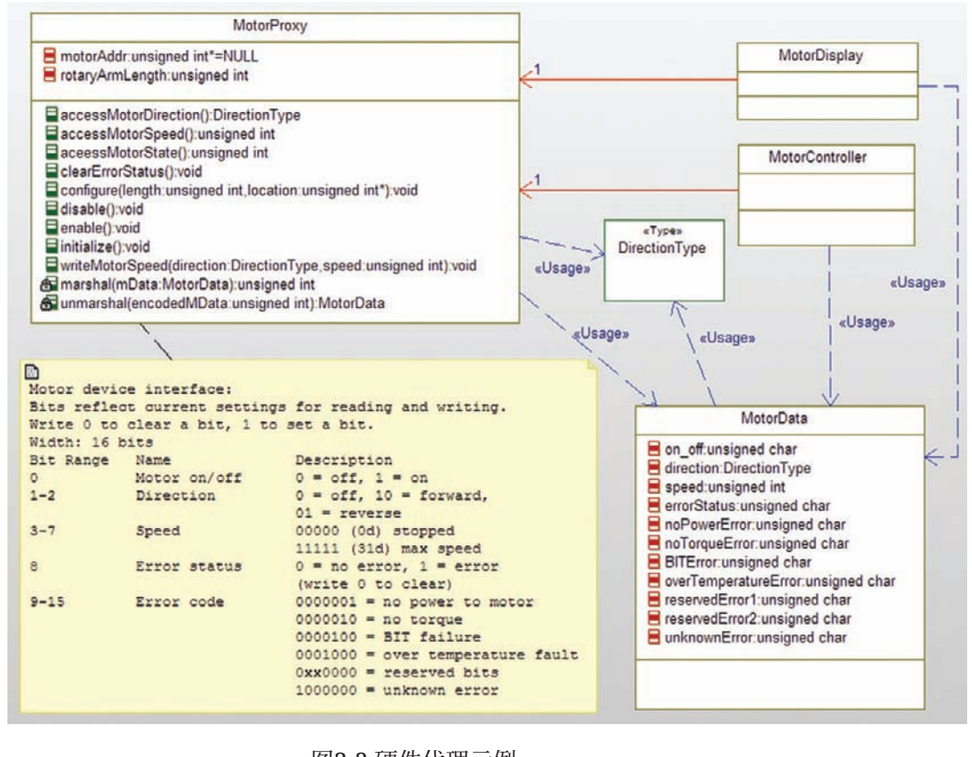


图3-2 硬件代理示例

——比特的具体编码方式。为此，电机代理模式提供了几种类型的服务：

电机管理功能

- configure() – 此函数设置电机的内存映射地址和旋转臂的长度；必须先调用此函数
- disable() – 此函数关闭电机，但保持设定值不变
- enable() – 此函数使用当前设定值开启电机
- initialize() – 此函数将电机开启至默认设定值（关闭）

电机状态函数

- accessMotorDirection() – 此函数返回当前电机方向（关闭、FORWARD 或 REVERSE）
- accessMotorSpeed() – 此函数返回电机的速度

Motor Control Functions

- writeMotorSpeed() – this function sets the speed of the motor and adjusts for the length of the rotary arm (if set)

Motor Error Management Functions

- clearErrorStatus() – this function clears all error bits
- accessMotorState() – this function returns the error status

Internal Data Formatting Functions (private)

These functions aren't provided to the clients, but are used internally to exchange the data between native and presentation formats.

- marshal() – converts presentation (client) data format to native (motor) format
- unmarshal() – converts native (motor) data format to presentation (client) format

The C source code for the data types and MotorProxy class are shown in the next several code segments. Code Listing 3-3 shows the data typing for the DirectionType while Code Listing 3-4 shows the presentation client format for MotorData.

```
#ifndef HWProxyExample_H
#define HWProxyExample_H

struct MotorController;
struct MotorData;
struct MotorDisplay;
struct MotorProxy;

typedef enum DirectionType {
        NO_DIRECTION,
        FORWARD,
        REVERSE
} DirectionType;
#endif
```

Code Listing 3-3: HardwareProxyExample.h

```
#ifndef MotorData_H
#define MotorData_H

#include "HWProxyExample.h"
typedef struct MotorData MotorData;
struct MotorData {
        unsigned char on_off;
        DirectionType direction;
        unsigned int speed;
        unsigned char errorStatus;
        unsigned char noPowerError;
```


电机控制函数

- writeMotorSpeed() – 此函数设置电机的速度，并根据旋转臂的长度进行调整（如果已设置）

电机错误管理函数

- clearErrorStatus() – 此函数清除所有错误位
- accessMotorState() – 此函数返回错误状态

内部数据格式化函数（私有）

这些函数不提供给客户端，而是在内部用于在本机格式和表示格式之间交换数据。

- marshal() – 将表示层（客户端）数据格式转换为本机（电机）格式
- unmarshal() – 将本机（电机）数据格式转换为表示层（客户端）格式

数据类型和MotorProxy类的C源代码在接下来的几个代码段中展示。代码清单3-3展示了DirectionType的数据类型，而代码清单3-4展示了MotorData的表示层客户端格式。

```
#ifndef HWProxyExample_H # define HWProxyExample_H
struct MotorController; struct MotorData; struct MotorDisplay; struct MotorProxy; typedef enum DirectionType { NO_DIRECTION, FORWARD, REVERSE } DirectionType; # endif
```

代码清单3-3：HardwareProxyExample.h

```
#ifndef MotorData_H #define MotorData_H #include " HWProxyExample.h" typedef struct MotorData MotorData; struct MotorData { 无符号字符 on_off; DirectionType direction; unsigned int speed; 无符号字符 errorStatus; 无符号字符 noPowerError;
```

```
            unsigned char noTorqueError;
            unsigned char BITError;
            unsigned char overTemperatureError;
            unsigned char reservedError1;
            unsigned char reservedError2;
            unsigned char unknownError;
};
#endif
```

Code Listing 3-4: MotorData.h

The next two code listings show the header and implementation files for the MotorProxy class itself including both the data structure and the member functions.

```
#ifndef MotorProxy_H
#define MotorProxy_H

#include "HWProxyExample.h"
#include "MotorData.h"
/* class MotorProxy */
typedef struct MotorProxy MotorProxy;
/* This is the proxy for the motor hardware.    */
/* Note that the speed of the motor is adjusted for the length of the rotary arm */
/* to keep a constant speed at the end of the arm. */
struct MotorProxy {
        unsigned int* motorAddr;
        unsigned int rotaryArmLength;
};

void MotorProxy_Init(MotorProxy* const me);
void MotorProxy_Cleanup(MotorProxy* const me);
DirectionType* MotorProxy_accessMotorDirection(MotorProxy* const me);
unsigned int MotorProxy_accessMotorSpeed(MotorProxy* const me);
unsigned int MotorProxy_aceessMotorState(MotorProxy* const me);
/* keep all settings the same but clear error bits */ void MotorProxy_clearErrorStatus
(MotorProxy* const me);
/* Configure must be called first, since it sets up the */
/* address of the device.    */
void MotorProxy_configure(MotorProxy* const me, unsigned int length, unsigned int*
location);

/* turn motor off but keep original settings */
void MotorProxy_disable(MotorProxy* const me);
/* Start up the hardware but leave all other settings of the */ /* hardware alone */ void
MotorProxy_enable(MotorProxy* const me);
/* precondition: must be called AFTER configure() function.    */ /* turn on the hardware to
a known default state. */ void MotorProxy_initialize(MotorProxy* const me);
/* update the speed and direction of the motor together */
void MotorProxy_writeMotorSpeed(MotorProxy* const me, const DirectionType* direction,
unsigned int speed);
```


无符号字符 noTorqueError; 无符号字符 BITError; 无符号字符 overTemperatureError; 无符号字符 reservedError1; 无符号字符 reservedError2; 无符号字符 unknownError; }; #endif

代码清单3-4：MotorData.h

接下来的两个代码清单展示了MotorProxy类本身的头文件和实现文件，包括数据结构和成员函数。

#ifndef MotorProxy_H #define MotorProxy_H #include "HWProxyExample.h" #include "MotorData.h" /* 类 MotorProxy */ typedef struct MotorProxy MotorProxy; /* 这是电机硬件的代理模式。 */ /* 注意，电机的速度会根据旋转臂的长度进行调整 */ /* 以保持臂末端的恒定速度。 */ struct MotorProxy { unsigned int* motorAddr; unsigned int rotaryArmLength; }; void MotorProxy_Init(MotorProxy* const me); void MotorProxy_Cleanup(MotorProxy* const me); DirectionType* MotorProxy_accessMotorDirection(MotorProxy* const me); unsigned int MotorProxy_accessMotorSpeed(MotorProxy* const me); unsigned int MotorProxy_aceessMotorState(MotorProxy* const me); /* 保持所有设置不变但清除错误位 */ void MotorProxy_clearErrorStatus (MotorProxy* const me); /* 必须先调用 configure，因为它设置了 */ /* 设备的地址。 */ void MotorProxy_configure(MotorProxy* const me, unsigned int length, unsigned int* location); /* 关闭电机但保留原始设置 */ void MotorProxy_disable(MotorProxy* const me); /* 启动硬件但保留硬件的所有其他设置 */ /* 不变 */ void MotorProxy_enable(MotorProxy* const me); /* 前置条件：必须在 configure() 函数之后调用。 */ /* 将硬件开启到已知的默认状态。 */ void MotorProxy_initialize(MotorProxy* const me); /* 同时更新电机的速度和方向 */ void MotorProxy_writeMotorSpeed(MotorProxy* const me, const DirectionType* direction, unsigned int speed);

```
MotorProxy * MotorProxy_Create(void); void MotorProxy_Destroy(MotorProxy* const me);
#endif
```

Code Listing 3-5: MotorProxy.h

```
#include "MotorProxy.h"
/* class MotorProxy */
/* This function takes a MotorData structure and creates     */
/* a device-specific unsigned int in device native format. */
static unsigned int marshal(MotorProxy* const me, const struct MotorData* mData);
static struct MotorData* unmarshal(MotorProxy* const me, unsigned int encodedMData);
void MotorProxy_Init(MotorProxy* const me) { me->motorAddr = NULL;
}

void MotorProxy_Cleanup(MotorProxy* const me) {
}

DirectionType* MotorProxy_accessMotorDirection(MotorProxy* const me) {
        MotorData mData;
        if (!me->motorData) return 0;
        mData = unmarshall(*me->motorAddr);
        return mData.direction;
}

unsigned int MotorProxy_accessMotorSpeed(MotorProxy* const me) {
        MotorData mData;
        if (!me->motorData) return 0;
        mData = unmarshall(*me->motorAddr);
        return mData.speed;
}

unsigned int MotorProxy_aceessMotorState(MotorProxy* const me) {
        MotorData mData;
        if (!me->motorData) return 0;
        mData = unmarshall(*me->motorAddr);
        return mData.errorStatus;
}

void MotorProxy_clearErrorStatus(MotorProxy* const me) {
        if (!me->motorData) return;
        *me->motorAddr &= 0xFF;
}

void MotorProxy_configure(MotorProxy* const me, unsigned int length, unsigned int* location) {
        me->rotaryArmLength = length;
        me->motorAddr = location;
}

void MotorProxy_disable(MotorProxy* const me) {
        // and with all bits set except for the enable bit
        if (!me->motorData) return;
        me->MotorAddr & = 0xFFFE;
}
```


MotorProxy * MotorProxy_Create(void); void MotorProxy_Destroy(MotorProxy* const me); #endif

代码清单 3-5：MotorProxy.h

#include "MotorProxy.h" /* 类 MotorProxy */ /* 此函数接收一个 MotorData 结构体，并创建一个 */ /* 设备特定的无符号整型，采用设备原生格式。 */ static unsigned int marshal(MotorProxy* const me, const struct MotorData* mData)

;

static struct MotorData* unmarshal(MotorProxy* const me, unsigned int encodedMDa ta);
void MotorProxy_Init(MotorProxy* const me) { me->motorAddr = 空; } void MotorProxy_清理(MotorProxy* const me) { } DirectionType* MotorProxy_accessMotorDirection(MotorProxy* const me) { MotorData mData; if (!me->motorData) return 0; mData = unmarshal(*me->motorAddr); return mData.direction; } unsigned int MotorProxy_accessMotorSpeed(MotorProxy* const me) { MotorData mData; if (!me->motorData) return 0; mData = unmarshal(*me->motorAddr); return mData.speed; } unsigned int MotorProxy_aceessMotorState(MotorProxy* const me) { MotorData mData; if (!me->motorData) return 0; mData = unmarshal(*me->motorAddr); return mData.errorStatus; } void MotorProxy_clearErrorStatus(MotorProxy* const me) { if (!me->motorData) return; *me->motorAddr &= 0xFF; } void MotorProxy_configure(MotorProxy* const me, unsigned int length, unsigned int* loc

ation) {

me->rotaryArmLength = length; me->motorAddr = location; } void MotorProxy_disable(MotorProxy* const me) { // 与所有位进行与运算，但使能位除外 if (!me->motorData) return; me->MotorAddr & = 0xFFFE; }

```
void MotorProxy_enable(MotorProxy* const me) {
        if (!me->motorData) return;
        *me->motorAddr |= 1;
}

void MotorProxy_initialize(MotorProxy* const me) {
        MotorData mData;
        if (!me->motorData) return;
        mData.on_off = 1;
        mData.direction = 0;
        mData.speed = 0;
        mData.errorStatus = 0;
        mData.noPowerError) = 0;
        mData.noTorqueError) = 0;
        mData.BITError) = 0;
        mData.overTemperatureError) = 0;
        mData.reservedError1) = 0;
        mData.reservedError2) = 0;
        Data.unknownError) = 0;
        *me->motorAddr = marshall(mData);
}

void MotorProxy_writeMotorSpeed(MotorProxy* const me, const DirectionType* direction,
unsigned int speed) {
        MotorData mData
        double dPi, dArmLength, dSpeed, dAdjSpeed;
        if (!me->motorData) return; mData = unmarshall(*me->motorAddr); mData.direction =
direction;
        // ok, let's do some math to adjust for
        // the length of the rotary arm times 10
        if (me->rotaryArmLength > 0) {
               dSpeed = speed;
               dArmLength = me->rotaryArmLength;
               dAdjSpeed = dSpeed / 2.0 / 3.14159 / dArmLength * 10.0;
               mData.speed = (int)dAdjSpeed;
        }
        else
            mData.speed = speed;
        *me->motorData = marshal(mData);
        return;
}

static unsigned int marshal(MotorProxy* const me, const struct MotorData* mData) {
        unsigned int deviceCmd;
        deviceCmd = 0; // set all bits to zero
        if (mData.on_off) deviceCmd |= 1; // OR in the appropriate bit
        if (mData.direction == FORWARD)
            deviceCmd |= (2 << 1);
        else if (mData.direction == REVERSE)
            deviceCmd |= (1 << 1);
        if (mData.speed < 32 && mData.speed >= 0)
            deviceCmd |= mData.speed << 3;
```


void MotorProxy_enable(MotorProxy* const me) { if (!me->motorData) return; *me->motorAddr |= 1; } void MotorProxy_initialize(MotorProxy* const me) { MotorData mData; if (!me->motorData) return; mData.on_off = 1; mData.direction = 0; mData.speed = 0; mData.errorStatus = 0; mData.noPowerError) = 0; mData.noTorqueError) = 0; mData.BITError) = 0; mData.overTemperatureError) = 0; mData.reservedError1) = 0; mData.reservedError2) = 0; Data.unknownError) = 0; *me->motorAddr = marshall(mData); } void MotorProxy_writeMotorSpeed(MotorProxy* const me, const DirectionType* direction, unsigned int speed) { MotorData mData double dPi, dArmLength, dSpeed, dAdjSpeed; if (!me->motorData) return; mData = unmarshall(*me->motorAddr); mData.direction =direction; // 好的，让我们做一些数学运算来调整 // 旋转臂长度乘以10 if (me->rotaryArmLength > 0) { dSpeed = speed; dArmLength = me->rotaryArmLength; dAdjSpeed = dSpeed / 2.0 / 3.14159 / dArmLength * 10.0; mData.speed = (int)dAdjSpeed; } else mData.speed = speed; *me->motorData = marshal(mData); return; } static unsigned int marshal(MotorProxy* const me, const struct MotorData* mData) { unsigned int deviceCmd; deviceCmd = 0; // 将所有位设置为零 if (mData.on_off) deviceCmd |= 1; // 或运算设置相应的位 if (mData.direction == FORWARD) deviceCmd |= (2 << 1); else if (mData.direction == REVERSE) deviceCmd |= (1 << 1); if (mData.speed < 32 && mData.speed >= 0) deviceCmd |= mData.speed << 3;

```
        if (mData.errorStatus) deviceCmd |= 1 << 8;
        if (mData.noPowerError) deviceCmd |= 1 << 9;
        if (mData.noTorqueError) deviceCmd |= 1 << 10;
        if (mData.BITError) deviceCmd |= 1 << 11;
        if (mData.overTemperatureError) deviceCmd |= 1 << 12;
        if (mData.reservedError1) deviceCmd |= 1 << 13;
        if (mData.reservedError2) deviceCmd |= 1 << 14;
        if (mData.unknownError) deviceCmd |= 1 << 15;
        return deviceCmd;
}

static struct MotorData* unmarshal(MotorProxy* const me, unsigned int encodedMData) {
        MotorData mData
        int temp;

        mData.on_off = encodedMData & 1;
        temp = (encodedMData & (3 << 1)) >> 1;     if (temp == 1)
            mData.direction = REVERSE;
        else if (temp == 2)
            mData.direction = FORWARD;
        else
            mData.direction = NO_DIRECTION;
        mData.speed = encodedMData & (31 << 3);
        mData.errorStatus = encodedMData & (1 << 8);
mData.noPowerError) = encodedMData & (1 << 9);
mData.noTorqueError) = encodedMData & (1 << 10); mData.BITError) = encodedMData & (1 <<
11); mData.overTemperatureError) = encodedMData & (1 << 12); mData.reservedError1) =
encodedMData & (1 << 13);     mData.reservedError2) = encodedMData & (1 << 14);
        Data.unknownError) = encodedMData & (1 << 15);
        return mData;
}

MotorProxy * MotorProxy_Create(void) {
        MotorProxy* me = (MotorProxy *) malloc(sizeof(MotorProxy));
        if(me!=NULL)
                {
                        MotorProxy_Init(me);
                }
        return me;
}

void MotorProxy_Destroy(MotorProxy* const me) {
        if(me!=NULL)
                {
                        MotorProxy_Cleanup(me);
                }
        free(me);
}
```

Code Listing 3-6: MotorProxy.c

if (mData.errorStatus) deviceCmd |= 1 << 8; if (mData.noPowerError) deviceCmd |= 1 << 9; if (mData.noTorqueError) deviceCmd |= 1 << 10; if (mData.BITError) deviceCmd |= 1 << 11; if (mData.overTemperatureError) deviceCmd |= 1 << 12; if (mData.reservedError1) deviceCmd |= 1 << 13; if (mData.reservedError2) deviceCmd |= 1 << 14; if (mData.unknownError) deviceCmd |= 1 << 15; return deviceCmd; } static struct MotorData* unmarshal(MotorProxy* const me, unsigned int encodedMData) { MotorData mData int temp; mData.on_off = encodedMData & 1; temp = (encodedMData & (3 << 1)) >> 1; if (temp == 1) mData.direction = REVERSE; else if (temp == 2) mData.direction = FORWARD; else mData.direction = NO_DIRECTION; mData.speed = encodedMData & (31 << 3); mData.errorStatus = encodedMData & (1 << 8); mData.noPowerError) = encodedMData & (1 << 9); mData.noTorqueError) = encodedMData & (1 << 10); mData.BITError) = encodedMData & (1 << 11); mData.overTemperatureError) = encodedMData & (1 << 12); mData.reservedError1) = encodedMData & (1 << 13); mData.reservedError2) = encodedMData & (1 << 14); Data.unknownError) = encodedMData & (1 << 15); return mData; } MotorProxy * MotorProxy_Create(void) { MotorProxy* me = (MotorProxy *) malloc(sizeof(MotorProxy)); if(me!=NULL) { MotorProxy_Init(me); } return me; } void MotorProxy_Destroy(MotorProxy* const me) { if(me!=NULL) { MotorProxy_Cleanup(me); } free(me); }

代码清单 3-6：MotorProxy.c

3.3 Hardware Adapter Pattern

The Hardware Adapter Pattern provides a way of adapting an existing hardware interface into the expectations of the application. This pattern is a straightforward derivative of the Adapter Pattern.

3.3.1 Abstract

The Hardware Adapter Pattern is useful when the application requires or uses one interface, but the actual hardware provides another. The pattern creates an element that converts between the two interfaces.

3.3.2 Problem

While hardware that performs similar functions tend to have similar interfaces, often the information they need and the set of services differ. Rather than rewrite the clients of the hardware device to use the provided interface, an adapter is created that provides the expected interface to the clients while converting the requests to and from the actual hardware interface.

The Hardware Adapter Pattern is useful when you have hardware that meets the semantic need of the system but that has an incompatible interface. The goal of the pattern is to minimize the reworking of code when one hardware design or implementation is replaced with another.

3.3.3 Pattern Structure

The pattern structure is shown in Figure 3-3. The pattern extends the Hardware Proxy Pattern by adding a Hardware Adapter and explicitly showing the interface the client expects the hardware to support.

3.3.4 Collaboration Roles

This section describes the roles for this pattern.

3.3.4.1 Adapter Client

The adapter client expects to be invoking the services of a software element that represents the hardware, as specified by the Hardware Interface to Client interface. It is an element in the application system that expects to control, monitor, or use the hardware device.

3.3.4.2 Hardware Adapter

The Hardware Adapter performs “impedance matching” between the client and the Hardware Proxy. That is, service requests made by the Adapter Client are converted into a sequence of services that are available from the Hardware Proxy. This potentially includes both factoring of the service invocations and reformatting and restructuring the data.

3.3 硬件适配器模式

硬件适配器模式提供了一种将现有硬件接口适配到应用程序期望的方式。该模式是适配器模式的直接衍生。

3.3.1 摘要

当应用需要或使用一种接口，而实际硬件提供另一种接口时，硬件适配器模式非常有用。该模式创建一个元素，在两个接口之间进行转换。

3.3.2 问题

虽然执行类似功能的硬件往往具有相似的接口，但它们所需的信息和服务集常常不同。与其重写硬件设备的客户端以使用所提供的接口，不如创建一个适配器，为客户端提供预期的接口，同时在请求与实际硬件接口之间进行转换。

当硬件满足系统的语义需求但接口不兼容时，硬件适配器模式非常有用。该模式的目标是最大限度地减少在一种硬件设计或实现被另一种替换时的代码重写工作。

3.3.3 模式结构

该模式的结构如图3-3所示。该模式通过添加硬件适配器并明确展示客户端期望硬件支持的接口，扩展了硬件代理模式。

3.3.4 协作角色

本节描述该模式的角色。

3.3.4.1 适配器客户端

适配器客户端期望调用代表硬件的软件元素的服务，如面向客户端的硬件接口所规定。它是应用系统中的一个元素，期望控制、监控或使用硬件设备。

3.3.4.2 硬件适配器

硬件适配器在客户端与硬件代理之间执行“阻抗匹配”。也就是说，适配器客户端发出的服务请求被转换为硬件代理所提供的一系列服务。这可能包括对服务调用进行分解，以及对数据进行重新格式化和重构。

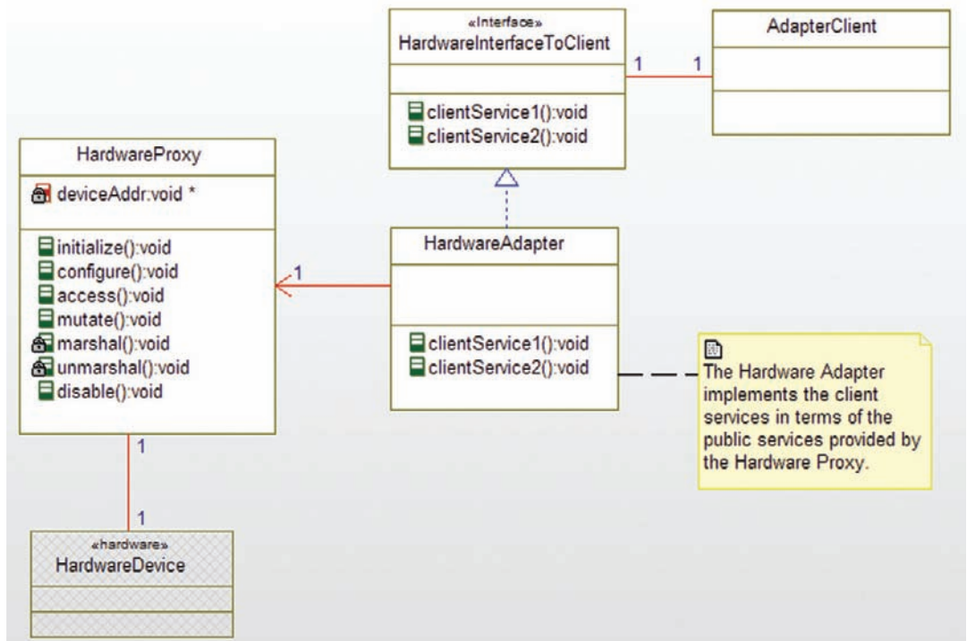


Figure 3-3 Hardware Adapter Pattern

3.3.4.3 Hardware Interface to Client

This interface represents the set of services and parameter lists that the client expects the Hardware Proxy to provide. As an interface, it is a collection of service specifications and has no implementation. The implementation in this case is provided by the Hardware Adapter Class.

3.3.4.4 Hardware Device

This element is described in the Hardware Proxy Pattern. See Section 3.2.4.1.

3.3.4.5 Hardware Proxy

This element is described in the Hardware Proxy Pattern. See Section 3.2.4.2.

3.3.5 Consequences

The use of this pattern allows various Hardware Proxies and their related hardware devices to be used as-is in different applications, while at the same time allowing existing applications to use different hardware devices without change. The key is that the adapter provides the connective glue to match the Hardware Proxy to the application. This means that it will be

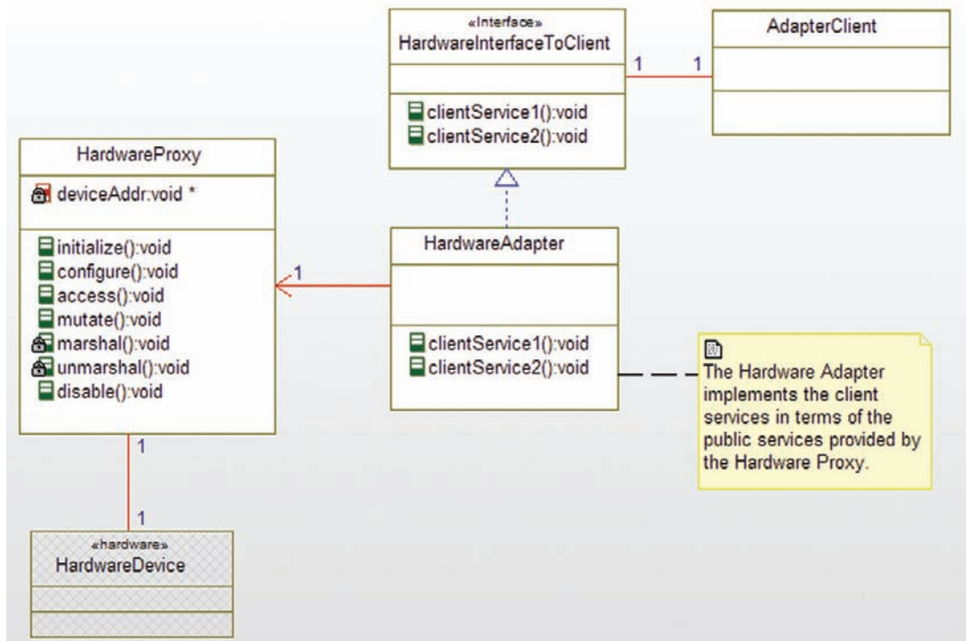


图 3-3 硬件适配器模式

3.3.4.3 面向客户端的硬件接口

该接口表示客户端期望硬件代理提供的一组服务和参数列表。作为一个接口，它是服务规范的集合，没有实现。此处的实现由硬件适配器类提供。

3.3.4.4 硬件设备

此 元素 在 硬件 代理 模式 中 描述 。 参见 第 3.2.4.1节。

3.3.4.5 硬件代理

This 元素 在 硬件 代理模式 中 进行 了 描述。 参见 第 3.2.4.2 节 .

3.3.5 结果

使用此模式可以让各种硬件代理及其相关硬件设备在不同应用中按原样使用，同时允许现有应用无需更改即可使用不同的硬件设备。关键在于适配器提供了将硬件代理与应用匹配的连接粘合。这意味着它将

easier, less error-prone, and faster to change hardware devices for an application or to reuse an existing hardware device in a new application.

The cost of using the pattern is that it adds a level of indirection and therefore decreases run-time performance slightly.

3.3.6 Implementation Strategies

The classic *Design Patterns*[9] book by Gamma, et. al., identifies two forms for this pattern. The *object adapter* form is as shown in Figure 3-3. In this form, the adapter subclasses along one interface and delegates to the other element (although we use realization in this case rather than inheritance). In the *class adapter* form, the Hardware Adapter subclasses from both interfaces and reimplements the client services in terms of the proxy services that it inherits. This requires a more elaborate implementation of classes than we've been using so far in this book since it requires the creation of a virtual function table to implement true polymorphism (see Section 1.2.3). The tradeoffs for the two different implementations are subtly different. The class adapter form is a bit more reusable but requires a more complex implementation in C.

3.3.7 Related Patterns

The Hardware Adapter extends the Hardware Proxy Pattern. The latter pattern encapsulates hardware interface detail but does not translate service requests into radically different ones. The Hardware Adapter Pattern adds a level of indirection between the client and the Hardware Proxy. This allows the clients to be unchanged in the application while at the same time permitting reuse of existing Hardware Proxies that may also have been created for other systems. The implementation of the Hardware Proxy and the Hardware Adapter can be merged, but that undermines the reusability of the Hardware Proxy.

3.3.8 Example

The example is shown graphically in Figure 3-4. This example shows a system that tracks both oxygen concentration and oxygen flow rates, such as might be found in a medical ventilator. Two clients are shown. The `Gas Display` client displays these data for the attending medical staff. The `Gas Mixer` client uses these data for the closed loop control of gas delivery. Both are implemented to use the interface as shown in the `iO2Sensor` interface. The two services the clients expect to see are `gimmeO2Conc()`, which returns an integer in the range of 0–100, and `gimmeO2Flow()`, which returns an integer type with units of cc/min.

[9] Gamma, E., Helm, R., Johnson, R., Vlissides, J., 1995. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.

更容易、更不易出错且更快速地更换应用的硬件设备，或在新的应用中复用现有的硬件设备。

使用该模式的成本在于它增加了一个间接层级，因此会略微降低运行时性能。

3.3.6 实现策略

Gamma等人所著的经典《设计模式》一书为该模式确定了两种形式。对象适配器形式如图3-3所示。在这种形式中，适配器沿一个接口进行子类化，并委托给另一个元素（尽管在此情况下我们使用的是实现而非继承）。在类适配器形式中，硬件适配器从两个接口进行子类化，并根据其继承的代理服务重新实现客户端服务。这需要比本书迄今为止所使用的更为精细的类实现，因为它需要创建虚函数表来实现真正的多态（参见第1.2.3节）。两种不同实现的权衡略有不同。类适配器形式更具可复用性，但在C语言中需要更复杂的实现。

3.3.7 相关模式

硬件适配器扩展了硬件代理模式。后者封装了硬件接口细节，但不会将服务请求转换为截然不同的请求。硬件适配器模式在客户端与硬件代理之间增加了一个间接层级。这使得应用程序中的客户端可以保持不变，同时允许复用可能为其他系统创建的现有硬件代理。硬件代理与硬件适配器的实现可以合并，但这会削弱硬件代理的可复用性。

3.3.8 示例

该示例以图形方式展示在图 3-4中。此示例展示了一个同时跟踪氧气浓度和氧气流速的系统，例如医用呼吸机中可能出现的系统。图中展示了两个客户端。`Gas Display` 客户端为值班医护人员显示这些数据。`Gas Mixer` 客户端使用这些数据对气体输送进行闭环控制。两者均按照`iO2Sensor` 接口中所示的接口进行实现。客户端期望看到的两个服务是`gimmeO2Conc()`（返回0–100范围内的整数）和`gimmeO2Flow()`，（返回以cc/min为单位的整数类型）。

[9] Gamma, E., Helm, R., Johnson, R., Vlissides, J., 1995. 设计模式：可复用面向对象软件基础. Addison-Wesley.

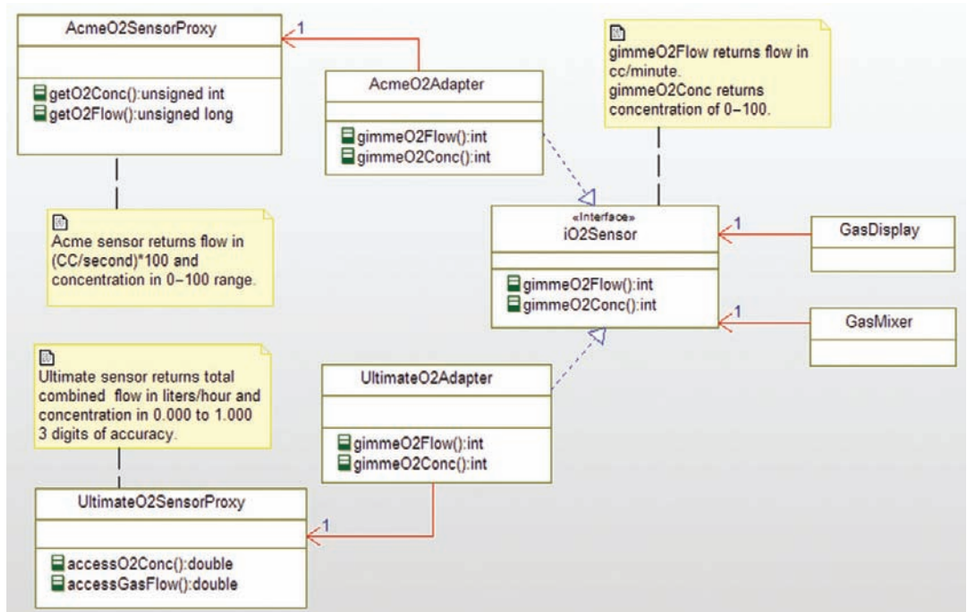


Figure 3-4 Hardware Adapter example

The system can be delivered with either of two physical sensors. These sensors do much the same job but they provide different programmatic interfaces. The `Acme O2 Sensor Proxy` provides two services. `getO2Conc()` returns the oxygen concentration in a range of 0–100 as an unsigned integer. `getO2Flow()` returns the flow oxygen flow as a scaled integer of 100* cc/sec. The `Ultimate O2 sensor` has two similar functions. `accessO2Conc()` returns concentration as a double in the range of 0.000 to 1.000 (3 digits of accuracy) and the `accessGasFlow()` returns the total gas flow (of which oxygen is just a part) as a double in liters/hour.

Each Hardware Proxy requires its own adapter to convert to the expected client interface.

The previous pattern showed code for the proxy classes, so this section will only show code for the adapters. The header files aren't very interesting, so we will concentrate solely on the implementation files of the two adapters. The implementation of the Acme O2 Adapter is shown in Code Listing 3-7 and the implementation for the Ultimate O2 Adapter is given in Code Listing 3-8.

```
int AcmeO2Adapter_gimmeO2Conc(AcmeO2Adapter* const me) {
       return me->itsAcmeO2SensorProxy->getO2Conc();
}
```

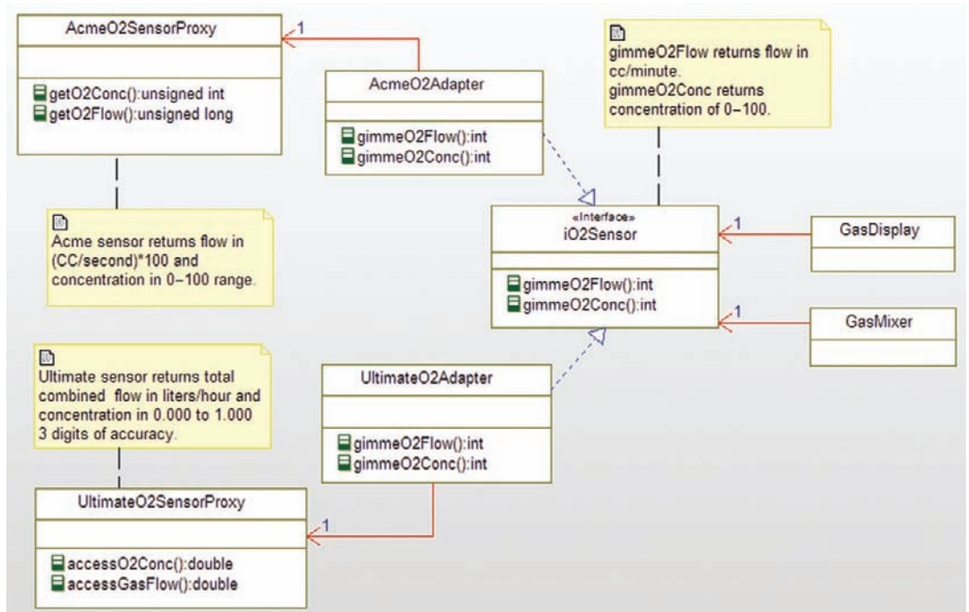



图 3-4 硬件适配器示例

该系统可配备两种物理传感器中的任意一种。这些传感器功能大致相同，但提供不同的编程接口。`Acme O2 Sensor Proxy`提供两个服务。`getO2Conc()` 以无符号整数形式返回0–100范围内的氧气浓度。`getO2Flow()` 以100* cc/sec的缩放整数形式返回氧气流量。`Ultimate O2 sensor` 有两个类似的函数。`accessO2Conc()` 以0.000到1.000范围内的double类型（3位精度）返回浓度，而`accessGasFlow()` 以升/小时为单位的double类型返回总气体流量（氧气只是其中一部分）。

每个硬件代理都需要自己的适配器来转换为预期的客户端接口。

上一个模式展示了代理类的代码，因此本节将只展示适配器的代码。头文件并不太有趣，所以我们将只专注于两个适配器的实现文件。Acme O2 适配器的实现如代码清单 3-7 所示，Ultimate O2 适配器的实现如代码清单 3-8 所示。

```
int AcmeO2Adapter_gimmeO2Conc(AcmeO2Adapter* const me) {
return me->itsAcmeO2SensorProxy->getO2Conc(); }
```

```
int AcmeO2Adapter_gimmeO2Flow(AcmeO2Adapter* const me) {
        return (me->itsAcmeO2SensorProxy->getO2Flow()*60)/100;
```

Code Listing 3-7: AcmeO2Adapter.c

```
int UltimateO2Adapter_gimmeO2Conc(UltimateO2Adapter* const me) {
        return int(me->getItsUltimateO2SensorProxy
                ->accessO2Conc()*100);
}
int UltimateO2Adapter_gimmeO2Flow(UltimateO2Adapter* const me) {
        double totalFlow;
        // convert from liters/hr to cc/min
        totalFlow = me->itsUltimateO2SensorProxy->accessGasFlow() *
            1000.0/60.0;
        // now return the portion of the flow due to oxygen
        // and return it as an integer
        return (int)(totalFlow *
                me->itsUltimateO2SensorProxy->accessO2Conc());
}
```

Code Listing 3-8: UltimateO2Adapter.c

3.4 Mediator Pattern

The Mediator Pattern provides a means of coordinating a complex interaction among a set of elements.

3.4.1 Abstract

The Mediator Pattern is particularly useful for managing different hardware elements when their behavior must be coordinated in well-defined but complex ways. It is particularly useful for C applications because it doesn't require a lot of specialization (subclassing), which can introduce its own complexities into the implementation.

3.4.2 Problem

Many embedded applications control sets of actuators that must work in concert to achieve the desired effect. For example, to achieve a coordinated movement of a multi-joint robot arm, all the motors must work together to provide the desired arm movement. Similarly, using reaction wheels or thrusters in a spacecraft in three dimensions requires many different such devices acting at precisely the right time and with the right amount of force to achieve attitude stabilization.

int AcmeO2Adapter_gimmeO2Flow(AcmeO2Adapter* const me) { return (me->itsAcmeO2SensorProxy->getO2Flow()*60)/100;

代码清单 3-7：AcmeO2Adapter.c

int UltimateO2Adapter_gimmeO2Conc(UltimateO2Adapter* const me) { return int(me->getItsUltimateO2SensorProxy ->accessO2Conc()*100); } int UltimateO2Adapter_gimmeO2Flow(UltimateO2Adapter* const me) { double totalFlow; // 将升/小时转换为立方厘米/分钟 totalFlow = me-> itsUltimateO2SensorProxy->accessGasFlow() * 1000.0/60.0; // 现在返回氧气部分对应的流量 // 并以整数形式返回 return (int)(totalFlow * me-> itsUltimateO2SensorProxy->accessO2Conc()); }

代码清单 3-8：UltimateO2Adapter.c

3.4 中介者模式

中介者模式提供了一种协调一组元素之间复杂交互的手段。

3.4.1 摘要

中介者模式在管理不同硬件元素时特别有用，尤其是当这些元素的行为必须以定义明确但复杂的方式进行协调时。它对C应用程序特别有用，因为它不需要大量的特化（子类化），而特化可能会给实现带来自身的复杂性。

3.4.2 问题

许多嵌入式应用控制一组执行器，这些执行器必须协同工作以实现预期效果。例如，要实现多关节机械臂的协调运动，所有电机必须协同工作以提供所需的机械臂运动。类似地，在航天器中使用反作用轮或推进器进行三维控制时，需要许多不同的此类设备在精确的正确时间以正确的力大小动作，以实现姿态稳定。

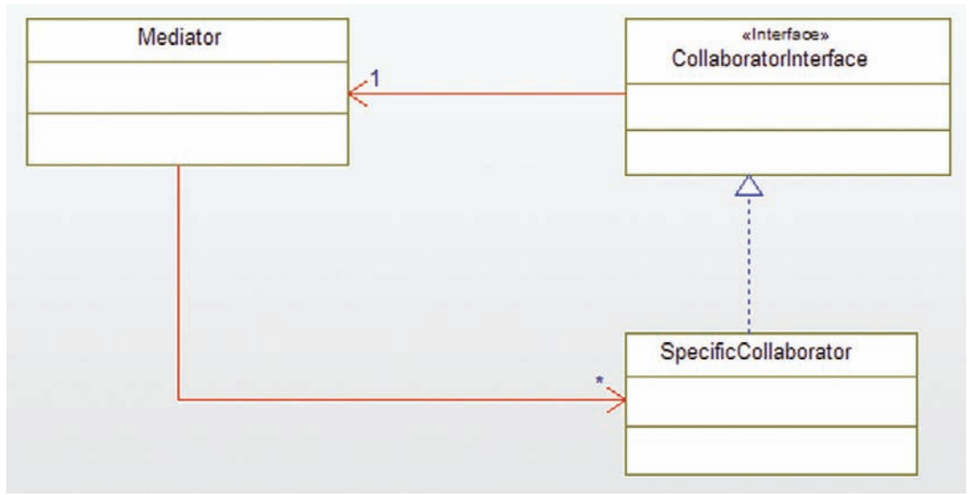


Figure 3-5 Mediator Pattern

3.4.3 Pattern Structure

The Mediator Pattern uses a mediator class to coordinate the actions of a set of collaborating devices to achieve the desired overall effect. The Mediator class coordinates the control of the set of multiple Specific Collaborators (their number is indicated by the '*' multiplicity on the association between the Mediator and the Specific Collaborator in Figure 3-5). Each Specific Collaborator must be able to contact the Mediator when an event of interest occurs.

3.4.4 Collaboration Roles

This section describes the roles for this pattern.

3.4.4.1 Collaborator Interface

The Collaborator Interface is a specification of a set of services common to all Specific Collaborators that may be invoked by the Mediator. For example, it is common to have reset(), shutdown, initialize() style operations for all the hardware devices to facilitate bringing them all to a known initial, recovery and/or shutdown state. Each of the common services must be implemented in each Specific Collaborator (although, of course, the implementation will usually be unique to each collaborator). If there are no services common to all Specific Collaborators, then this interface may be omitted.

3.4.4.2 Mediator

The Mediator class coordinates the Specific Collaborators in the pattern. It has a link to each Specific Collaborator so that it can send messages to it. In addition, each Specific Collaborator must be able to send the Mediator messages when events of interest occur. The Mediator

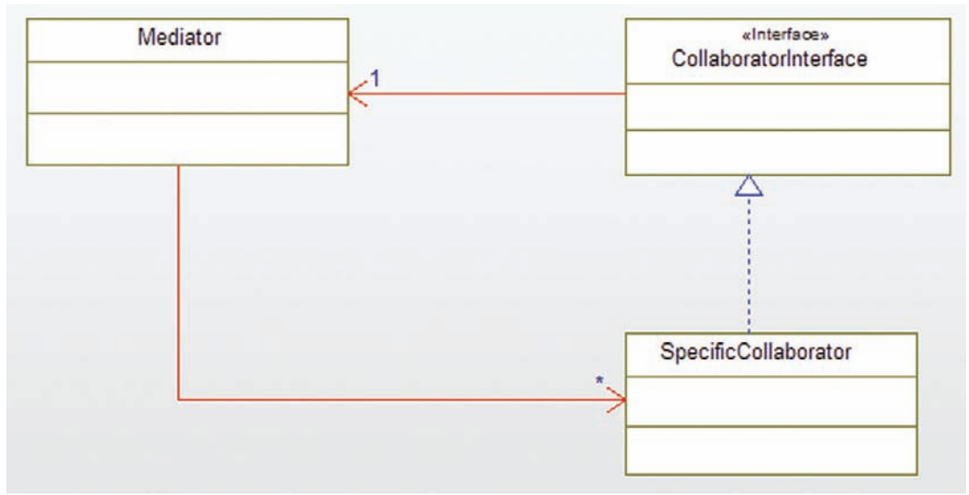


图 3-5 中介者模式

3.4.3 模式结构

中介者模式使用一个中介者类来协调一组协作者设备的动作，以实现预期的整体效果。中介者类协调对多个具体协作者的控制（其数量由图3-5中中介者与具体协作者之间关联上的"*"多重性表示）。每个具体协作者必须能够在发生感兴趣的事件时联系中介者。

3.4.4 协作角色

本节描述该模式的角色。

3.4.4.1 协作者接口

协作者接口是对一组服务的规范说明，这些服务是所有特定协作者共有的，可由中介者调用。例如，通常为所有硬件设备提供 reset()、shutdown、initialize() 风格的操作，以便将它们全部带入已知的初始、恢复和/或关闭状态。每个公共服务都必须在每个特定协作者中实现（当然，实现通常对每个协作者而言是独特的）。如果不存在所有特定协作者共有的服务，则可以省略此接口。

3.4.4.2 中介者

中介者类协调模式中的具体协作者。它持有指向每个具体协作者的链接，以便向其发送消息。此外，每个具体协作者必须能够在感兴趣的事件发生时向中介者发送消息。中介者

provides the coordination logic that would otherwise require Specific Collaborators to coordinate among themselves.

3.4.4.3 Specific Collaborator

The Specific Collaborator represents one device and so may be a device driver or Hardware Proxy. It receives command messages from the Mediator and also sends messages to the Mediator when events of interest occur.

3.4.5 Consequences

This pattern creates a mediator that coordinates the set of collaborating actuators but without requiring direct coupling of those devices. This greatly simplifies the overall design by minimizing the points of coupling and encapsulating the coordination within a single element. Whenever the Collaborator would have directly contacted another Collaborator, instead it notifies the Mediator who can decide how to respond as a collective collaborative whole.

Since many embedded systems must react with high time fidelity, delays between the actions may result in unstable or undesirable effects. It is important that the mediator class can react within those time constraints. This is of a particular concern when there is two-way collaboration between the actuators and the mediator.

3.4.6 Implementation Strategies

The Mediator must be able to link to each Specific Collaborator. This can be done with an array of pointers (the obvious implementation strategy), a separate pointer for each Specific Collaborator, or a combination of the two. A common rule of thumb is that if a set of Specific Collaborators are indistinguishable in terms of their use other than their position in the list (e.g., they provide a common interface), then an array of pointers is best. If the Specific Collaborators serve different purposes, then use a separate link for each such Specific Collaborator. The example in Section 3.4.8 shows the use of both an array and of specific pointers.

It is even possible to have grouped sets of different Specific Collaborators in which each class within a group provides a common interface.

3.4.7 Related Patterns

This pattern is similar to the Strategy Pattern from the classic Design Patterns book[10] and the architectural Rendezvous Pattern from my Real-Time Design Patterns book[11]. In this case we are focusing on its use for detailed hardware coordination.

[10] Gamma, E., Helm, R., Johnson, R., Vlissides, J., 1995. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
[11] Douglass, B.P., 2003. *Real-Time Design Patterns: Robust Scalable Architecture for Real-Time Systems*. Addison-Wesley.

提供协调逻辑，否则这些逻辑将需要具体协作者之间相互协调。

3.4.4.3 特定协作者

特定协作者代表一个设备，因此可以是设备驱动程序或硬件代理。它从中介者接收命令消息，并在发生感兴趣的事件时向中介者发送消息。

3.4.5 结果

该模式创建了一个中介者来协调一组协作执行器，但不需要这些设备直接耦合。这通过最小化耦合点并将协调封装在单个元素中，极大地简化了整体设计。每当协作者原本会直接联系另一个协作者时，它改为通知中介者，中介者可以决定如何作为集体协作整体来响应。

由于许多嵌入式系统必须以高时间保真度做出反应，动作之间的延迟可能导致不稳定或不良影响。中介者类能够在这些时间约束内做出反应非常重要。当执行器与中介者之间存在双向协作时，这一点尤其值得关注。

3.4.6 实现策略

中介者必须能够链接到每个具体协作者。这可以通过指针数组（显而易见的实现策略）、为每个具体协作者使用单独的指针，或两者的组合来实现。一条常见的经验法则是：如果一组具体协作者除了在列表中的位置之外在使用上无法区分（例如，它们提供公共接口），那么最好使用指针数组。如果具体协作者服务于不同的目的，则为每个这样的具体协作者使用单独的链接。第 3.4.8 节中的示例展示了数组和特定指针两者的使用。

甚至可以将不同的具体协作者分组，其中组内的每个类都提供公共接口。

3.4.7 相关模式

此模式类似于经典《设计模式》一书中的策略模式[10] 以及我的《实时设计模式》一书中的架构会合模式[11]。在本例中，我们关注的是其在详细硬件协调方面的应用。

[10] Gamma, E., Helm, R., Johnson, R., Vlissides, J., 1995. 设计模式：可复用面向对象软件基础. Addison-Wesley. [11] Douglass, B.P., 2003. Real-Time Design Patterns: Robust Scalable Architecture for Real-Time Systems. Addison-Wesley.

The Observer Pattern may be used instead of a direct link to the Mediator when there are multiple clients of the Specific Collaborator or when you wish to hide the Mediator from the Specific Collaborator.

3.4.8 Example

In this example of a robot system (See Figure 3-6), the Robot Arm Manager receives requests to grasp objects at specific points in space (x, y, z) and time (t). It first computes an arm trajectory via the `computeTrajectory()` function and produces a set of `nSteps` actions (up to 100). Each action is composed of a set of commands to each of the seven servos. If the goal is achievable, then Robot Arm Manager calls `executeStep()` function `nSteps` times to step through the computed action sequence. At each step, any command can return an error code (non-zero) which causes the `graspAt()` function to abort with an error code. Note that the computation of the arm path via the sum of the movements of the various arm joints is a complex exercise in geometry which, while interesting, is outside the scope of concern for this book. In addition, many paths

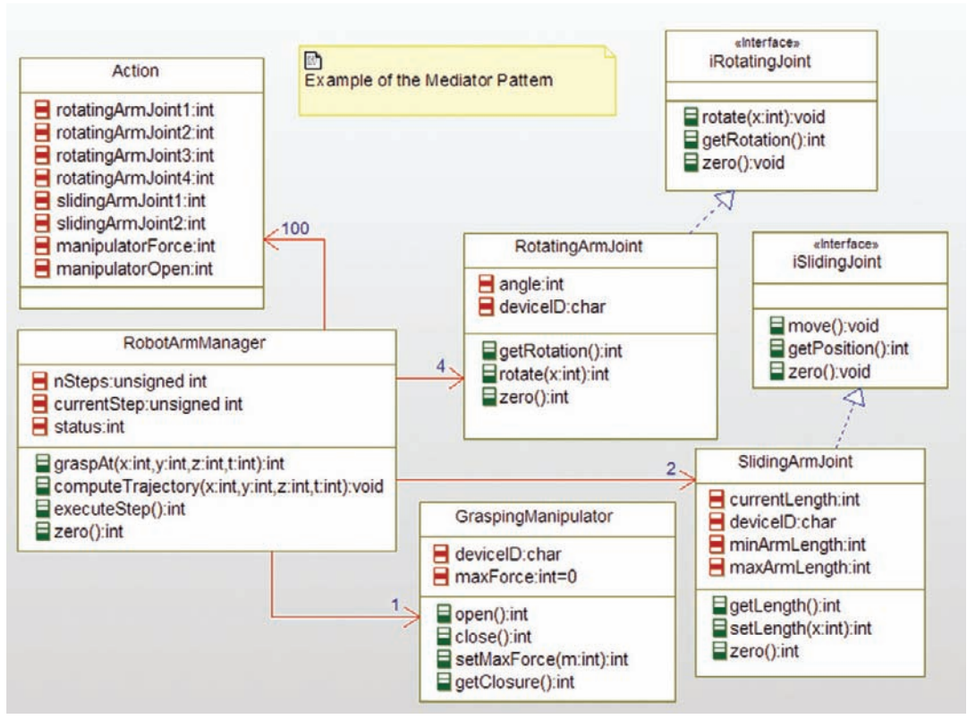


Figure 3-6 Mediator Example

当特定协作者存在多个客户端，或者你希望向特定协作者隐藏中介者时，可以使用观察者模式来代替与中介者的直接链接。

3.4.8 示例

在 这个 示例 中，一个 机器人 系统 （参见 图 3-6），机械臂管理器接收在空间中特定点（x、y、z）和特定时间（t）抓取对象的请求。它首先通过`computeTrajectory()`函数计算出一条机械臂轨迹，并生成一组 `nSteps` 动作（最多 100 个）。每个动作由发送给七个伺服电机的一组命令组成。如果目标可以实现，那么机械臂管理器会调用`executeStep()`函数 `nSteps` 次，以逐步执行计算出的动作序列。在每一步中，任何命令都可能返回一个错误码（非零），这会导致 `graspAt()` 函数以错误码中止。请注意，通过各机械臂关节运动之和来计算机械臂路径是一项复杂的几何学练习，虽然有趣，但超出了本书的关注范围。此外，许多路径

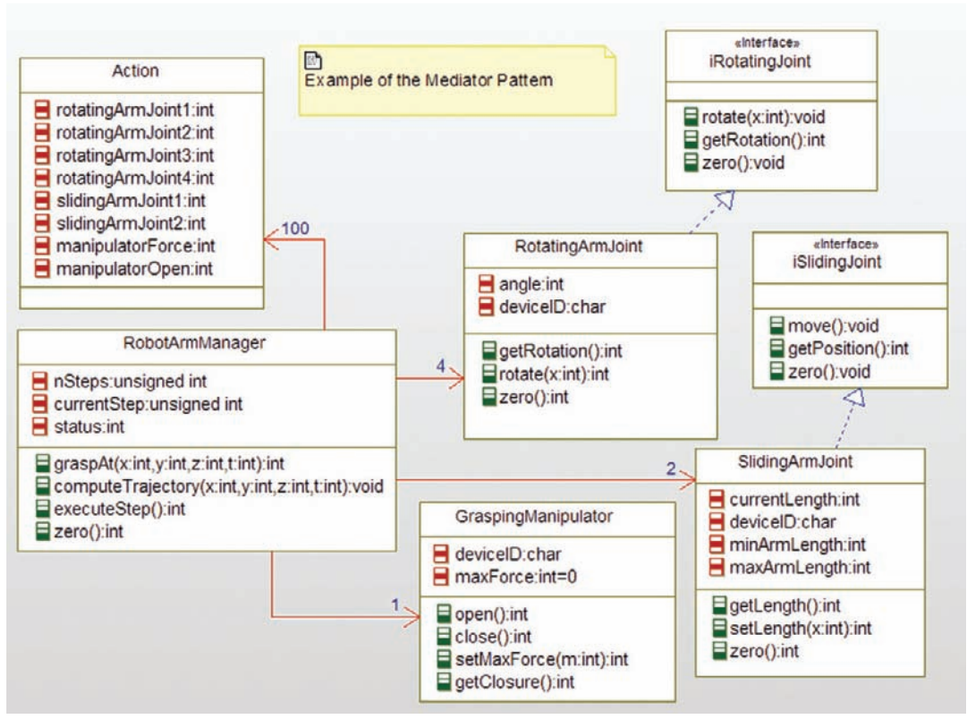


图 3-6 中介者示例

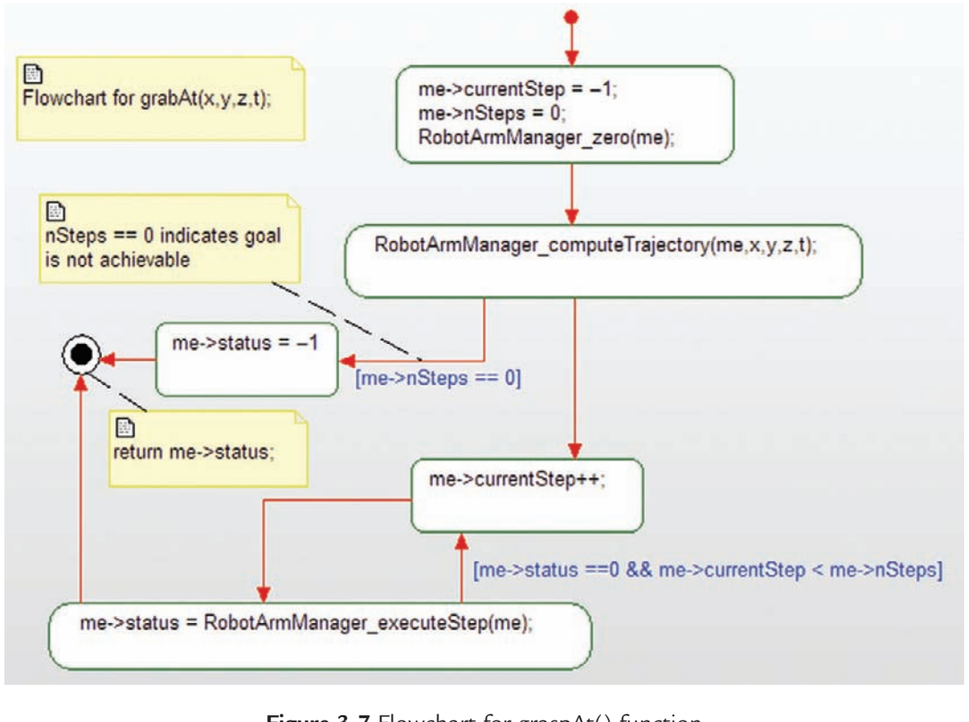


Figure 3-7 Flowchart for graspAt() function

may be disallowed for safety reasons (Rule 1: You Can't Hit the Operator)[12], because they may require a path through a physical object, or may not even be physically possible.

The point of the example is to illustrate the value of the Mediator (RobotArmManager). Without its coordinating influences, all of the actuators would have to collaborate directly with their peers. The computation of allowable versus illegal movement paths of the arm would be divided up among the collaborators and would be very difficult to manage.

Figure 3-7 shows a flowchart for the graspAt() function. This is the primary function that clients of the RobotArmManager will invoke when they want the arm to move to a position and grasp (or let go of) something.

The code for the RobotArmManager is given in Code Listing 3-9 and Code Listing 3-10. You can see that the RobotArmManager coordinates the various servos to achieve the desired arm actions.

[12] An entirely different Rule #1 than in *Zombieland*. See www.zombieland.com (Sony Pictures, 2009), possibly the most awesome zombie movie *ever*.

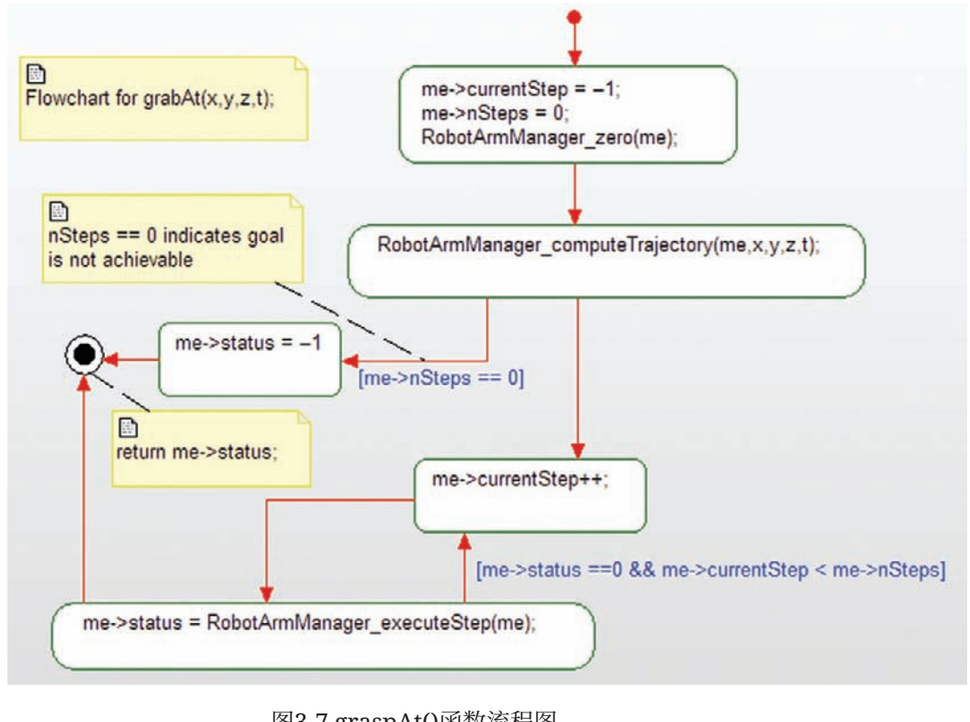


图3-7 graspAt()函数流程图

可能出于安全性原因而被禁止（规则1：你不能击中操作员）[12]，，因为它们可能需要穿过物理对象的路径，甚至可能在物理上无法实现。

本示例的重点在于说明中介者（机械臂管理器）的价值。如果没有它的协调作用，所有执行器都将不得不直接与其同级协作。机械臂允许与非法运动路径的计算将在协作者之间分配，并且将非常难以管理。

图3-7展示了graspAt()函数的流程图。这是机械臂管理器的客户端在希望机械臂移动到某个位置并抓取（或松开）某物时将调用的主要函数。

RobotArmManager 的代码见代码清单 3-9 和代码清单 3-10。可以看到，RobotArmManager 协调各个伺服电机以实现所需的机械臂动作。

[12] 与Zombieland中完全不同的第一条规则。参见www.zombieland.com（索尼影视，2009年），这可能是有史以来最棒的僵尸电影。

```
#include "GraspingManipulator.h"
#include "RotatingArmJoint.h"
#include "SlidingArmJoint.h"
#include "Action.h"

/*## class RobotArmManager */
typedef struct RobotArmManager RobotArmManager;
struct RobotArmManager {
        unsigned int currentStep;
        unsigned int nSteps;
        struct GraspingManipulator* itsGraspingManipulator;
        struct RotatingArmJoint *itsRotatingArmJoint[4];
        struct SlidingArmJoint *itsSlidingArmJoint[2];
        struct Action *itsAction[100];
        int status;
};
/* Constructors and destructors:*/
void RobotArmManager_Init(RobotArmManager* const me);
void RobotArmManager_Cleanup(RobotArmManager* const me);
/* Operations */
void RobotArmManager_computeTrajectory(RobotArmManager* const me, int x, int y, int z, int t);
int RobotArmManager_executeStep(RobotArmManager* const me);
int RobotArmManager_graspAt(RobotArmManager* const me, int x, int y, int z, int t);
int RobotArmManager_zero(RobotArmManager* const me);
struct GraspingManipulator*
RobotArmManager_getItsGraspingManipulator(const
RobotArmManager* const me);
void RobotArmManager_setItsGraspingManipulator(RobotArmManager* const me, struct
GraspingManipulator* p_GraspingManipulator);
int RobotArmManager_getItsRotatingArmJoint(const RobotArmManager* const me);
void RobotArmManager_addItsRotatingArmJoint(RobotArmManager* const me, struct
RotatingArmJoint * p_RotatingArmJoint);
void RobotArmManager_removeItsRotatingArmJoint(RobotArmManager* const me, struct
RotatingArmJoint * p_RotatingArmJoint);
void RobotArmManager_clearItsRotatingArmJoint(RobotArmManager* const me);
int RobotArmManager_getItsSlidingArmJoint(const RobotArmManager* const me);
void RobotArmManager_addItsSlidingArmJoint(RobotArmManager* const me, struct
SlidingArmJoint * p_SlidingArmJoint);
void RobotArmManager_removeItsSlidingArmJoint(RobotArmManager* const me, struct
SlidingArmJoint * p_SlidingArmJoint);
void RobotArmManager_clearItsSlidingArmJoint(RobotArmManager* const me);
RobotArmManager * RobotArmManager_Create(void);
void RobotArmManager_Destroy(RobotArmManager* const me);
int RobotArmManager_getItsAction(const RobotArmManager* const me);
void RobotArmManager_addItsAction(RobotArmManager* const me, struct Action * p_Action);
void RobotArmManager_removeItsAction(RobotArmManager* const me, struct Action * p_Action);
void RobotArmManager_clearItsAction(RobotArmManager* const me);
#endif
```

Code Listing 3-9: RobotArmManager.h

#include "GraspingManipulator.h" #include "RotatingArmJoint.h" #include "SlidingArmJoint.h" #i nclude "Action.h" /*## class RobotArmManager */ typedef struct RobotArmManager RobotArmManager; struct RobotArmManager { unsigned int currentStep; unsigned int nSteps; struct GraspingManipulator* itsGraspingManipulator; struct RotatingArmJoint *itsRotatingArmJoint[4]; struct SlidingArmJoint *itsSlidingArmJoint[2]; struct Action *itsAction[100]; int status; }; /* 构造函数与析构函数： */ void RobotArmManager_Init(RobotArmManager* const me); void RobotArmManager_Cleanup(RobotArmManager* const me); /* 操作 */ void RobotArmManager_computeTrajectory(RobotArmManager* const me, int x, int y, int z, int t); int RobotArmManager_executeStep(RobotArmManager* const me); int RobotArmManager_graspAt(RobotArmManager* const me, int x, int y, int z, int t); int RobotArmManager_zero(RobotArmManager* const me); struct GraspingManipulator* RobotArmManager_getItsGraspingManipulator(const RobotArmManager* const me); void RobotArmManager_setItsGraspingManipulator(RobotArmManager* const me, struct GraspingManipulator* p_GraspingManipulator); int RobotArmManager_getItsRotatingArmJoint(const RobotArmManager* const me); void RobotArmManager_addItsRotatingArmJoint(RobotArmManager* const me, struct RotatingArmJoint * p_RotatingArmJoint); void RobotArmManager_removeItsRotatingArmJoint(RobotArmManager* const me, struct RotatingArmJoint * p_RotatingArmJoint); void RobotArmManager_clearItsRotatingArmJoint(RobotArmManager* const me); int RobotArmManager_getItsSlidingArmJoint(const RobotArmManager* const me); void RobotArmManager_addItsSlidingArmJoint(RobotArmManager* const me, struct SlidingArmJoint * p_SlidingArmJoint); void RobotArmManager_removeItsSlidingArmJoint(RobotArmManager* const me, struct SlidingArmJoint * p_SlidingArmJoint); void RobotArmManager_clearItsSlidingArmJoint(RobotArmManager* const me); RobotArmManager * RobotArmManager_Create(void); void RobotArmManager_Destroy(RobotArmManager* const me); int RobotArmManager_getItsAction(const RobotArmManager* const me); void RobotArmManager_addItsAction(RobotArmManager* const me, struct Action * p_Action); void RobotArmManager_removeItsAction(RobotArmManager* const me, struct Action * p_Action); void RobotArmManager_clearItsAction(RobotArmManager* const me); #endif

代码清单 3-9：RobotArmManager.h

```
#include "RobotArmManager.h"
static void cleanUpRelations(RobotArmManager* const me);
void RobotArmManager_Init(RobotArmManager* const me) {
        int pos;
        for(pos = 0; pos < 100; ++pos) {
            me->itsAction[pos] = NULL;
        }

        me->itsGraspingManipulator = NULL;
        for(pos = 0; pos < 4; ++pos)   {
            me->itsRotatingArmJoint[pos] = NULL;
        }

        for(pos = 0; pos < 2; ++pos) {
            me->itsSlidingArmJoint[pos] = NULL;
              }
}

void RobotArmManager_Cleanup(RobotArmManager* const me) {
        cleanUpRelations(me);
}

/* operation computeTrajectory(x,y,z,t)
    This function computes a path for the robot arm to follow to position the manipulator at
    the desired end point. It produces a set of up to 100 actions, each action is a set of
    commands to the various servos to which the RobotArmManager connects. In actual practice
    this is a complex job. The implementation here is just a placeholder.
*/
void RobotArmManager_computeTrajectory(RobotArmManager* const me, int x, int y, int z,
int t) {
        Action* ap;
        int j;

        me->nSteps = 0;
        RobotArmManager_clearItsAction(me);
        /* move the arm to the position with manipulator open*/
        ap = Action_Create();
        RobotArmManager_addItsAction(me, ap);
        ap->rotatingArmJoint1=1;
        ap->rotatingArmJoint2=2;
        ap->rotatingArmJoint3=3;
        ap->rotatingArmJoint4=4;
        ap->slidingArmJoint1=10;
        ap->slidingArmJoint2=20;
        ap->manipulatorForce=0;
        ap->manipulatorOpen=1;

        /* grab the object */
        ap = Action_Create();
        RobotArmManager_addItsAction(me, ap);
        ap->rotatingArmJoint1=1;
        ap->rotatingArmJoint2=2;
        ap->rotatingArmJoint3=3;
```


#include "RobotArmManager.h" static void cleanUpRelations(RobotArmManager* const me); void RobotArmManager_Init(RobotArmManager* const me) { int pos; for(pos = 0; pos < 100; ++pos) { me->itsAction[pos] = NULL; } me->itsGraspingManipulator = NULL; for(pos = 0; pos < 4; ++pos) { me->itsRotatingArmJoint[pos] = NULL; } for(pos = 0; pos < 2; ++pos) { me->itsSlidingArmJoint[pos] = NULL; } } void RobotArmManager_Cleanup(RobotArmManager* const me) { cleanUpRelations(me); } /* 操作 computeTrajectory(x,y,z,t) 此函数为机械臂计算一条路径，以将机械手定位到期望的终点。它生成最多100个动作的集合，每个动作是一组发送到RobotArmManager所连接的各个伺服电机的命令。在实际实践中，这是一项复杂的工作。此处的实现只是一个占位符。 */ void RobotArmManager_computeTrajectory(RobotArmManager* const me, int x, int y, int z, int t) { Action* ap; int j; me->nSteps = 0; RobotArmManager_clearItsAction(me); /* 将机械臂移动到机械手打开的位置 */ ap = Action_Create(); RobotArmManager_addItsAction(me, ap); ap->rotatingArmJoint1=1; ap->rotatingArmJoint2=2; ap->rotatingArmJoint3=3; ap->rotatingArmJoint4=4; ap->slidingArmJoint1=10; ap->slidingArmJoint2=20; ap->manipulatorForce=0; ap->manipulatorOpen=1; /* 抓取对象 */ ap = Action_Create(); RobotArmManager_addItsAction(me, ap); ap->rotatingArmJoint1=1; ap->rotatingArmJoint2=2; ap->rotatingArmJoint3=3;

```
        ap->rotatingArmJoint4=4;
        ap->slidingArmJoint1=10;
        ap->slidingArmJoint2=20;
        ap->manipulatorForce=10;
        ap->manipulatorOpen=0;

        /* return to zero position */
        ap = Action_Create();
        RobotArmManager_addItsAction(me, ap);
        ap->rotatingArmJoint1=0;
        ap->rotatingArmJoint2=0;
        ap->rotatingArmJoint3=0;
        ap->rotatingArmJoint4=0;
        ap->slidingArmJoint1=0;
        ap->slidingArmJoint2=0;
        ap->manipulatorForce=10;
        ap->manipulatorOpen=0;

        me->nSteps = 3;
}

/*    operation executeStep()
    This operation executes a single step in the chain of actions by
    executing all of the commands within the current action
*/
int RobotArmManager_executeStep(RobotArmManager* const me) {
        int actionValue = 0;
        int step = me->currentStep;
        int status = 0;

        if (me->itsAction[step]) {
            actionValue = me->itsAction[step]->rotatingArmJoint1;
            status = RotatingArmJoint_rotate(me->itsRotatingArmJoint[0],actionValue);
            if (status) return status;
             actionValue = me->itsAction[step]->rotatingArmJoint2;
             status = RotatingArmJoint_rotate(me->itsRotatingArmJoint[1],actionValue);
            if (status) return status;
             actionValue = me->itsAction[step]->rotatingArmJoint3;
             status = RotatingArmJoint_rotate(me->itsRotatingArmJoint[2],actionValue);
            if (status) return status;
             actionValue = me->itsAction[step]->rotatingArmJoint4;
             status = RotatingArmJoint_rotate(me->itsRotatingArmJoint[3],actionValue);
            if (status) return status;
             actionValue = me->itsAction[step]->slidingArmJoint1;
             status = SlidingArmJoint_setLength(me->itsSlidingArmJoint[0],actionValue);
            if (status) return status;
             actionValue = me->itsAction[step]->rotatingArmJoint2;
             status = SlidingArmJoint_setLength(me->itsSlidingArmJoint[0],actionValue);
            if (status) return status;
               actionValue = me->itsAction[step]->manipulatorForce;
               status  =  GraspingManipulator_setMaxForce(me->itsGraspingManipulator,
actionValue);
```


ap->rotatingArmJoint4=4; ap->slidingArmJoint1=10; ap->slidingArmJoint2=20; ap->manipulatorForce=10; ap->manipulatorOpen=0; /* 返回零位 */ ap = Action_Create(); RobotArmManager_addItsAction(me, ap); ap->rotatingArmJoint1=0; ap->rotatingArmJoint2=0; ap->rotatingArmJoint3=0; ap->rotatingArmJoint4=0; ap->slidingArmJoint1=0; ap->slidingArmJoint2=0; ap->manipulatorForce=10; ap->manipulatorOpen=0; me->nSteps = 3; } /* 操作 executeStep() 此操作通过执行当前动作内的所有命令，来执行动作链中的单个步骤 */ int RobotArmManager_executeStep(RobotArmManager* const me) { int actionValue = 0; int step = me->currentStep; int status = 0; if (me->itsAction[step]) { actionValue = me->itsAction[step]->rotatingArmJoint1; status = RotatingArmJoint_rotate(me->itsRotatingArmJoint[0], actionValue); if (status) return status; actionValue = me->itsAction[step]->rotatingArmJoint2; status = RotatingArmJoint_rotate(me->itsRotatingArmJoint[1],actionValue); if (status) return status; actionValue = me->itsAction[step]->rotatingArmJoint3; status = RotatingArmJoint_rotate(me->itsRotatingArmJoint[2],actionValue); if (status) return status; actionValue = me->itsAction[step]->rotatingArmJoint4; status = RotatingArmJoint_rotate(me->itsRotatingArmJoint[3],actionValue); if (status) return status; actionValue = me->itsAction[step]->slidingArmJoint1; status = SlidingArmJoint_setLength(me->itsSlidingArmJoint[0],actionValue); if (status) return status; actionValue = me->itsAction[step]->slidingArmJoint2; status = SlidingArmJoint_setLength(me->itsSlidingArmJoint[0], actionValue); if (status) return status; actionValue = me->itsAction[step]->manipulatorForce; status = GraspingManipulator_setMaxForce(me->itsGraspingManipulator, actionValue);

```
            if (status) return status;
            if (me->itsAction[step]->manipulatorOpen)
                  status = GraspingManipulator_open(me->itsGraspingManipulator);
            else
                status = GraspingManipulator_close(me->itsGraspingManipulator);
        };
        return status;
}
/*    operation graspAt(x,y,z,t) is the main function called by clients of the
    RobotArmManager. This operation:
        1. zeros the servos
        2. computes the trajectory with a call to computeTrajectory()
        3. executes each step in the constructed action list
*/
int RobotArmManager_graspAt(RobotArmManager* const me, int x, int y, int z, int t) {
        me->currentStep = -1;
        me->nSteps = 0;
        RobotArmManager_zero(me);
        RobotArmManager_computeTrajectory(me,x,y,z,t);

        if ( me->nSteps == 0 ) {
          me->status = -1;
        }
        else {
            do {
                me->currentStep++;
                me->status = RobotArmManager_executeStep(me);
            }
            while (me->status ==0 && me->currentStep < me->nSteps);
        }
        return me->status;
}
/*    operation zero()
    This operation returns all servos to a starting default position
*/
int RobotArmManager_zero(RobotArmManager* const me) {
        /* zero all devices */
        int j;
        for (j=0; j<4; j++){
                if (me->itsRotatingArmJoint[j] == NULL) return -1;
            if (RotatingArmJoint_zero(me->itsRotatingArmJoint[j])) return -1;
            }

        for (j=0; j<2; j++) {
                if (me->itsSlidingArmJoint[j] == NULL) return -1;
            if (SlidingArmJoint_zero(me->itsSlidingArmJoint[j])) return -1;
            }

        if (me->itsGraspingManipulator == NULL) return -1;
        if (GraspingManipulator_open(me->itsGraspingManipulator)) return -1;
        return 0;
}
```


if (status) return status; if (me->itsAction[step]->manipulatorOpen) status = GraspingManipulator_open(me->itsGraspingManipulator); else status = GraspingManipulator_close(me->itsGraspingManipulator); }; return status; } /* 操作 graspAt(x,y,z,t) 是客户端调用 RobotArmManager 的主函数。此操作：1. 将伺服电机归零 2. 通过调用 computeTrajectory() 计算轨迹 3. 执行构建的动作列表中的每个步骤 */ int RobotArmManager_graspAt(RobotArmManager* const me, int x, int y, int z, int t) { me->currentStep = -1; me->nSteps = 0; RobotArmManager_zero(me); RobotArmManager_computeTrajectory(me,x,y,z,t); if (me->nSteps == 0) { me->status = -1; } else { do { me->currentStep++; me->status = RobotArmManager_executeStep(me); } while (me->status ==0 && me->currentStep < me->nSteps); } return me->status; } /* 操作 zero() 此操作将所有伺服电机返回到起始默认位置 */ int RobotArmManager_zero(RobotArmManager* const me) { /* 将所有设备归零 */ int j; for (j=0; j<4; j++){ if (me->itsRotatingArmJoint[j] == NULL) return -1; if (RotatingArmJoint_zero(me->itsRotatingArmJoint[j])) return -1; } for (j=0; j<2; j++) { if (me->itsSlidingArmJoint[j] == NULL) return -1; if (SlidingArmJoint_zero(me->itsSlidingArmJoint[j])) return -1; } if (me->itsGraspingManipulator == NULL) return -1; if (GraspingManipulator_open(me->itsGraspingManipulator)) return -1; return 0; }

```
struct GraspingManipulator*
RobotArmManager_getItsGraspingManipulator(const RobotArmManager* const me) {
        return (struct GraspingManipulator*)me->itsGraspingManipulator;
}

void RobotArmManager_setItsGraspingManipulator(RobotArmManager* const me, struct
GraspingManipulator* p_GraspingManipulator) {
        me->itsGraspingManipulator = p_GraspingManipulator;
}

int RobotArmManager_getItsRotatingArmJoint(const RobotArmManager* const me) {
        int iter = 0;
        return iter;
}

void RobotArmManager_addItsRotatingArmJoint(RobotArmManager* const me, struct
RotatingArmJoint * p_RotatingArmJoint) {
        int pos;
        for(pos = 0; pos < 4; ++pos) {
            if (!me->itsRotatingArmJoint[pos]) {
                me->itsRotatingArmJoint[pos] = p_RotatingArmJoint;
                break;
            }
        }
}

void RobotArmManager_removeItsRotatingArmJoint(RobotArmManager* const me, struct
RotatingArmJoint * p_RotatingArmJoint) {
        int pos;
        for(pos = 0; pos < 4; ++pos) {
            if (me->itsRotatingArmJoint[pos] == p_RotatingArmJoint) {
                me->itsRotatingArmJoint[pos] = NULL;
            }
        }
}

void RobotArmManager_clearItsRotatingArmJoint(RobotArmManager* const me) {
        {
                int pos;
                for(pos = 0; pos < 4; ++pos)
                {
                    me->itsRotatingArmJoint[pos] = NULL;
                }
        }
}

int RobotArmManager_getItsSlidingArmJoint(const RobotArmManager* const me) {
        int iter = 0;
        return iter;
}

void RobotArmManager_addItsSlidingArmJoint(RobotArmManager* const me, struct
SlidingArmJoint * p_SlidingArmJoint) {
```


struct GraspingManipulator* RobotArmManager_getItsGraspingManipulator(const RobotArmManager* const me) { return (struct GraspingManipulator*)me-> itsGraspingManipulator; } void RobotArmManager_ setItsGraspingManipulator(RobotArmManager* const me, struct GraspingManipulator* p_ GraspingManipulator) { me->itsGraspingManipulator = p_GraspingManipulator; } int RobotArmManager_getItsRotatingArmJoint(const RobotArmManager* const me) { int iter = 0; return iter; } void RobotArmManager_addItsRotatingArmJoint(RobotArmManager* const me, struct RotatingArmJoint * p_RotatingArmJoint) { int pos; for(pos = 0; pos < 4; ++pos) { if (!me-> itsRotatingArmJoint[pos]) { me->itsRotatingArmJoint[pos] = p_RotatingArmJoint; break; } } } void RobotArmManager_removeItsRotatingArmJoint(RobotArmManager* const me, struct RotatingArmJoint * p_RotatingArmJoint) { int pos; for(pos = 0; pos < 4; ++pos) { if (me-> itsRotatingArmJoint[pos] == p_RotatingArmJoint) { me->itsRotatingArmJoint[pos] = NULL; } } } void RobotArmManager_clearItsRotatingArmJoint(RobotArmManager* const me) { { int pos; for(pos = 0; pos < 4; ++pos) { me->itsRotatingArmJoint[pos] = NULL; } } } int RobotArmManager_ getItsSlidingArmJoint(const RobotArmManager* const me) { int iter = 0; return iter; } void RobotArmManager_addItsSlidingArmJoint(RobotArmManager* const me, struct SlidingAr mJoint * p_SlidingArmJoint) {

```
        int pos;
        for(pos = 0; pos < 2; ++pos) {
            if (!me->itsSlidingArmJoint[pos]) {
                me->itsSlidingArmJoint[pos] = p_SlidingArmJoint;
                break;
            }
        }
}
void RobotArmManager_removeItsSlidingArmJoint(RobotArmManager* const me, struct
SlidingArmJoint * p_SlidingArmJoint) {
        int pos;
        for(pos = 0; pos < 2; ++pos) {
            if (me->itsSlidingArmJoint[pos] == p_SlidingArmJoint) {
                me->itsSlidingArmJoint[pos] = NULL;
            }
        }
}
void RobotArmManager_clearItsSlidingArmJoint(RobotArmManager* const me) {
        {
                int pos;
                for(pos = 0; pos < 2; ++pos)
                {
                    me->itsSlidingArmJoint[pos] = NULL;
                }
        }
}
RobotArmManager * RobotArmManager_Create(void) {
        RobotArmManager* me = (RobotArmManager *)
        malloc(sizeof(RobotArmManager));
        if(me!=NULL)
                {
                        RobotArmManager_Init(me);
                }
        return me;
}
void RobotArmManager_Destroy(RobotArmManager* const me) {
        if(me!=NULL) {
                        RobotArmManager_Cleanup(me);
        }
        free(me);
}
static void cleanUpRelations(RobotArmManager* const me) {
        if(me->itsGraspingManipulator != NULL) {
                        me->itsGraspingManipulator = NULL;
        }
}
int RobotArmManager_getItsAction(const RobotArmManager* const me)
{
        int iter = 0;
```


int pos; for(pos = 0; pos < 2; ++pos) { if (!me->itsSlidingArmJoint[pos]) { me->itsSlidingArmJoint[pos] = p_SlidingArmJoint; break; } } } void RobotArmManager_removeItsSlidingArmJoint(RobotArmManager* const me, str

uct

SlidingArmJoint * p_SlidingArmJoint) { int pos; for(pos = 0; pos < 2; ++pos) { if (me->itsSlidingArmJoint[pos] == p_SlidingArmJoint) { me->itsSlidingArmJoint[pos] = 空; } } } void RobotArmManager_clearItsSlidingArmJoint(RobotArmManager* const me) { { int pos; for(pos = 0; pos < 2; ++pos) { me->itsSlidingArmJoint[pos] = 空; } } } RobotArmManager * RobotArmManager_Create(void) { RobotArmManager* me = (RobotArmManager *) malloc(sizeof(RobotArmManager)); if(me!=空) { RobotArmManager_Init(me); } return me; } void RobotArmManager_Destroy(RobotArmManager* const me) { if(me!=空) { RobotArmManager_Cleanup(me); } free(me); } static void cleanUpRelations(RobotArmManager* const me) { if(me->itsGraspingManipulator != 空) { me->itsGraspingManipulator = 空; } } int RobotArmManager_getItsAction(const RobotArmManager* const me) { int iter = 0;

```
        return iter;
}

void RobotArmManager_addItsAction(RobotArmManager* const me, struct Action * p_Action) {
        int pos;
        for(pos = 0; pos < 100; ++pos) {
            if (!me->itsAction[pos]) {
                me->itsAction[pos] = p_Action;
                break;
            }
        }
}

void  RobotArmManager_removeItsAction(RobotArmManager*  const  me,  struct  Action  *
p_Action) {
        int pos;
        for(pos = 0; pos < 100; ++pos) {
            if (me->itsAction[pos] == p_Action) {
                me->itsAction[pos] = NULL;
            }
        }
}

void RobotArmManager_clearItsAction(RobotArmManager* const me) {
          int pos;
        for(pos = 0; pos < 100; ++pos) {
                me->itsAction[pos] = NULL;
            }
    }
```

Code Listing 3-10: RobotArmManager.c

3.5 Observer Pattern

The Observer Pattern is one of the most common patterns around. When present, it provides a means for objects to "listen in" on others while requiring no modifications whatsoever to the data servers. In the embedded domain, this means that sensor data can be easily shared to elements that may not even exist when the sensor proxies are written.

3.5.1 Abstract

The Observer Pattern (also known as the "Publish-Subscribe Pattern") provides notification to a set of interested clients that relevant data have changed. It does this without requiring the data server to have any *a priori* knowledge about its clients. Instead, the clients simply offer a subscription function that allows clients to dynamically add (and remove) themselves to the notification list. The data server can then enforce whatever notification policy it desires. Most commonly, data are sent whenever new data arrive, but clients can also be updated periodically,

return iter; } void RobotArmManager_addItsAction(RobotArmManager* const me, struct Action * p_Action) { int pos; for(pos = 0; pos < 100; ++pos) { if (!me->itsAction[pos]) { me->itsAction[pos] = p_Action; break; } } }

void RobotArmManager_removeItsAction(RobotArmManager* const me, struct Action * p_Action) { int pos; for(pos = 0; pos < 100; ++pos) { if (me->itsAction[pos] == p_Action) { me->itsAction[pos] = 空; } } } void RobotArmManager_clearItsAction(RobotArmManager* const me) { int pos; for(pos = 0; pos < 100; ++pos) { me->itsAction[pos] = 空; } }

代码清单 3-10：RobotArmManager.c

3.5 观察者模式

观察者模式是最常见的模式之一。当存在该模式时，它提供了一种让对象"监听"其他对象的方式，同时无需对数据服务器做任何修改。在嵌入式领域中，这意味着传感器数据可以轻松共享给那些在编写传感器代理时甚至可能还不存在的元素。

3.5.1 摘要

观察者模式（也称为"发布-订阅模式"）向一组感兴趣的客户端提供相关数据已发生变化的通知。它无需数据服务器对其客户端有任何先验知识即可实现这一点。相反，客户端只需提供一个订阅函数，使其能够动态地将自己添加（和移除）到通知列表中。然后，数据服务器可以执行其所需的任何通知策略。最常见的情况是，每当新数据到达时就发送数据，但客户端也可以定期更新，

or with a minimum or maximum frequency. This reduces the computational burden on the clients to ensure that they have timely data.

3.5.2 Problem

In a naïve situation, each client can request data periodically from a data server in case the data have changed, but that is wasteful of compute and communication resources as the clients generally cannot know when new data are available. If the data server pushes the data out, then it must know who all of its clients are, breaking the basic rule of client-server relations[13] requiring changes to the server to add new clients.

The Observer Pattern addresses this concern by adding subscription and unsubscription services to the data server. Thus a client can dynamically add itself to the notification list without *a priori* knowledge of the client on the part of the server. On the server side, the server can enforce the appropriate update policy to the notification of its interested clients. In addition, the pattern allows dynamic modification of subscriber lists and so adds a great deal of flexibility to the software.

3.5.3 Pattern Structure

Figure 3-8 shows the basic structure of the pattern. The `AbstractSubject` is the data server plus the machinery to maintain a list of interested subscribers. A client adds itself to the notification list by passing a pointer to an `accept(Datum)` function and removes itself by calling unsubscribe with the same pointer. When the `AbstractSubject` decides to notify its clients, the notify() function walks through the client list, calling the pointed-to function and passing the relevant data. The `AbstractClient` provides the `accept(Datum)` function to receive and process the incoming data.

3.5.4 Collaboration Roles

This section describes the roles for this pattern.

3.5.4.1 AbstractClient Interface

The `AbstractClient` associates with the `AbstractSubject` so that it can invoke the latter's various services. It contains an `accept(Datum)` function to be called when the `AbstractClient` subscribes as well as whenever the `AbstractSubject` thinks it is appropriate to send data. The `AbstractClient` is *abstract* in the sense that while it specifies the functions, it does not provide any implementation. It is realized by a set of `ConcreteClients` that provide implementation that do things with the received data.

In addition to the accept(Datum) function, the AbstractClient associates with the data. This is normally realized with a pointer but it can also be a stack, global, or static variable.

[13] That is, the clients know about the server but the server doesn't know who its clients are.

或以最小或最大频率更新。这减轻了客户端的计算负担，以确保它们拥有及时的数据。

3.5.2 问题

在一种简单的情况下，每个客户端可以定期向数据服务器请求数据，以防数据发生变化，但这会浪费计算和通信资源，因为客户端通常无法知道新数据何时可用。如果数据服务器推送数据，那么它必须知道其所有客户端是谁，这打破了客户端-服务器关系的基本规则[13]，要求修改服务器才能添加新客户端。

观察者模式通过向数据服务器添加订阅和取消订阅服务来解决这一问题。因此，客户端可以动态地将自己添加到通知列表中，而服务器无需事先了解该客户端。在服务器端，服务器可以对其感兴趣的客户端的通知执行适当的更新策略。此外，该模式允许动态修改订阅者列表，从而为软件增加了极大的灵活性。

3.5.3 模式结构

图 3-8 展示了该模式的基本结构。抽象主题 是数据服务器加上维护感兴趣订阅者列表的机制。客户端通过传递指向`accept(Datum)`函数的指针将自己添加到通知列表，并通过使用相同指针调用unsubscribe来移除自己。当抽象主题 决定通知其客户端时，notify()函数会遍历客户端列表，调用所指向的函数并传递相关数据。抽象客户端 提供`accept(Datum)`函数来接收和处理传入的数据。

3.5.4 协作角色

本节描述该模式的角色。

3.5.4.1 抽象客户端接口

抽象客户端 与抽象主题 关联，以便它可以调用后者的各种服务。它包含一个`accept(Datum)`函数，当抽象客户端 订阅时以及每当抽象主题 认为适合发送数据时，该函数会被调用。抽象客户端 之所以是抽象的，是因为它虽然指定了函数，但不提供任何实现。它由一组具体客户端 实现，这些具体客户端提供对接收到的数据进行处理的实现。

除了 accept(Datum) 函数之外，AbstractClient 还与数据相关联。这通常通过指针实现，但也可以是栈、全局变量或静态变量。

[13] Th也就是说，客户端知道服务器，但服务器不知道它的客户端是谁。

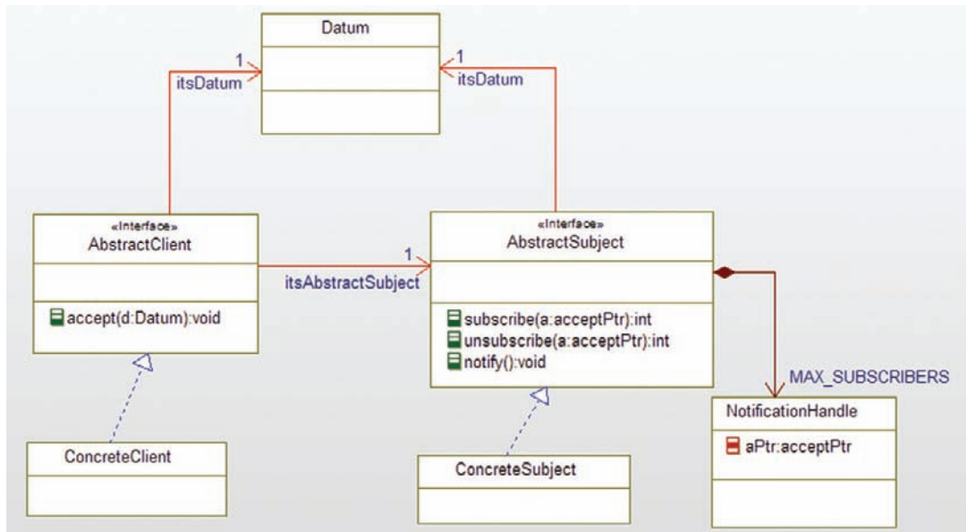


Figure 3-8 Observer Pattern

3.5.4.2 AbstractSubject Interface

The `AbstractSubject` is the data server in this pattern. Relevant to this pattern it provides three services. The `subscribe(acceptPtr)` service adds the pointer to the accept function to the notification list. Its return value is zero if the add is successful or non-zero if not. The `unsubscribe(acceptPtr)` function removes the accept function from the notification list, again returning zero if successful. Finally, the `notify()` function walks the notification list to notify the subscribed clients. This is done by invoking the function pointed to by each notification handle.

The `AbstractSubject` also contains a link to the `Datum` to be passed as well as a list of `NotificationHandles`. In the representation here, the `NotificationHandle` is implemented as an array of pointers, but it can also be a linked list or other data structure. Similarly, the `Datum` can be on the heap, stack, or static as appropriate.

3.5.4.3 ConcreteClient

The `ConcreteClient` is a concrete realization of the `AbstractClient` interface. That is, it provides an implementation of the `acceptPtr(Datum)` function as well as other functionality not relevant to the pattern *per se*.

3.5.4.4 ConcreteSubject

The `ConcreteSubject` is a concrete realization of the `AbstractSubject` interface. It provides implementation of the functions but also provides means to acquire and manage the

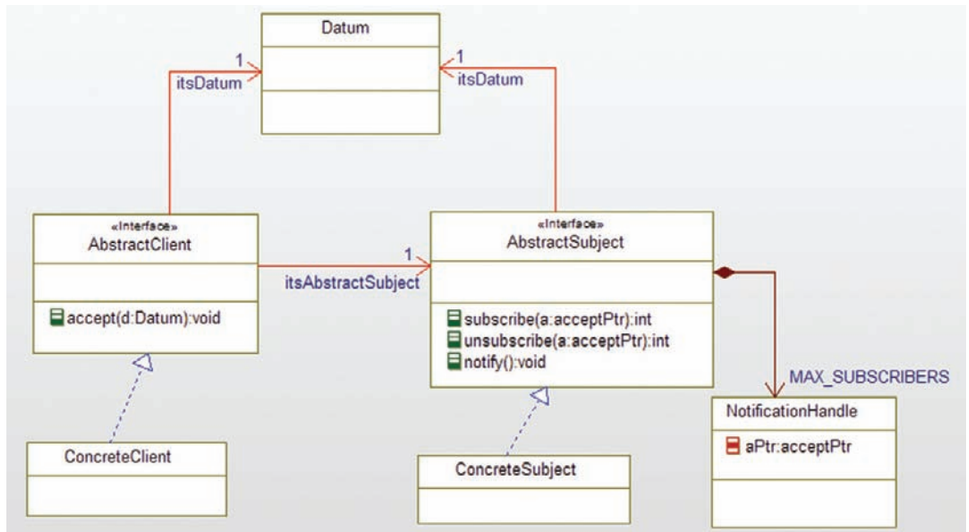


图 3-8 观察者模式

3.5.4.2 抽象主题接口

`AbstractSubject` 是该模式中的数据服务器。与该模式相关，它提供三项服务。`subscribe(acceptPtr)` 服务将接受函数的指针添加到通知列表。如果添加成功，其返回值为零；否则为非零。`unsubscribe(acceptPtr)` 函数从通知列表中移除接受函数，同样在成功时返回零。最后，`notify()` 函数遍历通知列表以通知已订阅客户端。这是通过调用每个通知句柄所指向的函数来完成的。

抽象主题 还包含一个指向要传递的数据 的链接，以及一个通知句柄列表。在此处的表示中，`NotificationHandle` 实现为指针数组，但也可以采用链表或其他数据结构。类似地，数据 可以根据需要放在堆、栈或静态存储区中。

3.5.4.3 具体客户端

具体客户端 是抽象客户端 接口的具体实现。也就是说，它提供了`acceptPtr(Datum)` 函数的实现，以及与模式本身无关的其他功能。

3.5.4.4 具体主题

具体主题 是抽象主题 接口的具体实现。它提供了函数的实现，同时也提供了获取和管理其所分发

data it distributes. The `ConcreteSubject` is often a `HardwareProxy` in addition to an `AbstractSubject`.

3.5.4.5 Datum

This element is a stand-in for the data of interest to the clients and subjects. It may be a simple primitive type such as an int or may be a complex struct.

3.5.4.6 NotificationHandle

The `NotificationHandle` is a representation of a means to invoke a client's `accept(Datum)` function. By far, the most common implementation is as a function pointer, but other implementations can be used as well.

3.5.5 Consequences

The Observer Pattern simplifies the process of distributing data to a set of clients who may not be known at design-time, and dynamically managing lists of interested clients during run-time. This pattern maintains the fundamental client-server relation while providing run-time flexibility through its subscription mechanism. Compute efficiency is maintained because clients are only updated when appropriate; the most common policy is that the clients are updated when the data change, but any appropriate policy can be implemented.

3.5.6 Implementation Strategies

The only complex aspects of this pattern are the implementation of the notification handle and the management of the notification handle list. The notification handle itself is almost always a function callback, that is, a pointer to a function with the right signature. Of course, this signature varies with the data being returned, but it also depends on the implementation of the software "classes" as discussed in Chapter 1. If using the struct approach with the `me` pointers, this will be the first parameter of the accept function. If you don't use the approach but write a more traditional C, then the realization of the `ConcreteClients` is more manual but you won't have to include the `me` pointer in the accept function parameter list. In the example later in this section, we shall use the former approach.

The easiest approach for the notification list is to declare an array big enough to hold all potential clients. This wastes memory in highly dynamic systems with many potential clients. An alternative is to construct the system as a linked list; that is, add another variable to each notification handle which is a pointer to the next one. Obviously, the last element in the list has a `NULL` next pointer value.

数据的手段。具体主题 通常除了是抽象主题之外，还是一个硬件代理 。

3.5.4.5 数据

该元素是客户端和主体所关注数据的替代物。它可以是简单的基本类型（如 int），也可以是复杂的结构体。

3.5.4.6 NotificationHandle

`NotificationHandle` 是对调用客户端 `accept(Datum)` 函数的一种方式的表示。到目前为止，最常见的实现是函数指针，但也可以使用其他实现。

3.5.5 结果

观察者模式简化了将数据分发给一组在设计时可能未知的客户端的过程，并在运行时动态管理感兴趣客户端的列表。该模式保持了基本的客户端-服务器关系，同时通过其订阅机制提供了运行时灵活性。计算效率得以保持，因为客户端仅在适当的时候被更新；最常见的策略是当数据变化时更新客户端，但也可以实现任何适当的策略。

3.5.6 实现策略

该模式唯一复杂的方面是通知句柄的实现和通知句柄列表的管理。通知句柄本身几乎总是一个函数回调，即一个具有正确签名的函数指针。当然，这个签名随返回的数据而变化，但它也取决于第1章中讨论的软件"类"的实现。如果使用带有 `me` 指针的结构体方法，这将是接受函数的第一个参数。如果不使用该方法而是编写更传统的C代码，那么 具体客户端 的实现更加手动化，但你不必在接受函数的参数列表中包含 `me` 指针。在本节后面的示例中，我们将使用前一种方法。

通知列表最简单的做法是声明一个足够大的数组来容纳所有潜在的客户端。在具有许多潜在客户端的高度动态系统中，这会浪费内存。另一种方法是将系统构造为链表；也就是说，为每个通知句柄添加另一个变量，该变量是指向下一个通知句柄的指针。显然，链表中的最后一个元素的 `NULL` 下一个指针值。

3.5.7 Related Patterns

This pattern can be freely mixed with the previous patterns in this chapter since its concerns are orthogonal. In embedded systems it is very common to add Observer functionality to a Hardware Proxy or Hardware Adapter, for example.

3.5.8 Example

Figure 3-9 provides a straightforward example of this pattern. In this example, the `GasSensor` is the `ConcreteClient` of the pattern; it provides the subscribe() and unsubscribe functions and manages the array of `GasNotificationHandles`. Because we implemented the pattern with the usage of our pseudo-object–oriented approach, all the operations have a first parameter that identifies the instance data. These "hidden parameters" are not shown in Figure 3-9 but they do show up in the code. This impacts the `GasNotificationHandle` in that not only do we need the function pointer for the acceptor function, we also need its instance data point to pass as its first argument.

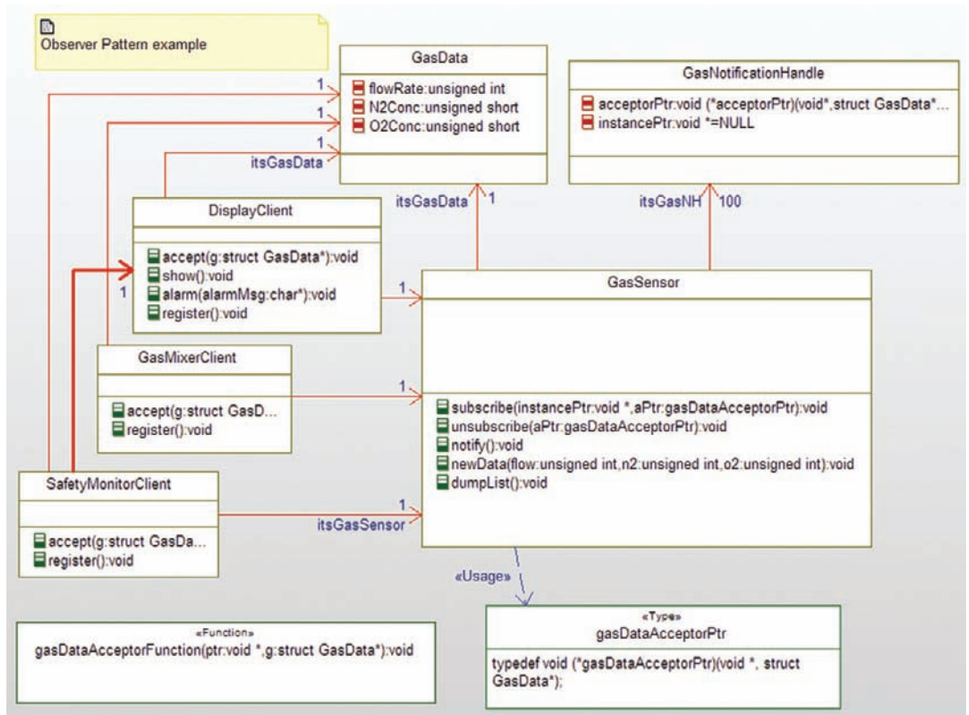


Figure 3-9 Observer Pattern example

3.5.7 相关模式

该模式可以与本章前面的模式自由混合使用，因为它关注的是正交的问题。例如，在嵌入式系统中，为硬件代理或硬件适配器添加观察者功能是非常常见的。

3.5.8 示例

图 3-9 提供了该模式的一个简单示例。在此示例中，`GasSensor`是该模式的具体客户端；它提供了 subscribe() 和 unsubscribe 函数，并管理 `GasNotificationHandles` 数组。由于我们使用伪面向对象方法实现了该模式，所有操作都有一个用于标识实例数据的第一个参数。这些"隐藏参数"未在图 3-9 中显示，但它们确实出现在代码中。这会影响 `GasNotificationHandle`，因为我们不仅需要接受器函数的函数指针，还需要其实例数据指针作为第一个参数传递。

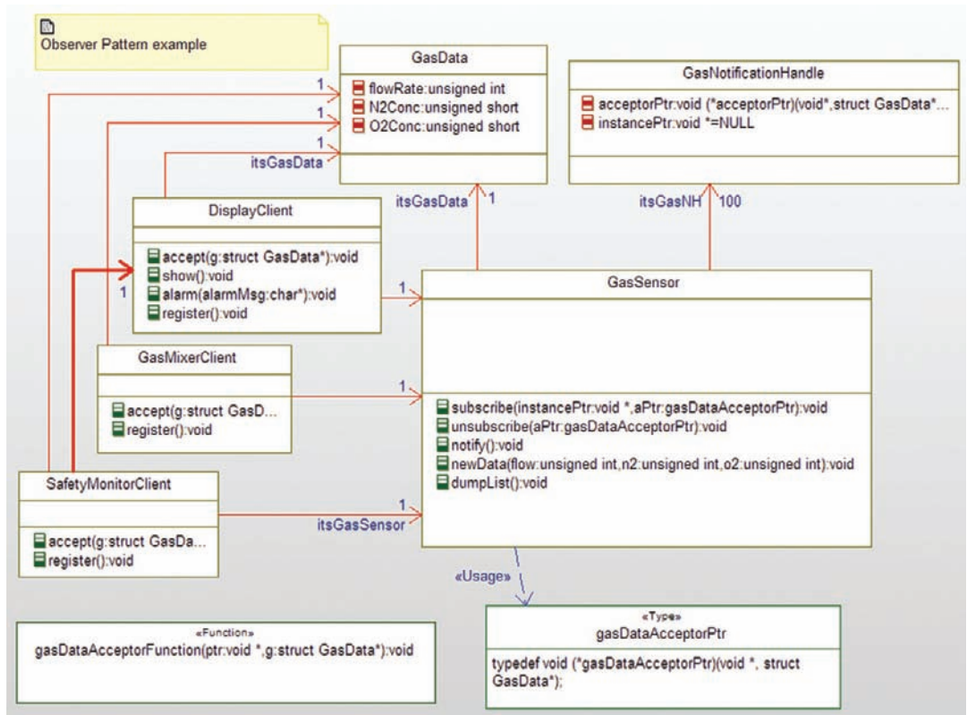


图 3-9 观察者模式示例

In addition to the subscribe() and unsubscribe() operations, the notify() function walks the list of registered subscribers and invokes the accept() operation using the simple statement

```
me->itsGasNH[pos]->acceptorPtr(me->itsGasNH[pos]->instancePtr,
me->itsGasData);
```

In this way, the function being pointed to is accessed via the function pointer, where the first parameter is the instance data of the client and the second parameter is a pointer to the new data.

The GasSensor class also includes a newData() function that calls the notify() function and a dumpList() function that prints out a list of the currently subscribed clients. The code for the GasSensor is shown in Code Listing 3-11 and Code Listing 3-12.

```
#ifndef GasSensor_H
#define GasSensor_H

#include "GasData.h"
/* the function pointer type
    The first value of the function pointer is to the instance
    data of the class. The second is a ptr to the new gas data
*/
typedef void (*gasDataAcceptorPtr)(void *, struct GasData*);
struct GasNotificationHandle;
/* class GasSensor */
typedef struct GasSensor GasSensor;
struct GasSensor {
       struct GasData* itsGasData;
       struct GasNotificationHandle *itsGasNH[100];
};

/* Constructors and destructors:*/
void GasSensor_Init(GasSensor* const me);
void GasSensor_Cleanup(GasSensor* const me);

/* Operations */
void GasSensor_dumpList(GasSensor* const me);
void GasSensor_newData(GasSensor* const me, unsigned int flow, unsigned int n2, unsigned
int o2);
void GasSensor_notify(GasSensor* const me);
void GasSensor_subscribe(GasSensor* const me, void * instancePtr, const gasDataAccep-
torPtr* aPtr);
void GasSensor_unsubscribe(GasSensor* const me, const gasDataAcceptorPtr* aPtr);
struct GasData* GasSensor_getItsGasData(const GasSensor* const me);
void GasSensor_setItsGasData(GasSensor* const me, struct GasData* p_GasData);
int GasSensor_getItsGasNH(const GasSensor* const me);
void  GasSensor_addItsGasNH(GasSensor*  const  me,  struct  GasNotificationHandle  *
p_GasNotificationHandle);
void  GasSensor_removeItsGasNH(GasSensor*  const  me,  struct  GasNotificationHandle  *
p_GasNotificationHandle);
void GasSensor_clearItsGasNH(GasSensor* const me);
```


除了 subscribe() 和 unsubscribe() 操作之外，notify() 函数还会遍历已注册订阅者的列表，并使用以下简单语句调用 accept() 操作

me->itsGasNH[pos]->acceptorPtr(me->itsGasNH[pos]->instancePtr, me->itsGasData);

通过这种方式，被指向的函数通过函数指针进行访问，其中第一个参数是客户端的实例数据，第二个参数是指向新数据的指针。

GasSensor 类还包含一个调用notify() 函数的newData() 函数，以及一个打印当前已订阅客户端列表的dumpList() 函数。GasSensor 的代码如代码清单3-11和代码清单3-12所示。

```
#ifndef GasSensor_H #
define GasSensor_H

#include "GasData.h" /* 函数指针类型 函数指针的第一个值指向类的实例数
据。第二个是指向新气体数据的指针 */ typedef void
(*gasDataAcceptorPtr)(void *, struct GasData*); struct
GasNotificationHandle; /* 类 GasSensor */ typedef struct GasSensor
GasSensor; struct GasSensor { struct GasData* itsGasData; struct
GasNotificationHandle *itsGasNH[100]; }; /* 构造函数与析构函数： */
void GasSensor_Init(GasSensor* const me); void GasSensor_
Cleanup(GasSensor* const me);

/* 操作 */
void GasSensor_dumpList(GasSensor* const me);
void GasSensor_newData(GasSensor* const me, unsigned int flow, unsigned int n2, unsigned int o2);

void GasSensor_notify(GasSensor* const me); void GasSensor_subscribe(GasSensor* const me,
void * instancePtr, const gasDataA                                    ccep-
torPtr* aPtr); void GasSensor_unsubscribe(GasSensor* const me, const gasDataAcceptorPtr*
aPtr); struct GasData* GasSensor_getItsGasData(const GasSensor* const me); void GasSensor_
setItsGasData(GasSensor* const me, struct GasData* p_GasData); int GasSensor_
getItsGasNH(const GasSensor* const me);

void GasSensor_addItsGasNH(GasSensor* const me, struct GasNotificationHandle * p_
GasNotificationHandle);
void GasSensor_removeItsGasNH(GasSensor* const me, struct GasNotificationHand                le  *
p_GasNotificationHandle); void GasSensor_cl
earItsGasNH(GasSensor* const me);
```

```
GasSensor * GasSensor_Create(void);
void GasSensor_Destroy(GasSensor* const me);
#endif
```

Code Listing 3-11: GasSensor.h

```
#include "GasSensor.h"
#include "GasData.h"
#include "GasNotificationHandle.h"

static void cleanUpRelations(GasSensor* const me);
void GasSensor_Init(GasSensor* const me) {
        me->itsGasData = NULL;
        int pos;
        for(pos = 0; pos < 100; ++pos) {
            me->itsGasNH[pos] = NULL;
        }
}

void GasSensor_Cleanup(GasSensor* const me) {
        cleanUpRelations(me);
}

void GasSensor_dumpList(GasSensor* const me) {
        int pos;
        char s[100];
        printf("Dumping registered elements/n");
        for (pos=0; pos<100; pos++)
            if (me->itsGasNH[pos])
                if (me->itsGasNH[pos]->acceptorPtr) {
                    printf("Client %d: InstancePtr=%p, acceptPtr=%p/n",
                        pos,me->itsGasNH[pos]->instancePtr, me->itsGasNH[pos]-
>acceptorPtr);
                };
}

void GasSensor_newData(GasSensor* const me, unsigned int flow, unsigned int n2, unsigned
int o2) {
        if (!me->itsGasData)
            me->itsGasData = GasData_Create();
        me->itsGasData->flowRate = flow;
        me->itsGasData->N2Conc = n2;
        me->itsGasData->O2Conc = o2;
        GasSensor_notify(me);
}

void GasSensor_notify(GasSensor* const me) {
        int pos;
        char s[100];
        for (pos=0; pos<100; pos++)
            if (me->itsGasNH[pos])
                if (me->itsGasNH[pos]->acceptorPtr)
```


GasSensor * GasSensor_Create(void); void
GasSensor_Destroy(GasSensor* const me); #endif

代码清单3-11：GasSensor.h

#include "GasSensor.h" #include "
GasData.h" #include "
GasNotificationHandle.h"

static void cleanUpRelations(GasSensor* const me); void GasSensor_Init(GasSensor* const me) { me->
itsGasData = 空; int pos; for(pos = 0; pos < 100; ++pos) { me->itsGasNH[pos] = 空; } } void GasSensor_
Cleanup(GasSensor* const me) { cleanUpRelations(me); } void GasSensor_dumpList(GasSensor* const
me) { int pos; char s[100]; printf("Dumping registered elements/n"); for (pos=0; pos<100; pos++) if (me->
itsGasNH[pos]) if (me->itsGasNH[pos]->acceptorPtr) { printf("Client %d: InstancePtr=%p, acceptPtr=%
p/n", pos,me->itsGasNH[pos]->instancePtr, me->itsGasNH[pos]->acceptorPtr); }; } void GasSensor_
newData(GasSensor* const me, unsigned int flow, unsigned int n2, unsigned int o2) { if (!me->
itsGasData) me->itsGasData = GasData_Create(); me->itsGasData->flowRate = flow; me->itsGasData->
N2Conc = n2; me->itsGasData->O2Conc = o2; GasSensor_notify(me); } void GasSensor_
notify(GasSensor* const me) { int pos; char s[100]; for (pos=0; pos<100; pos++) if (me->itsGasNH[pos]) if
(me->itsGasNH[pos]->acceptorPtr)

```
                me->itsGasNH[pos]->acceptorPtr(me->itsGasNH[pos]->instancePtr,
me->itsGasData);
}

void GasSensor_subscribe(GasSensor* const me, void * instancePtr, const gasDataAccep-
torPtr* aPtr) {
        struct GasNotificationHandle* gnh;
        gnh = GasNotificationHandle_Create();
        gnh->instancePtr = instancePtr;
        gnh->acceptorPtr = aPtr;
        GasSensor_addItsGasNH(me, gnh);
}

void GasSensor_unsubscribe(GasSensor* const me, const gasDataAcceptorPtr* aPtr) {
        int pos;
        for(pos = 0; pos < 100; ++pos) {
            if (me->itsGasNH[pos])
                if (me->itsGasNH[pos]->acceptorPtr == aPtr) {
                    GasNotificationHandle_Destroy(me->itsGasNH[pos]);
                    me->itsGasNH[pos] = NULL;
                }
        }
}

struct GasData* GasSensor_getItsGasData(const GasSensor* const me) {
        return (struct GasData*)me->itsGasData;
}

void GasSensor_setItsGasData(GasSensor* const me, struct GasData* p_GasData) {
        me->itsGasData = p_GasData;
}

int GasSensor_getItsGasNH(const GasSensor* const me) {
        int iter = 0;
        return iter;
}

void GasSensor_addItsGasNH(GasSensor* const me, struct GasNotificationHandle *
p_GasNotificationHandle) {
        int pos;
        for(pos = 0; pos < 100; ++pos) {
            if (!me->itsGasNH[pos]) {
                me->itsGasNH[pos] = p_GasNotificationHandle;
                break;
            }
        }
}

void  GasSensor_removeItsGasNH(GasSensor*  const  me,  struct  GasNotificationHandle  *
p_GasNotificationHandle) {
        int pos;
        for(pos = 0; pos < 100; ++pos) {
            if (me->itsGasNH[pos] == p_GasNotificationHandle) {
```


me->itsGasNH[pos]->acceptorPtr(me->itsGasNH[pos]->instancePt r, me->itsGasData); } void GasSensor_subscribe(GasSensor* const me, void * instancePtr, const gasData

Accep-

torPtr* aPtr) { struct GasNotificationHandle* gnh; gnh = GasNotificationHandle_Create(); gnh-> instancePtr = instancePtr; gnh->acceptorPtr = aPtr; GasSensor_addItsGasNH(me, gnh); } void GasSensor_unsubscribe(GasSensor* const me, const gasDataAcceptorPtr* aPtr) { int pos; for(pos = 0; pos < 100; ++pos) { if (me->itsGasNH[pos]) if (me->itsGasNH[pos]->acceptorPtr == aPtr) { GasNotificationHandle_Destroy(me->itsGasNH[pos]); me->itsGasNH[pos] = 空; } } } struct GasData* GasSensor_getItsGasData(const GasSensor* const me) { return (struct GasData*)me-> itsGasData; } void GasSensor_setItsGasData(GasSensor* const me, struct GasData* p_GasData) { me->itsGasData = p_GasData; } int GasSensor_getItsGasNH(const GasSensor* const me) { int iter = 0; return iter; } void GasSensor_addItsGasNH(GasSensor* const me, struct GasNotificationHandle * p_GasNotificationHandle) { int pos; for(pos = 0; pos < 100; ++pos) { if (! me->itsGasNH[pos]) { me->itsGasNH[pos] = p_GasNotificationHandle; break; } } }

void GasSensor_removeItsGasNH(GasSensor* const me, struct GasNotificationHandle * p_ GasNotificationHandle) { int pos; for(pos = 0; pos < 100; ++pos) { if (me->itsGasNH[pos] == p_ GasNotificationHandle) {

```
                me->itsGasNH[pos] = NULL;
            }
        }
}
void GasSensor_clearItsGasNH(GasSensor* const me) {
        int pos;
        for(pos = 0; pos < 100; ++pos)
        {
            me->itsGasNH[pos] = NULL;
        }
}
GasSensor * GasSensor_Create(void) {
        GasSensor* me = (GasSensor *) malloc(sizeof(GasSensor));
        if(me!=NULL) {
                        GasSensor_Init(me);
        }
        return me;
}
void GasSensor_Destroy(GasSensor* const me) {
        if(me!=NULL) {
                        GasSensor_Cleanup(me);
        }
        free(me);
}
static void cleanUpRelations(GasSensor* const me) {
        if(me->itsGasData != NULL) {
                        me->itsGasData = NULL;
                }
}
```

Code Listing 3-12: GasSensor.c

Figure 3-9 shows that our example system has three clients – the DisplayClient, the GasMixerClient, and the SafetyMonitorClient. In addition to the accept() function, the DisplayMonitorClient class includes a show() function that displays the data when its accept() function is called, an alarm function (invoked by the SafetyMonitorClient when appropriate), and a register() function that calls the subscribe function of the GasSensor. The code for the DisplayClient is given in Code Listing 3-13 and Code Listing 3-14.

```
#ifndef DisplayClient_H
#define DisplayClient_H
#include "GasSensor.h"

typedef struct DisplayClient DisplayClient;
struct DisplayClient {
        struct GasData* itsGasData;
        struct GasSensor* itsGasSensor;
};
```


me->itsGasNH[pos] = 空; } } } void GasSensor_ clearItsGasNH(GasSensor* const me) { int pos; for(pos = 0; pos < 100; ++ pos) { me->itsGasNH[pos] = 空; } } GasSensor * GasSensor_Create(void) { GasSensor* me = (GasSensor *) malloc(sizeof(GasSensor)); if(me!=空) { GasSensor_Init(me); } return me; } void GasSensor_Destroy(GasSensor* const me) { if(me!=空) { GasSensor_Cleanup(me); } free(me); } static void cleanUpRelations(GasSensor* const me) { if(me->itsGasData != 空) { me-> itsGasData = 空; } }

代码清单3-12：GasSensor.c

图3-9显示我们的示例系统有三个客户端——DisplayClient、GasMixerClient和SafetyMonitorClient。除了accept()函数之外，DisplayMonitorClient类还包含一个show()函数，当其accept()函数被调用时显示数据；一个alarm函数（由SafetyMonitorClient在适当的时候调用）；以及一个register()函数，该函数调用GasSensor的subscribe函数。DisplayClient的代码见代码清单 3-13 和代码清单 3-14。

#ifndef DisplayClient_H #define DisplayClient_H # include "GasSensor.h" typedef 结构体 DisplayClient DisplayClient; 结构体 DisplayClient { 结构体 GasData* itsGasData; 结构体 GasSensor* itsGasSensor; };

```
/* Constructors and destructors:*/
void DisplayClient_Init(DisplayClient* const me);
void DisplayClient_Cleanup(DisplayClient* const me);
/* Operations */
void DisplayClient_accept(DisplayClient* const me, struct GasData* g);
void DisplayClient_alarm(DisplayClient* const me, char* alarmMsg);
void DisplayClient_register(DisplayClient* const me);
void DisplayClient_show(DisplayClient* const me);
struct GasData* DisplayClient_getItsGasData(const DisplayClient* const me);
void DisplayClient_setItsGasData(DisplayClient* const me, struct GasData* p_GasData);
struct GasSensor* DisplayClient_getItsGasSensor(const DisplayClient* const me);
void DisplayClient_setItsGasSensor(DisplayClient* const me, struct GasSensor* p_GasSensor);
DisplayClient * DisplayClient_Create(void);
void DisplayClient_Destroy(DisplayClient* const me);
#endif
```

Code Listing 3-13: DisplayClient.h

```
#include "DisplayClient.h"
#include "GasData.h"
#include "GasSensor.h"

static void cleanUpRelations(DisplayClient* const me);
void DisplayClient_Init(DisplayClient* const me) { me->itsGasData = NULL;
me->itsGasSensor = NULL;
}

void DisplayClient_Cleanup(DisplayClient* const me) {
        cleanUpRelations(me);
}

void DisplayClient_accept(DisplayClient* const me, struct GasData* g) {
        if (!me->itsGasData)
            me->itsGasData = GasData_Create();
        if (me->itsGasData) {
            me->itsGasData->flowRate = g->flowRate;
            me->itsGasData->N2Conc = g->N2Conc;
            me->itsGasData->O2Conc = g->O2Conc;
            DisplayClient_show(me);
        };
}

void DisplayClient_alarm(DisplayClient* const me, char* alarmMsg)
{
        printf("ALERT! ");
        printf(alarmMsg);
        printf("/n/n");
}
void DisplayClient_register(DisplayClient* const me) {
        if (me->itsGasSensor)
            GasSensor_subscribe(me->itsGasSensor, me,&DisplayClient_accept);
}
```


/* 构造函数与析构函数：*/ void DisplayClient_Init(DisplayClient* const me); void DisplayClient_Cleanup(DisplayClient* const me); /* 操作 */ void DisplayClient_accept(DisplayClient* const me, struct GasData* g); void DisplayClient_alarm(DisplayClient* const me, char* alarmMsg); void DisplayClient_register(DisplayClient* const me); void DisplayClient_show(DisplayClient* const me); struct GasData* DisplayClient_getItsGasData(const DisplayClient* const me); void DisplayClient_setItsGasData(DisplayClient* const me, struct GasData* p_GasData); struct GasSensor* DisplayClient_getItsGasSensor(const DisplayClient* const me); void DisplayClient_setItsGasSensor(DisplayClient* const me, struct GasSensor* p_GasSensor); DisplayClient * DisplayClient_Create(void); void DisplayClient_Destroy(DisplayClient* const me); #endif

代码清单 3-13：DisplayClient.h

#include “DisplayClient.h” #include “GasData.h” #include “GasSensor.h” static void cleanUpRelations(DisplayClient* const me); void DisplayClient_Init(DisplayClient* const me) { me->itsGasData = 空; me->itsGasSensor = 空; } void DisplayClient_Cleanup(DisplayClient* const me) { cleanUpRelations(me); } void DisplayClient_accept(DisplayClient* const me, struct GasData* g) { if (!me->itsGasData) me->itsGasData = GasData_Create(); if (me->itsGasData) { me->itsGasData->flowRate = g->flowRate; me->itsGasData->N2Conc = g->N2Conc; me->itsGasData->O2Conc = g->O2Conc; DisplayClient_show(me); }; } void DisplayClient_alarm(DisplayClient* const me, char* alarmMsg) { printf(“ ALERT! ”); printf(alarmMsg); printf(“/n/n”); } void DisplayClient_register(DisplayClient* const me) { if (me->itsGasSensor) GasSensor_subscribe(me->itsGasSensor, me,& DisplayClient_accept); }

```
void DisplayClient_show(DisplayClient* const me) {
        if (me->itsGasData) {
            printf("Gas Flow Rate = %5d/n", me->itsGasData->flowRate);
            printf("O2 Concentration = %2d/n", me->itsGasData->N2Conc);
            printf("N2 Concentration = %2d/n/n", me->itsGasData->N2Conc);
        }
        else
            printf("No data available/n/n");
}

struct GasData* DisplayClient_getItsGasData(const DisplayClient* const me) {
        return (struct GasData*)me->itsGasData;
}

void DisplayClient_setItsGasData(DisplayClient* const me, struct GasData* p_GasData) {
        me->itsGasData = p_GasData;
}

struct GasSensor* DisplayClient_getItsGasSensor(const DisplayClient* const me) {
        return (struct GasSensor*)me->itsGasSensor;
}

void DisplayClient_setItsGasSensor(DisplayClient* const me, struct GasSensor* p_GasSen-
sor) {
        me->itsGasSensor = p_GasSensor;
}

DisplayClient * DisplayClient_Create(void) {
        DisplayClient* me = (DisplayClient *)
malloc(sizeof(DisplayClient));
        if(me!=NULL) {
                        DisplayClient_Init(me);
        }
        return me;
}
void DisplayClient_Destroy(DisplayClient* const me) {
        if(me!=NULL) {
                        DisplayClient_Cleanup(me);
        }
        free(me);
}
static void cleanUpRelations(DisplayClient* const me) {
        if(me->itsGasData != NULL) {
                        me->itsGasData = NULL;
        }
        if(me->itsGasSensor != NULL) {
                        me->itsGasSensor = NULL;
        }
}
```

Code Listing 3-14: DisplayClient.c

void DisplayClient_show(DisplayClient* const me) { if (me->itsGasData) { printf("气体流速 = %5d/n", me->itsGasData->flowRate); printf("氧气浓度 = %2d/n", me->itsGasData->N2Conc); printf("N2 浓度 = %2d/n/n", me->itsGasData->N2Conc); } else printf("无可用数据/n/n"); } struct GasData* DisplayClient_getItsGasData(const DisplayClient* const me) { return (struct GasData*)me->itsGasData; } void DisplayClient_setItsGasData(DisplayClient* const me, struct GasData* p_GasData) { me->itsGasData = p_GasData; } struct GasSensor* DisplayClient_getItsGasSensor(const DisplayClient* const me) { return (struct GasSensor*)me->itsGasSensor; } void DisplayClient_setItsGasSensor(DisplayClient* const me, struct GasSensor* p_GasSen- sor) { me->itsGasSensor = p_GasSensor; } DisplayClient * DisplayClient_Create(void) { DisplayClient* me = (DisplayClient *) malloc(sizeof(DisplayClient)); if(me!=NULL) { DisplayClient_Init(me); } return me; } void DisplayClient_Destroy(DisplayClient* const me) { if(me!= NULL) { DisplayClient_Cleanup(me); } free(me); } static void cleanUpRelations(DisplayClient* const me) { if(me->itsGasData != NULL) { me->itsGasData = NULL; } if(me->itsGasSensor != NULL) { me->its GasSensor = NULL; } }

代码清单 3-14：DisplayClient.c

Lastly, let's look at the code for the GasData class. This is a very straightforward data class with just a few helper functions, as seen in Code Listing 3-15 and Code Listing 3-16.

```
ifndef GasData_H
#define GasData_H

typedef struct GasData GasData;
struct GasData {
        unsigned short N2Conc;
        unsigned short O2Conc;
        unsigned int flowRate;
};

/* Constructors and destructors:*/
oid GasData_Init(GasData* const me);
void GasData_Cleanup(GasData* const me);
GasData * GasData_Create(void);
void GasData_Destroy(GasData* const me);
#endif
```

Code Listing 3-15: GasData.h

```
#include "GasData.h"
void GasData_Init(GasData* const me) {
}

void GasData_Cleanup(GasData* const me) {
}

GasData * GasData_Create(void) {
        GasData* me = (GasData *) malloc(sizeof(GasData));
        if(me!=NULL) {
                        GasData_Init(me);
        }
        return me;
}

void GasData_Destroy(GasData* const me) {
        if(me!=NULL) {
                        GasData_Cleanup(me);
        }
        free(me);
}
```

Code Listing 3-16: GasData.c

3.6 Debouncing Pattern

This simple pattern is used to reject multiple false events arising from intermittent contact of metal surfaces.

最后，让我们看看 GasData 类的代码。这是一个非常简单的数据类，只有几个辅助函数，如代码清单 3-15 和代码清单 3-16 所示。

ifndef GasData H_#define GasData_H typedef struct GasData GasData; struct GasData { unsigned short N2Conc; unsigned short O2Conc; unsigned int flowRate; }; /* 构造函数与析构函数: */ oid GasData_Init(GasData* const me); void GasData_Cleanup(GasData* const me); GasData * GasData_Create(void); void GasData_Destroy(GasData* const me); #endif

代码清单 3-15：GasData.h

#include "GasData.h" void GasData_Init(GasData* const me) { } void GasData_Cleanup(GasData* const me) { } GasData * GasData_Create(void) { GasData* me = (GasData *) malloc(sizeof(GasData)); if(me!=空) { GasData_Init(me); } return me; } void GasData_Destroy(GasData* const me) { if(me!=空) { GasData_Cleanup(me); } free(me); }

代码清单 3-16：GasData.c

3.6 去抖模式

这种简单的模式用于拒绝由金属表面间歇性接触产生的多个虚假事件。

3.6.1 Abstract

Push buttons, toggle switches, and electromechanical relays are input devices for digital systems that share a common problem – as metal connections make contact, the metal deforms or “bounces”, producing intermittent connections during switch open or closure. Since this happens very slowly (order of milliseconds) compared to the response speed of embedded systems (order of microseconds or faster), this results in multiple electronic signals to the control system. This pattern addresses this concern by reducing the multiple signals into a single one by waiting a period of time after the initial signal and then checking the state.

3.6.2 Problem

Many input devices for embedded systems use metal-on-metal contact to indicate events of interest, such as button presses, switch movement, and activating or deactivating relays. As the metal moves into contact, physical deformation occurs resulting in an intermittent bouncing contact until the vibrations dampen down. This results in an intermediate contact profile such as that shown in Figure 3-10.

3.6.3 Pattern Structure

The basic solution is to accept the initial event, wait for the vibrations to dampen out, and then sample the data source for its state. See Figure 3-11 for the pattern structure.

3.6.4 Collaboration Roles

This section describes the roles for this pattern.

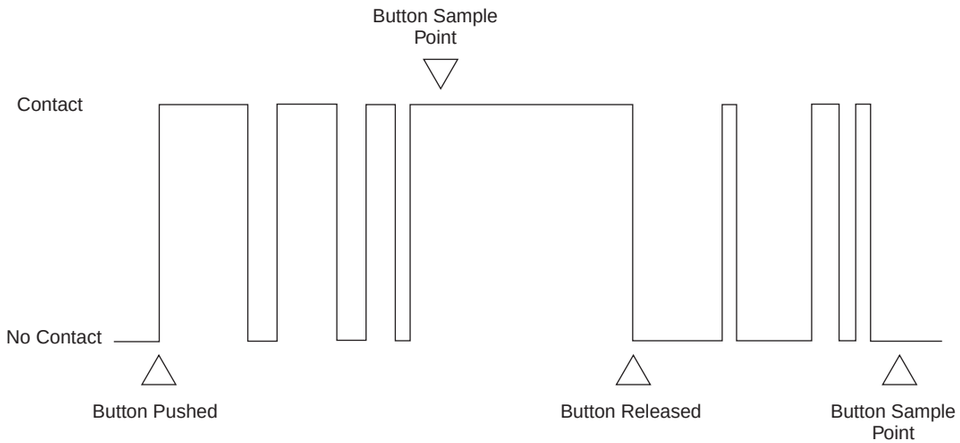


Figure 3-10 Bouncing electrical contacts

3.6.1 摘要

按钮、拨动开关和机电继电器是数字系统的输入设备，它们有一个共同的问题——当金属连接接触时，金属会变形或“弹跳”，在开关断开或闭合期间产生间歇性连接。由于这相对于嵌入式系统的响应速度（微秒级或更快）来说发生得非常缓慢（毫秒级），这会导致向控制系统发送多个电子信号。该模式通过在初始信号之后等待一段时间，然后检查状态，将多个信号减少为单个信号，从而解决这一问题。

3.6.2 问题

许多嵌入式系统的输入设备使用金属对金属接触来指示感兴趣的事件，例如按钮按下、开关移动以及激活或停用继电器。当金属移动至接触时，会发生物理变形，导致间歇性的弹跳接触，直到振动减弱。这会产生如图3-10所示的中间接触曲线。

3.6.3 模式结构

基本解决方案是接受初始事件，等待振动减弱，然后对数据源进行采样以获取其状态。模式结构见图3-11。

3.6.4 协作角色

本节描述该模式中的角色。

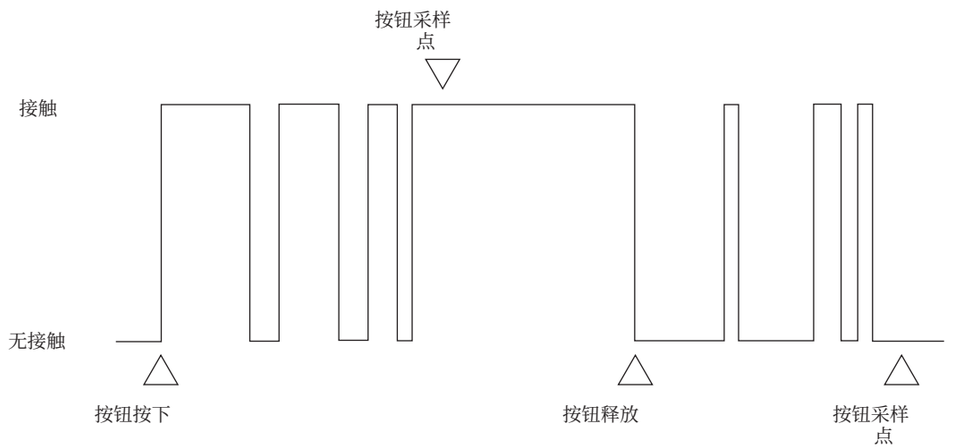


图3-10 电触点抖动

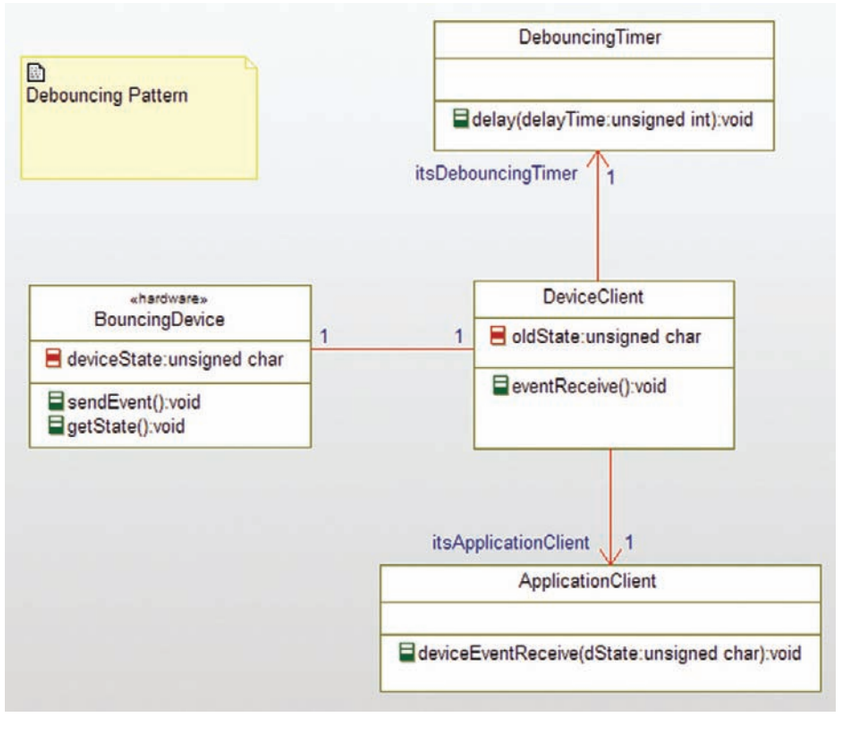


Figure 3-11 Debouncing Pattern

3.6.4.1 ApplicationClient

This element is the ultimate recipient of the debounced event. Its operation `deviceEventReceive()` is only activated when the event is real (i.e., results in a changed device state).

3.6.4.2 BouncingDevice

The `BouncingDevice` represents the hardware for the device itself. The most common implementation of this device is completely in hardware; in the case, the `sendEvent()` operation is simply an activation of an interrupt in the interrupt vector table of the embedded processor and the getState operation is implemented via a read of a memory location or IO port. `DeviceState` is normally a bivalued attribute, ON or OFF.

3.6.4.3 Debouncer

The `Debouncer` is the software element that processes the incoming event, debounces it, and makes sure that it represents an actual device state change. Its `eventReceive()` function is activated by the `sendEvent()` service of the `BouncingDevice`. In turn, it sets the delay timer (disabling interrupts from the device if necessary) and then checks the device state. If, after the

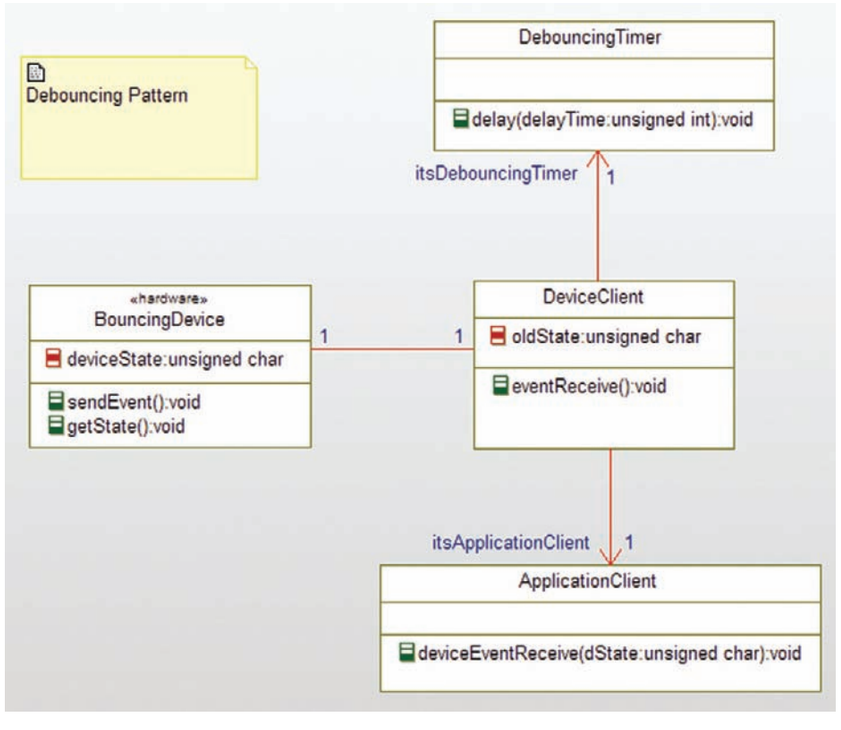


图 3-11 去抖模式

3.6.4.1 应用客户端

该元素是去抖事件的最终接收者。其操作`deviceEventReceive()`仅在事件为真实事件（即导致设备状态发生变化）时才会被激活。

3.6.4.2 弹跳设备

`BouncingDevice`表示设备本身的硬件。该设备最常见的实现完全由硬件完成；在这种情况下，`sendEvent()`操作仅仅是激活嵌入式处理器中断向量表中的一个中断，而获取状态操作则通过读取内存位置或IO端口来实现。`DeviceState`通常是一个二值属性，即开启或关闭。

3.6.4.3 去抖器

去抖器 是处理传入事件、对其进行去抖并确保其代表实际设备状态变化的软件元素。其`eventReceive()`函数由弹跳设备的`sendEvent()`服务激活。接着，它设置延迟定时器（必要时禁用来自设备的中断），然后检查设备状态。如果在

debouncing time, the state is different then the event must have been real, so it sends the appropriate message to the ApplicationClient. The old state of the button is stored in its variable oldState; this variable is updated whenever there is a detected change of state of the device.

3.6.4.4 DebouncingTimer

This timer provides a nonbusy wait via its `delay()` service. This is often done with an OS call but might be done with specialty timer hardware as well.

3.6.5 Consequences

This pattern is not necessary for hardware that debounces before announcing events. In my experience, however, it often falls on the software to perform this task. This is a simple pattern that performs the debouncing so that the application need only concern itself with true events resulting from a change in device state.

3.6.6 Implementation Strategies

It is common for the device to use an interrupt vector to contact its client. Indeed, it is common for the BouncingDevice to be entirely implemented in hardware so that when the event occurs, a hardware interrupt is created, and the device state must be verified by reading a memory location or port address. When the event interface is via an interrupt, the address of the DeviceClient::eventReceive() operation must be installed in the proper location in the interrupt vector table when the DeviceClient is initiated, and the address must be removed if the DeviceClient is deactivated. Care must be taken to ensure that the eventReceive operation has no parameters (not even hidden ones!) so that the Return from Interrupt instruction will work properly. If the hidden me parameter is used as in the other patterns here, then making the operation static will ensure there is no hidden parameter because static operations are class-wide and not instance-wide functions (that is, they use no instance data and so don't require a me pointer).

If the interrupt vector table is not used, this pattern can be mixed with the Observer Pattern to distribute the event signal to multiple clients, if desired.

The DebouncingTimer may use special timer hardware such as a programmable 555 timer. If the RTOS (Real-Time Operating System) timer is used, care must be taken with the time unit resolution. Windows, for example, has a standard timer resolution of between 10 and 25 milliseconds. You can only get a delay time as a multiple of the basic timer resolution so if you want a 45 ms delay, you will have to use the closest timer resolution that is greater than or equal to your desired time. For debouncing applications, this is usually fine but your needs may vary with your specific hardware and application.

If you don't mind fully occupying your embedded processor while you wait for the debounce timeout, it is a simple matter, as shown in Code Listing 3-17.

去抖时间之后状态不同，那么该事件必定是真实的，因此它向应用客户端发送相应的消息。按钮的旧状态存储在其变量oldState中；每当检测到设备状态变化时，该变量就会被更新。

3.6.4.4 去抖定时器

该定时器通过其 `delay()` 服务提供非忙等待。这通常通过操作系统调用来完成，但也可能通过专用定时器硬件来实现。

3.6.5 结果

对于在宣布事件之前进行去抖的硬件，此模式并非必需。然而，根据我的经验，这项任务通常落在软件上。这是一个简单的模式，它执行去抖，使应用程序只需关注由设备状态变化产生的真实事件。

3.6.6 实现策略

设备使用中断向量来联系其客户端是很常见的。实际上，BouncingDevice完全由硬件实现也是很常见的，这样当事件发生时，就会产生一个硬件中断，并且必须通过读取内存位置或端口地址来验证设备状态。当事件接口通过中断实现时，在DeviceClient被初始化时，必须将DeviceClient::eventReceive()操作的地址安装到中断向量表中的适当位置，并且在DeviceClient被停用时必须移除该地址。必须注意确保eventReceive操作没有参数（甚至没有隐藏参数！），以便中断返回指令能够正常工作。如果像这里的其他模式那样使用了隐藏的me参数，那么将操作设为静态将确保没有隐藏参数，因为静态操作是类范围的函数，而不是实例范围的函数（也就是说，它们不使用实例数据，因此不需要me指针）。

如果不使用中断向量表，如果需要的话，此模式可以与观察者模式混合使用，以将事件信号分发给多个客户端。

去抖定时器可以使用特殊的定时器硬件，例如可编程555定时器。如果使用RTOS（实时操作系统）定时器，则必须注意时间单位分辨率。例如，Windows的标准定时器分辨率在10到25毫秒之间。你只能获得基本定时器分辨率倍数的延迟时间，因此如果你想要45毫秒的延迟，就必须使用大于或等于所需时间的最接近的定时器分辨率。对于去抖应用，这通常没有问题，但你的需求可能会因具体硬件和应用而异。

如果你不介意在等待去抖超时时完全占用嵌入式处理器，那就很简单，如代码清单3-17所示。

```
/* LOOPS_PER_MS is the # of loops in the delay() function
   required to hit 1 ms, so it is processor and
   compiler-dependent
*/
#define LOOPS_PER_MS 1000
void delay(unsigned int ms) {
    long nLoops = ms * LOOPS_PER_MS;
    do {
    while (nLoops-);
    }
```

Code Listing 3-17: Busy wait timing

3.6.7 Related Patterns

As mentioned, the pattern is often used in conjunction with the Interrupt Pattern discussed elsewhere in this chapter. Timeouts are often shown within a tm() event on the state machine.

3.6.8 Example

In the example shown in Figure 3-12, we are modeling a toggle button for a microwave oven; press it once to turn on the microwave emitter and press it again to turn it off. This means that we need to call different operations in the MicrowaveEmitter depending if this is an even-numbered press of the button or an odd-numbered press. In addition, we have to debounce both the press and release of the button, although we only act on a button release.

The code for the button driver is given in Code Listing 18 and Code Listing 19. #ifndef ButtonDriver_H
#define ButtonDriver_H

#define LOOPS_PER_MS (1000)
#define DEBOUNCE_TIME (40)

struct Button;
struct MicrowaveEmitter;
struct Timer;

typedef struct ButtonDriver ButtonDriver;
struct ButtonDriver {
        unsigned char oldState;
        unsigned char toggleOn;
        struct Button* itsButton;
        struct MicrowaveEmitter* itsMicrowaveEmitter;
        struct Timer* itsTimer;
};

/* LOOPS_PER_MS 是 delay() 函数中达到 1 ms 所需的循环次数，因此它取决于处理器和编译器 */ #define LOOPS_PER_MS 1000 void delay(unsigned int ms) { long nLoops = ms * LOOPS_PER_MS; do { while (nLoops–); }

代码清单3-17：忙等待定时

3.6.7 相关模式

如前所述，该模式通常与本章其他部分讨论的中断模式结合使用。超时通常显示在状态机上的tm()事件中。

3.6.8 示例

在图3-12所示的示例中，我们正在为微波炉建模一个切换按钮；按一次开启微波发射器，再按一次将其关闭。这意味着我们需要根据这是按钮的偶数次按下还是奇数次按下，在MicrowaveEmitter中调用不同的操作。此外，我们必须对按钮的按下和释放都进行去抖，尽管我们只对按钮释放做出响应。

按钮驱动程序的代码见代码清单 18 和代码清单 19。#ifndef ButtonDriver H_#define ButtonDriver_H # define LOOPS_PER_MS (1000) #define DEBOUNCE_TIME (40) struct Button; struct MicrowaveEmitter; struct Timer; typedef struct ButtonDriver ButtonDriver; struct ButtonDriver { unsigned char oldState; unsigned char toggleOn; struct Button* itsButton; struct MicrowaveEmitter* itsMicrowaveEmitter; struct Timer* itsTimer; };

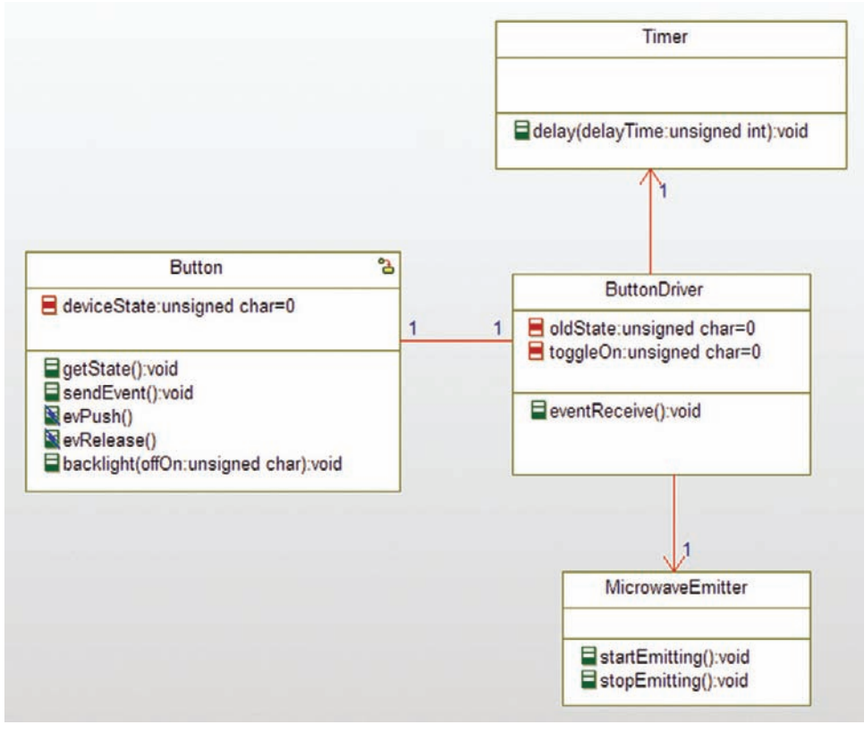


Figure 3-12 Debounce example

```
void ButtonDriver_Init(ButtonDriver* const me);
void ButtonDriver_Cleanup(ButtonDriver* const me);
/* Operations */ void ButtonDriver_eventReceive(ButtonDriver* const me);
struct Button* ButtonDriver_getItsButton(const ButtonDriver* const me);
void ButtonDriver_setItsButton(ButtonDriver* const me, struct Button* p_Button);
struct MicrowaveEmitter*
ButtonDriver_getItsMicrowaveEmitter(const ButtonDriver* const me);
void ButtonDriver_setItsMicrowaveEmitter(ButtonDriver* const me, struct MicrowaveEmitter*
p_MicrowaveEmitter);
struct Timer* ButtonDriver_getItsTimer(const ButtonDriver* const me);
void ButtonDriver_setItsTimer(ButtonDriver* const me, struct Timer* p_Timer);
ButtonDriver * ButtonDriver_Create(void);
void ButtonDriver_Destroy(ButtonDriver* const me);
void ButtonDriver___setItsButton(ButtonDriver* const me, struct Button* p_Button);
void ButtonDriver__setItsButton(ButtonDriver* const me, struct Button* p_Button);
void ButtonDriver__clearItsButton(ButtonDriver* const me);
#endif
```

Code Listing 3-18: ButtonDriver.h

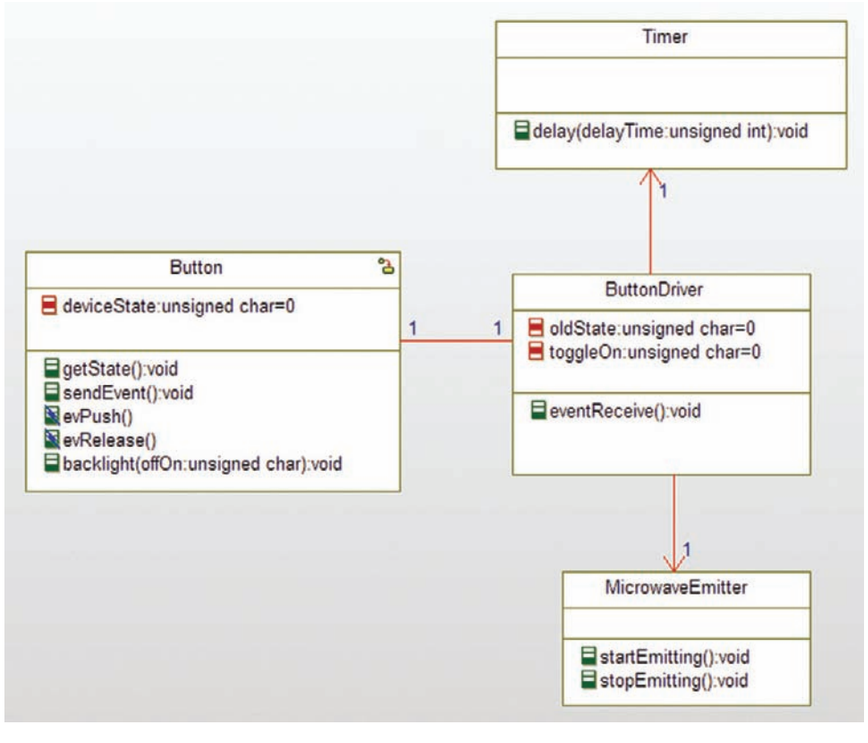


图 3-12 去抖示例

void ButtonDriver_Init(ButtonDriver* const me); void ButtonDriver_Cleanup(ButtonDriver* const me); /* 操作 */ void ButtonDriver_eventReceive(ButtonDriver* const me); struct Button* ButtonDriver_getItsButton(const ButtonDriver* const me); void ButtonDriver_setItsButton(ButtonDriver* const me, struct Button* p_Button); struct MicrowaveEmitter* ButtonDriver_getItsMicrowaveEmitter(const ButtonDriver* const me); void ButtonDriver_setItsMicrowaveEmitter(ButtonDriver* const me, struct MicrowaveEmitter* p_MicrowaveEmitter); struct Timer* ButtonDriver_getItsTimer(const ButtonDriver* const me); void ButtonDriver_setItsTimer(ButtonDriver* const me, struct Timer* p_Timer); ButtonDriver * ButtonDriver_Create(void); void ButtonDriver_Destroy(ButtonDriver* const me); void ButtonDriver___setItsButton(ButtonDriver* const me, struct Button* p_Button); void ButtonDriver__setItsButton(ButtonDriver* const me, struct Button* p_Button); void ButtonDriver__clearItsButton(ButtonDriver* const me); #endif

代码清单 3-18：ButtonDriver.h

```
#include "ButtonDriver.h"
#include "Button.h"
#include "MicrowaveEmitter.h"
#include "Timer.h"

static void cleanUpRelations(ButtonDriver* const me);
void ButtonDriver_Init(ButtonDriver* const me) {
        me->oldState = 0;
        me->toggleOn = 0;
        me->itsButton = NULL;
        me->itsMicrowaveEmitter = NULL;
        me->itsTimer = NULL;
}

void ButtonDriver_Cleanup(ButtonDriver* const me) {
        cleanUpRelations(me);
}

void ButtonDriver_eventReceive(ButtonDriver* const me) {
        Timer_delay(me->itsTimer, DEBOUNCE_TIME);
        if (Button_getState(me->itsButton) != me->oldState) {
            /* must be a valid button event */
            me->oldState = me->itsButton->deviceState;
            if (!me->oldState) {
                /* must be a button release, so update toggle value */
                if (me->toggleOn) {
                    me->toggleOn = 0; /* toggle it off */
                    Button_backlight(me->itsButton, 0);
                    MicrowaveEmitter_stopEmitting(me->itsMicrowaveEmitter);
                    }
            else {
                    me->toggleOn = 1; /* toggle it on */
                    Button_backlight(me->itsButton, 1);
                    MicrowaveEmitter_startEmitting(me->itsMicrowaveEmitter);
                    }
                }

            /* if it's not a button release, then it must
               be a button push, which we ignore.
            */
        }
}

struct Button* ButtonDriver_getItsButton(const ButtonDriver* const me) {
        return (struct Button*)me->itsButton;
}

void ButtonDriver_setItsButton(ButtonDriver* const me, struct Button* p_Button) {
        if(p_Button != NULL)
                {
                        Button__setItsButtonDriver(p_Button, me);
                }
```


#include "ButtonDriver.h" #include "Button.h" #include "MicrowaveEmitter.h" #include " Timer.h" static void cleanUpRelations(ButtonDriver* const me); void ButtonDriver_ Init(ButtonDriver* const me) { me->oldState = 0; me->toggleOn = 0; me->itsButton = NULL; me->itsMicrowaveEmitter = NULL; me->itsTimer = NULL; } void ButtonDriver_ Cleanup(ButtonDriver* const me) { cleanUpRelations(me); } void ButtonDriver_ eventReceive(ButtonDriver* const me) { Timer_delay(me->itsTimer, DEBOUNCE_TIME); if (Button_getState(me->itsButton) != me->oldState) { /* 必须是一个有效的按钮事件 */ me-> oldState = me->itsButton->deviceState; if (!me->oldState) { /* 必须是按钮释放，因此更新切换值 */ if (me->toggleOn) { me->toggleOn = 0; /* 将其切换为关闭 */ Button_backlight(me->itsButton, 0); MicrowaveEmitter_stopEmitting(me->itsMicrowaveEmitter); } else { me->toggleOn = 1; /* 将其切换为开启 */ Button_backlight(me->itsButton, 1); MicrowaveEmitter_startEmitting(me-> itsMicrowaveEmitter); } } /* 如果不是按钮释放，则必定是按钮按下，我们将其忽略。 */ } } struct Button* ButtonDriver_getItsButton(const ButtonDriver* const me) { return (struct Button*)me->itsButton; } void ButtonDriver_setItsButton(ButtonDriver* const me, struct Button* p_Butto

n) {
if(p_Button != 空) { Button__setItsButtonDriver(p_Button, me); }

```
        ButtonDriver__setItsButton(me, p_Button);
}
struct MicrowaveEmitter*
ButtonDriver_getItsMicrowaveEmitter(const ButtonDriver* const me)
{
        return (struct MicrowaveEmitter*)me->itsMicrowaveEmitter;
}
void ButtonDriver_setItsMicrowaveEmitter(ButtonDriver* const me, struct MicrowaveEmit-
ter* p_MicrowaveEmitter) {
        me->itsMicrowaveEmitter = p_MicrowaveEmitter;
}
struct Timer* ButtonDriver_getItsTimer(const ButtonDriver* const me) {
        return (struct Timer*)me->itsTimer;
}
void ButtonDriver_setItsTimer(ButtonDriver* const me, struct Timer* p_Timer) {
        me->itsTimer = p_Timer;
}
ButtonDriver * ButtonDriver_Create(void) {
        ButtonDriver* me = (ButtonDriver *)
        malloc(sizeof(ButtonDriver));
        if(me!=NULL)
                {
                        ButtonDriver_Init(me);
                }
        return me;
}
void ButtonDriver_Destroy(ButtonDriver* const me) {
        if(me!=NULL)
                {
                        ButtonDriver_Cleanup(me);
                }
        free(me);
}
static void cleanUpRelations(ButtonDriver* const me) {
        if(me->itsButton != NULL)
                {
                    struct ButtonDriver* p_ButtonDriver = Button_getItsButtonDriver(me-
>itsButton);
                    if(p_ButtonDriver != NULL)
                             {
                                    Button___setItsButtonDriver(me->itsButton, NULL);
                             }
                      me->itsButton = NULL;
                }
        if(me->itsMicrowaveEmitter != NULL)
                {
                        me->itsMicrowaveEmitter = NULL;
                }
```


ButtonDriver__setItsButton(me, p_Button); } 结构体 MicrowaveEmitter* ButtonDriver_getItsMicrowaveEmitter(const ButtonDriver* const me) { return (结构体 MicrowaveEmitter*)me->itsMicrowaveEmitter; } void ButtonDriver_setItsMicrowaveEmitter(ButtonDriver* const me, 结构体 MicrowaveEmi

t-

ter* p_MicrowaveEmitter) { me->itsMicrowaveEmitter = p_MicrowaveEmitter; } struct Timer* ButtonDriver_getItsTimer(const ButtonDriver* const me) { return (struct Timer*)me->itsTimer; } void ButtonDriver_setItsTimer(ButtonDriver* const me, struct Timer* p_Timer) { me->itsTimer = p_Timer; } ButtonDriver * ButtonDriver_Create(void) { ButtonDriver* me = (ButtonDriver *) malloc(sizeof(ButtonDriver)); if(me!=NULL) { ButtonDriver_Init(me); } return me; } void ButtonDriver_Destroy(ButtonDriver* const me) { if(me!=NULL) { ButtonDriver_Cleanup(me); } free(me); } static void cleanUpRelations(ButtonDriver* const me) { if(me->itsButton != NULL) { struct ButtonDriver* p_ButtonDriver = Button_getItsButtonDriver(m

e-

>itsButton); if(p_ButtonDriver != 空) { Button___setItsButtonDriver(me->itsButton, 空

);

} me->itsButton = 空; } if(me->itsMicrowaveEmitter != 空) { me->itsMicrowaveEmitter = 空; }

```
        if(me->itsTimer != NULL)
                {
                        me->itsTimer = NULL;
                }
}
void ButtonDriver___setItsButton(ButtonDriver* const me, struct Button* p_Button) {
        me->itsButton = p_Button;
}
void ButtonDriver__setItsButton(ButtonDriver* const me, struct Button* p_Button) {
        if(me->itsButton != NULL)
                {
                        Button___setItsButtonDriver(me->itsButton, NULL);
                }
        ButtonDriver___setItsButton(me, p_Button);
}
void ButtonDriver__clearItsButton(ButtonDriver* const me) {
        me->itsButton = NULL;
}
```

Code Listing 3-19: ButtonDriver.c

3.7 Interrupt Pattern

The physical world is fundamentally both concurrent and asynchronous; it's nonlinear too, but that's a different story. Things happen when they happen and if your embedded system isn't paying attention, those occurrences may be lost. Interrupt handlers (a.k.a. Interrupt Service Routines, or ISRs) are a useful way to be notified when an event of interest occurs even if your embedded system is off doing other processing.

3.7.1 Abstract

The Interrupt Pattern is a way of structuring the system to respond appropriately to incoming events. It does require some processor- and compiler-specific services but most embedded compilers, and even some operating systems provide services for just this purpose. Once initialized, an interrupt will pause its normal processing, handle the incoming event, and then return the system to its original computations.

3.7.2 Problem

In many systems, events have different levels of urgency[14]. Most embedded systems have at least some events with a high urgency that must be handled even when the system is busy doing

[14] Urgency is a measure of the rapidity with which an event must be handled, usually expressed as the nearness of a deadline. This topic will be discussed in more detail in Chapter 4.

if(me->itsTimer != 空) { me->itsTimer = 空; } } void ButtonDriver___setItsButton(ButtonDriver* const me, struct Button* p_Button) { me->itsButton = p_Button; } void ButtonDriver__setItsButton(ButtonDriver* const me, struct Button* p_Button) { if(me->itsButton != 空) { Button___setItsButtonDriver(me->itsButton, 空); } ButtonDriver___setItsButton(me, p_Button); } void ButtonDriver__clearItsButton(ButtonDriver* const me) { me->itsButton = 空; }

代码清单 3-19：ButtonDriver.c

3.7 中断模式

物理世界本质上是并发且异步的；它也是非线性的，但那是另一回事。事情在它们发生时发生，如果你的嵌入式系统没有关注，那些事件可能会丢失。中断处理程序（也称为中断服务例程，或ISR）是一种有用的方式，可以在感兴趣的事件发生时得到通知，即使你的嵌入式系统正在忙于其他处理。

3.7.1 摘要

中断模式是一种结构化系统以适当响应传入事件的方式。它确实需要一些处理器和编译器特定的服务，但大多数嵌入式编译器，甚至一些操作系统都为此目的提供了服务。一旦初始化，中断将暂停其正常处理，处理传入事件，然后将系统返回到其原始计算。

3.7.2 问题

在许多系统中，事件的紧迫性程度各不相同[14]。大多数嵌入式系统至少会有一些高紧迫性的事件，即使系统正忙于进行

[14] 紧迫性是衡量事件必须被处理的速度的指标，通常以截止时间的临近程度来表示。该主题将在第4章中更详细地讨论。

other processing. The Polling Pattern, discussed elsewhere in this chapter, looks for events of interest when it is convenient for the system. While this has the advantage that primary processing can proceed uninterrupted, it has the disadvantage that high urgency and high frequency events may not be handled in a timely fashion, or may be missed altogether. The Interrupt Pattern addresses this problem by immediately pausing the current processing, handling the incoming event, and then returning to the original computation.

3.7.3 Pattern Structure

Figure 3-13 shows the basic structure of the pattern. Because it is important to ensure that the interrupt handlers are parameterless, I've used the «File» stereotype to indicate that this pattern isn't using the class-structured approach with the me pointer.

3.7.4 Collaboration Roles

This section describes the roles for this pattern.

3.7.4.1 InterruptHandler

The interruptHandler is the only element that has behavior in the pattern. It has functions to install or deinstall an interrupt vector and the interrupt service routines themselves.

The install() function takes the interrupt number as an incoming parameter. When run, it copies the existing vector into the vector table and then replaces it with the address of the appropriate interrupt service routine. The deinstall() function does the reverse, restoring the original vector.

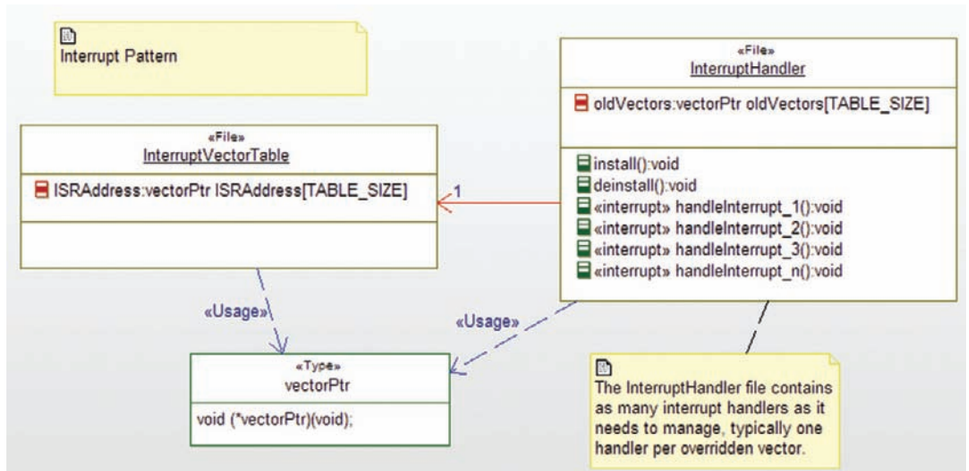


Figure 3-13 Interrupt Pattern

其他处理时，也必须处理这些事件。本章其他部分讨论的轮询模式会在系统方便时查找感兴趣的事件。虽然这样做的好处是主处理可以不受中断地进行，但缺点是高紧迫性和高频率的事件可能无法及时处理，或者可能被完全遗漏。中断模式通过立即暂停当前处理、处理传入事件，然后返回到原来的计算来解决这个问题。

3.7.3 模式结构

图3-13展示了该模式的基本结构。由于确保中断处理程序无参数非常重要，我使用了«File»构造型来表明该模式并未采用带有me指针的类结构化方法。

3.7.4 协作角色

本节描述该模式的角色。

3.7.4.1 InterruptHandler

InterruptHandler 是该模式中唯一具有行为的元素。它具有 install 或 deinstall 中断向量的函数，以及中断服务例程本身。

install() 函数将中断号作为传入参数。运行时，它将现有向量复制到向量表中，然后用相应中断服务例程的地址替换它。deinstall() 函数执行相反的操作，恢复原始向量。

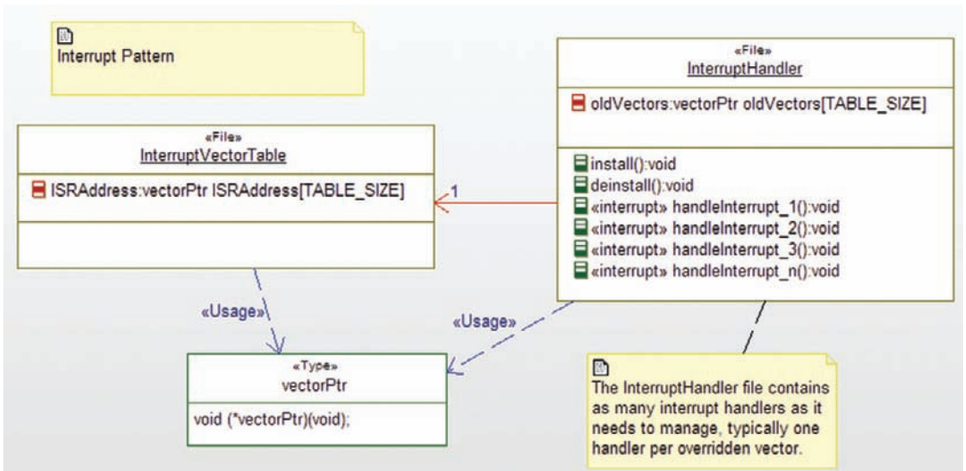


图 3-13 中断模式

Each handleIinterrupt_x() function handles a specific interrupt, completing with a Return From Interrupt (RTI) statement. This statement is compiler and processor dependent. As mentioned, it is crucial that the interrupt service routines have no parameters because otherwise when they attempt to return, the wrong value will be popped off the CPU stack.

3.7.4.2 InterruptVectorTable

The InterruptVectorTable is nothing more than an array of addresses to interrupt service routines. It is located in a specific memory location that is processor dependent. When interrupt number x occurs, the CPU suspends the current processing and indirectly invokes the address corresponding to the x-th index in this table. Upon an RTI, the CPU restores execution of the suspending computation.

3.7.4.3 vectorPtr

The vectorPtr is a data type; specifically it is a pointer to a function that takes no parameters and returns no values.

3.7.5 Consequences

This pattern allows for highly responsive processing of events of interest. It interrupts normal processing (provided that interrupts have not been disabled) and so should be used carefully when time-critical processing is going on. Normally, interrupts are disabled while an interrupt service routine is executing; this means that interrupt service routines must execute very quickly to ensure that other interrupts are not missed.

Because the interrupt service routines must be short, one must take care when using them to invoke other system services. To share data signaled by an interrupt, for example, the ISR might need to queue the data and quickly RTI; at some point in the future, the application software will discover the data in the queue. This mechanism is useful when the actual acquisition of the data is more urgent than its processing. In this way, a long interrupt handle can be broken up into two parts; the urgent part, done via the ISR *per se*, and the processing part done via a second function that periodically checks for waiting data or signals.

Problems arise with this pattern when the ISR processing takes too long, when an implementation mistake leaves interrupts disabled, or race conditions or deadlocks occur on shared resources. The last of these concerns is the most insidious.

A race condition is a computational situation in which the result depends on the order of execution of statements but that order isn't known or knowable. A deadlock is a situation in which two elements are waiting on a condition that cannot in principle occur. These are common problems that will be discussed in more detail in the next chapter. In this context, a variable or data structure (such as a queue) shared between an interrupt service routine and an application service is a resource with potential race and deadlock conditions because you can never know exactly when the ISR will be executed.

每个 handleIinterrupt_x() 函数处理一个特定的中断，并以中断返回 (RTI) 语句结束。该语句依赖于编译器和处理器。如前所述，中断服务例程没有参数至关重要，否则当它们尝试返回时，错误值将从 CPU 栈中弹出。

3.7.4.2 中断向量表

中断向量表不过是一个指向中断服务例程的地址数组。它位于一个特定的内存位置，该位置取决于处理器。当发生中断号 x 时，CPU 暂停当前的处理，并间接调用该表中第 x 个索引对应的地址。在 RTI 之后，CPU 恢复被暂停的计算的执行。

3.7.4.3 vectorPtr

vectorPtr 是一种数据类型；具体来说，它是一个函数指针，该函数不接受参数也不返回任何值。

3.7.5 结果

该模式允许对关注的事件进行高响应性处理。它会中断正常处理（前提是中断未被禁用），因此在时间关键的处理正在进行时应谨慎使用。通常，在中断服务例程执行期间会禁用中断；这意味着中断服务例程必须执行得非常快，以确保不会遗漏其他中断。

由于中断服务例程必须很短，在使用它们调用其他系统服务时必须小心。例如，为了共享由中断信号通知的数据，ISR可能需要将数据排队并快速RTI；在将来的某个时刻，应用软件将在队列中发现该数据。当数据的实际获取比其处理更紧急时，这种机制很有用。通过这种方式，一个长的中断处理可以被分解为两部分：紧急部分通过ISR本身完成，处理部分通过第二个函数完成，该函数周期性地检查等待的数据或信号。

当ISR处理耗时过长、实现错误导致中断被禁用，或共享资源上发生竞态条件或死锁时，该模式就会出现问题。最后一项担忧是最隐蔽的。

竞态条件是一种计算情境，其中结果取决于语句的执行顺序，但该顺序未知或不可知。死锁是一种情境，其中两个元素在等待一个原则上不可能发生的条件。这些是常见问题，将在下一章中更详细地讨论。在此上下文中，在中断服务例程和应用服务之间共享的变量或数据结构（如队列）是一种具有潜在竞态和死锁条件的资源，因为你永远无法确切知道ISR何时会被执行。

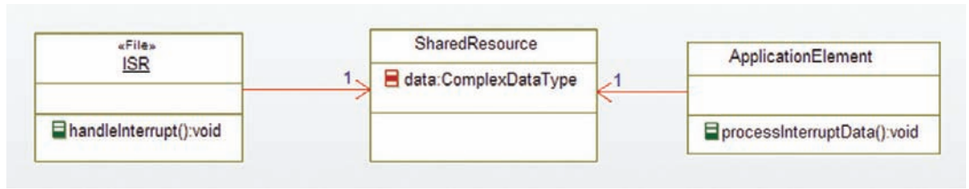


Figure 3-14 ISR potential race condition

Figure 3-14 shows a typical structure when the data acquired as a part of an ISR must be shared with normal system processing. The race condition arises when the interrupt occurs when ApplicationElement is accessing the SharedResource. Imagining that the ApplicationElement is halfway through reading the data when the interrupt occurs – unless the read takes place as an atomic uninterruptible access – the ISR will pause the ApplicationElement mid-read, modify the data, and then return. The ApplicationElement will see corrupted data – partially new data and partially old data.

There are a number of solutions to this, but they all involve serializing the access to the shared resource. One way is to disable interrupts in the ApplicationElement just prior to reading the data and to reenable interrupt just after access is complete. The primary downside to that approach is the discipline necessary to ensure that this is always done. Another approach is to use a mutex semaphore to guard the data, as shown in Figure 3-15.

In this figure, the SharedResource is protected with a mutex semaphore; this results in a lock when either the getData() or setData() functions are called and in the removal of the lock when the functions are complete. Deadlock can occur if the ISR waits on the semaphore lock when it tries to access the data. Since the ISR has interrupted the ApplicationElement that owns the

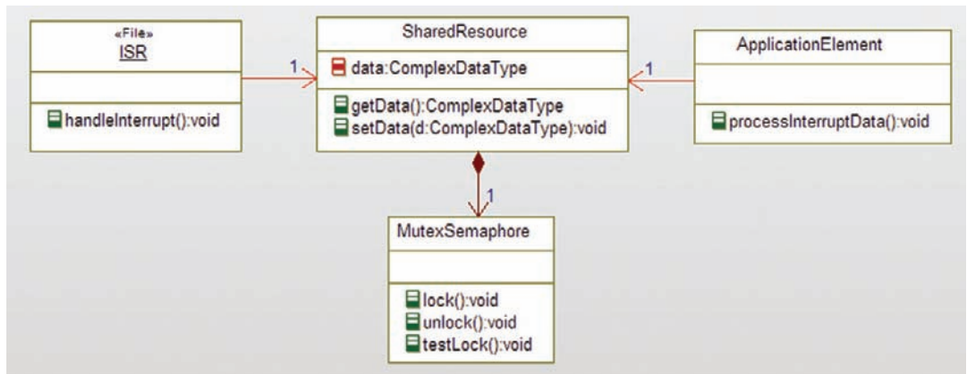


Figure 3-15 ISR potential deadlock condition

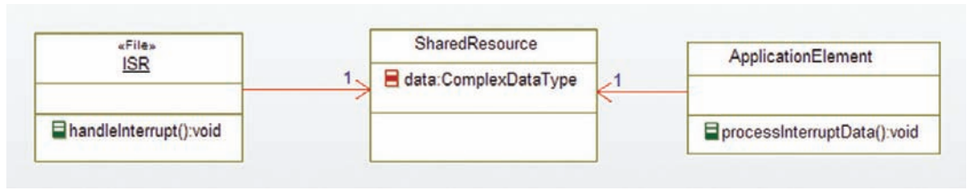


图 3-14 ISR 潜在的竞态条件

图3-14展示了当作为ISR一部分获取的数据必须与正常系统处理共享时的典型结构。当ApplicationElement正在访问SharedResource时发生中断，就会产生竞态条件。设想ApplicationElement在中断发生时读取数据进行到一半——除非读取以原子不可中断访问的方式进行——ISR将暂停ApplicationElement的读取过程，修改数据，然后返回。ApplicationElement将看到损坏的数据——部分新数据和部分旧数据。

针对这一问题有多种解决方案，但它们都涉及串行化对共享资源的访问。一种方法是在 ApplicationElement 中于读取数据之前禁用中断，并在访问完成后立即重新启用中断。该方法的主要缺点是需要严格的纪律来确保始终这样做。另一种方法是使用互斥信号量来守卫数据，如图 3-15 所示。

在此图中，SharedResource 由互斥信号量保护；当调用 getData() 或 setData() 函数时，这会导致加锁，并在函数完成时移除锁。如果中断服务例程在尝试访问数据时等待信号量锁，则可能发生死锁。由于中断服务例程中断了拥有

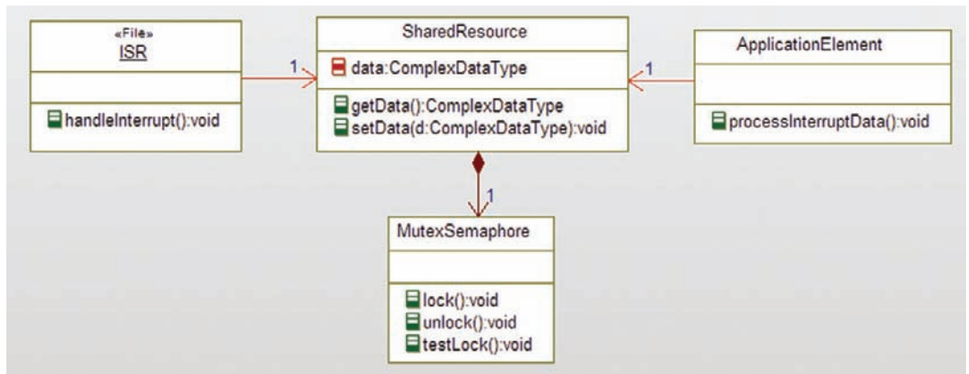


图 3-15 ISR 潜在的死锁条件

lock, if the ISR waits, the ApplicationElement will never have the opportunity to remove the lock. The solution of course, is that the ISR cannot wait on the lock. The new data can be discarded or two shared resources can be made with the proviso that the ISR or ApplicationElement can only lock one of the resources at a time. This latter solution is sometimes referred to as a "ping pong buffer."

3.7.6 Implementation Strategies

An ISR is different than a normal C function in that as long as interrupts are enabled, it can pretty much interrupt between any pair of CPU instructions. That means that the CPU registers must be saved and restored before the ISR statements can execute.

In fact, each interrupt service routine must:

- save the CPU registers, including the CPU instruction pointer and any processor flags, such as carry, parity and zero
- clear the interrupt bit
- perform the appropriate processing
- restore the CPU registers
- return

In "standard C," it is necessary to use assembly language keyword `asm` to save and restore registers, something like[15]:

```
void isr (void) {
    asm {
          DI            ; disable interrupts
          PUSH    AF    ; save registers
          PUSH    BC
          PUSH    DE
          PUSH    HL
          PUSH    IX
          PUSH    IY
          }
    /* normal C code here */
    asm {
          POP     IY
          POP     IX
          POP     HL
          POP     DE
          POP     BC
          POP     AF
          EI            ; enable interrupts
```

[15] This is Z80 assembly language (my personal favorite). Your processor mileage may vary.

锁的 ApplicationElement，如果中断服务例程等待，ApplicationElement 将永远没有机会移除锁。当然，解决方案是中断服务例程不能等待锁。新数据可以被丢弃，或者可以创建两个共享资源，条件是中断服务例程或 ApplicationElement 一次只能锁定其中一个资源。后一种解决方案有时被称为"乒乓缓冲区"。

3.7.6 实现策略

中断服务例程与普通C函数的不同之处在于，只要中断处于enable状态，它几乎可以在任意一对CPU指令之间产生中断。这意味着在执行中断服务例程的语句之前，必须保存并恢复CPU寄存器。

事实上，每个中断服务例程必须：

- 保存CPU寄存器，包括CPU指令指针以及任何处理器标志，如进位、奇偶和归零标志
- 清除中断位
- 执行适当的处理
- 恢复CPU寄存器
- 返回

在 "标准 C" 中，需要 使用 汇编 语言 关键字 asm 来 保存 和 恢复 寄存器，类似 这样：[15]：

void isr (void) { asm { DI ; 禁用中断 PUSH AF ; 保存寄存器 PUSH BC PUSH DE PUSH HL PUSH IX PUSH IY } /* 此处为普通C代码 */ asm { POP IY POP IX POP HL POP DE POP BC POP AF EI ; enable 中断

[15] 这是Z80汇编语言（我个人的最爱）。你的处理器情况可能有所不同 ary.

```
            RETI        ; return from interrupt
            }
    }
```

Some compilers support this by using a special `interrupt` keyword. When available, the keyword is used in the function declaration such as

```
interrupt void isr(void) {
    /* normal C code here */
};
```

and the compiler will insert the necessary instructions for you.

The GNU C Compiler (GCC) uses `__attribute__` specification to indicate an interrupt handler using the syntax

```
void isr(void) __attribute__ ((interrupt ("IRQ")));
void isr(void) {
    /* normal C code here */
}
```

but again this is processor dependent.

The interrupt keyword tells the compiler to generate register-saving instructions before the body of the function is executed and to restore the registers just before the ISRs return.

Many RTOSs provide functions for installing the vectors and some CPUs only have a single vector, requiring the ISR to figure out which interrupt occurred by checking the hardware.

3.7.7 Related Patterns

An alternative means to get hardware signals and data is to periodically check for them via the Polling Pattern.

3.7.8 Example

In the example shown in Figure 3-16, a button is the hardware device. One interrupt (index 0) is generated for the push and another (index 1) is generated for the release. The ButtonHandler is the interrupt handler for the interrupts and its install() functions set up the vectors for both pointers. The main() routine does the following:

- initializes the Button and LED classes
- sets all the elements of the RobotInterruptVectorTable and ButtonHandler::oldVectors to NULL
- sets the pointer from the ButtonHandler to the LED
- calls the install() function to set up the vectors to the two ISRs


```
            RETI ; 中断返回 } }
```

一些编译器通过使用特殊的中断 关键字来支持这一点。当可用时，该关键字用于函数声明中，例如

```
interrupt void isr(void) { /* 此
处为普通C代码 */ };
```

编译器将为你插入必要的指令。

GNU C编译器（GCC）使用__属性__ 规范来指示中断处理程序，语法为

```
void isr(void) __attribute__ ((interrupt ("IRQ"))); void
isr(void) { /* 此处为普通C代码 */ }
```

但这同样取决于处理器。

中断关键字告诉编译器在函数体执行之前生成保存寄存器的指令，并在ISR返回之前恢复寄存器。

许多实时操作系统提供了安装向量的函数，而某些CPU只有一个向量，需要ISR通过检查硬件来确定发生了哪个中断。

3.7.7 相关模式

获取硬件信号和数据的另一种方式是通过轮询模式定期检查它们。

3.7.8 示例

在图3-16所示的示例中，Button是硬件设备。推送时产生一个中断（索引0），释放时产生另一个中断（索引1）。ButtonHandler是这些中断的中断处理程序，其install()函数为两个指针设置向量。main()例程执行以下操作：

- 初始化Button和LED类
- 将RobotInterruptVectorTable和ButtonHandler::oldVectors的所有元素设置为空

- 设置从ButtonHandler到LED的指针
- 调用 install() 函数，将向量设置为两个 ISR

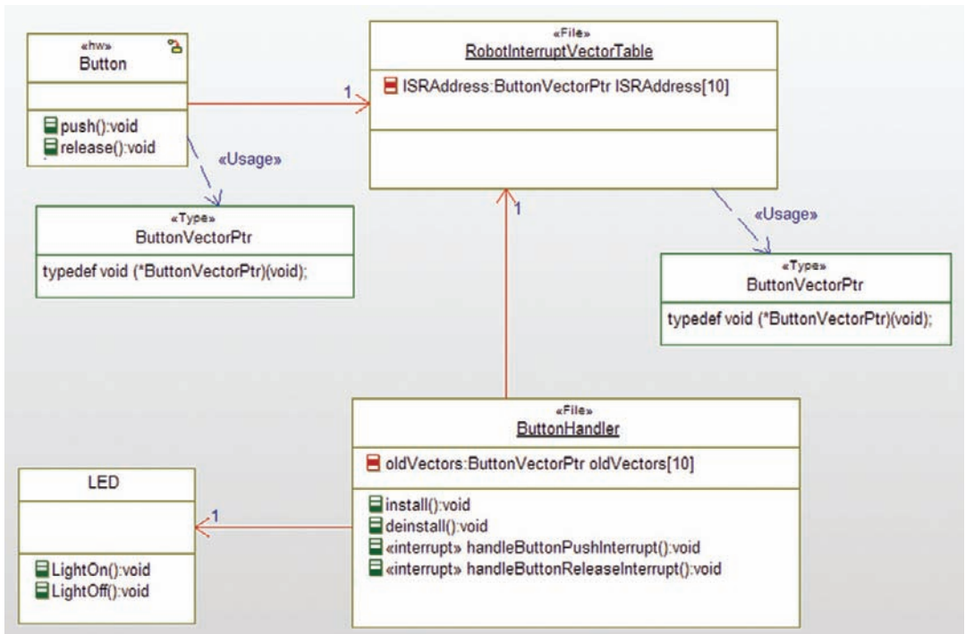


Figure 3-16 Interrupt Example

as you can see in Code Listing 20

```
int j;
Button itsButton;
LED itsLED;
itsButton = Button_Create();
itsLED = LED_Create();
for (j=0;j<9;j++) {
    ISRAddress[j] = NULL;
    oldVectors[j] = NULL;
    };
ButtonHandler_setItsLED(&itsLED);
install(); /* install interrupt vectors */
/* normal system execution stuff */
```

Code Listing 20: Interupt example main() code

The interesting part is the ButtonHandler itself. It installs the address of the two ISRs into positions 0 and 1 of the interrupt vector table so that when the interrupts occur, the appropriate

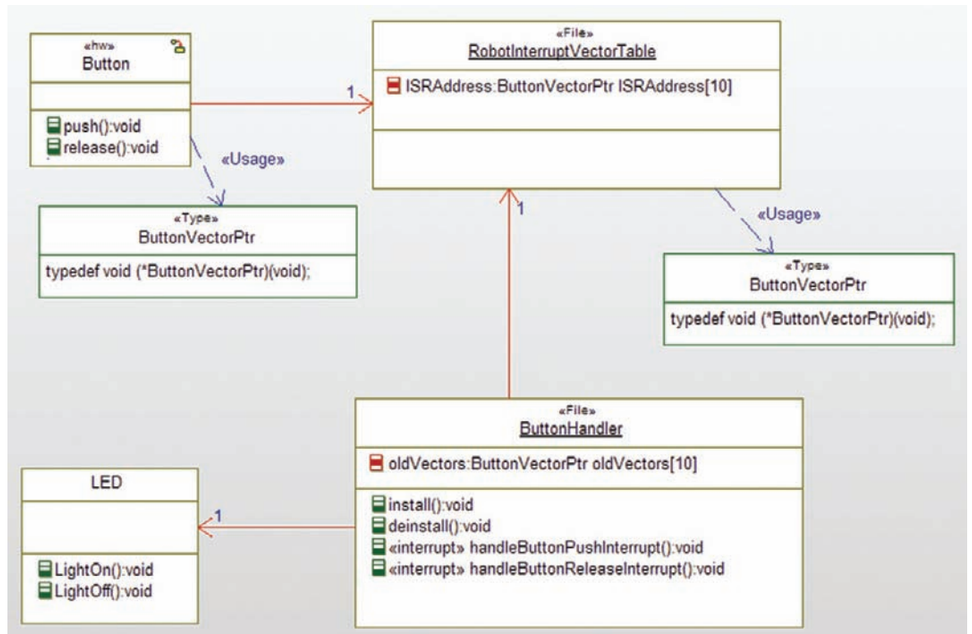


图3-16 中断示例

如你在代码清单20中所见

```
int j; Button itsButton; LED itsLED; itsButton =
Button_Create(); itsLED = LED_Create(); for (j=0;
j<9;j++) { ISRAddress[j] = NULL; oldVectors[j] =
NULL; }; ButtonHandler_setItsLED(&itsLED);
install(); /* 安装中断向量 */ /* 正常系统执行代码 */
```

代码清单20：中断示例 main() 代码

有趣的部分是 ButtonHandler 本身。它将两个 ISR 的地址安装到中断向量表的位置 0 和 1 中，以便当中断发生时，相应的

ISR is invoked. In the example, when the button is pushed, the ISR lights the LED and turns off the light when the button is released.

Note that this code uses the interrupt keyword to indicate to the compiler the interrupt service routines; you will have to use the appropriate syntax provided by your compiler. To execute this on standard desktop hardware, simply remove the interrupt keyword from the ISR's function headers and then the interrupt vector table will operate as a table of normal function pointers.

```
#ifndef ButtonHandler_H
#define ButtonHandler_H

typedef void (*ButtonVectorPtr)(void);
struct LED;

extern ButtonVectorPtr oldVectors[10];
/* Operations */
void install(void);
void deinstall(void);

interrupt void handleButtonPushInterrupt(void);
interrupt void handleButtonReleaseInterrupt(void);
struct LED* ButtonHandler_getItsLED(void); void ButtonHandler_setItsLED(struct
LED* p_LED);
#endif
```

Code Listing 21: ButtonHandler.h

```
#include "ButtonHandler.h"
#include "LED.h"
#include "RobotInterruptVectorTable.h"

ButtonVectorPtr oldVectors[10];
static struct LED* itsLED;
void deinstall(void) {
        ISRAddress[0] = oldVectors[0];
        ISRAddress[1] = oldVectors[1];
}

interrupt void handleButtonPushInterrupt(void) {
        LED_LightOn(itsLED);
}

interrupt void handleButtonReleaseInterrupt(void) {
        LED_LightOff(itsLED);
}

void install(void) {
        oldVectors[0] = ISRAddress[0];
        oldVectors[1] = ISRAddress[1];
        ISRAddress[0] = handleButtonPushInterrupt;
        ISRAddress[1] = handleButtonReleaseInterrupt;
}
```


ISR 被调用。在示例中，当按钮被按下时，ISR 点亮 LED，并在按钮释放时关闭指示灯。

请注意，此代码使用 interrupt 关键字向编译器指示中断服务例程；您必须使用编译器提供的适当语法。要在标准桌面硬件上执行此代码，只需从 ISR 的函数头中移除 interrupt 关键字，然后中断向量表将作为普通函数指针表运行。

```
#ifndef ButtonHandler_H #
define ButtonHandler_H

typedef void (*ButtonVectorPtr)(void); 结构体
LED;

extern ButtonVectorPtr oldVectors[10]; /* 操
作 */ void install(void); void deinstall(void);


中断 void handleButtonPushInterrupt(void); 中断 void handleButtonReleaseInterrupt(void); 结
构体 LED* ButtonHandler_getItsLED(void); void ButtonHandler_setItsLED(结构体 LED* p_
LED); #endif
```

代码清单21：ButtonHandler.h

```
#include "ButtonHandler.h" #include "LED.h" #include "
RobotInterruptVectorTable.h" ButtonVectorPtr oldVectors[10];
static struct LED* itsLED; void deinstall(void) { ISRAddress
[0] = oldVectors[0]; ISRAddress[1] = oldVectors[1]; }
interrupt void handleButtonPushInterrupt(void) { LED_
LightOn(itsLED); } interrupt void
handleButtonReleaseInterrupt(void) { LED_LightOff(itsLED); }
void install(void) { oldVectors[0] = ISRAddress[0]; oldVectors
[1] = ISRAddress[1]; ISRAddress[0] =
handleButtonPushInterrupt; ISRAddress[1] =
handleButtonReleaseInterrupt; }
```

```
struct LED* ButtonHandler_getItsLED(void) {
        return (struct LED*)itsLED;
}

void ButtonHandler_setItsLED(struct LED* p_LED) {
        itsLED = p_LED;
}
```

Code Listing 22: ButtonHandler.c

3.8 Polling Pattern

Another common pattern for getting sensor data or signals from hardware is to check periodically, a process known as *polling*. Polling is useful when the data or signals are not so urgent that they cannot wait until the next polling period to be received or when the hardware isn't capable of generating interrupts when data or signals become available.

3.8.1 Abstract

The Polling Pattern is the simplest way to check for new data or signals from the hardware. Polling can be periodic or opportunistic; periodic polling uses a timer to indicate when the hardware should be sampled whereas opportunistic polling is done when it is convenient for the system, such as between major system functions or at some point in a repeated execution cycle. Opportunistic polling is, by definition, less regular but has less impact on the timeliness of other activities in which the system may be engaged.

3.8.2 Problem

The Polling Pattern addresses the concern of getting new sensor data or hardware signals into the system as it runs when the data or events are not highly urgent and the time between data sampling can be guaranteed to be fast enough.

3.8.3 Pattern Structure

This pattern comes in two flavors. Figure 3-17 shows the pattern structure for opportunistic polling while Figure 3-18 shows the pattern for periodic polling. The difference is that in the former pattern the applicationFunction will embed calls to poll() when convenient, and in the latter, a timer is created to start polling.

3.8.4 Collaboration Roles

This section describes the roles for both variants of this pattern.

struct LED* ButtonHandler_getItsLED(void) { return (struct LED*)itsLED; } void ButtonHandler_setItsLED(struct LED* p_LED) { itsLED = p_LED; }

代码清单 22：ButtonHandler.c

3.8 轮询模式

另一种从硬件获取传感器数据或信号的常见模式是定期检查，这一过程称为轮询。当数据或信号并不紧急，可以等到下一个轮询周期再接收时，或者当硬件无法在数据或信号可用时产生中断时，轮询就很有用。

3.8.1 摘要

轮询模式是检查硬件是否有新数据或信号的最简单方式。轮询可以是周期性的或机会性的；周期性轮询使用Timer来指示何时应对硬件进行采样，而机会性轮询则在系统方便时进行，例如在主要系统函数之间或在重复执行周期的某个点。机会性轮询按定义不太规律，但对系统可能正在从事的其他活动的及时性影响较小。

3.8.2 问题

轮询模式解决的是在系统运行时将新的传感器数据或硬件信号引入系统的问题，适用于数据或事件并非高度紧急、且能够保证数据采样间隔足够快的情况。

3.8.3 模式结构

该模式有两种形式。图 3-17 展示了机会性轮询的模式结构，而图 3-18 展示了周期性轮询的模式。区别在于，前一种模式中应用函数会在方便时嵌入对 poll() 的调用，而后一种模式中则创建一个 Timer 来启动轮询。

3.8.4 协作角色

本节描述此模式两种变体的角色。

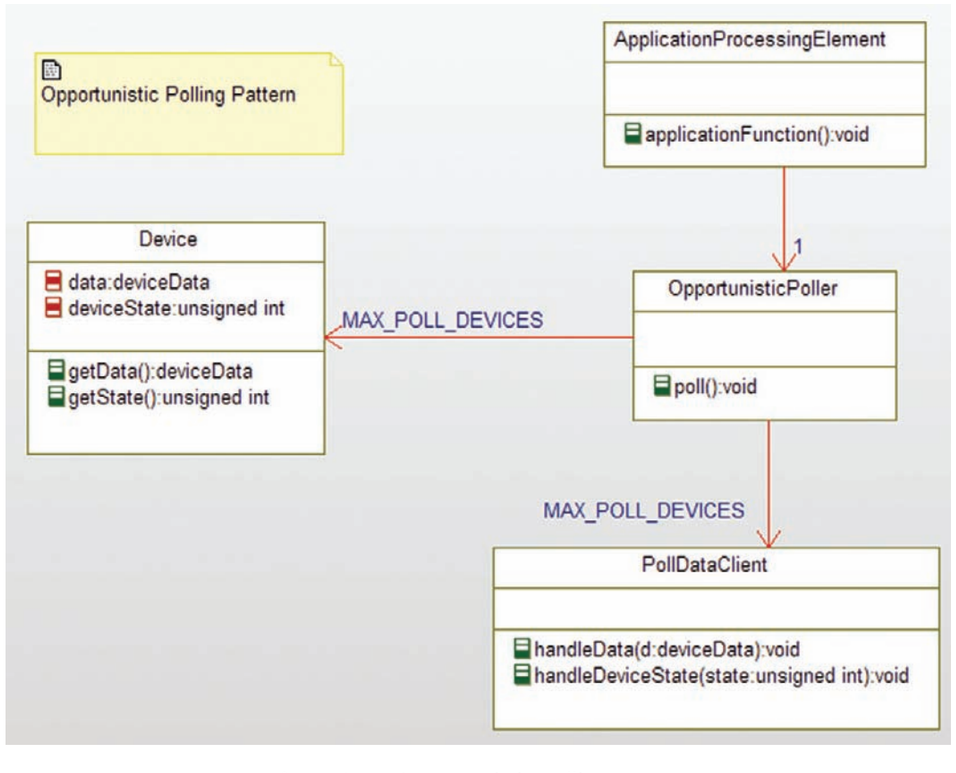


Figure 3-17 Opportunistic Polling Pattern

3.8.4.1 ApplicationProcessingElement

This elements has the applicationFunction that has a loop in which it invokes the poll() operation, such as

```
while(processing){
      action1();
      action2();
      action3();
      OpportunityPoller_poll(me->itsOpportunisticPoller);
}
```

3.8.4.2 Device

The *device* provides data and/or device state information via accessible functions. This element can be a device driver or can simply read from memory- or port-mapped devices. This class provides two functions, one to retrieve data (typed as deviceData in the example) and one to retrieve device state (typed as an unsigned int in the example).

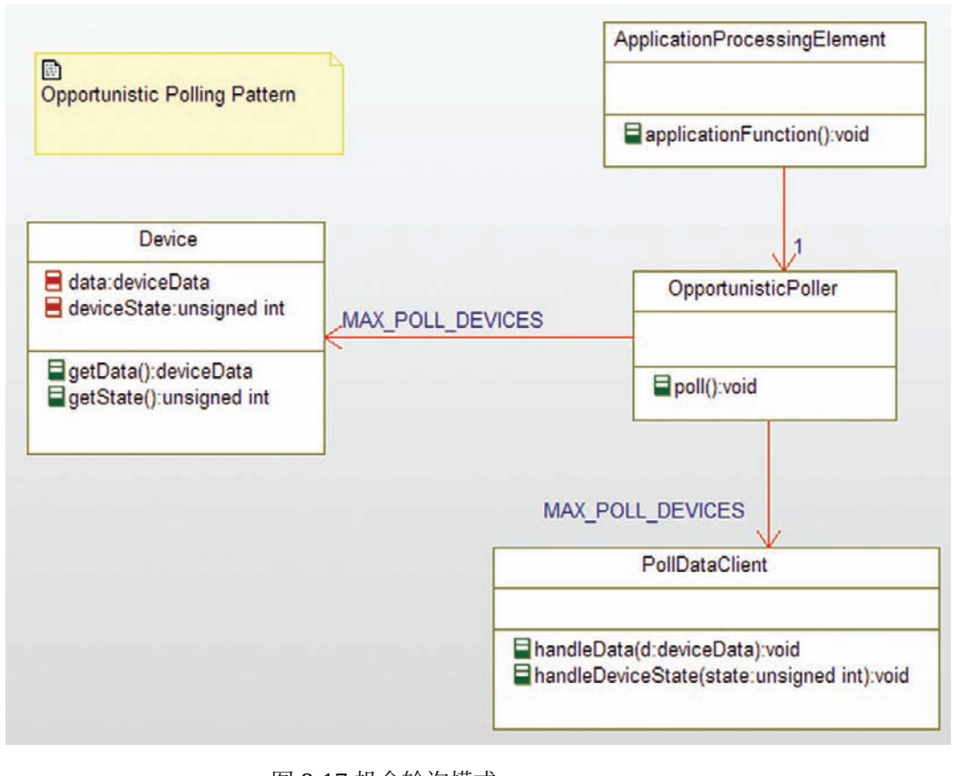


图 3-17 机会轮询模式

3.8.4.1 ApplicationProcessingElement

此元素具有应用函数，该函数包含一个循环，在循环中调用 poll() 操作，例如

```
while (processing) { action1(); action2(); action3(); OpportunityPoller_
poll(me->itsOpportunisticPoller); }
```

3.8.4.2 设备

设备通过可访问的函数提供数据和/或设备状态信息。该元素可以是设备驱动程序，也可以简单地读取内存映射设备或端口映射设备。该类提供两个函数，一个用于检索数据（在示例中类型为deviceData），另一个用于检索设备状态（在示例中类型为unsigned int）。

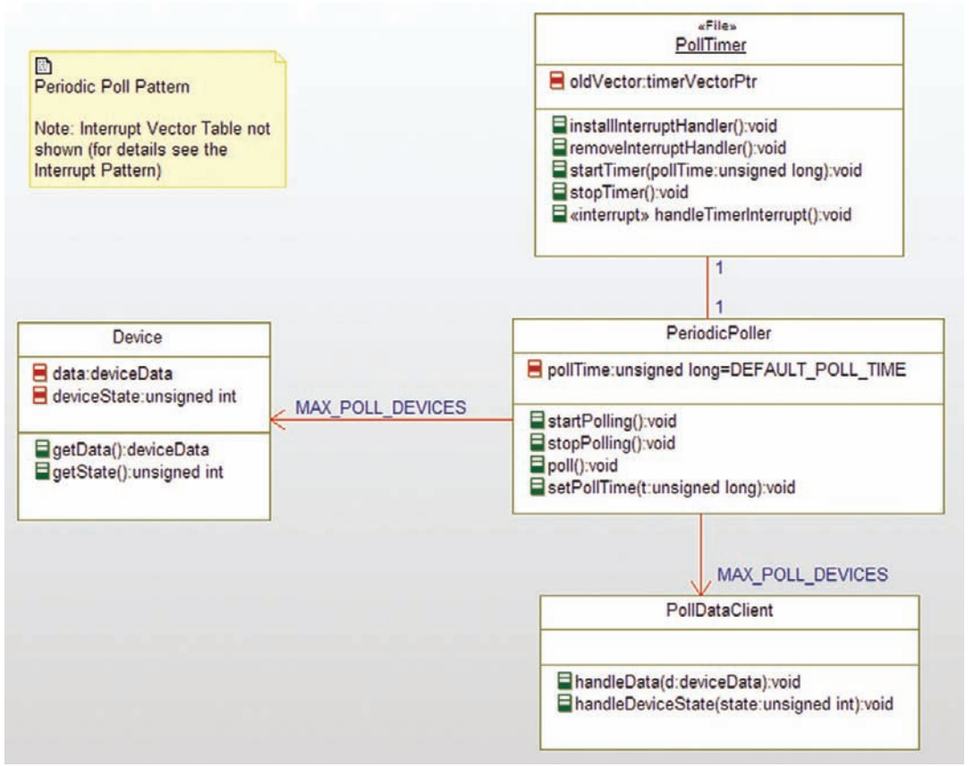


Figure 3-18 Periodic Polling Pattern

In the example, there are MAX_POLL_DEVICES connected to the polling element so that the poll() function scans them all and notifies their respective clients.

3.8.4.3 OpportunisticPoller

This element has the `poll()` function that scans the attached devices for data and device state and passes this information on to the appropriate client for each. The difference between the OpportunisticPoller and the PeriodicPoller is that the latter has timer initialization and shut-down functionality in addition to polling for the data.

3.8.4.4 PeriodicPoller

Like the `OpportunisticPoller`, the `PeriodicPoller` has the `poll()` function that scans the attached devices for data and device state and passes this information on to the appropriate client for each. In addition, it has a variable for the poll time (settable in its `setPollTime(t)` function) and services to start and stop polling. The `installInterruptTimer()` function inserts the address of

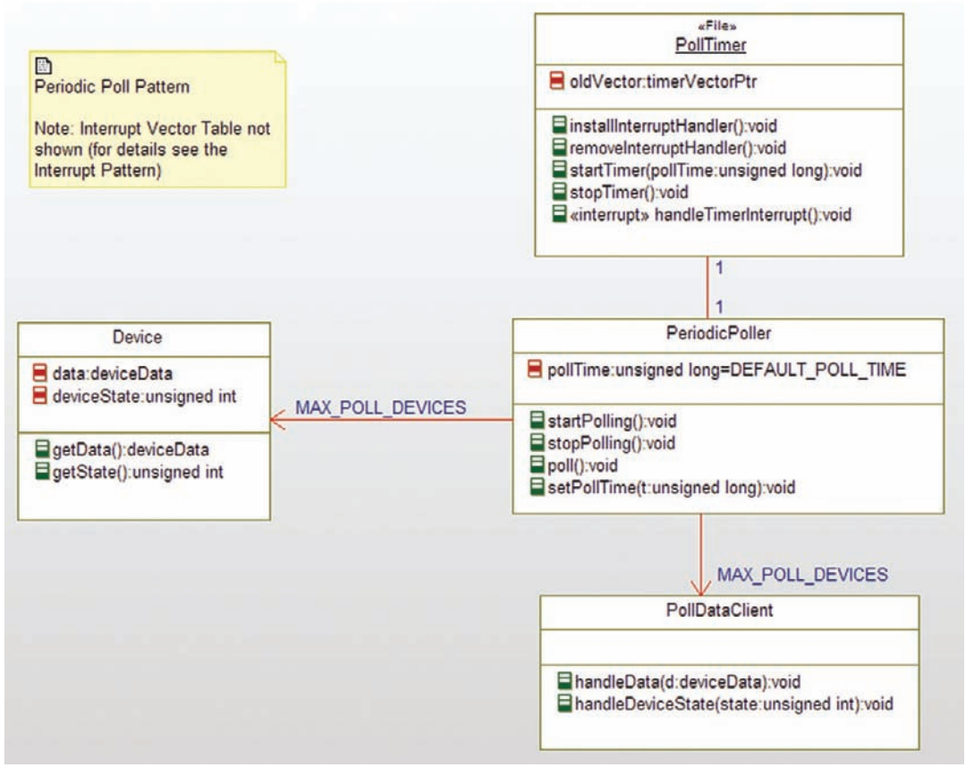


图 3-18 周期性轮询模式

在示例中，有MAX_POLL_DEVICES连接到轮询元素，以便poll()函数扫描所有设备并通知各自的客户端。

3.8.4.3 OpportunisticPoller

该元素具有 `poll()` 函数，用于扫描所连接的设备以获取数据和设备状态，并将此信息传递给各自对应的客户端。OpportunisticPoller 与 PeriodicPoller 的区别在于，后者除了轮询数据之外，还具有定时器初始化和关闭功能。

3.8.4.4 PeriodicPoller

与`OpportunisticPoller`类似，`PeriodicPoller` 具有`poll()` 函数，该函数扫描所连接的设备以获取数据和设备状态，并将此信息传递给每个设备对应的客户端。此外，它还有一个用于轮询时间的变量（可在其`setPollTime(t)` 函数中设置）以及用于启动和停止轮询的服务。`installInterruptTimer()` 函数将

the ISR into the interrupt vector table while the `removeInterruptHandler()` restores the original vector. The `startPolling()` function initializes the timer. The `stopPolling()` function stops the timer but leaves the vector in the interrupt vector table.

3.8.4.5 PollDataClient

This element is the client for data and state information from one or more of the devices. In some cases, there may be a single client for the data from all devices, but in general each device will have its own client.

3.8.4.6 PollTimer

The `PollTimer` element represents a timer and the services associated with using it. The startTimer() method installs the `handleTimerInterrupt()` interrupt service routine in the interrupt vector table and creates and initializes a timer with the `pollTime` attribute. Once the timer is started, it will set the hardware timer so that its ISR `handleTimerInterrupt()` will be invoked, which first resets the timer count and then calls the poll() function.

3.8.5 Consequences

Polling is simpler than the setup and use of the Interrupt Service Routines, although periodic polling is usually implemented with an ISR tied to a poll timer.

Polling can check many different devices at the same time for status changes but is usually less timely than interrupts. For this reason, care must be taken that if there are deadlines associated with the data or signals that the poll time plus the response time is always less than the deadlines involved. If data arrive faster than the poll time, then data will be lost. This is not a problem in many applications but is fatal (sometimes literally so) in others.

3.8.6 Implementation Strategies

The simplest implementation is to insert the hardware check in the middle of a main processing loop that executes as long as the system operates. This is called "symmetric opportunistic polling" because it operates the same way all the time, even though the length of time the processing loops take may vary widely. *Asymmetric opportunistic polling* refers to the practice of placing checks for new data at convenient but unrelated points throughout the processing. This approach allows for better tuning (if you need to be more responsive, add some more checks) but has a larger impact on the primary flow and is harder to maintain.

Periodic polling is polling that takes place with a regular time interval (known as the *period*) with a bounded variation (known as the *jitter*). To achieve this regularity, the checks for new data are initiated by a timer tied to an interrupt. Thus the periodic variant of the Polling Pattern is simply a special case of the Interrupt Pattern.

中断服务例程的地址插入中断向量表，而`removeInterruptHandler()`则恢复原始向量。`startPolling()`函数初始化定时器。`stopPolling()`函数停止定时器，但将向量保留在中断向量表中。

3.8.4.5 PollDataClient

该元素是用于从一个或多个设备获取数据和状态信息的客户端。在某些情况下，所有设备的数据可能只有一个客户端，但通常每个设备都会有自己的客户端。

3.8.4.6 PollTimer

`PollTimer`元素表示一个定时器以及与其使用相关的服务。startTimer()方法将`handleTimerInterrupt()`中断服务例程安装到中断向量表中，并使用`pollTime`属性创建并初始化一个定时器。定时器启动后，将设置硬件定时器，使其ISR `handleTimerInterrupt()`被调用，该ISR首先重置定时器计数，然后调用poll()函数。

3.8.5 结果

轮询比中断服务例程的设置和使用更简单，尽管周期性轮询通常通过绑定到轮询定时器的中断服务例程来实现。

轮询可以同时检查许多不同设备的状态变化，但通常不如中断及时。因此，必须注意：如果数据或信号存在相关的截止时间，则轮询时间加上响应时间必须始终小于所涉及的截止时间。如果数据到达速度快于轮询时间，那么数据将会丢失。这在许多应用中不是问题，但在其他应用中则是致命的（有时确实是字面意义上的致命）。

3.8.6 实现策略

最简单的实现方式是将硬件检查插入到系统运行期间持续执行的主处理循环中间。这被称为"对称机会轮询"，因为它的运行方式始终相同，尽管处理循环所花费的时间长度可能变化很大。非对称机会轮询是指在处理过程中将新数据的检查放置在方便但互不相关的各个点上。这种方法允许更好地调优（如果你需要更高的响应性，可以增加更多检查），但对主流程的影响更大，且更难维护。

周期性轮询是以规则的时间间隔（称为周期）进行的轮询，并具有有界的变化（称为抖动）。为实现这种规律性，对新数据的检查由与中断绑定的定时器发起。因此，轮询模式的周期性变体只是中断模式的一个特例。

3.8.7 Related Patterns

Periodic polling is a special case of the Interrupt Pattern. In addition, the hardware checks may be done by invoking data acquisition services of Hardware Proxies or Hardware Adapters. In addition, the Observer Pattern can be merged in as well, with the polling element (OpportunisticPoller or PeriodicPoller) serving as the data server and the PollDataClients serving as the data clients.

3.8.8 Example

For the example shown in Figure 3-19, I decided to use the Periodic Polling Pattern to check for three breathing circuit sensors. The first of these sensors monitors the oxygen concentration in the breathing circuit, the second monitors the gas pressure (and holds the gas flow state), and the third monitors the pressure in the circuit. When the `BEPeriodicPoller` is created, its initializer (`BCPeriodicPoller_Init()`) calls the `BCTimer::installInterruptHandler()` service;

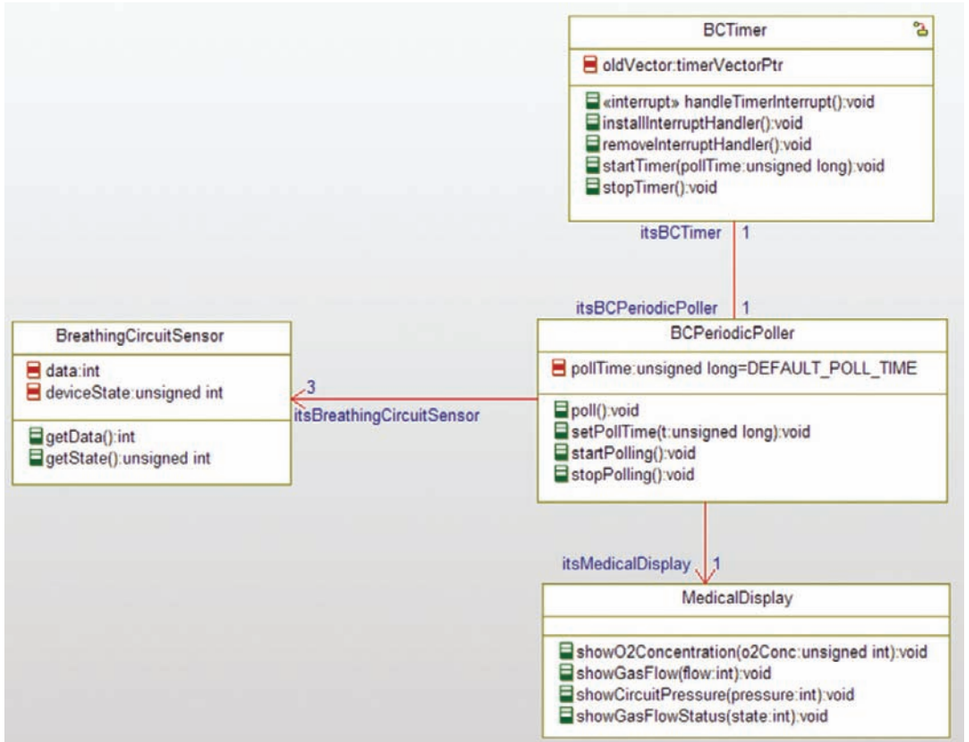


Figure 3-19 Polling example

3.8.7 相关模式

周期性轮询是中断模式的一个特例。此外，硬件检查可以通过调用硬件代理或硬件适配器的数据采集服务来完成。此外，观察者模式也可以合并进来，由轮询元素（OpportunisticPoller 或 PeriodicPoller）充当数据服务器，由 PollDataClients 充当数据客户端。

3.8.8 示例

对于 图 3-19 所示的示例，我决定使用周期性轮询模式来检查三个呼吸回路传感器。其中第一个传感器监测呼吸回路中的氧气浓度，第二个传感器监测气体压力（并保存气体流量状态），第三个传感器监测回路中的压力。当创建 `BEPeriodicPoller` 时，其初始化器（`BCPeriodicPoller_Init()`）会调用 `BCTimer::installInterruptHandler()` 服务；

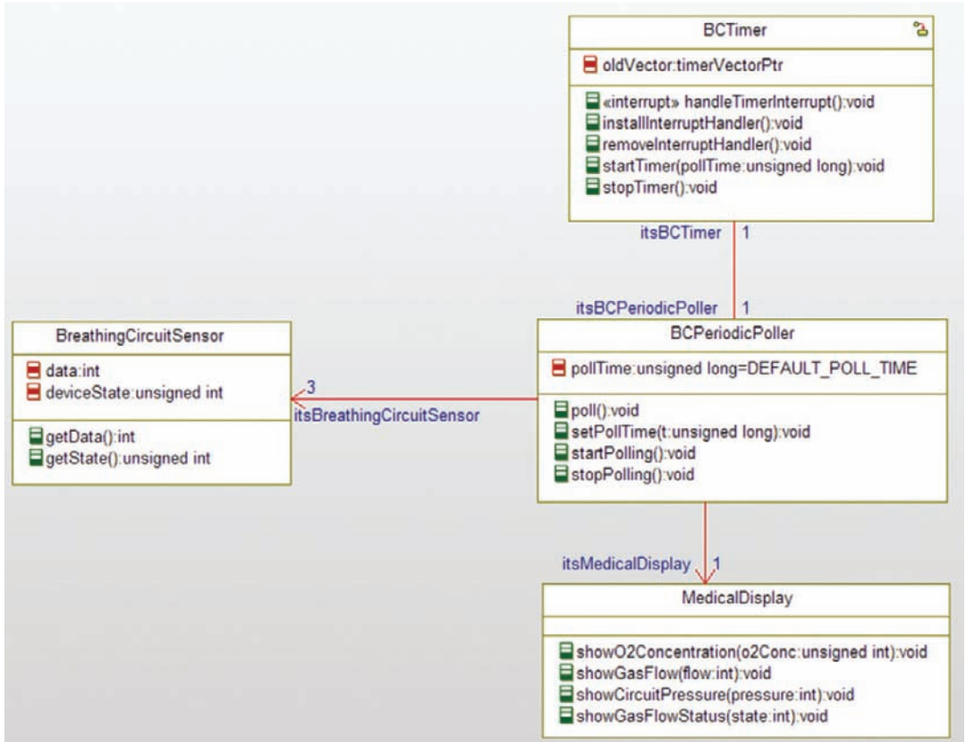


图 3-19 轮询示例

similarly, the clean-up operation done when the `BCPeriodicPoller` is destroyed calls `BCTimer_stopTiming()` and then BCTimer_removeInterruptHandler().

At some point, the `BCPeriodicPoller` is commanded to start polling via a call to `BCPeriodicPoller_startPolling()`. This function calls `BC_Timer_startTimer(timeout)` service, passing the desired poll period. From then on, until commanded otherwise, the timer will fire periodically and invoke the timer interrupt handler `BCTimer_handleTimeInterrupt()`, which resets the timer and calls `BCPeriodicPoller_poll()`. The poll() function then polls the data and state information from the three devices and sends them to the `MedicalDisplay`.

```
#ifndef BCPeriodicPoller_H
#define BCPeriodicPoller_H

typedef int deviceData;
typedef void (*timerVectorPtr)(void);
#define MAX_POLL_DEVICES (10)
#define DEFAULT_POLL_TIME (1000)

struct BCTimer;
struct BreathingCircuitSensor;
struct MedicalDisplay;

typedef struct BCPeriodicPoller BCPeriodicPoller;
struct BCPeriodicPoller {
        unsigned long pollTime;
        struct BCTimer* itsBCTimer;
        struct BreathingCircuitSensor *itsBreathingCircuitSensor[3];
        struct MedicalDisplay* itsMedicalDisplay;
};

void BCPeriodicPoller_Init(BCPeriodicPoller* const me);
void BCPeriodicPoller_Cleanup(BCPeriodicPoller* const me);
/* Operations */ void BCPeriodicPoller_poll(BCPeriodicPoller* const me);
void BCPeriodicPoller_setPollTime(BCPeriodicPoller* const me, unsigned long t);
void BCPeriodicPoller_startPolling(BCPeriodicPoller* const me);
void BCPeriodicPoller_stopPolling(BCPeriodicPoller* const me);
struct BCTimer* BCPeriodicPoller_getItsBCTimer(const BCPeriodicPoller* const me);
void  BCPeriodicPoller_setItsBCTimer(BCPeriodicPoller*  const  me,  struct  BCTimer*
p_BCTimer);
int BCPeriodicPoller_getItsBreathingCircuitSensor(const BCPeriodicPoller* const me);
void BCPeriodicPoller_addItsBreathingCircuitSensor(BCPeriodicPoller* const me, struct
BreathingCircuitSensor * p_BreathingCircuitSensor);
void  BCPeriodicPoller_removeItsBreathingCircuitSensor(BCPeriodicPoller*  const  me,
struct BreathingCircuitSensor * p_BreathingCircuitSensor);
void BCPeriodicPoller_clearItsBreathingCircuitSensor(BCPeriodicPoller* const me);
struct MedicalDisplay* BCPeriodicPoller_getItsMedicalDisplay(const BCPeriodicPoller*
const me);
void BCPeriodicPoller_setItsMedicalDisplay(BCPeriodicPoller* const me, struct Medical-
Display* p_MedicalDisplay);
BCPeriodicPoller * BCPeriodicPoller_Create(void);
```


类似地，当 `BCPeriodicPoller` 被销毁时执行的清理操作会调用`BCTimer_stopTiming()`，然后调用 BCTimer_removeInterruptHandler()。

在某个时刻，`BCPeriodicPoller` 通过调用`BCPeriodicPoller_startPolling()`被命令开始轮询。该函数调用`BC_Timer_startTimer(timeout)` 服务，并传入所需的轮询周期。从那时起，除非另有命令，定时器将周期性地触发并调用定时器中断处理程序`BCTimer_handleTimeInterrupt()`，该处理程序会重置定时器并调用`BCPeriodicPoller_poll()`。然后，poll()函数从三个设备轮询数据和状态信息，并将它们发送到`MedicalDisplay`。

```
#ifndef BCPeriodicPoller_H #define
BCPeriodicPoller_H typedef int deviceData;
typedef void (*timerVectorPtr)(void); #
define MAX_POLL_DEVICES (10) #define
DEFAULT_POLL_TIME (1000) struct
BCTimer; struct BreathingCircuitSensor;
struct MedicalDisplay;


typedef struct BCPeriodicPoller BCPeriodicPoller; struct BCPeriodicPoller {
unsigned long pollTime; struct BCTimer* itsBCTimer; struct
BreathingCircuitSensor *itsBreathingCircuitSensor[3]; struct MedicalDisplay*
itsMedicalDisplay; };


void BCPeriodicPoller_Init(BCPeriodicPoller* const me); void BCPeriodicPoller_
Cleanup(BCPeriodicPoller* const me); /* 操作 */ void BCPeriodicPoller_poll(BCPeriodicPoller* const
me); void BCPeriodicPoller_setPollTime(BCPeriodicPoller* const me, unsigned long t); void
BCPeriodicPoller_startPolling(BCPeriodicPoller* const me); void BCPeriodicPoller_
stopPolling(BCPeriodicPoller* const me); struct BCTimer* BCPeriodicPoller_getItsBCTimer(const
BCPeriodicPoller* const me); void BCPeriodicPoller_setItsBCTimer(BCPeriodicPoller* const me, struct
BCTimer* p_BCTimer); int BCPeriodicPoller_getItsBreathingCircuitSensor(const BCPeriodicPoller* co
nst me); void BCPeriodicPoller_addItsBreathingCircuitSensor(BCPeriodicPoller* const me, struct
BreathingCircuitSensor * p_BreathingCircuitSensor);


void BCPeriodicPoller_removeItsBreathingCircuitSensor(BCPeriodicPoller* const me, struct
BreathingCircuitSensor * p_BreathingCircuitSensor);
void BCPeriodicPoller_clearItsBreathingCircuitSensor(BCPeriodicPoller* const me); struct
MedicalDisplay* BCPeriodicPoller_getItsMedicalDisplay(const BCPeriodicPoller* const me); void
BCPeriodicPoller_setItsMedicalDisplay(BCPeriodicPoller* const me, struct Medical- Display* p_
MedicalDisplay); BCPeriodicPoller * BCPeriodicPoller_Create(void);
```

```
void BCPeriodicPoller_Destroy(BCPeriodicPoller* const me);
void BCPeriodicPoller___setItsBCTimer(BCPeriodicPoller* const me, struct BCTimer*
p_BCTimer);
void BCPeriodicPoller__setItsBCTimer(BCPeriodicPoller* const me, struct BCTimer*
p_BCTimer);
void BCPeriodicPoller__clearItsBCTimer(BCPeriodicPoller* const me);
#endif
```

Code Listing 3-23: BCPeriodicPoller.h

```
#include "BCPeriodicPoller.h"
#include "BCTimer.h"
#include "BreathingCircuitSensor.h"
#include "MedicalDisplay.h"

static void cleanUpRelations(BCPeriodicPoller* const me);
void BCPeriodicPoller_Init(BCPeriodicPoller* const me) { me->pollTime = DEFAULT_
POLL_TIME; me->itsBCTimer = NULL;
                    int pos;
                    for(pos = 0; pos < 3; ++pos) {
                        me->itsBreathingCircuitSensor[pos] = NULL;
                            me->itsMedicalDisplay = NULL;
        BCTimer_installInterruptHandler(me->itsBCTimer);
        me->pollTime = DEFAULT_POLL_TIME;
}

void BCPeriodicPoller_Cleanup(BCPeriodicPoller* const me) {
        BCTimer_stopTimer(me->itsBCTimer);
        BCTimer_removeInterruptHandler(me->itsBCTimer);
        cleanUpRelations(me);
}

void BCPeriodicPoller_poll(BCPeriodicPoller* const me) {
         int state, data;
         data = BreathingCircuitSensor_getData(me->itsBreathingCircuitSensor[0]);
         MedicalDisplay_showO2Concentration(me->itsMedicalDisplay, data);
         data = BreathingCircuitSensor_getData(me->itsBreathingCircuitSensor[1]);
         state = BreathingCircuitSensor_getState(me->itsBreathingCircuitSensor[1]);
         MedicalDisplay_showGasFlow(me->itsMedicalDisplay, data);
         MedicalDisplay_showGasFlowStatus(me->itsMedicalDisplay, state);
         data = BreathingCircuitSensor_getData(me->itsBreathingCircuitSensor[2]);
         MedicalDisplay_showCircuitPressure(me->itsMedicalDisplay, data);
}

void BCPeriodicPoller_setPollTime(BCPeriodicPoller* const me, unsigned long t) {
        me->pollTime = t;
}

void BCPeriodicPoller_startPolling(BCPeriodicPoller* const me) {
        BCTimer_startTimer(me->itsBCTimer, me->pollTime);
}
```


void BCPeriodicPoller_Destroy(BCPeriodicPoller* const me); void BCPeriodicPoller___ setItsBCTimer(BCPeriodicPoller* const me, struct BCTimer* p_BCTimer); void BCPeriodicPoller__ setItsBCTimer(BCPeriodicPoller* const me, struct BCTimer* p_BCTimer); void BCPeriodicPoller__ clearItsBCTimer(BCPeriodicPoller* const me); #endif

代码清单3-23：BCPeriodicPoller.h

#include "BCPeriodicPoller.h" #include " BCTimer.h" #include " BreathingCircuitSensor.h" #include "MedicalDisplay.h"

static void cleanUpRelations(BCPeriodicPoller* const me); void BCPeriodicPoller_ Init(BCPeriodicPoller* const me) { me->pollTime = DEFAULT_POLL_TIME; me->itsBCTimer = 空; int pos; for(pos = 0; pos < 3; ++pos) { me->itsBreathingCircuitSensor[pos] = 空; me->itsMedicalDisplay = 空; BCTimer_installInterruptHandler(me->itsBCTimer); me->pollTime = DEFAULT_POLL_TIME; } void BCPeriodicPoller_Cleanup(BCPeriodicPoller* const me) { BCTimer_stopTimer(me->it sBCTimer); BCTimer_removeInterruptHandler(me->itsBCTimer); cleanUpRelations(me); } void BCPeriodicPoller_poll(BCPeriodicPoller* const me) { int state, data; data = BreathingCircuitSensor_ getData(me->itsBreathingCircuitSensor[0]); MedicalDisplay_showO2Concentration(me-> itsMedicalDisplay, data); data = BreathingCircuitSensor_getData(me->itsBreathingCircuitSensor[1]); state = BreathingCircuitSensor_获取状态(me->itsBreathingCircuitSensor[1]); MedicalDisplay_ showGasFlow(me->itsMedicalDisplay, data); MedicalDisplay_showGasFlowStatus(me-> itsMedicalDisplay, state); data = BreathingCircuitSensor_getData(me->itsBreathingCircuitSensor[2]); MedicalDisplay_showCircuitPressure(me->itsMedicalDisplay, data); } void BCPeriodicPoller_ setPollTime(BCPeriodicPoller* const me, unsigned long t) { me->pollTime = t; } void BCPeriodicPoller_startPolling(BCPeriodicPoller* const me) { BCTimer_startTimer(me->itsBCTimer, me->pollTime); }

```
void BCPeriodicPoller_stopPolling(BCPeriodicPoller* const me) {
        BCTimer_stopTimer(me->itsBCTimer);
}
struct BCTimer* BCPeriodicPoller_getItsBCTimer(const BCPeriodicPoller* const me) {
        return (struct BCTimer*)me->itsBCTimer;
}

void BCPeriodicPoller_setItsBCTimer(BCPeriodicPoller* const me, struct BCTimer* p_
BCTimer) {
        if(p_BCTimer != NULL)
                {
                        BCTimer__setItsBCPeriodicPoller(p_BCTimer, me);
                }
        BCPeriodicPoller__setItsBCTimer(me, p_BCTimer);
}

int BCPeriodicPoller_getItsBreathingCircuitSensor(const BCPeriodicPoller* const me) {
        int iter = 0;
        return iter;
}

void BCPeriodicPoller_addItsBreathingCircuitSensor(BCPeriodicPoller* const me, struct
BreathingCircuitSensor * p_BreathingCircuitSensor) {
        int pos;
        for(pos = 0; pos < 3; ++pos) {
            if (!me->itsBreathingCircuitSensor[pos]) {
                me->itsBreathingCircuitSensor[pos] = p_BreathingCircuitSensor;
                break;
            }
        }
}

void  BCPeriodicPoller_removeItsBreathingCircuitSensor(BCPeriodicPoller*  const  me,
struct BreathingCircuitSensor * p_BreathingCircuitSensor) {
        int pos;
        for(pos = 0; pos < 3; ++pos) {
            if (me->itsBreathingCircuitSensor[pos] == p_BreathingCircuitSensor) {
                me->itsBreathingCircuitSensor[pos] = NULL;
            }
        }
}

void BCPeriodicPoller_clearItsBreathingCircuitSensor(BCPeriodicPoller* const me) {
        {
                int pos;
                for(pos = 0; pos < 3; ++pos)
                {
                    me->itsBreathingCircuitSensor[pos] = NULL;
                }
        }
}
```



```
void BCPeriodicPoller_stopPolling(BCPeriodicPoller* const me) { BCTimer_stopTimer(me->itsBCTimer); }
结构体 BCTimer* BCPeriodicPoller_getItsBCTimer(const BCPeriodicPoller* const me) { return (结构体
BCTimer*)me->itsBCTimer; } void BCPeriodicPoller_setItsBCTimer(BCPeriodicPoller* const me, 结构体
BCTimer* p_BCTimer) { if(p_BCTimer != 空) { BCTimer__setItsBCPeriodicPoller(p_BCTimer, me); }
BCPeriodicPoller__setItsBCTimer(me, p_BCTimer); } int BCPeriodicPoller_
getItsBreathingCircuitSensor(const BCPeriodicPoller* const me) { int iter = 0; return iter; } void
BCPeriodicPoller_addItsBreathingCircuitSensor(BCPeriodicPoller* const me, 结构体
BreathingCircuitSensor * p_BreathingCircuitSensor) { int pos; for(pos = 0; pos < 3; ++pos) { if (!me->
itsBreathingCircuitSensor[pos]) { me->itsBreathingCircuitSensor[pos] = p_BreathingCircuitSensor;
break; } } }

void BCPeriodicPoller_removeItsBreathingCircuitSensor(BCPeriodicPoller* const me                    ,
struct BreathingCircuitSensor * p_BreathingCircuitSensor) { int pos; for(pos = 0; pos < 3; ++pos) { if
(me->itsBreathingCircuitSensor[pos] == p_BreathingCircuitSensor) { me->i
tsBreathingCircuitSensor[pos] = 空; } } } void BCPeriodicPoller_
clearItsBreathingCircuitSensor(BCPeriodicPoller* const me) { { int pos; for(pos = 0; pos < 3; ++pos) {
me->itsBreathingCircuitSensor[pos] = 空; } } }
```

```
struct MedicalDisplay* BCPeriodicPoller_getItsMedicalDisplay(const BCPeriodicPoller*
const me) {
	return (struct MedicalDisplay*)me->itsMedicalDisplay;
}

void BCPeriodicPoller_setItsMedicalDisplay(BCPeriodicPoller* const me, struct Medical-
Display* p_MedicalDisplay) {
	me->itsMedicalDisplay = p_MedicalDisplay;
}

BCPeriodicPoller * BCPeriodicPoller_Create(void) {
	BCPeriodicPoller* me = (BCPeriodicPoller *) malloc(sizeof(BCPeriodicPoller));
	if(me!=NULL)
		{
			BCPeriodicPoller_Init(me);
		}
	return me;
}

void BCPeriodicPoller_Destroy(BCPeriodicPoller* const me) {
	if(me!=NULL)
		{
			BCPeriodicPoller_Cleanup(me);
		}
	free(me);
}

static void cleanUpRelations(BCPeriodicPoller* const me) {
	if(me->itsBCTimer != NULL)
		{
			struct BCPeriodicPoller* p_BCPeriodicPoller = BCTimer_
getItsBCPeriodicPoller(me->itsBCTimer);
			if(p_BCPeriodicPoller != NULL)
				{
				 BCTimer___setItsBCPeriodicPoller(me->itsBCTimer, NULL);
				}
			me->itsBCTimer = NULL;
		}
	if(me->itsMedicalDisplay != NULL)
		{
			me->itsMedicalDisplay = NULL;
		}
}

void BCPeriodicPoller___setItsBCTimer(BCPeriodicPoller* const me, struct BCTimer*
p_BCTimer) {
	me->itsBCTimer = p_BCTimer;
}

void BCPeriodicPoller__setItsBCTimer(BCPeriodicPoller* const me, struct BCTimer*
p_BCTimer) {
	if(me->itsBCTimer != NULL)
		{
```


struct MedicalDisplay* BCPeriodicPoller_getItsMedicalDisplay(const BCPeriodicPoller* const me) { return (struct MedicalDisplay*)me->itsMedicalDisplay; } void BCPeriodicPoller_setItsMedicalDisplay(BCPeriodicPoller* const me, struct Medical- Display* p_MedicalDisplay) { me->itsMedicalDisplay = p_MedicalDisplay; } BCPeriodicPoller * BCPeriodicPoller_Create(void) { BCPeriodicPoller* me = (BCPeriodicPoller *) malloc(sizeof(BCPeriodicPoller)); if(me!=空) { BCPeriodicPoller_Init(me); } return me; } void BCPeriodicPoller_Destroy(BCPeriodicPoller* const me) { if(me!=空) { BCPeriodicPoller_Cleanup(me); } free(me); } static void cleanUpRelations(BCPeriodicPoller* const me) { if(me->itsBCTimer != 空) { struct BCPeriodicPoller* p_BCPeriodicPoller = BCTimer_getItsBCPeriodicPoller(me->itsBCTimer); if(p_BCPeriodicPoller != 空) { BCTimer___setItsBCPeriodicPoller(me->itsBCTimer, 空); } me->itsBCTimer = 空; } if(me->itsMedicalDisplay != 空) { me->itsMedicalDisplay = 空; } } void BCPeriodicPoller___setItsBCTimer(BCPeriodicPoller* const me, struct BCTimer*

p_BCTimer) { me->itsBCTimer = p_
BCTimer; }

void BCPeriodicPoller__setItsBCTimer(BCPeriodicPoller* const me, struct BCTimer* p_BCTimer) { if(
me->itsBCTimer != 空) {

```
                    BCTimer___setItsBCPeriodicPoller(me->itsBCTimer, NULL);
                }
        BCPeriodicPoller___setItsBCTimer(me, p_BCTimer);
}

void BCPeriodicPoller__clearItsBCTimer(BCPeriodicPoller* const me) {
        me->itsBCTimer = NULL;
}
```

Code Listing 3-24: BCPeriodicPoller.c

3.9 So, What Did We Learn?

In this chapter, we've looked at a number of patterns useful for manipulating hardware. The Hardware Proxy Pattern focuses on encapsulation of hardware-specific details while the Hardware Adapter Pattern provides a straightforward means to adapt different but similar devices for the needs of the system. Many times, complex interactions of hardware are necessary for proper system functions, a benefit of the Mediator Pattern. The Observer Pattern supports dynamic addition and removal of clients for hardware data, such as from sensors.

The last three patterns provide different means for interfacing to hardware. The Debouncing Pattern handles the common problem of hardware that produces intermittent contact during open and closure of metal contact switches. The Interrupt Pattern supports the highly timely response to events of interest. The Polling Pattern repeatedly polls the hardware for new sensor data and device state information.

These are basic patterns in the sense that they all address low-level concerns about interfacing with the hardware. The more difficult design issues have to do with transforming the commands and data within the system to achieve the system goals. The next chapter provides patterns that deal with issues of concurrent execution, from basic concepts to scheduling execution, avoiding race conditions and deadlock, and synchronizing tasks when necessary.

Chapter 5 deals with finite state machines – their implementation and usage; while in Chapter 6 we look at safety and reliability.

BCTimer___setItsBCPeriodicPoller(me->itsBCTimer, 空); } BCPeriodicPoller___ setItsBCTimer(me, p_BCTimer); } void BCPeriodicPoller__clearItsBCTimer(BCPeriodicPoller* const me) { me->itsBCTimer = 空; }

代码清单3-24：BCPeriodicPoller.c

3.9 那么，我们学到了什么？

在本章中，我们研究了许多用于操作硬件的模式。硬件代理模式侧重于封装硬件特定的细节，而硬件适配器模式提供了一种直接的方式来适配不同但相似的设备以满足系统的需求。许多时候，为了实现正确的系统功能，硬件之间需要进行复杂的交互，这正是中介者模式的优点。观察者模式支持动态添加和移除硬件数据的客户端，例如来自传感器的数据。

最后三个模式提供了不同的硬件接口方式。去抖模式处理了硬件在金属接触开关断开和闭合期间产生间歇性接触这一常见问题。中断模式支持对关注事件的高度及时响应。轮询模式反复轮询硬件以获取新的传感器数据和设备状态信息。

这些是基本模式，因为它们都涉及与硬件接口相关的底层问题。更困难的设计问题在于如何转换系统内的命令和数据以实现系统目标。下一章将提供处理并发执行问题的模式，从基本概念到调度执行、避免竞态条件和死锁，以及在必要时同步任务。

第5章讨论有限状态机——其实现与使用；而在第6章中，我们探讨安全性与可靠性。

This page intentionally left blank

本页有意留白

CHAPTER 4

Design Patterns for Embedding Concurrency and Resource Management

Chapter Outline

4.1 Basic Concurrency Concepts 152
- 4.1.1 Identifying Tasks 160
- 4.1.2 Concurrency in the UML 161
- 4.1.3 Real-Time Operating Systems 163

Scheduling Patterns 164

4.2 Cyclic Executive Pattern 164
- 4.2.1 Abstract 165
- 4.2.2 Problem 165
- 4.2.3 Pattern Structure 165
- 4.2.4 Collaboration Roles 165
  - *4.2.4.1 AbstractCEThread 165*
  - *4.2.4.2 CyclicExecutive 165*
  - *4.2.4.3 CycleTimer 166*
- 4.2.5 ConcreteCEThread 166
- 4.2.6 Consequences 167
- 4.2.7 Implementation Strategies 167
- 4.2.8 Related Patterns 167
- 4.2.9 Example 167

4.3 Static Priority Pattern 170
- 4.3.1 Abstract 170
- 4.3.2 Problem 171
- 4.3.3 Pattern Structure 171
- 4.3.4 Collaboration Roles 172
  - *4.3.4.1 AbstractStaticThread 172*
  - *4.3.4.2 ConcreteThread 172*
  - *4.3.4.3 Mutex 172*
  - *4.3.4.4 PriorityQueue 172*
  - *4.3.4.5 SharedResource 172*
  - *4.3.4.6 Stack 173*
  - *4.3.4.7 StaticPriorityScheduler 173*
  - *4.3.4.8 StaticTaskControlBlock 173*
- 4.3.5 Consequences 173
- 4.3.6 Implementation Strategies 173

Design Patterns for Embedded Systems in C
DOI:10.1016/B978-1-85617-707-8.00004-2

Copyright © 2011 Elsevier Inc.

第4章

嵌入式并发与资源管理设计模式

本章大纲

4.1 基本 并发 概念 **152** 4 1 1 160 . . 识别任务 4.1.2 UML中的并发 161 4.1.3 实时操作系统 16调度 模式 **164 42** . 循环 执行 模式 **164** 4 2 1 165 . . 摘要 4.2.2 问题 3 165 4.2.3 模式结构 165 4.2.4 协作角色 165 4.2.4.1 AbstractCEThread 165 4.2.4.2 CyclicExecutive 165 4.2.4.3 CycleTimer 166 4.2.5 ConcreteCEThread 166 4.2.6 结果 167 4.2.7 实现策略 167 4.2.8 相关模式 167 4.2.9 示例 167**4.3** 静态 优先级 模式 **170**4.3.1 摘要 170 4.3.2 问题 171 4.3.3 模式结构 171 4.3.4 协作角色 172 4.3.4.1 AbstractStaticThread 172 4.3.4.2 ConcreteThread 172 4.3.4.3 互斥锁 172 4.3.4.4 优先级队列 172 4.3.4.5 共享资源 172 4.3.4.6 栈 173 4.3.4.7 StaticPriorityScheduler 173 4.3.4.8 StaticTaskControlBlock 173 4.3.5 结果 173 4.3.6 实现策略 173

C语言嵌入式系统设计模式
DOI:10.1016/B978-1-85617-707-8.00004-2 版权所有 © 2011 Elsevier Inc.

4.3.7 Related Patterns 174
4.3.8 Example 175
Task Coordination Patterns 182
4.4 Critical Region Pattern 182
4.4.1 Abstract 182
4.4.2 Problem 182
4.4.3 Pattern Structure 182
4.4.4 Collaboration Roles 183
4.4.4.1 CRSharedResource 183
4.4.4.2 TaskWithCriticalRegion 183
4.4.4.3 TaskWithSharedResource 184
4.4.5 Consequences 184
4.4.6 Implementation Strategies 184
4.4.7 Related Patterns 184
4.4.8 Example 184
4.5 Guarded Call Pattern 190
4.5.1 Abstract 190
4.5.2 Problem 190
4.5.3 Pattern Structure 190
4.5.4 Collaboration Roles 191
4.5.4.1 GuardedResource 191
4.5.4.2 PreemptiveTask 191
4.5.4.3 Semaphore 192
4.5.4.4 StaticPriorityScheduler 192
4.5.5 Consequences 192
4.5.6 Implementation Strategies 192
4.5.7 Related Patterns 193
4.5.8 Example 194
4.6 Queuing Pattern 207
4.6.1 Abstract 207
4.6.2 Problem 207
4.6.3 Pattern Structure 208
4.6.4 Collaboration Roles 208
4.6.4.1 Message 208
4.6.4.2 MessageQueue 208
4.6.4.3 Mutex 209
4.6.4.4 Qtask 209
4.6.5 Consequences 209
4.6.6 Implementation Strategies 210
4.6.7 Related Patterns 210
4.6.8 Example 210
4.7 Rendezvous Pattern 224
4.7.1 Abstract 225
4.7.2 Problem 226
4.7.3 Pattern Structure 226

4.3.7 相关模式 174 4.3.8 示例 175任务
协调 模式 **1824.4** 临界 区 模式 **182**
4 4 1 182 . . 摘要 4.4.2 问题 182 4.4.3 模式
结构 182 4.4.4 协作角色 183 4.4.4.1
CRSharedResource 183 4.4.4.2
TaskWithCriticalRegion 183 4.4.4.3
TaskWithSharedResource 18 4.4.5 结果
184 4.4.6 实现策略 184 4.4.7 相关模式 184
4.4.8 示例 184**4.5** 受保护 调用 模式
1904.5.1 摘要 190 4.5.2 问题 190 4.5.3 模式*4*
结构 190 4.5.4 协作角色 191 4.5.4.1
GuardedResource 191 4.5.4.2
PreemptiveTask 191 4.5.4.3 信号量 192
4.5.4.4 StaticPriorityScheduler 192 4.5.5
结果 192 4.5.6 实现策略 192 4.5.7 相关模式
193 4.5.8 示例 194

4.6 排队 模式 **207** 4 6 1 207 . . 摘要
4.6.2 问题 207 4.6.3 模式结构 208 4.6.4
协作角色 208 4.6.4.1 消息 208 4.6.4.2
MessageQueue 208 4.6.4.3 互斥锁 209
4.6.4.4 Qtask 209 4.6.5 结果 209 4.6.6 实
现策略 210 4.6.7 相关模式 210 4.6.8 示例
210

4.7 会合 模式 **224**4.7.1 摘要
225 4.7.2 问题 226 4.7.3 模式结
构 226

4.7.4 Collaboration Roles 226
4.7.4.1 Rendezvous 226
4.7.4.2 Semaphore 227
4.7.4.3 SynchronizingThread 227
4.7.5 Consequences 227
4.7.6 Implementation Strategies 228
4.7.7 Related Patterns 228
4.7.8 Example 228
Deadlock Avoidance Patterns 231
4.8 Simultaneous Locking Pattern 232
4.8.1 Abstract 232
4.8.2 Problem 233
4.8.3 Pattern Structure 233
4.8.4 Collaboration Roles 234
4.8.4.1 MasteredResource 234
4.8.4.2 MultimasteredResource 234
4.8.4.3 Mutex 235
4.8.4.4 QueryMutex 235
4.8.4.5 ResourceClient 235
4.8.4.6 Resource Master 235
4.8.4.7 ResourceMasterSimplified 235
4.8.5 Consequences 236
4.8.6 Implementation Strategies 236
4.8.7 Related Patterns 236
4.8.8 Example 236
4.9 Ordered Locking 242
4.9.1 Abstract 242
4.9.2 Problem 243
4.9.3 Pattern Structure 243
4.9.4 Collaboration Roles 244
4.9.4.1 Mutex 244
4.9.4.2 OrderedResource 244
4.9.4.3 ResourceClient 245
4.9.4.4 ResourceList 247
4.9.4.5 ResourceReference 247
4.9.5 Consequences 247
4.9.6 Implementation Strategies 248
4.9.7 Related Patterns 248
4.9.8 Example 248
4.10 So, What Have We Learned? 255

4.7.4 协作角色 226 4.7.4.1 会合
226 4.7.4.2 信号量 227 4.7.4.3
SynchronizingThread 227 4.7.5 结果
227 4.7.6 实现策略 228 4.7.7 相关模式
228 4.7.8 示例 228 死锁避免模式 231
4.8 同时锁定模式 232 4.8.1 摘要 232
4.8.2 问题 233 4.8.3 模式结构 233 4.8.4
协作角色 234 4.8.4.1 受控资源 234
4.8.4.2 多主控资源 234 4.8.4.3 互斥锁
235 4.8.4.4 查询互斥量 235 4.8.4.5 资源
客户端 235 4.8.4.6 资源主控 235
4.8.4.7 简化资源主控 235 4.8.5 结果
236 4.8.6 实现策略 236 4.8.7 相关模式
236 4.8.8 示例 236 4.9 有序锁定 242 4
9 1 242 . . 摘要 4.9.2 问题 243 4.9.3 模式
结构 243 4.9.4 协作角色 244 4.9.4.1 互斥
锁 244 4.9.4.2 有序资源 244 4.9.4.3 资源
客户端 245 4.9.4.4 资源列表 247
4.9.4.5 资源引用 247 4.9.5 结果 247
4.9.6 实现策略 248 4.9.7 相关模式 248
4.9.8 示例 248 4.10 那么，我们学到了什
么？ 255

Patterns in this chapter

Cyclic Executive Pattern – Schedule threads in an infinite loop

Static Priority Pattern – Schedule threads by priority

Critical Region Pattern – Protect resources by disabling task switching

Guarded Call Pattern – Protect resources through mutex semaphores

Queuing Pattern – Serialize access through queuing messages

Rendezvous Pattern – Coordinate complex task synchronization

Simultaneous Locking Pattern – Avoid deadlock by locking resources together

Ordered Locking Pattern – Avoid deadlock by locking resources only in specific orders

4.1 Basic Concurrency Concepts

Most embedded systems need to perform several activities simultaneously. The key to doing that is through the definition and management of the concurrency model of the system. In this section, I will provide an overview of the basic terms and concepts of concurrency to support the understanding and usage of the design patterns that come later in this chapter.

The most important terms to understand are given in the sidebar.

Concurrency Term Definitions

- *Action Sequence*: a series of actions, possibly with decision branch points, in which the order of execution is fully determined
- *Arrival Pattern*: how the event that begins a concurrency unit arrives; it may be periodic or aperiodic
- *Blocking*: see *priority inversion*
- *Blocking time*: the length of time a high-priority task is prevented from running because a lower-priority task owns a required resource
- *Concurrency*: the simultaneous execution of action sequences
- *Concurrency unit*: a set of actions executed sequentially in the same thread of execution. Also known as a *task* or *thread*
- *Criticality*: the importance of the completion of an action sequence
- *Deadline*: a point in time at which the completion of an action or action sequence becomes incorrect or irrelevant
- *Deadlock*: a situation in which multiple clients are simultaneously waiting on the same conditions but the conditions cannot, in principle, be satisfied

本章 的 模式 包括循环执行模式 – 在无限循环中调度线程 静态优先级模式 – 按优先级调度线程 临界区模式 – 通过禁用任务切换来保护资源 受保护调用模式 – 通过互斥信号量保护资源 排队模式 – 通过排队消息串行化访问 会合模式 – 协调复杂的任务同步 同时锁定模式 – 通过同时锁定资源来避免死锁 有序锁定模式 – 仅按特定顺序锁定资源来避免死锁

4.1 基本并发概念

大多数嵌入式系统需要同时执行多个活动。实现这一点的关键在于定义和管理系统的并发模型。在本节中，我将概述并发的基本术语和概念，以帮助理解和运用本章后续介绍的设计模式。

需要理解的最重要术语见侧栏。

并发术语定义

- 动作序列：一系列动作，可能带有决策分支点，其执行顺序完全确定
- 到达模式：启动并发单元的事件如何到达；它可以是周期性的或非周期性的
- 阻塞：参见优先级反转
- 阻塞时间：高优先级任务因低优先级任务拥有所需资源而无法运行的时间长度
- 并发：动作序列的同时执行
- 并发单元：在同一执行线程中顺序执行的一组动作。也称为任务或线程
- 关键性：动作序列完成的重要性
- 截止时间：动作或动作序列的完成变得不正确或无关紧要的时间点
- 死锁：多个客户端同时等待相同条件，但这些条件原则上无法被满足的情形

- *Execution time*: the length of time required for an action or action sequence
- *Interarrival time*: the time between task initiation for an aperiodic task
- *Jitter*: variation around the period of a periodic task
- *Multitasking Preemptive Scheduling*: a scheduling schema in which the highest priority task ready to run will do so, preempting any lower priority tasks that are currently running
- *Period*: the interval between periodic task invocations
- *Priority*: a numeric value used to determine which task, of a set of ready-to-run tasks, will execute preferentially
- *Priority inversion*: when a lower priority task is running even though a higher priority task is ready to run. Also known as *blocking*
- *Pseudoconcurrency*: execution of multiple tasks by executing a single task at a time and switching focus between tasks according to some scheduling policy
- *Race condition*: a computational situation in which the result depends upon the order of execution of actions, but that order is not inherently knowable
- *Real-Time Action*: an action for which timeliness is an aspect of correctness
- *Schedulability*: a task set is said to be schedulable if all of its tasks are timely
- *Synchronization Pattern*: how tasks will manage synchronization at specific points in their respective action sequences
- *Synchronization Point*: a specific action in a sequence at which the task will wait until other tasks reach corresponding synchronization points
- *Task*: see *concurrency unit*
- *Thread*: see *concurrency unit*
- *Timeliness*: the ability of a task to predictably complete prior to its deadline, or to meet its time-related performance requirements
- *Urgency*: the nearness of an action sequence's deadline
- *Worst Case Execution Time*: the longest time an action sequence execution can take

Figure 4-1 shows the relationship between actions, action sequences, and tasks[1]. Each task (also known as a concurrency unit) consists of a set of actions performed in a specific sequence. These action sequences are executed independently from the others. This means that within a task, the order of action execution is fully known; in the case of Task 1 in the figure, first Action A, then Action B, then Action C or Action D, then Action E and so on. What is not known is the order of action execution *between* tasks. Which action executes first? Action A? Action Z? Action Beta? The answer is *you don't know and you don't care* – if you care, then concurrency was a poor choice.

[1] Many people draw a distinction between *task, thread*, and *process*. However, these are simply the same concept at different scopes of concern. In this book, we will use these terms interchangeably.

- 执行时间：执行一个动作或动作序列所需的时间长度
- 到达间隔时间：非周期性任务的任务启动之间的时间
- 抖动：周期性任务周期周围的变动
- 多任务抢占式调度：一种调度方案，其中最高优先级的就绪任务将执行，抢占当前正在运行的任何较低优先级任务
- 周期：周期性任务调用之间的间隔
- 优先级：一个数值，用于确定一组就绪任务中哪个任务将优先执行
- 优先级反转：当较高优先级的任务已就绪时，较低优先级的任务仍在运行。也称为阻塞
- 伪并发：通过一次执行一个任务并根据某种调度策略在任务之间切换焦点来执行多个任务
- 竞态条件：一种计算情形，其结果取决于动作的执行顺序，但该顺序本质上不可知
- 实时动作：及时性是其正确性一个方面的动作
- 可调度性：如果一个任务集中的所有任务都是及时的，则称该任务集是可调度的
- 同步模式：任务将如何在其各自动作序列中的特定点管理同步
- 同步点：动作序列中的一个特定动作，任务将在该点等待，直到其他任务到达相应的同步点
- 任务：参见并发单元
- 线程：参见并发单元
- 及时性：任务在其截止时间之前可预测地完成，或满足其与时间相关的性能需求的能力
- 紧迫性：动作序列的截止时间的临近程度
- 最坏情况执行时间：动作序列执行所能花费的最长时间

图 4-1 展示了动作、动作序列和任务之间的关系[1]。每个任务（也称为并发单元）由一组按特定顺序执行的动作组成。这些动作序列彼此独立执行。这意味着在任务内部，动作执行的顺序是完全已知的；在图中任务 1 的情况下，首先是动作 A，然后是动作 B，然后是动作 C 或动作 D，然后是动作 E，依此类推。未知的是任务之间动作执行的顺序。哪个动作先执行？动作 A？动作 Z？动作 Beta？答案是你不知道，也不关心——如果你关心，那么并发就是一个糟糕的选择。

[1] 许多人对任务、线程和进程加以区分。然而，这些不过是同一概念在不同关注范围下的体现。在本书中，我们将互换使用这些术语。

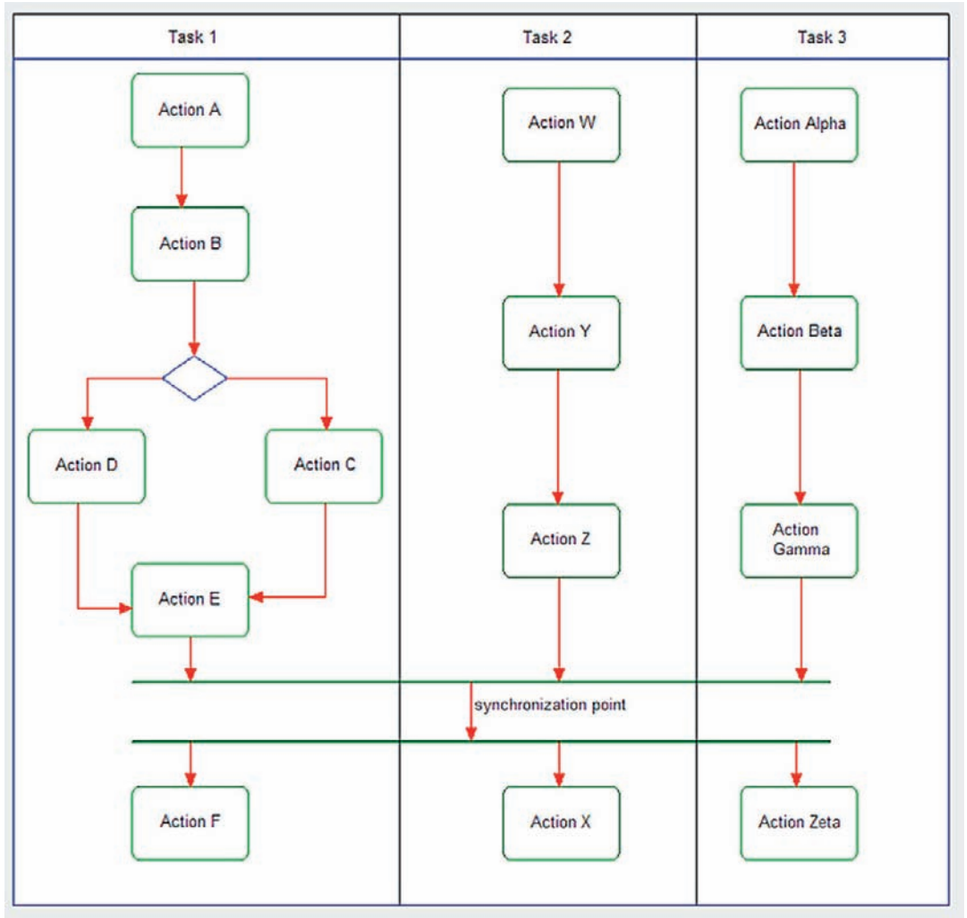


Figure 4-1: Concurrency as a set of action sequences

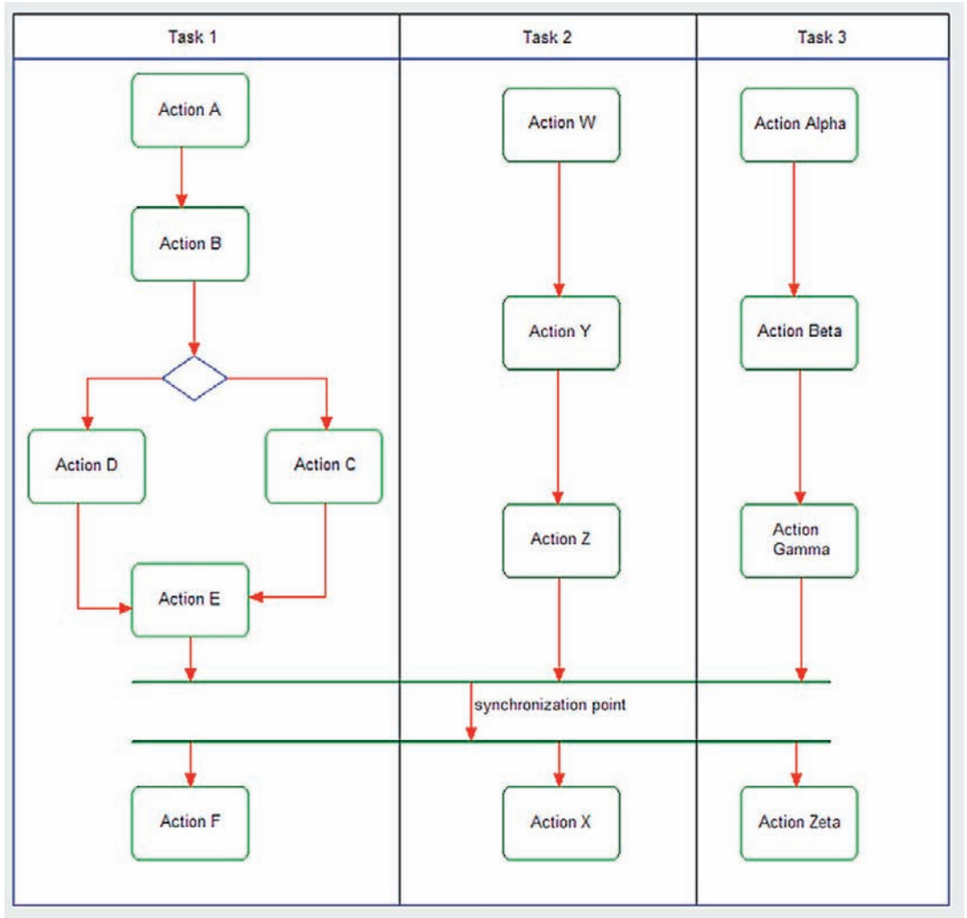


图4-1：并发作为一组动作序列

The only points at which you care about the order of execution between tasks are called *synchronization points*. This is indicated in the activity diagram of Figure 4-1 with a pair of horizontal bars. The first of these is called a *join*, since it joins together a set of actions from different tasks into, logically, a single thread. The second bar is called a fork, as it forks from the single thread into multiple. The semantics of using joins and forks in this way are that the flow cannot proceed past the synchronization point until all predecessor actions (in this case, Action E, Action Z, and Action Gamma) have completed. Once all three actions have

你唯一关心任务之间执行顺序的点称为同步点。这在图4-1的活动图中用一对水平条表示。其中第一个称为汇合，因为它将来自不同任务的一组动作在逻辑上汇合为单个线程。第二个条称为分叉，因为它从单个线程分叉为多个线程。以这种方式使用汇合和分叉的语义是，在所有的前驱动作（在本例中为动作E、动作Z和动作Gamma）完成之前，流程不能越过同步点。一旦所有三个动作都已

completed, control goes on to subsequent actions (Action F, Action X, and Action Zeta, not necessarily in that order). Synchronization points are common places in which tasks will share data or ensure preconditions are met before proceeding.

Tasks can communicate in two fundamental ways. Synchronous communications are like a phone call – both parties are engaged at the same time and invoke services and exchange data immediately. The sender "sends and waits" for a response. Synchronous communication is usually implemented via function calls. Asynchronous communication is like a postcard. The sender of the message "sends and forgets." At some later time, the receiver gets and processes the message or data. The message processing is less timely but the sender need not wait until the receiver is ready.

For tasks to execute concurrently, they must run on different CPUs or cores within a multicore CPU. Within a given CPU core, multiple tasks are run using pseudoconcurrency in which only one task executes at any instant in time, but the processor gives the appearance of concurrency by switching among the ready tasks. There are many different policies that can be used to select which task to run, some of which will be provided as patterns later in this chapter.

The basic structure of tasks depends a bit on the scheduling pattern that will invoke them. A task for a cyclic executive is simply sequential code that returns when it is done. A task that runs in a cooperative multitasking environment usually runs in a loop but contains points at which it voluntarily releases control back to the scheduler to allow other tasks to run. Code Listing 4-1 shows a typical cyclic executive. First, the initial global data declarations are followed by system initiation, and then (if initiation succeeds), the infinite cyclic executive task loop.

```
void main(void) {
     /*  global static and stack data  */
     static int nTasks  =  3;
     int currentTask;

     /*  initialization code  */
     currentTask  =  0;

     if (POST()) { /*  Power On Self Test succeeds  */
          /*  scheduling executive  */
          while (TRUE) {
               task1();
               task2();
               task3();
          }; /*  end cyclic processing loop  */
     }
}
```

Code Listing 4-1: Cyclic executive processing loop

完成后，控制继续执行后续动作（动作F、动作X和动作Zeta，不一定按此顺序）。同步点是任务共享数据或确保在继续之前满足前置条件的常见位置。

任务可以通过两种基本方式进行通信。同步通信就像打电话——双方同时参与，立即调用服务并交换数据。发送者"发送并等待"响应。同步通信通常通过函数调用来实现。异步通信就像明信片。消息的发送者"发送后即忘"。在稍后的某个时间，接收者获取并处理消息或数据。消息处理的及时性较差，但发送者无需等待接收者就绪。

要使任务并发执行，它们必须在不同的CPU或多核CPU内的不同核心上运行。在给定的CPU核心内，多个任务通过伪并发运行，即在任何时刻只有一个任务执行，但处理器通过在就绪任务之间切换来呈现并发的表象。有许多不同的策略可用于选择运行哪个任务，其中一些将在本章后面作为模式提供。

任务的基本结构在一定程度上取决于将调用它们的调度模式。循环执行程序的任务就是简单的顺序代码，执行完毕后返回。在协作式多任务环境中运行的任务通常在一个循环中运行，但其中包含一些点，在这些点上任务会自愿将控制权交还给调度器，以便其他任务得以运行。代码清单4-1展示了一个典型的循环执行程序。首先，初始的全局数据声明之后是系统初始化，然后（如果初始化成功），进入无限的循环执行程序任务循环。

```
void main(void) { /* 全局静态数据和栈数据 */ static int nTasks
  =  3; int currentTask; /* 初始化代码 */ currentTask  =  0; if
(POST()) { /* 上电自检成功 */ /* 调度执行程序 */ while (TRUE) {
task1(); task2(); task3(); }; /* 结束循环处理循环 */ } }
```

代码清单4-1：循环执行程序处理循环

The different ways to schedule the tasks have both benefits and costs. *Cyclic executives* are very simple; the scheduler consists of an infinite loop that executes the tasks one after the other. While simple, it isn't flexible and can have poor response to incoming events since an event processed by a task must wait until that task is run within the cycle. In addition, task executions must be short if other tasks are going to run in a timely fashion. A variant known as *time-triggered cyclic executive* starts the cycle at a time boundary; if it completes the cycle before the next time boundary, it sleeps until the start of the next epoch. A cooperative round-robin scheduler is basically a cyclic executive in which the tasks don't run to completion but instead run to specific points at which they release control back to the scheduling loop. This approach allows long tasks to run while still permitting other tasks to make progress in their processing.

While simple, there are a number of problems with both cyclic executives and round-robin schedulers. First, they are not responsive to urgent events; the task that processes the urgent event simply runs in its sequence. Secondly, they are not flexible since the schedule order is determined at design time. Thirdly, they don't scale well to large numbers of tasks. Lastly, a single misbehaving task stops the entire system from execution. These kinds of schedulers are best-suited to very simple systems or systems that are highly safety critical (since simple systems are far easier to prove correct and certify than complex ones).

The other primary kind of scheduling is *preemptive scheduling*. In preemptive scheduling, a scheduler stops the currently executing task when it decides to do so and runs the next task. The criterion for preempting a task and the selection of the next task in sequence depends on the particular scheduling policy.

At the high level, tasks in a preemptive scheduler have four significant states. A task is said to be *waiting* when the conditions for it to be run are not currently in force. When the conditions required for a task to run are true, then the task is said to be *ready to run*. When the scheduler initiates the task, the task is said to be *running*. When a task is ready to run but is being prevented because another task owns a necessary resource, the task is said to be *blocked*.

A time-slicing scheduler (also known as *time-driven multitasking executive* or TDME) operates on a fairness doctrine; that is, every task gets an equal opportunity to execute. The tasks are preempted after they run for a specific period of time. TDME schedulers are tuned by adjusting the duration of the time slices; too long and the other tasks don't run often enough to do their jobs, too short and too high a percentage of time is spent switching the task context.

The most common scheduling pattern for larger embedded systems is *priority-based scheduling*. In this approach, each task is assigned a priority, a scalar value that will be used to determine which task, from the set of ready to run tasks, will be selected for execution.

调度任务的不同方式各有其优点和成本。循环执行程序非常简单；调度器由一个无限循环组成，依次执行各个任务。虽然简单，但它不够灵活，并且对传入事件的响应可能较差，因为由某个任务处理的事件必须等到该任务在循环中被运行时才能得到处理。此外，如果其他任务要及时运行，任务执行必须简短。一种称为时间触发循环执行程序的变体在时间边界处启动循环；如果它在下一个时间边界之前完成了循环，就会休眠直到下一个纪元开始。协作式轮转调度器基本上是一种循环执行程序，其中任务不会运行到完成，而是运行到特定点后将控制权释放回调度循环。这种方法允许长任务运行，同时仍允许其他任务在其处理中取得进展。

虽然简单，但循环执行程序和轮转调度器都存在许多问题。首先，它们对紧急事件响应不佳；处理紧急事件的任务只是按其顺序运行。其次，它们不够灵活，因为调度顺序是在设计时确定的。第三，它们无法很好地扩展到大量任务。最后，单个行为异常的任务就会导致整个系统停止执行。这类调度器最适合非常简单的系统或高度安全关键的系统（因为简单系统比复杂系统更容易证明正确性并获得认证）。

另一种主要的调度类型是抢占式调度。在抢占式调度中，调度器在决定抢占时停止当前正在执行的任务，并运行下一个任务。抢占任务的准则以及按顺序选择下一个任务取决于特定的调度策略。

在高层次上，抢占式调度器中的任务有四种重要状态。当任务运行的条件当前不成立时，该任务被称为等待。当任务运行所需的条件为真时，该任务被称为就绪。当调度器启动该任务时，该任务被称为运行。当一个任务已就绪但因为有另一个任务拥有必要的资源而被阻止时，该任务被称为阻塞。

时间片调度器（也称为时间驱动多任务执行程序或TDME）基于公平原则运行；也就是说，每个任务都有平等的执行机会。任务在运行特定时间段后被抢占。TDME调度器通过调整时间片的持续时间来进行调优；时间片过长，其他任务运行不够频繁，无法完成其工作；时间片过短，则切换任务上下文所花费的时间占比过高。

对于较大的嵌入式系统，最常见的调度模式是基于优先级的调度。在这种方法中，每个任务被分配一个优先级，这是一个标量值，用于确定从就绪任务集合中将选择哪个任务执行。

A task running within either a time-slicing or priority-based preemptive scheduler runs in an infinite loop and will be interrupted by the scheduler when it is appropriate for other tasks to run. The basic structure for this latter task template looks like Code Listing 4-2. You can see in the code listing where the task static and stack data go. Following these data declarations, there is usually some initialization or set-up code followed by an infinite, or at least indefinite, loop where the task cycles endlessly waiting for a signal of interest – this might come from a queue insertion or event occurrence. When such a signal appears, a `switch case` statement then performs the processing. When complete, the task code loops back to wait for the next signal.

```
/*
      here is where private static data goes
*/
static int taskAInvocationCounter = 0;

void taskA(void) {
      /*  more private task data  */
      int stufftoDo;

      /*  initialization code  */
      stuffToDo  =  1;

      while (stuffToDo) {
            signal  =  waitOnSignal();
            switch (signal) {
                  case signal1:
                        /*  signal 1 processing here  */
                        break;
                  case signal2:
                        /*  signal 2 processing here  */
                  case signal3:
                        /*  signal 3 processing here  */
            };
      }; /*  end infinite while loop  */
}; /*  end task  */
```

Code Listing 4-2: Basic task structure with a preemptive scheduler

Priority can be set according to any criterion imaginable, but the two prevalent types of priority-based scheduling are urgency-based and criticality-based[2]. Many people use these terms imprecisely but the terms have precise meanings. *Urgency* refers to the nearness of a task's deadlines while *criticality* refers to how important the execution of the task is to the system. Figure 4-2 shows the difference. The curve itself is known as the utility function; this is the value to the system of the completion of an action or action sequence as a function of time.

[2] And, of course, these can be mixed in different ways to assign priorities, if desired.

在时间分片或基于优先级的抢占式调度器中运行的任务运行在无限循环中，当适合其他任务运行时将被调度器中断。后一种任务模板的基本结构如代码清单4-2所示。你可以在代码清单中看到任务静态数据和栈数据的位置。在这些数据声明之后，通常有一些初始化或设置代码，随后是一个无限（或至少是不定）循环，任务在其中无休止地循环等待感兴趣的信号——这可能来自队列插入或事件发生。当这样的信号出现时，一个`switch case`语句随后执行处理。完成后，任务代码循环回去等待下一个信号。

```
/* 这里是私有静态数据存放处 */ static int taskAInvocationCounter
= 0; void taskA(void) { /* 更多私有任务数据 */ int stufftoDo; /* 初始
化代码 */ stuffToDo  =  1; while (stuffToDo) { signal  =
waitOnSignal(); switch (signal) { case signal1: /* 此处处理信号1 */
break; case signal2: /* 此处处理信号2 */ case signal3: /* 此处处理信
号3 */ }; }; /* 无限while循环结束 */ }; /* 任务结束 */
```

代码清单4-2：使用抢占式调度器的基本任务结构

优先级可以根据任何可以想象的标准来设置，但两种普遍的基于优先级的调度类型是基于紧迫性的和基于关键性的[2]。许多人使用这些术语时并不精确，但这些术语有确切的含义。紧迫性指的是任务截止时间的临近程度，而关键性指的是任务的执行对系统有多重要。图4-2展示了这种区别。曲线本身被称为效用函数；这是动作或动作序列的完成作为时间的函数对系统的价值。

[2] 当然，如果需要，这些可以以不同方式混合来分配优先级。

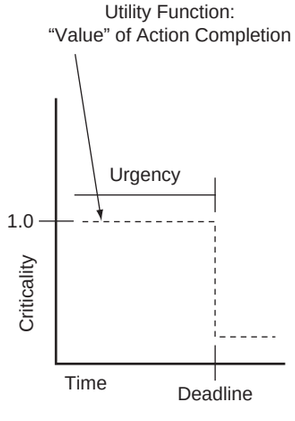


Figure 4-2: Criticality vs. urgency

The criticality is the height of the curve while urgency is the time distance to the point at which the criticality falls to zero. It should be noted that a task that is real-time does not necessarily have a deadline, but deadlines are the most common way to specify timeliness constraints for a real-time task.

When a task executes, there are several important time-related terms, summarized in Figure 4-3. For a periodic task (one that executes at a more-or-less fixed rate), two important values are the *period* (the duration of the time interval between task initiations) and the *jitter* (the variation in the period). Nonperiodic tasks (known commonly as *aperiodic tasks*) are characterized by the interarrival time (the time between task initiations) and the *minimum interarrival time* (the smallest such time). All tasks have an execution time (the time required to complete the activities of the task) although for computation of schedulability, the *worst-case execution time*

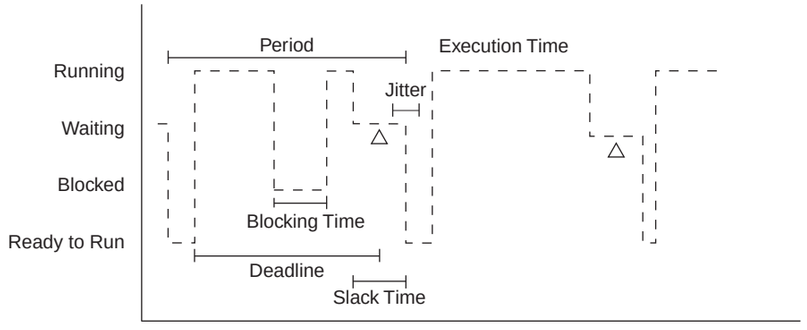


Figure 4-3: Time-related terms

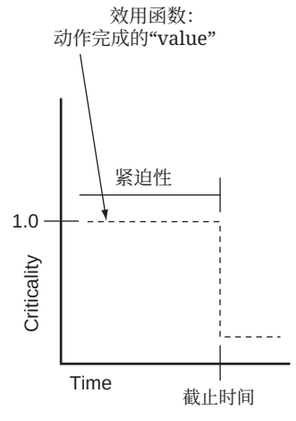


图4-2：关键性与紧迫性

关键性是曲线的高度，而紧迫性是到关键性归零那一点的时间距离。应当注意，实时任务不一定有截止时间，但截止时间是为实时任务指定时效性约束的最常见方式。

当一个任务执行时，有几个重要的时间相关术语，总结在图4-3中。对于周期性任务（以大致固定的速率执行的任务），两个重要的值是周期（任务启动之间的时间间隔持续时间）和抖动（周期的变化）。非周期性任务（通常称为非周期任务）的特征是到达间隔时间（任务启动之间的时间）和最小到达间隔时间（最小的此类时间）。所有任务都有执行时间（完成任务活动所需的时间），尽管为了计算可调度性，通常使用最坏情况执行时间

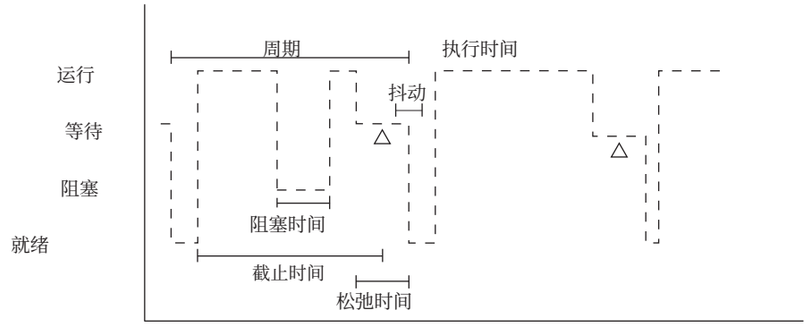


图4-3：时间相关术语

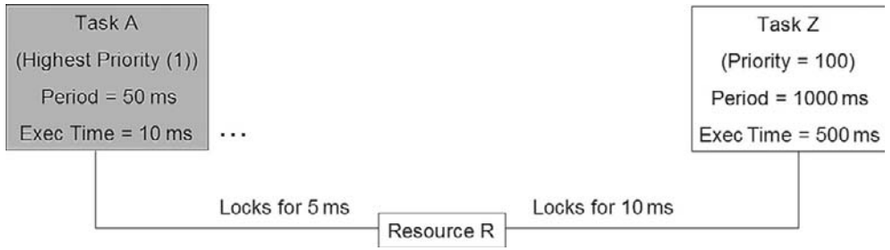


Figure 4-4: Blocking

is usually used. The *deadline* is the period of time between when the task becomes ready to run and when it must complete.

When you use semaphores to lock a resource, it can happen that a lower priority task locks a resource and then is preempted by a higher priority task that needs that resource. The high-priority task must block to allow the latter task to run and release the resource. This is known as *priority inversion* because, contrary to normal priority scheduling rules, the lower priority task is running even though there is a higher priority task ready to run.

Note that priority inversion cannot be avoided in multitasking preemptive schedule that includes the use of semaphores to guard resources. That isn't necessarily bad. What *is* almost always bad is unbounded priority inversion. Consider Figure 4-4. In this figure, there are two tasks. Task A is the higher priority[3], executing with a period of 50 ms and an execution time of 10 ms, 5 ms of which it locks resource R. Task Z is the lower priority task, with a period of 1000 ms and needing 500 ms of time to execute, 10 ms of which it locks resource R. Note that in this example, the deadline for each task occurs at the start of the next period.

Priority inversion occurs when Task Z is running with the resource R locked and then Task A becomes ready to run. It will immediately preempt Task Z but as soon as it gets to the point at which it needs to access the resource R it is "stuck"; it must block and allow Task Z to run until it releases the resource. Once the resource is release by Task Z, Task A again preempts Task Z and all is as it should be.

If schedulability of tasks is a concern (and it almost always should be), you should examine the effect of blocking on schedulability. In the case in Figure 4-4, what is the worst case? It happens when Task Z has just barely locked R and then Task A runs. In this case the total execution time for Task A is 10 ms plus the blocking time of 10 ms. Since Task A completes in 20 ms (less than the deadline) and the execution time for Z is 500 ms + 10 runs of Task A (at 10 ms each) = 600 ms, every one is, as they say "fat, dumb, and happy." Now consider Figure 4-5.

[3] Many Real-Time Operating Systems (RTOSs) use smaller numbers to indicate higher priority. Go figure…

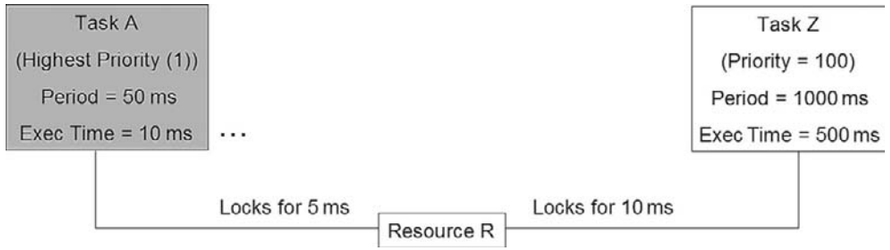


图 4-4：阻塞

。截止时间是任务变为就绪状态到必须完成之间的时间段。

当你使用信号量来锁定资源时，可能会出现这样的情况：一个较低优先级的任务锁定了某个资源，随后被一个需要该资源的更高优先级任务抢占。高优先级任务必须阻塞，以便让后者运行并释放该资源。这被称为优先级反转，因为与正常的优先级调度规则相反，即使有更高优先级的任务就绪，较低优先级的任务仍在运行。

请注意，在使用信号量来守卫资源的多任务抢占式调度中，优先级反转无法避免。这并不一定是坏事。几乎总是坏事的是无界优先级反转。考虑图 4-4。在该图中，有两个任务。任务A是较高优先级的[3]，，其周期为50 ms，执行时间为10 ms，其中5 ms用于锁定资源R。任务Z是较低优先级的任务，周期为1000 ms，需要500 ms的执行时间，其中10 ms用于锁定资源R。请注意，在此示例中，每个任务的截止时间出现在下一个周期的开始。

优先级反转发生在以下情况：任务Z正在运行并锁定了资源R，此时任务A变为就绪状态。任务A会立即抢占任务Z，但一旦它到达需要访问资源R的位置，它就会"卡住"；它必须阻塞并允许任务Z运行，直到任务Z释放该资源。一旦资源被任务Z释放，任务A会再次抢占任务Z，一切恢复正常。

如果任务的可调度性是一个关注点（而且几乎总是应该关注），你就应该考察阻塞对可调度性的影响。在图4-4的情况中，最坏情况是什么？它发生在任务Z刚刚加锁R之后任务A就运行时。在这种情况下，任务A的总执行时间为10 ms加上10 ms的阻塞时间。由于任务A在20 ms内完成（小于截止时间），而Z的执行时间为500 ms + 10 任务A的运行（每次10 ms）= 600 ms，每个任务都如他们所说"胖、笨且快乐"。现在考虑图4-5。

[3] 许多实时　　操作系统（RTOS）使用较小的数字表示较高的优先级。真是让人摸不着头脑.....

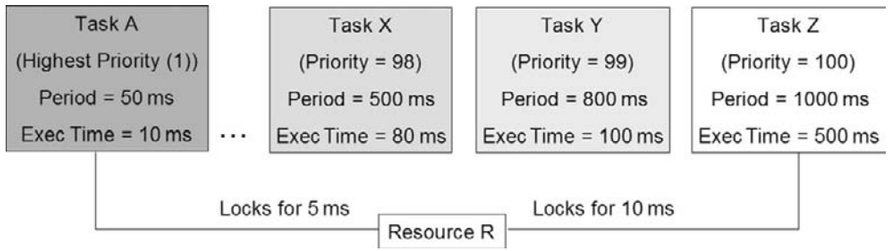


Figure 4-5: Unbounded blocking

In this example, only tasks A and Z use the resource. Task Y is the second-lowest priority task running every 800 ms for a total of 100 ms, while Task X is the third-lowest priority task with a period of 500 ms with an execution time of 80 ms. What happens to schedulability now?

The worst case is the following: Task Z runs and locks the resource and then Task A runs, preempts, runs until it needs to access R, and then blocks. At this point Task Z runs, but immediately Task Y runs. Since it is a higher priority, it preempts Z. As it runs, task X becomes ready to run and preempts Y. At this point, Task A is being blocked by three tasks. The resource will not become available for 190 ms (10+100+80), and Task A now misses its deadline. This is called *unbounded priority inversion* and is a critical problem with many embedded system designs.

There are a number of design solutions, such as using critical regions and variants of the priority inheritance pattern, that solve this problem. These patterns will be discussed later in this chapter.

4.1.1 Identifying Tasks

The entire previous discussion of what is a task and what are task properties avoids the crucial question of how to identify a good set of tasks for your particular embedded system. Generally speaking, good tasks have independent action sequences that interact with others only at specific points of interactions. However, selecting a good task set involves more than this because we create task sets to optimize performance and responsiveness. We could, in principle, run every object (or function, if you prefer) in a separate task thread or we can run them all in a single task thread. If ease of maintenance is the primary concern, then independence should be the primary task identification strategy. If schedulability is the primary concern, then the recurrence properties (e.g., the task period) should guide your choice. Table 4-1 shows common strategies for task identification[4].

In practice, the task set you select will use a combination of several of the strategies from Table 4-1.

[4] Borrowed (with permission) from the author's book *Real-Time Agility: The Harmony/ESW Method for Real-Time and Embedded Systems Development* (Addison-Wesley, 2009).

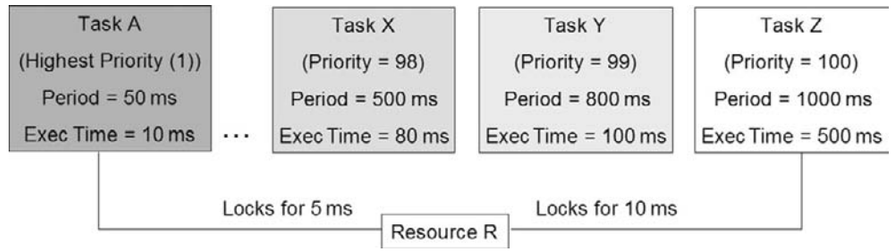


图4-5：无界阻塞

在这个示例中，只有任务A和Z使用该资源。任务Y是第二低优先级的任务，每800 ms运行一次，总共运行100 ms，而任务X是第三低优先级的任务，周期为500 ms，执行时间为80 ms。现在可调度性会发生什么变化?

最坏情况如下：任务Z运行并锁定资源，然后任务A运行，抢占，运行直到需要访问R，然后阻塞。此时任务Z运行，但任务Y立即运行。由于它具有更高的优先级，它抢占了Z。当它运行时，任务X变为就绪并抢占了Y。此时，任务A被三个任务阻塞。资源在190毫秒（10+100+80）内不会变为可用，任务A现在错过了它的截止时间。这被称为无界优先级反转，是许多嵌入式系统设计中的一个关键问题。

有许多设计方案可以解决这个问题，例如使用临界区和优先级继承模式的变体。这些模式将在本章后面讨论。

4.1.1 识别任务

前面关于什么是任务以及什么是任务属性的整个讨论回避了一个关键问题：如何为你的特定嵌入式系统识别一组好的任务。一般来说，好的任务具有独立的动作序列，仅在特定的交互点与其他任务交互。然而，选择好的任务集涉及的不仅仅是这些，因为我们创建任务集是为了优化性能和响应性。原则上，我们可以在单独的任务线程中运行每个对象（或者函数，如果你愿意），也可以将它们全部在一个任务线程中运行。如果易于维护是主要关注点，那么独立性应该是主要的任务识别策略。如果可调度性是主要关注点，那么循环属性（例如任务周期）应该指导你的选择。表4-1显示了任务识别的常见策略[4]。

在实践中，任务集 你将会使用表4-1中几种策略的组合 .

[4] 经作者许可，摘自其著作《实时敏捷：面向实时与嵌入式系统开发的Harmony/ESW方法》（Addison-Wesley，2009年）。

Table 4-1: Task Identification Strategies

Strategy	Description	Pros	Cons
Single Event Groups	Use a single event type per task	Very simple threading model	Doesn't scale to many events well; suboptimal performance
Interrupt Handler	Use a single event type to raise an interrupt	Simple to implement for handling a small set of incoming event types; highly efficient in terms of handling urgent events quickly	Doesn't scale well to many events; interrupt handles typically must be very short and atomic; possible to drop events; difficult to share data with other tasks
Event Source	Group all events from a single source so as to be handled by one task	Simple threading model	Doesn't scale to many event sources well; suboptimal performance
Related Information	Group all processing of information (usually around a resource) together in a task	Useful for "background" tasks that perform low-urgency processing	Same as Event Source
Independent Processing	Identify different sequences of actions as threads when there is no dependency between the difference sequences	Leads to simple tasking model	May result in too many or too few threads for optimality; doesn't address sharing of resources or task synchronization
Interface Device	A kind of event source	Useful for handling device (e.g., bus) interfaces	Same as Event Source
Recurrence Properties	Group together events of similar recurrence properties such as period, minimum interarrival time, or deadline	Best for schedulability; can lead to optimal scheduling in terms of timely response to events	More complex
Safety Assurance	Group special safety and reliability assurance behaviors together (e.g., watchdogs)	Good way to add on safety and reliability processing for safety-critical and high-reliability systems	Suboptimal performance

4.1.2 Concurrency in the UML

In the UML, there are three ways to represent concurrency: «active» objects, forks and joins in activity diagrams, and orthogonal regions in state machines[5]. Of these, the primary means is with «active» objects. An «active» object is one that owns a thread of execution. It is usually a structured object, meaning that it contains part objects inside itself that execute in the context of

[5] For more information about UML and concurrency, see the author's *Real-Time UML Third Edition: Advances in the UML for Real-Time Systems* (Addison-Wesley, 2004).

表 4-1：任务识别策略

策略	描述	Pros	Cons
单事件组	每个事件使用单一事件类型task	非常简单的线程模型	无法很好地扩展到大量事件；次优的性能
中断处理程序	使用单一事件类型来引发中断	易于实现，用于处理一小组传入事件类型；在以下方面非常高效处理紧急事件快速地	无法很好地扩展到大量事件；中断处理程序通常必须非常短且具有原子性；可以丢弃事件；难以与其他任务共享数据
事件源	将来自单一来源的所有事件分组，以便由一个任务处理	简单的线程模型	无法扩展到许多事件源良好；次优性能
相关信息	将所有处理信息（通常围绕某个资源）集中在一个任务中	适用于"后台"执行以下操作的任务低紧迫性处理	与事件源相同
独立处理	识别不同的序列将动作视为线程，当不存在依赖关系在不同序列之间	导致简单的任务模型	可能导致线程过多或线程过少，无法达到最优状态；未解决资源共享或任务同步
接口设备	一种事件源	适用于处理设备（例如，总线）接口	与事件源相同
递归属性	将以下事件分组在一起相似递归诸如周期、最小到达间隔时间或截止时间等属性，	最有利于可调度性；可能实现最优调度在及时响应方面对事件	更复杂
安全性保障	将特殊安全性与可靠性保证分组行为一起（例如，看门狗）	添加安全性与可靠性处理的好方法用于安全关键型和高可靠性系统	性能欠佳

4.1.2 UML中的并发

在UML中，有三种方式来表示并发：«active»对象、活动图中的分叉与汇合，以及状态机中的正交区域[5]。其中，主要手段是使用«active»对象。«active»对象是拥有一个执行线程的对象。它通常是一个结构化对象，这意味着它内部包含在其上下文中执行的部件对象

[5] 有关UML和并发的更多信息，请参阅作者的Real-Time UML Third Edition: Advances in the UML for Real-Time Systems（Addison-Wesley，2004年）。

«active» object's thread. The «active» object has the explicit responsibility of managing the event queue for the set of software elements that execute in its thread's context. The icon for an «active» object is an object with a heavy border on its left and right sides although often the stereotype «active» is shown instead. A *task diagram* is a UML class (or object[6]) diagram whose primary focus is on the concurrency units and their interaction. Figure 4-6 shows an example of an object task diagram containing five tasks («active» objects), one shared resource (with guarding semaphore), and comments showing concurrency-related metadata (priority, period, and deadline in this case[7]). Note that with some tasks we use ports (named connection points) to connect directly with objects inside the tasks while in other cases we just use links

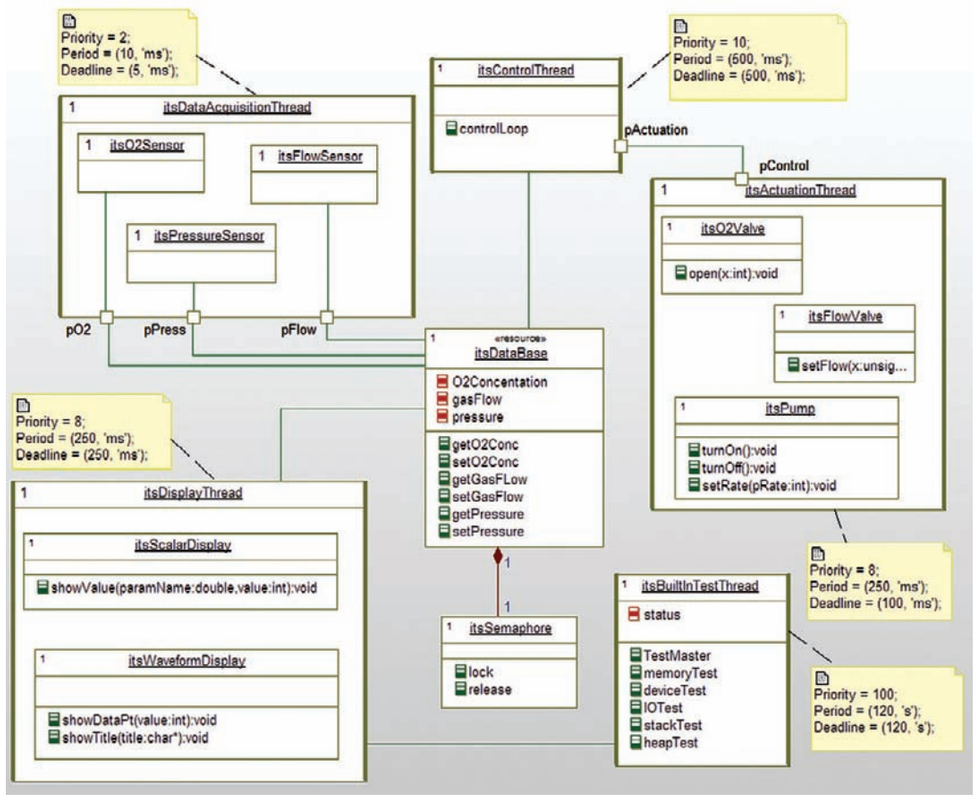


Figure 4-6: UML task diagram

[6] Remember from Chapter 1 that a class is a specification and an object is an instance. An «active» class is similar to a task type in Ada while an object is similar to an Ada task.

[7] To perform schedulability analysis we need additional metadata such as worst-case execution time and, since we have shared resources with blocking, worst-case blocking times.

«active»对象的线程。「active»对象明确负责管理在其线程上下文中执行的一组软件元素的事件队列。「active»对象的图标是一个左右两侧带有粗边框的对象，不过通常显示的是构造型«active»。任务图是一种UML类（或对象[6]）图，其主要关注点是并发单元及其交互。图4-6展示了一个对象任务图的示例，其中包含五个任务（«active»对象）、一个共享资源（带有保护信号量），以及展示并发相关元数据的注释（本例中为优先级、周期和截止时间[7]）。请注意，对于某些任务，我们使用端口（命名的连接点）直接与任务内部的对象连接，而在其他情况下我们只使用链接

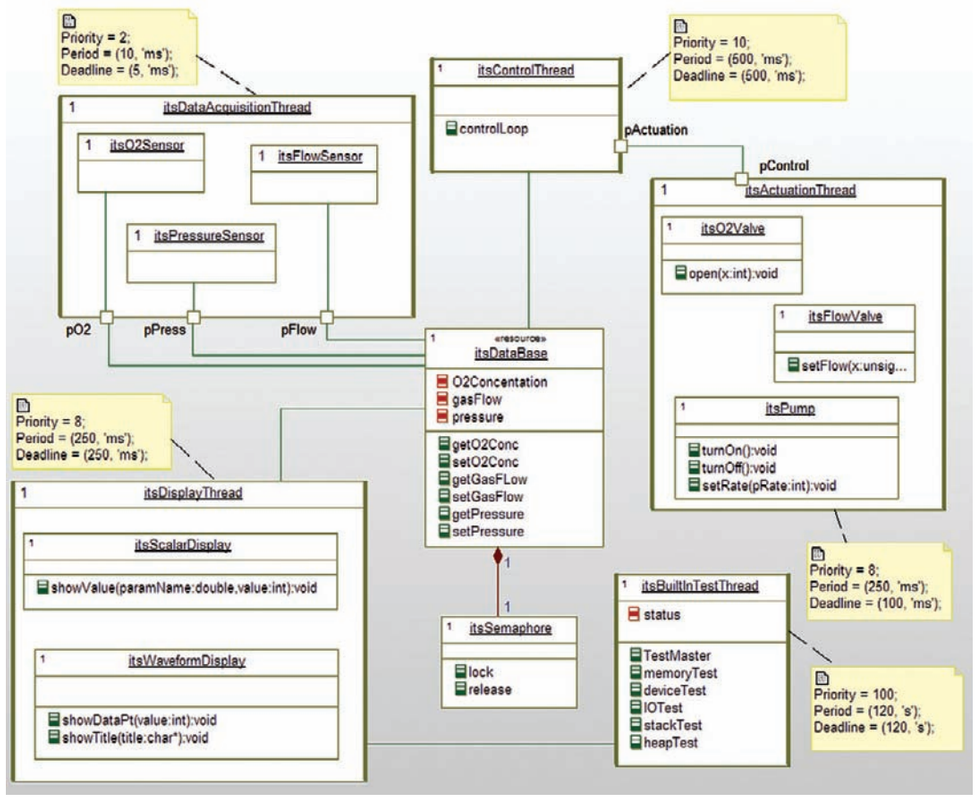


图 4-6：UML 任务图

[6] 回顾第1章，类是规范，对象是实例。「活动」类类似于 Ada 中的任务类型，而对象类似于 Ada 任务。

[7] 为了进行可调度性分析，我们需要额外的元数据，例如最坏情况执行时间，并且由于我们拥有带阻塞的共享资源，还需要最坏情况阻塞时间。

(most likely implemented as pointers). In the case of the `ControlThread` and the `BuildInTestThread`, these «active» objects do not contain internal objects but are objects that run in their own threads.

4.1.3 Real-Time Operating Systems

In many embedded systems, a real-time operating system (RTOS[8]) provides a set of services to the application system including:

- task services such as the creation, scheduling, and destruction of tasks
- event services such as pending on an event, posting, and event and management of event queues
- creation, destruction, and usage of mutex semaphores
- time services, such as delay for a specified period of time, and the creation and destruction of timers
- file system services
- network input/output services

This is no surprise – these services are offered by most desktop operating systems as well. RTOSs are different in a number of ways. First and foremost, they are embeddable; they are designed to be burned into ROM, Flash, or some nonvolatile memory. They may execute from that memory or copy themselves into volatile memory via a boot loader. Secondly, they are tuned to be both fast and predictable[9]. Thirdly, they are usually designed to be scalable; they are often designed using the Microkernel Architecture Pattern[10] and that enables the memory image of the RTOS to include only those services that the system needs and for which it is willing to allocate space.

Figure 4-7 shows a typical structure for an RTOS. An RTOS must be specialized for the target hardware. This is done via a hardware abstraction layer that most RTOS vendors refer to as the Board Support Package (BSP). Above this are kernel services such as memory, task, and timer services. Optionally, file services and network services may be added or removed as needed by the application. The various applications and their tasks run atop the RTOS, invoking the RTOS services via an application programming interface (API) as necessary.

It is now fairly common to embedded such RTOSs, but it wasn't so long ago that the application software had to provide these services. Indeed, many embedded systems running in

[8] Wind River's VxWorks™ and Green Hills' Integrity™ operating systems are popular RTOSs but there are more than 400 RTOSs in use.

[9] Predictability is a scale with chaos on one end and determinism on the other. RTOSs tend toward the deterministic end of the spectrum.

[10] See the author's *Real-Time Design Patterns: Robust Scalable Architecture for Real-Time Systems* (Addision-Wesley, 2003) for a detailed description of the pattern.

（最可能实现为指针）。对于`ControlThread`和`BuildInTestThread`，这些«active»对象不包含内部对象，而是在各自线程中运行的对象。

4.1.3 实时操作系统

在许多嵌入式系统中，一个实时操作系统(RTOS[8])为应用系统提供一组服务，包括：

- 任务服务，例如任务的创建、调度和销毁
- 事件服务，例如挂起等待事件、发布事件以及事件队列的管理
- 互斥信号量的创建、销毁和使用
- 时间服务，例如指定时间段的延时，以及定时器的创建和销毁
- 文件系统服务
- 网络输入/输出服务

这并不奇怪——大多数桌面操作系统也提供这些服务。实时操作系统在多个方面有所不同。首先也是最重要的，它们是可嵌入的；它们被设计为烧录到ROM、闪存或某种非易失性存储器中。它们可以从该存储器中执行，也可以通过引导加载程序将自己复制到易失性存储器中。其次，它们被调优为既快速又可预测[9]。第三，它们通常被设计为可伸缩的；它们通常使用微内核架构模式[10]进行设计，这使得实时操作系统的内存映像能够仅包含系统所需且愿意为其分配空间的那些服务。

图4-7展示了实时操作系统的典型结构。实时操作系统必须针对目标机硬件进行专门化。这是通过硬件抽象层实现的，大多数实时操作系统供应商将其称为板级支持包（BSP）。在此之上是内核服务，如内存、任务和定时器服务。可选地，文件服务和网络服务可以根据应用程序的需要添加或移除。各种应用程序及其任务运行在实时操作系统之上，根据需要经由应用程序编程接口（API）调用实时操作系统服务。

如今嵌入这样的实时操作系统已相当普遍，但不久之前，应用软件还必须自行提供这些服务。事实上，许多运行在

[8] Wind River 的 VxWorksTM 和 Green Hills 的 IntegrityTM 操作系统是流行的实时操作系统，但实际使用的实时操作系统超过 400 种。[9] 可预测性是一个尺度，一端是混沌，另一端是确定性。实时操作系统倾向于确定性的一端。[10] 关于该模式的详细描述，请参阅作者的 Real-Time Design Patterns: Robust Scalable Architecture for Real-Time Systems（Addision-Wesley，2003）。

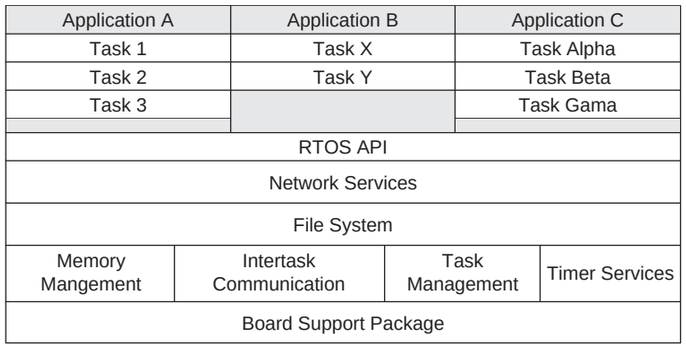


Figure 4-7: Basic RTOS structure

smaller memory space and processor target environments cannot afford the time, memory, or recurring cost overhead of commercial RTOSs.

Let's now look at some patterns useful for managing the concurrency aspects of embedded systems. It should be noted that the Interrupt Pattern of the previous chapter is a concurrency pattern in addition to a hardware access pattern.

Scheduling Patterns

This section contains patterns related to the invocation of tasks. This is a service that an RTOS will provide, if available. If not, then your application software must provide these services. Even if your system uses an RTOS to provide scheduling, understanding the benefits and costs of the different scheduling approaches gives you more information to select an optimal scheduling strategy.

4.2 Cyclic Executive Pattern

As mentioned earlier in this chapter, the Cyclic Executive Pattern has the advantage of being a very simple approach to scheduling multiple tasks but isn't very flexible or responsive to urgent events. The Cyclic Executive Pattern implements a fairness doctrine for scheduling that allows all tasks an equal opportunity to run. Although suboptimal in terms of responsiveness, the Cyclic Executive Pattern is easy to analyze for schedulability (that is, the ability of the system to predictably meet all task deadlines): the deadline for each task must be greater than or equal to the sum of all the worst-case execution times for all the tasks plus the loop overhead. Assuming that the deadline for each task is the duration of one complete task execution cycle, Equation 4-1 gives the relation that must be true for the task set to be schedulable, where:

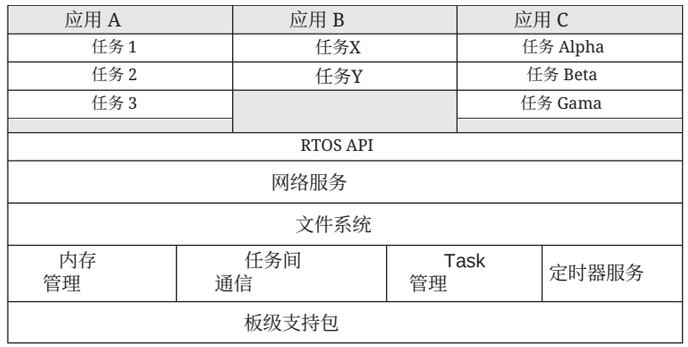


图4-7：基本RTOS结构

较小内存空间和处理器目标环境中的嵌入式系统无法承受商用实时操作系统的时间、内存或经常性成本开销。

现在让我们来看一些有助于管理嵌入式系统并发方面的模式。应当指出，上一章的中断模式除了是硬件访问模式之外，也是一种并发模式。

调度模式

本节包含与任务调用相关的模式。这是实时操作系统（RTOS）在可用时提供的一项服务。如果不可用，则您的应用软件必须提供这些服务。即使您的系统使用RTOS来提供调度，理解不同调度方法的优势与成本也能为您提供更多信息，以选择最优调度策略。

4.2 循环执行模式

正如本章前面所述，循环执行模式具有作为调度多任务的一种非常简单方法的优势，但不太灵活，对紧急事件的响应性也不佳。循环执行模式实现了一种调度公平原则，允许所有任务具有平等的运行机会。尽管在响应性方面并非最优，但循环执行模式易于进行可调度性分析（即系统可预测地满足所有任务截止时间的能力）：每个任务的截止时间必须大于或等于所有任务的所有最坏情况执行时间之和加上循环开销。假设每个任务的截止时间是一个完整任务执行周期的持续时间，则公式4-1给出了任务集可调度所必须成立的关系，其中：

- D_j is the deadline for task j
- C_j is the worst-case execution time for task j
- K is the cyclic executive loop overhead, including the overhead of the task invocation and return

for all tasks i, $D_i \geq \sum_{j=1}^{n} C_j + K$

Equation 4-1: Schedulability for Cyclic Executive with *n* tasks

4.2.1 Abstract

The Cyclic Executive Pattern is used primarily in two situations. First, for very small embedded applications, it allows multiple tasks to be run pseudoconcurrently without the overhead of a complex scheduler or RTOS. Secondly, in high-safety relevant systems, the Cyclic Executive Pattern is easy to certify, hence its prevalence in avionics and flight management systems. The scheduler is a simple loop that calls each of the tasks in sequence. Each task is simply a function invoked by the scheduler that runs to completion and then returns.

4.2.2 Problem

Many embedded systems are tiny applications that have extremely tight memory and time constraints and cannot possibly host even a scaled-down microkernel RTOS. The Cyclic Executive Pattern provides a low-resource means to accomplish this goal.

4.2.3 Pattern Structure

Figure 4-1 shows the structure of this simple pattern. The CyclicExecutive has a `controlLoop()` function that iteratively invokes the `run()` operation of each of the task threads. Each `run()` operation is relatively short and runs to completion.

4.2.4 Collaboration Roles

This section describes the roles for this pattern.

4.2.4.1 AbstractCEThread

The AbstractCEThread provides a standard interface for the threads by declaring a run() function. It is this function that the Cyclic Executive will invoke to execute the task. When this function completes, the CyclicExecutive runs the next task in the list.

4.2.4.2 CyclicExecutive

As mentioned in Section 4-1, this class contains the infinite loop that runs the tasks each in turn. In addition, the CyclicExecutive contains global stack and static data that are needed

• D_j 是任务j的截止时间 • C_j 是任务j的最坏情况执行时间 • K是循环执行体循环开销，包括任务调用和返回的开销

for all tasks i, $D_i \geq \sum_{j=1}^{n} C_j + K$

公式 4-1：n 个任务的循环执行程序可调度性

4.2.1 摘要

循环执行模式主要用于两种情况。首先，对于非常小的嵌入式应用，它允许多个任务以伪并发方式运行，而无需复杂调度器或实时操作系统的开销。其次，在高安全相关系统中，循环执行模式易于认证，因此它在航空电子和飞行管理系统中十分普遍。调度器是一个简单的循环，按顺序调用每个任务。每个任务只是一个由调度器调用的函数，运行至完成后返回。

4.2.2 问题

许多嵌入式系统是极小的应用，具有极其紧张的内存和时间约束，甚至不可能承载一个缩减版的微内核实时操作系统。循环执行模式提供了一种低资源的方式来实现这一目标。

4.2.3 模式结构

图 4-1 展示了这个简单模式的结构。CyclicExecutive 具有一个`controlLoop()`函数，该函数迭代地调用每个任务线程的 `run()` 操作。每个 `run()` 操作相对较短，并且运行至完成。

4.2.4 协作角色

本节描述该模式的角色。

4.2.4.1 AbstractCEThread

AbstractCEThread 通过声明 run() 函数为线程提供标准接口。循环执行程序将调用此函数来执行任务。当此函数完成时，CyclicExecutive 运行列表中的下一个任务。

4.2.4.2 CyclicExecutive

如第4-1节所述，该类包含依次运行各任务的无限循环。此外，CyclicExecutive包含所需的全局栈和静态数据

either by the tasks or by the scheduler itself. The code in Figure 4-4.1 is typical for a cyclic executive.

A variant of the pattern is the time-triggered Cyclic Executive. In this variant, the `CyclicExecutive` will set up and use the `CycleTimer` to initiate each epoch (execution of the task list). Note that while the Interrupt Pattern from Chapter 3 can be and the Cyclic Executive can simply wait for the interrupt to do the next cycle, unless there is background processing to perform, it can sit in a so-called "busy loop" and poll for the timer to elapse.

4.2.4.3 CycleTimer

The `CycleTimer` isn't used in the most common variant of the `CyclicExecutive`; the fact that it is optional is indicated with the "0,1" multiplicity on the directed association in Figure 4-8. This timer can invoke an interrupt when it expires or, even more simply, can return TRUE (non-zero) to the `hasElapsed()` function. The `CyclicExecutive` would then call `start()` to begin the timing interval for the next epoch.

4.2.5 ConcreteCEThread

The ConcreteCEThread stands for the functions and related data that implement a logical task in the system. Each has its own implementation of the `run()` function.

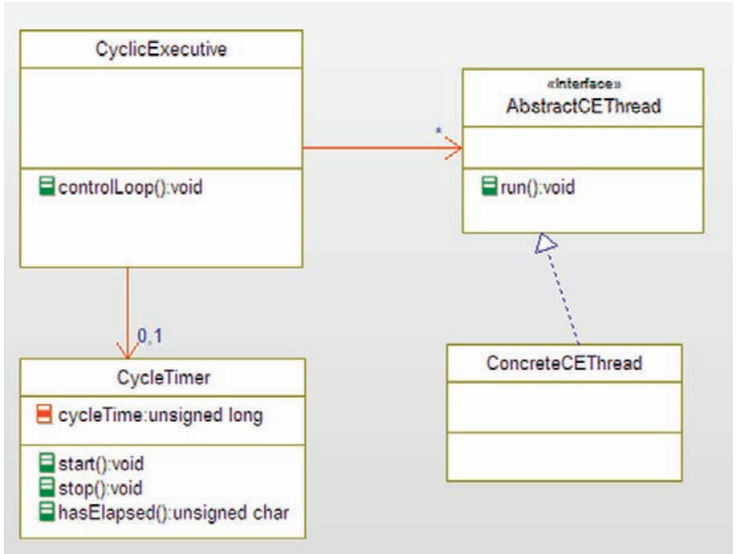


Figure 4-8: Cyclic Executive Pattern

这些 数据 可能 由任务 或 由 调度器 本身 使用。 图 4-4.1 中 的 代码 是 循环执行程序 的 典型 代码。

该模式的一个变体是时间触发循环执行程序。在这个变体中，`CyclicExecutive` 将设置并使用`CycleTimer` 来启动每个纪元（任务列表的执行）。请注意，虽然第3章中的中断模式可以做到这一点，并且循环执行程序可以简单地等待中断来进行下一个周期，但除非有后台处理要执行，否则它可以坐在所谓的"忙循环"中并轮询定时器是否已到期。

4.2.4.3 CycleTimer

`CycleTimer` 并未用于`CyclicExecutive`最常见的变体中；它是可选的，这一点通过图 4-8中定向关联上的"0,1"多重性来表示。该定时器可以在到期时调用中断，或者更简单地说，可以向`hasElapsed()` 函数返回TRUE（非零）。然后`CyclicExecutive` 将调用`start()` 来开始下一个纪元的计时间隔。

4.2.5 ConcreteCEThread

ConcreteCEThread 代表在系统中实现逻辑任务的功能和相关数据。每个都有自己的 `run()` 函数实现。

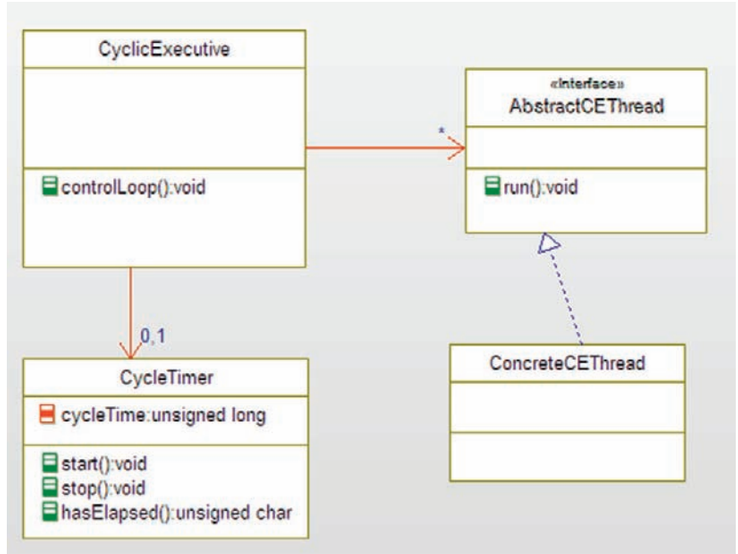


图 4-8：循环执行模式

4.2.6 Consequences

As mentioned, the primary benefit of this pattern is its simplicity. It's pretty difficult to get the scheduler wrong. On the other hand, the pattern is demonstrably less responsive to high-urgency events and this makes the pattern inappropriate for applications that have high responsiveness requirements. Another benefit of the pattern is that it is very lightweight in terms of required resources and so is appropriate for very small memory devices.

A downside of using the pattern is that the interaction of threads is made more challenging than other patterns. If an output of one task is needed by another, the data must be retained in global memory or a shared resource until that task runs (See the Task Coordination Patterns section later in this chapter). On the other hand, because of the lack of preemption, unbounded blocking is not a concern. Deadlock can only occur because of a mistake – a single misbehaved task (i.e., one that never returns control to the cyclic executive) shuts the entire system down. In a preemptive scheduling system, unless the misbehaving task has turned off task switching, the other tasks will continue to function even if one task fails.

4.2.7 Implementation Strategies

The implementation of this pattern is almost trivially simple. In most cases, the Cyclic Executive may simply be the application's `main()` function and related global data. In other cases, it will be invoked from `main()`.

4.2.8 Related Patterns

Because the pattern's responsiveness is demonstrably suboptimal, it is often augmented with some interrupt service routines (see the Interrupt Pattern in Chapter 3) for high urgency events. Another use for the Interrupt Pattern with the Cyclic Executive Pattern is to implement the epoch timer response with the time-triggered cyclic executive, although the Polling Pattern (also from Chapter 3) may be used to poll the timer as well.

The Cyclic Executive Pattern does not address data sharing between tasks. The Cyclic Executive does define global data structures that can be used for this purpose but it is possible to mix other patterns for this purpose (see the Task Coordination patterns section in this chapter). Because there is no preemption, global data do not require access serialization with this pattern.

4.2.9 Example

The example shown in Figure 4-9 shows the cyclic executive scheduler `GasControlExecutive`. Its `controlLoop()` function calls the run services in each of the threads. Because we used

4.2.6 结果

如前所述，该模式的主要优点是其简单性。调度器很难出错。另一方面，该模式对高紧急事件的响应性明显较差，这使得该模式不适用于具有高响应性需求的应用。该模式的另一个优点是，就所需资源而言，它非常轻量，因此适用于非常小的存储设备。

使用该模式的一个缺点是，线程之间的交互比其他模式更具挑战性。如果一个任务的输出需要被另一个任务使用，数据必须保留在全局内存或共享资源中，直到该任务运行（参见本章后面的任务协调模式部分）。另一方面，由于没有抢占，无界阻塞不是问题。死锁只会因错误而发生——单个行为不当的任务（即从不将控制权返回给循环执行程序的任务）会导致整个系统关闭。在抢占式调度系统中，除非行为不当的任务关闭了任务切换，否则即使一个任务失败，其他任务也会继续运行。

4.2.7 实现策略

该模式的实现几乎非常简单。在大多数情况下，循环执行程序可以只是应用程序的 `main()` 函数及相关全局数据。在其他情况下，它将从 `main()` 中调用。

4.2.8 相关模式

由于该模式的响应性明显欠佳，因此通常会针对高紧迫性事件辅以一些中断服务例程（参见第3章的中断模式）。中断模式与循环执行模式配合使用的另一个用途是，通过时间触发循环执行程序来实现纪元定时器响应，不过也可以使用轮询模式（同样来自第3章）来轮询定时器。

循环执行模式不涉及任务之间的数据共享。循环执行程序确实定义了可用于此目的的全局数据结构，但也可以混合使用其他模式来实现此目的（参见本章的任务协调模式部分）。由于不存在抢占，因此使用该模式时全局数据不需要访问串行化。

4.2.9 示例

图 4-9 所示的示例 展示了循环执行调度器 `GasControlExecutive`。其 `controlLoop()` 函数会调用每个线程中的运行服务。因为我们使用了

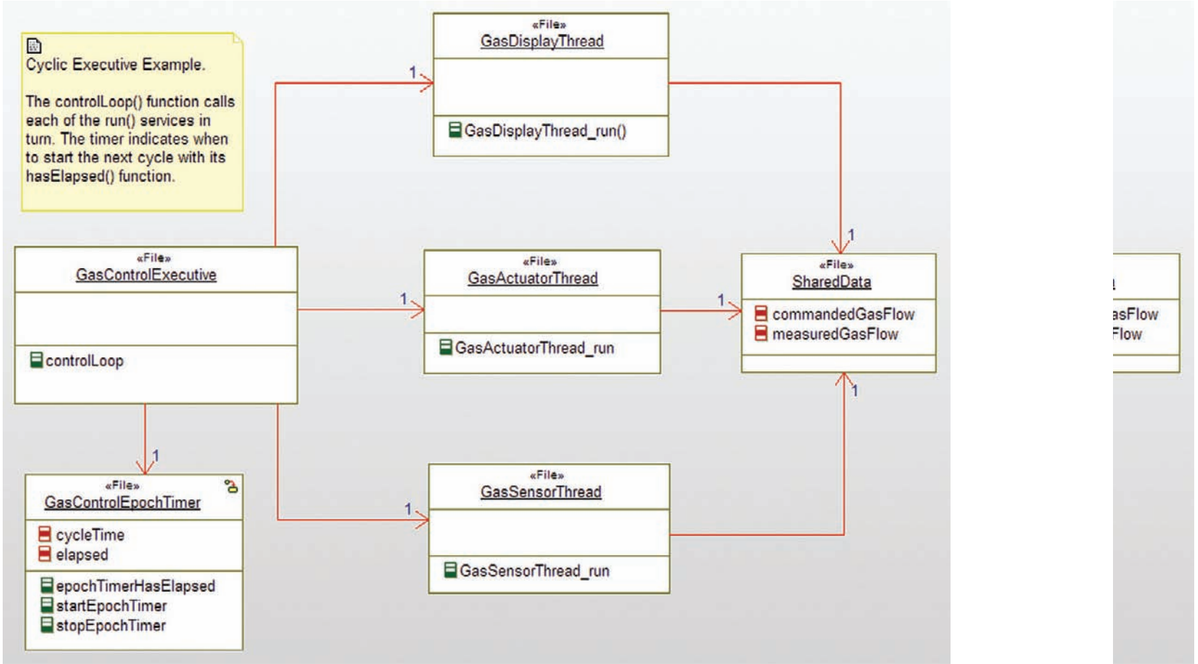


Figure 4-9: Cyclic Executive Example

the file implementation, it was necessary to encode the name of the file in the name of the run function of each to avoid global naming clash. In the example, I implemented the time-triggered variant of the pattern. As you can see in Code Listing 4-3, the `controlLoop()` function implements a busy wait for the elapse of the epoch time. Note that the timer `startEpochTimer()` function resets the elapsed variable to `FALSE (0)` before it starts the timer. When the timer elapses, this variable is set to `TRUE (1)`.

```
#include "GasControlExecutive.h"
#include "GasActuatorThread.h"
#include "GasControlEpochTimer.h"
#include "GasDisplayThread.h"
#include "GasSensorThread.h"
void controlLoop(void) {
    while (TRUE) {
      startEpochTimer();
      GasSensorThread_run();
      GasActuatorThread_run();
      GasDisplayThread_run();
      while(!epochTimerHasElapsed()) ;
    };
}
```

Code Listing 4-3: GasControlExecutive.c

Each of the code files for the other elements simply declare their data and functions and the related C file provides the implementation. For example, the implementation for the SharedData file is given in Code Listing 4-4 and Code Listing 4-5.

```
#ifndef SharedData_H
#define SharedData_H

extern int commandedGasFlow;
extern int measuredGasFlow;

#endif
```

Code Listing 4-4: SharedData.h

```
#include "SharedData.h"
int commandedGasFlow;
int measuredGasFlow;
```

Code Listing 4-5: SharedData.c

在文件实现中，有必要将文件名编码到每个运行函数的名称中，以避免全局命名冲突。在示例中，我实现了该模式的时间触发变体。如代码清单4-3所示，`controlLoop()`函数实现了对纪元时间流逝的忙等待。请注意，定时器`startEpochTimer()`函数在启动定时器之前将elapsed变量重置为`FALSE (0)`。当定时器到期时，该变量被设置为`TRUE (1)`。

```
#include "GasControlExecutive.h" #include "
GasActuatorThread.h" #include "
GasControlEpochTimer.h" #include "
GasDisplayThread.h" #include "
GasSensorThread.h" void controlLoop(void)
{ while (TRUE) { startEpochTimer();
GasSensorThread_run();
GasActuatorThread_run();
GasDisplayThread_run(); while(!
epochTimerHasElapsed()) ; }; }
```

代码清单 4-3：GasControlExecutive.c

其他各元素的代码文件仅声明其数据和函数，相关的 C 文件则提供实现。例如，SharedData 文件的实现见代码清单 4-4 和代码清单 4-5。

```
#ifndef SharedData_H #define
SharedData_H extern int
commandedGasFlow; extern int
measuredGasFlow;


#endif
```

代码清单 4-4：SharedData.h

```
#include "SharedData.h" int
commandedGasFlow; int
measuredGasFlow;
```

代码清单 4-5：SharedData.c

```
void GasDisplayThread_run(void) {
    printf("Measured Gas Flow %d\n", measuredGasFlow);
    printf("commandedGasFlow %d\n\n", commandedGasFlow);
}
```

Code Listing 4-6: GasDisplayThread.c

4.3 Static Priority Pattern

The Static Priority Pattern is supported by most real-time operating systems. It has good response to high-priority events and scales well to large numbers of tasks. Different schemas can be used to assign priorities, but, as mentioned earlier, priorities are usually assigned based on urgency, criticality, or some combination of the two. When assigned using rate monotonic scheduling (see the Implementation Strategies section below), this scheduling pattern is both optimal (you can't do better with other scheduling strategies) and stable (in an overload situation, which tasks will fail is predictable – the lowest priority tasks). Assuming that each task deadline occurs at the end of the period, the system is schedulable if the inequality in Equation 4-2 holds, where:

- C_j is the worst case execution time for task j
- T_j is the period (deadline occurs at the end of the period) for task j
- B_j is the worst-case blocking time for task j
- n is the number of tasks

$$\sum_j \frac{C_j}{T_j} + max\left(\frac{B_1}{T_1}, \cdots, \frac{B_{n-1}}{T_{n-1}}\right) \leq utilization(n) = n(2^{\frac{1}{n}} - 1)$$

Equation 4-2: Schedulability for Static Priority Pattern with RMS priorities for *n* tasks[11]

In the case where the task is not periodic, the minimum interarrival time for task j (the smallest time between successive invocation of the same task) is used. Note that the blocking terms only extend to the n-1 task; this is because by definition, the lowest priority task cannot be blocked.

4.3.1 Abstract

The Static Priority Pattern is a very common pattern in embedded systems because of its strong RTOS support, ease of use, and good responsiveness to urgent events. It may be overkill for very small, simple, or highly predictable systems that are not driven by urgent asynchronous events. The pattern is easy to analyze for schedulability but naïve implementation with blocking resource sharing can lead to unbounded priority inversion. Fortunately, there are resource sharing patterns that can avoid this problem or at least bound priority inversion to at

[11] For a proof of this equation, see Lui and Leyand, Scheduling algorithms for multiprogramming in a hard real-time environment. *Journal of ACM* 20, January 1973, 40–61.


```
void GasDisplayThread_run(void) { printf("Measured Gas Flow %
d\n", measuredGasFlow); printf("commandedGasFlow %d\n\n",
commandedGasFlow); }
```

代码清单 4-6：GasDisplayThread.c

4.3 静态优先级模式

静态优先级模式受到大多数实时操作系统的支持。它对高优先级事件具有良好的响应能力，并且能够很好地扩展到大量任务。可以使用不同的方案来分配优先级，但如前所述，优先级通常根据紧迫性、关键性或两者的某种组合来分配。当使用速率单调调度（参见下面的实现策略部分）进行分配时，这种调度模式既是最优的（使用其他调度策略无法做得更好），也是稳定的（在过载情况下，哪些任务会失败是可预测的——最低优先级的任务）。假设每个任务的截止时间发生在其周期结束时，如果等式4-2中的不等式成立，则系统是可调度的，其中：

- C_j 是任务 j 的最坏情况执行时间 • T_j 是任务 j 的周期（截止时间发生在周期结束时） • B_j 是任务 j 的最坏情况阻塞时间 • n 是任务数量

$$\sum_j \frac{C_j}{T_j} + max\left(\frac{B_1}{T_1}, \cdots, \frac{B_{n-1}}{T_{n-1}}\right) \leq utilization(n) = n(2^{\frac{1}{n}} - 1)$$

公式 4-2：调度　　具有 RMS 优先级的静态优先级模式对 n 个任务的可调度性[11]

在任务非周期性的情况下，使用任务 j 的最小到达间隔时间（同一任务连续调用之间的最小时间）。注意，阻塞项仅延伸到 n-1 个任务；这是因为根据定义，最低优先级任务不能被阻塞。

4.3.1 摘要

静态优先级模式是嵌入式系统中非常常见的模式，因为它具有强大的RTOS支持、易于使用，并且对紧急事件具有良好的响应性。对于非常小、简单或高度可预测且不受紧急异步事件驱动的系统来说，它可能过于复杂。该模式易于进行可调度性分析，但采用阻塞资源共享的朴素实现可能导致无界优先级反转。幸运的是，有一些资源共享模式可以避免此问题，或至少将优先级反转限制在至

[11] 关于此公式的证明，请参见 Lui 和 Leyand，Scheduling algorithms for multiprogramming in a hard real-time environment。ACM杂志 20，1973年1月，40–61。

most one level. The most common way to assign priorities is on the basis of deadline duration (that is, task urgency); the shorter the deadline, the higher the priority. If the deadline is at the end of the period, this schema is known as *rate monotonic scheduling*. It assumes that all tasks are periodic, but the minimum interarrival interval can be used for aperiodic tasks, although this often results in overdesigning the system with an impact on system recurring cost.

4.3.2 Problem

The Static Priority Pattern provides a simple approach to scheduling tasks based on priority. This pattern addresses systems with more tasks than the Cyclic Executive Pattern and also emphasizes responsiveness to urgent events over fairness.

4.3.3 Pattern Structure

Figure 4-10 shows the structure for the Static Priority Pattern. The colored classes indicate the element normally implemented for you by an RTOS. Note that the mapping constraint on the diagram indicates that there is a `StaticTaskControlBlock` for each `AbstractStaticThread`. This holds data about the task needed by the scheduler to support scheduling and preemption.

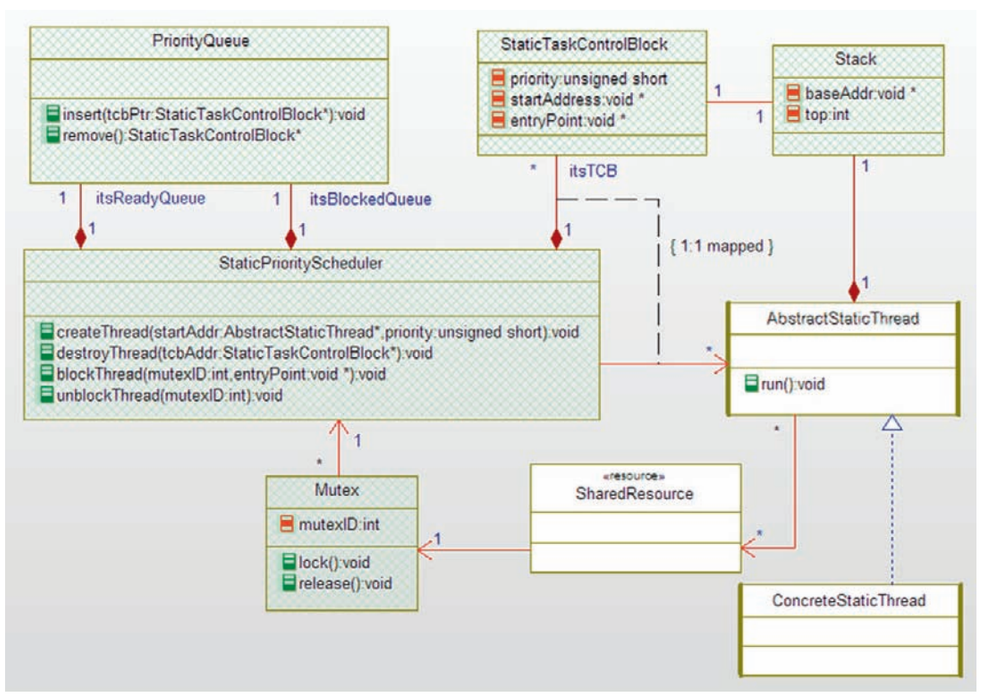


Figure 4-10: Static Priority Pattern

多一个级别。分配优先级最常见的方法是基于截止期限时长（即任务紧迫性）；截止时间越短，优先级越高。如果截止时间在周期末尾，这种方案被称为速率单调调度。它假设所有任务都是周期性的，但对于非周期任务可以使用最小到达间隔，尽管这通常会导致系统过度设计，从而影响系统经常性成本。

4.3.2 问题

静态优先级模式提供了一种基于优先级调度任务的简单方法。该模式适用于任务数量多于循环执行模式的系统，并且强调对紧急事件的响应性优先于公平性。

4.3.3 模式结构

图 4-10 展示了静态优先级模式的结构。彩色类表示通常由实时操作系统为你实现的元素。请注意，图上的映射约束表明，每个 `AbstractStaticThread` 都有一个 `StaticTaskControlBlock`。它保存了调度器支持调度和抢占所需的任务数据。

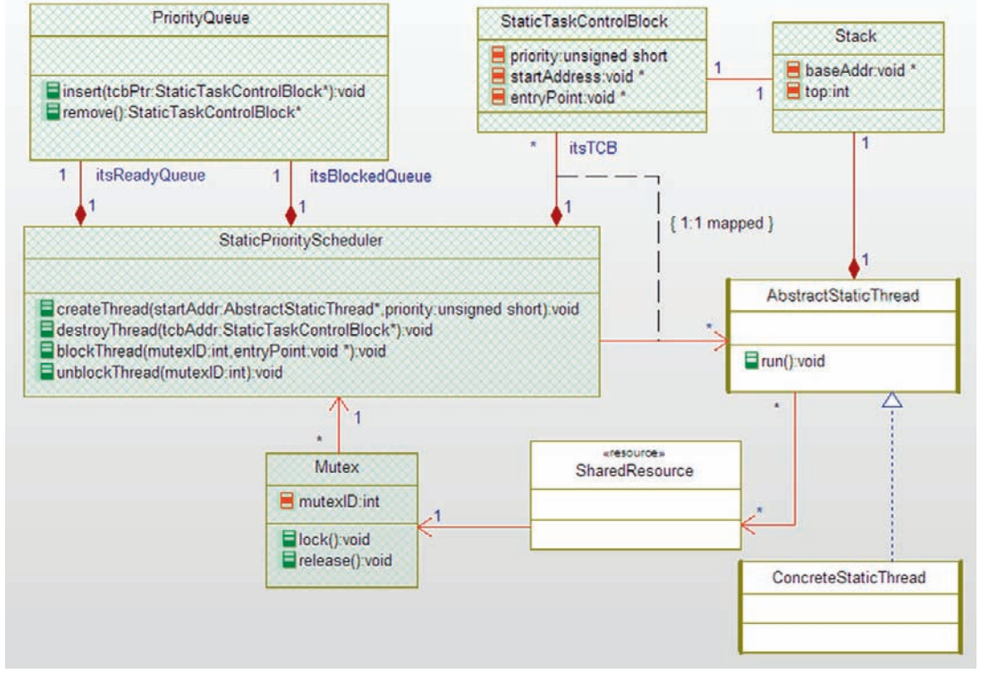


图 4-10：静态优先级模式

4.3.4 Collaboration Roles

This section describes the roles for this pattern.

4.3.4.1 AbstractStaticThread

The `AbstractStaticThread` class is an abstract (i.e., noninstantiable) super class for *Concrete Thread. Abstract Thread* associates with the `Scheduler`; specifically the Scheduler executes the run() function of the thread to run the thread. The `AbstractStaticThread` specifies the run interface to be realized by the `ConcreteThread`.

4.3.4.2 ConcreteThread

The `ConcreteThread` is an «active» class that serves as a container and thread context for a set of passive "semantic" objects that do the real work of the system. The `ConcreteThread` class provides a straightforward means of attaching these semantic objects into the concurrency architecture. `ConcreteThread` represents the threads in the application software.

4.3.4.3 Mutex

The `Mutex` is a mutual exclusion semaphore class that serializes access to a `SharedResource`. The guarded functions of the `SharedResource` invoke the `lock()` function of the `Mutex` whenever they are called and `release()` once the service is complete. Other client threads that attempt to invoke a service when its associated `Mutex` is locked become blocked until the `Mutex` is unlocked. The blocking is performed when the `Mutex` semaphore signals the `StaticPriorityScheduler` that a call attempt was made by the currently active thread. The `mutexID` identifies which `Mutex` is blocking the task. This element is usually provided by the RTOS.

4.3.4.4 PriorityQueue

The `PriorityQueue` is a queue of `StaticTaskControlBlock` pointers organized by priority. The pattern uses two distinct instances of `PriorityQueue`. The first is the ready queue. When a task becomes ready to run, a pointer to its `StaticTaskControlBlock` is inserted into the queue in order of its priority. If the priority of the task is higher than the currently running task (or becomes so when the current task terminates), then the task is removed from the ready queue and run. The other `PriorityQueue` is the blocked queue; this queue holds the set of tasks currently blocked because they tried to access `SharedResources` protected by a locked `Mutex`. This element is usually provided by the RTOS.

4.3.4.5 SharedResource

A `SharedResource` is a class whose instances are shared by one or more `ConcreteThreads`. For the system to operate properly in all cases, all shared resources must either be reentrant (meaning that corruption from simultaneous access cannot occur) or they must be protected via access serialization. Protection via mutual exclusion semaphores is discussed in the Guarded Call Pattern later in this chapter.

4.3.4 协作角色

本节描述该模式的角色。

4.3.4.1 AbstractStaticThread

`AbstractStaticThread` 类是具体线程的抽象（即不可实例化的）超类。抽象线程与调度器关联；具体而言，调度器执行线程的run()函数来运行线程。`AbstractStaticThread`指定了由`ConcreteThread`实现的run接口。

4.3.4.2 ConcreteThread

`ConcreteThread` 是一个«活动»类，用作一组执行系统实际工作的被动"语义"对象的容器和线程上下文。`ConcreteThread` 类提供了一种将这些语义对象附加到并发架构中的直接方式。`ConcreteThread` 表示应用软件中的线程。

4.3.4.3 互斥锁

`Mutex` 是一个互斥信号量类，用于串行化对`SharedResource`的访问。`SharedResource` 的受守卫函数在被调用时会调用Mutex 的 lock()函数，并在服务完成后调用 release()。其他客户端线程在关联的Mutex 被锁定时尝试调用服务会被阻塞，直到Mutex 被解锁。阻塞是在Mutex信号量向 StaticPriorityScheduler发出当前活动线程进行了调用尝试的信号时执行的。 mutexID 标识哪个Mutex 正在阻塞该任务。此元素通常由RTOS提供。

4.3.4.4 优先级队列

`PriorityQueue` 是一个按优先级组织的`StaticTaskControlBlock` 指针队列。该模式使用两个不同的`PriorityQueue`实例。第一个是就绪队列。当任务变为就绪时，指向其`StaticTaskControlBlock` 的指针会按其优先级顺序插入队列。如果该任务的优先级高于当前正在运行的任务（或在当前任务终止时变得更高），则该任务会从就绪队列中移除并运行。另一个`PriorityQueue` 是阻塞队列；该队列保存当前被阻塞的任务集合，这些任务因尝试访问由已锁定Mutex保护的SharedResources而被阻塞。此元素通常由RTOS提供。

4.3.4.5 共享资源

`SharedResource` 是一个类，其实例由一个或多个`ConcreteThreads`共享。为使系统在所有情况下都能正常运行，所有共享资源必须要么是可重入的（即不会因同时访问而发生损坏），要么必须通过访问串行化加以保护。通过互斥信号量进行保护将在本章后面的受保护调用模式中讨论。

4.3.4.6 Stack

Each `AbstractThread` has a `Stack` for return addresses and passed parameters. This is normally only explicit at the assembly language level within the application thread, but is an important part of the scheduling infrastructure. This element is usually provided by the RTOS.

4.3.4.7 StaticPriorityScheduler

This class orchestrates the execution of multiple threads based on their priority according to a simple rule: always run the highest priority ready thread. When the «active» thread object is created, it (or its creator) calls the `createThread()` operation to create a thread for the «active» object. Whenever this thread is executed by the `StaticPriorityScheduler`, it calls the `StartAddr` address (except when the thread has been blocked or preempted, in which case it calls the `EntryPoint` address). This element is usually provided by the RTOS.

4.3.4.8 StaticTaskControlBlock

The `StaticTaskControlBlock` contains the scheduling information for its corresponding *AbstractThread* object. This includes the priority of the thread, the default start address and the current entry address, if it was preempted or blocked prior to completion. The `StaticPriorityScheduler` maintains a `StaticTaskControlBlock` object for each existing `AbstractThread`. This element is usually provided by the RTOS.

4.3.5 Consequences

As previously mentioned, the Static Priority Pattern runs well with potentially large numbers of tasks and provides responsiveness to incoming events. Typically the tasks exist in waiting for initiation most of the time and become active only when an initiating event (such as the elapse of their period) occurs. Priority is only an issue when there is more than one task ready to run; in this case the highest priority task will be run preferentially.

It is common to use blocking to serialize access to resources. While this is highly effective, naïve implementation can lead to unbounded priority inversion in the manner described in the first section of this chapter. See the section on resource sharing patterns later in this chapter for patterns to address this concern.

4.3.6 Implementation Strategies

By far, the most common implementation of this pattern is to use a commercial RTOS to provide the scheduling infrastructure. While this wasn't true 20 years ago, today there are dozens of popular RTOSs that will fit the constraints of most embedded systems.

Regardless of whether you buy a commercial RTOS or roll your own, the problem of assigning priorities must still be addressed. RMS (rate monotonic scheduling) is the most common means to assign priorities. It is optimal and stable. By *optimal*, I mean that if the task set can be

4.3.4.6 栈

每个`AbstractThread`都有一个`Stack`，用于存放返回地址和传递参数。这通常仅在应用线程内的汇编语言层面显式存在，但它是调度基础设施的重要组成部分。该元素通常由实时操作系统提供。

4.3.4.7 StaticPriorityScheduler

该类根据一个简单的规则来编排多个线程的执行：始终运行优先级最高的就绪线程。当创建«活动»线程对象时，它（或其创建者）调用`createThread()`操作为«活动»对象创建一个线程。每当该线程由`StaticPriorityScheduler`执行时，它会调用`StartAddr`地址（除非该线程已被阻塞或抢占，在这种情况下它会调用`EntryPoint`地址）。该元素通常由实时操作系统提供。

4.3.4.8 StaticTaskControlBlock

`StaticTaskControlBlock`包含其对应AbstractThread对象的调度信息。这包括线程的优先级、默认起始地址以及当前入口地址（如果它在完成之前被抢占或阻塞）。`StaticPriorityScheduler`为每个现有的`AbstractThread`维护一个`StaticTaskControlBlock`对象。该元素通常由实时操作系统提供。

4.3.5 结果

如前所述，静态优先级模式在任务数量可能很大的情况下运行良好，并能对传入事件提供响应性。通常，任务在大部分时间处于等待触发状态，只有在触发事件（例如其周期到期）发生时才会变为活动状态。只有当有多个任务就绪时，优先级才会成为问题；在这种情况下，优先级最高的任务将被优先运行。

通常使用阻塞来串行化对资源的访问。虽然这非常有效，但朴素的实现方式可能导致本章第一节所述的无界优先级反转。有关解决此问题的模式，请参阅本章后面的资源共享模式部分。

4.3.6 实现策略

到目前为止，该模式最常见的实现方式是使用商用实时操作系统来提供调度基础设施。虽然20年前并非如此，但如今已有数十种流行的实时操作系统能够满足大多数嵌入式系统的约束。

无论你是购买商用实时操作系统还是自行开发，分配优先级的问题都必须解决。RMS（速率单调调度）是分配优先级最常用的方法。它是最优且稳定的。所谓最优，我的意思是，如果任务集能够

scheduled by any means whatsoever, it can also be scheduled by RMS. By *stable*, I mean that in the case of an overloaded situation in which some tasks will miss their deadlines, with static priority scheduling, it is clear which tasks will fail – those with the lowest priority. RMS simply makes the priority a function of the period – the shorter the period, the higher the priority[12].

RMS makes a number of assumptions about the tasks but is reasonably robust in the presence of violations to those assumptions:

- tasks are periodic
- task deadlines occur at the end of the period
- tasks are infinitely preemptable

As to the first assumption, if tasks are not periodic, then they can occur at a maximum rate (that is, with a minimum time between task invocations). This value can be used instead of the period to determine the priority.

For the second assumption, it sometimes happens that the deadline is less than the period. In this case, simply use the deadline (D_j) instead of the period (T_j) to assign the priority.

The last assumption is that a currently running task can be preempted as soon as a higher priority task becomes ready to run. This isn't true if the currently running task is in a critical region (a period of time in which task switching is disabled – see the Critical Region Pattern later in this chapter). However, if the critical regions are short, this is usually not a problem. Such situations can be analyzed by treating the duration of the longest critical region of all other tasks as the worst case blocking time for the task under analysis.

4.3.7 Related Patterns

The Static Priority Pattern offers a more responsive alternative to fairness doctrine approaches such as the Cyclic Executive Pattern. When resources must be shared with other tasks, this pattern must be mixed with resource sharing patterns such as those in this chapter. When tasks need to coordinate, this pattern can be mixed with task coordination patterns. The task coordination patterns overlap a bit with the resource sharing patterns since one of the ways that tasks coordinate is by sharing common resources.

A very similar pattern to the Static Priority Pattern is the Dynamic Priority Pattern. In this pattern, the design does not specify the priority of the task; this is computed at run-time. The most common priority assignment strategy is known as Earliest Deadline First (EDF) scheduling. In EDF, the task must specify the duration of its deadline from which the scheduler computes the absolute time of the elapse of this task's next deadline. The task is then put into

[12] Note that many RTOSs use small values to indicate higher priorities so that a priority of 5 is a higher priority than a priority of 10.

以任何方式被调度，那么它也能被RMS调度。所谓稳定，我的意思是，在过载情况下某些任务将错过截止时间时，使用静态优先级调度，可以清楚地知道哪些任务会失败——那些优先级最低的任务。RMS只是将优先级设为周期的函数——周期越短，优先级越高[12]。

RMS 对任务做出了若干假设，但在这些假设被违反的情况下仍具有相当的鲁棒性：

- 任务是周期性的
- 任务截止时间出现在周期末尾
- 任务是完全可抢占的

关于第一个假设，如果任务不是周期性的，那么它们可以以最大速率发生（即任务调用之间的最小时间间隔）。可以使用该值代替周期来确定优先级。

对于第二个假设，有时会出现截止时间小于周期的情况。在这种情况下，只需使用截止时间（D$_j$）而不是周期（T$_j$）来分配优先级。

最后一个假设是，当前正在运行的任务可以在更高优先级的任务就绪时立即被抢占。如果当前正在运行的任务处于临界区（即任务切换被禁用的一段时间——参见本章后面的临界区模式），则此假设不成立。然而，如果临界区很短，这通常不是问题。可以通过将所有其他任务中最长临界区的持续时间视为所分析任务的最坏情况阻塞时间来分析此类情况。

4.3.7 相关模式

静态优先级模式提供了比公平原则方法（如循环执行模式）更具响应性的替代方案。当资源必须与其他任务共享时，此模式必须与本章中的资源共享模式等资源共享模式混合使用。当任务需要协调时，此模式可以与任务协调模式混合使用。任务协调模式与资源共享模式有一定重叠，因为任务协调的方式之一就是共享公共资源。

与静态优先级模式非常相似的一种模式是动态优先级模式。在这种模式中，设计并不指定任务的优先级；优先级在运行时计算。最常见的优先级分配策略被称为最早截止时间优先（EDF）调度。在EDF中，任务必须指定其截止时间的持续时间，调度器据此计算该任务下一个截止时间到期的绝对时间。然后该任务被放入

[12] 请注意，许多实时操作系统使用较小的值来表示较高的优先级，因此优先级 5 比优先级 10 更高。

the ready queue based on the nearness of the next absolute deadline for all ready tasks. That is, if a task A is being queued that has a nearer next deadline than another task B, then it will be placed in front of task B in the queue; otherwise, it will be placed later in the queue. EDF is optimal (in the same sense that RMS is) but it is not stable; in an overload situation which tasks will fail to meet their deadlines cannot be computed at design time. If you use EDF scheduling, then schedulability can be computed in a manner similar to RMS scheduling, except that the upper utilization bound is 1.00 rather than a function of the number of tasks. This is shown in Equation 4-3.

$$\sum_{j} \frac{C_j}{T_j} + max\left(\frac{B_1}{T_1}, \cdots, \frac{B_{n-1}}{T_{n-1}}\right) \leq 1.00$$

Equation 4-3: Earliest Deadline First schedulability

4.3.8 Example

In this example, we are going to assume the use of a commercial RTOS so we won't explicitly model or code the infrastructure elements of the pattern. Instead, the example (See Figure 4-11) focuses on concrete threads and shared resources.

In the code listings below, the calls to initialize timers, pend on the OS events, and create tasks that are all RTOS-dependent.

```
void main(void) {
      /*  initialize the links and start the tasks  */
      MotorPositionSensor_initRelations();
      MotorDisplay_initRelations();
      MotorController_initRelations();

      /*  now let the tasks do their thing  */
      while (TRUE) {
      };
};
```

Code Listing 4-7: Static Priority Example main.c

The code listings for the `MotorPositionSensor`, `MotorDisplay`, and `MotorController` are very simple, as can be seen in Code Listing 4-8 through Code Listing 4-13. The interesting code can be found in the `_Init()` functions, where the OS event queues are created and the tasks created with the `corresponding _run()` functions. A queue size of 10 events is more than enough space if the events are handled in a timely fashion. Note also that each task is created with a 1024 int sized stack. Following the task

就绪队列中，其位置基于所有就绪任务中下一个绝对截止时间的临近程度。也就是说，如果任务A被排队时其下一个截止时间比另一个任务B更近，那么它将被放置在队列中任务B的前面；否则，它将被放置在队列中较后的位置。EDF是最优的（与RMS在相同意义上），但它不是稳定的；在过载情况下，哪些任务将无法满足其截止时间无法在设计时计算。如果使用EDF调度，则可调度性可以以类似于RMS调度的方式计算，不同之处在于利用率上界为1.00，而不是任务数量的函数。如公式4-3所示。

$$\sum_{j} \frac{C_j}{T_j} + max\left(\frac{B_1}{T_1}, \cdots, \frac{B_{n-1}}{T_{n-1}}\right) \leq 1.00$$

公式 4-3：最早截止时间优先可调度性

4.3.8 示例

在本示例中，我们将假设使用商用实时操作系统，因此不会显式地对模式的基础设施元素进行建模或编码。相反，本示例（参见图 4-11）重点关注具体线程和共享资源。

在下面的代码清单中，初始化定时器、挂起等待操作系统事件以及创建任务的调用均依赖于实时操作系统。

```
void main(void) { /* 初始化链接并启动任务 */
MotorPositionSensor_initRelations(); MotorDisplay_
initRelations(); MotorController_initRelations(); /* 现在让任务
执行其工作 */ while (TRUE) { }; };
```

代码清单 4-7：静态优先级示例 main.c

`MotorPositionSensor`、`MotorDisplay`和`MotorController` 的代码清单非常简单，如代码清单4-8至代码清单4-13所示。有趣的代码可以在`_Init()` 函数中找到，在这些函数中创建了操作系统事件队列，并使用相应的 `_run()` 函数创建了任务。如果事件能够及时处理，10个事件的队列大小绰绰有余。另请注意，每个任务创建时都分配了1024个int大小的栈。在任务

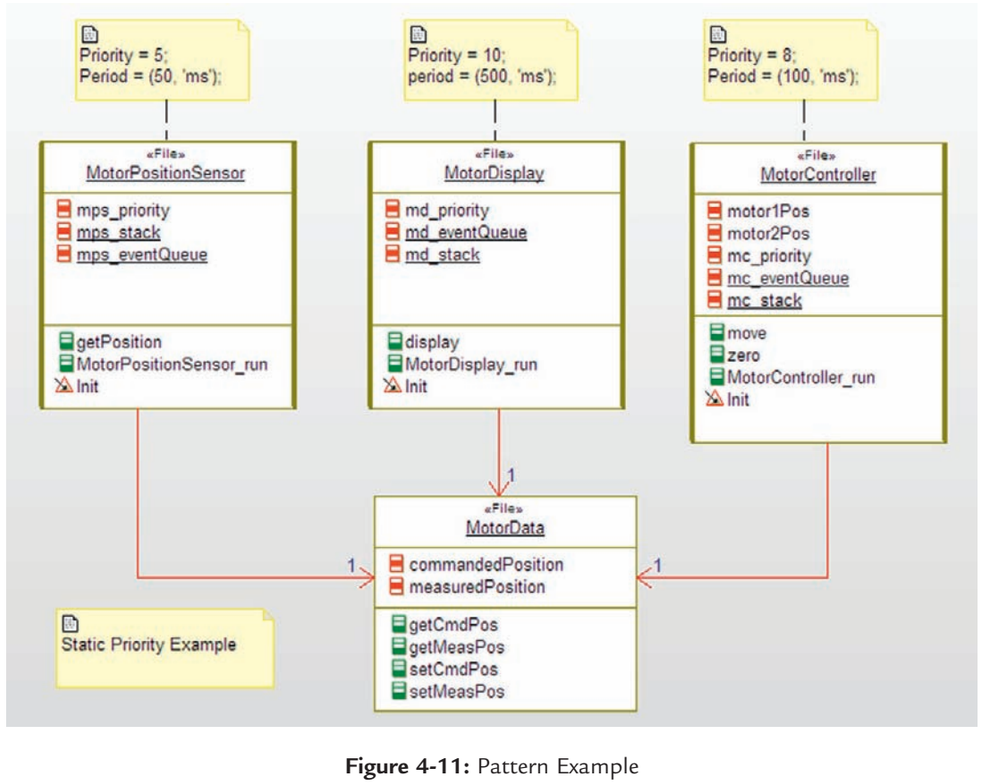


Figure 4-11: Pattern Example

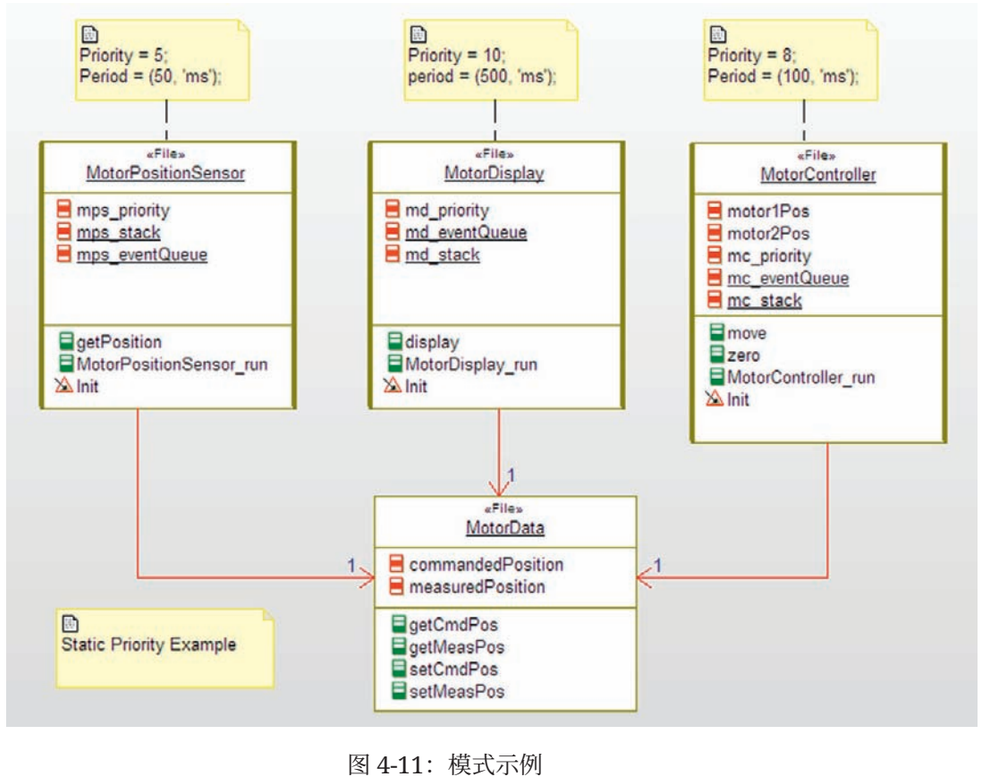


图 4-11：模式示例

creation, the corresponding `_run()` function creates an OS timer (with the `RETRIGGER` parameter so that the timer repeats automatically) and then runs an infinite loop. Within the loop, the task then pends on the event appearance. When the corresponding event with the proper id occurs, the `OSQPend()` function returns and the task is initiated. Since this is an infinite loop, once the loop completes, each task iterates and goes to sleep waiting for its next event.

创建之后，相应的`_run()`函数会创建一个操作系统定时器（带有`RETRIGGER`参数，以便定时器自动重复），然后运行一个无限循环。在循环中，任务随后等待事件出现。当具有正确id的相应事件发生时，`OSQPend()`函数返回，任务被启动。由于这是一个无限循环，一旦循环完成，每个任务都会迭代并进入休眠状态，等待下一个事件。

The code listings for `MotorData` (Code Listing 4-14 and Code Listing 4-15) are even simpler since it is just a passive resource with services invoked by the active threads. Again note that the calls to create, initialize, and use threads are RTOS-specific.

`MotorData`的代码清单（代码清单4-14和代码清单4-15）更加简单，因为它只是一个被动资源，其服务由活动线程调用。再次注意，创建、初始化和使用线程的调用是特定于实时操作系统的。

```
#ifndef MotorPositionSensor_H
#define MotorPositionSensor_H
```

```
#ifndef MotorPositionSensor_H #
define MotorPositionSensor_H
```

```
void MotorPositionSensor_Init(void);
void MotorPositionSensor_Cleanup(void);

void MotorPositionSensor_run(void);
void getPosition(void);

void MotorPositionSensor_initRelations(void);

#endif
```

Code Listing 4-8: MontorPositionSensor.h

```
#include "MotorData.h"
#include "MotorPositionSensor.h"
#include <stdlib.h>

static int mps_priority = 5;
static int mps_eventQueue[10];
static int mps_stack[1024];

void MotorPositionSensor_Cleanup(void) {
}
void MotorPositionSensor_Init(void) {
   /* create the OS event queue and the task */
   /* use a queue of 10 events, and a 1024 element stack */
   OSQCreate (&mps_eventQueue[0], 10);
   OSTaskCreate (MotorPositionSensor_run, NULL,
        (void *)&mps_stack[1024], 5);

   /* now run the task */
   MotorPositionSensor_run();
}
void MotorPositionSensor_run(void) {
   int os_pend_timer_id;
   os_pend_timer_id = OSCreateTimer(50, RETRIGGER);

   while (TRUE) {
     OSQPend(os_pend_timer_id, WAIT_FOREVER);
     getPosition();
   };
}
void getPosition(void) {
   int x;
   /* get position by reading the sensor */
   x = rand();
   setMeasPos(x);
}
```



```
void MotorPositionSensor_Init(void); void
MotorPositionSensor_Cleanup(void);

void MotorPositionSensor_run(void);
void getPosition(void);

void MotorPositionSensor_initRelations(void); #endif
```

代码清单 4-8：MontorPositionSensor.h

```
#include "MotorData.h" #include "
MotorPositionSensor.h" #include <
stdlib.h>

static int mps_优先级 = 5; static
int mps_事件队列[10]; static int
mps_栈[1024];

void MotorPositionSensor_Cleanup(void) { } void
MotorPositionSensor_Init(void) { /* 创建操作系统事件队列和任务 */ /*
使用包含10个事件的事件队列，以及一个1024元素的栈 */ OSQCreate (&
mps_eventQueue[0], 10); OSTaskCreate (MotorPositionSensor_r
un, NULL, (void *)&mps_stack[1024], 5); /* 现在运行任务 */
MotorPositionSensor_run(); } void MotorPositionSensor_run(void) {
int os_pend_timer_id; os_pend_timer_id = OSCreateTimer(50,
RETRIGGER); while (TRUE) { OSQPend(os_pend_timer_id, WAIT_
FOREVER); getPosition(); }; } void getPosition(void) { int x; /* 通过读取
传感器获取位置 */ x = rand(); setMeasPos(x); }
```

```
void MotorPositionSensor_initRelations(void) {
    MotorPositionSensor_Init();
}
```

Code Listing 4-2: MotorPositionSensor.c

```
#ifndef MotorDisplay_H
#define MotorDisplay_H



void MotorDisplay_Init(void);
void MotorDisplay_Cleanup(void);

void MotorDisplay_run(void);
void display(void);

void MotorDisplay_initRelations(void);
#endif
```

Code Listing 4-3: MotorDisplay.h

```
#include "MotorData.h"
#include <stdio.h>
#include "MotorDisplay.h"

static int md_priority  =  10;
static int md_eventQueue[10];
static int md_stack[1024];

void MotorDisplay_Init(void) {
    /*  create the OS event queue and the task  */
    /*  use a queue of 10 events, and a 1024 element stack  */
    OSQCreate (&md_eventQueue[0], 10);
    OSTaskCreate (MotorDisplay_run, NULL,
         (void  *)&md_stack[1024], 5);

    /*  now run the task  */
    MotorDisplay_run();
}

void MotorDisplay_Cleanup(void) {
}

void MotorDisplay_run(void) {
    int os_pend_timer_id;
    os_pend_timer_id  =  OSCreateTimer(500, RETRIGGER);
```



```
void MotorPositionSensor_initRelations(void) {
MotorPositionSensor_Init(); }
```

代码清单4-2：MotorPositionSensor.c

```
#ifndef MotorDisplay_H #
define MotorDisplay_H



void MotorDisplay_Init(void); void
MotorDisplay_Cleanup(void);

void MotorDisplay_run(void);
void display(void);

void MotorDisplay_initRelations(void); #endif
```

代码清单 4-3：MotorDisplay.h

```
#include "MotorData.h" #
include <stdio.h>#include "
MotorDisplay.h"

static int md_优先级  =  10; static
int md_事件队列[10]; static int md_
栈[1024];

void MotorDisplay_Init(void) {
    /*  创建操作系统事件队列和任务 */
    /*  使用包含 10 个事件的事件队列，以及 1024 个元素的栈 */
    OSQCreate (&md_事件队列[0], 10);
    OSTaskCreate (MotorDisplay_run, 空,
         (void *)&md_栈[1024]      , 5);

    /*  现在运行该任务 */
    MotorDisplay_run();
}

void MotorDisplay_Cleanup(void) {
}

void MotorDisplay_run(void) { int os_pend_timer_id; os_pend_
timer_id  =  OSCreateTimer(500, RETRIGGER);
```

```
    while (TRUE) {
      OSQPend(os_pend_timer_id, WAIT_FOREVER);
      display();
      };
}

void display(void) {
    printf("Commanded position  =  %d\n", getCmdPos());
    printf("Measured position  =  %d\n\n", getMeasPos());
}

void MotorDisplay_initRelations(void) {
    MotorDisplay_Init();
}
```

Code Listing 4-4: MotorDisplay.c

```
#ifndef MotorController_H
#define MotorController_H

void MotorController_Init(void);
void MotorController_Cleanup(void);

void MotorController_run(void);
void move(void);
void zero(void);

void MotorController_initRelations(void);

#endif
```

Code Listing 4-5: MotorController.h

```
#include "MotorData.h"
#include <stdlib.h>
#include "MotorController.h"

static int mc_priority  =  8;
static int mc_eventQueue[10];
static int mc_stack[1024];

int motor1Pos;
int motor2Pos;
```



```
    while (TRUE) { OSQPend(os_pend_timer_id,
WAIT_FOREVER); display(); }; }


void display(void) { printf("指令位置  =  %d\n", getCmdPos());
printf("测量位置  = %d\n\n", getMeasPos()); } void MotorDisplay_
initRelations(void) { MotorDisplay_Init(); }
```

代码清单 4-4：MotorDisplay.c

```
#ifndef MotorController_H #define
MotorController_H void MotorController_
Init(void); void MotorController_Cleanup(void);
void MotorController_run(void); void
move(void); void zero(void); void
MotorController_initRelations(void); #endif
```

代码清单 4-5：MotorController.h

```
#include "MotorData.h" #include <
stdlib.h>#include "
MotorController.h"

static int mc_优先级  =  8; static
int mc_事件队列[10]; static int mc_
栈[1024];

int motor1Pos;
int motor2Pos;
```

```
void MotorController_Init(void) {
    /* use a queue of 10 events, and a 1024 element stack */
    /* create the OS event queue and the task */
    OSQCreate (&mc_eventQueue[0], 10);
    OSTaskCreate (MotorController_run, NULL,
    (void *)&mc_stack[1024], 5);

    /* now run the task */
    MotorController_run();
}

void MotorController_Cleanup(void) {
}

void MotorController_run(void) {
    int os_pend_timer_id, x, y;
    os_pend_timer_id = OSCreateTimer(100, RETRIGGER);

    while (TRUE) {
      OSQPend(os_pend_timer_id, WAIT_FOREVER);;
      move();
      };
}

void move(void) {
    int m1Pos, m2Pos;
    /* this function would read the instrument
       panel to set the motor movement settings
       to set the position of the motors.
       Note that if you want to set only one
       of the motors, then set a negative value
       for the commanded position of the other
    */
    m1Pos = rand();
    m2Pos = rand();

    if (m1Pos >= 0)
      motor1Pos = m1Pos;
    if (m2Pos >=0)
      motor2Pos = m2Pos;
    setCmdPos(100*m1Pos + m2Pos);
}

void zero(void) {
    motor1Pos = 0;
    motor2Pos = 0;
    setCmdPos(0);
}

void MotorController_initRelations(void) {
    MotorController_Init();
}
```


void MotorController_Init(void) { /* 使用一个包含10个事件的队列，以及一个1024元素的栈 * /
/* 创建操作系统事件队列和任务 */ OSQCreate (&mc_eventQueue[0], 10); OSTaskCreate (MotorController_run, NULL, (void *)&mc_stack[1024], 5); /* 现在运行任务 */ MotorController_run(); } void MotorController_Cleanup(void) { } void MotorController_run(void) { int os_pend_timer_id, x, y; os_pend_timer_id = OSCreateTimer(100, RETRIGGER); while (TRUE) { OSQPend(os_pend_timer_id, WAIT_FOREVER);; move(); }; } void move(void) { int m1Pos, m2Pos; /* 此函数将读取仪表盘以设置电机运动设置，从而设置电机的位置。注意，如果只想设置其中一个电机，则将另一个电机的指令位置设为负值 */ m1Pos = rand(); m2Pos = rand(); if (m1Pos >= 0) motor1Pos = m1Pos; if (m2Pos >=0) motor2Pos = m2Pos; setCmdPos(100*m1Pos + m2Pos); } void zero(void) { motor1Pos = 0; motor2Pos = 0; setCmdPos(0); } void MotorController_initRelations(void) { MotorController_Init(); }

Code Listing 4-13: MotorController.c

```
#ifndef MotorData_H
#define MotorData_H

#define RETRIGGER (2)
#define WAIT_FOREVER (0)

extern int commandedPosition;
extern int measuredPosition;

int getCmdPos(void);
int getMeasPos(void);
void setCmdPos(int x);
void setMeasPos(int x);

#endif
```

Code Listing 4-14: MotorData.h

```
#include "MotorData.h"

int commandedPosition  =  0;
int measuredPosition  =  0;

int getCmdPos(void) {
    return commandedPosition;
}

int getMeasPos(void) {
    return measuredPosition;
}

void setCmdPos(int x) {
     commandedPosition  =  x;
}

void setMeasPos(int x) {
     measuredPosition  =  x;
}
```

Code Listing 4-15: MotorData.c

代码清单4-13：MotorController.c

```
#ifndef MotorData_H #
define MotorData_H

#define RETRIGGER (2) #
define 等待_FOREVER (0)

extern int commandedPosition;
extern int measuredPosition;

int getCmdPos(void); int
getMeasPos(void); void
setCmdPos(int x); void
setMeasPos(int x);

#endif
```

代码清单4-14：MotorData.h

```
#include "MotorData.h" int
commandedPosition  =  0; int
measuredPosition  =  0; int
getCmdPos(void) { return
commandedPosition; } int
getMeasPos(void) { return
measuredPosition; } void
setCmdPos(int x) {
commandedPosition  =  x; }
void setMeasPos(int x) {
measuredPosition  =  x; }
```

代码清单4-15：MotorData.c

Task Coordination Patterns

To be honest, when tasks are truly independent, the concurrency design is pretty simple. The problem is that independence is not the normal condition for concurrent threads. In virtually all embedded systems, tasks are "mostly independent"[13]. The four patterns of this section include the Critical Region, Guard Call, Queuing, and Rendezvous Patterns. The first three of these are often used to serialize access to shared resources as well and so could have been reasonably placed in that section of this chapter.

4.4 Critical Region Pattern

The critical region pattern is the simplest pattern for task coordination around. It consists of disabling task switching during a region where it is important that the current task executes without being preempted. This can be because it is accessing a resource that cannot be simultaneously accessed safely, or because it is crucial that the task executes for a portion of time without interruption.

4.4.1 Abstract

In a preemptive multitasking environment (see the Static Priority Pattern, above), there may be periods of time during which a task must not be interrupted or preempted. The Critical Region Pattern address these intervals by disabling task switching or even by disabling interrupts (see the Interrupt Pattern, above). This provides uninterrupted execution by the current task. Once beyond the critical region, the task must reenable task switching (or interrupts) or the system will fail.

4.4.2 Problem

There are two primary circumstances under which a task should not be interrupted or preempted. First, it may be accessing a resource that may not be safely accessed by multiple clients simultaneously. Secondly, the task may be performing actions that must be completed within a short period of time or that must be performed in a specific order. This pattern allows the active task to execute without any potential interference from other tasks.

4.4.3 Pattern Structure

Figure 4-12 shows just how structurally simple this pattern is. In a common variant, the protected element is a resource and not a task; this is shown in Figure 4-13. In either case,

[13] In the same sense that Miracle Max reports that Wesley is "mostly dead" (Goldman, W., 1973. *Princess Bride: S, Morgenstern's Classic Tale of True Love and High Adventure*. Harcourt).

任务协调模式

说实话，当任务真正独立时，并发设计相当简单。问题在于独立性并不是并发线程的常态。在几乎所有嵌入式系统中，任务都是"大部分独立"的[13]。本节中的四种模式包括临界区模式、保护调用模式、排队模式和会合模式。其中前三种模式也常用于串行化对共享资源的访问，因此本可以合理地放在本章的那一节中。

4.4 临界区模式

临界区模式是最简单的任务协调模式。它通过在某个区域内禁用任务切换来实现，在该区域内，当前任务必须在不被抢占的情况下执行，这一点非常重要。这可能是因为该任务正在访问一个无法同时安全访问的资源，或者是因为该任务必须在一段时间内不被打断地执行，这一点至关重要。

4.4.1 摘要

在抢占式多任务环境（参见上文的静态优先级模式）中，可能存在某些时间段，在此期间任务不得被中断或抢占。临界区模式通过禁用任务切换甚至禁用中断（参见上文的中断模式）来处理这些时间区间。这为当前任务提供了不间断的执行。一旦离开临界区，任务必须重新启用任务切换（或中断），否则系统将失败。

4.4.2 问题

在两种主要情况下，任务不应被中断或抢占。首先，任务可能正在访问一个无法被多个客户端同时安全访问的资源。其次，任务可能正在执行必须在短时间内完成或必须按特定顺序执行的动作。此模式允许活动任务在不受其他任务任何潜在干扰的情况下执行。

4.4.3 模式结构

图 4-12 展示了该模式在结构上是多么简单。在一个常见的变体中，受保护的元素是资源而不是任务；这一点如图 4-13 所示。在任一情况下，

[13] 同样地，奇迹麦克斯报告说韦斯利"基本已死"（Goldman, W., 1973. Princess Bride: S, Morgenstern's Classic Tale of True Love and High Adventure. Harcourt）。

Figure 4-12: Critical Region Pattern

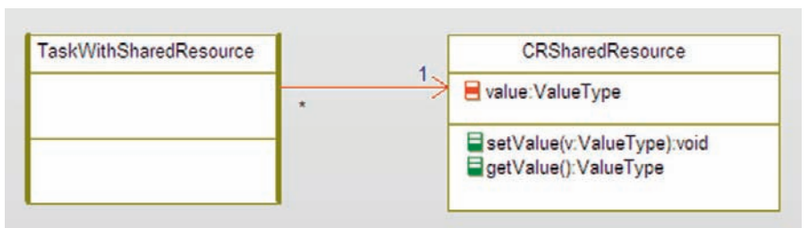


Figure 4-13: Critical Region Pattern with Resource

within the relevant services, task switching must be disabled before the critical region begins and reenabled before the service ends. Note that unlike the Guarded Call Pattern, the scheduler is not involved in the critical region begin/end process other than to provide services to disable/enable task switching. If the scheduler does not provide such services, then the critical region can be controlled with the user of the C `asm` directive to turn on or off interrupt processing at the hardware level.

4.4.4 Collaboration Roles

This section describes the roles for this pattern.

4.4.4.1 CRSharedResource

This element is used in the variant of the Critical Resource Pattern that disables task switching to prevent simultaneous access to the resource. In the case of the pattern, the protected resource is the `value` attribute. Note that all the relevant services must implement the critical region to protect the resource. In this case, it means that both the `setValue()` and `getValue()` functions must separately implement the critical region.

4.4.4.2 TaskWithCriticalRegion

This element is used when the task wants to disable task switching for reasons unrelated to sharing resources (usually related to timing). The criticalService() disables and enables task switching or interrupts during its critical region(s).

图 4-12：临界区模式

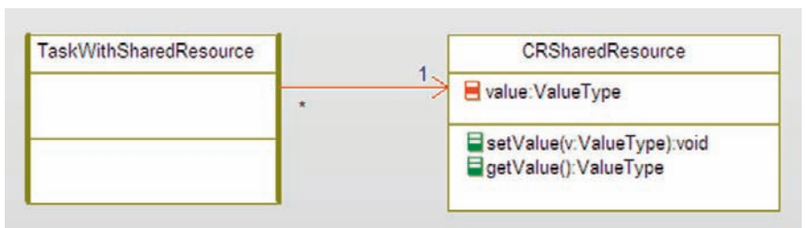


图 4-13：带资源的临界区模式

在相关服务内部，必须在临界区开始之前禁用任务切换，并在服务结束之前重新启用任务切换。请注意，与受保护调用模式不同，调度器除了提供禁用/启用任务切换的服务之外，并不参与临界区的开始/结束过程。如果调度器不提供此类服务，那么可以使用 C 的 `asm` 指令在硬件层面打开或关闭中断处理来控制临界区。

4.4.4 协作角色

本节描述该模式的角色。

4.4.4.1 CRSharedResource

该元素用于临界资源模式的一种变体，该变体通过禁用任务切换来防止对资源的并发访问。在该模式中，受保护的资源是`value`属性。请注意，所有相关服务都必须实现临界区以保护该资源。在这种情况下，这意味着`setValue()`和`getValue()`函数都必须分别实现临界区。

4.4.4.2 TaskWithCriticalRegion

当任务因与共享资源无关的原因（通常与时序相关）想要禁用任务切换时，使用此元素。criticalService() 在其临界区内禁用和启用任务切换或中断。

4.4.4.3 TaskWithSharedResource

This element stands for the set of all tasks that may want to access the shared resource (hence the '*' multiplicity on the directed association). These tasks have no knowledge about the method for protecting the resource, since that is encapsulated within the shared resource.

4.4.5 Consequences

This pattern enforces the critical region policies well and can be used to turn off the scheduled task switches or, more draconianly, disable interrupts all together. Care must be taken to reenable task switching once the element is out of its critical region. Additionally, the use of this pattern can affect the timing of other tasks. Critical regions are usually short in duration for this reason. There is no problem of unbounded priority inversion because all task switching is disabled during the critical region.

Care must be taken when nesting calls to functions with their own critical regions. If such a function is called within the critical region of another, the nest function call will reenable task switching and end the critical region of the caller function.

4.4.6 Implementation Strategies

This pattern is usually implemented either by using scheduler-provided services such as `OS_disable_task_switching()` and `OS_enable_task_switching()` (or something similar) or by introducing assembly language instructions to disable or enable interrupt processing at the hardware level.

4.4.7 Related Patterns

This pattern is only appropriate when multiple tasks can access the same resource or services or when other tasks could interrupt time-critical processing of the current task. For this reason, it is not appropriate with the Cyclic Executive Pattern, but is often used with the Static Priority Pattern.

4.4.8 Example

The example in Figure 4-14 shows a single task `CRRobotArmManager` with a function `moveRobotArm` that has a critical region during the actual movement of the arm.

In either case, the critical region is encapsulated entirely with the `moveRobotArm()` and `motorZero()` services, as can be seen in Code Listing 4-16 and Code Listing 4-17.

4.4.4.3 TaskWithSharedResource

此元素代表可能想要访问共享资源的所有任务的集合（因此有向关联上带有“*”多重性）。这些任务不知道保护资源的方法，因为该方法封装在共享资源内部。

4.4.5 结果

此模式很好地执行了临界区策略，可用于关闭调度任务切换，或者更严厉地，完全禁用中断。必须注意在元素离开其临界区后重新启用任务切换。此外，使用此模式可能影响其他任务的时序。出于这个原因，临界区通常持续时间很短。不存在无界优先级反转的问题，因为在临界区期间所有任务切换都被禁用。

在嵌套调用具有自身临界区的函数时必须小心。如果在另一个函数的临界区内调用这样的函数，嵌套函数调用将重新启用任务切换并结束调用者函数的临界区。

4.4.6 实现策略

此模式通常通过使用调度器提供的服务（如`OS_disable_task_switching()`和`OS_enable_task_switching()`（或类似功能））来实现，或者通过引入汇编语言指令在硬件层面禁用或启用中断处理来实现。

4.4.7 相关模式

此模式仅适用于多个任务可以访问同一资源或服务，或者其他任务可能中断当前任务的时间关键处理的情况。因此，它不适用于循环执行模式，但常与静态优先级模式一起使用。

4.4.8 示例

图 4-14 中 的 示例 展示了一个单一任务`CRRobotArmManager`，其函数`moveRobotArm`在机械臂实际移动期间具有临界区。

无论哪种情况，临界区都完全封装在`moveRobotArm()`和`motorZero()`服务中，如代码清单4-16和代码清单4-17所示。

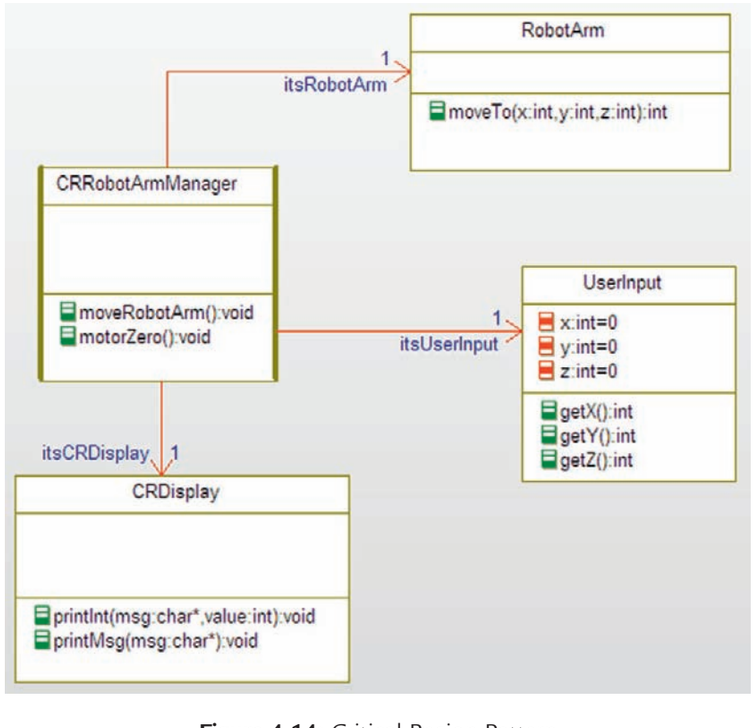


Figure 4-14: Critical Region Pattern

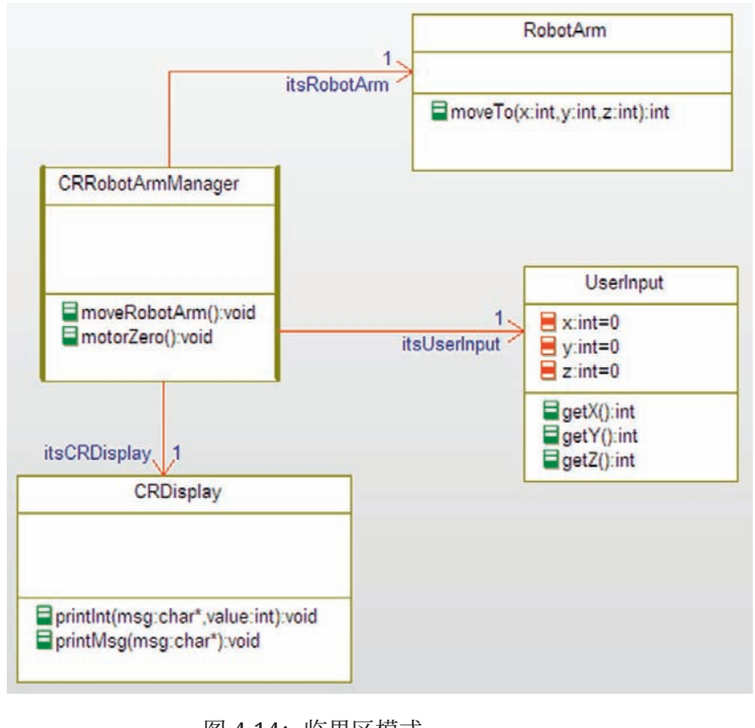


图 4-14：临界区模式

```
#ifndef CRRobotArmManager_H
#define CRRobotArmManager_H

struct CRDisplay;
struct RobotArm;
struct UserInput;

typedef struct CRRobotArmManager CRRobotArmManager;
struct CRRobotArmManager {
   struct CRDisplay*  itsCRDisplay;
   struct RobotArm*  itsRobotArm;
   struct UserInput*  itsUserInput;
};

/*  Constructors and destructors:*/
void CRRobotArmManager_Init(CRRobotArmManager*  const me);
void CRRobotArmManager_Cleanup(CRRobotArmManager*  const me);
```

```
#ifndef CRRobotArmManager_H #
define CRRobotArmManager_H

struct CRDisplay;
struct RobotArm;
struct UserInput;

typedef struct CRRobotArmManager CRRobotArmManager;
struct CRRobotArmManager { struct CRDisplay*
itsCRDisplay; struct RobotArm* itsRobotArm; struct
UserInput* itsUserInput; };


/* 构造函数与析构函数：*/ void CRRobotArmManager_
Init(CRRobotArmManager* const me); void CRRobotArmManager_
Cleanup(CRRobotArmManager* const me);
```

```
/* Operations */
void CRRobotArmManager_motorZero(CRRobotArmManager* const me);
void CRRobotArmManager_moveRobotArm(CRRobotArmManager* const me);
struct CRDisplay* CRRobotArmManager_getItsCRDisplay(const
CRRobotArmManager* const me);

void CRRobotArmManager_setItsCRDisplay(CRRobotArmManager* const
me, struct CRDisplay* p_CRDisplay);

struct RobotArm* CRRobotArmManager_getItsRobotArm(const
CRRobotArmManager* const me);

void CRRobotArmManager_setItsRobotArm(CRRobotArmManager* const me,
struct RobotArm* p_RobotArm);

struct UserInput* CRRobotArmManager_getItsUserInput(const
CRRobotArmManager* const me);

void CRRobotArmManager_setItsUserInput(CRRobotArmManager* const me,
struct UserInput* p_UserInput);

CRRobotArmManager * CRRobotArmManager_Create(void);

void CRRobotArmManager_Destroy(CRRobotArmManager* const me);

#endif
```

Code Listing 4-6: CRRobotArmManager.h

```
#include "CRRobotArmManager.h"
#include "CRDisplay.h"
#include "RobotArm.h"
#include "UserInput.h"

static void cleanUpRelations(CRRobotArmManager* const me);

void CRRobotArmManager_Init(CRRobotArmManager* const me) {
    me->itsCRDisplay = NULL;
    me->itsRobotArm = NULL;
    me->itsUserInput = NULL;
}

void CRRobotArmManager_Cleanup(CRRobotArmManager* const me) {
    cleanUpRelations(me);
}

void CRRobotArmManager_motorZero(CRRobotArmManager* const me) {
    int success = 1;
```



```
/* 操作 */
void CRRobotArmManager_motorZero(CRRobotArmManager* const me);
void CRRobotArmManager_moveRobotArm(CRRobotArmManager* const me);
struct CRDisplay* CRRobotArmManager_getItsCRDisplay(const
CRRobotArmManager* const me);

void CRRobotArmManager_setItsCRDisplay(CRRobotArmManager* const me,
struct CRDisplay* p_CRDisplay);

struct RobotArm* CRRobotArmManager_getItsRobotArm(const
CRRobotArmManager* const me);

void CRRobotArmManager_setItsRobotArm(CRRobotArmManager* const me,
struct RobotArm* p_RobotArm);

struct UserInput* CRRobotArmManager_getItsUserInput(const
CRRobotArmManager* const me);

void CRRobotArmManager_setItsUserInput(CRRobotArmManager* const me,
struct UserInput* p_UserInput);

CRRobotArmManager * CRRobotArmManager_Create(void);

void CRRobotArmManager_Destroy(CRRobotArmManager* const me);

#endif
```

代码清单 4-6：CRRobotArmManager.h

```
#include "CRRobotArmManager.h" #
include "CRDisplay.h" #include "
RobotArm.h" #include "UserInput.h"


static void cleanUpRelations(CRRobotArmManager* const me);

void CRRobotArmManager_Init(CRRobotArmManager* const me) { me->
itsCRDisplay = 空; me->itsRobotArm = 空; me->itsUserInput = 空; }
void CRRobotArmManager_清理(CRRobotArmManager* const me) {
cleanUpRelations(me); }




void CRRobotArmManager_motorZero(CRRobotArmManager* const me) { int
success = 1;
```

```
    OS_disable_task_switching();

    /* critical region code */
    success = RobotArm_moveTo(me->itsRobotArm,0,0,0);

    /* critical region ends */
    OS_enable_task_switching();

    if (!success)
      CRDisplay_printMsg(me->itsCRDisplay,"Cannot zero!");
}

void CRRobotArmManager_moveRobotArm(CRRobotArmManager* const me) {
    /* local stack variable declarations */
    int x, y, z, success = 1;
    /*noncritical region code */

    /* note that the function below has its
       own critical region and so cannot be
       called inside of the critical region
       of this function
    */
    CRRobotArmManager_motorZero(me);

    x = UserInput_getX(me->itsUserInput);
    y = UserInput_getY(me->itsUserInput);
    z = UserInput_getX(me->itsUserInput);

    /* critical region begins */
    OS_disable_task_switching();

    /* critical region code */
    success = RobotArm_moveTo(me->itsRobotArm,x,y,z);

    /* critical region ends */
    OS_enable_task_switching();

    /* more noncritical region code */
    CRDisplay_printInt(me->itsCRDisplay, "Result is ", success);
}

struct CRDisplay* CRRobotArmManager_getItsCRDisplay(const
CRRobotArmManager* const me) {
    return (struct CRDisplay*)me->itsCRDisplay;
}

void CRRobotArmManager_setItsCRDisplay(CRRobotArmManager* const
me, struct CRDisplay* p_CRDisplay) {
    me->itsCRDisplay = p_CRDisplay;
```


OS_disable_task_switching(); /* 临界区代码 */ success = RobotArm_moveTo(me->itsRobotArm,0,0,0); /* 临界区结束 */ OS_enable_task_switching(); if (!success) CRDisplay_printMsg(me->itsCRDisplay,"无法归零！"); } void CRRobotArmManager_moveRobotArm(CRRobotArmManager* const me) { /* 局部栈变量声明 */ int x, y, z, success = 1; /* 非临界区代码 */ /* 注意下面的函数有自己的临界区，因此不能在此函数的临界区内调用 */ CRRobotArmManager_motorZero(me); x = UserInput_getX(me->itsUserInput); y = UserInput_getY(me->itsUserInput); z = UserInput_getX(me->itsUserInput); /* 临界区开始 */ OS_disable_task_switching(); /* 临界区代码 */ success = RobotArm_moveTo(me->itsRobotArm,x,y,z); /* 临界区结束 */ OS_enable_task_switching(); /* 更多非临界区代码 */ CRDisplay_printInt(me->itsCRDisplay, "结果是 ", success); } struct CRDisplay* CRRobotArmManager_getItsCRDisplay(const CRRobotArmManager* const me) { return (struct CRDisplay*)me->itsCRDisplay; } void CRRobotArmManager_setItsCRDisplay(CRRobotArmManager* const me, struct CRDisplay* p_CRDisplay) { me->itsCRDisplay = p_CRDisplay;

```
}

struct RobotArm*  CRRobotArmManager_getItsRobotArm(const
CRRobotArmManager*  const me) {
    return (struct RobotArm*)me->itsRobotArm;
}

void CRRobotArmManager_setItsRobotArm(CRRobotArmManager*  const me,
struct RobotArm*  p_RobotArm) {
    me->itsRobotArm  =  p_RobotArm;
}

struct UserInput*  CRRobotArmManager_getItsUserInput(const
CRRobotArmManager*  const me) {
    return (struct UserInput*)me->itsUserInput;
}

void CRRobotArmManager_setItsUserInput(CRRobotArmManager*  const
me, struct UserInput*  p_UserInput) {
    me->itsUserInput  =  p_UserInput;
}

CRRobotArmManager  *  CRRobotArmManager_Create(void) {
    CRRobotArmManager*  me  =  (CRRobotArmManager  *)
malloc(sizeof(CRRobotArmManager));
    if(me!=NULL)
        {
            CRRobotArmManager_Init(me);
        }
    return me;
}

void CRRobotArmManager_Destroy(CRRobotArmManager*  const me) {
    if(me!=NULL)
        {
            CRRobotArmManager_Cleanup(me);
        }
    free(me);
}

static void cleanUpRelations(CRRobotArmManager*  const me) {
    if(me->itsCRDisplay !=  NULL)
        {
            me->itsCRDisplay  =  NULL;
        }
    if(me->itsRobotArm !=  NULL)
        {
            me->itsRobotArm  =  NULL;
        }
```



```
}

struct RobotArm* CRRobotArmManager_getItsRobotArm(const
CRRobotArmManager* const me) { return (struct RobotArm*)me->itsRobotArm; }
void CRRobotArmManager_setItsRobotArm(CRRobotArmManager* const me

                                                                                ,
struct RobotArm* p_RobotArm) { me->itsRobotArm    = p_RobotArm; } struct
UserInput* CRRobotArmManager_getItsUserInput(const
CRRobotArmManager* const me) { return (struct UserInput*)me->i
tsUserInput; } void CRRobotArmManager_
setItsUserInput(CRRobotArmManager* const me, struct UserInput* p_
UserInput) { me->itsUserInput    =    p_UserInput; } CRRobotArmManager *
CRRobotArmManager_Create(void) { CRRobotArmManager* me    =
(CRRobotArmManager *) malloc(sizeof(CRRobotArmManager)); if(me!=空) {
CRRobotArmManager_Init(me); } return me; } void CRRobotArmManager_
Destroy(CRRobotArmManager* const me) { if(me!=空) { CRRobotArmManager_
清理(me); } free(me); } static void cleanUpRelations(CRRobotArmManager*
const me) { if(me->itsCRDisplay !=    空) { me->itsCRDisplay    =    空; } if(me->
itsRobotArm !=    空) { me->itsRobotArm    =    空; }
```

```
    if(me->itsUserInput != NULL)
        {
            me->itsUserInput = NULL;
        }
}
```

Code Listing 4-17: CRRobotArmManager.c

A little bit about semaphores

Edsger Dijkstra invented the concept of a software semaphore for the coordination of tasks wanting to share resources. The most common implementation is a *counting semaphore* which allows, at most n tasks, to use a resource and blocks additional tasks should they attempt to access the resource until one of the tasks that has successfully locked (said to *own* the semaphore) unlocks the resource. n is said to be the *capacity* of the semaphore. A *binary semaphore* is a counting semaphore for which n = 1. A *mutual exclusion (mutex) semaphore* is a binary semaphore with some extra properties, which are OS dependent. Binary semaphores are used for both mutual exclusion and event notification, while mutexes are used only to enforce mutual exclusion.

Semaphore are initialized with n when whey are created, something like

```
semaPtr = OS_create_semaphore(int n);
```

which initializes an internal thread counter to n.

The semaphore offers two other services, `lock()` and `release()` (which, being Dutch, Dijkstra called P (for *Prolaag* or "test")[1] and V(for *verhoog* or "increase")). `lock()` works by decrementing the internal counter by 1. If the count has decremented to 0 or less, then the thread calling the `lock()` function is blocked by the scheduler. The `release()` function increments the semaphore counter, so that if there is are threads waiting on that lock, one of them will unblock and run (that is, its call to `lock()` now returns). Note that you usually can't test to see if a call to `lock()` will block or not before the thread calls it; it will either succeed immediately or block the thread and (hopefully) be released later. Put another way, there is usually no way to know if there are threads blocking on that semaphore or not before you call `lock()`. However, some RTOS give a `trylock()` function that locks if the resource is available or returns an error code if not.

```
void lock(Semaphore* s) {
    do {
        wait();
    } while (s->count <= 0);
    --s->count; /* this operation must be implemented as a
                   critical region */
```

[1] Or it could be P for "Piraat" (pirate). No one seems to know for sure.


```
    if(me->itsUserInput != 空) { me->
itsUserInput = 空; } }
```

代码清单4-17：CRRobotArmManager.c

关于信号量的简要介绍

艾兹格·迪杰斯特拉发明了软件信号量的概念，用于协调想要共享资源的任务。最常见的实现是计数信号量，它允许最多n个任务使用某个资源，并在其他任务尝试访问该资源时阻塞它们，直到某个已成功加锁（称为拥有该信号量）的任务解锁该资源。n被称为信号量的容量。二元信号量是n = 1的计数信号量。互斥（互斥锁）信号量是一种具有某些额外属性的二元信号量，这些属性依赖于操作系统。二元信号量用于互斥和事件通知，而互斥锁仅用于实现互斥。

信号量在创建时用n进行初始化，类似于

```
semaPtr = OS_create_semaphore(int n);
```

这会将内部线程计数器初始化为n。

信号量还提供另外两个服务：lock() 和 release()（作为荷兰人，Dijkstra 将其称为 P（代表 Prolaag 或"测试"）[1] 和 V（代表 verhoog 或"增加"））。lock() 的工作原理是将内部计数器减 1。如果计数减至 0 或更小，则调用 lock() 函数的线程会被调度器阻塞。release() 函数会增加信号量计数器，因此如果有线程正在等待该锁，其中一个将被解除阻塞并运行（即其对 lock() 的调用现在返回）。请注意，通常无法在调用 lock() 之前测试该调用是否会阻塞；它要么立即成功，要么阻塞线程并（希望）在稍后被释放。换句话说，通常无法在调用 lock() 之前知道是否有线程阻塞在该信号量上。然而，一些实时操作系统提供了 trylock() 函数，如果资源可用则加锁，否则返回错误码。

```
void lock(Semaphore* s) {
    do {
        等待();
    } while (s->count <= 0);
    --s->count; /* 此操作必须实现为
                   临界区 */
```

[1] 或者它可能是P，代表"Piraat"（海盗）。似乎没有人确切知道。

```
};

void release(Semaphore*  s) {
     ++s->count; /*  this operation must be implemented as a
                   critical region  */
}
```

Other terms for lock include *decrement* and *wait*; other terms for release include *free, increment*, and *signal*, used in symmetric pairs, (lock/free, lock/release, wait/signal, decrement/increment) depending on the usage of the semaphore.

Mutexes may be *recursive*, a useful property in which once a thread has successfully locked the semaphore, further calls it makes to `lock()` do not block the thread (that is, the thread cannot block itself through multiple calls to the same mutex). For a recursive mutex, the semaphore is freed by a thread when the number of calls it makes to free equals the number of calls it made to `lock()`.

4.5 Guarded Call Pattern

The Guarded Call Pattern serializes access to a set of services that could potentially interfere with each other in some way if called simultaneously by multiple callers. In this pattern, access is serialized by providing a locking mechanism that prevents other threads from invoking the services while locked.

4.5.1 Abstract

The Guarded Call Pattern uses semaphores to protect a related set of services (functions) from simultaneous access by multiple clients in a preemptive multitasking environment, in a process known as *mutual exclusion*. The Guarded Call Pattern provides timely access to services, provided that they are not in use by other tasks. However, if not mixed with other patterns, the use of this pattern can lead to unbounded priority inversion, such as that shown in Figure 4-5.

4.5.2 Problem

The problem this pattern addresses is the need for a timely synchronization or data exchange between threads. In such cases, it may not be possible to wait for an asynchronous (queuing) rendezvous. A synchronous (function call) rendezvous can be made more timely, but this must be done carefully to avoid data corruption and erroneous computation.

4.5.3 Pattern Structure

The pattern structure for this pattern is shown in Figure 4-15. In this case, multiple `PreemptiveTasks` access the `GuardedResource` through its functions. Inside those


```
};

void release(Semaphore* s) {++s->count; /* 此操作必须作为临界区实现 */ } lock 的其他术语包括 decrement 和 wait；release 的其他术语包括 free、increment 和 signal，根据信号量的用法以对称对的形式使用（lock/free、lock/release、wait/signal、decrement/increment）。互斥锁可以是递归的，这是一个有用的特性：一旦线程成功锁定了信号量，它后续对 lock() 的调用不会阻塞该线程（即线程不会因多次调用同一互斥锁而阻塞自身）。对于递归互斥量，当线程调用 free 的次数等于其调用 lock() 的次数时，信号量被该线程释放。
```

4.5 受保护调用模式

受保护调用模式对一组服务的访问进行串行化，如果多个调用者同时调用这些服务，它们可能会以某种方式相互干扰。在此模式中，通过提供一种加锁机制来实现访问的串行化，该机制在锁定期间阻止其他线程调用这些服务。

4.5.1 摘要

受保护调用模式使用信号量来保护一组相关服务（函数），在抢占式多任务环境中防止多个客户端同时访问，这一过程称为互斥。受保护调用模式提供对服务的及时访问，前提是这些服务未被其他任务使用。然而，如果未与其他模式结合使用，该模式的使用可能导致无界优先级反转，如图4-5所示。

4.5.2 问题

该模式所解决的问题是线程之间需要进行及时的同步或数据交换。在这种情况下，可能无法等待异步（排队）会合。同步（函数调用）会合可以更加及时，但必须谨慎操作以避免数据损坏和错误计算。

4.5.3 模式结构

该模式的模式结构如图 4-15所示。在这种情况下，多个抢占式任务 通过其函数访问受保护资源 。在这些

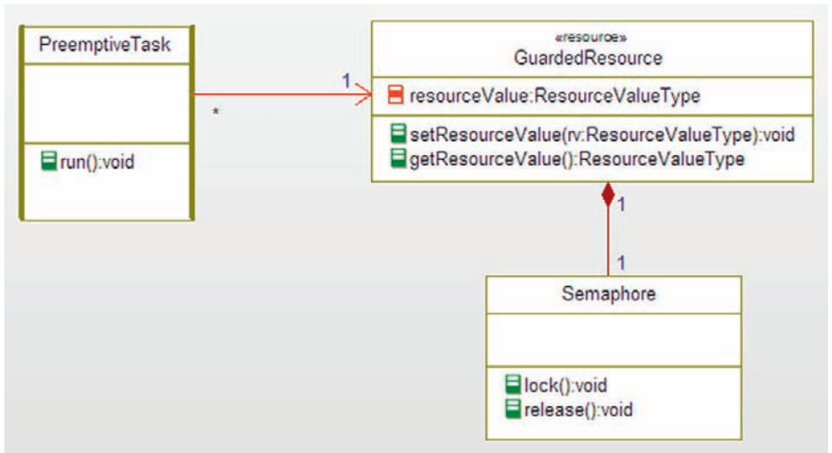


Figure 4-15: Guarded Call Pattern

functions, calls are made to lock and release the resource. The scheduler supports blocking by placing a task on the blocked queue when it invokes a locked semaphore and unblocks it when that semaphore is released. The scheduler must implement the semaphore `lock()` function as a critical region to eliminate the possibility of race conditions.

4.5.4 Collaboration Roles

This section describes the roles for this pattern.

4.5.4.1 GuardedResource

The `GuardedResource` is a shared resource that employs mutex semaphores to enforce mutual exclusion of access to its methods. Inside the relevant functions[14], and prior to the access of the resource *per se*, a call is made to the `lock()` function of the associated `Semaphore` instance. if the `Semaphore` is in the unlocked state, then it becomes locked; if it is in the locked state, then the `Semaphore` signals the `StaticPriorityScheduler` to block the currently running task. It is important that the relevant functions of a specific resource instance must all share the same instance of the `Semaphore` class. This ensures that they are protected together as a unit from simultaneous access by multiple `PreemptiveTasks`.

4.5.4.2 PreemptiveTask

The `PreemptiveTask` represents an «active» class that is run via a preemptive multitasking scheduler. It accesses the `GuardedResource` by invoking the latter's functions, which in turn are protected by the `Semaphore`.

[14] I say "relevant functions" because the class may offer some functions that don't require protection.

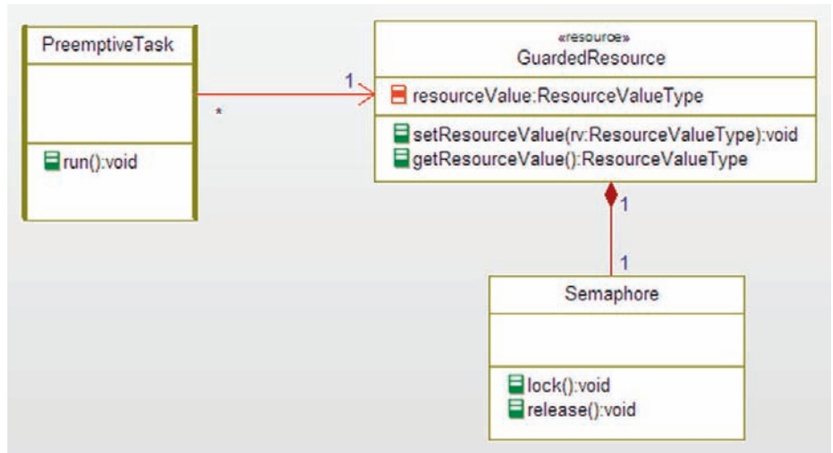


图 4-15：受保护调用模式

函数内部，会调用加锁和释放资源的操作。调度器支持阻塞机制：当任务调用已加锁的信号量时，将其放入阻塞队列；当该信号量被释放时，再将其解除阻塞。调度器必须将信号量`lock()`函数实现为临界区，以消除竞态条件的可能性。

4.5.4 协作角色

本节描述该模式的角色。

4.5.4.1 受保护资源

受保护资源是一种共享资源，它使用互斥信号量来强制对其方法的互斥访问。在相关函数[14]，内部，且在访问资源本身之前，会调用关联信号量实例的 lock() 函数。如果信号量处于解锁状态，则它变为加锁状态；如果它处于加锁状态，则信号量向 StaticPriorityScheduler 发出信号，以阻塞当前正在运行的任务。重要的是，特定资源实例的相关函数必须全部共享同一个信号量类的实例。这确保它们作为一个整体受到保护，免受多个抢占式任务的并发访问。

4.5.4.2 PreemptiveTask

`PreemptiveTask` 表示一个「活动」类，它通过抢占式多任务调度器运行。它通过调用`GuardedResource` 的函数来访问后者，而这些函数又受到`Semaphore`的保护。

[14] 我说"相关 函数"，是因为该类可能提供一些不需要保护的函数。

4.5.4.3 Semaphore

The Semaphore is a mutual exclusion semaphore class that serializes access to a GuardedResource. The guarded functions of the GuardedResource invoke the lock() function of the Semaphore whenever they are called and release()once the service is complete. Other client threads that attempt to invoke a service when its associated Semaphore is locked become blocked until the Semaphore is unlocked. The blocking is performed when the Semaphore signals the StaticPriorotyScheduler that a call attempt was made by the currently active thread. The mutexID identifies which Mutex is blocking the task. This element is usually provided by the RTOS. Note this is the same element that appears in the Static Priority Pattern earlier in this chapter.

4.5.4.4 StaticPriorityScheduler

This object orchestrates the execution of multiple threads based on their priority. In this pattern, we are focusing on the responsibility (not shown) of this element to block a task if it attempts to lock an already-locked Semaphore and to unblock all tasks blocked on the mutexID when this lock is released. This element is usually provided by the RTOS. Note this is the same element that appears in the Static Priority Pattern earlier in this chapter.

4.5.5 Consequences

This pattern provides a timely access to a resource and at the same time prevents multiple simultaneous accesses that can result in data corruption and system misbehavior. If the resource is not locked, then access proceeds without delay. If the resource is currently locked, then the caller must block until the lock on the resource is released. Naïve use can result in unbounded priority inversion in the manner described in Figure 4-5. See Section 4.5.7 Related Patterns for a discussion of an RTOS-supported solution.

4.5.6 Implementation Strategies

The tricky part of implementing the pattern lies in the implementation of the Mutex. Usually, the RTOS will provide a Semaphore instance, accessible through operations such as:

- OSSemaphore OS_create_semaphore()
  creates a semaphore and returns a pointer to it.
- void OS_destroy_semaphore(OSSemaphore*)
  destroys the semaphore pointed at.
- void OS_lock_semaphore(OSSemaphore*)
  locks the relevant semaphore. If called with an unlocked OSSemaphore, it becomes locked. If already locked, then it blocks the current task with its internally-stored mutexID. The scheduler must store the task data including the continuation entry point and stack.

4.5.4.3 信号量

Semaphore 是一个互斥信号量类，用于串行化对GuardedResource的访问。GuardedResource 的受保护函数在被调用时会调用Semaphore 的lock()函数，并在服务完成后调用release()。其他客户端线程在关联的Semaphore被锁定时尝试调用服务会被阻塞，直到Semaphore 被解锁。阻塞是在Semaphore 向StaticPriorotyScheduler 发出信号，表明当前活动线程进行了一次调用尝试时执行的。mutexID 标识哪个Mutex 正在阻塞该任务。此元素通常由RTOS提供。注意，这与本章前面静态优先级模式中出现的元素相同。

4.5.4.4 StaticPriorityScheduler

该对象根据优先级来编排多个线程的执行。在此模式中，我们关注的是该元素（未显示）的职责：当任务试图锁定一个已被锁定的信号量 时将其阻塞，并在该锁被释放时解除所有阻塞在互斥量ID 上的任务。该元素通常由实时操作系统提供。注意，这与本章前面静态优先级模式中出现的是同一个元素。

4.5.5 结果

此模式提供对资源的及时访问，同时防止可能导致数据损坏和系统行为异常的多次同时访问。如果资源未被锁定，则访问立即进行。如果资源当前已被锁定，则调用者必须阻塞，直到资源上的锁被释放。简单直接的使用方式可能导致如图4-5所述的无界优先级反转。关于实时操作系统支持的解决方案的讨论，请参见第4.5.7节相关模式。

4.5.6 实现策略

实现该模式的棘手之处在于 互斥锁 的实现。通常，实时操作系统会提供一个 信号量 实例，可通过如下操作访问：

- OSSemaphore OS_create_semaphore() 创建
一个信号量 并返回指向它的指针。
- void OS_destroy_se
maphore(OSSemaphore*) 销毁所指向的信号量。
- void OS_lock_semaphore(OSSemaphore*)锁定相关信号量。如果调用时OSSemaphore 处于未锁定状态，则将其变为锁定状态。如果已经锁定，则使用其内部存储的 mutexID 阻塞当前任务。调度器必须存储任务数据，包括续体入口点和栈。

- `void OS_release_semaphore(OSSemaphore*)`
  releases the lock on the relevant semaphore and unblocks all tasks blocked on its internally-stored `MutexID`.

Of course, the names and parameters of the services are OS specific.

4.5.7 Related Patterns

The Guarded Call Pattern is used with Static Priority or other preemptive multitasking environments. It is a less draconian policy than is enforced by the Critical Region Pattern (above) because it does not interfere with the execution of higher priority tasks that don't need access to the resource. It is more responsive than the Queuing Pattern (below) because it doesn't need to wait for the resource's thread to become active to check for the request.

There is one very important class of variants of this pattern that solves the problem of unbounded priority inversion that uses a concept of *priority inheritance.* The basic idea is that each resource has an addition attribute (variable) knows as its *priority ceiling,* which is equal to the priority of the highest-priority task that can ever access the resource. The underlying problem with naïve implementation of the Guarded Call Pattern is that intermediate priority tasks that don't need the resource can preempt the low-priority task that currently owns the resource needed by the blocked high-priority task. There can be any number of such intermediate-priority tasks, leading to chained blocking of the high priority task. The priority inheritance solution is to elevate the priority of the low-priority task as soon as the high-priority task attempts to lock the resource owned by the low-priority task. This can be done by creating a semaphore that implements the priority inversion. This is the approach taken, for example, by VxWorks™ from Wind River, which implements a set of mutex semaphore functions such as:

```
SEM_ID semM;
semM  =  semMCreate(. . .);
semTake(semM, FLAG);
semGive(semM);
```

Values for FLAG include WAIT_FOREVER, and – important to this discussion – SEM_INVERSION_SAFE. When the latter flag value is used, VxWorks' mutex semaphores implement priority inheritance.

In Figure 4-5, if the locking mutex is implemented in this way, the following sequence of events holds:

1. Task Z runs and locks Resource R at its nominal priority of 100.
2. Task A runs at priority 1, and attempts to lock the mutex guarding R and blocks. At this point, the priority of Task Z is elevated to that of Task A (i.e., priority 1).

- `void OS_release_semaphore(OSSemaphore*)`释放相关信号量上的锁，并解除所有因其内部存储的 `MutexID` 而被阻塞的任务的阻塞状态。

当然，服务的名称和参数是操作系统特定的。

4.5.7 相关模式

受保护调用模式与静态优先级或其他抢占式多任务环境配合使用。与临界区模式（上文）所强制执行的策略相比，它的策略不那么严苛，因为它不会干扰那些不需要访问该资源的更高优先级任务的执行。它比排队模式（下文）响应更及时，因为它不需要等待资源所属线程变为活动状态来检查请求。

该模式有一类非常重要的变体，它使用优先级继承的概念解决了无界优先级反转问题。基本思想是每个资源都有一个额外的属性（变量），称为其优先级天花板，它等于可能访问该资源的最高优先级任务的优先级。受保护调用模式的朴素实现存在的根本问题是，不需要该资源的中间优先级任务可以抢占当前拥有被阻塞的高优先级任务所需资源的低优先级任务。这样的中间优先级任务可以有任意多个，导致高优先级任务的链式阻塞。优先级继承的解决方案是，一旦高优先级任务尝试对低优先级任务所拥有的资源加锁，就立即提升低优先级任务的优先级。这可以通过创建一个实现优先级反转的信号量来实现。例如，Wind River 的 VxWorksTM 就采用了这种方法，它实现了一组互斥信号量函数，例如：

```
SEM_ID semM; semM   =
semMCreate(. .);
semTake(semM, FLAG);
semGive(semM);
```

FLAG 的取值包括 WAIT_FOREVER，以及——对本讨论很重要的——SEM_INVERSION_SAFE。当使用后一个标志值时，VxWorks 的互斥信号量实现优先级继承。

在 图 4-5中，如果加锁互斥锁是以这种方式实现的，则以下事件序列成立：

1. 任务Z运行，并以100的标称优先级锁定资源R。2. 任务A以优先级1运行，并尝试锁定守卫R的互斥锁，然后阻塞。此时，任务Z的优先级被提升到任务A的优先级（即优先级1）。

3. Task Y becomes ready to run. With a normal naïve mutex, it would run and preempt Task Z. However, its priority (99) is less than the priority at which Task Z is running so it remains on the Task Ready queue.
4. Task Z completes its use of the Resource R and unlocks it. At this point, the priority of Task Z is demoted to its nominal priority (100).
5. Task A unblocks and now uses the resource and completes within its deadline.
6. Task Y now, being the highest ready task, runs.
7. Task Y completes, and finally, Task Z is allowed to complete its execution.

You can see in this simple example how priority inheritance prevented the unbounded priority inversion that a normal counting semaphore would allow.

4.5.8 Example

The example shown in Figure 4-16 is a portion of a flight control system. In it, the ship KinematicData (position and attitude) are managed as a shared resource for multiple clients.

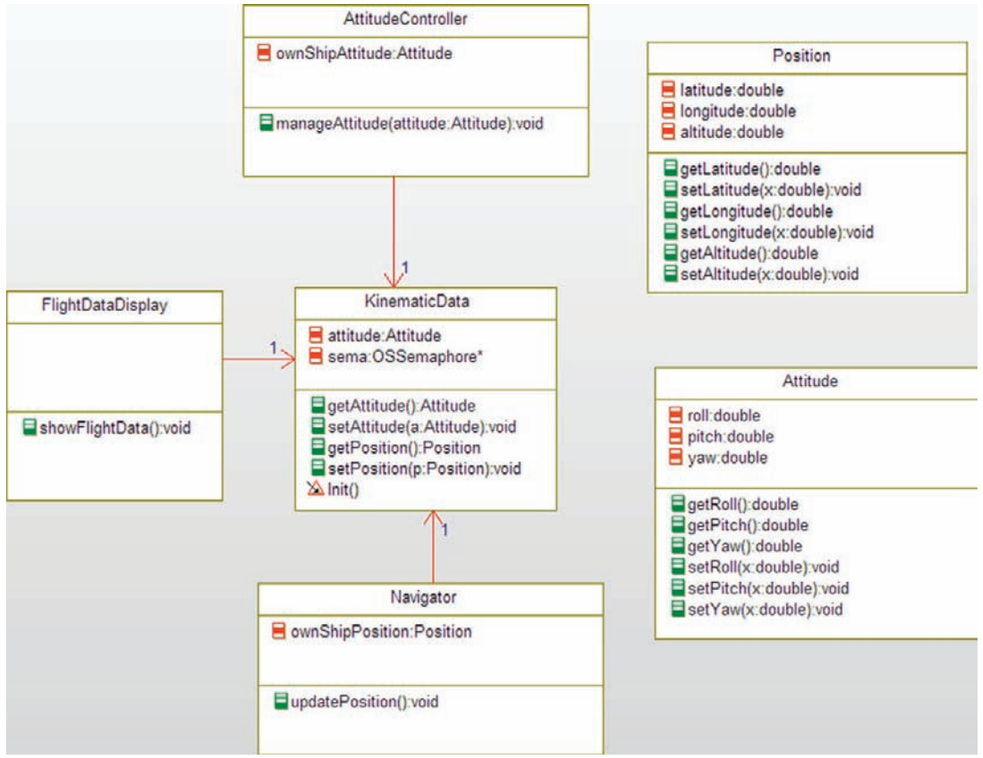


Figure 4-16: Guarded Call example

3. 任务Y变为就绪。使用普通的朴素互斥锁时，它会运行并抢占任务Z。然而，它的优先级（99）低于任务Z运行时的优先级，因此它留在任务就绪队列中。4. 任务Z完成对资源R的使用并解锁它。此时，任务Z的优先级被降回其标称优先级（100）。5. 任务A解除阻塞，现在使用该资源并在其截止时间内完成。6. 任务Y现在作为最高优先级的就绪任务运行。7. 任务Y完成，最后，任务Z被允许完成其执行。

在这个简单的示例中，你可以看到优先级继承如何防止了普通计数信号量所允许的无界优先级反转。

4.5.8 示例

图4-16所示的示例是飞行控制系统的一部分。在该示例中，飞船的KinematicData（位置和姿态）作为多个客户端的共享资源进行管理。

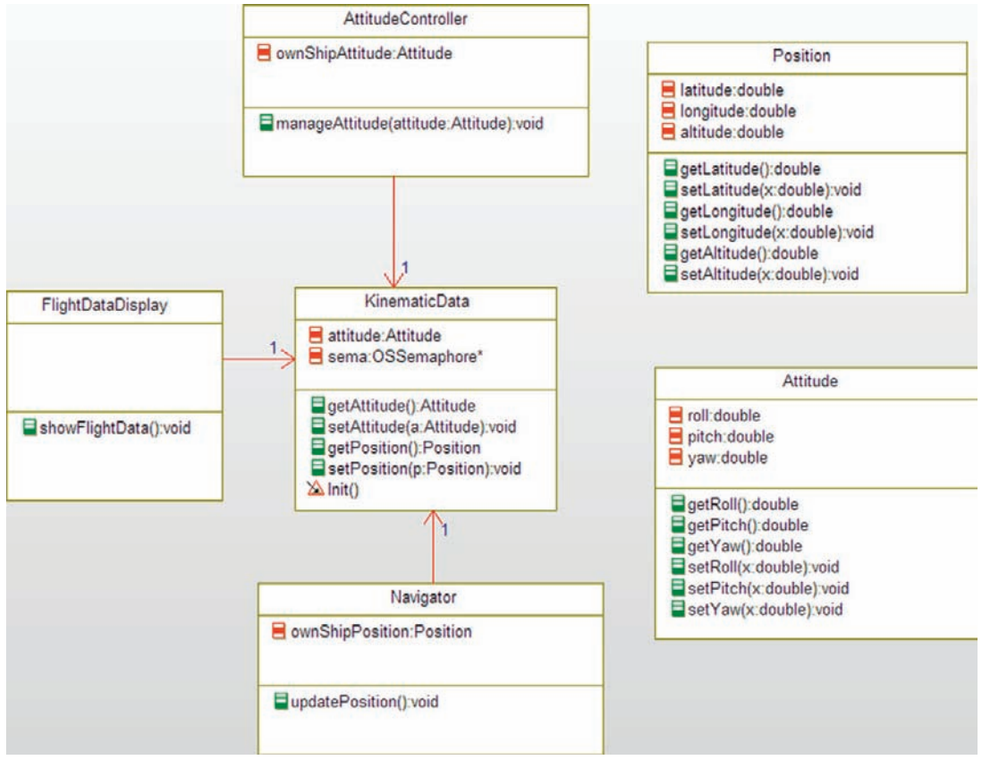


图4-16：受保护调用示例

Since the setting and getting of these values is not an atomic process, there is a possibility of displaying invalid data if, for example, the `FlightDataDisplay showFlightData()` function is preempted to update the attitude data by the `AttitudeController`. The use of semaphores in the methods ensures that this cannot happen.

The next several code listings (Code Listing 4-19 through Code Listing 4-30) give the code for the elements of the Guarded Call example in Figure 4-16. The first gives the specification for the OS calls used for the semaphores. The next four show the code for the complex data types `Position` and `Attitude`, respectively.

Code Listing 4-23 and Code Listing 4-24 give the code for the resource (`KinematicData`) that uses the `OSSemaphore` to guard its operations to set and get that data. The initializer `KinematicData_Init()` creates a semaphore via a call to `OS_create_semaphore()` while the clean-up function `KinematicData_CleanUp()` calls `OS_destroy_semaphore()` to free up its memory. Its accessor and mutator methods to get and set attitude and position data all invoke the `OS_lock_semaphore()`, do their processing, and then invoke `OS_release_semaphore()`.

The code listings after that show the code for the clients: the `FlightDataDisplay`, `AttitudeController`, and `Navigator`. These all share the information stored in the `KinematicData` object and so must be protected from data corruption.

```
#ifndef GuardedCallExample_H
#define GuardedCallExample_H


struct Attitude;
struct AttitudeController;
struct FlightDataDisplay;
struct GuardTester;
struct GuardedCallBuilder;
struct KinematicData;
struct Navigator;
struct Position;

/*  OS provided types and functions  */
struct OSSemaphore;
struct OSSemaphore*  OS_create_semaphore(void);
void OS_destroy_semaphore(struct OSSemaphore*  sPtr);
void OS_lock_semaphore(struct OSSemaphore*  sPtr);
void OS_release_semaphore(struct OSSemaphore*  sPtr);

#endif
```

Code Listing 4-7: GuardedCallExample.h

由于这些值的设置和获取不是原子过程，因此存在显示无效数据的可能性，例如，如果`FlightDataDisplay showFlightData()`函数被`AttitudeController`抢占以更新姿态数据。在这些方法中使用信号量可确保这种情况不会发生。

接下来的几个代码清单（代码清单 4-19 至代码清单 4-30）给出了图 4-16 中受保护调用示例各元素的代码。第一个给出了用于信号量的操作系统调用的规范。接下来的四个分别展示了复杂数据类型 Position 和 Attitude 的代码。

代码清单 4-23 和代码清单 4-24 给出了资源（KinematicData）的代码，该资源使用 OSSemaphore 来守卫其设置和获取数据的操作。初始化器 KinematicData_Init() 通过调用 OS_create_semaphore() 创建一个信号量，而清理函数 KinematicData_CleanUp() 则调用 OS_destroy_semaphore() 来释放其内存。其用于获取和设置姿态及位置数据的访问器方法和修改器方法都会调用 OS_lock_semaphore()，执行其处理，然后调用 OS_release_semaphore()。

其后的代码清单展示了客户端的代码：`FlightDataDisplay`、`AttitudeController`和`Navigator`。它们都共享存储在`KinematicData` 对象中的信息，因此必须受到保护以防止数据损坏。

```
#ifndef GuardedCallExample_H #
define GuardedCallExample_H


struct Attitude; struct
AttitudeController; struct
FlightDataDisplay; struct
GuardTester; struct
GuardedCallBuilder; struct
KinematicData; struct
Navigator; struct Position;

/* 操作系统提供的类型和函数 */ struct OSSemaphore; struct
OSSemaphore* OS_create_semaphore(void); void OS_destroy_
semaphore(struct OSSemaphore* sPtr); void OS_lock_
semaphore(struct OSSemaphore* sPtr); void OS_release_
semaphore(struct OSSemaphore* sPtr);


#endif
```

代码清单 4-7：GuardedCallExample.h

```
#ifndef Position_H
#define Position_H

#include "GuardedCallExample.h"

typedef struct Position Position;
struct Position {
    double altitude;         /*# # attribute altitude  */
    double latitude;         /*# # attribute latitude  */
    double longitude;        /*# # attribute longitude  */
};

/*  Constructors and destructors:*/
void Position_Init(Position*  const me);
void Position_Cleanup(Position*  const me);

/*  Operations  */
double Position_getAltitude(Position*  const me);
double Position_getLatitude(Position*  const me);
double Position_getLongitude(Position*  const me);
void Position_setAltitude(Position*  const me, double x);
void Position_setLatitude(Position*  const me, double x);
void Position_setLongitude(Position*  const me, double x);

Position  *  Position_Create(void);
void Position_Destroy(Position*  const me);

#endif
```

Code Listing 4-19: Position.h

```
#include "Position.h"

void Position_Init(Position*  const me) {
}

void Position_Cleanup(Position*  const me) {
}

double Position_getAltitude(Position*  const me) {
    return me->altitude;
}

double Position_getLatitude(Position*  const me) {
    return me->latitude;
}
```



```
#ifndef Position_H #
define Position_H

#include "GuardedCallExample.h"

typedef struct Position Position; struct Position { double altitude; /*#
# 属性 altitude */ double latitude; /*# # 属性 latitude */ double
longitude; /*# # 属性 longitude */ };

/* 构造函数与析构函数：*/ void Position_Init(Position*
const me); void Position_Cleanup(Position* const
me);

/* 操作 */
double Position_getAltitude(Position* const me); double Position_
getLatitude(Position* const me); double Position_
getLongitude(Position* const me); void Position_setAltitude(Position*
const me, double x); void Position_setLatitude(Position* const me,
double x); void Position_setLongitude(Position* const me, double x);

Position * Position_Create(void); void Position_
Destroy(Position* const me);

#endif
```

代码清单 4-19: Position.h

```
#include "Position.h" void Position_Init(Position* const
me) { } void Position_Cleanup(Position* const me) { }
double Position_getAltitude(Position* const me) { return
me->altitude; } double Position_getLatitude(Position* const
me) { return me->latitude; }
```

```
double Position_getLongitude(Position*  const me) {
    return me->longitude;
}

void Position_setAltitude(Position*  const me, double x) {
    me->altitude  =  x;
}

void Position_setLatitude(Position*  const me, double x) {
    me->latitude  =  x;
}

void Position_setLongitude(Position*  const me, double x) {
    me->longitude  =  x;
}

Position  *  Position_Create(void) {
    Position*  me  =  (Position  *) malloc(sizeof(Position));
    if(me!=NULL) {
           Position_Init(me);
    }
  return me;
}

void Position_Destroy(Position*  const me) {
    if(me!=NULL) {
       Position_Cleanup(me);
    }
    free(me);
}
```

Code Listing 4-8: Position.c

```
#ifndef Attitude_H
#define Attitude_H

#include "GuardedCallExample.h"

typedef struct Attitude Attitude;
struct Attitude {
    double pitch;        /*## attribute pitch  */
    double roll;         /*## attribute roll  */
    double yaw;          /*## attribute yaw  */
};

/*  Constructors and destructors:*/
void Attitude_Init(Attitude*  const me);
void Attitude_Cleanup(Attitude*  const me);
```


double Position_getLongitude(Position* const me) { return me-> longitude; } void Position_setAltitude(Position* const me, double x) { me->altitude = x; } void Position_setLatitude(Position* const me, double x) { me->latitude = x; } void Position_setLongitude(Position* const me, double x) { me->longitude = x; } Position * Position_ Create(void) { Position* me = (Position *) malloc(sizeof(Position)); if(me!=NULL) { Position_Init(me); } return me; } void Position_ Destroy(Position* const me) { if(me!=NULL) { Position_Cleanup(me); } free(me); }

代码清单 4-8：Position.c

#ifndef Attitude_H #define Attitude_H #include " GuardedCallExample.h" typedef struct Attitude Attitude; struct Attitude { double pitch; /*## 属性 pitch */ double roll; /*## 属性 roll */ double yaw; /*## 属性 yaw */ };

/* 构造函数与析构函数：*/ void Attitude_Init(Attitude* const me); void Attitude_Cleanup(Attitude* const me);

```
/*  Operations  */
double Attitude_getPitch(Attitude*  const me);
double Attitude_getRoll(Attitude*  const me);
double Attitude_getYaw(Attitude*  const me);
void Attitude_setPitch(Attitude*  const me, double x);
void Attitude_setRoll(Attitude*  const me, double x);
void Attitude_setYaw(Attitude*  const me, double x);

Attitude  *  Attitude_Create(void);
void Attitude_Destroy(Attitude*  const me);

#endif
```

Code Listing 4-9: Attitude.h

```
#include "Attitude.h"

void Attitude_Init(Attitude*  const me) {
}

void Attitude_Cleanup(Attitude*  const me) {
}

double Attitude_getPitch(Attitude*  const me) {
    return me->pitch;
}

double Attitude_getRoll(Attitude*  const me) {
    return me->roll;
}

double Attitude_getYaw(Attitude*  const me) {
    return me->yaw;
}

void Attitude_setPitch(Attitude*  const me, double x) {
    me->pitch  =  x;
}

void Attitude_setRoll(Attitude*  const me, double x) {
    me->roll  =  x;
}

void Attitude_setYaw(Attitude*  const me, double x) {
    me->yaw  =  x;
}
```


/* 操作 */ double Attitude_getPitch(Attitude* const me); double Attitude_getRoll(Attitude* const me); double Attitude_getYaw(Attitude* const me); void Attitude_setPitch(Attitude* const me, double x); void Attitude_setRoll(Attitude* const me, double x); void Attitude_setYaw(Attitude* const me, double x);

Attitude * Attitude_Create(void); void Attitude_Destroy(Attitude* const me);

#endif

代码清单 4-9: Attitude.h

#include "Attitude.h" void Attitude_Init(Attitude* const me) { } void Attitude_Cleanup(Attitude* const me) { } double Attitude_getPitch(Attitude* const me) { return me->pitch; } double Attitude_getRoll(Attitude* const me) { return me->roll; } double Attitude_getYaw(Attitude* const me) { return me->yaw; } void Attitude_setPitch(Attitude* const me, double x) { me->pitch = x; } void Attitude_setRoll(Attitude* const me, double x) { me->roll = x; } void Attitude_setYaw(Attitude* const me, double x) { me->yaw = x; }

```
Attitude  *  Attitude_Create(void){
    Attitude*  me  =  (Attitude  *)malloc(sizeof(Attitude));
    if(me!=NULL)
        {
            Attitude_Init(me);
        }
    return me;
}

void Attitude_Destroy(Attitude*  const me){
    if(me!=NULL)
        {
            Attitude_Cleanup(me);
        }
    free(me);
}
```

Code Listing 4-10: Attitude.c

```
#ifndef KinematicData_H
#define KinematicData_H

#include "GuardedCallExample.h"
#include "Attitude.h"
#include "OSSemaphore.h"
#include "Position.h"

typedef struct KinematicData KinematicData;
struct KinematicData {
    struct Attitude attitude;
    struct Position position;
    OSSemaphore*  sema;        /*## mutex semaphore  */
};

/*  Constructors and destructors:*/
void KinematicData_Init(KinematicData*  const me);
void KinematicData_Cleanup(KinematicData*  const me);

/*  Operations  */

Attitude KinematicData_getAttitude(KinematicData*  const me);
struct Position KinematicData_getPosition(KinematicData*  const me);

void KinematicData_setAttitude(KinematicData*  const me, Attitude a);

void KinematicData_setPosition(KinematicData*  const me, Position p);
```


姿态 * 姿态_Create(void) { 姿态* me = (姿态 *) malloc(sizeof(姿态));
if(me!=空) { 姿态_Init(me); } return me; } void 姿态_Destroy(姿态* const
me) { if(me!=空) { 姿态_Cleanup(me); } free(me); }

代码清单4-10：Attitude.c

#ifndef KinematicData_H #
define KinematicData_H

#include "GuardedCallExample.h" #
include "Attitude.h" #include "
OSSemaphore.h" #include "Position.h"

typedef struct KinematicData KinematicData; struct
KinematicData { struct Attitude attitude; struct Position
position; OSSemaphore* sema; /*## 互斥信号量 */ };

/* 构造函数与析构函数：*/ void KinematicData_Init(
KinematicData* const me); void KinematicData_
Cleanup(KinematicData* const me);

/* 操作 */

姿态 KinematicData_getAttitude(KinematicData* const me); 结构体 Position
KinematicData_getPosition(KinematicData* const me);

void KinematicData_setAttitude(KinematicData* const me, Attitude a);

v oid KinematicData_setPosition(KinematicData* const me, Position p) ;

```
KinematicData  *  KinematicData_Create(void);

void KinematicData_Destroy(KinematicData*  const me);

#endif
```

Code Listing 4-11: KinematicData.h

```
#include "KinematicData.h"

void KinematicData_Init(KinematicData*  const me) {
    Attitude_Init(&(me->attitude));
    Position_Init(&(me->position));
    me->sema  =  OS_create_semaphore();
}

void KinematicData_Cleanup(KinematicData*  const me) {
     OS_destroy_semaphore(me->sema);
}

struct Position KinematicData_getPosition(KinematicData*  const me) {
    Position p;

    /*  engage the lock  */
    OS_lock_semaphore(me->sema);

    p  =  me->position;

    /*  release the lock  */
    OS_release_semaphore(me->sema);

    return p;
}

void KinematicData_setAttitude(KinematicData*  const me, Attitude a) {
  /*  engage the lock  */
  OS_lock_semaphore(me->sema);

  me->attitude  =  a;

  /*  release the lock  */
  OS_release_semaphore(me->sema);
}

void KinematicData_setPosition(KinematicData*  const me, Position p) {
  /*  engage the lock  */
  OS_lock_semaphore(me->sema);
```


KinematicData * KinematicData_Create(void); void KinematicData_Destroy(KinematicData* const me); #endif

代码清单4-11：KinematicData.h

#include "KinematicData.h" void KinematicData_Init(KinematicData* const me) { Attitude_Init(&(me->attitude)); Position_Init(&(me->position)); me->sema = OS_create_semaphore(); } void KinematicData_Cleanup(KinematicData* const me) { OS_destroy_semaphore(me->sema); } struct Position KinematicData_getPosition(KinematicData* const me) { Position p; /* 加锁 */ OS_lock_semaphore(me->sema); p = me->position; /* 释放锁 */ OS_release_semaphore(me->sema); return p; } void KinematicData_setAttitude(KinematicData* const me, Attitude a) { /* 加锁 */ OS_lock_semaphore(me->sema); me->attitude = a; /* 释放锁 */ OS_release_semaphore(me->sema); } void KinematicData_setPosition(KinematicData* const me, Position p) { /* 加锁 */ OS_lock_semaphore(me->sema);

```
  me->position  =  p;

  /*  release the lock  */
  OS_release_semaphore(me->sema);

}

KinematicData  *  KinematicData_Create(void){
    KinematicData*  me  =  (KinematicData  *)
malloc(sizeof(KinematicData));
    if(me!=NULL){
         KinematicData_Init(me);
    }
    return me;
}
void KinematicData_Destroy(KinematicData*  const me){
    if(me!=NULL){
        KinematicData_Cleanup(me);
    }
    free(me);
}
Attitude KinematicData_getAttitude(KinematicData*  const me){
    Attitude a;

    /*  engage the lock  */
    OS_lock_semaphore(me->sema);

    a  =  me->attitude;

    /*  release the lock  */
    OS_release_semaphore(me->sema);

    return a;
}
```

Code Listing 4-24: KinematicData.c

```
#ifndef AttitudeController_H
#define AttitudeController_H

#include "GuardedCallExample.h"
#include "Attitude.h"
struct KinematicData;

typedef struct AttitudeController AttitudeController;
struct AttitudeController {
    struct Attitude ownShipAttitude;
```


me->position = p; /* 释放锁 */ OS_release_semaphore(me->sema); } KinematicData * KinematicData_Create(void) { KinematicData* me = (KinematicData *) malloc(sizeof(KinematicData)); if(me!=NULL) { KinematicData_Init(me); } return me; } void KinematicData_ Destroy(KinematicData* const me) { if(me!=NULL) { KinematicData_ Cleanup(me); } free(me); } Attitude KinematicData_ getAttitude(KinematicData* const me) { Attitude a; /* 加锁 */ OS_lock_ semaphore(me->sema); a = me->attitude; /* 释放锁 */ OS_release_ semaphore(me->sema); return a; }

代码清单4-24：KinematicData.c

#ifndef AttitudeController_H #define AttitudeController_H # include "GuardedCallExample.h" #include "Attitude.h" struct KinematicData; typedef struct AttitudeController AttitudeController; struct AttitudeController { struct Attitude ownShipAttitude;

```
    struct KinematicData*  itsKinematicData;
};

/*  Constructors and destructors:*/

void AttitudeController_Init(AttitudeController*  const me);
void AttitudeController_Cleanup(AttitudeController*  const me);

/*  Operations  */

void AttitudeController_manageAttitude(AttitudeController*  const me);

struct KinematicData*
AttitudeController_getItsKinematicData(const AttitudeController*
const me);

void AttitudeController_setItsKinematicData(AttitudeController*
const me, struct KinematicData*  p_KinematicData);

AttitudeController  *  AttitudeController_Create(void);

void AttitudeController_Destroy(AttitudeController*  const me);

#endif
```

Code Listing 4-12: AttitudeController.h

```
#include "AttitudeController.h"
#include "KinematicData.h"

static void cleanUpRelations(AttitudeController*  const me);

void AttitudeController_Init(AttitudeController*  const me) {
    Attitude_Init(&(me->ownShipAttitude));
    me->itsKinematicData  =  NULL;
}

void AttitudeController_Cleanup(AttitudeController*  const me) {
    cleanUpRelations(me);
}

void AttitudeController_manageAttitude(AttitudeController*  const me) {
    KinematicData_setAttitude(me->itsKinematicData, me->ownShipAttitude);
}

struct KinematicData*
AttitudeController_getItsKinematicData(const AttitudeController*  const me) {
```


struct KinematicData* itsKinematicData; }; /* 构造函数与析构函数: */ void AttitudeController_Init(AttitudeController* const me); void AttitudeController_Cleanup(AttitudeController* const me); /* 操作 */ void AttitudeController_manageAttitude(AttitudeController* const me); struct KinematicData* AttitudeController_getItsKinematicData(const AttitudeController* const me); void AttitudeController_setItsKinematicData(AttitudeController* const me, struct KinematicData* p_KinematicData); AttitudeController * AttitudeController_Create(void); void AttitudeController_Destroy(AttitudeController* const me); #endif

代码清单4-12: AttitudeController.h

#include "AttitudeController.h" #include "KinematicData.h" static void cleanUpRelations(AttitudeController* const me); void AttitudeController_Init(AttitudeController* const me) { Attitude_Init(&(me->ownShipAttitude)); me->itsKinematicData = 空; } void AttitudeController_清理(AttitudeController* const me) { cleanUpRelations(me); } void AttitudeController_manageAttitude(AttitudeController* const me) { KinematicData_setAttitude(me->itsKinematicData, me->本舰姿态); }

struct KinematicData* AttitudeController_getItsKinematicData(const AttitudeController* const me) {

```
  return (struct KinematicData*)me->itsKinematicData;
}

void AttitudeController_setItsKinematicData(AttitudeController*
const me, struct KinematicData*  p_KinematicData) {
    me->itsKinematicData  =  p_KinematicData;
}

AttitudeController  *  AttitudeController_Create(void) {
    AttitudeController*  me  =  (AttitudeController  *)
malloc(sizeof(AttitudeController));
    if(me!=NULL) {
        AttitudeController_Init(me);
    }
    return me;
}

void AttitudeController_Destroy(AttitudeController*  const me) {
    if(me!=NULL) {
        AttitudeController_Cleanup(me);
    }
    free(me);
}

static void cleanUpRelations(AttitudeController*  const me) {
    if(me->itsKinematicData !=  NULL) {
        me->itsKinematicData  =  NULL;
    }
}
```

Code Listing 4-13: AttitudeController.c


```
return (struct KinematicData*)me->itsKinematicData; } void
AttitudeController_setItsKinematicData(AttitudeController* const me, struct
KinematicData* p_KinematicData) { me->itsKinematicData    = p_
KinematicData; } AttitudeController * AttitudeController_Create(void) {
AttitudeController* me    =    (AttitudeController *)
malloc(sizeof(AttitudeController)); if(me!=空) { AttitudeController_Init(me); }
return me; } void AttitudeController_Destroy(AttitudeController* const me) {
if(me!=空) { AttitudeController_Cleanup(me); } free(me); } static void
cleanUpRelations(AttitudeController* const me) { if(me->itsKinematicData !=
空) { me->itsKinematicData    = 空; } }
```

代码清单4-13：AttitudeController.c

```
#ifndef Navigator_H
#define Navigator_H

#include "GuardedCallExample.h"
#include "Position.h"
struct KinematicData;

typedef struct Navigator Navigator;
struct Navigator {
    struct Position ownShipPosition;
    struct KinematicData*  itsKinematicData;
};

/*  Constructors and destructors:*/
void Navigator_Init(Navigator*  const me);
void Navigator_Cleanup(Navigator*  const me);
```

```
#ifndef Navigator_H #define Navigator_H #include "
GuardedCallExample.h" #include "Position.h" struct
KinematicData; typedef struct Navigator Navigator;
struct Navigator { struct Position ownShipPosition;
struct KinematicData* itsKinematicData; };

/* 构造函数与析构函数：*/ void Navigator_
Init(Navigator* const me); void Navigator_
Cleanup(Navigator* const me);
```

```
/*  Operations  */
void Navigator_updatePosition(Navigator*  const me);

struct KinematicData*  Navigator_getItsKinematicData(const Navigator*  const me);

void Navigator_setItsKinematicData(Navigator*  const me, struct
KinematicData*  p_KinematicData);

Navigator  *  Navigator_Create(void);

void Navigator_Destroy(Navigator*  const me);

#endif
```

Code Listing 4-14: Navigator.h

```
#include "Navigator.h"
#include "KinematicData.h"

static void cleanUpRelations(Navigator*  const me);

void Navigator_Init(Navigator*  const me) {
    Position_Init(&(me->ownShipPosition));
    me->itsKinematicData  =  NULL;
}

void Navigator_Cleanup(Navigator*  const me) {
    cleanUpRelations(me);
}

void Navigator_updatePosition(Navigator*  const me) {
    KinematicData_setPosition(me->itsKinematicData, me->ownShipPosition);
}

struct KinematicData*  Navigator_getItsKinematicData(const
Navigator*  const me) {
    return (struct KinematicData*)me->itsKinematicData;
}

void Navigator_setItsKinematicData(Navigator*  const me, struct
KinematicData*  p_KinematicData) {
    me->itsKinematicData  =  p_KinematicData;
}

Navigator  *  Navigator_Create(void) {
    Navigator*  me  =  (Navigator  *) malloc(sizeof(Navigator));
    if(me!=NULL) {
        Navigator_Init(me);
    }
```


/* 操作 */ void Navigator_updatePosition(Navigator* const me); struct KinematicData* Navigator_getItsKinematicData(const Navigator* const me); void Navigator_setItsKinematicData(Navigator* const me, struct KinematicData* p_KinematicData); Navigator * Navigator_Create(void); void Navigator_Destroy(Navigator* const me); #endif

代码清单4-14：Navigator.h

#include "Navigator.h" #include "KinematicData.h" static void cleanUpRelations(Navigator* const me); void Navigator_Init(Navigator* const me) { Position_Init(&(me->ownShipPosition)); me->itsKinematicData = 空; } void Navigator_Cleanup(Navigator* const me) { cleanUpRelations(me); } void Navigator_updatePosition(Navigator* const me) { KinematicData_setPosition(me->its KinematicData, me->ownShipPosition); } struct KinematicData* Navigator_getItsKinematicData(const Navigator* const me) { return (struct KinematicData*)me->itsKinematicData; } void Navigator_setItsKinematicData(Navigator* const me, struct KinematicData* p_KinematicData) { me->itsKinematicData = p_KinematicData; } Navigator * Navigator_Create(void) { Navigator* me = (Navigator *) malloc(sizeof(Navigator)); if(me!=空) { Navigator_Init(me); }

```
    return me;
}

void Navigator_Destroy(Navigator*  const me) {
    if(me!=NULL) {
        Navigator_Cleanup(me);
    }
    free(me);
}

static void cleanUpRelations(Navigator*  const me) {
    if(me->itsKinematicData !=  NULL) {
        me->itsKinematicData  =  NULL;
    }
}
```

Code Listing 4-15: Navigator.c

```
#ifndef FlightDataDisplay_H
#define FlightDataDisplay_H

#include "GuardedCallExample.h"
struct KinematicData;

typedef struct FlightDataDisplay FlightDataDisplay;
struct FlightDataDisplay {
    struct KinematicData*  itsKinematicData;
};

/*  Constructors and destructors:*/
void FlightDataDisplay_Init(FlightDataDisplay*  const me);
void FlightDataDisplay_Cleanup(FlightDataDisplay*  const me);

/*  Operations  */

void FlightDataDisplay_showFlightData(FlightDataDisplay*  const me);

struct KinematicData*  FlightDataDisplay_getItsKinematicData(const
FlightDataDisplay*  const me);

void FlightDataDisplay_setItsKinematicData(FlightDataDisplay*
const me, struct KinematicData*  p_KinematicData);

FlightDataDisplay  *  FlightDataDisplay_Create(void);

void FlightDataDisplay_Destroy(FlightDataDisplay*  const me);

#endif
```

Code Listing 4-16: FlightDataDisplay.h

return me; } void Navigator_Destroy(Navigator* const me) { if(me!=空) { Navigator_Cleanup(me); } free(me); } static void cleanUpRelations(Navigator* const me) { if(me->itsKinematicData != 空) { me->itsKinematicData = 空; } }

代码清单4-15：Navigator.c

#ifndef FlightDataDisplay_H #define FlightDataDisplay_H #include " GuardedCallExample.h" struct KinematicData; typedef struct FlightDataDisplay FlightDataDisplay; struct FlightDataDisplay { struct KinematicData* itsKinematicData; }; /* 构造函数与析构函数： */ void FlightDataDisplay_ Init(FlightDataDisplay* const me); void FlightDataDisplay_ Cleanup(FlightDataDisplay* const me); /* 操作 */ void FlightDataDisplay_ showFlightData(FlightDataDisplay* const me); struct KinematicData* FlightDataDisplay_getItsKinematicData(const FlightDataDisplay* const me); void FlightDataDisplay_setItsKinematicData(FlightDataDisplay* const me, struct KinematicData* p_KinematicData); FlightDataDisplay * FlightDataDisplay_ Create(void); void FlightDataDisplay_Destroy(FlightDataDisplay* const me); #e ndif

代码清单4-16：FlightDataDisplay.h

```
#include "FlightDataDisplay.h"
#include "KinematicData.h"
#include <stdio.h>

static void cleanUpRelations(FlightDataDisplay*  const me);

void FlightDataDisplay_Init(FlightDataDisplay*  const me){
    me->itsKinematicData  =  NULL;
}

void FlightDataDisplay_Cleanup(FlightDataDisplay*  const me){
    cleanUpRelations(me);
}

void FlightDataDisplay_showFlightData(FlightDataDisplay*  const me){
    Attitude a;
    Position p;
    a  =  KinematicData_getAttitude(me->itsKinematicData);
    p  =  KinematicData_getPosition(me->itsKinematicData);
    printf("Roll, pitch, yaw  =  %f, %f, %f  \n", a.roll, a.pitch, a.yaw);
    printf("Lat, Long, Altitude  =  %f, %f, %f\n", p.latitude, p.longitude, p.altitude);
}

struct KinematicData*  FlightDataDisplay_getItsKinematicData(const FlightDataDis-
play*  const me){
    return (struct KinematicData*)me->itsKinematicData;
}

void FlightDataDisplay_setItsKinematicData(FlightDataDisplay*  const me, struct Kine-
maticData*  p_KinematicData){
    me->itsKinematicData  =  p_KinematicData;
}

FlightDataDisplay  *  FlightDataDisplay_Create(void){
    FlightDataDisplay*  me  =  (FlightDataDisplay  *) malloc(sizeof
(FlightDataDisplay));
    if(me!=NULL){
    FlightDataDisplay_Init(me);
    }
    return me;
}

void FlightDataDisplay_Destroy(FlightDataDisplay*  const me){
    if(me!=NULL){
    FlightDataDisplay_Cleanup(me);
    }
    free(me);
}
```


#include "FlightDataDisplay.h" #include "KinematicData.h" #include <stdio.h>static void cleanUpRelations(FlightDataDisplay* const me); void FlightDataDisplay_Init(FlightDataDisplay* const me) { me->itsKinematicData = NULL; } void FlightDataDisplay_Cleanup(FlightDataDisplay* const me) { cleanUpRelations(me); } void FlightDataDisplay_showFlightData(FlightDataDisplay* const me) { Attitude a; Position p; a = KinematicData_getAttitude(me->itsKinematicData); p = KinematicData_getPosition(me->itsKinematicData); printf("横滚角、俯仰、偏航 = %f, %f, %f \n", a.roll, a.pitch, a.yaw); printf("纬度、经度、高度 = %f, %f, %f\n", p.latitude, p.longitude, p.altitude); } struct KinematicData* FlightDataDisplay_getItsKinematicData(const FlightDataDis- play* const me) { return (struct KinematicData*)me->itsKinematicData; } void FlightDataDisplay_setItsKinematicData(FlightDataDisplay* const me, struct Kine- maticData* p_KinematicData) { me->itsKinematicData = p_KinematicData; } FlightDataDisplay * FlightDataDisplay_Create(void) { FlightDataDisplay* me = (FlightDataDisplay *) malloc(sizeof (FlightDataDisplay)); if(me!=NULL) { FlightDataDisplay_Init(me); } return me; } void FlightDataDisplay_Destroy(FlightDataDisplay* const me) { if(me!=NULL) { FlightDataDisplay_Cleanup(me); } free(me); }

```
static void cleanUpRelations(FlightDataDisplay*  const me) {
    if(me->itsKinematicData !=  NULL) {
    me->itsKinematicData  =  NULL;
    }
}
```

Code Listing 4-30: FlightDataDisplay.c

4.6 Queuing Pattern

The Queuing Pattern is the most common implementation of task asynchronous communication. It provides a simple means of communication between tasks that are uncoupled in time. The Queuing Pattern accomplishes this communication by storing messages in a queue, a nominally first-in-first-out data structure. The sender deposits a message in the queue and some time later, the receiver withdraws the message from the queue. It also provides a simple means of serializing access to a shared resource. The access messages are queued and handled at a later time and this avoids the mutual exclusion problem common to sharing resources.

4.6.1 Abstract

The Message Queuing Pattern uses asynchronous communications, implemented via queued messages, to synchronize and share information among tasks. This approach has the advantage of simplicity and does not experience mutual exclusion problems because no resource is shared by reference. Any information shared among threads is *passed by value* to the separate thread. While this limits the complexity of the collaboration among the threads, this approach in its pure form is immune to the standard resource corruption problems that plague concurrent systems that share information *passed by reference*. In passed by value sharing, a copy of the information is made and sent to the receiving thread for processing. The receiving thread fully owns the data it receives and so may modify them freely without concern for corrupting data due to multiple writers or due to sharing them among a writer and multiple readers. A downside of the pattern is that the message isn't processed immediately after the sender posts it; the process waits until the receiver task runs and can process the waiting messages.

4.6.2 Problem

In multithreaded systems, tasks must synchronize and share information with others. Two primary things must be accomplished. First, the tasks must synchronize to permit sharing of the information. Secondly, the information must be shared in such a way that there is no chance of corruption or race conditions. This pattern addresses such task interaction.


```
static void cleanUpRelations(FlightDataDisplay* const me) { if(me->
itsKinematicData !=  空) { me->itsKinematicData  = 空; } }
```

代码清单 4-30：FlightDataDisplay.c

4.6 排队模式

排队模式是任务异步通信最常见的实现方式。它提供了一种在时间上解耦的任务之间进行通信的简单方法。排队模式通过将消息存储在队列（一种名义上的先进先出数据结构）中来实现这种通信。发送者将消息存入队列，一段时间后，接收者从队列中取出该消息。它还提供了一种对共享资源进行串行化访问的简单方法。访问消息被排队并在稍后处理，这避免了共享资源中常见的互斥问题。

4.6.1 摘要

消息排队模式使用异步通信，通过排队消息实现，以在任务之间同步和共享信息。这种方法具有简单性的优点，并且不会出现互斥问题，因为没有任何资源是按引用共享的。线程之间共享的任何信息都按值传递给单独的线程。虽然这限制了线程之间协作的复杂性，但这种方法的纯粹形式不受困扰并发系统的标准资源损坏问题的影响，这些并发系统共享按引用传递的信息。在按值传递共享中，信息的一个副本被创建并发送给接收线程进行处理。接收线程完全拥有它接收的数据，因此可以自由地修改它们，而不必担心由于多个写入者或由于在写入者和多个读取者之间共享数据而导致的数据损坏。该模式的一个缺点是，消息在发送者发布后不会立即被处理；进程会等待，直到接收者任务运行并能够处理等待中的消息。

4.6.2 问题

在多线程系统中，任务必须与其他任务同步并共享信息。必须完成两件主要的事情。首先，任务必须同步以允许信息共享。其次，信息的共享方式必须确保不存在损坏或竞态条件的可能。此模式解决了此类任务交互问题。

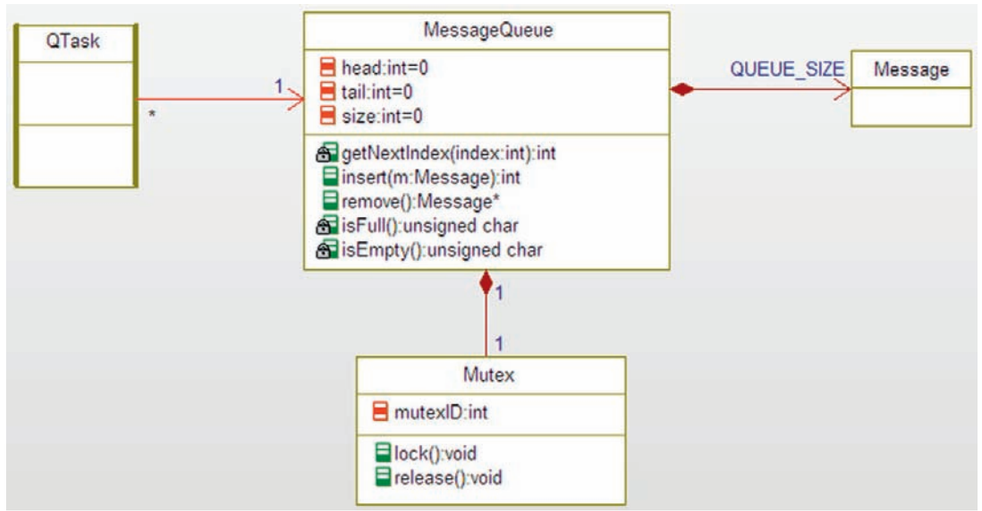


Figure 4-17: Queuing Pattern

4.6.3 Pattern Structure

The Queuing Pattern structure is shown in Figure 4-17. The `QUEUE_SIZE` declaration determines the maximum number of elements that the queue can retain. Care must be taken to ensure that this is large enough to handle the worst-case situation for your system usage, while not being so large as to waste memory.

4.6.4 Collaboration Roles

This section describes the roles for this pattern.

4.6.4.1 Message

The `Message` class is an abstraction of a message of relevance between `QTasks`. It can be anything from a simple data value to an elaborate datagram structure such as used in TCP/IP messaging.

4.6.4.2 MessageQueue

The `MessageQueue` is the storage of information being exchanged between `QTasks`. The `MessageQueue` can store up to a maximum of `QUEUE_SIZE` message elements. The `MessageQueue` provides the following methods:

- `int MessageQueue_getNextIndex(MessageQueue* me, int index)`
  A private (static) function that uses modulo arithmetic to compute the next valid index value.

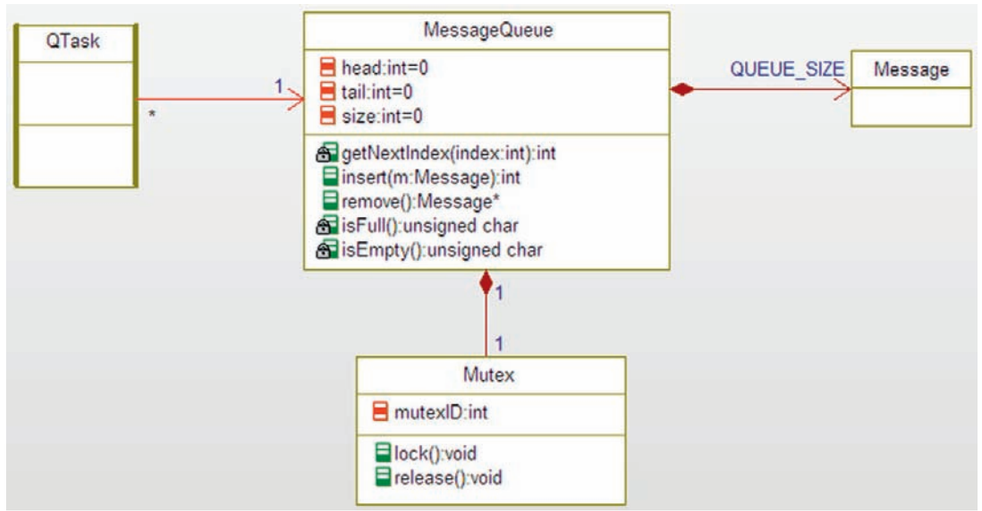


图 4-17：排队模式

4.6.3 模式结构

队列 模式 的 结构 如 图 4-17 所示 。 `QUEUE_SIZE` 声明决定了队列可保留的最大元素数量。必须注意确保该值足够大，以处理系统使用中的最坏情况，同时又不至于大到浪费内存。

4.6.4 协作角色

本节描述此模式的角色。

4.6.4.1 消息

`Message` 类是对`QTasks`之间相关消息的抽象。它可以是简单的数据值，也可以是复杂的数据报结构，例如TCP/IP消息传递中使用的数据报结构。

4.6.4.2 MessageQueue

`MessageQueue` 是`QTasks`之间交换信息的存储。 `MessageQueue` 最多可存储 `QUEUE_SIZE` 个消息元素。`MessageQueue` 提供以下方法：

- `int MessageQueue_getNextIndex(MessageQueue* me, int index)`一个私有（静态）函数，使用模运算计算下一个有效索引值。

- `int MessageQueue_insert(MessageQueue* me, Message m)`
  A public function that, if the `MessageQueue` isn't full, inserts a `Message` into the queue at the head position and updates the `head` index. It returns 0 if the insert succeeds and 1 if the `MessageQueue` was full.
- `Message* MessageQueue_remove(MessageQueue* me)`
  A public function that, if the `MessageQueue` isn't empty, removes the oldest message, creates a new heap variable to hold the data, updates the `tail` index, and returns that pointer to the caller. It returns a pointer to a `Message` if successful and a `NULL` pointer if not.
- `unsigned char MessageQueue_isFull(MessageQueue* me)`
  A private function that returns 1 if the `MessageQueue` is full and 0 otherwise.
- `unsigned char MessageQueue_isEmpty(MessageQueue* me)`
  A private function that returns 1 if the `MessageQueue` is empty and 1 otherwise.

4.6.4.3 Mutex

The `Mutex` is a mutual exclusion semaphore object that serializes access to a `MessageQueue`. The guarded functions of the `MessageQueue` invoke the `lock()` function of the `Mutex` whenever they are called and `release()` once the service is complete. Other client threads that attempt to invoke a service when its associated `Mutex` is locked become blocked until the `Mutex` is unlocked. This element is usually provided by the RTOS. Note this is the same element that appears in the Static Priority Pattern earlier in this chapter.

4.6.4.4 Qtask

The `QTask` class is a client of the `MessageQueue` and invokes either the `insert()` or `remove()` functions to access the data held there.

4.6.5 Consequences

The Queuing Pattern provides good serialization of access to the data as they are passed among tasks. The use of the mutex ensures that the MessageQueue itself isn't corrupted by simultaneous access to the MessageQueue's services. The reception of the data is less timely than the Guarded Call Pattern since the Queuing Pattern implements asynchronous communication.

The number of elements held by the queue can be quite large; this is useful as the products of the elements run in a "bursty" fashion, allowing for the consumer of the elements to catch up later in time. Poorly sized queues can lead to data loss (if too small) or wasted memory (if too large). Very small queues (that can hold one or two elements) can be used for simple asynchronous data exchange when the problem of data loss isn't present.

- `int MessageQueue_insert(MessageQueue* me, Message m)`一个公共函数，如果`MessageQueue` 未满，则将`Message` 插入到队列的head位置，并更新`head` 索引。如果插入成功则返回0，如果`MessageQueue` 已满则返回1。

- `Message* MessageQueue_remove(MessageQueue* me)`一个公共函数，如果`MessageQueue` 不为空，则移除最旧的消息，创建一个新的堆变量来保存数据，更新尾部 索引，并将该指针返回给调用者。成功时返回指向`Message` 的指针，失败时返回空 指针。

- unsigned char MessageQueue_isFull(MessageQueue* me) 一个私有函数，如果MessageQueue 已满则返回1，否则返回0。
- unsigned char MessageQueue_isEmpty(MessageQueue* me) 一个私有函数，如果MessageQueue 为空则返回1，否则返回1。

4.6.4.3 互斥锁

互斥锁 是一个互斥信号量对象，用于串行化对`MessageQueue`的访问。`MessageQueue` 的受守卫函数在每次被调用时都会调用互斥锁的`lock()` 函数，并在服务完成后调用`release()`。其他客户端线程在尝试调用服务时，如果其关联的互斥锁已加锁，则会变为阻塞状态，直到互斥锁 被解锁。此元素通常由实时操作系统提供。注意，这与本章前面静态优先级模式中出现的元素相同。

4.6.4.4 Qtask

`QTask` 类是`MessageQueue` 的客户端，并调用`insert()` 或`remove()` 函数来访问其中保存的数据。

4.6.5 结果

排队模式在任务之间传递数据时提供了良好的访问串行化。互斥锁的使用确保MessageQueue本身不会因同时访问MessageQueue的服务而损坏。由于排队模式实现的是异步通信，因此数据的接收及时性不如受保护调用模式。

队列所容纳的元素数量可能相当大；这很有用，因为元素的生产以"突发"方式进行，允许元素的消费者在稍后时间赶上。队列大小不当可能导致数据丢失（如果太小）或浪费内存（如果太大）。非常小的队列（只能容纳一两个元素）可用于简单的异步数据交换，前提是不存在数据丢失的问题。

4.6.6 Implementation Strategies

The simplest implementation of a queue is as an array of message elements. This has the advantage of simplicity but lacks the flexibility of a linked list.

Queues are fairly simple to implement but there are many possible variants. Sometimes some messages are more urgent or important than others and should be processed before waiting for lower priority messages. The MessageQueue can be modified to implement priority by adding either multiple buffers (one for each priority) or by inserting the element in the queue on the basis of the message priority (although this can increase the timing necessary for an element insertion and deletion). In this case, the message should have a priority attribute to enable the MessageQueue to properly insert and remove the element.

In complex systems, it may not be feasible to predict an optimal queue size. In such cases, extensible queues can be implemented. When the MessageQueue is full, more memory is allocated. A linked list implementation of the MessageQueue is more appropriate for an extensible queue. It may be difficult to determine under what circumstances it is appropriate to shrink the queue and release unused memory.

It sometimes happens that the potential number of elements exceeds memory capacity. In this case, a cached queue can be created; in such a queue, there are essentially three storage locations; a local memory newest data buffer, a local memory oldest data buffer, and a file that holds data of intermediate age. When the local new data buffer becomes full, it is emptied to the file system. When the local old data buffer becomes empty, the data from the file system are loaded into the local old data buffer and removed from the disk (or copied from the new data buffer if the file is empty). This allows storage of potentially huge volumes of data, but caching the data can be time intensive.

4.6.7 Related Patterns

The Queuing Pattern serializes access to data by queuing data or commands, allowing the recipient to handle them one at a time. It does this asynchronously, so the time between the sending of the message and its processing is uncoupled. This may not meet the performance needs of the system. The Guarded Call Pattern also serializes access but does so synchronously, so that data and command transfers generally happen closer together in time. However, the Guarded Call Pattern can lead to unbounded priority inversion if not used carefully. In addition, if the receiver isn't ready to receive the message from the sender, the sender must block or take another action.

4.6.8 Example

Figure 4-18 shows an example of producers and consumers that share data via a queue. The key element for this pattern is the `GasDataQueue`. It implements a small `10`-element queue of `GasData`. `GasData` is a struct that contains an enumerated GAS_TYPE (with enumerated values

4.6.6 实现策略

队列的最简单实现是作为消息元素的数组。这具有简单性的优点，但缺乏链表的灵活性。

队列的实现相当简单，但存在许多可能的变体。有时某些消息比其他消息更紧急或更重要，应当在等待较低优先级消息之前被处理。可以通过添加多个缓冲区（每个优先级一个）或根据消息优先级将元素插入队列来修改 MessageQueue 以实现优先级（尽管这可能会增加元素插入和删除所需的时序）。在这种情况下，消息应具有优先级属性，以使 MessageQueue 能够正确地插入和移除元素。

在复杂系统中，预测最优队列大小可能不可行。在这种情况下，可以实现可扩展队列。当 MessageQueue 已满时，分配更多内存。MessageQueue 的链表实现更适合可扩展队列。可能难以确定在何种情况下适合收缩队列并释放未使用的内存。

有时会出现潜在元素数量超过内存容量的情况。在这种情况下，可以创建缓存队列；在这种队列中，本质上存在三个存储位置：本地内存最新数据缓冲区、本地内存最旧数据缓冲区，以及一个保存中间 age 数据的文件。当本地新数据缓冲区变满时，将其清空到文件系统。当本地旧数据缓冲区变空时，从文件系统将数据加载到本地旧数据缓冲区并从磁盘中移除（如果文件为空，则从新数据缓冲区复制）。这允许存储潜在海量的数据，但缓存数据可能非常耗时。

4.6.7 相关模式

排队模式通过排队数据或命令来串行化对数据的访问，使接收者能够一次处理一个。它以异步方式执行此操作，因此消息发送与其处理之间的时间被解耦。这可能无法满足系统的性能需求。受保护调用模式也串行化访问，但以同步方式执行，因此数据和命令的传输通常在时间上更接近。然而，如果不小心使用，受保护调用模式可能导致无界优先级反转。此外，如果接收者尚未准备好接收来自发送者的消息，发送者必须阻塞或采取其他动作。

4.6.8 示例

图 4-18 展示了生产者与消费者通过队列共享数据的示例。该模式的关键元素是 `GasDataQueue`。它实现了一个包含 `10` 个 `GasData` 元素的小型队列。`GasData` 是一个结构体，包含一个枚举类型的 GAS_TYPE（其枚举值为

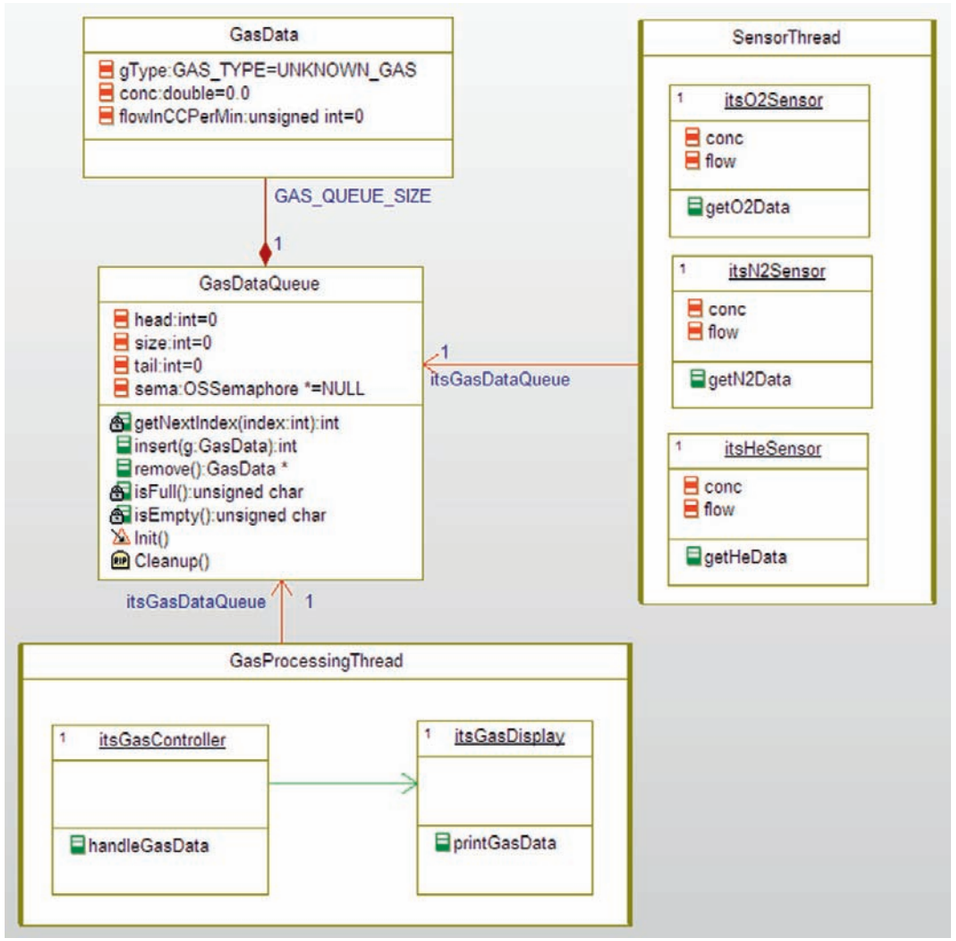


Figure 4-18: Queuing Example

O2_GAS, N2_GAS, HE_GAS, and UNKNOWN_GAS), the gas concentration (a double), and the flow rate in ccs per `minute (int)`. Code Listing 4-31 gives the `#defines` and general declarations. Code Listing 4-32 and Code Listing 4-33 give the code for the `GasData` struct.

```
#ifndef QueuingExample_H
#define QueuingExample_H

struct GasController;
struct GasData;
struct GasDataQueue;
struct GasDisplay;
```

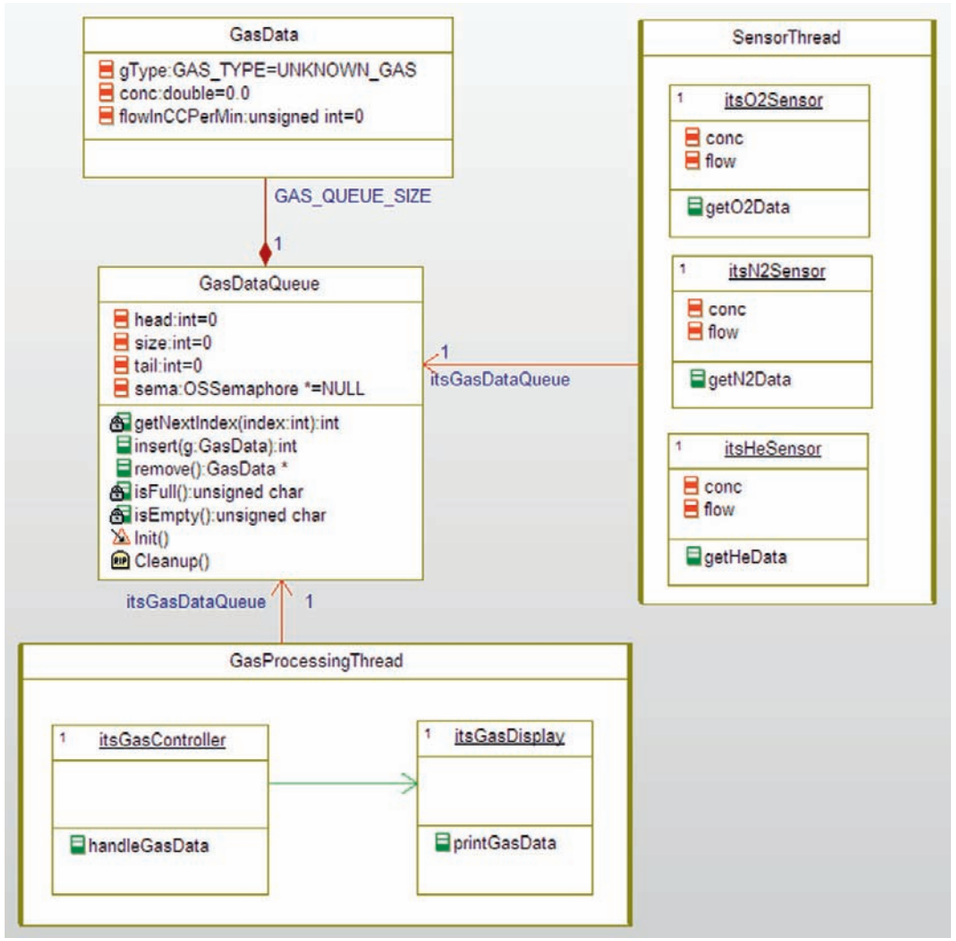


图 4-18：排队示例

O2_GAS、N2_GAS、HE_GAS 和 UNKNOWN_GAS）、气体浓度（double 类型）以及流速（单位为 cc/分钟，int 类型）。代码清单 4-31 给出了 `#defines` 和通用声明。代码清单 4-32 和代码清单 4-33 给出了 `GasData` 结构体的代码。

```
#ifndef QueuingExample_H #
define QueuingExample_H

struct GasController;
struct GasData; struct
GasDataQueue; struct
GasDisplay;
```

```
struct GasProcessingThread;
struct HeSensor;
struct N2Sensor;
struct O2Sensor;
struct OSSemaphore;
struct SensorThread;

typedef enum GAS_TYPE {
    O2_GAS,
    N2_GAS,
    HE_GAS,
    UNKNOWN_GAS
} GAS_TYPE;

/*  define the size of the queue  */
#define GAS_QUEUE_SIZE (10)

/*  OS semaphore services  */
struct OSSemaphore*  OS_create_semaphore(void);
void OS_destroy_semaphore(struct OSSemaphore*  sPtr);
void OS_lock_semaphore(struct OSSemaphore*  sPtr);
void OS_release_semaphore(struct OSSemaphore*  sPtr);

#endif
```

Code Listing 4-17: GasDataExample.h

```
#ifndef GasData_H
#define GasData_H

#include "QueuingExample.h"

typedef struct GasData GasData;
struct GasData {
    double conc;
    unsigned int flowInCCPerMin;
    GAS_TYPE gType;
};

/*  Constructors and destructors:*/

void GasData_Init(GasData*  const me);
void GasData_Cleanup(GasData*  const me);
GasData  *  GasData_Create(void);
void GasData_Destroy(GasData*  const me);

#endif
```

Code Listing 4-32: GasData.h


```
struct GasProcessingThread;
struct HeSensor; struct
N2Sensor; struct O2Sensor;
struct OSSemaphore; struct
SensorThread;


typedef enum GAS_TYPE
{ O2_GAS, N2_GAS, HE_
GAS, UNKNOWN GAS_}
GAS_TYPE;



/* 定义队列的 size */ #define GAS_QUEUE_
SIZE (10)

/* 操作系统信号量服务 */ struct OSSemaphore* OS_create_
semaphore(void); void OS_destroy_semaphore(struct
OSSemaphore* sPtr); void OS_lock_semaphore(struct
OSSemaphore* sPtr); void OS_release_semaphore(struct
OSSemaphore* sPtr);

#endif
```

代码清单4-17：GasDataExample.h

```
#ifndef GasData_H #
define GasData_H

#include "QueuingExample.h"

typedef struct GasData GasData; struct
GasData { double conc; unsigned int
flowInCCPerMin; GAS_TYPE gType; }; /*
构造函数与析构函数：*/



void GasData_Init(GasData* const me); void
GasData_Cleanup(GasData* const me); GasData *
GasData_Create(void); void GasData_
Destroy(GasData* const me);

#endif
```

代码清单 4-32：GasData.h

```
#include "GasData.h"

void GasData_Init(GasData*  const me) {
    me->conc  =  0.0;
    me->flowInCCPerMin  =  0;
    me->gType  =  UNKNOWN_GAS;
}

void GasData_Cleanup(GasData*  const me) {
}

GasData  *  GasData_Create(void) {
    GasData*  me  =  (GasData  *) malloc(sizeof(GasData));
    if(me!=NULL) {
        GasData_Init(me);
    }
    return me;
}

void GasData_Destroy(GasData*  const me) {
    if(me!=NULL) {
        GasData_Cleanup(me);
    }
    free(me);
}
```

Code Listing 4-33: GasData.c

The `GasDataQueue` is the center of the pattern. Its two operations insert() and remove() enqueue new data and dequeue oldest data, respectively. The insert() function returns `TRUE` if it succeeded and `FALSE` otherwise. The remove() function returns a pointer to a `GasData` instance for which the client is responsible for deleting. If the remove() function fails, it returns a `NULL` pointer.

The class also includes some private (static) helper functions such as `getNextIndex()` which computes the next index using modulo arithmetic to wrap around the top of the queue, `isEmpty()`, which returns `TRUE` if the queue has no elements, and `isFull()`, which returns `TRUE` if there is no space to hold additional elements.

Code Listing 4-34 and Code Listing 4-35 give the C code for the `GasDataQueue`.

```
#ifndef GasDataQueue_H
#define GasDataQueue_H

#include "QueuingExample.h"
#include "GasData.h"
```


#include "GasData.h" void GasData_Init(GasData* const me) { me-> conc = 0.0; me->flowInCCPerMin = 0; me->gType = UNKNOWN_GAS; } void GasData_Cleanup(GasData* const me) { } GasData * GasData_Create(void) { GasData* me = (GasData *) malloc(sizeof(GasData)); if(me!=NULL) { GasData_Init(me); } return me; } void GasData_Destroy(GasData* const me) { if(me!=NULL) { GasData_Cleanup(me); } free(me); }

代码清单 4-33：GasData.c

GasDataQueue 是该模式的核心。它的两个操作 insert() 和 remove() 分别将新数据入队和最旧数据出队。insert() 函数成功时返回 TRUE，否则返回 FALSE。remove() 函数返回一个指向 GasData 实例的指针，客户端负责删除该实例。如果 remove() 函数失败，则返回 NULL 指针。

该类还包含一些私有（静态）辅助函数，例如 getNextIndex()，它使用模运算计算下一个索引以在队列顶部回绕；isEmpty()，如果队列中没有元素则返回 TRUE；以及 isFull()，如果没有空间容纳更多元素则返回TRUE。

代码清单 4-34 和 代码 清单 4-35 给出 了 GasDataQueue 的 C 语言 代码 .

#ifndef GasDataQueue_H # define GasDataQueue_H

#include "QueuingExample.h" #include "GasData.h"

```
#include "OSSemaphore.h"

typedef struct GasDataQueue GasDataQueue;
struct GasDataQueue {
    int head;
    OSSemaphore  *  sema;
    int size;
    int tail;
    struct GasData itsGasData[GAS_QUEUE_SIZE];
};

/*  Constructors and destructors:*/

void GasDataQueue_Init(GasDataQueue*  const me);
void GasDataQueue_Cleanup(GasDataQueue*  const me);

/*  Operations  */
int GasDataQueue_insert(GasDataQueue*  const me, GasData g);

GasData  *  GasDataQueue_remove(GasDataQueue*  const me);

int GasDataQueue_getItsGasData(const GasDataQueue*  const me);

GasDataQueue  *  GasDataQueue_Create(void);

void GasDataQueue_Destroy(GasDataQueue*  const me);

#endif
```

Code Listing 4-34: GasDataQueue.h

```
#include "GasDataQueue.h"
#include <stdio.h>

/*  private (static) methods  */
static void cleanUpRelations(GasDataQueue*  const me);
static int getNextIndex(GasDataQueue*  const me, int index);
static unsigned char isEmpty(GasDataQueue*  const me);
static unsigned char isFull(GasDataQueue*  const me);
static void initRelations(GasDataQueue*  const me);

void GasDataQueue_Init(GasDataQueue*  const me) {
    me->head  =  0;
    me->sema  =  NULL;
    me->size  =  0;
    me->tail  =  0;
    initRelations(me);
```



```
#include "OSSemaphore.h" typedef struct
GasDataQueue GasDataQueue; struct GasDataQueue {
int head; OSSemaphore * sema; int size; int tail; struct
GasData itsGasData[GAS_QUEUE_SIZE]; }; /* 构造函数与
析构函数：*/

void GasDataQueue_Init(GasDataQueue* const me); void
GasDataQueue_Cleanup(GasDataQueue* const me);

/* 操作 */ int GasDataQueue_insert(GasDataQueue* const me, GasData
g);

GasData * GasDataQueue_移除(GasDataQueue* const me);

int GasDataQueue_getItsGasData(const GasDataQueue* const me);

GasDataQueue * GasDataQueue_Create(void);

void GasDataQueue_Destroy(GasDataQueue* const me);

#endif
```

代码清单 4-34：GasDataQueue.h

```
#include "GasDataQueue.h" #
include <stdio.h>

/*  private (static) methods  */
static void cleanUpRelations(GasDataQueue* const me); static int 获取下一个索
引(GasDataQueue* const me, int 索引); static 无符号字符
isEmpty(GasDataQueue* const me); static 无符号字符 isFull(GasDataQueue*
const me); static void initRelations(GasDataQueue* const me);


void GasDataQueue_Init(GasDataQueue* const me) { me->
head  =  0; me->信号量  =  空; me->size  =  0; me->尾部
  =  0; initRelations(me);
```

```
    me->sema = OS_create_semaphore();
}

void GasDataQueue_Cleanup(GasDataQueue* const me) {
    OS_destroy_semaphore(me->sema);
    cleanUpRelations(me);
}

/*
    Insert puts new gas data elements into the queue
    if possible. It returns 1 if successful, 0 otherwise.
*/
int GasDataQueue_insert(GasDataQueue* const me, GasData g) {
    OS_lock_semaphore(me->sema);
    if (!isFull(me)) {
        me->itsGasData[me->head] = g;
        me->head = getNextIndex(me, me->head);
        ++me->size;

        /* instrumentation */
        /* print stuff out, just to visualize the insertions */
        switch (g.gType) {
        case O2_GAS:
            printf("+++   Oxygen ");
            break;
        case N2_GAS:
            printf("+++   Nitrogen ");
            break;
        case HE_GAS:
            printf("+++   Helium ");
            break;
        default:
               printf("UNKNWON ");
               break;
        };
        printf("  at conc %f, flow %d\n",g.conc,g.flowInCCPerMin);
        printf("  Number of elements queued %d, head = %d, tail = %d\n",
            me->size, me->head, me->tail);
        /* end instrumentation */

            OS_release_semaphore(me->sema);
        return 1;
    }
    else {  /* return error indication */
        OS_release_semaphore(me->sema);
        return 0;
        }
}
/*
        remove creates a  new element on the heap, copies
```


me->sema = OS_create_semaphore(); } void GasDataQueue_Cleanup(GasDataQueue* const me) { OS_destroy_semaphore(me->sema); cleanUpRelations(me); } /* 插入操作在可能的情况下将新的气体数据元素放入队列。成功时返回1，否则返回0。 */ int GasDataQueue_insert(GasDataQueue* const me, GasData g) { OS_lock_semaphore(me->sema); if (!isFull(me)) { me->itsGasData[me->head] = g; me->head = getNextIndex(me, me->head);++me->size; /* 插桩 */ /* 打印内容，仅用于可视化插入操作 */ switch (g.gType) { case O2 GAS:_printf("+++ Oxygen "); break; case N2 GAS:_printf("+++ Nitrogen "); break; case HE GAS:_printf("+++ Helium "); break; default: printf("UNKNWON "); break; }; printf(" at conc %f, flow %d\n", g.conc,g.flowInCCPerMin); printf(" Number of elements queued %d, head = %d, tail = %d\n", me->size, me->head, me->tail); /* 插桩结束 */ OS_release_semaphore(me->sema); return 1; } else { /* 返回错误指示 */ OS_release_semaphore(me->sema); return 0; } } /* remove 在堆上创建一个新元素，复制

```
        the contents of the oldest data into it, and
        returns the pointer. Returns NULL if the queue
        is empty
    */
GasData * GasDataQueue_remove(GasDataQueue* const me) {
    GasData* gPtr;

    OS_lock_semaphore(me->sema);
    if (!isEmpty(me)) {
        gPtr = (GasData*)malloc(sizeof(GasData));
        gPtr->gType = me->itsGasData[me->tail].gType;
        gPtr->conc = me->itsGasData[me->tail].conc;
        gPtr->flowInCCPerMin = me->itsGasData[me->tail].flowInCCPerMin;
        me->tail = getNextIndex(me, me->tail);
        -me->size;
        /* instrumentation */
        switch (gPtr->gType) {
            case O2_GAS:
                printf("- Oxygen ");
                break;
            case N2_GAS:
                printf("- Nitrogen ");
                break;
            case HE_GAS:
                printf("- Helium ");
                break;
            default:
                   printf("- UNKNWON ");
                   break;
        };
        printf(" at conc %f, flow %d\n",gPtr->conc,gPtr->flowInCCPerMin);
        printf(" Number of elements queued %d, head = %d, tail = %d\n",
            me->size, me->head, me->tail);
        /* end instrumentation */

        OS_release_semaphore(me->sema);
        return gPtr;
        }
    else {  /* if empty return NULL ptr */
        OS_release_semaphore(me->sema);
        return NULL;
        }
}

static int getNextIndex(GasDataQueue* const me, int index) {
        /* this operation computes the next index from the
        first using modulo arithmetic
        */
    return (index+1) % QUEUE_SIZE;
}
```


将最旧的数据内容放入其中，并返回指针。如果队列为空则返回 NULL */ GasData * GasDataQueue_remove(GasDataQueue* const me) { GasData* gPtr; OS_lock_semaphore(me->sema); if (!isEmpty(me)) { gPtr = (GasData*)malloc(sizeof(GasData)); gPtr->gType = me->itsGasData[me->tail].gType; gPtr->conc = me->itsGasData[me->tail].conc; gPtr->flowInCCPerMin = me->itsGasData[me->tail].flowInCCPerMin; me->tail = getNextIndex(me, me->tail); –me->size; /* 插桩 */ switch (gPtr->gType) { case O2 GAS:_printf("— 氧气 "); break; case N2_GAS: printf("— 氮气 "); break; case HE GAS:_printf("— 氦气 "); break; default: printf("— 未知 "); break; }; printf(" 浓度 %f, 流量 %d\n", gPtr->conc,gPtr->flowInCCPerMin); printf(" 队列中元素数量 %d, head = %d, tail = %d\n", me->size, me->head, me->tail); /* 插桩结束 */ OS_release_semaphore(me->sema); return gPtr; } else { /* 如果为空则返回 NULL 指针 */ OS_release_semaphore(me->sema); return NULL; } } static int getNextIndex(GasDataQueue* const me, int index) { /* 此操作使用模运算从第一个索引计算下一个索引 */ return (index+1) % QUEUE_SIZE; }

```
static unsigned char isEmpty(GasDataQueue*  const me) {
    return (me->size  ==    0);
}

static unsigned char isFull(GasDataQueue*  const me) {
    return (me->size  ==    GAS_QUEUE_SIZE);
}

int GasDataQueue_getItsGasData(const GasDataQueue*  const me) {
    int iter  =  0;
    return iter;
}

GasDataQueue  *  GasDataQueue_Create(void) {
    GasDataQueue*  me  =  (GasDataQueue  *)
malloc(sizeof(GasDataQueue));
    if(me!=NULL) {
       GasDataQueue_Init(me);
    }
    return me;
}

void GasDataQueue_Destroy(GasDataQueue*  const me) {
    if(me!=NULL) {
         GasDataQueue_Cleanup(me);
    }
    free(me);
}

static void initRelations(GasDataQueue*  const me) {
    int iter  =  0;
    while (iter  <  GAS_QUEUE_SIZE){
          GasData_Init(&((me->itsGasData)[iter]));
          iter++;
    }
}

static void cleanUpRelations(GasDataQueue*  const me) {
    int iter  =  0;
    while (iter  <  GAS_QUEUE_SIZE){
          GasData_Cleanup(&((me->itsGasData)[iter]));
          iter++;
    }
}
```

Code Listing 4-35: GasDataQueue.c

In the example (Figure 4-18) two threads run clients. The `SensorThread` owns three clients itself, an `O2Sensor`, `N2Sensor`, and `HeSensor`. When this thread runs the main function of

static unsigned char isEmpty(GasDataQueue* const me) { return (me->size == 0); } static unsigned char isFull(GasDataQueue* const me) { return (me->size == GAS_QUEUE_SIZE); } int GasDataQueue_getItsGasData(const GasDataQueue* const me) { int iter = 0; return iter; } GasDataQueue * GasDataQueue_Create(void) { GasDataQueue* me = (GasDataQueue *) malloc(sizeof(GasDataQueue)); if(me!=空) { GasDataQueue_Init(me); } return me; } void GasDataQueue_Destroy(GasDataQueue* const me) { if(me!=空) { GasDataQueue_Cleanup(me); } free(me); } static void initRelations(GasDataQueue* const me) { int iter = 0; while (iter < GAS_QUEUE_SIZE){ GasData_Init(&((me->itsGasData)[iter])); iter++; } } static void cleanUpRelations(GasDataQueue* const me) { int iter = 0; while (iter < GAS_QUEUE_SIZE){ GasData_Cleanup(&((me->itsGasData)[iter])); iter++;} }

代码清单 4-35：GasDataQueue.c

在 示例 中 （图 4-18），两个线程运行客户端。`SensorThread` 自身拥有三个客户端：一个`O2Sensor`、`N2Sensor`和`HeSensor`。当该线程运行

this thread (SensorThread_updateData()), it calls the getXXData() function of each to produce the data, and then marshalls the data up into GasData structs and sends them off. The relevant code (less the task initiation and supporting functions) for the SensorThread is given in Code Listing 4-36 and Code Listing 4-37.

```
/*  ...    initial declaratons stuff above  ...    */

typedef struct SensorThread SensorThread;
struct SensorThread {
    struct GasDataQueue*  itsGasDataQueue;
    struct HeSensor itsHeSensor;
    struct N2Sensor itsN2Sensor;
    struct O2Sensor itsO2Sensor;
};

/*  Operations  */

void SensorThread_updateData(SensorThread*  const me);

/*  ...    other operations declared too  ...    */
#endif
```

Code Listing 4-36: SensorThread.h

```
/*
    updateData runs every period of the sensor thread
    and calls the getXXData() function of each of
    its sensors, then uses a  random number generator
    to decide which data should be published to the
    GasDataQueue.
*/
void SensorThread_updateData(SensorThread*  const me) {
    unsigned char success;
    GasData g;

    GasData_Init(&g);

    O2Sensor_getO2Data(&(me->itsO2Sensor));
    N2Sensor_getN2Data(&(me->itsN2Sensor));
    HeSensor_getHeData(&(me->itsHeSensor));

    if (rand()  >  RAND_MAX /  3) {
        g.gType  =  HE_GAS;
        g.conc  =  me->itsHeSensor.conc;
        g.flowInCCPerMin  =  me->itsHeSensor.flow;
```


该线程的主函数（SensorThread_updateData()）时，它会调用每个客户端的getXXData()函数来生成数据，然后将数据封送到GasData结构体中并发送出去。SensorThread的相关代码（除去任务启动和支持函数）见代码清单4-36和代码清单4-37。

/* ... 上方为初始声明内容 ... */ typedef struct SensorThread SensorThread; struct SensorThread { struct GasDataQueue* itsGasDataQueue; struct HeSensor itsHeSensor; struct N2Sensor itsN2Sensor; struct O2Sensor itsO2Sensor; }; /* 操作 */ void SensorThread_updateData(SensorThread* const me); /* ... 还声明了其他操作 ... */ #endif

代码清单 4-36：SensorThread.h

/* updateData 在传感器线程的每个周期运行，并调用其每个传感器的getXXData() 函数，然后使用随机数生成器决定哪些数据应发布到GasDataQueue。 */ void SensorThread_updateData(SensorThread* const me) { unsigned char success; GasData g; GasData_Init(&g); O2Sensor_getO2Data(&(me->itsO2Sensor)); N2Sensor_getN2Data(&(me->itsN2Sensor)); HeSensor_getHeData(&(me->itsHeSensor)); if (rand() > RAND_MAX / 3) { g.gType = HE_GAS; g.conc = me->itsHeSensor.conc; g.flowInCCPerMin = me->itsHeSensor.flow;

```
        success = GasDataQueue_insert(me->itsGasDataQueue, g);
        if (!success)
            printf("Helium Gas Data queue insertion failed!\n");
    };

    if (rand() > RAND_MAX / 3) {
        g.gType = N2_GAS;
        g.conc = me->itsN2Sensor.conc;
        g.flowInCCPerMin = me->itsN2Sensor.flow;
        success = GasDataQueue_insert(me->itsGasDataQueue, g);
        if (!success)
            printf("Nitrogen Gas Data queue insertion failed!\n");
    };

    if (rand() > RAND_MAX / 3) {
        g.gType = O2_GAS;
        g.conc = me->itsO2Sensor.conc;
        g.flowInCCPerMin = me->itsO2Sensor.flow;
        success = GasDataQueue_insert(me->itsGasDataQueue, g);
        if (!success)
            printf("Oxygen Gas Data queue insertion failed! \n");
    }
}
```

Code Listing 4-37: updateData function of SensorThread.c

The sensors in this example are pretty simple – they just augment their values over time but they show that they just produce data; the `SensorThread_updateData()` function is responsible for pushing the data out to the queue. Code Listing 4-38 and Code Listing 4-39 show the code for one of these sensors.

```
#ifndef O2Sensor_H
#define O2Sensor_H

#include "QueuingExample.h"

typedef struct O2Sensor O2Sensor;
struct O2Sensor {
    double conc;
    unsigned int flow;
};

/* Constructors and destructors:*/

void O2Sensor_Init(O2Sensor* const me);
void O2Sensor_Cleanup(O2Sensor* const me);
```


success = GasDataQueue_insert(me->itsGasDataQueue, g); if (!success) printf("氮气数据队列插入失败！\n"); }; if (rand() > RAND_MAX / 3) { g.gType = N2_GAS; g.conc = me->itsN2Sensor.conc; g.flowInCCPerMin = me->itsN2Sensor.flow; success = GasDataQueue_insert(me->itsGasDataQueue, g); if (!success) printf("氮气数据队列插入失败！\n"); }; if (rand() > RAND_MAX / 3) { g.gType = O2_GAS; g.conc = me->itsO2Sensor.conc; g.flowInCCPerMin = me->itsO2Sensor.flow; success = GasDataQueue_insert(me->itsGasDataQueue, g); if (!success) printf("氧气数据队列插入失败！ \n"); } }

代码清单 4-37：SensorThread.c 的 updateData 函数

本示例中的传感器非常简单——它们只是随时间增加其值，但它们表明它们只是产生数据；`SensorThread_updateData()` 函数负责将数据推送到队列。代码清单 4-38 和代码清单 4-39 展示了其中一个传感器的代码。

#ifndef O2Sensor_H #define O2Sensor_H #include "QueuingExample.h" typedef struct O2Sensor O2Sensor; struct O2Sensor { double conc; unsigned int flow; }; /* 构造函数与析构函数： */

void O2Sensor_Init(O2Sensor* const me); void O2Sensor_Cleanup(O2Sensor* const me);

```
/* Operations */

void O2Sensor_getO2Data(O2Sensor* const me);
O2Sensor * O2Sensor_Create(void);
void O2Sensor_Destroy(O2Sensor* const me);

#endif
```

Code Listing 4-38: O2Sensor.h

```
#include "O2Sensor.h"
#include <stdlib.h>
#include <stdio.h>

void O2Sensor_Init(O2Sensor* const me) {
    me->conc = 0.0;
    me->flow = 0;
}

void O2Sensor_Cleanup(O2Sensor* const me) {
}

/*
    getO2Data() is where the sensor class would
    actually acquire the data. Here it just
    augments it.
*/
void O2Sensor_getO2Data(O2Sensor* const me) {
    me->conc += 20;
    me->flow += 25;
}

O2Sensor * O2Sensor_Create(void) {
    O2Sensor* me = (O2Sensor *) malloc(sizeof(O2Sensor));
    if(me!=NULL) {
        O2Sensor_Init(me);
    }
    return me;
}

void O2Sensor_Destroy(O2Sensor* const me) {
    if(me!=NULL) {
        O2Sensor_Cleanup(me);
    }
    free(me);
}
```

Code Listing 4-39: O2Sensor.c


```
/* 操作 */

void O2Sensor_getO2Data(O2Sensor* const me);
O2Sensor * O2Sensor_Create(void); void O2Sensor_
Destroy(O2Sensor* const me);

#endif
```

代码清单 4-38：O2Sensor.h

```
#include "O2Sensor.h" #include <stdlib.h>#include <stdio.h>void
O2Sensor_Init(O2Sensor* const me) { me->conc = 0.0; me->flow = 0; }
void O2Sensor_Cleanup(O2Sensor* const me) { } /* getO2Data() 是传感器
类实际获取数据的地方。这里只是对其进行了增强。 */ void O2Sensor_
getO2Data(O2Sensor* const me) { me->conc += 20; me->flow
 += 25; } O2Sensor * O2Sensor_Create(void) { O2Sensor* me =
(O2Sensor *) malloc(sizeof(O2Sensor)); if(me!=NULL) { O2Sensor_
Init(me); } return me; } void O2Sensor_Destroy(O2Sensor* const me) {
if(me!=NULL) { O2Sensor_Cleanup(me); } free(me); }
```

代码清单 4-39：O2Sensor.c

On the consumer side, the `GasProcessingThread_processData()` runs periodically and pulls data out. If there are data available, it calls the function `GasController_handleGasData()` of its nested instance of `GasController`. This function just prints out the data (by calling the `GasDisplay_printGasData()` function) but one can imagine that it might find more interesting things to do with the data, such as maintain a constant gas delivery of appropriate concentrations. Code Listing 4-40 through Code Listing 4-43 show the code for these two elements.

```
#ifndef GasController_H
#define GasController_H

#include "QueuingExample.h"
#include "GasData.h"
struct GasDisplay;

typedef struct GasController GasController;
struct GasController {
    struct GasDisplay*  itsGasDisplay;
};

/*  Constructors and destructors:*/

void GasController_Init(GasController*  const me);
void GasController_Cleanup(GasController*  const me);

/*  Operations  */
void GasController_handleGasData(GasController*  const me, GasData
*  gPtr);

struct GasDisplay*  GasController_getItsGasDisplay(const
GasController*  const me);

void GasController_setItsGasDisplay(GasController*  const me,
struct GasDisplay*  p_GasDisplay);

GasController  *  GasController_Create(void);

void GasController_Destroy(GasController*  const me);

#endif
```

Code Listing 4-40: GasController.h

```
#include "GasController.h"
#include "GasDisplay.h"
#include <stdio.h>
static void cleanUpRelations(GasController*  const me);
```


在消费者一侧，`GasProcessingThread_processData()` 周期性地运行并提取数据。如果有可用数据，它会调用其嵌套的 `GasController` 实例的 `GasController_handleGasData()` 函数。该函数只是打印出数据（通过调用`GasDisplay_printGasData()` 函数），但可以想象它可能会找到更有趣的方式来处理这些数据，例如维持适当浓度的恒定气体输送。代码清单 4-40 至代码清单 4-43 展示了这两个元素的代码。

```
#ifndef GasController_H #
define GasController_H

#include "QueuingExample.h" #
include "GasData.h" struct
GasDisplay;

typedef struct GasController GasController; struct
GasController { struct GasDisplay* itsGasDisplay; };



/* 构造函数与析构函数：*/

void GasController_Init(GasController* const me); void
GasController_Cleanup(GasController* const me);

/* 操作 */ void GasController_handleGasData(GasController* const me, GasData
* gPtr);


struct GasDisplay* GasController_getItsGasDisplay(const
GasController* const me);

void GasController_setItsGasDisplay(GasController* const me, struct
GasDisplay* p_GasDisplay);

GasController * GasController_Create(void);

void GasController_Destroy(GasController* const me);

#endif
```

代码清单 4-40：GasController.h

```
#include "GasController.h" #include "GasDisplay.h" #include <
stdio.h>static void cleanUpRelations(GasController* const me);
```

```
void GasController_Init(GasController*  const me) {
    me->itsGasDisplay  =  NULL;
}

void GasController_Cleanup(GasController*  const me) {
    cleanUpRelations(me);
}

void GasController_handleGasData(GasController*  const me, GasData  *  gPtr) {
    if (gPtr) {
        GasDisplay_printGasData(me->itsGasDisplay, gPtr->gType, gPtr->conc,
        gPtr->flowInCCPerMin);
        free(gPtr);
        };
}

struct GasDisplay*  GasController_getItsGasDisplay(const GasController*  const me) {
    return (struct GasDisplay*)me->itsGasDisplay;
}
void GasController_setItsGasDisplay(GasController*  const me, struct GasDisplay*
p_GasDisplay) {
    me->itsGasDisplay  =  p_GasDisplay;
}

GasController  *  GasController_Create(void) {
    GasController*  me  =  (GasController  *)
malloc(sizeof(GasController));
    if(me!=NULL)
    {
    GasController_Init(me);
    }
    return me;
}

void GasController_Destroy(GasController*  const me) {
    if(me!=NULL)
    {
    GasController_Cleanup(me);
    }
    free(me);
}

static void cleanUpRelations(GasController*  const me) {
    if(me->itsGasDisplay !=  NULL)
    {
    me->itsGasDisplay  =  NULL;
    }
}
```

Code Listing 4-18: GasController.c

void GasController_Init(GasController* const me) { me->itsGasDisplay = 空; } void GasController_Cleanup(GasController* const me) { cleanUpRelations(me); } void GasController_handleGasData(GasController* const me, GasData * gPtr) { if (gPtr) { GasDisplay_printGasData(me->itsGasDisplay, gPtr->gType, gPtr->conc, gPtr->flowInCCPerMin); free(gPtr); }; } struct GasDisplay* GasController_getItsGasDisplay(const GasController* const me) { return (struct GasDisplay*)me->itsGasDisplay; } void GasController_setItsGasDisplay(GasController* const me, struct GasDisplay* p_GasDisplay) { me->itsGasDisplay = p_GasDisplay; } GasController * GasController_Create(void) { GasController* me = (GasController *) malloc(sizeof(GasController)); if(me!=空) { GasController_Init(me); } return me; } void GasController_Destroy(GasController* const me) { if(me!=空) { GasController_Cleanup(me); } free(me); } static void cleanUpRelations(GasController* const me) { if(me->itsGasDisplay != 空) { me->itsGasDisplay = 空; } }

代码清单 4-18：GasController.c

```
#include "QueuingExample.h"
#include "GasData.h"

typedef struct GasDisplay GasDisplay;
struct GasDisplay {
    int screenWidth;
};

/*  Constructors and destructors:*/
void GasDisplay_Init(GasDisplay*  const me);
void GasDisplay_Cleanup(GasDisplay*  const me);

/*  Operations  */
void GasDisplay_printGasData(GasDisplay*  const me, const GAS_TYPE gasType, double
gas_conc, unsigned int gas_flow);

GasDisplay  *  GasDisplay_Create(void);

void GasDisplay_Destroy(GasDisplay*  const me);

#endif
```

Code Listing 4-19: GasDisplay.h

#include "QueuingExample.h" #include "GasData.h" typedef struct GasDisplay GasDisplay; struct GasDisplay { int screenWidth; }; /* 构造函数与析构函数：*/ void GasDisplay_Init(GasDisplay* const me); void GasDisplay_Cleanup(GasDisplay* const me); /* 操作 */ void GasDisplay_printGasData(GasDisplay* const me, const GAS_TYPE gasType, double gas_conc, unsigned int gas_flow); GasDisplay * GasDisplay_Create(void); void GasDisplay_Destroy(GasDisplay* const me); #endif

代码清单 4-19：GasDisplay.h

```
#include "GasDisplay.h"
#include <stdio.h>

void GasDisplay_Init(GasDisplay*  const me) {
    me->screenWidth  =  80;
}

void GasDisplay_Cleanup(GasDisplay*  const me) {
}

void GasDisplay_printGasData(GasDisplay*  const me, const GAS_TYPE gasType, double
gas_conc, unsigned int gas_flow) {
    printf("\n");
    switch (gasType) {
        case O2_GAS:
            printf("Oxygen  ");
            break;
        case N2_GAS:
            printf("Nitrogen  ");
            break;
        case HE_GAS:
```

#include "GasDisplay.h" #include <stdio.h>void GasDisplay_Init(GasDisplay* const me) { me->screenWidth = 80; } void GasDisplay_Cleanup(GasDisplay* const me) { } void GasDisplay_printGasData(GasDisplay* const me, const GAS_TYPE gasType, double gas_conc, unsigned int gas_flow) { printf("\n");switch (gasType) { case O2 GAS:_printf("氧气 "); break; case N2 GAS:_printf("氮气 "); break; case HE_GAS:

```
            printf("Helium  ");
            break;
        default:
                    printf("UNKNWON  ");
                    break;
    };
    printf("Conc %f, Flow in CC/Min %d\n", gas_conc, gas_flow);
    printf("\n");
}

GasDisplay  *  GasDisplay_Create(void){
    GasDisplay*  me  =  (GasDisplay  *) malloc(sizeof(GasDisplay));
    if(me!=NULL){
    GasDisplay_Init(me);
    }
    return me;
}

void GasDisplay_Destroy(GasDisplay*  const me){
    if(me!=NULL){
    GasDisplay_Cleanup(me);
    }
    free(me);
}
```

Code Listing 4-43: GasDisplay.c

Figure 4-19 shows a four-second run of the example in which the `GasProcessingThread` runs first at a period of 1000 ms and the `SensorThread` runs second at a period of 500 ms and a queue size of 10. You can see that even though each of the three gas sensors only has a 1/3 chance of producing data during the interval, data are put in faster than they are removed and the queue eventually fills up. The insert operation is instrumented with a `printf()` statement to show when data are inserted. The `GasController's handleGasData()` is invoked every period of the `GasProcessingThread` and it calls the `GasDisplay's printGasData()` function.

4.7 Rendezvous Pattern

Tasks must synchronize in various ways. The Queuing and Guarded Call Patterns provide mechanisms when the synchronization occurs around a simple function call, sharing a single resource, or passing data. But what if the conditions required for synchronization are more complex? The Rendezvous Pattern reifies the strategy as an object itself and therefore supports even those most complex needs for task synchronization.

printf("氦气 "); break; default: printf("未知 "); break; }; printf("浓度 %f, 流量（CC/分钟）%d\n", gas_conc, gas_flow); printf("\n"); } GasDisplay * GasDisplay_Create(void) { GasDisplay* me = (GasDisplay *) malloc(sizeof(GasDisplay)); if(me!=NULL) { GasDisplay_Init(me); } return me; } void GasDisplay_Destroy(GasDisplay* const me) { if(me!=NULL) { GasDisplay_Cleanup(me); } free(me); }

代码清单 4-43：GasDisplay.c

图 4-19 展示了该示例运行四秒的情况，其中 `GasProcessingThread` 首先以 1000 毫秒的周期运行，`SensorThread` 随后以 500 毫秒的周期和 10 的队列大小运行。可以看到，尽管三个气体传感器在每个间隔内只有 1/3 的概率产生数据，但数据插入的速度快于移除的速度，队列最终会被填满。插入操作通过 `printf()` 语句进行了插桩，以显示数据何时被插入。`GasController` 的 `handleGasData()` 在每个 `GasProcessingThread` 周期被调用，并且它调用 `GasDisplay` 的 printGasData() 函数。

4.7 会合模式

任务必须以多种方式进行同步。排队模式和受保护调用模式在同步围绕简单函数调用、共享单个资源或传递数据时提供了机制。但如果同步所需的条件更为复杂呢？会合模式将策略本身具体化为一个对象，因此甚至支持那些最复杂的任务同步需求。

```
+++ Nitrogen  at conc 10.000000, flow 15
   Number of elements queued 1, head = 1, tail = 0
+++ Oxygen    at conc 20.000000, flow 25
   Number of elements queued 2, head = 2, tail = 0
--- Nitrogen  at conc 10.000000, flow 15
   Number of elements queued 1, head = 2, tail = 1

Nitrogen Conc 10.000000, Flow in CC/Min 15

+++ Helium    at conc 2.000000, flow 2
   Number of elements queued 2, head = 3, tail = 1
+++ Nitrogen  at conc 20.000000, flow 30
   Number of elements queued 3, head = 4, tail = 1
+++ Oxygen    at conc 40.000000, flow 50
   Number of elements queued 4, head = 5, tail = 1
+++ Oxygen    at conc 60.000000, flow 75
   Number of elements queued 5, head = 6, tail = 1
--- Oxygen    at conc 20.000000, flow 25
   Number of elements queued 4, head = 6, tail = 2

Oxygen   Conc 20.000000, Flow in CC/Min 25

+++ Helium    at conc 4.000000, flow 4
   Number of elements queued 5, head = 7, tail = 2
+++ Nitrogen  at conc 40.000000, flow 60
   Number of elements queued 6, head = 8, tail = 2
+++ Helium    at conc 5.000000, flow 5
   Number of elements queued 7, head = 9, tail = 2
+++ Nitrogen  at conc 50.000000, flow 75
   Number of elements queued 8, head = 0, tail = 2
--- Helium    at conc 2.000000, flow 2
   Number of elements queued 7, head = 0, tail = 3

Helium   Conc 2.000000, Flow in CC/Min 2

+++ Helium    at conc 6.000000, flow 6
   Number of elements queued 8, head = 1, tail = 3
+++ Oxygen    at conc 120.000000, flow 150
   Number of elements queued 9, head = 2, tail = 3
+++ Nitrogen  at conc 70.000000, flow 105
   Number of elements queued 10, head = 3, tail = 3
Oxygen Gas Data queue insertion failed!
--- Nitrogen  at conc 20.000000, flow 30
   Number of elements queued 9, head = 3, tail = 4
+++ Helium    at conc 8.000000, flow 8
Conc 20.000000, Flow in CC/Min 30Conc 20.000000, Flow in CC/Min 30
ued 10, head = 4, tail = 4
Oxygen Gas Data queue insertion failed!
-
```

Figure 4-19: Sample run of the Queuing example

4.7.1 Abstract

A *precondition* is something that is specified to be true prior to an action or activity. A precondition is a type of constraint that is usually generative, that is, it can be used to generate code either to force the precondition to be true or to check that a precondition is true.

The Rendezvous Pattern is concerned with modeling the preconditions for synchronization or rendezvous of threads as a separate explicit object with its own data and functions. It is a general pattern and easy to apply to ensure that arbitrarily complex sets of preconditions can be met at run-time. The basic behavioral model is that as each thread becomes ready to rendezvous, it registers with the `Rendezvous` class, and then blocks until the `Rendezvous`

+++ Nitrogen at conc 10.000000, flow 15
   Number of elements queued 1, head = 1, tail = 0
+++ Oxygen   at conc 20.000000, flow 25
   Number of elements queued 2, head = 2, tail = 0
--- Nitrogen at conc 10.000000, flow 15
   Number of elements queued 1, head = 2, tail = 1

Nitrogen Conc 10.000000, Flow in CC/Min 15

+++ Helium   at conc 2.000000, flow 2
   Number of elements queued 2, head = 3, tail = 1
+++ Nitrogen at conc 20.000000, flow 30
   Number of elements queued 3, head = 4, tail = 1
+++ Oxygen   at conc 40.000000, flow 50
   Number of elements queued 4, head = 5, tail = 1
+++ Oxygen   at conc 60.000000, flow 75
   Number of elements queued 5, head = 6, tail = 1
--- Oxygen   at conc 20.000000, flow 25
   Number of elements queued 4, head = 6, tail = 2

Oxygen   Conc 20.000000, Flow in CC/Min 25

+++ Helium   at conc 4.000000, flow 4
   Number of elements queued 5, head = 7, tail = 2
+++ Nitrogen at conc 40.000000, flow 60
   Number of elements queued 6, head = 8, tail = 2
+++ Helium   at conc 5.000000, flow 5
   Number of elements queued 7, head = 9, tail = 2
+++ Nitrogen at conc 50.000000, flow 75
   Number of elements queued 8, head = 0, tail = 2
--- Helium   at conc 2.000000, flow 2
   Number of elements queued 7, head = 0, tail = 3

Helium   Conc 2.000000, Flow in CC/Min 2

+++ Helium   at conc 6.000000, flow 6
   Number of elements queued 8, head = 1, tail = 3
+++ Oxygen   at conc 120.000000, flow 150
   Number of elements queued 9, head = 2, tail = 3
+++ Nitrogen at conc 70.000000, flow 105
   Number of elements queued 10, head = 3, tail = 3
Oxygen Gas Data queue insertion failed!
--- Nitrogen at conc 20.000000, flow 30
   Number of elements queued 9, head = 3, tail = 4
+++ Helium   at conc 8.000000, flow 8
Conc 20.000000, Flow in CC/Min 30Conc 20.000000, Flow in CC/Min 30
ued 10, head = 4, tail = 4
Oxygen Gas Data queue insertion failed!
-

图 4-19：排队示例的示例运行

4.7.1 摘要

前置条件是指在某个动作或活动之前被指定为真的条件。前置条件是一种约束，通常是生成式的，也就是说，它可用于生成代码，以强制前置条件为真或检查前置条件是否为真。

会合模式关注的是将线程同步或会合的前置条件建模为一个独立的显式对象，该对象拥有自己的数据和函数。它是一种通用模式，易于应用，可确保在运行时满足任意复杂的前置条件集合。其基本行为模型是：当每个线程准备好会合时，它会向Rendezvous类注册，然后阻塞，直到Rendezvous

class releases it to run. Once the set of preconditions is met, then the registered tasks are released to run using whatever scheduling policy is currently in force.

4.7.2 Problem

The Rendezvous Pattern solves the problems when a set of tasks must synchronize in a complex fashion.

4.7.3 Pattern Structure

Figure 4-20 shows the structure of the Rendezvous Pattern. Its structural simplicity belies its useful to manage complex task interactions.

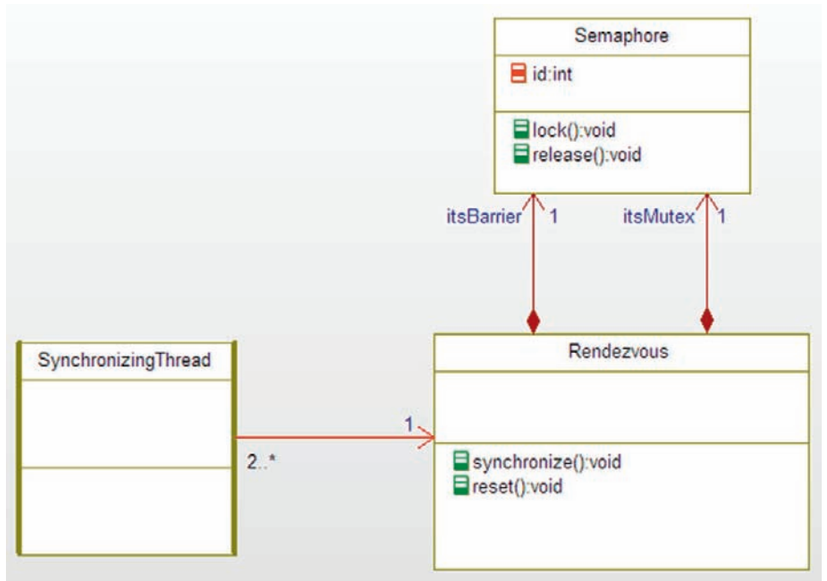


Figure 4-20: Rendezvous Pattern

4.7.4 Collaboration Roles

This section describes the roles for this pattern.

4.7.4.1 Rendezvous

The `Rendezvous` class manages the synchronization. It does this through its two primary functions:

- void reset(void)
  This function resets the synchronization criteria back to its initial condition.

类释放它运行。一旦前置条件集合得到满足，已注册的任务便会被释放，按照当前生效的调度策略运行。

4.7.2 问题

会合模式解决了当一组任务必须以复杂方式进行同步时的问题。

4.7.3 模式结构

图 4-20 展示了会合模式的结构。其结构的简洁掩盖了其管理复杂任务交互的实用性。

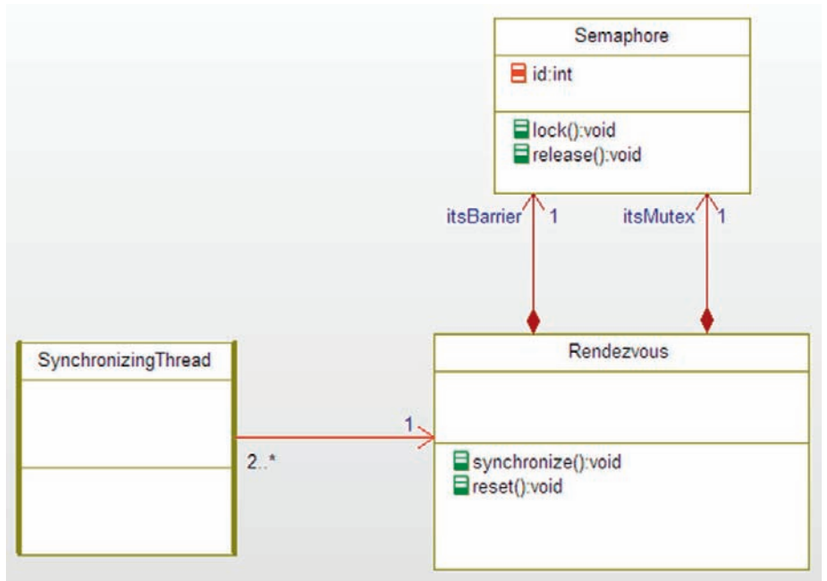


图4-20：会合模式

4.7.4 协作角色

本节描述该模式的角色。

4.7.4.1 会合

会合 类负责管理同步。它通过两个主要函数来实现这一点：

- void reset(void) 此函数将同步条件重置回其初始状态。

- void synchronize(void)
  This function is called when a task wants to synchronize. If the criteria are not met, then the task is blocked in some way (see Implementation Strategies, below). This is commonly done either through the use of the Observer Pattern (see Chapter 2) or the Guarded Call Pattern.

The complexity of this class resides primarily in the `synchronize()` function; this evaluates the synchronization preconditions to see if they are met. These conditions may be internal (such as the Thread Barrier variant of the pattern used for the example) or external, or any combination of the two.

It is important to note that the `Rendezvous` object itself doesn't normally identify the threads as they rendezvous; as far as the `Rendezvous` object is concerned the tasks are anonymous. If they are not, such as when the strategy calls for tasks to rendezvous in a specific order, then the `synchronize()` function can accept that information in the form of one or more parameters.

4.7.4.2 Semaphore

This is a normal counting semaphore. The semaphore provides the following operations:

- Semaphore* create_semaphore(void)
- destroy_semaphore(Semaphore*)
- lock_semaphore(Semaphore*)
- release_semaphore(Semaphore*)

4.7.4.3 SynchronizingThread

This element represents a thread that uses the `Rendezvous` object for synchronization. It will run until its synchronization point and then call `synchronize()` function.

There are a number of ways to implement the pattern that result in minor changes to the parameters and return types for the pattern elements.

4.7.5 Consequences

In this pattern, two or more tasks can be synchronized by any arbitrarily complex strategy that can be encoded in the Rendezvous class. The pattern itself is simple and flexible and easily lends itself to specialization and adaptation.

The pattern is most appropriate when the tasks must halt at their synchronization point; that is, the pattern implements a blocking rendezvous. If a nonblocking rendezvous is desired, then this pattern can be mixed with a Queuing Pattern or the Observer Pattern with callbacks to a `notify()` function within the `SynchronizingThread`.

- void synchronize(void)
  当任务想要同步时会调用此函数。如果未满足条件，则任务会以某种方式被阻塞（参见下文“实现策略”）。这通常通过使用观察者模式（参见第2章）或受保护调用模式来实现。

此类的复杂性主要体现在 `synchronize()` 函数中；该函数评估同步前置条件以检查是否满足。这些条件可以是内部的（例如示例中使用的模式的线程屏障变体）或外部的，或两者的任意组合。

需要注意的是，`Rendezvous` 对象本身通常不会在线程会合时标识它们；就 `Rendezvous` 对象而言，任务是匿名的。如果不是这样，例如当策略要求任务按特定顺序会合时，那么 `synchronize()` 函数可以通过一个或多个参数的形式接受该信息。

4.7.4.2 信号量

这是一个普通的 计数信号量。该信号量提供以下操作：

- 信号量* create_semaphore(void)
- destroy_信号量(Semaphore*)
- 加锁_信号量(Semaphore*)
- 释放_信号量(信号量*)

4.7.4.3 SynchronizingThread

该模式元素表示一个使用`Rendezvous` 对象进行同步的线程。它将运行至其同步点，然后调用`synchronize()` 函数。

实现该模式有多种方式，这些方式会导致模式元素的参数和返回类型发生细微变化。

4.7.5 结果

在该模式中，两个或多个任务可以通过任意复杂且可编码在Rendezvous类中的策略进行同步。该模式本身简单而灵活，易于进行特化和适配。

当任务必须在其同步点处停止时，该模式最为适用；也就是说，该模式实现的是阻塞式会合。如果需要非阻塞式会合，则可以将此模式与排队模式或观察者模式混合使用，通过回调 `SynchronizingThread` 内的 `notify()` 函数来实现。

4.7.6 Implementation Strategies

This pattern may be implemented with the Observer Pattern from Chapter 3 or with the Guarded Call Pattern from this chapter. If the pattern is implemented with the Observer Pattern, then the Tasks must register with the address of a function to call when the synchronization criteria have been met. If the pattern is implemented with a Guarded Call Pattern, then each `Rendezvous` object owns a single semaphore that is enabled when the `Rendezvous` object is created. Each calling task then calls the `register()` function when it wants to synchronize. When the synchronization criteria is met, the semaphore is released, and the tasks are then all free to run according to the normal scheduling strategy.

4.7.7 Related Patterns

This pattern may be overkill when simpler solutions such as Guarded Call or Queuing Patterns may suffice.

The pattern may be implemented in a number of different ways, as well, using the Guarded Call or Observer Patterns to manage the thread blocking and unblocking.

4.7.8 Example

In this example (Figure 4-21), we have implemented a specific form of the Rendezvous Pattern knows as the Thread Barrier Pattern. In this pattern, the `Rendezvous` object only knows that a certain number of tasks are expected to rendezvous. When that number of tasks have called the `synchronize()` function, the set of blocked tasks are released. The example also implemented the pattern with the use of semaphores (Guarded Call Pattern) and lets the operating system block and unlock the tasks.

The algorithm for the Thread Barrier is well understood using counting semaphores. See the code for the synchronize operation in Code Listing 4-45.

```
#ifndef ThreadBarrier_H
#define ThreadBarrier_H

/*# # auto_generated  */
#include <oxf/Ric.h>
/*# # auto_generated  */
#include "RendezvousExample.h"
/*# # auto_generated  */
#include <oxf/RiCTask.h>
/*# # package RendezvousPattern::RendezvousExample  */

/*# # class ThreadBarrier  */
typedef struct ThreadBarrier ThreadBarrier;
struct ThreadBarrier {
    int currentCount;
```


4.7.6 实现策略

该模式可以使用第3章的观察者模式或本章的受保护调用模式来实现。如果使用观察者模式实现该模式，则任务必须注册一个函数的地址，以便在满足同步条件时调用该函数。如果使用受保护调用模式实现该模式，则每个 `Rendezvous` 对象拥有一个信号量，该信号量在创建 `Rendezvous` 对象时启用。然后，每个调用任务在需要同步时调用 `register()` 函数。当满足同步条件时，信号量被释放，随后所有任务都可以按照正常的调度策略自由运行。

4.7.7 相关模式

当更简单的解决方案（如受保护调用或排队模式）已足够时，此模式可能过于复杂。

该模式也可以通过多种不同方式实现，使用受保护调用或观察者模式来管理线程阻塞和解除阻塞。

4.7.8 示例

在 本 示例 （图 4-21）中，我们实现了会合模式的一种特定形式，称为线程屏障模式。在该模式中，会合 对象只知道预期会有一定数量的任务进行会合。当该数量的任务调用了`synchronize()`函数后，被阻塞的任务集合便被释放。该示例还使用信号量（受保护调用模式）实现了该模式，并让操作系统阻塞和解锁任务。

使用计数信号量可以很好地理解线程屏障的算法。参见代码清单4-45中同步操作的代码。

```
#ifndef ThreadBarrier_H #
define ThreadBarrier_H

/*# # auto_generated */ #include <oxf/Ric.h>/*# # auto_
generated */ #include "RendezvousExample.h" /*# # auto_
generated */ #include <oxf/RiCTask.h>/*# # package
RendezvousPattern::RendezvousExample */



/*# # 类 ThreadBarrier */ typedef struct
ThreadBarrier ThreadBarrier; struct
ThreadBarrier { int currentCount;
```

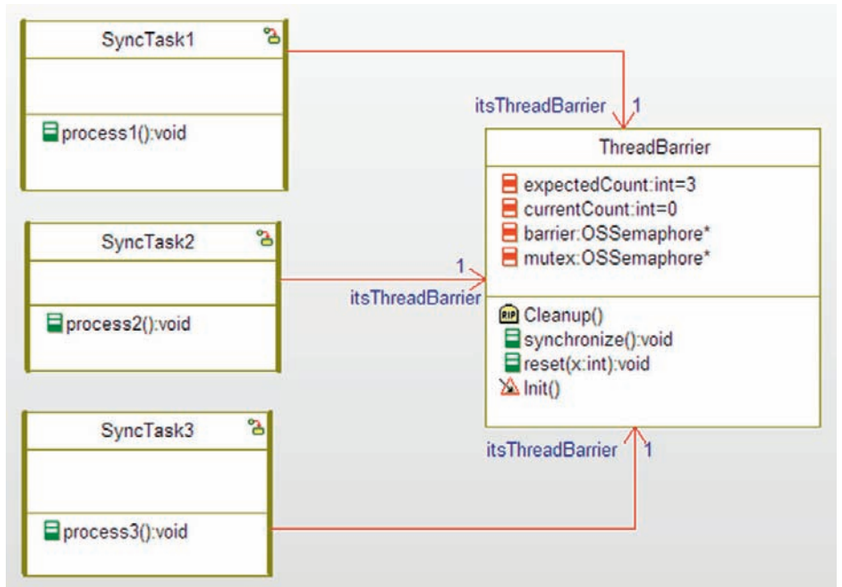


Figure 4-21: Rendezvous Example

```
    int expectedCount;
    OSSemaphore*  barrier;
    OSSemaphore*  mutex;
};

/*  Constructors and destructors:*/
void ThreadBarrier_Init(ThreadBarrier*  const me);
void ThreadBarrier_Cleanup(ThreadBarrier*  const me);

/*  Operations  */
void ThreadBarrier_reset(ThreadBarrier*  const me, int x);
void ThreadBarrier_synchronize(ThreadBarrier*  const me);
ThreadBarrier  *  ThreadBarrier_Create(void);
void ThreadBarrier_Destroy(ThreadBarrier*  const me);
#endif
```

Code Listing 4-20: ThreadBarrier.h

```
#include "ThreadBarrier.h"

void ThreadBarrier_Init(ThreadBarrier*  const me) {
    me->currentCount  =  0;
    me->expectedCount  =  3;
```

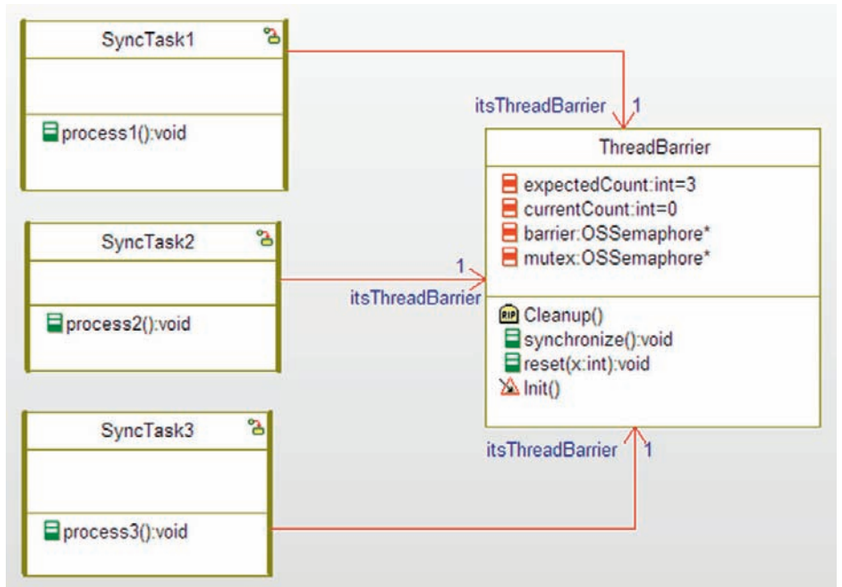



图4-21：会合示例

```
      int expectedCount;
OSSemaphore* barrier;
OSSemaphore* mutex; };

/* 构造函数与析构函数：*/ void ThreadBarrier_Init(ThreadBarrier*
const me); void ThreadBarrier_Cleanup(ThreadBarrier* const
me);

/* 操作 */
void ThreadBarrier_reset(ThreadBarrier* const me, int x); void
ThreadBarrier_synchronize(ThreadBarrier* const me);
ThreadBarrier * ThreadBarrier_Create(void); void ThreadBarrier_
Destroy(ThreadBarrier* const me); #endif
```

代码清单4-20：ThreadBarrier.h

```
#include "ThreadBarrier.h"

void ThreadBarrier_Init(ThreadBarrier* const me) { me->
currentCount  =  0; me->expectedCount  =  3;
```

```
    me->mutex = OSSemaphore_Create();
    me->barrier = OSSemaphore_Create();
    if(me->barrier){
        OSSemaphore_lock(me->barrier);
        printf("BARRIER IS LOCKED FIRST TIME\n");
    };
}

void ThreadBarrier_Cleanup(ThreadBarrier* const me){
    OSSemaphore_Destroy(me->barrier);
    OSSemaphore_Destroy(me->mutex);
}

void ThreadBarrier_reset(ThreadBarrier* const me, int x){
    me->expectedCount = x;
    me->currentCount = 0;
}

void ThreadBarrier_synchronize(ThreadBarrier* const me){
    /*
        protect the critical region around
        the currentCount
    */
    OSSemaphore_lock(me->mutex);
        ++me->currentCount;     /* critical region */
    OSSemaphore_release(me->mutex);

    /*
        are conditions for unblocking met?
        if so, then release the first blocked
        thread or the highest priority blocked
        thread (depending on the OS)
    */
    if(me->currentCount == me->expectedCount){
        printf("Conditions met\n");
        OSSemaphore_release(me->barrier);
    };

    /*
        lock the semaphore and when released
        then release it for the next blocked thread
    */
    OSSemaphore_lock(me->barrier);
    OSSemaphore_release(me->barrier);
}

ThreadBarrier * ThreadBarrier_Create(void){
    ThreadBarrier* me = (ThreadBarrier *)
  malloc(sizeof(ThreadBarrier));
    if(me!=NULL)
```


me->mutex = OSSemaphore_Create(); me->barrier = OSSemaphore_Create(); if (me->barrier) { OSSemaphore_lock(me->barrier); printf("BARRIER IS LOCKED FIRST TIME\n"); }; } void ThreadBarrier_Cleanup(ThreadBarrier* const me) { OSSemaphore_Destroy(me->barrier); OSSemaphore_Destroy(me->mutex); } void ThreadBarrier_reset(ThreadBarrier* const me, int x) { me->expectedCount = x; me->currentCount = 0; } void ThreadBarrier_synchronize(ThreadBarrier* const me) { /* 保护currentCount 周围的临界区 */ OSSemaphore_lock(me->mutex);++me->currentCount; /* 临界区 */ OSSemaphore_release(me->mutex); /* 解除阻塞的条件是否满足？如果满足，则释放第一个被阻塞的线程或优先级最高的被阻塞线程（取决于操作系统） */ if (me->currentCount == me->expectedCount) { printf("Conditions met\n"); OSSemaphore_release(me->barrier); }; /* 锁定信号量，当被释放时再将其释放给下一个被阻塞的线程 */ OSSemaphore_lock(me->barrier); OSSemaphore_release(me->barrier); } ThreadBarrier * ThreadBarrier_Create(void) { ThreadBarrier* me = (ThreadBarrier *) malloc(sizeof(ThreadBarrier)); if(me!=NULL)

```
    ThreadBarrier_Init(me);
    return me;
}

void ThreadBarrier_Destroy(ThreadBarrier*  const me) {
    if(me!=NULL)
      ThreadBarrier_Cleanup(me);
    free(me);
}
```

Code Listing 4-21: ThreadBarrier.c

Deadlock Avoidance Patterns

The last problem I want to address in this chapter is *deadlock.* Deadlock is a situation in which multiple clients are simultaneously waiting for conditions that cannot, in principle, occur.

Figure 4-22 shows an example of deadlock. In this example, the priority of Task 1 is higher than that of Task 2. The figure shows a timing diagram overlayed with a class diagram showing the structure. In addition to the tasks, there are two resources, R1 and R2, shared by both tasks. The problem arises if the tasks lock the resources in reverse order and Task 1 preempts Task 2

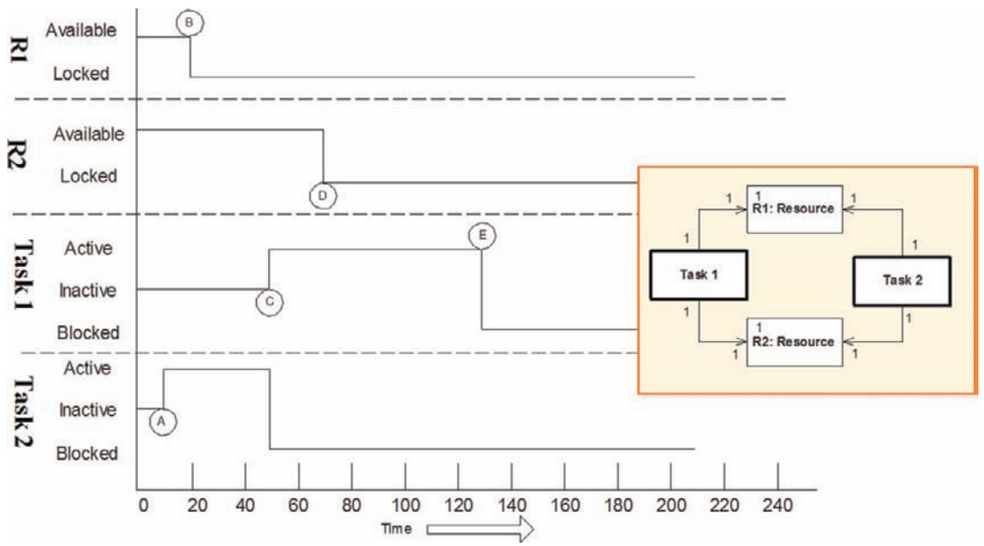


Figure 4-22: Deadlock example


```
    ThreadBarrier_Init(me); return me; } void ThreadBarrier_
Destroy(ThreadBarrier* const me) { if(me!=NULL)
ThreadBarrier_Cleanup(me); free(me); }
```

代码清单 4-21：ThreadBarrier.c

死锁避免模式

本章我要讨论的最后一个问题是死锁。死锁是指多个客户端同时等待原则上不可能发生的条件的情况。

图 4-22 展示了一个死锁的示例。在此示例中，任务 1 的优先级高于任务 2。该图展示了一个时序图，并叠加了显示结构的类图。除了任务之外，还有两个资源 R1 和 R2，由两个任务共享。如果任务以相反的顺序加锁资源，并且任务 1 抢占任务 2，就会出现问题

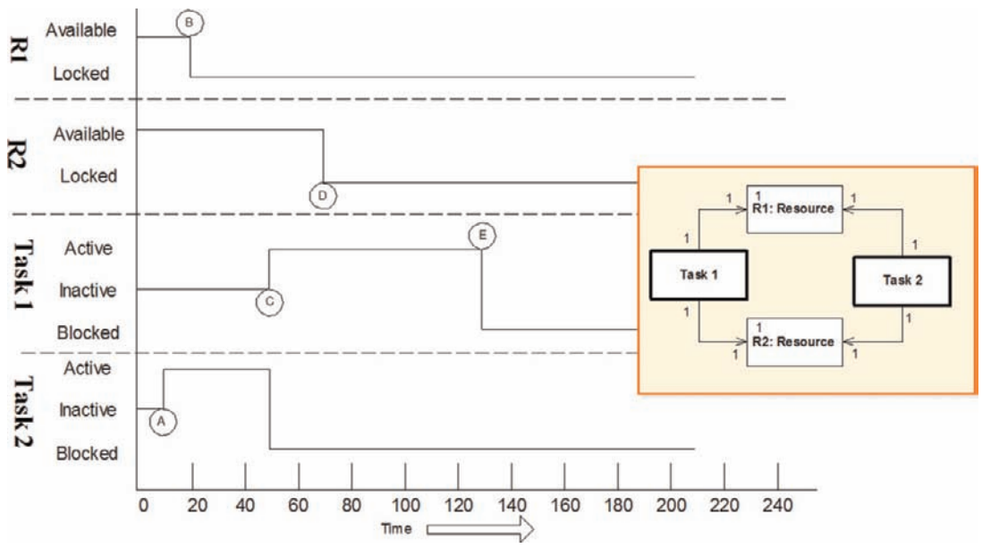


图 4-22：死锁示例

between Task 1 between locking R1 and R2. The following description refers to the named points in Figure 4-22.

A. At this point, Task 2 runs.
B. Task 2 locks R1 and is just about to lock R2.
C. Task 1 runs. Since it is higher priority than Task 2, it preempts Task 2.
D. Task 1 locks R2.
E. Task 1 now attempts to lock R1, however, R1 is currently locked by Task 2. If Task 1 blocks to allow Task 2 to run, Task 2 must block as soon as it tries to lock R1. The system is in deadlock.

It is enough to break any of the four required conditions for deadlock (below) conditions to ensure that deadlock cannot appear. The simultaneous locking, ordered locking, and critical region patterns all avoid the problems of deadlock. For example, had Task 2 enabled a critical region (i.e., disallowed task switching) before it locked R1, then Task 1 would not have been able to preempt it before it locked R2. If both tasks locked the resources in the same order, deadlock would have been prevented. If Task 2 locks both resources at the same time, there would be no problem. Deadlock is easy to prevent, but you must take careful steps to do so.

Leaving aside the common problems of software errors, deadlocks require four conditions to be true:

1. mutual exclusion locking of resources
2. some resources are locked while others are requested
3. preemption while resources are locked is allowed
4. a circular waiting condition exists

Deadlock can be avoided by breaking any of these four conditions. Using the Critical Region Pattern, for example, breaks rules #1 and #3. The Queuing Pattern, because it uses asynchronous message passing avoids #1 (other than the lock on the queue access itself). This chapter finishes up with two other patterns that avoid deadlock.

4.8 Simultaneous Locking Pattern

The Simultaneous Locking Pattern is a pattern solely concerned with deadlock avoidance. It achieves this by locking breaking condition #2 (holding resources while waiting for others). The pattern works in an all-or-none fashion. Either all needed resources are locked at once or none are locked.

4.8.1 Abstract

Deadlock can be solved by breaking any of the four required conditions. This pattern prevents the condition of holding some resources by requesting others by allocating them all at once.

在 任务 1 在 加锁 R1 和 R2 之间。 以下 描述 涉及 图 4-22 中的命名点。

A. 此时，任务 2 运行。B. 任务 2 锁定 R1，正准备锁定 R2。C. 任务 1 运行。由于它的优先级高于任务 2，它抢占了任务 2。D. 任务 1 锁定 R2。E. 任务 1 现在尝试锁定 R1，然而，R1 当前已被任务 2 锁定。如果任务 1 阻塞以允许任务 2 运行，那么任务 2 在尝试锁定 R1 时也必须阻塞。系统处于死锁状态。

只要打破死锁四个必要条件（见下文）中的任意一个，就足以确保死锁不会出现。同时锁定、有序锁定和临界区模式都能避免死锁问题。例如，如果任务 2 在锁定 R1 之前启用了临界区（即禁止任务切换），那么任务 1 就无法在它锁定 R2 之前抢占它。如果两个任务以相同的顺序锁定资源，死锁就可以被防止。如果任务 2 同时锁定两个资源，就不会有问题。死锁很容易预防，但你必须采取谨慎的步骤来做到这一点。

撇开常见的软件错误问题不谈，死锁需要满足四个条件：

1. 资源的互斥锁定 2. 某些资源已被锁定而其他资源仍被请求 3. 允许在资源被锁定时进行抢占 4. 存在循环等待条件

通过打破这四个条件中的任意一个，就可以避免死锁。例如，使用临界区模式可以打破规则#1和#3。排队模式由于使用异步消息传递，避免了#1（除了对队列访问本身的加锁之外）。本章最后还将介绍另外两种避免死锁的模式。

4.8 同时锁定模式

同时锁定模式是一种仅关注死锁避免的模式。它通过打破条件#2（在等待其他资源时持有资源）来实现这一点。该模式以全有或全无的方式工作。要么一次性锁定所有需要的资源，要么一个都不锁定。

4.8.1 摘要

死锁可以通过打破四个必要条件中的任意一个来解决。该模式通过一次性分配所有资源来请求其他资源，从而防止持有部分资源的条件。

However, this pattern has an advantage over the Critical Region Pattern by allowing higher priority tasks to run if they don't need any of the locked resources.

4.8.2 Problem

The problem of deadlock is a serious enough one in highly reliable computing that many systems design in specific mechanisms to detect it or avoid it. As previously discussed, deadlock occurs when a task is waiting on a condition that can never, in principle, be satisfied. There are four conditions that must be true for deadlock to occur, and it is sufficient to deny the existence of any one of these. The Simultaneous Locking Pattern breaks condition #2, not allowing any task to lock resources while waiting for other resources to be free.

4.8.3 Pattern Structure

The structure of the pattern is shown in Figure 4-23.

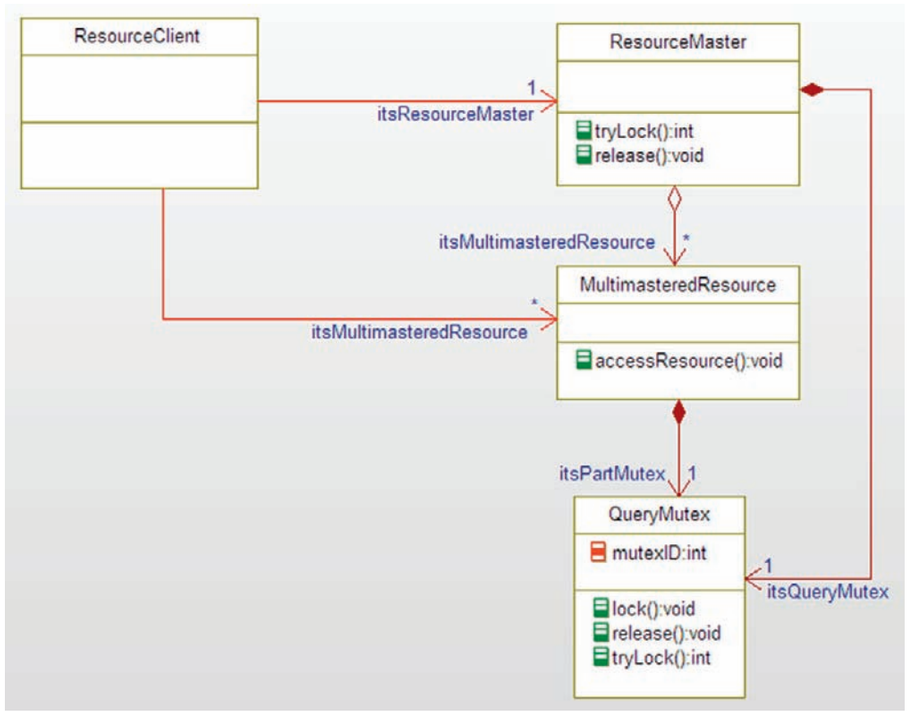


Figure 4-23: Simultaneous Locking Pattern general case

然而，该模式相较于临界区模式具有一个优势：如果更高优先级的任务不需要任何锁定资源，则允许它们运行。

4.8.2 问题

在高可靠性计算中，死锁问题非常严重，以至于许多系统在设计时采用特定机制来检测或避免死锁。如前所述，死锁发生在任务等待一个原则上永远无法满足的条件时。死锁的发生必须满足四个条件，只要否定其中任何一个条件即可避免死锁。同时锁定模式打破了条件#2，不允许任何任务在等待其他资源空闲时锁定资源。

4.8.3 模式结构

该 模式的结构如图 4-23所示。

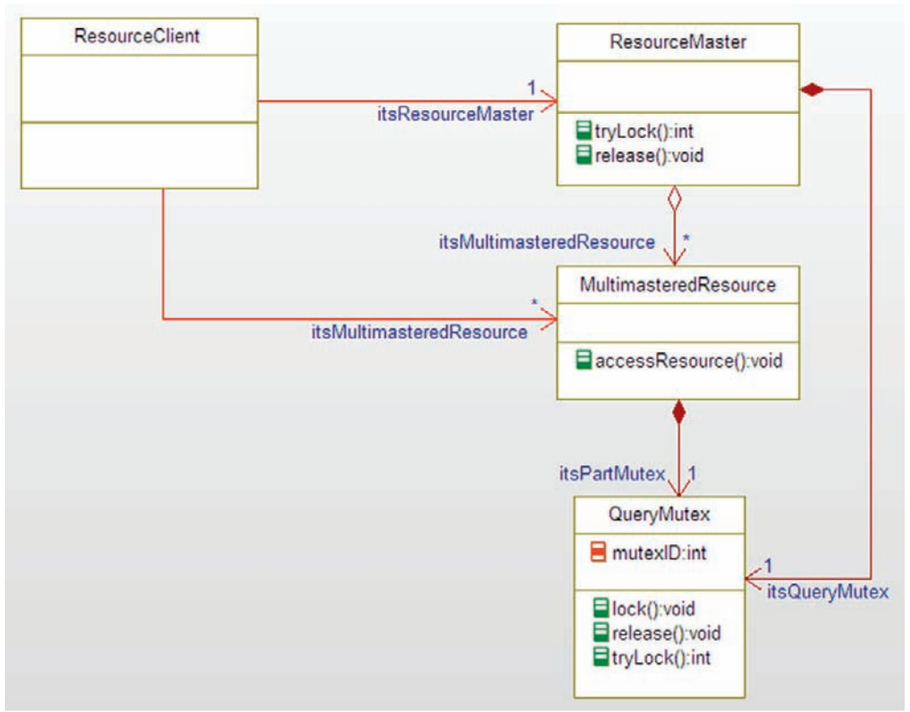


图4-23：同时锁定模式的一般情况

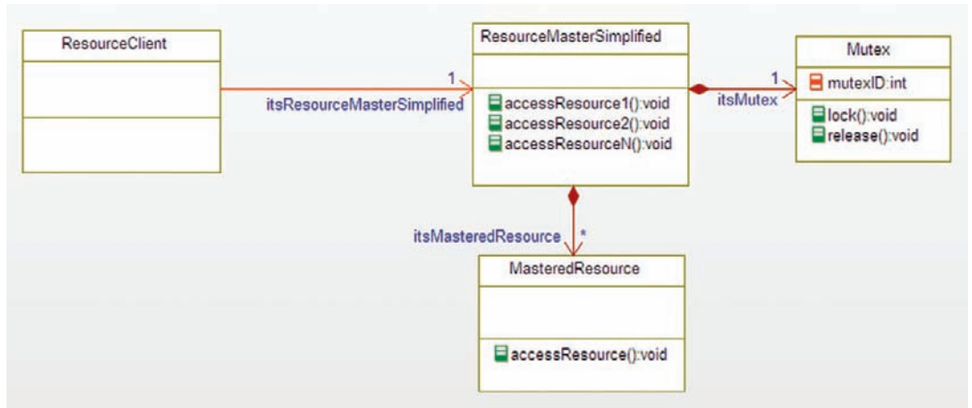


Figure 4-24: Simplified Simultaneous Locking Pattern in which each resource must be assigned to a single Resource Master

In general, a `MultimasteredResource` can be a part of any number of different sets of resources, where a separate instance of `ResourceMaster` manages one such set. This potential sharing of resources across multiple `ResourceMasters` complicates the situation. When this flexibility is not needed, the situation simplifies to that shown in Figure 4-24. While the difference may not appear to be great, it removes the requirement for a `tryLock()` function on the `Mutex` and it is algorithmically much simpler.

4.8.4 Collaboration Roles

This section describes the roles for this pattern.

4.8.4.1 MasteredResource

This class is an element of the Simplified Simultaneous Locking Pattern. It represents a shared resource that is owned by a single instance of ResourceMasterSimplified. If it can be determined that each `MasteredResource` can be uniquely assigned to a set controlled by a single instance of `ResourceMasterSimplified`, life is much simpler; in this case the resource does not need its own `Mutex` because it will execute under the auspices of the `Mutex` of the `ResourceMasterSimplified`. If the resource must be shared in multiple resource groups, then its becomes a `MultimasteredResource`, which is a part of the more general pattern.

4.8.4.2 MultimasteredResource

This class is an element of the general Simultaneous Locking Pattern. It is managed by multiple `ResourceMasters`, then it requires its own mutex (via its like `itsPartMutex`). The `Mutex` it uses must provide the `tryLock()` function so that it can be determined by the `ResourceMaster` whether all the attempted locks will succeed or fail.

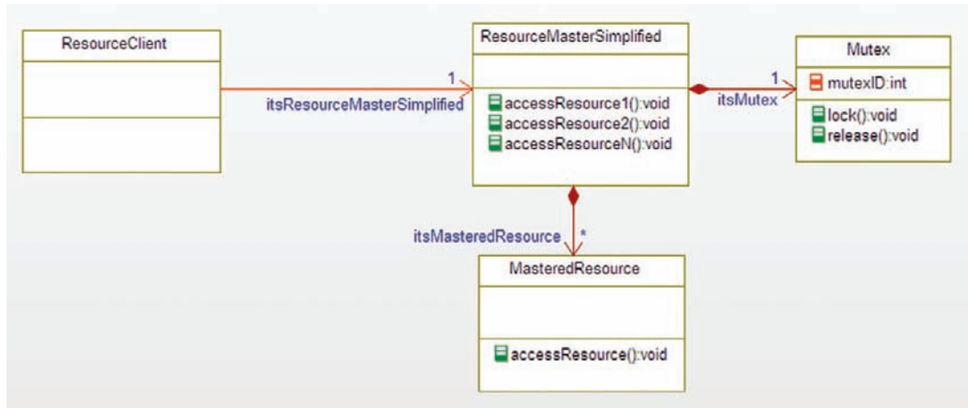


图4-24：简化同时锁定模式，其中每个资源必须分配给单个资源主控

一般来说，一个多主控资源可以属于任意数量的不同资源集合，其中每个集合由单独的 ResourceMaster 实例管理。这种跨多个 ResourceMaster 的潜在资源共享会使情况变得复杂。当不需要这种灵活性时，情况会简化为图4-24所示。虽然差异看起来可能不大，但它消除了对 Mutex 上 tryLock() 函数的需求，并且在算法上要简单得多。

4.8.4 协作角色

本节描述该模式的角色。

4.8.4.1 受控资源

该类是简化同时锁定模式的一个元素。它表示由单个ResourceMasterSimplified实例拥有的共享资源。如果可以确定每个受控资源 都能唯一地分配给由单个`ResourceMasterSimplified`实例控制的集合，那么事情就简单得多；在这种情况下，该资源不需要自己的互斥锁，因为它将在`ResourceMasterSimplified`的互斥锁 的保护下执行。如果该资源必须在多个资源组中共享，那么它就成为多主控资源，这是更通用模式的一部分。

4.8.4.2 多主控资源

该类是通用同时锁定模式的一个元素。它由多个资源主控管理，因此需要自己的互斥锁（通过类似`itsPartMutex`的方式）。它使用的互斥锁必须提供`tryLock()`函数，以便资源主控 能够判断所有尝试的加锁操作是会成功还是会失败。

4.8.4.3 Mutex

This element is a normal mutex semaphore as used in previous patterns; if offers `lock()` and `release()` functions that work in the way standard for a mutex semaphore.

4.8.4.4 QueryMutex

This element is a normal mutex semaphore except that it also provides a `tryLock()` function for the general pattern case. This function works like the normal `lock()` function except that it returns with an error code if the lock fails rather than blocking the current thread. In the specialized form of the pattern (in which a single resource is owned by a single `ResourceMaster`), then the `tryLock()` function isn't needed.

4.8.4.5 ResourceClient

This element is a client that wants to access a set of resources all at once to avoid potential deadlock. Although it has direct access (via its `itsMultimasteredResource` link) to the clients, it doesn't access them until it has successfully received a lock on the `ResourceMaster`. This requires discipline on the part of the developers of the `ResourceClients` to ensure that they never attempt to access any of the client services without first getting a lock, and once the client is done with the resources, it frees the `ResourceMaster`.

4.8.4.6 Resource Master

The `ResourceMaster` controls the lock of the entire set of resources. It is used in the general case of the pattern and it must return a lock if and only if it can lock all the resources. While it can be implemented with mutexes solely on the parts (instances of `ResourceMaster`) because it may be blocked multiple times while it attempts to lock the set of individual `MultimasteredResources`, it may have unacceptable execution times in practice. A solution, implemented here, is to use the `tryLock()` function of the mutex to test. If the `tryLock()` calls for the entire set of `MultimasteredResources` to all succeed, then the `ResourceMaster` itself is locked and the `ResourceClient` is allowed to use its link to the `MultimasteredResources`. If the `ResourceClient` cannot lock all of the `MultimasteredResources`, then the `ResourceMaster` unlocks all of the `MultimasteredResources` it locked (before it found one where it failed), and returns with an error code to the `ResourceClient`.

4.8.4.7 ResourceMasterSimplified

The Simplified Simultaneous Locking Pattern doesn't require the additional developer discipline required for the general case. The `ResourceClient` no longer has direct links to the `MasterResources` *per se*; instead, the `ResourceMasterSimplified` republishes all the services of its `MasteredResources` and delegates the functions directly to them. All of the functions share the same mutex.

4.8.4.3 互斥锁

该元素是前面模式中所使用的普通互斥信号量；它提供`lock()`和`release()`函数，其工作方式符合互斥信号量的标准。

4.8.4.4 查询互斥量

此元素是一个普通的互斥信号量，不同之处在于它还为通用模式场景提供了一个 tryLock() 函数。该函数的工作方式与普通的 lock() 函数类似，区别在于如果加锁失败，它会返回一个错误码，而不是阻塞当前线程。在该模式的专用形式中（即单个资源由单个 ResourceMaster 拥有），则不需要 tryLock() 函数。

4.8.4.5 资源客户端

此元素是一个客户端，它希望一次性访问一组资源以避免潜在的死锁。尽管它通过其 `itsMultimasteredResource` 链接可以直接访问这些客户端，但它在成功获得 `ResourceMaster` 上的锁之前不会访问它们。这要求 `ResourceClients` 的开发者遵守纪律，确保他们在未先获得锁的情况下绝不尝试访问任何客户端服务，并且一旦客户端用完这些资源，它就会释放 `ResourceMaster`。

4.8.4.6 资源主控

`ResourceMaster` 控制整个资源集合的锁。它用于该模式的通用情况，并且当且仅当它能够锁定所有资源时才必须返回一个锁。虽然它可以仅在各部分（`ResourceMaster`的实例）上使用互斥锁来实现，因为它在尝试锁定各个`MultimasteredResources`集合时可能会被多次阻塞，但在实践中它可能具有不可接受的执行时间。此处实现的一种解决方案是使用互斥锁的`tryLock()`函数进行测试。如果对整组`MultimasteredResources`的`tryLock()`调用全部成功，那么`ResourceMaster`自身被加锁，并且`ResourceClient`被允许使用其到`MultimasteredResources`的链接。如果`ResourceClient`无法锁定所有`MultimasteredResources`，那么`ResourceMaster`解锁它已锁定的所有`MultimasteredResources`（在它发现一个失败之前），并向`ResourceClient`返回一个错误码。

4.8.4.7 简化资源主控

简化同时锁定模式不需要一般情况所需的额外开发者纪律。`ResourceClient`不再直接链接到`MasterResources`本身；相反，`ResourceMasterSimplified`重新发布其`MasteredResources`的所有服务，并将函数直接委托给它们。所有函数共享同一个互斥锁。

4.8.5 Consequences

The Simultaneous Locking Pattern prevents deadlock by removing required condition #2 – some resources are locked while others are requested – by locking either all of the resources needed at once, or none of them. However, while the pattern removes the possibility of deadlock, it may increase the delay of the execution of other tasks. This delay may occur even in situations in which there is no actual resource conflict. The problem is far more pronounced for resources that are widely shared than for resources that are shared more narrowly. In addition, the pattern does not address priority inversion and in fact may make it worse, unless some priority inversion bounding pattern is deployed, such as the Guarded Call Pattern with priority inheritance. The general case for the pattern, shown in Figure 4-24, requires developer discipline in that they must avoid using the direct links to the `MultimasteredResources` until they have successfully been given a lock. In large project teams, this discipline can be ignored resulting in subtle and hard-to-identify errors. The simplified model, shown in Figure 4-24, doesn't have this problem because the `ResourceMasterSimplified` republishes the resource access methods.

Of course this pattern only applies when deadlock is a possibility, such as when multiple resources are shared with at least two in common by at least two threads.

4.8.6 Implementation Strategies

The implementation of this pattern should pose no difficulties other than the need to use, in the general case, a mutex semaphore that supports a `tryLock()` operation, and the need to ensure that the precondition of access (successful locking of the `MultimasteredResource`) is achieved before accessing the resource directly.

4.8.7 Related Patterns

Deadlock can be prevented by avoiding any of the required four conditions. The Critical Regions Pattern avoids by breaking condition #1 – mutual exclusion locking of resources. The Ordered Locking Pattern, described later in this chapter, prevents deadlock by breaking condition #4 – circular waiting condition.

Because this pattern doesn't address the concern of unbounded priority inversion, it can be mixed with the Guarded Call Pattern with priority inheritance.

4.8.8 Example

Figure 4-25 shows an example of the simplified form of Figure 4-24. In this case, the `SensorMaster` replicates the functions of the sensors (although the names are munged with the

4.8.5 结果

同时锁定模式通过移除所需条件#2——一些资源被锁定而另一些被请求——来防止死锁，方法是同时锁定所有需要的资源，或者一个都不锁定。然而，虽然该模式消除了死锁的可能性，但它可能增加其他任务执行的延迟。即使在没有实际资源冲突的情况下，这种延迟也可能发生。对于广泛共享的资源，这个问题比共享范围较窄的资源要明显得多。此外，该模式没有解决优先级反转问题，实际上可能使其更糟，除非部署了某种优先级反转限制模式，例如带优先级继承的受保护调用模式。该模式的一般情况如图4-24所示，要求开发者遵守纪律，即他们必须避免使用对MultimasteredResources的直接链接，直到成功获得锁。在大型项目团队中，这种纪律可能被忽视，导致微妙且难以识别的错误。简化模型如图4-24所示，没有这个问题，因为ResourceMasterSimplified重新发布了资源访问方法。

当然，此模式仅在死锁可能发生时适用，例如当多个资源被共享，且至少两个线程共享至少两个共同资源时。

4.8.6 实现策略

此模式的实现除了一般情况下需要使用支持 `tryLock()` 操作的互斥信号量，以及需要确保在直接访问资源之前满足访问的前置条件（成功锁定 `MultimasteredResource`）之外，不应带来其他困难。

4.8.7 相关模式

可以通过避免四个必要条件中的任意一个来预防死锁。临界区模式通过打破条件#1——资源的互斥锁定来避免死锁。本章后面介绍的有序锁定模式通过打破条件#4——循环等待条件来预防死锁。

由于此模式不解决无界优先级反转的问题，因此它可以与带优先级继承的受保护调用模式结合使用。

4.8.8 示例

图 4-25 展示了图 4-24 简化形式的一个示例。在此情况下，`SensorMaster` 复制了传感器的函数（尽管名称与服务器名称进行了拼接以消除服务歧义）。

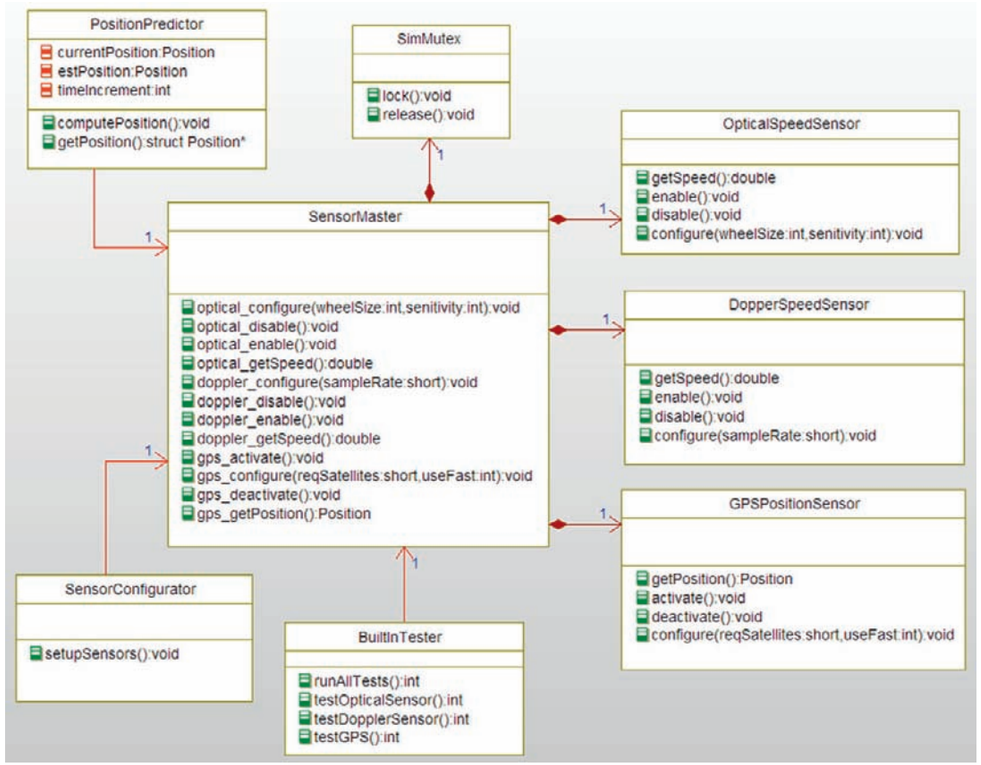


Figure 4-25: Simultaneous Locking Pattern example

图4-25：同时锁定模式示例

names of the servers to disambiguate services). When a client (such as the `PositionPredictor`) invokes one of the services, the `SensorMaster` locks the mutex and releases it before returning to the client. The `SensorMaster` owns a mutex semaphore for this purpose.

当客户端（例如位置预测器）调用其中一个服务时，`SensorMaster` 会加锁互斥锁，并在返回客户端之前释放它。`SensorMaster` 为此拥有一个互斥信号量。

```
#ifndef SensorMaster_H
#define SensorMaster_H
#include "Position.h"
struct DopplerSpeedSensor;
struct GPSPositionSensor;
struct OpticalSpeedSensor;
struct SimMutex;

typedef struct SensorMaster SensorMaster;
struct SensorMaster {
    struct DopplerSpeedSensor*  itsDopplerSpeedSensor;
    struct GPSPositionSensor*  itsGPSPositionSensor;
```

```
#ifndef SensorMaster_H #define
SensorMaster_H #include "
Position.h" 结构体 多普勒速度传感器;
结构体 GPSPositionSensor; 结构体
OpticalSpeedSensor; 结构体
SimMutex;


typedef struct SensorMaster SensorMaster; struct SensorMaster {
struct DopplerSpeedSensor* itsDopplerSpeedSensor; struct
GPSPositionSensor* itsGPSPositionSensor;
```

```
    struct OpticalSpeedSensor*  itsOpticalSpeedSensor;
    struct SimMutex*  itsSimMutex;
};

/*  Constructors and destructors:*/

void SensorMaster_Init(SensorMaster*  const me);
void SensorMaster_Cleanup(SensorMaster*  const me);

/*  Operations  */
void SensorMaster_doppler_configure(SensorMaster*  const me, short sampleRate);
void SensorMaster_doppler_disable(SensorMaster*  const me);
void SensorMaster_doppler_enable(SensorMaster*  const me);
double SensorMaster_doppler_getSpeed(SensorMaster*  const me);
void SensorMaster_gps_activate(SensorMaster*  const me);
void SensorMaster_gps_configure(SensorMaster*  const me, short
reqSatellites, int useFast);

void SensorMaster_gps_deactivate(SensorMaster*  const me);
struct Position SensorMaster_gps_getPosition(SensorMaster*  const me);
void SensorMaster_optical_configure(SensorMaster*  const me, int
wheelSize, int sensitivity);
void SensorMaster_optical_disable(SensorMaster*  const me);
void SensorMaster_optical_enable(SensorMaster*  const me);
double SensorMaster_optical_getSpeed(SensorMaster*  const me);

struct DopplerSpeedSensor*
SensorMaster_getItsDopplerSpeedSensor(const SensorMaster*  const me);
void SensorMaster_setItsDopplerSpeedSensor(SensorMaster*  const
me, struct DopplerSpeedSensor*  p_DopplerSpeedSensor);

struct GPSPositionSensor*
SensorMaster_getItsGPSPositionSensor(const SensorMaster*  const me);

void SensorMaster_setItsGPSPositionSensor(SensorMaster*  const me,
struct GPSPositionSensor*  p_GPSPositionSensor);

struct OpticalSpeedSensor*
SensorMaster_getItsOpticalSpeedSensor(const SensorMaster*  const me);
void SensorMaster_setItsOpticalSpeedSensor(SensorMaster*  const me,
struct OpticalSpeedSensor*  p_OpticalSpeedSensor);

struct SimMutex*  SensorMaster_getItsSimMutex(const SensorMaster*  const me);

void SensorMaster_setItsSimMutex(SensorMaster*  const me, struct
SimMutex*  p_SimMutex);

SensorMaster  *  SensorMaster_Create(void);
```



```
    struct OpticalSpeedSensor* itsOpticalSpeedSensor;
    struct SimMutex* itsSimMutex;
};

/* 构造函数与析构函数：*/

void SensorMaster_Init(SensorMaster* const me); void
SensorMaster_Cleanup(SensorMaster* const me);

/* 操作 */
void SensorMaster_doppler_configure(SensorMaster* const me, short 采样率); void
SensorMaster_doppler_disable(SensorMaster* const me); void SensorMaster_doppler_
enable(SensorMaster* const me); double SensorMaster_doppler_getSpeed(SensorMaster*
const me); void SensorMaster_gps_activate(SensorMaster* const me); void SensorMaster_gps_
configure(SensorMaster* const me, short reqSatellites, int useFast);


void SensorMaster_gps_deactivate(SensorMaster* const me); 结构体 Position
SensorMaster_gps_getPosition(SensorMaster* const me); void SensorMaster_optical_
configure(SensorMaster* const me, int wheelSize, int sensitivity); void Sensor
Master_optical_disable(SensorMaster* const me); void SensorMaster_optical_
enable(SensorMaster* const me); double SensorMaster_optical_
getSpeed(SensorMaster* const me);

struct DopplerSpeedSensor*
SensorMaster_getItsDopplerSpeedSensor(const SensorMaster* const me); void
SensorMaster_setItsDopplerSpeedSensor(SensorMaster* const me, struct
DopplerSpeedSensor* p_DopplerSpeedSensor);

struct GPSPositionSensor*
SensorMaster_getItsGPSPositionSensor(const SensorMaster* const me          );

void SensorMaster_setItsGPSPositionSensor(SensorMaster* const me, struct
GPSPositionSensor* p_GPSPositionSensor);

struct OpticalSpeedSensor* SensorMaster_getItsOpticalSpeedSensor(const
SensorMaster* const me); void SensorMaster_setItsOpticalSpeedSensor(SensorMaster*
const me, struct OpticalSpeedSensor* p_OpticalSpeedSensor);


struct SimMutex* SensorMaster_getItsSimMutex(const SensorMaster* const me);

void SensorMaster_setItsSimMutex(SensorMaster* const me, struct
SimMutex* p_SimMutex);

SensorMaster * SensorMaster_Create(void);
```

```
void SensorMaster_Destroy(SensorMaster*  const me);

#endif
```

Code Listing 4-22: SensorMaster.h

```
#include "SensorMaster.h"
#include "DopplerSpeedSensor.h"
#include "GPSPositionSensor.h"
#include "OpticalSpeedSensor.h"
#include "SimMutex.h"

static void cleanUpRelations(SensorMaster*  const me);

void SensorMaster_Init(SensorMaster*  const me) {
    me->itsDopplerSpeedSensor  =  NULL;
    me->itsGPSPositionSensor  =  NULL;
    me->itsOpticalSpeedSensor  =  NULL;
    me->itsSimMutex  =  NULL;
}

void SensorMaster_Cleanup(SensorMaster*  const me) {
    cleanUpRelations(me);
}

void SensorMaster_doppler_configure(SensorMaster*  const me, short sampleRate) {
    SimMutex_lock(me->itsSimMutex);
    DopplerSpeedSensor_configure(me->itsDopplerSpeedSensor,sampleRate);
    SimMutex_release(me->itsSimMutex);
}

void SensorMaster_doppler_disable(SensorMaster*  const me) {
    SimMutex_lock(me->itsSimMutex);
    DopplerSpeedSensor_disable(me->itsDopplerSpeedSensor);
    SimMutex_release(me->itsSimMutex);
}

void SensorMaster_doppler_enable(SensorMaster*  const me) {
    SimMutex_lock(me->itsSimMutex);
    DopplerSpeedSensor_enable(me->itsDopplerSpeedSensor);
    SimMutex_release(me->itsSimMutex);
}

double SensorMaster_doppler_getSpeed(SensorMaster*  const me) {
    double speed;
    SimMutex_lock(me->itsSimMutex);
    speed  =  DopplerSpeedSensor_getSpeed(me->itsDopplerSpeedSensor);
```



```
void SensorMaster_Destroy(SensorMaster* const me);

#endif
```

代码清单4-22：SensorMaster.h

```
#include "SensorMaster.h" #include "
DopplerSpeedSensor.h" #include "
GPSPositionSensor.h" #include "
OpticalSpeedSensor.h" #include "
SimMutex.h"

static void cleanUpRelations(SensorMaster* const me); void SensorMaster_Init(SensorMaster*
const me) { me->itsDopplerSpeedSensor  =  空; me->itsGPSPositionSensor  =  空; me->
itsOpticalSpeedSensor  =  空; me->itsSimMutex  =  空; } void SensorMaster_
Cleanup(SensorMaster* const me) { cleanUpRelations(me); } void SensorMaster_doppler_
configure(SensorMaster* const me, short sampleRate) { SimMutex_加锁(me->itsSimMutex);
DopplerSpeedSensor_configure(me->itsDopplerSpeedSensor,sampleRate); SimMutex_释放(me->
itsSimMutex); } void SensorMaster_doppler_disable(SensorMaster* const me) { SimMutex_加锁
(me->itsSimMutex); DopplerSpeedSensor_disable(me->itsDopplerSpeedSensor); SimMutex_释放
(me->itsSimMutex); } void SensorMaster_doppler_enable(SensorMaster* const me) { SimMutex_
加锁(me->itsSimMutex); DopplerSpeedSensor_enable(me->itsDopplerSpeedSensor); SimMutex_
释放(me->itsSimMutex); } double SensorMaster_doppler_getSpeed(SensorMaster* const me) {
double speed; SimMutex_加锁(me->itsSimMutex); speed  =  DopplerSpeedSensor_
getSpeed(me->itsDopplerSpeedSensor);
```

```
    SimMutex_release(me->itsSimMutex);
    return speed;
}

void SensorMaster_gps_activate(SensorMaster*  const me){
    SimMutex_lock(me->itsSimMutex);
    GPSPositionSensor_activate(me->itsGPSPositionSensor);
    SimMutex_release(me->itsSimMutex);
}

void SensorMaster_gps_configure(SensorMaster*  const me, short reqSatellites, int use-
Fast){
    SimMutex_lock(me->itsSimMutex);
    GPSPositionSensor_configure(me->itsGPSPositionSensor,reqSatellites,useFast);
    SimMutex_release(me->itsSimMutex);
}

void SensorMaster_gps_deactivate(SensorMaster*  const me){
    SimMutex_lock(me->itsSimMutex);
    GPSPositionSensor_deactivate(me->itsGPSPositionSensor);
    SimMutex_release(me->itsSimMutex);
}

struct Position SensorMaster_gps_getPosition(SensorMaster*  const me){
    Position p;
    SimMutex_lock(me->itsSimMutex);
    p  =  GPSPositionSensor_getPosition(me->itsGPSPositionSensor);
    SimMutex_release(me->itsSimMutex);
    return p;
}

void SensorMaster_optical_configure(SensorMaster*  const me, int wheelSize,
int sensitivity){
    SimMutex_lock(me->itsSimMutex);
    OpticalSpeedSensor_configure(me->itsOpticalSpeedSensor,wheelSize, sensitivity);
    SimMutex_release(me->itsSimMutex);
}

void SensorMaster_optical_disable(SensorMaster*  const me){
    SimMutex_lock(me->itsSimMutex);
    OpticalSpeedSensor_disable(me->itsOpticalSpeedSensor);
    SimMutex_release(me->itsSimMutex);
}

void SensorMaster_optical_enable(SensorMaster*  const me){
    SimMutex_lock(me->itsSimMutex);
    OpticalSpeedSensor_enable(me->itsOpticalSpeedSensor);
    SimMutex_release(me->itsSimMutex);
}
```


SimMutex_release(me->itsSimMutex); return speed; } void SensorMaster_gps_activate(SensorMaster* const me) { SimMutex_lock(me->itsSimMutex); GPSPositionSensor_activate(me->itsGPSPositionSensor); SimMutex_release(me->itsSimMutex); } void SensorMaster_gps_configure(SensorMaster* const me, short reqSatellites, int use- Fast) { SimMutex_lock(me->itsSimMutex); GPSPositionSensor_configure(me->itsGPSPositionSensor,reqSatellites,useFast); SimMutex_release(me->itsSimMutex); } void SensorMaster_gps_deactivate(SensorMaster* const me) { SimMutex_lock(me->itsSimMutex); GPSPositionSensor_deactivate(me->itsGPSPositionSensor); SimMutex_release(me->itsSimMutex); } struct Position SensorMaster_gps_getPosition(SensorMaster* const me) { Position p; SimMutex_lock(me->itsSimMutex); p = GPSPositionSensor_getPosition(me->itsGPSPositionSensor); SimMutex_release(me->itsSimMutex); return p; } void SensorMaster_optical_configure(SensorMaster* const me, int wheelSize, int sensitivity) { SimMutex_lock(me->itsSimMutex); OpticalSpeedSensor_configure(me->itsOpticalSpeedSensor,wheelSize, sensitivity); SimMutex_release(me->itsSimMutex); } void SensorMaster_optical_disable(SensorMaster* const me) { SimMutex_lock(me->itsSimMutex); OpticalSpeedSensor_disable(me->itsOpticalSpeedSensor); SimMutex_release(me->itsSimMutex); } void SensorMaster_optical_enable(SensorMaster* const me) { SimMutex_lock(me->itsSimMutex); OpticalSpeedSensor_enable(me->itsOpticalSpeedSensor); SimMutex_release(me->itsSimMutex); }

```
double SensorMaster_optical_getSpeed(SensorMaster* const me) {
    double speed;
    SimMutex_lock(me->itsSimMutex);
    speed = OpticalSpeedSensor_getSpeed(me->itsOpticalSpeedSensor);
    SimMutex_release(me->itsSimMutex);
    return speed;
}

struct DopplerSpeedSensor* SensorMaster_getItsDopplerSpeedSensor(const
SensorMaster* const me) {
    return (struct DopplerSpeedSensor*)me->itsDopplerSpeedSensor;
}

void SensorMaster_setItsDopplerSpeedSensor(SensorMaster* const me, struct Doppler-
SpeedSensor* p_DopplerSpeedSensor) {
    me->itsDopplerSpeedSensor = p_DopplerSpeedSensor;
}

struct GPSPositionSensor* SensorMaster_getItsGPSPositionSensor(const
SensorMaster* const me) {
    return (struct GPSPositionSensor*)me->itsGPSPositionSensor;
}

void SensorMaster_setItsGPSPositionSensor(SensorMaster* const me,
struct GPSPositionSensor* p_GPSPositionSensor) {
    me->itsGPSPositionSensor = p_GPSPositionSensor;
}

struct OpticalSpeedSensor* SensorMaster_getItsOpticalSpeedSensor(const
SensorMaster* const me) {
    return (struct OpticalSpeedSensor*)me->itsOpticalSpeedSensor;
}

void SensorMaster_setItsOpticalSpeedSensor(SensorMaster* const me, struct Optical-
SpeedSensor* p_OpticalSpeedSensor) {
    me->itsOpticalSpeedSensor = p_OpticalSpeedSensor;
}

struct SimMutex* SensorMaster_getItsSimMutex(const SensorMaster* const me) {
    return (struct SimMutex*)me->itsSimMutex;
}

void SensorMaster_setItsSimMutex(SensorMaster* const me, struct
SimMutex* p_SimMutex) {
    me->itsSimMutex = p_SimMutex;
}

SensorMaster * SensorMaster_Create(void) {
```


double SensorMaster_optical_getSpeed(SensorMaster* const me) { double speed; SimMutex_lock(me->itsSimMutex); speed = OpticalSpeedSensor_getSpeed(me->itsOpticalSpeedSensor); SimMutex_release(me->itsSimMutex); return speed; } struct DopplerSpeedSensor* SensorMaster_getItsDopplerSpeedSensor(const SensorMaster* const me) { return (struct DopplerSpeedSensor*) me->itsDopplerSpeedSensor; } void SensorMaster_setItsDopplerSpeedSensor(SensorMaster* const me, struct Doppler- SpeedSensor* p_DopplerSpeedSensor) { me->itsDopplerSpeedSensor = p_DopplerSpeedSensor; } struct GPSPositionSensor* SensorMaster_getItsGPSPositionSensor(const SensorMaster* const me) { return (struct GPSPositionSensor*)me->itsGPSPositionSensor; } void SensorMaster_setItsGPSPositionSensor(SensorMaster* const me, struct GPSPositionSensor* p_GPSPositionSensor) { me->itsGPSPositionSensor = p_GPSPositionSensor; } struct OpticalSpeedSensor* SensorMaster_getItsOpticalSpeedSensor(const SensorMaster* const me) { return (struct OpticalSpeedSensor*)me->itsOpticalSpeedSensor; } void SensorMaster_setItsOpticalSpeedSensor(SensorMaster* const me, struct Optical- SpeedSensor* p_OpticalSpeedSensor) { me->itsOpticalSpeedSensor = p_OpticalSpeedSensor; } struct SimMutex* SensorMaster_getItsSimMutex(const SensorMaster* const me) { return (struct SimMutex*)me->itsSimMutex; } void SensorMaster_setItsSimMutex(SensorMaster* const me, struct SimMutex* p_SimMutex) { me->itsSimMutex = p_SimMutex; } SensorMaster * SensorMaster_Create(void) {

```
    SensorMaster* me = (SensorMaster *)
malloc(sizeof(SensorMaster));
    if(me!=NULL)
      SensorMaster_Init(me);
    return me;
}

void SensorMaster_Destroy(SensorMaster* const me) {
    if(me!=NULL)
        SensorMaster_Cleanup(me);
    free(me);
}

static void cleanUpRelations(SensorMaster* const me) {
    if(me->itsDopplerSpeedSensor != NULL)
           me->itsDopplerSpeedSensor = NULL;
    if(me->itsGPSPositionSensor != NULL)
           me->itsGPSPositionSensor = NULL;
    if(me->itsOpticalSpeedSensor != NULL)
           me->itsOpticalSpeedSensor = NULL;
    if(me->itsSimMutex != NULL)
           me->itsSimMutex = NULL;
}
```

Code Listing 4-23: SensorMaster.c

4.9 Ordered Locking

The Ordered Locking Pattern is another way to ensure that deadlock cannot occur, this time by preventing condition 4 (circular waiting) from occurring. It does this by ordering the resources and requiring that they always be locked by a client in that specified order. If this is religiously enforced, then no circular waiting condition can ever occur.

4.9.1 Abstract

The Ordered Locking Pattern eliminates deadlock by ordering resources and enforcing a policy in which resources must be allocated only in a specific order. For monadic (that is, functions that can be used independently of others) operations, the client can invoke them without special consideration and the called function will lock and release the resource internally. For holding locks over the long term – accessing dyadic (that is, functions that require the caller to explicitly lock and unlock them) functions – the client must explicitly lock and release the resources, rather than doing it implicitly by merely invoking a service on a resource. This means that the potential for neglecting to unlock the resource exists.


```
    SensorMaster* me = (SensorMaster *)
malloc(sizeof(SensorMaster)); if(me!=NULL) SensorMaster_
Init(me); return me; } void SensorMaster_
Destroy(SensorMaster* const me) { if(me!=NULL) SensorMaster_
Cleanup(me); free(me); } static void
cleanUpRelations(SensorMaster* const me) { if(me->
itsDopplerSpeedSensor != NULL) me->itsDopplerSpeedSensor
= NULL; if(me->itsGPSPositionSensor != NULL) me->
itsGPSPositionSensor = NULL; if(me->itsOpticalSpeedSensor !
= NULL) me->itsOpticalSpeedSensor = NULL; if(me->
itsSimMutex != NULL) me->itsSimMutex = NULL; }
```

代码清单4-23：SensorMaster.c

4.9 有序锁定

有序锁定模式是另一种确保死锁不会发生的方法，这次是通过防止条件4（循环等待）的发生来实现的。它通过对资源进行排序，并要求客户端始终按照指定的顺序对资源加锁来实现这一点。如果严格遵守这一规则，那么循环等待条件就永远不会发生。

4.9.1 摘要

有序锁定模式通过对资源进行排序并强制执行一种策略来消除死锁，该策略要求资源只能按照特定顺序分配。对于一元操作（即可以独立于其他函数使用的函数），客户端无需特别考虑即可调用它们，被调用的函数将在内部加锁和释放资源。对于长期持有锁的情况——访问二元函数（即需要调用者显式加锁和解锁的函数）——客户端必须显式地加锁和释放资源，而不是仅通过对资源调用服务来隐式完成。这意味着存在忘记解锁资源的潜在风险。

4.9.2 Problem

The Ordered Locking Pattern solely addresses the problem of deadlock elimination, as does the previous Simultaneous Locking Pattern.

4.9.3 Pattern Structure

Figure 4-26 shows the pattern structure for the Ordered Locking Pattern.

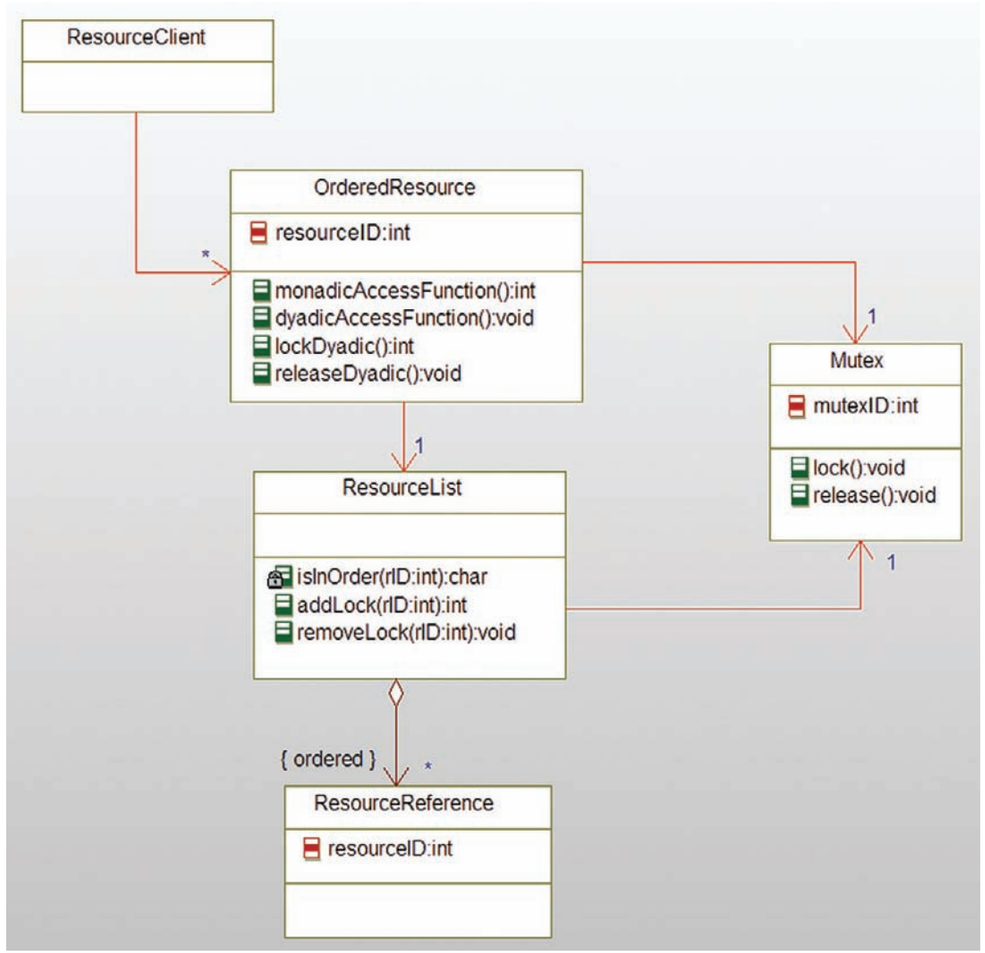


Figure 4-26: Ordered Locking Pattern

4.9.2 问题

有序锁定模式仅解决死锁消除问题，与之前的同时锁定模式相同。

4.9.3 模式结构

图 4-26 展示了 有序加锁 模式的　　结构。

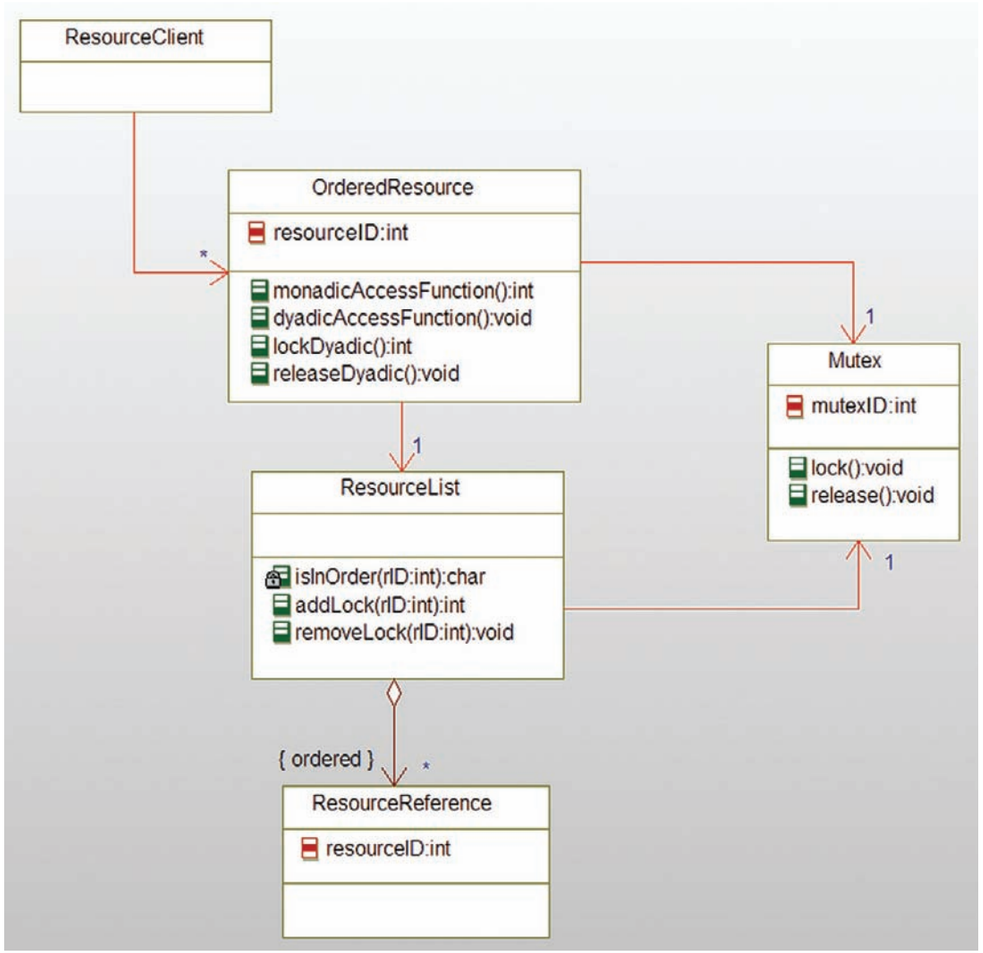


图4-26：有序锁定模式

4.9.4 Collaboration Roles

This section describes the roles for this pattern.

4.9.4.1 Mutex

The `Mutex` is used by two other pattern elements: `OrderedResource` and `ResourceList`. Of course, this means that each of these elements will use different instances of the `Mutex` class. As with the other patterns in this chapter, the `Mutex` class provides two basic services. The `lock()` function locks the resource (if currently unlocked) or blocks the caller (if already locked). The `release()` function removes the lock and allows any blocked callers to continue.

4.9.4.2 OrderedResource

The `OrderedResource` is really the heart of the pattern. It has a `resourceID` attribute. This is a unique ID associated with each resource instance and, along with the `ResourceList`, this class enforces the ordered locking rule:

A resource may only be locked if its `resourceID` *is greater than the largest* `resourceID` *of any locked resource.*

The `orderedResource` may have different kinds of functions. The first kind, called monadic functions, internally lock the mutex, perform their magic, unlock the `Mutex`, and then return. The magic inside the function may call functions of other resources, hence the need to use the `ResourceList` to prevent possible deadlocks. Figure 4-27 shows the flow of control for such a `monadicAccessFunction`.

The other kind of access function is dyadic, called so because it requires a separate pair of calls to manage the lock on the resource. That is, before a dyadic access function is called, the `ResourceClient` should first call `lockDyadic()` and after the `ResourceClient` is done with the resource, it should call `releaseDyadic()`. Because of the ordered locking enforcement, the `ResourceClient` can hold any number of resources in the locked state while it performs its work without fear of causing deadlock. However, the `ResourceClient` must lock them in the order of their `resourceIDs` or errors will be returned and access will be denied. Figure 4-28 shows the flowchart for locking the resource and Figure 4-29 shows the flowchart for its release.

An interesting question arises as to how to assign the `resourceIDs`. In many ways, the best approach is to assign them at design time so that the constructor `OrderedResource_Init()` simply initializes the value. This approach requires design-time analysis to ensure that the `resourceIDs` are unique but also provides an opportunity to use *a priori* information to identify an optimal assignment of `resourceIDs` depending on the usage patterns of the various resources. Design-time analysis may discover `resourceID` assignments that lessen blocking due to deadlock prevention mechanisms (that is, if we assign the `resourceIDs` in *this* order, we minimize the occurrence of resource lock rejection due to potential deadlocking).

4.9.4 协作角色

本节描述该模式的各个角色。

4.9.4.1 互斥锁

互斥锁 被另外两个模式元素使用：有序资源 和资源列表。当然，这意味着这些元素中的每一个都将使用互斥锁 类的不同实例。与本章中的其他模式一样，互斥锁 类提供两个基本服务。`lock()`函数对资源加锁（如果当前未锁定）或阻塞调用者（如果已锁定）。`release()` 函数移除锁并允许任何被阻塞的调用者继续执行。

4.9.4.2 有序资源

有序资源 实际上是该模式的核心。它有一个`resourceID` 属性。这是一个与每个资源实例关联的唯一ID，并且与资源列表一起，该类强制执行有序锁定规则：

只有当资源的resourceID大于任何已加锁资源的最大resourceID时，才能对该资源加锁。

有序资源可以具有不同种类的函数。第一种称为一元函数，它在内部对互斥锁加锁、执行其操作、解锁互斥锁，然后返回。函数内部的操作可能会调用其他资源的函数，因此需要使用资源列表来防止可能的死锁。图4-27展示了此类一元访问函数的控制流。

另一种访问函数是二元的，之所以这样称呼，是因为它需要一对独立的调用来管理资源上的锁。也就是说，在调用二元访问函数之前，资源客户端应首先调用lockDyadic()，并且在资源客户端完成对资源的使用后，应调用releaseDyadic()。由于有序锁定强制执行机制，资源客户端可以在执行其工作时以锁定状态持有任意数量的资源，而不必担心导致死锁。但是，资源客户端必须按照其resourceID的顺序对它们加锁，否则将返回错误并拒绝访问。图4-28展示了锁定资源的流程图，图4-29展示了释放资源的流程图。

一个有趣的问题出现了：如何分配 `resourceIDs`。在许多方面，最好的方法是在设计时分配它们，以便构造函数 `OrderedResource_Init()` 简单地初始化该值。这种方法需要设计时分析来确保 `resourceIDs` 是唯一的，但也提供了一个机会，利用先验信息来根据各种资源的使用模式确定 `resourceIDs` 的最优分配。设计时分析可能会发现 `resourceID` 分配方案，从而减少因死锁预防机制而导致的阻塞（也就是说，如果我们按此顺序分配 `resourceIDs`，就能最大限度地减少因潜在死锁而导致的资源锁拒绝的发生）。

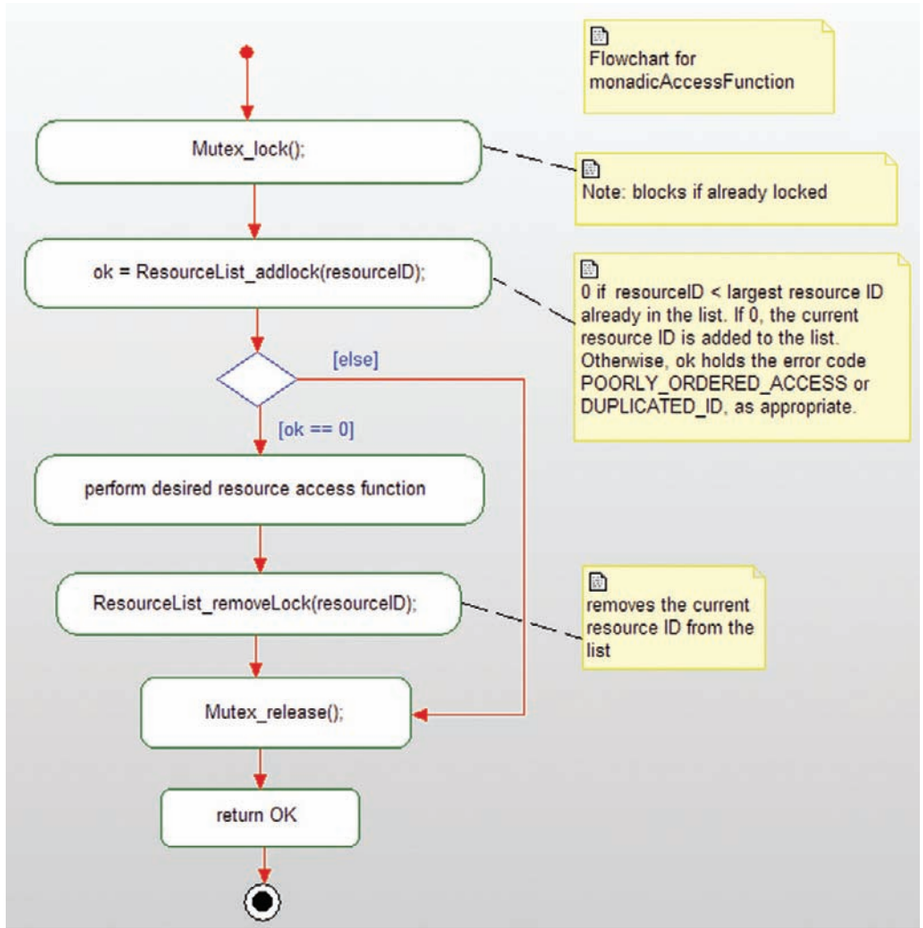


Figure 4-27: Flowchart for monadicAccessFunction

On the other hand, this analysis may be very difficult for larger systems. However, dynamically assigning the `resourceIDs` with a `resourceID` server is problematic as it may lead to pathological conditions under which the order in which a set of resources was called in one case leads to the permanent failure of another `ResourceClient`. For this reason, I recommend design-time assignment of `resourceIDs`.

4.9.4.3 ResourceClient

The `ResourceClient` represents the set of software elements that want to invoke the services of the `OrderedResources`. The `ResourceClient` doesn't need to know

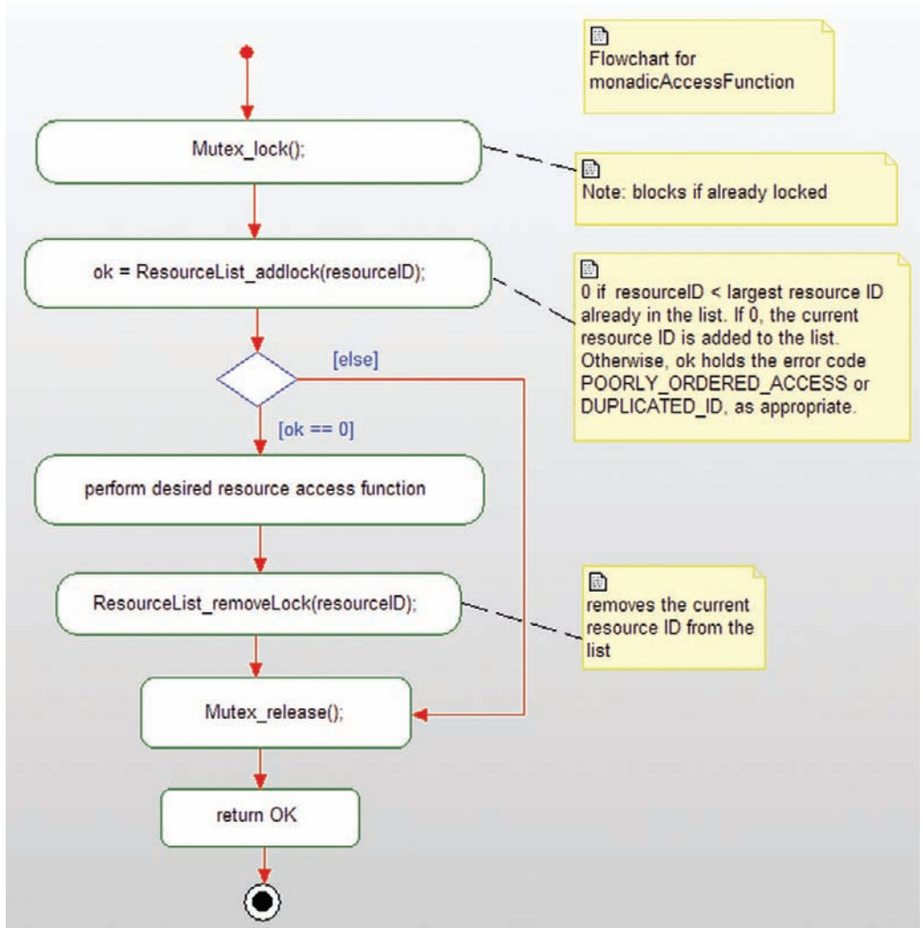


图4-27：一元访问函数流程图

另一方面，对于较大的系统，这种分析可能非常困难。然而，使用 `resourceID` 服务器动态分配 `resourceIDs` 是有问题的，因为它可能导致病态情况，即一组资源在一种情况下被调用的顺序会导致另一个 `ResourceClient` 的永久失败。因此，我建议在设计时分配 `resourceIDs`。

4.9.4.3 资源客户端

资源客户端 表示希望调用有序资源服务的一组软件元素。资源客户端 不需要知道

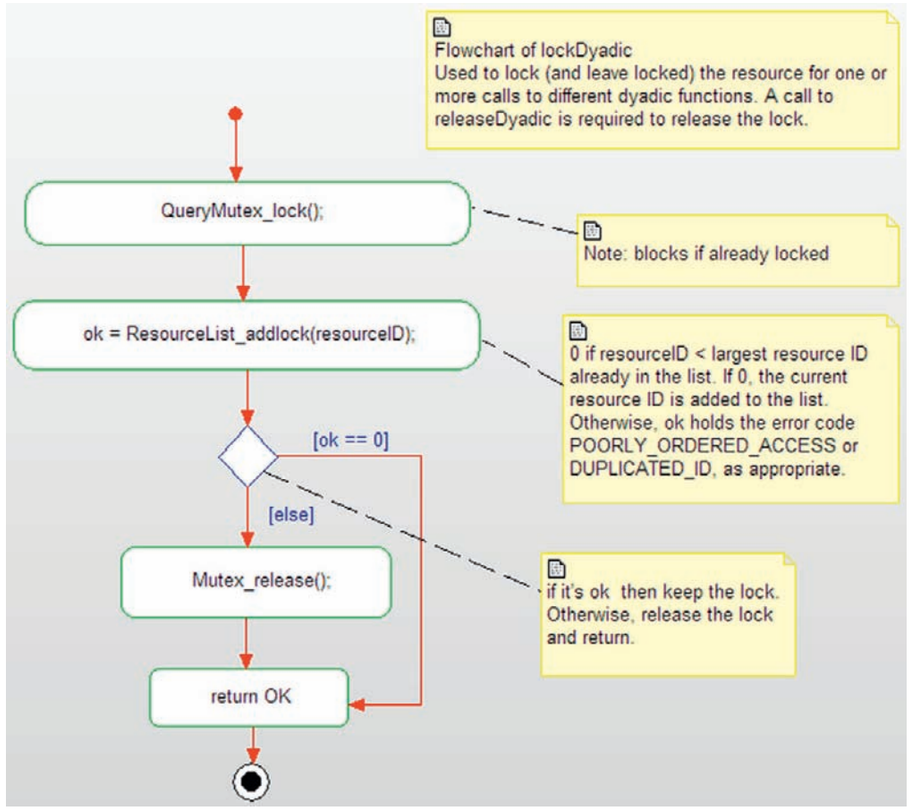


Figure 4-28: Flowchart for lockDyadic

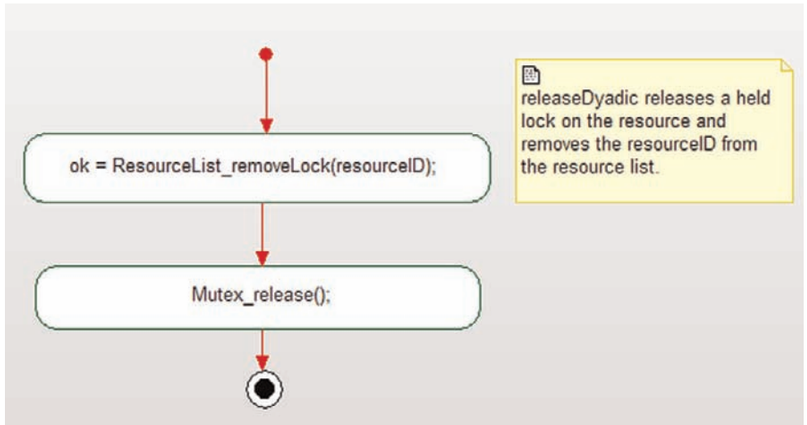


Figure 4-29: Flowchart for releaseDyadic

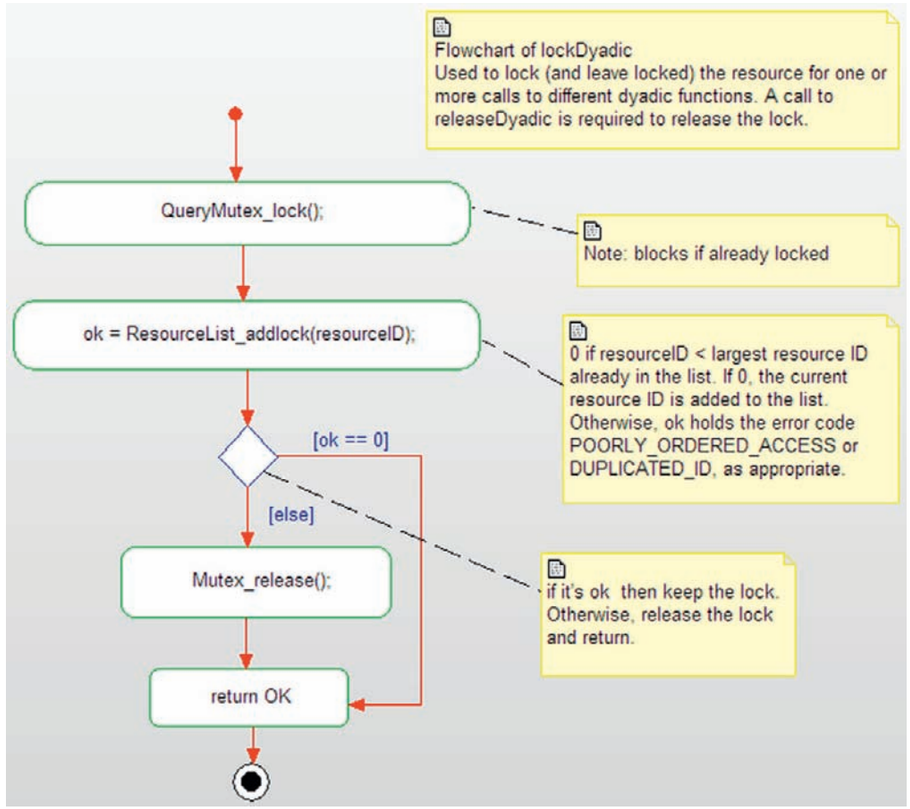


图4-28：lockDyadic 流程图

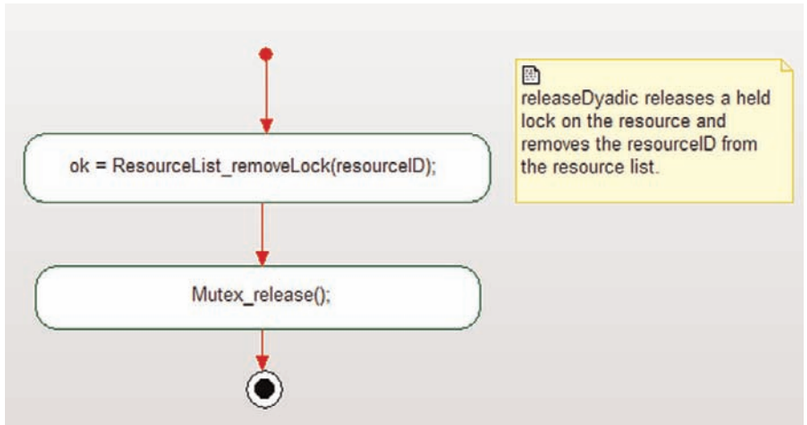


图4-29：releaseDyadic 流程图

anything about the `resourceIDs` *per se*, but as a part of testing, they need to be designed so that they only lock the resources in the correct order. For monadic functions, they don't need to do anything special – they will just invoke the `monadicAccessFunction()`. For dyadic access functions, the `ResourceClient` must explicitly lock the resource first, invoke the needed services, and then call unlock. The use of the dyadic form is more flexible – since the `ResourceClient` can hold the locks on any number of resources as long as it needs them – but requires additional discipline on the part of the developer.

4.9.4.4 ResourceList

This element provides two important functions. `addLock()`returns 0 ("`ok`") if the passed `resourceID` is greater than that of the largest currently locked resource. If the passed `referenceID` is *less than* the largest resource ID, then `POORLY_ORDERED_ACCESS` is returned; if the passed `referenceID` equals it, then `DUPLICATE_IDS` is returned. (Note that if the same resource is accessed while it is locked, its mutex will block before the call is made to `addLock()`, so the only way this can occur is if the `resourceID` is nonunique). It is also important to note that there must be only one `ResourceList` for the application, giving the `resourceIDs` an effective global scope.

4.9.4.5 ResourceReference

This is just the `resourceID` held in an ordered list; most likely an array. It is insufficient to just keep the largest value only since many resources might be locked at any one time.

4.9.5 Consequences

This pattern effectively removes the possibility of resource-based deadlocks by removing the possibility of condition 4 – circular waiting. For the algorithm to work, any ordering of `OrderedResources` will do provided that this ordering is global. However, some orderings are better than others and will result is less blocking overall. This may take some analysis at design time to identify the best ordering of the `OrderedResources`. As mentioned above, if `OrderedResources` are themselves `ResourceClients` (a reasonable possibility), then they should *only* invoke services of `OrderedResources` that have higher valued IDs than they do. If they invoke a lower-valued `OrderedResources`, then they are in effect violating the ordered locking protocol by the transitive property of locking (if A locks B and then B locks C, then A is in effect locking C). This results in a returned error code.

While draconian, one solution to the potential problem of transitive violation of the ordering policy is to enforce the rule that a `OrderedResources` may never invoke services or lock other `OrderedResources`. If your system design does allow such transitive locking, then each transitive path must be examined to ensure that the ordering policy is not violated.

关于`resourceIDs` 本身并不需要做任何特殊处理，但作为测试的一部分，它们需要被设计成只按正确的顺序锁定资源。对于一元函数，它们不需要做任何特殊的事情——它们只会调用

`monadicAccessFunction()`。对于二元访问函数，`ResourceClient` 必须显式地先锁定资源，调用所需的服务，然后调用解锁。使用二元形式更加灵活——因为 `ResourceClient` 可以根据需要持有任意数量资源的锁——但需要开发者具备额外的纪律性。

4.9.4.4 资源列表

该元素提供两个重要功能。`addLock()`如果传入的`resourceID` 大于当前已锁定资源中最大的资源ID，则返回0（"`ok`"）。如果传入的`referenceID` 小于最大的资源ID，则返回`POORLY_ORDERED_ACCESS`；如果传入的`referenceID` 等于它，则返回 `DUPLICATE_IDS`。（注意，如果在资源被锁定时访问同一资源，其互斥锁会在调用 `addLock()`之前阻塞，因此这种情况唯一可能发生的方式是`resourceID` 不唯一）。同样重要的是要注意，应用程序中必须只有一个`ResourceList`，从而使`resourceIDs` 具有有效的全局作用域。

4.9.4.5 ResourceReference

这只是保存在有序列表中的 `resourceID`；很可能是一个数组。仅保留最大值是不够的，因为在任意时刻可能有许多资源被加锁。

4.9.5 结果

该模式通过消除条件4——循环等待的可能性，有效地消除了基于资源的死锁的可能性。要使算法正常工作，`OrderedResources` 的任何排序都可以，只要该排序是全局的。然而，某些排序比其他排序更好，并且总体上会导致更少的阻塞。这可能需要设计时的一些分析来确定 `OrderedResources` 的最佳排序。如上所述，如果 `OrderedResources` 本身是 `ResourceClients` （一种合理的可能性），那么它们只应调用 ID 值高于自身的 `OrderedResources` 的服务。如果它们调用 ID 值较低的 `OrderedResources`，那么根据加锁的传递性（如果 A 锁定 B，然后 B 锁定 C，那么 A 实际上锁定了 C），它们实际上违反了有序加锁协议。这会导致返回错误码。

虽然这种方案较为严苛，但针对传递性违反排序策略这一潜在问题，一种解决方案是强制执行以下规则：有序资源 永远不得调用服务或锁定其他有序资源。如果你的系统设计确实允许这种传递加锁，则必须检查每条传递路径，以确保排序策略不被违反。

4.9.6 Implementation Strategies

The implementation of the pattern requires the addition of a `resourceID` to each `OrderedResource` and logic in the `ResourceList` to ensure that the `resourceID` of each `OrderedResource` is larger than any currently locked `resourceID`. In addition, a list of locked `resourceIDs` must be maintained so that as `OrderedResources` are released, additional ones can be locked appropriately.

The most common implementation for the `ResourceList` is as an array of integers representing the `resourceIDs` in the order in which they are locked.

4.9.7 Related Patterns

This pattern is specifically designed to remove the possibilities of deadlocks. The Simultaneous Locking Pattern also does this, although through the different mechanism of preventing condition 2 (some resources are locked while others are requested). The Critical Region Pattern also removes the possibility of deadlock by preventing condition 3 (preemption while resources are locked is allowed). The Cyclic Executive Pattern avoids deadlock by preventing condition 1 (mutual exclusion locking of resources).

4.9.8 Example

The example in Figure 4-30 shows an application of the Ordered Locking Pattern. In this (simplified) aircraft example, three servers hold related, but distinct information; `AttitudeSensor` has information about the attitude (roll, pitch, and yaw) of the aircraft, `PositionSensor` has information about where the aircraft is in the airspace (latitude, longitude, and altitude), and the `VelocitySensor` has information about how fast the aircraft is moving in three dimensions.

The example has two different clients. The `KinematicModelClient` needs to get time-coherent information about the state of the aircraft for the purpose of precise and detailed control of actuators and engines. The `RoutePlanningClient` needs the same information to determine updates to the route depending on fuel usage, threat assessments, mission criticality, and so on.

Let's suppose, for the purpose of this discussion that for both the clients it is crucial to get data in the same sample interval. A way to do that is to lock all the resources, gather and manipulate the data, and then release all the resources. If the KinematicModel locks them in the order 1) `PositionSensor`, 2) `VelocitySensor`, and 3) `AttitudeSensor` but the `RoutePlanningClient` (done by a different subcontractor) locks them in the reverse order, there is an opportunity for deadlock. Ergo, we've added the Ordered Locking Pattern to specifically prevent deadlock. We've also used the dyadic form of the pattern so the

4.9.6 实现策略

该模式的实现需要为每个有序资源 添加一个`resourceID`，并在资源列表 中加入逻辑，以确保每个有序资源 的`resourceID` 大于当前已锁定的任何`resourceID`。此外，还必须维护一个已锁定资源ID 的列表，以便在有序资源 被释放时，可以适当地锁定其他资源。

资源列表 最常见的实现方式是使用一个整数数组，按照加锁的顺序表示资源ID。

4.9.7 相关模式

该模式专门设计用于消除死锁的可能性。同时锁定模式也能实现这一点，不过它通过不同的机制来防止条件2（部分资源已加锁而其他资源正在被请求）。临界区模式同样通过防止条件3（资源加锁期间允许抢占）来消除死锁的可能性。循环执行模式则通过防止条件1（资源的互斥锁定）来避免死锁。

4.9.8 示例

图 4-30 中 的 示例 展示了有序锁定模式的一个应用。在这个（简化的）飞机示例中，三个服务器持有相关但不同的信息；姿态传感器 拥有关于飞机姿态（横滚角、俯仰和偏航）的信息，位置传感器 拥有关于飞机在空域中位置（纬度和高度）的信息，而`VelocitySensor` 拥有关于飞机在三维空间中移动速度的信息。

该示例有两个不同的客户端。运动模型客户端 需要获取关于飞机状态的时间一致信息，以便对执行器和发动机进行精确且详细的控制。航线规划客户端 需要相同的信息，以根据燃料消耗、威胁评估、任务关键性等来确定航线的更新。

为了便于讨论，我们假设两个客户端都必须在相同的采样间隔内获取数据。实现这一点的一种方法是锁定所有资源，采集并处理数据，然后释放所有资源。如果 KinematicModel 按以下顺序锁定它们：1) 姿态传感器，2) `VelocitySensor`，3) 姿态传感器，而航线规划客户端 （由不同的分包商完成）以相反的顺序锁定它们，则存在死锁的可能。因此，我们添加了有序锁定模式来专门防止死锁。我们还使用了该模式的二元形式，因此

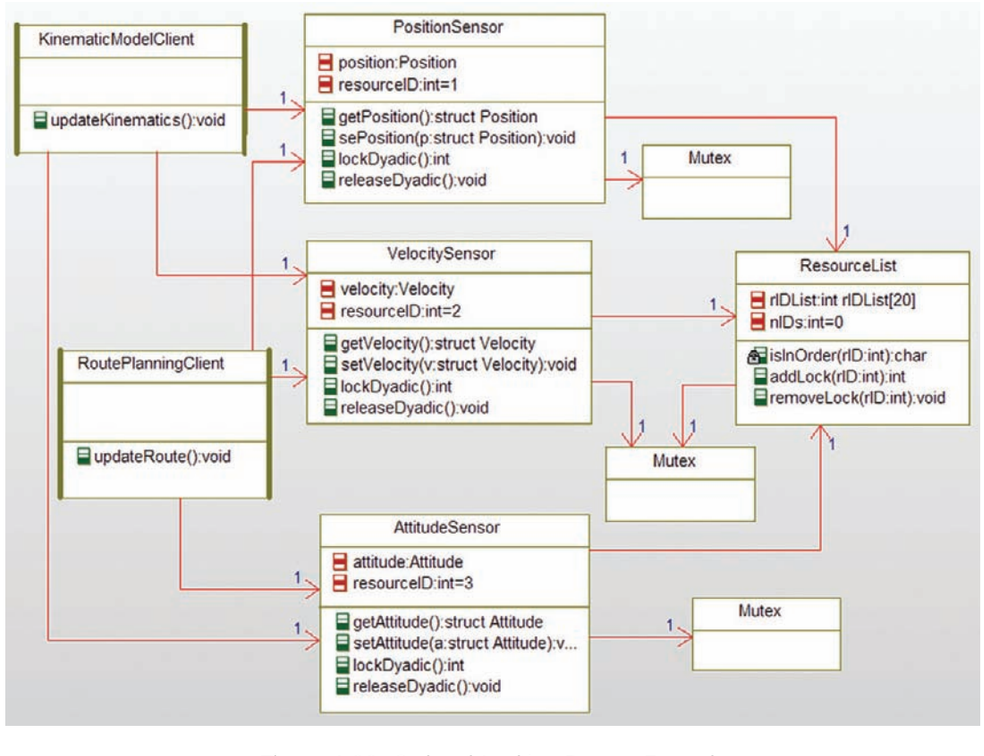


Figure 4-30: Ordered Locking Pattern Example

clients are expected to manually lock the resources before use and release the resources once done.

Note that the mutex class shows up multiple times in the diagram. This is done to minimize line crossing in the diagram. There is a single mutex class, but four distinct instances of that class, each providing a separate locking semaphore for its owner.

I've included the code for two of the classes, the `VelocitySensor` and the `ResourceList` to illustrate how this pattern may be implemented. The error codes are simply implemented in the OrdseredLockingExample.h file as

```
#define POORLY_ORDERED_ACCESS (1)
#define DUPLICATED_IDS (2)
```

Code Listing 4-48 has the VelocitySensor.h code and Code Listing 4-49 has the corresponding C code.

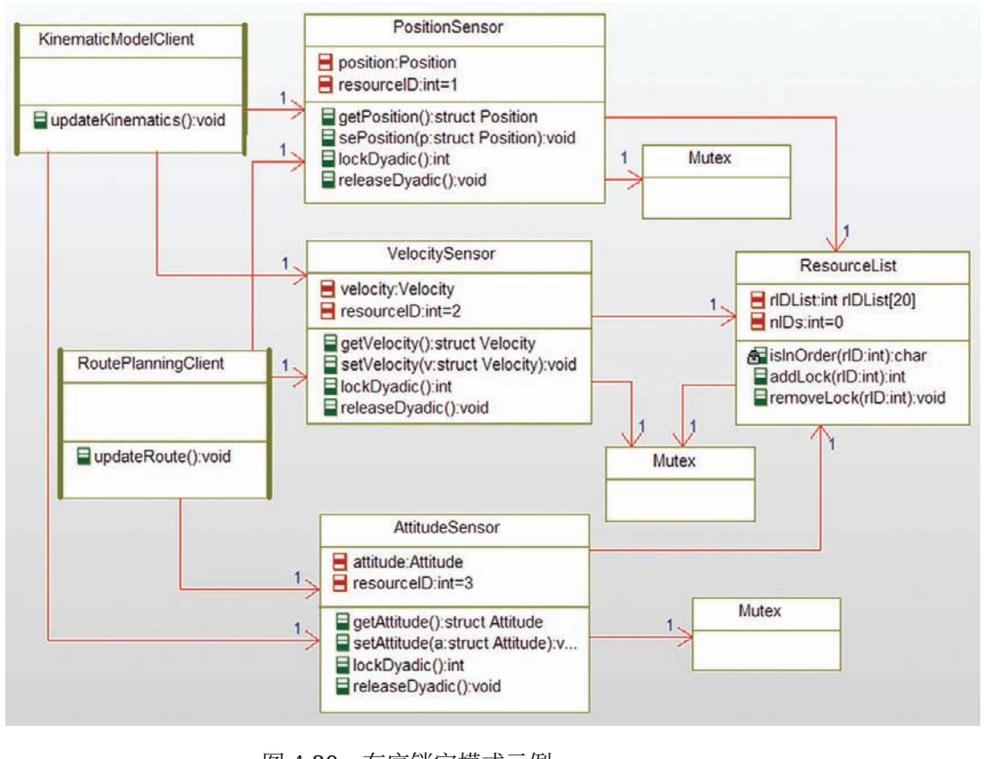


图 4-30：有序锁定模式示例

客户端应在使用前手动锁定资源，并在完成后释放资源。

请注意，互斥锁类在图中多次出现。这样做是为了尽量减少图中的线条交叉。只有一个互斥锁类，但该类有四个不同的实例，每个实例为其所有者提供一个单独的锁定信号量。

我包含了其中两个类的代码，即 `VelocitySensor` 和 `ResourceList`，以说明如何实现此模式。错误码只是在 OrdseredLockingExample.h 文件中简单地实现为

```
#define POORLY_ORDERED_ACCESS (1) #
define DUPLICATED_IDS (2)
```

代码清单 4-48 包含 VelocitySensor.h 代码，代码清单 4-49 包含对应的 C 代码。

```
#ifndef VelocitySensor_H
#define VelocitySensor_H
/* brings in the error codes */
#include "OrderedLockingExample.h"
#include "Velocity.h"
struct Mutex;

struct ResourceList;

typedef struct VelocitySensor VelocitySensor;
struct VelocitySensor {
    int resourceID;
    struct Velocity velocity;
    struct Mutex* itsMutex;
    struct ResourceList* itsResourceList;
};

void VelocitySensor_Init(VelocitySensor* const me);

void VelocitySensor_Cleanup(VelocitySensor* const me);

/* Operations */
struct Velocity VelocitySensor_getVelocity(VelocitySensor* const me);

int VelocitySensor_lockDyadic(VelocitySensor* const me);

void VelocitySensor_releaseDyadic(VelocitySensor* const me);

void VelocitySensor_setVelocity(VelocitySensor* const me, struct Velocity v);

struct Mutex* VelocitySensor_getItsMutex(const VelocitySensor* const me);

void VelocitySensor_setItsMutex(VelocitySensor* const me, struct Mutex* p_Mutex);

struct ResourceList* VelocitySensor_getItsResourceList(const
VelocitySensor* const me);

void VelocitySensor_setItsResourceList(VelocitySensor* const me,
struct ResourceList* p_ResourceList);

VelocitySensor * VelocitySensor_Create(void);

void VelocitySensor_Destroy(VelocitySensor* const me);

#endif
```

Code Listing 4-24: VelocitySensor.h


```
#ifndef VelocitySensor_H #define
VelocitySensor_H /* 引入错误码 */ #
include "OrderedLockingExample.h" #
include "Velocity.h" struct Mutex;



struct ResourceList;

typedef 结构体 VelocitySensor VelocitySensor; 结构体
VelocitySensor { int resourceID; 结构体 Velocity
velocity; 结构体 互斥锁* itsMutex; 结构体 资源列表*
itsResourceList; };



void VelocitySensor_Init(VelocitySensor* const me);

void VelocitySensor_Cleanup(VelocitySensor* const me);

/* 操作 */ struct Velocity VelocitySensor_getVelocity(VelocitySensor* const me);

int VelocitySensor_lockDyadic(VelocitySensor* const me);

void VelocitySensor_releaseDyadic(VelocitySensor* const me);

void VelocitySensor_setVelocity(VelocitySensor* const me, struct Velocity v);

结构体 Mutex* VelocitySensor_getItsMutex(const VelocitySensor* const me) ;

void Velocity 传感器_setItsMutex(VelocitySensor* const me, 结构体 互斥锁* p_Mutex );

struct ResourceList* VelocitySensor_getItsResourceList(const
VelocitySensor* const me);

void VelocitySensor_setItsResourceList(VelocitySensor* const me, struct
ResourceList* p_ResourceList);

VelocitySensor * VelocitySensor_Create(void);

void VelocitySensor_Destroy(VelocitySensor* const me);

#endif
```

代码清单4-24：VelocitySensor.h

```
#include "VelocitySensor.h"
#include "Mutex.h"
#include "ResourceList.h"

static void cleanUpRelations(VelocitySensor* const me);

void VelocitySensor_Init(VelocitySensor* const me) {
    me->resourceID = 2;
    {
        Velocity_Init(&(me->velocity));
    }
    me->itsMutex = NULL;
    me->itsResourceList = NULL;
}

void VelocitySensor_Cleanup(VelocitySensor* const me) {
    cleanUpRelations(me);
}

struct Velocity VelocitySensor_getVelocity(VelocitySensor* const me) {
    return me->velocity;
}

int VelocitySensor_lockDyadic(VelocitySensor* const me) {
    int ok;
    Mutex_lock(me->itsMutex);
    ok = ResourceList_addLock(me->itsResourceList, me->resourceID);
    if (ok != 0)
        Mutex_release(me->itsMutex);
    return ok;
}

void VelocitySensor_releaseDyadic(VelocitySensor* const me) {
    ResourceList_removeLock(me->itsResourceList, me->resourceID);
    Mutex_release(me->itsMutex);
}

void VelocitySensor_setVelocity(VelocitySensor* const me, struct Velocity v) {
    me->velocity = v;
}

struct Mutex* VelocitySensor_getItsMutex(const VelocitySensor* const me) {
    return (struct Mutex*)me->itsMutex;
}

void VelocitySensor_setItsMutex(VelocitySensor* const me, struct Mutex* p_Mutex) {
    me->itsMutex = p_Mutex;
}
```


#include "VelocitySensor.h" #include "Mutex.h" #include "ResourceList.h" static void cleanUpRelations(VelocitySensor* const me); void VelocitySensor_Init(VelocitySensor* const me) { me->resourceID = 2; { Velocity_Init(&(me->velocity)); } me->itsMutex =空; me->itsResourceList = 空; } void VelocitySensor_清理(VelocitySensor* const me) { cleanUpRelations(me); } struct Velocity VelocitySensor_getVelocity(VelocitySensor* const me) { return me->velocity; } int VelocitySensor_lockDyadic(VelocitySensor* const me) { int ok; Mutex_加锁(me->itsMutex); ok =资源列表_addLock(me->itsResourceList, me->resourceID); if (ok!= 0) Mutex_释放(me->itsMutex); return ok; } void VelocitySensor_releaseDyadic(VelocitySensor* const me) { 资源列表_removeLock(me->itsResourceList, me->resourceID); Mutex_释放(me->itsMutex); } void VelocitySensor_setVelocity(VelocitySensor* const me, struct Velocity v) { me->velocity = v; } 结构体 互斥锁* VelocitySensor_getItsMutex(const VelocitySensor* const me) { return (struct Mutex*)me->itsMutex; } void VelocitySensor_setItsMutex(VelocitySensor* const me, struct Mutex* p_Mutex) { me->itsMutex = p_互斥锁; }

```
struct ResourceList*  VelocitySensor_getItsResourceList(const
VelocitySensor*  const me) {
    return (struct ResourceList*)me->itsResourceList;
}

void VelocitySensor_setItsResourceList(VelocitySensor*  const me,
struct ResourceList*  p_ResourceList) {
    me->itsResourceList  =  p_ResourceList;
}

VelocitySensor  *  VelocitySensor_Create(void) {
    VelocitySensor*  me  =  (VelocitySensor  *)
malloc(sizeof(VelocitySensor));
    if(me!=NULL)
           VelocitySensor_Init(me);
    return me;
}

void VelocitySensor_Destroy(VelocitySensor*  const me) {
    if(me!=NULL)
    VelocitySensor_Cleanup(me);
    free(me);
}

static void cleanUpRelations(VelocitySensor*  const me) {
    if(me->itsMutex !=  NULL)
           me->itsMutex  =  NULL;
    if(me->itsResourceList !=  NULL)
           me->itsResourceList = NULL;
}
```

Code Listing 4-49: VelocitySensor.c

Of course, the key class for this pattern is the ResourceList. ResourceList.h is in Code Listing 4-50 with the corresponding C file shown in Code Listing 4-51.

```
#ifndef ResourceList_H
#define ResourceList_H

#include "OrderedLockingExample.h"
struct Mutex;

typedef struct ResourceList ResourceList;
struct ResourceList {
    int nIDs;
    int rIDList[20];
    struct Mutex*  itsMutex;
};
```


struct ResourceList* VelocitySensor_getItsResourceList(const VelocitySensor* const me) { return (struct ResourceList*)me->itsResourceList; } void VelocitySensor_setItsResourceList(VelocitySensor* const me, struct ResourceList* p_ResourceList) { me->itsResourceList = p_ResourceList; } VelocitySensor * VelocitySensor_Create(void) { VelocitySensor* me = (VelocitySensor *) malloc(sizeof(VelocitySensor)); if(me!=空) VelocitySensor_Init(me); return me; } void VelocitySensor_Destroy(VelocitySensor* const me) { if(me!=空) VelocitySensor_Cleanup(me); free(me); } static void cleanUpRelations(VelocitySensor* const me) { if(me->itsMutex != 空) me->itsMutex = 空; if(me->itsResourceList != 空) me->itsResourceList = 空; }

代码清单 4-49：VelocitySensor.c

当然，此模式的关键类是 ResourceList。ResourceList.h 见代码清单 4-50，对应的 C 文件见代码清单 4-51。

#ifndef ResourceList_H #define ResourceList_H #include "OrderedLockingExample.h" struct Mutex; typedef struct ResourceList ResourceList; struct ResourceList { int nIDs; int rIDList[20]; struct Mutex* itsMutex; };

```
/*  Constructors and destructors:*/
void ResourceList_Init(ResourceList*  const me);
void ResourceList_Cleanup(ResourceList*  const me);

/*  Operations  */
int ResourceList_addLock(ResourceList*  const me, int rID);

/*  The elements are added in order but  */
/*  can be removed in any order. Therefore,  */
/*  the stored resoiurceIDs above the current  */
/*  one in the list must be moved down to  */
/*  lower in the list.  */
void ResourceList_removeLock(ResourceList*  const me, int rID);

struct Mutex*  ResourceList_getItsMutex(const ResourceList*  const me);

void ResourceList_setItsMutex(ResourceList*  const me, struct Mutex*  p_Mutex);

ResourceList  *  ResourceList_Create(void);

void ResourceList_Destroy(ResourceList*  const me);

#endif
```

Code Listing 4-50: ResourceList.h

```
#include "ResourceList.h"
#include "Mutex.h"

static char isInOrder(ResourceList*  const me, int rID);

static void cleanUpRelations(ResourceList*  const me);

void ResourceList_Init(ResourceList*  const me) {
    me->nIDs  =  0;
    me->itsMutex  =  NULL;
}

void ResourceList_Cleanup(ResourceList*  const me) {
    cleanUpRelations(me);
}

int ResourceList_addLock(ResourceList*  const me, int rID) {
    int retVal;
    Mutex_lock(me->itsMutex);
    if (isInOrder(me,rID)) {
        me->rIDList[me->nIDs++]  =  rID;
        retVal  =  0;
        }
```



```
/* 构造函数与析构函数：*/ void ResourceList_Init(ResourceList*
const me); void ResourceList_Cleanup(ResourceList* const
me);

/* 操作 */ int ResourceList_addLock(ResourceList* const me, int rID);


/* 元素按顺序添加，但 */ /* 可以按任意顺序移除。因此， */ /* 列表中位于当前元
素之上的已存储 */ /* 资源ID必须向下移动 */ /* 到列表中的更低位置。 */ void
ResourceList_removeLock(ResourceList* const me, int rID);




struct Mutex* ResourceList_getItsMutex(const ResourceList* const me);

void ResourceList_setItsMutex(ResourceList* const me, struct Mutex* p_Mutex);

ResourceList * 资源列表_Create(void);

void ResourceList_Destroy(ResourceList* const me);

#endif
```

代码清单 4-50：ResourceList.h

```
#include "ResourceList.h" #include "Mutex.h" static char
isInOrder(ResourceList* const me, int rID); static void
cleanUpRelations(ResourceList* const me); void ResourceList_
Init(ResourceList* const me) { me->nIDs  =  0; me->itsMutex  = 空; }
void ResourceList_清理(ResourceList* const me) {
cleanUpRelations(me); } int ResourceList_addLock(ResourceList*
const me, int rID) { int retVal; Mutex_加锁(me->itsMutex); if
(isInOrder(me,rID)) { me->rIDList[me->nIDs++]  = rID; retVal  =  0; }
```

```
    else
        if (rID == me->rIDList[me->nIDs])
            retVal = DUPLICATED_IDS;
        else
            retVal = POORLY_ORDERED_ACCESS;
    Mutex_release(me->itsMutex);
    return retVal;
}

void ResourceList_removeLock(ResourceList* const me, int rID){
    int j,k;

    if (me->nIDs){
        for (j=0; j<me->nIDs; j++){
            if (rID == me->rIDList[j]){
                for (k=j; k<me->nIDs-1; k++)
                     me->rIDList[k] = me->rIDList[k+1];
                -me->nIDs;
                break;
                };
            };
    };
}

static char isInOrder(ResourceList* const me, int rID){
    if (me->nIDs)
        return rID > me->rIDList[me->nIDs-1];
    else
        return 1;
}

struct Mutex* ResourceList_getItsMutex(const ResourceList* const me){
    return (struct Mutex*)me->itsMutex;
}

void ResourceList_setItsMutex(ResourceList* const me, struct Mutex* p_Mutex){
    me->itsMutex = p_Mutex;
}

ResourceList * ResourceList_Create(void){
    ResourceList* me = (ResourceList *)
malloc(sizeof(ResourceList));
        if(me!=NULL)
            ResourceList_Init(me);
    return me;
}

void ResourceList_Destroy(ResourceList* const me){
    if(me!=NULL)
    ResourceList_Cleanup(me);
```


else if (rID == me->rIDList[me->nIDs]) retVal = DUPLICATED_IDS; else retVal = POORLY_ORDERED_ACCESS; Mutex_release(me->itsMutex); return retVal; } void ResourceList_removeLock(ResourceList* const me, int rID) { int j,k; if (me->nIDs) { for (j=0; j<me->nIDs; j++) { if (rID == me->rIDList[j]) { for (k=j; k<me->nIDs-1; k++) me->rIDList[k] = me->rIDList[k+1];–me->nIDs; break; }; }; } static char isInOrder(ResourceList* const me, int rID) { if (me->nIDs) return rID > me->rIDList[me->nIDs-1]; else return 1; } struct Mutex* ResourceList_getItsMutex(const ResourceList* const me) { return (struct Mutex*)me->itsMutex; } void ResourceList_setItsMutex(ResourceList* const me, struct Mutex* p_Mutex) { me->itsMutex = p_Mutex; } ResourceList * ResourceList_Create(void) { ResourceList* me = (ResourceList *) malloc(sizeof(ResourceList)); if(me!=NULL) ResourceList_Init(me); return me; } void ResourceList_Destroy(ResourceList* const me) { if(me!=NULL) ResourceList_Cleanup(me);

```
    free(me);
}

static void cleanUpRelations(ResourceList*  const me) {
    if(me->itsMutex != NULL)
            me->itsMutex  =  NULL;
}
```

Code Listing 4-51: ResourceList.c

4.10 So, What Have We Learned?

The chapter started off with a basic introduction to concurrency concepts, such as what do we mean by the terms concurrency, urgency, criticality, resource, deadlock, and so on. Concurrency is something that we deal with constantly in our daily lives; many many things happen in parallel and we must account for when to start, stop, and synchronize. It is no different in programming embedded systems, except that we must be more precise in our definition and usage of the terms to ensure our systems' predictably to the right thing all the time.

We also talked about how to represent concurrency in the Unified Modeling Language (UML) so that we could draw pictures of the patterns and understand what they mean. Primarily we use «active» classes to represent threads and nest (part) objects inside these structured classes to indicate the parts running within the thread context of the enclosing class. In the same thread, to use a pun, we talked about the structure and services of a typical Real-Time Operating System (RTOS).

This chapter then presented a number of patterns that dealt with concurrency and resource management that are in common use in embedded systems. The first two – the Cyclic Executive and the Static Priority Patterns – provided two different ways for scheduling threads. The Cyclic Executive Pattern is very simple and easy to implement but is demonstrably suboptimal in terms of responsiveness to incoming events. The Static Priority Pattern relies on scheduling services of an RTOS kernel and gives very good responsiveness to events, but it is less predictable and more complex than a Cyclic Executive.

The next three patterns – the Critical Region, the Guarded Call, and the Queuing Patterns – addressed the concern of serializing access to resources in a multitasking environment. The Critical Region Pattern turns off task switching during resource access, thus preventing the possibility of resource data corruption. This approach works but blocks higher-priority tasks that may never access the resource. The Guarded Call Pattern accomplishes the same resource protection goal by using mutex semaphore. The naïve implementation of this pattern can lead to unbounded priority inversion, so the pattern can be augmented with others, such as the Priority

free(me); } static void cleanUpRelations(ResourceList* const me) { if(me->itsMutex != 空) me->itsMutex = 空; }

代码清单 4-51：ResourceList.c

4.10 那么，我们学到了什么?

本章首先对并发概念进行了基本介绍，例如并发、紧迫性、关键性、资源、死锁等术语的含义。并发是我们日常生活中不断处理的事情；许多事情并行发生，我们必须考虑何时启动、停止和同步。在嵌入式系统编程中也是如此，只不过我们必须更精确地定义和使用这些术语，以确保系统始终可预测地执行正确的操作。

我们还讨论了如何在统一建模语言（UML）中表示并发，以便我们能够绘制模式的图形并理解其含义。我们主要使用«活动»类来表示线程，并在这些结构化类中嵌套（部件）对象，以表示在封闭类的线程上下文中运行的部件。在同一线程中——借用一句双关语——我们讨论了典型实时操作系统（RTOS）的结构和服务。

本章随后介绍了若干在嵌入式系统中常用的、涉及并发与资源管理的模式。前两个模式——循环执行模式和静态优先级模式——提供了两种不同的线程调度方式。循环执行模式非常简单且易于实现，但在对传入事件的响应性方面明显次优。静态优先级模式依赖于RTOS内核的调度服务，对事件具有非常好的响应性，但与循环执行程序相比，它可预测性较差且更为复杂。

接下来的三个模式——临界区模式、受保护调用模式和排队模式——解决了在多任务环境中对资源访问进行串行化的问题。临界区模式在资源访问期间关闭任务切换，从而防止资源数据损坏的可能性。这种方法有效，但会阻塞那些可能永远不会访问该资源的更高优先级任务。受保护调用模式通过使用互斥信号量来实现相同的资源保护目标。该模式的朴素实现可能导致无界优先级反转，因此该模式可以与其他模式（如优先级

Inheritance Pattern, to limit priority inversion to a single level. The last pattern in this group, the Queuing Pattern, serializes access to a resource by putting the requests in a queue and the resource removes them from the queue in a first-in-first-out (FIFO) order. The Queuing is conceptually very simple but can lead to delays in response to a request made of the resource.

The last two patterns – the Simultaneous Locking and the Ordered Locking Patterns – concerned themselves with deadlock prevention. Deadlock is a very real problem that occurs when there are conditions waited for that can never actually occur, and this happens far more often than most people would suspect. Four conditions are required for deadlock to occur and deadlock can be eliminated by violating any of these four conditions. The Simultaneous Locking Pattern violates the requirement of holding one resource while requesting others through the simple expedient of locking all resource it needs at once. The Ordered Locking Pattern breaks another requirement of deadlocks, that a circular waiting condition occurs; by ensuring that all clients locking the resource in the same order, circular waiting (and deadlock) can be eliminated.

继承模式）结合使用，以将优先级反转限制在单一级别。该组中的最后一个模式——排队模式——通过将请求放入队列来串行化对资源的访问，资源按先进先出（FIFO）顺序从队列中移除请求。排队在概念上非常简单，但可能导致对资源请求的响应延迟。

最后两个模式——同时锁定模式和有序锁定模式——关注的是死锁预防。死锁是一个非常现实的问题，当存在永远无法实际发生的等待条件时就会发生，而这种情况发生的频率远超大多数人的想象。死锁的发生需要满足四个条件，只要违反其中任何一个条件就可以消除死锁。同时锁定模式通过一次性锁定所需的所有资源这一简单手段，违反了在请求其他资源时仍持有一个资源的要求。有序锁定模式打破了死锁的另一个要求，即循环等待条件的发生；通过确保所有客户端以相同的顺序锁定资源，可以消除循环等待（以及死锁）。

CHAPTER 5

Design Patterns for State Machines

Chapter Outline

5.1 Oh Behave! 259

5.2 Basic State Machine Concepts 261

5.2.1 OR-States 263

5.2.2 AND-States 265

5.2.3 Special Stuff: Timeouts, Conditionals, and More 267

5.2.4 Synchronous versus Asynchronous? 270

5.3 Single Event Receptor Pattern 274

5.3.1 Abstract 274

5.3.2 Problem 274

5.3.3 Pattern Structure 274

5.3.4 Collaboration Roles 275

5.3.4.1 Event 275

5.3.4.2 EventType 275

5.3.4.3 Mutex 276

5.3.4.4 TokenizerAsyncSingleReceptor 276

5.3.4.5 TokenizerSyncSingleReceptor 277

5.3.4.6 TSRClient 277

5.3.4.7 TSREventQueue 277

5.3.4.8 TSTATE 278

5.3.4.9 TSUBSTATE 278

5.3.5 Consequences 278

5.3.6 Implementation Strategies 278

5.3.7 Related Patterns 278

5.3.8 Example 278

5.4 Multiple Event Receptor Pattern 287

5.4.1 Abstract 287

5.4.2 Problem 287

5.4.3 Pattern Structure 288

5.4.4 Collaboration Roles 288

5.4.4.1 Mutex 288

5.4.4.2 TMRClient 289

5.4.4.3 TokenizeMultiReceptor 289

5.4.4.4 TSTATE 289

5.4.4.5 TSUBSTATE 290

5.4.5 Consequences 290

5.4.6 Implementation Strategies 290

Design Patterns for Embedded Systems in C
DOI:10.1016/B978-1-85617-707-8.00005-4

Copyright © 2011 Elsevier Inc.

第 5 章

状态机设计模式

第 章 大纲 哦 表现好点！ **2595.2** 基本 状态 机 概念 **261**5.2.1 或状态 263 5.2.2 与状态 265 5.2.3 特殊内容：超时、条件，以及

More 267

5.2.4 同步与异步？ 270**5.3** 单 事件 接收器 模式 **274** 5 3 1 274 . . 摘要 5.3.2 问题 274 5.3.3 模式结构 274 5.3.4 协作角色 275 5.3.4.1 事件 275 5.3.4.2 EventType 275 5.3.4.3 互斥锁 276 5.3.4.4 TokenizerAsyncSingleReceptor 276 5.3.4.5 TokenizerSyncSingleReceptor 277 5.3.4.6 TSRClient 277 5.3.4.7 TSREventQueue 277 5.3.4.8 TSTATE 278 5.3.4.9 TSUBSTATE 278 5.3.5 结果 278 5.3.6 实现策略 278 5.3.7 相关模式 278 5.3.8 示例 278**5.4** 多 事件 接收器 模式 **287**5.4.1 摘要 287 5.4.2 问题 287 5.4.3 模式结构 288 5.4.4 协作角色 288 5.4.4.1 互斥锁 288 5.4.4.2 TMRClient 289 5.4.4.3 TokenizeMultiReceptor 289 5.4.4.4 TSTATE 289 5.4.4.5 TSUBSTATE 290 5.4.5 结果 290 5.4.6 实现策略 290

C语言嵌入式系统设计模式
DOI:10.1016/B978-1-85617-707-8.00005-4 版权所

有 © 2011 Elsevier Inc.

5.4.7 Related Patterns 290
5.4.8 Example 290
5.5 State Table Pattern 296
5.5.1 Abstract 296
5.5.2 Problem 296
5.5.3 Pattern Structure 296
5.5.4 Collaboration Roles 297
5.5.4.1 ActionPtr0 298
5.5.4.2 ActionPtr1 298
5.5.4.3 ActionType 298
5.5.4.4 Event 298
5.5.4.5 EventType 298
5.5.4.6 GuardType 298
5.5.4.7 Mutex 298
5.5.4.8 TableEntryType 298
5.5.4.9 TokenizerStateTable 298
5.5.4.10 TSTATETYPE 298
5.5.5 Consequences 299
5.5.6 Implementation Strategies 299
5.5.7 Related Patterns 299
5.5.8 Example 300
5.6 State Pattern 308
5.6.1 Abstract 308
5.6.2 Problem 308
5.6.3 Pattern Structure 308
5.6.4 Collaboration Roles 310
5.6.4.1 ActionPtr0 310
5.6.4.2 ActionPtr1 311
5.6.4.3 Context 311
5.6.4.4 Mutex 311
5.6.4.5 State Classes 311
5.6.4.6 StateActionSet 312
5.6.4.7 TSTATETYPE 312
5.6.5 Consequences 312
5.6.6 Implementation Strategies 313
5.6.7 Related Patterns 313
5.6.8 Example 314
5.7 AND-States 327
5.8 Decomposed AND-State Pattern 330
5.8.1 Abstract 331
5.8.2 Problem 331
5.8.3 Pattern Structure 331
5.8.4 Collaboration Roles 331
5.8.4.1 AbstractAndStateClass 331
5.8.4.2 ActionSet 331
5.8.4.3 AndStateList 331

5.4.7 相关模式 290 5.4.8
示例 290
5.5 状态 表 模式 **296**5.5.1 摘要 296
5.5.2 问题 296 5.5.3 模式结构 296 5.5.4 协
作角色 297 5.5.4.1 ActionPtr0 298 5.5.4.2
ActionPtr1 298 5.5.4.3 ActionType 298
5.5.4.4 事件 298 5.5.4.5 EventType 298
5.5.4.6 GuardType 298 5.5.4.7 互斥锁
298 5.5.4.8 TableEntryType 298 5.5.4.9
TokenizerStateTable 298 5.5.4.10
TSTATETYPE 298 5.5.5 结果 299 5.5.6 实
现策略 299 5.5.7 相关模式 299 5.5.8 示例
300**5.6** 状态 模式 **308**5.6.1 摘要 308
5.6.2 问题 308 5.6.3 模式结构 308 5.6.4 协
作角色 310 5.6.4.1 ActionPtr0 310 5.6.4.2
ActionPtr1 311 5.6.4.3 上下文 311 5.6.4.4
互斥锁 311 5.6.4.5 状态类 311 5.6.4.6
StateActionSet 312 5.6.4.7 TSTATETYPE
312 5.6.5 结果 312 5.6.6 实现策略 313
5.6.7 相关模式 313 5.6.8 示例 314**5.7**
AND状态 **327****5.8** 分解 AND状态 模式
35.8.1 摘要 331 5.8.2 问题 331 5.8.3 模式
结构 331 5.8.4 协作角色 331 5.8.4.1
AbstractAndStateClass 331 5.8.4.2
ActionSet 331 5.8.4.3 AndStateList 331

5.8.4.4 ConcreteAndStateClass 333
5.8.4.5 ConcreteAndStateType 333
5.8.4.6 Context 333
5.8.5 Consequences 333
5.8.6 Implementation Strategies 333
5.8.7 Related Patterns 334
5.8.8 Example 334
5.9 OK, What Have We Learned? 356

Patterns in this chapter

Single Event Receptor Pattern – Implement state machines with a single event receptor

Multiple Event Receptor Pattern – Implement state machines with many event receptors

State Table Pattern – Implement a table-driven state machine

State Design Pattern – Implement state machines with creation of state objects

Decomposed And-State Pattern – Implement and-states through the decomposition of the composite state

5.1 Oh Behave!

Behavior can be defined as a change in condition or value over time. It is often a response to an external event or request, but autonomous systems decide for themselves when and how to behave. I like to further characterize behavior into four basic kinds[1]. *Simple behavior* is behavior independent of any history of the object or system. For example, computing $\cos(\frac{\pi}{4})$returns the same result regardless of what value the cos() function was invoked with before or when it was computed. Similarly computing the maximum of two values is self-contained in the sense that it doesn't rely on the history of the system. *Continuous behavior*, on the other hand, does rely upon the history of the object or system. For example, a moving average digital filter computes an output that depends not only on current data but also on the history of the data (see Equation 5-1).

$$f_j = \frac{d_j + d_{j-1} + d_{j-2} + d_{j-3}}{4}$$

Equation 5-1: Moving average digital filter

Similarly, a Proportional Integral-Differential (PID) control loop uses feedback to update its output, as we see in Figure 5-1. In both cases, the output depends not only on current information

[1] See my previous book *Real-Time UML Third Edition: Advances in the UML for Real-Time Systems* (Addison-Wesley, 2004) for more detailed information. We won't deal with all of the rich semantics available in UML state machines in this book.

5.8.4.4 ConcreteAndStateClass
3335.8.4.5 ConcreteAndStateType *3335.8.4.6*
上下文 *333*5.8.5 结果 333 5.8.6 实现
策略 333 5.8.7 相关 模式 334 5.8.8
示例 334 5.9 好，我们学到了什么?
356

本章中的模式

单事件接收器模式 – 使用单个事件接收器实现状态机 多事件接收器模式 – 使用多个事件接收器实现状态机 状态表模式 – 实现表驱动的状态机 状态设计模式 – 通过创建状态对象实现状态机 分解AND状态模式 – 通过分解复合状态实现AND状态

5.1 Oh Behave!

行为可以定义为条件或值随时间的变化。它通常是对外部事件或请求的响应，但自主系统会自行决定何时以及如何行为。我喜欢进一步将行为分为四种基本类型[1]。简单行为是独立于对象或系统任何历史的行为。例如，计算$\cos(\frac{\pi}{4})$返回的结果与之前调用cos()函数时使用的值或计算时间无关。类似地，计算两个值的最大值是自包含的，因为它不依赖于系统的历史。另一方面，连续行为确实依赖于对象或系统的历史。例如，移动平均数字滤波器计算的输出不仅取决于当前数据，还取决于数据的历史（见公式5-1）。

$$f_j = \frac{d_j + d_{j-1} + d_{j-2} + d_{j-3}}{4}$$

公式5-1：移动平均数字滤波器

同样，比例积分微分（PID）控制回路利用反馈来更新其输出，如图5-1所示。在这两种情况下，输出不仅取决于当前信息，

[1] 更多详细信息请参阅我之前的著作《Real-Time UML Third Edition: Advances in the UML for Real-Time Systems》（Addison-Wesley，2004年）。本书不会涉及UML状态机中所有丰富的语义。

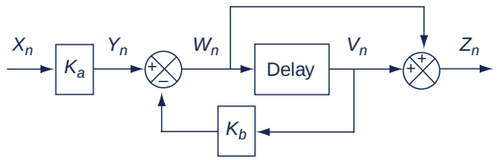


Figure 5-1: Simple PID control loop

but also on previous information, but it does so in a smooth, continuous way. Put another way, the output varies in value but not in kind as a function of new data and the system history.

The third kind of behavior, to which this is dedicated, is *discontinuous* or *stateful behavior*. In stateful systems. the behavior depends on the new data (typically incoming events of interest) and the current state of the system, which are determined by that system's history. However, the difference is the kind of actions the system performs not the value of the outputs. In Figure 5-2, we see a state machine for a microwave oven. The history of the system is represented in the states and the behaviors are the actions, such as `initialize()` (executed on entry into the state `Ready`) and `timeTocook -= currentTime()` (executed on the transition from `Microwaving` to

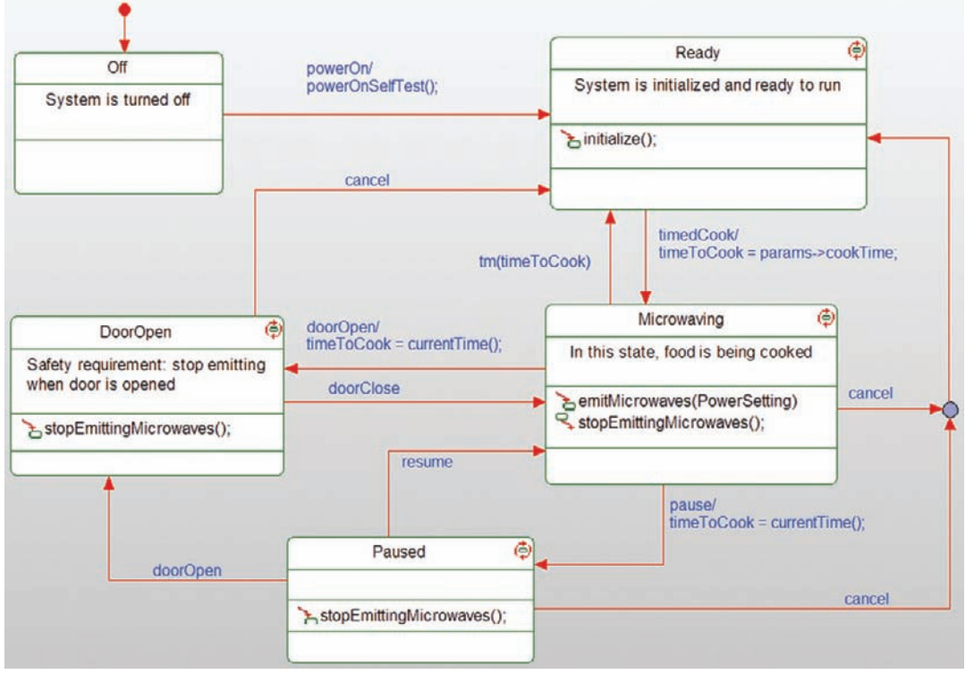


Figure 5-2: Stateful behavior

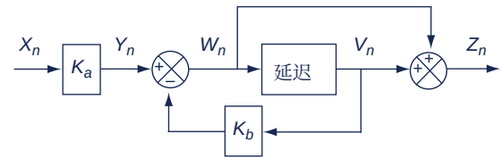


图5-1：简单PID控制回路

还取决于先前的信息，但它以平滑、连续的方式实现这一点。换句话说，输出在新数据和系统历史的函数作用下，其值会变化，但种类不变。

第三种行为，也就是本章所专门讨论的，是不连续行为或有状态行为。在有状态系统中，行为取决于新数据（通常是关注的传入事件）以及系统的当前状态，而当前状态由该系统的历史所决定。然而，区别在于系统所执行的动作种类，而非输出的值。在图5-2中，我们看到一个微波炉的状态机。系统的历史体现在状态中，而行为则是动作，例如 initialize()（在进入就绪状态时执行）和 timeTocook -= currentTime()（在从微波加热到

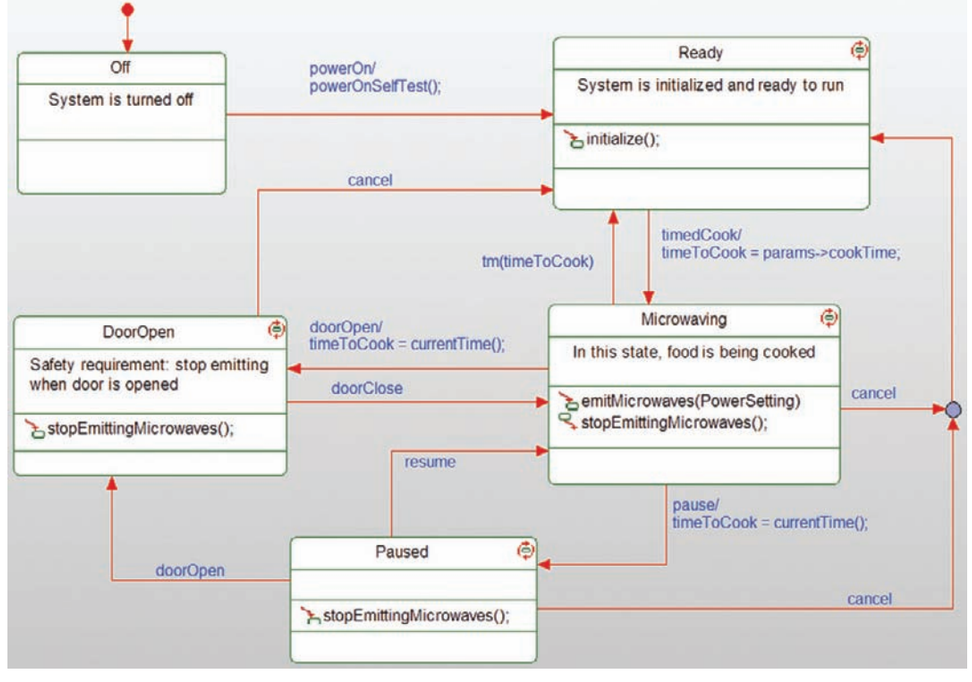


图5-2：有状态行为

Table 5-1: State transition table

	Initial	powerOn	timedCook	doorOpen	doorClose	cancel	pause	resime	resume	Null
ROOT	Off									
Off		Ready								
Ready			Microwaving							
Microwaving				DoorOpen		junctionconnector_11	Paused			Ready
DoorOpen					Microwaving	Ready				
Paused				DoorOpen		junctionconnector_11			Microwaving	
junctionconnector_11										Ready

the `DoorOpen` or `Paused` states). The important thing to note here is that the kind of actions the system performs varies on the basis of its state (and therefore on its history). Microwaves are only emitted in the `Microwaving` state; the system is only initialized in the `Ready` state, and so on.

The state machine may also be visualized as a state x transition table, such as Table 5-1. Many people prefer this notation. It has some advantages but, as you can see, the actual behaviors are not shown in the table. Instead, the rows indicate the current state and the columns are the transitions; in the cells of the table are the next states after the events are processed by the state machine.

State machines are an extremely powerful means by which the behavior of a complex, embedded system can be controlled and orchestrated. As mentioned, it will be the subject of this and the next chapters.

Lastly, the fourth type of behavior is a combination of the previous two, known as *piece-wise continuous behavior* (see Figure 5.3). In PC behavior, within a state, continuous (and, for that matter, simple) the behaviors that occur are of the same kind, but between states, different kinds of behaviors occur. A simple example of this is using different sets of differential equations depending on the state of the system.

5.2 Basic State Machine Concepts

A Finite State Machine (FSM) is a directed graph composed of three primary elements: states, transitions, and actions. A *state* is a condition of a system or element (usually a class in an object-oriented system). Looking back to , the element is the microwave oven (or at least the collection of elements involved in cooking) and the states are the conditions of that system – `Off`, `Ready`, `Microwaving`, `DoorOpen`, and `Paused`. Usual names for states are verbs, adverbs, and verb phrases.

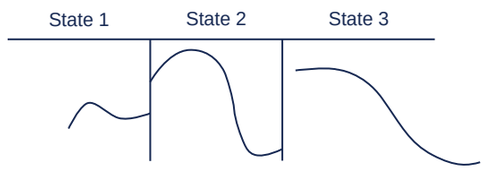


Figure 5-3: Piece-wise continuous behavior

表5-1：状态转移表

	Initial	powerOn	timedCook	doorOpen	doorClose	cancel	pause	resime	resume	Null
ROOT	Off									
Off		Ready								
Ready			Microwaving							
Microwaving				DoorOpen		junctionconnector_11	Paused			Ready
DoorOpen					Microwaving	Ready				
Paused				DoorOpen		junctionconnector_11			Microwaving	
junctionconnector_11										Ready

门开或暂停状态的迁移时执行）。这里需要注意的重要一点是，系统所执行的动作种类因其状态（因而也因其历史）而异。微波仅在微波加热状态下发射；系统仅在就绪状态下初始化，依此类推。

状态机也可以可视化为状态×转移表，如表5-1所示。许多人更喜欢这种表示法。它有一些优点，但正如你所看到的，实际行为并未在表中显示。相反，行表示当前状态，列表示转换；表格单元格中则是状态机处理事件后的下一个状态。

状态机是一种极其强大的手段，可用于控制和编排复杂嵌入式系统的行为。如前所述，这将是本章及下一章的主题。

最后，第四种行为类型是前两种的组合，称为分段连续行为（见图5.3）。在PC行为中，在一个状态内，发生的连续（以及就此而言，简单的）行为属于同一类型，但在状态之间，会发生不同类型的行为。一个简单的例子是根据系统状态使用不同的微分方程组。

5.2 基本状态机概念

有限状态机 (FSM) 是由三个主要元素组成的有向图：状态、转换和动作。状态是系统或元素（通常是面向对象系统中的一个类）所处的条件。回顾前文，该元素是微波炉（或者至少是参与烹饪的元素集合），而状态则是该系统的各种条件——关闭、就绪、微波加热、门开和暂停。状态的常用名称是动词、副词和动词短语。

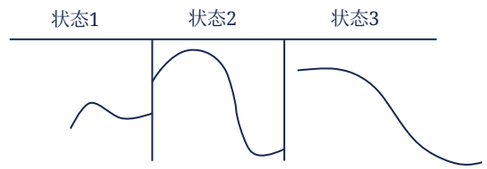


图5-3：分段连续行为

A *transition* is a path from one state to another, usually initiated by an event of interest; it connects a predecessor state with a subsequent state when the element is in the predecessor state and receives the triggering event. The events of interest in Figure 5-2 include the `powerOn`, `timedCook`, `doorOpen`, `cancel`, `pause`, `resume`, and `tm` (timeout) events. If an event occurs while the element is in a state that doesn't respond to that particular event, the event is "quietly discarded" with no effect – that is, state machines only make affirmative statements "When I'm in this state and this occurs, I'll do this") not negative statements ("If I'm not in this state or this doesn't happen, then I'll do that"). Events not explicitly handled within a state are discarded should they occur when the element is in that state.

The actual behaviors executed by the element are represented in *actions*. In UML state machine, actions may be associated with entry into a state, exit from a state, a transition *per se*, or what is called an "internal transition" (we will get into this in more detail shortly). Actions are specified in an *action language*, which for us will be C. Actions shown on Figure 5-2 include `initialize()`, `emitMicrowaves()`, `stopEmittingMicrowaves()`, and `timeToCook -= currentTime()`.

Figure 5-4 shows the basic syntax for UML state machines. The rounded rectangles represent the states of the element. The element remains in a state until a transition causes it to change state. Within a state, three annotations may be made. Entry actions are actions the element executes when the state is entered regardless of which transition path was taken (it is possible to

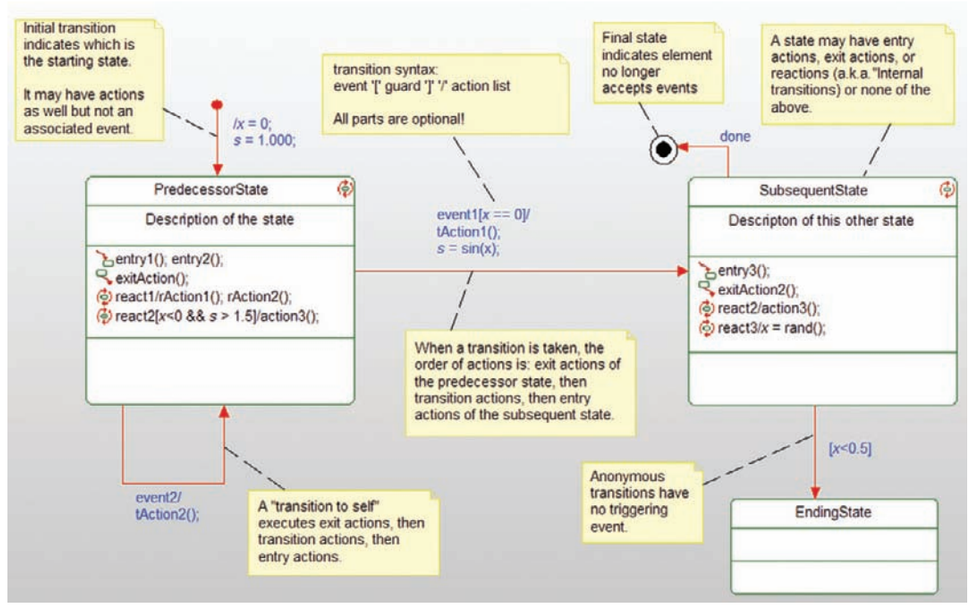


Figure 5-4: Basic state machine syntax

转换是从一个状态到另一个状态的路径，通常由所关注的事件触发；当元素处于前驱状态并接收到触发事件时，它将前驱状态与后继状态连接起来。图5-2中关注的事件包括powerOn、timedCook、doorOpen、cancel、pause、resume 和 tm（超时）事件。如果事件发生时元素处于不响应该特定事件的状态，则该事件会被"静默丢弃"且不产生任何效果——也就是说，状态机只做出肯定陈述（"当我处于此状态且此事件发生时，我将执行此操作"），而非否定陈述（"如果我不处于此状态或此事件未发生，那么我将执行那个操作"）。在某个状态中未被显式处理的事件，若在元素处于该状态时发生，则会被丢弃。

元素执行的实际行为通过动作来表示。在UML状态机中，动作可以与进入状态、退出状态、迁移本身或所谓的"内部转移"相关联（我们稍后将更详细地讨论这一点）。动作使用动作语言来指定，对我们而言就是C语言。图5-2中显示的动作包括initialize()、emitMicrowaves()、stopEmittingMicrowaves()和timeToCook -= currentTime()。

图5-4展示了UML状态机的基本语法。圆角矩形表示元素的状态。元素会一直保持某个状态，直到某个转换使其改变状态。在一个状态内，可以做出三种标注。进入动作是元素在进入该状态时执行的动作，无论走的是哪条转换路径（有可能

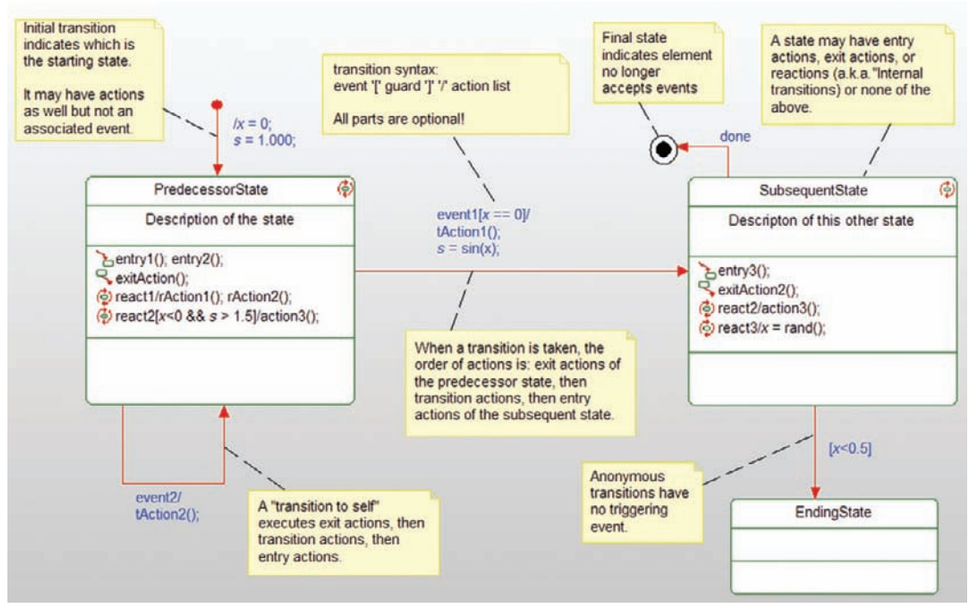


图5-4：基本状态机语法

make many different transitions terminate on any given state). The exit actions are actions taken whenever the state is left, regardless of which path is taken. Reactions ("internal transitions") are taken when the element is in the specified state and the triggering event occurs; in this case, the actions are taken but the state is not exited. Contrast the `PredecessorState` response to the event `react1` with the transition taken with `event2`. In both cases, the element ends up in the same state (`PredecessorState`), but in the former case, only the actions `rAction1()` and `rAction2()` are executed. In the latter case, the exit actions (in this case `exitAction()`) are taken, then the transition actions (in this case `tAction2()`), and finally the entry actions are taken (in this case `entry1()` and `entry2()`).

Transitions are shown as directed arrows. They are normally initiated by the element receiving a triggering event. Transitions have three syntactic fields with three special characters delimiting them. The first field is the *event name*; this is the name of the event type that causes this transition to be taken (provided, of course, that the element is in the predecessor state when the event occurs). The second field is known as the *guard*. The guard is a Boolean condition (without side effects!); if the event occurs AND the guard evaluates to true, then the transition is taken. If the event occurs and the guard is false, then the event is "quietly discarded" and no actions are taken, including the exit actions of the predecessor state. This means that the guard, if present, *must* be evaluated before any actions are executed. The guard, when present, is delimited by square brackets. Lastly, there may be a list of actions that are executed when the transition is taken. The *action list* is preceded by a slash mark.

It should be noted that all of these fields are optional. When the triggering event is missing (known sometimes as an "anonymous transition"), then the transition is triggered as soon as the element enters the predecessor state, that is, as soon as the entry actions for the predecessor state are completed. If the anonymous transition has a guard that evaluates to false, then the anonymous event is discarded and the element remains in the predecessor state even if the guard condition should later become true.

There is a special kind of transition, known as the *initial transition* (or "default pseudostate" if you want to be fancy) that indicates the starting state and actions when the element is created. This transition may not have a triggering event.

There is a special state, known as the *final pseudostate*, that indicates the element no longer accepts events. Usually, this means that the element is about to be deleted. The icon for the final state is a filled circle within an unfilled circle. In Figure 5-4, the pseudostate is entered when the element is in the `SubsequentState` and the element receives the `done` event.

5.2.1 OR-States

In simple state machines, an element must always be in *exactly* one of its states, no more and no less. For this reason, such states are called *OR-states*. UML state machines extend the notion of OR-states by adding the notion of nesting within states. Nested states add a level of abstraction

许多不同的转换终止于任意给定状态）。退出动作是每当离开该状态时执行的动作，无论走的是哪条路径。反应（"内部转换"）是在元素处于指定状态且触发事件发生时执行的；在这种情况下，动作会被执行，但不会退出该状态。请将`PredecessorState`对事件`react1`的响应与使用`event2`所进行的转换进行对比。在两种情况下，元素最终都处于同一状态（`PredecessorState`），但在前一种情况下，只执行动作`rAction1()`和`rAction2()`。在后一种情况下，先执行退出动作（在此例中为`exitAction()`），然后执行转换动作（在此例中为`tAction2()`），最后执行进入动作（在此例中为`entry1()`和`entry2()`）。

转换以有向箭头表示。它们通常由接收触发事件的元素发起。转换具有三个语法字段，并由三个特殊字符分隔。第一个字段是事件名称；这是导致执行此转换的事件类型的名称（当然，前提是事件发生时该元素处于前驱状态）。第二个字段称为守卫条件。守卫条件是一个布尔条件（无副作用！）；如果事件发生且守卫条件求值为真，则执行转换。如果事件发生且守卫条件为假，则该事件被"静默丢弃"，不执行任何动作，包括前驱状态的退出动作。这意味着守卫条件（如果存在）必须在执行任何动作之前进行求值。守卫条件存在时，由方括号分隔。最后，可能有一个动作列表，在转换执行时被执行。动作列表前面有一个斜杠标记。

应当注意，所有这些字段都是可选的。当触发事件缺失时（有时称为"匿名转换"），则一旦元素进入前驱状态，即前驱状态的进入动作完成后，该迁移就会被触发。如果匿名转换的守卫条件求值为假，则匿名事件被丢弃，即使守卫条件之后变为真，元素仍保持在前驱状态。

有一种特殊的迁移，称为初始转换（如果想显得更专业，也可称为"默认伪状态"），它指示元素创建时的起始状态和动作。该迁移可以没有触发事件。

有一种特殊的状态，称为最终伪状态，它指示元素不再接受事件。通常，这意味着该元素即将被删除。最终状态的图标是一个未填充圆圈内的实心圆。在图5-4中，当元素处于SubsequentState且元素接收到完成事件时，进入该伪状态。

5.2.1 或状态

在简单状态机中，一个元素必须始终恰好处于其状态之一，不多也不少。因此，这类状态被称为或状态。UML状态机通过引入状态内嵌套的概念扩展了或状态的概念。嵌套状态为有限状态机世界增加了一个抽象层次

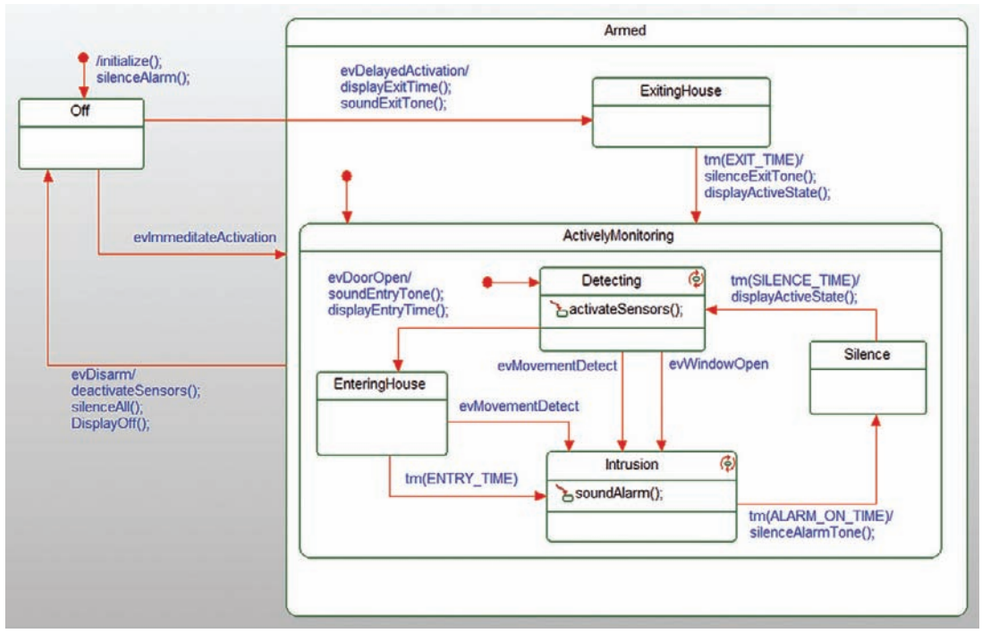


Figure 5-5: OR-states[2]

to the FSM world and this requires a minor modification to the basic rule: the element must be in exactly one state *at each level of abstraction*.

Figure 5-5 shows some nested states for the monitoring part of a home alarm system. The system must either be `Off` or `Armed` – no other conditions are possible. If it is `Armed`, then it must either be in the `ExitingHouse` or `ActivelyMonitoring` states. And, if it is `ActivelyMonitoring`, then it must be in exactly one of the following states: `Detecting`, `EnteringHouse`, `Intrusion`, or `Silence`.

There are a few interesting things to note. First, there must be a default pseudostate at every level of abstraction and it indicates the default state when that level of abstraction is entered. This default can be bypassed when desired, however. If you take the "default" transition path from the `Off` state via the `evImmediateActivation` transition, the system ends up in the `Armed::ActivelyMonitoring::Detecting` state. However, if the user activates the alarm from inside the house, the system gives them a couple of minutes to get out of the house before beginning detecting – this is the `evDelayedActivation` transition path. Also note that regardless of which deeply nested state of `Armed` the system is in, when the system is deactivated

[2] Thanks to Mark Richardson of LDRA for the idea of this state machine example.

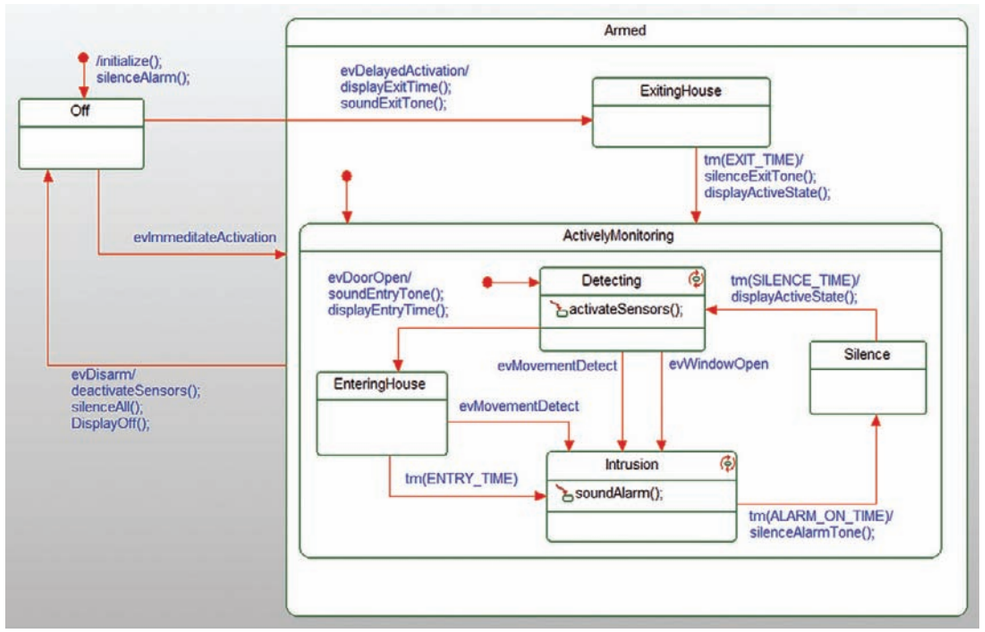


图5-5：或状态[2]

，这需要对基本规则做一个小修改：该元素在每个抽象层次上必须恰好处于一个状态。

图 5-5 展示了家庭报警系统监控部分的一些嵌套状态。系统必须处于关闭 或布防 状态——不可能存在其他条件。如果处于布防状态，则必须处于离家 或主动监控 状态。并且，如果处于主动监控状态，则必须恰好处于以下状态之一：检测、进屋、入侵或静音。

有几点值得注意。首先，在每个抽象层次上都必须有一个默认伪状态，它指示进入该抽象层次时的默认状态。不过，在需要时可以绕过这个默认状态。如果从 `Off` 状态沿 `evImmediateActivation` 迁移走"默认"迁移路径，系统最终会进入 `Armed::ActivelyMonitoring::Detecting` 状态。然而，如果用户从房屋内部激活 alarm，系统会给他们几分钟时间离开房屋，然后才开始检测——这就是 `evDelayedActivation` 迁移路径。另请注意，无论系统处于 `Armed` 的哪个深层嵌套状态，当系统被停用时

[2] 感谢LDRA的Mark Richardson提供这个状态机示例的创意。

Table 5-2: OR-state table

	Initial	evDelayedActivation	evImmediateActivation	evDisarm	evDoorOpen	evMovementDetect	evWindowOpen	Null
ROOT	Off							
Off		ExitingHouse	Armed					
Armed	ActivelyMonitoring			Off				
ExitingHouse								ActivelyMonitoring
ActivelyMonitoring	Detecting							
Detecting					EnteringHouse	Intrusion	Intrusion	
EnteringHouse						Intrusion		Intrusion
Intrusion								Silence
Silence								Detecting

via the `evDisarm` transition, the system transitions to the `Off` state. This transition can be activated whenever the system is in the `Armed` state, regardless of which nested state it is currently in.

As a side note, the state diagram in Figure 5-5 can also be shown in tabular form, as it is in Table 5-2.

5.2.2 AND-States

Sometimes state machines are complex in the sense that they have independent (or at least mostly so) aspects. Consider the state machine for a light that has two independent aspects; color (`Off`, `Red`, `Yellow`, and `Green`) and display style (`Off`, `Steady`, `Flashing Slowly`, and `Flashing Quickly`). How many states of this light are there? If you look at Figure 5-6,

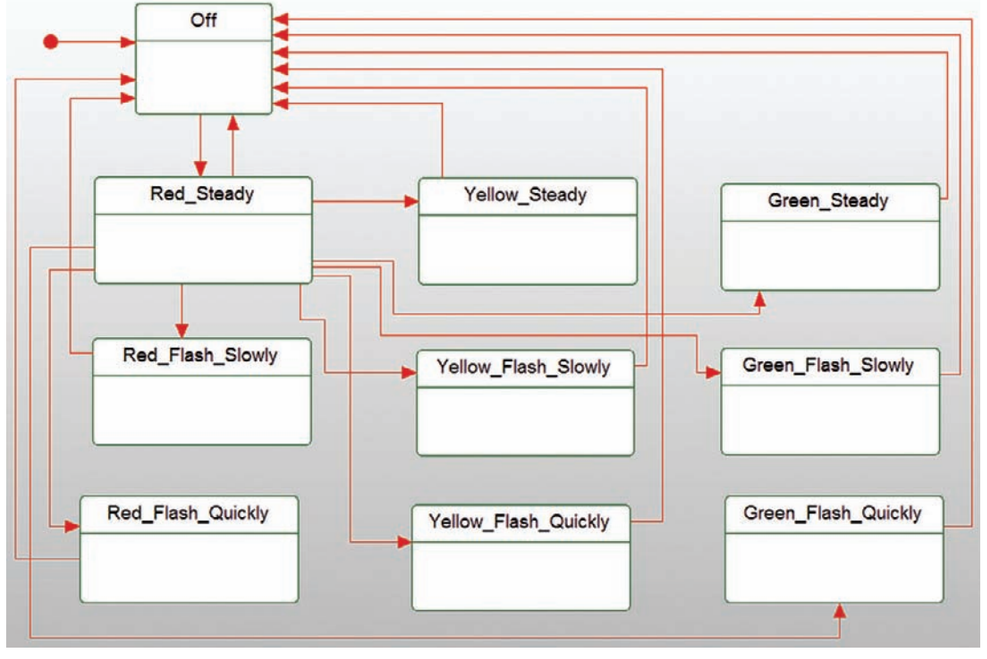


Figure 5-6: Light states

表5-2：OR状态表

	Initial	evDelayedActivation	evImmediateActivation	evDisarm	evDoorOpen	evMovementDetect	evWindowOpen	Null
ROOT	Off							
Off		ExitingHouse	Armed					
Armed	ActivelyMonitoring			Off				
ExitingHouse								ActivelyMonitoring
ActivelyMonitoring	Detecting							
Detecting					EnteringHouse	Intrusion	Intrusion	
EnteringHouse						Intrusion		Intrusion
Intrusion								Silence
Silence								Detecting

通过 `evDisarm` 迁移，系统会迁移到 `Off` 状态。只要系统处于 `Armed` 状态，无论当前处于哪个嵌套状态，都可以激活此迁移。

作为补充说明，图 5-5 中的状态图也可以以表格形式表示，正如表 5-2 中所示。

5.2.2 AND状态

有时状态机之所以复杂，是因为它们具有独立的（或至少大部分独立）方面。考虑一个具有两个独立方面的灯的状态机：颜色（关闭、红色、黄色和绿色）和显示样式（关闭、常亮、慢速闪烁和快速闪烁）。这盏灯有多少个状态？如果你看图5-6，

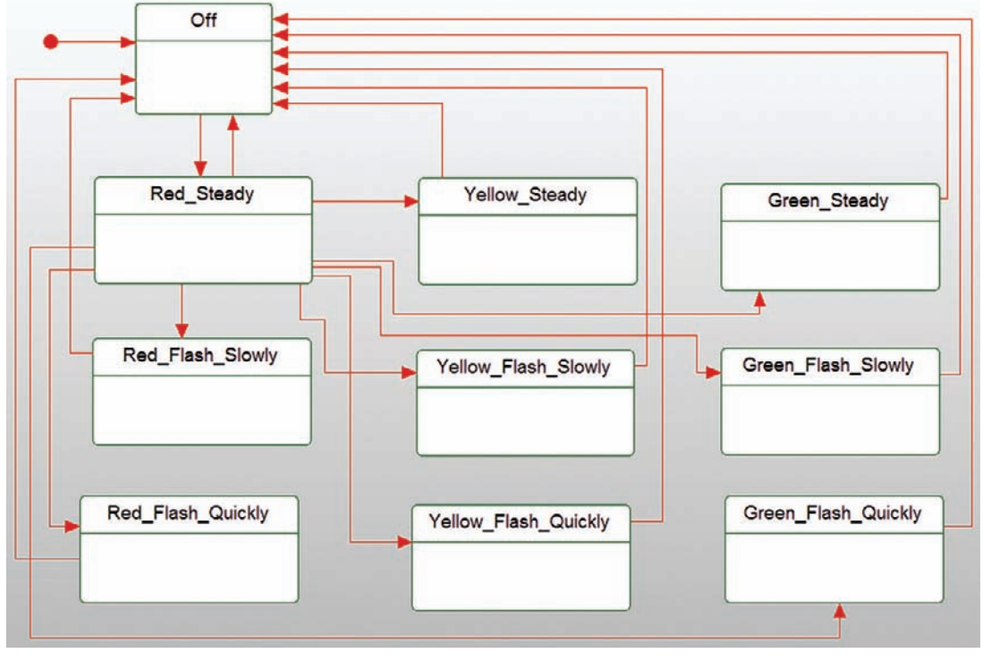


图5-6：灯状态

you see that there are 10. Further, in this light, it is possible to go from any of these states to any other. The figure shows one source state going to all other states (`Red_Steady`) and one target state receiving transitions from all other states (`Off`); you can imagine the rat's nest that would result from actually drawing all transition paths.

You can see that to consider all the states of the independent aspects, you must multiply the independent state spaces together, resulting in a possibly enormous set of states (imagine now adding a third independent aspect, brightness – low, medium, and high – into the mix). This problem is so common it even has a name – *state explosion*. UML state machines have a solution to the state explosion problem – AND-states.

AND-states are special independent states that are always nested inside other states. The special rule for AND-states is that the element must be in exactly one of each active AND-state at each level of abstraction. In the case of the light, we'll have one AND-state for color and another for display rate. You can see that this simplifies the problem so much that we have no problem representing all the transitions in the state machine even though we couldn't before (and maintain any semblance of readability). This is shown in Figure 5-7. The dashed line in the middle of the `On` state is an AND-line and separates the two AND-states.

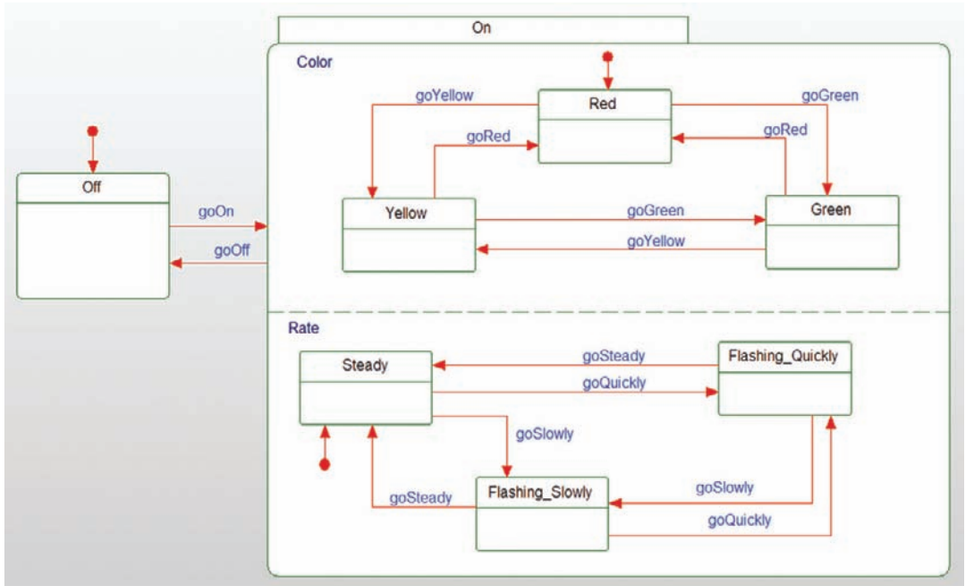


Figure 5-7: Light AND-states

你会发现共有10个状态。此外，在这盏灯中，可以从这些状态中的任何一个转移到任何其他状态。该图展示了一个源状态指向所有其他状态（红色_常亮），以及一个目标状态接收来自所有其他状态的迁移（关闭）；你可以想象，如果实际绘制所有迁移路径，将会形成多么错综复杂的网状结构。

可以看到，要考虑独立方面的所有状态，必须将独立的状空间相乘，从而产生可能极其庞大的状态集（想象一下现在再加入第三个独立方面——亮度——低、中、高——到混合中）。这个问题如此普遍，甚至有一个名称——状态爆炸。UML状态机对状态爆炸问题有一个解决方案——AND状态。

AND状态是特殊的独立状态，它们总是嵌套在其他状态内部。AND状态的特殊规则是：在每一抽象层次上，元素必须恰好处于每个活动AND状态之一中。对于灯的例子，我们将有一个用于颜色的AND状态和另一个用于显示速率的AND状态。可以看到，这极大地简化了问题，以至于我们能够毫无困难地表示状态机中的所有迁移，而在此之前我们无法做到这一点（同时保持任何可读性）。如图5-7所示。On状态中间的虚线是一条AND线，将两个AND状态分隔开。

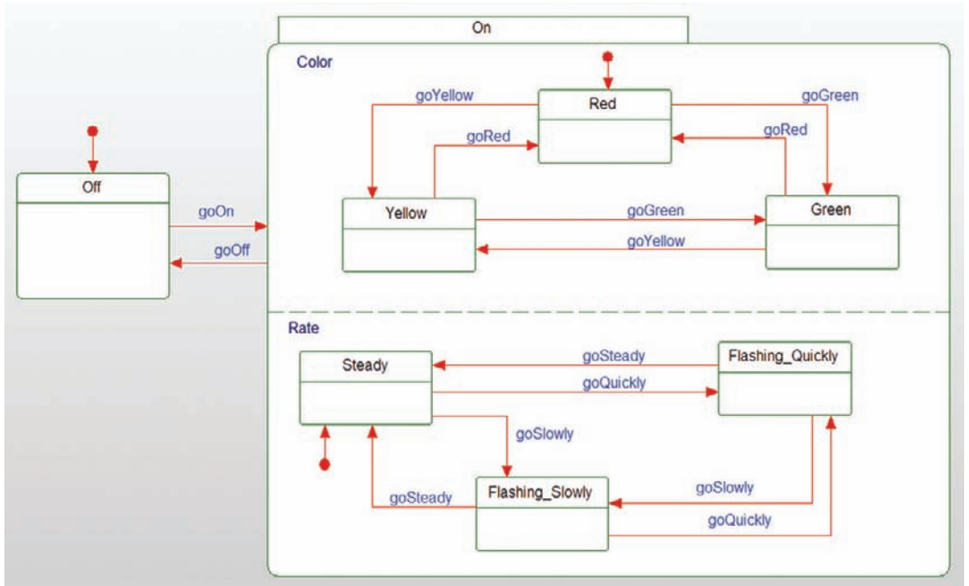


图5-7：Light AND状态

Table 5-3: Light AND-state table

	Initial	goSteady	goQuickly	goSlowly	goYellow	goGreen	goRed	goOn	goOff
ROOT	Off								
Off								On	
On									Off
Color	Red								
Green					Yellow		Red		
Yellow						Green	Red		
Red					Yellow	Green			
Rate	Steady								
Flashing_Slowly		Steady	Flashing_Quickly						
Steady			Flashing_Quickly	Flashing_Slowly					
Flashing_Quickly		Steady		Flashing_Slowly					

And, yes, the state machine in Figure 5-7 can also be shown in a table. See Table 5-3. You just have to pay special attention to the icon of the `On` state to notice that the nested states are AND-states and not OR-states to correctly interpret the table.

5.2.3 Special Stuff: Timeouts, Conditionals, and More

All FSMs can be ultimately reduced to the core elements discussed above. However, the UML standard adds a number of syntactic elements that simplify representing complex situations. Most of these fall into the category of *pseudostates*, an unfortunate name for a set of annotations that are in *no way* states, but are nonetheless useful.

First, let's discuss timeouts semantics. The UML has the notion that elapsed time is a kind of event that can cause a change of state in a state machine. The notation for this is normally either `tm(<time interval>)` or `after(<time interval>)`. The way to think about a timeout transition is as follows. When entering a state that has an exiting timeout transition, a logical timer begins as soon as the state is entered (i.e., as soon as the state's exit actions complete). If the element is still in the state when the timeout expires, then the transition triggered by the timeout fires. If the element leaves that state and reenters it later, the timer starts over from zero.

The state machine in Figure 5-8 shows how timeout transitions might be used. When the system is created, it begins in the Off state. Once it receives the start event, the system transitions to the `Waiting` state. In `WAIT_TIME` milliseconds later[3], the system transitions to the `DoneWaiting` state. Let's suppose that `WAIT_TIME` is set to 1000 milliseconds. If the system remains in the `Waiting` state for a full second, then it will transition to the `DoneWaiting` state. However, let's suppose that 950 ms after entering the `Waiting` state, the system receives the pause event; in this case, the timer (logically) disappears and the system enters the `Paused` state. If, two days later, the system receives the resume event, it will transition back to the `Waiting` state and resume timing, but it will again wait a full second before timing out. Similarly, let's set `QUIT_TIME` to

[3] Milliseconds are the most common units for state implementation and will be used here, but you are free to use whatever timing resolution makes sense for you.

表 5-3：轻量级与状态表

	Initial	goSteady	goQuickly	goSlowly	goYellow	goGreen	goRed	goOn	goOff
ROOT	Off								
Off								On	
On									Off
Color	Red								
Green					Yellow		Red		
Yellow						Green	Red		
Red					Yellow	Green			
Rate	Steady								
Flashing_Slowly		Steady	Flashing_Quickly						
Steady			Flashing_Quickly	Flashing_Slowly					
Flashing_Quickly		Steady		Flashing_Slowly					

而且， 是的， 图 5-7 中的 状态 机 也 可以用表格来展示。参见表 5-3。你只需特别注意 `On` 状态的图标，以注意到嵌套状态是AND状态而非OR状态，从而正确地解读该表格。

5.2.3 特殊内容：超时、条件及其他

所有有限状态机最终都可以归结为上文讨论的核心元素。然而，UML 标准增加了若干语法元素，以简化对复杂情况的表示。其中大多数属于伪状态这一类别——这个名称对于一组注解来说并不恰当，因为它们绝不是状态，但仍然很有用。

首先，让我们讨论超时的语义。UML 中有这样一个概念：经过的时间是一种可以导致状态机状态改变的事件。其表示法通常为`tm(<时间 间隔>)` 或 `after(<时间 间隔>)`。理解超时迁移的方式如下。当进入一个具有外出超时迁移的状态时，逻辑定时器在进入该状态时立即启动（即在该状态的退出动作完成后立即启动）。如果超时到期时元素仍处于该状态，则由超时触发的迁移就会触发。如果元素离开该状态并在之后重新进入，定时器将从零重新开始。

图 5-8 中的状态机展示了如何使用超时迁移。当系统被创建时，它从关闭状态开始。一旦接收到启动事件，系统就迁移到等待 状态。在`WAIT_TIME` 毫秒之后[3]，系统迁移到`DoneWaiting`状态。假设`WAIT_TIME` 被设置为1000毫秒。如果系统在等待 状态停留整整一秒，那么它将迁移到`DoneWaiting` 状态。然而，假设在进入等待 状态950毫秒后，系统接收到暂停事件；在这种情况下，定时器（逻辑上）消失，系统进入暂停 状态。如果两天后系统接收到恢复事件，它将迁移回等待 状态并恢复计时，但它将再次等待整整一秒后才超时。类似地，假设将`QUIT_TIME` 设置为

[3] 毫秒是状态实现中最常用的单位，此处也将使用毫秒，但您可以自由选择适合您的任何时序分辨率。

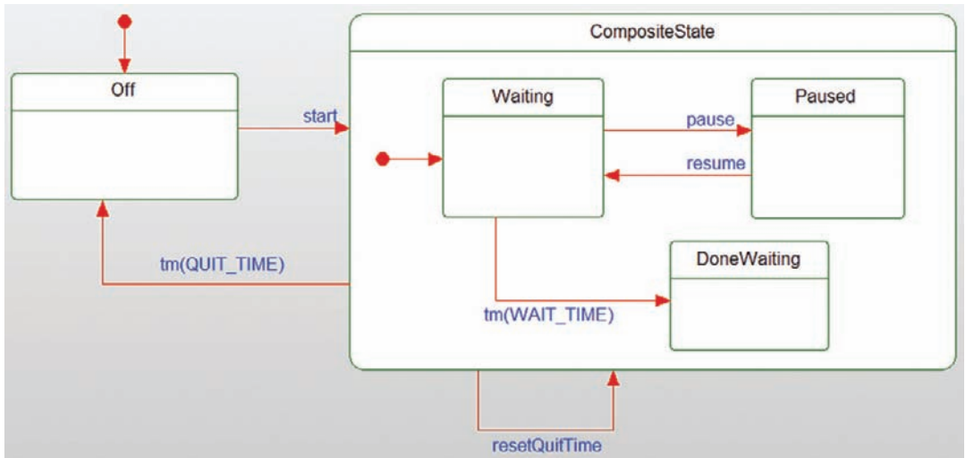


Figure 5-8: Timeouts

20,000 ms. We can effectively reset the system by issuing the `resetQuitTime` event. Let's suppose the system is in the `Paused` state and the `QUIT_TIME` timer has counted down 10,000 ms. What happens then?

The transition triggered by the `resetQuitTime` causes the system to reenter the `WaitingState` with the `WAIT_TIME` counter starting over and the `QUIT_TIME` counter also restarted. What if we want to reenter the `Paused` state? UML has a solution for that concern too – the history pseudostate.

The UML provides a number of pseudostates but we will only discuss the most common of these:

- Default pseudostate – identifies the starting state for a system or within a level of abstraction
- Conditional pseudostate – shows branch points where a transition may terminate on different target states and/or have different actions depending on the results of guards
- Fork pseudostate – allows bypassing the defaults when going into a composite with AND nested states
- Join pseudostate – allows specific nested AND-states as source states for an exit from a composite state with AND nested states
- History pseudostate – sets the default nested state to be the "last active" nested state
- Final pseudostate – indicates that the state machine no longer receives events, usually because it is about to be destroyed

Figure 5-9 shows these different annotations. The default and final pseudostates we've seen before. The conditional pseudostate is activated while the system is in the `Ready` state and the `go` event is received. If the guard `useDefaults()` returns `TRUE`, then the default nest AND-states

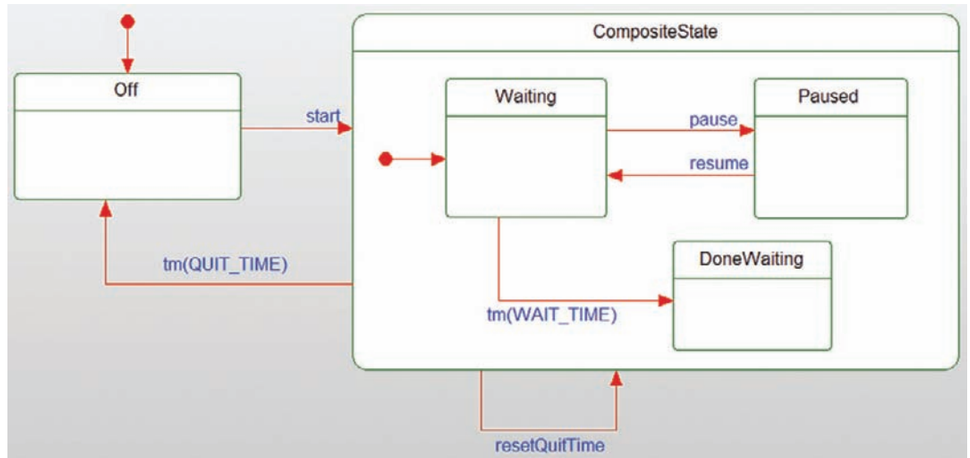


图5-8：超时

20,000毫秒。我们可以通过发出`resetQuitTime`事件来有效地重置系统。假设系统处于暂停 状态，并且`QUIT_TIME`定时器已倒计时10,000毫秒。那么会发生什么?

由`resetQuitTime`触发的迁移使系统重新进入`WaitingState`，同时`WAIT_TIME`计数器重新开始计数，`QUIT_TIME`计数器也重新启动。如果我们想重新进入`Paused`状态呢? UML对此也有解决方案——历史伪状态。

UML提供了多种伪状态，但我们只讨论其中最常见的几种:

- 默认伪状态——标识系统或某一抽象层次内的起始状态
- 条件伪状态——显示分支点，在分支点上，迁移可能终止于不同的目标状态，和/或根据守卫条件的结果执行不同的动作
- 分叉伪状态 – 允许在进入带有AND嵌套状态的复合状态时绕过默认设置
- 汇合伪状态 – 允许将特定的嵌套AND状态作为从带有AND嵌套状态的复合状态退出的源状态
- 历史伪状态 – 将默认嵌套状态设置为"最后活动"的嵌套状态
- 最终伪状态 – 表示状态机不再接收事件，通常是因为它即将被销毁

图 5-9 展示了这些不同的注解。默认伪状态和最终伪状态我们之前已经见过。条件伪状态在系统处于就绪 状态且接收到`go`事件时被激活。如果守卫条件`useDefaults()`返回`TRUE`，则进入默认的嵌套AND状态,

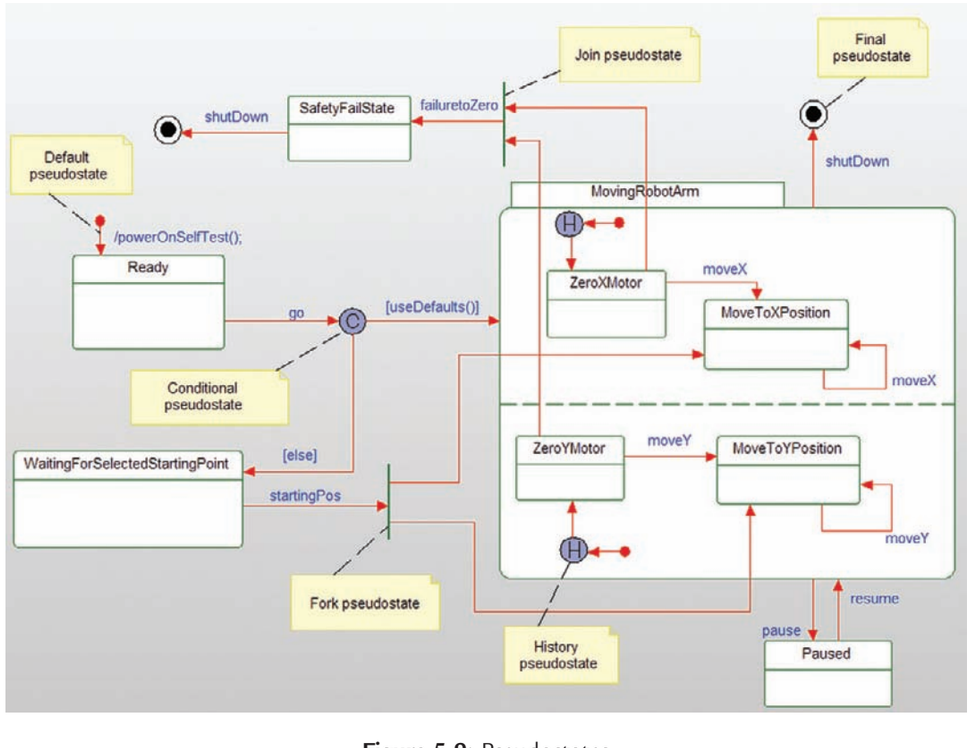


Figure 5-9: Pseudostates

are entered. Alternatively, if it is `FALSE`, the (optional) else clause routes the transition to the `WaitingForSelectedStartingPoint` state. If used, the else clause path is taken if and only if all other branching guards are `FALSE`. If all the guards evaluate to `FALSE` (impossible if the else clause is included), then the event is discarded without taking a transition.

The fork is used between the `WaitingForSelectedStartingPoint` and the `MovingRobotArm` states. In this case, the system bypasses the defaults (taken in the go [`useDefaults()`] transition) and instead transitions to two nondefault states inside the `MovingRobotArm` composite state.

Similarly, a join occurs between the `ZeroXMotor` and `ZeroYMotor` nested AND-states on one side and the `SafetyFailState` on the other. That means that the system must be in *both* the predecessor states AND the event `failuretoZero` must occur; if the system is in any other condition, the event is discarded.

Lastly, the history pseudostate is used to return to the last active nested state. In the example, when the system is in the `MovingRobotArm` state and receives the event `pause`, it transitions to the `Paused` state. A subsequent `resume` event will return to the same nested states the system

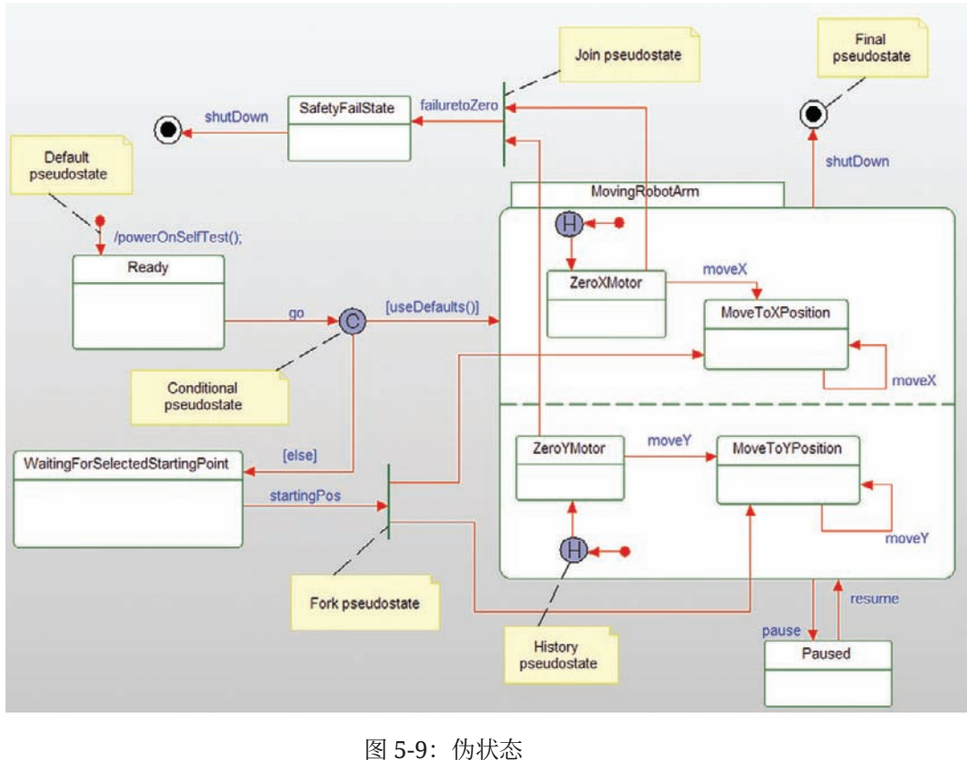


图 5-9：伪状态

或者，如果返回`FALSE`，则（可选的）else子句将迁移路由到`WaitingForSelectedStartingPoint`状态。如果使用了else子句，则当且仅当所有其他分支守卫均为`FALSE`时才会走else子句路径。如果所有守卫都求值为`FALSE`（如果包含else子句则不可能发生），则该事件被丢弃，不进行迁移。

分叉用于`WaitingForSelectedStartingPoint`和`MovingRobotArm`状态之间。在这种情况下，系统绕过默认状态（在 go [`useDefaults()`] 迁移中采用），转而迁移到`MovingRobotArm`复合状态内部的两个非默认状态。

类似地，汇合发生在一侧的`ZeroXMotor`和`ZeroYMotor`嵌套AND状态与另一侧的`SafetyFailState`之间。这意味着系统必须同时处于两个前驱状态，并且事件`failuretoZero`必须发生；如果系统处于任何其他条件，该事件将被丢弃。

最后，历史伪状态用于返回到最后一个活动的嵌套状态。在示例中，当系统处于`MovingRobotArm`状态并接收到事件`pause`时，它会迁移到`Paused`状态。随后的`resume`事件将返回到系统

was in when it was in the `MovingRobotArm` state. For example, imagine the system is in the `ZeroXMotor` and `MoveToYPosition` states when the `pause` occurs. With the `resume` event, the system will return to those two states. Without the history state, it would return to the `ZeroXMotor` and `ZeroYMotor` nested states, since they are the defaults.

5.2.4 Synchronous versus Asynchronous?

A synchronous event is activated by a call to an operation on the target state machine and the caller waits (blocked) until the state machine completes its response to the event. In contrast, an asynchronous event is sent via a send-and-forget mechanism by the sender and the sender may continue on regardless of the actions of the target state machine.

The state machine itself doesn't care how the event arrives, but must handle the event in either case. This means that to avoid race conditions, synchronous state machines must block callers (using the Guarded Call or Critical Region Patterns of the previous chapter) and asynchronous state machines must use queuing to store their events until they can be handled. While a state machine can handle both synchronous and asynchronous events, a given state machine usually handles one or the other but not both to eliminate potential race conditions.

We really want to be able to have the system respond in the same way to the same sequence of events; such a system is said to have *deterministic semantics.* There are a couple of conditions under which determinism is threatened with which any good implementation of a state machine must deal.

What should happen, for example, if a state machine is in the middle of executing an action and an incoming event occurs? Should the state machine stop immediately and handle the event? The answer, of course, is *no*. The state machine must be idle and waiting for events before handling a new one. Mealy-Moore state machines "handle" this by assuming that actions take zero time, a proposition that is not only clearly false but also complicates timing analysis. The UML has a different answer – run-to-completion semantics. That is, actions may take time, but any pending events must wait until the state machine has completed the entire exit action-transition action-entry action sequence. The problem is simplified if asynchronous events are used, since the delivery mechanism for the event can be triggered when the state machine enters a state. For synchronous events in the presence of multiple threads, though, the action sequence must block any attempt to invoke an event receptor operation until the action executions are complete.

Let's consider the basic elements of both synchronous and asynchronous state machines.

For a synchronous state machine, the owning element of the state machine will provide one or more event handler functions. The most obvious way to handle the deterministic semantics problem is to lock the semaphore when the event receptor is called and unlock it when the event receptor returns.

处于 `MovingRobotArm` 状态时所处的相同嵌套状态。例如，假设当 `pause` 发生时，系统处于 `ZeroXMotor` 和 `MoveToYPosition` 状态。通过 `resume` 事件，系统将返回到这两个状态。如果没有历史状态，它将返回到 `ZeroXMotor` 和 `ZeroYMotor` 嵌套状态，因为它们是默认状态。

5.2.4 同步与异步?

同步事件通过调用目标状态机上的操作来激活，调用者会等待（阻塞），直到状态机完成对该事件的响应。相比之下，异步事件由发送者通过发送后不管机制发送，发送者可以继续执行，而不受目标状态机动作的影响。

状态机本身并不关心事件如何到达，但无论哪种情况都必须处理该事件。这意味着为了避免竞态条件，同步状态机必须阻塞调用者（使用上一章的受保护调用或临界区模式），而异步状态机必须使用排队来存储其事件，直到能够处理它们。虽然一个状态机可以同时处理同步事件和异步事件，但给定的状态机通常只处理其中一种，而不是两者兼有，以消除潜在的竞态条件。

我们确实希望系统能够对相同的事件序列以相同的方式响应；这样的系统被称为具有确定性语义。有几种情况会威胁确定性，任何良好的状态机实现都必须处理这些问题。

例如，如果状态机正在执行某个动作的过程中，一个传入事件发生了，应该怎么办?状态机应该立即停止并处理该事件吗？答案当然是否定的。状态机必须处于空闲并等待事件的状态，然后才能处理新的事件。Mealy-Moore状态机通过假设动作耗时为归零来"处理"这一问题，这一命题不仅显然是错误的，而且还会使时序分析变得复杂。UML给出了不同的答案——运行至完成语义。也就是说，动作可以耗时，但任何待处理的事件都必须等待，直到状态机完成整个退出动作-迁移动作-进入动作序列。如果使用异步事件，问题就会简化，因为事件的投递机制可以在状态机进入某个状态时被触发。然而，对于存在多个线程情况下的同步事件，动作序列必须阻塞任何调用事件接收器操作的尝试，直到动作执行完成。

让我们考虑一下同步状态机和异步状态机的基本要素 .

对于同步状态机，状态机的所属元素将提供一个或多个事件处理函数。处理确定性语义问题最明显的方法是在调用事件接收器时对信号量加锁，并在事件接收器返回时解锁。

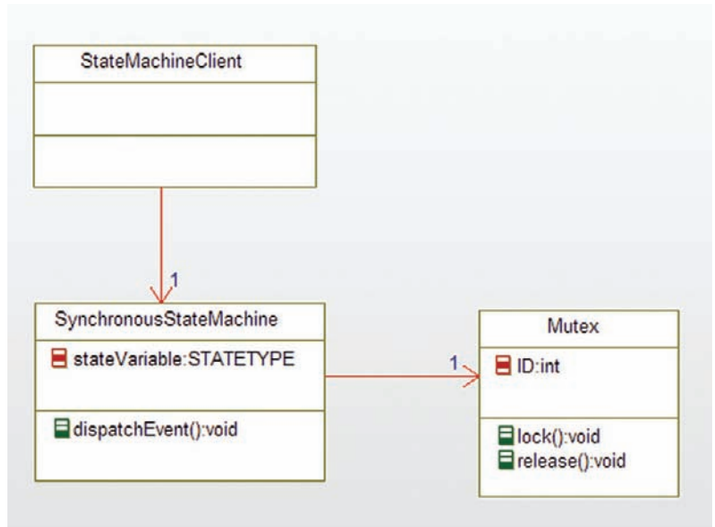


Figure 5-10: Synchronous state machine basic interaction model

Figure 5-10 shows this basic model. In the figure, the `SynchronousStateMachine` has a single `stateVariable` (of an enumerated type `STATETYPE`, although most people just encode this with integers) to track the current state of the FSM. It also has a single function `dispatchEvent` that receives the event and any data that it passes. The basic flow of logic for the function would be:

```
Mutex_lock(me->itsMutex);
/* perform list of actions */
/* update stateVariable */
Mutex_release(me->itsMutex);
```

Code Listing 5-1: Basic dispatchEvent Flow

Figure 5-11 shows the basic form for an asynchronous state machine. The first difference you see is that the client actually sends events to the queue via the `post()` operation and then queues them. This locks the queue's mutex, stores it in the queue, unlocks its `Mutex`, calls the `dispatchEvent` on the `AsynchronousStateMachine` and returns. Code Listing 5-2 shows the basic flow for the `post()` function.

```
Mutex_lock(me->itsMutex);
insert(ev); /* insert into the queue */
```

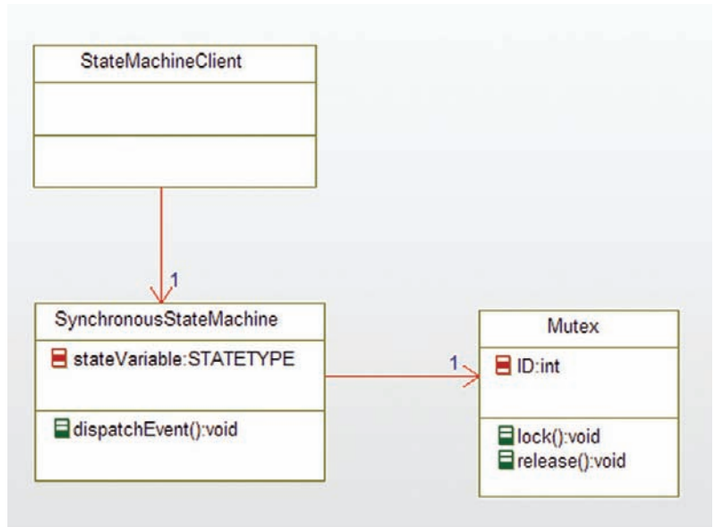



图 5-10：同步状态机基本交互模型

图 5-10 展示了这一基本模型。在图中，`SynchronousStateMachine` 有一个单一的状态变量（类型为枚举类型`STATETYPE`，尽管大多数人只是用整数来编码它）来跟踪有限状态机的当前状态。它还有一个单一的函数`dispatchEvent`，用于接收事件及其传递的任何数据。该函数的基本逻辑流程为：

```
Mutex_lock(me->itsMutex); /* 执行
动作列表 */ /* 更新 stateVariable */
Mutex_release(me->itsMutex);
```

代码清单 5-1：基本 dispatchEvent 流程

图 5-11 展示了异步状态机的基本形式。你看到的第一个区别是，客户端实际上通过`post()`操作将事件发送到队列，然后将其排队。这会锁定队列的互斥锁，将其存储在队列中，解锁其互斥锁，在`AsynchronousStateMachine`上调用`dispatchEvent`并返回。代码清单5-2展示了`post()`函数的基本流程。

```
Mutex_lock(me->itsMutex); insert(ev); /* 插入
到队列中 */
```

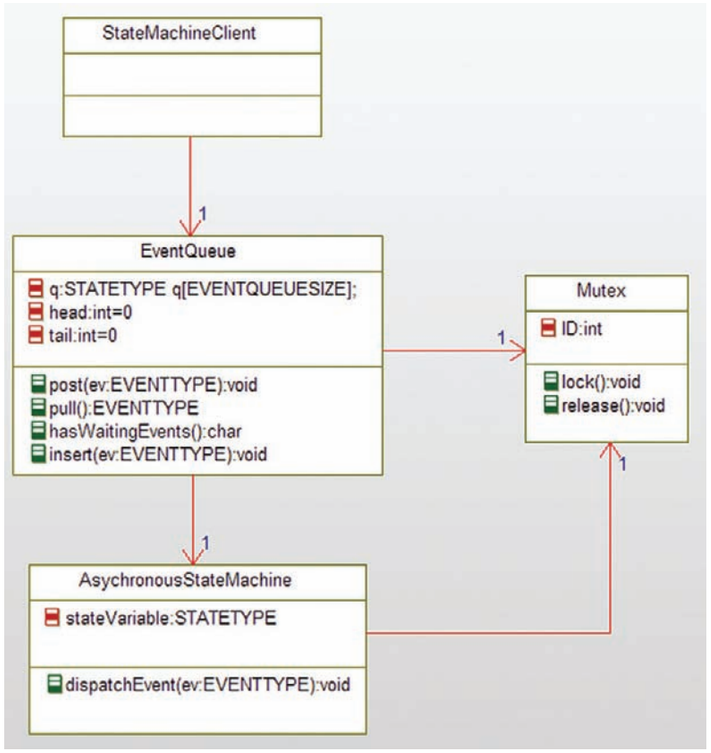


Figure 5-11: Asynchronous state machine basic interaction model

```
Mutex_release(me->itsMutex);
AsynchronousStateMachine_dispatchEvent(ev);
```

Code Listing 5-2: Basic post flow

This ensures that the `AsynchronousStateMachine` is notified when an event is posted (note that the `Mutex` associated with the `AsynchronousStateMachine` protects any active action sequence from interruption). Note that we expect the mutex in Figure 5-11 to be a recursive mutex (see the discussion in the previous chapter) so that the `EventQueue` can invoke (and be blocked by) the `Mutex` associated with the `AsynchronousStateMachine` as many times as necessary.

The `EventType` referenced in Figure 5-11 may be a simple enumerated type or something more complex. If you want to pass data along with the event, then it needs to be a struct (usually with a union clause) to hold the data. That is because the type must be stored within an event queue regardless of the size and subcomponents of the data being passed. In the case of a synchronous

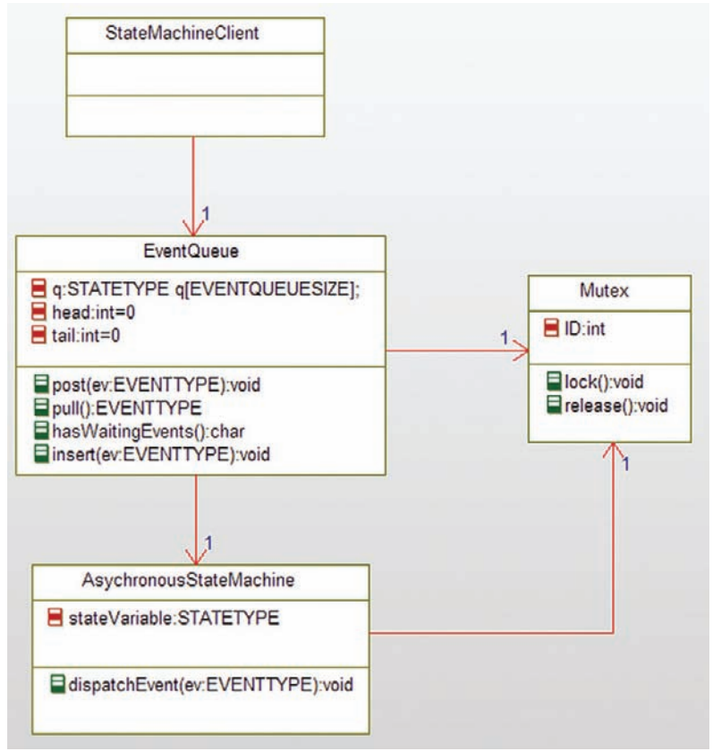


图5-11：异步状态机基本交互模型

```
Mutex_release(me->itsMutex);
AsynchronousStateMachine_dispatchEvent(ev);
```

代码清单5-2：基本投递流程

这确保了当事件被投递时，AsynchronousStateMachine会收到通知（注意，与AsynchronousStateMachine关联的互斥锁保护任何活动动作序列免受中断）。请注意，我们期望图5-11中的互斥锁是递归互斥量（参见上一章的讨论），以便EventQueue可以根据需要多次调用（并被阻塞于）与AsynchronousStateMachine关联的互斥锁。

图5-11中引用的EventType可以是一个简单的枚举类型，也可以是更复杂的内容。如果希望随事件一起传递数据，那么它需要是一个结构体（通常带有联合体子句）来保存数据。这是因为无论所传递数据的大小和子组件如何，该类型都必须存储在事件队列中。在同步

state machine, this can be encoded directly in the various event receptors' argument lists. For example, the follow code snippet shows the definition of three event receptors for a synchronous state machine that each take different arguments:

```
void event1(FSM* me); /* no arguments */
void event2(FMS* me, int n, double torque); /* two scalar arguments */
void event3(FSM* me, struct NavData* nd); /* pointer to struct */
```

Code Listing 5-3: Event receptor data passing

Note that because the synchronous state machine can call a specific function for each different event separately, there is no need to encode the event type explicitly, unlike the case in which events must be queued for later consumption.

The above discussion is applicable regardless of exactly how we implement the state machine. Let us now turn our attention to different implementation patterns for finite state machines. In all cases, we will implement the state machine shown in Figure 5-12. This state machine implements a simple parser for input characters and computes a numeric result. This works for such character strings as "12.34" and ".1234").

It is assumed that the client has already determined, on a character-by-character basis, whether the character is a digit ('0' to '9'), a dot ('.') or whitespace (' ', or '\t') and has generated one of

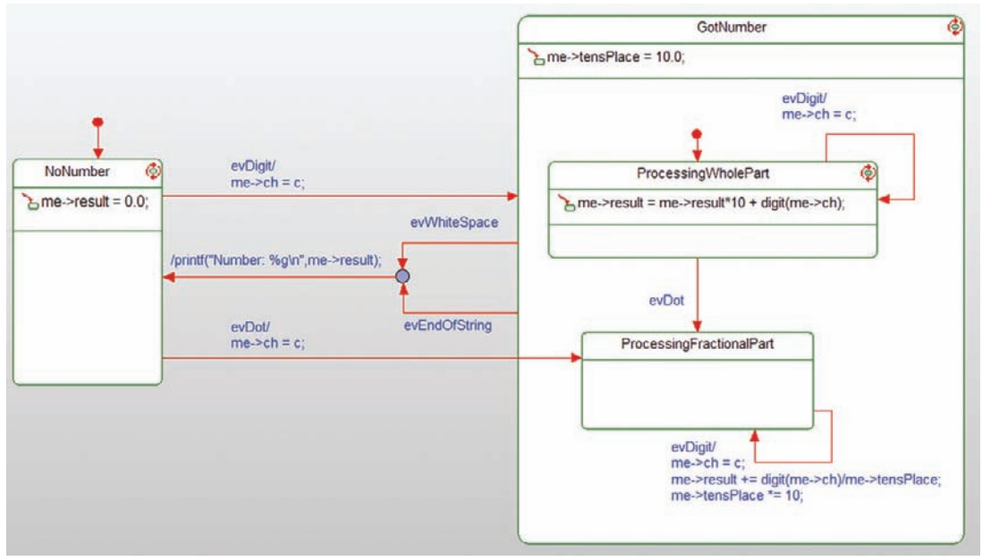


Figure 5-12: Example state machine

状态机的情况下，这可以直接编码在各个事件接收器的参数列表中。例如，以下代码片段展示了同步状态机的三个事件接收器的定义，每个接收器接受不同的参数：

```
void event1(FSM* me); /* 无参数 */ void event2(FMS* me, int n, double torque); /* 两个
标量参数 */ void event3(FSM* me, struct NavData* nd); /* 指向结构体的指针 */
```

代码清单5-3：事件接收器数据传递

请注意，由于同步状态机可以为每个不同的事件分别调用特定的函数，因此无需显式编码事件类型，这与事件必须排队以供后续消费的情况不同。

上述讨论适用于我们实现状态机的任何具体方式。现在让我们将注意力转向有限状态机的不同实现模式。在所有情况下，我们都将实现图5-12所示的状态机。该状态机实现了一个针对输入字符的简单解析器，并计算数值结果。这适用于诸如"12.34"和".1234"之类的字符串。

假定客户端已经逐字符判断了该字符是数字（'0'到'9'）、点（'.'）还是空白字符（' '或'\t'），并相应地生成了以下事件之一

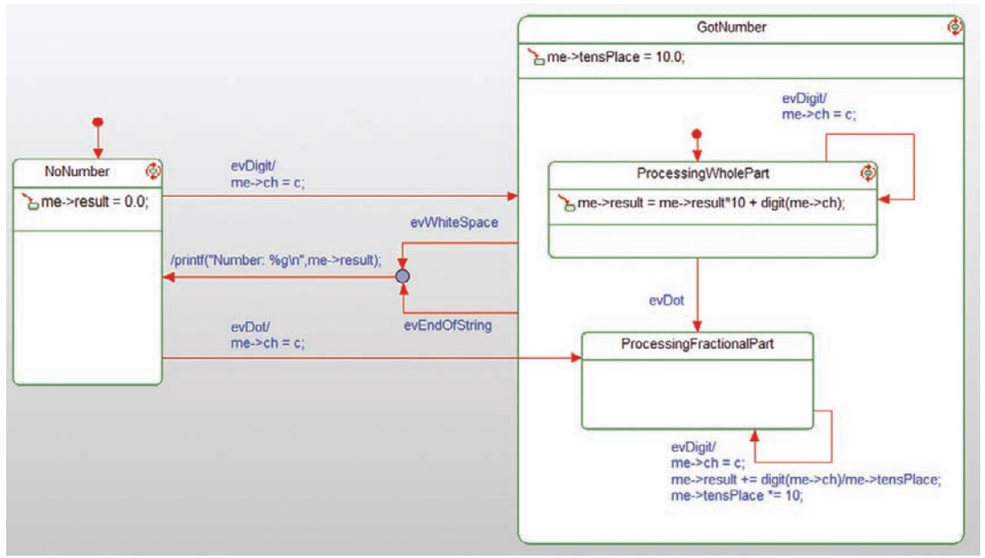


图5-12：示例状态机

the events accordingly. For multiple receptors, the client will generate one of the following events: `evDigit` (passing the character as a parameter), `evDot`, `evWhiteSpace`, or `evEndOfString`. For a single event receptor, there will be a single event `evChar` and it will have to carry data of the character itself. It is easy enough to make the character type identification within the state machine but it is left as an exercise to the reader.

5.3 Single Event Receptor Pattern

Single event receptor state machines (henceforth known as SERSMs) can be used for both synchronous and asynchronous events.

5.3.1 Abstract

The Single Event Receptor Pattern relies on a single event receptor to provide the interface between the clients and the state machine. Internally, this single event receptor must accept an event data type that not only identifies which event has occurred but also any data that accompanies the event. This pattern may be used in synchronous state machines – receiving events directly from the clients – or asynchronous, via an intermediate event queue.

5.3.2 Problem

This pattern addresses the problem of providing simple implementation of a state machine that applies to both synchronous and asynchronous event delivery.

5.3.3 Pattern Structure

This pattern can be used in two ways – for either asynchronous or for synchronous state machines. The asynchronous version context for this pattern is shown in Figure 5-13. In this case, it should be noted that at the end of the `TSREventQueue_post()` function, the OS function `postSignal()` is called. The code for the `post()` function is given in Code Listing 5-4. On the state machine side, the asynchronous version of the state machine receptor class has a task loop that waits for the signal, indicating that an event has been queued, and then iterates over all queued events. That code is shown in Code Listing 5-5.

The synchronous version of the pattern is shown in Figure 5-14. The most notable differences between the two is the absence of the event queue and that the class that owns the state machine `TokenizeSyncSingleReceptor` has an association to a mutex to ensure run-to-completion semantics for the state machine actions. Note that the state machine class has many operations but the only one called by the client is `TokenizerSyncSingleReceptor_eventDispatch()`. Internally, this function will determine which event is being passed and process the incoming event.

。对于多受体的情况，客户端将生成以下事件之一：`evDigit`（将字符作为参数传入）、`evDot`、`evWhiteSpace`或`evEndOfString`。对于单事件接收器，将只有一个事件`evChar`，并且它必须携带字符本身的数据。在状态机内部进行字符类型识别是很容易的，但这留给读者作为练习。

5.3 单事件接收器模式

单事件接收器状态机（以下简称SERSMs）既可用于同步事件，也可用于异步事件。

5.3.1 摘要

单事件接收器模式依赖单个事件接收器来提供客户端与状态机之间的接口。在内部，该单个事件接收器必须接受一种事件数据类型，该类型不仅能标识发生了哪个事件，还能包含伴随该事件的任何数据。此模式既可用于同步状态机——直接从客户端接收事件——也可用于异步状态机，通过中间事件队列接收事件。

5.3.2 问题

该模式解决的问题是：提供一种适用于同步和异步事件交付的简单状态机实现。

5.3.3 模式结构

该模式可以以两种方式使用——用于异步状态机或同步状态机。该模式的异步版本上下文如图5-13所示。在这种情况下，应当注意，在TSREventQueue_post()函数的末尾，调用了操作系统函数postSignal()。post()函数的代码见代码清单5-4。在状态机一侧，状态机接收器类的异步版本具有一个任务循环，该循环等待信号（表示有事件已入队），然后遍历所有已入队的事件。该代码见代码清单5-5。

该模式的同步版本如图 5-14所示。两者最显著的差异在于没有事件队列，并且拥有状态机的类`TokenizeSyncSingleReceptor`与一个互斥锁相关联，以确保状态机动作的运行至完成语义。注意，状态机类有许多操作，但客户端唯一调用的是`TokenizerSyncSingleReceptor_eventDispatch()`。在内部，该函数将确定正在传递的是哪个事件，并处理传入事件。

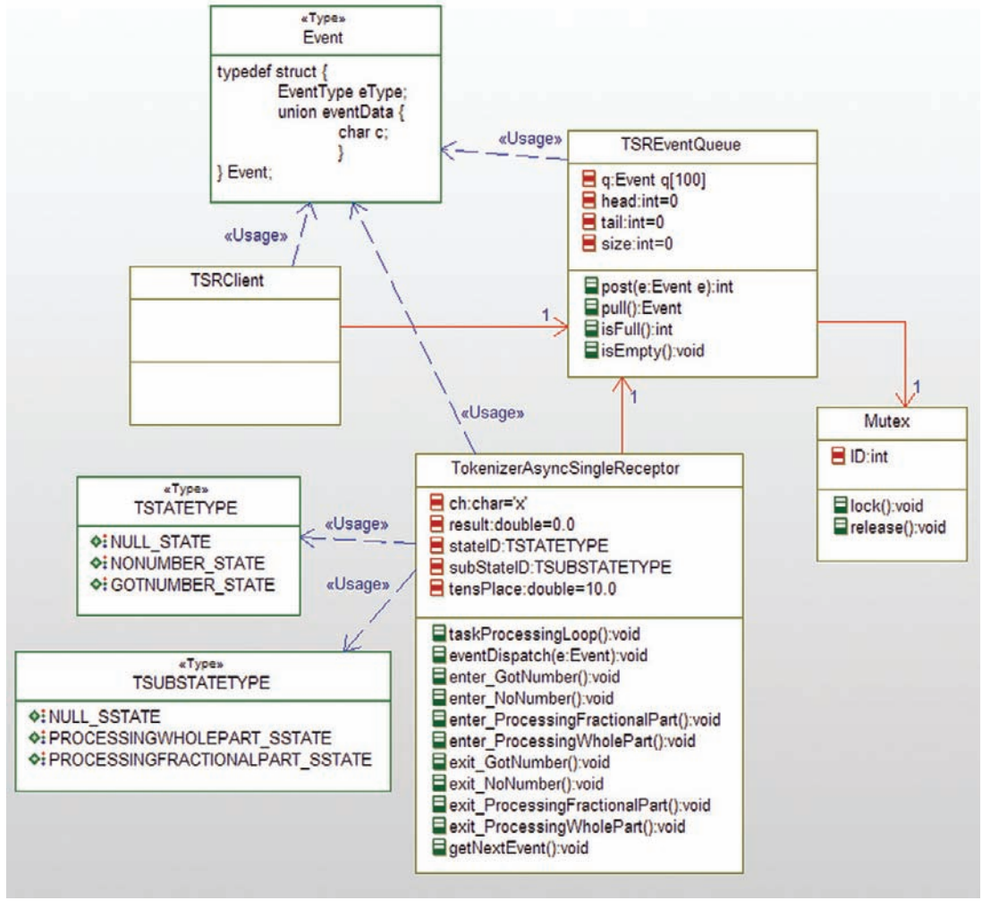


Figure 5-13: Single event receptor state machine context for asynchronous events

5.3.4 Collaboration Roles

This section describes the roles for this pattern.

5.3.4.1 Event

The `event` is a struct that contains an attribute to identify the type of the event and other data that is passed along with the different events. This is normally implemented as a `union` in C.

5.3.4.2 EventType

The `EventType` is an enumerated type for the events. Alternatively, `ints` can be used but I recommend `enums` as being more informative.

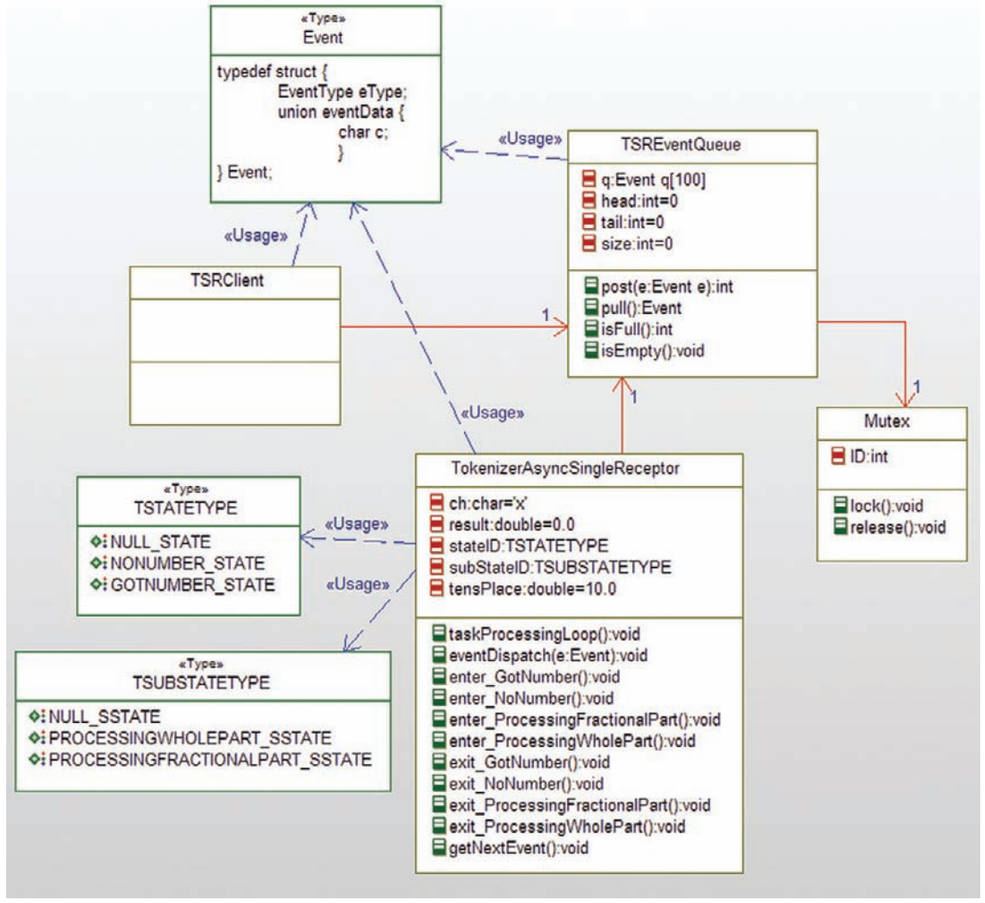


图 5-13：用于异步事件的单事件接收器状态机上下文

5.3.4 协作角色

本节描述该模式的角色。

5.3.4.1 事件

事件 是一个结构体，包含一个用于标识事件类型的属性，以及随不同事件传递的其他数据。在 C 语言中，这通常通过联合体 来实现。

5.3.4.2 EventType

`EventType` 是事件的枚举类型。也可以使用整数，但我推荐使用枚举，因为信息更丰富。

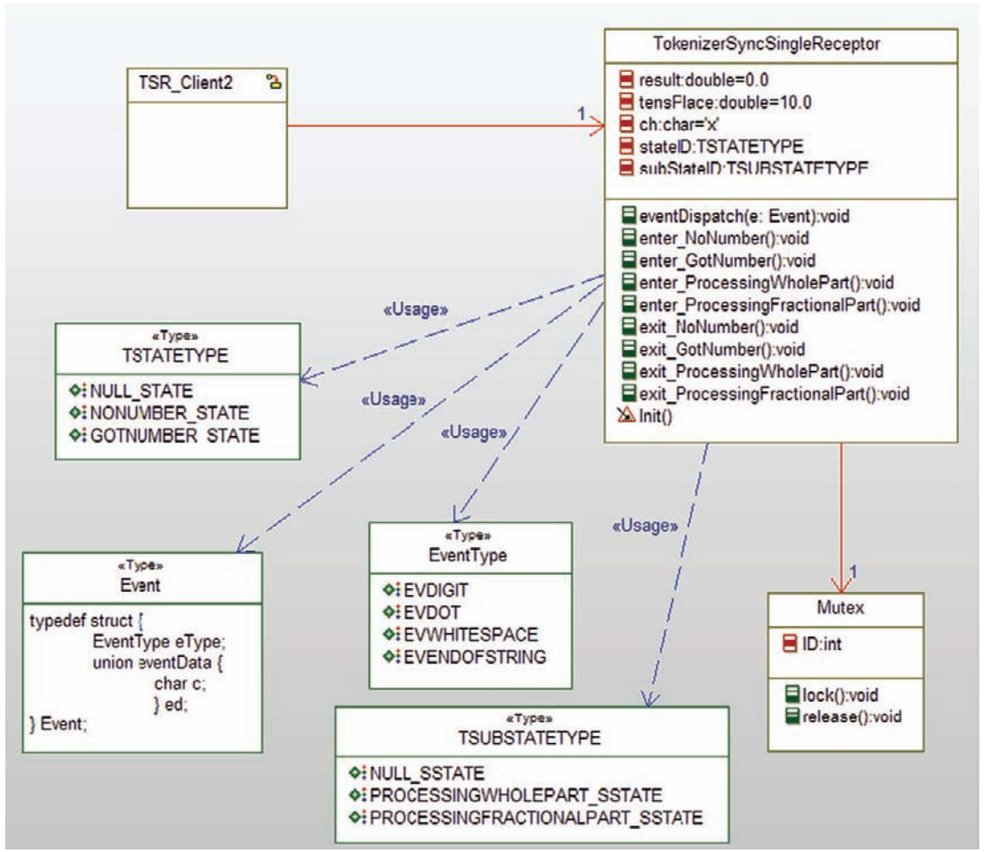


Figure 5-14: Single event receptor state machine context for synchronous events

5.3.4.3 Mutex

For the asynchronous version of the state machine, the `Mutex` ensures the integrity of the `TSREventQueue` in the case that multiple clients are simultaneously trying to post events or if a client tries to post an event while the state machine class is removing one.

For the synchronous version of the pattern, the mutex ensures the deterministic run-to-completion semantics of transitions by preventing other clients from interrupting the state machine execution of an action sequence. This is accomplished by locking the semaphore before the event is processed and releasing it once the transition processing is complete.

5.3.4.4 TokenizerAsyncSingleReceptor

This element is the owner and implementer of the desired state machine (see Figure 5-12). It has two kinds of attributes: those involved with the intent of the state machine (such as `result`)

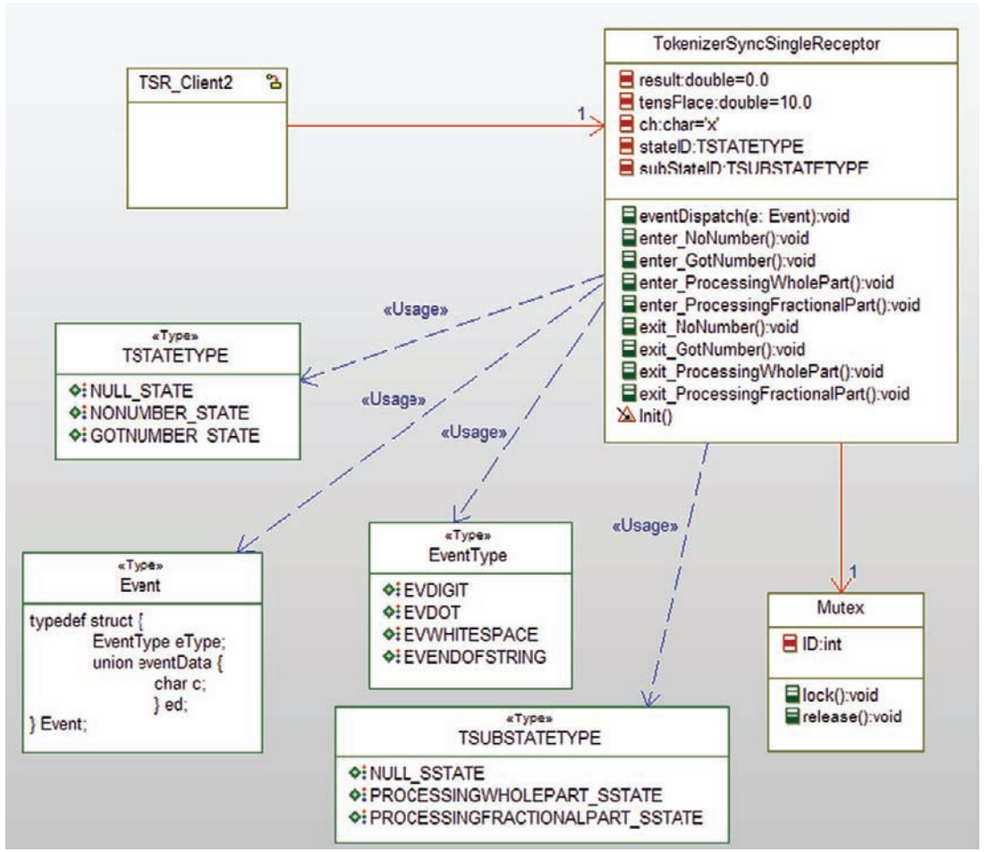


图 5-14：同步事件的单事件接收器状态机上下文

5.3.4.3 互斥锁

对于状态机的异步版本，互斥锁 确保`TSREventQueue` 的完整性，以应对多个客户端同时尝试投递事件，或某个客户端在状态机类移除事件时尝试投递事件的情况。

对于该模式的同步版本，互斥锁通过防止其他客户端中断状态机对动作序列的执行，来确保转换的确定性运行至完成语义。这是通过在事件被处理之前对信号量加锁，并在转换处理完成后释放它来实现的。

5.3.4.4 TokenizerAsyncSingleReceptor

此 元素 是 所需 状态机 的 拥有者 和 实现者 （参见 图 5-12）。它有两类属性：与状态机意图相关的属性（如 `result`）

and those involved with the management of the state machine (such as `stateID`). Of the former type, we find:

- `result` which holds the current value of the number being parsed
- `tensPlace` which is used in computing fractional parts of numbers, as in 0.345 of the number 10.345
- `ch` which holds the current character being processed

Of the latter type, we see:

- `stateID` which stores the current high level state (`NULL_STATE`, `NONUMBER_STATE`, or `GOTNUMBER_STATE`)
- `subStateID` which stores the current substate (`NULL_SSTATE`, `PROCESSINGWHOLEPART_SSTATE`, or `PROCESSINGFRACTIONALPART_SSTATE`)

These values are defined in the enumerated types `TSTATETYPE` and `TSUBSTATETYPE`, respectively.

Functions of this state machine are likewise of three types. The first is the task loop that pends on the signal (which is posted when an event is enqueued). The second is the eventDispatch that identifies and processes the event. The third type of operations are for the entry and exit actions of the states. Note that the transitions actions are all embedded within the event-receptors since they only occur once, but there are often many paths into and out of a state, so we explicitly create entry and exit functions for each state, making them reusable.

5.3.4.5 TokenizerSyncSingleReceptor

This class is exactly like the previously described `TokenizerAsyncSingleReceptor` except that 1) it does not have a task loop because it is invoked synchronously, and 2) it uses a mutex to enforce run-to-completion semantics of its state machine. The only function directly invoked by the client is `TokenizerSyncSingleReceptor_eventDispatch()`.

5.3.4.6 TSRClient

This is the client of the state machine. For the asynchronous version, it queues the event by calling `TSREventQueue_post()` function using an instance of the event type as the passed parameter. For the synchronous case, it invokes the single event receptor `TokenizerSyncSingleReceptor` and passes the appropriate event (and associated data) as typed with the event type.

5.3.4.7 TSREventQueue

This is a queue that holds events. It is only used in the asynchronous event processing case, to hold the incoming events until the state machine gets around to processing them. It provides two primary functions. The `TSREventQueue_post()` function enqueues an event and signals

以及与状态机管理相关的属性（如 `stateID`）。对于前一种类型，我们发现：

- `result` 保存正在解析的数字的当前值
- `tensPlace` 用于计算数字的小数部分，如数字 10.345 中的 0.345

- `ch` 保存当前正在处理的字符

对于后一种类型，我们可以看到：

- stateID 存储当前的高层状态（NULL_STATE、NONUMBER_STATE 或 GOTNUMBER_STATE）
- subStateID 用于 存储 当前 子状态 的 (NULL_SSTATE、PROCESSINGWHOLEPART_SSTATE，或 PROCESSINGFRACTIONALPART_SSTATE)

这些值分别定义在枚举类型 `TSTATETYPE` 和 `TSUBSTATETYPE` 中。

该状态机的函数同样分为三种类型。第一种是任务循环，它挂起在信号上（当事件入队时会发送该信号）。第二种是 eventDispatch，用于识别并处理事件。第三种操作是用于状态的进入和退出动作。请注意，转移动作全部嵌入在事件接收器内，因为它们只发生一次，但进入和离开一个状态通常有许多路径，因此我们为每个状态显式创建进入和退出函数，使其可复用。

5.3.4.5 TokenizerSyncSingleReceptor

这个类与前面描述的 `TokenizerAsyncSingleReceptor` 完全相同，区别在于：1) 它没有任务循环，因为它是同步调用的；2) 它使用互斥锁来强制其状态机的运行至完成语义。客户端直接调用的唯一函数是 `TokenizerSyncSingleReceptor_eventDispatch()`。

5.3.4.6 TSRClient

这是状态机的客户端。对于异步版本，它通过使用事件类型的实例作为传入参数调用 `TSREventQueue_post()` 函数来将事件排队。对于同步情况，它调用单事件接收器 `TokenizerSyncSingleReceptor`并传递以事件类型标注的适当事件（及相关数据）。

5.3.4.7 TSREventQueue

这是一个存放事件的队列。它仅用于异步事件处理场景，用于保存传入事件，直到状态机开始处理它们。它提供两个主要函数。`TSREventQueue_post()` 函数将事件入队并发出信号

the OS that an event has arrived. The `TSREventQueue_pull()` function is invoked by the `TSRAsyncSingleReceptor` class to get the next pending event.

5.3.4.8 TSTATE

`TSTATE` is an enumerated type of the high-level states.

5.3.4.9 TSUBSTATE

`TSUBSTATE` is an enumerated type of the nested states. In this case, there are only two levels of nesting. If there were more, then more enumerated types could be added.

5.3.5 Consequences

This pattern provides a straightforward state machine implementation easily adapted to either synchronous or asynchronous state machine event passing. The fact that all the state logic is encapsulated within a single event receptor limits the scalability of the approach.

5.3.6 Implementation Strategies

It should be noted that *one way* of implementing this pattern is by simply taking a Multiple Event Receptor Pattern and adding a single publicly available event receptor (to be invoked by clients) that calls the multiple event receptors already present. You'll still need to create the abstract event type to be passed into the single event receptor. This is one way to take a synchronous state machine done using that pattern and enabling it to be used in an asynchronous way – add an event queue, a polling mechanism to look for new events, a single event receptor and you can use it without other modification.

A more common implementation of the Single Event Receptor Pattern is to encode all the logic in a big switch-case statement within the single event receptor.

5.3.7 Related Patterns

The synchronous version of the pattern uses the Guarded Call Pattern (see Chapter 4) to ensure deterministic run-to-completion of transitions while the asynchronous version uses the Queuing Pattern. The Critical Region Pattern can be used instead for either synchronous or asynchronous state machines to ensure the run-to-completion semantics.

5.3.8 Example

The asynchronous version differs in a couple of ways from the synchronous version. First, let's consider what is special about the asynchronous version.

The most obvious difference is, of course, the presence of the `TSREventQueue`. The clients don't invoke the event receptor on the state machine directly; instead they send the events to the event queue, which then posts the signal to the OS on which the state machine pends (see Code

通知操作系统有事件到达。`TSREventQueue_pull()` 函数由`TSRAsyncSingleReceptor`类调用，以获取下一个待处理事件。

5.3.4.8 TSTATE

`TSTATE` 是高层状态的枚举类型。

5.3.4.9 TSUBSTATE

`TSUBSTATE` 是嵌套状态的枚举类型。在这种情况下，只有两个嵌套层级。如果有更多层级，则可以添加更多枚举类型。

5.3.5 结果

该模式提供了一种直接的状态机实现方式，易于适配同步或异步状态机的事件传递。所有状态逻辑都封装在单个事件接收器内，这一事实限制了该方法的可扩展性。

5.3.6 实现策略

应当指出，实现该模式的一种方式是简单地采用多事件接收器模式，并添加一个公开可用的事件接收器（供客户端调用），该接收器调用已存在的多个事件接收器。你仍然需要创建要传入单事件接收器的抽象事件类型。这是一种将使用该模式完成的同步状态机改造为可异步使用的方法——添加一个事件队列、一个用于查找新事件的轮询机制、一个单事件接收器，即可无需其他修改地使用它。

单事件接收器模式更常见的实现方式是将所有逻辑编码在单事件接收器内的一个大型 switch-case 语句中。

5.3.7 相关模式

该模式的同步版本使用受保护调用模式（见第4章）来确保迁移的确定性运行至完成，而异步版本则使用排队模式。对于同步或异步状态机，可以改用临界区模式来确保运行至完成语义。

5.3.8 示例

异步版本与同步版本在几个方面有所不同。首先，让我们考虑异步版本的特殊之处。

当然，最明显的区别是 `TSREventQueue` 的存在。客户端不直接调用状态机上的事件接收器；相反，它们将事件发送到事件队列，然后事件队列将信号发送到操作系统，状态机在该操作系统上等待（见代码

Listing 5-4). The state machine class has its own task processing loop that waits for the signal to be posted, and when it is, loops to remove and process all events that are currently queued (Code Listing 5-5).

```
int TSREventQueue_post(Event e) {
    Mutex_lock(me->itsMutex);
    if (!TSRQueue_isFull(me)) {
        me->q[me->head] = e;
        me->head = (me->head + 1) % QSIZE;
        me->size += 1;
        Mutex_release(me->itsMutex);
        postSignal(); /* signal that an event is present */
        return 1;
    }
    else {
        Mutex_release(me->itsMutex);
        return 0;
        };
};
```

Code Listing 5-4: TSREventQueue_post() function

```
void TokenizerAsyncSingleReceptor_taskLoop() {
    while (1) {
        waitOnSignal(); /* wait until event occurs */
        while (! TSREventQueue_isEmpty(me->itsTSREventQueue)) {
            TokenizerAsyncSingleReceptor_eventDispatch(
                TSREventQueue_pull(me->itsTSREventQueue));
            }
    }
}
```

Code Listing 5-5: TokenizeAsyncSingleReceptor_taskLoop() function

Code Listing 5-1 shows the enumerated types and other definitions used by the elements of the pattern.

```
/* this is the size of the event queue */
#define QSIZE 100

typedef enum EventType {
  EVDIGIT,
  EVDOT,
  EVWHITESPACE,
  EVENDOFSTRING
} EventType;

typedef struct {
    EventType eType;
    union eventData {
        char c;
        } ed;
```


清单5-4）。状态机类有自己的任务处理循环，该循环等待信号被发送，当信号被发送时，循环移除并处理当前队列中的所有事件（代码清单5-5）。

```
int TSREventQueue_post(Event e) { Mutex_lock(me->itsMutex); if (!
TSRQueue_isFull(me)) { me->q[me->head] = e; me->head = (me->
head + 1) % QSIZE; me->size += 1; Mutex_release(me->itsMutex);
postSignal(); /* 发出信号表示有事件存在 */ return 1; } else { Mutex_
release(me->itsMutex); return 0; }; };
```

代码清单5-4：TSREventQueue_post() 函数

```
void TokenizerAsyncSingleReceptor_taskLoop() { while (1) {
waitOnSignal(); /* 等待直到事件发生 */ while (! TSREventQueue_
isEmpty(me->itsTSREventQueue)) { TokenizerAsyncSingleReceptor_
eventDispatch( TSREventQueue_pull(me->itsTSREventQueue)); } } }
```

代码清单5-5：TokenizeAsyncSingleReceptor_taskLoop() 函数

代码清单5-1展示了该模式的元素所使用的枚举类型及其他定义。

```
/* 这是事件队列的大小 */ #define QSIZE 100
typedef enum EventType { EVDIGIT, EV
DOT, EVWHITESPACE, EVENDOFSTRING }
EventType; typedef struct { EventType
eType; union eventData { char c; } ed;
```

```
} Event;

typedef enum TSTATETYPE {
  NULL_STATE,
  NONUMBER_STATE,
  GOTNUMBER_STATE
} TSTATETYPE;

typedef enum TSUBSTATETYPE {
  NULL_SSTATE,
  PROCESSINGWHOLEPART_SSTATE,
  PROCESSINGFRACTIONALPART_SSTATE
} TSUBSTATETYPE;

/* helper function returns the digit */
/* held by a char */
int digit(char c) {
  return c-'0';
}

/*
  OS signal and wait functions for
  task synchronization
*/

void postSignal(void);
void waitOnSignal(void);
```

Code Listing 5-6: TokenizeAsyncSinglePkg.h

Code Listing 5-7 and Code Listing 5-8 show the code for the event queue used in the pattern.

```
#include "TokenizerAsyncSinglePkg.h"

struct Mutex;

typedef struct TSREventQueue TSREventQueue;
struct TSREventQueue {
  Event q[100];
  int size;
  int head;
  int tail;
  struct Mutex* itsMutex;
};

/* Constructors and destructors:*/
void TSREventQueue_Init(TSREventQueue* const me);

void TSREventQueue_Cleanup(TSREventQueue* const me);

/* Operations */

void TSREventQueue_isEmpty(TSREventQueue* const me);
int TSREventQueue_isFull(TSREventQueue* const me);
/* puts the passed event in */
/* the event queue and then */
```


} 事件;

typedef enum TSTATETYPE { NULL_STATE, NONUMBER_STATE, GOTNUMBER STATE_} TSTATETYPE; typedef enum TSUBSTATETYPE { NULL_SSTATE, PROCESSINGWHOLEPART_SSTATE, PROCESSINGFRACTIONALPART_SSTATE } TSUBSTATETYPE; /* 辅助函数返回由 char 保存的数字 */ int digit(char c) { return c-'0';} /* 用于任务同步的操作系统信号和等待函数 */ void postSignal(void); void waitOnSignal(void);

代码清单5-6：TokenizeAsyncSinglePkg.h

代码清单5-7和代码清单5-8展示了该模式中使用的事件队列的代码。

#include "TokenizerAsyncSinglePkg.h" struct Mutex; typedef struct TSREventQueue TSREventQueue; struct TSREventQueue { Event q[100]; int size; int head; int tail; struct Mutex* itsMutex; }; /* 构造函数与析构函数： */ void TSREventQueue_Init(TSREventQueue* const me); void TSREventQueue_Cleanup(TSREventQueue* const me); /* 操作 */ void TSREventQueue_isEmpty(TSREventQueue* const me); int TSREventQueue_isFull(TSREventQueue* const me); /* 将传入的事件放入 */ /* 事件队列，然后 */

```
/* calls the event receptor of */
/* the state machine. Note */
/* if the queue overflows an */
/* error code (1) is returned. */
int TSREventQueue_post(TSREventQueue* const me, Event e);

/* It is assumed that the caller has */
/* ensured that there is at least one */
/* event in the queue (via the isEmpty() */
/* funtion) prior to calling this function. */
/* Otherwise they get a default event. */
Event TSREventQueue_pull(TSREventQueue* const me);

struct Mutex* TSREventQueue_getItsMutex(const TSREventQueue* const me);

void TSREventQueue_setItsMutex(TSREventQueue* const me, struct Mutex* p_Mutex);

TSREventQueue * TSREventQueue_Create(void);

void TSREventQueue_Destroy(TSREventQueue* const me);

#endif
```

Code Listing 5-7: TSREventQueue.h

```
#include "TSREventQueue.h"
#include "Mutex.h"

static void cleanUpRelations(TSREventQueue* const me);

void TSREventQueue_Init(TSREventQueue* const me) {
  me->head = 0;
  me->size = 0;
  me->tail = 0;
  me->itsMutex = NULL;
}

void TSREventQueue_Cleanup(TSREventQueue* const me) {
  cleanUpRelations(me);
}

void TSREventQueue_isEmpty(TSREventQueue* const me) {
  return me->size <= 0;
}

int TSREventQueue_isFull(TSREventQueue* const me) {
  return me->size >= QSIZE-1;
}

/* post enqueues an event and signals that fact */
int TSREventQueue_post(TSREventQueue* const me, Event e) {
  Mutex_lock(me->itsMutex);
  if (!TSRQueue_isFull(me)) {
    me->q[me->head] = e;
        me->head = (me->head + 1) % QSIZE;
        me->size += 1;
        Mutex_release(me->itsMutex);
```


/* 调用状态机的 */ /* 事件接收器。注意 */ /* 如果队列溢出，则 */ /* 返回错误码 (1)。 */ int TSREventQueue_post(TSREventQueue* const me, Event e); /* 假定调用者在调用此函数之前 */ /* 已确保队列中至少有一个 */ /* 事件（通过 isEmpty() */ /* 函数）。 */ /* 否则他们将得到一个默认事件。 */ Event TSREventQueue_pull(TSREventQueue* const me); struct Mutex* TSREventQueue_getItsMutex(const TSREventQueue* const me); void TSREventQueue_setItsMutex(TSREventQueue* const me, struct Mutex* p_Mutex); TSREventQueue * TSREventQueue_Create(void); void TSREventQueue_Destroy(TSREventQueue* const me); #endif

代码清单5-7：TSREventQueue.h

#include "TSREventQueue.h" #include "Mutex.h" static void cleanUpRelations(TSREventQueue* const me); void TSREventQueue_Init(TSREventQueue* const me) { me->head = 0; me->size = 0; me->tail = 0; me->itsMutex = NULL; } void TSREventQueue_Cleanup(TSREventQueue* const me) { cleanUpRelations(me); } void TSREventQueue_isEmpty(TSREventQueue* const me) { return me->size <= 0; } int TSREventQueue_isFull(TSREventQueue* const me) { return me->size >= QSIZE-1; } /* post 将事件入队并发出信号表示该事实 */ int TSREventQueue_post(TSREventQueue* const me, Event e) { Mutex_lock(me->itsMutex); if (!TSRQueue_isFull(me)) { me->q[me->head] = e; me->head = (me->head + 1) % QSIZE; me->size += 1; Mutex_release(me->itsMutex);

```
        postSignal(); /* signal that an event is present */
        return 1;
    }
    else {
      Mutex_release(me->itsMutex);
      return 0;
      };
}

/* pulls removes the oldest event from the queue */
/* pull should only be called when there is an event waiting */
Event TSREventQueue_pull(TSREventQueue* const me) {
  Event e;
  Mutex_lock(me->itsMutex);
  if (!TSREventQueue_isEmpty(me)) {
     e = me->q[me->tail];
     me->tail = (me->tail + 1) % QSIZE
     size -= 1;
  };
  Mutex_release(me->itsMutex);
  return e;
}

struct Mutex* TSREventQueue_getItsMutex(const TSREventQueue* const me) {
  return (struct Mutex*)me->itsMutex;
}

void TSREventQueue_setItsMutex(TSREventQueue* const me, struct Mutex* p_Mutex) {
  me->itsMutex = p_Mutex;
}

TSREventQueue * TSREventQueue_Create(void) {
  TSREventQueue* me = (TSREventQueue *) malloc(sizeof(TSREventQueue));
  if(me!=NULL)
  TSREventQueue_Init(me);
  return me;
}

void TSREventQueue_Destroy(TSREventQueue* const me) {
  if(me!=NULL)
  TSREventQueue_Cleanup(me);
  free(me);
}

static void cleanUpRelations(TSREventQueue* const me) {
  if(me->itsMutex != NULL)
  me->itsMutex = NULL;
}
```

Code Listing 5-8: TSREventQueue.c

Of course, the real action here takes place in the class with the state machine. In this case, we'll show just the code for the asynchronous version in Code Listing 5-9 and Code Listing 5-10.

postSignal(); /* 发出信号表示存在事件 */ return 1; } else { Mutex_release(me->itsMutex); return 0; }; } /* pull 从队列中移除最旧的事件 */ /* pull 只应在有事件等待时调用 */ Event TSREventQueue_pull(TSREventQueue* const me) { Event e; Mutex_lock(me->itsMutex); if (!TSREventQueue_isEmpty(me)) { e = me->q[me->tail]; me->tail = (me->tail + 1) % QSIZE size -= 1; }; Mutex_release(me->itsMutex); return e; } struct Mutex* TSREventQueue_getItsMutex(const TSREventQueue* const me) { return (struct Mutex*)me->itsMutex; } void TSREventQueue_setItsMutex(TSREventQueue* const me, struct Mutex* p_Mutex) { me->itsMutex = p_Mutex; } TSREventQueue * TSREventQueue_Create(void) { TSREventQueue* me = (TSREventQueue *) malloc(sizeof(TSREventQueue)); if(me!=NULL)TSREventQueue_Init(me); return me; } void TSREventQueue_Destroy(TSREventQueue* const me) { if(me!=NULL)TSREventQueue_Cleanup(me); free(me); } static void cleanUpRelations(TSREventQueue* const me) { if(me->itsMutex != NULL) me->itsMutex = NULL; }

代码清单5-8：TSREventQueue.c

当然，真正的动作发生在这里带有状态机的类中。在这种情况下，我们将在代码清单5-9和代码清单5-10中仅展示异步版本的代码。

```
#include <stdio.h>
#include "TokenizerSyncSingleReceptorPkg.h"

struct Mutex;

typedef struct TokenizerSyncSingleReceptor TokenizerSyncSingleReceptor;
struct TokenizerSyncSingleReceptor {
  char ch;
  double result;
  TSTATETYPE stateID;
  TSUBSTATETYPE subStateID;
  double tensPlace;
  struct Mutex* itsMutex;
};

void
TokenizerSyncSingleReceptor_enter_GotNumber(TokenizerSyncSingleReceptor* const me);

void
TokenizerSyncSingleReceptor_enter_NoNumber(TokenizerSyncSingleReceptor* const me);

void
TokenizerSyncSingleReceptor_enter_ProcessingFractionalPart(TokenizerSyncSingleRe-
ceptor* const me);

void
TokenizerSyncSingleReceptor_enter_ProcessingWholePart(TokenizerSyncSingleReceptor*
const me);

void
TokenizerSyncSingleReceptor_exit_GotNumber(TokenizerSyncSingleReceptor* const me);

void
TokenizerSyncSingleReceptor_exit_NoNumber(TokenizerSyncSingleReceptor* const me);

void
TokenizerSyncSingleReceptor_exit_ProcessingFractionalPart(TokenizerSyncSingleRecep-
tor* const me);

void
TokenizerSyncSingleReceptor_exit_ProcessingWholePart(TokenizerSyncSingleReceptor*
const me);

void
TokenizerSyncSingleReceptor_Init(TokenizerSyncSingleReceptor* const me);

void
TokenizerSyncSingleReceptor_Cleanup(TokenizerSyncSingleReceptor* const me);

void
TokenizerSyncSingleReceptor_eventDispatch(TokenizerSyncSingleReceptor*   const   me,
Event e);

struct Mutex* TokenizerSyncSingleReceptor_getItsMutex(const TokenizerSyncSingleRecep-
tor* const me);

void
TokenizerSyncSingleReceptor_setItsMutex(TokenizerSyncSingleReceptor* const me, struct
Mutex* p_Mutex);
```


#include <stdio.h>#include "TokenizerSyncSingleReceptorPkg.h" 结构体 互斥锁; typedef 结构体 TokenizerSyncSingleReceptor TokenizerSyncSingleReceptor; 结构体 TokenizerSyncSingleReceptor { char ch; double result; TSTATETYPE stateID; TSUBSTATETYPE subStateID; double tensPlace; 结构体 互斥锁* itsMutex; }; void TokenizerSyncSingleReceptor_enter_GotNumber(TokenizerSyncSingleReceptor* const me); void TokenizerSyncSingleReceptor_enter_NoNumber(TokenizerSyncSingleReceptor* const me); void TokenizerSyncSingleReceptor_enter_ProcessingFractionalPart(TokenizerSyncSingleReceptor* const me); void TokenizerSyncSingleReceptor_enter_ProcessingWholePart(TokenizerSyncSingleReceptor* const me); void TokenizerSyncSingleReceptor_exit_GotNumber(TokenizerSyncSingleReceptor* const me); void TokenizerSyncSingleReceptor_exit_NoNumber(TokenizerSyncSingleReceptor* const me); void TokenizerSyncSingleReceptor_exit_ProcessingFractionalPart(TokenizerSyncSingleRecep

tor* const me); void TokenizerSyncSingleReceptor_exit_ProcessingWholePart(TokenizerSyncSingleReceptor* const me); void TokenizerSyncSingleReceptor_Init(TokenizerSyncSingleReceptor* const me); void TokenizerSyncSingleReceptor_Cleanup(TokenizerSyncSingleReceptor* const me); void TokenizerSyncSingleReceptor_eventDispatch(TokenizerSyncSingleReceptor* const me

事件 e); 结构体 互斥锁* TokenizerSyncSingleReceptor_getItsMutex(const TokenizerSyncSingleRecep

tor* const me); void TokenizerSyncSingleReceptor_setItsMutex(TokenizerSyncSingleReceptor* const me, struc

Mutex* p_Mutex);

```
TokenizerSyncSingleReceptor *
TokenizerSyncSingleReceptor_Create(void);

void
TokenizerSyncSingleReceptor_Destroy(TokenizerSyncSingleReceptor* const me);

#endif
```

Code Listing 5-9: TSRSyncSingleReceptor.h

```
#include "TSRSyncSingleReceptor.h"
#include "Mutex.h"

void
TokenizerSyncSingleReceptor_enter_GotNumber(TokenizerSyncSingleReceptor* const me) {
  me->tensPlace = 10.0;
}

void
TokenizerSyncSingleReceptor_enter_NoNumber(TokenizerSyncSingleReceptor* const me) {
  me->result = 0.0;
}

void
TokenizerSyncSingleReceptor_enter_ProcessingFractionalPart(TokenizerSyncSingle
Receptor* const me) {
    /* enter appropriate actions, if any here */
}

void
TokenizerSyncSingleReceptor_enter_ProcessingWholePart(TokenizerSyncSingleReceptor*
const me) {
  me->result = me->result*10 + digit(me->ch);
}

void
TokenizerSyncSingleReceptor_exit_GotNumber(TokenizerSyncSingleReceptor* const me) {
  me->subStateID = NULL_SSTATE;
}

void
TokenizerSyncSingleReceptor_exit_NoNumber(TokenizerSyncSingleReceptor* const me) {
    /* enter appropriate actions, if any here */
}

void
TokenizerSyncSingleReceptor_exit_ProcessingFractionalPart(TokenizerSyncSingle
Receptor* const me) {
  /* enter appropriate actions, if any here */
}

void
TokenizerSyncSingleReceptor_exit_ProcessingWholePart(TokenizerSyncSingleReceptor*
const me) {
  /* enter appropriate actions, if any here */
}
```


TokenizerSyncSingleReceptor * TokenizerSyncSingleReceptor_Create(void); void TokenizerSyncSingleReceptor_Destroy(TokenizerSyncSingleReceptor* const me); #endif

代码清单5-9：TSRSyncSingleReceptor.h

#include "TSRSyncSingleReceptor.h" #include "Mutex.h" void TokenizerSyncSingleReceptor_enter_GotNumber(TokenizerSyncSingleReceptor* const me) { me->tensPlace = 10.0; } void TokenizerSyncSingleReceptor_enter_NoNumber(TokenizerSyncSingleReceptor* const me) { me->result = 0.0; } void TokenizerSyncSingleReceptor_enter_ProcessingFractionalPart(TokenizerSyncSingle Receptor* const me) { /* 在此处输入适当的动作（如有） */ } void TokenizerSyncSingleReceptor_enter_ProcessingWholePart(TokenizerSyncSingleReceptor* const me) { me->result = me->result*10 + digit(me->ch); } void TokenizerSyncSingleReceptor_exit_GotNumber(TokenizerSyncSingleReceptor* const me) { me->subStateID = NULL_SSTATE; } void TokenizerSyncSingleReceptor_exit_NoNumber(TokenizerSyncSingleReceptor* const me) { /* 在此处输入适当的动作（如有） */ } void TokenizerSyncSingleReceptor_exit_ProcessingFractionalPart(TokenizerSyncSingle Receptor* const me) { /* 在此处输入适当的动作（如有） */ } void TokenizerSyncSingleReceptor_exit_ProcessingWholePart(TokenizerSyncSingleReceptor* const me) { /* 在此处输入适当的动作（如有） */ }

```
void
TokenizerSyncSingleReceptor_Init(TokenizerSyncSingleReceptor* const me) {
  me->ch = 'x';
  me->result = 0.0;
  me->tensPlace = 10.0;
  me->itsMutex = NULL;

  me->stateID = NONUMBER_STATE;
  me->subStateID = NULL_SSTATE;
}
void
TokenizerSyncSingleReceptor_eventDispatch(TokenizerSyncSingleReceptor*   const   me,
Event e) {
  Mutex_lock(me->itsMutex);
  switch (e.eType) {
     case EVDIGIT:
         switch (me->stateID) {
             case NONUMBER_STATE:
                 /* transition to GotNumber state, default substate */

             TokenizerSyncSingleReceptor_exit_NoNumber(me);
                    me->ch = e.ed.c;

             TokenizerSyncSingleReceptor_enter_GotNumber(me);
                    me->stateID = GOTNUMBER_STATE;

             TokenizerSyncSingleReceptor_enter_ProcessingWholePart(me);
                    me->subStateID = PROCESSINGWHOLEPART_SSTATE;
                    printf("Current value of result: %g\n", me->result);
                    break;
         case GOTNUMBER_STATE:
             switch (me->subStateID) {
                 case PROCESSINGWHOLEPART_SSTATE:

                 TokenizerSyncSingleReceptor_exit_ProcessingWholePart(me);
                     me->ch = e.ed.c;
                 TokenizerSyncSingleReceptor_enter_ProcessingWholePart(me);
                     printf("Current value of result: %g\n", me->result);
                     break;
                 case PROCESSINGFRACTIONALPART_SSTATE:

                 TokenizerSyncSingleReceptor_exit_ProcessingFractionalPart(me);
                     me->ch = e.ed.c;
                     me->result += digit(me->ch) / me->tensPlace;
                     me->tensPlace *= 10.0;

                 TokenizerSyncSingleReceptor_enter_ProcessingFractionalPart(me);
                     printf("Current value of result: %g\n", me->result);
                     break;
                 };
         };
     break;
case EVDOT:
     me->ch = '.';
```


void TokenizerSyncSingleReceptor_Init(TokenizerSyncSingleReceptor* const me) { me->ch = 'x'; me->result = 0.0; me->tensPlace = 10.0; me->itsMutex = 空; me->stateID = NoNumber_状态; me->subStateID = 空_SSTATE; } void TokenizerSyncSingleReceptor_eventDispatch(TokenizerSyncSingleReceptor* const me

,

事件 e) { 互斥锁_加锁(me->itsMutex); switch (e.eType) { case EVDIGIT: switch (me->stateID) { case NONUMBER状态:_/* 迁移到GotNumber状态，默认子状态 */ TokenizerSyncSingleReceptor_退出_NoNumber(me); me->ch = e.ed.c; TokenizerSyncSingleReceptor_进入_GotNumber(me); me->stateID = GOTNUMBER_状态; TokenizerSyncSingleReceptor_进入_ProcessingWholePart(me); me->subStateID = PROCESSINGWHOLEPART_SSTATE; printf("当前result的值: %g\n", me->result); break; case GOTNUMBER状态:_switch (me->subStateID) { case PROCESSINGWHOLEPART_SSTATE: TokenizerSyncSingleReceptor_退出_ProcessingWholePart(me); me->ch = e.ed.c; TokenizerSyncSingleReceptor_进入_ProcessingWholePart(me); printf("当前result的值: %g\n", me->result); break; case PROCESSINGFRACTIONALPART_SSTATE: TokenizerSyncSingleReceptor_退出_ProcessingFractionalPart(me); me->ch = e.ed.c; me->result += digit(me->ch) / me->tensPlace; me->tensPlace *= 10.0; TokenizerSyncSingleReceptor_进入_ProcessingFractionalPart(me); printf("当前result的值: %g\n", me->result); break; }; }; break; case EVDOT: me->ch = '.';

```
    switch (me->stateID) {
        case NONUMBER_STATE:
            /* transition to GotNumber state, default substate */
            TokenizerSyncSingleReceptor_exit_NoNumber(me);
            TokenizerSyncSingleReceptor_enter_GotNumber(me);
                me->stateID = GOTNUMBER_STATE;
            TokenizerSyncSingleReceptor_enter_ProcessingFractionalPart(me);
                me->subStateID = PROCESSINGFRACTIONALPART_SSTATE;
            break;
        case GOTNUMBER_STATE:
            switch (me->subStateID) {
                case PROCESSINGWHOLEPART_SSTATE:
                TokenizerSyncSingleReceptor_exit_ProcessingWholePart(me);
                TokenizerSyncSingleReceptor_enter_ProcessingFractionalPart(me);
                    me->subStateID = PROCESSINGFRACTIONALPART_SSTATE;
                    break;
            };
        };
    break;
case EVWHITESPACE:
case EVENDOFSTRING:
    switch (me->stateID) {
        case GOTNUMBER_STATE:
            switch (me->subStateID) {
                case PROCESSINGWHOLEPART_SSTATE:
                TokenizerSyncSingleReceptor_exit_ProcessingWholePart(me);
                    break;
                case PROCESSINGFRACTIONALPART_SSTATE:
                TokenizerSyncSingleReceptor_exit_ProcessingFractionalPart(me);
                    break;
                };
            TokenizerSyncSingleReceptor_exit_GotNumber(me);
                printf("Number: %g\n", me->result);
            TokenizerSyncSingleReceptor_enter_NoNumber(me);
                me->stateID = NONUMBER_STATE;
                break;
        };
        break;
    }; // end switch e.eType
  Mutex_release(me->itsMutex);
}
struct Mutex* TokenizerSyncSingleReceptor_getItsMutex(const TokenizerSyncSingleReceptor*
const me) {
  return (struct Mutex*)me->itsMutex;
}
void
TokenizerSyncSingleReceptor_setItsMutex(TokenizerSyncSingleReceptor* const me, struct
Mutex* p_Mutex) {
```


switch (me->stateID) { case NONUMBER状态:_/* 迁移到GotNumber状态，默认子状态 */ TokenizerSyncSingleReceptor_退出_NoNumber(me); TokenizerSyncSingleReceptor_进入_GotNumber(me); me->stateID = GOTNUMBER_状态; TokenizerSyncSingleReceptor_进入_ProcessingFractionalPart(me); me->subStateID = PROCESSINGFRACTIONALPART_SSTATE; break; case GOTNUMBER状态:_switch (me->subStateID) { case PROCESSINGWHOLEPART SSTATE:_TokenizerSyncSingleReceptor_退出_ProcessingWholePart(me); TokenizerSyncSingleReceptor_进入_ProcessingFractionalPart(me); me->subStateID = PROCESSINGFRACTIONALPART_SSTATE; break; }; }; break; case EVWHITESPACE: case EVENDOFSTRING: switch (me->stateID) { case GOTNUMBER状态:_switch (me->subStateID) { case PROCESSINGWHOLEPART SSTATE:_TokenizerSyncSingleReceptor_退出_ProcessingWholePart(me); break; case PROCESSINGFRACTIONALPART SSTATE:_TokenizerSyncSingleReceptor_退出_ProcessingFractionalPart(me); break; }; TokenizerSyncSingleReceptor_退出_GotNumber(me); printf("数字: %g\n", me->result); TokenizerSyncSingleReceptor_进入_NoNumber(me); me->stateID = NONUMBER_状态; break; }; break; }; // end switch e.eType 互斥锁_释放(me->itsMutex); } struct Mutex* TokenizerSyncSingleReceptor_getItsMutex(const TokenizerSyncSingleRecepto

r*

const me) { return (struct Mutex*)me->itsMutex; } void TokenizerSyncSingleReceptor_setItsMutex(TokenizerSyncSingleReceptor* const me, struc

t

Mutex* p_Mutex) {

```
  me->itsMutex = p_Mutex;
}

TokenizerSyncSingleReceptor *
TokenizerSyncSingleReceptor_Create(void) {
  TokenizerSyncSingleReceptor*  me  =  (TokenizerSyncSingleReceptor  *)  malloc(sizeof
(TokenizerSyncSingleReceptor));
  if(me!=NULL)
  TokenizerSyncSingleReceptor_Init(me);
  return me;
}

void
TokenizerSyncSingleReceptor_Destroy(TokenizerSyncSingleReceptor* const me) {
  if(me!=NULL)
  TokenizerSyncSingleReceptor_Cleanup(me);
  free(me);
}

static void cleanUpRelations(TokenizerSyncSingleReceptor* const me) {
  if(me->itsMutex != NULL)
  me->itsMutex = NULL;
}

void
TokenizerSyncSingleReceptor_Cleanup(TokenizerSyncSingleReceptor* const me) {
  cleanUpRelations(me);
}
```

Code Listing 5-10: TSRSyncSingleReceptor.c

5.4 Multiple Event Receptor Pattern

Multiple event receptor finite state machines (MERSMs) are generally only used for synchronous state machines because the client is often aware of the set of events that it might want to send to the server state machine. In this pattern, there is a single event receptor for each event sent from the client.

5.4.1 Abstract

The MERSM approach for state machine implementation is probably the most common implementation pattern for synchronous state machines. It is usually the simplest because each event receptor concerns itself only with the handling of a single event and the execution of the associated actions.

5.4.2 Problem

This pattern addresses the problem of providing a simple robust implementation for synchronous state machines.

me->itsMutex = p_Mutex; } TokenizerSyncSingleReceptor * TokenizerSyncSingleReceptor_Create(void) { TokenizerSyncSingleReceptor* me = (TokenizerSyncSingleReceptor *) malloc(sizeof (TokenizerSyncSingleReceptor)); if(me!=NULL)TokenizerSyncSingleReceptor_Init(me); return me; } void TokenizerSyncSingleReceptor_Destroy(TokenizerSyncSingleReceptor* const me) { if(me!=NULL) TokenizerSyncSingleReceptor_Cleanup(me); free(me); } static void cleanUpRelations(TokenizerSyncSingleReceptor* const me) { if(me->itsMutex != 空) me->itsMutex = 空; } void TokenizerSyncSingleReceptor_Cleanup(TokenizerSyncSingleReceptor* const me) { cleanUpRelations(me); }

代码清单5-10：TSRSyncSingleReceptor.c

5.4 多事件接收器模式

多事件接收器有限状态机（MERSM）通常仅用于同步状态机，因为客户端通常知道它可能想要发送给服务器状态机的事件集合。在此模式中，从客户端发送的每个事件都有一个单独的事件接收器。

5.4.1 摘要

用于状态机实现的MERSM方法可能是同步状态机最常见的实现模式。它通常是最简单的，因为每个事件接收器只关心单个事件的处理以及相关动作的执行。

5.4.2 问题

该模式解决了为同步状态机提供简单且健壮的实现这一问题。

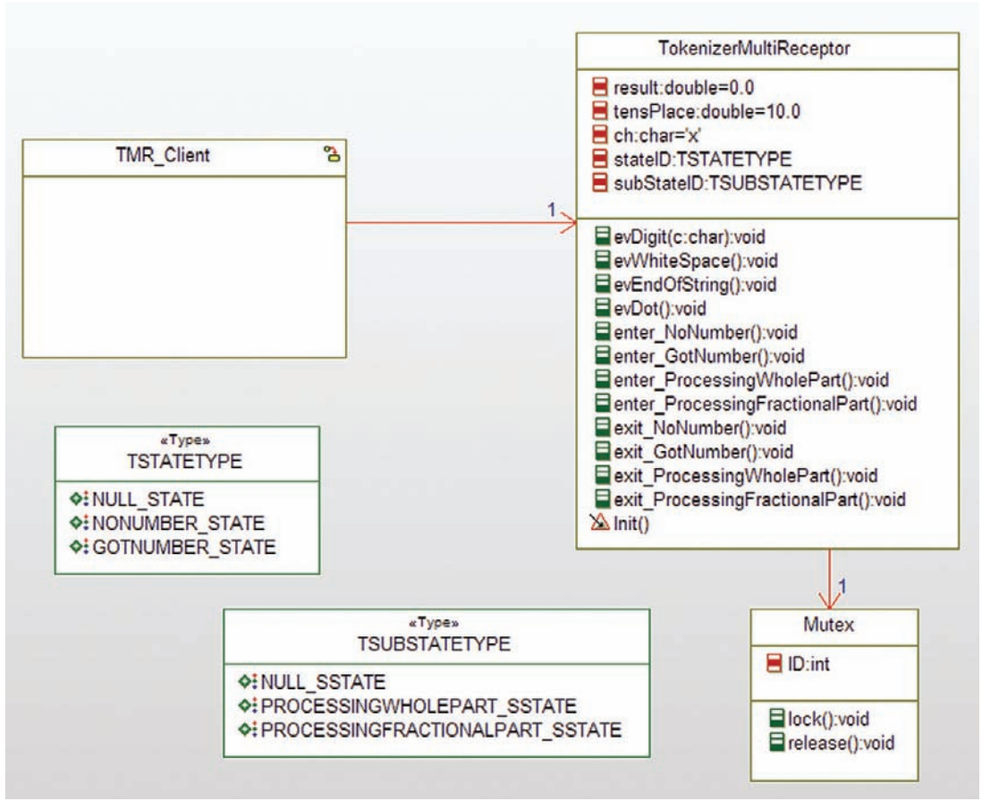


Figure 5-15: Multiple event receptor state machine context

5.4.3 Pattern Structure

The context for the state machine implementation is shown in Figure 5-15. You can see that the `TokenizerMultiReceptor` has an event receptor for each event passed to the state machine plus additional functions representing the entry and exit actions of each of the states. In addition, the figure explicitly shows the enumerated types used for the state and substates (as mentioned, it is also common to simply use integer values). The event type need not be explicitly represented because the client implicitly indicates it by invoking the appropriate event receptor.

5.4.4 Collaboration Roles

This section describes the roles for this pattern.

5.4.4.1 Mutex

The mutex ensures the deterministic run-to-completion semantics of transitions by preventing other clients from interrupting the state machine execution of an action sequence. This is

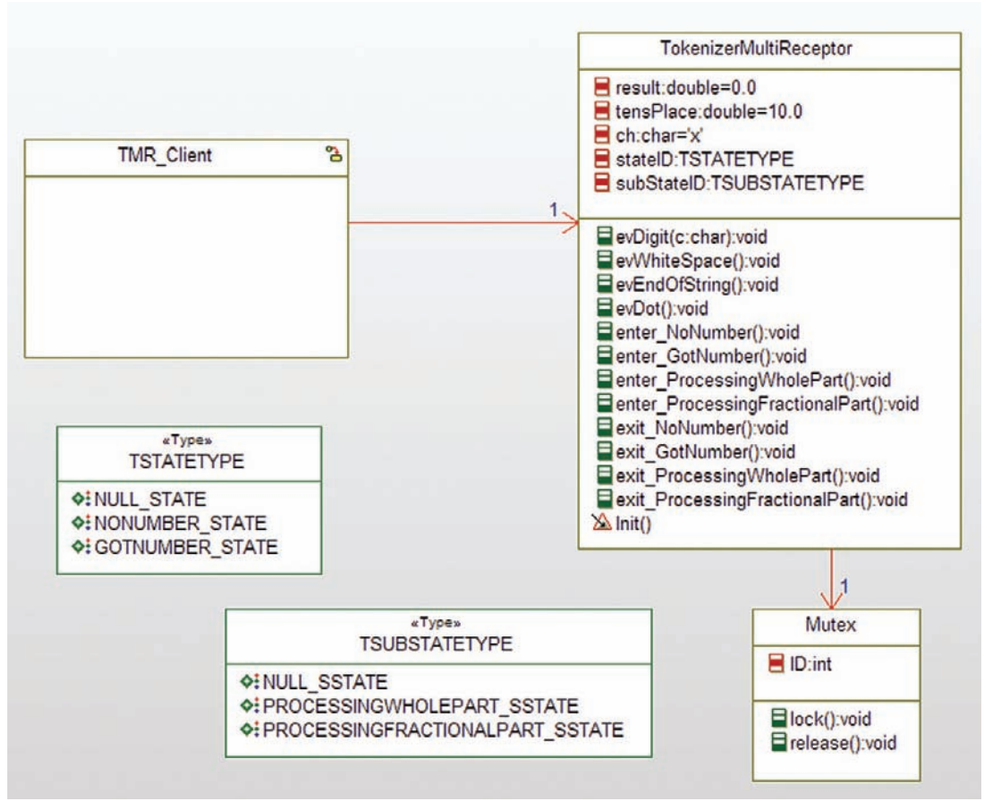


图5-15：多事件接收器状态机上下文

5.4.3 模式结构

状态机 实现的上下文如图 5-15所示。 可以看到，TokenizerMultiReceptor 为传递给状态机的每个事件都有一个事件接收器，此外还有表示每个状态进入和退出动作的附加函数。此外，该图明确展示了用于状态和子状态的枚举类型（如前所述，通常也直接使用整数值）。事件类型无需显式表示，因为客户端通过调用相应的事件接收器隐式地指明了它。

5.4.4 协作角色

本节描述该模式的角色。

5.4.4.1 互斥锁

互斥锁通过防止其他客户端中断状态机对动作序列的执行，确保迁移具有确定性的运行至完成语义。这是

accomplished by locking the semaphore before the event is processed and releasing it once the transition processing is complete.

5.4.4.2 TMRClient

This is the client of the state machine. It invokes the event receptors (in this case, those functions with the "ev" prefix, such as `evDigit` and `evDot`.

5.4.4.3 TokenizeMultiReceptor

This element is the owner and implementer of the desired state machine (see Figure 5-12). It has two kinds of attributes: those involved with the intent of the state machine (such as result) and those involved with the management of the state machine (such as `stateID`). Of the former type, we find:

- `result`, which holds the current value of the number being parsed
- `tensPlace`, which is used in computing fractional parts of numbers, as in 0.345 of the number 10.345
- `ch`, which holds the current character being processed

Of the latter type, we see:

- `stateID`, which stores the current high level state (`NULL_STATE`, `NONUMBER_STATE`, or `GOTNUMBER_STATE`)
- `subStateID`, which stores the current substate (`NULL_SSTATE`, `PROCESSINGWHOLEPART_SSTATE`, or `PROCESSINGFRACTIONALPART_SSTATE`)

These values are defined in the enumerated types `TSTATETYPE` and `TSUBSTATETYPE`, respectively.

Functions of this state machine are likewise of two types. The first type is the event receptors. These are invoked by the client to "send" the event and pass data as needed. These include:

- `evDigit(c: char)` sends the `evDigit` event and passes the character of the digit as a parameter
- `evDot` sends the dot event, indicating that the decimal point was the next character in the string
- `evEndOfString` indicates that there are no more characters in the string
- `evWhiteSpace` indicates that whitespace, such as a blank or tab character, was found

The other operations are for the entry and exit actions of the states. Note that the transitions actions are all embedded within the event receptors since they only occur once, but there are often many paths into and out of a state so we explicitly create entry and exit functions for each state. These are not called directly by the clients.

5.4.4.4 TSTATE

`TSTATE` is an enumerated type of the high level states.

通过在处理事件之前对信号量加锁，并在迁移处理完成后释放信号量来实现的。

5.4.4.2 TMRClient

这是状态机的客户端。它调用事件接收器（在本例中，即那些带有"ev"前缀的函数，例如 `evDigit` 和 `evDot`。

5.4.4.3 TokenizeMultiReceptor

此元素是所需状态机的所有者和实现者（参见图 5-12）。它具有两类属性：与状态机意图相关的属性（例如 result）以及与状态机管理相关的属性（例如 `stateID`）。对于前一种类型，我们发现：

- `result`，它保存正在解析的数字的当前值
- `tensPlace`，用于计算数字的小数部分，例如数字 10.345 中的 0.345
- `ch`，保存当前正在处理的字符

在后一种类型中，我们可以看到：

- stateID，存储当前高层状态（NULL_STATE、NONUMBER_STATE 或 GOTNUMBER_STATE）
- subStateID，它存储当前子状态（（NULL_SSTATE、PROCESSINGWHOLEPART_SSTATE、或 PROCESSINGFRACTIONALPART_SSTATE）

这些值分别在枚举类型 `TSTATETYPE` 和 `TSUBSTATETYPE` 中定义。

该状态机的函数同样分为两种类型。第一种类型是事件接收器。这些由客户端调用以"发送"事件并按需传递数据。它们包括：

- `evDigit(c: char)` 发送 `evDigit` 事件，并将数字字符作为参数传递
- `evDot` 发送点事件，表示字符串中的下一个字符是小数点
- `evEndOfString` 表示字符串中不再有字符
- `evWhiteSpace` 表示发现了空白字符，例如空格或制表符

其他操作用于状态的进入和退出动作。请注意，转移动作全部嵌入在事件接收器中，因为它们只发生一次，但进入和退出一个状态通常有许多路径，因此我们为每个状态显式创建进入和退出函数。这些函数不由客户端直接调用。

5.4.4.4 TSTATE

`TSTATE` 是高层状态的枚举类型。

5.4.4.5 TSUBSTATE

TSUBSTATE is an enumerated type of the nested states. In this case, there are only two levels.

5.4.5 Consequences

This pattern simplifies the implementation by dividing up the state logic into a set of event receptors. Each of which provides a switch-case statement to implement the appropriate behavior dependent on states. The pattern is almost exclusively applied to synchronous state machines because the event queues do not, generally, parse the internal content of the data they manage, and so do not know which event receptor to call, if there is more than one.

5.4.6 Implementation Strategies

The primary variation points for the implementation of this pattern are to use If-then statements instead of switch-case statements and to use raw integers for state variables rather than enumerated types.

5.4.7 Related Patterns

This pattern uses the Guarded Call Pattern (see Chapter 4) to ensure deterministic run-to-completion of transitions. It is also possible to use the Critical Region Pattern to simply disable task switching or even disable interrupts instead. The single event receptor provides a similar approach that can be easily adapted for asynchronous event processing.

5.4.8 Example

The implementation of the sample state machine with this pattern is shown the following code lists. Code Listing 5-11 shows the enumerated types and the definition of the `digit()` function. The next two code listings provide the header and implementation files for the `TokenizeMultiReceptor` class. Note that this class defines functions for entry and exit of each state even if empty. You may choose to omit the empty ones if desired. The `init()` function operates as a constructor and puts the object into its starting state and invokes the relevant entry action.

```
typedef enum TSTATETYPE {
  NULL_STATE,
  NONUMBER_STATE,
  GOTNUMBER_STATE
} TSTATETYPE;

typedef enum TSUBSTATETYPE {
  NULL_SSTATE,
  PROCESSINGWHOLEPART_SSTATE,
  PROCESSINGFRACTIONALPART_SSTATE
} TSUBSTATETYPE;
```


5.4.4.5 TSUBSTATE

TSUBSTATE 是嵌套状态的枚举类型。在本例中，只有两个层级。

5.4.5 结果

该模式通过将状态逻辑划分为一组事件接收器来简化实现。每个事件接收器提供一个 switch-case 语句，以根据状态实现相应的行为。该模式几乎专门应用于同步状态机，因为事件队列通常不解析其所管理数据的内部内容，因此如果有多个事件接收器，就不知道应该调用哪一个。

5.4.6 实现策略

该模式实现的主要变化点是使用 If-then 语句代替 switch-case 语句，以及使用原始整数作为状态变量而非枚举类型。

5.4.7 相关模式

该模式使用受保护调用模式（参见第4章）来确保迁移的确定性运行至完成。也可以使用临界区模式来简单地禁用任务切换，甚至禁用中断。单事件接收器提供了一种类似的方法，可以很容易地适配用于异步事件处理。

5.4.8 示例

使用该模式的示例状态机的实现如下面的代码清单所示。代码清单5-11展示了枚举类型和`digit()`函数的定义。接下来的两个代码清单提供了`TokenizeMultiReceptor`类的头文件和实现文件。注意，该类为每个状态的进入和退出定义了函数，即使为空。如果需要，您可以选择省略空函数。`init()`函数作为构造函数运行，将对象置于其起始状态并调用相关的进入动作。

```
typedef enum TSTATETYPE { NULL_
STATE, NONUMBER_STATE,
GOTNUMBER STATE_} TSTATETYPE;
typedef enum TSUBSTATETYPE { NULL_
SSTATE, PROCESSINGWHOLEPART_
SSTATE,
PROCESSINGFRACTIONALPART_
SSTATE } TSUBSTATETYPE;
```

```
int digit(char c) {
  return c-'0';
}
```

Code Listing 5-11: Basic types used by TokenizeMultiReceptor

```
#include "TokenizeMultiReceptor.h"
#include <stdio.h>

struct Mutex;

typedef struct TokenizerMultiReceptor TokenizerMultiReceptor;
struct TokenizerMultiReceptor {
  char ch;
  double result;
  TSTATETYPE stateID;
  TSUBSTATETYPE subStateID;
  double tensPlace;
  struct Mutex* itsMutex;
};

/* Constructors and destructors:*/
void TokenizerMultiReceptor_Cleanup(TokenizerMultiReceptor* const me);
/* Operations */

void TokenizerMultiReceptor_evDigit(TokenizerMultiReceptor* const me, char c);

void TokenizerMultiReceptor_evDot(TokenizerMultiReceptor* const me);

void TokenizerMultiReceptor_evEndOfString(TokenizerMultiReceptor* const me);

void TokenizerMultiReceptor_evWhiteSpace(TokenizerMultiReceptor* const me);

void
TokenizerMultiReceptor_enter_GotNumber(TokenizerMultiReceptor* const me);

void
TokenizerMultiReceptor_enter_NoNumber(TokenizerMultiReceptor* const me);

void
TokenizerMultiReceptor_enter_ProcessingFractionalPart(TokenizerMultiReceptor*  const
me);

void
TokenizerMultiReceptor_enter_ProcessingWholePart(TokenizerMultiReceptor* const me);

void
TokenizerMultiReceptor_exit_GotNumber(TokenizerMultiReceptor* const me);

void
TokenizerMultiReceptor_exit_NoNumber(TokenizerMultiReceptor* const me);

void
TokenizerMultiReceptor_exit_ProcessingFractionalPart(TokenizerMultiReceptor* const me);

void
TokenizerMultiReceptor_exit_ProcessingWholePart(TokenizerMultiReceptor* const me);
TokenizerMultiReceptor * TokenizerMultiReceptor_Create(void);
```



```
int digit(char c) {
return c-'0'; }
```

代码清单5-11：TokenizeMultiReceptor 使用的基本类型

```
#include "TokenizeMultiReceptor.h" #
include <stdio.h>

struct Mutex;

typedef struct TokenizerMultiReceptor TokenizerMultiReceptor; struct Tokenizer
MultiReceptor { char ch; double result; TSTATETYPE stateID; TSUBSTATETYPE subStateID;
double tensPlace; struct Mutex* itsMutex; }; /* 构造函数与析构函数：*/ void
TokenizerMultiReceptor_Cleanup(TokenizerMultiReceptor* const me); /* 操作 */ void
TokenizerMultiReceptor_evDigit(TokenizerMultiReceptor* const me, char c); void
TokenizerMultiReceptor_evDot(TokenizerMultiReceptor* const me); void
TokenizerMultiReceptor_evEndOfString(TokenizerMultiReceptor* const me); void
TokenizerMultiReceptor_evWhiteSpace(TokenizerMultiReceptor* const me); void
TokenizerMultiReceptor_enter_GotNumber(TokenizerMultiReceptor* const me); void
TokenizerMultiReceptor_enter_NoNumber(TokenizerMultiReceptor* const me); void
TokenizerMultiReceptor_enter_ProcessingFractionalPart(TokenizerMultiReceptor*
const
me); void TokenizerMultiReceptor_enter_ProcessingWholePart(TokenizerMultiReceptor* cons
t me);

void TokenizerMultiReceptor_exit_GotNumber(TokenizerMultiReceptor* const me); void
TokenizerMultiReceptor_exit_NoNumber(TokenizerMultiReceptor* const me); void
TokenizerMultiReceptor_exit_ProcessingFractionalPart(TokenizerMultiReceptor* con
st me);

void TokenizerMultiReceptor_exit_ProcessingWholePart(TokenizerMultiReceptor* const me
);
TokenizerMultiReceptor * TokenizerMultiReceptor_Create(void);
```

```
void TokenizerMultiReceptor_Destroy(TokenizerMultiReceptor* const me);

void TokenizerMultiReceptor_Init(TokenizerMultiReceptor* const me);

struct Mutex* TokenizerMultiReceptor_getItsMutex(const TokenizerMultiReceptor* const
me);

void  TokenizerMultiReceptor_setItsMutex(TokenizerMultiReceptor*  const  me,  struct
Mutex* p_Mutex);

#endif
```

Code Listing 5-12: TokenizeMultiReceptor.h

```
#include "TokenizerMultiReceptor.h"
#include "Mutex.h"

static void cleanUpRelations(TokenizerMultiReceptor* const me);

void TokenizerMultiReceptor_Cleanup(TokenizerMultiReceptor* const me) {
  cleanUpRelations(me);
}

/*
    process the evDigit event, passing the character
    c as a parameter
*/
void TokenizerMultiReceptor_evDigit(TokenizerMultiReceptor* const me, char c) {
  Mutex_lock(me->itsMutex);
  switch (me->stateID) {
    case NONUMBER_STATE:
        /* transition to GotNumber state, default substate */
        TokenizerMultiReceptor_exit_NoNumber(me);
        me->ch = c;
        TokenizerMultiReceptor_enter_GotNumber(me);
        me->stateID = GOTNUMBER_STATE;
        TokenizerMultiReceptor_enter_ProcessingWholePart(me);
        me->subStateID = PROCESSINGWHOLEPART_SSTATE;
        printf("Current value of result: %g\n", me->result);
        break;
    case GOTNUMBER_STATE:
        switch (me->subStateID) {
            case PROCESSINGWHOLEPART_SSTATE:

        TokenizerMultiReceptor_exit_ProcessingWholePart(me);
                me->ch = c;

        TokenizerMultiReceptor_enter_ProcessingWholePart(me);
                printf("Current value of result: %g\n", me->result);
                break;
            case PROCESSINGFRACTIONALPART_SSTATE:

        TokenizerMultiReceptor_exit_ProcessingFractionalPart(me);
                me->ch = c;
                me->result += digit(me->ch) / me->tensPlace;
                me->tensPlace *= 10.0;
```


void TokenizerMultiReceptor_Destroy(TokenizerMultiReceptor* const me); void TokenizerMultiReceptor_Init(TokenizerMultiReceptor* const me); struct 互斥锁* TokenizerMultiReceptor_getItsMutex(const TokenizerMultiReceptor* const me); void TokenizerMultiReceptor_setItsMutex(TokenizerMultiReceptor* const me, struct 互斥锁* p_Mutex); # endif

代码清单5-12：TokenizeMultiReceptor.h

#include "TokenizerMultiReceptor.h" #include "Mutex.h" static void cleanUpRelations(TokenizerMultiReceptor* const me); void TokenizerMultiReceptor_ Cleanup(TokenizerMultiReceptor* const me) { cleanUpRelations(me); } /* 处理evDigit事件，将字符c作为参数传入 */ void TokenizerMultiReceptor_evDigit(TokenizerMultiReceptor* const me, char c) { Mutex_lock(me->itsMutex); switch (me->stateID) { case NONUMBER_STATE: /* 迁移到GotNumber状态，默认子状态 */ TokenizerMultiReceptor_exit_NoNumber(me); me->ch = c; TokenizerMultiReceptor_enter_GotNumber(me); me->stateID = GOTNUMBER_STATE; TokenizerMultiReceptor_enter_ProcessingWholePart(me); me->subStateID = PROCESSINGWHOLEPART_SSTATE; printf("result的当前值: %g\n", me->result); break; case GOTNUMBER STATE:_switch (me->subStateID) { case PROCESSINGWHOLEPART_SSTATE: TokenizerMultiReceptor_exit_ProcessingWholePart(me); me->ch = c; Tokenizer MultiReceptor_enter_ProcessingWholePart(me); printf("result的当前值: %g\n", me->result); break; case PROCESSINGFRACTIONALPART SSTATE:_TokenizerMultiReceptor_exit_ ProcessingFractionalPart(me); me->ch = c; me->result += digit(me->ch) / me->tensPlace; me->tensPlace *= 10.0;

```
            TokenizerMultiReceptor_enter_ProcessingFractionalPart(me);
                printf("Current value of result: %g\n", me->result);
                break;
            };
        };
  Mutex_release(me->itsMutex);
}

/*
    process the evDot
*/
void TokenizerMultiReceptor_evDot(TokenizerMultiReceptor* const me) {
  Mutex_lock(me->itsMutex);
  me->ch = '.';
  switch (me->stateID) {
     case NONUMBER_STATE:
         /* transition to GotNumber state, default substate */
         TokenizerMultiReceptor_exit_NoNumber(me);
         TokenizerMultiReceptor_enter_GotNumber(me);
         me->stateID = GOTNUMBER_STATE;

    TokenizerMultiReceptor_enter_ProcessingFractionalPart(me);
         me->subStateID = PROCESSINGFRACTIONALPART_SSTATE;
         break;
     case GOTNUMBER_STATE:
         switch (me->subStateID) {
             case PROCESSINGWHOLEPART_SSTATE:
    TokenizerMultiReceptor_exit_ProcessingWholePart(me);

             TokenizerMultiReceptor_enter_ProcessingFractionalPart(me);
                 me->subStateID = PROCESSINGFRACTIONALPART_SSTATE;
                 break;
             };
     };
  Mutex_release(me->itsMutex);
}

/*
    process the evEndOfString event
*/
void TokenizerMultiReceptor_evEndOfString(TokenizerMultiReceptor* const me) {
  Mutex_lock(me->itsMutex);
  switch (me->stateID) {
     case GOTNUMBER_STATE:
         switch (me->subStateID) {
             case PROCESSINGWHOLEPART_SSTATE:
    TokenizerMultiReceptor_exit_ProcessingWholePart(me);
                 break;
             case PROCESSINGFRACTIONALPART_SSTATE:
    TokenizerMultiReceptor_exit_ProcessingFractionalPart(me);
                 break;
             };
```


TokenizerMultiReceptor_enter_ProcessingFractionalPart(me); printf("result的当前值: %g\n", me->result); break; }; }; Mutex_release(me->itsMutex); } /* 处理evDot */ void TokenizerMultiReceptor_evDot(TokenizerMultiReceptor* const me) { Mutex_lock(me->itsMutex); me->ch ＝ '.';switch (me->stateID) { case NONUMBER STATE:_/* 迁移到GotNumber状态，默认子状态 */ TokenizerMultiReceptor_exit_NoNumber(me); TokenizerMultiReceptor_enter_GotNumber(me); me->stateID = GOTNUMBER_STATE; TokenizerMultiReceptor_enter_ProcessingFractionalPart(me); me->subStateID = PROCESSINGFRACTIONALPART_SSTATE; break; case GOTNUMBER STATE:_switch (me->subStateID) { case PROCESSINGWHOLEPART SSTATE:_TokenizerMultiReceptor_exit_ProcessingWholePart(me); TokenizerMultiReceptor_enter_ProcessingFractionalPart(me); me->subStateID = PROCESSINGFRACTIONALPART_SSTATE; break; }; }; Mutex_release(me->itsMutex); } /* 处理evEndOfString事件 */ void TokenizerMultiReceptor_evEndOfString(TokenizerMultiReceptor* const me)

{

互斥锁_加锁(me->itsMutex); switch (me->stateID) { case GOTNUMBER_STATE: switch (me->subStateID) { case PROCESSINGWHOLEPART子状态:_TokenizerMultiReceptor_exit_ProcessingWholePart(me); break; case PROCESSINGFRACTIONALPART SSTATE:_TokenizerMultiReceptor_exit_ProcessingFractionalPart(me); break; };

```
        TokenizerMultiReceptor_exit_GotNumber(me);
        printf("Number: %g\n", me->result);

        TokenizerMultiReceptor_enter_NoNumber(me);
        me->stateID = NONUMBER_STATE;
        break;
    };
  Mutex_release(me->itsMutex);
}

/*
    process the evWhiteSpace event
*/
void TokenizerMultiReceptor_evWhiteSpace(TokenizerMultiReceptor* const me) {
  Mutex_lock(me->itsMutex);
  switch (me->stateID) {
    case GOTNUMBER_STATE:
        switch (me->subStateID) {
            case PROCESSINGWHOLEPART_SSTATE:

    TokenizerMultiReceptor_exit_ProcessingWholePart(me);
                break;
            case PROCESSINGFRACTIONALPART_SSTATE:

    TokenizerMultiReceptor_exit_ProcessingFractionalPart(me);
                break;
            };
            TokenizerMultiReceptor_exit_GotNumber(me);
             printf("Number: %g\n", me->result);
            TokenizerMultiReceptor_enter_NoNumber(me);
            me->stateID = NONUMBER_STATE;
            break;
    };
  Mutex_release(me->itsMutex);
}

/*
    entry and exit actions for each state
*/
void
TokenizerMultiReceptor_enter_GotNumber(TokenizerMultiReceptor* const me) {
  me->tensPlace = 10.0;
}

void
TokenizerMultiReceptor_enter_NoNumber(TokenizerMultiReceptor* const me) {
  me->result = 0.0;
}

void
TokenizerMultiReceptor_enter_ProcessingFractionalPart(TokenizerMultiReceptor*  const
me) {
}
```


TokenizerMultiReceptor_exit_GotNumber(me); printf("Number: %g\n", me->result); TokenizerMultiReceptor_enter_NoNumber(me); me->stateID = NONUMBER_STATE; break; }; Mutex_release(me->itsMutex); } /* 处理evWhiteSpace事件 */ void TokenizerMultiReceptor_ evWhiteSpace(TokenizerMultiReceptor* const me) { Mutex_lock(me->itsMutex); switch (me-> stateID) { case GOTNUMBER STATE:_switch (me->subStateID) { case PROCESSINGWHOLEPART SSTATE:_TokenizerMultiReceptor_exit_ ProcessingWholePart(me); break; case PROCESSINGFRACTIONALPART SSTATE:_ TokenizerMultiReceptor_exit_ProcessingFractionalPart(me); break; }; TokenizerMultiReceptor_exit_GotNumber(me); printf("Number: %g\n", me->result); TokenizerMultiReceptor_enter_NoNumber(me); me->stateID = NONUMBER_STATE; break; }; Mutex_release(me->itsMutex); } /* 每个状态的进入和退出动作 */ void TokenizerMultiReceptor_ enter_GotNumber(TokenizerMultiReceptor* const me) { me->tensPlace = 10.0; } void TokenizerMultiReceptor_enter_NoNumber(TokenizerMultiReceptor* const me) { me->result = 0.0; } void TokenizerMultiReceptor_enter_ ProcessingFractionalPart(TokenizerMultiRecepto

r* const me) { }

```
void
TokenizerMultiReceptor_enter_ProcessingWholePart(TokenizerMultiReceptor* const me) {
  me->result = me->result*10 + digit(me->ch);
}

void
TokenizerMultiReceptor_exit_GotNumber(TokenizerMultiReceptor* const me) {
  me->subStateID = NULL_SSTATE;
}

void TokenizerMultiReceptor_exit_NoNumber(TokenizerMultiReceptor* const me) {
}

void
TokenizerMultiReceptor_exit_ProcessingFractionalPart(TokenizerMultiReceptor*   const
me) {
}

void
TokenizerMultiReceptor_exit_ProcessingWholePart(TokenizerMultiReceptor* const me) {
}

/* helper functions */
TokenizerMultiReceptor * TokenizerMultiReceptor_Create(void) {
  TokenizerMultiReceptor* me = (TokenizerMultiReceptor *) malloc(sizeof
  (TokenizerMultiReceptor));
  if(me!=NULL)
    TokenizerMultiReceptor_Init(me);
  return me;
}

void TokenizerMultiReceptor_Destroy(TokenizerMultiReceptor* const me) {
  if(me!=NULL)
  TokenizerMultiReceptor_Cleanup(me);
  free(me);
}

void TokenizerMultiReceptor_Init(TokenizerMultiReceptor* const me) {
  me->ch = 'x';
  me->result = 0.0;
  me->tensPlace = 10.0;
  me->itsMutex = NULL;

  /* initialize state variables */
  me->stateID = NONUMBER_STATE;
  me->subStateID = NULL_SSTATE;
}

struct Mutex* TokenizerMultiReceptor_getItsMutex(const TokenizerMultiReceptor* const me) {
  return (struct Mutex*)me->itsMutex;
}

void  TokenizerMultiReceptor_setItsMutex(TokenizerMultiReceptor*  const  me,  struct
Mutex* p_Mutex) {
  me->itsMutex = p_Mutex;
}
```


void TokenizerMultiReceptor_enter_ProcessingWholePart(TokenizerMultiReceptor* const me) { me->result = me->result*10 + digit(me->ch); } void TokenizerMultiReceptor_exit_GotNumber(TokenizerMultiReceptor* const me) { me->subStateID = NULL_SSTATE; } void TokenizerMultiReceptor_exit_NoNumber(TokenizerMultiReceptor* const me) { } void TokenizerMultiReceptor_exit_ProcessingFractionalPart(TokenizerMultiReceptor* const me) { } void TokenizerMultiReceptor_exit_ProcessingWholePart(TokenizerMultiReceptor* const me) { } /* 辅助函数 */ TokenizerMultiReceptor * TokenizerMultiReceptor_Create(void) { TokenizerMultiReceptor* me = (TokenizerMultiReceptor *) malloc(sizeof (TokenizerMultiReceptor)); if(me!=NULL) TokenizerMultiReceptor_Init(me); return me; } void TokenizerMultiReceptor_Destroy(TokenizerMultiReceptor* const me) { if(me!=NULL)TokenizerMultiReceptor_Cleanup(me); free(me); } void TokenizerMultiReceptor_Init(TokenizerMultiReceptor* const me) { me->ch = 'x'; me->result = 0.0; me->tensPlace = 10.0; me->itsMutex = NULL; /* 初始化状态变量 */ me->stateID = NONUMBER_STATE; me->subStateID = NULL_SSTATE; } struct Mutex* TokenizerMultiReceptor_getItsMutex(const TokenizerMultiReceptor* const me) { return (struct Mutex*)me->itsMutex; } void TokenizerMultiReceptor_setItsMutex(TokenizerMultiReceptor* const me, struct Mutex* p_Mutex) { me->itsMutex = p_Mutex; }

```
static void cleanUpRelations(TokenizerMultiReceptor* const me) {
  if(me->itsMutex != NULL)
  me->itsMutex = NULL;
}
```

Code Listing 5-13:TokenizeMultiReceptor.c

5.5 State Table Pattern

The State Table Pattern is a create pattern for large, flat (i.e., without nesting or AND-states) state machines. It gives good performance (O(constant)) in use although it requires more initialization time before it becomes available. They are relatively easy to extend to more states but don't handle nesting very easily.

5.5.1 Abstract

The State Table Pattern uses a two-dimensional array to hold the state transition information. This table is usually constructed as a state x event table; the state variable is the first index and the received event ID is the second index. The contents of the table is a structure containing links to the appropriate actions and guards and the new state, should the transition be taken. This requires a somewhat more elaborate data structuring than the other state implementation patterns.

5.5.2 Problem

The problem addressed by the State Table Pattern is to implement a state machine for potentially large state spaces that are flat or can easily be made so. Also, this pattern is applicable when you want the performance of the state machine, in terms of the response time to a given event, independent of the size of the state space but you are unconcerned about the time necessary to initialize the state machine.

5.5.3 Pattern Structure

Figure 5-16 shows the structure for the State Table Pattern. It shows two important aspects of the data typing. First, the table itself is set up as a two-dimensional array of which the first index is the state (held within the state variable `stateID`) and the second index is the event ID received. The event ID, as before, is a field within the struct `Event`. To add levels of nesting requires the nesting of such tables and most developers feel that it is easier to flatten the state space (that is always possible – proof is left to the interested reader) than to nest the tables.

The other notable complexity of the data structuring is in the `TableEntry` type. This is a struct containing a set of `ActionTypes`, a `GuardType`, and the state to enter when the transition is complete. The `GuardType` is a parameterless function (except for the ubiquitous

static void cleanUpRelations(TokenizerMultiReceptor* const me) { if(me->
itsMutex != 空) me->itsMutex = 空; }

代码清单5-13：TokenizeMultiReceptor.c

5.5 状态表模式

状态表模式是一种用于大型扁平（即无嵌套或无AND状态）状态机的创建型模式。它在使用时具有良好的性能（O(常数)），尽管在可用之前需要更多的初始化时间。它们相对容易扩展到更多状态，但不太容易处理嵌套。

5.5.1 摘要

状态表模式使用二维数组来保存状态转换信息。该表通常构造为状态×事件表；状态变量是第一个索引，接收到的事件ID是第二个索引。表的内容是一个结构体，包含指向相应动作和守卫条件的链接以及若发生迁移后的新状态。这需要比其他状态实现模式更为复杂的数据结构化。

5.5.2 问题

状态表模式所解决的问题是为可能较大的平坦状态空间（或可以容易地使其平坦的状态空间）实现状态机。此外，该模式适用于以下情况：你希望状态机的性能（即对给定事件的响应时间）与状态空间的大小无关，但不关心初始化状态机所需的时间。

5.5.3 模式结构

图 5-16 展示了状态表模式的结构。它展示了数据类型设计的两个重要方面。首先，表本身设置为一个二维数组，其第一个索引是状态（保存在状态变量`stateID`中），第二个索引是接收到的事件ID。事件ID如前所述，是结构体`Event`中的一个字段。要添加嵌套层级需要嵌套这样的表，而大多数开发者认为展平状态空间（这总是可能的——证明留给感兴趣的读者）比嵌套表更容易。

数据结构化中另一个值得注意的复杂性在于 `TableEntry` 类型。这是一个结构体，包含一组 `ActionTypes`、一个 `GuardType`，以及迁移完成后要进入的状态。`GuardType` 是一个无参函数（除了那个无处不在的

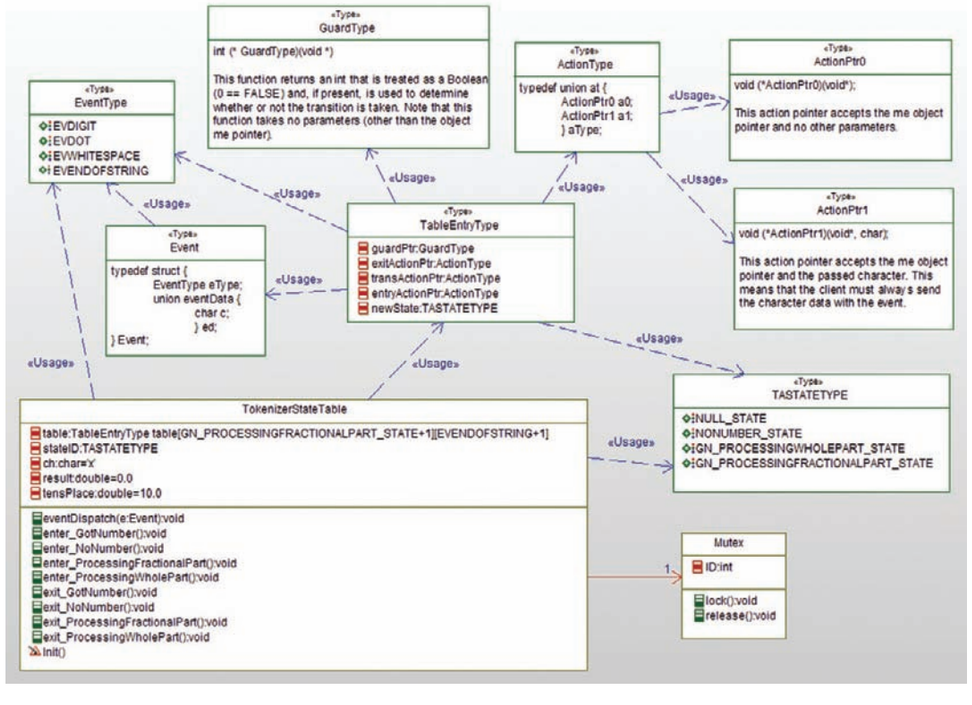


Figure 5-16: State Table Pattern

object `me` pointer) that returns an int indication `TRUE` (non-zero) or `FALSE` (zero). The `ActionType`, however, is a union with either a parameterless function pointer (`Action0`) or a pointer to a function that has a `char` parameter (`Action1`). The `TableEntry` struct has separate entries for entry, exit, and transition actions. If an action field doesn't apply, the pointer should be set to `NULL`. This means that if there are multiple actions to perform within one of those fields, they need to be composed into a single action that calls them in sequence. The initialization of all the table entries is done in the `init()` function of the `TokenizerStateTable` class.

The `eventDispatch()` function now has a straightforward task. Given an incoming event, invoke the guard (if any) in the cell `table[stateID][eType]` and if `TRUE` (or missing) then invoke the exit actions, transition actions, and entry actions and set the `stateID` equal to the value of the `newState` field found within the cell.

5.5.4 Collaboration Roles

This section describes the roles for this pattern.

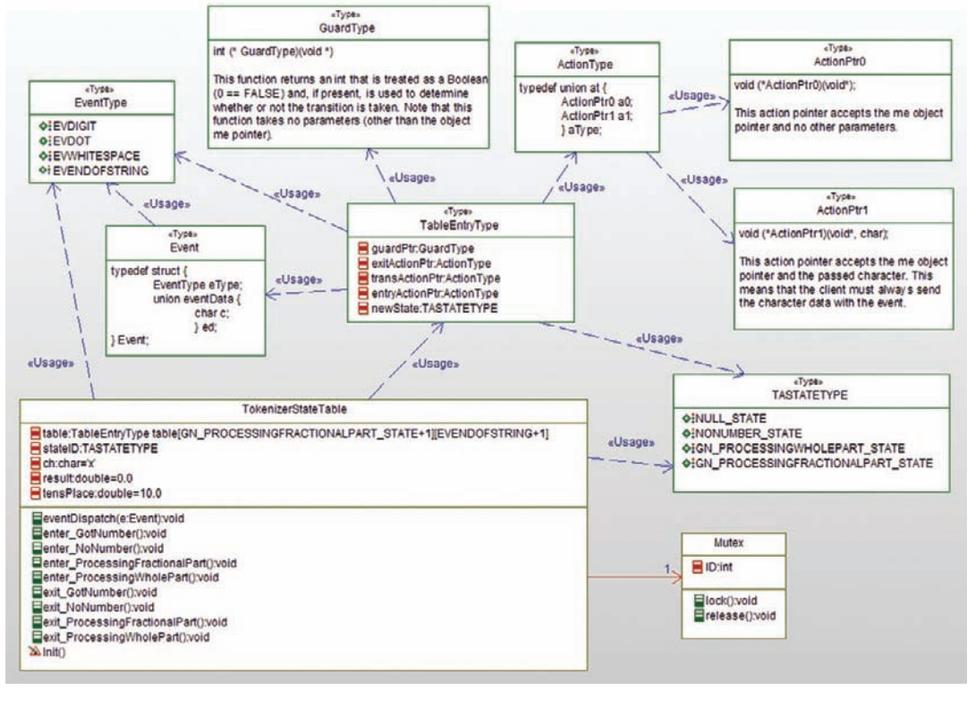


图 5-16：状态表模式

对象 `me` 指针之外），它返回一个 int 值，指示 `TRUE`（非零）或 `FALSE`（零）。然而，`ActionType` 是一个联合体，包含一个无参函数指针（`Action0`）或一个指向带有 `char` 参数的函数的指针（`Action1`）。`TableEntry` 结构体分别为进入、退出和迁移动作设有独立的条目。如果某个动作字段不适用，则应将指针设置为 `NULL`。这意味着，如果其中一个字段内需要执行多个动作，则必须将它们组合成一个按顺序调用它们的单一动作。所有表条目的初始化都在 `TokenizerStateTable` 类的 `init()` 函数中完成。

`eventDispatch()` 函数现在有一个简单的任务。给定一个传入事件，调用单元格`table[stateID][eType]` 中的守卫条件（如果有），如果为`TRUE`（或缺失），则调用退出动作、迁移动作和进入动作，并将`stateID` 设置为该单元格中`newState` 字段的值。

5.5.4 协作角色

本节描述该模式的角色。

5.5.4.1 ActionPtr0

This type is a pointer to a function that has 0 parameters, other than the obligatory `me` object pointer.

5.5.4.2 ActionPtr1

This type is a pointer to a function that takes 1 parameter of type char in addition to the obligatory `me` object pointer.

5.5.4.3 ActionType

This type is a union of the two `ActionPtr types`; as a union it only has a single pointer, but it is either of type `ActionPtr0` or `ActionPtr1`. The `ActionType` defines the `TableEntryType` fields for entry, transition, and exit actions.

5.5.4.4 Event

This type is a struct that holds the event type and the associated data passed within some of the events – in this case, a `char`. This is normally implemented as a `union` in C.

5.5.4.5 EventType

The `EventType` is an enumerated type for the events. Alternatively, `int`s can be used but I recommend `enums` as being more informative.

5.5.4.6 GuardType

This type implements a function pointer that takes no arguments but returns an `int` that will be treated as a Boolean and used to determine whether a triggered transition will be taken or not. Note that in this example, no guards are used so all these elements for this example will be `NULL`.

5.5.4.7 Mutex

As with the other implementation patterns, the `Mutex` is used to ensure the run-to-completion semantics required for deterministic state behavior.

5.5.4.8 TableEntryType

This composite type specifies the elements within the cells of the state table. It consists of a set of function pointers for exit, transition, entry, and guard actions plus the new state of the state machine should the transition be taken.

5.5.4.9 TokenizerStateTable

This class implements the state machine. Internally it contains the state table *per se* as one of its data members (along with the `stateID` and other attributes used for the computation required of the state machine). It also includes the functions necessary to implement the actions called.

5.5.4.10 TSTATETYPE

`TSTATE` is an enumerated type of the high level states. Note that the nested state machine is flattened to remove nesting. This greatly simplifies the state table. The enumerated values for this type are:

5.5.4.1 ActionPtr0

该类型是一个函数指针，该函数除必需的`me` 对象指针外，有0个参数。

5.5.4.2 ActionPtr1

此类型是一个函数指针，该函数除了必需的 `me` 对象指针外，还接受一个 char 类型的参数。

5.5.4.3 ActionType

此类型是两种 `ActionPtr` 类型的联合体；作为联合体，它只有一个指针，但该指针的类型为 `ActionPtr0` 或 `ActionPtr1`。`ActionType` 定义了用于进入、迁移和退出动作的 `TableEntryType` 字段。

5.5.4.4 事件

该类型是一个结构体，用于保存事件类型以及部分事件中传递的相关数据——在本例中为一个 `char`。在 C 语言中，这通常通过 联合体 来实现。

5.5.4.5 EventType

`EventType` 是用于表示事件的枚举类型。也可以使用 `int`，但我推荐使用 枚举，因为其信息更丰富。

5.5.4.6 GuardType

该类型实现了一个函数指针，该函数指针不接受参数，但返回一个 `int`，该返回值将被视为布尔值，用于确定是否执行触发的转移。请注意，在本示例中未使用守卫条件，因此本示例中所有这些元素都将为 空。

5.5.4.7 互斥锁

与其他实现模式一样，互斥锁 用于确保确定性状态行为所需的运行至完成语义。

5.5.4.8 TableEntryType

该复合类型指定了状态表单元格内的元素。它由一组用于退出、迁移、进入和守卫动作的函数指针组成，以及如果执行迁移时状态机的新状态。

5.5.4.9 TokenizerStateTable

该类实现了状态机。在内部，它将状态表本身作为其数据成员之一（连同`stateID` 以及状态机计算所需的其他属性）。它还包括实现所调用动作所需的函数。

5.5.4.10 TSTATETYPE

`TSTATE` 是高层状态的枚举类型。注意，嵌套状态机被扁平化以移除嵌套。这极大地简化了状态表。该类型的枚举值为：

- NULL_STATE
- NONUMBER_STATE
- GN_PROCESSINGWHOLEPART_STATE
- GN_PROCESSINGFRACTIONALPART_STATE

Where the GN_ prefix is a reminder that the last two states were nested states of the GotNumber state.

5.5.5 Consequences

This pattern has excellent execution performance that is independent of the size of the state space (i.e., the number of cells in the state table). This is because it is directly indexed with a combination of the current stateID and the incoming eType. On the other hand, the initialization of the table, while not particularly complex, is involved, since the fields for every StateTableEntry must be initialized separately in the init() function. This means that the initialization of this pattern is much slower than for the other patterns, with the possible exception of the State Pattern, discussed in the next section. Thus, this implementation pattern is a good choice for circumstances in which the start-up time of the system is relatively unimportant but the state space is potentially large.

The State Table Pattern is also easier to extend than the other patterns since extension is mostly a matter of adding new elements to the state table and defining the StateTableEntries for the relevant transitions.

It should be noted that this pattern will waste memory if the state space is sparse, that is, when many of the cells are vacant, because the table allocates StateTableEntries for all cells. This can be addressed by storing pointers to StateTableEntries rather than the elements themselves if memory is tight.

This pattern, as presented, supports only synchronous state machines. However, it can be mixed with the elements of the asynchronous version of the State Single Receptor Pattern (with suitable enhancement of the TokenizerStateTable class with a task processing loop to pull events from the event queue), it can likewise handle asynchronous events.

5.5.6 Implementation Strategies

Normally, the table is implemented as a two-dimensional array of StateTableEntries. This can waste memory if the table is sparsely populated – memory can be saved by implementing the table as a two-dimensional array of pointers to StateTableEntries instead (at the cost of an additional pointer dereference during execution).

5.5.7 Related Patterns

This pattern employs the Mutex of the Guarded Call Pattern (see Chapter 4). It can also use a critical region within the eventDispatch() function but that would not permit higher

- 空状态_
- NONUMBER状态_
- GN PROCESSINGWHOLEPART 状态_
- GN PROCESSINGFRACTIONALPART STATE

其中GN_ 前缀用于提示最后两个状态是GotNumber 状态的嵌套状态。

5.5.5 结果

该模式具有出色的执行性能，且与状态空间的大小（即状态表中的单元格数量）无关。这是因为它直接使用当前 stateID 和传入的 eType 的组合进行索引。另一方面，表的初始化虽然不算特别复杂，但涉及的内容较多，因为每个 StateTableEntry 的字段都必须在 init() 函数中单独初始化。这意味着该模式的初始化速度远慢于其他模式，可能只有下一节讨论的状态模式例外。因此，该实现模式非常适合系统启动时间相对不重要但状态空间可能很大的场景。

状态表模式也比其他模式更容易扩展，因为扩展主要是在状态表中添加新元素，并为相关转换定义 StateTableEntries 。

需要注意的是，如果状态空间是稀疏的，即许多单元格为空，该模式会浪费内存，因为表会为所有单元格分配 StateTableEntries 。如果内存紧张，可以通过存储指向 StateTableEntries 的指针而非元素本身来解决此问题。

如所呈现的，该模式仅支持同步状态机。然而，它可以与状态单接收器模式的异步版本中的元素混合使用（通过对 TokenizerStateTable 类进行适当增强，加入任务处理循环以从事件队列中拉取事件），它同样可以处理异步事件。

5.5.6 实现策略

通常，该表实现为 StateTableEntries 的二维数组。如果表中填充稀疏，这会浪费内存——可以通过将表实现为指向 StateTableEntries 的指针的二维数组来节省内存（代价是在执行期间增加一次指针解引用）。

5.5.7 相关模式

此模式采用了受保护调用模式的互斥锁 （参见第4章）。它也可以在eventDispatch() 函数内使用临界区，但那样将不允许更高

priority tasks to run during the state processing even when they could not possibly interfere with the execution of the state machine. As noted above, the Queuing Pattern could be mixed in as well, to support asynchronous event handling.

5.5.8 Example

This pattern has some interesting programming aspects with respect to the data typing, event dispatching, and initialization. Code Listing 5-14 shows the definitions of the types used by the `TokenizerStateTable` class. Code Listing 5-15 and Code Listing 5-16 show the header and implementation files for the `TokenizerStateTable` class respectively.

```
#ifndef StateTablePattern_H
#define StateTablePattern_H

typedef enum TSTATETYPE {
  NULL_STATE,
  NONUMBER_STATE,
  GN_PROCESSINGWHOLEPART_STATE,
  GN_PROCESSINGFRACTIONALPART_STATE
} TSTATETYPE;

/* This action pointer accepts the me object pointer and no other parameters. */
typedef void (*ActionPtr0)(void*);
/* This action pointer accepts the me object pointer and the passed character. This means
that the client must always send the character data with the event. */
typedef void (*ActionPtr1)(void*, char);

typedef enum EventType {
  EVDIGIT,
  EVDOT,
  EVWHITESPACE,
  EVENDOFSTRING
} EventType;
typedef struct {
    EventType eType;
    union eventData {
        char c;
        } ed;
} Event;

typedef struct ap {
    int nParams; /* 0 or 1 in this case */
    union {
        ActionPtr0 a0;
        ActionPtr1 a1;
    } aPtr;
} ActionType;

typedef ActionType* ActionPtr;
```


优先级任务在状态处理期间运行，即使它们不可能干扰状态机的执行。如上所述，排队模式也可以混合使用，以支持异步事件处理。

5.5.8 示例

该模式在数据类型、事件分发和初始化方面有一些有趣的编程要点。代码清单5-14展示了`TokenizerStateTable`类所使用的类型定义。代码清单5-15和代码清单5-16分别展示了`TokenizerStateTable`类的头文件和实现文件。

```
#ifndef StateTablePattern_H #define
StateTablePattern_H typedef 枚举
TSTATETYPE { 空_STATE, NoNumber_
STATE, GN_PROCESSINGWHOLEPART_
STATE, GN PROCESSINGFRACTIONALPART
STATE_                                  _}
TSTATETYPE;


/* 该动作指针接受me对象指针，不接受其他参数。 */ typedef void (*ActionPtr0)(void*);
/* 该动作指针接受me对象指针和传入的字符。这意味着客户端必须始终随事件发送字符数据。 */ typedef void
(*ActionPtr1)(void*, char);

typedef enum EventType { EVDIGIT, EVDOT,
EVWHITESPACE, EVENDOFSTRING }
EventType; typedef struct { EventType
eType; union eventData { char c; } ed; }
Event; typedef struct ap { int nParams; /* 此处
为 0 或 1 */ union { ActionPtr0 a0; ActionPtr1
a1; } aPtr; } ActionType; typedef ActionType*
ActionPtr;
```

```
/* This function returns an int that is treated as a Boolean (0 == FALSE) and, if present, is
used to determine whether or not the transition is taken. Note that this function takes no
parameters (other than the object me pointer). */
typedef int (*GuardType)(void *);

typedef struct TableEntryType {
  ActionPtr entryActionPtr;
  ActionPtr exitActionPtr;
  GuardType guardPtr;
  TSTATETYPE newState;
  ActionPtr transActionPtr;
} TableEntryType;

/* digit returns an int value of the char
  That is: return c-'0' */
int digit(char c); */

#endif
```

Code Listing 5-14: StateTablePattern.h

```
#ifndef TokenizerStateTable_H
#define TokenizerStateTable_H

#include "StateTablePattern.h"
struct Mutex;

typedef struct TokenizerStateTable TokenizerStateTable;
struct TokenizerStateTable {
  char ch;
  double result;
  TSTATETYPE stateID;
  TableEntryType
table[GN_PROCESSINGFRACTIONALPART_STATE+1][EVENDOFSTRING+1];
  double tensPlace;
  struct Mutex* itsMutex;
};

/* Constructors and destructors:*/
void TokenizerStateTable_Init(TokenizerStateTable* const me);

void TokenizerStateTable_Cleanup(TokenizerStateTable* const me);

/* Operations */
void TokenizerStateTable_eventDispatch(TokenizerStateTable* const me, Event e);

void TokenizerStateTable_enter_GotNumber(TokenizerStateTable* const me);

void TokenizerStateTable_enter_NoNumber(TokenizerStateTable* const me);

void
TokenizerStateTable_enter_ProcessingFractionalPart(TokenizerStateTable* const me);

void
TokenizerStateTable_enter_ProcessingWholePart(TokenizerStateTable* const me);

void TokenizerStateTable_exit_GotNumber(TokenizerStateTable* const me);
```


/* 此函数返回一个 int，该值被视为布尔值（0 == FALSE），并且如果存在，则用于确定是否执行该迁移。注意，此函数不接受任何参数（除了对象 me指针）。 */ typedef int (*GuardType)(void *);

typedef struct TableEntryType { 进入
动作指针 entryActionPtr; 退出动作指
针 exitActionPtr; 守卫指针 g
uardPtr; TSTATETYPE newState;
transActionPtr transActionPtr; }
TableEntryType;

/* digit 返回 char 的 int 值，即：return c-'0' */
int digit(char c); */ #endif

代码清单 5-14：StateTablePattern.h

#ifndef TokenizerStateTable_H #
define TokenizerStateTable_H

#include "StateTablePattern.h" struct
Mutex;

typedef 结构体 TokenizerStateTable TokenizerStateTable;
结构体 TokenizerStateTable {
  char ch;
  double result;
  TSTATETYPE stateID;
  TableEntryType
table[GN_PROCESSINGFRACTIONALPART_STATE+1][EVENDOFSTRING+1];
  double tensPlace;
  struct Mutex* itsMutex;
};

/* 构造函数与析构函数： */
void TokenizerStateTable_Init(TokenizerStateTable* const me);

void TokenizerStateTable_Cleanup(TokenizerStateTable* const me);

/* 操作 */
void TokenizerStateTable_eventDispatch(TokenizerStateTable* const me, 事件 e);

void TokenizerStateTable_enter_GotNumber(TokenizerStateTable* const me);

void TokenizerStateTable_enter_NoNumber(TokenizerStateTable* const me);

void
TokenizerStateTable_enter_ProcessingFractionalPart(TokenizerStateTable* const me);

void
TokenizerStateTable_enter_ProcessingWholePart(TokenizerStateTable* const me);

void TokenizerStateTable_exit_GotNumber(TokenizerStateTable* const me) ;

```
void TokenizerStateTable_exit_NoNumber(TokenizerStateTable* const me);
void
TokenizerStateTable_exit_ProcessingFractionalPart(TokenizerStateTable* const me);
void
TokenizerStateTable_exit_ProcessingWholePart(TokenizerStateTable* const me);
struct Mutex* TokenizerStateTable_getItsMutex(const TokenizerStateTable* const me);
void TokenizerStateTable_setItsMutex(TokenizerStateTable* const me, struct Mutex*
p_Mutex);
TokenizerStateTable * TokenizerStateTable_Create(void);
void TokenizerStateTable_Destroy(TokenizerStateTable* const me);
void TokenizerStateTable_assignCh(TokenizerStateTable* const me, char c);
void TokenizerStateTable_NoNum2GN(TokenizerStateTable* const me, char c);
void TokenizerStateTable_Frac2Frac(TokenizerStateTable* const me, char c);
void TokenizerStateTable_printResult(TokenizerStateTable* const me);
#endif
```

Code Listing 5-15: TokenizerStateTable.h

```
#include "TokenizerStateTable.h"
#include "Mutex.h"
static void cleanUpRelations(TokenizerStateTable* const me);
void TokenizerStateTable_Init(TokenizerStateTable* const me) {
  me->ch = 'x';
  me->result = 0.0;
  me->tensPlace = 10.0;
  me->itsMutex = NULL;
  TSTATETYPE st;
  EventType ev;
  TableEntryType te;

  /* initialize each cell of the table to null condition */
  for (st=NULL_STATE; st<=GN_PROCESSINGFRACTIONALPART_STATE; st++) {
     for (ev=EVDIGIT; ev<=EVENDOFSTRING; ev++) {
        me->table[st][ev].newState = NULL_STATE;
        me->table[st][ev].guardPtr = (GuardType)NULL;
        me->table[st][ev].exitActionPtr = NULL;
        me->table[st][ev].transActionPtr = NULL;
        me->table[st][ev].entryActionPtr = NULL;
        }
  }

  /*                                  */
  /* now add the specific transitions */
  /*                                  */
  /* evDigit from NoNumber to ProcessingWholePart */
```


void TokenizerStateTable_exit_NoNumber(TokenizerStateTable* const me); void TokenizerStateTable_ exit_ProcessingFractionalPart(TokenizerStateTable* const me); void TokenizerStateTable_exit_ ProcessingWholePart(TokenizerStateTable* const me); struct Mutex* TokenizerStateTable_ getItsMutex(const TokenizerStateTable* const me); void TokenizerStateTable_ setItsMutex(TokenizerStateTable* const me, struct Mutex* p_Mutex); TokenizerStateTable * TokenizerStateTable_Create(void); void TokenizerStateTable_Destroy(TokenizerStateTable* const me); void TokenizerStateTable_assignCh(TokenizerStateTable* const me, char c); void TokenizerStateTable_ NoNum2GN(TokenizerStateTable* const me, char c); void TokenizerStateTable_ Frac2Frac(TokenizerStateTable* const me, char c); void TokenizerStateTable_ printResult(TokenizerStateTable* const me); #endif

代码清单5-15：TokenizerStateTable.h

#include "TokenizerStateTable.h" #include "Mutex.h" static void cleanUpRelations(TokenizerStateTable* const me); void TokenizerStateTable_Init(TokenizerStateTable* const me) { me->ch = 'x'; me->result = 0.0; me->tensPlace = 10.0; me->itsMutex = 空; TSTATETYPE st; EventType ev; TableEntryType te;

/* 将表中的每个单元格初始化为空条件 */ for (st=NULL_STATE; st<=GN_ PROCESSINGFRACTIONALPART_STATE; st++) { for (ev=EVDIGIT; ev<= EVENDOFSTRING; ev++) { me->table[st][ev].newState = NULL_STATE; me-> table[st][ev].guardPtr = (GuardType)NULL; me->table[st][ev].exitActionPtr = NULL; me->table[st][ev].transActionPtr = NULL; me->table[st][ev]. entryActionPtr = NULL; } }

/* */ /* 现在添加具体的转换 */ /* */ /* evDigit 从 NoNumber 到 ProcessingWholePart */

```
  te.guardPtr = (GuardType)NULL;
  te.exitActionPtr = (ActionPtr)malloc(sizeof(ActionPtr));
  te.exitActionPtr->nParams = 0;
  te.exitActionPtr->aPtr.a0 = (ActionPtr0)TokenizerStateTable_exit_NoNumber;
  te.transActionPtr = (ActionPtr)malloc(sizeof(ActionPtr));
  te.transActionPtr->nParams = 1;
  te.transActionPtr->aPtr.a1 = (ActionPtr1)TokenizerStateTable_NoNum2GN;
  te.entryActionPtr = (ActionPtr)malloc(sizeof(ActionPtr));
  te.entryActionPtr->nParams = 0;
  te.entryActionPtr->aPtr.a0 = (ActionPtr0)
TokenizerStateTable_enter_ProcessingWholePart;
  te.newState = GN_PROCESSINGWHOLEPART_STATE;
  me->table[NONUMBER_STATE][EVDIGIT] = te;

  /* evDigit from Proessing Whole Part to itself */
  te.guardPtr = (GuardType)NULL;
  te.exitActionPtr = (ActionPtr)malloc(sizeof(ActionPtr));
  te.exitActionPtr->nParams = 0;
  te.exitActionPtr->aPtr.a0 = (ActionPtr0)TokenizerStateTable_exit_NoNumber;
  te.transActionPtr = (ActionPtr)malloc(sizeof(ActionPtr));
  te.transActionPtr->nParams = 1;
  te.transActionPtr->aPtr.a1 = (ActionPtr1)TokenizerStateTable_assignCh;
  te.entryActionPtr = (ActionPtr)malloc(sizeof(ActionPtr));
  te.entryActionPtr->nParams = 0;
  te.entryActionPtr->aPtr.a0 = (ActionPtr0)
TokenizerStateTable_enter_ProcessingWholePart;
  te.newState = GN_PROCESSINGWHOLEPART_STATE;
  me->table[GN_PROCESSINGWHOLEPART_STATE][EVDIGIT] = te;

  /* evDot from NoNumber to ProcessingFractionalPart */
  te.guardPtr = (GuardType)NULL;
  te.exitActionPtr = (ActionPtr)malloc(sizeof(ActionPtr));
  te.exitActionPtr->nParams = 0;
  te.exitActionPtr->aPtr.a0 = (ActionPtr0)TokenizerStateTable_exit_NoNumber;
  te.transActionPtr = (ActionPtr)malloc(sizeof(ActionPtr));
  te.transActionPtr->nParams = 1;
  te.transActionPtr->aPtr.a1 = (ActionPtr1)TokenizerStateTable_NoNum2GN;
  te.entryActionPtr = (ActionPtr)malloc(sizeof(ActionPtr));
  te.entryActionPtr->nParams = 0;
  te.entryActionPtr->aPtr.a0 = (ActionPtr0)
TokenizerStateTable_enter_ProcessingFractionalPart;
  te.newState = GN_PROCESSINGFRACTIONALPART_STATE;
  me->table[NONUMBER_STATE][EVDOT] = te;

  /* evDot from ProcessingWholePart to ProcessingFractionalPart */
  te.guardPtr = (GuardType)NULL;
  te.exitActionPtr = (ActionPtr)malloc(sizeof(ActionPtr));
  te.exitActionPtr->nParams = 0;
  te.exitActionPtr->aPtr.a0 = (ActionPtr0)
TokenizerStateTable_exit_ProcessingWholePart;
  te.transActionPtr = (ActionPtr)NULL;
```


te.guardPtr = (GuardType)NULL; te.exitActionPtr = (ActionPtr)malloc(sizeof(ActionPtr)); te.exitActionPtr->nParams = 0; te.exitActionPtr->aPtr.a0 = (ActionPtr0)TokenizerStateTable_exit_NoNumber; te.transActionPtr = (ActionPtr)malloc(sizeof(ActionPtr)); te.transActionPtr->nParams = 1; te.transActionPtr-> aPtr.a1 = (ActionPtr1)TokenizerStateTable_NoNum2GN; te.entryActionPtr = (ActionPtr)malloc(sizeof(ActionPtr)); te.entryActionPtr->nParams = 0; te.entryActionPtr-> aPtr.a0 = (ActionPtr0)

TokenizerStateTable_enter_ProcessingWholePart; te.newState = GN_PROCESSINGWHOLEPART_STATE; me-> table[NONUMBER_STATE][EVDIGIT] = te;

/* evDigit 从 ProcessingWholePart 到自身 */ te.guardPtr = (GuardType)NULL; te.exitActionPtr = (ActionPtr)malloc(sizeof(ActionPtr)); te.exitActionPtr->nParams = 0; te.exitActionPtr->aPtr.a0 = (ActionPtr0)TokenizerStateTable_exit_NoNumber; te.transActionPtr = (ActionPtr)malloc(sizeof(ActionPtr)); te.transActionPtr->nParams = 1; te.transActionPtr->aPtr.a1 = (ActionPtr1)TokenizerStateTable_assignCh; te.entryActionPtr = (ActionPtr)malloc(sizeof(ActionPtr)); te.entryActionPtr->nParams = 0; te.entryActionPtr-> aPtr.a0 = (ActionPtr0) TokenizerStateTable_enter_ProcessingWholePart; te.newState = GN_ PROCESSINGWHOLEPART_STATE; me->table[GN_PROCESSINGWHOLEPART_STATE][EVDIGIT] = te;

/* evDot 从 NoNumber 到 ProcessingFractionalPart */ te.guardPtr = (GuardType)NULL; te.exitActionPtr = (ActionPtr)malloc(sizeof(ActionPtr)); te.exitActionPtr->nParams = 0; te.exitActionPtr->aPtr.a0 = (ActionPtr0)TokenizerStateTable_exit_NoNumber; te.transActionPtr = (ActionPtr)malloc(sizeof(ActionPtr)); te.transActionPtr->nParams = 1; te.transActionPtr->aPtr.a1 = (ActionPtr1)TokenizerStateTable_NoNum2GN; te.e ntryActionPtr = (ActionPtr)malloc(sizeof(ActionPtr)); te.entryActionPtr->nParams = 0; te.entryActionPtr->aPtr.a0 = (ActionPtr0) TokenizerStateTable_enter_ ProcessingFractionalPart; te.newState = GN_PROCESSINGFRACTIONALPART_STATE; me-> table[NONUMBER_STATE][EVDOT] = te;

/* evDot 从 ProcessingWholePart 到 ProcessingFractionalPart */ te.guardPtr = (GuardType)NULL; te.exitActionPtr = (ActionPtr)malloc(sizeof(ActionPtr)); te.exitActionPtr->nParams = 0; te.exitActionPtr->aPtr.a0 = (ActionPtr0) TokenizerStateTable_exit_ ProcessingWholePart; te.transActionPtr = (ActionPtr)NULL;

```
  te.entryActionPtr = (ActionPtr)malloc(sizeof(ActionPtr));
  te.entryActionPtr->nParams = 0;
  te.entryActionPtr->aPtr.a0 = (ActionPtr0)
TokenizerStateTable_enter_ProcessingFractionalPart;
  te.newState = GN_PROCESSINGFRACTIONALPART_STATE;
  me->table[GN_PROCESSINGWHOLEPART_STATE][EVDOT] = te;

  /* evDigit from ProcessingFractionalPart to ProcessingFractionalPart */
  te.guardPtr = (GuardType)NULL;
  te.exitActionPtr = (ActionPtr)malloc(sizeof(ActionPtr));
  te.exitActionPtr->nParams = 0;
  te.exitActionPtr->aPtr.a0 = (ActionPtr0)
TokenizerStateTable_exit_ProcessingFractionalPart;
  te.transActionPtr = (ActionPtr)malloc(sizeof(ActionPtr));
  te.transActionPtr->nParams = 1;
  te.transActionPtr->aPtr.a1 = (ActionPtr1)TokenizerStateTable_Frac2Frac;
  te.entryActionPtr = (ActionPtr)malloc(sizeof(ActionPtr));
  te.entryActionPtr->nParams = 0;
  te.entryActionPtr->aPtr.a0 = (ActionPtr0)
TokenizerStateTable_enter_ProcessingFractionalPart;
  te.newState = GN_PROCESSINGFRACTIONALPART_STATE;
  me->table[GN_PROCESSINGFRACTIONALPART_STATE][EVDIGIT] = te;

  /* evWhiteSpace from ProcessingWholePart to NoNumber */
  te.guardPtr = (GuardType)NULL;
  te.exitActionPtr = (ActionPtr)malloc(sizeof(ActionPtr));
  te.exitActionPtr->nParams = 0;
  te.exitActionPtr->aPtr.a0 = (ActionPtr0)
TokenizerStateTable_exit_ProcessingWholePart;
  te.transActionPtr = (ActionPtr)malloc(sizeof(ActionPtr));
  te.transActionPtr->nParams = 0;
  te.transActionPtr->aPtr.a0 = (ActionPtr0)TokenizerStateTable_printResult;
  te.entryActionPtr = (ActionPtr)malloc(sizeof(ActionPtr));
  te.entryActionPtr->nParams = 0;
  te.entryActionPtr->aPtr.a0 = (ActionPtr0)TokenizerStateTable_enter_NoNumber;
  te.newState = NONUMBER_STATE;
  me->table[GN_PROCESSINGWHOLEPART_STATE][EVWHITESPACE] = te;

  /* evWhiteSpace from ProcessingFractionalPart to NoNumber */
  te.guardPtr = (GuardType)NULL;
  te.exitActionPtr = (ActionPtr)malloc(sizeof(ActionPtr));
  te.exitActionPtr->nParams = 0;
  te.exitActionPtr->aPtr.a0 = (ActionPtr0)
TokenizerStateTable_exit_ProcessingFractionalPart;
  te.transActionPtr = (ActionPtr)malloc(sizeof(ActionPtr));
  te.transActionPtr->nParams = 0;
  te.transActionPtr->aPtr.a0 = (ActionPtr0)TokenizerStateTable_printResult;
  te.entryActionPtr = (ActionPtr)malloc(sizeof(ActionPtr));
  te.entryActionPtr->nParams = 0;
  te.entryActionPtr->aPtr.a0 = (ActionPtr0)TokenizerStateTable_enter_NoNumber;
```



```
te.entryActionPtr = (ActionPtr)malloc(sizeof(ActionPtr));
te.entryActionPtr->nParams = 0; te.entryActionPtr->aPtr.a0 =
(ActionPtr0) TokenizerStateTable_enter_ProcessingFractionalPart;
te.newState = GN_PROCESSINGFRACTIONALPART_STATE; me->table[
GN_PROCESSINGWHOLEPART_STATE][EVDOT] = te;

/* evDigit 从 ProcessingFractionalPart 到 ProcessingFractionalPart */ te.guardPtr =
(GuardType)NULL; te.exitActionPtr = (ActionPtr)malloc(sizeof(ActionPtr));
te.exitActionPtr->nParams = 0; te.exitActionPtr->aPtr.a0 = (ActionPtr0)
TokenizerStateTable_exit_ProcessingFractionalPart; te.transActionPtr =
(ActionPtr)malloc(sizeof(ActionPtr)); te.transActionPtr->nParams = 1; te.transActionPtr->
aPtr.a1 = (ActionPtr1)TokenizerStateTable_Frac2Frac; te.entryActionPtr =
(ActionPtr)malloc(sizeof(ActionPtr)); te.entryActionPtr->nParams = 0; te.entryActionPtr->
aPtr.a0 = (ActionPtr0) TokenizerStateTable_enter_ProcessingFractionalPart; te.ne
wState = GN_PROCESSINGFRACTIONALPART_STATE; me->table[GN_
PROCESSINGFRACTIONALPART_STATE][EVDIGIT] = te;

/* evWhiteSpace 从 ProcessingWholePart 到 NoNumber */ te.guardPtr = (
GuardType)NULL; te.exitActionPtr = (ActionPtr)malloc(sizeof(ActionPtr)); te.exitActionPtr->
nParams = 0; te.exitActionPtr->aPtr.a0 = (ActionPtr0) TokenizerStateTable_exit_
ProcessingWholePart; te.transActionPtr = (ActionPtr)malloc(sizeof(ActionPtr));
te.transActionPtr->nParams = 0; te.transActionPtr->aPtr.a0 = (ActionPtr0)TokenizerStateTable_
printResult; te.entryActionPtr = (ActionPtr)malloc(sizeof(ActionPtr)); te.entryActionPtr->
nParams = 0; te.entryActionPtr->aPtr.a0 = (ActionPtr0)TokenizerStateTable_enter_NoNumber;
te.newState = NONUMBER_STATE; me->table[GN_PROCESSINGWHOLEPART_STATE][
EVWHITESPACE] = te;

/* evWhiteSpace 从 ProcessingFractionalPart 到 NoNumber */ te.guardPtr =
(GuardType)NULL; te.exitActionPtr = (ActionPtr)malloc(sizeof(ActionPtr)); te.exitActionPtr->
nParams = 0; te.exitActionPtr->aPtr.a0 = (ActionPtr0) TokenizerStateTable_exit_
ProcessingFractionalPart; te.transActionPtr = (ActionPtr)malloc(sizeof(ActionPtr));
te.transActionPtr->nParams = 0; te.transActionPtr->aPtr.a0 = (ActionPtr0)TokenizerStateTable_
printResult; te.entryActionPtr = (ActionPtr)malloc(sizeof(ActionPtr)); te.entryActionPtr->
nParams = 0; te.entryActionPtr->aPtr.a0 = (ActionPtr0)TokenizerStateTable_enter_NoNumber;
```

```
  te.newState = NONUMBER_STATE;
  me->table[GN_PROCESSINGFRACTIONALPART_STATE][EVWHITESPACE] = te;

  /* evEndOfString from ProcessingWholePart to NoNumber */
  te.guardPtr = (GuardType)NULL;
  te.exitActionPtr = (ActionPtr)malloc(sizeof(ActionPtr));
  te.exitActionPtr->nParams = 0;
  te.exitActionPtr->aPtr.a0 = (ActionPtr0)
TokenizerStateTable_exit_ProcessingWholePart;
  te.transActionPtr = (ActionPtr)malloc(sizeof(ActionPtr));
  te.transActionPtr->nParams = 0;
  te.transActionPtr->aPtr.a0 = (ActionPtr0)TokenizerStateTable_printResult;
  te.entryActionPtr = (ActionPtr)malloc(sizeof(ActionPtr));
  te.entryActionPtr->nParams = 0;
  te.entryActionPtr->aPtr.a0 = (ActionPtr0)TokenizerStateTable_enter_NoNumber;
  te.newState = NONUMBER_STATE;
  me->table[GN_PROCESSINGWHOLEPART_STATE][EVENDOFSTRING] = te;

  /* evEndOfString from ProcessingFractionalPart to NoNumber */
  te.guardPtr = (GuardType)NULL;
  te.exitActionPtr = (ActionPtr)malloc(sizeof(ActionPtr));
  te.exitActionPtr->nParams = 0;
  te.exitActionPtr->aPtr.a0 = (ActionPtr0)
TokenizerStateTable_exit_ProcessingFractionalPart;
  te.transActionPtr = (ActionPtr)malloc(sizeof(ActionPtr));
  te.transActionPtr->nParams = 0;
  te.transActionPtr->aPtr.a0 = (ActionPtr0)TokenizerStateTable_printResult;
  te.entryActionPtr = (ActionPtr)malloc(sizeof(ActionPtr));
  te.entryActionPtr->nParams = 0;
  te.entryActionPtr->aPtr.a0 = (ActionPtr0)TokenizerStateTable_enter_NoNumber;
  te.newState = NONUMBER_STATE;
  me->table[GN_PROCESSINGFRACTIONALPART_STATE][EVENDOFSTRING] = te;

  /* Last initialization activity - enter the initial state */
  me->stateID = NONUMBER_STATE;
  TokenizerStateTable_enter_NoNumber(me);
}

void TokenizerStateTable_Cleanup(TokenizerStateTable* const me) {
  cleanUpRelations(me);
}

void TokenizerStateTable_eventDispatch(TokenizerStateTable* const me, Event e) {
  int takeTransition = 0;
  Mutex_lock(me->itsMutex);
  /* first ensure the entry is within the table boundaries */
  if (me->stateID >= NULL_STATE && me->stateID <= GN_PROCESSINGFRACTIONALPART_STATE) {
    if (e.eType >= EVDIGIT && e.eType <= EVENDOFSTRING) {
        /* is there a valid transition for the current state and event? */
      if (me->table[me->stateID][e.eType].newState != NULL_STATE) {
         /* is there a guard? */
```


te.newState = NONUMBER_STATE; me->table[GN_PROCESSINGFRACTIONALPART_STATE][EVWHITESPACE] = te; /* evEndOfString 从 ProcessingWholePart 到 NoNumber */ te.guardPtr = (GuardType)NULL; te.exitActionPtr = (ActionPtr)malloc(sizeof(ActionPtr)); te.exitActionPtr->nParams = 0; te.exitActionPtr->aPtr.a0 = (ActionPtr0) TokenizerStateTable_exit_ProcessingWholePart; te.transActionPtr = (ActionPtr)malloc(sizeof(ActionPtr)); te.transActionPtr->nParams = 0; te.transActionPtr->aPtr.a0 = (ActionPtr0)TokenizerStateTable_printResult; te.entryActionPtr = (ActionPtr)malloc(sizeof(ActionPtr)); te.entryActionPtr->nParams = 0; te.entryActionPtr->aPtr.a0 = (ActionPtr0)TokenizerStateTable_enter_NoNumber; te.newState = NONUMBER_STATE; me->table[GN_PROCESSINGWHOLEPART_STATE][EVENDOFSTRING] = te; /* evEndOfString 从 ProcessingFractionalPart 到 NoNumber */ te.guardPtr = (GuardType)NULL; te.exitActionPtr = (ActionPtr)malloc(sizeof(ActionPtr)); te.exitActionPtr->nParams = 0; te.exitActionPtr->aPtr.a0 = (ActionPtr0) TokenizerStateTable_exit_ProcessingFractionalPart; te.transActionPtr = (ActionPtr)malloc(sizeof(ActionPtr)); te.transActionPtr->nParams = 0; te.transActionPtr->aPtr.a0 = (ActionPtr0)TokenizerStateTable_printResult; te.entryActionPtr = (ActionPtr)malloc(sizeof(ActionPtr)); te.entryActionPtr->nParams = 0; te.entryActionPtr->aPtr.a0 = (ActionPtr0)TokenizerStateTable_enter_NoNumber; te.newState = NONUMBER_STATE; me->table[GN_PROCESSINGFRACTIONALPART_STATE][EVENDOFSTRING] = te; /* 最后的初始化活动 - 进入初始状态 */ me->stateID = NONUMBER_STATE; TokenizerStateTable_enter_NoNumber(me); } void TokenizerStateTable_Cleanup(TokenizerStateTable* const me) { cleanUpRelations(me); } void TokenizerStateTable_eventDispatch(TokenizerStateTable* const me, Event e) { int takeTransition = 0; Mutex_lock(me->itsMutex); /* 首先确保条目在表边界内 */ if (me->stateID >= NULL_STATE && me->stateID <= GN_PROCESSINGFRACTIONALPART_STATE) {

if (e.eType >= EVDIGIT && e.eType <= EVENDOFSTRING) { /* 当前状态和事件是否存在有效的迁移？ */ if (me->table[me->stateID][e.eType].newState != NULL_STATE) { /* 是否存在守卫条件？ */

```
            if (me->table[me->stateID][e.eType].guardPtr == NULL)
                /* is the guard TRUE? */
                takeTransition = TRUE; /* if no guard, then it "evaluates" to TRUE */
            else
                takeTransition =(me->table[me->stateID][e.eType].guardPtr(me));
            if (takeTransition) {
                if (me->table[me->stateID][e.eType].exitActionPtr != NULL)
                    if (me->table[me->stateID][e.eType].exitActionPtr->nParams == 0)
                        me->table[me->stateID][e.eType].exitActionPtr->aPtr.a0(me);
                    else
                      me->table[me->stateID][e.eType].exitActionPtr->aPtr.a1(me, e.ed.c);
                if (me->table[me->stateID][e.eType].transActionPtr != NULL)
                    if (me->table[me->stateID][e.eType].transActionPtr->nParams == 0)
                        me->table[me->stateID][e.eType].transActionPtr->aPtr.a0(me);
                    else
                     me->table[me->stateID][e.eType].transActionPtr->aPtr.a1(me, e.ed.c);
                if (me->table[me->stateID][e.eType].entryActionPtr != NULL)
                    if (me->table[me->stateID][e.eType].entryActionPtr->nParams == 0)
                        me->table[me->stateID][e.eType].entryActionPtr->aPtr.a0(me);
                    else
                     me->table[me->stateID][e.eType].entryActionPtr->aPtr.a1(me, e.ed.c);
                me->stateID = me->table[me->stateID][e.eType].newState;
            }
        }
    }
}
  Mutex_release(me->itsMutex);
}

void TokenizerStateTable_enter_GotNumber(TokenizerStateTable* const me) {
  me->tensPlace = 10.0;
}

void TokenizerStateTable_enter_NoNumber(TokenizerStateTable* const me) {
  me->result = 0.0;
}

void
TokenizerStateTable_enter_ProcessingFractionalPart(TokenizerStateTable*
const me) {
}

void
TokenizerStateTable_enter_ProcessingWholePart(TokenizerStateTable* const me) {
  me->result = me->result*10 + digit(me->ch);
}

void TokenizerStateTable_exit_GotNumber(TokenizerStateTable* const me) {
}

void TokenizerStateTable_exit_NoNumber(TokenizerStateTable* const me) {
}
```



```
if (me->table[me->stateID][e.eType].guardPtr == NULL) /* 守卫条件是否为真？ */ takeTransition = TRUE; /* 如果没有守卫条件，则其“求值”为真 */ else takeTransition =(me->table[me->stateID][e.eType].guardPtr(me)); if (takeTransition) { if (me->table[me->stateID][e.eType].exitActionPtr != NULL) if (me->table[me->stateID][e.eType].exitActionPtr->nParams == 0) me->table[me->stateID][e.eType].exitActionPtr->aPtr.a0(me); else me->table[me->stateID][e.eType].exitActionPtr->aPtr.a1(me, e.ed.c); if (me->table[me->stateID][e.eType].transActionPtr != NULL) if (me->table[me->stateID][e.eType].transActionPtr->nParams == 0) me->table[me->stateID][e.eType].transActionPtr->aPtr.a0(me); else me->table[me->stateID][e.eType].transActionPtr->aPtr.a1(me, e.ed.c); if (me->table[me->stateID][e.eType].entryActionPtr != NULL) if (me->table[me->stateID][e.eType].entryActionPtr->nParams == 0) me->table[me->stateID][e.eType].entryActionPtr->aPtr.a0(me); else me->table[me->stateID][e.eType].entryActionPtr->aPtr.a1(me, e.ed.c); me->stateID = me->table[me->stateID][e.eType].newState; } } } } Mutex_release(me->itsMutex); } void TokenizerStateTable_enter_GotNumber(TokenizerStateTable* const me) { me->tensPlace = 10.0; } void TokenizerStateTable_enter_NoNumber(TokenizerStateTable* const me) { me->result = 0.0; } void TokenizerStateTable_enter_ProcessingFractionalPart(TokenizerStateTable* const me) { } void TokenizerStateTable_enter_ProcessingWholePart(TokenizerStateTable* const me) { me->result = me->result*10 + digit(me->ch); } void TokenizerStateTable_exit_GotNumber(TokenizerStateTable* const me) { } void TokenizerStateTable_exit_NoNumber(TokenizerStateTable* const me) { }
```

```
void
TokenizerStateTable_exit_ProcessingFractionalPart(TokenizerStateTable*
const me) {
}
void
TokenizerStateTable_exit_ProcessingWholePart(TokenizerStateTable* const me) {
}
struct Mutex* TokenizerStateTable_getItsMutex(const TokenizerStateTable* const me) {
  return (struct Mutex*)me->itsMutex;
}
void  TokenizerStateTable_setItsMutex(TokenizerStateTable*  const  me,  struct  Mutex*
p_Mutex) {
  me->itsMutex = p_Mutex;
}
TokenizerStateTable * TokenizerStateTable_Create(void) {
  TokenizerStateTable* me = (TokenizerStateTable *) malloc(sizeof
(TokenizerStateTable));
  if(me!=NULL)
  TokenizerStateTable_Init(me);
  return me;
}
void TokenizerStateTable_Destroy(TokenizerStateTable* const me) {
  if(me!=NULL)
  TokenizerStateTable_Cleanup(me);
  free(me);
}
static void cleanUpRelations(TokenizerStateTable* const me) {
  if(me->itsMutex != NULL)
  me->itsMutex = NULL;
}
void TokenizerStateTable_assignCh(TokenizerStateTable* const me, char c) {
  me->ch = c;
}
void TokenizerStateTable_NoNum2GN(TokenizerStateTable* const me, char c) {
  TokenizerStateTable_assignCh(me, c);
  me->tensPlace = 10.0;
}
void TokenizerStateTable_Frac2Frac(TokenizerStateTable* const me, char c) {
  me->ch = c;
  me->result += digit(me->ch)/me->tensPlace;
  me->tensPlace *= 10;
}
void TokenizerStateTable_printResult(TokenizerStateTable* const me) {
  printf("Number: %g\n", me->result);
}
```

Code Listing 5-16: TokenizerStateTable.c

void TokenizerStateTable_exit_ProcessingFractionalPart(TokenizerStateTable* const me) { } void TokenizerStateTable_exit_ProcessingWholePart(TokenizerStateTable* const me) { } struct Mutex* TokenizerStateTable_getItsMutex(const TokenizerStateTable* const me) { return (struct Mutex*)me->itsMutex; } void TokenizerStateTable_setItsMutex(TokenizerStateTable* const me, struct Mut

ex*

p_Mutex) { me->itsMutex = p_Mutex; } TokenizerStateTable * TokenizerStateTable_Create(void) { TokenizerStateTable* me = (TokenizerStateTable *) malloc(sizeof (TokenizerStateTable)); if(me!=NULL)TokenizerStateTable_Init(me); return me; } void TokenizerStateTable_Destroy(TokenizerStateTable* const me) { if(me!=NULL) TokenizerStateTable_Cleanup(me); free(me); } static void cleanUpRelations(TokenizerStateTable* const me) { if(me->itsMutex != 空) me->itsMutex = 空; } void TokenizerStateTable_assignCh(TokenizerStateTable* const me, char c) { me->ch = c; } void TokenizerStateTable_NoNum2GN(TokenizerStateTable* const me, char c) { TokenizerStateTable_assignCh(me, c); me->tensPlace = 10.0; } void TokenizerStateTable_Frac2Frac(TokenizerStateTable* const me, char c) { me->ch = c; me->result += digit(me->ch)/me->tensPlace; me->tensPlace *= 10; } void TokenizerStateTable_printResult(TokenizerStateTable* const me) { printf("数字: %g\n", me->result); }

代码清单5-16：TokenizerStateTable.c

Figure 5-17 shows an example execution of the system in which a client sends the characters of the string "1.3" to the `TokenizerStateTable`.

5.6 State Pattern

This, perhaps poorly named, pattern comes from *Design Patterns: Elements of Reusable Object Oriented Software* by Gamma, et. al.[4] It provides an implementation strategy that can be optimized either for data memory at the expense of run-time performance or vice versa. It is applicable for sparse and dense state spaces alike and has relatively low initial set-up costs. However, it makes rather heavy use of dynamic memory. It is simple to implement in C++ because of its use of polymorphism and therefore somewhat more complex in C.

5.6.1 Abstract

The State Pattern implements a state machine through the creation of state objects, one per state. The class owning the state machine, known as the `Context`, maintains a list of these objects with an internal variable that identifies which of these is the current state. All event receptors are passed to the currently active state object.

5.6.2 Problem

This pattern provides a means for implementing state machines that simplifies the stateful class by creating a set of objects, each of which implements the behavior of a single state. It makes the states explicit, rather easy to modify, and enables the extension of the state space by adding new state objects as needed.

5.6.3 Pattern Structure

Figure 5-18 shows the structure for the pattern for the parsing example. Note that the states are all implemented as different classes, and each of these classes provides a function for each of the events (even if the implementation is empty). The `Context` passes the control off to the current state (implemented as the `currentState` index into the `stateList` array). Because the actions are almost always implemented in terms of manipulating variables and invoking functions of the `Context`, each state class requires a pointer back to the `Context`. In a fashion similar to the State Table Pattern, the pattern is made much easier if you flatten the state machine (i.e., eliminate nesting).

[4] Gamma, E., Helm, R., Johnson, R., Vlissides, J., 1995. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.

图 5-17 展示了系统的一个示例执行过程，其中客户端将字符串"1.3"的字符发送给 `TokenizerStateTable`。

5.6 状态模式

这个命名或许不太恰当的模式来自 Gamma 等人的《设计模式：可复用面向对象软件的基础》。[4] 它提供了一种实现策略，可以针对数据内存进行优化而牺牲运行时性能，或者反之。它同样适用于稀疏和稠密的状态空间，并且初始设置成本相对较低。然而，它相当多地使用了动态内存。它在 C++ 中实现起来较为简单，因为它使用了多态，因此在 C 中则稍显复杂。

5.6.1 摘要

状态模式通过创建状态对象来实现状态机，每个状态对应一个对象。拥有该状态机的类称为 上下文，它维护一个这些对象的列表，并通过一个内部变量来标识其中哪个是当前状态。所有事件接收器都被传递给当前活动的状态对象。

5.6.2 问题

该模式提供了一种实现状态机的方法，通过创建一组对象来简化有状态类，其中每个对象实现单个状态的行为。它使状态显式化，易于修改，并能够通过按需添加新的状态对象来扩展状态空间。

5.6.3 模式结构

图 5-18 展示了解析示例中模式的结构。注意，所有状态都实现为不同的类，并且每个类都为每个事件提供了一个函数（即使实现为空）。上下文 将控制权传递给当前状态（通过`currentState` 索引到`stateList` 数组来实现）。由于动作几乎总是通过操作变量和调用上下文的函数来实现，因此每个状态类都需要一个指回上下文的指针。与状态表模式类似，如果展平状态机（即消除嵌套），该模式会变得容易得多。

[4] Gamma, E., Helm, R., Johnson, R., Vlissides, J., 1995. 设计模式：可复用面向对象软件基础. Addison-Wesley.

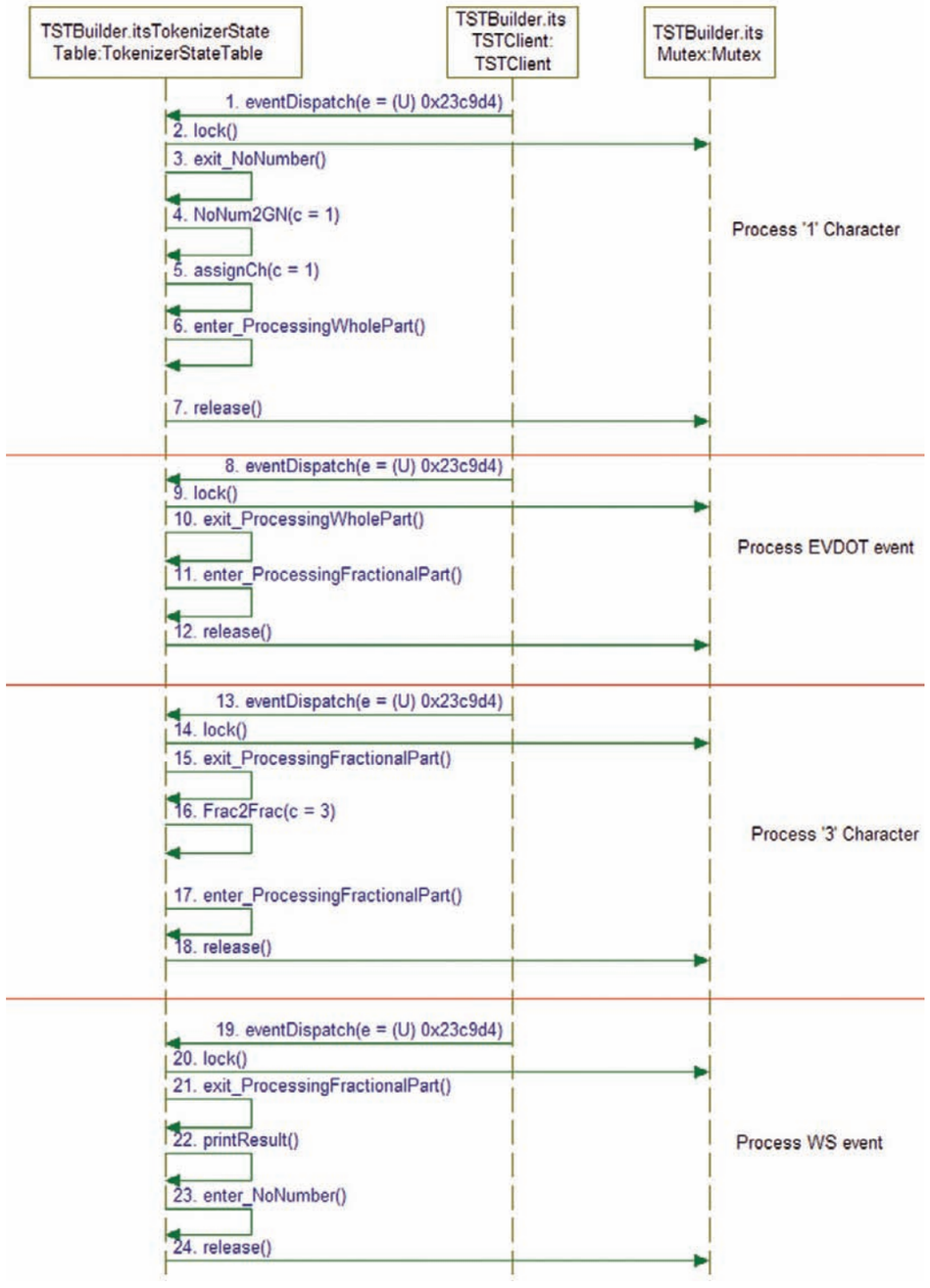


Figure 5-17: State Table Pattern sequence for execution of "1.3" string

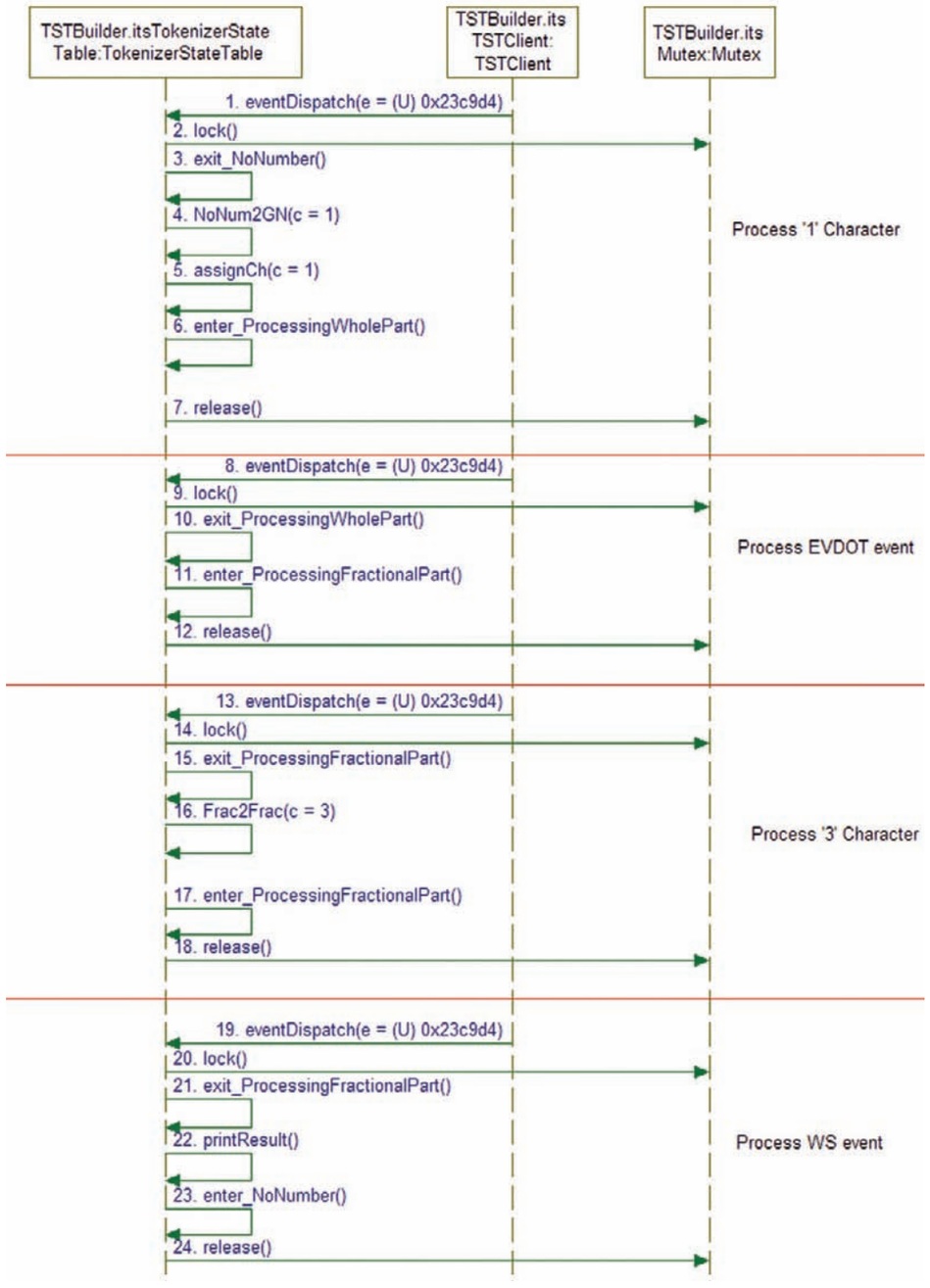


图 5-17：执行"1.3"字符串的状态表模式序列

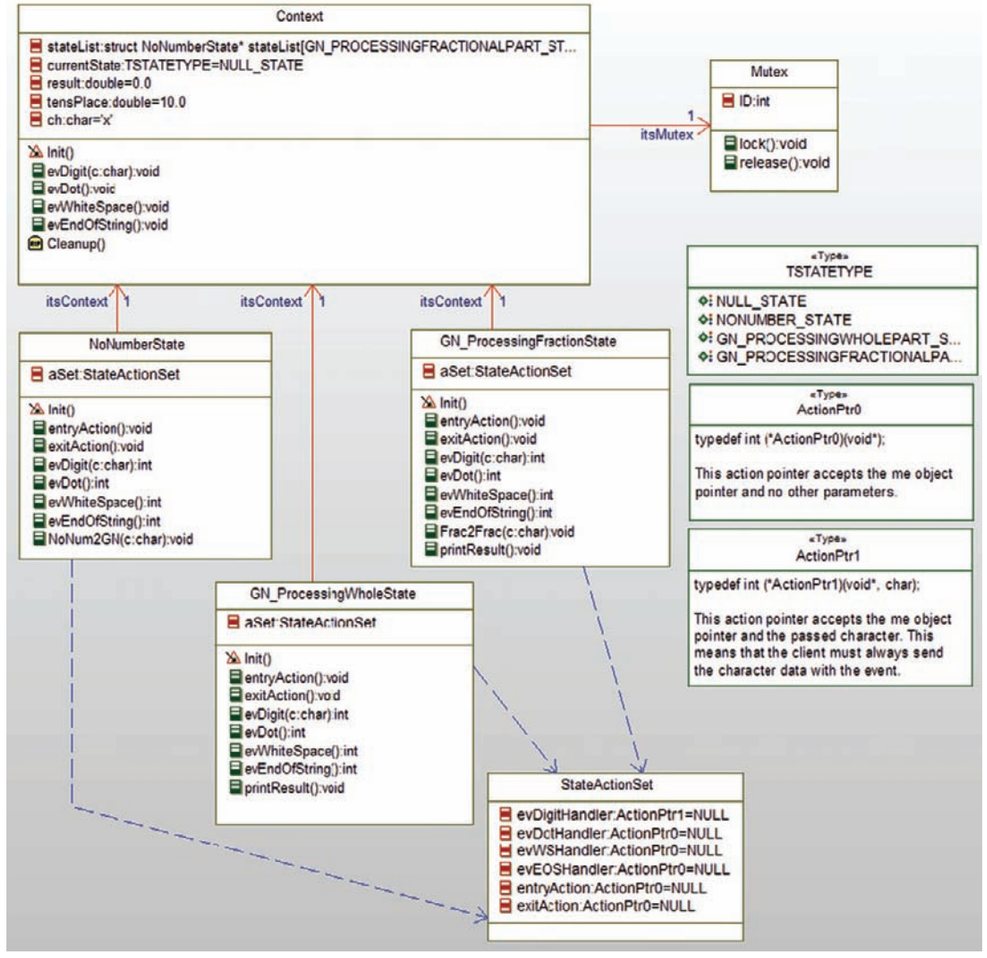


Figure 5-18: State Pattern

5.6.4 Collaboration Roles

5.6.4.1 ActionPtr0

This type is a pointer to a function that has 0 parameters, other than the obligatory `me` object pointer, and returns an `int`. The value of the return is either 0 (event discarded) or 1 (event handled).

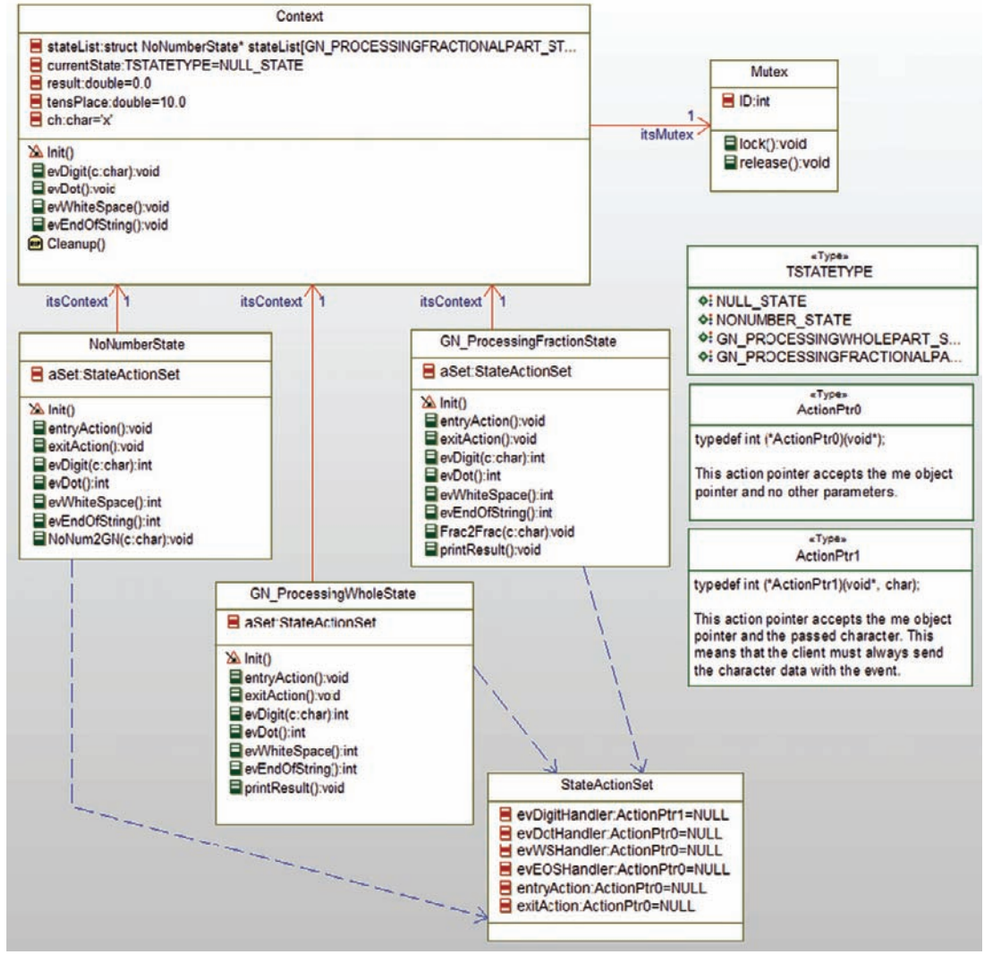


图5-18：状态模式

5.6.4 协作角色

5.6.4.1 ActionPtr0

此类型是一个函数指针，该函数除必需的 `me` 对象指针外，有 0 个参数，并返回一个 `int`。返回值为 0（事件丢弃）或 1（事件处理）。

5.6.4.2 ActionPtr1

This type is a pointer to a function that takes 1 parameter of type `char` in addition to the obligatory `me` object pointer and returns an `int`. The value of the return is either 0 (event discarded) or 1 (event handled).

5.6.4.3 Context

The `Context` is the class that owns the state machine and has the data manipulated by the actions. It implements a `stateList` as an array of pointers to the state classes and a `currentState` variable that indicates which of the current states is active. Since the structure of the state classes is all the same (in terms of the struct), the elements of the stateList array can be typed as pointers to one of them.

The `Context` provides a set of event handlers similar to the Multiple Event Receptor Pattern. The implementation of each event handler delegates the event handling to the current state object, which is free to act on, or discard the event as appropriate. If it handles the event, the state object will update the `Context`'s `currentState` variable; once the event handler returns (and if it handled the event), the `Context` will execute the entry actions of the new state.

During initialization, the `Context` creates each of the state objects, puts it in the `stateList`, and then enters the initial state.

5.6.4.4 Mutex

The `Mutex` ensures the deterministic run-to-completion semantics of transitions by preventing other clients from interrupting the state machine execution of an action sequence. This is accomplished by locking the semaphore before the event is processed and releasing it once the transition processing is complete.

5.6.4.5 State Classes

The state classes each implement one of the states of the `Context` and implements all of the event handlers. The event handlers return 1 if they handled the event and 0 if they discarded the event. If it handles the event, the event handler is expected to execute the exit actions of the current state, then the transition actions, and finally set the `Context`'s `currentState` to the new state. The entry actions of the new state will be executed by the `Context` after the return of the state object's event handler. The state classes are:

- `NoNumberState`
- `GN_ProcessingFractionState`
- `GN_ProcessingWholeState`

Each event handler may have additional helper functions invoked by its event handlers, particularly for the transition actions, in addition to the entry and exit actions.

Each state object has an instance of the `StateActionSet` class that implements function pointers to each of the event handlers. During the state object's initialization, these function pointers are set

5.6.4.2 ActionPtr1

此类型是一个函数指针，该函数除必需的 `me` 对象指针外，还接受 1 个类型为 `char` 的参数，并返回一个 `int`。返回值为 0（事件丢弃）或 1（事件处理）。

5.6.4.3 上下文

上下文 是拥有状态机并持有由动作操作的数据的类。它将`stateList` 实现为指向状态类的指针数组，以及一个`currentState` 变量，用于指示当前状态中哪个处于活动状态。由于状态类的结构（就结构体而言）完全相同，stateList 数组的元素可以类型化为指向其中之一的指针。

上下文 提供了一组类似于多事件接收器模式的事件处理器。每个事件处理器的实现将事件处理委托给当前状态对象，该对象可以酌情处理或丢弃该事件。如果它处理了该事件，状态对象将更新上下文的`s currentState` 变量；一旦事件处理器返回（并且如果它处理了该事件），上下文 将执行新状态的进入动作。

在初始化期间，上下文 创建每个状态对象，将其放入`stateList`，然后进入初始状态。

5.6.4.4 互斥锁

互斥锁 通过阻止其他客户端中断状态机对动作序列的执行，确保迁移具有确定性的运行至完成语义。这是通过在事件被处理之前对信号量加锁，并在迁移处理完成后释放它来实现的。

5.6.4.5 状态类

每个状态类分别实现上下文 的其中一个状态，并实现所有的事件处理器。如果事件处理器处理了该事件则返回 1，如果丢弃了该事件则返回 0。如果它处理该事件，事件处理器应执行当前状态的退出动作，然后执行转换动作，最后将上下文的 `currentState` 设置为新状态。新状态的进入动作将在状态对象的事件处理器返回之后由上下文 执行。这些状态类包括：

- NoNumberState
- GN_处理分数状态
- GN_处理整个状态

每个事件处理器除了进入和退出动作之外，还可以拥有由其事件处理器调用的额外辅助函数，特别是用于转换动作的辅助函数。

每个状态对象都有一个 `StateActionSet` 类的实例，该类实现了指向每个事件处理器的函数指针。在状态对象的初始化过程中，这些函数指针被设置

to point to the appropriate event handler functions of the state class. In this way, polymorphism is addressed to support general event handling regardless of which is the current state.

The actions of each state class is normally a manipulation of the values owned by the `Context`. For this reason, each state class has an association (pointer) back to the `Context` to access those values. An alternative implementation, discussed later, is to pass the `Context`'s `me` pointer to each action to make the state classes truly context-free.

5.6.4.6 StateActionSet

This class is the key to implementing the required polymorphism. Each state class owns its own instance of this class in a variable called `aSet`. Each event handler (and the entry and exit actions) have a function pointer that will be dereferenced to invoke the event handler for the current state. In this case, the handlers are either of type `ActionPtr0` (no parameters other than the me pointer) and `ActionPtr1` (one char parameter plus the `me` pointer). To implement events with other parameter lists, new action pointer types must be created.

This class has no behavior other than an initializer that sets all the pointers to `NULL` initially. It is expected that the owner of each instance of the `StateActionSet` will set the function pointers to access the correct functions.

5.6.4.7 TSTATETYPE

`TSTATE` is an enumerated type of the high level states. Note that the nested state machine is flattened to remove nesting. This greatly simplifies the state pattern implementation. The enumerated values for this type are:

- `NULL_STATE`
- `NONUMBER_STATE`
- `GN_PROCESSINGWHOLEPART_STATE`
- `GN_PROCESSINGFRACTIONALPART_STATE`

Where the `GN_` prefix is a reminder that the last two states were nested states of the `GotNumber` state.

5.6.5 Consequences

This pattern localizes the behavior specific to a state to a single object, simplifying and encapsulating state-specific behavior. This means new states can be added by adding a new state class. New events are more work to add, since they must be at least added to the `StateActionSet` and usually to each state class as well. The State Pattern makes the event transitions explicit by providing an event handler for each incoming event in each state class.

This pattern uses more memory than a state table but simplifies the implementation by distributing the complexity among the various state objects.

为指向状态类中相应的事件处理函数。通过这种方式，多态得以实现，从而支持通用的事件处理，而无需考虑当前状态是哪一个。

每个状态类的动作通常是对上下文所拥有的值进行操作。因此，每个状态类都有一个指回上下文 的关联（指针），以访问这些值。另一种实现方式（稍后讨论）是将上下文的me指针传递给每个动作，从而使状态类真正与上下文无关。

5.6.4.6 StateActionSet

该类是实现所需多态的关键。每个状态类都在名为`aSet`的变量中拥有该类自己的实例。每个事件处理器（以及进入和退出动作）都有一个函数指针，该指针将被解引用以调用当前状态的事件处理器。在这种情况下，处理器要么是`ActionPtr0` 类型（除me指针外无其他参数），要么是`ActionPtr1` 类型（一个char参数加上`me` 指针）。要实现具有其他参数列表的事件，必须创建新的动作指针类型。

该类除了一个初始化器外没有其他行为，该初始化器最初将所有指针设置为空 。预期每个`StateActionSet` 实例的拥有者将设置函数指针以访问正确的函数。

5.6.4.7 TSTATETYPE

`TSTATE` 是高层状态的枚举类型。注意，嵌套状态机已被扁平化以移除嵌套。这极大地简化了状态模式的实现。该类型的枚举值为：

- NULL STATE_
- NONUMBER状态_
- GN PROCESSINGWHOLEPART 状态_
- GN PROCESSINGFRACTIONALPART STATE

其中 `GN_` 前缀提醒我们，最后两个状态是 `GotNumber` 状态的嵌套状态。

5.6.5 结果

该模式将特定于某个状态的行为集中到单个对象中，从而简化和封装了状态特定的行为。这意味着可以通过添加新的状态类来添加新状态。添加新事件则需要更多工作，因为它们至少必须添加到 `StateActionSet` 中，而且通常还需要添加到每个状态类中。状态模式通过在每个状态类中为每个传入事件提供事件处理器，使事件转换变得明确。

此模式比状态表占用更多内存，但通过将复杂性分散到各个状态对象中，简化了实现。

Another advantage of this pattern is that with a small change in implementation, it is possible to share state objects among multiple instances of the `Context` class – for example, if the system has multiple sensors all typed by the same class. This makes it very lightweight in terms of memory usage in those cases.

5.6.6 Implementation Strategies

This pattern is straightforward in C++ but rather more complex in C because it uses inheritance and polymorphism. In the implementation shown here, polymorphism is implemented with a combination of two techniques. First, all the state classes implement the same event handling interface; that is each state implements an event handler for all events even if the event is discarded. The event handlers return a 1 if the event was handled and 0 if the event was discarded. This is necessary because the `Context` needs to know if it should execute the entry actions of the new state.

The `StateActionSet` class then provided the polymorphic dispatch of the functions by providing a set of pointers for the event handlers and the entry and exit handlers. Each state class implemented the event handlers appropriately and then put a function pointer in its `StateActionSet` instance. The `Context` class uses the `currentState` variable as the index to dereference the pointer to the state that is "current," and further dereferences the state's `StateActionSet` to invoke the appropriate event handler.

Two primary implementation variants occur around whether to create all the state objects at once and simply switch among them based on the `currentState` variable of the `Context`, or to dynamically create and destroy the instances of the different state. In general, if the system changes state only infrequently, then memory can be saved by only creating the state as it is needed and destroying it when the `Context` is no longer in that state. If the state changes are frequent, then this approach will likely impose a significant performance penalty, so at the cost of some additional memory, it is best to keep all the state instances in memory.

An interesting property of the pattern is that the state objects themselves have no state context other than the `itsContext` pointer. This means that the state objects can be shared by multiple instances of the `Context` class. This might occur if, for example, the system has many sensor instances all typed by the same `Context` class. In this case the State Pattern can implement shared state objects, resulting in significant memory savings. To decontextualize the state classes, remove the class's `itsContext` pointer and add a `Context` pointer (that's the `Context`'s `me` pointer) to each event handler parameter list (and also to the entry and exit actions). When this is done, each state class is completely independent of its `Context`.

5.6.7 Related Patterns

The State Pattern implements a separate event receptor for each event, similar to the Single Event Receptor Pattern. The State Pattern is useful for more complex state machines because

此模式的另一个优点是，只需对实现稍作修改，就可以在上下文 类的多个实例之间共享状态对象——例如，如果系统有多个传感器，且它们都属于同一个类。在这些情况下，这使得内存使用非常轻量。

5.6.6 实现策略

此模式在 C++ 中很直接，但在 C 中则相当复杂，因为它使用了继承和多态。在本文所示的实现中，多态是通过两种技术的组合来实现的。首先，所有状态类都实现相同的事件处理接口；也就是说，每个状态都为所有事件实现一个事件处理器，即使该事件被丢弃。如果事件被处理，事件处理器返回 1；如果事件被丢弃，则返回 0。这是必要的，因为 上下文 需要知道是否应执行新状态的进入动作。

然后，`StateActionSet` 类通过为事件处理器以及进入和退出处理器提供一组指针，来提供函数的多态分派。每个状态类都适当地实现了事件处理器，然后在其 `StateActionSet` 实例中放置一个函数指针。上下文 类使用 `currentState` 变量作为索引，来解引用指向"当前"状态的指针，并进一步解引用该状态的`StateActionSet` 以调用相应的事件处理器。

两种主要的实现变体围绕以下选择产生：是一次性创建所有状态对象，然后仅根据 Context 的 currentState 变量在它们之间切换，还是动态创建和销毁不同状态的实例。一般来说，如果系统状态变化不频繁，那么可以通过仅在需要时创建状态并在 `Context` 不再处于该状态时销毁它来节省内存。如果状态变化频繁，那么这种方法可能会带来显著的性能损失，因此以一些额外内存为代价，最好将所有状态实例保留在内存中。

该模式的一个有趣特性是，状态对象本身除了 `itsContext` 指针之外没有状态上下文。这意味着状态对象可以由 `Context` 类的多个实例共享。例如，如果系统有许多传感器实例，且它们都由同一个 `Context` 类定型，就可能出现这种情况。在这种情况下，状态模式可以实现共享状态对象，从而显著节省内存。要去上下文化状态类，请移除该类的 `itsContext` 指针，并向每个事件处理器参数列表（以及进入和退出动作）添加一个 `Context` 指针（即 `Context` 的 me 指针）。完成此操作后，每个状态类都完全独立于其 `Context`。

5.6.7 相关模式

状态模式为每个事件实现一个单独的事件接收器，类似于单事件接收器模式。状态模式对于更复杂的状态机很有用，因为

the event handling is decomposed across different state objects. On the other hand, implementation of nested states is somewhat simpler in the case of the Single Event Receptor Pattern.

The State Table Pattern is similar in usage to the State Pattern, in that both are greatly simplified by flattening the state hierarchy. However, the former cannot easily share implementation so when the system has multiple instances of the `Context`, the latter may be more memory efficient. Both patterns have good performance.

5.6.8 Example

This section shows the implementation of the various elements of this pattern for the tokenizer example in Figure 5-12. Code Listing 5-17 shows the common types used by the other elements.

```
#ifndef StatePattern_H
#define StatePattern_H

struct Context;
struct GN_ProcessingFractionState;
struct GN_ProcessingWholeState;
struct Mutex;
struct NoNumberState;
struct SPClient;
struct StateActionSet;
struct StatePatternBulder;

typedef enum TSTATETYPE {
  NULL_STATE,
  NONUMBER_STATE,
  GN_PROCESSINGWHOLEPART_STATE,
  GN_PROCESSINGFRACTIONALPART_STATE
} TSTATETYPE;

typedef int (*ActionPtr0)(void*);
typedef int (*ActionPtr1)(void*, char);

int digit(char c); /* returns c-'0' */

#endif
```

Code Listing 5-17: StatePattern.h

Next we have the `Context` code in Code Listing 5-18 and Code Listing 5-19.

```
#ifndef Context_H
#define Context_H

#include "StatePattern.h"
#include "GN_ProcessingFractionState.h"
```


事件处理被分解到不同的状态对象上。另一方面，在单事件接收器模式的情况下，嵌套状态的实现要简单一些。

状态表模式在使用上与状态模式相似，两者都通过展平状态层次结构而大大简化。然而，前者难以共享实现，因此当系统有多个 上下文 实例时，后者可能更节省内存。两种模式都具有良好的性能。

5.6.8 示例

本节展示了图5-12中分词器示例的该模式各元素的实现。代码清单5-17展示了其他元素所使用的公共类型。

```
#ifndef StatePattern_H #define StatePattern_
H struct Context; struct GN_
ProcessingFractionState; struct GN_
ProcessingWholeState; struct Mutex; struct
NoNumberState; struct SPClient; struct
StateActionSet; struct StatePatternBulder;
typedef enum TSTATETYPE { NULL_STATE,
NONUMBER_STATE, GN_
PROCESSINGWHOLEPART_STATE, GN
PROCESSINGFRACTIONALPART STATE
_                                    _}
TSTATETYPE; typedef int (
*ActionPtr0)(void*); typedef int
(*ActionPtr1)(void*, char); int digit(char c); /*
返回 c-'0' */ #endif
```

代码清单 5-17：StatePattern.h

接下来，我们在代码清单 5-18 和代码清单 5-19 中给出了 上下文 代码。

```
#ifndef Context_H #define Context_H #include "
StatePattern.h" #include "GN_
ProcessingFractionState.h"
```

```
#include "GN_ProcessingWholeState.h"
#include "NoNumberState.h"

struct Mutex;

typedef struct Context Context;
struct Context {
  char ch;
  TSTATETYPE currentState;
  double result;
  struct NoNumberState*
stateList[GN_PROCESSINGFRACTIONALPART_STATE+1];
  double tensPlace;
  struct Mutex* itsMutex;
};

/* Constructors and destructors:*/
void Context_Init(Context* const me);
void Context_Cleanup(Context* const me);

/* Operations */
void Context_evDigit(Context* const me, char c);
void Context_evDot(Context* const me);
void Context_evEndOfString(Context* const me);
void Context_evWhiteSpace(Context* const me);

struct Mutex* Context_getItsMutex(const Context* const me);
void Context_setItsMutex(Context* const me, struct Mutex* p_Mutex);
Context * Context_Create(void);
void Context_Destroy(Context* const me);

#endif
```

Code Listing 5-18: Context.h

```
#include "Context.h"
#include "Mutex.h"

static void cleanUpRelations(Context* const me);

void Context_Init(Context* const me) {
  me->ch = 'x';
  me->currentState = NULL_STATE;
  me->result = 0.0;
  me->tensPlace = 10.0;
  me->itsMutex = NULL;

  me->stateList[NULL_STATE] = NULL;

  /* set up state list */
  me->stateList[NONUMBER_STATE] = NoNumberState_Create();
  me->stateList[NONUMBER_STATE]->itsContext = me;
  me->stateList[GN_PROCESSINGWHOLEPART_STATE] = (NoNumberState*) GN_ProcessingWhole
State_Create();
```



```
#include "GN_ProcessingWholeState.h" #
include "NoNumberState.h"

struct Mutex;

typedef struct Context Context; struct Context { char ch;
TSTATETYPE currentState; double result; struct
NoNumberState* stateList[GN_
PROCESSINGFRACTIONALPART_STATE+1];double
tensPlace; struct Mutex* itsMutex; };

/* 构造函数与析构函数：*/ void Context_
Init(Context* const me); void Context_
Cleanup(Context* const me);

/* 操作 */
void Context_evDigit(Context* const me, char c)          ;
void Context_evDot(Context* const me); void Context_
evEndOfString(Context* const me); void Context_
evWhiteSpace(Context* const me);

struct Mutex* Context_getItsMutex(const Context* const me); void Context_
setItsMutex(Context* const me, struct Mutex* p_Mutex);
Context * Context_Create(void); void Context_
Destroy(Context* const me);

#endif
```

代码清单 5-18: Context.h

```
#include "Context.h" #
include "Mutex.h"

static void cleanUpRelations(Context* const me);

void Context_Init(Context* const me) { me->
ch = 'x';me->currentState = 空_STATE;
me->result = 0.0; me->tensPlace = 10.0; me->
itsMutex = 空;

  me->stateList[空_状态] = 空;

    /* 设置状态列表 */ me->stateList[NONUMBER_STATE] = NoNumberState_Create(); me->stateList[
NONUMBER_STATE]->itsContext = me; me->stateList[GN_PROCESSINGWHOLEPART_STATE] =
(NoNumberState*) GN_ProcessingWhole State_Create();
```

```
  me->stateList[GN_PROCESSINGWHOLEPART_STATE]->itsContext = me;
  me->stateList[GN_PROCESSINGFRACTIONALPART_STATE] = (NoNumberState*) GN_Processing
FractionState_Create();
  me->stateList[GN_PROCESSINGFRACTIONALPART_STATE]->itsContext = me;

  /* enter starting state */
  me->stateList[NONUMBER_STATE]->aSet.entryAction(me->stateList[NONUMBER_STATE]);
}
void Context_Cleanup(Context* const me) {
  NoNumberState_Destroy(me->stateList[NONUMBER_STATE]);
  GN_ProcessingWholeState_Destroy((GN_ProcessingWholeState *)me->stateList
[GN_PROCESSINGWHOLEPART_STATE]);
GN_ProcessingFractionState_Destroy((GN_ProcessingFractionState *)me->stateList
[GN_PROCESSINGFRACTIONALPART_STATE]);
  cleanUpRelations(me);
}
void Context_evDigit(Context* const me, char c) {
  Mutex_lock(me->itsMutex);
  /* this call does the entry and transition actions */
  /* and updates the currentState variable */
  if (me->stateList[me->currentState]->aSet.evDigitHandler(me->stateList[me->curren-
tState], c))
     me->stateList[me->currentState]->aSet.entryAction(me->stateList
[me->currentState]);
  Mutex_release(me->itsMutex);
}
void Context_evDot(Context* const me) {
  Mutex_lock(me->itsMutex);
  /* this call does the entry and transition actions */
  /* and updates the currentState variable */
  if (me->stateList[me->currentState]->aSet.evDotHandler(me->stateList
[me->currentState]))
    me->stateList[me->currentState]->aSet.entryAction(me->stateList
[me->currentState]);
  Mutex_release(me->itsMutex);
}
void Context_evEndOfString(Context* const me) {
  Mutex_lock(me->itsMutex);
  /* this call does the entry and transition actions */
  /* and updates the currentState variable */
  if (me->stateList[me->currentState]->aSet.evEOSHandler(me->stateList
[me->currentState]))
    me->stateList[me->currentState]->aSet.entryAction(me->stateList
[me->currentState]);
  Mutex_release(me->itsMutex);
}
void Context_evWhiteSpace(Context* const me) {
  Mutex_lock(me->itsMutex);
```


me->stateList[GN_PROCESSINGWHOLEPART_STATE]->itsContext = me; me->stateList[GN_PROCESSINGFRACTIONALPART_STATE] = (NoNumberState*) GN_Processing FractionState_Create(); me->stateList[GN_PROCESSINGFRACTIONALPART_STATE]->itsContext = me; /* 进入起始状态 */ me->stateList[NONUMBER_STATE]->aSet.entryAction(me->stateList[NONUMBER_STATE]); } void Context_Cleanup(Context* const me) { NoNumberState_Destroy(me->stateList[NONUMBER_STATE]); GN_ProcessingWholeState_Destroy((GN_ProcessingWholeState *)me->stateList[GN_PROCESSINGWHOLEPART_STATE]); GN_ProcessingFractionState_Destroy((GN_ProcessingFractionState *)me->stateList[GN_PROCESSINGFRACTIONALPART_STATE]); cleanUpRelations(me); } void Context_evDigit(Context* const me, char c) { Mutex_lock(me->itsMutex); /* 此调用执行进入动作和迁移动作 */ /* 并更新 currentState 变量 */ if (me->stateList[me->currentState]->aSet.evDigitHandler(me->stateList[me->curren- tState], c)) me->stateList[me->currentState]->aSet.entryAction(me->stateList[me->currentState]); Mutex_release(me->itsMutex); } void Context_evDot(Context* const me) { Mutex_lock(me->itsMutex); /* 此调用执行进入动作和迁移动作 */ /* 并更新 currentState 变量 */ if (me->stateList[me->currentState]->aSet.evDotHandler(me->stateList[me->currentState])) me->stateList[me->currentState]->aSet.entryAction(me->stateList[me->currentState]); Mutex_release(me->itsMutex); } void Context_evEndOfString(Context* const me) { Mutex_lock(me->itsMutex); /* 此调用执行进入动作和迁移动作 */ /* 并更新 currentState 变量 */ if (me->stateList[me->currentState]->aSet.evEOSHandler(me->stateList[me->currentState])) me->stateList[me->currentState]->aSet.entryAction(me->stateList[me->currentState]); Mutex_release(me->itsMutex); } void Context_evWhiteSpace(Context* const me) { Mutex_lock(me->itsMutex);

```
  /* this call does the entry and transition actions */
  /* and updates the currentState variable */
  if (me->stateList[me->currentState]->aSet.evWSHandler(me->stateList
[me->currentState]))
     me->stateList[me->currentState]->aSet.entryAction(me->stateList
[me->currentState]);
  Mutex_release(me->itsMutex);
}

struct Mutex* Context_getItsMutex(const Context* const me) {
  return (struct Mutex*)me->itsMutex;
}

void Context_setItsMutex(Context* const me, struct Mutex* p_Mutex) {
  me->itsMutex = p_Mutex;
}

Context * Context_Create(void) {
  Context* me = (Context *) malloc(sizeof(Context));
  if(me!=NULL)
          Context_Init(me);
  return me;
}

void Context_Destroy(Context* const me) {
  if(me!=NULL)
          Context_Cleanup(me);
  free(me);
}

static void cleanUpRelations(Context* const me) {
  if(me->itsMutex != NULL)
          me->itsMutex = NULL;
}
```

Code Listing 5-19: Context.c

The code for the `StateActionSet` is straightforward as we see in Code Listing 5-20 and Code Listing 5-21.

```
#ifndef StateActionSet_H
#define StateActionSet_H

#include "StatePattern.h"

typedef struct StateActionSet StateActionSet;
struct StateActionSet {
  ActionPtr0 entryAction;
  ActionPtr1 evDigitHandler;
  ActionPtr0 evDotHandler;
  ActionPtr0 evEOSHandler;
```


/* 此调用执行进入动作和迁移动作 */ /* 并更新 currentState 变量 */ if (me->stateList[me->currentState]->aSet.evWSHandler(me->stateList[me->currentState])) me-> stateList[me->currentState]->aSet.entryAction(me->stateList[me->currentState]); Mutex_release(me->itsMutex); } struct Mutex* Context_getItsMutex(const Context* const me) { return (struct Mutex*)me->itsMutex; } void Context_ setItsMutex(Context* const me, struct Mutex* p_Mutex) { me->itsMutex = p_Mutex; } Context * Context_Create(void) { Context* me = (Context *) malloc(sizeof(Context)); if(me!=NULL)Context_Init(me); return me; } void Context_Destroy(Context* const me) { if(me!=NULL)Context_Cleanup(me); free(me); } static void cleanUpRelations(Context* const me) { if(me->itsMutex != NULL) me->itsMutex = NULL; }

代码清单 5-19：Context.c

`StateActionSet` 的代码很直接，如代码清单 5-20 和代码清单 5-21 所示。

#ifndef StateActionSet_H #define StateActionSet_H # include "StatePattern.h" typedef struct State ActionSet StateActionSet; struct StateActionSet { ActionPtr0 entryAction; ActionPtr1 evDigitHandler; ActionPtr0 evDotHandler; ActionPtr0 evEOS Handler;

```
  ActionPtr0 evWSHandler;
  ActionPtr0 exitAction;
};

/* Constructors and destructors:*/
void StateActionSet_Init(StateActionSet* const me);
void StateActionSet_Cleanup(StateActionSet* const me);

StateActionSet * StateActionSet_Create(void);
void StateActionSet_Destroy(StateActionSet* const me);

#endif
```

Code Listing 5-20: StateActionSet.h

```
#include "StateActionSet.h"
void StateActionSet_Init(StateActionSet* const me) {
  me->entryAction = NULL;
  me->evDigitHandler = NULL;
  me->evDotHandler = NULL;
  me->evEOSHandler = NULL;
  me->evWSHandler = NULL;
  me->exitAction = NULL;
}

void StateActionSet_Cleanup(StateActionSet* const me) {
}

StateActionSet * StateActionSet_Create(void) {
  StateActionSet* me = (StateActionSet *)
malloc(sizeof(StateActionSet));
  if(me!=NULL)
        StateActionSet_Init(me);
  return me;
}

void StateActionSet_Destroy(StateActionSet* const me) {
  if(me!=NULL)
        StateActionSet_Cleanup(me);
  free(me);
}
```

Code Listing 5-21: StateActionSet.c

The `NoNumberState` is the first state class. Its implementation is in Code Listing 5-22 and Code Listing 5-23.

```
#ifndef NoNumberState_H
#define NoNumberState_H

#include "StatePattern.h"
#include "StateActionSet.h"
```


ActionPtr0 evWSHandler; ActionPtr0 exitAction; }; /* 构造函数与析构函数: */ void StateActionSet_Init(StateActionSet* const me); void StateActionSet_Cleanup(StateActionSet* const me); StateActionSet * StateActionSet_Create(void); void StateActionSet_Destroy(StateActionSet* const me); #endif

代码清单5-20：StateActionSet.h

#include "StateActionSet.h" void StateActionSet_Init(StateActionSet* const me) { me->entryAction = NULL; me->evDigitHandler = NULL; me->evDotHandler = NULL; me->evEOSHandler = NULL; me->evWSHandler = NULL; me->exitAction = NULL; } void StateActionSet_Cleanup(StateActionSet* const me) { } StateActionSet * StateActionSet_Create(void) { StateActionSet* me = (StateActionSet *) malloc(sizeof(StateActionSet)); if(me!=NULL) StateActionSet_Init(me); return me; } void StateActionSet_Destroy(StateActionSet* const me) { if(me!=NULL)StateActionSet_Cleanup(me); free(me); }

代码清单5-21：StateActionSet.c

`NoNumberState` 是第一个状态类。其实现位于代码清单5-22和代码清单5-23中。

#ifndef NoNumberState_H # define NoNumberState_H # include "StatePattern.h" # include "StateActionSet.h"

```
struct Context;

typedef struct NoNumberState NoNumberState;
struct NoNumberState {
  struct StateActionSet aSet;
  struct Context* itsContext;
};

/* Constructors and destructors:*/
void NoNumberState_Init(NoNumberState* const me);
void NoNumberState_Cleanup(NoNumberState* const me);

/* Operations */
void NoNumberState_NoNum2GN(NoNumberState* const me, char c);
void NoNumberState_entryAction(NoNumberState* const me);
int NoNumberState_evDigit(NoNumberState* const me, char c);
int NoNumberState_evDot(NoNumberState* const me);
int NoNumberState_evEndOfString(NoNumberState* const me);
int NoNumberState_evWhiteSpace(NoNumberState* const me);
void NoNumberState_exitAction(NoNumberState* const me);

struct Context* NoNumberState_getItsContext(const NoNumberState* const me);
void NoNumberState_setItsContext(NoNumberState* const me, struct Context* p_Context);
NoNumberState * NoNumberState_Create(void);
void NoNumberState_Destroy(NoNumberState* const me);

#endif
```

Code Listing 5-22: NoNumberState.h

```
#include "NoNumberState.h"
#include "Context.h"
#include <stdio.h>

static void cleanUpRelations(NoNumberState* const me);

void NoNumberState_Init(NoNumberState* const me) {
  {
      StateActionSet_Init(&(me->aSet));
  }
  me->itsContext = NULL;

  /* set up the aSet function pointers */
  /* remember that the initializer for the StateActionSet */
  /* sets all pointers to NULL as the default */
  me->aSet.evDigitHandler = (ActionPtr1) NoNumberState_evDigit;
  me->aSet.evDotHandler = (ActionPtr0) NoNumberState_evDot;
  me->aSet.evWSHandler = (ActionPtr0) NoNumberState_evWhiteSpace;
  me->aSet.evEOSHandler = (ActionPtr0) NoNumberState_evEndOfString;
  me->aSet.entryAction = (ActionPtr0) NoNumberState_entryAction;
  me->aSet.exitAction = (ActionPtr0) NoNumberState_exitAction;
}
```



```
struct Context;

typedef struct NoNumberState NoNumberState;
struct NoNumberState { struct StateActionSet aSet;
struct Context* itsContext; };


/* 构造函数与析构函数：*/ void NoNumberState_
Init(NoNumberState* const me); void NoNumberState_
Cleanup(NoNumberState* const me);

/* 操作 */
void NoNumberState_NoNum2GN(NoNumberState* const me, char c); void
NoNumberState_entryAction(NoNumberState* const me); int NoNumberState_
evDigit(NoNumberState* const me, char c); int NoNumberState_
evDot(NoNumberState* const me); int NoNumberState_
evEndOfString(NoNumberState* const me); int NoNumberState_
evWhiteSpace(NoNumberState* const me); void NoNumberState_
exitAction(NoNumberState* const me);

struct Context* NoNumberState_getItsContext(const NoNumberState* const me); void NoNumberState_
setItsContext(NoNumberState* const me, struct Context* p_Context); NoNumberState * NoNumberState_
Create(void); void NoNumberState_Destroy(NoNumberState* const me);


#endif
```

代码清单5-22：NoNumberState.h

```
#include "NoNumberState.h" #include "Context.h" #include <stdio.h>static void
cleanUpRelations(NoNumberState* const me); void NoNumberState_
Init(NoNumberState* const me) { { StateActionSet_Init(&(me->aSet)); } me->
itsContext = NULL; /* 设置 aSet 函数指针 */ /* 请记住 StateActionSet 的初始化器 */ /
* 默认将所有指针设置为 NULL */ me->aSet.evDigitHandler = (ActionPtr1)
NoNumberState_evDigit; me->aSet.evDotHandler = (ActionPtr0)
NoNumberState_evDot; me->aSet.evWSHandler = (ActionPtr0) NoNumberState_
evWhiteSpace; me->aSet.evEOSHandler = (ActionPtr0) NoNumberState_
evEndOfString; me->aSet.entryAction = (ActionPtr0) NoNumberState_
entryAction; me->aSet.exitAction = (ActionPtr0) NoNumberState_exitAction; }
```

```
void NoNumberState_Cleanup(NoNumberState* const me) {
  cleanUpRelations(me);
}

void NoNumberState_NoNum2GN(NoNumberState* const me, char c) {
  me->itsContext->ch = c;
  me->itsContext->tensPlace = 10.0;
}

void NoNumberState_entryAction(NoNumberState* const me) {
  me->itsContext->result = 0.0;
}

int NoNumberState_evDigit(NoNumberState* const me, char c) {
  NoNumberState_exitAction(me);
  NoNumberState_NoNum2GN(me, c); /* transition action */
  /* set new state */
  me->itsContext->currentState = GN_PROCESSINGWHOLEPART_STATE;
  return 1; /* processed event */
}

int NoNumberState_evDot(NoNumberState* const me) {
  return 0; /* discarded event */
}

int NoNumberState_evEndOfString(NoNumberState* const me) {
  return 0; /* discarded event */
}

int NoNumberState_evWhiteSpace(NoNumberState* const me) {
  return 0; /* discarded event */
}

void NoNumberState_exitAction(NoNumberState* const me) {
}

struct Context* NoNumberState_getItsContext(const NoNumberState* const me) {
  return (struct Context*)me->itsContext;
}

void NoNumberState_setItsContext(NoNumberState* const me, struct Context* p_Context) {
  me->itsContext = p_Context;
}

NoNumberState * NoNumberState_Create(void) {
  NoNumberState* me = (NoNumberState *)
malloc(sizeof(NoNumberState));
  if(me!=NULL)
        NoNumberState_Init(me);
  return me;
}

void NoNumberState_Destroy(NoNumberState* const me) {
  if(me!=NULL)
        NoNumberState_Cleanup(me);
  free(me);
}
```


void NoNumberState_Cleanup(NoNumberState* const me) { cleanUpRelations(me); } void NoNumberState_NoNum2GN(NoNumberState* const me, char c) { me->itsContext->ch = c; me->itsContext->tensPlace = 10.0; } void NoNumberState_entryAction(NoNumberState* const me) { me->itsContext->result = 0.0; } int NoNumberState_evDigit(NoNumberState* const me, char c) { NoNumberState_exitAction(me); NoNumberState_NoNum2GN(me, c); /* 迁移动作 */ /* 设置新状态 */ me->itsContext->currentState = GN_PROCESSINGWHOLEPART_STATE; return 1; /* 已处理事件 */ } int NoNumberState_evDot(NoNumberState* const me) { return 0; /* 丢弃事件 */ } int NoNumberState_evEndOfString(NoNumberState* const me) { return 0; /* 丢弃事件 */ } int NoNumberState_evWhiteSpace(NoNumberState* const me) { return 0; /* 丢弃事件 */ } void NoNumberState_exitAction(NoNumberState* const me) { } struct Context* NoNumberState_getItsContext(const NoNumberState* const me) { return (struct Context*)me->itsContext; } void NoNumberState_setItsContext(NoNumberState* const me, struct Context* p_Context) { me->itsContext = p_Context; } NoNumberState * NoNumberState_Create(void) { NoNumberState* me = (NoNumberState *) malloc(sizeof(NoNumberState)); if(me!=NULL)NoNumberState_Init(me); return me; } void NoNumberState_Destroy(NoNumberState* const me) { if(me!=NULL)NoNumberState_Cleanup(me); free(me); }

```
static void cleanUpRelations(NoNumberState* const me) {
  if(me->itsContext != NULL)
        me->itsContext = NULL;
}
```

Code Listing 5-23: NoNumberState.c

The next state class is GN_ProcessingWholeState. Its implementation is in Code Listing 5-24 and Code Listing 5-25.

```
#ifndef GN_ProcessingWholeState_H
#define GN_ProcessingWholeState_H

#include "StatePattern.h"
#include "StateActionSet.h"

struct Context;

typedef struct GN_ProcessingWholeState GN_ProcessingWholeState;
struct GN_ProcessingWholeState {
  struct StateActionSet aSet;
  struct Context* itsContext;
};

/* Constructors and destructors:*/
void GN_ProcessingWholeState_Init(GN_ProcessingWholeState* const me);
void GN_ProcessingWholeState_Cleanup(GN_ProcessingWholeState* const me);

/* Operations */
void GN_ProcessingWholeState_entryAction(GN_ProcessingWholeState* const me);
int GN_ProcessingWholeState_evDigit(GN_ProcessingWholeState* const me, char c);
int GN_ProcessingWholeState_evDot(GN_ProcessingWholeState* const me);
int
GN_ProcessingWholeState_evEndOfString(GN_ProcessingWholeState* const me);

int
GN_ProcessingWholeState_evWhiteSpace(GN_ProcessingWholeState* const me);
void GN_ProcessingWholeState_exitAction(GN_ProcessingWholeState* const me);
void GN_ProcessingWholeState_printResult(GN_ProcessingWholeState* const me);

struct Context* GN_ProcessingWholeState_getItsContext(const GN_ProcessingWholeState*
const me);

void
GN_ProcessingWholeState_setItsContext(GN_ProcessingWholeState* const me, struct Con-
text* p_Context);

GN_ProcessingWholeState * GN_ProcessingWholeState_Create(void);
void GN_ProcessingWholeState_Destroy(GN_ProcessingWholeState* const me);

#endif
```

Code Listing 5-24: GN_ProcessingWholeState.h


```
static void cleanUpRelations(NoNumberState* const me) { if(
me->itsContext != 空) me->itsContext = 空; }
```

代码清单 5-23：NoNumberState.c

下一个状态类是 GN_ProcessingWholeState。其实现见代码清单 5-24 和代码清单 5-25。

```
#ifndef GN_ProcessingWholeState_H #
define GN_ProcessingWholeState_H

#include "StatePattern.h" #
include "StateActionSet.h"

struct Context;

typedef struct GN_ProcessingWholeState GN_ProcessingWholeState; struct
GN_ProcessingWholeState { struct StateActionSet aSet; struct Context*
itsContext; };

/* 构造函数与析构函数：*/ void GN_ProcessingWholeState_Init(GN_ProcessingWholeState*
const me); void GN_ProcessingWholeState_Cleanup(GN_ProcessingWholeState* const me);

/* 操作 */
void GN_ProcessingWholeState_entryAction(GN_ProcessingWholeState* const me); int GN_
ProcessingWholeState_evDigit(GN_ProcessingWholeState* const me, char c); int GN_
ProcessingWholeState_evDot(GN_ProcessingWholeState* const me); int

GN_ProcessingWholeState_evEndOfString(GN_ProcessingWholeState* const me);

int
GN_ProcessingWholeState_evWhiteSpace(GN_ProcessingWholeState* const me); void GN_
ProcessingWholeState_退出动作(GN_ProcessingWholeState* const me); void GN_
ProcessingWholeState_打印结果(GN_ProcessingWholeState* const me);

struct Context* GN_ProcessingWholeState_getItsContext(const GN_ProcessingWholeState* const me);

void
GN_ProcessingWholeState_setItsContext(GN_ProcessingWholeState* const me, struct 上下文* p_
Context);

GN_ProcessingWholeState * GN_ProcessingWholeState_Create(void); void GN_
ProcessingWholeState_Destroy(GN_ProcessingWholeState* const me);

#endif
```

代码清单 5-24：GN_ProcessingWholeState.h

```
#include "GN_ProcessingWholeState.h"
#include "Context.h"

static void cleanUpRelations(GN_ProcessingWholeState* const me);

void GN_ProcessingWholeState_Init(GN_ProcessingWholeState* const me) {
  StateActionSet_Init(&(me->aSet));
  me->itsContext = NULL;

  /* set up the aSet function pointers */
  /* remember that the initializer for the StateActionSet */
  /* sets all pointers to NULL as the default */
  me->aSet.evDigitHandler = (ActionPtr1) GN_ProcessingWholeState_evDigit;
  me->aSet.evDotHandler = (ActionPtr0) GN_ProcessingWholeState_evDot;
  me->aSet.evWSHandler = (ActionPtr0) GN_ProcessingWholeState_evWhiteSpace;
  me->aSet.evEOSHandler = (ActionPtr0) GN_ProcessingWholeState_evEndOfString;
  me->aSet.entryAction = (ActionPtr0) GN_ProcessingWholeState_entryAction;
  me->aSet.exitAction = (ActionPtr0) GN_ProcessingWholeState_exitAction;
}

void GN_ProcessingWholeState_Cleanup(GN_ProcessingWholeState* const me) {
  cleanUpRelations(me);
}

void GN_ProcessingWholeState_entryAction(GN_ProcessingWholeState* const me) {
  me->itsContext->result = me->itsContext->result*10 + digit(me->itsContext->ch);
}

int GN_ProcessingWholeState_evDigit(GN_ProcessingWholeState* const me, char c) {
  GN_ProcessingWholeState_exitAction(me);
  me->itsContext->ch = c; /* transitiona action */
  /* note: same state is reentered in this case */
  /* so no change to itsContext->currentState */
  return 1; /* event handled */
}

int GN_ProcessingWholeState_evDot(GN_ProcessingWholeState* const me) {
  GN_ProcessingWholeState_exitAction(me);
  /* no transition action */
  me->itsContext->currentState = GN_PROCESSINGFRACTIONALPART_STATE;
  return 1; /* event handled */
}

int
GN_ProcessingWholeState_evEndOfString(GN_ProcessingWholeState* const me) {
  GN_ProcessingWholeState_exitAction(me);
  GN_ProcessingWholeState_printResult(me); /* transition action */
  me->itsContext->currentState = NONUMBER_STATE;
  return 1; /* event handled */
}

int GN_ProcessingWholeState_evWhiteSpace(GN_ProcessingWholeState* const me) {
  GN_ProcessingWholeState_exitAction(me);
  GN_ProcessingWholeState_printResult(me); /* transition action */
  me->itsContext->currentState = NONUMBER_STATE;
```


#include "GN_ProcessingWholeState.h" #include "Context.h" static void cleanUpRelations(GN_ProcessingWholeState* const me); void GN_ProcessingWholeState_Init(GN_ProcessingWholeState* const me) { StateActionSet_Init(&(me->aSet)); me->itsContext = NULL; /* 设置 aSet 函数指针 */ /* 请记住 StateActionSet 的初始化器 */ /* 默认将所有指针设置为 NULL */ me->aSet.evDigitHandler = (ActionPtr1) GN_ProcessingWholeState_evDigit; me->aSet.evDotHandler = (ActionPtr0) GN_ProcessingWholeState_evDot; me->aSet.evWSHandler = (ActionPtr0) GN_ProcessingWholeState_evWhiteSpace; me->aSet.evEOSHandler = (ActionPtr0) GN_ProcessingWholeState_evEndOfString; me->aSet.entryAction = (ActionPtr0) GN_ProcessingWholeState_entryAction; me->aSet.exitAction = (ActionPtr0) GN_ProcessingWholeState_exitAction; } void GN_ProcessingWholeState_Cleanup(GN_ProcessingWholeState* const me) { cleanUpRelations(me); } void GN_ProcessingWholeState_entryAction(GN_ProcessingWholeState* const me) { me->itsContext->result = me->itsContext->result*10 + digit(me->itsContext->ch); } int GN_ProcessingWholeState_evDigit(GN_ProcessingWholeState* const me, char c) { GN_ProcessingWholeState_exitAction(me); me->itsContext->ch = c; /* 迁移动作 */ /* 注意：此情况下重新进入相同状态 */ /* 因此不改变 itsContext->currentState */ return 1; /* 事件处理 */ } int GN_ProcessingWholeState_evDot(GN_ProcessingWholeState* const me) { GN_ProcessingWholeState_exitAction(me); /* 无迁移动作 */ me->itsContext->currentState = GN_PROCESSINGFRACTIONALPART_STATE; return 1; /* 事件处理 */ } int GN_ProcessingWholeState_evEndOfString(GN_ProcessingWholeState* const me) { GN_ProcessingWholeState_exitAction(me); GN_ProcessingWholeState_printResult(me); /* 迁移动作 */ me->itsContext->currentState = NONUMBER_STATE; return 1; /* 事件处理 */ } int GN_ProcessingWholeState_evWhiteSpace(GN_ProcessingWholeState* const me) { GN_ProcessingWholeState_exitAction(me); GN_ProcessingWholeState_printResult(me); /* 迁移动作 */ me->itsContext->currentState = NONUMBER_STATE;

```
  return 1; /* event handled */
}

/*## operation exitAction() */
void GN_ProcessingWholeState_exitAction(GN_ProcessingWholeState* const me) {
}

void GN_ProcessingWholeState_printResult(GN_ProcessingWholeState* const me) {
  printf("Number: %g\n", me->itsContext->result);
}

struct Context* GN_ProcessingWholeState_getItsContext(const GN_ProcessingWholeState*
const me) {
  return (struct Context*)me->itsContext;
}

void
GN_ProcessingWholeState_setItsContext(GN_ProcessingWholeState* const me, struct Con-
text* p_Context) {
  me->itsContext = p_Context;
}

GN_ProcessingWholeState * GN_ProcessingWholeState_Create(void) {
  GN_ProcessingWholeState* me = (GN_ProcessingWholeState *) malloc(sizeof
(GN_ProcessingWholeState));
  if(me!=NULL)
        GN_ProcessingWholeState_Init(me);
  return me;
}

void GN_ProcessingWholeState_Destroy(GN_ProcessingWholeState* const me) {
  if(me!=NULL)
        GN_ProcessingWholeState_Cleanup(me);
  free(me);
}

static void cleanUpRelations(GN_ProcessingWholeState* const me) {
  if(me->itsContext != NULL)
        me->itsContext = NULL;
}
```

Code Listing 5-25: GN_ProcessingWholeState.c

And finally, the code for the `GN_ProcessingFractionState` is given in Code Listing 5-26 and Code Listing 5-27.

```
#ifndef GN_ProcessingFractionState_H
#define GN_ProcessingFractionState_H

#include "StatePattern.h"
#include "StateActionSet.h"

struct Context;

typedef struct GN_ProcessingFractionState GN_ProcessingFractionState;
struct GN_ProcessingFractionState {
```


return 1; /* 事件处理 */ } /*## 操作 exitAction() */ void GN_ProcessingWholeState_exitAction(GN_ProcessingWholeState* const me) { } void GN_ProcessingWholeState_printResult(GN_ProcessingWholeState* const me) { printf("Number: %g\n", me->itsContext->result); } struct Context* GN_ProcessingWholeState_getItsContext(const GN_ProcessingWholeState* const me) { return (struct Context*)me->itsContext; } void GN_ProcessingWholeState_setItsContext(GN_ProcessingWholeState* const me, struct Con- text* p_Context) { me->itsContext = p_Context; } GN_ProcessingWholeState * GN_ProcessingWholeState_Create(void) { GN_ProcessingWholeState* me = (GN_ProcessingWholeState *) malloc(sizeof (GN_ProcessingWholeState)); if(me!=NULL)GN_ProcessingWholeState_Init(me); return me; } void GN_ProcessingWholeState_Destroy(GN_ProcessingWholeState* const me) { if(me!=NULL)GN_ProcessingWholeState_Cleanup(me); free(me); } static void cleanUpRelations(GN_ProcessingWholeState* const me) { if(me->itsContext != NULL) me->itsContext = NULL; }

代码清单 5-25：GN_ProcessingWholeState.c

最后，`GN_ProcessingFractionState` 的代码在代码清单 5-26 和代码清单 5-27 中给出。

#ifndef GN_ProcessingFractionState_H #define GN_ProcessingFractionState_H # include "StatePattern.h" #include "StateActionSet.h" struct Context; typedef struct GN_ProcessingFractionState GN_ProcessingFractionState; struct GN_ProcessingFractionState {

```
  struct StateActionSet aSet;
  struct Context* itsContext;
};

void GN_ProcessingFractionState_Init(GN_ProcessingFractionState* const me);
void
GN_ProcessingFractionState_Cleanup(GN_ProcessingFractionState* const me);

/* Operations */

void
GN_ProcessingFractionState_Frac2Frac(GN_ProcessingFractionState* const me, char c);
void
GN_ProcessingFractionState_entryAction(GN_ProcessingFractionState* const me);
int
GN_ProcessingFractionState_evDigit(GN_ProcessingFractionState* const me, char c);
int
GN_ProcessingFractionState_evDot(GN_ProcessingFractionState* const me);
int
GN_ProcessingFractionState_evEndOfString(GN_ProcessingFractionState* const me);
int
GN_ProcessingFractionState_evWhiteSpace(GN_ProcessingFractionState* const me);
void
GN_ProcessingFractionState_exitAction(GN_ProcessingFractionState* const me);
void
GN_ProcessingFractionState_printResult(GN_ProcessingFractionState* const me);

struct Context* GN_ProcessingFractionState_getItsContext(const GN_Processing
FractionState* const me);

void
GN_ProcessingFractionState_setItsContext(GN_ProcessingFractionState* const me, struct
Context* p_Context);

GN_ProcessingFractionState *
GN_ProcessingFractionState_Create(void);

void GN_ProcessingFractionState_Destroy(GN_ProcessingFractionState* const me);

#endif
```

Code Listing 5-26: GN_ProcessingFractionState.h

```
#include "GN_ProcessingFractionState.h"
#include "Context.h"

static void cleanUpRelations(GN_ProcessingFractionState* const me);

void GN_ProcessingFractionState_Init(GN_ProcessingFractionState* const me) {
  StateActionSet_Init(&(me->aSet));
  me->itsContext = NULL;

  /* set up the aSet function pointers */
  /* remember that the initializer for the StateActionSet */
```



```
struct StateActionSet aSet; struct Context* itsContext; }; void GN_
ProcessingFractionState_Init(GN_ProcessingFractionState* const me); void GN_
ProcessingFractionState_Cleanup(GN_ProcessingFractionState* const me);

/* 操作 */

void GN_ProcessingFractionState_Frac2Frac(GN_ProcessingFractionState* const me, char c); void GN_
ProcessingFractionState_entryAction(GN_ProcessingFractionState* const me); int GN_
ProcessingFractionState_evDigit(GN_ProcessingFractionState* const me, char c); int GN_
ProcessingFractionState_evDot(GN_ProcessingFractionState* const me); int GN_
ProcessingFractionState_evEndOfString(GN_ProcessingFractionState* const me); int GN_
ProcessingFractionState_evWhiteSpace(GN_ProcessingFractionState* const me); void GN_
ProcessingFractionState_exitAction(GN_ProcessingFractionState* const me); void GN_
ProcessingFractionState_printResult(GN_ProcessingFractionState* const me); struct Context* GN_
ProcessingFractionState_getItsContext(const GN_Processing FractionState* const me); void GN_
ProcessingFractionState_setItsContext(GN_ProcessingFractionState* const me, struc
                                                                                t
Context* p_Context); GN_ProcessingFractionState * GN_ProcessingFractionState_Create(void);
void GN_ProcessingFractionState_Destroy(GN_ProcessingFractionState* const me); #endif
```

代码清单 5-26: GN_ProcessingFractionState.h

```
#include "GN_ProcessingFractionState.h" #
include "Context.h"

static void cleanUpRelations(GN_ProcessingFractionState* const me);

void GN_ProcessingFractionState_Init(GN_ProcessingFractionState* const me) {
StateActionSet_Init(&(me->aSet)); me->itsContext = 空;

  /* 设置 aSet 函数指针 */ /* 请记住 StateActionSet 的初始化器 */
```

```
  /* sets all pointers to NULL as the default */
  me->aSet.evDigitHandler = (ActionPtr1) GN_ProcessingFractionState_evDigit;
  me->aSet.evDotHandler = (ActionPtr0) GN_ProcessingFractionState_evDot;
  me->aSet.evWSHandler = (ActionPtr0) GN_ProcessingFractionState_evWhiteSpace;
  me->aSet.evEOSHandler = (ActionPtr0) GN_ProcessingFractionState_evEndOfString;
  me->aSet.entryAction = (ActionPtr0) GN_ProcessingFractionState_entryAction;
  me->aSet.exitAction = (ActionPtr0) GN_ProcessingFractionState_exitAction;
}

void
GN_ProcessingFractionState_Cleanup(GN_ProcessingFractionState* const me) {
  cleanUpRelations(me);
}

void
GN_ProcessingFractionState_Frac2Frac(GN_ProcessingFractionState* const me, char c) {
  me->itsContext->ch = c;
  me->itsContext->result += digit(me->itsContext->ch)/me->itsContext->tensPlace;
  me->itsContext->tensPlace *= 10;
}

void
GN_ProcessingFractionState_entryAction(GN_ProcessingFractionState* const me) {
}

int
GN_ProcessingFractionState_evDigit(GN_ProcessingFractionState* const me, char c) {
  GN_ProcessingFractionState_entryAction(me);
  GN_ProcessingFractionState_Frac2Frac(me, c); /* transition action */
  /* note: state doesn't change, so the same */
  /* state is reentered */
  return 1; /* event handled */
}

int
GN_ProcessingFractionState_evDot(GN_ProcessingFractionState* const me) {
  return 0; /* event discarded */
}

int
GN_ProcessingFractionState_evEndOfString(GN_ProcessingFractionState* const me) {
  GN_ProcessingFractionState_exitAction(me);
  GN_ProcessingFractionState_printResult(me); /* transition action */
  me->itsContext->currentState = NONUMBER_STATE;
  return 1; /* event handled */
}

int
GN_ProcessingFractionState_evWhiteSpace(GN_ProcessingFractionState* const me) {
  GN_ProcessingFractionState_exitAction(me);
  GN_ProcessingFractionState_printResult(me); /* transition action */
  me->itsContext->currentState = NONUMBER_STATE;
  return 1; /* event handled */
}
```


/* 默认将所有指针设置为 NULL */ me->aSet.evDigitHandler = (ActionPtr1) GN_ProcessingFractionState_evDigit; me->aSet.evDotHandler = (ActionPtr0) GN_ProcessingFractionState_evDot; me->aSet.evWSHandler = (ActionPtr0) GN_ProcessingFractionState_evWhiteSpace; me->aSet.evEOSHandler = (ActionPtr0) GN_ProcessingFractionState_evEndOfString; me->aSet.entryAction = (ActionPtr0) GN_ProcessingFractionState_entryAction; me->aSet.exitAction = (ActionPtr0) GN_ProcessingFractionState_exitAction; } void GN_ProcessingFractionState_Cleanup(GN_ProcessingFractionState* const me) { cleanUpRelations(me); } void GN_ProcessingFractionState_Frac2Frac(GN_ProcessingFractionState* const me, char c) { me->itsContext->ch = c; me->itsContext->result += digit(me->itsContext->ch)/me->itsContext->tensPlace; me->itsContext->tensPlace *= 10; } void GN_ProcessingFractionState_entryAction(GN_ProcessingFractionState* const me) { } int GN_ProcessingFractionState_evDigit(GN_ProcessingFractionState* const me, char c) { GN_ProcessingFractionState_entryAction(me); GN_ProcessingFractionState_Frac2Frac(me, c); /* 迁移动作 */ /* 注意：状态不会改变，因此 */ /* 重新进入同一状态 */ return 1; /* 事件处理 */ } int GN_ProcessingFractionState_evDot(GN_ProcessingFractionState* const me) { return 0; /* 事件丢弃 */ } int GN_ProcessingFractionState_evEndOfString(GN_ProcessingFractionState* const me) { GN_ProcessingFractionState_exitAction(me); GN_ProcessingFractionState_printResult(me); /* 迁移动作 */ me->itsContext->currentState = NONUMBER_STATE; return 1; /* 事件处理 */ } int GN_ProcessingFractionState_evWhiteSpace(GN_ProcessingFractionState* const me) { GN_ProcessingFractionState_exitAction(me); GN_ProcessingFractionState_printResult(me); /* 迁移动作 */ me->itsContext->currentState = NONUMBER_STATE; return 1; /* 事件处理 */ }

```
void
GN_ProcessingFractionState_exitAction(GN_ProcessingFractionState* const me) {
}

void
GN_ProcessingFractionState_printResult(GN_ProcessingFractionState* const me) {
  printf("Number: %g\n", me->itsContext->result);
}

struct Context* GN_ProcessingFractionState_getItsContext(const GN_ProcessingFraction-
State* const me) {
  return (struct Context*)me->itsContext;
}

void
GN_ProcessingFractionState_setItsContext(GN_ProcessingFractionState* const me, struct
Context* p_Context) {
  me->itsContext = p_Context;
}

GN_ProcessingFractionState *
GN_ProcessingFractionState_Create(void) {
  GN_ProcessingFractionState* me = (GN_ProcessingFractionState *) malloc(sizeof
(GN_ProcessingFractionState));
  if(me!=NULL)
        GN_ProcessingFractionState_Init(me);
  return me;
}

void
GN_ProcessingFractionState_Destroy(GN_ProcessingFractionState* const me) {
  if(me!=NULL)
        GN_ProcessingFractionState_Cleanup(me);
  free(me);
}

static void cleanUpRelations(GN_ProcessingFractionState* const me) {
  if(me->itsContext != NULL)
        me->itsContext = NULL;
}
```

Code Listing 5-27: GN_ProcessingFractionState.c

It is interesting to see how these elements interact in execution of a simple case. Figure 5-19 shows the interaction flow for processing the string “1.3” sent from a client of the state machine. In this figure, you can see how the event handlers of the `Context` invoke the event handlers of the then-current state to process the incoming events. You may find it interesting to compare this figure with the corresponding figure for the State Table Pattern, Figure 5-17.

void GN_ProcessingFractionState_退出动作(GN_ProcessingFractionState* const me) { } void GN_ProcessingFractionState_打印结果(GN_ProcessingFractionState* const me) { printf("Number: %g\n", me->itsContext->result); } struct Context* GN_ProcessingFractionState_getItsContext(const GN_ProcessingFraction- State* const me) { return (struct Context*)me->itsContext; } void GN_ProcessingFractionState_setItsContext(GN_ProcessingFractionState* const me, struct Context* p_Context) { me->itsContext = p_Context; } GN_ProcessingFractionState * GN_ProcessingFractionState_Create(void) { GN_ProcessingFractionState* me = (GN_ProcessingFractionState *) malloc(sizeof (GN_ProcessingFractionState)); if(me!=NULL)GN_ProcessingFractionState_Init(me); return me; } void GN_ProcessingFractionState_Destroy(GN_ProcessingFractionState* const me) { if(me!=NULL)GN_ProcessingFractionState_Cleanup(me); free(me); } static void cleanUpRelations(GN_ProcessingFractionState* const me) { if(me->itsContext != 空) me->itsContext = 空; }

代码清单 5-27：GN_ProcessingFractionState.c

观察这些元素在一个简单案例的执行中如何交互是很有意思的。图 5-19 展示了处理从状态机客户端发送的字符串“1.3”的交互流程。在此图中，你可以看到 Context 的事件处理器如何调用当前状态的事件处理器来处理传入事件。你可能会觉得将此图与状态表模式对应的图 5-17 进行比较很有意思。

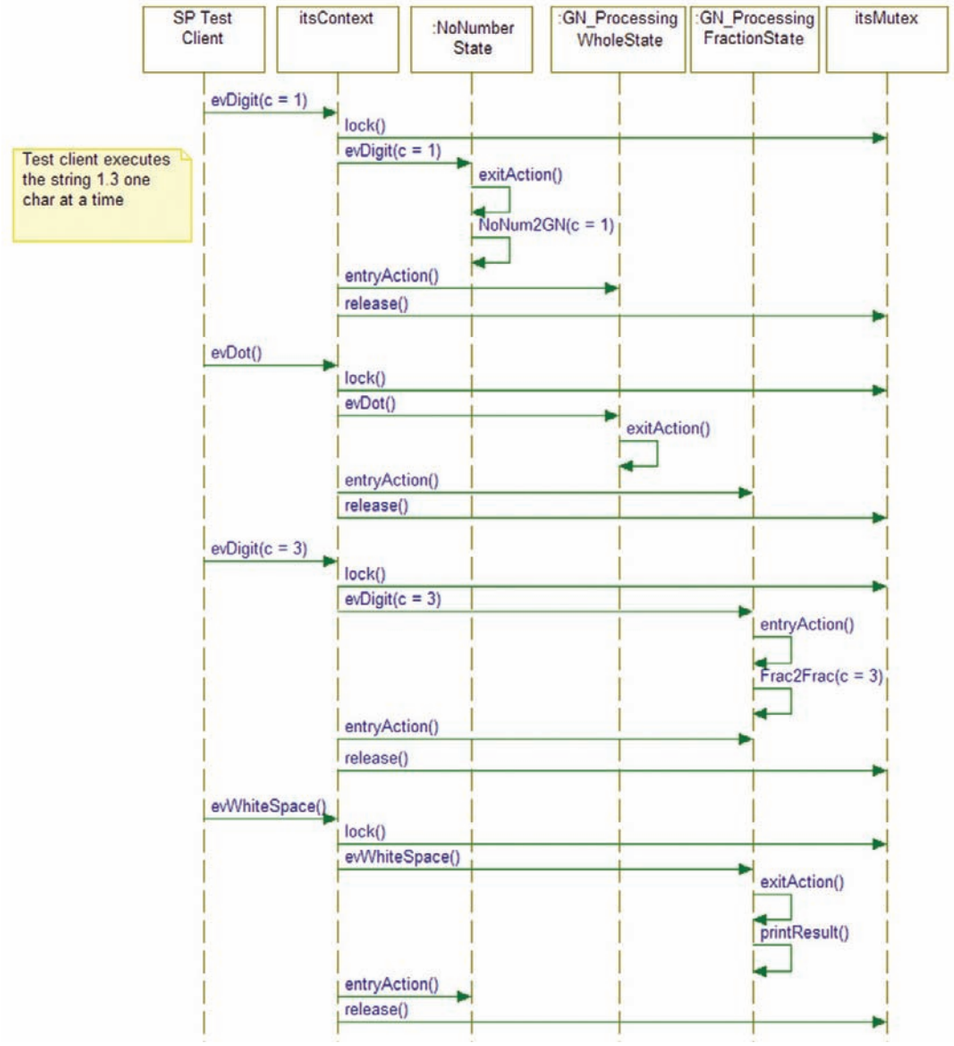


Figure 5-19: State Pattern sequence for execution of "1.3" string

5.7 AND-States

The last two patterns in the chapter will deal with the implementation of AND-states. For these patterns, we'll use the state machine from Figure 5-7 as the example state machine with AND-states and we'll add a few actions to make it more "real." Figure 5-20 shows the example state machine we will demonstrate with the pattern.

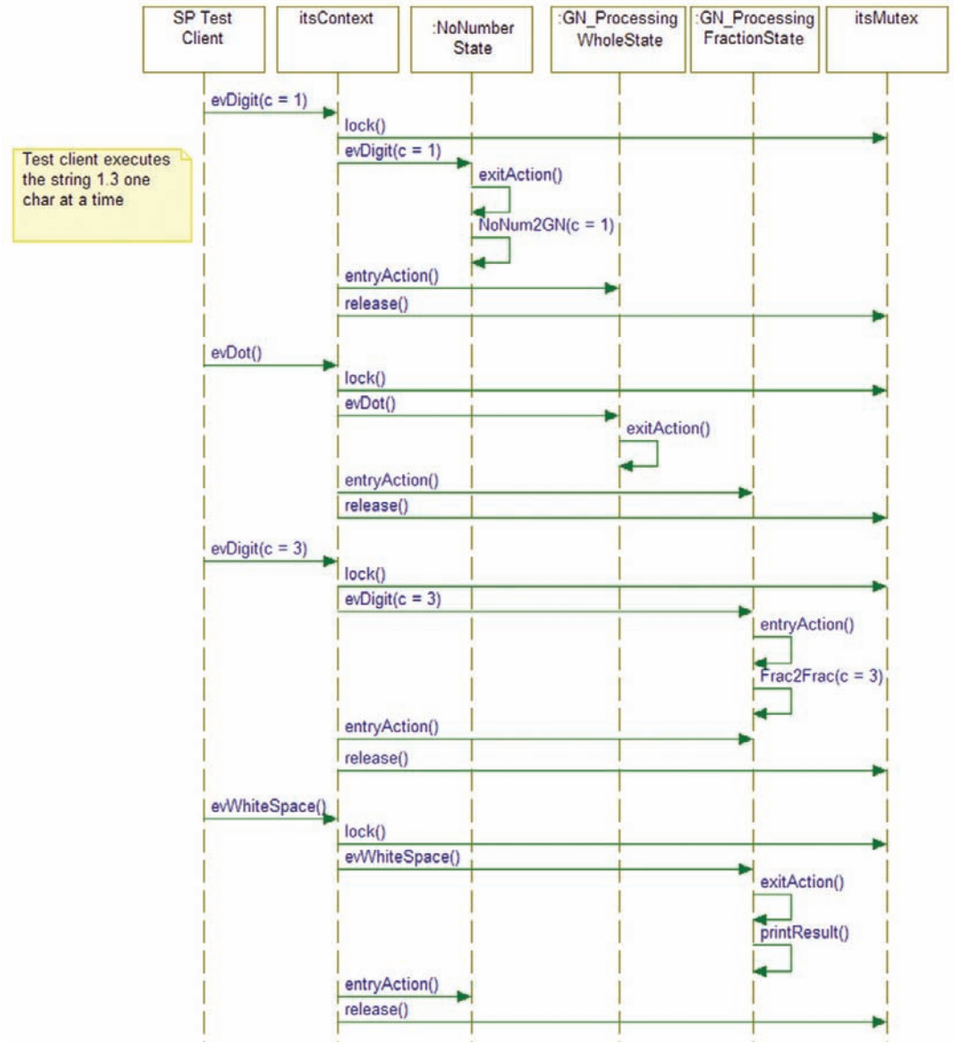


图 5-19：执行"1.3"字符串的状态模式序列

5.7 AND状态

本章的最后两个模式将处理 AND状态的实现。对于这些模式，我们将使用图 5-7 中的状态机作为带有 AND状态的示例状态机，并添加一些动作使其更加"真实"。图 5-20 展示了我们将用该模式演示的示例状态机。

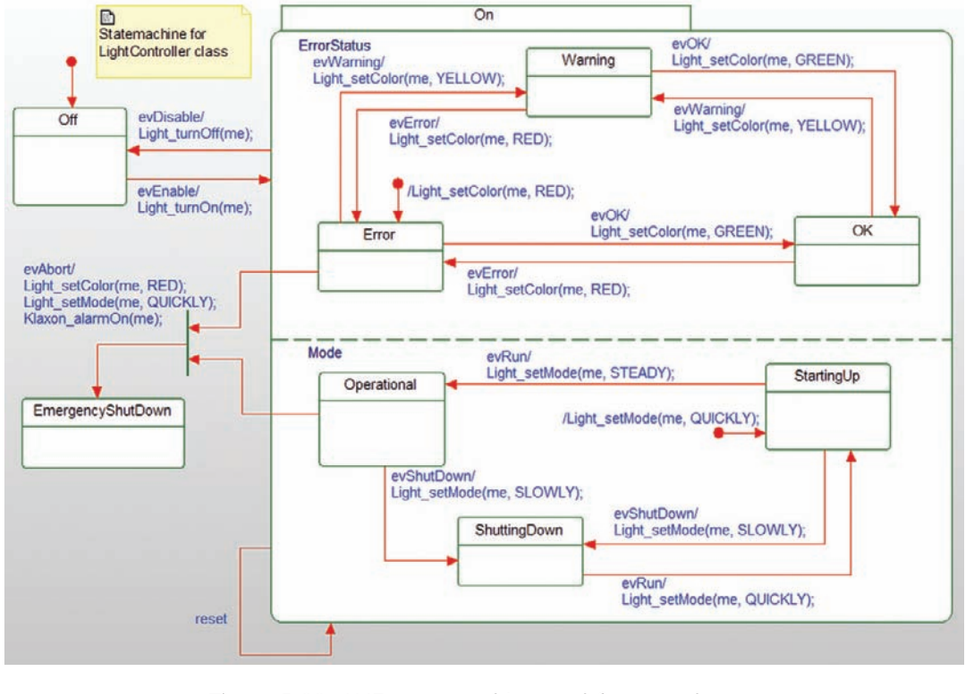


Figure 5-20: AND-state machine used for example case

The semantics of AND-states is that they are "logically concurrent." This means that in some sense they execute simultaneously, but what does that mean in a world of run-to-completion semantics? If two AND-states process different events, then AND-states are more of a (significant) notational convenience. Simultaneity only has importance when the same event is processed within two AND-states. In that case, what is the meaning of concurrency? In this context, it means that *the semantics of the system are not dependent upon which AND-state processes the event first.* It also implies that when AND-states are active, each active AND-state must receive, in some sense, its own copy of the event sent to the state machine, and is free to act on it or discard it as appropriate. What we cannot rely upon is which AND-state will act on the event first[5]. If we need to ensure that they execute in a particular order, then perhaps AND-states were not the appropriate mechanism to employ.

A naïve but overly heavyweight approach would be to spawn threads for the different state machines. While this can be made to work, in practice I have never seen AND-states implemented in this fashion, and for good reason. The logical concurrency really only requires:

[5] The UML has no notion of event or transition priority. Events are always handled in order of their arrival.

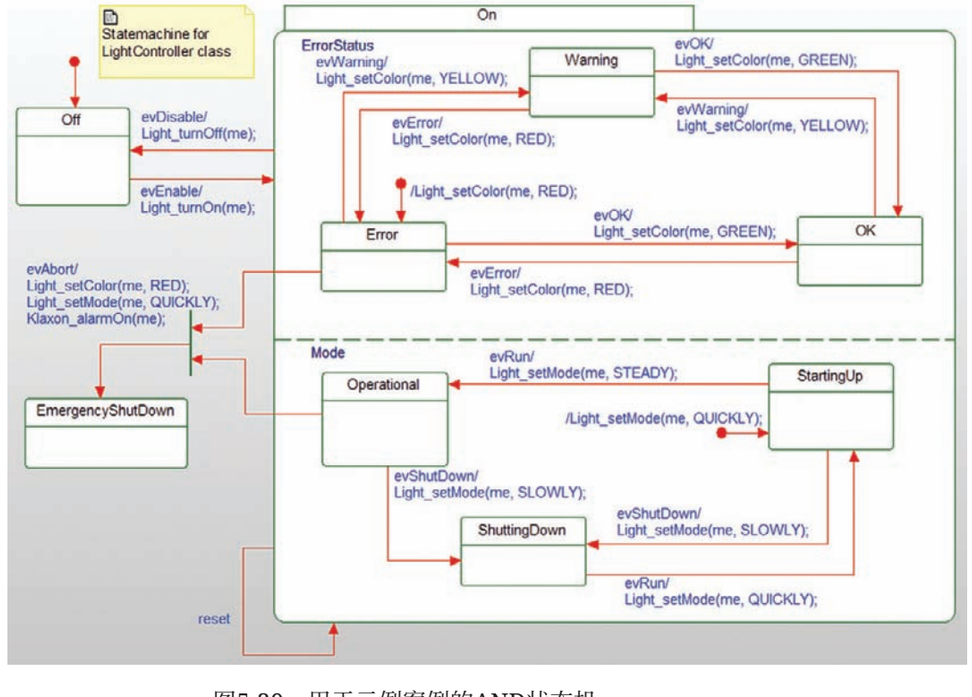


图5-20：用于示例案例的AND状态机

AND状态的语义是它们"逻辑上并发"。这意味着在某种意义上它们同时执行，但在运行至完成语义的世界中这意味着什么？如果两个AND状态处理不同的事件，那么AND状态更多是一种（显著的）表示法便利。只有当同一事件在两个AND状态内被处理时，同时性才具有重要性。在这种情况下，并发的含义是什么？在此上下文中，它意味着系统的语义不依赖于哪个AND状态先处理该事件。它还意味着当AND状态处于活动状态时，每个活动的AND状态必须在某种意义上接收发送给状态机的自己的事件副本，并可以自由地酌情对其采取行动或丢弃它。我们不能依赖的是哪个AND状态将先对事件采取行动[5]。如果我们需要确保它们按特定顺序执行，那么也许AND状态并不是适合采用的机制。

一种简单但过于笨重的方法是为不同的状态机生成线程。虽然这样做可以奏效，但在实践中我从未见过以这种方式实现AND状态，而且理由充分。逻辑并发实际上只需要：

[5] UML 没有　　事件或迁移优先级的概念。事件始终按照其到达顺序进行处理　　.

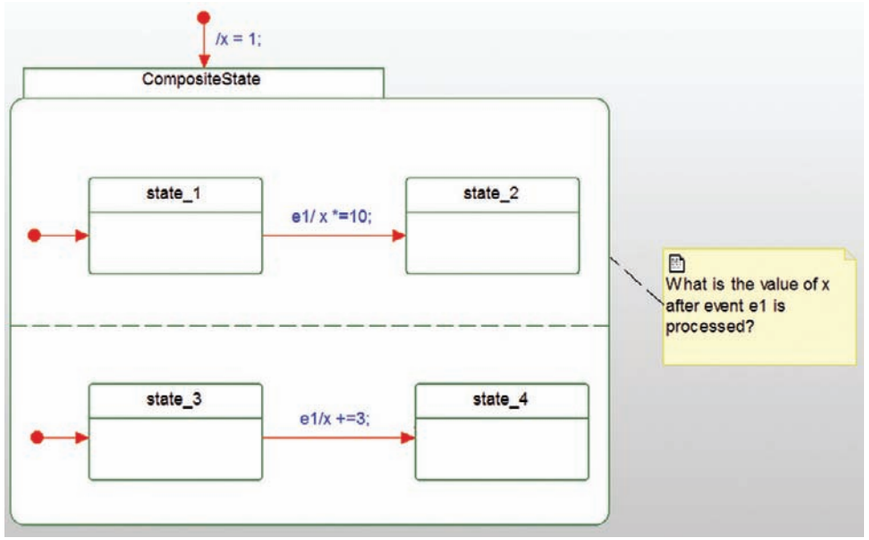


Figure 5-21: Race condition

- events are handled run-to-completion
- all AND-state event handlers must complete before the next event sent to the state machine is processed

So which AND-state processes the event first? The upper one? The one you drew first? The one that appears first in the code? *It doesn't matter.* That's the point.

Note that this does raise the specter of race conditions, as discussed in Chapter 4. For example, consider the state machine in Figure 5-21. It contains a race condition. The `e1` event is handled in two AND-states; each of which has an action manipulating the variable `x`. If the upper AND-state processes its event first[6], the resulting value of `x` is 33. If the lower one processes it first, the answer is 40.

Don't do that![7]

If you need to force a specific order of event execution, then you can use a simple pattern called *event propagation*. If you want to force the upper AND-state to execute first, then create a different event for the second transition and generate that event in an action of the first transition. This pattern, applied to the previous figure, is shown in Figure 5-22, where the GEN

[6] Notice that x is set to the value 1 before entering the `CompositeState`.

[7] A Law of Douglass is that any language sufficiently powerful to be useful is also sufficiently powerful to say nonsense. My rule is to try to say only smart things.

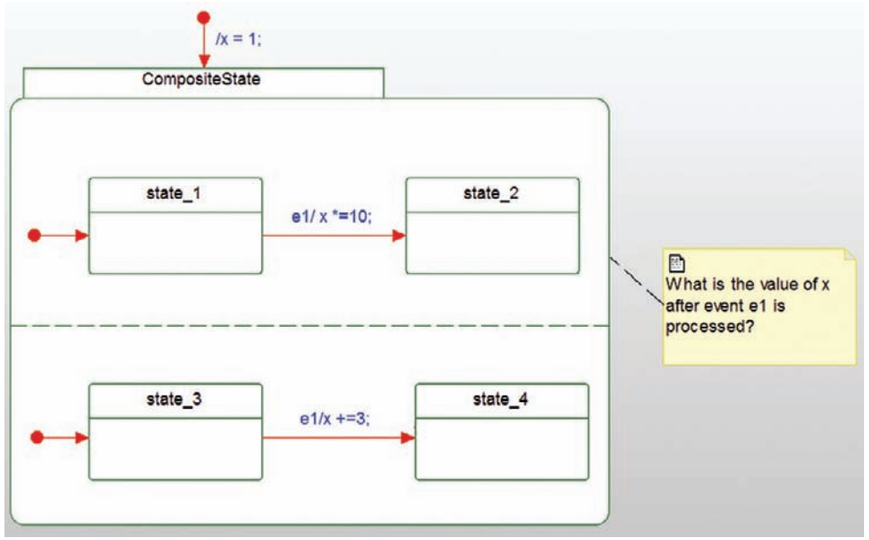


图5-21：竞态条件

- 事件以运行至完成的方式处理
- 所有AND状态的事件处理器必须在发送给状态机的下一个事件被处理之前完成

那么哪个AND状态先处理事件？上面的那个？你先画的哪个？代码中先出现的那个？这并不重要。这正是关键所在。

请注意，这确实引发了竞态条件的隐患，如第4章所述。例如，考虑图5-21中的状态机。它包含一个竞态条件。e1事件在两个AND状态中被处理；每个AND状态都有一个操作变量x的动作。如果上面的AND状态先处理其事件[6]，，则x的结果值为33。如果下面的AND状态先处理，则结果为40。

不要那样做！[7]

如果你需要强制特定的事件执行顺序，那么可以使用一种称为事件传播的简单模式。如果你想强制上层与状态先执行，那么为第二个迁移创建一个不同的事件，并在第一个迁移的动作中生成该事件。应用于前一张图的该模式如图5-22所示，其中GEN

[6] 请注意，在进入复合状态之前，x 已被设置为值 1。[7] 道格拉斯定律指出，任何足够强大到有用的语言，也同样足够强大到可以胡说八道。我的准则是尽量只说有意义的话。

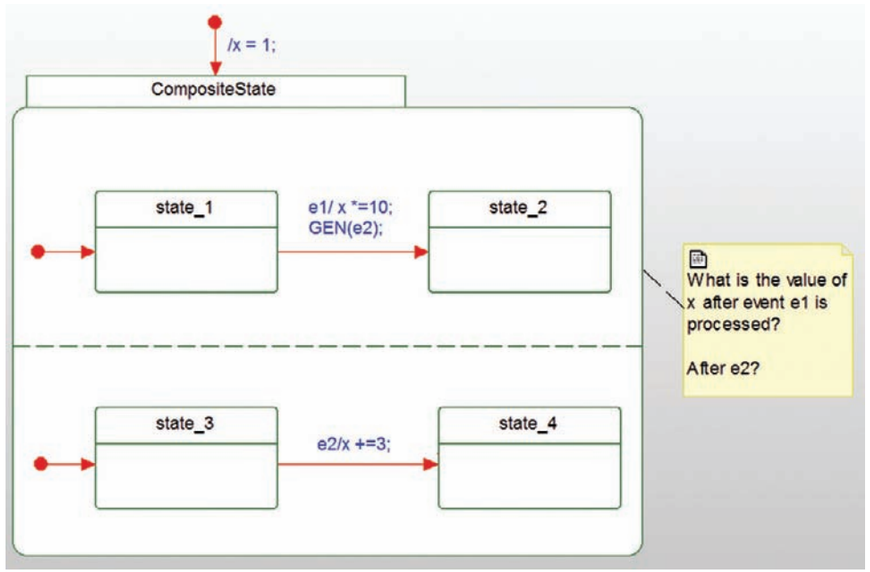


Figure 5-22: Event propagation

macro generates the event using whatever synchronous or asynchronous approach you've implemented for your state machine.

There are a number of ways commonly used to implement AND-states. The first, and least desirable, is to flatten the state space. This is problematic, as was pointed out in Figure 5-6 because of state explosion – you must compute the cross product of the state spaces and that's often very ugly and difficult to read and understand because of the sheer number of resulting states. It does work, however, and in simple cases it might be easiest.

A more common solution is to decompose the object owning the AND-states into a set of different objects that communicate. This is the Decomposed AND-State Pattern discussed below. It is also possible to elaborate one of the previous patterns we've presented into managing multiple AND-states in the event receptors. There are other solutions as well, but these are the most common.

5.8 Decomposed AND-State Pattern

This pattern extends the State Pattern to implement AND-states. It does so by creating a collaboration of different objects, each of which is allocated one of the AND-states.

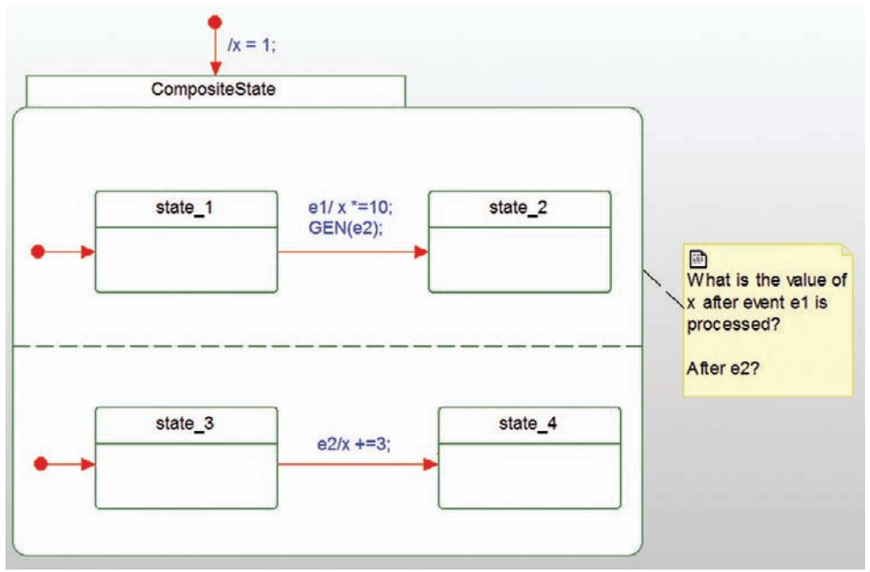


图5-22：事件传播

宏使用你为状态机实现的任何同步或异步方法来生成事件。

实现AND状态有几种常用方法。第一种，也是最不理想的一种，是展平状态空间。正如在图5-6中指出的，这种方法存在问题，因为会导致状态爆炸——你必须计算状态空间的笛卡尔积，而由于结果状态数量庞大，这通常非常丑陋且难以阅读和理解。不过它确实可行，在简单情况下可能还是最简单的做法。

更常见的解决方案是将拥有AND状态的对象分解为一组相互通信的不同对象。这就是下面要讨论的分解AND状态模式。也可以将我们之前介绍的某种模式扩展为在事件接收器中管理多个AND状态。还有其他解决方案，但这些是最常见的。

5.8 分解AND状态模式

此模式扩展了状态模式以实现AND状态。它通过创建不同对象的协作来实现这一点，每个对象被分配一个AND状态。

5.8.1 Abstract

The Decomposed And-State Pattern takes an object that owns a state machine and decomposes it into a set of objects that interact. The main object owns the overarching state machine and other objects own individual AND-states.

5.8.2 Problem

This pattern addresses the problem of implementing state machines that contain orthogonal regions (AND-states) in a way that preserves the AND-state design.

5.8.3 Pattern Structure

Figure 5-23 shows the structure of the pattern. Instead of a singular class that implements the state machine, the state problem is decomposed into a collaboration of objects that relate to the Context.

5.8.4 Collaboration Roles

This section describes the pattern roles.

5.8.4.1 AbstractAndStateClass

This class is an abstraction of the various concrete AND-state classes. It provides an event handler for each event received by the `Context` and a link back to the `Context` for execution of actions and manipulation of variables within the `Context`. In the C implementation, this is a notional class, not usually represented directly but it does show what is inherited by the subclasses.

5.8.4.2 ActionSet

This class contains a set of pointers, one per incoming event. It is owned by the `AbstractAndStateClass` and implemented by the `ConcreteAndStateClass`. In the initializer of the latter class, the function pointers are set to point to the functions that implement the actions within that concrete state.

5.8.4.3 AndStateList

This class maintains a list of orthogonal regions (containing all the nested states within an AND-state) for a given composite. There is an entry for every state; if the list is empty, then that state has no nested AND-states. If it is not empty, then the list entry contains a list of the orthogonal regions of AND-states; each region is represented by a `ConcreteAndStateClass` and it implements all the nested states within that region.

Consider the example state machine in Figure 5-20. `STATE_N` in this case is 1. The 0-th state is the `Off` state; this state has no nested AND-states so this list entry is empty. The 1st state is the `On` state; this state has two nested orthogonal regions, `Color` and `Rate`. Thus the

5.8.1 摘要

分解AND状态模式接收一个拥有状态机的对象，并将其分解为一组相互交互的对象。主对象拥有总体状态机，其他对象拥有各自的AND状态。

5.8.2 问题

此模式解决的问题是实现包含正交区域（AND状态）的状态机，同时保持AND状态设计。

5.8.3 模式结构

图5-23展示了该模式的结构。状态问题不是由单个类来实现状态机，而是被分解为与上下文相关联的对象协作。

5.8.4 协作角色

本节描述该模式的各个角色。

5.8.4.1 AbstractAndStateClass

该类是各种具体与状态类的抽象。它为 上下文 接收的每个事件提供一个事件处理器，并提供一个指回 上下文 的链接，用于执行动作和操作 上下文中的变量。在 C 实现中，这是一个概念上的类，通常不直接表示，但它确实展示了子类所继承的内容。

5.8.4.2 ActionSet

该类包含一组指针，每个传入事件对应一个指针。它由 AbstractAndStateClass 拥有，并由 `ConcreteAndStateClass` 实现。在后者的初始化器中，函数指针被设置为指向实现该具体状态内动作的函数。

5.8.4.3 AndStateList

该类为给定的复合状态维护一个正交区域列表（包含AND状态内的所有嵌套状态）。每个状态都有一个条目；如果列表为空，则该状态没有嵌套AND状态。如果列表不为空，则列表条目包含AND状态的正交区域列表；每个区域由一个

`ConcreteAndStateClass` 表示，并实现该区域内的所有嵌套状态。

考虑图 5-20 中的示例状态机。此例中`N` 为1。第0个状态是关闭 状态；该状态没有嵌套AND状态，因此该列表项为空。第1个状态是开启 状态；该状态有两个嵌套正交区域：颜色 和速率。因此，

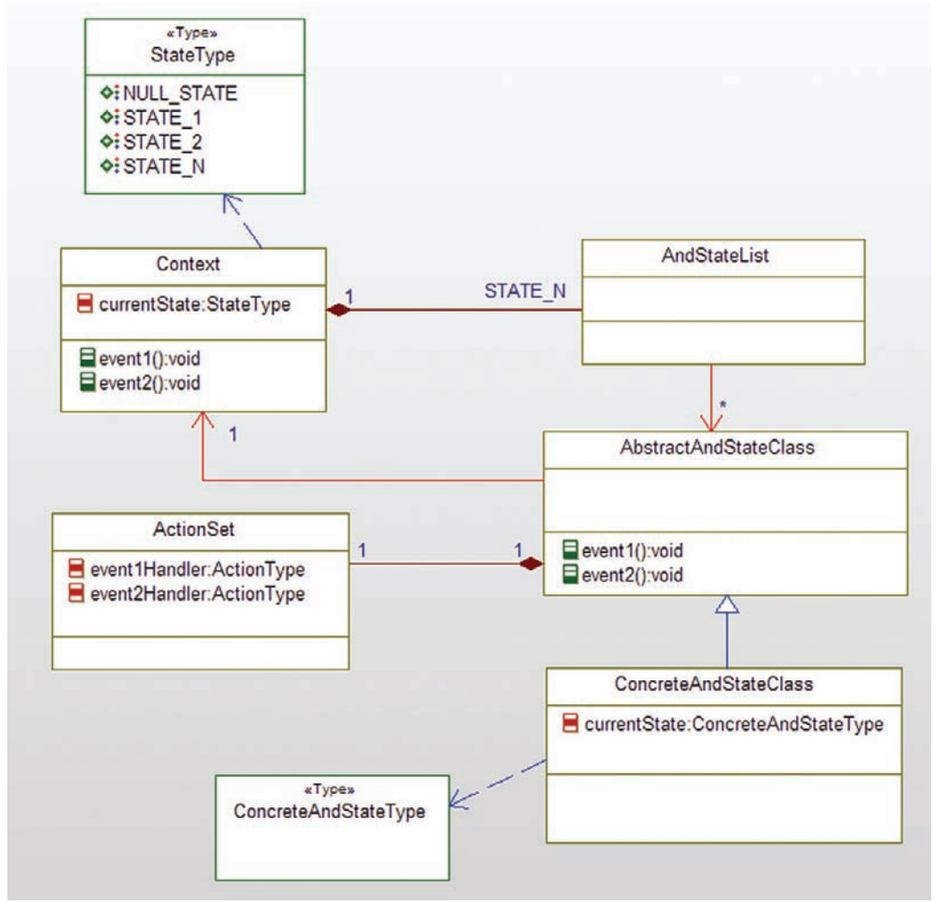


Figure 5-23: Decomposed And-State Pattern structure

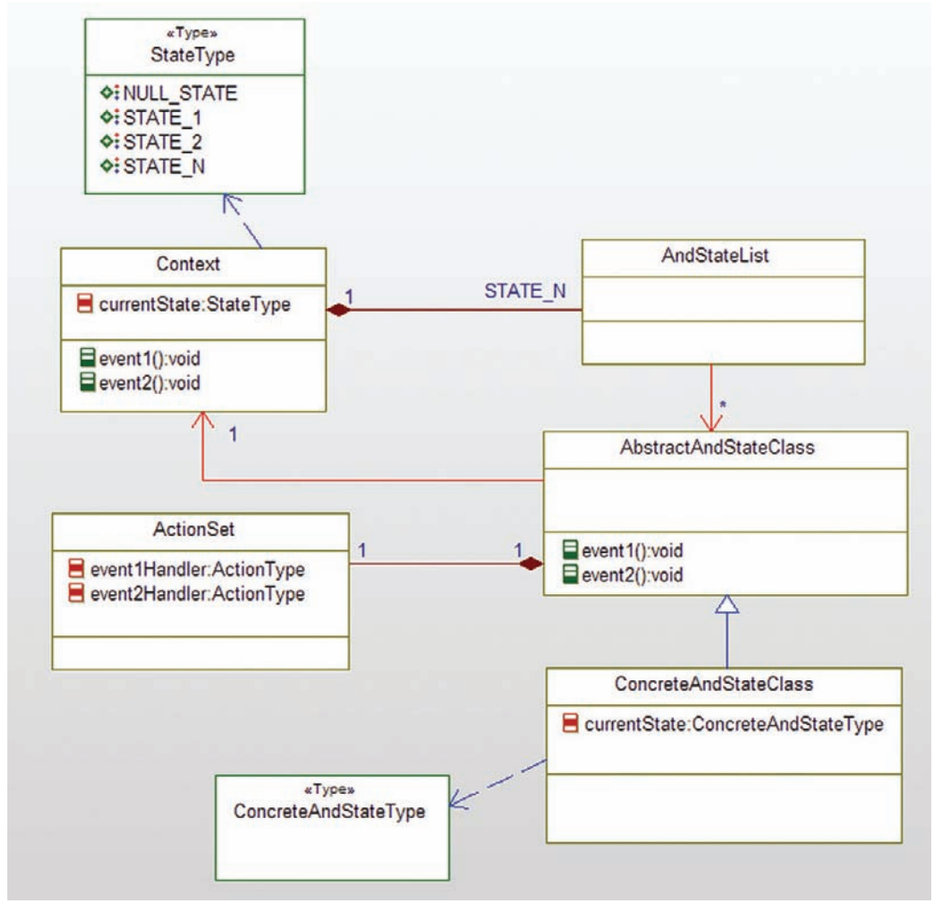


图5-23：分解AND状态模式结构

`Context` list entry for State 1 is not empty. In this case, the list entry for State 1 has two lists, one for each orthogonal region. The `ConcreteAndStateClass Color` manages three states – `Red`, `Yellow`, and `Green`. The `ConcreteAndStateClass Rate` also manages three states – `Steady`, `Flashing_Slowly`, and `Flashing_Quickly`. So the lists look like this:

- `Context` state list:
  - State 0 `(Off)` – Empty
  - State 1 `(On)` list:
    - `Color` (manages `Red`, `Yellow`, `Green` states)
    - `Rate` (manages `Steady`, `Flashing_Slowly`, `Flashing_Quickly` states)

上下文 中状态1的列表项不为空。在此情况下，状态1的列表项包含两个列表，每个正交区域各一个。`ConcreteAndStateClass` 颜色 管理三个状态——红色、黄色和绿色。`ConcreteAndStateClass` 速率 也管理三个状态——常亮、闪烁_缓慢和闪烁_快速。因此列表如下所示：

- 上下文 状态列表：
  - 状态0 （关闭） – 空
  - 状态1 （开启） 列表：
    - ○ 颜色 （管理红色、黄色、绿色 状态）
    - ○ `Rate` （管理常亮、闪烁_缓慢、闪烁_快速 状态）

You'll notice that this gives us lists of lists of states. This is often implemented with arrays because the sizes of these lists are known at design time. If you need to further nest AND-states, you can add additional levels of lists, with appropriate behavior at each level.

5.8.4.4 ConcreteAndStateClass

This represents one of the AND-state classes. The design will contain one such class, with its own implementation of its event receptors, for each AND-state. In the basic pattern, each and-state class will itself use the Multiple Event Receptor Pattern, but other approaches are possible.

5.8.4.5 ConcreteAndStateType

Each AND-state has its unique set of nested states; this type is an enumerated list of them. One such type will exist for each `ConcreteAndStateClass`. This class is not visible to the clients of the `Context`.

5.8.4.6 Context

The `Context` is the ultimate owner of the state machine. It has the variables and functions that are manipulated by the `ConcreteAndStateClasses`. This class presents a set of event receptors to its clients (alternatively, the Single Event Receptor Pattern or other state pattern can be mixed in here as well). The basic rules of state change are enforced here:

- If the new state being entered has no nested AND-states, it executes normally as in the Multiple Event Receptor Pattern.
- If the new state being entered has nested AND-states, then its entry action is performed prior to the entry actions of the entered nested AND-states.
- If the composite state having AND-states is unchanged (that is, the event transition is local within the current composite state), then the event is delegated to its set of AND-state classes for handling, including the execution of the actions.
- If the composite state is left, then the exit actions of the composite state execute after the exit actions for each of its nested AND-states.

5.8.5 Consequences

This pattern provides extensible mechanisms for managing state machines that contain AND-states. Each class is relatively simple because of the delegation of responsibilities for state management – the `Context` manages the high level states, and the `ConcreteAndStateClasses` manage the nested AND-states.

On the other hand, this pattern does require the management of lists, of lists which will normally be managed as lists of pointers to lists of pointers. This level of indirection, while powerful, can be the source of obscure mistakes if not carefully managed.

5.8.6 Implementation Strategies

As mentioned in the previous section, the common implementation of the lists is as arrays of pointers to arrays of pointers.

你会注意到，这为我们提供了状态的列表的列表。这通常用数组来实现，因为这些列表的大小在设计时是已知的。如果你需要进一步嵌套AND状态，可以添加额外的列表层级，并在每一层实现相应的行为。

5.8.4.4 ConcreteAndStateClass

这代表其中一个与状态类。设计中将为每个与状态包含一个这样的类，并带有其自身的事件接收器实现。在基本模式中，每个与状态类本身将使用多事件接收器模式，但其他方法也是可能的。

5.8.4.5 ConcreteAndStateType

每个与状态都有其独特的嵌套状态集合；此类型是这些状态的枚举列表。每个 `ConcreteAndStateClass` 都将存在一个这样的类型。此类对 上下文 的客户端不可见。

5.8.4.6 上下文

上下文 是状态机的最终拥有者。它拥有由具体AND状态类所操作的变量和函数。该类向其客户端提供一组事件接收器（或者，也可以在此混合使用单事件接收器模式或其他状态模式）。状态变化的基本规则在此处强制执行：

- 如果正在进入的新状态没有嵌套AND状态，则其按多事件接收器模式中的方式正常执行。
- 如果正在进入的新状态具有嵌套AND状态，则其进入动作在进入的嵌套AND状态的进入动作之前执行。
- 如果具有AND状态的复合状态未发生变化（即事件迁移在当前复合状态内部是局部的），则事件被委托给其AND状态类集合进行处理，包括执行动作。
- 如果离开复合状态，则复合状态的退出动作在其每个嵌套AND状态的退出动作之后执行。

5.8.5 结果

该模式为管理包含AND状态的状态机提供了可扩展的机制。由于职责委派用于状态管理，每个类都相对简单——上下文 管理高层状态，而具体AND状态类 管理嵌套AND状态。

另一方面，该模式确实需要管理列表，而这些列表的列表通常将作为指向指针列表的指针列表来管理。这种间接层级虽然强大，但如果不仔细管理，可能会成为难以察觉的错误的来源。

5.8.6 实现策略

如上一节所述，列表的常见实现是指针数组的指针数组。

5.8.7 Related Patterns

This pattern can be used in conjunction with the state pattern – in this case, each state class will be the `Context` for its AND-states.

5.8.8 Example

In the example shown in Figure 5-24, the `LightController` plays the part of the `Context` pattern role and the `ErrorStateClass` and `ModeStateClass` play the part of the `ConcreteAndStateClass` in the pattern. Supporting elements, such as enumerated types and the `StateActionSet` are also shown.

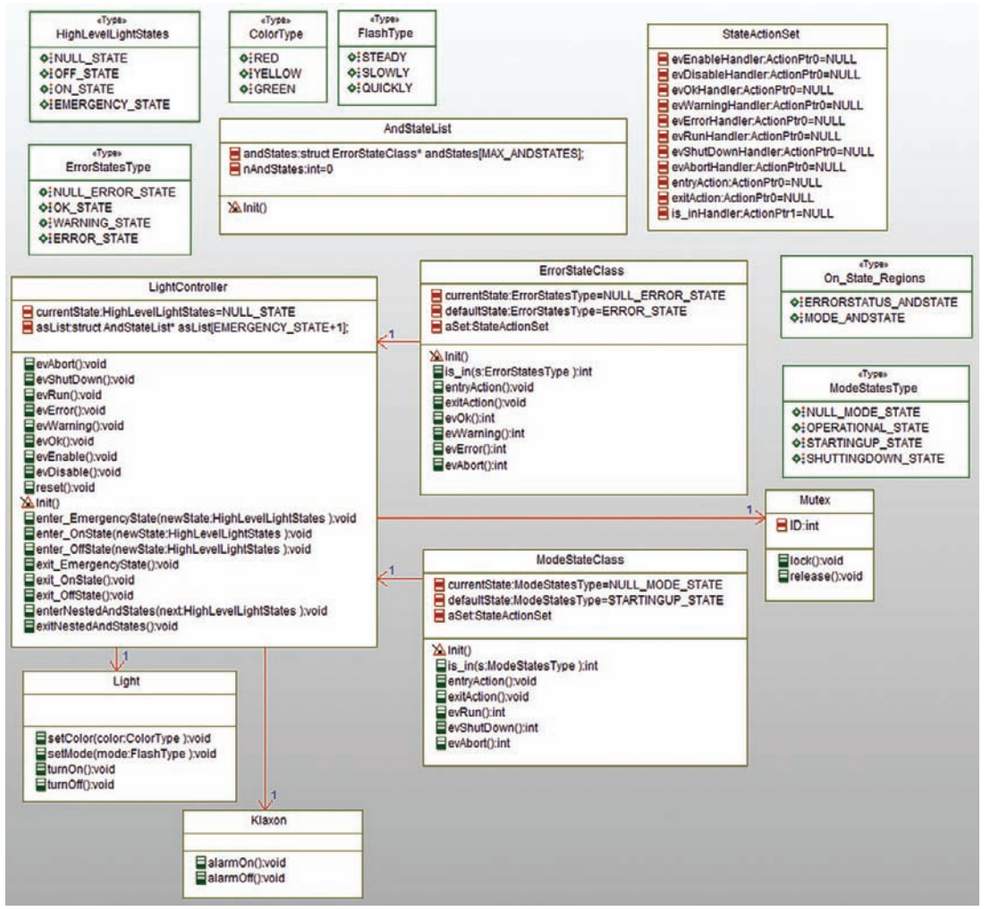


Figure 5-24: Example problem with Decomposed And-State Pattern

5.8.7 相关模式

此模式可与状态模式结合使用——在这种情况下，每个状态类都将成为其AND状态的上下文。

5.8.8 示例

在 图 5-24 所示的示例中，`LightController` 扮演了模式中上下文的角色，而 `ErrorStateClass` 和`ModeStateClass` 则扮演了模式中`ConcreteAndStateClass` 的角色。同时还展示了支持性元素，例如枚举类型和`StateActionSet` 。

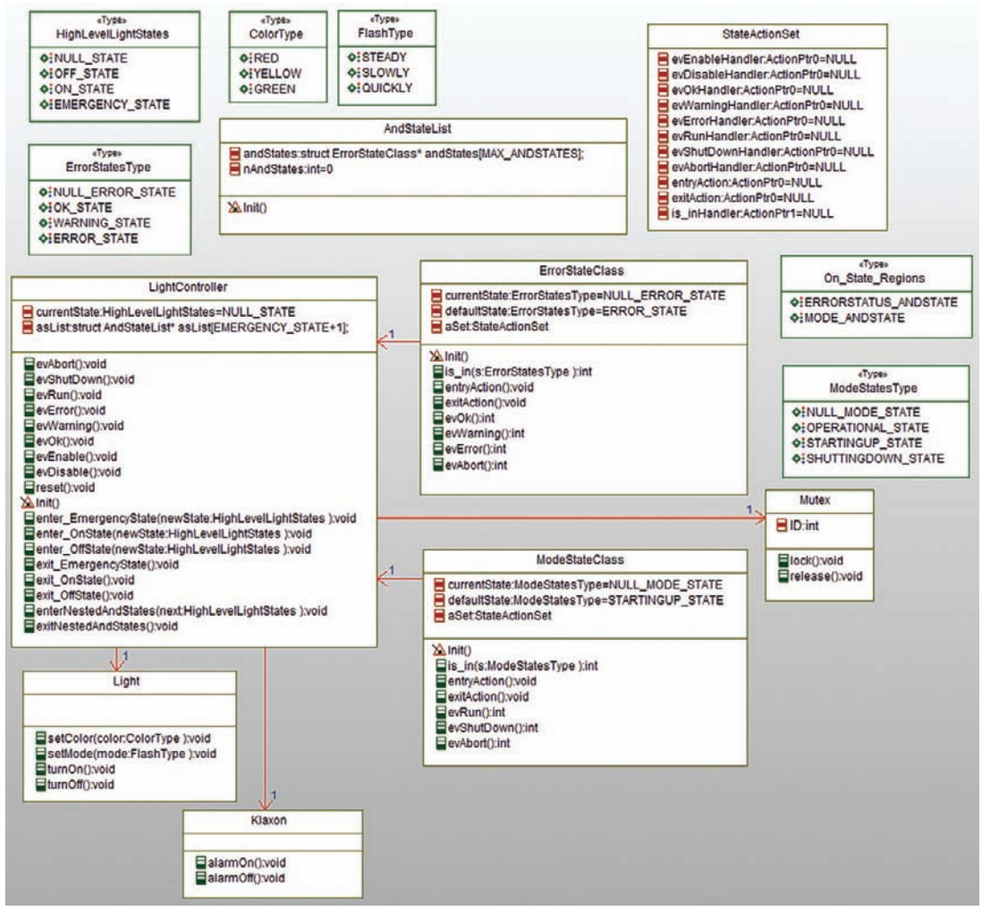


图5-24：分解AND状态模式的示例问题

The code for this example is shown in the next several code listings. The LightPkg.h file in Code Listing 5-28 holds the basic types used in the other files.

```
#ifndef LightPkg_H
#define LightPkg_H

struct AndStateList;
struct DecompBuilder;
struct DecompClient;
struct ErrorStateClass;
struct Klaxon;
struct Light;
struct LightController;
struct ModeStateClass;
struct Mutex;
struct StateActionSet;

typedef enum HighLevelLightStates {
  NULL_STATE,
  OFF_STATE,
  ON_STATE,
  EMERGENCY_STATE
} HighLevelLightStates;

typedef enum ColorType {
  RED,
  YELLOW,
  GREEN
} ColorType;

typedef enum FlashType {
  STEADY,
  SLOWLY,
  QUICKLY
} FlashType;

typedef enum ErrorStatesType {
  NULL_ERROR_STATE,
  OK_STATE,
  WARNING_STATE,
  ERROR_STATE
} ErrorStatesType;

typedef enum ModeStatesType {
  NULL_MODE_STATE,
  OPERATIONAL_STATE,
  STARTINGUP_STATE,
  SHUTTINGDOWN_STATE
} ModeStatesType;

typedef enum On_State_Regions {
  ERRORSTATUS_ANDSTATE,
  MODE_ANDSTATE
} On_State_Regions;
```


该示例的代码将在接下来的几个代码清单中展示。代码清单5-28中的LightPkg.h文件包含了其他文件中使用的基本类型。

```
#ifndef LightPkg_H #define LightPkg_H
struct AndStateList; struct
DecompBuilder; struct DecompClient;
struct ErrorStateClass; struct Klaxon;
struct Light; struct LightController;
struct ModeStateClass; struct Mutex;
struct StateActionSet; typedef enum
HighLevelLightStates { NULL_STATE,
OFF_STATE, ON_STATE, EMERGENCY
STATE_} HighLevelLightStates; typedef
enum ColorType { RED, YELLOW,
GREEN } ColorType; typedef enum
FlashType { STEADY, SLOWLY,
QUICKLY } FlashType; typedef enum
ErrorStatesType { NULL_ERROR_ST
ATE, OK_STATE, WARNING_STATE,
ERROR STATE_} ErrorStatesType;
typedef enum ModeStatesType { NULL_
MODE_STATE, OPERATIONAL_STATE,
STARTINGUP_STATE, SHUTTINGDOWN
STATE_} ModeStatesType; typedef e
num On_State_Regions { ERRORSTATUS_
ANDSTATE, MODE_ANDSTATE } On_
State_Regions;
```

```
#define MAX_ANDSTATES 2

/* This action pointer accepts the me object pointer and no other parameters. */
typedef int (*ActionPtr0)(void*);
/* This action pointer accepts the me object pointer and no other parameters. */
typedef int (*ActionPtr1)(void*, int);

#endif
```

Code Listing 5-28: LightPkg.h

The LightController class provides the state machine context for the example. It is shown in Code Listing 5-29 and Code Listing 5-30.

```
#ifndef LightController_H
#define LightController_H

#include "LightPkg.h"
#include "AndStateList.h"
#include "ErrorStateClass.h"
#include "ModeStateClass.h"

struct Klaxon;
struct Light;
struct Mutex;

typedef struct LightController LightController;
struct LightController {
  struct AndStateList* asList[EMERGENCY_STATE+1];
  HighLevelLightStates currentState;
  struct Klaxon* itsKlaxon;
  struct Light* itsLight;
  struct Mutex* itsMutex;
};

void LightController_evEnable(LightController* const me);
void LightController_evDisable(LightController* const me);
void LightController_reset(LightController* const me);
void LightController_evOk(LightController* const me);
void LightController_evWarning(LightController* const me);
void LightController_evError(LightController* const me);
void LightController_evRun(LightController* const me);
void LightController_evShutDown(LightController* const me);
void LightController_evAbort(LightController* const me);
void LightController_Init(LightController* const me);
void LightController_Cleanup(LightController* const me);

void LightController_enter_OffState(LightController* const me, HighLevelLightStates
newState);

void LightController_enter_OnState(LightController* const me, HighLevelLightStates
newState);
```



```
#define MAX_ANDSTATES 2

/* 此动作指针接受me对象指针，不接受其他参数。 */ typedef int (*ActionPtr0)(void*); /* 此动作
指针接受me对象指针，不接受其他参数。 */ typedef int (*ActionPtr1)(void*, int);

#endif
```

代码清单5-28：LightPkg.h

LightController类为示例提供了状态机上下文。其内容见代码清单5-29和代码清单5-30。

```
#ifndef LightController_H #
define LightController_H

#include "LightPkg.h" #include "
AndStateList.h" #include "
ErrorStateClass.h" #include "
ModeStateClass.h"

struct Klaxon;
struct Light;
struct Mutex;

typedef struct LightController LightController; struct
LightController { struct AndStateList* asList[EMERGENCY_
STATE+1];HighLevelLightStates currentState; struct
Klaxon* itsKlaxon; struct Light* itsLight; struct Mutex*
itsMutex; };

void LightController_evEnable(LightController* const me); void
LightController_evDisable(LightController* const me); void LightController_
reset(LightController* const me); void LightController_evOk(LightController*
const me); void LightController_evWarning(LightController* const me); void
LightController_evError(LightController* const me); void LightController_
evRun(LightController* const me); void LightController_关闭事件
(LightController* const me); void LightController_evAbort(LightController*
const me); void LightController_Init(LightController* const me); void
LightController_清理(LightController* const me);

void LightController_enter_OffState(LightController* const me, HighLevelLightStates newState);

void LightController_enter_OnState(LightController* const me, HighLevelLightStates newState);
```

```
void LightController_enter_EmergencyState(LightController* const me, HighLevelLight-
States newState);

void LightController_exit_OffState(LightController* const me);

void LightController_exit_OnState(LightController* const me);

void LightController_exit_EmergencyState(LightController* const me);

void LightController_enterNestedAndStates(LightController* const me, HighLevelLight-
States next);

void LightController_exitNestedAndStates(LightController* const me);

struct Klaxon* LightController_getItsKlaxon(const LightController* const me);

void LightController_setItsKlaxon(LightController* const me, struct Klaxon* p_Klaxon);

struct Light* LightController_getItsLight(const LightController* const me);

void LightController_setItsLight(LightController* const me, struct Light* p_Light);

struct Mutex* LightController_getItsMutex(const LightController* const me);

void LightController_setItsMutex(LightController* const me, struct Mutex* p_Mutex);
LightController * LightController_Create(void);

void LightController_Destroy(LightController* const me);

#endif
```

Code Listing 5-29: LightController.h

```
#include "LightController.h"
#include "Klaxon.h"
#include "Light.h"
#include "Mutex.h"

static void cleanUpRelations(LightController* const me);

void LightController_evEnable(LightController* const me) {
  HighLevelLightStates newState;

  Mutex_lock(me->itsMutex);
  switch (me->currentState) {
     case OFF_STATE:
         newState = ON_STATE;
         LightController_exit_OffState(me);

         /* transition to On state */
         Light_turnOn(me->itsLight);
         Light_setColor(me->itsLight, RED);
         Light_setMode(me->itsLight, QUICKLY);

         /* enter new state */
         LightController_enter_OnState(me, newState);
         me->currentState = newState;
```



```
void LightController_enter_EmergencyState(LightController* const me, HighLevelLight-States
newState); void LightController_exit_OffState(LightController* const me); void LightController_exit_
OnState(LightController* const me); void LightController_exit_EmergencyState(LightController* const
me); void LightController_enterNestedAndStates(LightController* const me, HighLevelLight-States
next); void LightController_exitNestedAndStates(LightController* const me); struct Klaxon*
LightController_getItsKlaxon(const LightController* const me); void LightController_
setItsKlaxon(LightController* const me, struct Klaxon* p_Klaxon); struct Light* LightController_
getItsLight(const LightController* const me); void LightController_setItsLight(LightController* const
me, struct Light* p_Light); struct Mutex* LightController_getItsMutex(const LightController* const
me); void LightController_setItsMutex(LightController* const me, struct Mutex* p_Mutex);
LightController * LightController_Create(void); void LightController_Destroy(LightController* const
me); #endif
```

代码清单 5-29：LightController.h

```
#include "LightController.h" #
include "Klaxon.h" #include "
Light.h" #include "Mutex.h"

static void cleanUpRelations(LightController* const me);

void LightController_evEnable(LightController* const me) {
HighLevelLightStates newState;

  互斥锁_加锁(me->itsMutex); switch (me->
  currentState) { case OFF_STATE: newState = ON_
  STATE; LightController_exit_OffState(me);


        /* 迁移到On状态 */ Light_turnOn(me->
        itsLight); Light_setColor(me->itsLight, 红色);
        Light_setMode(me->itsLight, QUICKLY);


        /* 进入新状态 */ LightController_enter_OnState(me,
        newState); me->currentState = newState;
```

```
        break;
    case ON_STATE:
    case EMERGENCY_STATE:
        break;
    };
  Mutex_release(me->itsMutex);
}
void LightController_evDisable(LightController* const me) {
  HighLevelLightStates newState;

  Mutex_lock(me->itsMutex);
  switch (me->currentState) {
     case ON_STATE:
         newState = OFF_STATE;
           /* exit nested states */
               LightController_exit_OnState(me);

               /* transition to OFF state */
               Light_turnOff(me->itsLight);

        /* enter new state */
        LightController_enter_OffState(me, newState);
        me->currentState = newState;
        break;
  case OFF_STATE:
  case EMERGENCY_STATE:
     break;
     };
  Mutex_release(me->itsMutex);
}
void LightController_reset(LightController* const me) {
  HighLevelLightStates newState;

  Mutex_lock(me->itsMutex);
  switch (me->currentState) {
    case ON_STATE:
     /* exit and-states, then outer state */
    newState = ON_STATE;
         LightController_exit_OnState(me);

         /* note no transition actions */
         /* enter outer state, then and-states */
         LightController_enter_OnState(me, newState);
         me->currentState = newState;
         break;

  case OFF_STATE:
  case EMERGENCY_STATE:
         break;
```


break; case On状态:_case EMERGENCY_状态: break; }; 互斥锁_释放(me->itsMutex); } void LightController_ev Disable(LightController* const me)

{

HighLevelLightStates newState; Mutex_lock(me->itsMutex); switch (me->currentState) { case ON STATE:_newState = OFF_ STATE; /* 退出嵌套状态 */ LightController_exit_OnState(me); /* 迁移到OFF状态 */ Light_turnOff(me->itsLight); /* 进入新状态 */ LightController_enter_OffState(me, newState); me->currentState = newState; break; case OFF STATE:_case EMERGENCY STATE:_ break; }; Mutex_release(me->itsMutex); } void LightController_ reset(LightController* const me) { HighLevelLightStates newState; Mutex_lock(me->itsMutex); switch (me->currentState) { case ON STATE:_/* 退出与状态，然后退出外层状态 */ newState = ON_STATE; LightController_exit_OnState(me); /* 注意没有转换动作 */ /* 进入外层状态，然后进入与状态 */ LightController_enter_ OnState(me, newState); me->currentState = newState; break; case OFF_STATE: case EMERGENCY STATE:_break;

```
    };
  Mutex_release(me->itsMutex);
}

void LightController_evOk(LightController* const me) {
  ErrorStateClass* sPtr;
  ActionPtr0 aPtr;
  HighLevelLightStates newState;

  Mutex_lock(me->itsMutex);
  switch (me->currentState) {
      case OFF_STATE:
        break;
      case ON_STATE:
      /* note we're not exiting On state */
      /* so just delegate to and-states */
      newState = me->currentState;
      if (me->asList[newState]) {
        sPtr = me->asList[newState]->andStates[ERRORSTATUS_ANDSTATE];
        if (sPtr) {
            aPtr = sPtr->aSet.evOkHandler;
            if (aPtr)
                aPtr(sPtr);
            };
        sPtr = me->asList[newState]->andStates[MODE_ANDSTATE];
        if (sPtr) {
            aPtr = sPtr->aSet.evOkHandler;
            if (aPtr)
                aPtr(sPtr);
            };
       };
     break;
  case EMERGENCY_STATE:
     break;
  };
  Mutex_release(me->itsMutex);
}

void LightController_evWarning(LightController* const me) {
  ErrorStateClass* sPtr;
  ActionPtr0 aPtr;
  HighLevelLightStates newState;

  Mutex_lock(me->itsMutex);
  switch (me->currentState) {
    case OFF_STATE:
       break;
    case ON_STATE:
        /* note we're not exiting On state */
        /* so just delegate to and-states */
        newState = me->currentState;
```



```
}; Mutex_release(me->itsMutex); } void LightController_evOk(LightController*
const me) { ErrorStateClass* sPtr; ActionPtr0 aPtr; HighLevelLightStates newState;
Mutex_lock(me->itsMutex); switch (me->currentState) { case OFF STATE:_break;
case ON STATE:  /* 注意我们不退出On状态 */ /* 因此只需委托给与状态 */ newState =
me->currentState; if (me->asList[newState]) { sPtr = me->asList[newState]->
andStates[ERRORSTATUS_ANDSTATE]; if (sPtr) { aPtr = sPtr->aSet.evOkHandler; if
(aPtr) aPtr(sPtr); }; sPtr = me->asList[newState]->andStates[MODE_ANDSTATE]; if
(sPtr) { aPtr = sPtr->aSet.evOkHandler; if (aPtr) aPtr(sPtr); }; }; break; case
EMERGENCY STATE:_break; }; Mutex_release(me->itsMutex); } void
LightController_evWarning(LightController* const me) { ErrorStateClass* sPtr;
ActionPtr0 aPtr; HighLevelLightStates newState; Mutex_lock(me->itsMutex); s
witch (me->currentState) { case OFF STATE:_break; case ON_STATE: /* 注意我们不退
出On状态 */ /* 因此只需委托给与状态 */ newState = me->currentState;
```

```
        if (me->asList[newState]) {
          sPtr = me->asList[newState]->andStates[ERRORSTATUS_ANDSTATE];
          if (sPtr) {
            aPtr = sPtr->aSet.evWarningHandler;
            if (aPtr)
                aPtr(sPtr);
            };
         sPtr = me->asList[newState]->andStates[MODE_ANDSTATE];
         if (sPtr) {
          aPtr = sPtr->aSet.evWarningHandler;
            if (aPtr)
                aPtr(sPtr);
            };
        };
      break;
  case EMERGENCY_STATE:
      break;
  };
  Mutex_release(me->itsMutex);
  /*#]*/
}

void LightController_evError(LightController* const me) {
  ErrorStateClass* sPtr;
  ActionPtr0 aPtr;
  HighLevelLightStates newState;
  Mutex_lock(me->itsMutex);
  switch (me->currentState) {
    case OFF_STATE:
      break;
    case ON_STATE:
        /* note we're not exiting On state */
        /* so just delegate to AND-states */
        newState = me->currentState;
        if (me->asList[newState]) {
          sPtr = me->asList[newState]->andStates[ERRORSTATUS_ANDSTATE];
          if (sPtr) {
            aPtr = sPtr->aSet.evErrorHandler;
            if (aPtr)
                aPtr(sPtr);
            };
          sPtr = me->asList[newState]->andStates[MODE_ANDSTATE];
          if (sPtr) {
          aPtr = sPtr->aSet.evErrorHandler;
            if (aPtr)
                aPtr(sPtr);
            };
          };
    break;
case EMERGENCY_STATE:
```


if (me->asList[newState]) { sPtr = me->asList[newState]->andStates[ERRORSTATUS_ANDSTATE]; if (sPtr) { aPtr = sPtr->aSet.evWarningHandler; if (aPtr) aPtr(sPtr); }; sPtr = me->asList[newState]->andStates[MODE_ANDSTATE]; if (sPtr) { aPtr = sPtr->aSet.evWarningHandler; if (aPtr) aPtr(sPtr); }; }; break; case EMERGENCY STATE:_break; }; Mutex_release(me->itsMutex); /*#]*/ } void LightController_evError(LightController* const me) { ErrorStateClass* sPtr; ActionPtr0 aPtr; HighLevelLightStates newState; Mutex_lock(me->itsMutex); switch (me->currentState) { case OFF STATE:_break; case ON STATE: /* 注意我们未退出On状态 */ /* 因此仅委托给 AND 状态 */ newState = me->currentState; if (me->asList[newState]) { sPtr = me->asList[newState]->andStates[ERRORSTATUS_ANDSTATE]; if (sPtr) { aPtr = sPtr->aSet.evErrorHandler; if (aPtr) aPtr(sPtr); }; sPtr = me->asList[newState]->andStates[MODE_ANDSTATE]; if (sPtr) { aPtr = sPtr->aSet.evErrorHandler; if (aPtr) aPtr(sPtr); }; }; break; case EMERGENCY STATE:_

```
    break;
  };
  Mutex_release(me->itsMutex);
}
void LightController_evRun(LightController* const me) {
  ErrorStateClass* sPtr;
  ActionPtr0 aPtr;
  HighLevelLightStates newState;

  Mutex_lock(me->itsMutex);
  switch (me->currentState) {
    case OFF_STATE:
      break;
    case ON_STATE:
        /* note we're not exiting On state */
        /* so just delegate to and-states */
        newState = me->currentState;
        if (me->asList[newState]) {
          sPtr = me->asList[newState]->andStates[ERRORSTATUS_ANDSTATE];
          if (sPtr) {
            aPtr = sPtr->aSet.evRunHandler;
            if (aPtr)
                aPtr(sPtr);
            };
          sPtr = me->asList[newState]->andStates[MODE_ANDSTATE];
          if (sPtr) {
           aPtr = sPtr->aSet.evRunHandler;
             if (aPtr)
                 aPtr(sPtr);
             };
          };
      break;
  case EMERGENCY_STATE:
      break;
    };
  Mutex_release(me->itsMutex);
}
void LightController_evShutDown(LightController* const me) {
  ErrorStateClass* sPtr;
  ActionPtr0 aPtr;
  HighLevelLightStates newState;

  Mutex_lock(me->itsMutex);
  switch (me->currentState) {
    case OFF_STATE:
      break;
    case ON_STATE:
      /* note we're not exiting On state */
      /* so just delegate to and-states */
      newState = me->currentState;
```


break; }; Mutex_release(me->itsMutex); } void LightController_evRun(LightController* const me) { ErrorStateClass* sPtr; ActionPtr0 aPtr; HighLevelLightStates newState; Mutex_lock(me->itsMutex); switch (me->currentState) { case OFF STATE:_break; case ON_STATE: /* 注意我们未退出On状态 */ /* 因此仅委托给AND状态 */ newState = me->currentState; if (me->asList[newState]) { sPtr = me->asList[newState]->andStates[ERRORSTATUS_ANDSTATE]; if (sPtr) { aPtr = sPtr->aSet.evRunHandler; if (aPtr) aPtr(sPtr); }; sPtr = me->asList[newState]->andStates[MODE_ANDSTATE]; if (sPtr) { aPtr = sPtr->aSet.evRunHandler; if (aPtr) aPtr(sPtr); }; }; break; case EMERGENCY_STATE: break; }; Mutex_release(me->itsMutex); } void LightController_evShutDown(LightController* const me) { ErrorStateClass* sPtr; ActionPtr0 aPtr; HighLevelLightStates newState; Mutex_lock(me->itsMutex); switch (me->currentState) { case OFF_STATE: break; case ON STATE: /* 注意我们未退出On状态 */ /* 因此仅委托给AND状态 */ newState = me->currentState;

```
        if (me->asList[newState]) {
          sPtr = me->asList[newState]->andStates[ERRORSTATUS_ANDSTATE];
          if (sPtr) {
              aPtr = sPtr->aSet.evShutDownHandler;
              if (aPtr)
                  aPtr(sPtr);
              };
          sPtr = me->asList[newState]->andStates[MODE_ANDSTATE];
          if (sPtr) {
           aPtr = sPtr->aSet.evShutDownHandler;
              if (aPtr)
                  aPtr(sPtr);
              };
          };
      break;
    case EMERGENCY_STATE:
          break;
      };
    Mutex_release(me->itsMutex);
}

void LightController_evAbort(LightController* const me) {
  ErrorStateClass* sPtr;
  ActionPtr1 is_inPtr;
  HighLevelLightStates newState;
  int precond1=0, precond2=0;
  Mutex_lock(me->itsMutex);
  switch (me->currentState) {
    case ON_STATE:
      /* are the preconditions true: */
      /* ie. Error and Operational states? */
      if (me->asList[me->currentState]) {
       sPtr = me->asList[me->currentState]->andStates[ERRORSTATUS_ANDSTATE];
       if (sPtr) {
           is_inPtr = sPtr->aSet.is_inHandler;
           if (is_inPtr)
               precond1 = is_inPtr(sPtr, ERROR_STATE);
           };
       sPtr = me->asList[me->currentState]->andStates[MODE_ANDSTATE];
       if (sPtr) {
           is_inPtr = sPtr->aSet.is_inHandler;
           if (is_inPtr)
               precond2 = is_inPtr(sPtr, OPERATIONAL_STATE);
           };
         };
    if (precond1 && precond2) {
     newState = EMERGENCY_STATE;
     /* exit nested states */
          LightController_exit_OnState(me);
```


if (me->asList[newState]) { sPtr = me->asList[newState]->andStates[ERRORSTATUS_ANDSTATE]; if (sPtr) { aPtr = sPtr->aSet.evShutDownHandler; if (aPtr) aPtr(sPtr); }; sPtr = me->asList[newState]->andStates[MODE_ANDSTATE]; if (sPtr) { aPtr = sPtr->aSet.evShutDownHandler; if (aPtr) aPtr(sPtr); }; }; break; case EMERGENCY STATE:_break; }; Mutex_release(me->itsMutex); } void LightController_evAbort(LightController* const me) { ErrorStateClass* sPtr; ActionPtr1 is_inPtr; HighLevelLightStates newState; int precond1=0, precond2=0 ;Mutex_lock(me->itsMutex); switch (me->currentState) { case ON STATE:_/* 前置条件是否为真： */ /* 即错误状态和运行状态? */ if (me->asList[me->currentState]) { sPtr = me->asList[me->currentState]->andStates[ERRORSTATUS_ANDSTATE]; if (sPtr) { is_inPtr = sPtr->aSet.is_inHandler; if (is_inPtr) precond1 = is_inPtr(sPtr, ERROR_STATE); }; sPtr = me->asList[me->currentState]->andStates[MODE_ANDSTATE]; if (sPtr) { is_inPtr = sPtr->aSet.is_inHandler; if (is_inPtr) precond2 = is_inPtr(sPtr, OPERATIONAL_STATE); }; }; if (precond1 && precond2) { newState = EMERGENCY_STATE; /* 退出嵌套状态 */ LightController_exit_OnState(me);

```
            /* transition to OFF state */
            Light_setColor(me->itsLight, RED);
            Light_setMode(me->itsLight, QUICKLY);
            Klaxon_alarmOn(me->itsKlaxon);

            /* enter new state */
            LightController_enter_EmergencyState(me, newState);
            me->currentState = newState;
          };
      break;
    case OFF_STATE:
    case EMERGENCY_STATE:
      break;
      };
    Mutex_release(me->itsMutex);
}

void LightController_Init(LightController* const me) {
  me->currentState = NULL_STATE;
  me->itsKlaxon = NULL;
  me->itsLight = NULL;
  me->itsMutex = NULL;
  int j;
  HighLevelLightStates st;
  ErrorStateClass* andStatePtr = NULL;
  for (st=NULL_STATE;st<=EMERGENCY_STATE; st++)
    me->asList[st] = NULL;

  /* create all and-state lists and states */
  /* In this case, there is only one, the ON_STATE */
  /* which has two nested orthogonal regions */
  me->asList[ON_STATE] = AndStateList_Create();
  me->asList[ON_STATE]->nAndStates = 2;
  andStatePtr = ErrorStateClass_Create();
  if (andStatePtr) {
    me->asList[ON_STATE]->andStates[ERRORSTATUS_ANDSTATE] = andStatePtr;
    andStatePtr->itsLightController = me;
    };
  andStatePtr = (ErrorStateClass*)ModeStateClass_Create();
  if (andStatePtr) {
    me->asList[ON_STATE]->andStates[MODE_ANDSTATE] = andStatePtr;
    andStatePtr->itsLightController = me;
    };

  /* enter initial state */
  me->currentState = OFF_STATE;
  LightController_enter_OffState(me, OFF_STATE);
  /* any nested and-states? */
  if (me->asList[me->currentState])
    /* enter the default state in each and-state */
```


/* 迁移到关闭状态 */ Light_setColor(me->itsLight, RED); Light_setMode(me->itsLight, QUICKLY); Klaxon_alarmOn(me->itsKlaxon); /* 进入新状态 */ LightController_enter_EmergencyState(me, newState); me->currentState = newState; }; break; case OFF STATE:_case EMERGENCY STATE:_break; }; Mutex_release(me->itsMutex); } void LightController_Init(LightController* const me) { me->currentState = NULL_STATE; me->itsKlaxon = NULL; me->itsLight = NULL; me->itsMutex = NULL; int j; HighLevelLightStates st; ErrorStateClass* andStatePtr = NULL; for (st =NULL_STATE; st<=EMERGENCY_STATE; st++)me->asList[st] = NULL; /* 创建所有与状态列表和状态 */ /* 在本例中，只有一个，即ON_STATE */ /* 它有两个嵌套正交区域 */ me->asList[ON_STATE] = AndStateList_Create(); me->asList[ON_STATE]->nAndStates = 2; andStatePtr = ErrorStateClass_Create(); if (andStatePtr) { me->asList[ON_STATE]->andStates[ERRORSTATUS_ANDSTATE] = andStatePtr; andStatePtr->itsLightController = me; }; andStatePtr = (ErrorStateClass*)ModeStateClass_Create(); if (andStatePtr) { me->asList[ON_STATE]->andStates[MODE_ANDSTATE] = andStatePtr; andStatePtr->itsLightController = me; }; /* 进入初始状态 */ me->currentState = OFF_STATE; LightController_enter_OffState(me, OFF_STATE); /* 是否有嵌套与状态? */ if (me->asList[me->currentState]) /* 进入每个与状态中的默认状态 */

```
    for (j=0; j<me->asList[me->currentState]->nAndStates; j++)
        me->asList[me->currentState]->andStates[j]->aSet.entryAction(me->asList[me-
>currentState]->andStates[j]);
}

void LightController_Cleanup(LightController* const me) {
  cleanUpRelations(me);
}

void LightController_enter_OffState(LightController* const me, HighLevelLightStates
newState) {
  LightController_enterNestedAndStates(me, newState);
}

void LightController_enter_OnState(LightController* const me, HighLevelLightStates
newState) {
  LightController_enterNestedAndStates(me, newState);
}

void LightController_enter_EmergencyState(LightController* const me, HighLevelLight-
States newState) {
  LightController_enterNestedAndStates(me, newState);
}

void LightController_exit_OffState(LightController* const me) {
  LightController_exitNestedAndStates(me);
}

void LightController_exit_OnState(LightController* const me) {
  LightController_exitNestedAndStates(me);
}

void LightController_exit_EmergencyState(LightController* const me) {
  LightController_exitNestedAndStates(me);
}

void LightController_enterNestedAndStates(LightController* const me, HighLevelLight-
States next) {
  ErrorStateClass* sPtr;
  ActionPtr0 aPtr;
  int j;

  if (me->asList[next]) {
    /* and-states to enter? */
    for (j=0; j<me->asList[next]->nAndStates; j++ ) {
        sPtr = me->asList[next]->andStates[j];
        if (sPtr) {
            aPtr = sPtr->aSet.entryAction;
            if (aPtr)
                aPtr(sPtr);
            sPtr->currentState = sPtr->defaultState;
            };
        };
    };
}
```


for (j=0; j<me->asList[me->currentState]->nAndStates; j++)me->asList[me->currentState]->andStates[j]->aSet.entryAction(me->asList[me->currentState]->andStates[j]); } void LightController_Cleanup(LightController* const me) { cleanUpRelations(me); } void LightController_enter_OffState(LightController* const me, HighLevelLightStates newState) { LightController_enterNestedAndStates(me, newState); } void LightController_enter_OnState(LightController* const me, HighLevelLightStates newState) { LightController_enterNestedAndStates(me, newState); } void LightController_enter_EmergencyState(LightController* const me, HighLevelLight- States newState) { LightController_enterNestedAndStates(me, newState); } void LightController_exit_OffState(LightController* const me) { LightController_exitNestedAndStates(me); } void LightController_exit_OnState(LightController* const me) { LightController_exitNestedAndStates(me); } void LightController_exit_EmergencyState(LightController* const me) { LightController_exitNestedAndStates(me); } void LightController_enterNestedAndStates(LightController* const me, HighLevelLight- States next) { ErrorStateClass* sPtr; ActionPtr0 aPtr; int j; if (me->asList[next]) { /* 要进入的与状态? */ for (j=0; j<me->asList[next]->nAndStates; j++) { sPtr = me->asList[next]->andStates[j]; if (sPtr) { aPtr = sPtr->aSet.entryAction; if (aPtr) aPtr(sPtr); sPtr->currentState = sPtr->defaultState; }; }; }; }

```
void LightController_exitNestedAndStates(LightController* const me) {
  ErrorStateClass* sPtr;
  ActionPtr0 aPtr;
  int j;

  if (me->asList[me->currentState]) { /* any nested states to exit? */
     for (j=0; j<me->asList[me->currentState]->nAndStates; j++ ) {
         sPtr = me->asList[me->currentState]->andStates[j];
         if (sPtr) {
             aPtr = sPtr->aSet.exitAction;
             if (aPtr)
                 aPtr(sPtr);
             sPtr->currentState = NULL_ERROR_STATE;
             }; // if sPtr
         }; // for
     }; // if asList
}

struct Klaxon* LightController_getItsKlaxon(const LightController* const me) {
  return (struct Klaxon*)me->itsKlaxon;
}

void LightController_setItsKlaxon(LightController* const me, struct Klaxon* p_Klaxon) {
  me->itsKlaxon = p_Klaxon;
}

struct Light* LightController_getItsLight(const LightController* const me) {
  return (struct Light*)me->itsLight;
}

void LightController_setItsLight(LightController* const me, struct Light* p_Light) {
  me->itsLight = p_Light;
}

struct Mutex* LightController_getItsMutex(const LightController* const me) {
  return (struct Mutex*)me->itsMutex;
}

void LightController_setItsMutex(LightController* const me, struct Mutex* p_Mutex) {
  me->itsMutex = p_Mutex;
}

LightController * LightController_Create(void) {
  LightController* me = (LightController *)
malloc(sizeof(LightController));
  if(me!=NULL)
      LightController_Init(me);
  return me;
}

void LightController_Destroy(LightController* const me) {
  if(me!=NULL)
      LightController_Cleanup(me);
  free(me);
}
```



```
void LightController_exitNestedAndStates(LightController* const me) { ErrorStateClass* sPtr;
ActionPtr0 aPtr; int j; if (me->asList[me->currentState]) { /* 是否有要退出的嵌套状态?  */ for (j=0; j<me->
asList[me->currentState]->nAndStates; j++ ) { sPtr = me->asList[me->currentState]->andStates[j]; if
(sPtr) { aPtr = sPtr->aSet.exitAction; if (aPtr) aPtr(sPtr); sPtr->currentState = NULL_ERROR_STATE; }; // if
sPtr }; // for }; // if asList } struct Klaxon* LightController_getItsKlaxon(const LightController* const me)
{ return (struct Klaxon*)me->itsKlaxon; } void LightController_setItsKlaxon(LightController* const me,
struct Klaxon* p_Klaxon) { me->itsKlaxon = p_Klaxon; } struct Light* LightController_getItsLight(const
LightController* const me) { return (struct Light*)me->itsLight; } void LightController_
setItsLight(LightController* const me, struct Light* p_Light) { me->itsLight = p_Light; } struct Mutex*
LightController_getItsMutex(const LightController* const me) { return (struct Mutex*)me->itsMutex; }
void LightController_setItsMutex(LightController* const me, struct Mutex* p_Mutex) { me->itsMutex =
p_Mutex; } LightController * LightController_Create(void) { LightController* me = (LightController *)
malloc(sizeof(LightController)); if(me!=NULL)LightController_Init(me); return me; } void
LightController_Destroy(LightController* const me) { if(me!=NULL)LightController_Cleanup(me);
free(me); }
```

```
static void cleanUpRelations(LightController* const me) {
  if(me->itsKlaxon != NULL)
     me->itsKlaxon = NULL;
  if(me->itsLight != NULL)
     me->itsLight = NULL;
  if(me->itsMutex != NULL)
     me->itsMutex = NULL;
}
```

Code Listing 5-30: LightController.c

The `AndStateList` class provides a level of indirection to a `ConcreteAndState` class that implements the state machine for all its nested states. It primarily provides an array of pointers to `ConcreteAndState` classes. Its code appears in Code Listing 5-31 and Code Listing 5-32.

```
#ifndef AndStateList_H
#define AndStateList_H

#include "LightPkg.h"

struct ErrorStateClass;

typedef struct AndStateList AndStateList;
struct AndStateList {
  struct ErrorStateClass* andStates[MAX_ANDSTATES];
  int nAndStates;
};

void AndStateList_Init(AndStateList* const me);
void AndStateList_Cleanup(AndStateList* const me);
AndStateList * AndStateList_Create(void);
void AndStateList_Destroy(AndStateList* const me);

#endif
```

Code Listing 5-31: AndStateList.h

```
#include "AndStateList.h"
#include "ErrorStateClass.h"

void AndStateList_Init(AndStateList* const me) {
  me->nAndStates = 0;
  int j;
  for (j=0; j<MAX_ANDSTATES; j++)
    me->andStates[j] = NULL;
}

void AndStateList_Cleanup(AndStateList* const me) {
}

AndStateList * AndStateList_Create(void) {
```


static void cleanUpRelations(LightController* const me) { if(me-> itsKlaxon != 空) me->itsKlaxon = 空; if(me->itsLight != 空) me-> itsLight = 空; if(me->itsMutex != 空) me->itsMutex = 空; }

代码清单5-30：LightController.c

AndStateList 类为ConcreteAndState 类提供了一个间接层级，后者为其所有嵌套状态实现了状态机。它主要提供了一个指向ConcreteAndState 类的指针数组。其代码见代码清单5-31和代码清单5-32。

#ifndef AndStateList_H #define AndStateList_H #include " LightPkg.h" struct ErrorStateClass; typedef struct AndStateList AndStateList; struct AndStateList { struct ErrorStateClass* andStates[MAX_ANDSTATES]; int nAndStates; }; void AndStateList_Init(AndStateList* const me); void AndStateList_Cleanup(AndStateList* const me); AndStateList * AndStateList_Create(void); void AndStateList_ Destroy(AndStateList* const me); #endif

代码清单5-31：AndStateList.h

#include "AndStateList.h" #include "ErrorStateClass.h" void AndStateList_Init(AndStateList* const me) { me-> nAndStates = 0; int j; for (j=0; j<MAX_ANDSTATES; j++) me->andStates[j] = NULL; } void AndStateList_ Cleanup(AndStateList* const me) { } AndStateList * AndStateList_Create(void) {

```
  AndStateList* me = (AndStateList *)
malloc(sizeof(AndStateList));
  if(me!=NULL)
      AndStateList_Init(me);
  return me;
}

void AndStateList_Destroy(AndStateList* const me) {
  if(me!=NULL)
      AndStateList_Cleanup(me);
  free(me);
}
```

Code Listing 5-32: AndStateList.c

Similarly, the `StateActionSet` is a collection of function pointers, one per event handler plus a couple of the entry and exit actions and the `is_in()` function (which identifies if the state class is in the specified state). The reason I made it a class is to provide the initialization behavior (setting all the pointers to `NULL`).

```
#ifndef StateActionSet_H
#define StateActionSet_H

#include "LightPkg.h"

typedef struct StateActionSet StateActionSet;
struct StateActionSet {
  ActionPtr0 entryAction;
  ActionPtr0 evAbortHandler;
  ActionPtr0 evDisableHandler;
  ActionPtr0 evEnableHandler;
  ActionPtr0 evErrorHandler;
  ActionPtr0 evOkHandler;
  ActionPtr0 evRunHandler;
  ActionPtr0 evShutDownHandler;
  ActionPtr0 evWarningHandler;
  ActionPtr0 exitAction;
  ActionPtr1 is_inHandler;
};

void StateActionSet_Init(StateActionSet* const me);
void StateActionSet_Cleanup(StateActionSet* const me);
StateActionSet * StateActionSet_Create(void);
void StateActionSet_Destroy(StateActionSet* const me);

#endif
```

Code Listing 33: StateActionSet.h

AndStateList* me = (AndStateList *)
malloc(sizeof(AndStateList)); if(me!=NULL)AndStateList_
Init(me); return me; } void AndStateList_
Destroy(AndStateList* const me) { if(me!=NULL)
AndStateList_Cleanup(me); free(me); }

代码清单5-32：AndStateList.c

类似地，`StateActionSet`是一组函数指针的集合，每个事件处理器对应一个，再加上几个进入和退出动作以及`is_in()`函数（该函数用于判断状态类是否处于指定状态）。我将其设计为类的原因是为了提供初始化行为（将所有指针设置为空）。

#ifndef StateActionSet_H #define StateActionSet_H #include "
LightPkg.h" typedef struct StateActionSet StateActionSet; struct
StateActionSet { ActionPtr0 entryAction; ActionPtr0
evAbortHandler; ActionPtr0 evDisableHandler; ActionPtr0
evEnableHandler; ActionPtr0 evErrorHandler; ActionPtr0
evOkHandler; ActionPtr0 evRunHandler; ActionPtr0
evShutDownHandler; ActionPtr0 evWarningHandler;
ActionPtr0 exitAction; ActionPtr1 is_inHandler; }; void
StateActionSet_Init(StateActionSet* const me); void
StateActionSet_Cleanup(StateActionSet* const me);
StateActionSet * StateActionSet_Create(void); void
StateActionSet_Destroy(StateActionSet* const me); #endif

代码清单33：StateActionSet.h

```
#include "StateActionSet.h"

void StateActionSet_Init(StateActionSet* const me) {
  me->entryAction = NULL;
  me->evAbortHandler = NULL;
  me->evDisableHandler = NULL;
  me->evEnableHandler = NULL;
  me->evErrorHandler = NULL;
  me->evOkHandler = NULL;
  me->evRunHandler = NULL;
  me->evShutDownHandler = NULL;
  me->evWarningHandler = NULL;
  me->exitAction = NULL;
  me->is_inHandler = NULL;
}

void StateActionSet_Cleanup(StateActionSet* const me) {
}

StateActionSet * StateActionSet_Create(void) {
  StateActionSet* me = (StateActionSet *)
malloc(sizeof(StateActionSet));
  if(me!=NULL)
     StateActionSet_Init(me);
  return me;
}

void StateActionSet_Destroy(StateActionSet* const me) {
  if(me!=NULL)
     StateActionSet_Cleanup(me);
  free(me);
}
```

Code Listing 34: StateActionSet.c

The last four code lists are for the header and implementation files of the `ErrorStateClass` and `ModeStateClass`, which each implement one of the orthogonal regions of the composite state On. Note that the `aSet` attribute variable is in the same position for both these classes, meaning that we can invoke them using a cast pointer without getting into trouble – that is, if we have a pointer to an `ErrorStateClass` struct, in reality it might be a pointer to a `ModeStateClass` but it still works because the `aSet` that holds the function pointers to the functions to invoke is in the same position offset in both classes.

```
#ifndef ErrorStateClass_H
#define ErrorStateClass_H

#include "LightPkg.h"
#include "StateActionSet.h"

struct LightController;
```


#include "StateActionSet.h" void StateActionSet_ Init(StateActionSet* const me) { me->entryAction = NULL; me-> evAbortHandler = NULL; me->evDisableHandler = NULL; me-> evEnableHandler = NULL; me->evErrorHandler = NULL; me-> evOkHandler = NULL; me->evRunHandler = NULL; me-> evShutDownHandler = NULL; me->evWarningHandler = NULL; me->exitAction = NULL; me->is_inHandler = NULL; } void StateActionSet_Cleanup(StateActionSet* const me) { } StateActionSet * StateActionSet_Create(void) { StateActionSet* me = (StateActionSet *) malloc(sizeof(StateActionSet)); if(me! =NULL)StateActionSet_Init(me); return me; } void StateActionSet_ Destroy(StateActionSet* const me) { if(me!=NULL)StateActionSet_ Cleanup(me); free(me); }

代码清单 34：StateActionSet.c

最后四个代码清单是 `ErrorStateClass` 和 `ModeStateClass` 的头文件和实现文件，它们各自实现复合状态 On 的一个正交区域。请注意，`aSet` 属性变量在这两个类中的位置相同，这意味着我们可以通过强制转换指针来调用它们而不会出现问题——也就是说，如果我们有一个指向 `ErrorStateClass` 结构体的指针，实际上它可能是指向 `ModeStateClass` 的指针，但它仍然可以正常工作，因为保存要调用的函数的函数指针的 `aSet` 在两个类中的偏移位置相同。

#ifndef ErrorStateClass_H # define ErrorStateClass_H # include "LightPkg.h" #include " StateActionSet.h" struct LightController;

```
typedef struct ErrorStateClass ErrorStateClass;
struct ErrorStateClass {
  struct StateActionSet aSet;
  ErrorStatesType currentState;
  ErrorStatesType defaultState;
  struct LightController* itsLightController;
};

void ErrorStateClass_Init(ErrorStateClass* const me);
void ErrorStateClass_Cleanup(ErrorStateClass* const me);

/* Operations */
void ErrorStateClass_entryAction(ErrorStateClass* const me);
int ErrorStateClass_evAbort(ErrorStateClass* const me);
int ErrorStateClass_evError(ErrorStateClass* const me);
int ErrorStateClass_evOk(ErrorStateClass* const me);
int ErrorStateClass_evWarning(ErrorStateClass* const me);

void ErrorStateClass_exitAction(ErrorStateClass* const me);
int ErrorStateClass_is_in(ErrorStateClass* const me, ErrorStatesType s);

struct LightController* ErrorStateClass_getItsLightController(const ErrorStateClass*
const me);

void ErrorStateClass_setItsLightController(ErrorStateClass* const me, struct LightCon-
troller* p_LightController);

ErrorStateClass * ErrorStateClass_Create(void);

void ErrorStateClass_Destroy(ErrorStateClass* const me);
#endif
```

Code Listing 5-35: ErrorStateClass.h

```
#include "ErrorStateClass.h"
#include "LightController.h"

static void cleanUpRelations(ErrorStateClass* const me);

void ErrorStateClass_Init(ErrorStateClass* const me) {
  me->currentState = NULL_ERROR_STATE;
  me->defaultState = ERROR_STATE;
  StateActionSet_Init(&(me->aSet));
  me->itsLightController = NULL;
  /*
  set up the aSet function pointers
  remember that the initializer for the StateActionSet
  sets all pointers to NULL as the default

  Note: In this implementation, pointers are only set to non-NULL
  for actual handlers (except for entry() and exit()). The caller
  can tell if the event is handled by checking if the pointer is NULL.

  The higher level On state handles events reset and evDisable.
 */
```



```
typedef struct ErrorStateClass ErrorStateClass; struct
ErrorStateClass { struct StateActionSet aSet;
ErrorStatesType currentState; ErrorStatesType
defaultState; struct LightController* itsLightController; };

void ErrorStateClass_Init(ErrorStateClass* const me); void
ErrorStateClass_Cleanup(ErrorStateClass* const me);

/* 操作 */
void ErrorStateClass_entryAction(ErrorStateClass* const me); int
ErrorStateClass_evAbort(ErrorStateClass* const me); int ErrorStateClass_
evError(ErrorStateClass* const me); int ErrorStateClass_
evOk(ErrorStateClass* const me); int ErrorStateClass_
evWarning(ErrorStateClass* const me);

void ErrorStateClass_exitAction(ErrorStateClass* const me); int ErrorStateClass_is_
in(ErrorStateClass* const me, ErrorStatesType s);

struct LightController* ErrorStateClass_getItsLightController(const ErrorStateClass* const me);

void ErrorStateClass_setItsLightController(ErrorStateClass* const me, struct LightController* p_
LightController);

ErrorStateClass * ErrorStateClass_Create(void);

void ErrorStateClass_Destroy(ErrorStateClass* const me); #endif
```

代码清单 5-35: ErrorStateClass.h

```
#include "ErrorStateClass.h" #
include "LightController.h"

static void cleanUpRelations(ErrorStateClass* const me);

void ErrorStateClass_Init(ErrorStateClass* const me) { me->
currentState = NULL_ERROR_STATE; me->defaultState =
ERROR_STATE; StateActionSet_Init(&(me->aSet)); me->
itsLightController = NULL; /* 设置 aSet 函数指针，请记住
StateActionSet 的初始化器默认将所有指针设置为 NULL

注意：在此实现中，只有实际处理函数（entry() 和 exit() 除外）的指针才会被设置
为非 NULL。调用者可以通过检查指针是否为 NULL 来判断事件是否被处理。

更高层次的On状态处理reset和evDisable事件。*/
```

```
  me->aSet.evOkHandler = (ActionPtr0) ErrorStateClass_evOk;
  me->aSet.evWarningHandler = (ActionPtr0) ErrorStateClass_evWarning;
  me->aSet.evErrorHandler = (ActionPtr0) ErrorStateClass_evError;
  me->aSet.evAbortHandler = (ActionPtr0) ErrorStateClass_evAbort;
  me->aSet.entryAction = (ActionPtr0) ErrorStateClass_entryAction;
  me->aSet.exitAction = (ActionPtr0) ErrorStateClass_exitAction;
  me->aSet.is_inHandler = (ActionPtr1) ErrorStateClass_is_in
}
void ErrorStateClass_Cleanup(ErrorStateClass* const me) {
  cleanUpRelations(me);
}
void ErrorStateClass_entryAction(ErrorStateClass* const me) {
}
int ErrorStateClass_evAbort(ErrorStateClass* const me) {
  switch(me->currentState) {
     case OK_STATE:
     case WARNING_STATE:
     case ERROR_STATE:

  case NULL_ERROR_STATE:
        break;
     };
  return 1;
}
int ErrorStateClass_evError(ErrorStateClass* const me) {
  switch(me->currentState) {
     case OK_STATE:
        Light_setColor(me->itsLightController->itsLight, RED);
        me->currentState = ERROR_STATE;
        break;
     case WARNING_STATE:
        Light_setColor(me->itsLightController->itsLight, RED);
        me->currentState = ERROR_STATE;
        break;
     case ERROR_STATE:
        case NULL_ERROR_STATE:
        break;
     };
  return 1;
}
int ErrorStateClass_evOk(ErrorStateClass* const me) {
  switch(me->currentState) {
        Light_setColor(me->itsLightController->itsLight, GREEN);
        me->currentState = OK_STATE;
        break;
     case ERROR_STATE:
        Light_setColor(me->itsLightController->itsLight, GREEN);
        me->currentState = OK_STATE;
        break;
```


me->aSet.evOkHandler = (ActionPtr0) ErrorStateClass_evOk; me->
aSet.evWarningHandler = (ActionPtr0) ErrorStateClass_evWar ning;
me->aSet.evErrorHandler = (ActionPtr0) ErrorStateClass_evError ;
me->aSet.evAbortHandler = (ActionPtr0) ErrorStateClass_evAbort ;
me->aSet.entryAction = (ActionPtr0) ErrorStateClass_entryActio n;
me->aSet.exitAction = (ActionPtr0) ErrorStateClass_exitAction; me->
aSet.is_inHandler = (ActionPtr1) ErrorStateClass_is_in } void
ErrorStateClass_Cleanup(ErrorStateClass* const me) { cl
eanUpRelations(me); } void ErrorStateClass_entryAction(ErrorStateClass*
const me) { } int ErrorStateClass_evAbort(ErrorStateClass* const me) {
switch(me->currentState) { case 正常状态:_case 警告状态:_case 错误_状态:
case 空错误状态:_ _break; }; return 1; } int ErrorStateClass_
evError(ErrorStateClass* const me) { switch(me->currentState) { case 正常状态:
_Light_setColor(me->itsLightController->itsLight, 红色); me->currentState = 错
误_状态; break; case 警告_状态: Light_setColor(me->itsLightController->
itsLight, 红色); me->currentState = 错误_状态; break; case 错误状态:_case 空错
误状态:_ _break; }; return 1; } int ErrorStateClass_
evOk(ErrorStateClass* const me) { switch(me->currentState) { Light_
setColor(me->itsLightController->itsLight, 绿色)

;
me->currentState = OK_STATE; break; case ERROR_STATE: Light_
setColor(me->itsLightController->itsLight, GREEN)

;
me->currentState = 正常_状态;
break;

```
    case WARNING_STATE:
    case OK_STATE:
       case NULL_ERROR_STATE:
        break;
    };
  return 1;
  /*#]*/
}

/*## operation evWarning() */
int ErrorStateClass_evWarning(ErrorStateClass* const me) {
  /*#[ operation evWarning() */
  switch(me->currentState) {
    case OK_STATE:
        Light_setColor(me->itsLightController->itsLight, YELLOW);
        me->currentState = WARNING_STATE;
        break;
    case ERROR_STATE:
        Light_setColor(me->itsLightController->itsLight, YELLOW);
        me->currentState = WARNING_STATE;
        break;
    case WARNING_STATE:
  case NULL_ERROR_STATE:
        break;
    };
  return 1;
}

void ErrorStateClass_exitAction(ErrorStateClass* const me) {
}

int ErrorStateClass_is_in(ErrorStateClass* const me, ErrorStatesType s) {
  return s == me->currentState;
}

struct LightController* ErrorStateClass_getItsLightController(const ErrorStateClass*
const me) {
  return (struct LightController*)me->itsLightController;
}

void ErrorStateClass_setItsLightController(ErrorStateClass* const me, struct LightCon-
troller* p_LightController) {
  me->itsLightController = p_LightController;
}

ErrorStateClass * ErrorStateClass_Create(void) {
  ErrorStateClass* me = (ErrorStateClass *)
malloc(sizeof(ErrorStateClass));
  if(me!=NULL)
      ErrorStateClass_Init(me);
  return me;
}

void ErrorStateClass_Destroy(ErrorStateClass* const me) {
  if(me!=NULL)
```


case 警告状态:_case 正常状态:_case 空_错误_状态: break; }; return 1; /*#]*/ } /*## 操作 evWarning() */ int ErrorStateClass_evWarning(ErrorStateClass* const me) { /*#[操作 evWarning() */ switch(me->currentState) { case 正常状态:_Light_setColor(me->itsLightController->itsLight, 黄色); me->currentState = 警告_状态; break; case 错误状态:_Light_setColor(me->itsLightController->itsLight, 黄色); me->currentState = 警告_状态; break; case 警告_状态: case 空错误状态:_ _break; }; return 1; } void ErrorStateClass_exitAction(ErrorStateClass* const me) { } int ErrorStateClass_is_in(ErrorStateClass* const me, ErrorStatesType s) { return s == me->currentState; } struct LightController* ErrorStateClass_getItsLightController(const ErrorStateClass* const me) { return (struct LightController*)me->itsLightController; } void ErrorStateClass_setItsLightController(ErrorStateClass* const me, struct LightCon- troller* p_LightController) { me->itsLightController = p_LightController; } ErrorStateClass * ErrorStateClass_Create(void) { ErrorStateClass* me = (ErrorStateClass *) malloc(sizeof(ErrorStateClass)); if(me!=NULL)ErrorStateClass_Init(me); return me; } void ErrorStateClass_Destroy(ErrorStateClass* const me) { if(me!=NULL)

```
    ErrorStateClass_Cleanup(me);
  free(me);
}

static void cleanUpRelations(ErrorStateClass* const me) {
  if(me->itsLightController != NULL)
  me->itsLightController = NULL;
}
```

Code Listing 5-36: ErrorState.c

```
#ifndef ModeStateClass_H
#define ModeStateClass_H

#include "LightPkg.h"
#include "StateActionSet.h"
struct LightController;

typedef struct ModeStateClass ModeStateClass;
struct ModeStateClass {
  struct StateActionSet aSet;
  ModeStatesType currentState;
  ModeStatesType defaultState;
  struct LightController* itsLightController;
};

void ModeStateClass_Init(ModeStateClass* const me);

void ModeStateClass_Cleanup(ModeStateClass* const me);

/* Operations */
void ModeStateClass_entryAction(ModeStateClass* const me);
int ModeStateClass_evAbort(ModeStateClass* const me);
int ModeStateClass_evRun(ModeStateClass* const me);
int ModeStateClass_evShutDown(ModeStateClass* const me);
void ModeStateClass_exitAction(ModeStateClass* const me);
int ModeStateClass_is_in(ModeStateClass* const me, ModeStatesType s);

struct  LightController*  ModeStateClass_getItsLightController(const  ModeStateClass*
const me);

void ModeStateClass_setItsLightController(ModeStateClass* const me, struct LightCon-
troller* p_LightController);

ModeStateClass * ModeStateClass_Create(void);

void ModeStateClass_Destroy(ModeStateClass* const me);
#endif
```

Code Listing 5-37: ModeStateClass.h

```
#include "ModeStateClass.h"
#include "LightController.h"

static void cleanUpRelations(ModeStateClass* const me);

void ModeStateClass_Init(ModeStateClass* const me) {
```


ErrorStateClass_Cleanup(me); free(me); } static void cleanUpRelations(ErrorStateClass* const me) { if(me-> itsLightController != 空) me->itsLightController = 空; }

代码清单5-36：ErrorState.c

#ifndef ModeStateClass_H # define ModeStateClass_H

#include "LightPkg.h" #include " StateActionSet.h" struct LightController;

typedef struct ModeStateClass ModeStateClass; struct ModeStateClass { struct StateActionSet aSet; ModeStatesType currentState; ModeStatesType defaultState; struct LightController* itsLightController; };

void ModeStateClass_Init(ModeStateClass* const me);

void ModeStateClass_Cleanup(ModeStateClass* const me);

/* 操作 */ void ModeStateClass_entryAction(ModeStateClass* const me); int ModeStateClass_evAbort(ModeStateClass* const me); int ModeStateClass_ evRun(ModeStateClass* const me); int ModeStateClass_ evShutDown(ModeStateClass* const me); void ModeStateClass_ exitAction(ModeStateClass* const me); int ModeStateClass_is_in(ModeStateClass* const me, ModeStatesType s);

struct LightController* ModeStateClass_getItsLightController(const ModeStateClass* const me);

void ModeStateClass_setItsLightController(ModeStateClass* const me, struct LightController* p_ LightController);

ModeStateClass * ModeStateClass_Create(void);

void ModeStateClass_Destroy(ModeStateClass* const me); #endif

代码清单5-37：ModeStateClass.h

#include "ModeStateClass.h" #include "LightController.h" static void cleanUpRelations(ModeStateClass* const me); void ModeStateClass_Init(ModeStateClass* const me) {

```
  me->currentState = NULL_MODE_STATE;
  me->defaultState = STARTINGUP_STATE;
  {
      StateActionSet_Init(&(me->aSet));
  }
  me->itsLightController = NULL;
  /*
  set up the aSet function pointers
  remember that the initializer for the StateActionSet
  sets all pointers to NULL as the default

  Note: In this implementation, pointers are only set to non-NULL
  for actual handlers (except for entry() and exit()). The caller
  can tell if the event is handled by checking if the pointer is NULL.

  The higher level On state handles events reset and evDisable.
*/
  me->aSet.evRunHandler = (ActionPtr0) ModeStateClass_evRun;
  me->aSet.evShutDownHandler = (ActionPtr0) ModeStateClass_evShutDown;
  me->aSet.evAbortHandler = (ActionPtr0) ModeStateClass_evAbort;
  me->aSet.entryAction = (ActionPtr0) ModeStateClass_entryAction;
  me->aSet.exitAction = (ActionPtr0) ModeStateClass_exitAction;
  me->aSet.is_inHandler = (ActionPtr1) ModeStateClass_is_in;
}

void ModeStateClass_Cleanup(ModeStateClass* const me) {
  cleanUpRelations(me);
}

void ModeStateClass_entryAction(ModeStateClass* const me) {
}

int ModeStateClass_evAbort(ModeStateClass* const me) {
  return 1;
}

int ModeStateClass_evRun(ModeStateClass* const me) {
  switch (me->currentState) {
     case STARTINGUP_STATE:
         Light_setMode(me->itsLightController->itsLight, STEADY);
         me->currentState = OPERATIONAL_STATE;
         break;
     case SHUTTINGDOWN_STATE:
         Light_setMode(me->itsLightController->itsLight, QUICKLY);
         me->currentState = STARTINGUP_STATE;
         break;
     case OPERATIONAL_STATE:
     case NULL_MODE_STATE:
         break;
     };
  return 1;
}

int ModeStateClass_evShutDown(ModeStateClass* const me) {
```


me->currentState = NULL_MODE_STATE; me->defaultState = STARTINGUP_STATE; { StateActionSet_Init(&(me->aSet)); } me->itsLightController = NULL; /* 设置 aSet 函数指针，请记住 StateActionSet 的初始化器默认将所有指针设置为 NULL。注意：在此实现中，只有实际处理函数（entry() 和 exit() 除外）的指针才会被设置为非 NULL。调用者可以通过检查指针是否为 NULL 来判断事件是否被处理。更高层的 On状态 处理 reset 和 evDisable 事件。 */ me->aSet.evRunHandler = (ActionPtr0) ModeStateClass_evRun; me->aSet.evShutDownHandler = (ActionPtr0) ModeStateClass_evShutDown; me->aSet.evAbortHandler = (ActionPtr0) ModeStateClass_evAbort; me->aSet.entryAction = (ActionPtr0) ModeStateClass_entryAction; me->aSet.exitAction = (ActionPtr0) ModeStateClass_exitAction; me->aSet.is_inHandler = (ActionPtr1) ModeStateClass_is_in; } void ModeStateClass_Cleanup(ModeStateClass* const me) { cleanUpRelations(me); } void ModeStateClass_entryAction(ModeStateClass* const me) { } int ModeStateClass_evAbort(ModeStateClass* const me) { return 1; } int ModeStateClass_evRun(ModeStateClass* const me) { switch (me->currentState) { case STARTINGUP STATE:_Light_setMode(me->itsLightController->itsLight, STEADY); me->currentState = OPERATIONAL_STATE; break; case SHUTTINGDOWN_STATE: Light_setMode(me->itsLightController->itsLight, QUICKLY); me->currentState = STARTINGUP_STATE; break; case OPERATIONAL STATE:_case NULL MODESTATE:_ _break; }; return 1; } int ModeStateClass

¯evShutDown(ModeStateClass* const me) {

```
  switch (me->currentState) {
    case OPERATIONAL_STATE:
        Light_setMode(me->itsLightController->itsLight, SLOWLY);
        me->currentState = SHUTTINGDOWN_STATE;
        break;
    case STARTINGUP_STATE:
        Light_setMode(me->itsLightController->itsLight, SLOWLY);
        me->currentState = SHUTTINGDOWN_STATE;
        break;
    case SHUTTINGDOWN_STATE:
        break;
    case NULL_MODE_STATE:
        break;
    };
  return 1;
}

void ModeStateClass_exitAction(ModeStateClass* const me) {
}

int ModeStateClass_is_in(ModeStateClass* const me, ModeStatesType s) {
  return s == me->currentState;
}

struct LightController* ModeStateClass_getItsLightController(const ModeStateClass*
const me) {
  return (struct LightController*)me->itsLightController;
}

void ModeStateClass_setItsLightController(ModeStateClass* const me, struct LightCon-
troller* p_LightController) {
  me->itsLightController = p_LightController;
}

ModeStateClass * ModeStateClass_Create(void) {
  ModeStateClass* me = (ModeStateClass *)
malloc(sizeof(ModeStateClass));
  if(me!=NULL)
     ModeStateClass_Init(me);
  return me;
}

void ModeStateClass_Destroy(ModeStateClass* const me) {
  if(me!=NULL)
  ModeStateClass_Cleanup(me);
  free(me);
}

static void cleanUpRelations(ModeStateClass* const me) {
  if(me->itsLightController != NULL)
  me->itsLightController = NULL;
}
```

Code Listing 5-38: ModeStateClass.c

switch (me->currentState) { case 运行状态:_Light_setMode(me->itsLightController->itsLight, SLOWLY); me->currentState = SHUTTINGDOWN_STATE; break; case 启动状态:_Light_setMode(me->itsLightController->itsLight, SLOWLY); me->currentState = SHUTTINGDOWN_STATE; break; case SHUTTINGDOWN STATE:_break; case 空模式状态:_ _break; }; return 1; } void ModeStateClass_exitAction(ModeStateClass* const me) { } int ModeStateClass_is_in(ModeStateClass* const me, ModeStatesType s) { return s == me->currentState; } struct LightController* ModeStateClass_getItsLightController(const ModeStateClass* const me) { return (struct LightController*)me->itsLightController; } void ModeStateClass_setItsLightController(ModeStateClass* const me, struct LightCon- troller* p_LightController) { me->itsLightController = p_LightController; } ModeStateClass * ModeStateClass_Create(void) { ModeStateClass* me = (ModeStateClass *) malloc(sizeof(ModeStateClass)); if(me!=NULL)ModeStateClass_Init(me); return me; } void ModeStateClass_Destroy(ModeStateClass* const me) { if(me!=NULL)ModeStateClass_Cleanup(me); free(me); } static void cleanUpRelations(ModeStateClass* const me) { if(me->itsLightController != NULL) me->itsLightController = NULL; }

代码清单 5-38：ModeStateClass.c

Figure 5-25 shows a sample run of the model shown in Figure 5-24 in which the `LightControllerClient` sends the events `evEnable`, `evError`, `evRun`, and then `evAbort`. Note the use of the `is_in()` function to make sure that the preconditions of the join are met before the event is taken.

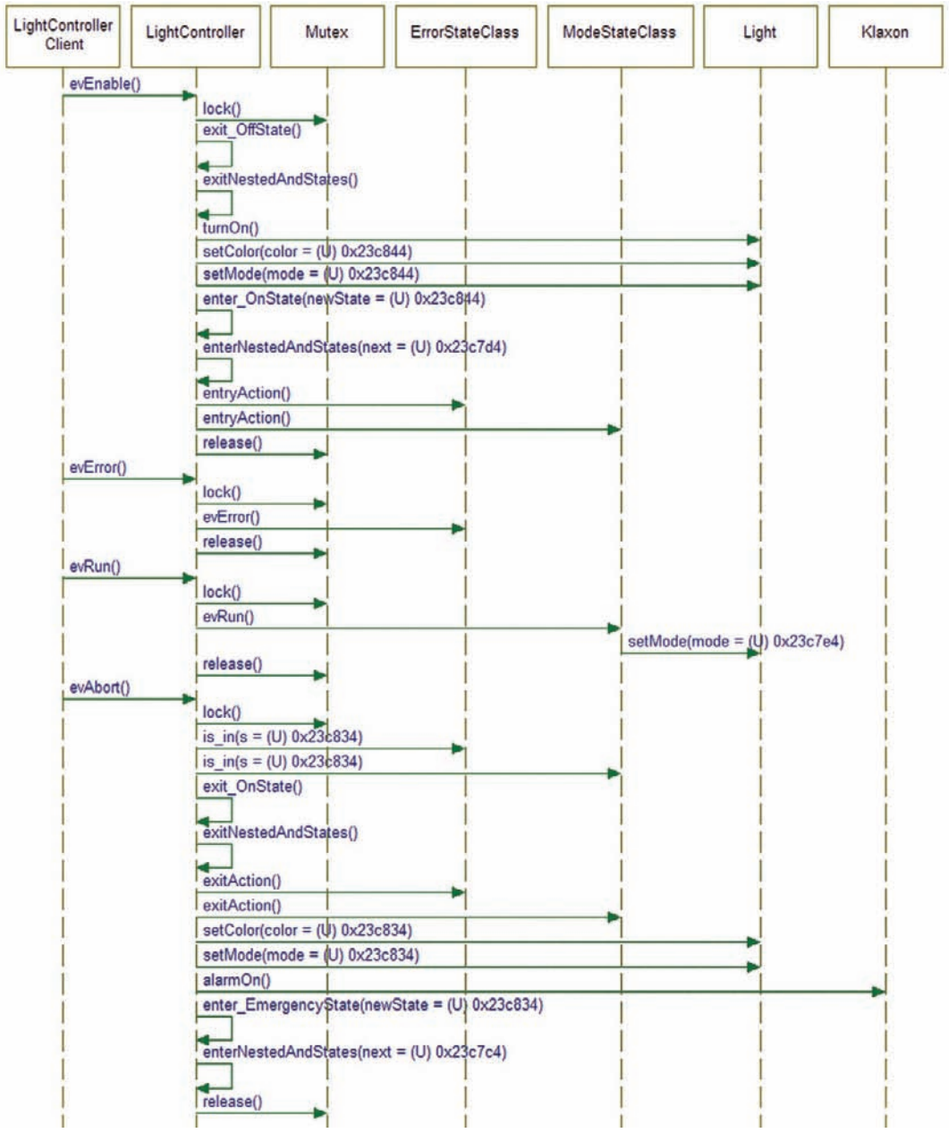


Figure 5-25: Sample execution of and-state example

图 5-25 展示了图 5-24 中所示模型的一次示例运行，其中 `LightControllerClient` 依次发送事件`evEnable`、`evError`、`evRun`，然后发送 `evAbort`。请注意使用`is_in()` 函数来确保在接收事件之前满足汇合的前置条件。

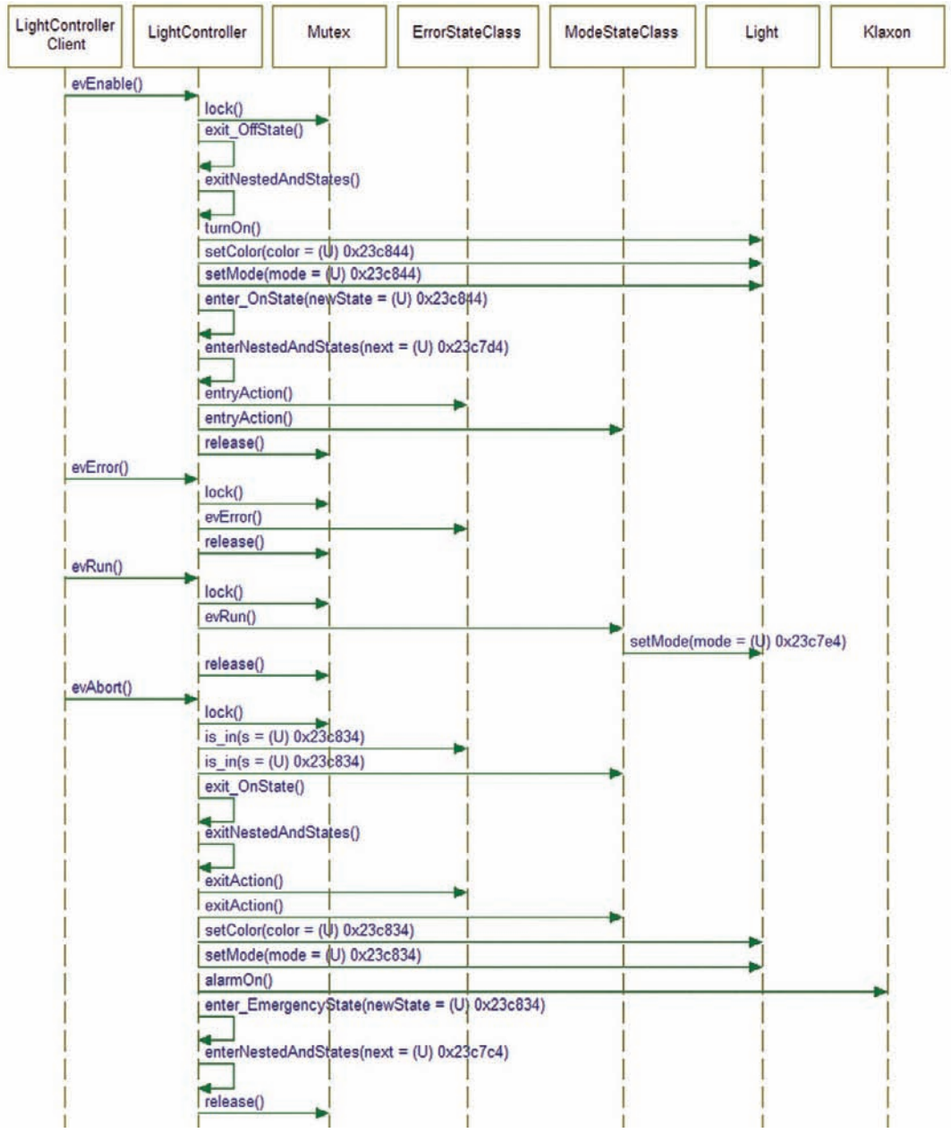


图5-25：与状态示例的示例执行

5.9 OK, What Have We Learned?

This chapter presents a number of patterns for the implementation of state machines. Each of these implementation patterns offers benefits and costs, so which you use depends upon what you are trying to optimize. The Single Event Receptor Pattern uses a single event receptor and implements the state behavior internally as a large switch-case statement. It requires the creation of a type associated with the event to pass to the receptor. In contrast, the Multiple Event Receptor Pattern uses separate event handlers for each event so that the event type is not explicitly represented. This makes the interface to the client slightly richer. The State Table Pattern scales up to large state spaces and provides performance that is independent of the size of the state space – good for large state spaces. However, it takes more set-up time than the other approaches and may waste memory for sparse state spaces. The State Pattern creates separate objects for each state. The implementation is less complex than for the State Table Pattern and distributes the complexity of the state machine across several objects. The Decomposed And-State Pattern provides a straightforward way of implementing AND-states but requires the use of polymorphism.

5.9 好，我们学到了什么?

本章介绍了多种用于实现状态机的模式。每种实现模式都有其优点和成本，因此使用哪种模式取决于您想要优化的目标。单事件接收器模式使用单个事件接收器，并在内部通过大型 switch-case 语句实现状态行为。它需要创建一个与事件关联的类型以传递给接收器。相比之下，多事件接收器模式为每个事件使用单独的事件处理器，因此事件类型不会被显式表示。这使得客户端的接口稍微更加丰富。状态表模式可扩展到大型状态空间，并提供与状态空间大小无关的性能——适合大型状态空间。然而，它比其他方法需要更多的设置时间，并且对于稀疏状态空间可能会浪费内存。状态模式为每个状态创建单独的对象。其实现比状态表模式更简单，并将状态机的复杂性分布到多个对象中。分解AND状态模式提供了一种实现AND状态的直接方式，但需要使用多态。

CHAPTER 6

Safety and Reliability Patterns

Chapter Outline

6.1 A Little Bit About Safety and Reliability 359
- 6.1.1 Safety and Reliability Related Faults 360
- 6.1.2 Achieving Safety and Reliability 361

6.2 One's Complement Pattern 362
- 6.2.1 Abstract 362
- 6.2.2 Problem 362
- 6.2.3 Pattern Structure 362
- 6.2.4 Collaboration Roles 362
  - *6.2.4.1 DataClient 362*
  - *6.2.4.2 OnesCompProtectedDataElement 362*
- 6.2.5 Consequences 363
- 6.2.6 Implementation Strategies 363
- 6.2.7 Related Patterns 363
- 6.2.8 Example 363

6.3 CRC Pattern 367
- 6.3.1 Abstract 367
- 6.3.2 Problem 367
- 6.3.3 Pattern Structure 368
- 6.3.4 Collaboration Roles 368
  - *6.3.4.1 computeCRCData Function 368*
  - *6.3.4.2 CRCProtectedData 368*
  - *6.3.4.3 DataClient 368*
  - *6.3.4.4 DataType 368*
- 6.3.5 Consequences 369
- 6.3.6 Implementation Strategies 369
- 6.3.7 Related Patterns 369
- 6.3.8 Example 369

6.4 Smart Data Pattern 380
- 6.4.1 Abstract 380
- 6.4.2 Problem 381
- 6.4.3 Pattern Structure 381
- 6.4.4 Collaboration Roles 382
  - *6.4.4.1 ErrorCodeType 382*
  - *6.4.4.2 ErrorManager 382*
  - *6.4.4.3 ServerClass 382*
  - *6.4.4.4 SmartDataType 382*
- 6.4.5 Consequences 382

Design Patterns for Embedded Systems in C
DOI:10.1016/B978-1-85617-707-8.00006-6

Copyright © 2011 Elsevier Inc.

第 6 章

安全性与可靠性模式

本章大纲

6.1 关于 安全性 与 可靠性 的 一点 说明 **359** 6 1 1 360 . . 安全性与可靠性相关故障 6.1.2 实现安全性与可靠性 361

6.2 反码 模式 **362** 6 2 1 362 . . 摘要 6.2.2 问题 362 6.2.3 模式结构 362 6.2.4 协作角色 362 6.2.4.1 DataClient 362 6.2.4.2 OnesCompProtectedDataElement 3 6.2.5 结果 363 6.2.6 实现策略 363 6.2.7 相关模式 363 6.2.8 示例 363**6.3 CRC** 模式 **367**6.3.1 摘要 367 6.3.2 问题 367 6.3.3 模式结构 368 6.3.4 协作角色 368 6.3.4.1 computeCRCData函数 368 6.3.4.2 CRCProtectedData 368 6.3.4.3 DataClient 368 6.3.4.4 DataType 368 6.3.5 结果 369 6.3.6 实现策略 369 6.3.7 相关模式 369 6.3.8 示例 369

6.4 Smart Data 模式 **380**6.4.1 摘要 380 6.4.2 问题 381 6.4.3 模式结构 381 6.4.4 协作角色 382 6.4.4.1 ErrorCodeType 382 6.4.4.2 ErrorManager 382 6.4.4.3 ServerClass 382 6.4.4.4 SmartDataType 382 6.4.5 结果 382

C语言嵌入式系统设计模式
DOI:10.1016/B978-1-85617-707-8.00006-6 版权所有 © 2011 Elsevier Inc.

6.4.6 Implementation Strategies 383
6.4.7 Related Patterns 383
6.4.8 Example 383
6.5 Channel Pattern 395
6.5.1 Abstract 395
6.5.2 Problem 395
6.5.3 Pattern Structure 395
6.5.4 Collaboration Roles 395
6.5.4.1 AbstractDataTransform 395
6.5.4.2 ActuatorDeviceDriver 396
6.5.4.3 ConcreteDataTransform 396
6.5.4.4 SensorDeviceDriver 396
6.5.5 Consequences 396
6.5.6 Implementation Strategies 397
6.5.7 Related Patterns 397
6.5.8 Example 397
6.6 Protected Single Channel Pattern 402
6.6.1 Abstract 402
6.6.2 Problem 402
6.6.3 Pattern Structure 403
6.6.4 Collaboration Roles 404
6.6.4.1 AbstractDataTransform 404
6.6.4.2 AbstractTransformChecker 404
6.6.4.3 ActuatorDeviceDriver 404
6.6.4.4 ConcreteDataTransform 404
6.6.4.5 ConcreteTransformChecker 404
6.6.4.6 SensorDeviceDriver 404
6.6.5 Implementation Strategies 405
6.6.6 Related Patterns 405
6.6.7 Example 405
6.7 Dual Channel Pattern 413
6.7.1 Abstract 413
6.7.2 Problem 414
6.7.3 Pattern Structure 414
6.7.4 Collaboration Roles 414
6.7.4.1 AbstractDataTransform 414
6.7.4.2 AbstractTransformChecker 414
6.7.4.3 ActuatorDeviceDriver 415
6.7.4.4 Channel 416
6.7.4.5 ConcreteDataTransform 416
6.7.4.6 ConcreteTransformChecker 417
6.7.4.7 SensorDeviceDriver 417
6.7.5 Consequences 417
6.7.6 Implementation Strategies 417
6.7.7 Related Patterns 417
6.7.7.1 Homogeneous Redundancy Pattern 417
6.7.7.2 Heterogeneous Redundancy Pattern 418

6.4. 6 实现策略 383 6.4.7 相关模式
383 6.4.8 示例 383

6.5 通道 模式 **395**6.5.1 摘要 395 6.5.2 问题
395 6.5.3 模式结构 395 6.5.4 协作角色 395
6.5.4.1 抽象数据变换 395 6.5.4.2 执行器设备驱
动 396 6.5.4.3 ConcreteDataTransform 396
6.5.4.4 传感器设备驱动 396 6.5.5 结果 396
6.5.6 实现策略 397 6.5.7 相关模式 397 6.5.8 示
例 397**6.6** 受保护 单 通道 模式 **402** 6 6 1
402 . . 摘要 6.6.2 问题 402 6.6.3 模式结构 403
6.6.4 协作角色 404 6.6.4.1 抽象数据变换 404
6.6.4.2 抽象变换检查器 404 6.6.4.3 执行器设备
驱动 404 6.6.4.4 ConcreteDataTransform
404 6.6.4.5 具体变换检查器 404 6.6.4.6 传感器
设备驱动 404 6.6.5 实现策略 405 6.6.6 相关模式
405 6.6.7 示例 405**6.7** 双 通道 模式
4136.7.1 摘要 413 6.7.2 问题 414 6.7.3 模式结
构 414 6.7.4 协作角色 414 6.7.4.1 抽象数据变换
414 6.7.4.2 抽象变换检查器 414 6.7.4.3 执行器
设备驱动 415 6.7.4.4 通道 416 6.7.4.5
ConcreteDataTransform 416 6.7.4.6 具体变换
检查器 417 6.7.4.7 传感器设备驱动 417 6.7.5 结
果 417 6.7.6 实现策略 417 6.7.7 相关模式 417
6.7.7.1 同构冗余模式 6.7.7.2 异构冗余模式

417
418

6.7.7.3 Triple Modular Redundancy (TMR) Pattern 418
6.7.7.4 Sanity-Check Pattern 419
6.7.7.5 Monitor-Actuator Pattern 420
6.7.8 Example 421
6.8 Summary 422

Patterns in this chapter

One's Complement Pattern – Adds a bitwise inverted copy of primitive data elements to identify when data is corrupted *in vivo*

CRC Pattern – Adds a cyclic redundancy check to identify when bits of the data have been corrupted *in vivo*

Smart Data Pattern – Adds behavior to the data to ensure that the data's preconditions and constraints are adhered to

Channel Pattern – Arranges the processing of sensory data as a series of transformational steps to provide a unit of large-scale redundancy

Protected Single Channel Pattern – Elaborates the channel pattern by adding data and processing validation checks at various points within the single channel

Dual Channel Pattern – Creates multiple channels to aid in the identification of errors and failures and, optionally, allows continuation of service in the presence of such as fault

6.1 A Little Bit About Safety and Reliability

Reliability is a measure of the "up-time" or "availability" of a system – specifically, it is the probability that a computation will successfully complete before the system fails. Put another way, it is a stochastic measure of the probability that the system can deliver a service. It is normally estimated with Mean Time Between Failure (MTBF) or a probability of successful service delivery (e.g., 0.9999). Reliability analysis is normally managed within Fault Means and Effect Analysis (FMEA) such as shown in Figure 6.1.

Safety is distinct from reliability. A safe system is one that does not incur too much risk to persons or equipment. A *hazard* is an event or condition that leads to an *accident* or *mishap* – a loss of some kind, such as injury or death. The risk associated with a hazard is the product of the severity of the hazard and its likelihood.

Hazards arise in five fundamental ways:

- Release of energy
- Release of toxins
- Interference of life support functions
- Supplying misleading information to safety personnel or control systems
- Failure to alarm when hazardous conditions arise

6.7.7.3 三重 模块化 冗余 (TMR) 模式 418
6.7.7.4 健全性检查 模式 419 6.7.7.5 监控器-执行器 模式 4206.7.8 示例 4216.8 总结 422

本章中的模式

反码模式 – 添加原始数据元素的按位取反副本，以识别数据在运行中何时被损坏 CRC模式 – 添加循环冗余校验，以识别数据位在运行中何时被损坏 智能数据模式 – 为数据添加行为，以确保遵守数据的前置条件和约束 通道模式 – 将传感数据的处理安排为一系列转换步骤，以提供大规模冗余单元 受保护单通道模式 – 通过在单通道内的各个点添加数据和进程验证检查来详细说明通道模式 双通道模式 – 创建多个通道以帮助识别错误和故障，并且可选地允许在存在此类故障时继续提供服务

6.1 关于安全性与可靠性的一点介绍

可靠性是系统"正常运行时间"或"可用性"的度量——具体而言，它是计算在系统发生故障之前成功完成的概率。换句话说，它是系统能够提供服务概率的随机度量。通常用平均故障间隔时间（MTBF）或成功服务交付的概率（例如0.9999）来估计。可靠性分析通常在故障模式与影响分析（FMEA）中进行管理，如图6.1所示。

安全性不同于可靠性。安全的系统是指不会对人员或设备造成过多风险的系统。危险是导致事故或灾难的事件或条件——某种损失，例如受伤或死亡。与危险相关的风险是危险的严重度与其可能性的乘积。

危险以五种基本方式产生：

- 能量释放
- 毒素释放
- 生命支持功能受到干扰
- 向安全人员或控制系统提供误导性信息
- 危险状况出现时未能发出alarm

Description	Function	Failure Rate / million hrs	Failure Mode	Failure Effects	Potential Cases	Corrective Actions	Criticality
O2 Sensor	Senses O2 conc in the blood	110	LED fails	System cannot determine O2 conc	• Power disconnect • LED emitter fails	• use gold contacts • Use high rated LED	High
		1750	Connection to system fails	System cannot determine O2 conc	• RS232 wire disconnect • EMI	• Use high rated connectors • Use message CRCs	High
Display of O2 conc	Displays patient data to physician	6000	Wrong value displayed	Misdiagnosis and mistreatment of acute conditions	• Internal data corruption • Computation error	• Data stored redundantly • Computation done with two separate algorithms	High
…							
…							
…							
…							

Figure 6-1: FMEA example

6.1.1 Safety and Reliability Related Faults

Accidents can occur either because the system contains errors or because some aspect of the system fails. An error is a systematic fault and may be due to incorrect requirements, poor design or poor implementation. An error is always present in the system even though it may not be visible. When an error is visible, it is said to be *manifest*. Failures are different in that they occur at some point in time and result in a failed state. That is, the system worked properly before but something changed, such as a metal rod breaking in the drive train of your car (a *persistent failure*) or a bit was flipped in your car's engine control computer but is repaired as the system runs (a *volatile failure*). A fault is either an error or a failure.

I make this distinction about the different kinds of faults because the kinds of mitigation behavior for the two kinds is different. Specifically, failures can be addressed by *homogeneous redundancy* – multiple replicas of the processing channel (a set of software elements). If you assume the system is safe enough if it remains safe in the case of a single point failure, then having an identical backup channel can address such a failure. However, if the fault is an error, then all identical copies of the channel will exhibit the same fault. In this case, you must have *heterogeneous redundancy* (also known as *n-way programming*) – multiple channels that provide the service using different code and/or designs-in order to continue functioning safely.

Faults have relations to various elements in your system. Behavior around faults may be specified in requirements. An element can be the source of the fault. Another element may be the identifier of the fault. Yet another element may mitigate or extenuate the fault.

描述	函数	失效 Rate / 百万小时	失效模式	失效影响	潜在案例	纠正措施 动作	关键性
氧传感器	感测氧浓度在血液	110	LED故障	系统无法确定氧气conc	• 电源断开 • LED发射器失效	• 使用镀金触点 • 使用高额定LED	High
		1750	连接到系统故障	系统无法确定氧气conc	• RS232线断开 • 电磁干扰	• 使用高额定连接器 • 使用消息CRCs	High
显示氧浓度	显示器患者数据至医生	6000	错误值display	误诊and错误治疗急性病症	• 内部数据损坏 • 计算错误	• 数据存储冗余地 • 计算完成于两个独立的算法	High
…							
…							
…							
…							

图6-1：FMEA示例

6.1.1 安全性与可靠性相关故障

事故的发生可能源于系统包含错误，也可能源于系统的某个方面发生故障。错误是一种系统性故障，可能由不正确的需求、糟糕的设计或糟糕的实现导致。错误始终存在于系统中，即使它可能不可见。当错误变得可见时，就称其为显性。故障则不同，它发生在某个时间点并导致故障状态。也就是说，系统之前运行正常，但某些东西发生了变化，例如汽车传动系统中的金属杆断裂（持续性故障），或者汽车发动机控制计算机中的某个位被翻转但在系统运行过程中被修复（易失性故障）。故障要么是错误，要么是失效。

我对不同种类的故障做出这一区分，是因为针对这两类故障的缓解行为有所不同。具体而言，故障可以通过同构冗余来解决——即处理通道（一组软件元素）的多个副本。如果你认为系统在发生单点故障时仍能保持安全就足够安全，那么拥有一个相同的备份通道就可以应对此类故障。然而，如果该故障是一个错误，那么通道的所有相同副本都会表现出相同的故障。在这种情况下，你必须采用异构冗余（也称为N版本编程）——即使用不同的代码和/或设计来提供服务的多个通道——以便继续安全地运行。

故障与系统中的各种元素存在关系。围绕故障的行为可以在需求中加以规定。一个元素可以是故障的来源。另一个元素可以是故障的标识符。还有一个元素可以缓解或减轻故障。

I believe that there is no such thing as "safe software" because it is the composite system – the electronics, mechanical, and software aspects working together in their operational environment – that is safe or unsafe. Hardware faults can be mitigated with software and vice versa. The system as a whole is safe or unsafe. Similarly, the reliability of a system is a function of all its constituent parts, since other parts can be designed to continue to provide services normally provided by failed parts. There is extensive literature on how to assess safety and reliability and how to construct safe and reliable systems, but a detailed discussion of those topics is beyond the scope of this book. More information can be had in my *Real-Time UML Workshop for Embedded Systems* or my *Real-Time Agility* books[1] as well as many other sources.

6.1.2 Achieving Safety and Reliability

In general, all safety-critical systems and high-reliability systems must contain and properly manage redundancy to achieve their safety and reliability requirements. How these elements are managed is different depending on the particular mix of safety and reliability concerns. This redundancy can be "redundancy-in-the-large" (architectural redundancy) or "redundancy-in-the-small" (detailed design redundancy). In this chapter we will consider a few patterns of both varieties. More safety-related architectural patterns can be found in my book *Real-Time Design Patterns*[2].

In the safety and reliability patterns in this chapter, there are three kinds of related elements. *Fault Manifestors* are elements that can, if they contain a fault, lead to a safety or reliability concern. *Fault Identifiers* are elements that can identify when a failure has occurred or when an error has become manifest. *Fault Extenuators* are elements that extenuate (reduce) a fault by either making the fault-related hazard less severe or by making the manifestation of the fault at the system level less likely. The patterns in this chapter differ in how these elements are organized and managed. Feedforward error correction patterns address the concern by providing enough redundancy to identify and deal with the fault and continue processing. Feedback error correction patterns address the faults by repeating one or more computational steps. Fault-safe state patterns address safety by going to a fault-safe state, but this reduces the reliability of the system[3].

And now, let's get to the patterns. First we'll start with some detailed design-level patterns, which some people call *idioms*.

[1] Douglass, B.P., 2009. *Real-Time UML Workshop for Embedded Systems*. Elsevier. Douglass, B.P., 2009. *Real-Time Agility*. Addison-Wesley.
[2] Douglass, B.P., 2002. *Real-Time Design Patterns: Robust Scalable Architecture for Real-Time Systems*. Addison-Wesley.
[3] In general, safety and reliability are opposing concerns when there is a fault-safe state and they are cooperating concerns when there is not.

我认为不存在所谓的"安全软件"，因为真正安全或不安全的是复合系统——即电子、机械和软件等方面在其运行环境中协同工作的整体。硬件故障可以通过软件来缓解，反之亦然。整个系统才是安全或不安全的。同样，系统的可靠性是其所有组成部分的函数，因为其他部分可以被设计为继续提供通常由故障部分提供的服务。关于如何评估安全性与可靠性以及如何构建安全可靠的系统，已有大量文献，但对这些主题的详细讨论超出了本书的范围。更多信息可以在我的《嵌入式系统实时UML工作坊》或《实时敏捷》书籍[1] 以及许多其他来源中找到。

6.1.2 实现安全性与可靠性

一般而言，所有安全关键系统和高可靠性系统都必须包含并妥善管理冗余，以满足其安全性与可靠性需求。如何管理这些要素取决于安全性与可靠性关注点的具体组合。这种冗余可以是"大粒度冗余"（架构冗余）或"小粒度冗余"（详细设计冗余）。在本章中，我们将考虑这两种类型的若干模式。更多与安全性相关的架构模式可以在我的《实时设计模式》[2]一书中找到。

在本章的安全性与可靠性模式中，有三类相关要素。故障显现者是如果包含故障则可能导致安全性或可靠性问题的要素。故障识别者是能够识别故障何时发生或错误何时显现的要素。故障缓解者是通过使与故障相关的危险不那么严重或使故障在系统层面显现的可能性降低来缓解（减少）故障的要素。本章中的模式在这些要素的组织和管理方式上有所不同。前馈纠错模式通过提供足够的冗余来识别和处理故障并继续处理，从而解决该问题。反馈纠错模式通过重复一个或多个计算步骤来解决故障。故障安全状态模式通过进入故障安全状态来解决安全性问题，但这会降低系统的可靠性[3]。

现在，让我们进入模式部分。首先从一些详细设计层面的模式开始，有些人将其称为惯用法。

[1] Douglass, B.P., 2009. 嵌入式系统实时UML工作坊. 爱思唯尔. Douglass, B.P., 2009. 实时敏捷性. Addison-Wesley. [2] Douglass, B.P., 2002. 实时设计模式：实时系统的健壮可扩展架构. Addison-Wesley. [3] 一般来说，当存在故障安全状态时，安全性与可靠性是相互对立的关注点；而当不存在故障安全状态时，它们是相互协作的关注点。

6.2 One's Complement Pattern

The One's Complement Pattern is useful to detect when memory is corrupted by outside influences or hardware faults.

6.2.1 Abstract

The One's Complement Pattern provides a detailed design pattern for the identification of single or multiple memory bit corruption. This can occur due to EMI (Electromagnetic interference), heat, hardware faults, software faults, or other external causes. The pattern works by storing critical data twice – once in normal form and once in one's complement (bit-wise inverted) form. When the data are read, the one's complement form is reinverted and then compared to the value of the normal form. If the values match, then that value is returned, otherwise error processing ensues.

6.2.2 Problem

This pattern addresses the problem that variables may be corrupted by a variety of causes such as environmental factors (e.g., EMI, heat, radiation), hardware faults (e.g., power fluctuation, memory cell faults, address line shorts), or software faults (other software erroneously modifying memory). This pattern addresses the problem of identifying data corruption for small sets of critical data values.

6.2.3 Pattern Structure

The basic pattern structure is shown in Figure 6-2. Since this is a detailed design pattern, its scope is primarily within a single class.

6.2.4 Collaboration Roles

This section describes the roles for this pattern.

6.2.4.1 DataClient

The `DataClient` is an element that either sets or retrieves the value of the datum.

6.2.4.2 OnesCompProtectedDataElement

The `OnesCompProtectedDataElement` is the class that combines the storage of the data with services that can detect bit-level data corruption. This class stores the values of the data twice; once in regular form and once as a bit-wise complement. When data are retrieved, inverted data are reinverted and compared to the value stored normally. If the values match, that value is returned; if not, the class provides a function to handle the error. The actual behavior of the error handler is system- and context-specific.

6.2 反码模式

反码模式可用于检测内存是否被外部影响或硬件故障所破坏。

6.2.1 摘要

反码模式提供了一种详细的设计模式，用于识别单个或多个内存位损坏。这种损坏可能由电磁干扰（EMI）、热量、硬件故障、软件故障或其他外部原因引起。该模式的工作原理是将关键数据存储两次——一次以正常形式存储，一次以反码（按位取反）形式存储。当读取数据时，反码形式被重新取反，然后与正常形式的值进行比较。如果值匹配，则返回该值，否则进行错误处理。

6.2.2 问题

该模式解决的问题是：变量可能因多种原因而损坏，例如环境因素（如电磁干扰、热量、辐射）、硬件故障（如电源波动、存储单元故障、地址线短路）或软件故障（其他软件错误地修改内存）。该模式解决了针对小型关键数据值集合识别数据损坏的问题。

6.2.3 模式结构

基本的模式结构如图6-2所示。由于这是一种详细设计模式，其作用范围主要在一个类内部。

6.2.4 协作角色

本节描述该模式的角色。

6.2.4.1 DataClient

`DataClient` 是一个用于设置或获取数据值的元素。

6.2.4.2 OnesCompProtectedDataElement

`OnesCompProtectedDataElement` 是一个将数据存储与能够检测位级数据损坏的服务相结合的类。该类将数据的值存储两次：一次以常规形式存储，一次以按位补码形式存储。当检索数据时，反转后的数据被重新反转，并与正常存储的值进行比较。如果值匹配，则返回该值；如果不匹配，该类提供一个函数来处理错误。错误处理程序的实际行为是系统和上下文特定的。

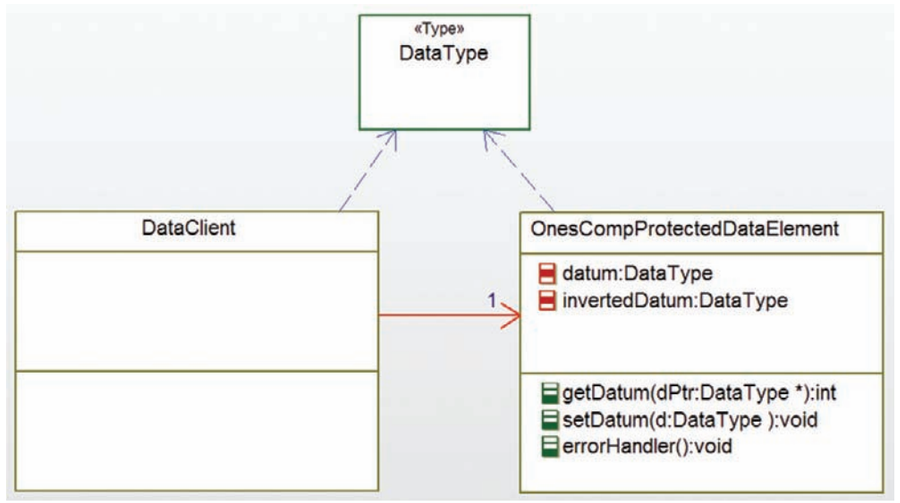


Figure 6-2: One's Complement Pattern

6.2.5 Consequences

This pattern uses twice the memory for storage of critical data; in addition, there is some performance hit on both data storage and retrieval. On the plus side, the pattern can identify errors due to EMI or due to stuck or shorted memory or address bits.

6.2.6 Implementation Strategies

This pattern is very easy to implement in C. For primitive types, the ~ operator computes the bit inversion. For structured types, you must iterate over the primitive parts of the data (see the example below).

6.2.7 Related Patterns

This pattern provides a very reliable way to identify faults that affect a single memory location. For very large data structures, the memory impact of complete data replication may be too large; in such cases, the Cyclic Redundancy Check (CRC) Pattern provides a reasonably reliable alternative.

6.2.8 Example

Figure 6-3 shows a simple application of this pattern. `OwnShipAttitude` stores the attitude (orientation, as specified in its attributes roll, pitch, and yaw) using the One's Complement Pattern. When the `getAttitude()` function is called, it reinverts the inverted copy and

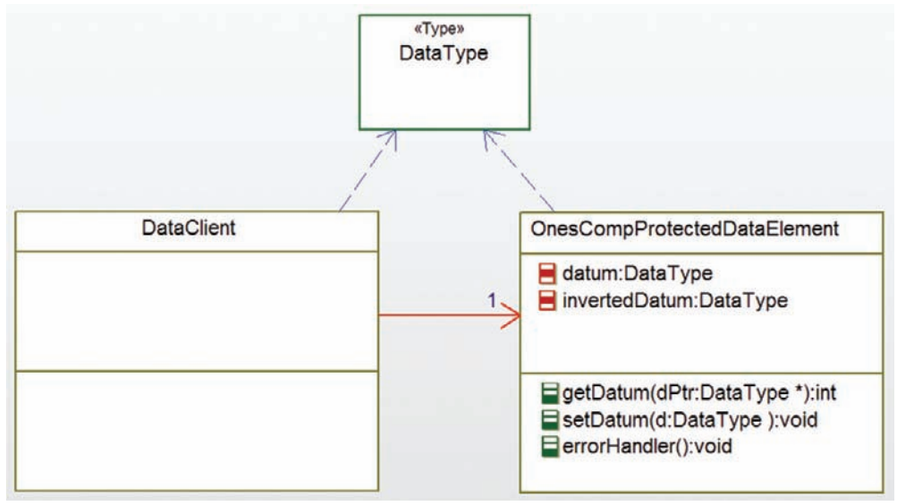


图 6-2：反码模式

6.2.5 结果

该模式使用两倍的内存来存储关键数据；此外，在数据存储和检索方面都会有一定的性能损失。有利的一面是，该模式可以识别由电磁干扰或由内存或地址位卡死或短路引起的错误。

6.2.6 实现策略

该模式在 C 语言中非常容易实现。对于基本类型，~运算符计算位反转。对于结构化类型，必须遍历数据的各个基本类型部分（参见下面的示例）。

6.2.7 相关模式

该模式提供了一种非常可靠的方法来识别影响单个内存位置的故障。对于非常大的数据结构，完整数据复制的内存影响可能过大；在这种情况下，循环冗余校验（CRC）模式提供了一种相当可靠的替代方案。

6.2.8 示例

图 6-3 展示了该模式的一个简单应用。`OwnShipAttitude` 使用反码模式存储姿态（即方向，由其属性横滚角、俯仰和偏航指定）。当调用`getAttitude()`函数时，它会将反转后的副本重新反转，并

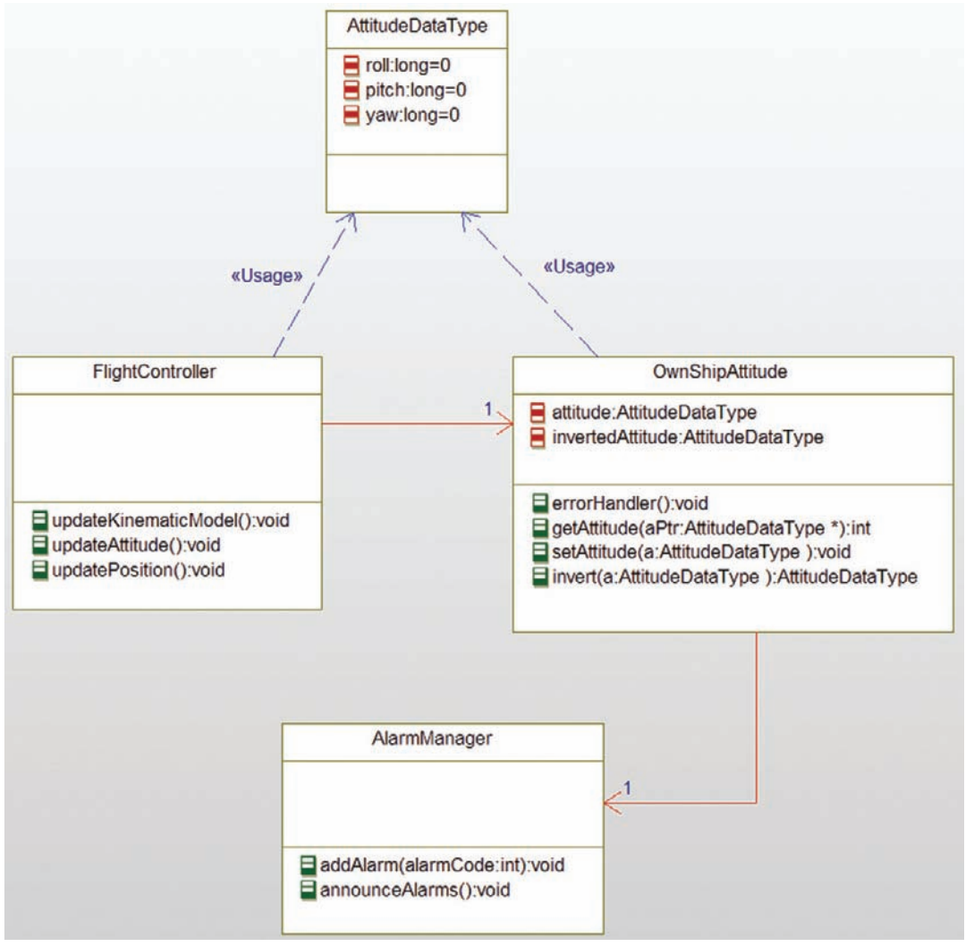


Figure 6-3: One's Complement example

compares it to normally stored values. If there is a difference, then the `errorHandler()` function is invoked which, in this case, calls the `addAlarm()` function of the `AlarmHandler`. If the values are the same, the value is returned to the client.

The code for the `OwnShipAttitude` (fulfilling the role of the `OnesCompProtectedDataElement` in the pattern) is shown in Code Listing 6-1 and Code Listing 6-2. Note that the error code passed to the alarm manager `ATTITUDE_MEMORY_FAULT` is simply a #define-d value.

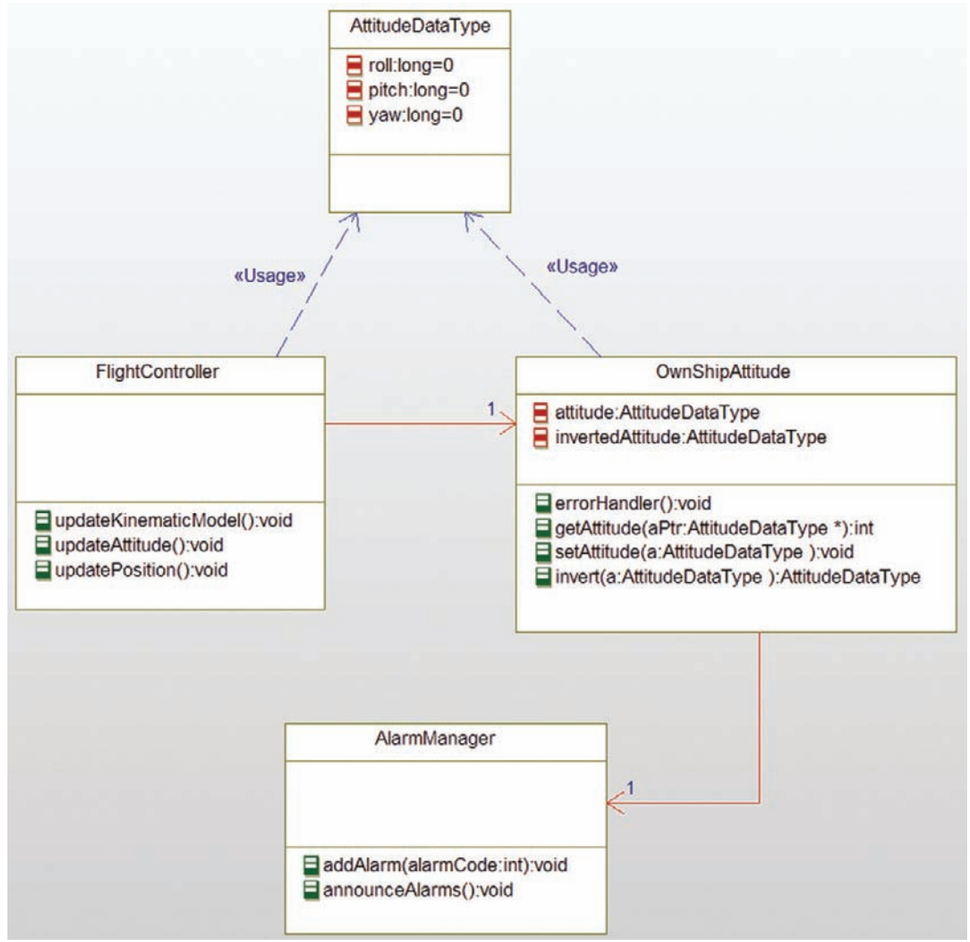


图6-3：反码示例

将其与正常存储的值进行比较。如果存在差异，则调用`errorHandler()`函数，在此情况下，该函数会调用`AlarmHandler`的`addAlarm()`函数。如果值相同，则将该值返回给客户端。

`OwnShipAttitude`的代码（在模式中履行`OnesCompProtectedDataElement`的角色）如代码清单 6-1 和代码清单 6-2 所示。请注意，传递给报警管理器的错误码`ATTITUDE_MEMORY_FAULT`只是一个 #define 的值。

```
#ifndef OwnShipAttitude_H
#define OwnShipAttitude_H

#include "AttitudeDataType.h"
struct AlarmManager;

typedef struct OwnShipAttitude OwnShipAttitude;
struct OwnShipAttitude {
  struct AttitudeDataType attitude;
  struct AttitudeDataType invertedAttitude;
  struct AlarmManager* itsAlarmManager;
};

void OwnShipAttitude_Init(OwnShipAttitude* const me);

void OwnShipAttitude_Cleanup(OwnShipAttitude* const me);

/* Operations */
void OwnShipAttitude_errorHandler(OwnShipAttitude* const me);

int OwnShipAttitude_getAttitude(OwnShipAttitude* const me, AttitudeDataType * aPtr);

AttitudeDataType OwnShipAttitude_invert(OwnShipAttitude* const me, AttitudeDataType a);

void OwnShipAttitude_setAttitude(OwnShipAttitude* const me, AttitudeDataType a);

struct AlarmManager* OwnShipAttitude_getItsAlarmManager(const OwnShipAttitude* const me);

void OwnShipAttitude_setItsAlarmManager(OwnShipAttitude* const
me, struct AlarmManager* p_AlarmManager);

OwnShipAttitude * OwnShipAttitude_Create(void);

void OwnShipAttitude_Destroy(OwnShipAttitude* const me);
#endif
```

Code Listing 6-1: OwnShipAttitude.h

```
#include "OwnShipAttitude.h"
#include "AlarmManager.h"

static void cleanUpRelations(OwnShipAttitude* const me);

void OwnShipAttitude_Init(OwnShipAttitude* const me) {
    AttitudeDataType_Init(&(me->attitude));
    AttitudeDataType_Init(&(me->invertedAttitude));
    me->itsAlarmManager = NULL;
}
void OwnShipAttitude_Cleanup(OwnShipAttitude* const me) {
```



```
#ifndef OwnShipAttitude_H #
define OwnShipAttitude_H

#include "AttitudeDataType.h"
struct AlarmManager;

typedef struct OwnShipAttitude OwnShipAttitude; struct
OwnShipAttitude { struct AttitudeDataType attitude; struct
AttitudeDataType invertedAttitude; struct AlarmManager*
itsAlarmManager; }; void OwnShipAttitude_Init(OwnShipAttitude* const
me); void OwnShipAttitude_Cleanup(OwnShipAttitude* const me); /* 操作
*/ void OwnShipAttitude_errorHandler(OwnShipAttitude* const me);

int OwnShipAttitude_getAttitude(OwnShipAttitude* const me, 姿态数据类型 * aPtr);

姿态数据类型 本舰姿态_invert(本舰姿态* const me, 姿态数据类型 a);

void OwnShipAttitude_setAttitude(OwnShipAttitude* const me, 姿态数据类型 a); struct Alar
mManager* OwnShipAttitude_getItsAlarmManager(const OwnShipAttitude* const me);

void OwnShipAttitude_setItsAlarmManager(OwnShipAttitude* const me,
struct AlarmManager* p_AlarmManager);

OwnShipAttitude * OwnShipAttitude_Create(void);

void OwnShipAttitude_Destroy(OwnShipAttitude* const me); #endif
```

代码清单 6-1: OwnShipAttitude.h

```
#include "OwnShipAttitude.h" #include "AlarmManager.h" static
void cleanUpRelations(OwnShipAttitude* const me); void
OwnShipAttitude_Init(OwnShipAttitude* const me) {
AttitudeDataType_Init(&(me->attitude)); AttitudeDataType_Init(&
(me->invertedAttitude)); me->itsAlarmManager = 空; } void
OwnShipAttitude_清理(OwnShipAttitude* const me) {
```

```
  cleanUpRelations(me);
}

void OwnShipAttitude_errorHandler(OwnShipAttitude* const me) {
  AlarmManager_addAlarm(me->itsAlarmManager,
ATTITUDE_MEMORY_FAULT);
}

int OwnShipAttitude_getAttitude(OwnShipAttitude* const me,
AttitudeDataType * aPtr) {
  AttitudeDataType ia = OwnShipAttitude_invert(me, me-  >invertedAttitude);

  if (me->attitude.roll == ia.roll && me->attitude.yaw == ia.yaw &&
      me->attitude.pitch == ia.pitch ) {
      *aPtr = me->attitude;
      return 1;
      }
  else {
      OwnShipAttitude_errorHandler(me);
      return 0;
      };
}

AttitudeDataType OwnShipAttitude_invert(OwnShipAttitude* const
me, AttitudeDataType a) {
  a.roll = ~a.roll;
  a.yaw = ~a.yaw;
  a.pitch = ~a.pitch;
  return a;
}

void OwnShipAttitude_setAttitude(OwnShipAttitude* const me,
AttitudeDataType a) {
  me->attitude = a;
  me->invertedAttitude = OwnShipAttitude_invert(me, a);
}

struct AlarmManager* OwnShipAttitude_getItsAlarmManager(const
OwnShipAttitude* const me) {
  return (struct AlarmManager*)me->itsAlarmManager;
}

void OwnShipAttitude_setItsAlarmManager(OwnShipAttitude* const
me, struct AlarmManager* p_AlarmManager) {
  me->itsAlarmManager = p_AlarmManager;
}

OwnShipAttitude * OwnShipAttitude_Create(void) {
  OwnShipAttitude* me = (OwnShipAttitude *)
malloc(sizeof(OwnShipAttitude));
```


cleanUpRelations(me); } void 本舰姿态_errorHandler(本舰姿态* const me) { AlarmManager_addAlarm(me->itsAlarmManager, 姿态_内存_故障); } int 本舰姿态_getAttitude(本舰姿态* const me, 姿态数据类型 * aPtr) { 姿态数据类型 ia = 本舰姿态_invert(me, me- >inverte

dAttitude);

if (me->attitude.roll == ia.roll && me->attitude.yaw == ia.yaw && me->attitude.pitch == ia.pitch) { *aPtr = me->attitude; return 1; } else { OwnShipAttitude_errorHandler(me); return 0; }; } AttitudeDataType OwnShipAttitude_invert(OwnShipAttitude* const me, AttitudeDataType a) { a.roll = ~a.roll; a.yaw = ~a.yaw; a.pitch = ~a.pitch; return a; } void OwnShipAttitude_setAttitude(OwnShipAttitude* const me, AttitudeDataType a) { me->attitude = a; me->invertedAttitude = OwnShipAttitude_invert(me, a); } struct AlarmManager* OwnShipAttitude_getItsAlarmManager(const OwnShipAttitude* const me) { return (struct AlarmManager*)me->itsAlarmManager; } void OwnShipAttitude_setItsAlarmManager(OwnShipAttitude* const me, struct AlarmManager* p_AlarmManager) { me->itsAlarmManager = p_AlarmManager; }

OwnShipAttitude * OwnShipAttitude_Create(void) { OwnShipAttitude* me = (OwnShipAttitude *) malloc(sizeof(OwnShipAttitude));

```
  if(me!=NULL)
    OwnShipAttitude_Init(me);
  return me;
}

void OwnShipAttitude_Destroy(OwnShipAttitude* const me) {
  if(me!=NULL)
    OwnShipAttitude_Cleanup(me);
  free(me);
}

static void cleanUpRelations(OwnShipAttitude* const me) {
  if(me->itsAlarmManager != NULL)
      me->itsAlarmManager = NULL;
}
```

Code Listing 6-2: OwnShipAttitude.c

6.3 CRC Pattern

The Cyclic Redundancy Check (CRC) Pattern calculates a fixed-size error-detection code based on a cyclic polynomial that can detect corruption in data sets far larger than the size of the code.

6.3.1 Abstract

The CRC Pattern computes a fixed-length binary code, called a CRC value, on your data to detect whether or not they have been corrupted. This code is stored in addition to the data values and is set when data are updated and checked when data are read.

A detailed explanation of the mathematics is beyond the scope of this book[4] but CRCs are both common and useful in practice. CRCs are characterized by the bit-length of the polynomials used to compute them. While algorithmic computation can be complex and time consuming, table-driven algorithms are quite time efficient. CRCs provide good detection of single and multiple bit errors for arbitrary length data fields and so are good for large data structures.

6.3.2 Problem

This pattern addresses the problem that variables may be corrupted by a variety of causes such as environmental factors (e.g., EMI, heat, radiation), hardware faults (e.g., power fluctuation, memory cell faults, address line shorts), or software faults (other software erroneously modifying memory). This pattern addresses the problem of data corruption in large data sets.

[4] See for example, "A Painless Guide to CRC Error Detection Algorithms" at http://www.ross.net/crc/download/crc_v3.txt for more information.

if(me!=NULL) 本舰姿态_Init(me); return me; } void 本舰姿态_Destroy(本舰姿态* const me) { if(me!=NULL) 本舰姿态_Cleanup(me); free(me); } static void cleanUpRelations(本舰姿态* const me) { if(me->itsAlarmManager != NULL) me->itsAlarmManager = NULL; }

代码清单 6-2：OwnShipAttitude.c

6.3 CRC模式

循环冗余校验（CRC）模式基于循环多项式计算固定大小的错误检测码，能够检测远大于该码大小的数据集中的损坏。

6.3.1 摘要

CRC模式对数据计算固定长度的二进制码（称为CRC值），以检测数据是否已损坏。该码与数据值一同存储，在数据更新时设置，在数据读取时校验。

对数学细节的详细解释超出了本书的范围[4] 但CRC在实践中既常见又实用。CRC的特征在于用于计算它的多项式的位长。虽然算法计算可能复杂且耗时，但表驱动算法在时间上相当高效。CRC能够很好地检测任意长度数据字段中的单个和多个位错误，因此适用于大型数据结构。

6.3.2 问题

该模式解决的问题是：变量可能因多种原因而损坏，例如环境因素（如电磁干扰、热量、辐射）、硬件故障（如电源波动、存储单元故障、地址线短路）或软件故障（其他软件错误地修改内存）。该模式解决的是大型数据集中的数据损坏问题。

[4] 例如， 请 参阅 "A Painless Guide to CRC Error Detection Algorithms"， 网址为 http://www.ross.net/crc/download/ crc v3.txt， 以 获取更多信息。_

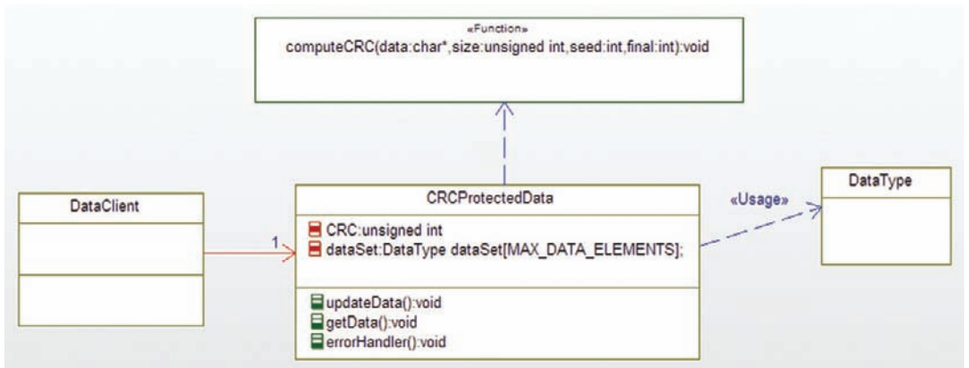


Figure 6-4: CRC Pattern

6.3.3 Pattern Structure

The CRC Pattern structure is shown in Figure 6-4. The `CRCProtectedData` class uses a generic `computeCRC()` function to calculate the CRC over its data. While data are shown as an array, it can be any contiguous block of data.

6.3.4 Collaboration Roles

This section describes the roles for this pattern.

6.3.4.1 computeCRCData Function

This is a general function for the computation of a CRC of a specific polynomial bit-length. Common bit lengths include 12, 16, and 32. In practice, this is usually implemented with a table-driven algorithm for computational efficiency.

6.3.4.2 CRCProtectedData

This is the class that owns and protects the data. When data are stored, the CRC is calculated. When data are read, the CRC is recalculated and compared to the stored CRC; if the CRCs differ, then the `errorHandler()` is called, otherwise the retrieved data are returned to the client.

6.3.4.3 DataClient

The `DataClient` is the element that stores and/or retrieves data from the `CRCProtectedData` class.

6.3.4.4 DataType

The is the base data type of the data structure owned by the `CRCProtectedDataClass`. This might be a primitive type, such as int or double, or it may be a complex data type such as an array or struct.

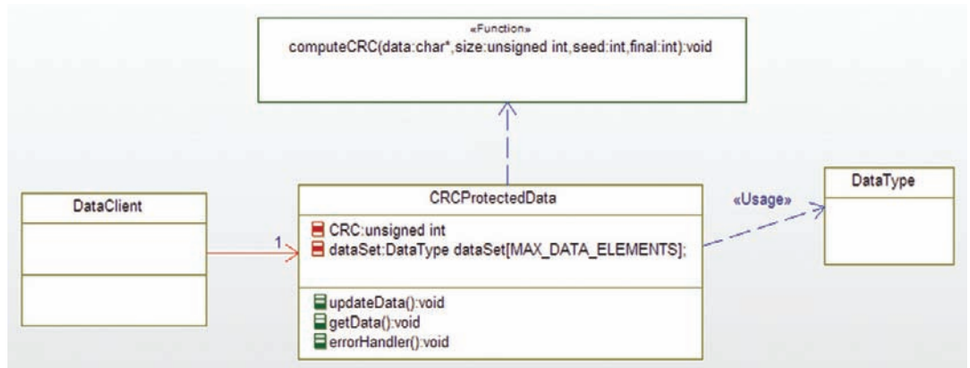


图6-4：CRC模式

6.3.3 模式结构

CRC 模式 的 结构 如 图 6-4 所示 。`CRCProtectedData` 类使用通用的 `computeCRC()` 函数来计算其数据上的 CRC。虽然数据以数组形式展示，但它可以是任何连续的数据块。

6.3.4 协作角色

本节描述该模式的角色。

6.3.4.1 computeCRCData函数

这是一个用于计算特定多项式位长的CRC的通用函数。常见的位长包括12、16和32。在实践中，为了计算效率，通常使用表驱动算法来实现。

6.3.4.2 CRCProtectedData

这是拥有并保护数据的类。当存储数据时，计算CRC。当读取数据时，重新计算CRC并与存储的CRC进行比较；如果CRC不同，则调用`errorHandler()`，否则将检索到的数据返回给客户端。

6.3.4.3 DataClient

`DataClient` 是从`CRCProtectedData`类中存储和/或检索数据的元素。

6.3.4.4 DataType

这是由`CRCProtectedDataClass`所拥有的数据结构的基础数据类型。它可以是基本类型，例如int或double，也可以是复杂数据类型，例如数组或结构体。

6.3.5 Consequences

With a little bit of memory used by the CRC data table and a relatively small amount of computation time to compute the polynomial, this pattern provides good detection of single (and small number of) bit errors. It is used often in communications messaging because such connections tend to be extremely unreliable. In addition, it can be used for in-memory error detection for harsh EMI environments or for mission-critical data. It is excellent for large data sets with relatively few bit errors.

6.3.6 Implementation Strategies

As mentioned, the table-driven algorithm is almost always used because it provides good performance with only a minor memory space penalty. If memory is tight, the polynomial can be computed algorithmically but this has a significant impact on run-time performance.

6.3.7 Related Patterns

The One's Complement Pattern replicates the data in a bit-inverted form, and so is useful for small data sets. This pattern is better for large data sets because it has a smaller memory impact.

6.3.8 Example

Figure 6-5 shows a medical application of this pattern. In this case, several clients set or get patient data, which is defined by the `PatientDataType`. The `PatientData` class provides functions to set and get the data, protecting them via calls to the `CRCCalculator`. A single CRC protects the entire patient data set.

The code listing in Code Listing 6-3 is adapted from free software written by Jack Klein, and released under the GNU General Public License Agreement[5]. It computes a CRC-16 but other polynomial bit-lengths are available for use.

```
/* crcccitt.c - a demonstration of look up table based CRC
 *              computation using the non-reversed CCITT_CRC
 *              polynomial 0x1021 (truncated)
 *
 * Copyright (C) 2000 Jack Klein
 *              Macmillan Computer Publishing
 *
```

[5] For information concerning usage of this code, contact the Free Software Foundation, Inc., 675 Mass Ave, Cambridge, MA 02139, USA

6.3.5 结果

该模式仅需CRC数据表占用少量内存，以及相对较少的计算多项式的时间，就能很好地检测单个（及少量）位错误。它常用于通信消息传递，因为此类连接往往极其不可靠。此外，它还可用于恶劣电磁干扰环境或关键任务数据的内存内错误检测。它非常适合位错误相对较少的大型数据集。

6.3.6 实现策略

如前所述，表驱动算法几乎总是被采用，因为它以较小的内存空间开销提供了良好的性能。如果内存紧张，可以通过算法计算多项式，但这会对运行时性能产生显著影响。

6.3.7 相关模式

反码模式以位反转形式复制数据，因此适用于小数据集。此模式更适合大型数据集，因为它的内存影响较小。

6.3.8 示例

图 6-5 展示了该模式的一个医疗应用。在此情况下，多个客户端设置或获取患者数据，这些数据由`PatientDataType`定义。`PatientData`类提供了设置和获取数据的函数，并通过调用`CRCCalculator`来保护这些数据。单个CRC保护整个患者数据集。

代码清单6-3中的代码改编自Jack Klein编写的自由软件，并根据GNU通用公共许可证协议[5]发布。它计算CRC-16，但也可以使用其他多项式位长。

```
/* crcccitt.c - 演示基于查找表的CRC计算，使用非反转CCITT_CRC多项式
0x1021（截断） * * 版权所有 (C) 2000 Jack Klein * Macmillan Computer
Publishing *
```

[5] 有关此代码使用的信息，请联系自由软件基金会，地址：675 Mass Ave, Cambridge, MA 02139, USA

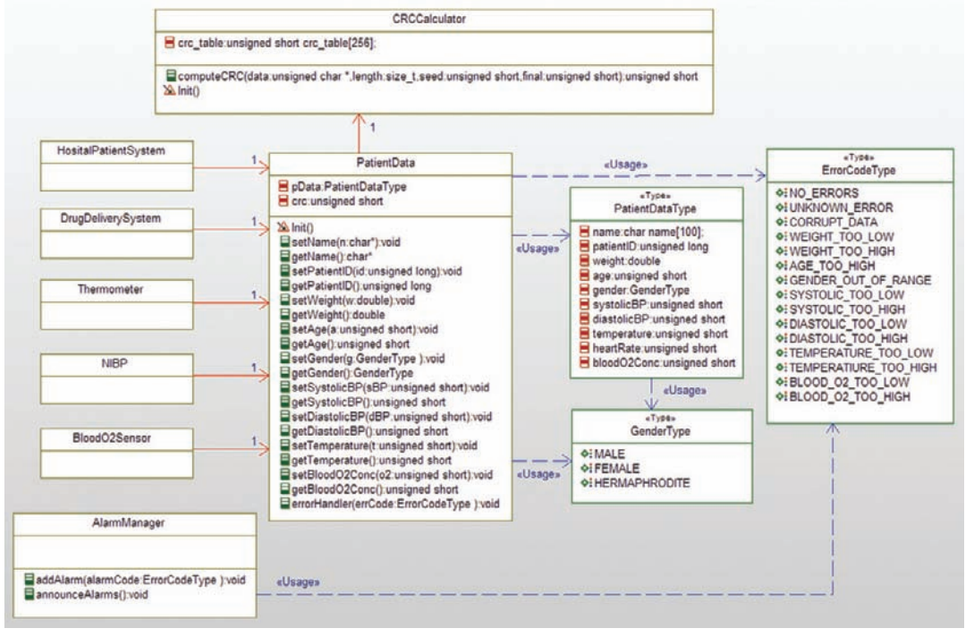


Figure 6-5: CRC example

```
* This program is free software; you can redistribute it
* and/or modify it under the terms of the GNU General
* Public License as published by the Free Software
* Foundation; either version 2 of the License, or
* (at your option) any later version.
*
* This program is distributed in the hope that it will
* be useful, but WITHOUT ANY WARRANTY; without even the
* implied warranty of MERCHANTABILITY or FITNESS FOR A
* PARTICULAR PURPOSE. See the GNU General Public License
* for more details.
*
* You should have received a copy of the GNU General
* Public License along with this program; if not, write
* to the Free Software Foundation, Inc., 675 Mass Ave,
* Cambridge, MA 02139, USA.
*
* Jack Klein may be contacted by email at:
*   The_C_Guru@yahoo.com
*
```

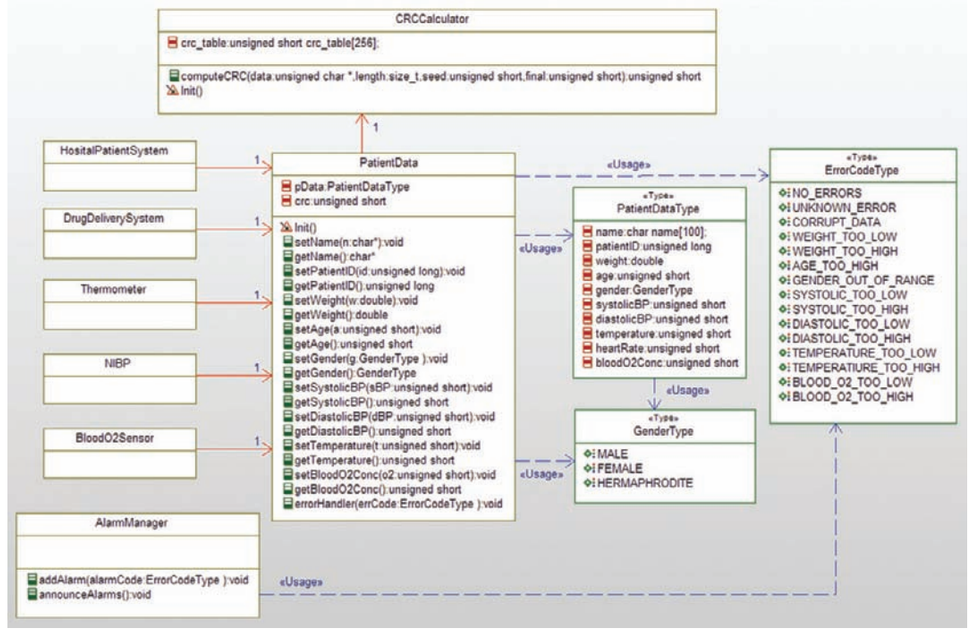



图6-5：CRC示例

```
* 本程序是自由软件；你可以根据自由软件基金会发布的GNU通用公共许可证条款 * 重新分发和/或修改它；许可证可以是第2版， * 或（根据你的选择）任何更高版本。
*
* 本程序的发布是希望其有用，但不提供任何担保；* 甚至不提供适销性或特定用途适用性的默示担保。* 详情请参阅GNU通用公共许可证。
*
* * 您应当已随本程序收到 GNU 通用公共许可证的副本；如果没有，请 * 写信至自由软件基金会，Inc.，675 Mass Ave, * Cambridge, MA 02139, USA。 * * 可通过电子邮件联系 Jack Klein： * The_C_Guru@yahoo.com *
```

```
* Modified slightly by Bruce Powel Douglass, 2010
*/

/*# # dependency stdio */
#include <stdio.h>
/*# # dependency stdlib */
#include <stdlib.h>
/*# # dependency string */
#include <string.h>

static unsigned short crc_table[256] = {
  0x0000, 0x1021, 0x2042, 0x3063, 0x4084, 0x50a5,
  0x60c6, 0x70e7, 0x8108, 0x9129, 0xa14a, 0xb16b,
  0xc18c, 0xd1ad, 0xe1ce, 0xf1ef, 0x1231, 0x0210,
  0x3273, 0x2252, 0x52b5, 0x4294, 0x72f7, 0x62d6,
  0x9339, 0x8318, 0xb37b, 0xa35a, 0xd3bd, 0xc39c,
  0xf3ff, 0xe3de, 0x2462, 0x3443, 0x0420, 0x1401,
  0x64e6, 0x74c7, 0x44a4, 0x5485, 0xa56a, 0xb54b,
  0x8528, 0x9509, 0xe5ee, 0xf5cf, 0xc5ac, 0xd58d,
  0x3653, 0x2672, 0x1611, 0x0630, 0x76d7, 0x66f6,
  0x5695, 0x46b4, 0xb75b, 0xa77a, 0x9719, 0x8738,
  0xf7df, 0xe7fe, 0xd79d, 0xc7bc, 0x48c4, 0x58e5,
  0x6886, 0x78a7, 0x0840, 0x1861, 0x2802, 0x3823,
  0xc9cc, 0xd9ed, 0xe98e, 0xf9af, 0x8948, 0x9969,
  0xa90a, 0xb92b, 0x5af5, 0x4ad4, 0x7ab7, 0x6a96,
  0x1a71, 0x0a50, 0x3a33, 0x2a12, 0xdbfd, 0xcbdc,
  0xfbbf, 0xeb9e, 0x9b79, 0x8b58, 0xbb3b, 0xab1a,
  0x6ca6, 0x7c87, 0x4ce4, 0x5cc5, 0x2c22, 0x3c03,
  0x0c60, 0x1c41, 0xedae, 0xfd8f, 0xcdec, 0xddcd,
  0xad2a, 0xbd0b, 0x8d68, 0x9d49, 0x7e97, 0x6eb6,
  0x5ed5, 0x4ef4, 0x3e13, 0x2e32, 0x1e51, 0x0e70,
  0xff9f, 0xefbe, 0xdfdd, 0xcffc, 0xbf1b, 0xaf3a,
  0x9f59, 0x8f78, 0x9188, 0x81a9, 0xb1ca, 0xa1eb,
  0xd10c, 0xc12d, 0xf14e, 0xe16f, 0x1080, 0x00a1,
  0x30c2, 0x20e3, 0x5004, 0x4025, 0x7046, 0x6067,
  0x83b9, 0x9398, 0xa3fb, 0xb3da, 0xc33d, 0xd31c,
  0xe37f, 0xf35e, 0x02b1, 0x1290, 0x22f3, 0x32d2,
  0x4235, 0x5214, 0x6277, 0x7256, 0xb5ea, 0xa5cb,
  0x95a8, 0x8589, 0xf56e, 0xe54f, 0xd52c, 0xc50d,
  0x34e2, 0x24c3, 0x14a0, 0x0481, 0x7466, 0x6447,
  0x5424, 0x4405, 0xa7db, 0xb7fa, 0x8799, 0x97b8,
  0xe75f, 0xf77e, 0xc71d, 0xd73c, 0x26d3, 0x36f2,
  0x0691, 0x16b0, 0x6657, 0x7676, 0x4615, 0x5634,
  0xd94c, 0xc96d, 0xf90e, 0xe92f, 0x99c8, 0x89e9,
  0xb98a, 0xa9ab, 0x5844, 0x4865, 0x7806, 0x6827,
  0x18c0, 0x08e1, 0x3882, 0x28a3, 0xcb7d, 0xdb5c,
  0xeb3f, 0xfb1e, 0x8bf9, 0x9bd8, 0xabbb, 0xbb9a,
  0x4a75, 0x5a54, 0x6a37, 0x7a16, 0x0af1, 0x1ad0,
  0x2ab3, 0x3a92, 0xfd2e, 0xed0f, 0xdd6c, 0xcd4d,
  0xbdaa, 0xad8b, 0x9de8, 0x8dc9, 0x7c26, 0x6c07,
```



```
* 由 Bruce Powel Douglass 于 2010 年稍作修改 */

/*# # 依赖 stdio */ #include <
stdio.h>/*# # 依赖 stdlib */ #
include <stdlib.h>/*# # 依赖
string */ #include <string.h>

static unsigned short crc_table[256] = { 0x0000,
0x1021, 0x2042, 0x3063, 0x4084, 0x50a5, 0x60c6, 0x70e7,
0x8108, 0x9129, 0xa14a, 0xb16b, 0xc18c, 0xd1ad, 0xe1ce,
0xf1ef, 0x1231, 0x0210, 0x3273, 0x2252, 0x52b5, 0x4294,
0x72f7, 0x62d6, 0x9339, 0x8318, 0xb37b, 0xa35a,
0xd3bd, 0xc39c, 0xf3ff, 0xe3de, 0x2462, 0x3443, 0x0420,
0x1401, 0x64e6, 0x74c7, 0x44a4, 0x5485, 0xa56a, 0x
b54b, 0x8528, 0x9509, 0xe5ee, 0xf5cf, 0xc5ac, 0xd58d,
0x3653, 0x2672, 0x1611, 0x0630, 0x76d7, 0x66f6, 0x5695,
0x46b4, 0xb75b, 0xa77a, 0x9719, 0x8738, 0xf7df, 0xe7fe,
0xd79d, 0xc7bc, 0x48c4, 0x58e5, 0x6886, 0x78a7, 0x0840,
0x1861, 0x2802, 0x3823, 0xc9cc, 0xd9ed, 0xe98e, 0xf9af,
0x8948, 0x9969, 0xa90a, 0xb92b, 0x5af5, 0x4ad4, 0x7a
b7, 0x6a96, 0x1a71, 0x0a50, 0x3a33, 0x2a12, 0xdbfd,
0xcbdc, 0xfbbf, 0xeb9e, 0x9b79, 0x8b58, 0xbb3b, 0xa
b1a, 0x6ca6, 0x7c87, 0x4ce4, 0x5cc5, 0x2c22, 0x3c03,
0x0c60, 0x1c41, 0xedae, 0xfd8f, 0xcdec, 0xddcd, 0xad2a,
0xbd0b, 0x8d68, 0x9d49, 0x7e97, 0x6eb6, 0x5ed5,
0x4ef4, 0x3e13, 0x2e32, 0x1e51, 0x0e70, 0xff9f, 0xefbe,
0xdfdd, 0xcffc, 0xbf1b, 0xaf3a, 0x9f59, 0x8f78, 0x9188,
0x81a9, 0xb1ca, 0xa1eb, 0xd10c, 0xc12d, 0xf14e, 0xe16f,
0x1080, 0x00a1, 0x30c2, 0x20e3, 0x5004, 0x4025, 0x7046,
0x6067, 0x83b9, 0x9398, 0xa3fb, 0xb3da, 0xc33d, 0xd31c,
0xe37f, 0xf35e, 0x02b1, 0x1290, 0x22f3, 0x32d2, 0x4235,
0x5214, 0x6277, 0x7256, 0xb5ea, 0xa5cb, 0x95a8, 0x8589,
0xf56e, 0xe54f, 0xd52c, 0xc50d, 0x34e2, 0x24c3, 0x14a0,
0x0481, 0x7466, 0x6447, 0x5424, 0x4405, 0xa7db, 0xb7fa,
0x8799, 0x97b8, 0xe75f, 0xf77e, 0xc71d, 0xd73c, 0x26d3,
0x36f2, 0x0691, 0x16b0, 0x6657, 0x7676, 0x4615, 0x5634,
0xd94c, 0xc96d, 0xf90e, 0xe92f, 0x99c8, 0x89e9, 0xb98a,
0xa9ab, 0x5844, 0x4865, 0x7806, 0x6827, 0x18c0, 0x08e1,
0x3882, 0x28a3, 0xcb7d, 0xdb5c, 0xeb3f, 0xfb1e, 0x8bf9,
0x9bd8, 0xabbb, 0xbb9a, 0x4a75, 0x5a54, 0x6a37,
0x7a16, 0x0af1, 0x1ad0, 0x2ab3, 0x3a92, 0xfd2e, 0xed0f,
0xdd6c, 0xcd4d, 0xbdaa, 0xad8b, 0x9de8, 0x8dc9,
0x7c26, 0x6c07,
```

```
  0x5c64, 0x4c45, 0x3ca2, 0x2c83, 0x1ce0, 0x0cc1,
  0xef1f, 0xff3e, 0xcf5d, 0xdf7c, 0xaf9b, 0xbfba,
  0x8fd9, 0x9ff8, 0x6e17, 0x7e36, 0x4e55, 0x5e74,
  0x2e93, 0x3eb2, 0x0ed1, 0x1ef0
};

unsigned short computeCRC(unsigned char * data, size_t length, unsigned short seed,
unsigned short final) {
  size_t count;
  unsigned int crc = seed;
  unsigned int temp;

  for (count = 0; count < length; ++count)
  {
    temp = (*data++ ^ (crc >> 8)) & 0xff;
    crc = crc_table[temp] ^ (crc << 8);
  }
  return (unsigned short)(crc ^ final);
}
```

Code Listing 6-3: CRCCalculator.c

The data types used in the example appear in Code Listing 6-4. Note that `ErrorCodeType` includes other errors, such as data element out of range, as is common in such applications.

```
#ifndef CRCExample_H
#define CRCExample_H

struct AlarmManager;
struct BloodO2Sensor;
struct CRCBuilderClass;
struct CRCClient2;
struct DrugDeliverySystem;
struct HositalPatientSystem;
struct NIBP;
struct PatientData;
struct Thermometer;

typedef enum ErrorCodeType {
  NO_ERRORS,
  UNKNOWN_ERROR,
  CORRUPT_DATA,
  WEIGHT_TOO_LOW,
  WEIGHT_TOO_HIGH,
  AGE_TOO_HIGH,
  GENDER_OUT_OF_RANGE,
  SYSTOLIC_TOO_LOW,
```


0x5c64, 0x4c45, 0x3ca2, 0x2c83, 0x1ce0, 0x0cc1, 0xef1f, 0xff3e, 0xcf5d, 0xdf7c, 0xaf9b, 0xbfba, 0x8fd9, 0x9ff8, 0x6e17, 0x7e36, 0x4e55, 0x5e74, 0x2e93, 0x3eb2, 0x0ed1, 0x1ef0 }; unsigned short computeCRC(unsigned char * 数据, size_t length, unsigned short seed, unsigned short final) { size_t count; unsigned int crc = seed; unsigned int temp; for (count = 0; count < length; ++count) { temp = (* 数据++ ^ (crc >> 8)) & 0xff; crc = crc_table[temp] ^ (crc << 8); } return (unsigned short)(crc ^ final); }

代码清单 6-3：CRCCalculator.c

示例中使用的数据类型见代码清单 6-4。请注意，`ErrorCodeType`包含其他错误，例如数据元素超出范围，这在此类应用中很常见。

#ifndef CRCExample_H #define CRCExample_H struct AlarmManager; struct BloodO2Sensor; struct CRCBuilderClass; struct CRCClient2; struct DrugDeliverySystem; struct HositalPatientSystem; struct NIBP; struct PatientData; struct Thermometer; typedef enum ErrorCodeType { NO_ERRORS, UNKNOWN_ERROR, CORRUPT_DATA, WEIGHT_TOO_LOW, WEIGHT_TOO_HIGH, AGE_TOO_HIGH, GENDER_OUT_OF_RANGE, SYSTOLIC_TOO_LOW,

```
  SYSTOLIC_TOO_HIGH,
  DIASTOLIC_TOO_LOW,
  DIASTOLIC_TOO_HIGH,
  TEMPERATURE_TOO_LOW,
  TEMPERATIURE_TOO_HIGH,
  BLOOD_O2_TOO_LOW,
  BLOOD_O2_TOO_HIGH
} ErrorCodeType;

typedef enum GenderType {
  MALE,
  FEMALE,
  HERMAPHRODITE
} GenderType;

typedef struct PatientDataType {
  unsigned short age;
  unsigned short bloodO2Conc;
  unsigned short diastolicBP;
  GenderType gender;
  unsigned short heartRate;
  char name[100];
  unsigned long patientID;
  unsigned short systolicBP;
  unsigned short temperature;
  double weight;
} PatientDataType;

#endif
```

Code Listing 6-4: CRCExample.c

Of course the main action is in the `PatientData` class code, shown in Code Listing 6-5 and Code Listing 6-6.

```
#ifndef PatientData_H
#define PatientData_H

#include "CRCExample.h"
struct AlarmManager;

typedef struct PatientData PatientData;
struct PatientData {
  PatientDataType pData;
  unsigned short crc;
  struct AlarmManager* itsAlarmManager;
};
```



```
    SYSTOLIC_TOO_HIGH,
DIASTOLIC_TOO_LOW,
DIASTOLIC_TOO_HIGH,
TEMPERATURE_TOO_LOW,
TEMPERATIURE_TOO_HIGH,
BLOOD_O2_TOO_LOW,
BLOOD O2 TOO HIGH_  _    _}
ErrorCodeType;

typedef enum GenderType {
MALE, FEMALE,
HERMAPHRODITE }
GenderType;


typedef struct PatientDataType {
unsigned short age; unsigned short
bloodO2Conc; unsigned short
diastolicBP; GenderType gender;
unsigned short heartRate; char
name[100]; unsigned long
patientID; unsigned short systolic
BP; unsigned short temperature;
double weight; } PatientDataType;

#endif
```

代码清单 6-4：CRCExample.c

当然，主要动作在 `PatientData` 类代码中，如代码清单 6-5 和代码清单 6-6 所示。

```
#ifndef PatientData_H #
define PatientData_H

#include "CRCExample.h"
struct AlarmManager;

typedef struct PatientData PatientData; struct
PatientData { PatientDataType pData;
unsigned short crc; struct AlarmManager*
itsAlarmManager; };
```

```
void PatientData_Init(PatientData* const me);
void PatientData_Cleanup(PatientData* const me);

/* Operations */

void PatientData_errorHandler(PatientData* const me,
ErrorCodeType errCode);

unsigned short PatientData_getAge(PatientData* const me);

unsigned short PatientData_getBloodO2Conc(PatientData* const me);

unsigned short PatientData_getDiastolicBP(PatientData* const me);

GenderType PatientData_getGender(PatientData* const me);

char* PatientData_getName(PatientData* const me);

unsigned long PatientData_getPatientID(PatientData* const me);

unsigned short PatientData_getSystolicBP(PatientData* const me);

unsigned short PatientData_getTemperature(PatientData* const me);

double PatientData_getWeight(PatientData* const me);

void PatientData_setAge(PatientData* const me, unsigned short a);

void PatientData_setBloodO2Conc(PatientData* const me, unsigned short o2);

void PatientData_setDiastolicBP(PatientData* const me, unsigned short dBP);

void PatientData_setGender(PatientData* const me, GenderType g);

void PatientData_setName(PatientData* const me, char* n);

void PatientData_setPatientID(PatientData* const me, unsigned long id);

void PatientData_setSystolicBP(PatientData* const me, unsigned short sBP);

void PatientData_setTemperature(PatientData* const me, unsigned short t);

void PatientData_setWeight(PatientData* const me, double w);

int PatientData_checkData(PatientData* const me);

struct AlarmManager* PatientData_getItsAlarmManager(const PatientData* const me);
```


void PatientData_Init(PatientData* const me); void PatientData_Cleanup(PatientData* const me); /* 操作 */ void PatientData_errorHandler(PatientData* const me, ErrorCodeType errCode); unsigned short PatientData_getAge(PatientData* const me); unsigned short PatientData_getBloodO2Conc(PatientData* const me); unsigned short PatientData_getDiastolicBP(PatientData* const me); GenderType PatientData_getGender(PatientData* const me); char* PatientData_getName(PatientData* const me); unsigned long PatientData_getPatientID(PatientData* const me); unsigned short PatientData_getSystolicBP(PatientData* const me); unsigned short PatientData_getTemperature(PatientData* const me); double PatientData_getWeight(PatientData* const me); void PatientData_setAge(PatientData* const me, unsigned short a); void PatientData_setBloodO2Conc(PatientData* const me, unsigned short o2); void PatientData_setDiastolicBP(PatientData* const me, unsigned short dBP); void PatientData_setGender(PatientData* const me, GenderType g); void PatientData_setName(PatientData* const me, char* n); void PatientData_setPatientID(PatientData* const me, unsigned long id); void PatientData_setSystolicBP(PatientData* const me, unsigned short sBP); void PatientData_setTemperature(PatientData* const me, unsigned short t); void PatientData_setWeight(PatientData* const me, double w); int PatientData_checkData(PatientData* const me); struct AlarmManager* PatientData_getItsAlarmManager(const PatientData* const me);

```
void PatientData_setItsAlarmManager(PatientData* const me, struct
AlarmManager* p_AlarmManager);

PatientData * PatientData_Create(void);

void PatientData_Destroy(PatientData* const me);
#endif
```

Code Listing 6-5: PatientData.h

```
#include "PatientData.h"
#include "AlarmManager.h"
#include <string.h>
#include <stdio.h>

static void cleanUpRelations(PatientData* const me);

void PatientData_Init(PatientData* const me) {
  me->itsAlarmManager = NULL;
  strcpy(me->pData.name, "");
  me->pData.patientID = 0;
  me->pData.weight = 0;
  me->pData.age = 0;
  me->pData.gender = HERMAPHRODITE;
  me->pData.systolicBP = 0;
  me->pData.diastolicBP = 0;
  me->pData.temperature = 0;
  me->pData.heartRate = 0;
  me->pData.bloodO2Conc = 0;
  me->crc = computeCRC((unsigned char *)&me->pData, sizeof(me->pData), 0xffff, 0);
  }
}

void PatientData_Cleanup(PatientData* const me) {
  cleanUpRelations(me);
}

int PatientData_checkData(PatientData* const me) {
  unsigned short result = computeCRC((unsigned char *)&me->pData,
sizeof(me->pData), 0xffff, 0);
  printf("computed CRC = %d, stored CRC = %d\n", result, me->crc);
  return result == me->crc;
}

void PatientData_errorHandler(PatientData* const me,
ErrorCodeType errCode) {
  AlarmManager_addAlarm(me->itsAlarmManager, errCode);
}
```



```
void PatientData_setItsAlarmManager(PatientData* const me, struct
AlarmManager* p_AlarmManager);

PatientData * PatientData_Create(void);

void PatientData_Destroy(PatientData* const me); #endif
```

代码清单 6-5：PatientData.h

```
#include "PatientData.h" #
include "AlarmManager.h" #
include <string.h>#include <
stdio.h>

static void cleanUpRelations(PatientData* const me);

void PatientData_Init(PatientData* const me) { me->itsAlarmManager = 空; strcpy(me->
pData.name, "");  me->pData.patientID = 0; me->pData.weight = 0; me->pData.age = 0; me->
pData.gender = HERMAPHRODITE; me->pData.systolicBP = 0; me->pData.diastolicBP = 0; me->
pData.temperature = 0; me->pData.heartRate = 0; me->pData.bloodO2Conc = 0; me->crc =
computeCRC((unsigned char *)&me->pData, sizeof(me->pData), 0xffff, 0); } }

void PatientData_Cleanup(PatientData* const me) {
cleanUpRelations(me); }

int PatientData_checkData(PatientData* const me) { unsigned short result =
computeCRC((无符号字符 *)&me->pData, sizeof(me->pData), 0xffff, 0); printf("
computed CRC = %d, stored CRC = %d\n", result, me->crc); return result ==
me->crc; }

void PatientData_errorHandler(PatientData* const me,
ErrorCodeType errCode) { AlarmManager_addAlarm(me->
itsAlarmManager, errCode); }
```

```
unsigned short PatientData_getAge(PatientData* const me) {
  if (PatientData_checkData(me))
      return me->pData.age;
  else {
      PatientData_errorHandler(me, CORRUPT_DATA);
      return 0;
      };
}

unsigned short PatientData_getBloodO2Conc(PatientData* const me)
{
  if (PatientData_checkData(me))
      return me->pData.bloodO2Conc;
  else {
      PatientData_errorHandler(me, CORRUPT_DATA);
      return 0;
      };
}

unsigned short PatientData_getDiastolicBP(PatientData* const me)
{
  if (PatientData_checkData(me))
      return me->pData.diastolicBP;
  else {
      PatientData_errorHandler(me, CORRUPT_DATA);
      return 0;
      };
}

GenderType PatientData_getGender(PatientData* const me) {
  if (PatientData_checkData(me))
      return me->pData.gender;
  else {
      PatientData_errorHandler(me, CORRUPT_DATA);
      return 0;
      };
}

char* PatientData_getName(PatientData* const me) {
  if (PatientData_checkData(me))
      return me->pData.name;
  else {
      PatientData_errorHandler(me, CORRUPT_DATA);
      return 0;
      };
}

unsigned long PatientData_getPatientID(PatientData* const me) {
  if (PatientData_checkData(me))
      return me->pData.patientID;
```


unsigned short PatientData_getAge(PatientData* const me) { if (PatientData_checkData(me)) return me->pData.age; else { PatientData_errorHandler(me, CORRUPT_DATA); return 0; }; } unsigned short PatientData_getBloodO2Conc(PatientData* const me) { if (PatientData_checkData(me)) return me->pData.bloodO2Conc; else { PatientData_errorHandler(me, CORRUPT_DATA); return 0; }; } unsigned short PatientData_getDiastolicBP(PatientData* const me) { if (PatientData_checkData(me)) return me->pData.diastolicBP; else { PatientData_errorHandler(me, CORRUPT_DATA); return 0; }; } GenderType PatientData_getGender(PatientData* const me) { if (PatientData_checkData(me)) return me->pData.gender; else { PatientData_errorHandler(me, CORRUPT_DATA); return 0; }; } char* PatientData_getName(PatientData* const me) { if (PatientData_checkData(me)) return me->pData.name; else { PatientData_errorHandler(me, CORRUPT_DATA); return 0; }; } unsigned long PatientData_getPatientID(PatientData* const me) { if (PatientData_checkData(me)) return me->pData.patientID;

```
  else {
      PatientData_errorHandler(me, CORRUPT_DATA);
      return 0;
      };
}

unsigned short PatientData_getSystolicBP(PatientData* const me) {
  if (PatientData_checkData(me))
      return me->pData.systolicBP;
  else {
      PatientData_errorHandler(me, CORRUPT_DATA);
      return 0;
      };
}

unsigned short PatientData_getTemperature(PatientData* const me)
{
  if (PatientData_checkData(me))
      return me->pData.temperature;
  else {
      PatientData_errorHandler(me, CORRUPT_DATA);
      return 0;
      };
}

double PatientData_getWeight(PatientData* const me) {
  if (PatientData_checkData(me))
      return me->pData.weight;
  else {
      PatientData_errorHandler(me, CORRUPT_DATA);
      return 0;
      };
}

void PatientData_setAge(PatientData* const me, unsigned short a) {
  if (PatientData_checkData(me)) {
      me->pData.age = a;
      me->crc = computeCRC((unsigned char *)&me->pData, sizeof(me->pData), 0xffff, 0);
      if (!PatientData_checkData(me))
          PatientData_errorHandler(me, CORRUPT_DATA);
      }
  else {
      printf("Set failed\n");
      PatientData_errorHandler(me, CORRUPT_DATA);
      };
}

void PatientData_setBloodO2Conc(PatientData* const me, unsigned
short o2) {
  if (PatientData_checkData(me)) {
```



```
      else { PatientData_errorHandler(me, CORRUPT_DATA); return 0; }; }
unsigned short PatientData_getSystolicBP(PatientData* const me) { if
(PatientData_checkData(me)) return me->pData.systolicBP; else { Patient
Data_errorHandler(me, CORRUPT_DATA); return 0; }; } unsigned short
PatientData_getTemperature(PatientData* const me) { if (PatientData_
checkData(me)) return me->pData.temperature; else { PatientData_
errorHandler(me, CORRUPT_DATA); return 0; }; } double PatientData_
getWeight(PatientData* const me) { if (PatientData_checkData(me)) return
me->pData.weight; else { PatientData_errorHandler(me, CORRUPT_DATA);
return 0; }; }

void PatientData_setAge(PatientData* const me, unsigned short a) { if (PatientData_checkData(me)) {
me->pData.age = a; me->crc = computeCRC((无符号字符 *)&me->pData, sizeof(me->pData), 0xffff, 0); if (!
PatientData_checkData(me)) PatientData_errorHandler(me, CORRUPT_DATA); } else { printf("Set
failed\n");PatientData_errorHandler(me, CORRUPT_DATA); }; }

void PatientData_setBloodO2Conc(PatientData* const me, unsigned short
o2) { if (PatientData_checkData(me)) {
```

```
        me->pData.bloodO2Conc = o2;
        me->crc = computeCRC((unsigned char *)&me->pData, sizeof(me->pData), 0xffff, 0);
        if (!PatientData_checkData(me))
            PatientData_errorHandler(me, CORRUPT_DATA);
        }
    else {
        printf("Set failed\n");
        PatientData_errorHandler(me, CORRUPT_DATA);
        };
}

void PatientData_setDiastolicBP(PatientData* const me, unsigned short dBP) {
    if (PatientData_checkData(me)) {
        me->pData.diastolicBP = dBP;
        me->crc = computeCRC((unsigned char *)&me->pData, sizeof(me->pData), 0xffff, 0);
        if (!PatientData_checkData(me))
            PatientData_errorHandler(me, CORRUPT_DATA);
        }
    else {
        printf("Set failed\n");
        PatientData_errorHandler(me, CORRUPT_DATA);
        };
}

void PatientData_setGender(PatientData* const me, GenderType g) {
    if (PatientData_checkData(me)) {
        me->pData.gender = g;
        me->crc = computeCRC((unsigned char *)&me->pData, sizeof(me->pData), 0xffff, 0);
        if (!PatientData_checkData(me))
            PatientData_errorHandler(me, CORRUPT_DATA);
        }
    else {
        printf("Set failed\n");
        PatientData_errorHandler(me, CORRUPT_DATA);
        };
}

void PatientData_setName(PatientData* const me, char* n) {
    if (PatientData_checkData(me)) {
        strcpy(me->pData.name, n);
        me->crc = computeCRC((unsigned char *)&me->pData, sizeof(me->pData), 0xffff, 0);
        if (!PatientData_checkData(me))
            PatientData_errorHandler(me, CORRUPT_DATA);
        }
    else {
        printf("Set failed\n");
        PatientData_errorHandler(me, CORRUPT_DATA);
        };
}

void PatientData_setPatientID(PatientData* const me, unsigned long id) {
    if (PatientData_checkData(me)) {
```


me->pData.bloodO2Conc = o2; me->crc = computeCRC((unsigned char *)&me->pData, sizeof(me->pData), 0xffff, 0); if (!PatientData_checkData(me)) PatientData_errorHandler(me, CORRUPT_DATA); } else { printf("设置失败\n");PatientData_errorHandler(me, CORRUPT_DATA); }; }

void PatientData_setDiastolicBP(PatientData* const me, unsigned short dBP) { if (PatientData_checkData(me)) { me->pData.diastolicBP = dBP; me->crc = computeCRC((unsigned char *)&me->pData, sizeof(me->pData), 0xffff, 0); if (!PatientData_checkData(me)) PatientData_errorHandler(me, CORRUPT_DATA); } else { printf("设置失败\n");PatientData_errorHandler(me, CORRUPT_DATA); }; } void PatientData_setGender(PatientData* const me, GenderType g) { if (PatientData_checkData(me)) { me->pData.gender = g; me->crc = computeCRC((unsigned char *)&me->pData, sizeof(me->pData), 0xffff, 0); if (!PatientData_checkData(me)) PatientData_errorHandler(me, CORRUPT_DATA); } else { printf("设置失败\n");PatientData_errorHandler(me, CORRUPT_DATA); }; } void PatientData_setName(PatientData* const me, char* n) { if (PatientData_checkData(me)) { strcpy(me->pData.name, n); me->crc = computeCRC((unsigned char *)&me->pData, sizeof(me->pData), 0xffff, 0); if (!PatientData_checkData(me)) PatientData_errorHandler(me, CORRUPT_DATA); } else { printf("设置失败\n"); PatientData_errorHandler(me, CORRUPT_DATA); }; }

void PatientData_setPatientID(PatientData* const me, unsigned long id) { if (PatientData_checkData(me)) {

```
      me->pData.patientID = id;
      me->crc = computeCRC((unsigned char *)&me->pData, sizeof(me->pData), 0xffff, 0);
      if (!PatientData_checkData(me))
          PatientData_errorHandler(me, CORRUPT_DATA);
      }
  else {
      printf("Set failed\n");
      PatientData_errorHandler(me, CORRUPT_DATA);
      };
}

void PatientData_setSystolicBP(PatientData* const me, unsigned short sBP) {
  if (PatientData_checkData(me)) {
      me->pData.systolicBP = sBP;
      me->crc = computeCRC((unsigned char *)&me->pData, sizeof(me->pData), 0xffff, 0);
      if (!PatientData_checkData(me))
          PatientData_errorHandler(me, CORRUPT_DATA);
      }
  else {
      printf("Set failed\n");
      PatientData_errorHandler(me, CORRUPT_DATA);
      };
}

void PatientData_setTemperature(PatientData* const me, unsigned short t) {
  if (PatientData_checkData(me)) {
      me->pData.temperature = t;
      me->crc = computeCRC((unsigned char *)&me->pData, sizeof(me->pData), 0xffff, 0);
      if (!PatientData_checkData(me))
          PatientData_errorHandler(me, CORRUPT_DATA);
      }
  else {
      printf("Set failed\n");
      PatientData_errorHandler(me, CORRUPT_DATA);
      };
}

void PatientData_setWeight(PatientData* const me, double w) {
  if (PatientData_checkData(me)) {
      me->pData.weight = w;
      me->crc = computeCRC((unsigned char *)&me->pData, sizeof(me->pData), 0xffff, 0);
      if (!PatientData_checkData(me))
          PatientData_errorHandler(me, CORRUPT_DATA);
      }
  else {
      printf("Set failed\n");
      PatientData_errorHandler(me, CORRUPT_DATA);
      };
}

struct AlarmManager* PatientData_getItsAlarmManager(const PatientData* const me) {
  return (struct AlarmManager*)me->itsAlarmManager;
}
```


me->pData.患者ID = id; me->crc = computeCRC((无符号字符 *)&me->pData, sizeof(me->pData), 0xffff, 0); if (!PatientData_checkData(me)) PatientData_errorHandler(me, CORRUPT_DATA); } else { printf("Set failed\n"); PatientData_errorHandler(me, CORRUPT_DATA); }; } void PatientData_setSystolicBP(PatientData* const me, unsigned short sBP) { if (PatientData_checkData(me)) { me->pData.systolicBP = sBP; me->crc = computeCRC((无符号字符 *)&me->pData, sizeof(me->pData), 0xffff, 0); if (!PatientData_checkData(me)) PatientData_errorHandler(me, CORRUPT_DATA); } else { printf("Set failed\n"); PatientData_errorHandler(me, CORRUPT_DATA); }; } void PatientData_setTemperature(PatientData* const me, unsigned short t) { if (PatientData_checkData(me)) { me->pData.temperature = t; me->crc = computeCRC((无符号字符 *)&me->pData, sizeof(me->pData), 0xffff, 0); if (!PatientData_checkData(me)) PatientData_errorHandler(me, CORRUPT_DATA); } else { printf("Set failed\n"); PatientData_errorHandler(me, CORRUPT_DATA); }; } void PatientData_setWeight(PatientData* const me, double w) { if (PatientData_checkData(me)) { me->pData.weight = w; me->crc = computeCRC((无符号字符 *)&me->pData, sizeof(me->pData), 0xffff, 0); if (!PatientData_checkData(me)) PatientData_errorHandler(me, CORRUPT_DATA); } else { printf("Set failed\n"); PatientData_errorHandler(me, CORRUPT_DATA); }; } 结构体 AlarmMana

) { return (结构体 AlarmManager*)me->itsAlarmManager; } er* PatientData_getItsAlarmMana 实现(const PatientData* const me

```
void PatientData_setItsAlarmManager(PatientData* const me, struct AlarmManager*
p_AlarmManager) {
  me->itsAlarmManager = p_AlarmManager;
}
PatientData * PatientData_Create(void) {
  PatientData* me = (PatientData *) malloc(sizeof(PatientData));
  if(me!=NULL)
    PatientData_Init(me);
  return me;
}

void PatientData_Destroy(PatientData* const me) {
  if(me!=NULL)
    PatientData_Cleanup(me);
  free(me);
}
static void cleanUpRelations(PatientData* const me) {
  if(me->itsAlarmManager != NULL)
    me->itsAlarmManager = NULL;
}
```

Code Listing 6-6: PatientData.c

6.4 Smart Data Pattern

One of the biggest problems I see with embedded C code in actual application is that functions all have preconditions for their proper execution but these functions don't explicitly check that those preconditons are actually satisfied. This falls under the heading of what is commonly known as "defensive design" – a development paradigm in which we design with proactive run-time defenses in place to detect problems. The Smart Data Pattern codifies that paradigm for scalar data elements.

6.4.1 Abstract

One of the reasons that languages such as Ada are considered "safer[6]" than C is that they have run-time checking. These checks include array index range checking, definition of subtypes and subranges (such as defining a subtype `Color` to be not only limited to a set of colors but generate errors if that range is exceeded), and parameter range checks. Indeed although the C++ language is not inherently "safe" by this standard, because data structures may be bound together with operations that provide run-time checking, it can be made "safe" by intention if not by default. This pattern, although landing more within the accepted range of "idiom" rather than "pattern," addresses those concerns by creating C classes that explicitly perform these checks.

[6] Using the term loosely and not in the precise sense defined in this chapter.

void PatientData_setItsAlarmManager(PatientData* const me, struct AlarmManager* p_AlarmManager) { me->itsAlarmManager = p_AlarmManager; } PatientData * PatientData_Create(void) { PatientData* me = (PatientData *) malloc(sizeof(PatientData)); if(me!=NULL)PatientData_Init(me); return me; } void PatientData_Destroy(PatientData* const me) { if(me!=NULL)PatientData_Cleanup(me); free(me); } static void cleanUpRelations(PatientData* const me) { if(me->itsAlarmManager != 空) me->itsAlarmManager = 空; }

代码清单 6-6：PatientData.c

6.4 智能数据模式

我在实际应用中看到的嵌入式C代码的最大问题之一是，函数对其正确执行都有前置条件，但这些函数并不显式检查这些前置条件是否真正得到满足。这属于通常所说的"防御性设计"的范畴——一种开发范式，我们在其中设计主动的运行时防御来检测问题。智能数据模式将这一范式编纂为针对标量数据元素的规范。

6.4.1 摘要

Ada 等 语言 被认为 比 C 更 " 安全[6]"的 原因 之一是它们具有运行时检查。这些检查包括数组索引范围检查、子类型和子范围的定义（例如定义一个子类型颜色，不仅限于一组颜色，而且如果超出该范围就会产生错误），以及参数范围检查。确实，尽管C语言按此标准并非本质上"安全"，但由于数据结构可以与提供运行时检查的操作绑定在一起，因此它可以通过有意设计而变得"安全"，即使不是默认如此。这种模式虽然更多地属于公认的"惯用法"范畴而非"模式"，但通过创建显式执行这些检查的C类来解决这些问题。

[6] 此处使用该术语较为宽泛，并非本章所定义的精确含义。

6.4.2 Problem

The most common idiom for error checking of data ranges in C is for functions that have no other return value: return a 0 value for function success and a -1 value for failure and set `errno` to an error code.

The problem is, of course, that the most common way to "handle" the return values is to ignore them, resulting in difficult-to-debug systems. Some people even put error checking into the system during development but remove such checks in the final release. I would argue that the time when you *really* want to know that the altitude determination program in your aircraft isn't producing the right value is when you are in the flying aircraft.

The problem this pattern addresses is to build functions and data types that essentially check themselves and provide error detection means that cannot be easily ignored.

6.4.3 Pattern Structure

As mentioned above, this pattern falls into the idiomatic (tiny pattern) range rather than a collaboration pattern. The key concepts of the pattern are to 1) build self-checking types whenever possible; 2) check incoming parameter values for appropriate range checking; and 3) check consistency and reasonableness among one or a set of parameters. Figure 6-6 shows the structure of the pattern.

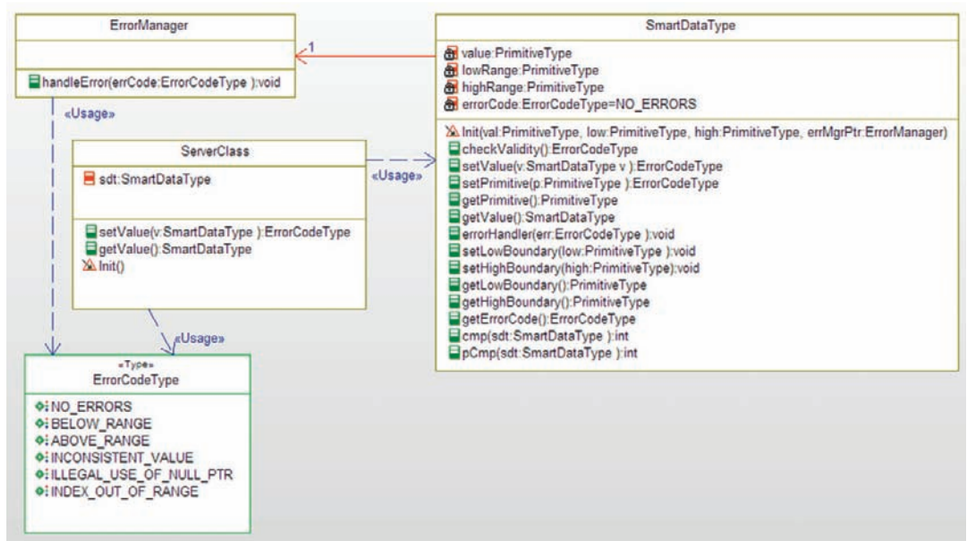


Figure 6-6: Smart Data Pattern

6.4.2 问题

在C中对数据范围进行错误检查的最常见惯用法适用于没有其他返回值的函数：函数成功时返回0值，失败时返回-1值，并将`errno`设置为错误码。

当然，问题在于"处理"返回值最常见的方式就是忽略它们，从而导致系统难以调试。有些人甚至在开发期间将错误检查放入系统中，但在最终发布时却移除了这些检查。我认为，你真正需要知道飞机中的高度测定程序没有产生正确值的时候，正是你身处飞行中的飞机上的时候。

该模式所解决的问题是构建能够自我检查的函数和数据类型，并提供不易被忽略的错误检测手段。

6.4.3 模式结构

如上所述，该模式属于惯用（微型模式）范围，而非协作模式。该模式的关键概念是：1）尽可能构建自检类型；2）检查传入参数值以进行适当的范围检查；3）检查一个或一组参数之间的一致性和合理性。图6-6展示了该模式的结构。

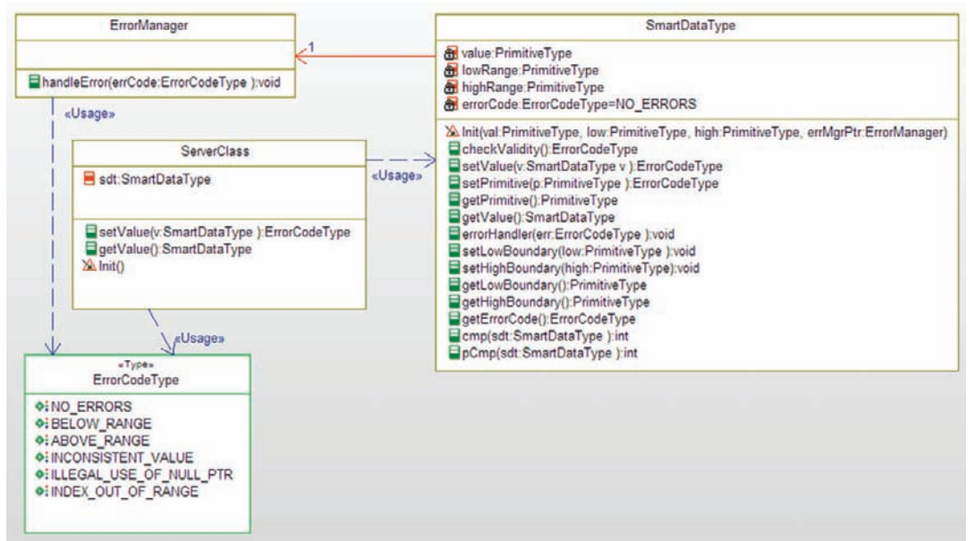


图 6-6：智能数据模式

6.4.4 Collaboration Roles

This section describes the roles for this pattern.

6.4.4.1 ErrorCodeType

The `ErrorCodeType` is an enumeration of possible error codes. This can be, and is often, done with ints but enums clarify the intention and usage of the codes more clearly.

6.4.4.2 ErrorManager

The `ErrorManager` class handles the errors identified within the `SubrangeType`; the actions are system-, context-, and error-specific.

6.4.4.3 ServerClass

This role in the pattern is an element that uses the SmartDataType.

6.4.4.4 SmartDataType

One of the disadvantages of data structures is that while they all have preconditions and rules for proper usage, they have no intrinsic behavior to enforce the conditions and rules. This class binds together the data (of type `PrimitiveType`) with operations that ensure they stay in range.

This class provides a variety of functions. The `SmartDataType_Init()` function takes five parameters – the `me` pointer to point to the instance data, the initial value being set, the low and high values specifying the valid data subrange, and the address of the `ErrorManager`. If the entire range of the primitive type is valid, then you can simply use the primitive type or use the low and high values to the min and max for the type (such as `SHRT_MIN` and `SHRT_MAX`, or `INT_MIN` and `INT_MAX` declared in <limits.h>, for example). To create a new instance on the heap, simply use `SmartDataType_Create()`, which takes three parameters – the value, the low boundary, and the high boundary and returns a pointer to the newly created instance.

`SmartDataTypes` can be assigned to each other with the `SmartDataType_setValue()` and `SmartDataType_getValue()` functions and compared with the boundary with the `SmartDataType_cmp()` function and without the boundaries with the `SmartDataType_pCmp()` function. The held data can be independently set with the `SmartDataType_setPrimitive()` and `SmartDataType_getPrimitive()` functions. The low and high boundaries can also be set and read independently.

6.4.5 Consequences

The downside for using smart data types is the performance overhead for executing the operations. The upside is that data are self-protecting and provide automatic checking when data are set. It is also possible for the programmers to avoid using the functions

6.4.4 协作角色

本节描述该模式的角色。

6.4.4.1 ErrorCodeType

`ErrorCodeType` 是可能错误码的枚举。这可以并且经常用整数来实现，但枚举能更清晰地阐明错误码的意图和用法。

6.4.4.2 ErrorManager

`ErrorManager` 类处理在`SubrangeType`中识别出的错误；这些动作是系统、上下文和错误特定的。

6.4.4.3 ServerClass

该模式中的这个角色是使用 SmartDataType 的元素。

6.4.4.4 SmartDataType

数据结构的缺点之一是，虽然它们都有前置条件和正确使用的规则，但它们没有内在行为来强制执行这些条件和规则。该类将数据（类型为 `PrimitiveType`）与确保其保持在范围内的操作绑定在一起。

该类提供了多种函数。`SmartDataType_Init()` 函数接受五个参数——指向实例数据的 `me` 指针、所设置的初始值、指定有效数据子范围的低值和高值，以及`ErrorManager`的地址。如果基本类型的整个范围都有效，那么你可以直接使用该基本类型，或者使用该类型的最小值和最大值作为低值和高值（例如，使用 `SHRT_MIN` 和 `SHRT_MAX`，或 `INT_MIN` 和 `INT_MAX`，这些在 <limits.h> 中声明）。要在堆上创建新实例，只需使用 `SmartDataType_Create()`，它接受三个参数——值、低边界和高边界，并返回指向新创建实例的指针。

`SmartDataTypes` 可以通过 `SmartDataType_setValue()` 和 `SmartDataType_getValue()` 函数相互赋值，并通过 `SmartDataType_cmp()` 函数与边界进行比较，以及通过 `SmartDataType_pCmp()` 函数在不涉及边界的情况下进行比较。所持有的数据可以通过 `SmartDataType_setPrimitive()` 和 `SmartDataType_getPrimitive()` 函数独立设置。低边界和高边界也可以独立设置和读取。

6.4.5 结果

使用智能数据类型的缺点是执行操作时的性能开销。优点在于数据具有自我保护能力，并在设置数据时提供自动检查。程序员也可以选择不使用这些函数，

and access the values directly if they are so inclined, defeating the purpose of the smart data type.

6.4.6 Implementation Strategies

The key implementation strategy is to create a struct with a set of defined operations, and then only use those operations to access the values. The pattern can be extended to include an enumerated type of the units of the value when dealing with numbers with units, such as currency (e.g., euro and dollar), mass (such as pounds, kilograms), and distance (such as inches, feet, yards, fathoms, miles, and even meters). This is a useful extension when dealing with data from different clients that may require conversion. The conversion functions can be built into the class so that you both set and get the values in a specified unit.

6.4.7 Related Patterns

The idea of building smart types is extended pointers with the Smart Pointer Pattern.

6.4.8 Example

Figure 6-7 shows a simple medical example of using smart data types. The `ServerClass` role from the pattern is fulfilled by the `PatientDataClass` in the figure. It has three *SmartInt* variables (weight, age, and heart rate) and two `SmartColor` variables (foreground color and background color). I included the use of an enumerated type `ColorType` to show how the pattern can be used for types other than built-in types.

Code Listing 6-7 gives the basic use types for the example.

```
#ifndef SmartDataExample_H
#define SmartDataExample_H

struct AlarmManager;
struct PatientDataClass;
struct SmartColor;
struct SmartDataClient;
struct SmartInt;

typedef enum ErrorCodeType {
  NO_ERRORS,
  BELOW_RANGE,
  ABOVE_RANGE,
  INCONSISTENT_VALUE,
  ILLEGAL_USE_OF_NULL_PTR,
  INDEX_OUT_OF_RANGE
} ErrorCodeType;
```


如果愿意的话直接访问值，从而违背了智能数据类型的目的。

6.4.6 实现策略

关键的实现策略是创建一个包含一组已定义操作的结构体，然后仅使用这些操作来访问值。当处理带有单位的数字时，该模式可以扩展为包含值单位的枚举类型，例如货币（如欧元和美元）、质量（如磅、千克）和距离（如英寸、英尺、码、英寻、英里，甚至米）。当处理来自不同客户端且可能需要转换的数据时，这是一个有用的扩展。转换函数可以构建到类中，以便您以指定的单位设置和获取值。

6.4.7 相关模式

构建智能类型的理念通过智能指针模式扩展到了指针。

6.4.8 示例

图 6-7 展示了使用智能数据类型的一个简单医疗示例。图中 `PatientDataClass` 承担了模式中的 `ServerClass` 角色。它有三个 SmartInt 变量（weight、age 和 heart rate）以及两个 `SmartColor` 变量（前景色和背景色）。我加入了枚举类型 `ColorType` 的使用，以展示该模式如何用于内置类型之外的类型。

代码清单 6-7 给出了该示例的基本使用类型。

```
#ifndef SmartDataExample_H #
define SmartDataExample_H

struct AlarmManager; struct
PatientDataClass; struct
SmartColor; struct
SmartDataClient; struct
SmartInt;

typedef enum ErrorCodeType {
NO_ERRORS, BELOW_RANGE,
ABOVE_RANGE, I
NCONSISTENT_VALUE,
ILLEGAL_USE_OF_NULL_PTR,
INDEX OUT OF RANGE_    _   _}
ErrorCodeType;
```

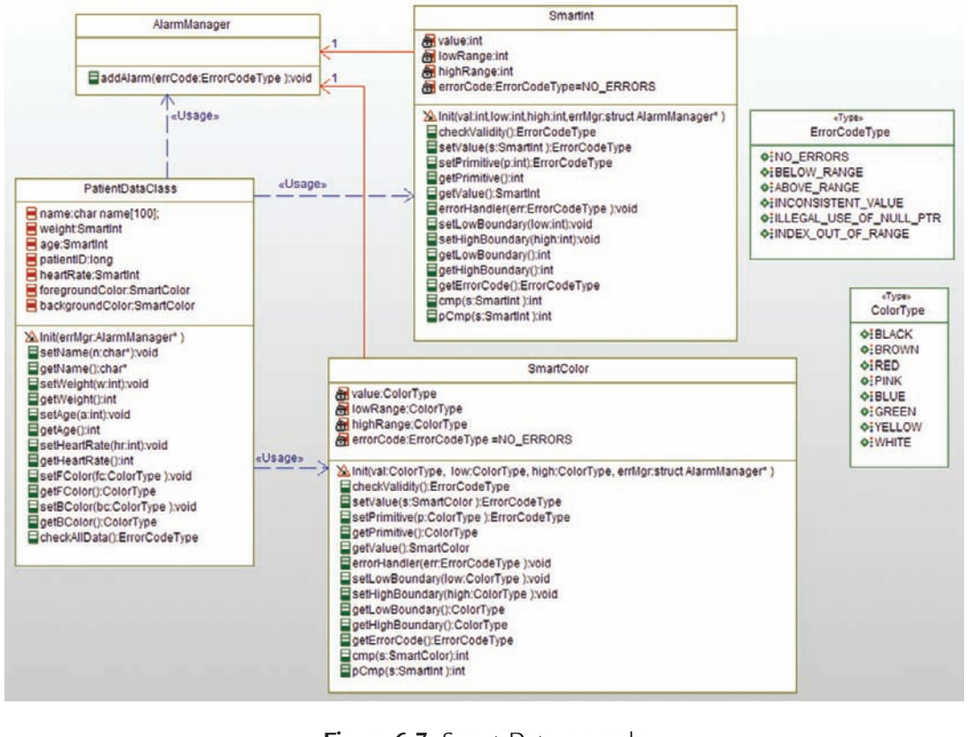


Figure 6-7: Smart Data example

```
typedef enum ColorType {
  BLACK,
  BROWN,
  RED,
  PINK,
  BLUE,
  GREEN,
  YELLOW,
  WHITE
}
ColorType;

#endif
```

Code Listing 6-7: SmartDataExample.h

The `SmartInt` data type is used for three variables in the `PatientDataClass`. It provides the basic functions identified in the `SmartData` role of the pattern, where int is the primitive type. Its code is shown in Code Listing 6-8 and Code Listing 6-9.

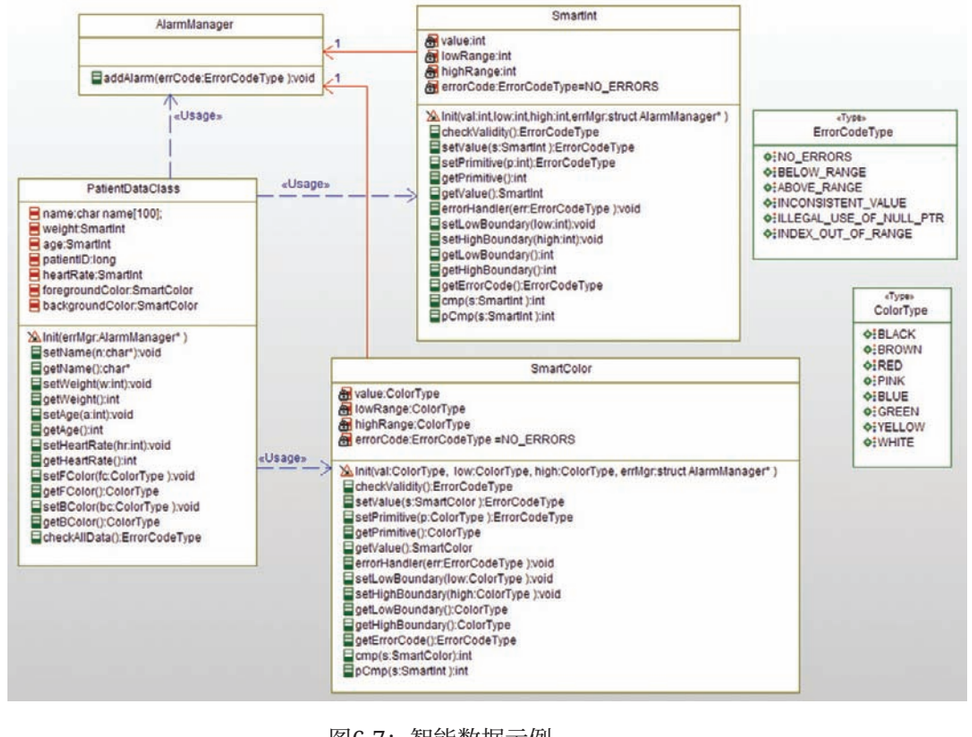


图6-7：智能数据示例

```
typedef enum ColorType {
BLACK, BROWN, RED,
PINK, BLUE, 绿色, 黄色,
WHITE } ColorType; #
endif
```

代码清单6-7：SmartDataExample.h

`SmartInt` 数据类型用于`PatientDataClass`中的三个变量。它提供了`SmartData` 模式角色中定义的基本函数，其中int是基本类型。其代码如代码清单6-8和代码清单6-9所示。

```
#ifndef SmartInt_H
#define SmartInt_H

#include "SmartDataExample.h"
struct AlarmManager;

typedef struct SmartInt SmartInt;
struct SmartInt {
  ErrorCodeType errorCode;
  int highRange;
  int lowRange;
  int value;
  struct AlarmManager* itsAlarmManager;
};

/* Constructors and destructors:*/
void SmartInt_Init(SmartInt* const me, int val, int low, int
high, struct AlarmManager* errMgr);

void SmartInt_Cleanup(SmartInt* const me);

/* Operations */
ErrorCodeType SmartInt_checkValidity(SmartInt* const me);

int SmartInt_cmp(SmartInt* const me, SmartInt s);

void SmartInt_errorHandler(SmartInt* const me, ErrorCodeType err);

ErrorCodeType SmartInt_getErrorCode(SmartInt* const me);

int SmartInt_getHighBoundary(SmartInt* const me);

int SmartInt_getLowBoundary(SmartInt* const me);

int SmartInt_getPrimitive(SmartInt* const me);

SmartInt SmartInt_getValue(SmartInt* const me);

int SmartInt_pCmp(SmartInt* const me, SmartInt s);

void SmartInt_setHighBoundary(SmartInt* const me, int high);

void SmartInt_setLowBoundary(SmartInt* const me, int low);

ErrorCodeType SmartInt_setPrimitive(SmartInt* const me, int p);
ErrorCodeType SmartInt_setValue(SmartInt* const me, SmartInt s);
struct AlarmManager* SmartInt_getItsAlarmManager(const SmartInt* const me);                                                    me);
```


#ifndef SmartInt_H #define SmartInt_H #include "SmartDataExample.h" struct AlarmManager; typedef struct SmartInt SmartInt; struct SmartInt { ErrorCodeType errorCode; int highRange; int lowRange; int value; struct AlarmManager* itsAlarmManager; }; /* 构造函数与析构函数： */ void SmartInt_Init(SmartInt* const me, int val, int low, int high, struct AlarmManager* errMgr); void SmartInt_Cleanup(SmartInt* const me); /* 操作 */ ErrorCodeType SmartInt_checkValidity(SmartInt* const me); int SmartInt_cmp(SmartInt* const me, SmartInt s); void SmartInt_errorHandler(SmartInt* const me, ErrorCodeType err); ErrorCodeType SmartInt_getErrorCode(SmartInt* const me); int SmartInt_getHighBoundary(SmartInt* const me); int SmartInt_getLowBoundary(SmartInt* const me); int SmartInt_getPrimitive(SmartInt* const me); SmartInt SmartInt_getValue(SmartInt* const me); int SmartInt_pCmp(SmartInt* const me, SmartInt s); void SmartInt_setHighBoundary(SmartInt* const me, int high); void SmartInt_setLowBoundary(SmartInt* const me, int low); ErrorCodeType SmartInt_setPrimitive(SmartInt* const me, int p); ErrorCodeType SmartInt_setValue(SmartInt* const me, SmartInt s); struct AlarmManager* SmartInt_getItsAlarmManager(const SmartInt* const

```
void SmartInt_setItsAlarmManager(SmartInt* const me, struct
AlarmManager* p_AlarmManager);

SmartInt * SmartInt_Create(int val, int low, int high, struct AlarmManager* errMgr);

void SmartInt_Destroy(SmartInt* const me);
#endif
```

Code Listing 6-8: SmartInt.h

```
#include "SmartInt.h"
#include "AlarmManager.h"

static void cleanUpRelations(SmartInt* const me);

void SmartInt_Init(SmartInt* const me, int val, int low, int
high, struct AlarmManager* errMgr) {
  me->errorCode = NO_ERRORS;
  me->itsAlarmManager = NULL;

  me->lowRange = low;
  me->highRange = high;
  me->value = val;
  me->itsAlarmManager = errMgr;
}

void SmartInt_Cleanup(SmartInt* const me) {
  cleanUpRelations(me);
}

ErrorCodeType SmartInt_checkValidity(SmartInt* const me) {
  if (me->value < me->lowRange)
      return BELOW_RANGE;
  else if (me->value > me->highRange)
      return ABOVE_RANGE;
  else
      return NO_ERRORS;
}

int SmartInt_cmp(SmartInt* const me, SmartInt s) {
  return memcmp(me, &s, sizeof(s));
}

void SmartInt_errorHandler(SmartInt* const me, ErrorCodeType err) {
  AlarmManager_addAlarm(me->itsAlarmManager, err);
}

ErrorCodeType SmartInt_getErrorCode(SmartInt* const me) {
  return me->errorCode;
}
```



```
void SmartInt_setItsAlarmManager(SmartInt* const me, 结构体
AlarmManager* p_AlarmManager);

SmartInt * SmartInt_Create(int val, int low, int high, 结构体 AlarmManager* errMgr);

void SmartInt_Destroy(SmartInt* const me); #endif
```

代码清单 6-8：SmartInt.h

```
#include "SmartInt.h" #
include "AlarmManager.h"

static void cleanUpRelations(SmartInt* const me);

void SmartInt_Init(SmartInt* const me, int val, int low, int high, struct
AlarmManager* errMgr) { me->errorCode = NO_ERRORS; me->
itsAlarmManager = 空; me->lowRange = low; me->highRange = high;
me->value = val; me->itsAlarmManager = errMgr; } void SmartInt_
Cleanup(SmartInt* const me) { cleanUpRelations(me); } ErrorCodeType
SmartInt_checkValidity(SmartInt* const me) { if (me->value < me->
lowRange) return BELOW_RANGE; else if (me->value > me->
highRange) return ABOVE_RANGE; else return NO_ERRORS; } int
SmartInt_cmp(SmartInt* const me, SmartInt s) { return memcmp(me, &
s, sizeof(s)); } void SmartInt_errorHandler(SmartInt* const me,
ErrorCodeType

                                                                    err) {
    AlarmManager_addAlarm(me->itsAlarmManager, err); }
ErrorCodeType SmartInt_getErrorCode(SmartInt* const me) {
return me->errorCode; }
```

```
int SmartInt_getHighBoundary(SmartInt* const me) {
  return me->highRange;
}
int SmartInt_getLowBoundary(SmartInt* const me) {
  return me->lowRange;
}

int SmartInt_getPrimitive(SmartInt* const me) {
  return me->value;
}

SmartInt SmartInt_getValue(SmartInt* const me) {
  /*#[ operation getValue() */
  return *me;
  /*#]*/
}

int SmartInt_pCmp(SmartInt* const me, SmartInt s) {
  int a = SmartInt_getPrimitive(&s);
  return memcmp(&me->value, &a, sizeof(me->value));
}

void SmartInt_setHighBoundary(SmartInt* const me, int high) {
  me->highRange = high;
}

void SmartInt_setLowBoundary(SmartInt* const me, int low) {
  me->lowRange = low;
}

ErrorCodeType SmartInt_setPrimitive(SmartInt* const me, int p) {
  me->errorCode = NO_ERRORS;
  if (p < me->lowRange) {
      me->errorCode = BELOW_RANGE;
      AlarmManager_addAlarm(me->itsAlarmManager, me->errorCode);
      }
  else if (p > me->highRange) {
      me->errorCode = ABOVE_RANGE;
      AlarmManager_addAlarm(me->itsAlarmManager, me->errorCode);
      }
  else
      me->value = p;

  return me->errorCode;
}

ErrorCodeType SmartInt_setValue(SmartInt* const me, SmartInt s) {
  me->errorCode = SmartInt_checkValidity(&s);
  if (me->errorCode == NO_ERRORS)
      *me = s;
  else
      SmartInt_errorHandler(me, me->errorCode);
```



```
int SmartInt_getHighBoundary(SmartInt* const me) { return me->highRange; }
int SmartInt_getLowBoundary(SmartInt* const me) { return me->lowRange; }
int SmartInt_getPrimitive(SmartInt* const me) { return me->value; }
SmartInt SmartInt_getValue(SmartInt* const me) { /*#[ 操作 getValue() */
return *me; /*#]*/ } int SmartInt_pCmp(SmartInt* const me, SmartInt s) { int
a = SmartInt_getPrimitive(&s); return memcmp(&me->value, &a, sizeof(me->
value)); } void SmartInt_setHighBoundary(SmartInt* const me, int high) {
me->highRange = high; } void SmartInt_setLowBoundary(SmartInt* const
me, int low) { me->lowRange = low; } ErrorCodeType SmartInt_
setPrimitive(SmartInt* const me, int p) { me->errorCode = NO_ERRORS; if (p <
me->lowRange) { me->errorCode = BELOW_RANGE; AlarmManager_
addAlarm(me->itsAlarmManager, me->errorCode); } else if (p > me->
highRange) { me->errorCode = ABOVE_RANGE; AlarmManager_
addAlarm(me->itsAlarmManager, me->errorCode); } else me->value = p;
return me->errorCode; } ErrorCodeType SmartInt_setValue(SmartInt* const
me, SmartInt s) { me->errorCode = SmartInt_checkValidity(&s); if (me->
errorCode == NO_ERRORS) *me = s; else SmartInt_errorHandler(me, me->
errorCode);
```

```
  return me->errorCode;
}
struct AlarmManager* SmartInt_getItsAlarmManager(const SmartInt* const me) {
  return (struct AlarmManager*)me->itsAlarmManager;
}

void SmartInt_setItsAlarmManager(SmartInt* const me, struct
AlarmManager* p_AlarmManager) {
  me->itsAlarmManager = p_AlarmManager;
}

SmartInt * SmartInt_Create(int val, int low, int high, struct AlarmManager* errMgr) {
  SmartInt* me = (SmartInt *) malloc(sizeof(SmartInt));
  if(me!=NULL)
    SmartInt_Init(me, val, low, high, errMgr);
  return me;
}

void SmartInt_Destroy(SmartInt* const me) {
  if(me!=NULL)
    SmartInt_Cleanup(me);
  free(me);
}

static void cleanUpRelations(SmartInt* const me) {
  if(me->itsAlarmManager != NULL)
    me->itsAlarmManager = NULL;
}
```

Code Listing 6-9: SmartInt.c

Similarly, the code for the `SmartColor` class is given in Code Listing 6-10 and Code Listing 6-11.

```
#ifndef SmartColor_H
#define SmartColor_H

#include "SmartDataExample.h"
#include "SmartInt.h"
struct AlarmManager;

typedef struct SmartColor SmartColor;
struct SmartColor {
  ErrorCodeType errorCode;
  ColorType highRange;
  ColorType lowRange;
  ColorType value;
```



```
return me->errorCode; } struct AlarmManager* SmartInt_getItsAlarmManager(const S
martInt* const me) { return (struct AlarmManager*)me->itsAlarmManager; } void SmartInt_
setItsAlarmManager(SmartInt* const me, struct AlarmManager* p_AlarmManager) { me->
itsAlarmManager = p_AlarmManager; } SmartInt * SmartInt_Create(int val, int low, int high, struct
AlarmManager* errMgr) { SmartInt* me = (SmartInt *) malloc(sizeof(SmartInt)); if(me!=NULL)
SmartInt_Init(me, val, low, high, errMgr); return me; } void SmartInt_Destroy(SmartInt* const me) {
if(me!=NULL)SmartInt_Cleanup(me); free(me); } static void cleanUpRelations(SmartInt* const me) {
if(me->itsAlarmManager != 空) me->itsAlarmManager = 空; }
```

代码清单 6-9：SmartInt.c

类似地，`SmartColor` 类的代码在代码清单 6-10 和代码清单 6-11 中给出。

```
#ifndef SmartColor_H #define SmartColor_
H #include "SmartDataExample.h" #i
nclude "SmartInt.h" struct Alar
mManager; typedef struct SmartColor
SmartColor; struct SmartColor {
ErrorCodeType errorCode; ColorType
highRange; ColorType lowRange;
ColorType value;
```

```
  struct AlarmManager* itsAlarmManager;
};
/* Constructors and destructors:*/

void SmartColor_Init(SmartColor* const me, ColorType val, ColorType low,
ColorType high, struct AlarmManager* errMgr);

void SmartColor_Cleanup(SmartColor* const me);

/* Operations */
ErrorCodeType SmartColor_checkValidity(SmartColor* const me);

int SmartColor_cmp(SmartColor* const me, SmartColor s);

void SmartColor_errorHandler(SmartColor* const me, ErrorCodeType err);

ErrorCodeType SmartColor_getErrorCode(SmartColor* const me);

ColorType SmartColor_getHighBoundary(SmartColor* const me);

ColorType SmartColor_getLowBoundary(SmartColor* const me);

ColorType SmartColor_getPrimitive(SmartColor* const me);

SmartColor SmartColor_getValue(SmartColor* const me);

int SmartColor_pCmp(SmartColor* const me, SmartInt s);

void SmartColor_setHighBoundary(SmartColor* const me, ColorType high);

void SmartColor_setLowBoundary(SmartColor* const me, ColorType low);

ErrorCodeType SmartColor_setPrimitive(SmartColor* const me, ColorType p);

ErrorCodeType SmartColor_setValue(SmartColor* const me, SmartColor s);

struct AlarmManager* SmartColor_getItsAlarmManager(const SmartColor* const me);

void SmartColor_setItsAlarmManager(SmartColor* const me,
struct AlarmManager* p_AlarmManager);

SmartColor * SmartColor_Create(ColorType val, ColorType low, ColorType high, struct
AlarmManager* errMgr);

void SmartColor_Destroy(SmartColor* const me);
#endif
```

Code Listing 6-10: SmartColor.h

```
#include "SmartColor.h"
#include "AlarmManager.h"
static void cleanUpRelations(SmartColor* const me);

void SmartColor_Init(SmartColor* const me, ColorType val, ColorType low, ColorType high,
struct AlarmManager* errMgr) {
  me->errorCode = NO_ERRORS;
  me->itsAlarmManager = NULL;
```



```
    struct AlarmManager* its
AlarmManager; }; /* 构造函数与析构函数：*/

void SmartColor_Init(SmartColor* const me, ColorType val, ColorType low,
ColorType high, struct AlarmManager* errMgr);

void SmartColor_Cleanup(SmartColor* const me);

/* 操作 */
ErrorCodeType SmartColor_checkValidity(SmartColor* const me);

int SmartColor_cmp(SmartColor* const me, SmartColor s);

void SmartColor_errorHandler(SmartColor* const me, ErrorCodeType err);

ErrorCodeType SmartColor_getErrorCode(SmartColor* const me);

ColorType SmartColor_getHighBoundary(SmartColor* const me);

ColorType SmartColor_getLowBoundary(SmartColor* const me);

ColorType SmartColor_getPrimitive(SmartColor* const me);

SmartColor SmartColor_getValue(SmartColor* const me);

int SmartColor_pCmp(SmartColor* const me, SmartInt s);

void SmartColor_setHighBoundary(SmartColor* const me, ColorType high)          ;

void SmartColor_setLowBoundary(SmartColor* const me, ColorType low);

错误  CodeType SmartColor_setPrimitive(SmartColor* const me, ColorType p);

ErrorCodeType SmartColor_setValue(SmartColor* const me, SmartColor s);

struct AlarmManager* SmartColor_getItsAlarmManager(const SmartColor* const me);

void SmartColor_setItsAlarmManager(SmartColor* const me,
struct AlarmManager* p_AlarmManager);

SmartColor * SmartColor_Create(ColorType val, ColorType low, ColorType high, struct AlarmManager*
errMgr);

void SmartColor_Destroy(SmartColor* const me); #endif
```

代码清单 6-10：SmartColor.h

```
#include "SmartColor.h" #include "AlarmManager.h" static
void cleanUpRelations(SmartColor* const me);

void SmartColor_Init(SmartColor* const me, ColorType val, ColorType low, ColorType high, struct
AlarmManager* errMgr) { me->errorCode = NO_ERRORS; me->itsAlarmManager = 空;
```

```
  me->lowRange = low;
  me->highRange = high;
  me->value = val;
  me->itsAlarmManager = errMgr;
}

void SmartColor_Cleanup(SmartColor* const me) {
  cleanUpRelations(me);
}

ErrorCodeType SmartColor_checkValidity(SmartColor* const me) {
  if (me->value < me->lowRange)
      return BELOW_RANGE;
  else if (me->value > me->highRange)
      return ABOVE_RANGE;
  else
      return NO_ERRORS;
}

int SmartColor_cmp(SmartColor* const me, SmartColor s) {
  return memcmp(me, &s, sizeof(s));
}

void SmartColor_errorHandler(SmartColor* const me, ErrorCodeType err) {
  AlarmManager_addAlarm(me->itsAlarmManager, err);
}

ErrorCodeType SmartColor_getErrorCode(SmartColor* const me) {
  return me->errorCode;
}

ColorType SmartColor_getHighBoundary(SmartColor* const me) {
  return me->highRange;
}

ColorType SmartColor_getLowBoundary(SmartColor* const me) {
  return me->lowRange;
}

ColorType SmartColor_getPrimitive(SmartColor* const me) {
  return me->value;
}

SmartColor SmartColor_getValue(SmartColor* const me) {
  return *me;
}

int SmartColor_pCmp(SmartColor* const me, SmartInt s) {
  ColorType a = SmartInt_getPrimitive(&s);
  return memcmp(&me->value, &a, sizeof(me->value));
}

void SmartColor_setHighBoundary(SmartColor* const me, ColorType high) {
  me->highRange = high;
}
```



```
me->lowRange = low; me->highRange = high; me->value = val; me->
itsAlarmManager = errMgr; } void SmartColor_Cleanup(SmartColor* const me) {
cleanUpRelations(me); } ErrorCodeType SmartColor_checkValidity(SmartColor* c
onst me) { if (me->value < me->lowRange) return BELOW_RANGE; else if (me->value >
me->highRange) return ABOVE_RANGE; else return NO_ERRORS; } int SmartColor_
cmp(SmartColor* const me, SmartColor s) { return memcmp(me, &s, sizeof(s)); } void
SmartColor_errorHandler(SmartColor* const me, ErrorCodeType err) {
AlarmManager_addAlarm(me->itsAlarmManager, err); } ErrorCodeType SmartColor_
getErrorCode(SmartColor* const me) { return me->errorCode; } ColorType
SmartColor_getHighBoundary(SmartColor* const me) { return me->highRange; }
ColorType SmartColor_getLowBoundary(SmartColor* const me) { return me->
lowRange; } ColorType SmartColor_getPrimitive(SmartColor* const me) { return me->
value; } SmartColor SmartColor_getValue(SmartColor* const me) { return *me; } int
SmartColor= pCmp(SmartColor* const me, SmartInt s) { ColorType a SmartInt_
getPrimitive(&s); return memcmp(&me->value, &a, sizeof(me->value)); } void
SmartColor_setHighBoundary(SmartColor* const me, ColorType high) { me->
highRange = high; }
```

```
void SmartColor_setLowBoundary(SmartColor* const me, ColorType low) {
  me->lowRange = low;
}
ErrorCodeType SmartColor_setPrimitive(SmartColor* const me, ColorType p) {
  me->errorCode = NO_ERRORS;
  if (p < me->lowRange) {
      me->errorCode = BELOW_RANGE;
      AlarmManager_addAlarm(me->itsAlarmManager, me->errorCode);
      }
  else if (p > me->highRange) {
      me->errorCode = ABOVE_RANGE;
      AlarmManager_addAlarm(me->itsAlarmManager, me->errorCode);
      }
  else
      me->value = p;

  return me->errorCode;
}

ErrorCodeType SmartColor_setValue(SmartColor* const me, SmartColor s) {
  me->errorCode = SmartColor_checkValidity(&s);
  if (me->errorCode == NO_ERRORS)
      *me = s;
  else
      SmartColor_errorHandler(me, me->errorCode);
  return me->errorCode;
}

struct AlarmManager* SmartColor_getItsAlarmManager(const SmartColor* const me) {
  return (struct AlarmManager*)me->itsAlarmManager;
}

void SmartColor_setItsAlarmManager(SmartColor* const me, struct AlarmManager* p_Alarm-
Manager) {
  me->itsAlarmManager = p_AlarmManager;
}

SmartColor * SmartColor_Create(ColorType val, ColorType low, ColorType high, struct
AlarmManager* errMgr) {
  SmartColor* me = (SmartColor *) malloc(sizeof(SmartColor));
  if(me!=NULL)
    SmartColor_Init(me, val, low, high, errMgr);
  return me;
}

void SmartColor_Destroy(SmartColor* const me) {
  if(me!=NULL)
    SmartColor_Cleanup(me);
  free(me);
}

static void cleanUpRelations(SmartColor* const me) {
```



```
void SmartColor_setLowBoundary(SmartColor* const me, ColorType low) { me->lowRange low; }
ErrorCodeType SmartColor_setPrimitive(SmartColor* const me, ColorType p) { me->errorCode = NO_
ERRORS; if (p < me->lowRange) { me->errorCode = BELOW_RANGE; AlarmManager_addAlarm(me->
itsAlarmManager, me->errorCode); } else if (p > me->highRange) { me->errorCode = ABOVE_RANGE;
AlarmManager_addAlarm(me->itsAlarmManager, me->errorCode); } else me->value = p; return me->
errorCode; } ErrorCodeType SmartColor_setValue(SmartColor* const me, SmartColor s) { me->error
Code = SmartColor_checkValidity(&s); if (me->errorCode == NO_ERRORS) *me = s; else SmartColor_
errorHandler(me, me->errorCode); return me->errorCode; } struct AlarmManager* SmartColor_
getItsAlarmManager(const SmartColor* const me) { return (struct AlarmManager*)me->
itsAlarmManager; } void SmartColor_setItsAlarmManager(SmartColor* const me, struct
AlarmManager* p_Alarm- Manager) { me->itsAlarmManager = p_AlarmManager; } SmartColor *
SmartColor_Create(ColorType val, ColorType low, ColorType high, struct AlarmManager* errMgr) {
SmartColor* me = (SmartColor *) malloc(sizeof(SmartColor)); if(me!=NULL)SmartColor_Init(me, val,
low, high, errMgr); return me; } void SmartColor_Destroy(SmartColor* const me) { if(me!=NULL)
SmartColor_Cleanup(me); free(me); } static void cleanUpRelations(SmartColor* const me) {
```

```
  if(me->itsAlarmManager != NULL)
    me->itsAlarmManager = NULL;
}
```

Code Listing 6-11: SmartColor.c

Finally, the `PatientDataClass` uses these types to store values. In this case, it provides the access functions for the primitive types to its clients but uses smart data types for itself.

```
#ifndef PatientDataClass_H
#define PatientDataClass_H

#include "SmartDataExample.h"
#include "AlarmManager.h"
#include "SmartColor.h"
#include "SmartInt.h"

typedef struct PatientDataClass PatientDataClass;
struct PatientDataClass {
  SmartInt age;
  SmartColor backgroundColor;
  SmartColor foregroundColor;
  SmartInt heartRate;
  char name[100];
  long patientID;
  SmartInt weight;
};

/* Constructors and destructors:*/
void PatientDataClass_Init(PatientDataClass* const me, AlarmManager* errMgr);

void PatientDataClass_Cleanup(PatientDataClass* const me);

/* Operations */
ErrorCodeType PatientDataClass_checkAllData(PatientDataClass* const me);

int PatientDataClass_getAge(PatientDataClass* const me);
ColorType PatientDataClass_getBColor(PatientDataClass* const me);
ColorType PatientDataClass_getFColor(PatientDataClass* const me);
int PatientDataClass_getHeartRate(PatientDataClass* const me);
char* PatientDataClass_getName(PatientDataClass* const me);
int PatientDataClass_getWeight(PatientDataClass* const me);
void PatientDataClass_setAge(PatientDataClass* const me, int a);
void PatientDataClass_setBColor(PatientDataClass* const me, ColorType bc);
void PatientDataClass_setFColor(PatientDataClass* const me, ColorType fc);
void PatientDataClass_setHeartRate(PatientDataClass* const me, int hr);
```



```
    if(me->itsAlarmManager != 空)
me->itsAlarmManager = 空; }
```

代码清单 6-11：SmartColor.c

最后，`PatientDataClass` 使用这些类型来存储值。在这种情况下，它向客户端提供基本类型的访问函数，但自身使用智能数据类型。

```
#ifndef PatientDataClass_H #
define PatientDataClass_H

#include "SmartDataExample.h" #
include "AlarmManager.h" #include "
SmartColor.h" #include "SmartInt.h"


typedef struct PatientDataClass PatientDataClass; struct
PatientDataClass { SmartInt age; SmartColor
backgroundColor; SmartColor foregroundColor; SmartInt
heartRate; char name[100]; long patientID; SmartInt
weight; };

/* 构造函数与析构函数：*/ void PatientDataClass_Init(PatientDataClass* const me,
AlarmManager* errMgr);

void PatientDataClass_Cleanup(PatientDataClass* const me);

/* 操作 */ ErrorCodeType PatientDataClass_checkAllData(PatientDataClass* const me);


int PatientDataClass_getAge(PatientDataClass* const me); ColorType PatientDataClass_
getBColor(PatientDataClass* const me); ColorType PatientDataClass_
getFColor(PatientDataClass* const me); int PatientDataClass_
getHeartRate(PatientDataClass* const me); char* PatientDataClass_
getName(PatientDataClass* const me); int PatientDataClass_
getWeight(PatientDataClass* const me); void PatientDataClass_setAge(PatientDataClass*
const me, int a); void PatientDataClass_setBColor(PatientDataClass* const me, ColorType
bc); void PatientDataClass_setFColor(PatientDataClass* const me, ColorType fc); void
PatientDataClass_setHeartRate(PatientDataClass* const me, int hr);
```

```
void PatientDataClass_setName(PatientDataClass* const me, char* n);
void PatientDataClass_setWeight(PatientDataClass* const me, int w);
PatientDataClass * PatientDataClass_Create(AlarmManager* errMgr);
void PatientDataClass_Destroy(PatientDataClass* const me);
#endif
```

Code Listing 6-12: PatientDataClass.h

```
#include "PatientDataClass.h"
void PatientDataClass_Init(PatientDataClass* const me,
AlarmManager* errMgr) {
  strcpy(me->name, " ");
  me->patientID = 0;
  /* initialize smart variables */
  SmartInt_Init(&me->weight, 0, 0, 500, errMgr);
  SmartInt_Init(&me->age, 0, 0, 130, errMgr);
  SmartInt_Init(&me->heartRate, 0, 0, 400, errMgr);
  SmartColor_Init(&me->foregroundColor, WHITE, BLACK, WHITE, errMgr);
  SmartColor_Init(&me->backgroundColor, BLACK, BLACK, WHITE, errMgr);
}
void PatientDataClass_Cleanup(PatientDataClass* const me) {
}
ErrorCodeType PatientDataClass_checkAllData(PatientDataClass* const me) {
  ErrorCodeType res;
  res = SmartInt_checkValidity(&me->weight);
  if (res != NO_ERRORS)
      return res;
  res = SmartInt_checkValidity(&me->age);
  if (res != NO_ERRORS)
      return res;
  res = SmartInt_checkValidity(&me->heartRate);
  if (res != NO_ERRORS)
      return res;
  res = SmartColor_checkValidity(&me->foregroundColor);
  if (res != NO_ERRORS)
      return res;
  res = SmartColor_checkValidity(&me->backgroundColor);
  if (res != NO_ERRORS)
      return res;
}
int PatientDataClass_getAge(PatientDataClass* const me) {
  return SmartInt_getPrimitive(&me->age);
}

ColorType PatientDataClass_getBColor(PatientDataClass* const me) {
  return SmartColor_getPrimitive(&me->backgroundColor);
}
```


void PatientDataClass_setName(PatientDataClass* const me, char* n); void PatientDataClass_setWeight(PatientDataClass* const me, int w); PatientDataClass * PatientDataClass_Create(AlarmManager* errMgr);

void PatientDataClass_Destroy(PatientDataClass* const me); #endif

代码清单 6-12：PatientDataClass.h

#include "PatientDataClass.h" void PatientDataClass_Init(PatientDataClass* const me, AlarmManager* errMgr) { strcpy(me->name, " ") ;me->patientID = 0; /* 初始化智能变量 */ SmartInt_Init(&me->weight, 0, 0, 500, errMgr); SmartInt_Init(&me->age, 0, 0, 130, errMgr); SmartInt_Init(&me->heartRate, 0, 0, 400, errMgr); SmartColor_Init(&me->foregroundColor, WHITE, BLACK, WHITE, errMgr); SmartColor_Init(&me->backgroundColor, BLACK, BLACK, WHITE, errMgr); } void PatientDataClass_Cleanup(PatientDataClass* const me) { } ErrorCodeType PatientDataClass_checkAllData(PatientDataClass* const me) { ErrorCodeType res; res = SmartInt_checkValidity(&me->weight); if (res != NO_ERRORS) return res; res = SmartInt_checkValidity(&me->age); if (res != NO_ERRORS) return res; res = SmartInt_checkValidity(&me->heartRate); if (res != NO_ERRORS) return res; res = SmartColor_checkValidity(&me->foregroundColor); if (res != NO_ERRORS) return res; res = SmartColor_checkValidity(&me->backgroundColor); if (res != NO_ERRORS) return res; } int PatientDataClass_getAge(PatientDataClass* const me) { return SmartInt_getPrimitive(&me->age); } ColorType PatientDataClass_getBColor(PatientDataClass* const me) { return SmartColor_getPrimitive(&me->backgroundColor); }

```
ColorType PatientDataClass_getFColor(PatientDataClass* const me) {
  return SmartColor_getPrimitive(&me->foregroundColor);
}

int PatientDataClass_getHeartRate(PatientDataClass* const me) {
  return SmartInt_getPrimitive(&me->heartRate);
}

char* PatientDataClass_getName(PatientDataClass* const me) {
  return me->name;
}

int PatientDataClass_getWeight(PatientDataClass* const me) {
  return SmartInt_getPrimitive(&me->weight);
}

void PatientDataClass_setAge(PatientDataClass* const me, int a) {
  SmartInt_setPrimitive(&me->age, a);
}

void PatientDataClass_setBColor(PatientDataClass* const me, ColorType bc) {
  SmartColor_setPrimitive(&me->backgroundColor, bc);
}

void PatientDataClass_setFColor(PatientDataClass* const me, ColorType fc) {
  SmartColor_setPrimitive(&me->foregroundColor, fc);
}

void PatientDataClass_setHeartRate(PatientDataClass* const me, int hr) {
  SmartInt_setPrimitive(&me->heartRate, hr);
}

void PatientDataClass_setName(PatientDataClass* const me, char* n) {
  strcpy(me->name, n);
}

void PatientDataClass_setWeight(PatientDataClass* const me, int w) {
  SmartInt_setPrimitive(&me->weight, w);
}

PatientDataClass * PatientDataClass_Create(AlarmManager* errMgr) {
  PatientDataClass* me = (PatientDataClass *)
malloc(sizeof(PatientDataClass));
  if(me!=NULL)
    PatientDataClass_Init(me, errMgr);
  return me;
}
void PatientDataClass_Destroy(PatientDataClass* const me) {
  if(me!=NULL)
    PatientDataClass_Cleanup(me);
  free(me);
}
```

Code Listing 6-13: PatientDataClass.c

ColorType PatientDataClass_getFColor(PatientDataClass* const me) { return SmartColor_getPrimitive(&me->foregroundColor); } int PatientDataClass_getHeartRate(PatientDataClass* const me) { return SmartInt_getPrimitive(&me->heartRate); } char* PatientDataClass_getName(PatientDataClass* const me) { return me->name; } int PatientDataClass_getWeight(PatientDataClass* const me) { return SmartInt_getPrimitive(&me->weight); } void PatientDataClass_setAge(PatientDataClass* const me, int a) { SmartInt_setPrimitive(&me->age, a); } void PatientDataClass_setBColor(PatientDataClass* const me, ColorType bc) { SmartColor_setPrimitive(&me->backgroundColor, bc); } void PatientDataClass_setFColor(PatientDataClass* const me, ColorType fc) { SmartColor_setPrimitive(&me->foregroundColor, fc); } void PatientDataClass_setHeartRate(PatientDataClass* const me, int hr) { SmartInt_setPrimitive(&me->heartRate, hr); } void PatientDataClass_setName(PatientDataClass* const me, char* n) { strcpy(me->name, n); } void PatientDataClass_setWeight(PatientDataClass* const me, int w) { SmartInt_setPrimitive(&me->weight, w); } PatientDataClass * PatientDataClass_Create(AlarmManager* errMgr) { PatientDataClass* me = (PatientDataClass *) malloc(sizeof(PatientDataClass)); if(me!=NULL) PatientDataClass_Init(me, errMgr); return me; } void PatientDataClass_Destroy(PatientDataClass* const me) { if(me!=NULL)PatientDataClass_Cleanup(me); free(me); }

代码清单 6-13：PatientDataClass.c

6.5 Channel Pattern

The Channel Pattern is a larger-scale pattern than the previous patterns we've discussed. The Channel Pattern is used to support medium- to large-scale redundancy to help identify when run-time faults occur and possibly (depending on how it's used) continue to provide services in the presence of such a fault.

6.5.1 Abstract

A *channel* is an architectural structure that contains software (and possibly hardware as well) that performs end-to-end processing; that is, it processes data from raw acquisition, through a series of data processing steps, to physical actuation in the real world. The advantage of a channel is that it provides an independent, self-contained unit of functionality that can be replicated in different ways to address different safety and reliability concerns.

6.5.2 Problem

This pattern provides a basic element of large-scale redundancy that can be used in different ways to address safety and reliability concerns.

6.5.3 Pattern Structure

The basic structure of the pattern is shown in Figure 6-8. The generalization shown in the figure can be notional only for implementations without true inheritance in C. The basic idea is that the software objects are arranged through a series of links so that data are acquired and processed through a series of data transformational steps until it results in an output that controls one or more actuators. The data types may change (for example, it may be acquired as a `double` but result in an `int` output to the actuator) or they may remain the same throughout the processing.

6.5.4 Collaboration Roles

This section describes the roles for this pattern.

6.5.4.1 AbstractDataTransform

This class is a placeholder for one of several possible `ConcreteDataTransforms`. In an object-oriented language or implementation, true generalization can be used but in C, it is more common to use this as a notional concept to give the intent rather than drive the implementation. It has two optional[7] associations. One is to itself – that is, the next transformation in the list. The other is to the output `ActuationDeviceDriver` – this is in lieu of the former

[7] Common implementation is a pointer, set to `NULL` if the link isn't active.

6.5 通道模式

通道模式是比我们之前讨论的模式规模更大的模式。通道模式用于支持中型到大型的冗余，以帮助识别运行时故障何时发生，并可能（取决于其使用方式）在存在此类故障的情况下继续提供服务。

6.5.1 摘要

通道是一种架构结构，包含执行端到端处理的软件（也可能包含硬件）；也就是说，它处理从原始采集开始，经过一系列数据处理步骤，直到现实世界中物理驱动的数据。通道的优势在于它提供了一个独立的、自包含的功能单元，可以以不同方式复制，以应对不同的安全性与可靠性问题。

6.5.2 问题

该模式提供了大规模冗余的基本要素，可以以不同方式使用它来应对安全性与可靠性问题。

6.5.3 模式结构

该 模式的基本 结构如图 6-8所示。对于在C中没有真正继承的实现，图中所示的泛化可能仅是概念上的。其基本思想是，软件对象通过一系列链接进行组织，使得数据通过一系列数据转换步骤被获取和处理，直到产生控制一个或多个执行器的输出。数据类型可能会改变（例如，它可能以`double` 类型获取，但最终以`int` 类型输出到执行器），也可能在整个处理过程中保持不变。

6.5.4 协作角色

本节描述该模式的角色。

6.5.4.1 抽象数据变换

该类是若干可能的具体数据变换之一的占位符。在面向对象语言或实现中，可以使用真正的泛化，但在C语言中，更常见的是将其作为一个概念性概念来传达意图，而不是驱动实现。它有两个可选的[7] 关联。一个是与自身——即列表中的下一个变换。另一个是与输出执行设备驱动 ——这是用来替代前者的

[7] 常见的实现方式是一个指针，如果链接未处于活动状态，则将其设置为空 。

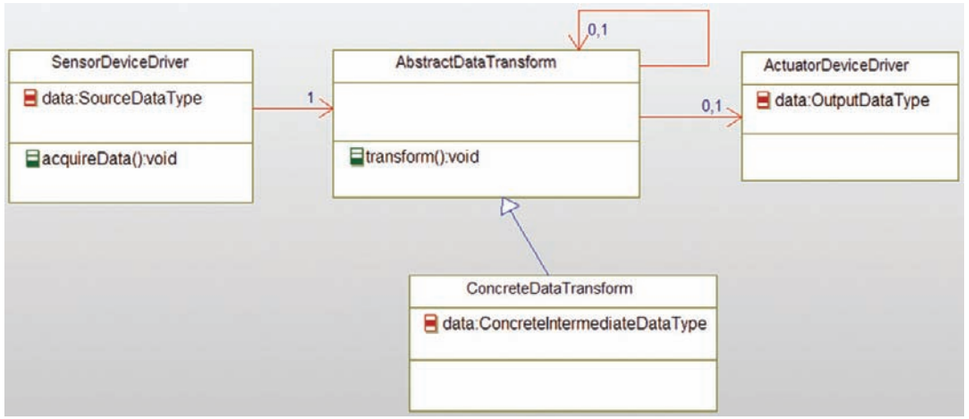


Figure 6-8: Channel Pattern

association. Put another way, the `AbstractDataTransform` has a link to the next processing step in the chain or to the output device driver, but not both.

6.5.4.2 ActuatorDeviceDriver

This class uses the computed output values to drive one or more actuators, such as a motor, light, or heating unit.

6.5.4.3 ConcreteDataTransform

This class performs a mathematical transformation on the incoming data value (from either the sensor or a previous `ConcreteDataTransform`) and produces an output (sent to the next `ConcreteDataTransform` in the sequence or the `ActuatorDeviceDriver`). In simple cases, there may be only a single `ConcreteDataTransform` class, but in more complex situations there may be dozens.

6.5.4.4 SensorDeviceDriver

This class acquires physical data from one or more sensors to put into the channel for processing.

6.5.5 Consequences

The channel in this simple form provides only a little value. Its true value lies in the use of multiple instances of the channel. There are a number of different ways to combine the channels, as well as the data transformations, to address safety and reliability concerns.

In general, the benefits are providing a clear means for identifying faults and, depending on the pattern, providing redundant channels to continue to provide services or to go to a

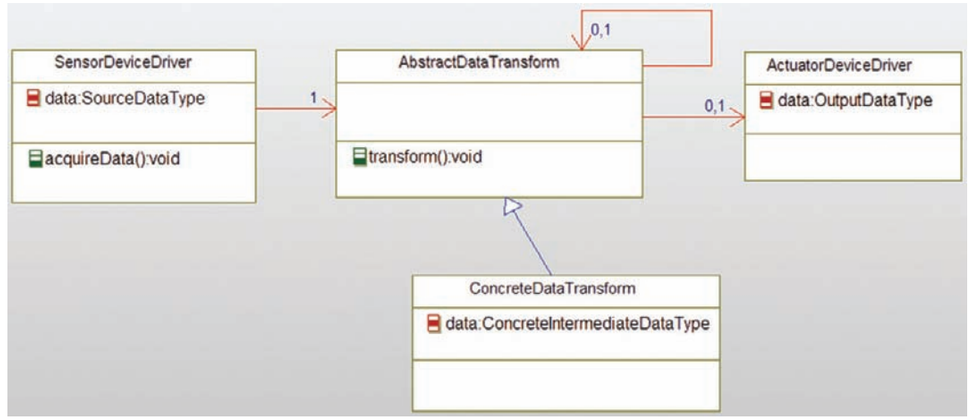


图 6-8：通道模式

关联。换句话说，抽象数据变换 有一个链接指向链中的下一个处理步骤或输出设备驱动程序，但不能同时指向两者。

6.5.4.2 执行器设备驱动

该类使用计算得到的输出值来驱动一个或多个执行器，例如电机、灯或加热单元。

6.5.4.3 ConcreteDataTransform

该类对传入的数据 value（来自传感器或前一个 `ConcreteDataTransform`）执行数学变换，并产生一个输出（发送到序列中的下一个 `ConcreteDataTransform` 或 `ActuatorDeviceDriver`）。在简单情况下，可能只有一个 `ConcreteDataTransform` 类，但在更复杂的情况下可能会有数十个。

6.5.4.4 传感器设备驱动

该类从一个或多个传感器获取物理数据，并将其放入通道进行处理。

6.5.5 结果

这种简单形式的通道提供的价值有限。其真正的价值在于使用通道的多个实例。有多种不同的方式可以组合通道以及数据转换，以解决安全性与可靠性问题。

一般来说，其优势在于提供了一种清晰的故障识别手段，并且根据具体模式的不同，还能提供冗余通道以继续提供服务，或转入

"fault-safe state." The drawbacks of using the pattern are the extra memory (both code and data), processing time, and hardware required, depending on the specific usage of the channel.

6.5.6 Implementation Strategies

As mentioned, the pattern may be implemented with true generalization but it is more common in C to think of the `AbstractDataTransformation` as a placeholder for some particular `ConcreteDataTransform` class; the links are from different types of `ConcereteDataTransform` classes that each perform some special step in the overall data transformation function.

Data are passed through using one of two primary strategies. The simplest is to acquire a datum and pass it through the series of transformations and out to the actuator before acquiring the next. In higher bandwidth applications multiple instances of the data may be present in the channel at the same time, one data instance per transform, with data passed from one transform to the next at the same time as all the other data instances.

6.5.7 Related Patterns

This pattern provides medium- to large-scale redundancy. It is the basis for many other patterns including the Protected Single-Channel Pattern and Homogeneous Redundancy Pattern discussed later in this chapter. Other patterns include Triple Modular Redundancy (TMR) Pattern, in which three channels run in parallel and vote on the outcome, and the Sanity Check Pattern in which a primary channel is checked for reasonableness by a lighter-weight secondary channel. A number of such patterns are discussed in my *Real-Time Design Patterns: Robust Scalable Architecture for Real-Time Systems* (Addison-Wesley, 2002).

6.5.8 Example

I will provide a relatively simple example here of the use of the pattern. The classes shown in Figure 6-9 implement a simple electromyography biofeedback device. It works by measuring the voltage across two electrodes placed on the skin over a muscle. As the muscle tension changes, the voltage potential between the electrodes changes. This is gathered (as an `int`) by the `EMGSensorDeviceDriver`. Then it is converted into a frequency by the first data transformation class; this is done by taking a set of samples and computing the fundamental frequency of those data. This results in a computed frequency that is then filtered with a moving average filter; this is a filter that smoothes out rapid frequency changes by averaging the change over several (frequency) data samples measured sequentially in time. Lastly, the filtered frequency is converted into a pleasing color (specified via RGB values) and this color is sent to the light device driver.

"故障安全状态"。使用该模式的缺点在于需要额外的内存（包括代码和数据）、处理时间以及硬件资源，具体取决于通道的使用方式。

6.5.6 实现策略

如前所述，该模式可以通过真正的泛化来实现，但在 C 语言中更常见的做法是将 `AbstractDataTransformation` 视为某个特定 `ConcreteDataTransform` 类的占位符；链接来自不同类型的 `ConcereteDataTransform` 类，每个类在整体数据变换函数中执行某个特定步骤。

数据通过两种主要策略之一进行传递。最简单的方法是获取一个数据，将其通过一系列变换并输出到执行器，然后再获取下一个数据。在更高带宽的应用中，通道中可能同时存在多个数据实例，每个变换对应一个数据实例，数据从一个变换传递到下一个变换的同时，所有其他数据实例也在同步传递。

6.5.7 相关模式

该模式提供中等至大规模的冗余。它是许多其他模式的基础，包括本章后面讨论的受保护单通道模式和同构冗余模式。其他模式包括三模冗余（TMR）模式——其中三个通道并行运行并对结果进行表决——以及合理性检查模式——其中主通道由较轻量级的辅助通道检查其合理性。我的《Real-Time Design Patterns: Robust Scalable Architecture for Real-Time Systems》（Addison-Wesley，2002）一书中讨论了许多此类模式。

6.5.8 示例

我将在此提供一个相对简单的示例来说明该模式的使用。图6-9中所示的类实现了一个简单的肌电生物反馈设备。它通过测量放置在肌肉上方皮肤上的两个电极之间的电压来工作。随着肌肉张力的变化，电极之间的电压电位也会变化。该信号由EMGSensorDeviceDriver采集（作为int类型）。然后，第一个数据转换类将其转换为频率；这是通过采集一组样本并计算这些数据的基本频率来实现的。由此得到一个计算出的频率，随后用移动平均滤波器对其进行滤波；该滤波器通过在时间上顺序测量的若干（频率）数据样本上对变化取平均值，来平滑快速的频率变化。最后，滤波后的频率被转换为一种令人愉悦的颜色（通过RGB值指定），并将该颜色发送到灯光设备驱动程序。

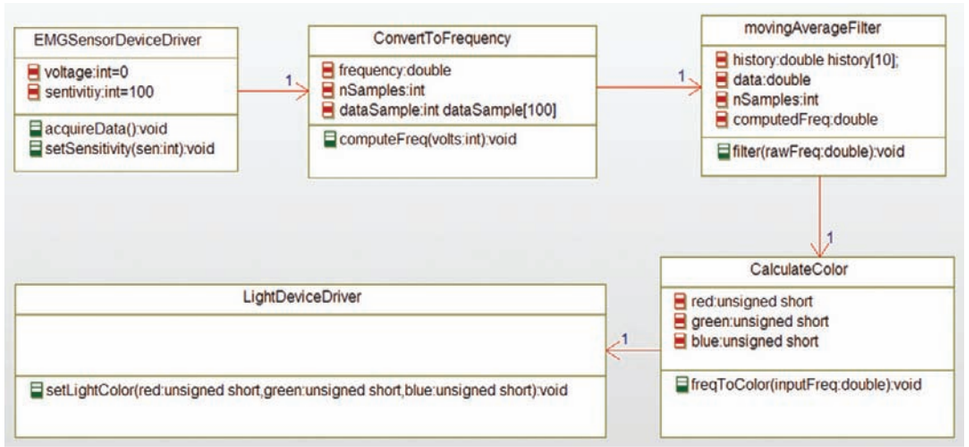


Figure 6-9: Channel Pattern example

Figure 6-10 shows the same pattern but with the channel objects as a part of a larger assembly[8] in which the elements of the channel are parts. The boxes inside the EMG Channel are part objects, and this is indicated with an underlined name.

The code is very straightforward for the pattern. For initialization, the key is that the EMG channel instantiates the objects and then initializes the links (see the `initRelations()` function in Code Listing 6-15) between them. For operation, the EMG Channel provides the services of the start of the chain (that is, the EMG Device Driver); when `acquireData()` is called on the EMG Channel, it calls `EMGSensorDeviceDriver_acquireData()` which gets the data and passes them on to the `ConvertToFrequency_computeFreq()` function. This function does the conversion and then calls the `MovingAverageFilter_filter()` function which smooths out high frequency artifacts and then passes the output of that onto `CalculateColor_freqToColor()`. This function computes the RGB values associated with the frequency and calls `LightDeviceDriver_setLightColor()` to turn the light to the correct color. In this case, the processing is all synchronous. It is possible to pass the data among the `ConcreteDataTransformations` asynchronously as well, by putting queues between them and having them pend on data input to those queues.

The next two code listings, Code Listing 6-14 and Code Listing 6-15 give the code for the `EMGChannel` class header and implementation, respectively.

[8] In UML, such an assembly is known as a *structured class*.

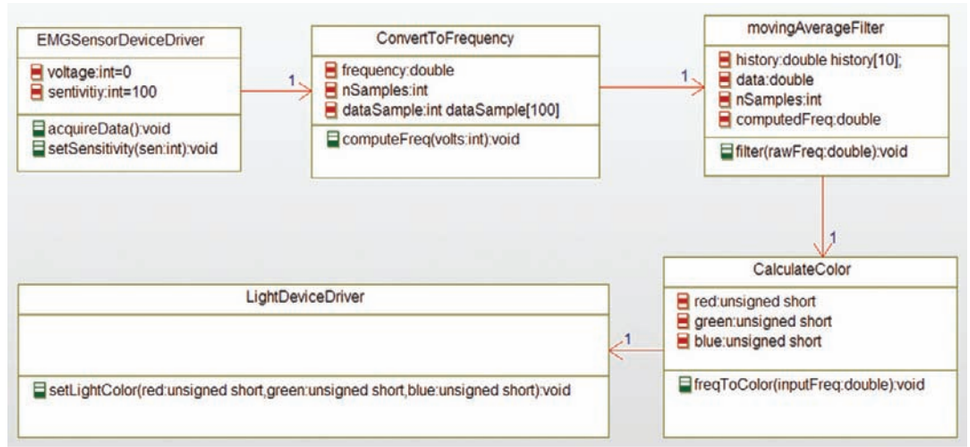


图 6-9：通道模式示例

图6-10展示了相同的模式，但通道对象作为一个更大装配体[8]的一部分，其中通道的元素是部件。EMG通道内的方框是部件对象，这通过带下划线的名称来表示。

该模式的代码非常直接。对于初始化，关键在于EMG通道实例化对象，然后初始化它们之间的链接（参见代码清单6-15中的`initRelations()`函数）。对于操作，EMG通道提供链起始端（即EMG设备驱动程序）的服务；当在EMG通道上调用`acquireData()`时，它会调用`EMGSensorDeviceDriver_acquireData()`，该函数获取数据并将其传递给`ConvertToFrequency_computeFreq()`函数。该函数执行转换，然后调用`MovingAverageFilter_filter()`函数，该函数平滑掉高频伪影，然后将输出传递给`CalculateColor_freqToColor()`。该函数计算与频率关联的RGB值，并调用`LightDeviceDriver_setLightColor()`将灯光调整为正确的颜色。在这种情况下，处理完全是同步的。也可以通过在各`ConcreteDataTransformations`之间放置队列，并让它们挂起等待这些队列的数据输入，从而异步地传递数据。

接下来的两个代码清单，代码清单 6-14 和代码清单 6-15 分别给出了`EMGChannel`类的头文件和实现代码。

[8] 在UML中，这样的装配被称为结构化类。

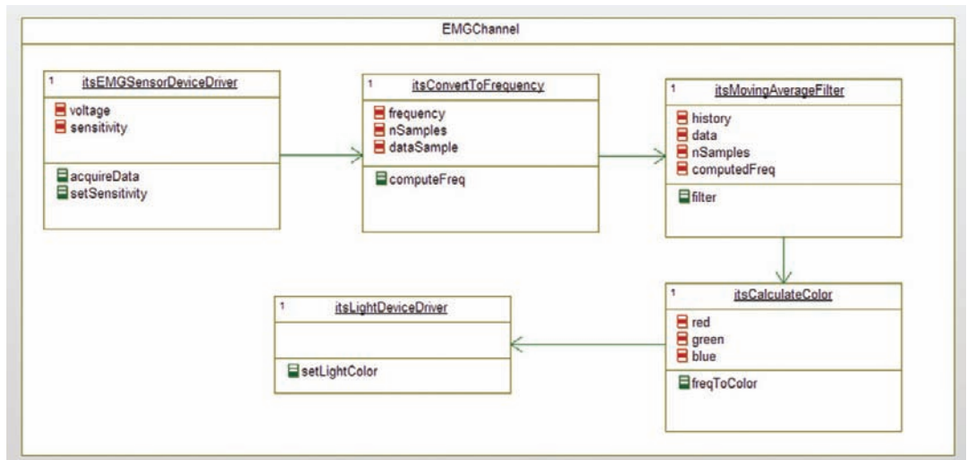


Figure 6-10: Channel Pattern (object view)

```
#ifndef EMGChannel_H
#define EMGChannel_H

#include "CalculateColor.h"
#include "ConvertToFrequency.h"
#include "EMGSensorDeviceDriver.h"
#include "LightDeviceDriver.h"
#include "movingAverageFilter.h"

typedef struct EMGChannel EMGChannel;
struct EMGChannel {
  struct CalculateColor itsCalculateColor;
  struct ConvertToFrequency itsConvertToFrequency;
  struct EMGSensorDeviceDriver itsEMGSensorDeviceDriver;
  struct LightDeviceDriver itsLightDeviceDriver;
  struct movingAverageFilter itsMovingAverageFilter;
};

/* Constructors and destructors:*/
void EMGChannel_Init(EMGChannel* const me);
void EMGChannel_Cleanup(EMGChannel* const me);

/* Operations */
void EMGChannel_acquireData(EMGChannel* const me);
double EMGChannel_getFrequency(EMGChannel* const me);
long EMGChannel_getLightColor(EMGChannel* const me);
int EMGChannel_getVoltage(EMGChannel* const me);
void EMGChannel_setSensitivity(EMGChannel* const me, int sen);
```

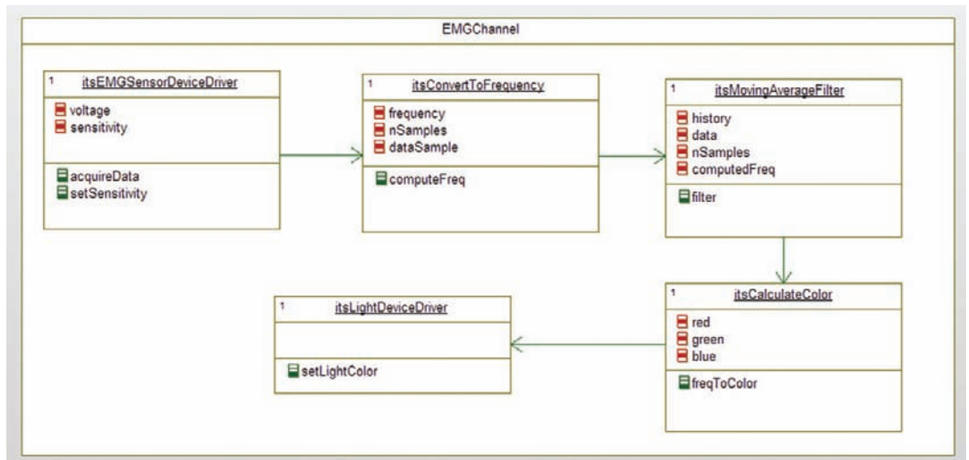



图 6-10：通道模式（对象视图）

```
#ifndef EMGChannel_H #
define EMGChannel_H

#include "CalculateColor.h" #include "
ConvertToFrequency.h" #include "
EMGSensorDeviceDriver.h" #include "
LightDeviceDriver.h" #include "
movingAverageFilter.h"

typedef struct EMGChannel EMGChannel; struct EMGChannel {
struct CalculateColor itsCalculateColor; struct ConvertToFrequency
itsConvertToFrequency; struct EMGSensorDeviceDriver
itsEMGSensorDeviceDriver; struct LightDeviceDriver
itsLightDeviceDriver; struct movingAverageFilter
itsMovingAverageFilter; };

/* 构造函数与析构函数：*/ void EMGChannel_
Init(EMGChannel* const me); void EMGChannel_
Cleanup(EMGChannel* const me);

/* 操作 */
void EMGChannel_acquireData(EMGChannel* const me); double
EMGChannel_getFrequency(EMGChannel* const me); 长整型 EMGChannel_
getLightColor(EMGChannel* const me); int EMGChannel_
getVoltage(EMGChannel* const me); void EMGChannel_
setSensitivity(EMGChannel* const me, int sen);
```

```
struct CalculateColor* EMGChannel_getItsCalculateColor(const EMGChannel* const me);

struct ConvertToFrequency* EMGChannel_getItsConvertToFrequency(const
EMGChannel* const me);

struct EMGSensorDeviceDriver* EMGChannel_getItsEMGSensorDeviceDriver(const
EMGChannel* const me);

struct LightDeviceDriver* EMGChannel_getItsLightDeviceDriver(const
EMGChannel* const me);

struct movingAverageFilter* EMGChannel_getItsMovingAverageFilter(const
EMGChannel* const me);

EMGChannel * EMGChannel_Create(void);

void EMGChannel_Destroy(EMGChannel* const me);
#endif
```

Code Listing 6-14: EMGChannel.h

```
#include "EMGChannel.h"

static void initRelations(EMGChannel* const me);

static void cleanUpRelations(EMGChannel* const me);

void EMGChannel_Init(EMGChannel* const me) {
  initRelations(me);
}

void EMGChannel_Cleanup(EMGChannel* const me) {
  cleanUpRelations(me);
}

void EMGChannel_acquireData(EMGChannel* const me) {
  /* delegate to the appropriate part */
  EMGSensorDeviceDriver_acquireData(&me->itsEMGSensorDeviceDriver);
}

double EMGChannel_getFrequency(EMGChannel* const me) {
  return me->itsMovingAverageFilter.computedFreq;
}

long EMGChannel_getLightColor(EMGChannel* const me) {
  return me->itsCalculateColor.red<<16 + me->itsCalculateColor.green<<8 + me->its-
CalculateColor.blue;
}
```


struct CalculateColor* EMGChannel_getItsCalculateColor(const EMGChannel* const me);

struct ConvertToFrequency* EMGChannel_getItsConvertToFrequency(const E MGChannel* const me); struct EMGSensorDeviceDriver* EMGChannel_ getItsEMGSensorDeviceDriver(const EMGChannel* const me); struct LightDeviceDriver* EMGChannel_getItsLightDeviceDriver(const EMGChannel* const me); struct movingAverageFilter* EMGChannel_getItsMovingAverageFilter(const EMGChannel* const me); EMGChannel * EMGChannel_Create(void); void EMGChannel_Destroy(EMGChannel* const me); #endif

代码清单 6-14: EMGChannel.h

#include "EMGChannel.h" static void initRelations(EMGChannel* const me); static void cleanUpRelations(EMGChannel* const me); void EMGChannel_Init(EMGChannel* c onst me) { initRelations(me); } void EMGChannel_Cleanup(EMGChannel* const me) { cleanUpRelations(me); } void EMGChannel_acquireData(EMGChannel* const me) { /* 委托给相应的部分 */ EMGSensorDeviceDriver_acquireData(&me-> itsEMGSensorDeviceDriver); } double EMGChannel_getFrequency(EMGChannel* const me) { return me->itsMovingAverageFilter.computedFreq; } long EMGChannel_ getLightColor(EMGChannel* const me) { return me->itsCalculateColor.red<<16 + me->itsCalculateColor.green<

<8 + me->its-CalculateColor.blue; }

```
int EMGChannel_getVoltage(EMGChannel* const me) {
  return me->itsEMGSensorDeviceDriver.voltage;
}

void EMGChannel_setSensitivity(EMGChannel* const me, int sen) {
  EMGSensorDeviceDriver_setSensitivity(&me->itsEMGSensorDeviceDriver, sen);
}

struct CalculateColor* EMGChannel_getItsCalculateColor(const EMGChannel* const me) {
  return (struct CalculateColor*)&(me->itsCalculateColor);
}

struct  ConvertToFrequency*  EMGChannel_getItsConvertToFrequency(const  EMGChannel*
const me) {
  return (struct ConvertToFrequency*)&(me->itsConvertToFrequency);
}

struct  EMGSensorDeviceDriver*  EMGChannel_getItsEMGSensorDeviceDriver(const  EMGChan-
nel* const me) {
  return (struct EMGSensorDeviceDriver*)&(me->itsEMGSensorDeviceDriver);
}

struct LightDeviceDriver* EMGChannel_getItsLightDeviceDriver(const EMGChannel* const
me) {
  return (struct LightDeviceDriver*)&(me->itsLightDeviceDriver);
}

struct movingAverageFilter*
EMGChannel_getItsMovingAverageFilter(const EMGChannel* const me) {
  return (struct movingAverageFilter*)&(me->itsMovingAverageFilter);
}

EMGChannel * EMGChannel_Create(void) {
  EMGChannel* me = (EMGChannel *) malloc(sizeof(EMGChannel));
  if(me!=NULL)
    EMGChannel_Init(me);
  return me;
}

void EMGChannel_Destroy(EMGChannel* const me) {
  if(me!=NULL)
    EMGChannel_Cleanup(me);
  free(me);
}

static void initRelations(EMGChannel* const me) {
  CalculateColor_Init(&(me->itsCalculateColor));
  ConvertToFrequency_Init(&(me->itsConvertToFrequency));
  EMGSensorDeviceDriver_Init(&(me->itsEMGSensorDeviceDriver));
```


int EMGChannel_getVoltage(EMGChannel* const me) { return me->itsEMGSensorDeviceDriver.voltage; } void EMGChannel_setSensitivity(EMGChannel* const me, int sen) { EMGSensorDeviceDriver_ setSensitivity(&me->itsEMGSensorDeviceDriver, sen); } struct CalculateColor* EMGChannel_ getItsCalculateColor(const EMGChannel* const me) { return (struct CalculateColor*)&(me-> itsCalculateColor); } struct ConvertToFrequency* EMGChannel_getItsConvertToFrequency(const EMGChannel* const me) { return (struct ConvertToFrequency*)&(me->itsConvertToFrequency); } struct EMGSensorDeviceDriver* EMGChannel_getItsEMGSensorDeviceDriver(const EMGChan- nel* const me) { return (struct EMGSensorDeviceDriver*)&(me->itsEMGSensorDeviceDriver); } struct Light DeviceDriver* EMGChannel_getItsLightDeviceDriver(const EMGChannel* const me) { return (struct LightDeviceDriver*)&(me->itsLightDeviceDriver); } struct movingAverageFilter* EMGChannel_ getItsMovingAverageFilter(const EMGChannel* const me) { return (struct movingAverageFilter*)&(me-> itsMovingAverageFilter); } EMGChannel * EMGChannel_Create(void) { EMGChannel* me = (EMGChannel *) malloc(sizeof(EMGChannel)); if(me!=NULL)EMGChannel_Init(me); return me; } void EMGChannel_Destroy(EMGChannel* const me) { if(me!=NULL)EMGChannel_Cleanup(me); free(me); } static void initRelations(EMGChannel* const me) { CalculateColor_Init(&(me->itsCalculateColor)); ConvertToFrequency_Init(&(me->itsConvertToFrequency)); EMGSensorDeviceDriver_Init(&(me-> itsEMGSensorDeviceDriver));

```
  LightDeviceDriver_Init(&(me->itsLightDeviceDriver));
  movingAverageFilter_Init(&(me->itsMovingAverageFilter));
  EMGSensorDeviceDriver_setItsConvertToFrequency(&(me-
>itsEMGSensorDeviceDriver),&(me->itsConvertToFrequency));
  ConvertToFrequency_setItsMovingAverageFilter(&(me->itsConvertToFrequency),&(me-
>itsMovingAverageFilter));
  movingAverageFilter_setItsCalculateColor(&(me->itsMovingAverageFilter),&(me-
>itsCalculateColor));
  CalculateColor_setItsLightDeviceDriver(&(me->itsCalculateColor),&(me-
>itsLightDeviceDriver));
}

static void cleanUpRelations(EMGChannel* const me) {
  movingAverageFilter_Cleanup(&(me->itsMovingAverageFilter));
  LightDeviceDriver_Cleanup(&(me->itsLightDeviceDriver));
  EMGSensorDeviceDriver_Cleanup(&(me-
>itsEMGSensorDeviceDriver));
  ConvertToFrequency_Cleanup(&(me->itsConvertToFrequency));
  CalculateColor_Cleanup(&(me->itsCalculateColor));
}
```

Code Listing 6-15: EMGChannel.c

6.6 Protected Single Channel Pattern

The Protected Single Channel Pattern is a simple elaboration of the Channel pattern in which data checks are added at one or more `ConcreteDataTransformations`. It provides light-weight redundancy but typically cannot continue to provide service if a fault is discovered.

6.6.1 Abstract

The Protected Single Channel Pattern uses a single channel to handle the processing of sensory data up to and including actuation based on that data. Safety and reliability are enhanced through the addition of checks at key points in the channel, which may require some additional hardware. Because there is only a single channel, this pattern will not be able to continue to function in the presence of persistent faults, but it detects and may be able to handle transient faults.

6.6.2 Problem

Redundancy is expensive, not only in terms of development effort but also in terms of replicated hardware. In some cases, fully redundant channels may not be necessary. For example, if there is a fault-safe state (a state known to always be safe), the system must only be able to detect a fault and enter that state. This lowers the reliability of the system, but it may still meet the user needs.

LightDeviceDriver_Init(&(me->itsLightDeviceDriver)); movingAverageFilter_Init(&(me->itsMovingAverageFilter)); EMGSensorDeviceDriver_setItsConvertToFrequency(&(me->itsEMGSensorDeviceDriver),&(me->itsConvertToFrequency)); ConvertToFrequency_setItsMovingAverageFilter(&(me->itsConvertToFrequency),&(me->itsMovingAverageFilter)); movingAverageFilter_setItsCalculateColor(&(me->itsMovingAverageFilter),&(me->itsCalculateColor)); CalculateColor_setItsLightDeviceDriver(&(me->itsCalculateColor),&(me->itsLightDeviceDriver)); } static void cleanUpRelations(EMGChannel* const me) { movingAverageFilter_Cleanup(&(me->itsMovingAverageFilter)); LightDeviceDriver_Cleanup(&(me->itsLightDeviceDriver)); EMGSensorDeviceDriver_Cleanup(&(me->itsEMGSensorDeviceDriver)); ConvertToFrequency_Cleanup(&(me->itsConvertToFrequency)); CalculateColor_Cleanup(&(me->itsCalculateColor)); }

代码清单 6-15：EMGChannel.c

6.6 受保护单通道模式

受保护单通道模式是通道模式的一种简单扩展，在一个或多个具体数据转换处添加了数据检查。它提供了轻量级冗余，但通常无法在发现故障后继续提供服务。

6.6.1 摘要

受保护单通道模式使用单个通道来处理传感数据，直至基于该数据执行执行操作。通过在通道的关键点添加检查（这可能需要一些额外的硬件），安全性与可靠性得以增强。由于只有一个通道，该模式在存在持续故障时无法继续运行，但它能够检测并可能处理瞬态故障。

6.6.2 问题

冗余的代价很高，不仅体现在开发工作量上，也体现在复制的硬件上。在某些情况下，可能不需要完全冗余的通道。例如，如果存在故障安全状态（一种已知始终安全的状态），系统只需能够检测到故障并进入该状态即可。这会降低系统的可靠性，但仍可能满足用户需求。

6.6.3 Pattern Structure

The pattern, shown in Figure 6-11, is a simple elaboration of the Channel Pattern (see Figure 6-8). All we've added here are some checks on some of the transformation steps. In fact, these checks need not be implemented in separate classes, but can be implemented internal to the `ConcreteTransformations` themselves. These checks can be any kind of checks, such as:

- Range checking (are data in the required range?)
- Reasonableness checks (are data in the expected range for the given situation?)
- Consistency checks (is the value consistent with other data being computed at the same time?)
- Backwards transformational checks (can we recompute the input values from the output values?)
- Data validity checks, such as we've seen before with the One's Complement and CRC Patterns

and so on.

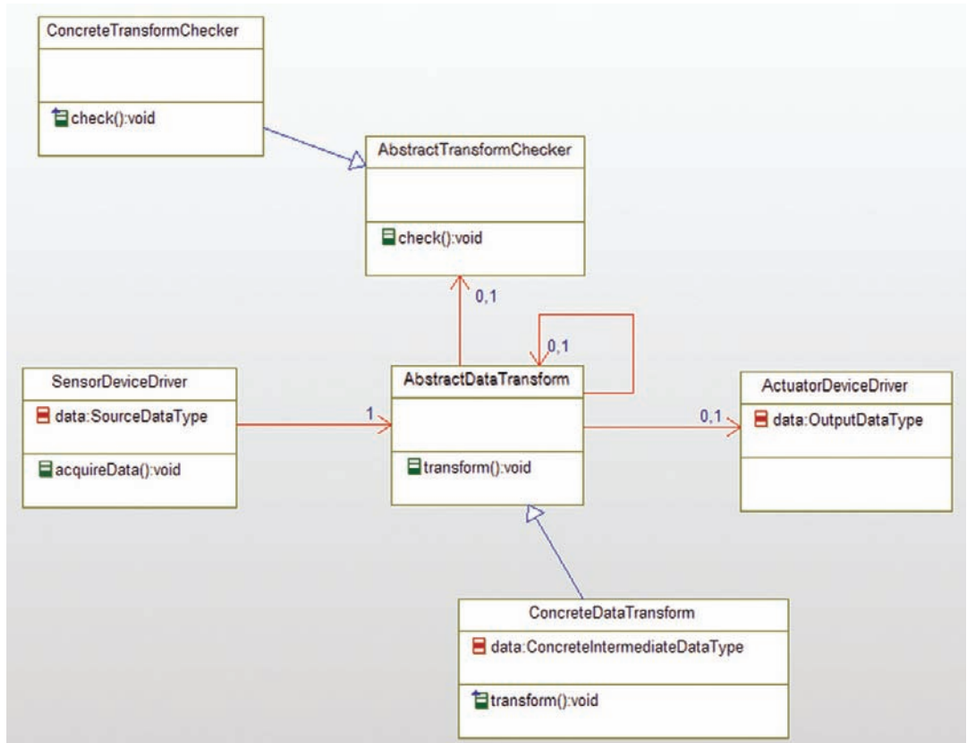


Figure 6-11: Protected Single Channel Pattern

6.6.3 模式结构

图 6-11 所示的模式是通道模式（见图6-8）的简单扩展。我们在这里添加的只是对某些转换步骤的一些检查。事实上，这些检查不必在单独的类中实现，而是可以在具体转换 内部实现。这些检查可以是任何类型的检查，例如：

- 范围检查（数据是否在要求的范围内？）
- 合理性检查（数据是否在给定情况的预期范围内？）
- 一致性检查（value是否与同时计算的其他数据一致？）
- 反向转换检查（我们能否从输出值重新计算出输入值？）

- 数据有效性检查，例如我们之前见过的反码模式和CRC模式

依此类推。

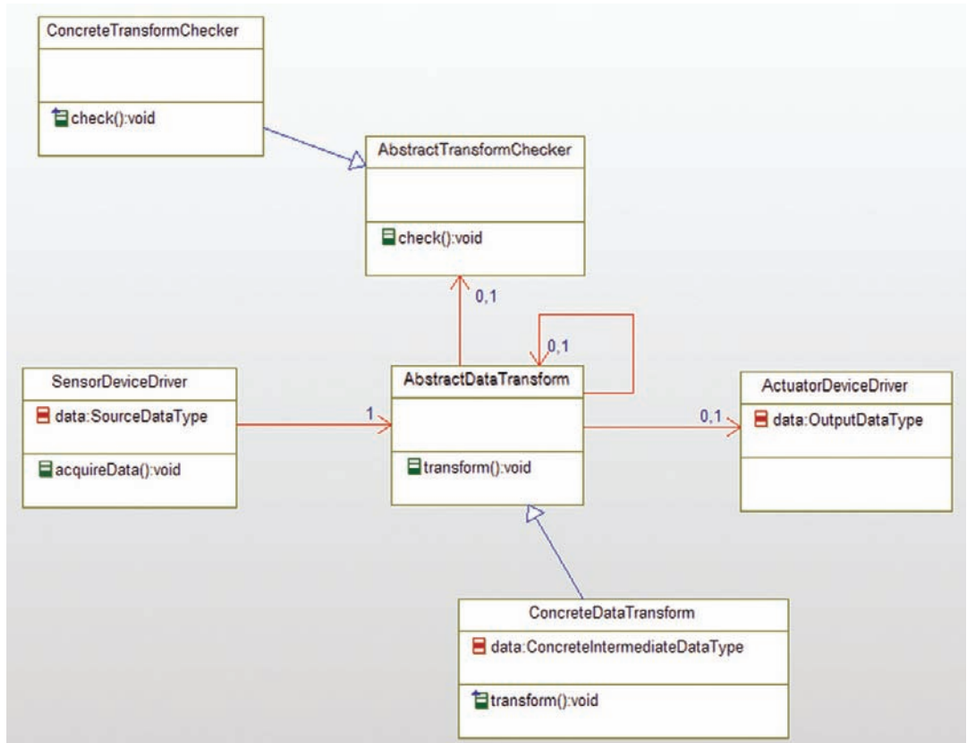


图6-11：受保护单通道模式

6.6.4 Collaboration Roles

This section describes the roles for this pattern.

6.6.4.1 AbstractDataTransform

This class is a placeholder for one of several possible `ConcreteDataTransforms`. In an object-oriented language or implementation, true generalization can be used but in C, it is more common to use this as a notional concept to give the intent rather than drive the implementation. It has two optional[9] associations. One is to itself – that is, the next transformation in the list. The other is to the output `ActuationDeviceDriver` – this is in lieu of the former association. Put another way, the `AbstractDataTransform` has a link to the next processing step in the chain or to the output device driver, but not both.

This class has also an optional link to `AbstractTransformChecker` for points in the transformation chain where checking is appropriate.

6.6.4.2 AbstractTransformChecker

This is a placeholder for an element that can check on the validity of the processing or data in their current state.

6.6.4.3 ActuatorDeviceDriver

This class uses the computed output values to drive one or more actuators, such as a motor, light, or heating unit.

6.6.4.4 ConcreteDataTransform

This class performs a mathematical transformation on the incoming data value (from either the sensor or a previous `ConcreteDataTransform`) and produces an output (sent to the next `ConcreteDataTransform` in the sequence or the `ActuatorDeviceDriver`). In simple cases, there may be only a single `ConcreteDataTransform` class, but in more complex situations there may be dozens.

Certain of these classes form checkpoints – points in the transformation chain at which the validality of the transformations will be checked. For those cases, the optional link to the specific `ConcreteTransformChecker` will be implemented.

6.6.4.5 ConcreteTransformChecker

This is a specific transformation checking class that checks the transformation at a single point in the chain.

6.6.4.6 SensorDeviceDriver

This class acquires physical data from one or more sensors to put into the channel for processing.

[9] Common implementation is a pointer, set to `NULL` if the link isn't active.

6.6.4 协作角色

本节描述该模式的角色。

6.6.4.1 抽象数据变换

该类是若干可能的具体数据变换之一的占位符。在面向对象语言或实现中，可以使用真正的泛化，但在C语言中，更常见的是将其作为一个概念性概念来传达意图，而不是驱动实现。它有两个可选的[9] 关联。一个是与自身——即列表中的下一个变换。另一个是与输出执行设备驱动——这代替了前一个关联。换句话说，抽象数据变换有一个链接指向链中的下一个处理步骤或指向输出设备驱动程序，但不会同时指向两者。

该类还有一个可选链接指向抽象变换检查器，用于变换链中适合进行检查的点。

6.6.4.2 抽象变换检查器

这是一个占位符，用于表示可以检查当前状态下处理或数据有效性的元素。

6.6.4.3 执行器设备驱动

该类使用计算得到的输出值来驱动一个或多个执行器，例如电机、灯或加热单元。

6.6.4.4 ConcreteDataTransform

该类对传入的数据值（来自传感器或前一个`ConcreteDataTransform`）执行数学变换，并产生输出（发送到序列中的下一个`ConcreteDataTransform`或`ActuatorDeviceDriver`）。在简单情况下，可能只有一个`ConcreteDataTransform`类，但在更复杂的情况下可能有数十个。

其中某些类构成检查点——即变换链中将要检查变换有效性的点。对于这些情况，将实现指向特定`ConcreteTransformChecker`的可选链接。

6.6.4.5 具体变换检查器

这是一个特定的变换检查类，用于检查链中单个点处的变换。

6.6.4.6 传感器设备驱动

该类从一个或多个传感器获取物理数据，并将其放入通道以进行处理。

[9] 常见实现是一个指针，如果链接未活动，则将其设置为空。

6.6.5 Implementation Strategies

In general, the benefits are providing a clear means for identifying faults and, depending on the pattern, providing redundant channels to continue to provide services or to go to a "fault-safe state." The drawbacks of using the pattern are the extra memory (both code and data), processing time, and hardware required, depending on the specific usage of the channel.

As mentioned, the pattern may be implemented with true generalization, but it is more common in C to think of the `AbstractDataTransformation` as a placeholder for some particular `ConcreteDataTransform` class; the links are from different subtypes of `ConcereteDataTransform` classes that each perform some special step in the overall data transformation function.

Data are passed through using one of two primary strategies. The simplest is to acquire a datum and pass it through the series of transformations and out to the actuator before acquiring the next. In higher bandwidth applications multiple instances of the data may be present in the channel at the same time, one data instance per transform, with the data passed from one transform to the next at the same time as all the other data instances.

In addition, the data checks may be put "in place" in some of the transformation steps rather than implement them as a separate class. That is, in fact, the specialization of this pattern from the more generic Channel Pattern.

6.6.6 Related Patterns

This pattern provides medium- to large-scale redundancy. It is one of many patterns based on the Channel Pattern discussed earlier in this chapter. A number of such patterns are discussed in my *Real-Time Design Patterns: Robust Scalable Architecture for Real-Time Systems* (Addison-Wesley, 2002).

The transformation checks can themselves be based on patterns such as the One's Complement and CRC Patterns discussed earlier.

6.6.7 Example

In Figure 6-12, I purposely chose a situation that demanded a bit more creative application of the pattern. This is a heating system with sensor sets for multiple rooms, each having sensors for fan speed, temperature, and thermostat settings. Because the example isn't laid out exactly as the pattern, I indicated which pattern role each class plays by adding a stereotype[10] to each.

[10] A stereotype is "a special kind of" marking that appears inside «» marks. In this case, the stereotypes are the names of the pattern roles.

6.6.5 实现策略

一般来说，其优点在于提供了一种清晰的故障识别手段，并且根据具体模式的不同，还能提供冗余通道以继续提供服务或进入"故障安全状态"。使用该模式的缺点在于，根据通道的具体使用情况，需要额外的内存（包括代码和数据）、处理时间以及硬件。

如前所述，该模式可以通过真正的泛化来实现，但在 C 语言中更常见的做法是将 `AbstractDataTransformation` 视为某个特定 `ConcreteDataTransform` 类的占位符；链接来自 `ConcereteDataTransform` 类的不同子类型，每个子类型在整体数据变换函数中执行某个特定步骤。

数据通过两种主要策略之一进行传递。最简单的方法是获取一个数据，将其依次通过一系列变换并输出到执行器，然后再获取下一个数据。在更高带宽的应用中，通道中可能同时存在多个数据实例，每个变换对应一个数据实例，数据从一个变换传递到下一个变换的同时，所有其他数据实例也在同步传递。

此外，数据检查可以"就地"放在某些变换步骤中，而不是将它们实现为单独的类。事实上，这正是该模式从更通用的通道模式中特化而来的方式。

6.6.6 相关模式

该模式提供中等至大规模冗余。它是本章前面讨论的通道模式所衍生的众多模式之一。我在《Real-Time Design Patterns: Robust Scalable Architecture for Real-Time Systems》（Addison-Wesley，2002年）一书中讨论了若干此类模式。

变换检查本身可以基于前面讨论的反码模式和CRC模式等模式。

6.6.7 示例

在图6-12中，我特意选择了一个需要对模式进行更具创造性应用的情境。这是一个具有多个房间传感器组的供暖系统，每个房间都有用于风扇速度、温度和恒温器设置的传感器。由于该示例并未完全按照模式来布局，我通过为每个类添加构造型[10] 来标明每个类所扮演的模式角色。

[10] 构造型是出现在«»标记内的"一种特殊类型"的标记。在本例中，构造型即为模式角色的名称。

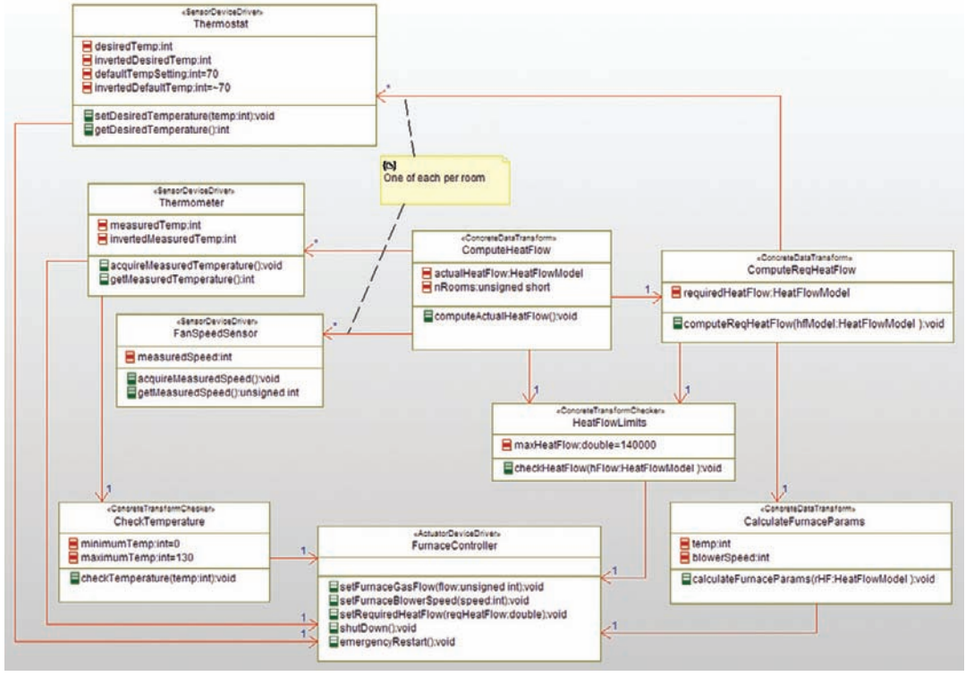


Figure 6-12: Protected Single Channel example

I further modified the pattern by making the first `ConcreteDataTransform` pull the data from its `SensorDeviceDrivers` (the thermometer and room fan) associated with each room. The next transform, `ComputeRequiredHeatFlow`, not only gets the actual heat flow from the previous transform but reads each room's thermostat to get the desired temperature and computes the desired output heat flow from the furnace. The difference between actual and desired temperatures results in a desired heat flow. This is passed on to the `CalculateFurnaceParams`, which computes the necessary furnace gas flow, furnace blower speeds, and commands the `FurnaceController`.

In terms of safety, there are several checks in this protected single channel. The Thermometers are checked for actual temperature, and if the temperature is out of bounds, this leads to either emergency restart of the furnace (if below 0º F) or shutting the furnace down (if over 130º F)[11]. The actual heat flow and (computed) desired heat flow are checked to ensure that they don't exceed 14,000 BTUs. If this limit is exceeded, the furnace is shut down.

[11] That's -18º C and 54º C for you metric-speaking types.

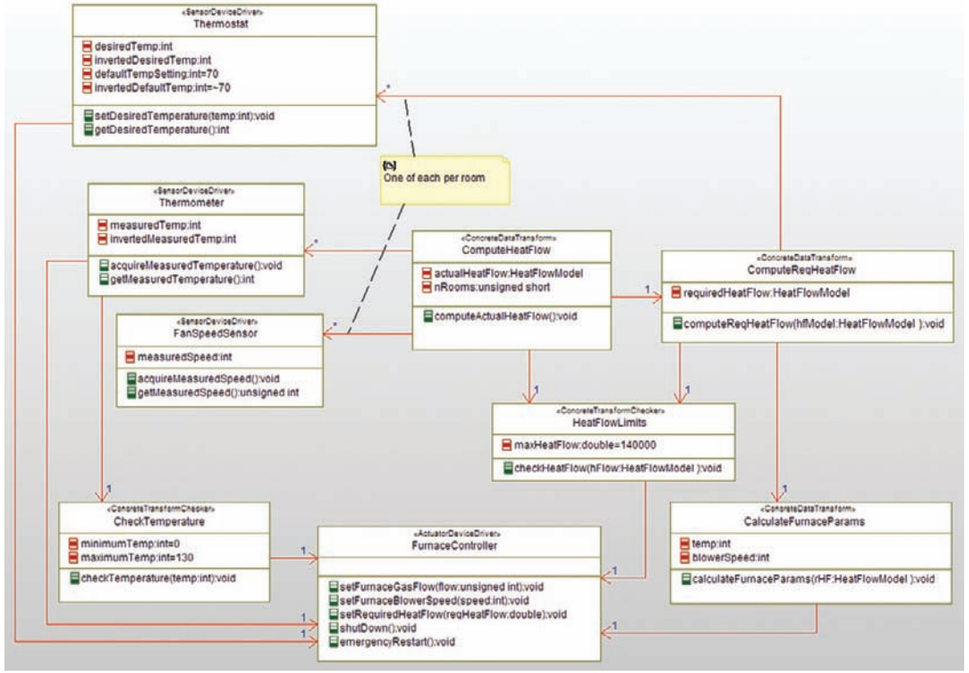


图 6-12：受保护单通道示例

我进一步修改了该模式，让第一个`ConcreteDataTransform`从与每个房间关联的`SensorDeviceDrivers`（温度计和房间风扇）中拉取数据。下一个转换`ComputeRequiredHeatFlow`不仅从上一个转换获取实际热流，还读取每个房间的恒温器以获取期望温度，并计算炉子的期望输出热流。实际温度与期望温度之间的差异产生期望热流。该热流被传递给`CalculateFurnaceParams`，由其计算所需的炉子燃气流量、炉子鼓风机速度，并向`FurnaceController`发送命令。

在安全性方面，这个受保护单通道中有若干项检查。Thermometer 会检查实际温度，如果温度超出范围，则会导致炉子紧急重启（如果低于 0o F）或关闭炉子（如果超过 130o F）[11]。实际热流和（计算出的）期望热流会被检查，以确保它们不超过 14,000 BTU。如果超过此限制，炉子将被关闭。

[11] 对于使用公制的类型来说，那是 -18°C 和 54°C。

In addition, both the `Thermometer` and `Thermostat` use the One's Complement Pattern (described earlier in this chapter) to detect memory corruption. In the case of the `Thermometer`, the behavior on corruption is to shut the furnace down, while in the case of the `Thermostat`, the behavior is to use the default temperature setting of 70º F. If the default is corrupted, then it calls `FurnaceController_shutDown()`.

I don't want to give all the code here, but we can look at a few relevant classes. The `Thermostat` class implements a One's Complement Pattern to ensure its data isn't corrupted. Its code is shown in Code Listing 6-16 and Code Listing 6-17.

```
#ifndef Thermostat_H
#define Thermostat_H

struct FurnaceController;

typedef struct Thermostat Thermostat;
struct Thermostat {
  int defaultTempSetting;
  int desiredTemp;
  int invertedDefaultTemp;
  int invertedDesiredTemp;
  struct FurnaceController* itsFurnaceController;
};

/* Constructors and destructors:*/
void Thermostat_Init(Thermostat* const me);
void Thermostat_Cleanup(Thermostat* const me);

/* Operations */
int Thermostat_getDesiredTemperature(Thermostat* const me);

void Thermostat_setDesiredTemperature(Thermostat* const me, int temp);

struct FurnaceController* Thermostat_getItsFurnaceController(const
Thermostat* const me);

void Thermostat_setItsFurnaceController(Thermostat* const me,
struct FurnaceController* p_FurnaceController);

Thermostat * Thermostat_Create(void);

void Thermostat_Destroy(Thermostat* const me);
#endif
```

Code Listing 6-16: Thermostat.h

此外，`Thermometer` 和`Thermostat` 都使用反码模式（本章前面已描述）来检测内存损坏。对于`Thermometer`，发生损坏时的行为是关闭炉子，而对于`Thermostat`，行为是使用 70o F 的默认温度设置。如果默认值被损坏，则它会调用 `FurnaceController_shutDown()`。

我不想在这里给出所有代码，但我们可以看看几个相关的类。`Thermostat` 类实现了反码模式以确保其数据不被损坏。其代码如代码清单 6-16 和代码清单 6-17 所示。

```
#ifndef Thermostat_H #
define Thermostat_H

struct FurnaceController;

typedef struct Thermostat Thermostat; struct Thermostat {
int defaultTempSetting; int desiredTemp; int
invertedDefaultTemp; int invertedDesiredTemp; struct
FurnaceController* itsFurnaceController; };

/* 构造函数与析构函数：*/ void Thermostat_
Init(Thermostat* const me); void Thermostat_
Cleanup(Thermostat* const me);

/* 操作 */ int Thermostat_getDesiredTemperature(Thermostat* const
me);

void Thermostat_setDesiredTemperature(Thermostat* const me, int temp);

struct FurnaceController* Thermostat_getItsFurnaceController(const Thermostat*
const me);

void Thermostat_setItsFurnaceController(Thermostat* const me, struct
FurnaceController* p_FurnaceController);

恒温器 * 恒温器_Create(void);

void Thermostat_Destroy(Thermostat* const me); #endif
```

代码清单 6-16：Thermostat.h

```
#include "Thermostat.h"
#include "FurnaceController.h"

static void cleanUpRelations(Thermostat* const me);

void Thermostat_Init(Thermostat* const me) {
  me->defaultTempSetting = 70;
  me->invertedDefaultTemp = ~70;
  me->itsFurnaceController = NULL;
}

void Thermostat_Cleanup(Thermostat* const me) {
  cleanUpRelations(me);
}

int Thermostat_getDesiredTemperature(Thermostat* const me) {
  if (me->desiredTemp == ~me->invertedDesiredTemp)
      return me->desiredTemp;
  else
      if (me->defaultTempSetting == ~me->invertedDefaultTemp) {
          Thermostat_setDesiredTemperature(me, me->defaultTempSetting);
          return me->defaultTempSetting;
          }
      else {
          FurnaceController_shutDown(me->itsFurnaceController);
          return me->defaultTempSetting;
          };
}

void Thermostat_setDesiredTemperature(Thermostat* const me, int temp) {
  if (me->desiredTemp == ~me->invertedDesiredTemp) {
      me->desiredTemp = temp;
      me->invertedDesiredTemp = ~temp;
      if (me->desiredTemp == ~me->invertedDesiredTemp)
          return;
      }
  else
      FurnaceController_shutDown(me->itsFurnaceController);
}

struct FurnaceController* Thermostat_getItsFurnaceController(const
Thermostat* const me) {
  return (struct FurnaceController*)me->itsFurnaceController;
}

void Thermostat_setItsFurnaceController(Thermostat* const me, struct FurnaceControl-
ler* p_FurnaceController) {
  me->itsFurnaceController = p_FurnaceController;
}
```


#include "Thermostat.h" #include "FurnaceController.h" static void cleanUpRelations(Thermostat* const me); void Thermostat_Init(Thermostat* const me) { me->defaultTempSetting = 70; me->invertedDefaultTemp = ~70; me->itsFurnaceController = 空; } void Thermostat_清理(Thermostat* const me) { cleanUpRelations(me); } int Thermostat_getDesiredTemperature(Thermostat* const me) { if (me->desiredTemp == ~me->invertedDesiredTemp)return me->desiredTemp; else if (me->defaultTempSetting == ~me->invertedDefaultTemp) { Thermostat_setDesiredTemperature(me, me->defaultTempSetting); return me->defaultTempSetting; } else { FurnaceController_关闭(me->itsFurnaceController); return me->defaultTempSetting; }; } void Thermostat_setDesiredTemperature(Thermostat* const me, int temp) { if (me->desiredTemp == ~me->invertedDesiredTemp) { me->desiredTemp = temp; me->invertedDesiredTemp = ~temp; if (me->desiredTemp == ~me->invertedDesiredTemp)return; } else FurnaceController_关闭(me->itsFurnaceController); } struct FurnaceController* Thermostat_getItsFurnaceController(const Thermostat* const me) { return (struct FurnaceController*)me->itsFurnaceController; } void Thermostat_setItsFurnaceController(Thermostat* const me, struct FurnaceController* p_FurnaceController) { me->itsFurnaceController = p_FurnaceController; }

```
Thermostat * Thermostat_Create(void) {
  Thermostat* me = (Thermostat *) malloc(sizeof(Thermostat));
  if(me!=NULL)
    Thermostat_Init(me);
  return me;
}

void Thermostat_Destroy(Thermostat* const me) {
  if(me!=NULL)
    Thermostat_Cleanup(me);
  free(me);
}

static void cleanUpRelations(Thermostat* const me) {
  if(me->itsFurnaceController != NULL)
    me->itsFurnaceController = NULL;
}
```

Code Listing 6-17: Thermostat.c

The `CheckTemperature` class provides a reasonableness check on the temperature. Code Listing 6-18 and Code Listing 6-19 give the header and implementation.

```
#ifndef CheckTemperature_H
#define CheckTemperature_H

struct FurnaceController;

typedef struct CheckTemperature CheckTemperature;
struct CheckTemperature {
  int maximumTemp;
  int minimumTemp;
  struct FurnaceController* itsFurnaceController;
};

void CheckTemperature_Init(CheckTemperature* const me);

void CheckTemperature_Cleanup(CheckTemperature* const me);

void CheckTemperature_checkTemperature(CheckTemperature* const me, int temp);

struct FurnaceController*
CheckTemperature_getItsFurnaceController(const CheckTemperature* const me);

void CheckTemperature_setItsFurnaceController(CheckTemperature* const
me, struct FurnaceController* p_FurnaceController);

CheckTemperature * CheckTemperature_Create(void);
```


Thermostat * Thermostat_Create(void) { Thermostat* me = (Ther mostat *) malloc(sizeof(Thermostat)); if(me!=NULL)Thermostat_I nit(me); return me; } void Thermostat_Destroy(Thermostat* const me) { if(me!=NULL)Thermostat_Cleanup(me); free(me); } static void cleanUpRelations(Thermostat* const me) { if(me->itsFurnaceController ! = 空) me->itsFurnaceController = 空; }

代码清单 6-17：Thermostat.c

`CheckTemperature` 类对温度进行合理性检查。代码清单 6-18 和代码清单 6-19 给出了头文件和实现。

#ifndef CheckTemperature_H #define CheckTemperature_H 结构体 FurnaceController; typedef 结构体 CheckTemperature CheckTemperature; 结构体 CheckTemperature { int maximumTemp; int minimumTemp; 结构体 FurnaceController* itsFurnaceController; }; void CheckTemperature_Init(CheckTemperature* const me); void CheckTemperature_ Cleanup(CheckTemperature* const me); void CheckTemperature_ checkTemperature(CheckTemperature* const me, int temp); 结构体 FurnaceController* CheckTemperature_getItsFurnaceController(const CheckTemperature* const me); void CheckTemperature_setItsFurnaceController(CheckTemperature* const me, 结构体 FurnaceController* p_FurnaceController); CheckTemperature * CheckTemperature_ Create(void);

```
void CheckTemperature_Destroy(CheckTemperature* const me);
#endif
```

Code Listing 6-18: CheckTemperature.h

```
#include "CheckTemperature.h"
#include "FurnaceController.h"

static void cleanUpRelations(CheckTemperature* const me);

void CheckTemperature_Init(CheckTemperature* const me) {
  me->maximumTemp = 130;
  me->minimumTemp = 0;
  me->itsFurnaceController = NULL;
}

void CheckTemperature_Cleanup(CheckTemperature* const me) {
  cleanUpRelations(me);
}

void CheckTemperature_checkTemperature(CheckTemperature* const me, int temp) {
  if (temp < me->minimumTemp)
      FurnaceController_emergencyRestart(me-
>itsFurnaceController);
  if (temp > me->maximumTemp)
      FurnaceController_shutDown(me->itsFurnaceController);
}

struct FurnaceController*
CheckTemperature_getItsFurnaceController(const CheckTemperature* const me) {
  return (struct FurnaceController*)me->itsFurnaceController;
}

void CheckTemperature_setItsFurnaceController(CheckTemperature*
const me, struct FurnaceController* p_FurnaceController) {
  me->itsFurnaceController = p_FurnaceController;
}

CheckTemperature * CheckTemperature_Create(void) {
  CheckTemperature* me = (CheckTemperature *)
malloc(sizeof(CheckTemperature));
  if(me!=NULL)
    CheckTemperature_Init(me);
  return me;
}

void CheckTemperature_Destroy(CheckTemperature* const me) {
  if(me!=NULL)
    CheckTemperature_Cleanup(me);
  free(me);
}
```


void CheckTemperature_Destroy(CheckTemperature* const me); #endif

代码清单 6-18：CheckTemperature.h

#include "CheckTemperature.h" #include "FurnaceController.h" static void _ cleanUpRelations(CheckTemperature* const me); void CheckTemperature= 130 Init(CheckTemperature* const me) { me->maximumTemp ; me->minimumTemp = 0; me-> itsFurnaceController = 空; } void CheckTemperature_清理(CheckTemperature* const me) { cleanUpRelations(me); } void CheckTemperature_checkTemperature(CheckTemperature* const me, int temp) { if (temp < me->minimumTemp) FurnaceController_紧急重启(me-> itsFurnaceController); if (temp > me->maximumTemp) FurnaceController_关闭(me-> itsFurnaceController); } struct FurnaceController* CheckTemperature_ getItsFurnaceController(const CheckTemperature* const me) { return (struct FurnaceController*)me->itsFurnaceController; } void CheckTemperature_ setItsFurnaceController(CheckTemperature* const me, struct FurnaceController* p_ FurnaceController) { me->itsFurnaceController = p_FurnaceController; } CheckTe mperature * CheckTemperature_Create(void) { CheckTemperature* me = (CheckTemperature *) malloc(sizeof(CheckTemperature)); if(me!=NULL)CheckTemperature_ Init(me); return me; } void CheckTemperature_Destroy(CheckTemperature* const me) { if(me!=NULL)CheckTemperature_清理(me); free(me); }

```
static void cleanUpRelations(CheckTemperature* const me) {
  if(me->itsFurnaceController != NULL)
           me->itsFurnaceController = NULL;
}
```

Code Listing 6-19: CheckTemperature.c

The `CheckTemperature` class is used by the `Thermometer`. The last code listings, **Code Listing 6-20** and **Code Listing 6-21**, give the header and implementation for this class.

```
#ifndef Thermometer_H
#define Thermometer_H

struct CheckTemperature;
struct FurnaceController;

typedef struct Thermometer Thermometer;
struct Thermometer {
  int invertedMeasuredTemp;
  int measuredTemp;
  struct CheckTemperature* itsCheckTemperature;
  struct FurnaceController* itsFurnaceController;
};

/* Constructors and destructors:*/

void Thermometer_Init(Thermometer* const me);

void Thermometer_Cleanup(Thermometer* const me);

/* Operations */
void Thermometer_acquireMeasuredTemperature(Thermometer* const me);

int Thermometer_getMeasuredTemperature(Thermometer* const me);

struct CheckTemperature* Thermometer_getItsCheckTemperature(const
Thermometer* const me);

void Thermometer_setItsCheckTemperature(Thermometer* const me,
struct CheckTemperature* p_CheckTemperature);

struct FurnaceController* Thermometer_getItsFurnaceController(const
Thermometer* const me);

void Thermometer_setItsFurnaceController(Thermometer* const me,
struct FurnaceController* p_FurnaceController);
```


static void cleanUpRelations(CheckTemperature* const me) { if(me->
itsFurnaceController != 空) me->itsFurnaceController = 空; }

代码清单 6-19：CheckTemperature.c

`CheckTemperature` 类由 `Thermometer` 使用。最后两个代码清单，代码清单 6-20 和代码清单 6-21，给出了该类的头文件和实现。

#ifndef Thermometer_H #define Thermometer_H struct CheckTemperature; struct FurnaceController; typedef struct Thermometer Thermometer; struct Thermometer { int invertedMeasuredTemp; int measuredTemp; struct CheckTemperature* itsCheckTemperature; struct FurnaceController* itsFurnaceController; }; /* 构造函数与析构函数：*/

void Thermometer_Init(Thermometer* const me);

void Thermometer_Cleanup(Thermometer* const me);

/* 操作 */ void Thermometer_acquireMeasuredTemperature(Thermometer* const me);

int Thermometer_getMeasuredTemperature(Thermometer* const me);

struct CheckTemperature* Thermometer_getItsCheckTemperature(const Thermometer* const me);

void Thermometer_setItsCheckTemperature(Thermometer* const me, 结构体 CheckTemperature* p_CheckTemperature);

struct FurnaceController* Thermometer_getItsFurnaceController(const Thermometer* const me);

void Thermometer_setItsFurnaceController(Thermometer* const me, str uct FurnaceController* p_FurnaceController);

```
Thermometer * Thermometer_Create(void);

void Thermometer_Destroy(Thermometer* const me);

struct CheckTemperature*
Thermometer_getItsCheckTemperature_1(const Thermometer* const me);

void Thermometer_setItsCheckTemperature_1(Thermometer* const me,
struct CheckTemperature* p_CheckTemperature);

#endif
```

Code Listing 6-20: Thermometer.h

```
#include "Thermometer.h"
#include "CheckTemperature.h"
#include "FurnaceController.h"

static void cleanUpRelations(Thermometer* const me);

void Thermometer_Init(Thermometer* const me) {
  me->itsCheckTemperature = NULL;
  me->itsFurnaceController = NULL;
}
void Thermometer_Cleanup(Thermometer* const me) {
  cleanUpRelations(me);
}
void Thermometer_acquireMeasuredTemperature(Thermometer* const me) {
  /* access the hardware here */
}
int Thermometer_getMeasuredTemperature(Thermometer* const me) {
  if (me->measuredTemp == ~me->invertedMeasuredTemp) {
      CheckTemperature_checkTemperature(me->itsCheckTemperature,
 me->measuredTemp);
      return me->measuredTemp;
      }
  else {
      FurnaceController_shutDown(me->itsFurnaceController);
      return 0;
      }
}
struct CheckTemperature* Thermometer_getItsCheckTemperature(const
Thermometer* const me) {
  return (struct CheckTemperature*)me->itsCheckTemperature;
}
void Thermometer_setItsCheckTemperature(Thermometer* const me,
struct CheckTemperature* p_CheckTemperature) {
  me->itsCheckTemperature = p_CheckTemperature;
}
```



```
Thermometer * Thermometer_Create(void);

void Thermometer_Destroy(Thermometer* const me);

struct CheckTemperature* Thermometer_getItsCheckTemperature_1(const
Thermometer* const me);

void Thermometer_setItsCheckTemperature_1(Thermometer* const me, 结构
体 CheckTemperature* p_CheckTemperature);

#endif
```

代码清单 6-20: Thermometer.h

#include "Thermometer.h" #include "CheckTemperature.h" #include "
FurnaceController.h" static void cleanUpRelations(Thermometer* const me); void
Thermometer_Init(Thermometer* const me) { me->itsCheckTemperature =
NULL; me->itsFurnaceController = NULL; } void Thermometer_
Cleanup(Thermometer* const me) { cleanUpRelations(me); } void Thermometer_
acquireMeasuredTemperature(Thermometer* const me) { /* 在此处访问硬件 */ }
int Thermometer_getMeasuredTemperature(Thermometer* const me) { if (me->
measuredTemp == ~me->invertedMeasuredTemp) { CheckTemperature_
checkTemperature(me->itsCheckTemperature, me->measuredTemp); return me->
measuredTemp; } else { FurnaceController_shutDown(me->itsF
urnaceController); return 0; } }

struct CheckTemperature* Thermometer_getItsCheckTemperature(const
Thermometer* const me) { return (struct CheckTemperature*)me->
itsCheckTemperature; } void Thermometer_
setItsCheckTemperature(Thermometer* const me, struct CheckTemperature* p_
CheckTemperature) { me->itsCheckTemperature = p_CheckTemperature; }

```
struct FurnaceController* Thermometer_getItsFurnaceController(const
Thermometer* const me) {
  return (struct FurnaceController*)me->itsFurnaceController;
}

void Thermometer_setItsFurnaceController(Thermometer* const me,
struct FurnaceController* p_FurnaceController) {
  me->itsFurnaceController = p_FurnaceController;
}

Thermometer * Thermometer_Create(void) {
  Thermometer* me = (Thermometer *)
  malloc(sizeof(Thermometer));
  if(me!=NULL)
    Thermometer_Init(me);
  return me;
}

void Thermometer_Destroy(Thermometer* const me) {
  if(me!=NULL)
    Thermometer_Cleanup(me);
  free(me);
}

static void cleanUpRelations(Thermometer* const me) {
  if(me->itsCheckTemperature != NULL)
    me->itsCheckTemperature = NULL;
  if(me->itsFurnaceController != NULL)
          me->itsFurnaceController = NULL;
}
```

Code Listing 6-21: Thermometer.c

6.7 Dual Channel Pattern

The Dual Channel Pattern is primarily a pattern to improve reliability by offering multiple channels, thereby addressing redundancy concerns at the architectural level. If the channels are identical (called the Homogeneous Redundancy Channel), the pattern can address random faults (failures) but not systematic faults (errors). If the channels use a different design or implementation, the pattern is called the Heterogeneous Redundancy Pattern (also known as Diverse Design Pattern) and can address both random and systematic faults.

6.7.1 Abstract

The Dual Channel Pattern provides architectural redundancy to address safety and reliability concerns. It does this by replicating channels and embedding logic that manages them and determines when the channels will be "active."

struct FurnaceController* Thermometer_getItsFurnaceController(const Thermometer* const me) { return (struct FurnaceController*)me-> itsFurnaceController; } void Thermometer_ setItsFurnaceController(Thermometer* const me, struct FurnaceController* p_ FurnaceController) { me->itsFurnaceController = p_FurnaceController; } Thermometer * Thermometer_Create(void) { Thermometer* me = (Ther mometer *) malloc(sizeof(Thermometer)); if(me!=NULL)Thermometer_Init(me); return me; } void Thermometer_Destroy(Thermometer* const me) { if(me!=NULL) Thermometer_Cleanup(me); free(me); } static void cleanUpRelations(Thermometer* const me) { if(me->itsCheckTemperature != 空) me->itsCheckTemperature = 空; if(me->itsFurnaceController != 空) me-> itsFurnaceController = 空; }

代码清单 6-21：Thermometer.c

6.7 双通道模式

双通道模式主要是一种通过提供多个通道来提高可靠性的模式，从而在架构层面解决冗余问题。如果通道完全相同（称为同构冗余通道），该模式可以处理随机故障（失效），但无法处理系统性故障（错误）。如果通道采用不同的设计或实现，该模式称为异构冗余模式（也称为多样性设计模式），可以同时处理随机故障和系统性故障。

6.7.1 摘要

双通道模式提供架构冗余，以解决安全性与可靠性问题。它通过复制通道并嵌入管理这些通道的逻辑以及确定通道何时处于"活动"状态的逻辑来实现这一点。

6.7.2 Problem

This pattern provides protection against single-point faults (either failures or both failures and errors, depending on the specific subpattern selected). Depending on which pattern, the system may detect a fault in one channel by comparing it to the other and then transition to the fault-safe state OR it may use other means to detect the fault in one channel and switch to the other when a fault occurs.

6.7.3 Pattern Structure

Figure 6-13 shows the basic structure for the pattern.

Figure 6-14 shows the detailed internal structure within the pattern. It is almost exactly what appears in the Protected Single Channel Pattern, with the exception of the association from the `ConcreteTransformChecker` to the current and alternative channel. Note that the `itsChannel` association points to the *current* channel that owns the `ConcreteTransformChecker` instance while the `itsOtherChannel` points to the alternative channel. This allows the checker to disable the current channel and enable the alternative channel.

6.7.4 Collaboration Roles

This section describes the roles for this pattern. Note that the roles that are internal to the pattern can be seen in the previous section on the Protected Single Channel Pattern.

6.7.4.1 AbstractDataTransform

This class is a placeholder for one of several possible `ConcreteDataTransforms`. In an object-oriented language or implementation, true generalization can be used but in C, it is more common to use this as a notional concept to give the intent rather than drive the implementation. It has two optional[12] associations. One is to itself – that is, the next transformation in the list. The other is to the output `ActuationDeviceDriver` – this is in lieu of the former association. Put another way, the `AbstractDataTransform` has a link to the next processing step in the chain or to the output device driver, but not both.

This class also an optional link to `AbstractTransformChecker` for points in the transformation chain where checking is appropriate.

6.7.4.2 AbstractTransformChecker

This is a placeholder for an element that can check on the validity of the processing or data in its current state.

[12] Common implementation is a pointer, set to NULL if the link isn't active.

6.7.2 问题

该模式提供针对单点故障的保护（根据所选的具体子模式，可以是故障，也可以是故障和错误两者）。根据具体模式的不同，系统可以通过将一个通道与另一个通道进行比较来检测其中一个通道中的故障，然后迁移到故障安全状态；或者它可以使用其他手段来检测其中一个通道中的故障，并在发生故障时切换到另一个通道。

6.7.3 模式结构

图 6-13 展示了该 基本 结构 。

图 6-14 展示了该模式内部的详细结构。它与受保护单通道模式几乎完全相同，区别在于从具体变换检查器 到当前通道和替代通道的关联。注意，`itsChannel` 关联指向拥有具体变换检查器 实例的当前通道，而`itsOtherChannel` 指向替代通道。这使得检查器能够禁用当前通道并启用替代通道。

6.7.4 协作角色

本节描述该模式的角色。请注意，模式内部的角色可以在上一节关于受保护单通道模式的内容中看到。

6.7.4.1 抽象数据变换

该类是若干可能的具体数据变换之一的占位符。在面向对象语言或实现中，可以使用真正的泛化，但在C语言中，更常见的是将其作为一个概念性概念来表达意图，而不是驱动实现。它有两个可选的[12] 关联。一个是与自身——即列表中的下一个变换。另一个是与输出执行设备驱动 ——这是替代前一个关联的方式。换句话说，抽象数据变换 链接到链中的下一个处理步骤或输出设备驱动程序，但不会同时链接到两者。

该类还有一个可选链接指向抽象变换检查器 ，用于变换链中适合进行检查的点。

6.7.4.2 抽象变换检查器

这是一个占位符，用于表示可以检查其当前状态下的处理或数据有效性的元素。

[12] 常见实现是一个指针，如果链接未活动则设为空。

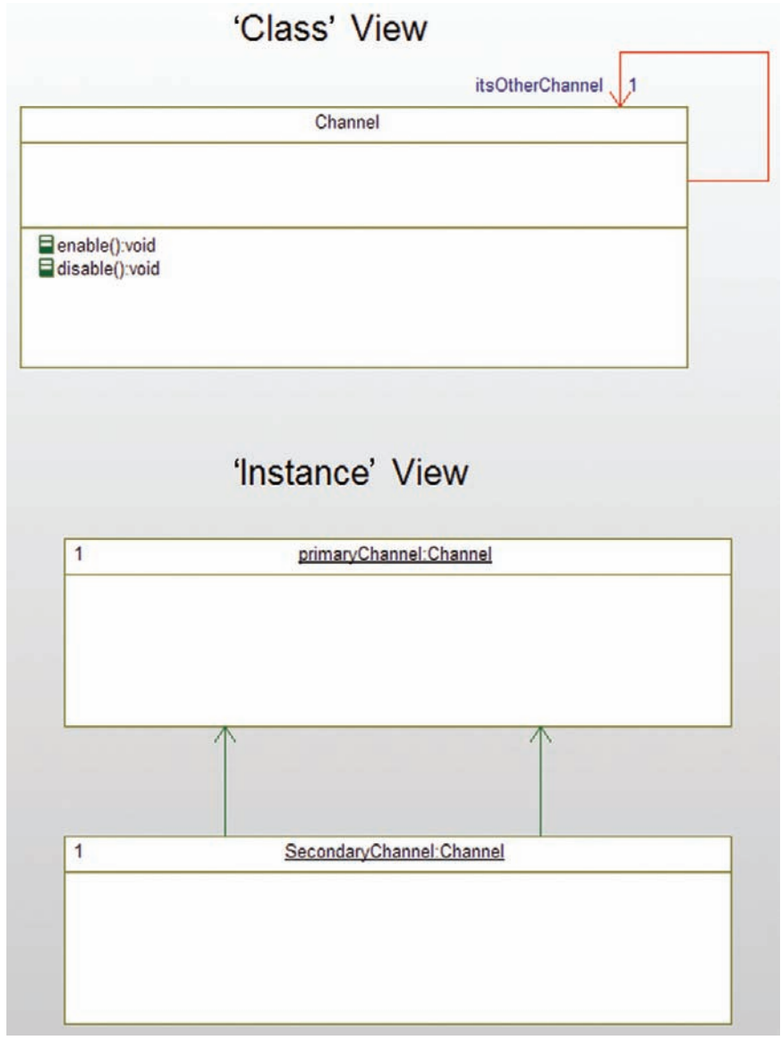


Figure 6-13: Dual Channel Pattern

6.7.4.3 ActuatorDeviceDriver

This class uses the computed output values to drive one or more actuator, such as a motor, light, or heating unit.

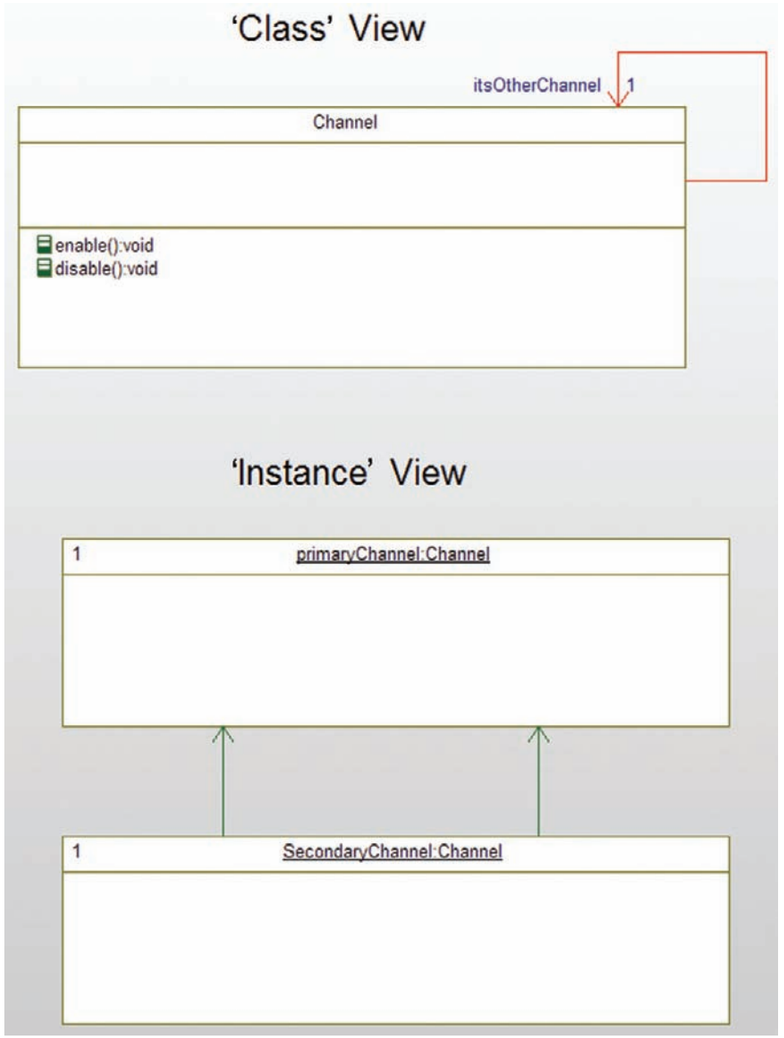


图 6-13：双通道模式

6.7.4.3 执行器设备驱动

该类使用计算出的输出值来驱动一个或多个执行器，例如电机、Light或加热单元。

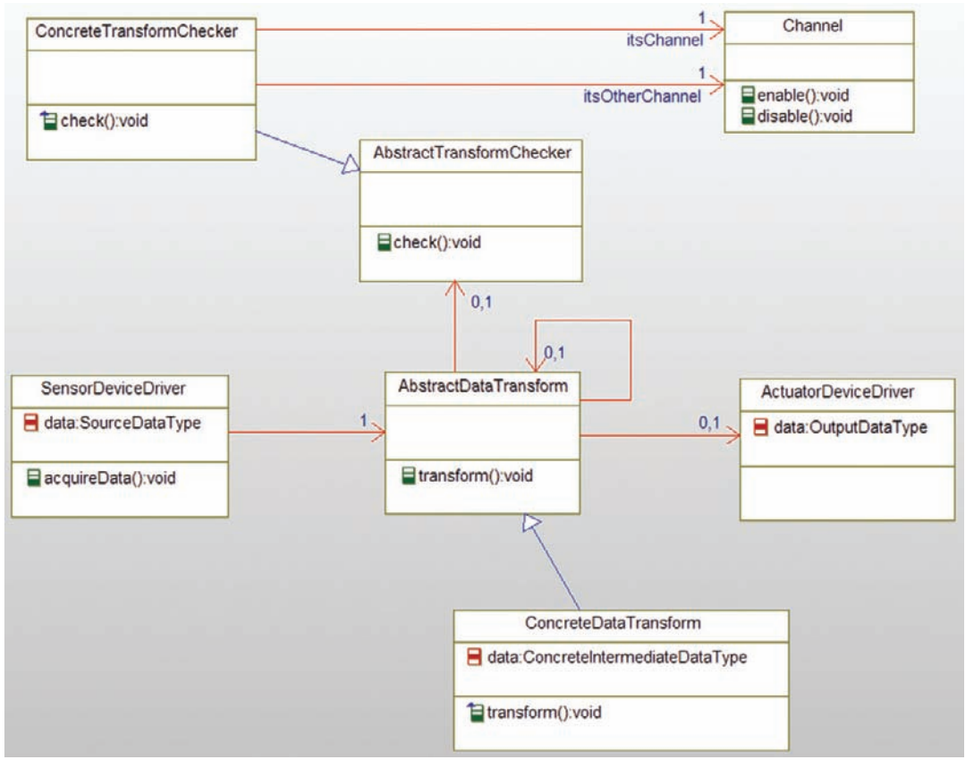


Figure 6-14: Dual Channel Pattern detailed class view

6.7.4.4 Channel

This class is the container for the instances of the others. It is important in this context because it provides a "unit of redundancy." It provides two services of itself – `enable()` and `disable()`.

6.7.4.5 ConcreteDataTransform

This class performs a mathematical transformation on the incoming data value (from either the sensor or a previous `ConcreteDataTransform`) and produces an output (sent to the next `ConcreteDataTransform` in the sequence or the `ActuatorDeviceDriver`). In simple cases, there may be only a single `ConcreteDataTransform` class, but in more complex situations there may be dozens.

Certain of these classes form checkpoints – points in the transformation chain at which the validity of the transformations will be checked. For those cases, the optional link to the specific `ConcreteTransformChecker` will be implemented.

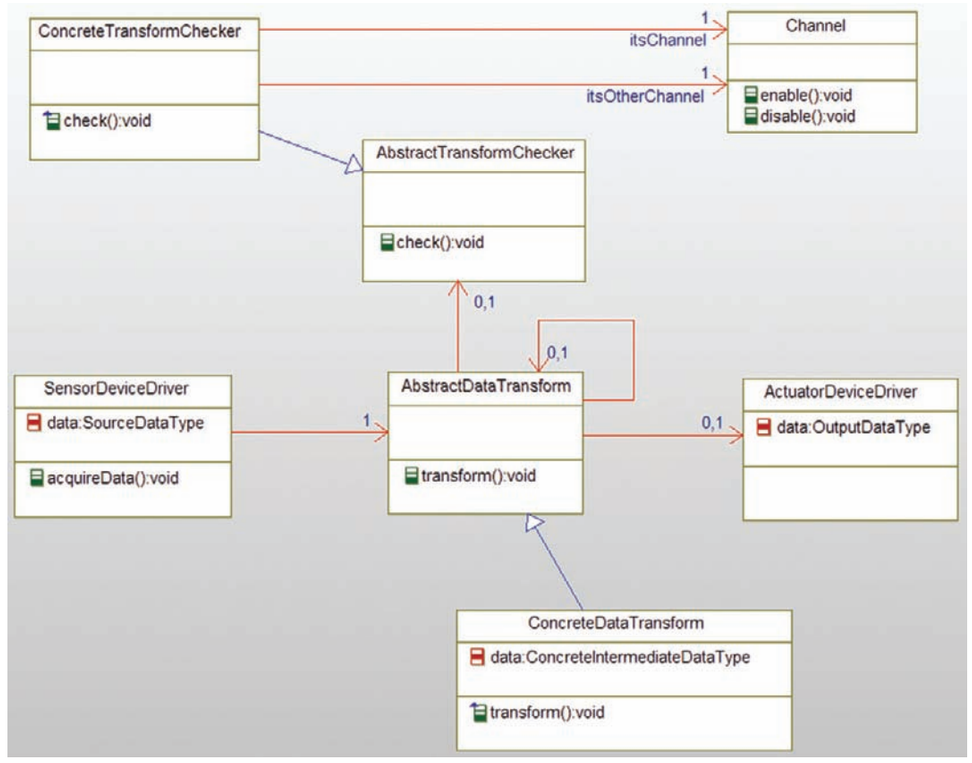


图 6-14：双通道模式详细类视图

6.7.4.4 通道

该类是其他类实例的容器。在此上下文中它很重要，因为它提供了一个"冗余单元"。它提供自身的两个服务——`enable()` 和 `disable()`。

6.7.4.5 ConcreteDataTransform

该类对传入的数据值（来自传感器或前一个 `ConcreteDataTransform`）执行数学变换，并产生输出（发送到序列中的下一个`ConcreteDataTransform` 或 `ActuatorDeviceDriver`）。在简单情况下，可能只有一个 `ConcreteDataTransform` 类，但在更复杂的情况下可能有数十个。

其中某些类构成检查点——即变换链中将在这些点检查变换有效性的点。对于这些情况，将实现指向特定具体变换检查器 的可选链接。

6.7.4.6 ConcreteTransformChecker

This is a specific transformation checking class that checks the transformation at a single point in the chain.

6.7.4.7 SensorDeviceDriver

This class acquires physical data from one or more sensors to put into the channel for processing.

6.7.5 Consequences

This pattern addresses safety- or reliability-related faults by replicating the channel; this usually requires replication of some amount of hardware as well, giving it a rather high recurring (production) cost. If the channels are identical, then all replicants contain the same errors and so will manifest the error under the same circumstances.

6.7.6 Implementation Strategies

The large-scale implementation strategies are commonly identified as different subpatterns of this one. These are discussed in the next section. In addition, the management of the two channels can be implemented differently. Both channels can be run simultaneously and checked against each other – if the outputs differ above some threshold, the system transitions to a fault-safe state. Alternatively, one channel can run until a fault is detected and then the other channel can be enabled, allowing continuation of services in the presence of the fault.

6.7.7 Related Patterns

There are a number of variants on this pattern. Each gives a different cost/benefit ratio.

6.7.7.1 Homogeneous Redundancy Pattern

Description

This pattern variant uses identical designs and implementations for the different channels. It can effectively address single-point faults – provided that the faults are isolated within a single channel. The system can continue functionality in the presence of a failure within a channel.

Consequences

This pattern variant has a relatively high recurring cost (cost per shipped item) but a relatively low design cost (since each channel must only be designed once). Typically sensors and actuators are replicated to provide fault isolation, and often processors, memory, and other hardware must be replicated.

6.7.4.6 具体变换检查器

这是一个特定的变换检查类，它在链中的单个点检查变换。

6.7.4.7 传感器设备驱动

该类从一个或多个传感器获取物理数据，并将其放入通道以供处理。

6.7.5 结果

该模式通过复制通道来解决与安全性或可靠性相关的故障；这通常还需要复制一定数量的硬件，因此其经常性（生产）成本相当高。如果通道完全相同，那么所有副本都包含相同的错误，因此会在相同的情况下显现该错误。

6.7.6 实现策略

大规模实现策略通常被识别为该模式的不同子模式。这些将在下一节中讨论。此外，两个通道的管理可以以不同方式实现。两个通道可以同时运行并相互校验——如果输出差异超过某个阈值，系统将迁移到故障安全状态。或者，一个通道可以运行直到检测到故障，然后启用另一个通道，从而在存在故障的情况下继续提供服务。

6.7.7 相关模式

该模式有许　多变体。每种变体提供不同的成本/收益比　　.

6.7.7.1 同构冗余模式

描述

此模式变体对不同通道使用相同的设计和实现。它可以有效处理单点故障——前提是故障被隔离在单个通道内。当某个通道内发生故障时，系统仍可继续运行功能。

结果

该模式变体具有相对较高的经常性成本（每件出货产品的成本），但设计成本相对较低（因为每个通道只需设计一次）。通常，传感器和执行器会被复制以提供故障隔离，并且通常处理器、内存和其他硬件也必须复制。

6.7.7.2 Heterogeneous Redundancy Pattern

Description

This pattern variant uses dual channels of different designs or different implementations to address both random and systematic faults. The system can continue to deliver services in the presence of a fault, provided that the fault is properly isolated within a single channel. Typically, sensors and actuators are replicated to provide fault isolation, and often processors, memory, and other hardware must be replicated as well as providing multiple (different) implementations of the same software functionality.

Consequences

This pattern variant has a relatively high recurring cost (cost per shipped item) and a relatively high design cost (since each channel must be designed twice). Typically, sensors and actuators are replicated to provide fault isolation, and often processors, memory, and other hardware must be replicated in addition to the software functionality. Note that this pattern variant addresses both random and systematic faults, although Nancy Leveson notes that there is usually some correlation between errors even in diverse implementations[13].

6.7.7.3 Triple Modular Redundancy (TMR) Pattern

Description

This pattern variant uses three channels of usually identical designs to address both faults. The theory of use is that if there is a single-point fault then one of the channels will disagree with the other two and the outlier is discarded. The voting mechanism (the `ChannelArbiter`) is designed to "fail on" so that at least one channel (it doesn't matter which) will result in an output if the voting mechanism fails. The system can continue to deliver services in the presence of a fault, provided that the fault is properly isolated within a single channel. Typically, sensors and actuators are replicated to provide fault isolation, and often processors, memory, and other hardware must be replicated as well as providing multiple (different) implementations of the same software functionality.

Consequences

This pattern has a very high recurring cost because the channel must be replicated three times. If all the channels have the same design (most common), then the design cost is relatively low; the only additional cost is the `ChannelArbiter` used to reject the outlier output. If the channels are of different designs, then the cost of this pattern variant is very high since each channel must be designed three times. This is a very common pattern in avionics.

[13] See Leveson, N., 1995. *Safeware: System Safety and Computers*. Addison-Wesley.

6.7.7.2 异构冗余模式

描述

该模式变体使用不同设计或不同实现的双通道来应对随机故障和系统性故障。只要故障在单个通道内被正确隔离，系统就能在出现故障时继续提供服务。通常，传感器和执行器会被复制以提供故障隔离，并且通常处理器、内存和其他硬件也必须复制，同时为相同的软件功能提供多种（不同的）实现。

结果

该模式变体具有相对较高的经常性成本（每件交付产品的成本）和相对较高的设计成本（因为每个通道必须设计两次）。通常，传感器和执行器会被复制以提供故障隔离，并且除了软件功能之外，处理器、内存和其他硬件通常也必须复制。请注意，该模式变体同时处理随机故障和系统性故障，尽管Nancy Leveson指出，即使在多样化实现中，错误之间通常也存在某种相关性[13]。

6.7.7.3 三模冗余（TMR）模式

描述

此模式变体使用通常为相同设计的三个通道来应对这两种故障。其使用理论是：如果存在单点故障，则其中一个通道将与另外两个通道不一致，从而将离群值丢弃。表决机制（`ChannelArbiter`）被设计为"故障导通"，以便在表决机制发生故障时，至少有一个通道（哪个通道无关紧要）能够产生输出。只要故障被正确隔离在单个通道内，系统就可以在存在故障的情况下继续提供服务。通常，传感器和执行器会被复制以提供故障隔离，并且通常处理器、内存和其他硬件也必须被复制，同时提供同一软件功能的多重（不同）实现。

结果

该模式的经常性成本非常高，因为通道必须复制三次。如果所有通道采用相同的设计（最常见的情况），那么设计成本相对较低；唯一的额外成本是用于剔除离群值输出的`ChannelArbiter`。如果各通道采用不同的设计，那么该模式变体的成本非常高，因为每个通道都必须设计三次。这是航空电子领域非常常见的模式。

[13] 参见 Leveson, N., 1995. Safeware: System Safety and Computers. Addison-Wesley

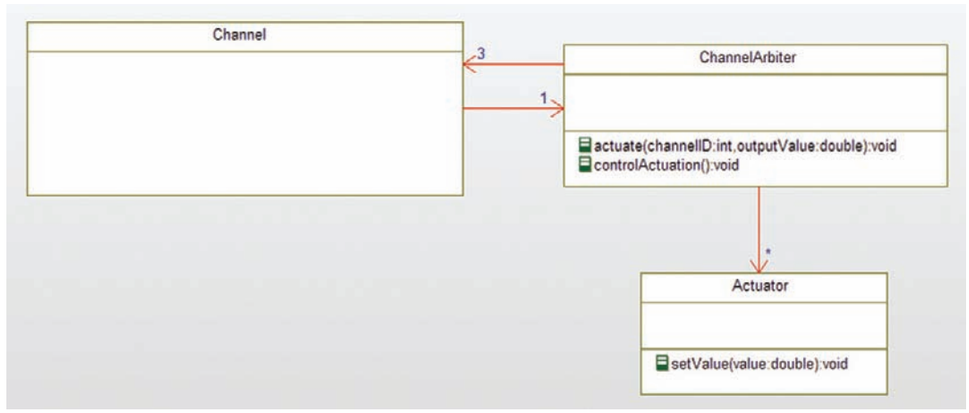


Figure 6-15: Triple Modular Redundancy Pattern

6.7.7.4 Sanity-Check Pattern

Description

This pattern variant uses two heterogeneous channels; one is the primary actuation channel and the second is a lightweight channel that checks on the output using lower fidelity computations and hardware. If the primary channel has a fault that can be detected by the lower fidelity checking channel, then the system enters its fault-safe state.

Consequences

This pattern has a low recurring cost and a medium design cost since it requires additional design work but lower fidelity redundancy. It cannot continue in the presence of a single fault.

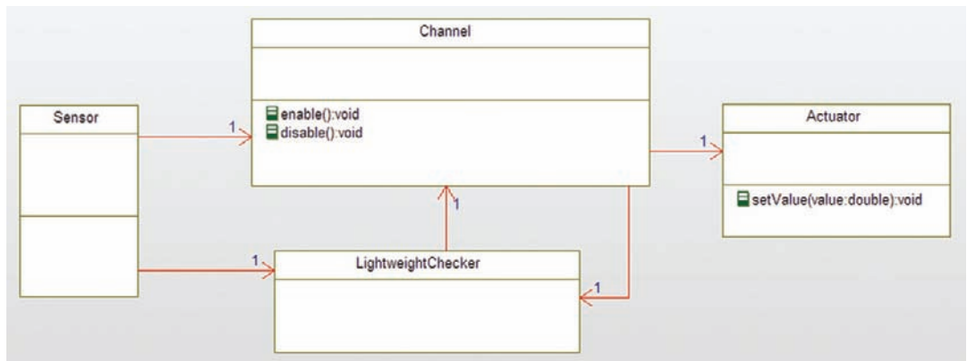


Figure 6-16: Sanity Check Pattern

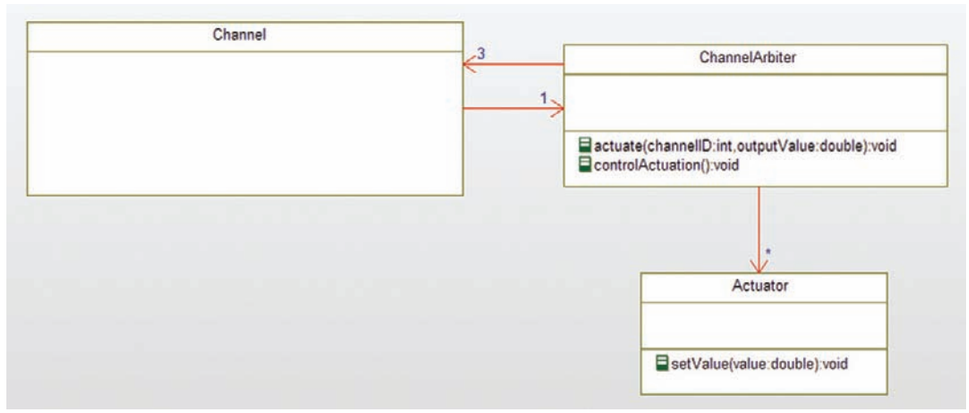


图 6-15：三模冗余模式

6.7.7.4 合理性检查模式

描述

该模式变体使用两个异构通道；一个是主驱动通道，第二个是轻量级通道，它使用低保真度计算和硬件对输出进行检查。如果主通道出现故障且该故障可被低保真度检查通道检测到，则系统进入其故障安全状态。

结果

该模式具有较低的经常性成本和中等设计成本，因为它需要额外的设计工作，但冗余的保真度较低。在出现单一故障时，它无法继续运行。

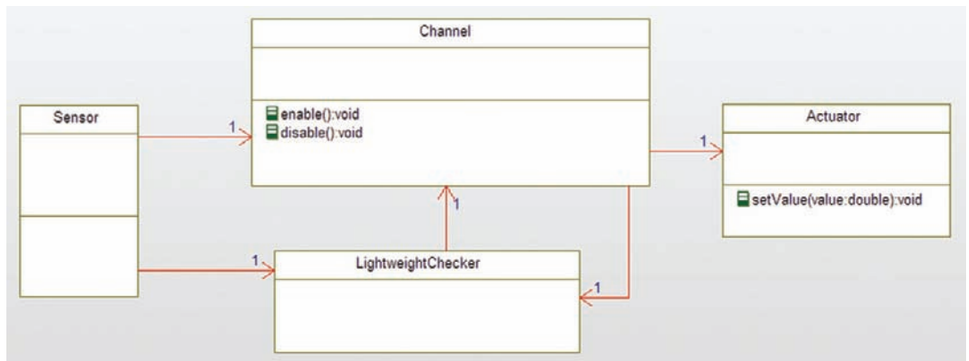


图 6-16：合理性检查模式

6.7.7.5 Monitor-Actuator Pattern

Description

This pattern variant uses two extremely heterogeneous channels. The first, just as in the Sanity Check Pattern, is the actuator channel. This channel provides the system service. The second channel monitors the physical result of the actuation channel using one or more independent sensors. The concept of the pattern is that if the actuator channel has a fault and performs incorrectly, the monitoring channel identifies it and can command the system to a fault-safe state. If the monitoring channel has a fault, then the actuation channel is still behaving correctly. This pattern assumes there is a fault-safe state and that the sensor(s) used by the monitoring channel are not used by the actuation channel.

The primary differences between the Monitor-Actuator Pattern and the Sanity Check Pattern are:

1. The Sanity Check Pattern is a low-fidelity check on the output of the computation of the actuation channel; this does not require an independent sensor.
2. The Monitor-Actuator is a high-fidelity check on the physical result of the actuator channel and therefore requires an independent sensor.

Consequences

This is a medium-weight pattern. It requires some additional recurring cost because of the additional sensor(s) and processing capability for the monitoring channel. The extra monitoring channel requires a modest amount of additional design work as well. As mentioned, this channel provides single-point fault safety for systems in which there is a fault-safe state.

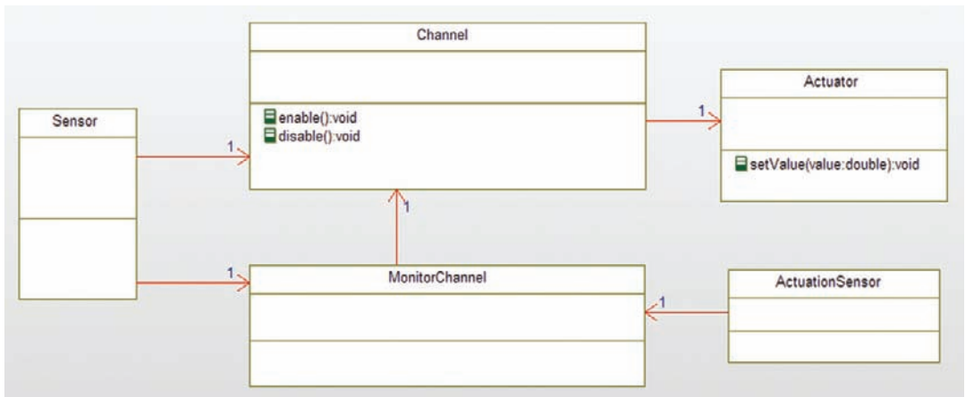


Figure 6-17: Monitor-Actuator Pattern

6.7.7.5 监控器-执行器模式

描述

该模式变体使用两个极其异构的通道。第一个通道与合理性检查模式中一样，是执行器通道。该通道提供系统服务。第二个通道使用一个或多个独立传感器来监控执行器通道的物理结果。该模式的理念是，如果执行器通道发生故障并执行不正确，监控通道会识别出该故障，并可以命令系统进入故障安全状态。如果监控通道发生故障，则执行器通道仍能正确运行。该模式假设存在故障安全状态，并且监控通道所使用的传感器不被执行器通道使用。

监控器-执行器模式与合理性检查模式的主要区别在于：

1. 合理性检查模式是对执行通道计算输出的低保真度检查；这不需要独立传感器。
2. 监控器-执行器模式是对执行器通道物理结果的高保真度检查，因此需要独立传感器。

结果

这是一种中等权重的模式。由于需要额外的传感器以及监控通道的处理能力，它需要一些额外的经常性成本。额外的监控通道也需要适量的额外设计工作。如前所述，该通道为具有故障安全状态的系统提供单点故障安全。

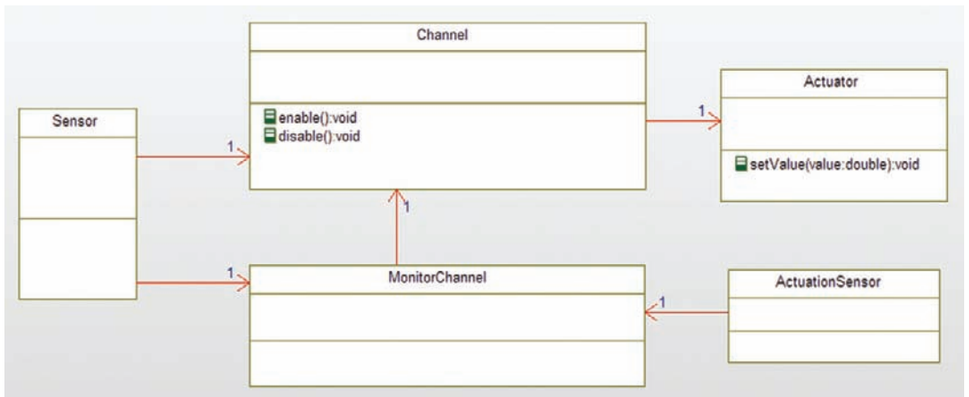


图 6-17：监控器-执行器模式

6.7.8 Example

The example shown in Figure 6-18 is a heterogeneous dual channel pattern for a train speed control system. One channel uses a light shining on the inside of a train wheel. By knowing the circumference of the wheel and by measuring the frequency of the mark appearance (measured by the optical sensor), one can compute the speed of the train. These data are then filtered, validated, and used to compute a desired engine output. This engine output is checked for reasonableness and, if acceptable, then sent to the engine. This is the `OpticalWheelSpeedChannel` shown in Figure 6-19.

The other channel uses a GPS to determine train speed. This channel receives ephemeris data from some set of (at least three) satellites and using the time sent (and comparing it to the time within the system), it computes the distance to the satellite. Through a process called 3D triangulation, the train's position can be calculated. This location is validated and then used to compute the speed by looking at the rate of change of positions. This speed is then validated and, if valid, is used to compute the desired engine output to maintain current speed. This is the GPSSpeedChannel shown in Figure 6-20.

In either channel, if data are identified as invalid, that channel does not command the engine speed. Because there is another channel to take over the task of determining desired engine output, the system can continue to operate even if a single point fault results in a channel shutting down. Further, because this is an extremely heterogeneous design, an error in one channel will not be replicated in the other. This design provides protection against both random and systematic faults.

There is nothing profound about the code that is produced to implement this example beyond what has been described earlier in this chapter. The interesting part is how the channels are

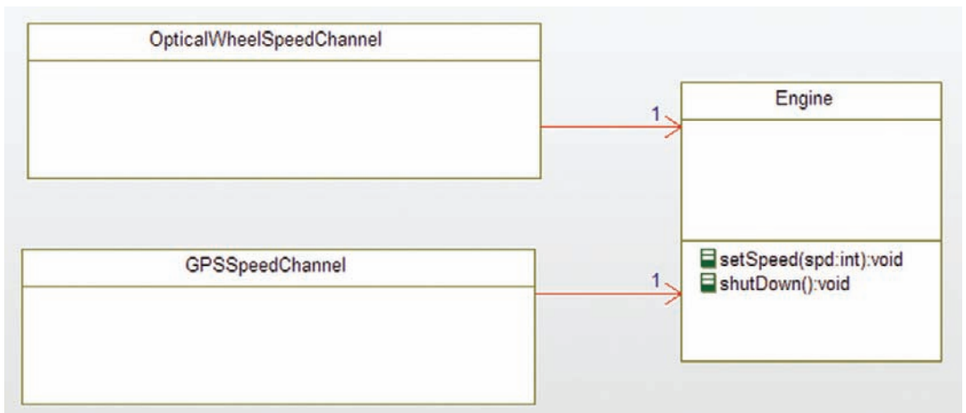


Figure 6-18: Dual Channel example

6.7.8 示例

图6-18所示的示例是用于列车速度控制系统的异构双通道模式。一个通道使用照射在列车车轮内侧的光。通过已知车轮的周长并测量标记出现频率（由光学传感器测量），可以计算列车的速度。然后对这些数据进行滤波、验证，并用于计算期望的发动机输出。该发动机输出经过合理性检查，如果可接受，则发送至发动机。这就是图6-19所示的OpticalWheelSpeedChannel。

另一个通道使用 GPS 来确定列车速度。该通道从一组（至少三颗）卫星接收星历数据，并利用发送的时间（将其与系统内的时间进行比较）计算出到卫星的距离。通过称为三维三角测量的过程，可以计算出列车的位置。该位置经过验证后，通过查看位置的变化率来计算速度。然后对该速度进行验证，如果有效，则用于计算所需的发动机输出以保持当前速度。这就是图 6-20 中所示的 GPSSpeedChannel。

在任一通道中，如果数据被识别为无效，该通道就不会发出引擎速度指令。由于有另一个通道接管确定所需发动机输出的任务，因此即使单点故障导致某个通道关闭，系统仍可继续运行。此外，由于这是一种极度异构的设计，一个通道中的错误不会在另一个通道中重现。这种设计提供了对随机故障和系统性故障的防护。

用于实现此示例的代码并没有什么深奥之处，超出了本章前面已经描述的内容。有趣的部分在于通道是如何

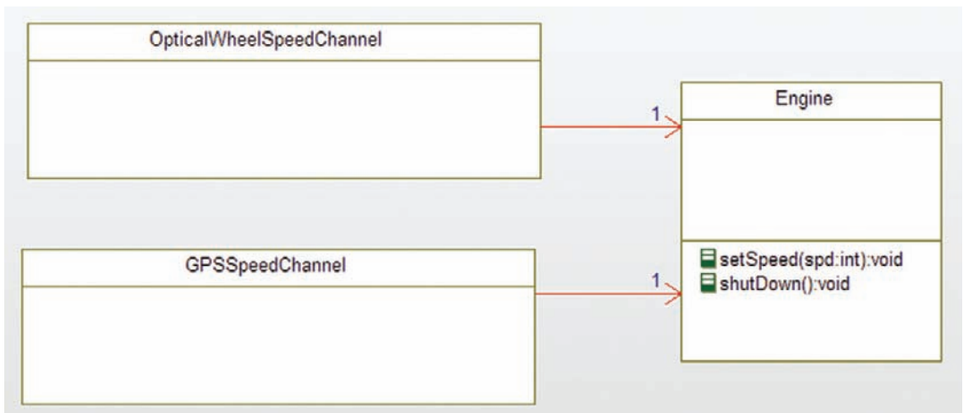


图 6-18：双通道示例

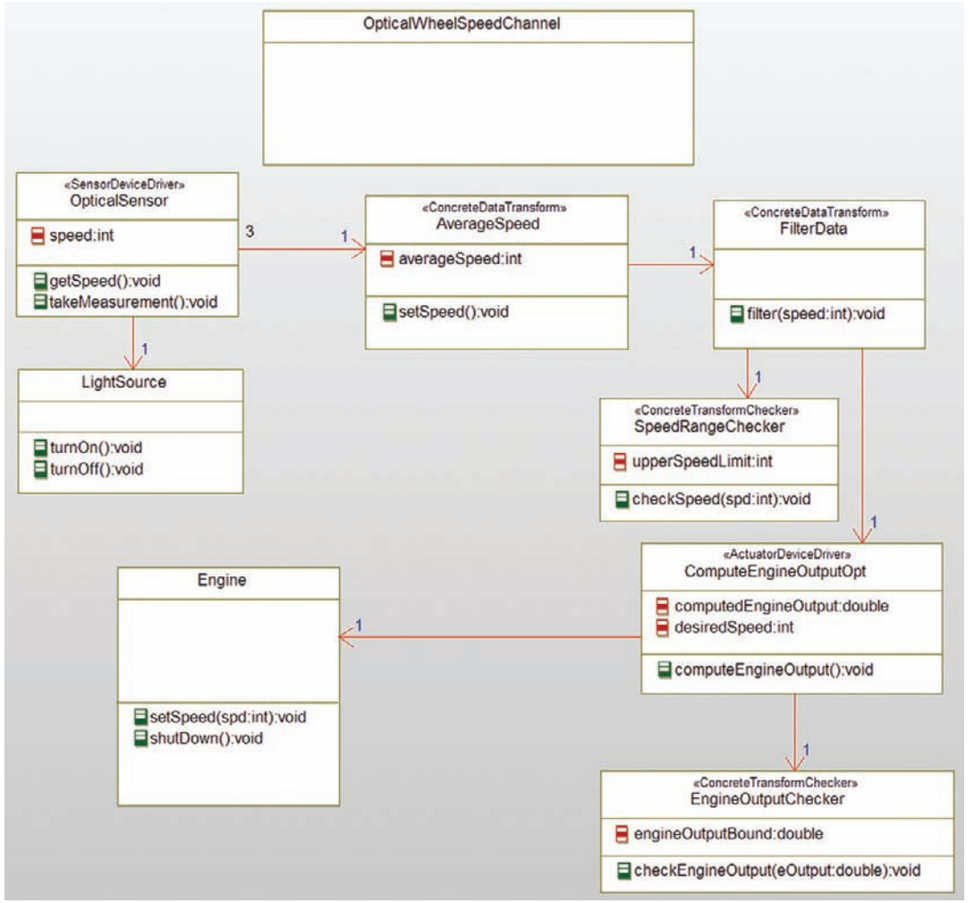


Figure 6-19: Details of Optical Wheel Speed Channel

connected to provide the ability of the system to detect a fault and continue safe operation in the presence of the fault.

6.8 Summary

This chapter has presented a number of different patterns that can be used to improve the safety and reliability of a design executing in its operational context. The first few patterns were small in scope (often referred to as “design idioms” instead of “design patterns”). The One’s Complement Pattern replicated primitive data to identify run-time corruption – a problem common to harsh environments including medical operating rooms, industrial factory settings,

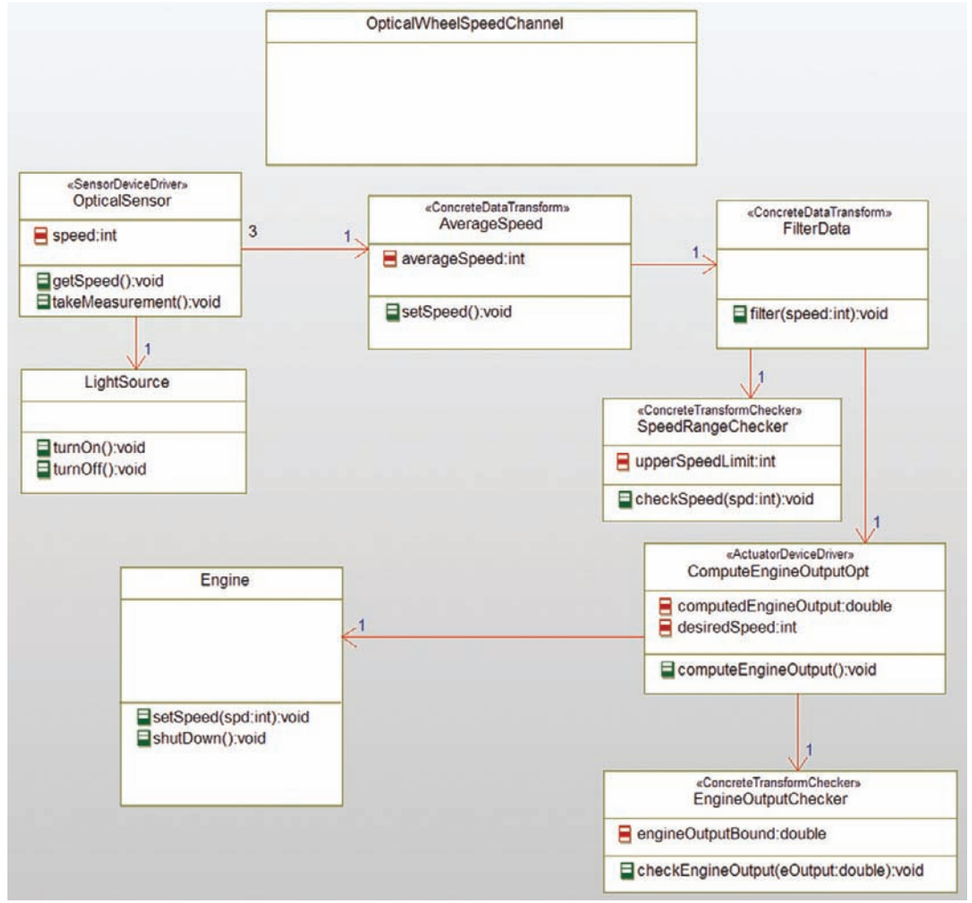


图6-19：光学轮速通道详解

连接起来，以提供系统检测故障并在存在故障的情况下继续安全运行的能力。

6.8 总结

本章介绍了多种不同的模式，可用于提高设计在其运行上下文中的安全性与可靠性。最初的几个模式范围较小（通常被称为“设计惯用法”而非“设计模式”）。反码模式通过复制原始数据来识别运行时损坏——这是包括医疗手术室、工业工厂环境和

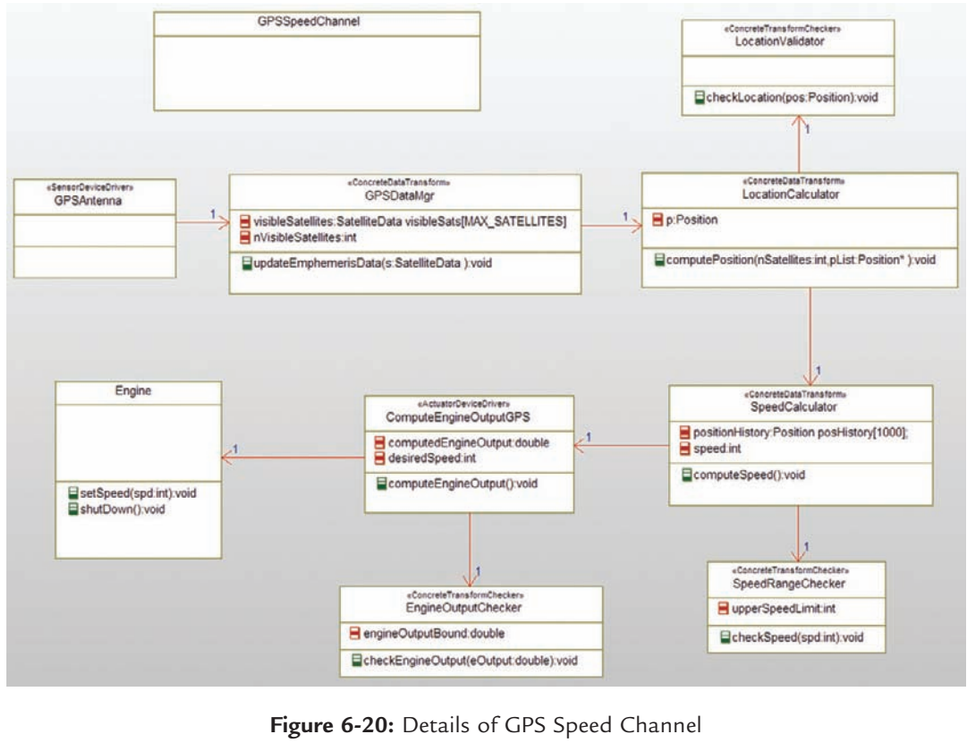


Figure 6-20: Details of GPS Speed Channel

and space. The CRC Pattern is useful in protecting larger volumes of data. The Smart Data Pattern adds behavior to data (that is, creates classes) to ensure that the data's preconditions and constraints are met.

The other patterns in this chapter have a larger scope. The fundamental pattern for the rest of the chapter is the Channel Pattern, in which the processing of sensory data into actionable commands is set up as a series of data transform steps. This pattern is elaborated into the Protected Single Channel Pattern in which checks are added at various steps to validate the processing within the channel. The Dual Channel Pattern adds two channels so that even if one fails, the other can continue providing the required service.

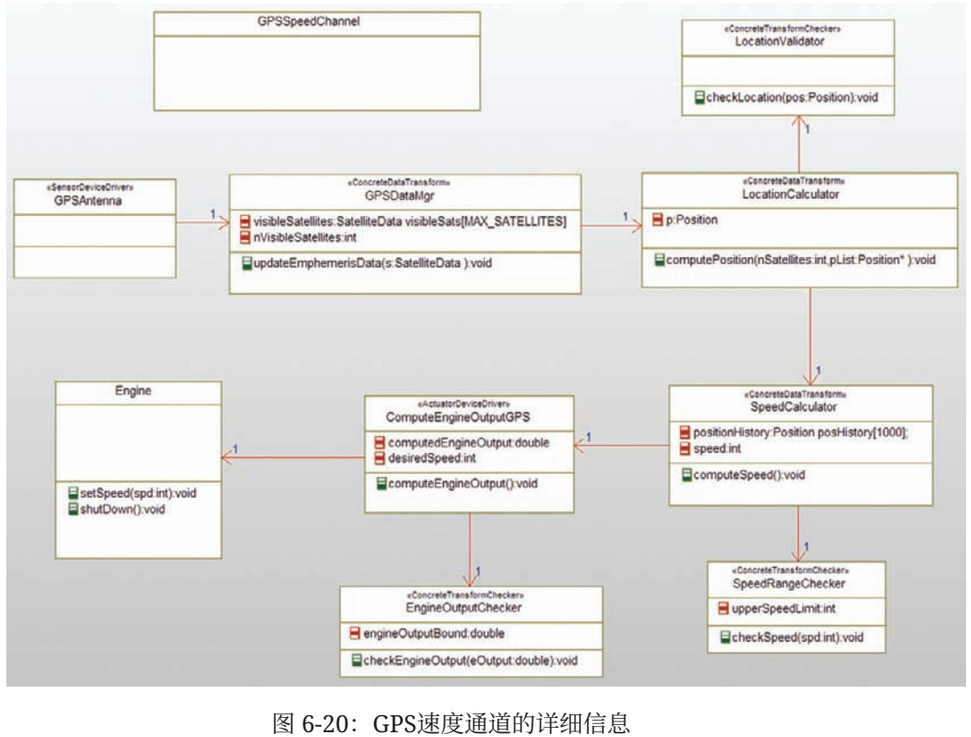


图 6-20：GPS速度通道的详细信息

太空在内的恶劣环境中常见的问题。CRC模式可用于保护更大容量的数据。智能数据模式为数据添加行为（即创建类），以确保满足数据的前置条件和约束。

本章中的其他模式具有更大的范围。本章其余部分的基础模式是通道模式，其中将传感数据处理为可执行命令的过程被设置为一系列数据转换步骤。该模式被扩展为受保护单通道模式，其中在各个步骤中添加检查以验证通道内的处理。双通道模式增加了两个通道，以便即使一个通道发生故障，另一个通道也能继续提供所需的服务。

This page intentionally left blank

本页有意留白

APPENDIX A

UML Notation

The UML is a rich and expressive language. The purpose of this appendix is to help you understand the most important notations of the language. For more detailed understanding of the richness of the UML language, the reader is referred to any of a number of books exploring the UML language in detail, such as my own *Real-Time UML: Advances in the UML for Real-Time Systems* (Addison-Wesley, 2004).

In the context of this book, UML was used to graphically depict both the structure and the behavior of the patterns. Three of the set of diagrams of the UML used in this book are discussed here: the Class Diagram, the Sequence Diagram, and the State Diagram. The UML provides a number of other diagrammatic types (not discussed here) and even for the diagrams discussed, there are other lesser-used features that are not discussed in this appendix.

1.1 Class Diagram

The Class Diagram is for showing system structure. It does this by showing Classes (specifications of software elements), Objects (instances of classes), and relations of various kinds. The UML is quite flexible about the amount of detail shown on a diagram.

1.1.1 Semantic Elements

This section describes the various elements on the Class Diagram.

1.1.1.1 Nodal Elements

1.1.1.1.1 Class

A class is a representation of an element that combines data elements (called attributes) with functions that act on them (called operations). Classes are normally implemented as `structs` containing the attributes.

1.1.1.1.2 Instance

An instance is a creation of a new variable of the type of the struct that defines the class.

1.1.1.1.3 Interface

An interface is a specification of a set of services (operations and/or event receptions) that a class must implement. An interface has no implementation and no attributes; it cannot be

Design Patterns for Embedded Systems in C
DOI:10.1016/B978-1-85617-707-8.00007-8
Copyright © 2011 Elsevier Inc.

附录A

UML表示法

UML是一门丰富且富有表现力的语言。本附录的目的是帮助您理解该语言中最重要的表示法。如需更详细地了解UML语言的丰富性，读者可参阅诸多深入探讨UML语言的书籍，例如我本人所著的《Real-Time UML: Advances in the UML for Real-Time Systems》（Addison-Wesley，2004）。

在本书的上下文中，UML被用于以图形方式描绘模式的结构和行为。此处讨论本书所使用的UML图中的三种：类图、序列图和状态图。UML还提供了许多其他图类型（此处不予讨论），而且即使对于所讨论的图，本附录也未讨论其他较少使用的特性。

1.1 类图

类图用于展示系统结构。它通过展示类（软件元素的规格说明）、对象（类的实例）以及各种关系来实现这一点。UML对于图上所展示的细节程度相当灵活。

1.1.1 语义元素

本节描述类图上的各种元素。

1.1.1.1 节点元素

1.1.1.1.1 类

类是一种元素的表示，它将数据元素（称为属性）与作用于这些数据元素的函数（称为操作）组合在一起。类通常实现为包含这些属性的结构体。

1.1.1.1.2 实例

实例是　　创建的一个新变量，其类型为定义该类的结构体　　.

1.1.1.1.3 接口

接口是对一组服务（操作和/或事件接收）的规范，类必须实现这些服务。接口没有实现，也没有属性；它不能被

C语言嵌入式系统设计模式
DOI:10.1016/B978-1-85617-707-8.00007-8 版权所
有 © 2011 Elsevier Inc.

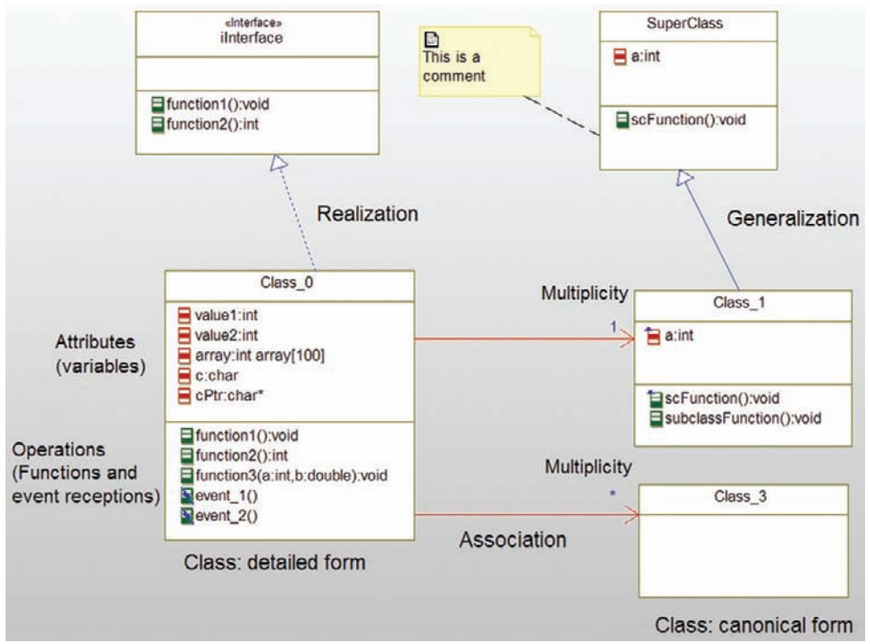


Figure A-1 Basic Class Diagram

instantiated. An interface is depicted with the same kind of box as a class, so it is visually distinguished with the «Interface» stereotype.

1.1.1.1.4 Attribute

A variable held within a class.

1.1.1.1.5 Operation

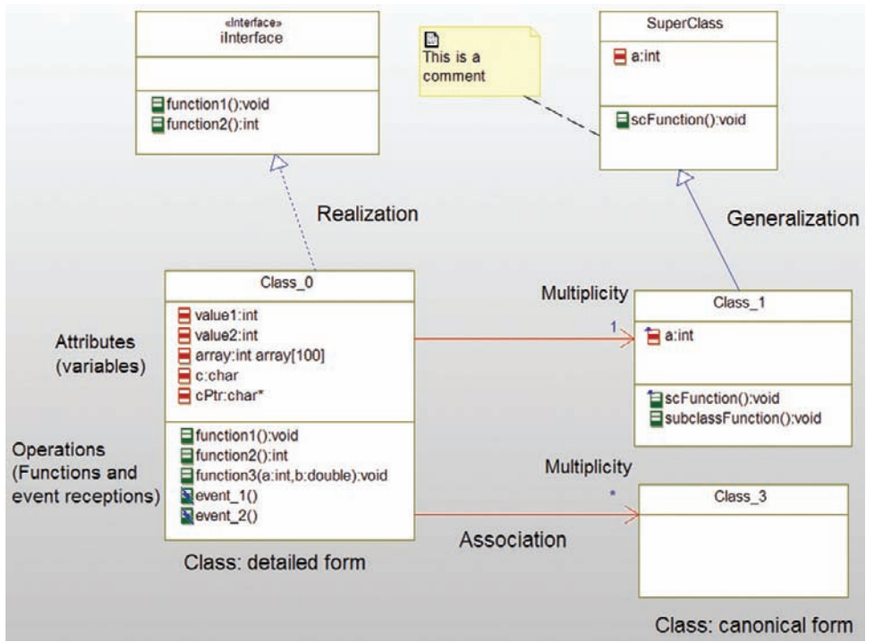
A function held within a class that typically manipulates the class attributes. In this book, the common implementation style of an operation is to use the notation <Class Name>_<Operation Name>, such as `SensorClass_getData()`. The first parameter passed to the class is a pointer to the instance of the class.

1.1.1.1.6 Event Reception

A special kind of operation that is invoked asynchronously and causes event transitions to take place on the state machine associated with the class.

1.1.1.1.7 Comment

A graphical way to representing textual comments. It may be anchored to one or more elements on the diagram or may be unanchored.



图A-1 基本类图

实例化。接口使用与类相同的方框来描绘，因此通过«Interface»构造型在视觉上加以区分。

1.1.1.1.4 属性

类中持有的变量。

1.1.1.1.5 操作

类中持有的函数，通常用于操作类的属性。在本书中，操作的常见实现风格是使用表示法<类名>_<操作名>，例如`SensorClass_getData()`。传递给类的第一个参数是指向该类实例的指针。

1.1.1.1.6 事件接收

一种特殊类型的操作，它以异步方式被调用，并促使与该类关联的状态机上发生事件转换。

1.1.1.1.7 注释

一种用于表示文本注释的图形化方式。它可以锚定到图上的一个或多个元素，也可以不锚定。

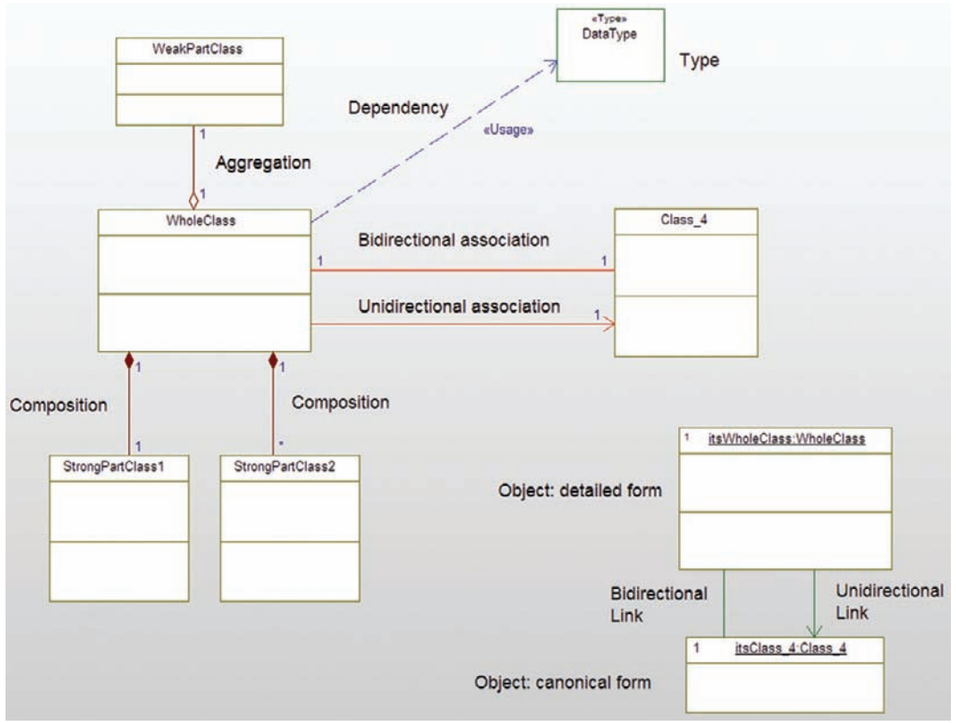


Figure A-2 More Class Diagram notations

1.1.1.2 Relations

1.1.1.2.1 Association

The specification of a navigable connection among classes. It may be unidirectional or bidirectional. Each end of the association is called an association role end and has a *multiplicity.* The multiplicity is the number of instances of the target class that participates in the interaction. "1" is the most common multiplicity and "*" means "zero or more".

An association is most commonly implemented as a pointer or collection of pointers.

1.1.1.2.2 Link

A link is an instance of an association; that is, a link exists when the pointer is populated with the address of the instance; if the pointer is NULL, then the link doesn't exist.

1.1.1.2.3 Generalization

Generalization is an is-a-specialized-kind-of relation with the arrow pointing to the more general class. The super- (more general) class's features, including attributes, operations, and relations, are all inherited by the sub- (more specific) class.

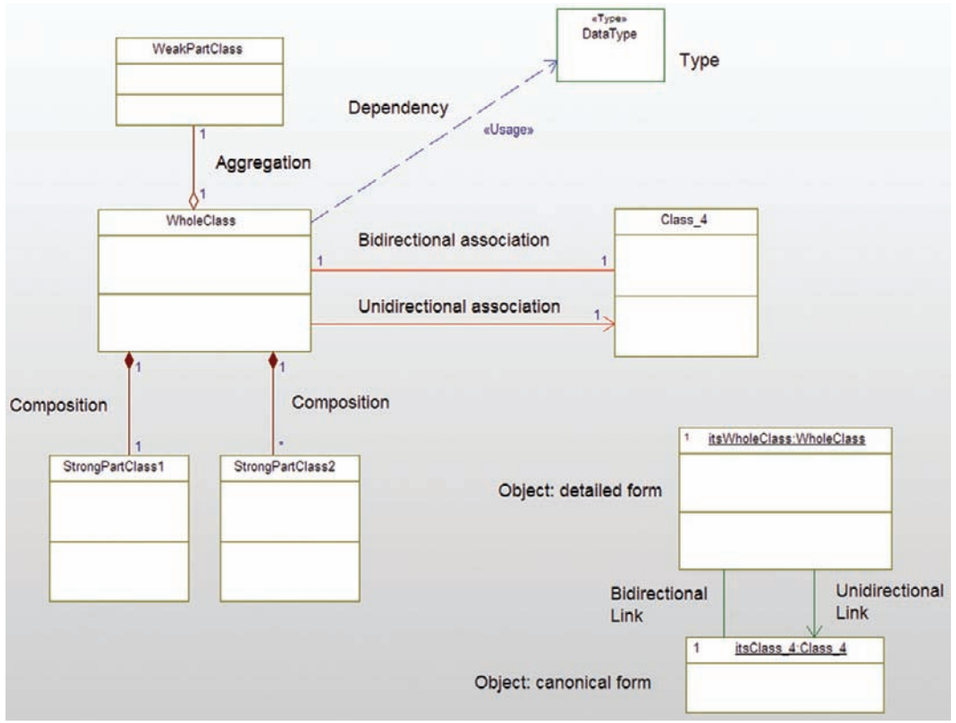


图 A-2 更多类图表示法

1.1.1.2 关系

1.1.1.2.1 关联

类之间可导航连接的规范。它可以是单向的或双向的。关联的每一端称为关联角色端，并具有多重性。多重性是参与交互的目标类的实例数量。"1"是最常见的多重性，"*"表示"零个或多个"。

关联最常见的实现方式是指针或指针集合。

1.1.1.2.2 链接

链接是关联的实例；也就是说，当指针中填充了实例的地址时，链接就存在；如果指针为空，则链接不存在。

1.1.1.2.3 泛化

泛化是一种特殊类型的关系，其箭头指向更一般类。超类（更一般的类）的特性，包括属性、操作和关系，均由子类（更具体的类）继承。

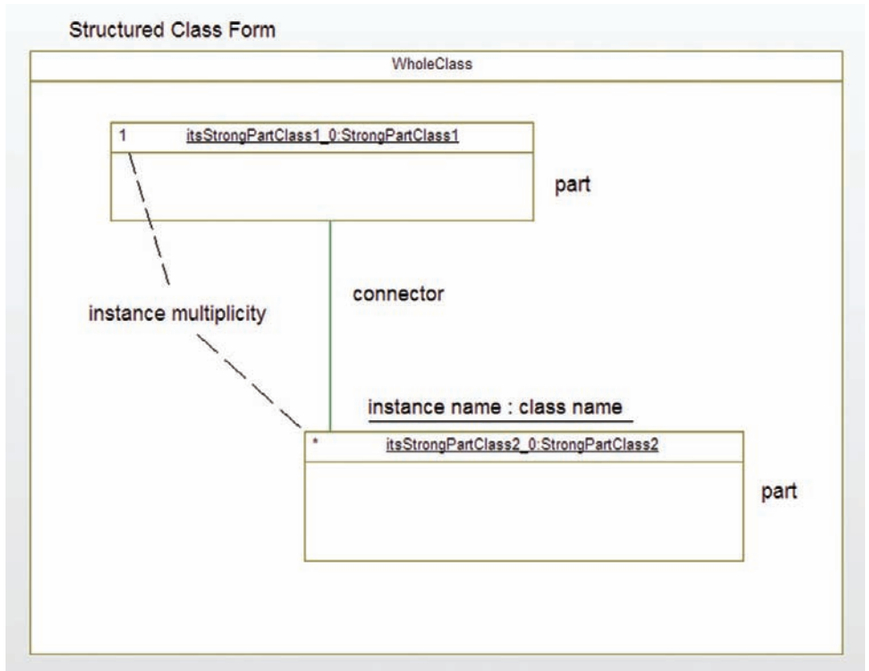


Figure A-3 Structured class notation

1.1.1.2.4 Realization

Realization is the relation between an interface and a class that implements the services specified by an interface.

1.1.1.2.5 Aggregation

Aggregation is a weak form of a whole-part association between two classes where the diamond indicates the "whole" end. It is weak in the sense that this relation does not imply creation and destruction responsibilities on the part of the whole.

1.1.1.2.6 Composition

Composition is a strong form of a whole-part association between two classes; again, the (now filled) diamond is at the end of the whole. It is strong in the sense that the whole is responsible for both the creation and destruction of the part instance.

1.1.1.2.7 Dependency

This is a general relationship not otherwise denoted. It is common to add a stereotype (such as «Usage») to indicate what kind of dependency you want. The «Usage» dependency is common

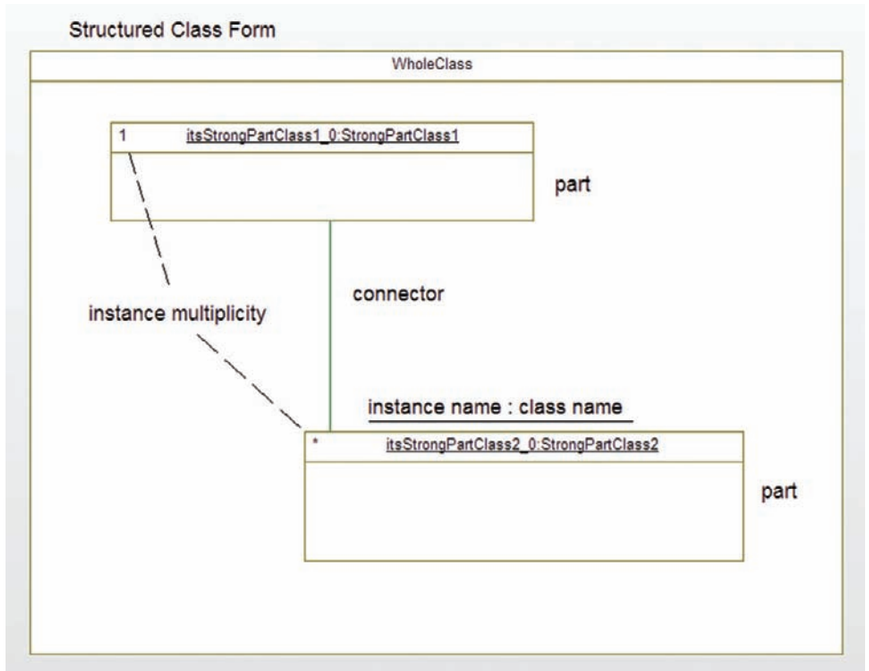


图 A-3 结构化类表示法

1.1.1.2.4 实现

实现是接口与实现该接口所指定服务的类之间的关系。

1.1.1.2.5 聚合

聚合是两个类之间整体-部分关联的一种弱形式，其中菱形指示"整体"端。它之所以是弱的，是因为这种关系不意味着整体对部分负有创建和销毁的责任。

1.1.1.2.6 组合

组合是两个类之间整体-部分关联的一种强形式；同样，菱形（此时为实心）位于整体一端。其"强"体现在整体负责部分实例的创建与销毁。

1.1.1.2.7 依赖

这是一种未另行表示的通用关系。通常会添加一个构造型（例如 «Usage»）来指示您想要的依赖类型。«Usage» 依赖很常见

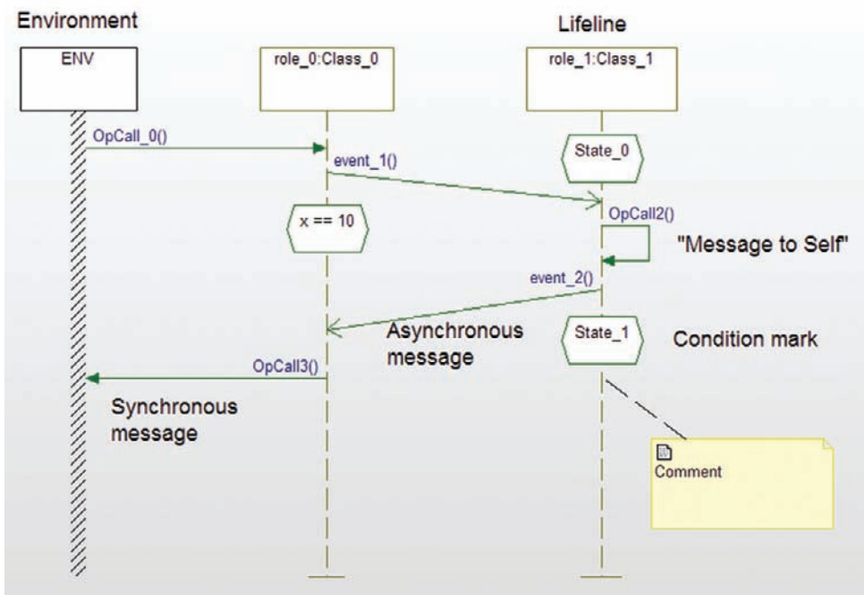


Figure A-4 Sequence Diagram

and indicates that the header of the target class or element should be included by the source class during compilation.

1.1.1.2.8 Type

A type is just that – a data type without the richness of a class to add behavior. It can be an enumerated type (`enum`), structured type (`struct`), array, `typedef`, or even a `#define`.

1.2 Sequence Diagram

A Sequence Diagram shows a particular sequence of messages exchanged between a set of instances. The instances are shown as vertical lines known as lifelines. Messages are shown as horizontal or downward slanted lines. Time flows (more or less) down the page. In addition, condition marks can be used to show state or the current value of attributes during the sequence.

1.2.1 Semantic Elements

Lifeline

A lifeline represents the usage of an instance. At the top of the lifeline, it is common to both name the instance and identify the type of class of the instance.

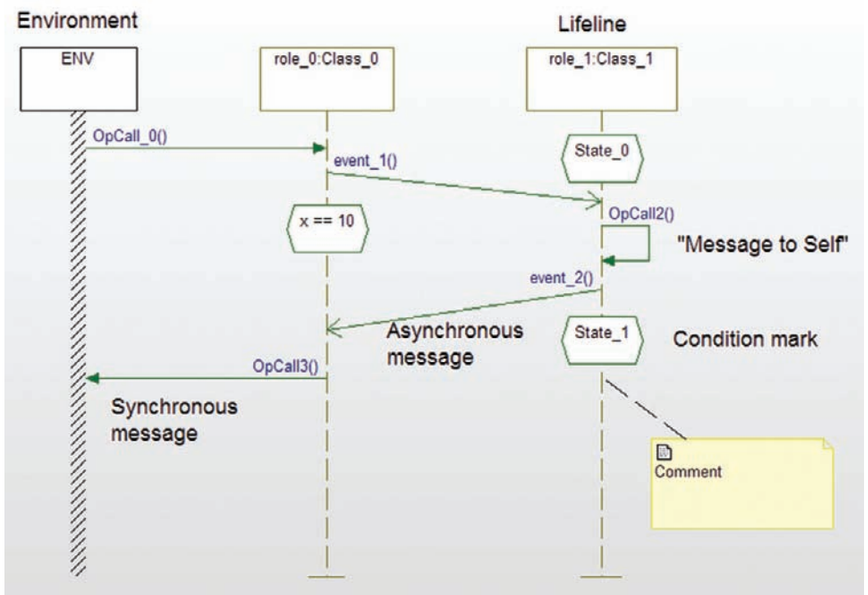


图 A-4 序列图

并且表示源类在编译期间应包含目标类或元素的头文件。

1.1.1.2.8 类型

类型就是如此——一种数据类型，没有类那样丰富的能力来添加行为。它可以是枚举类型（`enum`）、结构化类型（`struct`）、数组、typedef，甚至可以是 `#define`。

1.2 序列图

序列图展示了一组实例之间交换的特定消息序列。实例以垂直线表示，称为生命线。消息以水平线或向下倾斜的线表示。时间（大致）沿页面从上往下流动。此外，条件标记可用于在序列过程中显示状态或属性的当前值。

1.2.1 语义元素

生命线

生命线表示实例的用法。在生命线的顶部，通常既命名实例，又标识该实例的类类型。

Environment Lifeline

This is a special lifeline that stands for "everything else." It provides a notational shorthand for elements omitted from the diagram.

Message

A representation of an interaction between two instances. It may be synchronous (as in a function call) or asynchronous (as in an asynchronous event exchange). A message is shown as a directed line that may be either horizontal or downwards sloping.

Synchronous Message

A synchronous event (function call) is shown with a solid arrowhead. It is most commonly draw as a horizontal line.

Asynchronous Message

An asynchronous message (event reception) is shown with an open arrowhead. It is common (but not required) to draw it as a downward sloping line.

Condition Mark

This is a hexagon that contains a textual condition that is true at that point in the sequence, such as the value of a variable or the current state in a state machine.

Comment

It may be anchored to one or more elements on the diagram or may be unanchored.

1.3 State Diagram

A State Diagram is a directed graph consisting of states (rounded rectangles) and transitions (arrowed lines). It defines the order of action execution for its owner class.

1.3.1 Semantic Elements

1.3.1.1 State

A state is a condition of an instance of a class. A state may contain entry actions, exit actions, and internal transitions. It is drawn as a rounded rectangle.

1.3.1.2 Nested State

States may contain lower level, nested states. For example, while the `DOOR_IS_OPEN` state is true, it might be in `DOOR_OPENING`, `DOOR_CLOSING`, or `DOOR_WAITING_OPEN` nested state. Nested states are drawn by graphically including the nested state inside a composite state.

环境生命线

这是一条特殊的生命线，代表“其他所有内容”。它为图中省略的元素提供了一种表示法简写。

消息

两个实例之间交互的一种表示。它可以是同步的（如函数调用）或异步的（如异步事件交换）。消息显示为一条有向线，可以是水平的或向下倾斜的。

同步消息

同步事件（函数调用）用实心箭头表示。它最常绘制为水平线。

异步消息

异步消息（事件接收）用空心箭头表示。通常（但并非必须）将其绘制为向下倾斜的线条。

条件标记

这是一个六边形，其中包含一个文本条件，该条件在序列中的该点处为真，例如变量的值或状态机中的当前状态。

注释

它可能是 锚定到图上的一个或多个元素，或者可以不锚定 .

1.3 状态图

状态图是一个有向图，由状态（圆角矩形）和迁移（带箭头的线）组成。它为其所有者类定义了动作执行的顺序。

1.3.1 语义元素

1.3.1.1 状态

状态是类的实例的一种条件。状态可以包含进入动作、退出动作和内部转换。它被绘制为圆角矩形。

1.3.1.2 嵌套状态

状态可以包含更低层次的嵌套状态。例如，当 `DOOR_IS_OPEN` 状态为真时，它可能处于 `DOOR_OPENING`, `DOOR_CLOSING`, 或 `DOOR_WAITING_OPEN` 嵌套状态。嵌套状态通过将嵌套状态图形化地包含在复合状态内部来绘制。

1.3.1.3 Composite State

A state that contains nested states. These may be OR-states (as in Figure A-5 and Figure A-6) or AND-states (as in Figure A-7).

1.3.1.4 OR-State

A set of states at a given level of abstraction; the semantics of state machines are that at a given level of abstraction, the state machine must be in exactly one of its OR-States. In Figure A-5, `state_0` and `state_1` are OR-states; similarly, `NestedState1` and `NestedState2` are OR-states.

1.3.1.5 AND-State

AND-states are orthogonal regions within a composite state with the semantics that the instance must be in exactly one AND-state in each orthogonal region. AND-states are indicated with a dashed line in the composite state. While it is theoretically possible to implement AND-states in separate threads, this is almost never done. If an event is shared between AND-states, each AND-state must be allowed to independently act on it or reject it, but the order of execution of the transitions triggered by the same event is not known in principle. This leads to the possibility of race conditions.

If you look at Figure A-7, the transition from `state_0` to `state_1` is triggered by the `event1` event. In this case, the instance enters both states `UpperState1` and `LowerState3`; the instance is in both states at the same time. AND-states are most commonly used to model independent properties of an instance.

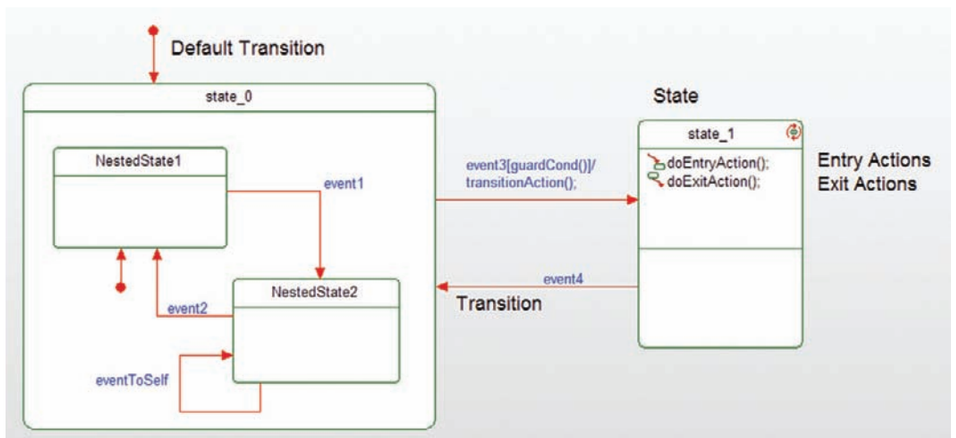


Figure A-5 Basic State Diagram

1.3.1.3 复合状态

包含嵌套状态的状态。这些状态可以是或状态（如图 A-5 和图 A-6 所示）或与状态（如图 A-7 所示）。

1.3.1.4 或状态

在给定抽象层次上的一组状态；状态机的语义是：在给定的抽象层次上，状态机必须恰好处于其或状态之一。在图A-5中，`状态_0` 和`状态_1` 是或状态；类似地，`NestedState1` 和`NestedState2` 也是或状态。

1.3.1.5 与状态

与状态是复合状态内的正交区域，其语义是：实例在每个正交区域中必须恰好处于一个与状态。与状态在复合状态中用虚线表示。虽然理论上可以在单独的线程中实现与状态，但这几乎从未被采用。如果事件在与状态之间共享，则每个与状态必须被允许独立地对其采取动作或拒绝它，但由同一事件触发的转换的执行顺序原则上未知。这导致了竞态条件的可能性。

如果查看图A-7，从`状态_0` 到`状态_1` 的迁移由`event1`事件触发。在这种情况下，实例同时进入`UpperState1` 和`LowerState3`两个状态；该实例同时处于这两个状态中。AND状态最常用于对实例的独立属性进行建模。

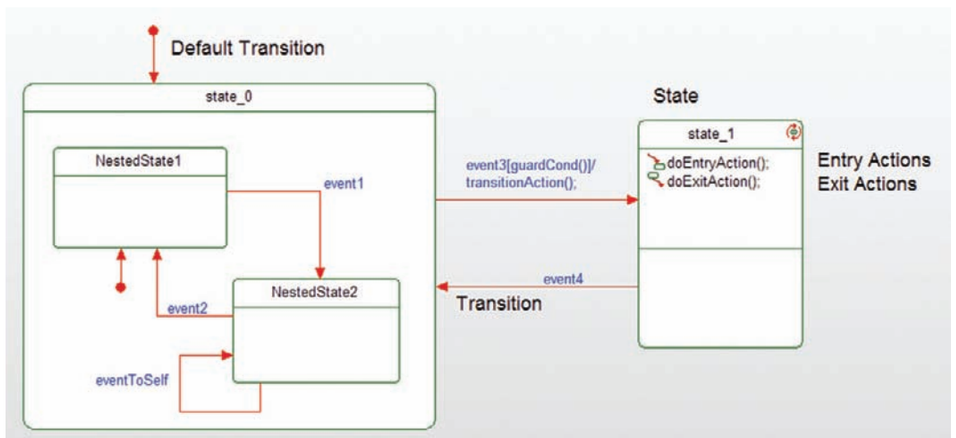


图 A-5 基本状态图

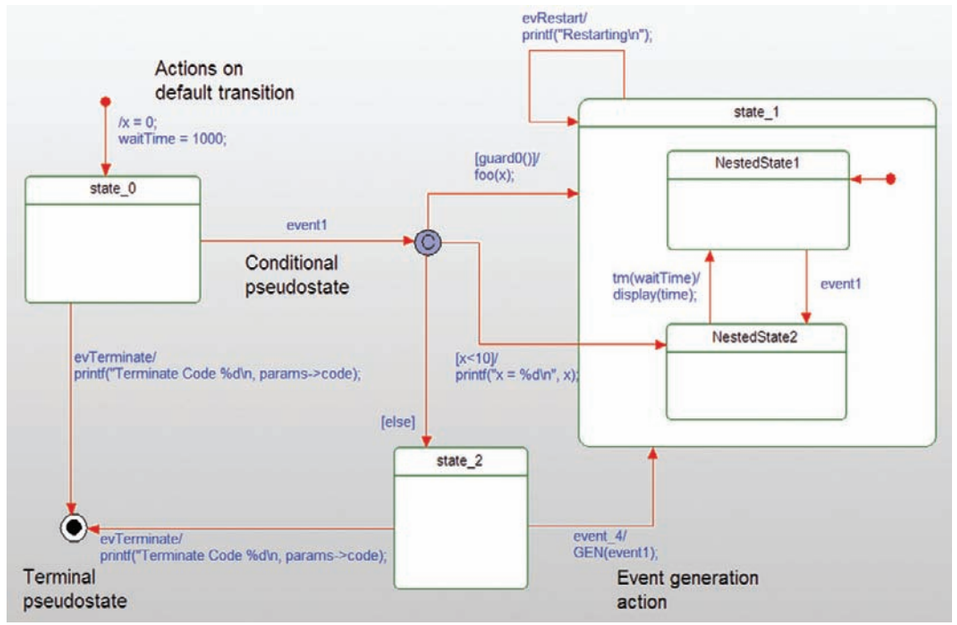


Figure A-6 Advanced State Diagram

Note also in the figure the transition from `state_1` to `state_0` triggered by the event `evGoBack`. In this case, it doesn't matter which combination of nested states is currently true; as long as the instance is in `state_1`, the transition is valid. However, in the event from nested state `LowerState3` to `state_2`, triggered by event `event10`, which nested state in `ANDStateRegion1` is true doesn't matter, but the instance must be currently in `LowerState3` state for that transition to be taken. If it is not, then the event is discarded.

1.3.1.6 Transition

A transition is a representation of a response to an event while the instance is in a specified state. It typically has an event signature consisting of an event trigger, guard, and list of actions, specified using the following syntax:

```
event trigger '[' guard condition ']' '/' action list
```

All of the fields in the event are optional. A transition is shown on a state machine as an arrowed line coming from a predecessor state and terminating on a subsequent state.

If the event-trigger is omitted, the transition "fires" as soon as the predecessor state is entered, that is, as soon as the entry actions of the predecessor state complete. See Figure A-7 for an example.

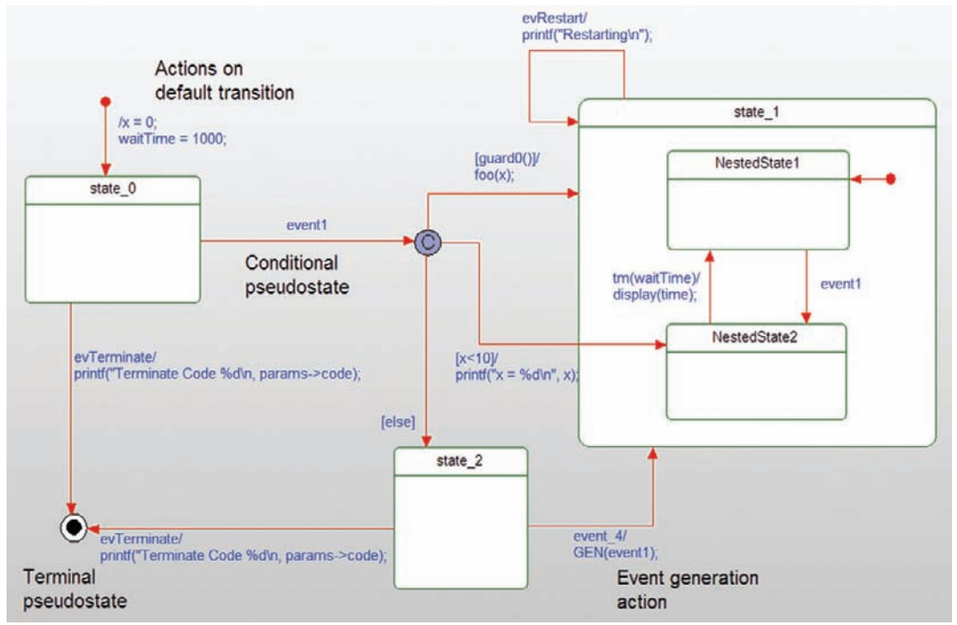


图 A-6 高级状态图

另请注意图中由事件`evGoBack`触发的从状态`_1`到状态`_0`的迁移。在这种情况下，当前哪个嵌套状态组合为真并不重要；只要实例处于状态`_1`，该迁移就是有效的。然而，在由事件`event10`触发的从嵌套状态`LowerState3`到状态`_2`的事件中，`ANDStateRegion1`中哪个嵌套状态为真并不重要，但实例当前必须处于`LowerState3`状态才能执行该迁移。如果不是，则该事件被丢弃。

1.3.1.6 迁移

迁移表示实例处于指定状态时对事件的响应。它通常具有一个事件签名，由事件触发器、守卫条件和动作列表组成，使用以下语法指定：

```
事件触发器 '[' 守卫条件 ']' '/' 动作列表
```

事件中的所有字段都是可选的。迁移在状态机上显示为一条带箭头的线，从前驱状态出发，终止于后继状态。

如果省略事件触发器，则迁移会在进入前驱状态后立即"触发"，也就是说，一旦前驱状态的进入动作完成就立即触发。示例见图 A-7。

1.3.1.7 Action

An action is a primitive behavior performed under some circumstance in a state machine. This is most often one of the following:

- A function call, for example `foo(x)`
- A direct primitive action, such as `x += 10`
- The generation of an event sent to the current instance or another instance known to the current instance; a common way to indicate this is with a `GEN(event)` macro
- A list of actions

Actions are separated from the rest of the event signature with a slash (‘/’) character.

1.3.1.8 Entry Action

An entry action is an action performed whenever a state is entered regardless of which transition is taken.

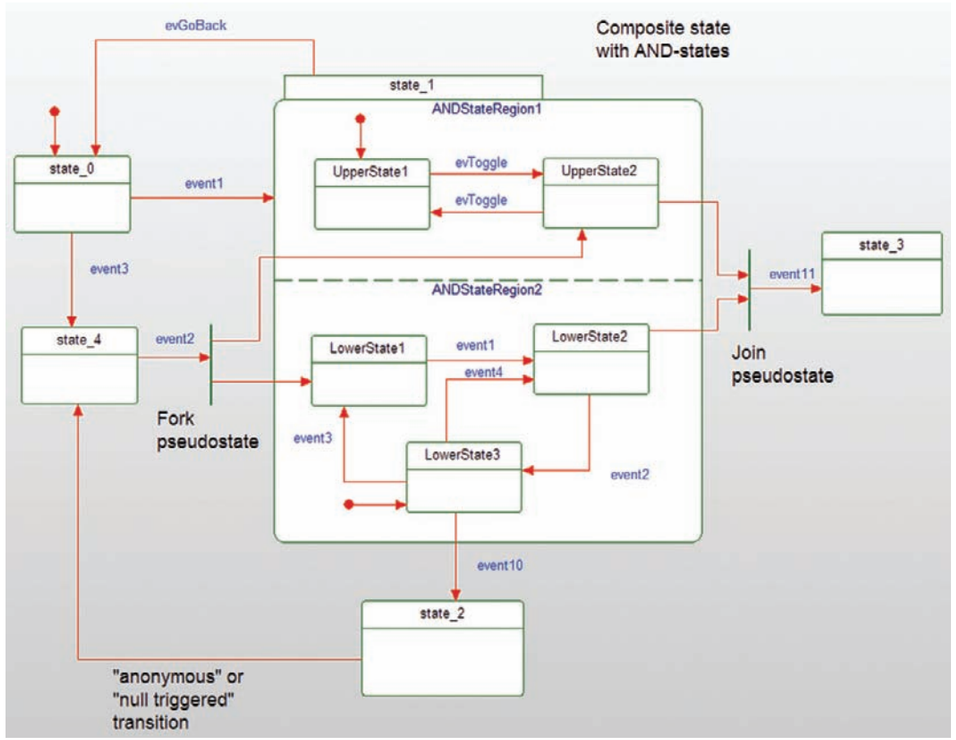


Figure A-7 AND-states on State Diagram

1.3.1.7 动作

动作是在状态机中某种情况下执行的基本行为。这通常是以下之一：

- A 函数 调用， 例如 foo(x)
- 一个直接的原语动作，例如 `x += 10`
- 生成一个发送给当前实例或当前实例已知的另一实例的事件；表示此操作的常见方式是使用 `GEN(event)` 宏
- 一个动作列表

动作与事件签名的其余部分用斜杠（'/'）字符分隔。

1.3.1.8 进入动作

进入动作是指无论采用哪条迁移，只要进入某个状态时都会执行的动作。

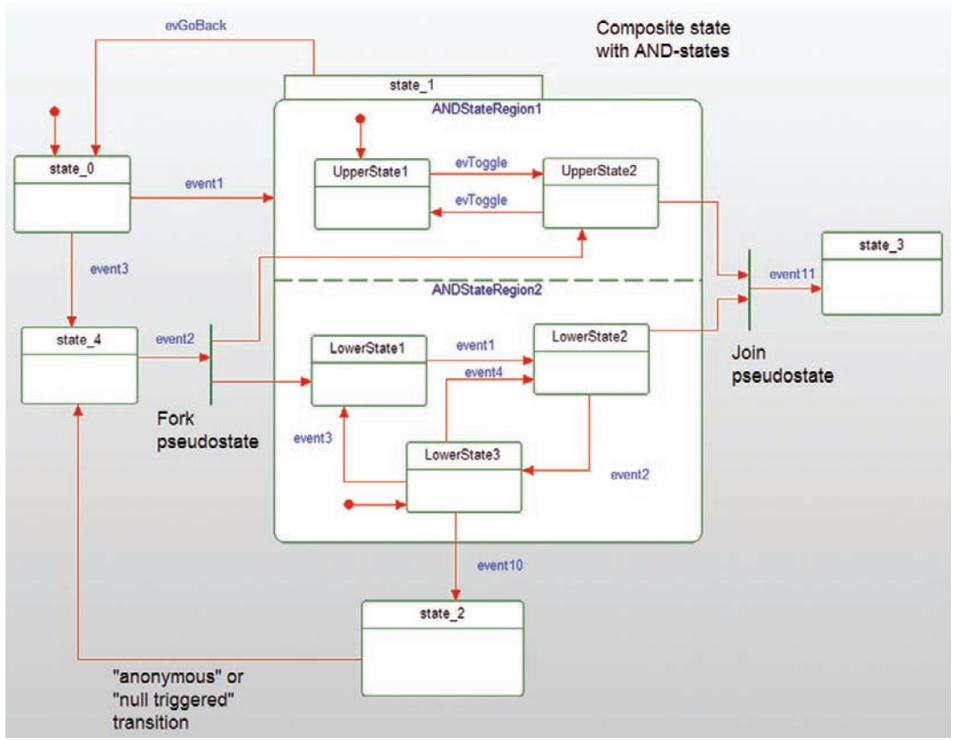


图 A-7 状态图上的 AND 状态

1.3.1.9 Exit Action

An exit action is an action performed whenever a state is left, regardless of which transition is taken.

1.3.1.10 Transition Action

A transition action is an action performed only when a specific transition is taken.

1.3.1.11 Internal Transition

An internal transition is when an action is performed when an event occurs but a state transition is not taken (that is, the exit and entry actions are not performed).

1.3.1.12 Event

An event is a specification of an event of interest that is received by the instance. It may be received synchronously or asynchronously. It may carry parameters, just like a function call. The syntax used here is that the event that carries data put those data into a `struct` called `params`. This struct may be dereferenced to access the data. For example, in Figure A-6, the `evTerminate` event carries a data element code. The actions in the state machine access the parameter by dereferencing the params `struct`.

Events may trigger a transition or an internal transition; if they do not, events are quietly discarded with no actions taken.

An event is consumed by the state machine; that is, once the state machine has completed its processing step for the current event, it is destroyed. Even if the newly assumed state has an event triggered by the same event, the event must occur again for this transition to be taken.

1.3.1.13 Event Trigger

The event trigger is the specification of an event. This may be asynchronous (which requires event queuing), synchronous, a timeout (usually indicated with a `tm(duration)` or `after (duration)` syntax), or when a state variable is changed.

1.3.1.14 Guard

A guard is an (optional) Boolean expression. If the guard evaluates to `TRUE`, then the transition is taken; if it evaluates to `FALSE`, then the event is discarded and the transition is not taken.

1.3.1.15 Default Transition

The default transition indicates that at a given level of abstraction, which, among a set of states, is entered first by default. It is shown as a transition with no event trigger or guard (although it may have actions) terminating on a state, but not beginning on one.

1.3.1.16 Pseudostate

A pseudostate is a perhaps poorly-named annotation of special semantics. The UML defines many pseudostates. In this limited description here, pseudostates include the default transition, conditional, fork, join, and terminal pseudostates.

1.3.1.9 退出动作

退出动作是指无论采用哪条迁移，只要离开某个状态时都会执行的动作。

1.3.1.10 迁移动作

迁移　　动作是仅当特定迁移被触发时才执行的动作　　n.

1.3.1.11 内部转移

内部转移是指当事件发生时执行某个动作，但不进行状态迁移（即不执行退出动作和进入动作）。

1.3.1.12 事件

事件是对实例所接收的感兴趣事件的规格说明。它可以同步或异步接收。它可以像函数调用一样携带参数。此处使用的语法是，携带数据的事件将这些数据放入一个名为`params`的结构体 中。可以对该结构体进行解引用来访问数据。例如，在图A-6中，`evTerminate` 事件携带一个数据元素code。状态机中的动作通过解引用params 结构体来访问该参数。

事件可以触发迁移或内部转移；如果它们不触发，事件将被静默丢弃，不执行任何动作。

事件被状态机消费；也就是说，一旦状态机完成了对当前事件的处理步骤，该事件即被销毁。即使新进入的状态具有由同一事件触发的事件，该事件也必须再次发生才能进行该迁移。

1.3.1.13 事件触发器

事件触发器是对事件的规格说明。它可以是异步的（需要事件排队）、同步的、超时（通常用 `tm(duration)` 或 `after(duration)` 语法表示），或当状态变量发生变化时触发。

1.3.1.14 守卫条件

守卫条件是一个（可选的）布尔表达式。如果守卫条件求值为 `TRUE`，则执行该迁移；如果求值为 `FALSE`，则丢弃该事件且不执行迁移。

1.3.1.15 默认迁移

默认迁移表示在给定的抽象层次上，在一组状态中默认首先进入哪个状态。它表示为一条没有事件触发器或守卫条件（但可以有动作）的迁移，终止于某个状态，但不从某个状态开始。

1.3.1.16 伪状态

伪状态是一种命名可能不太恰当的特殊语义标注。UML定义了许多伪状态。在此处的有限描述中，伪状态包括默认迁移、条件伪状态、分叉伪状态、汇合伪状态和终止伪状态。

1.3.1.17 Conditional Pseudostate

The conditional pseudostate indicates multiple potential outgoing transitions initiated by a single event with the proviso that only one such path will actually be taken. The different paths are selected on the basis of guards; that is, each transition segment will have a guard that will be evaluated to `TRUE` or `FALSE`. If multiple transition segment guards evaluate to TRUE, then any one of the true paths can be taken. If no guards evaluate to true then the event is discarded and no transition is taken. A special guard "else" is provided; if present, this transition path will be taken only if all the other transition segments are rejected.

1.3.1.18 Fork

When entering a composite state with nested AND-states, a fork can be used to specify nondefault states to enter, as in Figure A-7. The event `event2` triggers the transition from `state_4` to `state_1`, but specifically enters nested AND-states `UpperState2` and `LowerState1`, bypassing the default states of `UpperState1` and `LowerState3`.

1.3.1.19 Join

When leaving a composite state with nested AND-states, a join can be used to state the precondition that the transition should only be taken if the specific set of states (one per AND-state) is true when the event occurs. See Figure A-7.

1.3.1.20 Terminal Pseudostate

The terminal pseudostate is a special indicator that means that the instance no longer accepts events, usually because the instance is about to be destroyed.

1.3.1.17 条件伪状态

条件伪状态表示由单个事件引发的多个潜在出转换，但前提是实际只会选择其中一条路径。不同路径基于守卫条件进行选择；也就是说，每个转换段都有一个守卫条件，其求值结果为`TRUE`或`FALSE`。如果多个转换段守卫条件的求值结果为TRUE，则可以走其中任意一条为真的路径。如果没有守卫条件的求值结果为真，则丢弃该事件，不进行任何迁移。系统提供了一个特殊的守卫条件"else"；如果存在该守卫条件，则仅当所有其他转换段都被拒绝时，才会走这条转换路径。

1.3.1.18 分叉

当进入带有嵌套AND状态的复合状态时，可以使用分叉来指定要进入的非默认状态，如图A-7所示。事件`event2`触发从`状态_4`到`状态_1`的迁移，但具体进入嵌套AND状态`UpperState2`和`LowerState1`，绕过`UpperState1`和`LowerState3`的默认状态。

1.3.1.19 汇合

当离开带有嵌套AND状态的复合状态时，可以使用汇合来声明前置条件：仅当事件发生时特定的状态集合（每个AND状态一个）为真时，才应执行该迁移。参见图A-7。

1.3.1.20 终止伪状态

终止伪状态是一种特殊指示符，表示该实例不再接受事件，通常是因为该实例即将被销毁。

This page intentionally left blank

本页有意留白

Index

& (ampersand), bit-wise operator, 83
^ (caret), bit-wise operator, 83
<< (less than signs), bit-wise operator, 83
>> (more than signs), bit-wise operator, 83
| (pipe sign), bit-wise operator, 83
~ (tilde), bit-wise operator, 83
802.11x, 82

A

Abstract
  for Channel Pattern, 395
  for CRC, 367
  for Critical Region Pattern, 182
  for Cyclic Executive Pattern, 165
  for Debouncing Pattern, 123
  for Decomposed And-State Pattern, 331
  for Dual-Channel Pattern, 413
  for Guarded Call Pattern, 190
  for Hardware Adapter Pattern, 96
  for Hardware Proxy Pattern, 85–86
  for Harmony/ERT, 46
  for Interrupt Pattern, 130
  for Mediator Pattern, 100
  for Multiple Event Receptor Pattern, 287
  for Observer Pattern, 66, 111–112
  for One's Complement Pattern, 362
  for Polling Pattern, 138
  for Protected Single Channel Pattern, 402
  for Queuing Pattern, 207
  for Rendezvous Pattern, 225–226
  for Simultaneous Locking Pattern, 232–233
  for Single Event Receptor Pattern, 274
  for Smart Data Pattern, 380
  for State Table Pattern, 296
  for Static Priority Pattern, 170–171
Abstract Subject
  inheritance and, 68
  for Observer Pattern, 66
  TMDQueue and, 68, 69
AbstractAndStateClass, 331
AbstractCEThread, 165
AbstractDataTransform, 395–396, 404, 414
AbstractStaticThread, 172
AbstractSubject, 113
  notify() and, 112
AbstractThread, Stack and, 173
AbstractTransformChecker, 404, 414
accept(Datum), 112
access() for Hardware Proxy class, 87
accessGasFlow(), 99
accessMotorDirection() as Motor Status Functions, 90
accessMotorSpeed() as Motor Status Functions, 90
accessMotorState() as Motor Error Management Function, 91
Accidents, 359, 360
AcmeO2Adapter.c, 100
AcquireData(), 398
Action, 262
Action list, 26
  for transitions, 27
  transitions and, 263
Action Sequence, 152
ActionPtr0, 298, 310, 312
ActionPtr1, 298, 311, 312
ActionSet, 331
ActionType, 296–297, 298
<<active>>, 161–162
  createThread() and, 173
  PreemptiveTask and, 191
ActivelyMonitoring, 264
ActuatorDeviceDriver, 396, 404, 414, 415
Actuators, 82
Ada, 380
Adapter client, for Hardware Adapter Pattern, 96
addLock(), ResourceList and, 247
After(), 27, 267
Analysis pattern, 44
AND, bit-wise operator, 83
AndStateList, 331–333
  ConcreteAndState and, 346
AndStateList.c, 347
AndStateList.h, 346–347
AND-states, 25, 265–267, 327–330
  nested states and, 266
Anonymous transition, 263
Aperiodic tasks, 158
API. See Application Program Interface
Application Program Interface (API), 5
  RTOS and, 163
ApplicationClient, 124
ApplicationElement, SharedResource and, 133
ApplicationProcessingElement, 139
Architectural design
  for Harmony/ERT, 36, 40
  patterns for, 45
Armed, 264, 265
Armed::ActivelyMonitoring::Detecting, 264
Arrival Pattern, 152
ArrythmiaDetector, 64, 68
ASet, 312
asm, 134

索引

&（和号），按位运算符，83
^（脱字符），按位运算符，83
<<（小于号），按位运算符，83
>>（大于号），按位运算符，83
|（竖线符号），按位运算符，83
~（波浪号），按位运算符，83
802.11x，82

A

通道模式摘要，395
对应 CRC，367
对应临界区模式，182
对应循环执行模式，165
对应去抖模式，123
对应分解与状态模式，331
对应双通道模式，413
对应受保护调用模式，190
对应硬件适配器模式，96
对应硬件代理模式，85–86
对应 Harmony/ERT，46
对应中断模式，130
对应中介者模式，100
对应多事件接收器模式，287
对应观察者模式，66、111–112
对应反码模式，362
对应轮询模式，138
对应受保护单通道模式，402
对应排队模式，207
对应会合模式，225–226
对应同时锁定模式，232–233
对应单事件接收器模式，274
对应智能数据模式，380
对应状态表模式，296
对应静态优先级模式，170–171
抽象主题继承及，68
对应观察者模式，66
TMDQueue 及，68、69
AbstractAndStateClass，331
AbstractCEThread，165
AbstractDataTransform，395–396、404、414
AbstractStaticThread，172
AbstractSubject，113
  notify() 及，112
AbstractThread、栈及，173
AbstractTransformChecker，404、414
accept(Datum)，112
access() 对应硬件代理类，87
accessGasFlow()，99
accessMotorDirection() 作为电机状态函数，90
accessMotorSpeed() 作为电机状态函数，90
accessMotorState() 作为电机错误管理函数，91
事故，359、360
AcmeO2Adapter.c，100
AcquireData()，398
动作，262
动作列表，26
对应迁移，27
迁移及，263
动作序列，152
ActionPtr0，298、310、312
ActionPtr1，298、311、312
ActionSet，331
ActionType，296、297、298
–，活动，–<< >>，161、162
  createThread() 及，173
  PreemptiveTask 与，191
ActivelyMonitoring，264
ActuatorDeviceDriver，396、404、414、415
Actuators，82
Ada，380
适配器客户端，用于硬件适配器模式，96
addLock()，ResourceList 与，247
After()，27、267
分析模式，44
AND，按位运算符，83
AndStateList，331–333
  ConcreteAndState 与，346
AndStateList.c，347
AndStateList.h，346–347
AND状态，25、265–267、327–330
  嵌套状态与，266
匿名转换，263
非周期任务，158
API。参见应用程序接口
应用程序接口（API），5
  RTOS 与，163
ApplicationClient，124
ApplicationElement，SharedResource 与，133
ApplicationProcessingElement，139
架构设计
  用于 Harmony/ERT，36、40
  模式用于，45
布防，264、265
Armed::ActivelyMonitoring::Detecting，264
到达模式，152
ArrythmiaDetector，64、68
ASet，312
asm，134

Asymmetric opportunistic polling, 141
Asynchronous communication, 155
  for Message Queuing Pattern, 207
Asynchronous events
  state machines and, 270–274
  transfer, 26
AsynchronousStateMachine, 271
  Mutex and, 272
Attitude.c, 199–200
AttitudeController, 195
AttitudeController.c, 203–204
AttitudeController.h, 202–203
Attitude.h, 198–199
ATTITUDE_MEMORY_FAULT, 364
AttitudeSensor, 248
attribute, 135
Attributes, 9
  of Abstract Subject, 68
  in classes, 10

B

Bareback, 6
Base class. See Parent class
BCPeriodicPoller, 142–143
BCPeriodicPoller.c, 147
BCPeriodicPoller.h, 144–147
BCPeriodicPoller_Init(), 142
BCPeriodicPoller_startPolling(), 143
BCTimer::installInterruptHandler(), 142
Behavior, 259–261
  discontinuous, 260
  initialize() and, 260
  piece-wise continuous, 261
  simple, 259
  stateful, 260
BEPeriodicPoller, 142
Binary semaphores, 189
BIT. See Built-in test
Bit fields, 82–83
  for bit-mapping device interfaces, 84
  problems with, 85
Bit masks, 83
Bit-wise operators, 83
Blocking, 152, 156
  PriorityQueue and, 172
  schedulability and, 159
Board Support Package (BSP), 163
Boolean expression, guards as, 27
BouncingDevice, 124
Boundary tests, 8
BSP. See Board Support Package
BuildInTestThread, 163
Built-in test (BIT), 3
Busy loops, 166
ButtonDriver.c, 130
ButtonDriver.h, 127–130
ButtonHandler, 135
ButtonHandler.c, 138
ButtonHandler.h, 137–138

C

C4ISR. See Command, Control, Communications, Computers, Intelligence, Surveillance, and Reconnaissance
CachedQueue
  header file, 22–25
  implementation of, 25
  UML for, 16
CachedQueue_GetSize(), 21
CachedQueue_Init(), 21
CalculateColor_freqToColor(), 398
CalculateFurnaceParams, 406
CallEvent, 26
Cancel event, 262
ChangeEvent, 26
Channel, 416
Channel Pattern, 395–402
  Protected Single Channel Pattern and, 405
ChannelArbiter, 418
CheckTemperature, 409–410
CheckTemperature.c, 411
CheckTemperature.h, 410–411
Child class, 15
Circular waiting condition, deadlock and, 232
Classes, 9, 10–13. See also Subclasses
  adapters, 98
  child, 15
  diagram, for UML, 16–17
  events in, 29
  mediator, for Mediator Pattern, 101–102
  objects in, 13–14
  parent, 15
    subclass and, 21
  super class, 68
Cleanup(), 69
clearErrorStatus(), as Motor Error Management Function, 91
Collaboration roles
  for Channel Pattern, 395–396
  for CRC, 368
  for Critical Region Pattern, 183
  for Cyclic Executive Pattern, 165–166
  for Debouncing Pattern, 123–125
  for Decomposed And-State Pattern, 331–333
  for Dual-Channel Pattern, 414–417
  for Guarded Call Pattern, 191
  for Hardware Adapter Pattern, 96–97
  for Hardware Proxy Pattern, 87–88
  for Interrupt Pattern, 131–132
  for Mediator Pattern, 101–102
  for Multiple Event Receptor Pattern, 288–290
  for Observer Pattern, 112–114
  for One's Complement Pattern, 362
  for Ordered Locking Pattern, 244–247
  for Polling Pattern, 138–141
  for Protected Single Channel Pattern, 404
  for Queuing Pattern, 208–209
  for Rendezvous Pattern, 226–227
  for Single Event Receptor Pattern, 275–278
  for Smart Data Pattern, 382
  for State Pattern, 310–312
  for State Table Pattern, 297–299
  for Static Priority Pattern, 172–173
Collaborator Interface, for Mediator Pattern, 101
ColorType, 383
Command, Control, Communications, Computers, Intelligence, Surveillance, and Reconnaissance (C4ISR), 2

非对称机会轮询，141 异步通信，155 用于消息排队模式，207 异步事件 状态机与，270–274 传输，26
AsynchronousStateMachine，271 互斥锁与，272 Attitude.c，199–200 AttitudeController，195 AttitudeController.c，203–204 AttitudeController.h，202–203 Attitude.h，198–199 ATTITUDE_MEMORY_FAULT，364 AttitudeSensor，248 _attribute_，135 属性，9 抽象主题的，68 类中的，10

B

裸机，6 基类。参见父类
BCPeriodicPoller，142–143
BCPeriodicPoller.c，147
BCPeriodicPoller.h，144–147
BCPeriodicPoller_Init()，142
BCPeriodicPoller_startPolling()，143
B
C
T
i
mer::installInterruptHandler()，142 行为，259–261 不连续，260 initialize() 与，260 分段连续，261 简单，259 有状态，260
BEPeriodicPoller，142 二元信号量，189 BIT。参见内置测试 位域，82–83 用于位映射设备接口，84 存在的问题，85 位掩码，83 位运算符，83 阻塞，152，156 PriorityQueue 与，172 可调度性与，159
板级支持包（BSP），163 布尔表达式，守卫作为，27
BouncingDevice，124 边界测试，8 BSP。参见板级支持包
BuildInTestThread，163 内置测试（BIT），3 忙循环，166
ButtonDriver.c，130
ButtonDriver.h，127–130
ButtonHandler，135
ButtonHandler.c，138
ButtonHandler.h，137–138

C

C4ISR。参见指挥、控制、通信、计算机、情报、监视与侦察
CachedQueue 头文件，22–25 的实现，25 的 UML，16
CachedQueue_GetSize()，21
CachedQueue_Init()，21
CalculateColor_freqToColor()，398 CalculateFurnacePara ms，406 调用事件，26 cancel 事件，262 变更事件，26 通道，416 通道模式，395–402 受保护单通道模式与，405
ChannelArbiter，418
CheckTemperature，409–410
CheckTemperature.c，411
CheckTemperature.h，410–411 子类，15 循环等待条件，死锁与，232 类，9，10–13。另见子类 适配器，98 子类，15 图，用于 UML，16–17 中的事件，29 中介者，用于中介者模式，101–102
对象，13–14 父类，15 子类与，21 超类，68 Cleanup()，69 clearErrorStatus()，作为电机错误管理函数，91 通道模式的协作角色，395–396 用于 CRC，368 用于临界区模式，183 用于循环执行模式，165–166 用于去抖模式，123–125 用于分解与状态模式，331–333 用于双通道模式，414–417 用于受保护调用模式，191 用于硬件适配器模式，96–97 用于硬件代理模式，87–88 用于中断模式，131–132 用于中介者模式，101–102 用于多事件接收器模式，288–290 用于观察者模式，112–114 用于反码模式，362 用于有序锁定模式，244–247 用于轮询模式，138–141 用于受保护单通道模式，404 用于排队模式，208–209 用于会合模式，226–227 用于单事件接收器模式，275–278 用于智能数据模式，382 用于状态模式，310–312 用于状态表模式，297–299 用于静态优先级模式，172–173 协作者接口，用于中介者模式，101 ColorType，383 指挥、控制、通信、计算机、情报、监视与侦察（C4ISR），2

Common Object Request Broker Architecture (CORBA), 6
Communication
  hardware, 81–82
  with tasks, 155
Compilers, bit fields and, 85
Completion events, 27
Complexity, as Harmony/ERT criteria, 42
computCRC(), 368
ComputeCRCData, 368
Concrete Observer, for Observer Pattern, 66
Concrete Subject, for Observer Pattern, 67
ConcreteAndState, AndStateList and, 346
ConcreteAndStateClass, 333, 334
ConcreteAndStateType, 333
ConcreteCEThread, 166
ConcreteClient, 113
ConcreteDataTransform, 396, 398, 404, 416
  SensorDeviceDriver and, 406
ConcreteSubject, 113–114
ConcreteThread, 172
  SharedResource and, 172
ConcreteTransformChecker, 404, 414, 417
Concurrency
  CPUs and, 155
  definition of, 152
  patterns for, 149–256
  tasks and, 155
  in UML, 161–163
Concurrency unit, 152
Conditional pseudostate, 268
configure()
  for Hardware Proxy class, 87
  as Motor Management Function, 90
Consequences
  for Channel Pattern, 396–397
  for CRC, 369
  for Critical Region Pattern, 184
  for Cyclic Executive Pattern, 167
  for Debouncing Pattern, 125
  for Decompose And-State Pattern, 333
  for Dual-Channel Pattern, 417
  for Hardware Adapter Pattern, 97–98
  for Hardware Proxy Pattern, 88–89
  for Harmony/ERT, 45–46, 46
  for Heterogeneous Redundancy Pattern, 418
  for Homogeneous Redundancy Pattern, 417
  for Interrupt Pattern, 132–134
  for Mediator Pattern, 102
  for Monitor-Actuator Pattern, 420
  for Multiple Event Receptor Pattern, 290
  for Observer Pattern, 67, 114
  for One's Complement Pattern, 363
  for Ordered Locking Pattern, 247
  for Polling Pattern, 141
  for Queuing Pattern, 209
  for Rendezvous Pattern, 227
  for Sanity Check Pattern, 419
  for Simultaneous Locking Pattern, 236
  for Single Event Receptor Pattern, 278
  for Smart Data Pattern, 382–383
  for State Pattern, 312–313
  for State Table Pattern, 299
  for Static Priority Pattern, 173
  for TMR, 418
Constraints
  on embedded design, 3
  preconditions and, 225
Constructors, 11
Context, 308, 311, 313
  Decompose AND-State Pattern and, 333
  me and, 312
Context.c, 317–318
Context.h, 315–317
Continuous behavior, 259
Continuous integration, 7–8
controlLoop(), 167
ControlThread, 163
ConvertToFrequency_computeFreq (), 398
CORBA. See Common Object Request Broker Architecture
cos(), 259
Counting semaphores, 189
  Thread Barrier Pattern and, 228
Coverage tests, 8
CPUs
  concurrency and, 155
  Interrupt Pattern and, 134
CRC. See Cycle Redundancy Check Pattern
CRCCalculator, 369
CRCCalculator.c, 372
CRCExample.c, 373
CRCProtectedData, 368
createThread(), > and, 173
Critical Region Pattern, 182–190
  deadlock and, 232
  Hardware Proxy Pattern and, 89
  Multiple Event Receptor Pattern and, 290
  Ordered Locking Pattern and, 248
  race condition and, 270
  Single Event Receptor Pattern and, 278
Criticality, 152
  tasks and, 157
CRRobotArmManager.c, 189
CRRobotArmManager.h, 186–189
CRSharedResource, 183
CurrentState, 308, 311, 313
Cycle Redundancy Check Pattern (CRC), 363, 367–380
  Protected Single Channel Pattern and, 405
CycleTimer, 166
Cyclic Executive Pattern, 164–170
Cyclic executive scheduling, 5
  problems with, 156
  processing loop, 155–157
  tasks for, 155
CyclicExecutive, 165–166

D

Data Distribution Service (DDS), 6
DataClient, 362
  CRC and, 368
DataType, 368
DataValue, 51
Datum
  AbstractSubject and, 113
  Observer Pattern and, 114
DDS. See Data Distribution Service

公共对象请求代理体系结构（CORBA），6
通信硬件，81–82
  与任务，155
编译器，位域与，85
完成事件，27
复杂性，作为Harmony/ERT准则，42
computCRC()，368
ComputeCRCData，368
具体观察者，用于观察者模式，66
具体主题，用于观察者模式，67
ConcreteAndState，AndStateList与，346
ConcreteAndStateClass，333，334
ConcreteAndStateType，333
ConcreteCEThread，166
ConcreteClient，113
ConcreteDataTransform，396，398，404，416
  SensorDeviceDriver与，406
ConcreteSubject，113–114
ConcreteThread，172
  SharedResource与，172
ConcreteTransformChecker，404，414，417
并发
  CPU与，155
  的定义，152
  用于并发的模式，149–256
  任务与，155
  在UML中，161–163
并发单元，152
条件伪状态，268
configure()
  用于硬件代理类，87
  作为电机管理功能，90
结果
  用于通道模式，396–397
  用于CRC，369
  用于临界区模式，184
  用于循环执行模式，167
  用于去抖模式，125
  用于分解与状态模式，333
  用于双通道模式，417
  硬件适配器模式，97–98
  硬件代理模式，88–89
Harmony/ERT，45–46, 46
异构冗余模式，418
同构冗余模式，417
中断模式，132–134
中介者模式，102
监控器-执行器模式，420
多事件接收器模式，290
观察者模式，67, 114
反码模式，363
有序锁定模式，247
轮询模式，141
排队模式，209
会合模式，227
合理性检查模式，419
同时锁定模式，236
单事件接收器模式，278
智能数据模式，382–383
状态模式，312–313
状态表模式，299
静态优先级模式，173
TMR，418
嵌入式设计约束，3
前置条件与，225
构造函数，11
上下文，308, 311, 313
  分解与状态模式与，333
  me与，312
Context.c，317–318
Context.h，315–317
连续行为，259
持续集成，7–8
controlLoop()，167
ControlThread，163
ConvertToFrequency_computeFreq ()，398
CORBA。参见 公共对象请求代理体系结构
cos()，259
计数信号量，189
线程屏障模式与，228
覆盖率测试，8
CPU
  并发与，155
  中断模式与，134
CRC。参见循环冗余校验模式
CRCCalculator，369
CRCCalculator.c，372
CRCExample.c，373
CRCProtectedData，368
createThread()，>与，173
临界区模式，182–190
  死锁与，232
  硬件代理模式与，89
  多事件接收器模式与，290
  有序锁定模式与，248
  竞态条件与，270
  单事件接收器模式与，278
关键性，152
  任务与，157
CRRobotArmManager.c，189
CRRobotArmManager.h，186–189
CRSharedResource，183
CurrentState，308，311，313
循环冗余校验模式（CRC），363，367–380
  受保护单通道模式与，405
CycleTimer，166
循环执行模式，164–170
  循环执行调度，5
  相关问题，156
  处理循环，155–157
  任务，155
CyclicExecutive，165–166

D

数据分发服务（DDS），6
DataClient，362
  CRC与，368
DataType，368
DataValue，51
Datum AbstractSubject与，113
  观察者模式与，114
DDS。参见 数据分发服务

Deadlines, 2
  Cyclic Executive Pattern and, 164
  definition of, 152
  tasks and, 159
Deadlock, 152
  avoidance patterns, 231–255
  Interrupt Pattern and, 132
Debouncer, 124–125
Debouncing Pattern, 122–130
DebouncingTimer, 125–126
Debugging, 4, 8–9
Decomposed And-State Pattern, 330–355
Default pseudostate, 268
Defensive design, 380
#define, 83, 364
Deinstall function, for Interrupt Pattern, 131
delay(), 125
Derived class. See Child class
Design
  defensive, 380
  detailed, 37
  for Harmony/ERT, 36–37
    criteria for, 40–42
    validation of, 44
  mechanistic, 37
Design pattern. See Patterns
Destroy_semaphore(Semaphore*), 227
Destructors, 11
Detailed design, for Harmony/ERT, 37
Detecting, 264
Deterministic semantics, 270
Device, 139–140
DeviceAddr, for Hardware Proxy class, 87–88
Dijkstra, Edsger, 189
Direct Memory Access (DMA), 82
disable()
  for Hardware Proxy class, 87
  as Motor Management Functions, 90
Discontinuous behavior, 260
DispatchEvent, 271–272
DisplayClient.c, 121
DisplayClient.h, 120–121
Distributability, as Harmony/ERT criteria, 42
Diverse Design Pattern, 413
DMA. See Direct Memory Access
Done event, 263
DoneWaiting, 267
DoorOpen event 261, 262
Dual-Channel Pattern, 413–422
dumpList(), 116
Dyadic access functions, resource Client and, 247

E

Earliest deadline first (EDF), 5
  for Dynamic Priority Pattern, 174–175
ECG_Module header file, 54
ECG_Module.c, 58–59
ECGPkg.h, 52–53
EDF. See Earliest deadline first
Embedded tool chain, 4–5
EMGChannel.c, 402
EMGChannel.h, 400–402
EMGSensorDeviceDriver, 397
emitMicrowaves(), as action, 262
enable(), as Motor Management Functions, 90
Encapsulation boundary, 10, 13
Encryption, security and, 89
Energy consumption, as Harmony/ERT criteria, 42
EnteringHouse, 264
entry1(), 263
entry2(), 263
EntryPoint, StartAddr and, 173
enum, 275
Errors, 360
ErrorCodeType, 372, 382
errorHandler(), 368
ErrorManager, 382
ErrorState.c, 352
ErrorStateClass, 334, 348–349
ErrorStateClass.h, 349–352
Ethernet, 82
evAbort, 355
evDelayedActivation, 264
evDigit(c:char), 289
evDisarm, 265
evDot, 289
evEnable, 355
evEndOfString, 289
Event(s), 296, 298
  in classes, 29
  handlers, 27
    And-states and, 329
  name, 263
  parameters of, 27
  propagation, 329
  receptors, event dispatcher function and, 29
  RTOS and, 163
  signature, 26
  Single Event Receptor Pattern and, 275
  source, for task identification, 161
  states and, 27
  triggers, transitions and, 27
  in UML, 26
Event dispatcher function, event receptors and, 29
eventDispatch(), 297, 299–300
EventQueue, 272
eventReceive(), 124
EventType, 272, 275, 298
evError, 355
evImmediateActivation, 264
evRun, 355
evWhiteSpace, 289
executeStep(), 103
Execution efficiency, of Observer Pattern, 64–65
Execution time, 153
exitAction(), 263
ExitingHouse, 264

F

FailuretoZero, 269
Faults
  reliability and, 360—361
  safety and, 360—361
Fault Extenuators, 361
Fault Identifiers, 361
Fault Manifestors, 361
Fault Means and Effect Analysis (FMEA), 359
Fault Seeding tests, 8
File system services, RTOS and, 163
Final pseudostate, 263, 268
Finite state machine (FSM), 25–33, 261–274

截止时间，2 循环执行模式与，164 定义，152 任务与，159 死锁，152 避免模式，231–255 中断模式与，132 去抖器，124–125 去抖模式，122–130 去抖定时器，125–126 调试，4，8–9 分解AND状态模式，330–355 默认伪状态，268 防御性设计，380 #define，83，364 卸载函数，用于中断模式，131 delay()，125 派生类。参见 子类 设计 防御性，380 详细，37 用于Harmony/ERT，36–37 准则，40–42 验证，44 机制性，37 设计模式。参见 模式 Destroy_semaphore(Semaphore*)，227 析构函数，11 详细设计，用于 Harmony/ERT，37 检测，264 确定性语义，270 设备，139–140 DeviceAddr，用于硬件代理类，87–88 Edsger Dijkstra，189 直接内存访问（DMA），82 disable() 用于硬件代理类，87 作为电机管理功能，90 不连续行为，260 DispatchEvent，271–272 DisplayClient.c，121 DisplayClient.h，120–121 可分布性，作为 Harmony/ERT 准则，42
多样性设计模式，413 直接内存访问（DMA）。参见直接内存访问完成事件，263 DoneWaiting，267 DoorOpen事件 261，262 双通道模式，413–422 dumpList()，116 二元访问函数，资源客户端与，247

E

最早截止时间优先（EDF），5 用于动态优先级模式，174–175 ECG_模块头文件，54 ECG_模块.c，58–59 ECGPkg.h，52–53 EDF。参见最早截止时间优先 嵌入式工具链，4–5 EMGChannel.c，402 EMGChannel.h，400–402 EMGSensorDeviceDriver，397 emitMicrowaves()，作为动作，262 enable()，作为电机管理功能，90 封装边界，10、13 加密，安全与，89 能耗，作为Harmony/ERT标准，42 进屋，264 entry1()，263 entry2()，263 EntryPoint，StartAddr与，173 枚举，275 错误，360 ErrorCodeType，372、382 errorHandler()，368 ErrorManager，382 ErrorState.c，352 ErrorStateClass，334、348–349 ErrorStateClass.h，349–352 以太网，82 evAbort，355 evDelayedActivation，264 evDigit(c:char)，289 evDisarm，265 evDot，289 evEnable，355 evEndOfString，289 事件，296、298
  在类中，29 处理程序，27 与状态及，329 名称，263 的参数，27 传播，329 受体，事件分发函数与，29 RTOS与，163 签名，26 单事件接收器模式与，275 源，用于任务识别，161 状态与，27 触发器，转换与，27 在UML中，26 事件分发函数，事件接收器与，29 eventDispatch()，297、299–300 EventQueue，272 eventReceive()，124 EventType，272、275、298 evError，355 evImmediateActivation，264 evRun，355 evWhiteSpace，289 executeStep()，103 执行效率，观察者模式的，64–65 执行时间，153 exitAction()，263 离家，264

F

归零，269 故障可靠性与，360—361 安全性与，360—361 故障缓解者，361 故障识别者，361 故障显现者，361 故障模式与影响分析 (FMEA)，359 故障注入测试，8 文件系统服务，RTOS 与，163 最终伪状态，263，268 有限状态机 (FSM)，2533，261 274 – –

FLAG, WAIT_FOREVER and, 193
FlightDataDisplay, 195
FlightDataDisplay.c, 207
FlightDataDisplay.h, 205–207
FMEA. See Fault Means and Effect Analysis
Fork, threads and, 154
Fork pseudostate, 268
FSM. See Finite state machine
Function pointers
  OO and, 13
  in specialization, 16, 18
Functional tests, 8
FurnaceController, 406

G

Gas Display, 98
Gas Mixer, 98
GasControlExec, 167
GasControlExecutive.c, 169
GasController, handleGasData() and, 224
GasController.c, 222–223
GasController.h, 221–222
GasController_handleGasData(), 221
GasData, 210–211
GasData.c, 122, 213
GasDataExample.h, 212
GasData.h, 122, 212–213
GasDataQueue
  insert() and, 213
  remove() and, 213
GasDataQueue.c, 217–218
GasDataQueue.h, 214–217
GasDisplay, printGasData() and, 224
GasDisplay.c, 224
GasDisplay.h, 223–224
GasDisplay_printGasData(), 221
GasDisplayThread.c, 170
GasMixerClient, 119
GasNotificationHandle, 115
GasProcessingThread, SensorThread and, 224
GasProcessingThread_processData(), 221
GasSensor, 116
GasSensor.c, 119
GasSensor.h, 117–119
GAS_TYPE, 210
GCC. See GNU C Compiler
Generalization, in UML, 15
get02Conc(), 99
get02Flow(), 99
getData(), 133
getNextIndex(), 213
getValue(), 183
getXXData(), 218
gimme02Conc(), 98
gimme02Flow(), 98
Global data declarations, 155
GN_PROCESSING FRACTIONAL PART_STATE, 299, 312
GN_Processing Fraction State, 311, 323–324
GN_Processing Fraction State.c, 326
GN_Processing Fraction State.h, 324–326
GN_PROCESSING WHOLE PART_STATE, 299, 312
GN_Processing Whole State, 311, 321
GN_Processing Whole State.c, 323
GN_Processing Whole State.h, 321–323
GNU C Compiler (GCC), 4, 135
Got Number, 299
GPSSpeedChannel, 421
graspAt(), 103–104
Green Hills Software, Integrity by, 5
Guards
  as Boolean expression, 27
  transitions and, 263
Guarded Call Pattern, 190–207
  Hardware Proxy Pattern and, 89
  Multiple Event Receptor Pattern and, 290
  Mutex and, 299
  priority inversion from, 210
  race condition and, 270
  Rendezvous Pattern and, 228
  Single Event Receptor Pattern and, 278
GuardedCallExample.h, 195–196
GuardedResource
  lock() and, 192
  Mutex and, 191–192
GuardedSource, 190
GuardType, 296–297, 298

H

handleGasData(), gasController and, 224
handleInterrupt_x(), 132
handleTimerInterrupt(), 141
Hard real-time systems, 2
Hardware
  categories of, 81–82
  codevelopment, 7–8
  design patterns for accessing, 79–147
  infrastructure, 81
Hardware Adapter, for Hardware Adapter Pattern, 96
Hardware Adapter Class, for Hardware Adapter Pattern, 97
Hardware Adapter Pattern, 96–100
  Polling Pattern and, 142
Hardware Device
  for Hardware Adapter Pattern, 97
  for Hardware Proxy Pattern, 87
Hardware Interface to Client, for Hardware Adapter Pattern, 97
Hardware Proxy
  for Hardware Adapter Pattern, 97
  for Hardware Proxy Pattern, 87
  Specific Collaborator and, 102
Hardware Proxy Pattern, 85–95
  Hardware Adapter Pattern and, 98
  Polling Pattern and, 142
HardwareProxyExample.h, 91–92
Harmony for Embedded RealTime (Harmony/ERT), 35–78
  architectural design for, 36, 40
  design for, 36–37
    criteria for, 40–42
    patterns and technologies for, 42–44
    validation of, 44
  detailed design for, 37
  macrocycle for, 36
  mechanistic design for, 37
  microcycle for, 36, 38–39
  nanocycle for, 36
  timescales for, 36

FLAG，WAIT_FOREVER 与，193 FlightDataDisplay，195 FlightDataDisplay.c，207 FlightDataDisplay.h，205–207 FMEA。参见 故障模式与影响分析 分叉，线程与，154 分叉伪状态，268 FSM。参见 有限状态机 函数指针 面向对象与，13 在特化中，16，18 功能测试，8 FurnaceController，406

G

气体显示器，98 气体混合器，98 GasControlExec，167 GasControlExecutive.c，169 GasController，handleGasData() 以及，224 GasController.c，222–223 GasController.h，221–222 GasController_handleGasData()，221 GasData，210–211 GasData.c，122，213 GasDataExample.h，212 GasData.h，122，212–213 GasDataQueue insert() 以及，213 remove() 以及，213 GasDataQueue.c，217–218 GasDataQueue.h，214–217 GasDisplay，printGasData() 以及，224 GasDisplay.c，224 GasDisplay.h，223–224 GasDisplay_printGasData()，221 GasDisplayThread.c，170 GasMixerClient，119 GasNotificationHandle，115 GasProcessingThread，SensorThread 以及，224 GasProcessingThread_processData ()，221 GasSensor，116 GasSensor.c，119 GasSensor.h，117–119 GAS_TYPE，210 GCC。参见 GNU C 编译器 泛化，在 UML 中，15 get02Conc()，99 get02Flow()，99 getData()，133 getNextIndex()，213 getValue()，183 getXXData()，218 gimme02Conc()，98 gimme02Flow()，98 全局数据声明，155 GN PROCESSING_FRACTIONAL PART_STATE，299，312 GN_Processing Fraction State，311，323 324–GN_Processing Fraction State.c，326 GN_Processing Fraction State.h，324–326 GN PROCESSING WHOLE_PART_STATE，299，312 GN_Processing Whole State，311，321 GN_Processing Whole State.c，323 GN_Processing Whole State.h，321–323 GNU C 编译器（GCC），4，135 Got Number，299 GPSSpeedChannel，421 graspAt()，103–104 Green Hills Software，Integrity by，5 守卫条件作为布尔表达式，27 转换与，263 受保护调用模式，190–207 硬件代理模式与，89 多事件接收器模式与，290 互斥锁与，299 优先级反转来自，210 竞态条件与，270 会合模式与，228 单事件接收器模式与，278 GuardedCallExample.h，195–196 受保护资源 lock() 与，192 互斥锁与，191–192 GuardedSource，190 GuardType，296–297，298

H

handleGasData()、gasController 以及，224 handleInterrupt_x()、132 handleTimerInterrupt()、141 硬实时系统，2 硬件类别，81–82 协同开发，7–8 用于访问的设计模式，79–147 基础设施，81 硬件适配器，用于硬件适配器模式，96 硬件适配器类，用于硬件适配器模式，97 硬件适配器模式，96–100 轮询模式与，142 硬件设备用于硬件适配器模式，97 用于硬件代理模式，87 面向客户端的硬件接口，用于硬件适配器模式，97 硬件代理用于硬件适配器模式，97 用于硬件代理模式，87 特定协作者与，102 硬件代理模式，85–95 硬件适配器模式与，98 轮询模式与，142 HardwareProxyExample.h，91–92 Harmony for Embedded RealTime (Harmony/ERT)，35–78 架构设计，36、40 设计，36–37 准则，40–42 模式与技术，42–44 验证，44 详细设计，37 宏周期，36 机制设计，37 微周期，36、38–39 纳周期，36 时间尺度，36

Harmony/ERT. See Harmony for Embedded RealTime
Hazards, 359
Head and, Message, 209
HESensor, 217–218
Heterogeneous redundancy, 360
Heterogeneous Redundancy Pattern, 413, 418
HistogramDisplay, 64, 68
HistogramDisplay header file, 54–56
HistogramDisplay.c implementation file, 59–61
  with Observer Pattern, 77
Homogeneous redundancy, 360
Homogeneous Redundancy Pattern, 413, 417

I

IBM Rational, 36
IDEs. See Integrated Development Environments
Idioms, 361
Impedance matching, 96
Implementation strategies
  for Channel Pattern, 397
  for CRC, 369
  for Critical Region Pattern, 184
  for Cyclic Executive Pattern, 167
  for Debouncing Pattern, 125–126
  for Decompose And-State Pattern, 333
  for Guarded Call Pattern, 192–193
  for Hardware Adapter Pattern, 98
  for Hardware Proxy Pattern, 89
  for Interrupt Pattern, 134–135
  for Mediator Pattern, 102
  for Multiple Event Receptor Pattern, 290
  for Observer Pattern, 114
  for One's Complement Pattern, 363
  for Ordered Locking Pattern, 248
  for Polling Pattern, 141
  for Protected Single Channel Pattern, 405
  for Queuing Pattern, 210
  for Rendezvous Pattern, 228
  for Simultaneous Locking Pattern, 236
  for Single Event Receptor Pattern, 278
  for Smart Data Pattern, 383
  for State Pattern, 313
  for State Table Pattern, 299
  for Static Priority Pattern, 173–174
Independent processing, for task identification, 161
Infrastructure hardware, 81
Inheritance, 15
  Abstract Subject and, 68
  priority, 193
  subclass and, 68
  super class and, 68
  virtual functions and, 68
Init(), 69
  TokenizerStateTable and, 297
Initial transition, 263
initialize()
  as action, 262
  behaviors and, 260
  for Hardware Proxy class, 88
  Mediator Pattern and, 101
  as Motor Management Functions, 90
  states and, 260
Initializers, 11
  KinematicData_Init(), 195
InitRelations(), 398
insert()
  GasDataQueue and, 213
  QTask and, 209
installInterruptTimer(), 140
Install function, for Interrupt Pattern, 131
Instances. See Objects
int, 13, 275, 310
int*, for Hardware Proxy class, 87
int MessageQueue_getNextindex (MessageQueue* me, int index), 208
int MessageQueue_insert (MessageQueue* me, Message m), 209
Integrated Development Environments (IDEs), 4
Integrity, by Green Hills Software, 5
Interarrival time, 153
Interface device recurrence properties, for task identification, 161
Internal Data Formatting Functions (private), as motor proxy, 91
Interrupt, 135
Interrupt handler, for task identification, 161
Interrupt Pattern, 130–138
  Polling Pattern and, 141
InterruptHandler, 131–132
InterruptVectorTable, 132
Intrusion, 264
ISR
  Interrupt Pattern and, 132–134
  Polling Pattern and, 141
itsContext, 313
itsOtherChannel, 414
itsPartMutex, 234

J

Jitter, 153
  Polling Pattern and, 141
Join pseudostate, 268
Joins, threads and, 154
Joint Test Architecture Group (JTAG), 4, 9
JTAG. See Joint Test Architecture Group

K

Keypress, 26
KinematicData.c, 201–202
KinematicData_Cleanup(), 195
KinematicData.h, 200–201
KinematicData_Init(), 195
KinematicModelClient, 248
Klein, Jack, 369

L

Leaky queue, 51
Light Controller Client, 355
LightController.c, 346
LightController.h, 337–346
LightPkg.h, 335–337
Linkers, 4
Load testing, 8

Harmony/ERT。参见 Harmony for Embedded RealTime 危险，359 头部与，消息，209 HESensor，217–218 异构冗余，360 异构冗余模式，413、418 HistogramDisplay，64、68 HistogramDisplay头文件，54–56 HistogramDisplay.c实现文件，59–61 与观察者模式，77 同构冗余，360 同构冗余模式，413、417

I

IBM Rational，36 集成开发环境。参见集成开发环境惯用法，361 阻抗匹配，96 通道模式的实现策略，397 CRC 的实现策略，369 临界区模式的实现策略，184 循环执行模式的实现策略，167 去抖模式的实现策略，125–126 分解与状态模式的实现策略，333 受保护调用模式的实现策略，192–193 硬件适配器模式的实现策略，98 硬件代理模式的实现策略，89 中断模式的实现策略，134–135 中介者模式的实现策略，102 多事件接收器模式的实现策略，290 观察者模式的实现策略，114 反码模式的实现策略，363 有序锁定模式的实现策略，248 轮询模式的实现策略，141 受保护单通道模式的实现策略，405 排队模式的实现策略，210 会合模式的实现策略，228
同时锁定模式的实现策略，236 单事件接收器模式的实现策略，278 智能数据模式的实现策略，383 状态模式的实现策略，313 状态表模式的实现策略，299 静态优先级模式的实现策略，173–174 独立处理，用于任务识别，161 基础设施硬件，81 继承，15 抽象主题与继承，68 优先级，193 子类与继承，68 超类与继承，68 虚函数与继承，68 Init()，69 TokenizerStateTable 与 Init()，297 初始转换，263 initialize() 作为动作，262 行为与 initialize()，260 用于硬件代理类，88 中介者模式与 initialize()，101 作为电机管理功能，90 状态与 initialize()，260 初始化器，11 KinematicData_Init()，195 InitRelations()，398 insert() 与 GasDataQueue，213 QTask 与 insert()，209 installInterruptTimer()，140 安装函数，用于中断模式，131 实例。参见对象 int，13，275，310 int*，用于硬件代理类，87 int MessageQueue_getNextindex (MessageQueue* me, int index)，208 int MessageQueue_insert (MessageQueue* me, Message m)，209 集成开发环境（IDEs），4 完整性，由 Green Hills Software 提供，5
到达间隔时间，153 接口设备重现属性，用于任务识别，161 内部数据格式化函数（私有），作为电机代理，91 中断，135 中断处理程序，用于任务识别，161 中断模式，130–138 轮询模式与，141 InterruptHandler，131–132 中断向量表，132 入侵，264 中断服务例程 中断模式与，132–134 轮询模式与，141 itsContext，313 itsOtherChannel，414 itsPartMutex，234

J

抖动，153 轮询模式与，141 汇合伪状态，268 汇合，线程与，154 联合测试架构组（JTAG），4，9 JTAG。参见 联合测试架构组

K

按键，26 KinematicData.c，201–202 KinematicData_Cleanup()，195 KinematicData.h，200–201 KinematicData_Init()，195 运动模型客户端，248 Klein, Jack，369

L

泄漏队列，51 灯光控制器客户端，355 LightController.c，346 LightController.h，337–346 LightPkg.h，335–337 链接器，4 负载测试，8

Loaders, 4
lock()
  GuardedResource and, 192
  MessageQueue and, 209
  Mutex and, 235
  QueryMutex and, 235
  semaphores and, 189, 191
lockDyadic(), resourceClient and, 244
Locking, deadlock and, 232
lock_semaphore(Semaphore*), 227
Long, 51
Loops
  busy, 166
  Cyclic Executive Pattern and, 164
  PID, 259
  for priority-based scheduling, 157
  for time-slicing schedule, 157

M

Macrocycle, for Harmony/ERT, 36
main(), 136–137
Main.c, 63–64
  Static Priority Pattern and, 175–177
Maintainability
  as Harmony/ERT criteria, 42
  of Observer Pattern, 64–65
marshal()
  for Hardware Proxy class, 88
  as Internal Data Formatting Function, 91
MasteredResource, 234
MasterResources, SimplifiedResourceMaster and, 235
MAX_POLL_DEVICES, 140
me, 10, 310
  Context and, 312
Mealy-Moore FSM, 25
Mean Time Between Failure (MTBF), 359
Mechanistic design, for Harmony/ERT, 37
Mediator class, for Mediator Pattern, 101–102
Mediator Pattern, 100–111
MedicalDisplay, 143
Memory. See Resource usage
MERSMs. See Multiple event receptor finite state machines
Message
  Head and, 209
  QTasks and, 208
Message* MessageQueue_remove (MessageQueue* me), 209
Message Queuing Pattern, asynchronous communication for, 207
MessageQueue
  lock() and, 209
  Mutex and, 209
  QTask and, 209
  release() and, 209
Microcycle, for Harmony/ERT, 36, 38–39
MicrowaveEmitter, 126
Microwaving, 261
Middleware, 6–7
Milliseconds, states and, 267
Minimum interarrival time, 158
Mishaps, 359
Mix-in, 68
ModeStateClass, 334, 348–349
ModeStateClass.c, 354
ModeStateClass.h, 352–354
Monadic functions, 244
MonadicAccessFunction, 244–245
Monitor-Actuator Pattern, 420
Motor Control Functions, for motor proxy, 91
Motor Error Management Functions, for motor proxy, 91
Motor Management Functions, for motor proxy, 90
Motor proxy, 89—91
Motor Status Functions, for motor proxy, 90
MotorController.c, 181
MotorController.h, 179–180
MotorData.c, 181
MotorData.h, 92–93, 177–178, 181
MotorDisplay.c, 179
MotorDisplay.h, 178–179
MotorPositionSensor.c, 178
MotorProxy.c, 95
MotorProxy.h, 93–95
MoveToYPosition, 270
MovingAverageFilter_filter(), 398
MovingRobotArm, 269–270
MTBF. See Mean Time Between Failure
MultimasteredResource, 234
  ResourceMaster and, 235
  trylock() and, 235
Multiple event receptor finite state machines (MERSMs), 287
Multiple Event Receptor Pattern, 287–296
  Context and, 311
Multitasking preemptive scheduling
  definition of, 153
  mutual exclusion and, 189
  priority inversion and, 159
Mutate(), for Hardware Proxy class, 88
Mutex
  AsynchronousStateMachine and, 272
  Guarded Call Pattern and, 299
  GuardedResource and, 191–192
  lock() and, 235
  MessageQueue and, 209
  Multiple Event Receptor Pattern and, 289
  release() and, 172, 235
  SharedResource and, 172
  Single Event Receptor Pattern and, 276
  State Pattern and, 311
  State Table Pattern and, 298
  StaticPriorityScheduler and, 172, 192
  trylock() and, 234
mutexID, 192
Mutual exclusion. See also Mutex
  deadlock and, 232
  multitasking preemptive scheduling and, 190
  semaphores, 190
    trylock() and, 236

N

N2Sensor, 217–218
Name, for Harmony/ERT, 45
Nanocycle, for Harmony/ERT, 36
Navigator, 195

加载器，4 lock() 与 GuardedResource，192 与 MessageQueue，209 与 Mutex，235 与 QueryMutex，235 与信号量，189、191 lockDyadic()，与 resourceClient，244 加锁，与死锁，232 lock_semaphore(Semaphore*)，227 长整型，51 循环 忙等待，166 与循环执行模式，164 PID，259 用于基于优先级的调度，157 用于时间片调度，157

M

宏周期，用于Harmony/ERT，36 main()，136–137 Main.c，63–64 静态优先级模式与，175–177 可维护性作为 Harmony/ERT准则，42 观察者模式的，64–65 marshal()用于硬件代理类，88 作为内部数据格式化函数，91 受控资源，234 资源主控，简化资源主控与，235 MAX_POLL_DEVICES，140 me，10，310 上下文与，312 Mealy-Moore 有限状态机，25 平均故障间隔时间（MTBF），359 机制设计，用于 Harmony/ERT，37 中介者类，用于中介者模式，101–102 中介者模式，100–111 MedicalDisplay，143 内存。参见资源使用
MERSM。参见多事件接收器有限状态机 消息头与，209 QTasks与，208 Message* MessageQueue_remove (MessageQueue* me)，209 消息排队模式，用于异步通信，207 MessageQueue lock()与，209 互斥锁与，209 QTask与，209 release()与，209 微周期，用于Harmony/ERT，36，38 39– 微波发射器，126 微波加热，261 中间件，6–7 毫秒，状态与，267 最小到达间隔时间，158 事故，359 混入，68 ModeStateClass，334，348–349 ModeStateClass.c，354 ModeStateClass.h，352–354 一元函数，244 MonadicAccessFunction，244–245 监控器-执行器模式，420 电机控制函数，用于电机代理，91 电机错误管理函数，用于电机代理，91 电机管理功能，用于电机代理，90 电机代理，89–91 电机状态函数，用于电机代理，90 MotorController.c，181 MotorController.h，179–180 MotorData.c，181 MotorData.h，92–93，177–178，181 MotorDisplay.c，179 MotorDisplay.h，178–179 MotorPositionSensor.c，178 MotorProxy.c，95 MotorProxy.h，93–95
MoveToYPosition，270 MovingAverageFilter_filter()，398 MovingRobotArm，269–270 MTBF。参见平均故障间隔时间 MultimasteredResource，234 ResourceMaster 与，235 trylock() 与，235 多事件接收器有限状态机（MERSM），287 多事件接收器模式，287–296 上下文与，311 多任务抢占式调度 定义，153 互斥与，189 优先级反转与，159 Mutate()，用于硬件代理类，88 互斥锁 异步状态机与，272 受保护调用模式与，299 受保护资源与，191–192 lock() 与，235 MessageQueue 与，209 多事件接收器模式与，289 release() 与，172，235 共享资源与，172 单事件接收器模式与，276 状态模式与，311 状态表模式与，298 StaticPriorityScheduler 与，172，192 trylock() 与，234 互斥量ID，192 互斥。另见互斥锁 死锁与，232 多任务抢占式调度与，190 信号量，190 trylock() 与，236

N

N2Sensor，217–218 名称，用于 Harmony/ERT，45 纳周期，用于 Harmony/ERT，36 Navigator，195

Navigator.c, 205
Navigator.h, 204–205
Nested states, 25, 263–264
  And-states and, 266
Network input/output services, RTOS and, 163
NewState, 297
Nonperiodic tasks, 158
NoNumberState, 311
NONUMBER_STATE, 299, 312
NoNumberState.c, 321
NoNumberState.h, 319–321
NOT, bit-wise operator, 83
Notification handle, for Observer Pattern, 66
NotificationHandle
  AbstractSubject and, 113
  Observer Pattern and, 114
  TMDQueue and, 69
NotificationHandle.c, 75–77
NotificationHandle.h, 74
notify(), 69, 113
  AbstractSubject and, 112
  SynchronizingThread and, 227
Null events, 27, 29
Null_id, 29
NULL_STATE, 299, 312
N-way programming, 360

O

O2Sensor, 217–218
O2Sensor.c, 220
O2Sensor.h, 220
Object adapters, 98
Object Analysis, 36
Object Management Group (OMG), 6
Object-oriented programming (OO)
  function pointers and, 13
  vs. structured language, 9–10
Objects, in classes, 11, 13–14
Observer Pattern, 51–77, 111–122
  HistogramDisplay.c with, 77
  Rendezvous Pattern and, 227, 228
  TMDQueue header file with, 70–74
  TMDQueue implementation file with, 74
Off, 261, 265
OMG. See Object Management Group
One's Complement Pattern, 362–367
  CRC and, 369
  Protected Single Channel Pattern and, 405
OnesCompProtectedDataElement, 362
OO. See Object-oriented programming
Operating system (OS), 5–6
Operations, 9
  of Abstract Subject, 68
  in classes, 10
  states and, 25
Opportunistic Polling Pattern, 139
OpportunisticPoller, 140
OpticalWheelSpeedChannel, 421
OR, bit-wise operator, 83
Ordered Locking Pattern, 242–255
  Hardware Proxy Pattern and, 89
OrderedResource
  ResourceClient and, 245, 247
  resourceID and, 244
OrderedResource_Init(), resourceID and, 244
OR-states, 263–265
OS. See Operating system
OS_create_semaphore, 195
OS_disable_task_switching(), 184
OS_enable_task_switching(), 184
OS_lock_semaphore(), 195
OSQPend(), 176
OS_release_semaphore(), 195
OSSemaphore, 195
OutputQueue, 21
OwnShipAttitude, 363–364
OwnShipAttitude.c, 367
OwnShipAttitude.h, 365–367

P

Parameters, of events, 27
Parent class, 15
  subclass and, 21
Passed by reference, 207
Passed by value, threads and, 207
PatientData, 369, 373–375
PatientData.c, 380
PatientDataClass, 383, 392–393
PatientDataClass.c, 394
PatientDataClass.h, 393–394
PatientData.h, 375–380
PatientDataType, 369
Patterns
  for accessing hardware, 79–147
  analysis, 44
  Arrival Pattern, 152
  Channel Pattern, 395–402
  for concurrency, 149–256
  CRC, 363, 367–380
    Protected Single Channel Pattern and, 405
  Critical Region Pattern, 182–190
    deadlock and, 232
    Hardware Proxy Pattern and, 89
    Multiple Event Receptor Pattern and, 290
    Ordered Locking Pattern and, 248
    race condition and, 270
    Single Event Receptor Pattern and, 278
  Cyclic Executive Pattern, 164–170
  Debouncing Pattern, 122–130
  Decomposed And-State Pattern, 330–355
  Diverse Design Pattern, 413
  Dual-Channel Pattern, 413–422
  Guarded Call Pattern, 190–207
    Hardware Proxy Pattern and, 89
    Multiple Event Receptor Pattern and, 290
    Mutex and, 299
    priority inversion from, 210
    race condition and, 270
    Rendezvous Pattern and, 228
    Single Event Receptor Pattern and, 278
  Hardware Adapter Pattern, 96–100
    Polling Pattern and, 142
  Hardware Proxy Pattern, 85–95
    Hardware Adapter Pattern and, 98
    Polling Pattern and, 142
  for Harmony/ERT design, 44–77

Navigator.c，205
Navigator.h，204–205 嵌套状态，25，263–264 与状态及，266 网络输入/输出服务，RTOS 与，163 NewState，297 非周期性任务，158
NoNumberState，311
NONUMBER_STATE，299，312 NoNumberState.c，321
NoNumberState.h，319–321 NOT，按位运算符，83 通知句柄，用于观察者模式，66
NotificationHandle 抽象主题与，113 观察者模式与，114
TMDQueue 与，69
NotificationHandle.c，75–77 NotificationHandle.h，74 notify()，69，113 抽象主题与，112
SynchronizingThread 与，227 空事件，27，29 空_id，29 NULL_STATE，299，312
N版本编程，360

O

O2Sensor，217–218
O2Sensor.c，220 O2Sensor.h，220 对象适配器，98 对象分析，36 对象管理组织（OMG），6 面向对象编程（OO）函数指针与，13 与结构化语言对比，9–10 对象，在类中，11，13–14 观察者模式，51–77，111–122
HistogramDisplay.c 与，77 会合模式与，227，228
TMDQueue头文件与，70–74
TMDQueue实现文件与，74 关闭，261，265
OMG。参见对象管理组织 反码模式，362–367 CRC与，369 受保护单通道模式与，405
O
n
esCompProtectedDataElement，362 OO。参见面向对象编程 操作系统（OS），5–6 操作，9 抽象主题的，68 在类中，10 状态与，25 机会轮询模式，139
OpportunisticPoller，140
OpticalWheelSpeedChannel，421 OR，按位运算符，83 有序锁定模式，242–255 硬件代理模式与，89 OrderedResource
ResourceClient与，245，247
resourceID与，244
OrderedResource_Init()，resourceID与，244 或状态，263–265 OS。参见操作系统 OS_create_semaphore，195 OS_disable_task_switching()，184
OS_enable_task_switching()，184 OS_lock_semaphore()，195
OSQPend()，176 OS_release_semaphore()，195
OSSemaphore，195
OutputQueue，21 本舰姿态，363–364 OwnShipAttitude.c，367 OwnShipAttitude.h，365–367

P

参数，事件的，27 父类，15 子类与，21 按引用传递，207 按值传递，线程与，207
PatientData，369，373–375
PatientData.c，380
PatientDataClass，383，392–393
PatientDataClass.c，394
PatientDataClass.h，393–394
PatientData.h，375–380
PatientDataType，369 访问硬件的模式，79–147 分析，44 到达模式，152 通道模式，395–402 用于并发，149–256 CRC，363，367–380 受保护单通道模式与，405 临界区模式，182–190 死锁与，232 硬件代理模式与，89 多事件接收器模式与，290 有序锁定模式与，248 竞态条件与，270 单事件接收器模式与，278 循环执行模式，164–170 去抖模式，122–130 分解AND状态模式，330–355 多样性设计模式，413 双通道模式，413–422 受保护调用模式，190–207 硬件代理模式与，89 多事件接收器模式与，290 互斥锁与，299 优先级反转来自，210 竞态条件与，270 会合模式与，228 单事件接收器模式与，278 硬件适配器模式，96–100 轮询模式与，142 硬件代理模式，85–95 硬件适配器模式与，98 轮询模式与，142 用于Harmony/ERT设计，44–77

basic structure of, 45–46
development and, 47–65
formats for, 46
Heterogeneous Redundancy Pattern, 413, 418
Homogeneous Redundancy Pattern, 413, 417
Interrupt Pattern, 130–138
Polling Pattern and, 141
Mediator Pattern, 100–111
Message Queuing Pattern, asynchronous communication for, 207
Monitor-Actuator Pattern, 420
Multiple Event Receptor Pattern, 287–296
Context and, 311
Observer Pattern, 51–77, 111–122
HistogramDisplay.c implementation file with, 77
Rendezvous Pattern and, 227, 228
TMDQueue header file with, 70–74
TMDQueue implementation file with, 74
One's Complement Pattern, 362–367
CRC and, 369
Protected Single Channel Pattern and, 405
Opportunistic Polling Pattern, 139
Ordered Locking Pattern, 242–255
Hardware Proxy Pattern and, 89
Periodic Polling Pattern, 140
Polling Pattern, 138–147
Interrupt Pattern and, 135
Protected Single Channel Pattern, 402–413
Channel Pattern and, 397
Protected Single Channel Pattern and, 405
Queuing Pattern, 207–224
Hardware Proxy Pattern and, 89
Rendezvous Pattern and, 227
Single Event Receptor Pattern and, 278
reliability, 357–423
Rendezvous Pattern, 224–231
Mediator Pattern and, 102–103
threads and, 227
for resource management, 149–256
safety, 357–423
Sanity Check Pattern, 419
Channel Pattern and, 397
Simultaneous Locking Pattern, 232–242
Hardware Proxy Pattern and, 89
Ordered Locking Pattern and, 248
Single Event Receptor Pattern, 274–287
Smart Data Pattern, 380–394
Smart Pointer Pattern, Smart Data Pointer and, 383
for state machines, 257–356
State Pattern, 308–327
State Table Pattern, 296–308
Static Priority Pattern, 170–181
Related Patterns and, 193
Strategy Pattern, Mediator Pattern and, 102–103
Synchronization Pattern, 153
Task Coordination Pattern, Cyclic Executive Pattern and, 167
Thread Barrier Pattern, counting semaphores and, 228
TMR, 418–419
Channel Pattern and, 397
Pattern hatching, for Harmony/ERT pattern designs, 47–49
Pattern instantiation
design pattern solution and, 45
for Harmony/ERT design, 44, 50–51
Pattern mining, for Harmony/ERT, 49–50
Pattern structure
for Channel Pattern, 395
for CRC, 368
for Critical Region Pattern, 182–183
for Cyclic Executive Pattern, 165
for Debouncing Pattern, 123
for Decomposed And-State Pattern, 331
for Dual-Channel Pattern, 414
Guarded Call Pattern, 190–191
for Hardware Adapter Pattern, 96
for Hardware Proxy Pattern, 86
for Harmony/ERT, 46
for Interrupt Pattern, 131
for Mediator Pattern, 101
for Multiple Event Receptor Pattern, 288
for Observer Pattern, 66, 112
for One's Complement Pattern, 362
for Ordered Locking Pattern, 243
for Polling Pattern, 138
for Protected Single Channel Pattern, 403
for Queuing Pattern, 208
for Rendezvous Pattern, 226
for Simultaneous Locking Pattern, 233–234
for Single Event Receptor Pattern, 274–275
for Smart Data Pattern, 381
for State Table Pattern, 296–297
for Static Priority Pattern, 171
Pause event, 262
Paused, 261, 268
PDF. See Probability density function
Performance, as Harmony/ERT criteria, 41
Period, 153
Polling Pattern and, 141
tasks and, 158
Periodic Polling Pattern, 140
PeriodicPoller, 140–141
Persistent failure, 360
PID. See Proportional Integral-Differential
Piece-wise continuous behavior, 261
poll(), 138
PollDataClient, 141
Polling Pattern, 138–147
Interrupt Pattern and, 135
PollTimer, 141
Polymorphism, 14–15

基本结构，45–46 开发与，47–65 格式，46 异构冗余模式，413，418 同构冗余模式，413，417 中断模式，130–138 轮询模式与，141 中介者模式，100–111 消息排队模式，异步通信用于，207 监控器-执行器模式，420 多事件接收器模式，287–296 上下文与，311 观察者模式，51–77，111–122
HistogramDisplay.c实现文件与，77 会合模式与，227，228 TMDQueue头文件与，70–74 TMDQueue实现文件与，74 反码模式，362–367 CRC与，369 受保护单通道模式与，405 机会轮询模式，139 有序锁定模式，242–255 硬件代理模式与，89 周期性轮询模式，140 轮询模式，138–147 中断模式与，135 受保护单通道模式，402–413 通道模式与，397 受保护单通道模式与，405 排队模式，207–224 硬件代理模式与，89 会合模式与，227
单事件接收器模式与，278 可靠性，357–423 会合模式，224–231 中介者模式与，102–103 线程与，227 用于资源管理，149–256 安全性，357–423 合理性检查模式，419 通道模式与，397 同时锁定模式，232–242 硬件代理模式与，89 有序锁定模式与，248 单事件接收器模式，274–287 智能数据模式，380–394 智能指针模式，智能数据指针与，383 用于状态机，257–356 状态模式，308–327 状态表模式，296–308 静态优先级模式，170–181 相关模式与，193 策略模式，中介者模式与，102–103 同步模式，153 任务协调模式，循环执行模式与，167 线程屏障模式，计数信号量与，228 TMR，418–419 通道模式与，397
去抖模式见第123页，分解AND状态模式见第331页，双通道模式见第414页，受保护调用模式见第190–191页，硬件适配器模式见第96页，硬件代理模式见第86页，Harmony/ERT见第46页，中断模式见第131页，中介者模式见第101页，多事件接收器模式见第288页，观察者模式见第66、112页，反码模式见第362页，有序锁定模式见第243页，轮询模式见第138页，受保护单通道模式见第403页，排队模式见第208页，会合模式见第226页，同时锁定模式见第233–234页，单事件接收器模式见第274–275页，智能数据模式见第381页，状态表模式见第296–297页，静态优先级模式见第171页，暂停事件见第262页，暂停见第261、268页，PDF。见概率密度函数 性能，作为Harmony/ERT准则，见第41页 周期，见第153页 轮询模式与周期，见第141页 任务与周期，见第158页 周期性轮询模式，见第140页 PeriodicPoller，见第140–141页 持续性故障，见第360页 PID。见比例积分微分 分段连续行为，见第261页 poll()，见第138页 PollDataClient，见第141页 轮询模式，见第138–147页 中断模式与轮询模式，见第 135 页 PollTimer，见第141页 多态，见第14–15页
P模式孵化，用于Harmony/ERT模式设计，47–49
P模式实例化设计模式解决方案以及，45，用于Harmony/ERT设计，44，50 51–
P模式挖掘，用于Harmony/ERT，49–50
P模式结构，用于通道模式，395，用于CRC，368，用于临界区模式，182–183，用于循环执行模式，165

POORLY_ORDERED_ACCESS, 247
Portability, as Harmony/ERT criteria, 42
Position.c, 197–198
Position.h, 196–197
PositionPredictor, 237
PositionSensor, 248
POST. See Power on self-test
post(), 271
PostSignal(), 274
Power on self-test (POST), 3
PowerOn event, 262
Precondition tests, 8
Preconditions, constraints and, 225
PredecessorState, 263
Predictability, 3
  as Harmony/ERT criteria, 41
Predictor-corrector algorithm, 7
Preemptive scheduling, 156
  basic task structure with, 157–160
  deadlock and, 232
PreemptiveTask, 190–191
  > and, 191
printf(), 224
printf tests, 9
printGasData() and, GasDisplay, 224
Priority ceiling, 193
Priority inheritance, 193
Priority inversion, 153
  from Guarded Call Pattern, 210
  multitasking preemptive scheduling and, 159
  unbounded, 160
Priority-based scheduling, 156
  loops for, 157
PriorityQueue
  blocking and, 172
  StaticTaskControlBlock and, 172
Private functions, Queuing Pattern and, 208–209
Probability density function (PDF), 8
Problem
  for Channel Pattern, 395
  for CRC, 367
  for Critical Region Pattern, 182
  for Cyclic Executive Pattern, 165
  for Debouncing Pattern, 123
  for Decomposed And-State Pattern, 331
  for Dual-Channel Pattern, 414
Guarded Call Pattern, 190
  for Hardware Adapter Pattern, 96
  for Hardware Proxy Pattern, 86
  for Harmony/ERT, 46
  for Interrupt Pattern, 130–131
  for Mediator Pattern, 100
  for Multiple Event Receptor Pattern, 287–288
  for Observer Pattern, 66, 112
  for One's Complement Pattern, 362
  for Ordered Locking Pattern, 243
  for Polling Pattern, 138
  for Protected Single Channel Pattern, 402
  for Queuing Pattern, 207–208
  for Rendezvous Pattern, 226
  for Simultaneous Locking Pattern, 233
  for Single Event Receptor Pattern, 274
  for Smart Data Pattern, 381
  for State Table Pattern, 296
  for Static Priority Pattern, 171
Procedures, 9
Processors, bit fields and, 85
Proportional Integral-Differential (PID), 259
Protected Single Channel Pattern, 402–413
  Channel Pattern and, 397
Pseudoconcurrency, 153
Pseudostates, 25, 267
  in UML, 268
Public functions, Queuing functions and, 209
Publish-Subscribe Pattern. See Observer Pattern
Purpose, for Harmony/ERT, 45
Push implementation, 64

Q

QoS. See Qualities of service
QRSDetector, 64, 68
QTask
  insert() and, 209
  Message and, 208
  MessageQueue and, 209
  remove() and, 209
Qualities of service (QoS), 3
  design patterns and, 44
  testing, 8
QueryMutex
  lock() and, 235
  tryLock() and, 235
Queue, 16–22
  code listing, 18–20
  implementation, 20
  leaky, 51
  testing, 21–22
  UML for, 16–17
Queue_Create(), 21
QUEUE_SIZE, 208
Queuing Pattern, 207–224
  Hardware Proxy Pattern and, 89
  Rendezvous Pattern and, 227
  Single Event Receptor Pattern and, 278
QUIT_TIME, 267, 268

R

Race condition, 153
  And-states and, 329
  Critical Region Pattern and, 270
  Guarded Call Pattern and, 270
  Interrupt Pattern and, 132–133
  synchronous state machines and, 270
rAction1(), 263
rAction2(), 263
Range tests, 8
Rate-monotonic scheduling, 5, 171
Reactive elements, statecharts and, 25
Ready, 260–261, 261
Ready to run, 156
Real-Time Action, 153
Real-time operating systems (RTOSs), 4, 5–6, 163–164
  Interrupt Pattern and, 135
Real-time systems, 2
Recurring cost, 3
  as Harmony/ERT criteria, 42
Redundancy, 360, 363
  for Protected Single Channel Pattern, 402
Reference model, 44
register(), Rendezvous Pattern and, 228
Regression tests, 8

POORLY_ORDERED_ACCESS，见第247页 可移植性，作为Harmony/ERT准则，见第42页
Position.c，见第197–198页
Position.h，见第196–197页
PositionPredictor，见第237页
PositionSensor，见第248页
POST。见上电自检 post()，见第271页 PostSignal()，见第274页 上电自检（POST），见第3页
PowerOn事件，见第262页 前置条件测试，见第8页 前置条件与约束，见第225页 PredecessorState，见第263页 可预测性，见第3页 作为Harmony/ERT准则，见第41页 预测-校正算法，见第7页 抢占式调度，见第156页 使用抢占式调度的基本任务结构，见第157–160页 死锁与抢占式调度，见第232页
PreemptiveTask，190 191 –与printf()，> 191
见第224页 printf测试，见第9页
printGasData()与GasDisplay，见第224页 优先级天花板，见第193页 优先级继承，见第193页 优先级反转，见第153页 来自受保护调用模式，见第210页 多任务抢占式调度与优先级反转，见第159页 无界优先级反转，见第160页 基于优先级的调度，见第156页 基于优先级的调度循环，见第157页
PriorityQueue阻塞与，见第172页
StaticTaskControlBlock 与，见第172页 私有函数与排队模式，见第208–209页 概率密度函数（PDF），见第8页 通道模式的问题，见第395页 CRC的问题，见第367页 临界区模式的问题，见第182页 循环执行模式的问题，见第165页 去抖模式的问题，见第123页
分解AND状态模式，331 双通道模式，414 受保护调用模式，190 硬件适配器模式，96 硬件代理模式，86 Harmony/ERT，46 中断模式，130–131 中介者模式，100 多事件接收器模式，287–288 观察者模式，66、112 反码模式，362 有序锁定模式，243 轮询模式，138 受保护单通道模式，402 排队模式，207–208 会合模式，226 同时锁定模式，233 单事件接收器模式，274 智能数据模式，381 状态表模式，296 静态优先级模式，171 过程，9 处理器、位域与，85 比例积分微分（PID），259 受保护单通道模式，402–413 通道模式与，397 伪并发，153 伪状态，25、267 在UML中，268 公共函数、排队函数与，209 发布-订阅模式。参见观察者模式 目的，用于Harmony/ERT，45 推送实现，64
remove() 与，209 服务质量（QoS），3 设计模式与，44 测试，8 QueryMutex
lock() 与，235 tryLock() 与，235 队列，16–22 代码清单，18–20 实现，20 泄漏，51 测试，21–22 队列的 UML，16–17 Queue_Create()，21 QUEUE_SIZE，208 排队模式，207–224 硬件代理模式与，89 会合模式与，227 单事件接收器模式与，278 QUIT_TIME，267，268

R

竞态条件，153 AND状态与，329 临界区模式与，270 受保护调用模式与，270 中断模式与，132–133 同步状态机与，270
rAction1()，263 rAction2()，263 范围测试，8 速率单调调度，5，171 反应式元素，状态图与，25 就绪，260–261，261 就绪，156 实时动作，153 实时操作系统（RTOSs），4，5–6，163–164 中断模式与，135 实时系统，2 经常性成本，3 作为Harmony/ERT准则，42 冗余，360，363 用于受保护单通道模式，402 参考模型，44
register()，会合模式与，228 回归测试，8

Q

QoS。参见服务质量
QRSDetector，64，68
QTask insert() 与，209
Message 与，208
MessageQueue 与，209

Related information, for task identification, 161
Related patterns
  for Channel Pattern, 397
  for CRC, 369
  for Critical Region Pattern, 184
  for Cyclic Executive Pattern, 167
  for Debouncing Pattern, 126
  for Decompose And-State Pattern, 334
  for Dual-Channel Pattern, 417–420
  for Guarded Call Pattern, 193–194
  for Hardware Adapter Pattern, 98
  for Hardware Proxy Pattern, 89
  for Mediator Pattern, 102–103
  for Multiple Event Receptor Pattern, 290
  for Observer Pattern, 115
  for One's Complement Pattern, 363
  for Ordered Locking Pattern, 248
  for Polling Pattern, 142
  for Protected Single Channel Pattern, 405
  for Queuing Pattern, 210
  for Rendezvous Pattern, 228
  for Simultaneous Locking Pattern, 236
  for Single Event Receptor Pattern, 278
  for Smart Data Pattern, 383
  for State Pattern, 313–314
  for State Table Pattern, 299–300
  for Static Priority Pattern, 174–175
  Static Priority Pattern and, 193
Relations, of Abstract Subject, 68
release()
  MessageQueue and, 209
  Mutex and, 172, 235
  semaphores and, 189
releaseDyadic(), 244
Release_semaphore(Semaphore*), 227
Reliability
  faults and, 360—361
  as Harmony/ERT criteria, 42
  patterns, 357–423
  up-time and, 359
remove()
  GasDataQueue and, 213
  QTask and, 209
RemoveInterruptHandler, 141
Rendezvous Pattern, 224–231
  Mediator Pattern and, 102–103
  threads and, 227
reset(), Mediator Pattern and, 101
ResetQuitTime, 268
Resource management, patterns for, 149–256
Resource usage
  as Harmony/ERT criteria, 42
  of Observer Pattern, 64–65
ResourceClient, 235, 245–247
  OrderedResources and, 245
  resourceID and, 245
ResourceClient
  dyadic access functions and, 247
  lockDyadic() and, 244
ResourceClients, OrderedResources and, 247
ResourceID
  OrderedResource and, 244
  OrderedResource_Init() and, 244
  ResourceClient and, 245
ResourceList, 244
  addLock() and, 247
ResourceList.c, 255
ResourceList.h, 252–255
ResourceMaster, 234
  MultimasteredResources and, 235
ResourceReference, 247
Resume event, 262
Return From Interrupt (RTI), 132
Reusability, as Harmony/ERT criteria, 42
Richardson, Mark, 264
Robot Arm Manager, 103–104
RobotArmManager.c, 111
RobotArmManager.h, 105–111
Round robin scheduling, 5
  problems with, 156
RoutePlanningClient, 248
RS-232, 82
RS-485, 82
RTI. See Return From Interrupt
RTOSs. See Real-time operating systems
run(), 166
_run(), 176
Running, 156
Run-time checking, 380
Run-time flexibility, of Observer Pattern, 64–65
Run-to-completion, 328–329

S

Safety
  faults and, 360—361
  as Harmony/ERT criteria, 42
  patterns, 357–423
  security and, 359
Safety assurance, for task identification, 161
SafetyFailState, 269
SafetyMonitorClient, 119
Sanity Check Pattern, 419
  Channel Pattern and, 397
Scalability, as Harmony/ERT criteria, 42
Schedulability
  blocking and, 159
  Cyclic Executive Pattern and, 164–165
  definition of, 153
  as Harmony/ERT criteria, 41
  Static Priority Pattern and, 170
Scheduling
  cyclic executive, 5
    problems with, 156
    processing loop, 155–157
    tasks for, 155
  multitasking preemptive
    definition of, 153
    mutual exclusion and, 189
    priority inversion and, 159
  preemptive, 156
    basic task structure with, 157–160
    deadlock and, 232
  priority-based, 156
    loops for, 157
  rate-monotonic, 5, 171
  round robin, 5
    problems with, 156
  time-slicing, 156
    loops for, 157
Scrum, 36

相关信息，用于任务识别，161
相关模式 用于通道模式，397 用于CRC，369 用于临界区模式，184 用于循环执行模式，167 用于去抖模式，126 用于分解与状态模式，334 用于双通道模式，417–420 用于受保护调用模式，193–194 用于硬件适配器模式，98 用于硬件代理模式，89 用于中介者模式，102–103 用于多事件接收器模式，290 用于观察者模式，115 用于反码模式，363 用于有序锁定模式，248 用于轮询模式，142 用于受保护单通道模式，405 用于排队模式，210 用于会合模式，228 用于同时锁定模式，236 用于单事件接收器模式，278 用于智能数据模式，383 用于状态模式，313–314 用于状态表模式，299–300 用于静态优先级模式，174–175 静态优先级模式与，193 关系，抽象主题的，68 release()
MessageQueue与，209 互斥锁与，172，235 信号量与，189 releaseDyadic()，244 Release_semaphore(Semaphore*)，227 可靠性 故障与，360—361 作为Harmony/ERT准则，42 模式，357–423
正常运行时间以及，359 remove() GasDataQueue 以及，213 QTask 以及，209 RemoveInterruptHandler，141 会合模式，224–231 中介者模式以及，102–103 线程以及，227 reset()，中介者模式以及，101 ResetQuitTime，268 资源管理，相关模式，149–256 资源使用作为 Harmony/ERT 准则，42 观察者模式的，64–65 ResourceClient，235，245–247 OrderedResources 以及，245 resourceID 以及，245 ResourceClient 二元访问函数以及，247 lockDyadic() 以及，244 ResourceClients，OrderedResources 以及，247 ResourceID
OrderedResource 以及，244 OrderedResource_Init() 以及，244 ResourceClient 以及，245 ResourceList，244 addLock() 以及，247 ResourceList.c，255 ResourceList.h，252–255 ResourceMaster，234 MultimasteredResources 以及，235 ResourceReference，247 恢复事件，262 中断返回 (RTI)，132 可复用性，作为 Harmony/ERT 准则，42 Richardson, Mark，264 机械臂管理器，103–104 RobotArmManager.c，111 RobotArmManager.h，105–111 轮转调度，5 相关问题，156 RoutePlanningClient，248 RS-232，82 RS-485，82 RTI。参见 中断返回 实时操作系统。参见 实时操作系统
run()，166 run()，176
跑步，156 运行时检查，380 观察者模式的运行时灵活性，64–65 运行至完成，328–329

S

安全故障以及，360—361 作为 Harmony/ERT 准则，42 模式，357–423 安全加密与，359 安全保证，用于任务识别，161 SafetyFailState，269 SafetyMonitorClient，119 合理性检查模式，419 通道模式与，397 可扩展性，作为 Harmony/ERT 准则，42 可调度性阻塞与，159 循环执行模式与，164–165 其定义，153 作为 Harmony/ERT 准则，41 静态优先级模式与，170 调度循环执行程序，5 相关问题，156 处理循环，155–157 用于调度的任务，155 多任务抢占式定义，153 互斥与，189 优先级反转与，159 抢占式，156 基本任务结构，157–160 死锁与，232 基于优先级，156 循环，157 速率单调，5，171 轮询，5 相关问题，156 时间分片，156 循环，157
Scrum，36

Security
  encryption and, 89
  safety and, 359
Semaphore* create_semaphore (void), 227
Semaphores, 159, 189
  binary, 189
  counting, 189
    Thread Barrier Pattern and, 228
  lock() and, 191
  mutual exclusion and, 189
  Rendezvous Pattern and, 227
  RTOS and, 163
SEM_INVERSION_SAFE, 193
sendEvent(), 124
SensorDeviceDriver, 396, 404, 417
  ConcreteDataTransform and, 406
SensorMaster, 236–237
SensorMaster.c, 242
SensorMaster.h, 239–242
Sensors, 82
SensorThread, 217–218
  GasProcessingThread and, 224
SensorThread.c, updateData of, 219–220
SensorThread.h, 218–219
SensorThread_updateData(), 218
SERSMs. See Single event receptor state machines
ServerClass, 382, 383
setData(), 133
setPollTime(t), 140
setValue(), 183
SharedData.c, 169–170
SharedData.h, 169
SharedResource
  ApplicationElement and, 133
  ConcreteThread and, 172
  Mutex and, 172
show(), 119
SignalEvent, 26
Silence, 264
Simple behavior, 259
SimplifiedResourceMaster, 234
  MasterResources and, 235
Simulating on host, 9
Simultaneous Locking Pattern, 232–242
  Hardware Proxy Pattern and, 89
  Ordered Locking Pattern and, 248
Single event groups, for task identification, 161
Single Event Receptor Pattern, 274–287
Single event receptor state machines (SERSMs), 274
Singletons, 13
Smart Data Pattern, 380–394
Smart devices, 81
Smart Pointer Pattern, Smart Data Pointer and, 383
SmartColor, 388–389
SmartColor.c, 392
SmartColor.h, 389–392
SmartDataExample.h, 384–386
SmartDataType, 382
SmartDataType_cmp(), 382
SmartDataType_getPrimitive(), 382
SmartDataType_getValue(), 382
SmartDataType_Init(), 382
SmartDataType_setValue(), 382
SmartInt.c, 388
SmartInt.h, 386–388
Soft real-time systems, 2
Solution, for Harmony/ERT, 45
Specialization
  function pointers in, 16, 18
  subclass and, 15–16
Specific Collaborator, for Mediator Pattern, 101–102
Stack, AbstractThread and, 173
start(), 166
StartAddr, EntryPoint and, 173
startTimer(), 141
States
  And-states, 25, 265–267, 327–330
    nested states and, 266
  definition of, 261
  events and, 27
  initialize() and, 260
  milliseconds and, 267
  nested, 25, 263–264
    And-states and, 266
  operations and, 25
  OR-states, 263–265
  pseudostates, 25, 267
    in UML, 268
State explosion, UML and, 266
State machines
  asynchronous events and, 270–274
  FSM, 25–33, 261–274
  MERSMs, 287
  patterns for, 257–356
  SERSMs, 274
  synchronous, 270–274
    State Table Pattern and, 299
  UML and, 263–264
State Pattern, 308–327
State space, 330
State Table Pattern, 296–308
StateActionSet, 311, 312, 313, 334
StateActionSet.c, 318–319, 348
StateActionSet.h, 318, 347–348
Statecharts
  reactive elements and, 25
  UML and, 25
Stateful behavior, 260
stateID, 277
  State Table Pattern and, 296
  TokenizeMultiReceptor and, 289
stateList, 308, 311
StatePattern.h, 314–315
StateTableEntries, 299
StateTablePattern.h, 301
STATETYPE, 271
stateVariable, 271
Static Priority Pattern, 170–181
  Related Patterns and, 193
StaticPriorityScheduler, 173
  Mutex and, 172, 192
StaticTaskControlBlock, PriorityQueue and, 172
Statistical tests, 8
Stereotypes, 405
stopEmittingMicrowaves(), as action, 262
stopPolling(), 141
Strategy Pattern, Mediator Pattern and, 102–103
Stress tests, 8
Structured language, vs. OO, 9–10
Subclasses, 15–25
  base class and, 21
  inheritance and, 68
  Mediator Pattern and, 100
  specialization and, 15–16
  super class and, 68
subscribe(), 69

安全加密与，89 安全性，359 信号量* create_semaphore (void)，227 信号量，159，189 二值，189 计数，189 线程屏障模式与，228 lock() 与，191 互斥与，189 会合模式与，227 实时操作系统与，163 SEM_INVERSION_SAFE，193 sendEvent()，124 SensorDeviceDriver，396，404，417 ConcreteDataTransform 与，406 SensorMaster，236–237 SensorMaster.c，242 SensorMaster.h，239–242 传感器，82 SensorThread，217–218 GasProcessingThread 与，224 SensorThread.c，其 updateData，219–220 SensorThread.h，218–219 SensorThread_updateData()，218 SERSMs。见单事件受体状态机 ServerClass，382，383 setData()，133 setPollTime(t)，140 setValue()，183 SharedData.c，169–170 SharedData.h，169 SharedResource ApplicationElement 与，133 ConcreteThread 与，172 Mutex 与，172 show()，119 SignalEvent，26 静音，264 简单行为，259 SimplifiedResourceMaster，234 MasterResources 与，235 主机仿真，9 同时锁定模式，232–242 硬件代理模式与，89 有序锁定模式与，248
单事件组，用于任务识别，161 单事件接收器模式，274–287 单事件受体状态机（SERSMs），274 单例，13 智能数据模式，380–394 智能设备，81 智能指针模式，智能数据指针与，383 SmartColor，388–389 SmartColor.c，392 SmartColor.h，389–392 SmartDataExample.h，384–386 SmartDataType，382 SmartDataType_cmp()，382 SmartDataType_getPrimitive()，382 SmartDataType_getValue()，382 SmartDataType_Init()，382 SmartDataType_setValue()，382 SmartInt.c，388 SmartInt.h，386–388 软实时系统，2 解决方案，用于 Harmony/ERT，45 特化 函数指针在，16, 18 子类与，15–16 特定协作者，用于中介者模式，101–102 栈，AbstractThread 与，173 start()，166 StartAddr，EntryPoint 与，173 startTimer()，141 状态 与状态，25, 265–267, 327–330 嵌套状态与，266 定义，261 事件与，27 initialize() 与，260 毫秒与，267 嵌套，25, 263–264 与状态与，266 操作与，25 或状态，263–265 伪状态，25，267 在 UML 中，268 状态爆炸，UML 与，266 状态机
异步事件以及，270–274 有限状态机，25–33，261–274 MERSM，287 相关模式，257–356 SERSM，274 同步，270–274 状态表模式与，299 UML 与，263–264 状态模式，308–327 状态空间，330 状态表模式，296–308 StateActionSet，311，312，313，334 StateActionSet.c，318–319，348 StateActionSet.h，318，347–348 状态图反应式元素与，25 UML 与，25 有状态行为，260 stateID，277 状态表模式与，296 TokenizeMultiReceptor 与，289 stateList，308，311 StatePattern.h，314–315 StateTableEntries，299 StateTablePattern.h，301 STATETYPE，271 stateVariable，271 静态优先级模式，170–181 相关模式与，193 StaticPriorityScheduler，173 互斥锁与，172，192 StaticTaskControlBlock，优先级队列与，172 统计测试，8 构造型，405 stopEmittingMicrowaves()，作为动作，262 stopPolling()，141 策略模式，中介者模式与，102–103 压力测试，8 结构化语言，与面向对象对比，9–10 子类，15–25 基类与，21 继承与，68 中介者模式与，100 特化与，15–16 超类与，68 subscribe()，69

subscribe(acceptPtr), 113
SubsequentState, 263
Super class
  inheritance and, 68
  subclass and, 68
Switch case, 157
Switch-case statement, 16
Synchronization, Rendezvous Pattern and, 226–227
Synchronization Pattern, 153
Synchronization points
  definition, 153
  tasks and, 154–155
synchronize(), Rendezvous Pattern and, 227
SynchronizingThread, notify() and, 227
Synchronous communications, 155
Synchronous event transfer, 26
Synchronous events, state machines and, 270–274
Synchronous state machines, State Table Pattern and, 299
SynchronousStateMachine, 271

T

TableEntry, 296
TableEntryType, 298
table[stateID][eType], 297
Tasks
  communication with, 155
  concurrency and, 155
  coordination patterns, 182–231
  criticality and, 157
  for cyclic executive scheduling, 155
  definition of, 153
  diagrams, 162
  identification of, 160–161
  preemptive scheduling and, 157–160
  priority, 5
  RTOS and, 163
  synchronization points and, 154–155
  waiting by, 156
Task Coordination Pattern, Cyclic Executive Pattern and, 167
TaskWithCriticalRegion, 183
TaskWithSharedResource, 184
TDD. See Test-driven development
TensPlace, 289
Test buddies, 9
TestBuilder.c, 62–63
TestBuilder.h, 61–62
Test-driven development (TDD), 8
Testing, 8–9
  on host, 9
  Queue, 21–22
Thermometer, 406–407, 411–412
Thermometer.c, 413
Thermometer.h, 412–413
Thermostat, 406–407
Thermostat.c, 409
Thermostat.h, 407–409
Thread Barrier Pattern, counting semaphores and, 228
ThreadBarrier.c, 231
ThreadBarrier.h, 229–231
Threads, 153
  forks and, 154
  joins and, 154
  passed by value and, 207
  Rendezvous Pattern and, 227
Throughput, 3
  as Harmony/ERT criteria, 42
Timed Marked Data queue (TMDQueue), 52–53
TimedCook event, 262
Time-driven multitasking executive (TMDE), 156
TimeEvent, 26
Timeliness, 153
Timeouts, 267
Timers, RTOS and, 163
Timescales, for Harmony/ERT, 36
Time-slicing schedule, 156
  loops for, 157
timeToCook-=currentTime(), 260
  as action, 262
Time-triggered cyclic executive, 156
Tm() event, 27, 262, 267
TMDE. See Time-driven multitasking executive
TMDQueue. See Timed Marked Data queue
  Abstract Subject and, 68, 69
  header file, with Observer Pattern, 70–74
  implementation file, with Observer Pattern, 74
  NotificationHandle and, 69
TMDQueue.c, 56–58
TMDQueue_insert(), 53
TMDQueue_remove(), 58
TMR. See Triple Modular Redundancy Pattern
TMRClient, 289
Tokenize Async Single Pkg.h, 280–281
Tokenize Async Single Receptor_task Loop(), 279–280
Tokenize Multi Receptor, 289, 290–292
Tokenize Multi Receptor.c, 296
Tokenize Multi Receptor.h, 292–296
Tokenizer Async Single Receptor, 276–277
Tokenizer Multi Receptor, 288
Tokenizer StateTable, 298, 300–301
  init() and, 297
Tokenizer State Table.c, 307
Tokenizer State Table.h, 302–307
Tokenizer Sync Single Receptor, 277
Tokenizer Sync Single Receptor_event Dispatch(), 274
Tokenize Sync Single Receptor, 274
Tool chain, 4–5
Trade study, 42
Transitions, 263
  action list for, 27
  definition of, 262
  event triggers and, 27
Triggered operations, 27
Triple Modular Redundancy Pattern (TMR), 418–419
  Channel Pattern and, 397
tryLock()
  MultimasteredResources and, 235
  Mutex and, 234
  mutual exclusion semaphores and, 236
  Query Mutex and, 235
  semaphores and, 189

subscribe(acceptPtr)，113
SubsequentState，263 超类继承与，68 子类与，68 Switch case，157 Switch-case 语句，16 同步，会合模式与，226–227 同步模式，153 同步点定义，153 任务与，154–155
synchronize()，会合模式与，227 SynchronizingThread，notify() 与，227 同步通信，155 同步事件传输，26 同步事件，状态机与，270–274 同步状态机，状态表模式与，299
SynchronousStateMachine，271

T

TableEntry，296
TableEntryType，298 表[stateID][eType]，297 任务与...
...的通信，155 并发与，155 协调模式，182–231 关键性与，157 用于循环执行调度，155 ...
...的定义，153 图，162 .....的标识，160–161 抢占式调度与，157–160 优先级，5 RTOS与，163 同步点与，154–155 通过...
...等待，156 任务协调模式，循环执行模式与，167
TaskWithCriticalRegion，183
TaskWithSharedResource，184
TDD。参见测试驱动开发
TensPlace，289 测试伙伴，9
TestBuilder.c，62–63
TestBuilder.h，61–62 测试驱动开发（TDD），8 测试，8–9 在主机上，9 队列，21–22
Thermometer，406–407，411–412 Thermometer.c，413 Thermometer.h，412–413 Thermostat，406–407
Thermostat.c，409
Thermostat.h，407–409 线程屏障模式，计数信号量与，228
ThreadBarrier.c，231
ThreadBarrier.h，229–231 线程，153 分叉与，154 汇合与，154 按值传递与，207 会合模式与，227 吞吐量，3 作为Harmony/ERT准则，42 时间标记数据队列（TMDQueue），52–53 TimedCook事件，262 时间驱动多任务执行程序（TMDE），156 TimeEvent，26 及时性，153 超时，267 定时器，RTOS与，163 时间尺度，用于Harmony/ERT，36 时间片调度，156 .....的循环，157
timeToCook-=currentTime()，260 作为动作，262 时间触发循环执行程序，156 Tm()事件，27，262，267 TMDE。参见时间驱动多任务执行程序
TMDQueue。参见时间标记数据队列 抽象主题与，68，69 头文件，使用观察者模式，70–74
  实现文件，含观察者模式，74 NotificationHandle 以及，69 TMDQueue.c，56–58
TMDQueue_insert()，53
TMDQueue_remove()，58
TMR。参见三模冗余模式
TMRClient，289 Tokenize Async Single Pkg.h，280–281 Tokenize Async Single Receptor_task Loop()，279–280 Tokenize Multi Receptor，289，290 292– Tokenize Multi Receptor.c，296 Tokenize Multi Receptor.h，292–296 Tokenizer Async Single Receptor，276–277 Tokenizer Multi Receptor，288 Tokenizer StateTable，298，300–301 init() 及其，297 Tokenizer State Table.c，307 Tokenizer State Table.h，302–307 Tokenizer Sync Single Receptor，277 Tokenizer Sync Single Receptor_event Dispatch()，274 Tokenize Sync Single Receptor，274 工具链，4–5 权衡研究，42 迁移，263 其动作列表，27 定义，262 事件触发器及其，27 触发操作，27 三模冗余模式（TMR），418–419 通道模式及其，397
tryLock() 与 MultimasteredResources 及其，235 互斥锁及其，234 互斥信号量及其，236 查询互斥量及其，235 信号量及其，189

TSR Client, 277
TSR Event Queue, 277–278
TSR Event Queue.c, 282–284
TSR Event Queue.h, 281–282
TSR Event Queue_post(), 274, 277, 279
TSR Event Queue_pull(), 278
TSR Sync Single Receptor.c, 287
TSR Sync Single Receptor.h, 284–287
TSTATE, 278, 289
TSTATETYPE, 277, 298, 312
TSUBSTATE, 278, 290
TSUBSTATETYPE, 277

U

UltimateO2Adapter.c, 100
UML. See Unified Modeling Language
Unbounded priority inversion, 160
Unified Modeling Language (UML)
  class diagram for, 16–17
  concurrency in, 161–163
  events in, 26
  generalization in, 15
  pseudostates in, 268
  state explosion and, 266
  state machines and, 263–264
  statecharts and, 25
union, 275
unmarshal()
  for Hardware Proxy class, 88
  as Internal Data Formatting Function, 91
unsigned char MessageQueue_isEmpty (MessageQueue* me), 209
unsigned char MessageQueue_isFull (MessageQueue* me), 209
unsubscribe(), 69
unsubscribe(acceptPtr), 113
update(), 69
updateData, of SensorThread.c, 219–220
Up-time, reliability and, 359
Urgency, 153
  Interrupt Pattern and, 130
  tasks and, 157
<<Usage>>, 10
USB, 82
useDefaults(), 268

V

Validation, of Harmony/ERT design, 44
Value, 183
vectorPtr, 132
VelocitySensor, 248
VelocitySensor.c, 252
VelocitySensor.h, 250–252
Virtual functions, 14–15
  inheritance and, 68
void*, for Hardware Proxy class, 87
void reset(void), Rendezvous Pattern and, 226
void synchronize(void), Rendezvous Pattern and, 227
Volatile failure, 360
Volume tests, 8
VxWorks, by Wind River, 5, 193

W

WAIT_FOREVER, FLAG and, 193
Waiting, 267
  by tasks, 156
WaitingForSelectedStartingPoint, 269
WAIT_TIME, 267
WaveformDisplay, 64, 68
Wind River, VxWorks by, 5, 193
Worst-case execution time, 158–159
  Cyclic Executive Pattern and, 164
  definition of, 153
  Queuing Pattern and, 208
writeMotorSpeed(), as Motor Control Function, 91

X

XOR, bit-wise operator, 83

Z

ZeroXMotor, 269, 270
ZeroYMotor, 269, 270

TSR客户端，277 TSR事件队列，277–278 TSR Event Queue.c，282–284 TSR Event Queue.h，281–282 TSR Event Queue_post()，274，277，279
TSR Event Queue_pull()，278
TSR Sync Single Receptor.c，287
TSR Sync Single Receptor.h，284–287 TSTATE，278，289
TSTATETYPE，277，298，312
TSUBSTATE，278，290
TSUBSTATETYPE，277

U

UltimateO2Adapter.c，100
UML。参见统一建模语言 无界优先级反转，160 统一建模语言（UML）类图，16–17 中的并发，161–163 中的事件，26 中的泛化，15 中的伪状态，268 状态爆炸与，266 状态机与，263–264 状态图与，25 联合体，275 用于硬件代理类的 unmarshal()，88
作为内部数据格式化函数，91 无符号字符 MessageQueue_isEmpty (MessageQueue* me)，209 无符号字符 MessageQueue_isFull (MessageQueue* me)，209 unsubscribe()，69
unsubscribe(acceptPtr)，113 update()，69
updateData，属于 SensorThread.c，219–220 正常运行时间，可靠性与，359 紧迫性，153 中断模式与，130 任务与，157<<Usage>>，10
USB，82 useDefaults()，268

V

验证，Harmony/ERT设计，44 值，183 vectorPtr，132
VelocitySensor，248
VelocitySensor.c，252
VelocitySensor.h，250–252 虚函数，14–15 继承与，68
void*，用于硬件代理类，87
void reset(void)，会合模式与，226 void synchronize(void)，会合模式与，227 易失性故障，360 容量测试，8 VxWorks，由 Wind River，5，193

W

WAIT_FOREVER、FLAG 和、193 等待、267 由任务、156
W aitingForSelectedStartingPoint、269 WAIT_TIME、267
WaveformDisplay、64、68
Wind River、VxWorks 由、5、193 最坏情况执行时间、158–159 循环执行模式与、164 定义、153 排队模式与、208
writeMotorSpeed()、作为电机控制函数、91

X

异或、按位运算符、83

Z

ZeroXMotor、269、270
ZeroYMotor、269、270